LOGIX CONNECT

PLC TRAINING WORKSTATION

✓ Placing 27 devices on the floor
✓ Opening the Start Page
DN · App_Ready = 1

1769-L33ER · V1.47 CREATED AND DESIGNED BY DEVON CALHOUN

The Programmer’s Guide

Ladder logic from the first rung to a running bottling line — a textbook, a parts catalogue and a step-by-step build of the bottle line from an empty sandbox

How to use this book

This is three books in one. **Part I and II** are the textbook: what a controller does, what a rung is, every instruction you will use, and how to read the machine while it runs. **Part III** is the catalogue: every part in the sandbox palette with its parameters and I/O, generated from the program itself so it cannot drift. **Part IV** is the build: a bottling line laid out, wired and programmed from an empty floor, one routine at a time, with something to run and watch at the end of every lesson. **Part V** points past the bottle line, and the appendices hold the complete program listing, the tag list, the register bit map, the shortcuts and the glossary.

If you have never programmed a PLC, read Parts I and II in order and do the small exercises; they take an afternoon. If you have, skim Part II for the house rules (rung text, tag naming, the alias convention) and go to Part IV. Keep Part III open while you build anything of your own.

Conventions. Rung text is printed exactly as you type it into the editor, in a box like this:

A rung, with its comment above it

```
XIC(Start_PB)XIC(Stop_PB)OTE(Run);
```

Names in `fixed type` are tags, signals and addresses as they appear in the program. Menu paths are written **File → New Project…**. A *Try it* box is an exercise with a known answer; a *Watch for* box is a mistake people make at that point. Everything in this book was checked against v1.47 of the page: the program's own regression suite builds the line and runs the lessons' program every release.

Every rung of the lessons, the levels and the classic scenes comes with four helpers, so that you work out what you type instead of copying it. **Predict it** (the dashed purple box) gives you a moment on the machine, which bits are on and what the numbers read, and asks what the rung will do; work it out from the ladder before you read on, and check yourself against the answers at the end of the chapter. **How this rung works** (the blue box) says the whole rung in plain words, then goes through it left to right, instruction by instruction, in the words of the tags' own descriptions, branch by branch; then it names the pattern the rung is built on and why. **Think of it like this** (green) puts an idea you have not met before into everyday terms: a timer you have to keep your finger on, a recipe card with one finger on it, the two doors of an airlock. **Watch for** (brown) is the trap in that pattern, the mistake that looks right and is not. The picture and the trap appear in full the first time a pattern turns up in a chapter; after that the box just reminds you.

Contents

Part I — Getting started

1. What Logix Connect is 6
2. The workbench: a tour 9
3. Projects: new, save, open, export 17

Part II — Ladder logic from zero

4. How a controller thinks: the scan and the rung 21
5. Tags, data types and I/O addresses 27
6. Bit instructions 31
7. Timers and counters 34
8. Compare, move and math 37
9. Routines, programs and the main routine 39
10. Shift registers: tracking parts down a line 45
11. Reading the machine while it runs 48

Part III — The sandbox and its parts

12. The sandbox floor 52
13. Parts catalogue 66
14. Operating the Process HMI 133

Part IV — Building the bottle line from ground zero

15. The line we are going to build 156
16. Lesson 1 — Lay out the machine 160
17. Lesson 2 — Name the I/O: the tag list 169
18. Lesson 3 — Step 1: the run circuit 177
19. Lesson 4 — Step 2: the dwell timer and the feeder 182
20. Lesson 5 — Step 3: index one pitch 186
21. Lesson 6 — Step 4: the shift registers 192
22. Lesson 7 — Step 5: filling 196
23. Lesson 8 — Step 6: the reservoir and the stack light 204
24. Lesson 9 — Step 7: capping and the cap feeder 211
25. Lesson 10 — Step 8: the gates, the divert conveyor, the boxes and the shredder 223
26. Lesson 11 — Step 9: the displays, the preset and Enter 232
27. Lesson 12 — Step 10: the labeler, FLS logic 242
28. Lesson 13 — Step 11: alarms and machine faults 260
29. Lesson 14 — Step 12: the case packer, both sizes, and the VFD 293

30. Lesson 15 — Step 13: the mix kitchen 317
31. Commissioning: the whole line, start to finish 351

Part V — The graded levels

32. The graded levels: the machine, the bins and the score 354
33. Level 1 — Height sort 360
34. Level 2 — Fill by label 380
35. Level 3 — Colour sort 404
36. Level 4 — Two on the line 427
37. Level 5 — Full sort 466
38. Level 6 — Pack it 511
39. Level 7 — Batch Mix 543
40. Level 8 — Vision check 575
41. Level 9 — Changeover 607
42. Level 10 — Servo pick 638
43. Level 11 — Lot code and verify 657
44. Level 12 — Kitchen changeover 687

Part VI — Beyond the bottle line

45. Save, share and replay 727
46. A servo axis on a Festo FHPP drive 731
47. Classic scenes: the elevator 757
48. Classic scenes: the traffic intersection 814
49. The sample sorter, programmed 870
50. Classic scenes: doors and gates 911
51. Classic scene: the car wash 989
52. Where to go next 1057

Appendices

Appendix A — The bottle line tag list 1059
Appendix B — The complete program 1068
Appendix C — The register bit map 1085
Appendix D — Sensor reference 1086
Appendix E — Keyboard shortcuts 1092
Appendix F — Glossary 1093
Appendix G — Instruction set 1104
Appendix H — Walkthrough log 1105

Part I — Getting started

1 What Logix Connect is

Logix Connect is a PLC training workstation that runs in a browser (and, wrapped, as a Windows app and a phone app). It puts three things on one screen that in a plant are three separate things: a **controller** styled on a CompactLogix with Studio 5000-style editing, a **chassis of I/O modules** that the controller scans, and a **machine** wired to that I/O: either one of the graded bottle-line levels or the sandbox, where you lay the machine out yourself from a palette of parts.

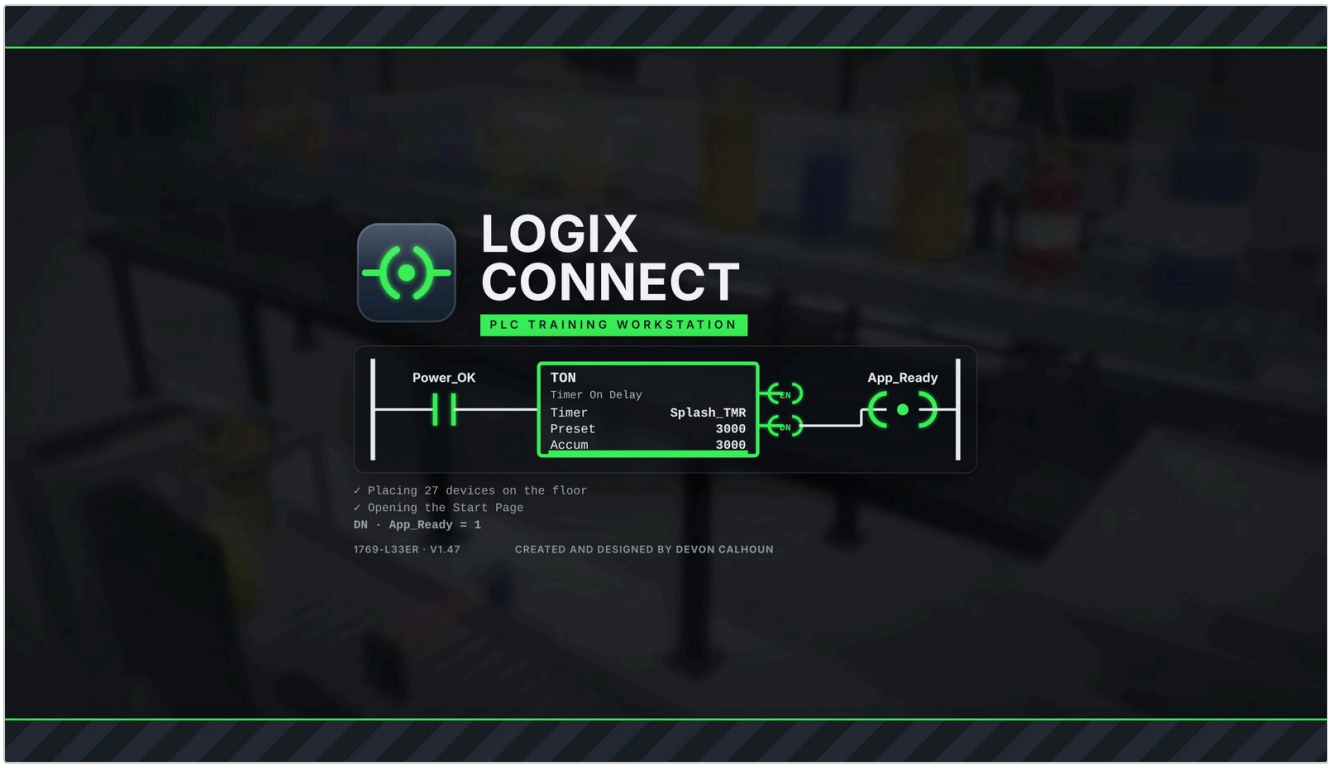


Figure 1. The splash is itself a rung: Power_OK closes, a TON times 3000 ms, and the App_Ready coil opens the workbench. By the end of chapter 7 you will read it without thinking.

The controller is real in the ways that matter for learning. It scans: inputs are read, the ladder is solved rung by rung, top to bottom, left to right, outputs are written, and it does that again every 20 ms. Tags are typed (BOOL, DINT, TIMER…), I/O lives at module addresses (`Local:1:I.Data.0`), alias tags give those addresses names, routines are called from a main routine with JSR, and a rung is written in the same neutral text Studio 5000 uses when you copy a rung. Projects export to L5X and import from it. What it leaves out is the parts of a real controller that teach nothing about logic: no firmware, no download time, no licence.

The machine is also real in the ways that matter. Conveyors carry parts at a speed, sensors see what is physically in front of them, a gate that fires late pushes the wrong bottle, a shredder that runs with no box under it spills glass onto the floor, a capper with an empty chute leaves bottles open, and a tank runs dry if

the inlet valve is never opened. The simulator does not know what you meant; it only knows what your outputs did. That is the whole point.

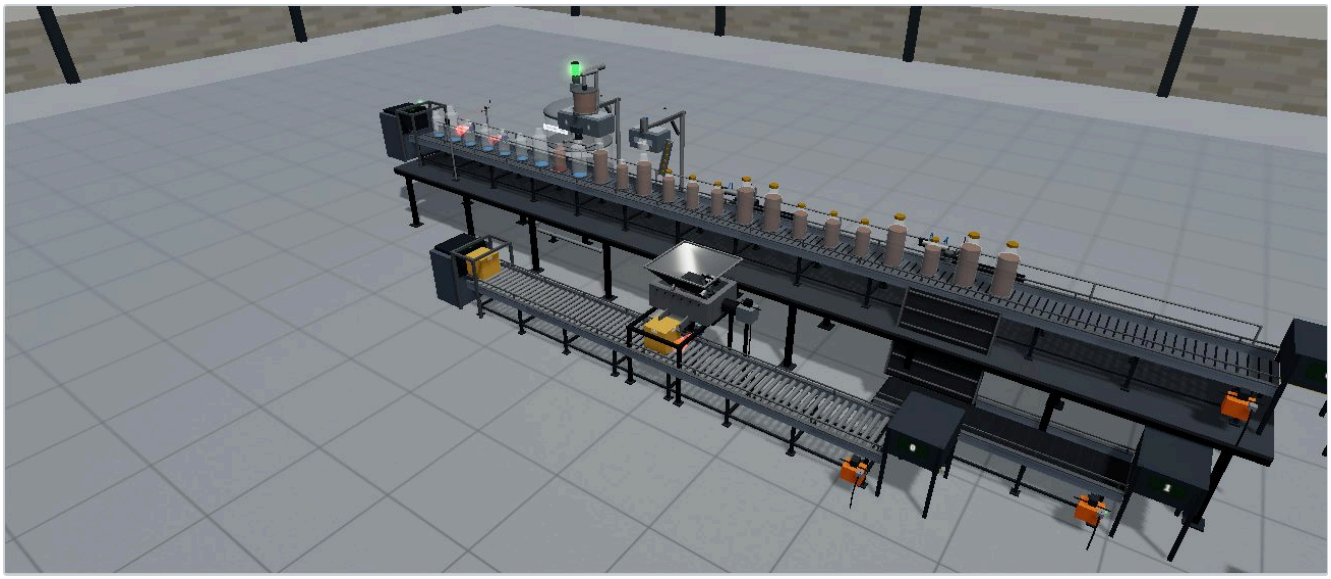


Figure 2. The LogixPro-style bottle line of Part IV, in the 3D view: a raised main conveyor, the fill station and capper, the scrap gate over the shredder and its box conveyor, the divert gate and its incline.

The chassis

Every sandbox project has the same chassis. Slot 0 is the controller, slots 1–3 are 16-point input modules, slots 4–6 are 16-point output modules, slots 7 and 9 are 4-channel analog input modules and slot 8 is a 4-channel analog output module. A point is written the way Studio 5000 writes it: `Local:<slot>:I.Data.<bit>` for an input bit, `Local:<slot>:O.Data.<bit>` for an output bit and `Local:<slot>:I.Ch<n>Data` for an analog channel.

Slot	Module	Name	Points
0	1769-L33ER	LogixConnect	controller
1	1769-IQ16	DI_1	16 digital inputs, Local:1:I.Data.0–15
2	1769-IQ16	DI_2	16 digital inputs, Local:2:I.Data.0–15
3	1769-IQ16	DI_3	16 digital inputs, Local:3:I.Data.0–15
4	1769-OB16	DO_1	16 digital outputs, Local:4:O.Data.0–15
5	1769-OB16	DO_2	16 digital outputs, Local:5:O.Data.0–15
6	1769-OB16	DO_3	16 digital outputs, Local:6:O.Data.0–15
7	1769-IF4	AI_1	4 analog inputs, Local:7:I.Ch0Data–Ch3Data
8	1769-OF4	AO_1	4 analog outputs, Local:8:O.Ch0Data–Ch3Data
9	1769-IF4	AI_2	4 analog inputs, Local:9:I.Ch0Data–Ch3Data

Slot	Module	Name	Points
10	1769-IQ16	DI_4	16 digital inputs, Local:10:I.Data.0–15
11	1769-IF4	AI_3	4 analog inputs, Local:11:I.Ch0Data–Ch3Data
12	1769-IQ16	DI_5	16 digital inputs, Local:12:I.Data.0–15

That is 80 digital inputs, 48 digital outputs, 12 analog inputs and 4 analog outputs. When you place a part in the sandbox, each of its signals takes the next free point of the right kind; the Scene sheet shows how many are in use. If a kind runs out, the part is placed but the signal has no address, and the status bar says so.

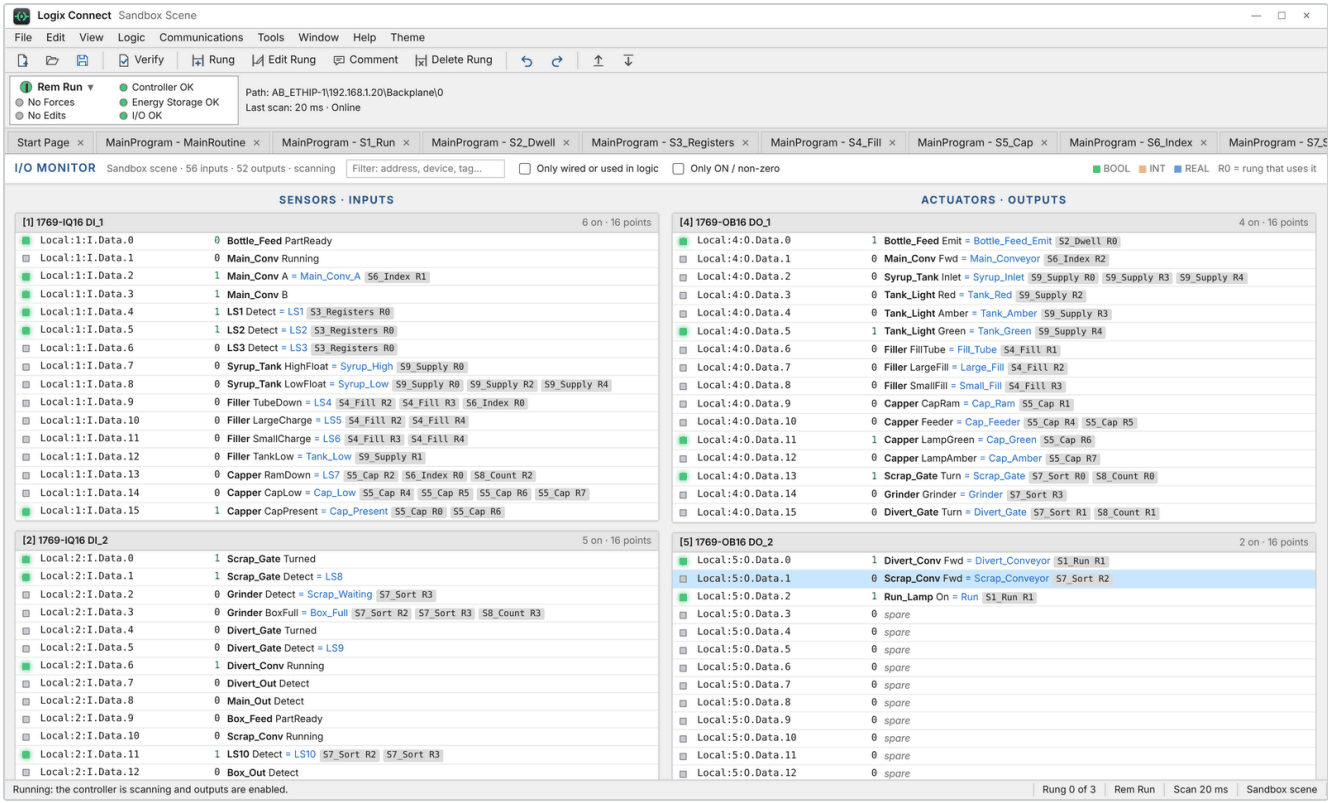


Figure 3. The I/O Monitor: every point of every module, live, with the device and signal wired to it.

Two kinds of project

A **level** is a graded puzzle on a machine built from the sandbox parts, with a score. The **sandbox** is the open floor: you choose the parts, the I/O and the program. This book is about the sandbox, because that is where you learn to build a machine and not just drive one; the levels are a good place to practise the instructions once you know them.

2 The workbench: a tour

Open the page and the splash gives way to the workbench. It is laid out like Logix Designer because the habits transfer: the **Controller Organizer** on the left, document tabs across the middle, the Errors / Simulator Log / Search Results / Cross Reference / Watch panes at the bottom, and a status bar under everything.

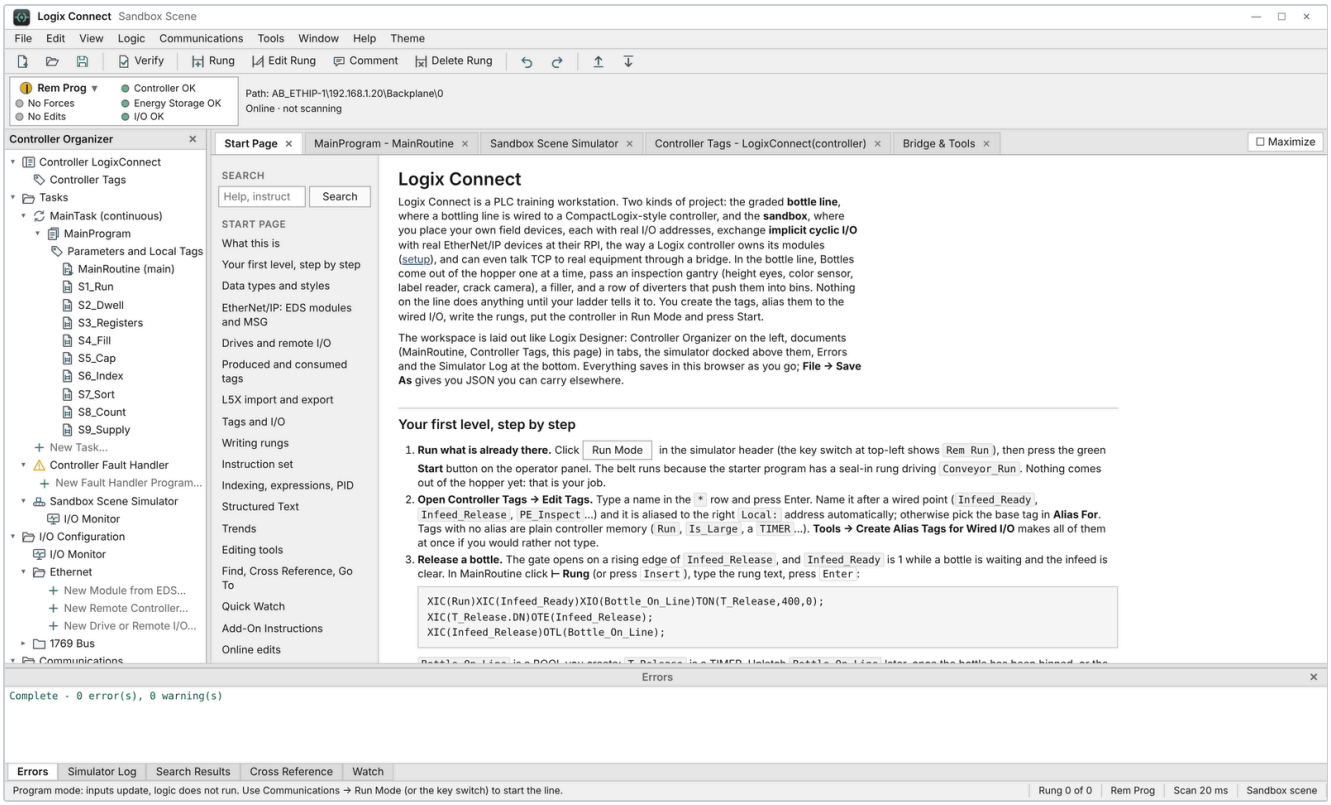


Figure 4. The workbench with the bottle-line project open: menus and toolbar, the key switch and controller status, the Controller Organizer, the document tabs (Start Page showing), the Errors pane and the status bar.

The title bar and the key switch

Top-left is the controller status: the key switch (*Program*, *Run*, *Rem Run*, *Test*), Forces, Edits, Controller OK, Energy Storage OK, I/O OK, and the path (`AB_ETHIP-1\192.168.1.20\Backplane\0`). The same switch is on the simulator's own header: **Running — click for Program Mode** and back. In Program mode inputs still update (you can watch a sensor change in Monitor Tags) but the ladder does not run and outputs are off. In Run mode the ladder scans and outputs drive the machine. Test mode scans without energising outputs.

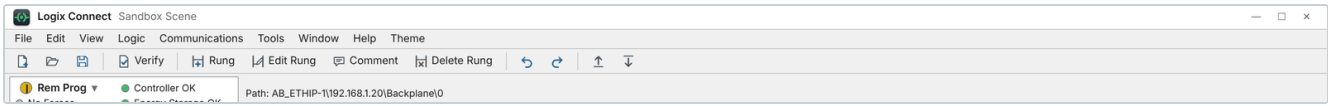


Figure 5. The title bar, menus, the rung toolbar and the controller status block with the key switch and the path.

The menus

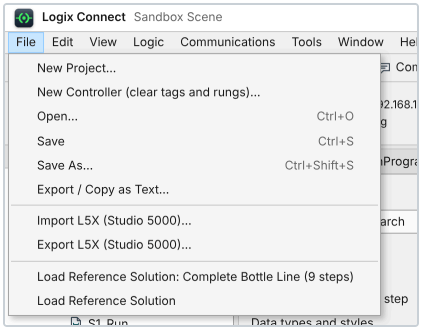


Figure 6. File

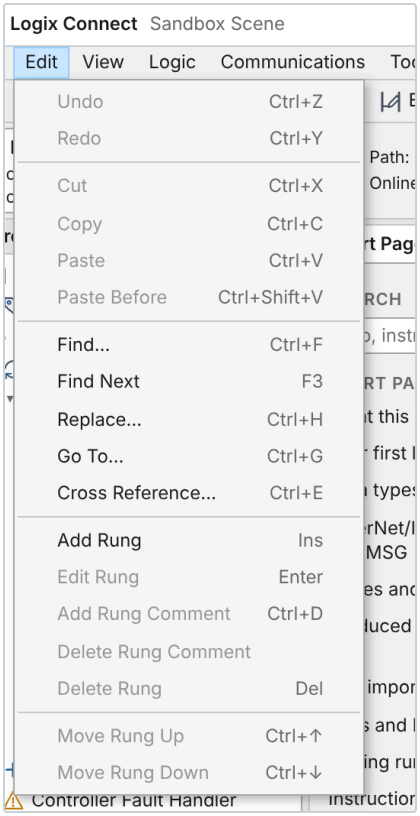


Figure 7. Edit

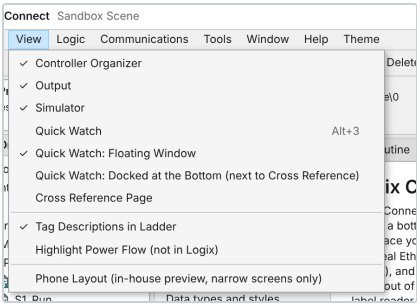


Figure 8. View

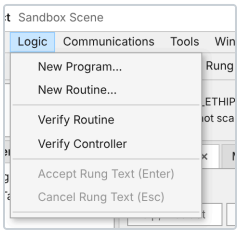


Figure 9. Logic

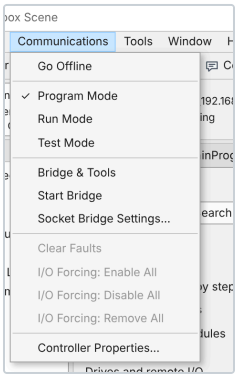


Figure 10. Communications

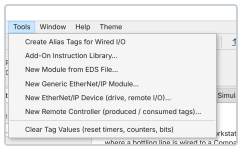


Figure 11. Tools

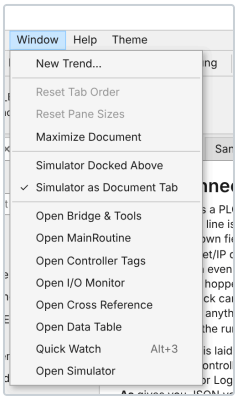


Figure 12. Window

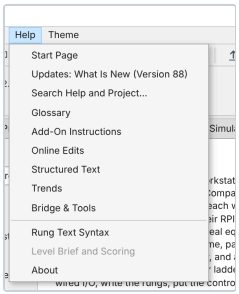


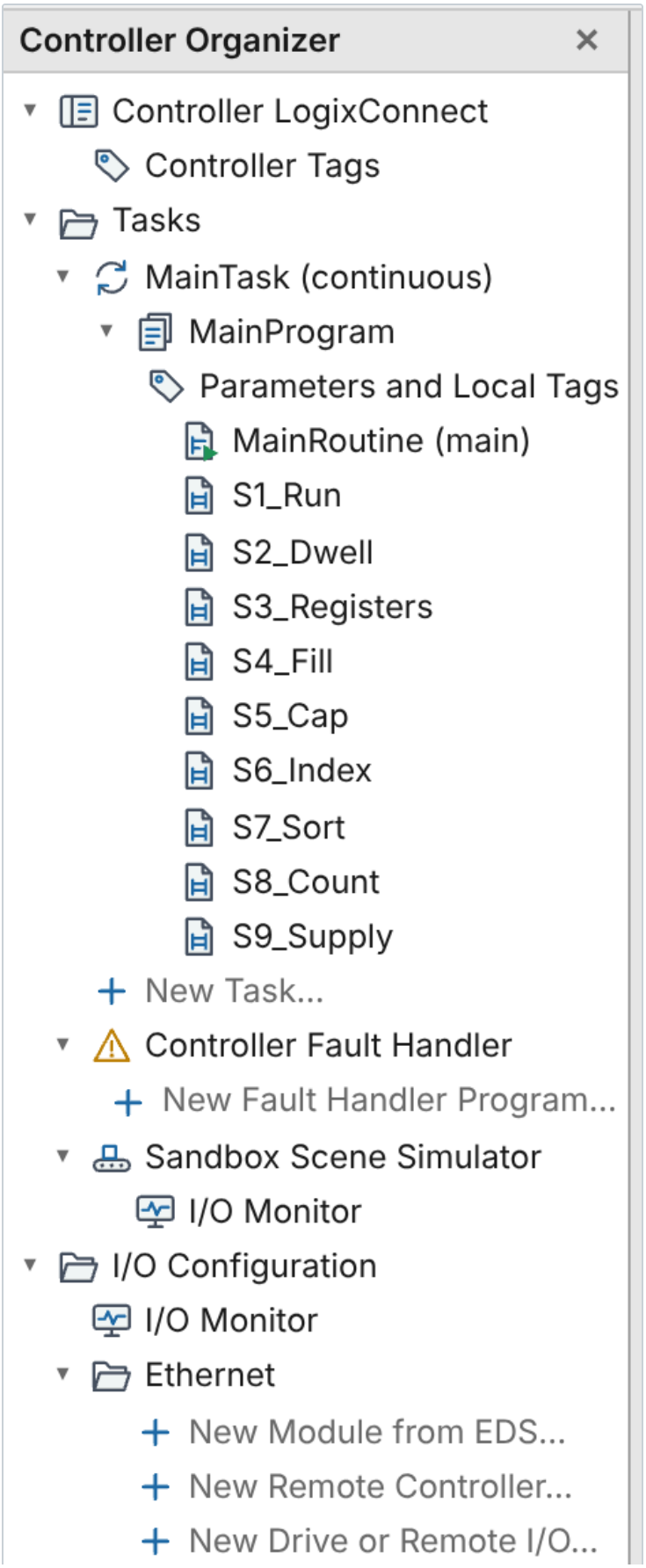
Figure 13. Help

- **File** — *New Project…, New Controller (clear tags and rungs)…, Open…, Save, Save As…, Export / Copy as Text…, Share Link…, Open Link…, Import L5X (Studio 5000)…, Export L5X (Studio 5000)…, Load Reference Solution: Complete Bottle Line (13 steps), Load Reference Solution.*
- **Edit** — *Undo, Redo, Cut, Copy, Paste, Paste Before, Find…, Find Next, Replace…, Go To…, Cross Reference…, Add Rung, Edit Rung, Add Rung Comment, Delete Rung Comment, Delete Rung, Move Rung Up, Move Rung Down.*
- **View** — *Controller Organizer, Output, Simulator, Quick Watch, Quick Watch: Floating Window, Quick Watch: Docked at the Bottom (next to Cross Reference), Cross Reference Page, Tag Descriptions in Ladder, Highlight Power Flow (not in Logix), Phone Layout (in-house preview, narrow screens only).*
- **Logic** — *New Program…, New Routine…, Verify Routine, Verify Controller, Accept Rung Text (Enter), Cancel Rung Text (Esc).*
- **Communications** — *Go Offline, Program Mode, Run Mode, Test Mode, Bridge & Tools, Start Bridge, Socket Bridge Settings…, Clear Faults, I/O Forcing: Enable All, I/O Forcing: Disable All, I/O Forcing: Remove All, Controller Properties….*
- **Tools** — *Inject Machine Fault…, Clear Machine Faults, Create Alias Tags for Wired I/O, Add-On Instruction Library…, New Module from EDS File…, New Generic EtherNet/IP Module…, New EtherNet/IP Device (drive, remote I/O)…, New Remote Controller (produced / consumed tags)…, Clear Tag Values (reset timers, counters, bits).*
- **Window** — *New Trend…, Reset Tab Order, Reset Pane Sizes, Maximize Document, Simulator Docked Above, Simulator as Document Tab, Open Bridge & Tools, Open MainRoutine, Open Controller Tags, Open I/O Monitor, Open Operator View, Open Cross Reference, Open Data Table, Quick Watch, Open Simulator.*
- **Help** — *Start Page, Guided Mode: Build the Bottle Line, Updates: What Is New (v1.47), Search Help and Project…, Glossary, Add-On Instructions, Online Edits, Structured Text, Trends, Bridge & Tools, Rung Text Syntax, Level Brief and Scoring, About.*
- **Theme** — *Logix Connect (default), Logix Designer Blue, Graphite (control room), Holiday splash: automatic (Halloween Oct 24–31), Holiday splash: Halloween always, Holiday splash: off, Preview the Halloween splash, 3D graphics: High (ambient occlusion, glow on lamps, fine shadows), 3D graphics: Medium (textured surfaces, sky, sharp shadows), 3D graphics: Low (the light renderer: phones, older PCs), 3D graphics: Off (Top-down and Side only), 3D graphics: step down by itself when slow.*

The Controller Organizer

The tree on the left is the project. **Controller Tags** opens the tag editor. **Tasks → MainTask → MainProgram** holds the routines; the main routine is marked *(main)* and is the only one the controller calls by itself — every other routine runs only when a JSR in a running routine reaches it. Right-click a program for *New Routine…*, right-click a routine to rename it, set it as main or delete it. **Sandbox Scene Simulator**

and **I/O Monitor** open the machine and the live chassis. **I/O Configuration → Ethernet** is where EtherNet/IP devices (a vision camera from its EDS file, a drive, remote I/O, another controller's produced tags) are added. **Add-On Instructions**, **Trends**, **Data Types** and the **Data Table** are below.


Controller Organizer
Controller LogixConnect
Controller Tags
Tasks
MainTask (continuous)
MainProgram
Parameters and Local Tags
MainRoutine (main)
S1_Run
S2_Dwell
S3_Registers
S4_Fill
S5_Cap
S6_Index
S7_Sort
S8_Count
S9_Supply
New Task...
Controller Fault Handler
New Fault Handler Program...
Sandbox Scene Simulator
I/O Monitor
I/O Configuration
I/O Monitor
Ethernet
New Module from EDS...
New Remote Controller...
New Drive or Remote I/O...

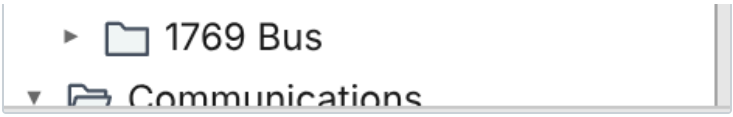


Figure 14. The Controller Organizer: Controller Tags, the task and program with its routines (MainRoutine is main), the simulator and I/O Monitor, the I/O Configuration tree, Communications, Add-On Instructions, Watch, Trends, Data Types and Help.

Document tabs

The **Start Page** is where you begin (four ways in, the levels with your best scores, what is new) and the built-in help (this book is its long form). A **routine** tab is the ladder editor. **Controller Tags** has Monitor and Edit views. The **Sandbox Scene Simulator** is the machine. **Bridge & Tools** is the link to real hardware. Tabs can be dragged into any order (Window → Reset Tab Order puts them back) and the simulator can be docked above the ladder (Window → Simulator Docked Above) so you can watch the machine and the rungs at once — the arrangement most people settle on.

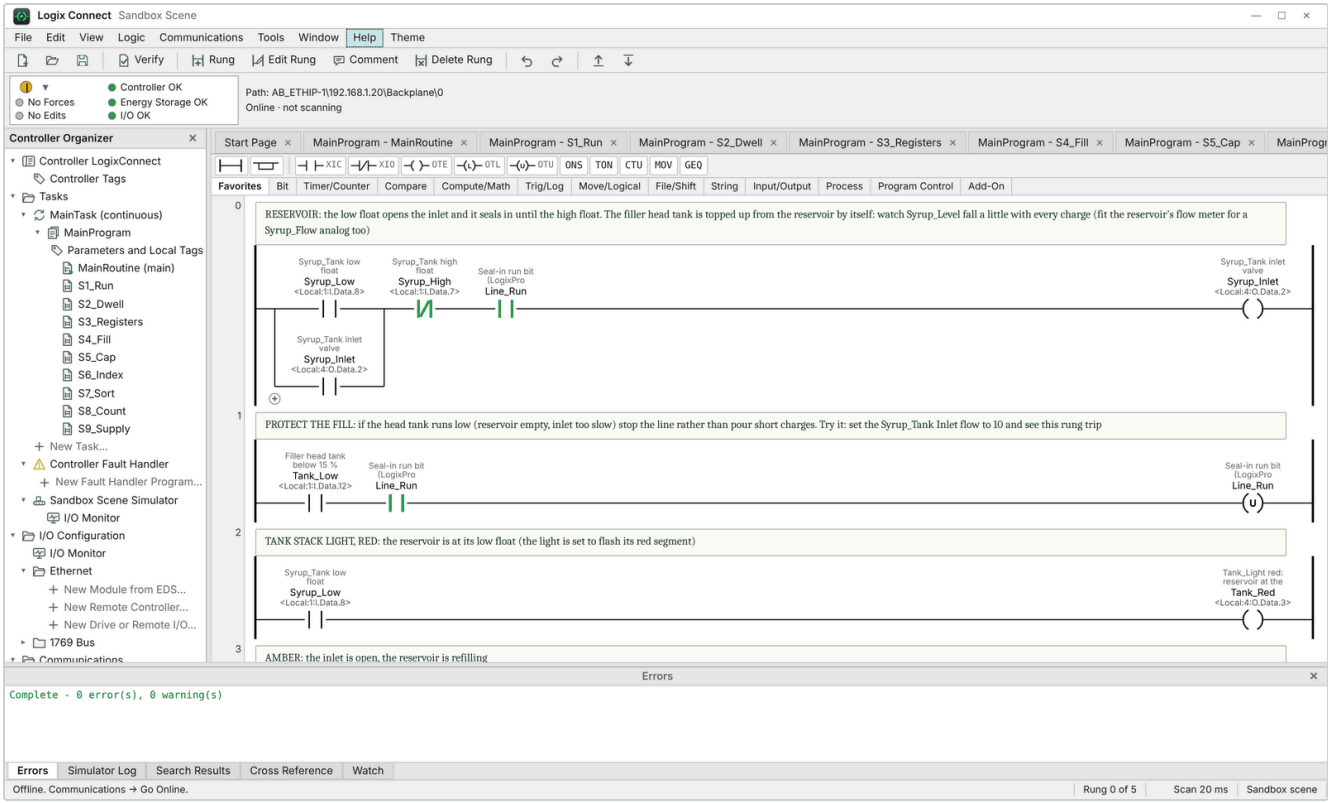


Figure 15. A routine tab with the simulator docked above it (Window → Simulator Docked Above): the machine and the rungs in one view.

The simulator toolbar

Figure 16. The simulator toolbar: mode toggle, Speed, Clear Parts, Reset Sim, Zoom, Floor, View, Labels, the three drawers, and the counters on the right.

Left to right: the Running/Program toggle; **Speed** (½× to 4×: slow the machine to watch a gate, speed it up to fill a box); **Clear Parts** (removes every part, zeroes the simulator's own counters, leaves the program's tags alone); **Reset Sim** (starts over: parts gone, every device back to its starting state — tank levels, cap chute, revolver drum, counters, timers — and every controller tag back to its initial value, in Program mode, with the scene and program untouched); **Zoom**; **Floor** (1600 × 520 up to 4000 × 800 px; the floor grows headroom by itself when a tall device sits near its top edge, and fills the window to the right and below); **View** (Top-down for laying out, Side for watching fills and caps, 3D to walk the line; on a phone 3D is off until you turn it on under Theme → 3D graphics); **Labels** (name tags shown, or hidden until you hover). **Palette**, **Properties** and **I/O** open the three side drawers; **?** opens the help for the selected part.

On the right are the counters: **Emitted** (every part the feeders have made), then three counters that read *program tags* — `Large_Bottle_Count`, `Small_Bottle_Count` and `Scrap_Bottle_Count` by default; click one to point it at any DINT tag, and a tag that does not exist shows a dash — then **Total processed** (parts that reached a remover, a bin or the shredder), with **lost** in red only when parts have fallen off the line with nothing to land on, and the simulated clock.

The bottom panes

Errors is where Verify reports (*Complete - 0 error(s), 0 warning(s)* is what you want to see). The **Simulator Log** is the machine's own record: every part made, filled, capped, scrapped, diverted, lost or spilled, with the device that did it and the simulated time — read it before you read your ladder when something goes wrong. **Search Results**, **Cross Reference** (every place a tag is used, click to go there) and **Watch** (Quick Watch: a short list of tags with live values) complete the set.

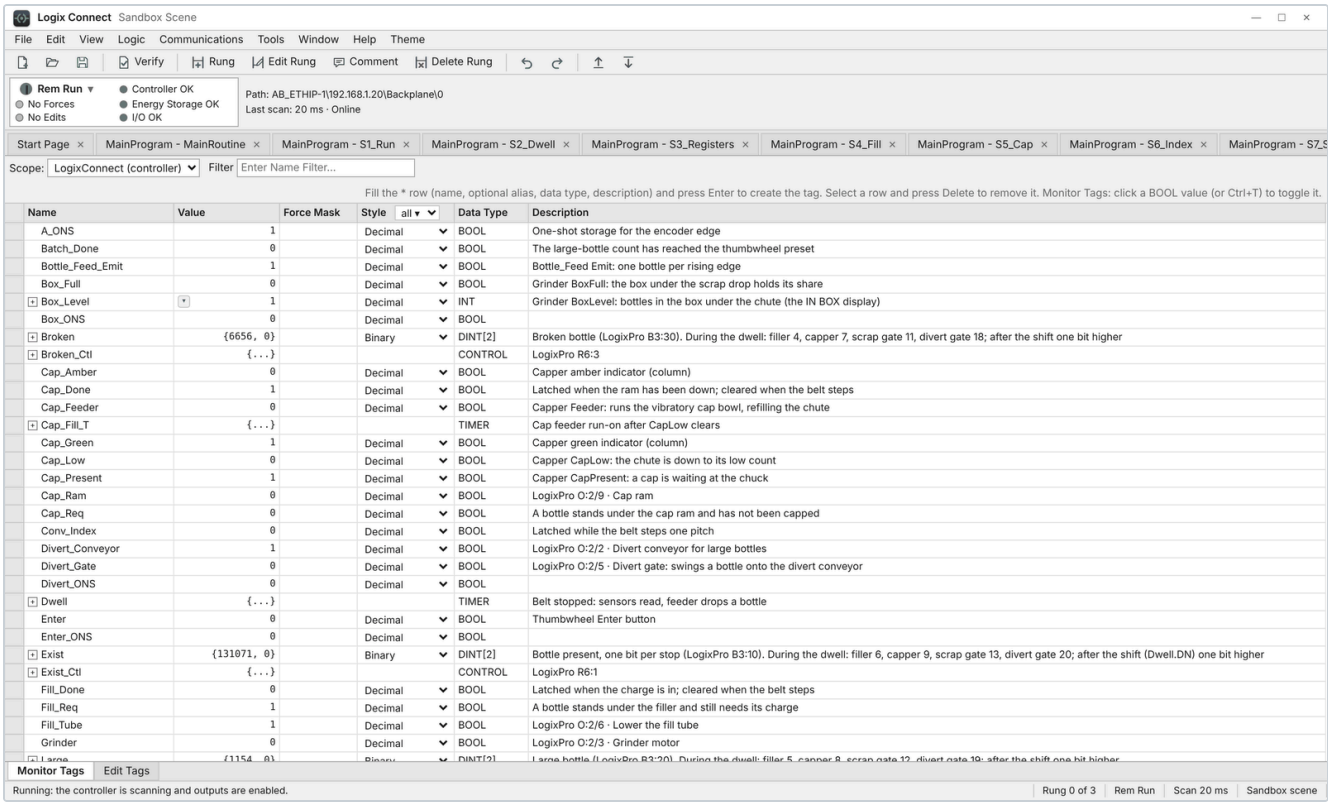


Figure 17. Controller Tags in the Monitor view while the line runs, with a DINT expanded to its bits and timers showing their accumulators.

On a phone

The phone layout has a bottom bar (Start, Ladder, Tags, Sim, Output, Run) and the simulator's own bar (Add Device, Select Several, I/O, Fit, Scene). The **Scene** sheet carries what the small screen has no toolbar room for: View (Top-down / Side), Floor, Name tags, Background, Clear Parts and Reset Sim. 3D is a desktop view; a scene saved in 3D opens Top-down on a phone.

3 Projects: new, save, open, export

File → New Project… offers these starts: an **empty project** (a bare floor and an empty routine, where Part IV begins), **Guided: build the bottle line** (the same, with the lesson panel), **a graded level** (Part V; the list shows your best score on each and which ones are in progress), a **sample** scene, and the **Sandbox: LogixPro-style Bottle Line** — the finished machine of Part IV with its thirteen-step program already written, mix kitchen included. The last one is your answer key: build from the empty floor, and when a lesson will not behave, open the finished line in a second tab and compare. When you come back to the app it also offers **Continue where I left off**, your last file, and **My sandbox project** when you have put one aside to play a level.

Nothing is thrown away when you switch. Each level keeps its own floor and program, so opening Level 3 and then Level 5 and back finds Level 3 as you left it; the sandbox project you were on when you opened a level is kept as *My sandbox project*. Programs written for the old fixed-machine levels (before Version 99) are carried onto the level's sandbox parts the first time you open it.

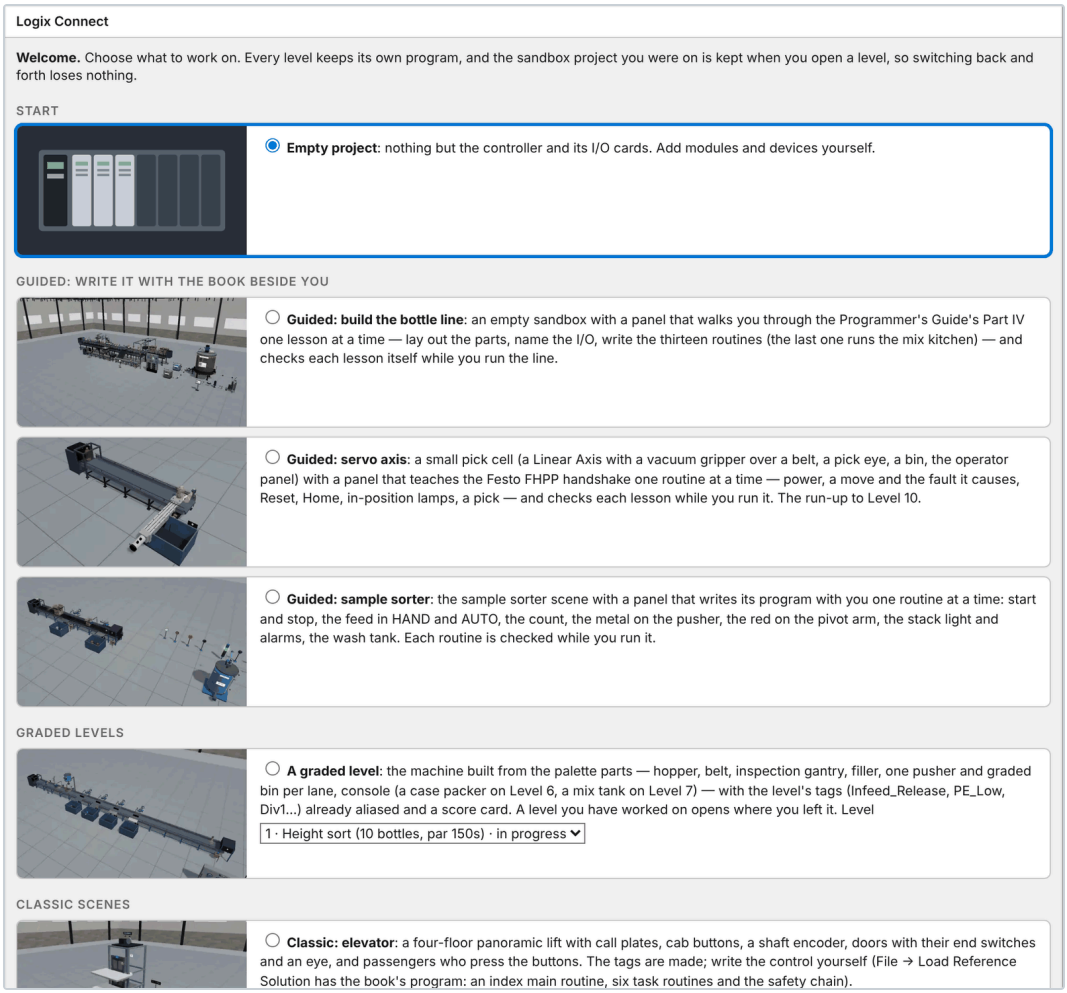


Figure 18. File → New Project...: every project as a card with a picture of it running, grouped: start empty, the guided tracks, the graded levels (the picture follows the level you pick), the classic scenes and the sandboxes.

Save writes the whole project — scene, tags, routines, panel layout, trends — as one JSON file; **Open…** reads it back. **Export / Copy as Text…** gives the program as rung text you can paste anywhere, which is also how this book's listings were made. **Export L5X** writes a Studio 5000 import file with your tags and routines; **Import L5X** reads one, including Add-On Instructions. **New Controller** clears tags and rungs but keeps the scene — the fastest way to start Part IV's programming lessons on a machine you have already built.

The page also keeps your work between visits by itself (the browser's storage); Save is for a copy you can carry, send or keep.

WATCH FOR

The two *Load Reference Solution* entries replace your program. *Complete Bottle Line (13 steps)* installs Part IV's thirteen routines on the LogixPro-style scene and adds any part the floor is missing, the mix kitchen too; the plain one installs the older single-routine answer. Save first if you have rungs you want to keep.

Part II — Ladder logic from zero

4 How a controller thinks: the scan and the rung

A relay panel is wired; a controller is scanned. Every scan — here every 20 ms — the controller does three things in order: it copies the state of every input point into memory, it solves every rung of the running routines from the top down, and it writes the result of every output rung to the output points. Nothing happens between scans. A sensor that flickers for 2 ms between two scans is never seen; an output written by rung 3 and overwritten by rung 9 is only ever what rung 9 said. Hold on to those two facts and most ladder mysteries dissolve.

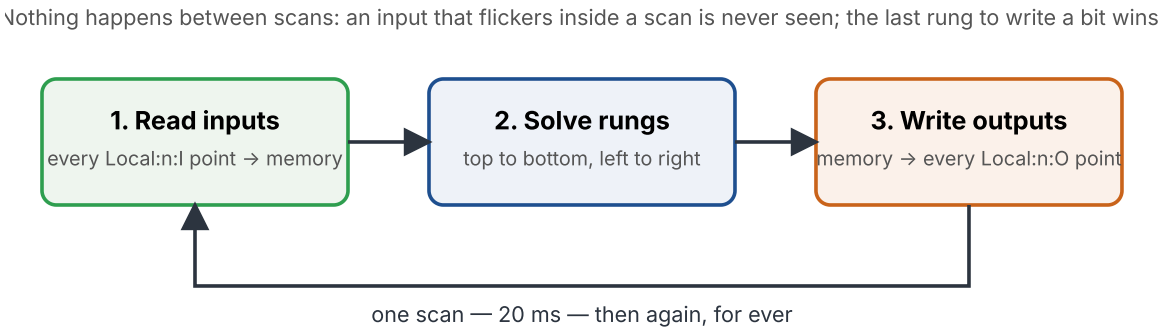


Figure 19. The scan. The controller never "watches" an input; it looks once per scan, decides everything, then writes everything.

Reading a rung

A rung is a path from the left power rail to the right. Instructions on the left (*inputs*, or conditions) either pass power or block it; the instruction on the right (*output*) does something when power reaches it. In Logix Connect you write the rung as text, exactly the way Studio 5000 copies a rung to the clipboard:

```
XIC(Start_PB)XIO(Stop_PB)OTE(Motor);
```

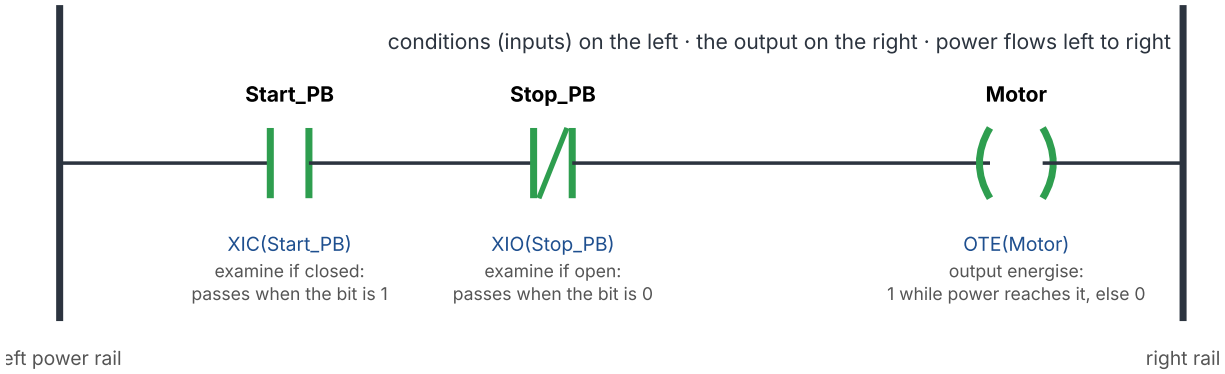


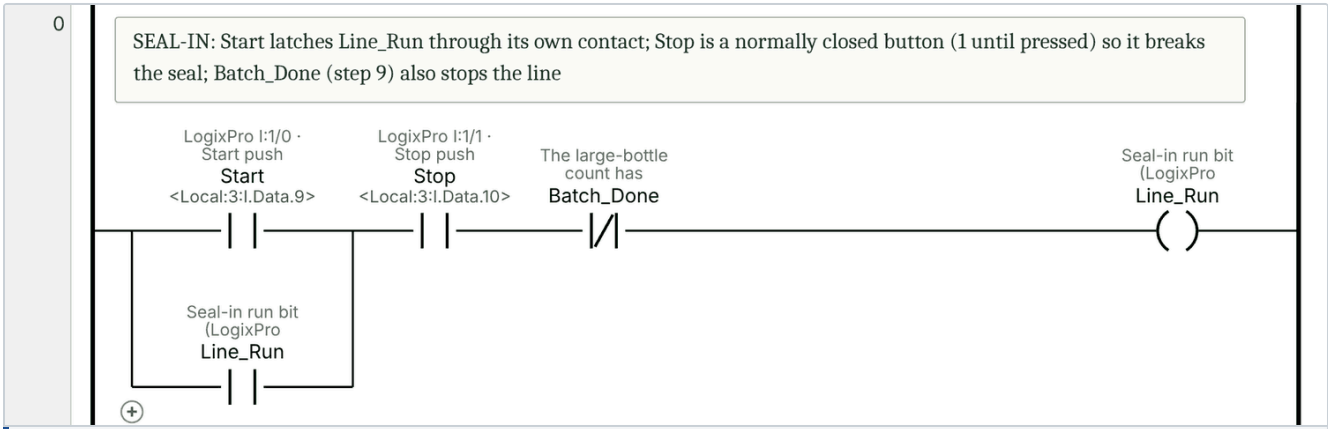
Figure 20. Anatomy of a rung, and the text you type for it: `XIC(Start_PB)XIO(Stop_PB)OTE(Motor);`

Read it left to right: *examine Start_PB if closed* (true when the tag is 1), *examine Stop_PB if open* (true when the tag is 0), *output energise Motor*. Instructions written one after another are in series: all must be true. A branch — instructions that can each complete the path — is written in square brackets with a comma between the legs:

The classic seal-in: Start or Motor itself, and not Stop, keeps Motor on

```
[XIC(Start_PB),XIC(Motor)]XIO(Stop_PB)OTE(Motor);
```

The editor draws it like this (this is the real one from Part IV's step 1, with Stop normally closed and a batch-done condition added; the tag descriptions are printed above the contacts):




```
[XIC(Start),XIC(Line_Run)]XIC(Stop)XIO(Batch_Done)OTE(Line_Run);
```

That rung is the most important one in the book. Press Start: the left leg is true, Motor comes on. Release Start: the right leg, `XIC(Motor)`, is now true because Motor is on, so the rung stays true — the output seals itself in. Press Stop: `XIO(Stop_PB)` goes false, Motor drops, and both legs are now false. Every *Run* bit on every machine you will ever program is this rung.

Entering rungs

Open a routine. Click ⊢ **Rung** (or press **Insert**): a text field opens for the new rung. Type the rung and press **Enter**; **Esc** cancels. Click a rung and choose **Edit Rung** (or double-click) to change it. Click an operand to give it a value: the box takes a tag name or a number; type a number and the top row of the list reads *Use 1600*, so a tap, Enter or the phone keyboard's return or Done puts it in. To change one instruction for a relative, select it: a *change to* bar under it offers the others of its family (XIC and XIO; OTE, OTL and OTU; TON, TOF and RTO; CTU and CTD; the compares; the math), and the operands stay. Right-click gives the same choices. A branch can go without what is on it: select the branch by its left bar and choose *Remove branch, keep instructions*, and the instructions on its legs stay on the rung in series. Click a leg's wire to select that leg alone; *Delete leg* (or Del) takes the leg and what is on it, and a branch left with one leg becomes plain series logic. With a leg selected, Ctrl+V pastes the copied instruction at the end of that leg and Ctrl+Shift+V at its start; right-click the leg for the same. **Comment** puts a note above the rung — use it; the lessons in Part IV comment every rung and you should too. **Delete Rung**, **Move Rung Up / Down** and the arrow buttons reorder. The semicolon at the end is the rung terminator; the editor adds it if you forget.

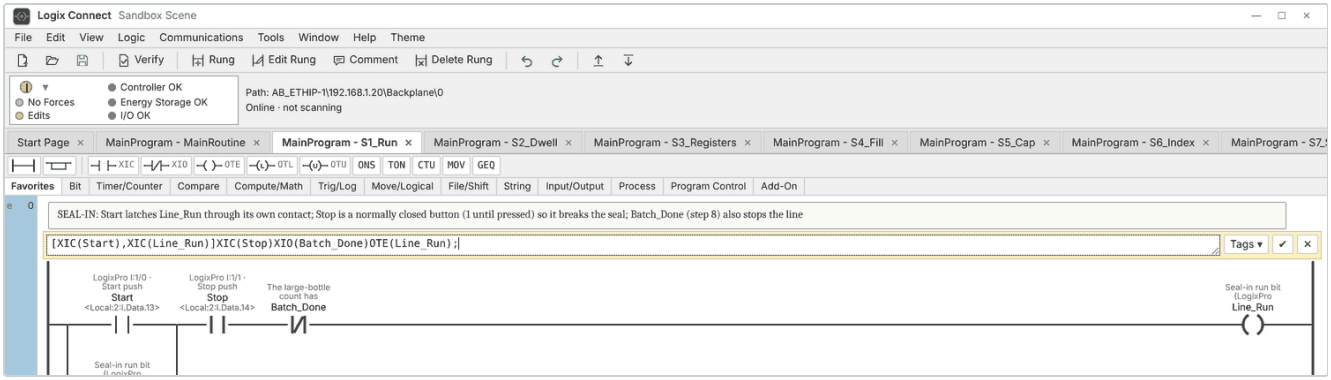


Figure 21. Editing a rung: the text field opens under the rung with Tags ▾ (browse tags for the operand at the cursor), accept and cancel. The rung draws itself as you type.

> **TRY IT**
>
> Enter `[XIC(Start_PB),XIC(Motor)]XIO(Stop_PB)OTE(Motor);`. Click the `OTE` and choose **OTL** on the *change to* bar, then click the lower leg's wire and press **Del**: the rung reads `XIC(Start_PB)XIO(Stop_PB)OTL(Motor);`. Ctrl+Z twice brings the seal-in back. Now select the branch by its left bar and choose *Remove branch, keep instructions*: both contacts stay, in series, and the rung needs Start *and* Motor to turn Motor on, so it never will.

Logic → Verify Routine (or just Verify on the toolbar) checks every rung: unknown tags, wrong parameter counts, a tag used as the wrong type. Errors appear in the Errors pane with the rung number. A routine that does not verify does not run. Tags you name in a rung before creating them are reported as unknown — create them (chapter 5) and verify again.

Rung text: the syntax on one page

Keep this open while you type. The editor accepts exactly what Studio 5000 puts on the clipboard when you copy a rung, so everything here transfers.

You want	You type	Notes
an instruction	`XIC(Start)`	mnemonic in capitals, parameters in brackets, no spaces needed
instructions in series (AND)	`XIC(A)XIC(B)OTE(C);`	one after another; all must be true
a branch (OR)	`[XIC(A),XIC(B)]OTE(C);`	square brackets around the legs, a comma between legs
a leg with several instructions	`[XIC(A)XIC(B),XIC(C)]OTE(D);`	A and B, or C
a branch inside a branch	`[XIC(A) [XIC(B),XIC(C)],XIC(D)]OTE(E);`	nest as deep as you need; count your brackets
two outputs on one rung	`XIC(A)[OTE(B),OTE(C)];`	an output branch: both energised together
a bit of a word	`XIC(Word.3)` `XIC(Exist[0].12)`	dot, then the bit number
a member of a structure	`XIC(Dwell.DN)` `GEQ(Dwell.ACC,500)`	timers: .PRE .ACC .EN .TT .DN; counters: .CU .DN; control: .DN .UL .POS
an array element	`MOV(Vals[2],Out)`	index in square brackets; a tag used as the index is fine
a number	`GEQ(Count,20)` `TON(T1,1500,0)`	plain digits; a decimal point makes it REAL
a timer / counter	`TON(Dwell,1000,0)` `CTU(Boxes,10,0)`	tag, preset, accumulator (start value)
a shift register	`BSL(Exist[0],Exist_Ctl,LS1,48)`	array, CONTROL tag, source bit, length
the end of the rung	`;`	the editor adds it if you forget

THE MISTAKES EVERYONE MAKES ONCE

- **Unbalanced brackets.** Every `[` needs a `]`, every `(` a `)`. The editor marks the rung with an *e* and the Errors pane says where.
- **A space in a tag name.** `Start PB` is two things; `Start_PB` is one.
- **The same tag on two OTEs.** Only the last one in the scan counts. Use a branch on one rung, or OTL/OTU.
- **An ONS without its own storage bit.** `ONS(X_ONS)` needs a BOOL used nowhere else; two one-shots sharing a bit both fail.
- **Reading a timer bit of a timer that never resets.** A TON resets when its rung goes false; if the rung is always true, .DN comes on once and stays.
- **Forgetting the JSR.** A routine nobody calls does nothing; Verify warns you.

Online

Switch to Run mode and the ladder is *online*: instructions that are true light green, the two power rails light green to show the rungs are powered, timers and counters show their accumulators, and a change you make to a rung is accepted into the running controller on Enter — like Studio 5000's online edits, with the edited rung marked until you accept it. Values (tag values, timer accumulators) carry over an edit; only Reset Sim or Tools → Clear Tag Values zero them.

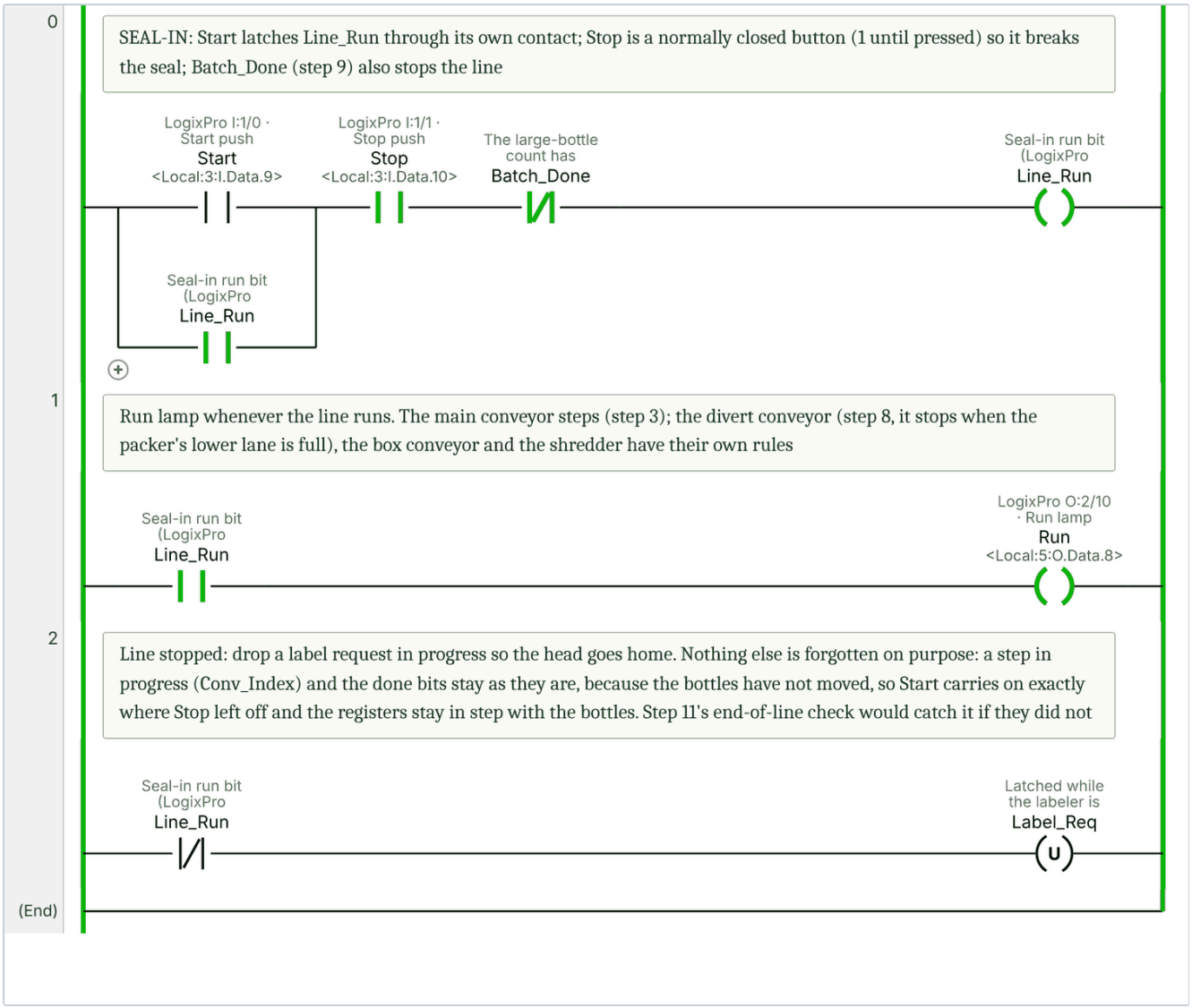


Figure 22. The same routine online, in Run mode: true instructions and energised coils light green; the seal-in is holding Line_Run.

TRY IT

In an empty sandbox place a *Push Button*, a second Push Button with *NC* ticked (its signal is then 1 until pressed, like a real stop button) and an *Indicator Light*. Name them Start_PB, Stop_PB and Lamp. Write the seal-in rung with `Lamp_On` as the output (the alias the light gets by itself, see chapter 5), go to Run mode and press the buttons. Then swap `XIO(Stop_PB_OK)` for `XIC(Stop_PB_OK)` and see why a stop button is wired normally closed.

5 Tags, data types and I/O addresses

A tag is a named piece of controller memory with a type. Logix Connect uses the Logix types you will meet in Studio 5000:

Type	What it holds	Where you use it
`BOOL`	one bit, 0 or 1	every input and output, every latch, every one-shot storage bit
`SINT / INT / DINT`	an 8 / 16 / 32-bit integer	counts, analog values (INT from the modules), shift registers (DINT, one bit per stop)
`REAL`	a floating-point number	scaled analog values, calculations
`TIMER`	a structure: .PRE (preset, ms), .ACC (accumulated, ms), .EN, .TT (timing), .DN (done)	TON / TOF / RTO
`COUNTER`	.PRE, .ACC, .CU, .CD, .DN, .OV, .UN	CTU / CTD
`CONTROL`	.LEN, .POS, .EN, .DN, .EM, .ER, .UL	BSL / BSR / FFL / FFU / SQO
arrays	`Exist[0..1]`: a DINT with Dim 2	a 64-bit shift register, addressed `Exist[0].12` for bit 12

Open **Controller Tags → Edit Tags**. Type a name in the `*` row and press Enter; pick the type; add a description. A description is free and is shown in the ladder when View → Tag Descriptions in Ladder is on — the lessons rely on them. A tag's **style** (Decimal, Binary, Hex) is only how it is displayed; the shift registers in Part IV are Binary so you can watch the bits walk.

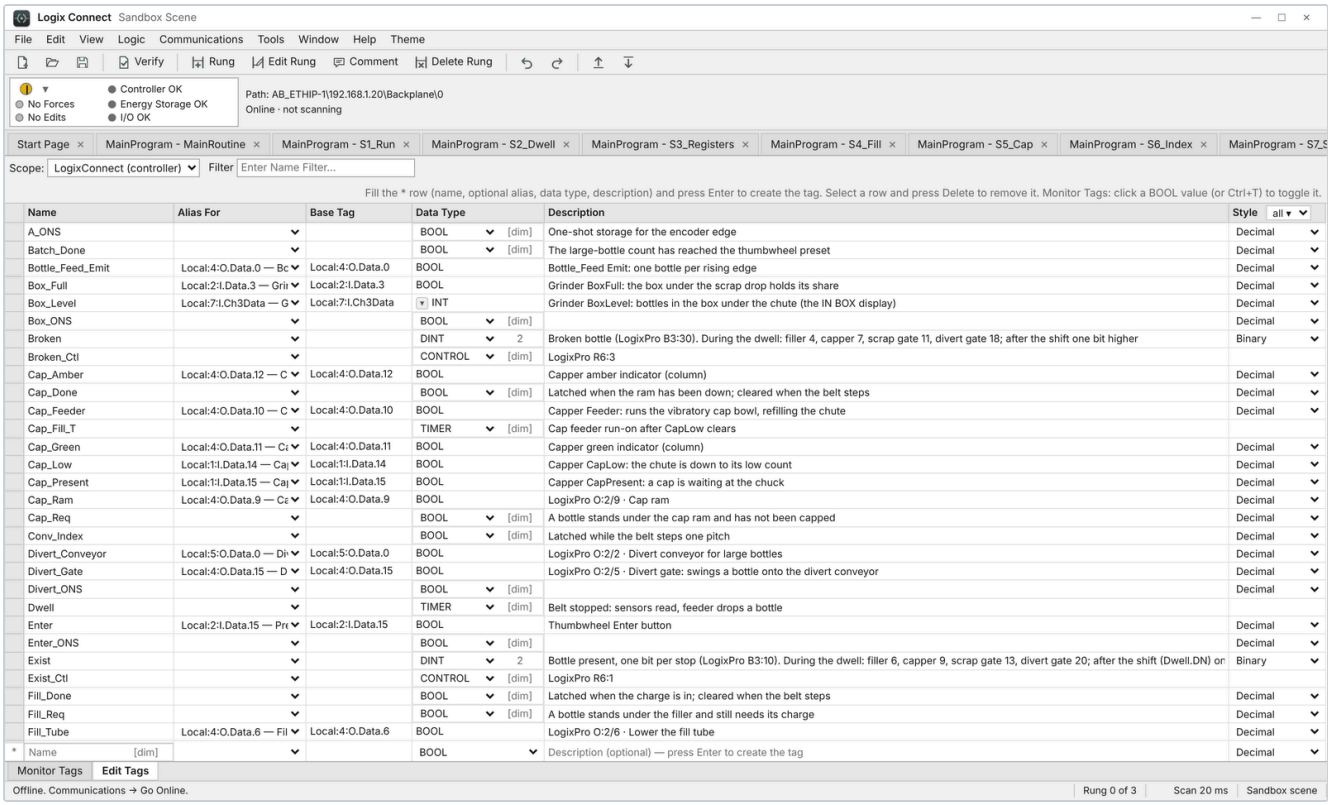


Figure 23. Controller Tags → Edit Tags: name, alias target, data type, style and description. The bottle line's LogixPro-style aliases are at the top.

Base tags: the I/O points

The chassis points exist as tags already: `Local:1:I.Data.0`, `Local:4:O.Data.3`, `Local:7:I.Ch1Data`. You can write them straight into a rung, and the I/O Monitor shows them live, but a program full of slot numbers is unreadable. That is what aliases are for.

Alias tags

An alias is a tag that *is* another tag under a better name: `Start_PB` aliased to `Local:1:I.Data.0`. Reading or writing the alias reads or writes the point. In the tag editor, choose *Alias* and pick the base in *Alias For*.

Logix Connect has a house rule that saves most of that work: **a tag named `Device_Signal` aliases itself.** Place a push button called `Start_PB`; its signal is `OK`; create a tag called `Start_PB_OK` and it is aliased to the button's point automatically. The Properties drawer of every part lists its signals with their addresses and a *tag Device_Signal* link that creates the alias in one click; the Scene sheet's *Alias all I/O* and **Tools → Create Alias Tags for Wired I/O** do it for every signal on the floor at once. Rename a device and its alias tags follow (`Supply_Tank` to `Milk` turns `Supply_Inlet` into `Milk_Inlet` and rewrites every rung).

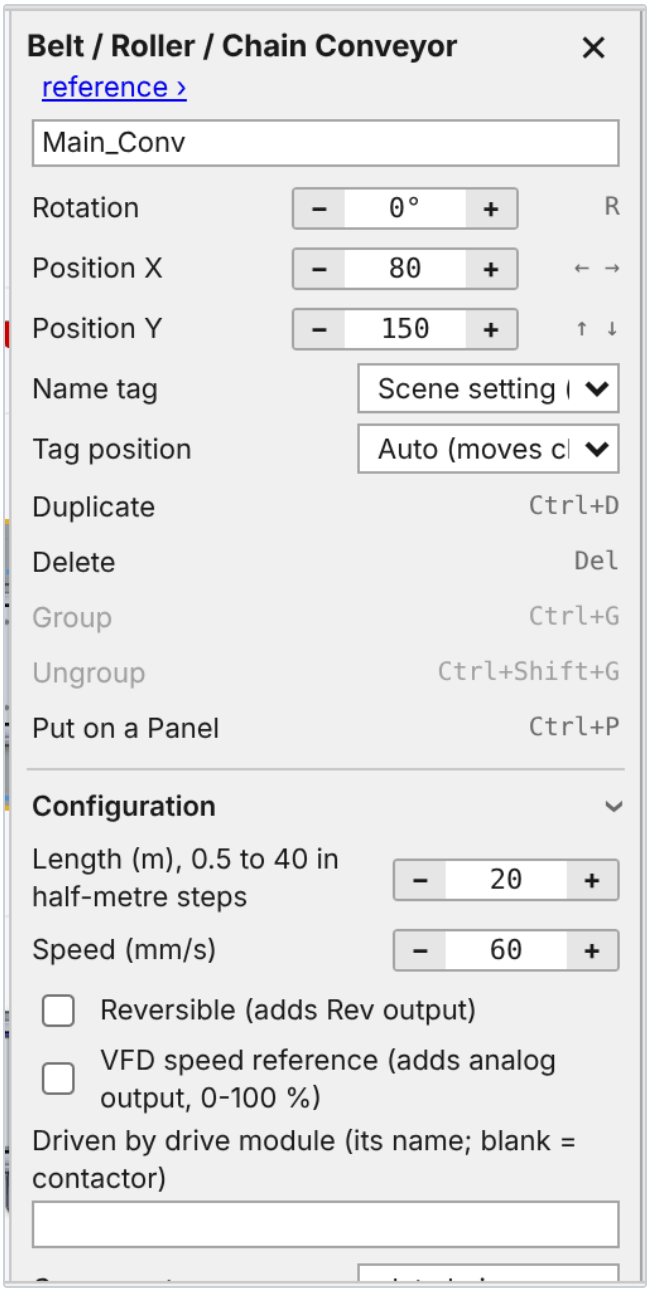


Figure 24. The Properties drawer of LS1: its settings and, under Signals and Addresses, the point each signal has and the tag Device_Signal link that creates the alias.

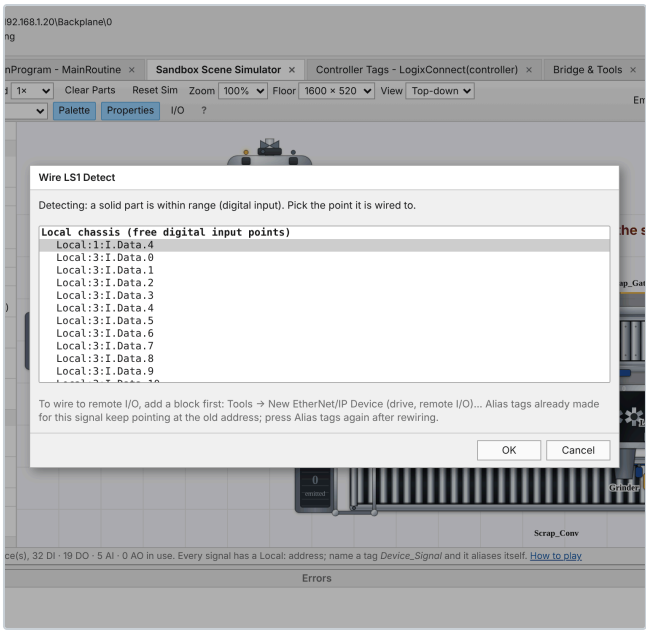


Figure 25. Re-wiring a signal to another point, or binding it straight to a tag of your own.

Part IV uses shorter, LogixPro-style names for the bottle line (`LS1`, `Cap_Ram`, `Supply_Inlet`), made as explicit aliases; the full list is in Appendix A. Either style is fine; the rule is that a rung should read like the machine, not like the chassis.

Bits of a word

Any integer tag can be used a bit at a time: `Exist[0].12` is bit 12 of the first DINT of the array `Exist`. The Controller Tags browser expands a DINT into its 32 bits and shades the ones the program uses; the Data Table (Window → Open Data Table) shows whole words in the radix of your choice, the way RSLogix's data files did.

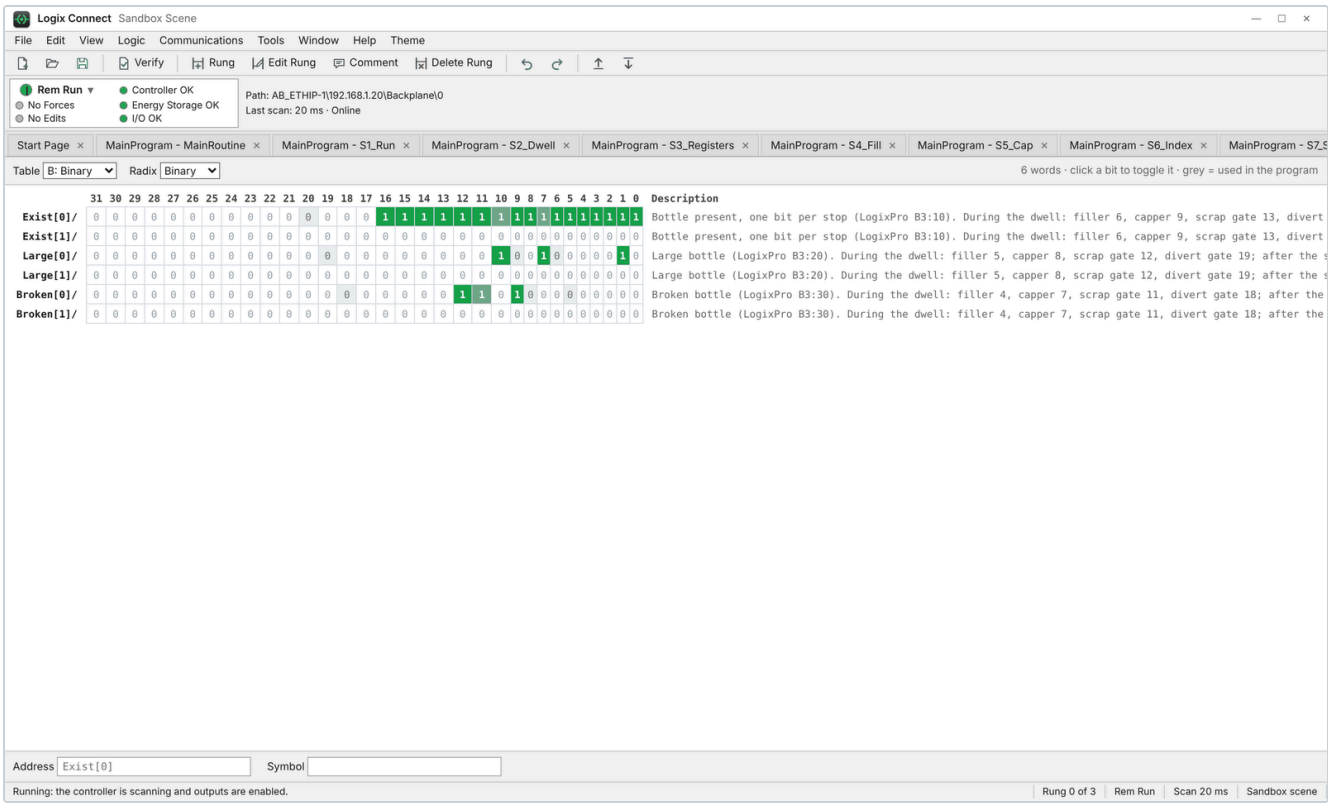


Figure 26. The Data Table (Window → Open Data Table): whole words in the radix of your choice, the way RSLogix data files looked. The shift registers are easiest to read here.

> **WATCH FOR**
>
> Names: letters, digits and underscores, not starting with a digit, up to 40 characters, and no two tags the same ignoring case. `Run` and `run` are the same tag. The editor tidies a device name into that shape when you type one with spaces.

6 Bit instructions

Everything on a machine is, at the bottom, a bit: a sensor sees or does not, a valve is open or shut. These six instructions are the whole language of bits.

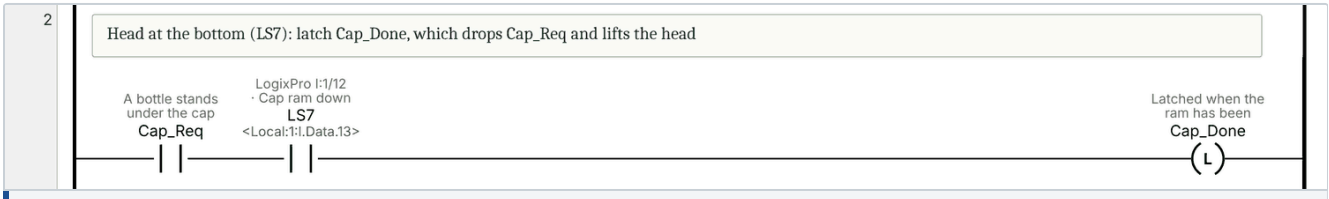
Instruction	Reads	Does
`XIC(tag)`	Examine If Closed	passes power when the bit is 1
`XIO(tag)`	Examine If Open	passes power when the bit is 0
`OTE(tag)`	Output Energise	writes 1 when the rung is true, 0 when it is false — every scan
`OTL(tag)`	Output Latch	writes 1 when the rung is true; leaves it alone when false
`OTU(tag)`	Output Unlatch	writes 0 when the rung is true; leaves it alone when false
`ONS(storage)`	One Shot	passes power for exactly one scan after the rung ahead of it becomes true; the storage bit remembers the previous scan

OTE: the output that follows the rung

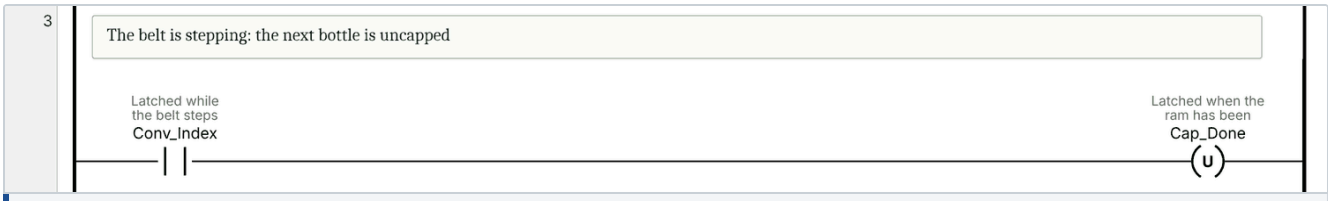
OTE is a mirror. The bit is whatever the rung is, scan after scan. Two rules follow. First, write each bit with OTE in one place only; the last OTE wins and the earlier one is wasted. Second, an OTE rung with a seal-in branch is how you make a bit remember (chapter 4).

OTL / OTU: memory without a seal-in

A latch sets the bit and walks away; nothing turns it off except an unlatch. Use the pair when the set and the reset belong to different events in different places — a request raised by one routine and cleared by another:




```
XIC(Cap_Req)XIC(LS7)OTL(Cap_Done);
```



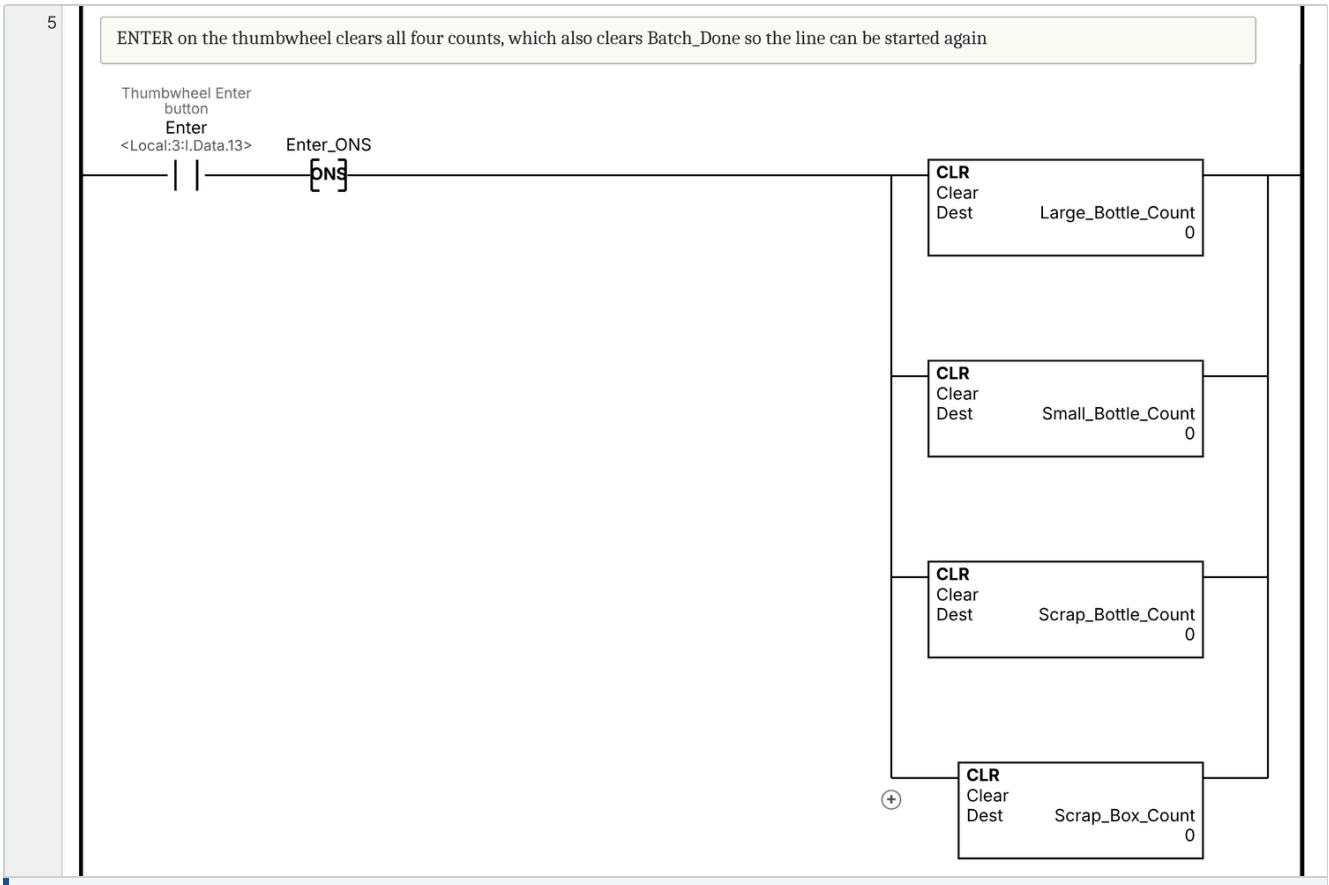

```
XIC(Conv_Index)OTU(Cap_Done);
```

(Part IV, step 7: the chuck reached the bottom, so this bottle has its cap; and when the belt steps, the next bottle is a new bottle.)

Latches survive a power cycle in a real controller and here survive everything short of Reset Sim, which is exactly why they need an unlatch you can find. Put the OTU right after the OTL in the listing when you can, and comment both.

ONS: once, not while

A push button held down is true for fifty scans. If the rung that counts presses were `XIC(PB)ADD(Count,1,Count)`, one press would add fifty. ONS lets the rung through on the first scan only:




```
XIC(Enter)ONS(Enter_ONS)
[CLR(Large_Bottle_Count),CLR(Small_Bottle_Count),CLR(Scrap_Bottle_Count),CLR(Sc
rap_Box_Count)];
```

(Part IV, step 9: one press of Enter clears the counts once, however long it is held.)

The storage bit (`Enter_ONS`) is a BOOL you create and never use anywhere else. OSR and OSF are the same idea as output instructions with their own storage and output bits.

> **TRY IT**
>
> With the button-and-lamp sandbox from chapter 4, add a DINT `Presses` and a BOOL `PB_ONS`. Write `XIC(Start_PB_OK)ONS(PB_ONS)ADD(Presses,1,Presses);` and watch Presses in Quick Watch while you press. Then delete the ONS and press once.

7 Timers and counters

TON, TOF, RTO

A timer tag has a preset (`.PRE`, milliseconds), an accumulator (`.ACC`) and three bits: `.EN` (the rung is true), `.TT` (timing: true while the accumulator is running toward the preset) and `.DN` (done). `TON(Dwell,1000,0)` means: timer `Dwell`, preset 1000 ms, accumulator starts at 0.

Instruction	Behaviour
`TON`	On-delay. Rung true: ACC counts up; at PRE, DN turns on. Rung false: ACC and DN reset. The ordinary "wait a second" timer.
`TOF`	Off-delay. DN turns on the moment the rung is true and stays on for PRE after the rung goes false. A fan that runs on after the heater stops.
`RTO`	Retentive. Like TON but the accumulator holds when the rung goes false and resumes when it comes back; only RES clears it. Run-hour meters, cumulative faults.

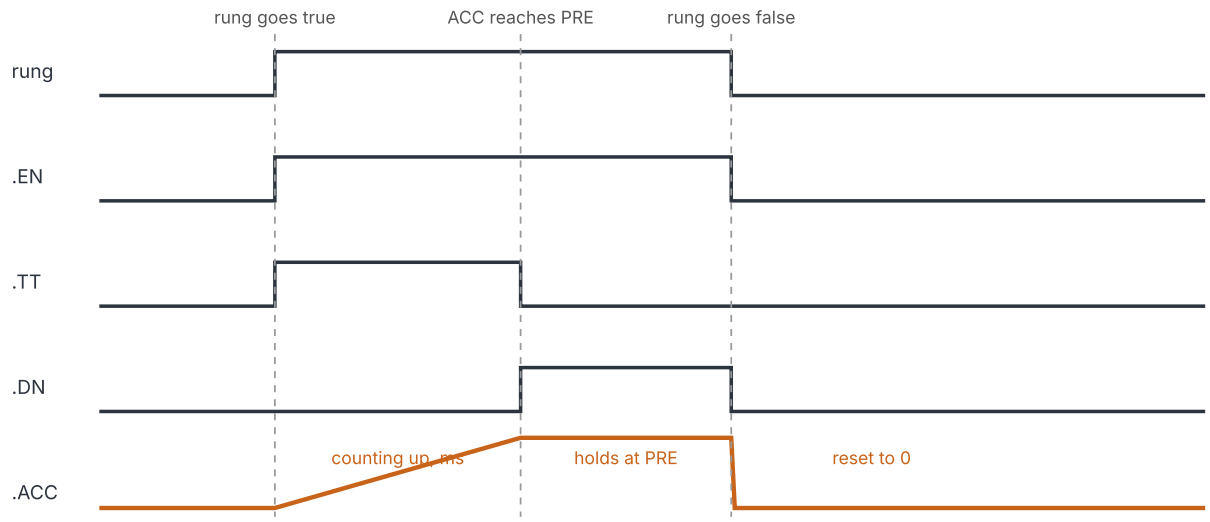
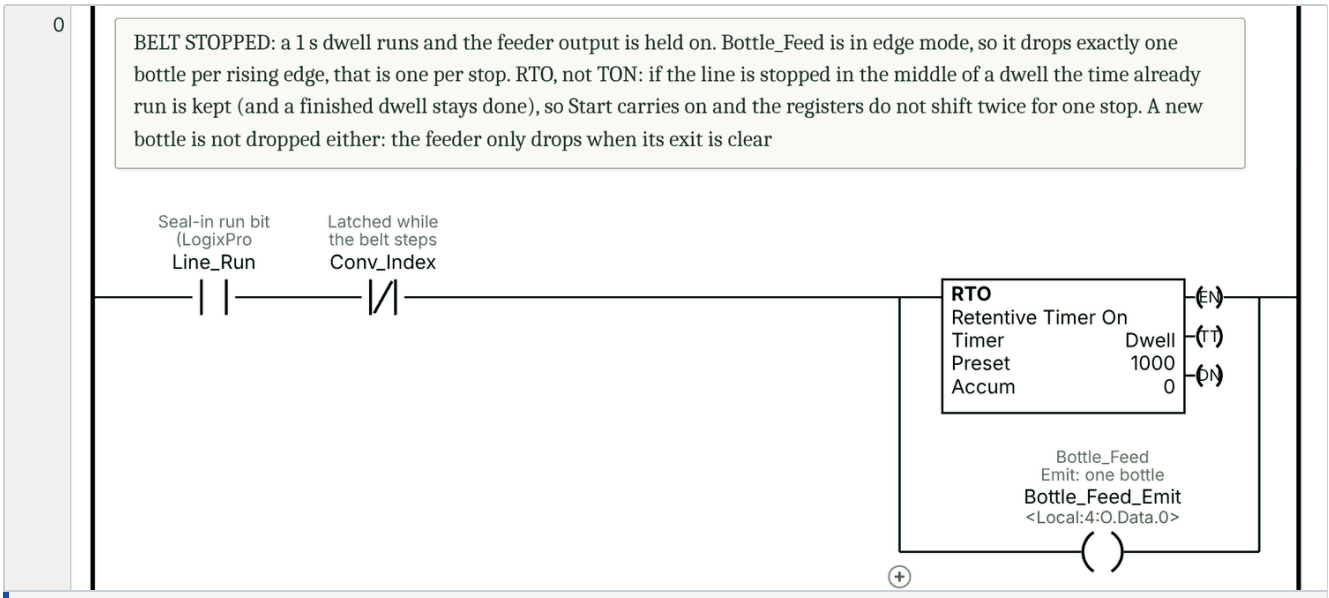


Figure 27. TON timing. .EN follows the rung; .TT is on only while the accumulator is climbing; .DN comes on at the preset and stays on until the rung drops. The dwell rung in Part IV reads .DN to say "the stop is over" and .TT to say "the bottles are standing still".

The timer in Part IV's step 2 is the heartbeat of the whole line:




```
XIC(Line_Run)XIO(Conv_Index)[RTO(Dwell,1000,0),OTE(Bottle_Feed_Emit)];
```

The rung is true whenever the line runs and the belt is not stepping, so the timer runs while the belt stands still; when `Dwell.DN` comes on, step 4 shifts the registers and step 3 steps the belt: `Conv_Index` goes true, the rung goes false, and step 3 resets the timer for the next stop (an RTO keeps its time until a RES clears it). The feeder's `Emit` is on the same rung: it fires on the rising edge, once per stop.

A flasher is two timers chasing each other, or one timer that resets itself:

resets itself the scan after it finishes: DN is true for one scan every 500 ms

```
XIO(Flash.DN)TON(Flash,500,0);
```

and that one scan toggles the lamp — though a latch toggled on one branch and unlatched on the next needs the ONS, or both fire

```
XIC(Flash.DN)ONS(F_ONS)[XIO(Lamp)OTL(Lamp),XIC(Lamp)OTU(Lamp)];
```

(An easier flasher: `XIO(T_Off.DN)TON(T_On,500,0); XIC(T_On.DN)TON(T_Off,500,0); XIC(T_On.TT)OTE(Lamp);` — on while T_On times, off while T_Off times. Stack lights in the sandbox also have a *flash* setting that does this for you.)

CTU, CTD, RES

A counter counts rung transitions, false to true, into `.ACC`; `.DN` comes on at `.PRE`. `CTU(Boxes,10,0)` counts up; `CTD` counts down; `RES(Boxes)` zeroes the accumulator (RES also resets timers). Because a counter has its own edge detection you do not need an ONS in front of it — but you do need the rung to go false between counts, which is why a counter fed by a sensor counts parts and a counter fed by a held button counts once.

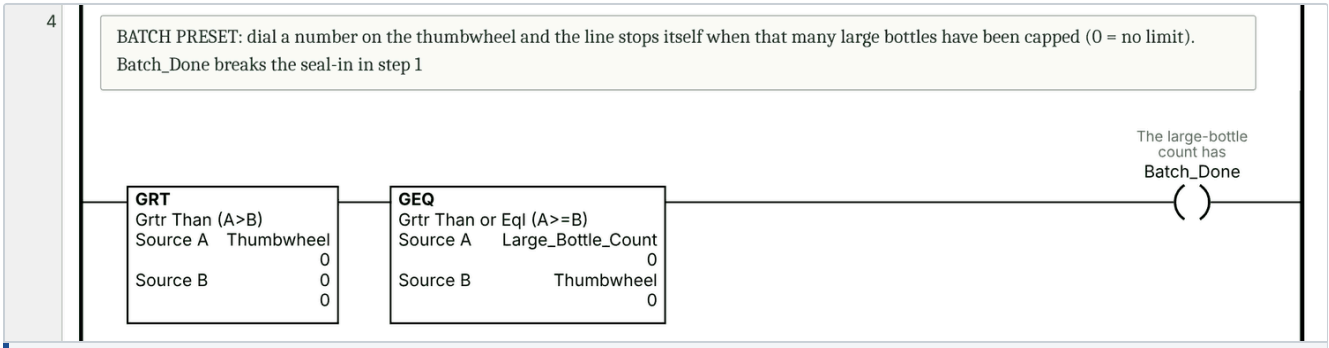
Part IV keeps its counts with ADD rather than CTU, for a reason worth knowing: the four displays show plain DINTs, the thumbwheel compares against a plain DINT, and a CTU's accumulator is just a DINT inside a structure. Both are right; ADD keeps the tag list flat.

> **TRY IT**
>
> Build a belt with a photoeye and a feeder set to automatic. Count the eye with `XIC(PE_1_Detect)CTU(Parts,5,0);` and light a lamp on `Parts.DN`; reset it with a button through RES. Then replace CTU with `ONS(P_ONS)ADD(Count,1,Count)` and show Count on a Digital Display.

8 Compare, move and math

Instruction	True when / does
`EQU(a,b)` `NEQ(a,b)`	a = b / a ≠ b
`GRT(a,b)` `GEQ(a,b)`	a > b / a ≥ b
`LES(a,b)` `LEQ(a,b)`	a < b / a ≤ b
`LIM(low,test,high)`	low ≤ test ≤ high (with low > high the window wraps, as in Logix)
`MEQ(source,mask,compare)`	the bits of source selected by mask equal compare
`MOV(source,dest)`	copies a value; `CLR(dest)` writes zero
`ADD SUB MUL DIV MOD`	dest = a op b, e.g. `ADD(Count,1,Count)`
`CPT(dest,expression)`	evaluates an expression: `CPT(Pct,(Level*100)/Cap)`
`COP / CPS / FLL`	copy arrays, copy synchronously, fill

Compares are input instructions — they sit on the left and pass power — and the math instructions are outputs that act every scan the rung is true, which is why a math rung nearly always wants an ONS, a one-scan timer bit or a counter edge in front of it. The batch preset in Part IV is a compare:




```
GRT(Thumbwheel,0)GEQ(Large_Bottle_Count,Thumbwheel)OTE(Batch_Done);
```

(Step 9: the line stops itself at the dialled number of large bottles; 0 means no limit.)

Scaling an analog input

Analog values arrive as INT counts on a channel of the 1769-IF4. By default a reservoir's `Level` is already 0–100 (percent), an encoder's pulses count up and wrap, a potentiometer is 0–100: the part's signal description (Part III) tells you the range. On a real panel the transmitter is a 4–20 mA loop and the card gives you counts, so the Reservoir and Tank parts have a *Level transmitter signal* setting that hands you what the card would: **Raw counts 0–32767** or **Engineering units 4000–20000** (4 mA = 4000, 20 mA = 20000). Scaling that into a number a person can read is one CPT into a REAL tag:

4–20 mA in engineering units → percent, as a REAL. The 100.0 keeps the arithmetic in floating point

```
CPT(Level_Pct,(Supply_Level-4000)*100.0/16000);
```

The pattern is always *(raw − raw_min) × (eu_max − eu_min) / (raw_max − raw_min) + eu_min*. For raw counts it is `CPT(Level_Pct,Supply_Level*100.0/32767)`. Put the result on a Digital Display with *Decimal places* set to 1 or 2 and the console reads `34.9 %`. A compare against a REAL works the same as against a DINT: `LES(Level_Pct,20.0)`.

Two more ways to get a float: a device signal can be bound straight to a tag of yours (*or use a tag directly* under the signal in Properties) and a REAL tag bound that way receives the unrounded value, bypassing the card; and the filler's own head tank, the weighing conveyor and the flow meter report the same way. The card route is the one you will meet in a plant, so the lessons use it.

TRY IT

On the bottle line, set Supply_Tank's *Level transmitter signal* to *Engineering units*, make a REAL tag `Level_Pct`, add the CPT rung above to S6_Supply, and point a Digital Display at `Level_Pct` with two decimals. Watch it fall with every charge. Then try the raw-counts setting and change the rung to match.

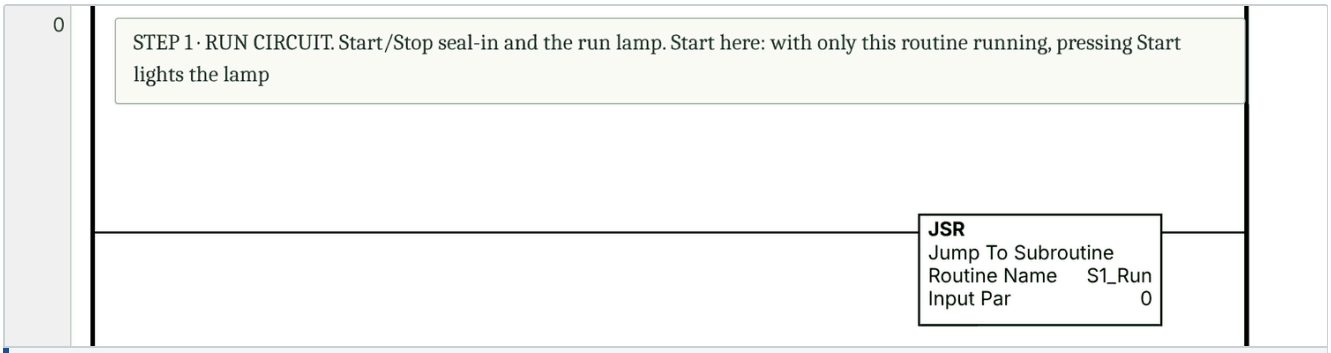
IN THE APP: COLOUR BANDS ON A DISPLAY

From Version 138 a Digital Display can change its digit colour with the value it shows. Its *Colour bands* property is a list of value:colour pairs: `1500:yellow, 3000:red` shows the digit colour below 1500, yellow from 1500, red from 3000; `<100:blue` shows blue below 100. The alarm limits still win over the bands, and *Flash the digits while in alarm* makes an alarm blink. Put `Level_Pct` on one with `<15:red, 15:amber, 30:green` and you have the operator's whole reading of the tank in one glance, without a rung.

9 Routines, programs and the main routine

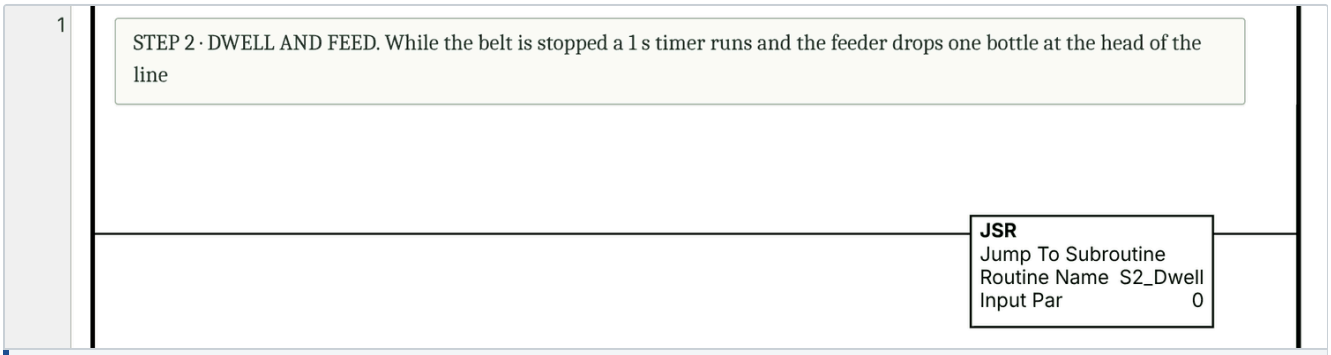
A program is a set of routines; one of them is the **main routine** and is the only one the controller scans on its own. Every other routine runs because a `JSR(Name)` in a running routine jumps to it, scans it top to bottom, and returns. A main routine made of nothing but JSRs is the cleanest way to organise a machine — and it is how Part IV is written:

RUNG 0



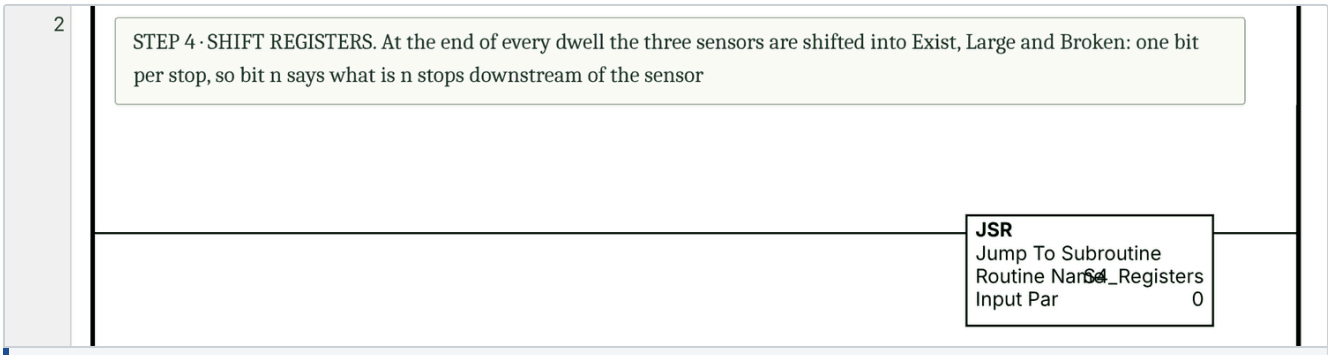

```
JSR(S1_Run);
```

RUNG 1



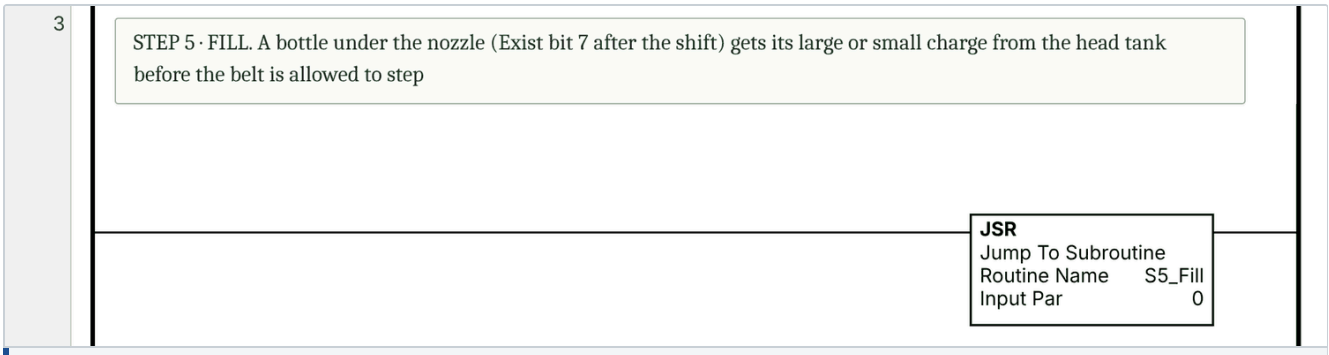

```
JSR(S2_Dwell);
```

RUNG 2



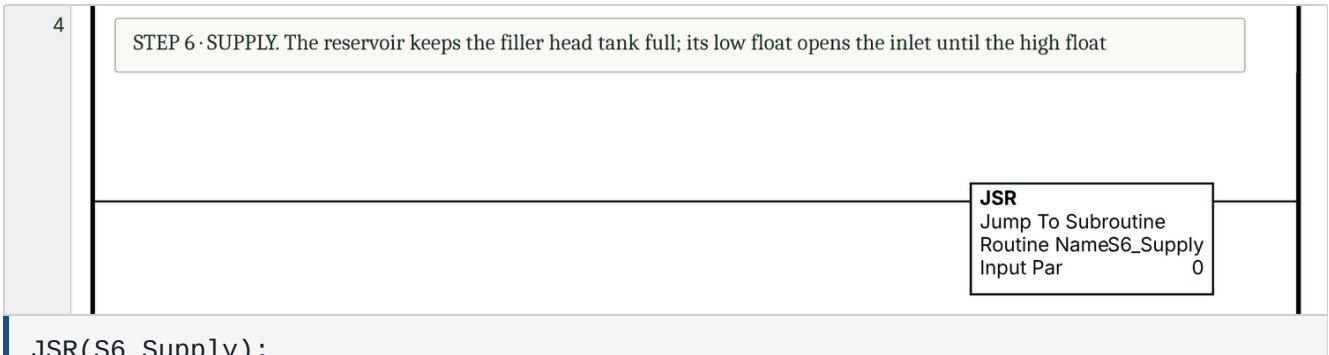

```
JSR(S4_Registers);
```

RUNG 3



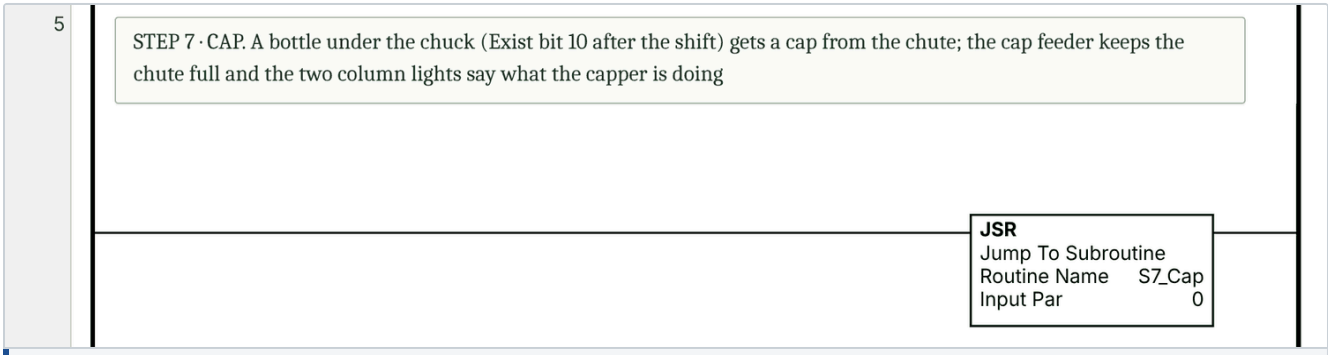

```
JSR(S5_Fill);
```

RUNG 4



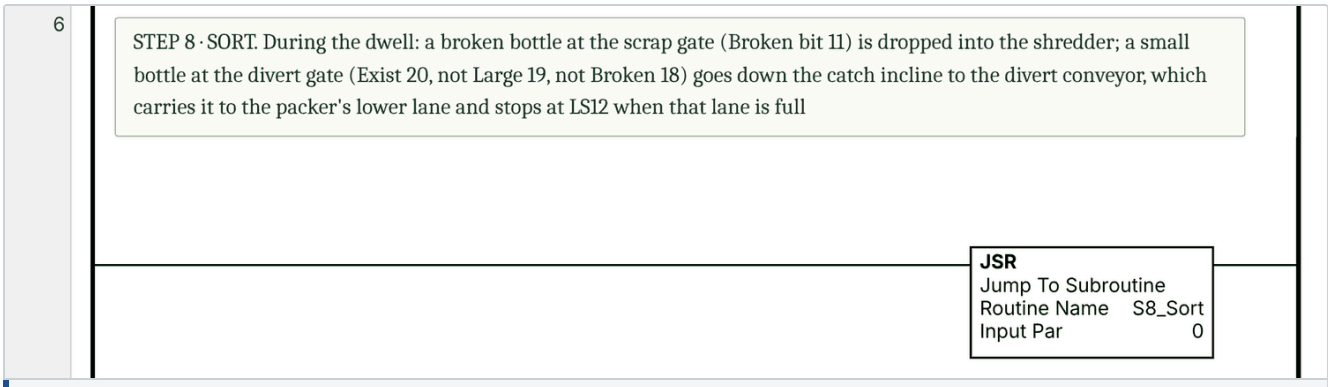

```
JSR(S6_Supply);
```

RUNG 5



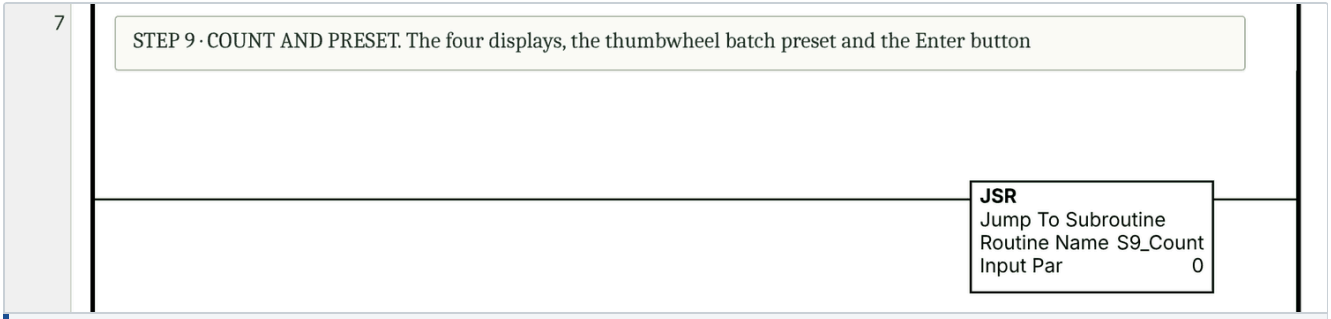

```
JSR(S7_Cap);
```

RUNG 6



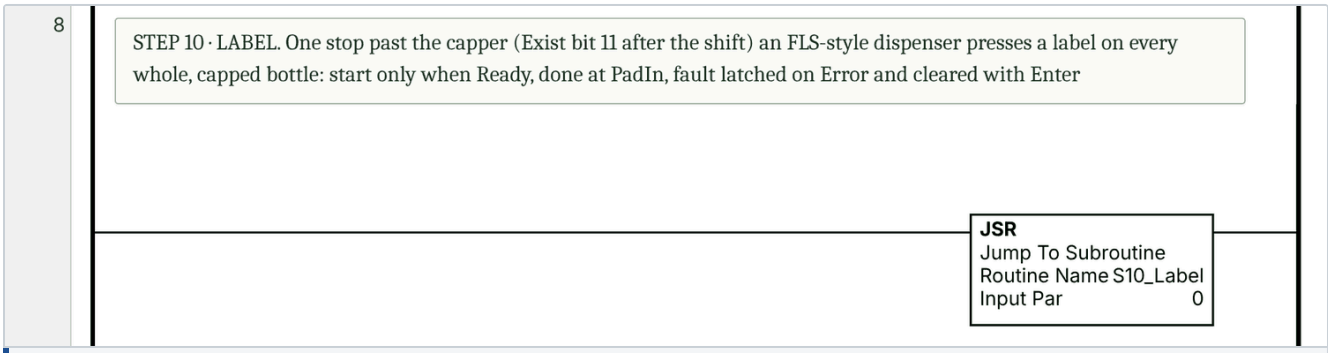

```
JSR(S8_Sort);
```

RUNG 7



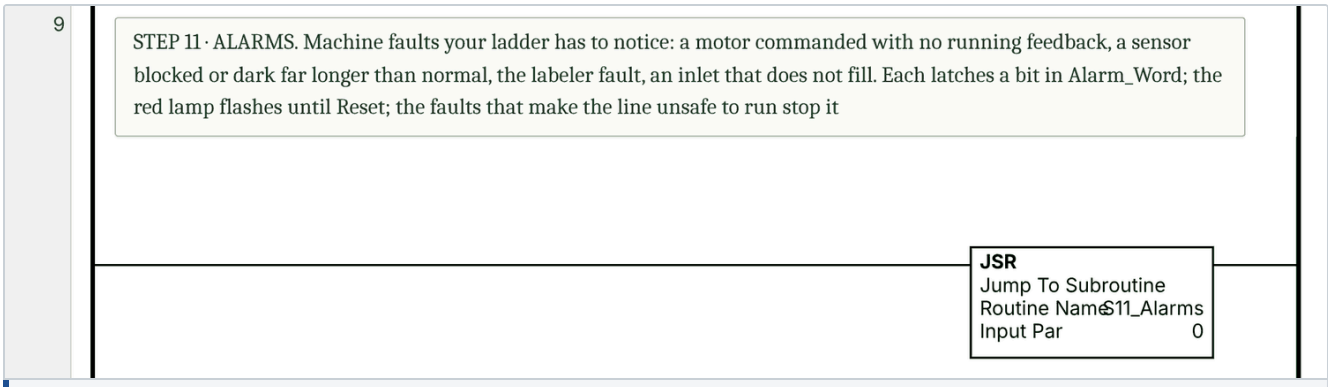

```
JSR(S9_Count);
```

RUNG 8



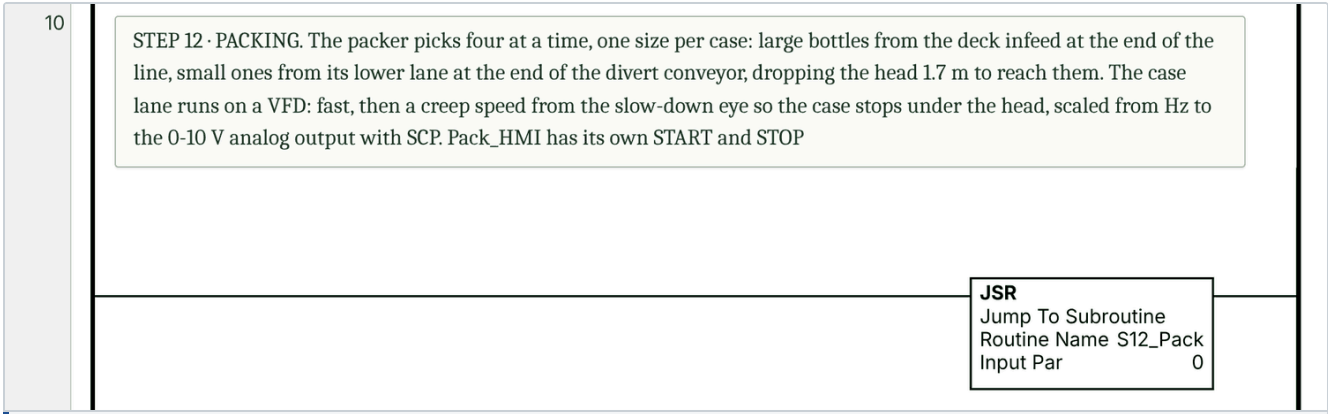

```
JSR(S10_Label);
```

RUNG 9



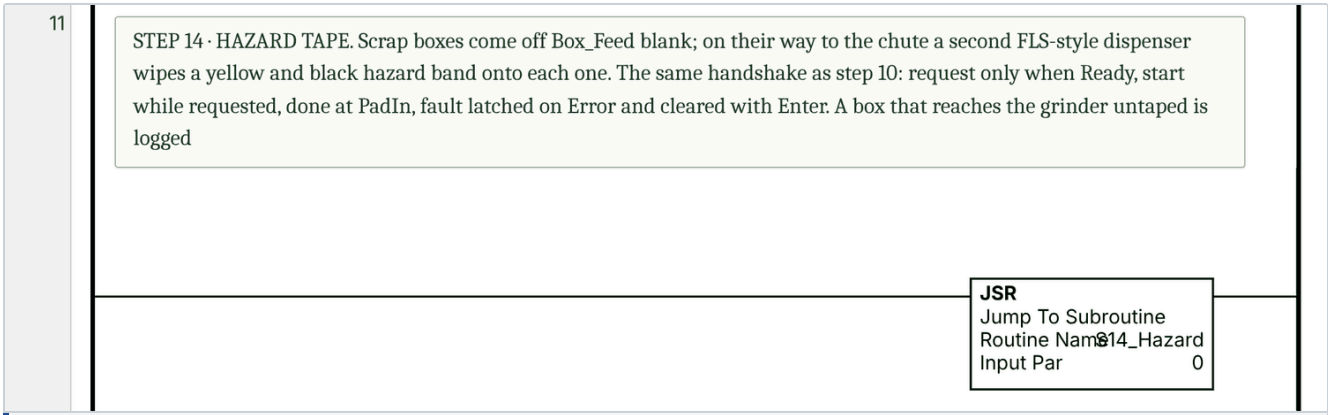

```
JSR(S11_Alarms);
```

RUNG 10



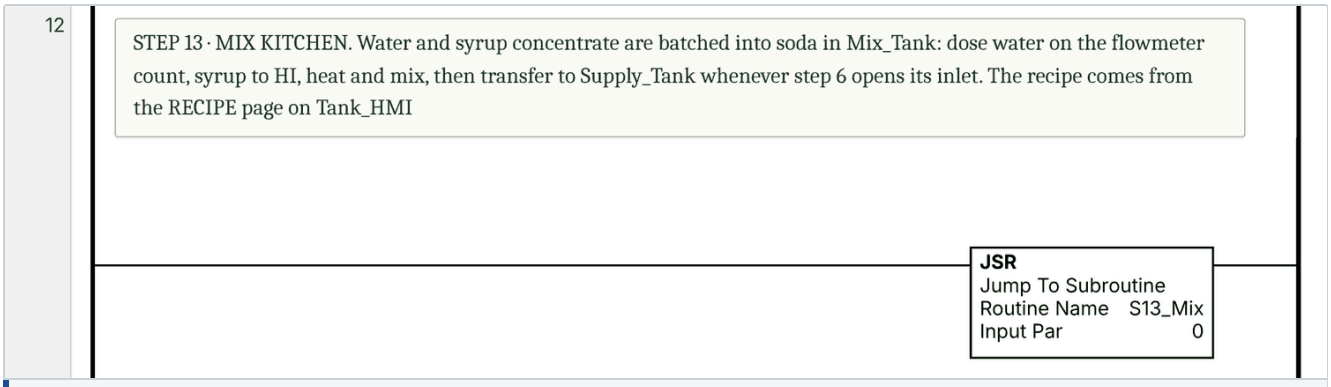

```
JSR(S12_Pack);
```

RUNG 11



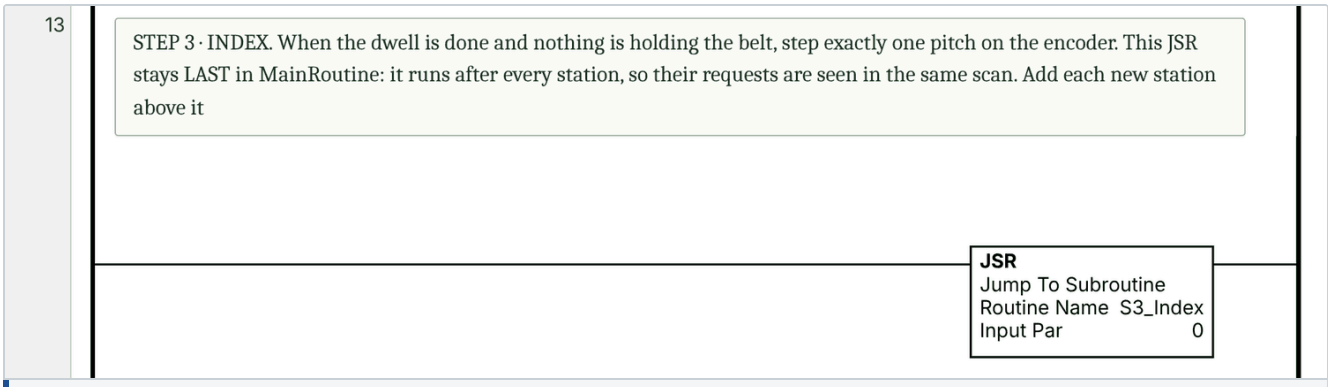

```
JSR(S14_Hazard);
```

RUNG 12




```
JSR(S13_Mix);
```

RUNG 13




```
JSR(S3_Index);
```

Order matters, because of the scan. Step 3 (index the belt) is written early, because every station needs bottles to arrive, but its JSR is always the *last* one in MainRoutine: it is called after the registers, the fill and the cap (steps 4, 5 and 7) so that a fill request raised this scan is seen by the index logic in the same scan. Put the JSR for step 3 above step 5 and the belt can step before the request is noticed; the *Try it* in step 5 asks you to do exactly that and watch. Inside each routine the rungs are written top to bottom, and each one works the moment it is written; MainRoutine is the one place where the order is about the scan.

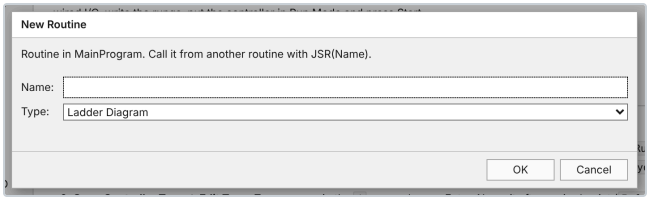


Figure 28. Logic → New Routine...

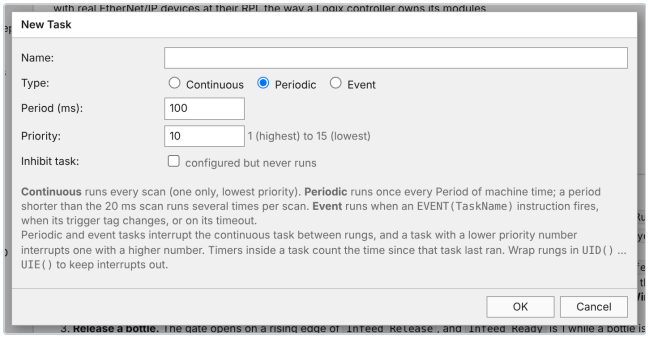
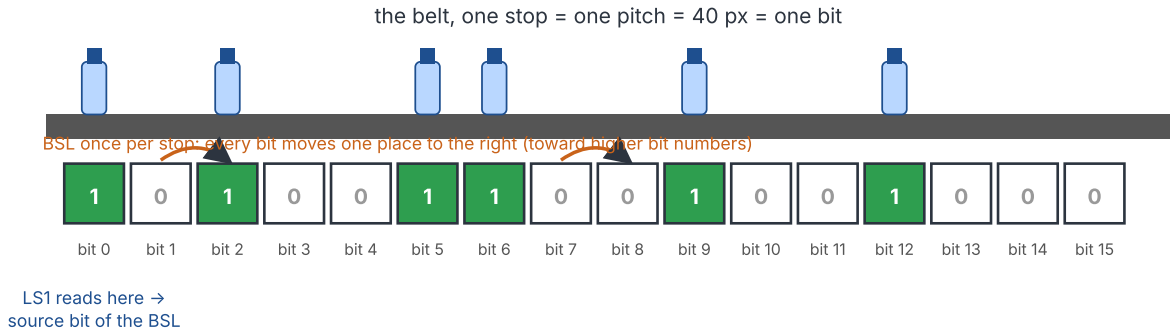


Figure 29. Task properties: continuous or periodic, period and priority.

Logic → New Routine… (or right-click the program) adds a routine; a routine with no JSR to it is dead code and Verify says so. `SBR` / `RET` pass parameters to and from a routine; `JMP(Label)` / `LBL(Label)` skip rungs within one; `AFI` (always false) parks a rung you want to keep but not run; `NOP` is a placeholder. Programs can be scheduled in tasks (continuous or periodic) under *Tasks*, and Add-On Instructions package a routine with its own parameters as a reusable instruction — see Help → Add-On Instructions when you get there. Structured text routines are also supported (Help → Structured Text).

10 Shift registers: tracking parts down a line

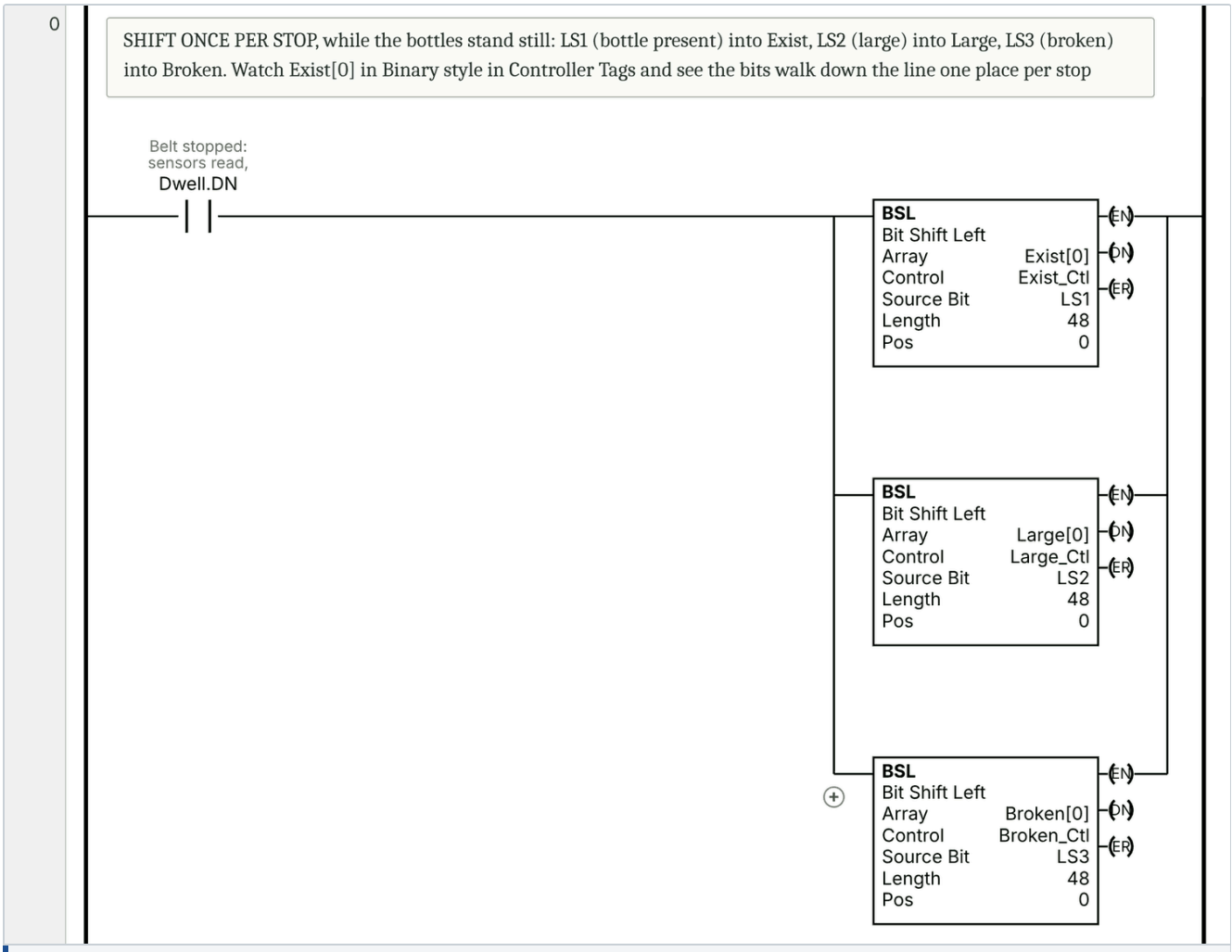
The bottle line's central idea is older than PLCs: an indexing conveyor moves a fixed pitch and stops; a sensor at the head reads what arrived; and a **shift register** — one bit per stop — moves what it knows one place down the line each time the belt steps. Bit 0 is the stop at the sensor; bit *n* is *n* stops downstream. A station at stop 12 reads bit 12 and knows what is standing in front of it without a sensor of its own.



A station n stops past the sensor reads bit n: the register is a model of the belt, one bit per bottle position.

Figure 30. A shift register as a picture of the belt. `Exist` shows a 1 wherever a bottle was seen; each index moves every 1 one stop down the line, in step with the bottles.

`BSL(array,control,source_bit,length)` — Bit Shift Left — does the move. Every time the rung goes true it shifts the whole array up one bit, copies the source bit into bit 0, and reports the bit that fell off the far end in `control.UL`. `BSR` shifts the other way. The control tag is a CONTROL; the length is the number of bits in use.




```
XIC(Dwell.DN)
[BSL(Exist[0],Exist_Ctl,LS1,48),BSL(Large[0],Large_Ctl,LS2,48),BSL(Broken[0],Br
oken_Ctl,LS3,48)];
```

Three registers, three questions about every stop: is there a bottle (`Exist`, from LS1), is it large (`Large`, from LS2, a retro beam set at a height only a large bottle breaks), is it broken (`Broken`, from LS3, a diffuse eye that only reports broken bottles). Downstream, a station asks its question with one XIC.

Which bit is my station?

Count the stops. The belt indexes 40 px. The head sensors sit on the stop grid at x = 160, 200 and 240 — one stop apart — and the stations at 440 (filler), 560 (capper), 720 (scrap gate) and 1000 (divert gate). The scrap gate is (720 − 240) / 40 = 12 stops past LS3. So `Broken[0].12` … almost.

The frame: before or after the shift

The shift happens at the *end* of the dwell, just before the belt steps. During the dwell, while the bottles stand still, the register was last shifted before the previous step: bit *n* describes the bottle that is now at stop *n + 1*. A rung that runs during the dwell — the gates in step 8 — therefore reads one bit *lower* than the count: `Broken[0].11` for the scrap gate. A rung that runs after the shift and before the step — the fill and cap requests in steps 5 and 7, which wait on `Dwell.DN` — reads the count itself. The table every lesson in Part IV refers to:

Station	x	Exist (LS1 @160)	Large (LS2 @200)	Broken (LS3 @240)
Filler	440	6 during the dwell · **7** after the shift	5 · **6**	4 · **5**
Capper	560	9 · **10**	8 · **9**	7 · **8**
Labeler	600	10 · **11**	9 · **10**	8 · **9**
Scrap gate	720	**13** · 14	**12** · 13	**11** · 12
Divert gate	1000	**20** · 21	**19** · 20	**18** · 19

Bold is the frame each station's rung actually uses. These numbers are not assumed: the program's test suite runs the line and correlates every bit against the bottle physically standing at each station, and the comments in the routines say which frame they are in.

WATCH FOR

A sensor that is not on the stop grid sees the same bottle at two stops, and a broken bottle then occupies two bits of the Broken register — the scrap gate fires twice and the good bottle behind the broken one goes into the shredder. The head sensors in the template sit at 160 / 200 / 240 for this reason; if you move one, re-measure your bits with Monitor Tags and a slow belt.

11 Reading the machine while it runs

Programming is mostly watching. Logix Connect gives you six windows onto a running controller, and each answers a different question.

- **The ladder, online.** True instructions light green; a timer shows ACC counting; a rung that should be true and is not shows you which instruction breaks the path. This is where you start.
- **Controller Tags → Monitor Tags**: every tag, live. Expand a DINT to see its bits; open a TIMER to see .ACC and .DN. Right-click a BOOL for *Toggle Bit* (Ctrl+T) to force a value while testing. The toolbar's Toggle button toggles the selected bit.
- **Quick Watch** (View → Quick Watch, floating or docked): a short list of the tags you care about right now, with values.
- **I/O Monitor**: the chassis, point by point, with the device and signal wired to each point. When a sensor "does not work", this is where you find that it is wired to a point your rung does not read.
- **Trends** (Window → New Trend…): pens on a strip chart. The tank level falling with every charge and rising when the inlet opens is a thing you have to see once.
- **The Simulator Log**: what the machine did, in its own words. *Capper cycled with no bottle under it*, *Blue bottle (s) pushed off by Divert_Gate with no bin or conveyor to land on*, *Grinder: ground with no box under it* — each line is a bug report with the device's name in it.

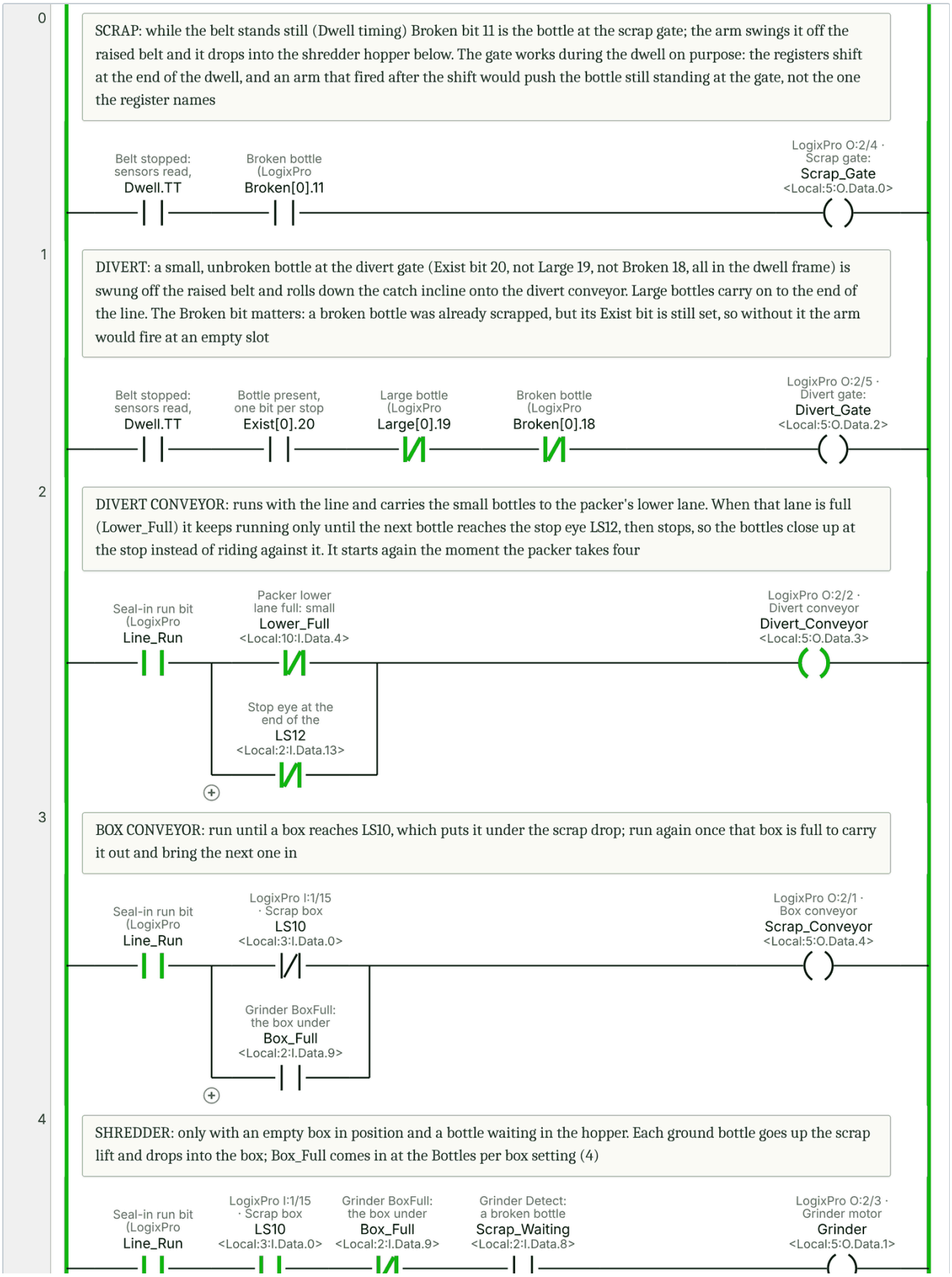

0
SCRAP: while the belt stands still (Dwell timing) Broken bit 11 is the bottle at the scrap gate; the arm swings it off the raised belt and it drops into the shredder hopper below. The gate works during the dwell on purpose: the registers shift at the end of the dwell, and an arm that fired after the shift would push the bottle still standing at the gate, not the one the register names
Belt stopped: sensors read, Dwell.TT
Broken bottle (LogixPro Broken[0].11
LogixPro O:2/4 · Scrap gate: Scrap_Gate <Local:5:O.Data.0>
1
DIVERT: a small, unbroken bottle at the divert gate (Exist bit 20, not Large 19, not Broken 18, all in the dwell frame) is swung off the raised belt and rolls down the catch incline onto the divert conveyor. Large bottles carry on to the end of the line. The Broken bit matters: a broken bottle was already scrapped, but its Exist bit is still set, so without it the arm would fire at an empty slot
Belt stopped: sensors read, Dwell.TT
Bottle present, one bit per stop Exist[0].20
Large bottle (LogixPro Large[0].19
Broken bottle (LogixPro Broken[0].18
LogixPro O:2/5 · Divert gate: Divert_Gate <Local:5:O.Data.2>
2
DIVERT CONVEYOR: runs with the line and carries the small bottles to the packer's lower lane. When that lane is full (Lower_Full) it keeps running only until the next bottle reaches the stop eye LS12, then stops, so the bottles close up at the stop instead of riding against it. It starts again the moment the packer takes four
Seal-in run bit (LogixPro Line_Run
Packer lower lane full: small Lower_Full <Local:10:I.Data.4>
LogixPro O:2/2 · Divert conveyor Divert_Conveyor <Local:5:O.Data.3>
Stop eye at the end of the LS12 <Local:2:I.Data.13>
3
BOX CONVEYOR: run until a box reaches LS10, which puts it under the scrap drop; run again once that box is full to carry it out and bring the next one in
Seal-in run bit (LogixPro Line_Run
LogixPro I:1/15 · Scrap box LS10 <Local:3:I.Data.0>
LogixPro O:2/1 · Box conveyor Scrap_Conveyor <Local:5:O.Data.4>
Grinder BoxFull: the box under Box_Full <Local:2:I.Data.9>
4
SHREDDER: only with an empty box in position and a bottle waiting in the hopper. Each ground bottle goes up the scrap lift and drops into the box; Box_Full comes in at the Bottles per box setting (4)
Seal-in run bit (LogixPro Line_Run
LogixPro I:1/15 · Scrap box LS10 <Local:3:I.Data.0>
Grinder BoxFull: the box under Box_Full <Local:2:I.Data.9>
Grinder Detect: a broken bottle Scrap_Waiting <Local:2:I.Data.8>
LogixPro O:2/3 · Grinder motor Grinder <Local:5:O.Data.1>

Figure 31. S8_Sort online while the line runs. Green is power; a rung that should be true and is not shows you the instruction that breaks the path.

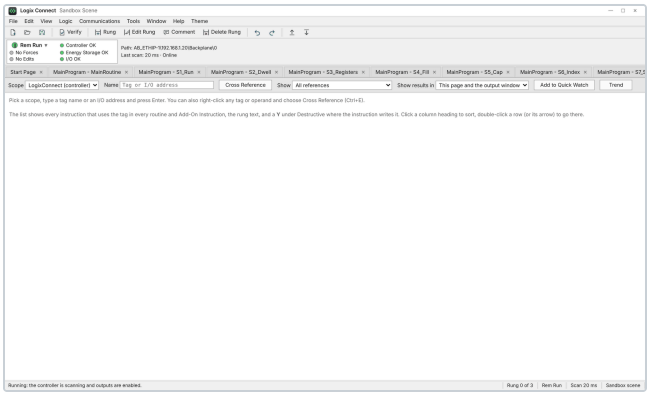


Figure 32. Cross Reference: every rung a tag is used in; click to go there.

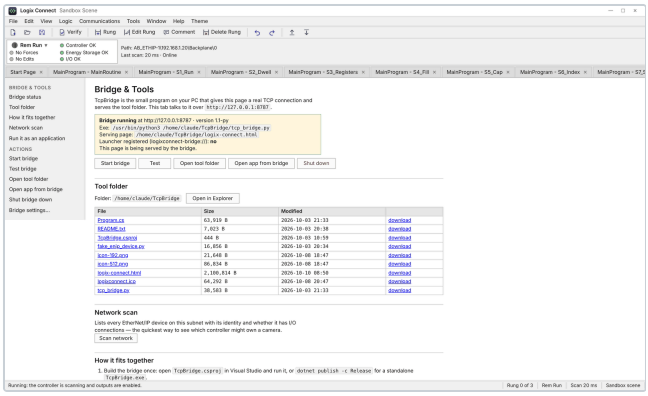


Figure 33. Bridge & Tools: the link to the Python bridge, TCP sockets and EtherNet/IP devices.

Cross Reference (Edit → Cross Reference…, or the bottom tab) lists every rung a tag appears in; click to jump. **Communications → I/O Forcing** lets you force a point regardless of the machine, the way you would on a real controller to test an output rung before the sensor is fitted; *Clear Faults* clears a faulted controller (a major fault stops the scan: an array index out of range, say). **Tools → Clear Tag Values** zeroes timers, counters and bits without touching the machine; **Reset Sim** does the machine as well.

Part III — The sandbox and its parts

12 The sandbox floor

File → New Project… → empty sandbox. The floor is a grid, 1600 × 520 px by default (Floor on the toolbar: up to 4000 × 800). One pixel is 1/60 of a metre in the 3D view, so a 20 m conveyor is 1200 px and the main line of Part IV runs most of the floor.

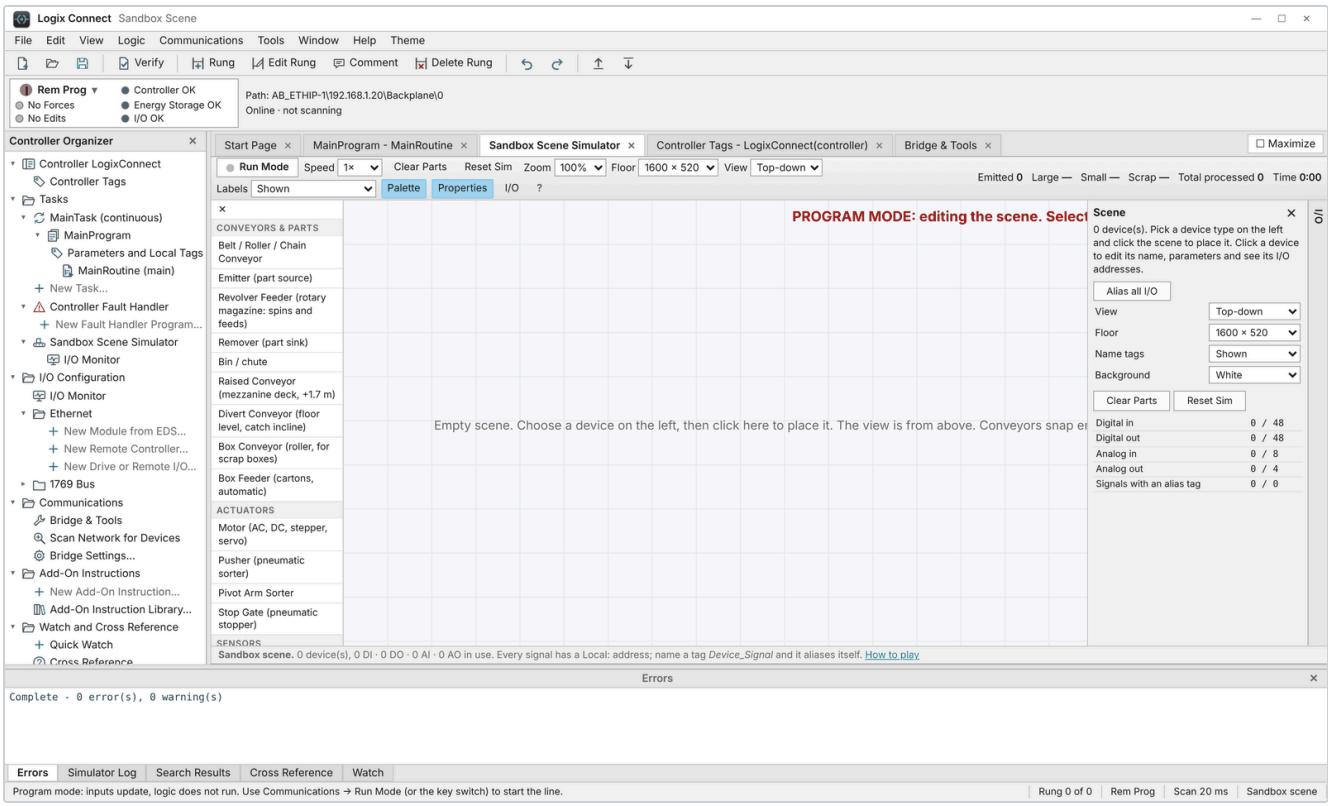


Figure 34. An empty sandbox: the floor, the Palette drawer on the left and the Properties drawer (Scene sheet) on the right.

Placing and arranging

Open the **Palette** drawer (or *Add Device* on a phone), click a part, click the floor: it is placed, named (`PE_1`, `Belt_2`…) and wired — every signal takes the next free point. Hold Shift while clicking to place several. Drag to move; parts snap to a 10 px grid, conveyors snap end to end and line up, and a sensor or station dragged near a conveyor snaps onto its centreline and takes its rotation. **R** rotates the selected part; the Properties drawer has Rotation and Position fields for exact placement, and the arrow keys nudge. Ctrl+click selects several; right-click (or long-press) for Duplicate, Delete, Group, Rotate, Put on a Panel; on a phone the long-press menu starts with Properties…, which opens the part's settings. Program mode only: the floor is locked while the controller runs.

×

CONVEYORS & PARTS

- Belt / Roller / Chain Conveyor
- Emitter (part source)
- Revolver Feeder (rotary magazine: spins and feeds)
- Remover (part sink)
- Bin / chute
- Raised Conveyor (mezzanine deck, +1.7 m)
- Divert Conveyor (floor level, catch incline)
- Box Conveyor (roller, for scrap boxes)
- Box Feeder (cartons, automatic)

ACTUATORS

- Motor (AC, DC, stepper, servo)
- Pusher (pneumatic sorter)

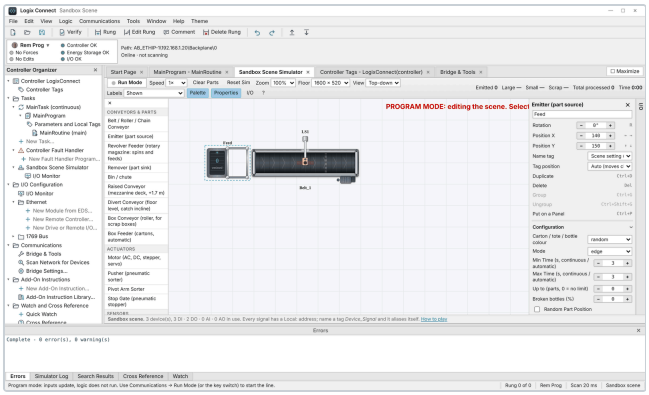


Figure 36. Three parts placed: a feeder, a belt and a photoeye, each wired to the next free point.

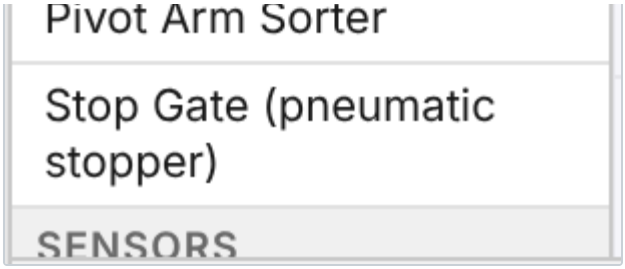


Figure 35. The Palette: every part in chapter 13, by category. Click a part, click the floor.

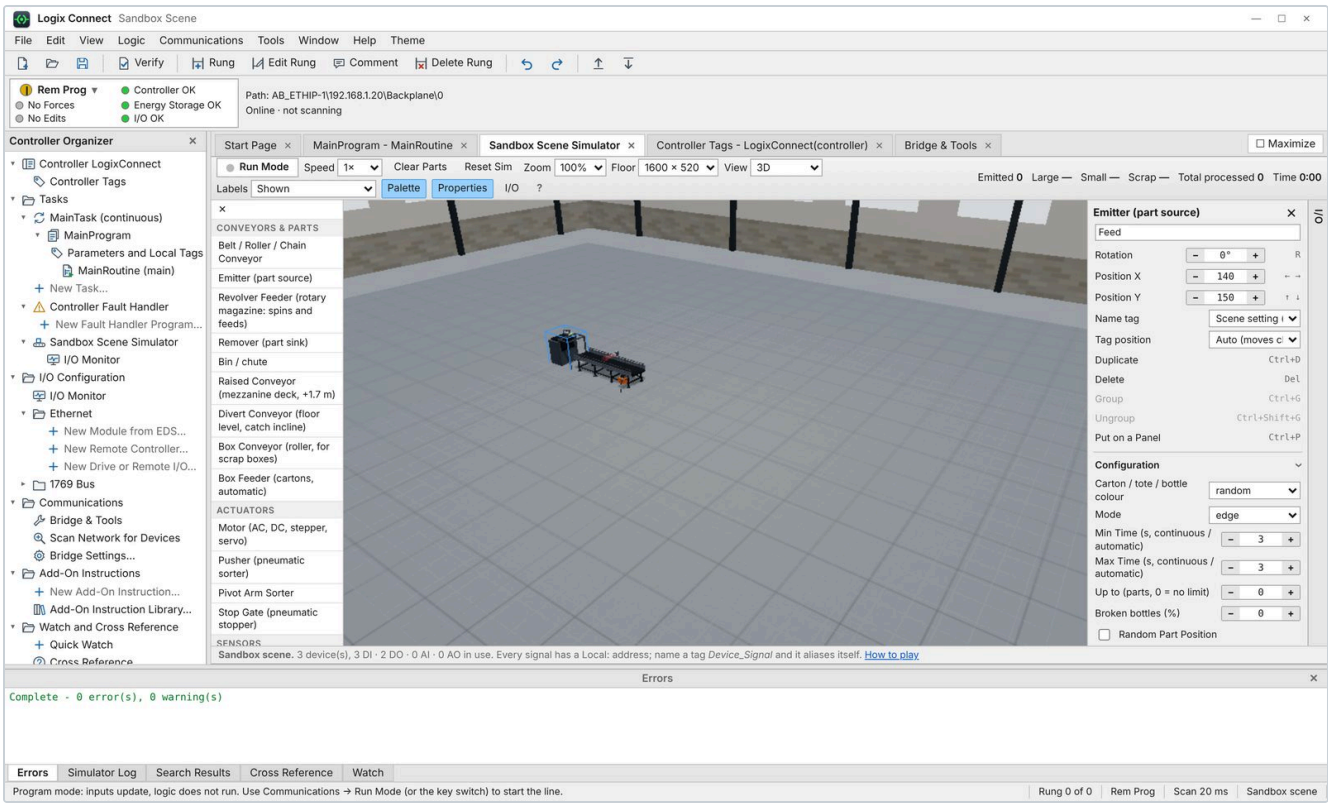


Figure 37. The same three parts in the 3D view.

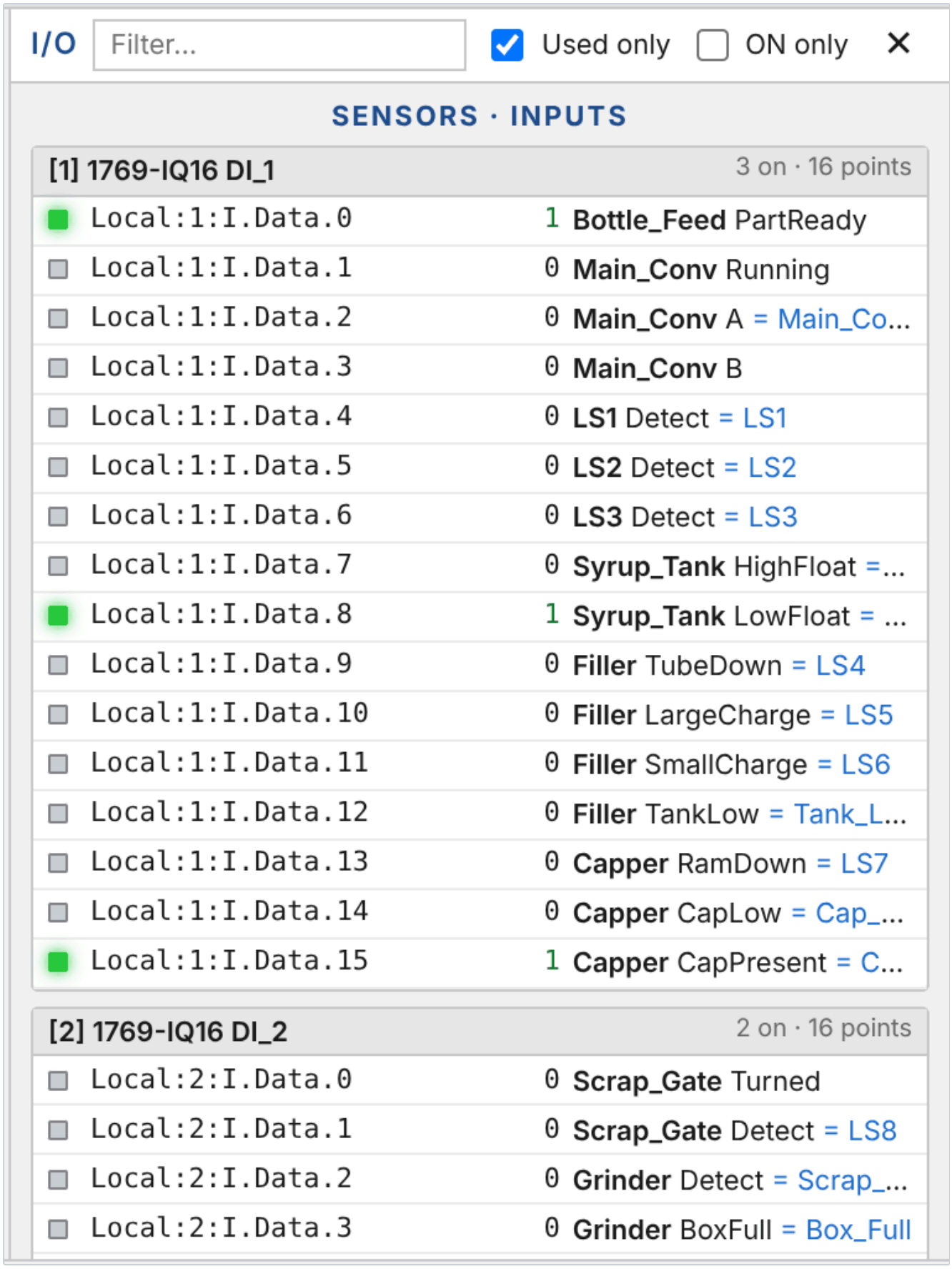

I/O
Filter...
Used only
ON only
SENSORS · INPUTS
[1] 1769-IQ16 DI_1
3 on · 16 points
Local:1:I.Data.0 1 Bottle_Feed PartReady
Local:1:I.Data.1 0 Main_Conv Running
Local:1:I.Data.2 0 Main_Conv A = Main_Co...
Local:1:I.Data.3 0 Main_Conv B
Local:1:I.Data.4 0 LS1 Detect = LS1
Local:1:I.Data.5 0 LS2 Detect = LS2
Local:1:I.Data.6 0 LS3 Detect = LS3
Local:1:I.Data.7 0 Syrup_Tank HighFloat =...
Local:1:I.Data.8 1 Syrup_Tank LowFloat = ...
Local:1:I.Data.9 0 Filler TubeDown = LS4
Local:1:I.Data.10 0 Filler LargeCharge = LS5
Local:1:I.Data.11 0 Filler SmallCharge = LS6
Local:1:I.Data.12 0 Filler TankLow = Tank_L...
Local:1:I.Data.13 0 Capper RamDown = LS7
Local:1:I.Data.14 0 Capper CapLow = Cap_...
Local:1:I.Data.15 1 Capper CapPresent = C...
[2] 1769-IQ16 DI_2
2 on · 16 points
Local:2:I.Data.0 0 Scrap_Gate Turned
Local:2:I.Data.1 0 Scrap_Gate Detect = LS8
Local:2:I.Data.2 0 Grinder Detect = Scrap_...
Local:2:I.Data.3 0 Grinder BoxFull = Box_Full

Figure 38. The I/O drawer: the chassis points in use and the signals on them.

Properties

Click a part and the **Properties** drawer shows its name (edit it there; the alias tags follow), rotation and position, the name-tag setting, the *Configuration* block (the parameters listed in chapter 13) and *Signals and Addresses*: every signal, its kind, its point, and the *tag Device_Signal* link that creates the alias. A signal can also be bound straight to any tag of yours (*or use a tag directly: click to pick*), which bypasses the chassis entirely; the lessons use addresses and aliases because that is what you will meet on a real panel.

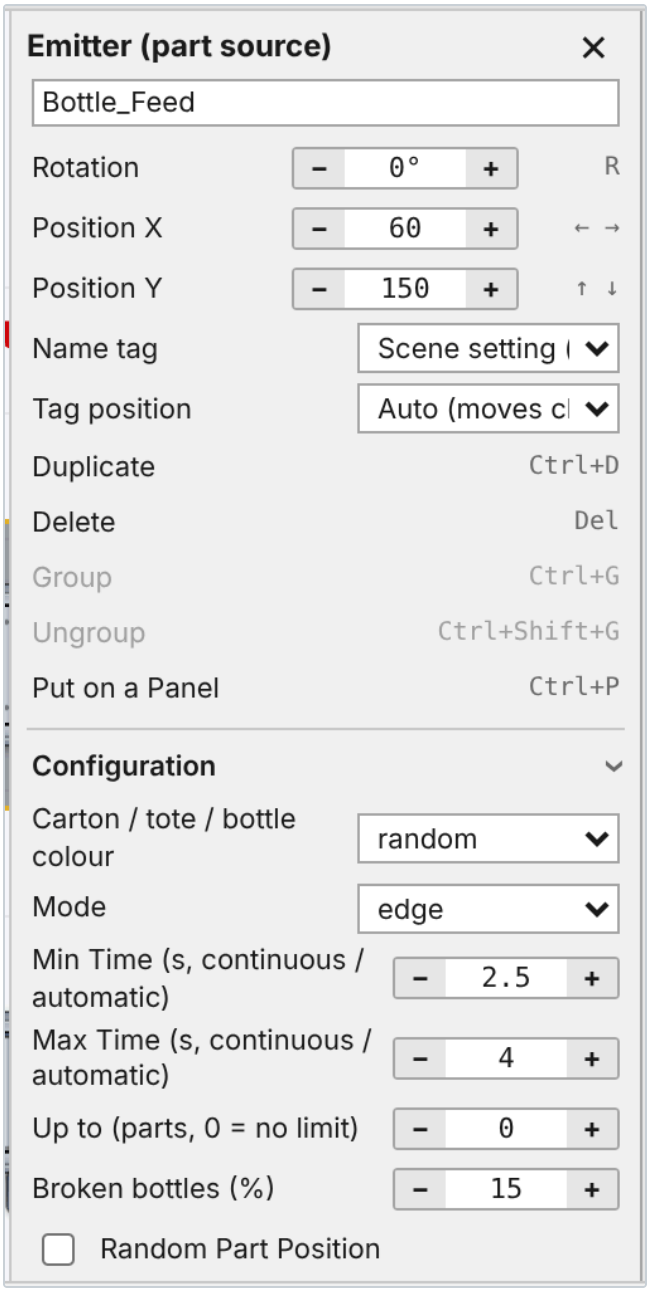


Figure 39. Properties of a conveyor: length, speed, style, Height (floor or raised deck), encoder.

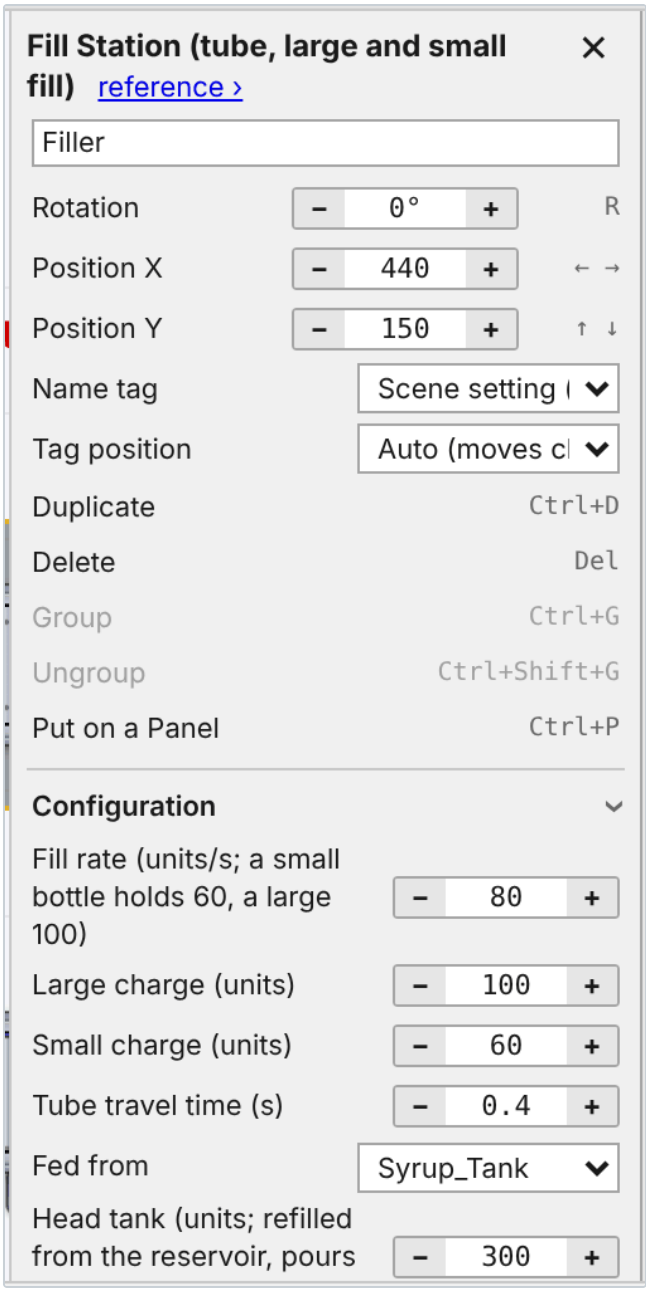


Figure 40. Properties of the Inline Cap Feeder: caps in the chute, magazine rate, the lamp signals.

Panels

A **Panel / Enclosure** is a drawn operator station. Drop buttons, switches, lights, thumbwheels and displays onto it and they move with it; *Put on a Panel* builds one around the selected parts. In 3D the panel stands on a pedestal with the devices on its face. A panel can also be given its own remote I/O block (right-click → its own I/O) so the operator station talks over EtherNet/IP like a real one.

Views

Top-down is the plan: lay the line out here. **Side** is the elevation: bottles stand on the belt, the fill tube comes down, the ram hits, tanks show their level. **3D** is the shop floor: drag to orbit, wheel to zoom, right-drag or Shift+drag to pan, double-click a device to look at it, double-click the floor to reset; hover a device for its live I/O tag. All three are the same scene and the same I/O. Tall parts near the top of the floor get headroom automatically, and the floor fills the window.

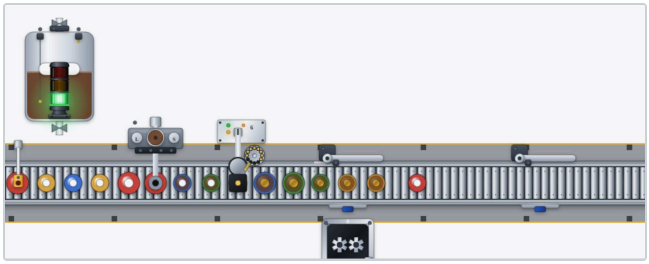


Figure 41. Top-down: the plan.

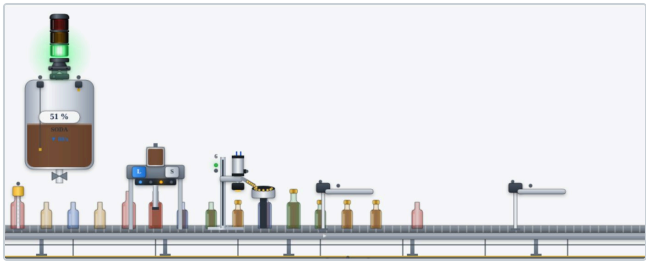


Figure 42. Side: the elevation.

Figure 43. 3D: the raised deck from the head of the line.

3D graphics

How much work the 3D view does is set under **Theme → 3D graphics**, one setting for the desktop layout and one for the phone layout:

Level	What you get	Default on
High	Medium, plus ambient occlusion (the soft dark where a part meets the floor or a guard meets a frame), a glow around lamps and lit lenses that grows at night, and the finest shadows.	a PC with a graphics card

Level	What you get	Default on
Medium	Textured asphalt, concrete, grass, belting and brushed metal, a sky over the outdoor scenes, shadows sharpened around whatever the camera looks at, and smoothed edges.	a PC on the graphics built into its processor (Intel Iris Xe or UHD, AMD Radeon Graphics)
Low	The light renderer: one coarse shadow map, no reflections, every other frame drawn. Lamps get a painted halo instead of a real glow.	a PC with 4 cores or fewer; a phone with 3D turned on
Off	No 3D view: Top-down and Side only. 3D leaves the View list.	a phone

Nothing about the machine changes with the level: the same parts, the same I/O, the same scan. It is only how the picture is drawn. The level starts from the graphics chip the browser reports, as in the table, until you pick one yourself. If the 3D view runs under 24 frames a second for about two and a half seconds the level steps down by itself (High to Medium, Medium to Low) and the status bar says so; it never turns 3D off by itself, and *Theme → 3D graphics: step down by itself when slow* turns the step off.

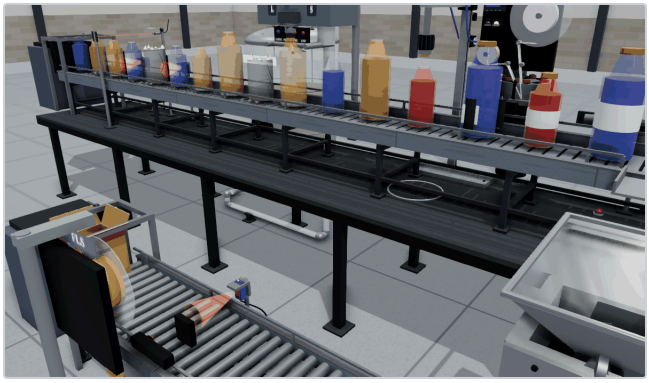


Figure 44. High: ambient occlusion under the guards and legs, textured floor and belting, lit lenses glowing.

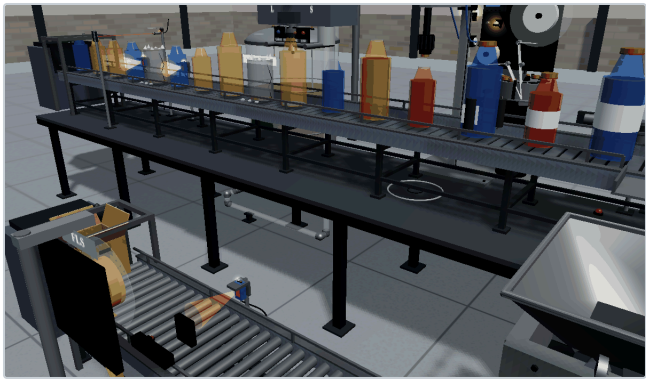


Figure 45. Low: the same moment on the light renderer.

What slows a 3D view down is mostly how many separate pieces are sent to the graphics chip each frame and how many lights fall on them, not how detailed a machine looks. So when a machine is built in 3D, the pieces that never move are joined into one piece per material, while everything that moves, lights up or shows a count stays separate. The big hall over the bottle line carries only six real lights however many ceiling fixtures it shows, and a lamp that is off is left out until it comes on. You will not see any of this; it is why the bottle line, the biggest scene, runs about as smoothly as the small ones.

TRY IT: THE SAME LAMP AT EVERY LEVEL

Open the bottle line in 3D, start it, and double-click the operator console to look at it. Switch Theme → 3D graphics from High to Medium to Low. On High the RUN lamp glows from inside its lens with a soft halo; on Medium the lens is lit but has no halo; on Low a painted halo stands in for it. Turn the lamp off with Stop: on every level the lens keeps its green, only darker, the way a coloured lens looks with the bulb out.

WATCH FOR

A status bar message that the graphics *stepped down* is the frame rate talking, not a fault: a big floor on High asks a lot of a laptop on battery. Put it back with Theme → 3D graphics once it is plugged in. On a phone, leave 3D off for writing and testing logic; Top-down and Side show every sensor and output, and they cost the battery almost nothing.

Real models

The machines are being rebuilt one by one as the real thing, starting with the fill station and the cars. Nothing a model changes reaches the program: the same signals, the same tags, the same rungs. What changes is what you can read off the machine while it runs.

The **fill station** stands on a stainless frame over the conveyor. Its head tank is the product bowl on top: the feed pipe from the reservoir comes in at the bowl's inlet, and the **sight glass** on its side shows the head tank level, dropping as bottles are filled and coming back up as it refills. The LARGE and SMALL valves sit on pneumatic cylinders on the manifold under the bowl, the nozzle rides a lift cylinder down into the bottle, and the guard has an interlocked door. The station reports on itself twice: the **LARGE, SMALL, TUBE and LOW** pilot lights on the panel on its front post, and the **stack light** on its top deck, green while it is fed and filling, amber when the head tank is low, red when it spills.

Figure 46. The fill station on the bottle line: the bowl with its sight glass, the valves on their cylinders, the guard, the lamp panel on the post and the stack light on top.

Figure 47. A car in the car wash tunnel; sedans, SUVs and pickups also drive the intersection and park in the garage.

The **cars** in the car wash, the intersection and the parking garage are sedans, SUVs and pickups. Their lamps do what they did before: brake lamps at a red light, headlamps at night, the wash's lamps as a car drives through.

The **parking garage** was reworked in the same release. The drive into LEVEL 1 is clear: no stall and no column stands in front of the entry or the exit, and arrows painted on the street and across LEVEL 1 show the way in and the way out. The back of the garage is a solid wall. The ramps sit in their own bay beside it, lighter concrete with kerbs and yellow chevrons, and the decks are cut open over them so you can follow a ramp from LEVEL 1 up to the roof. With the drive cleared, LEVEL 1 holds 18 cars and each floor above it 24: 90 in all, which is what the reference loads into `Lot_Capacity` on the first scan.

Figure 48. The parking garage: the arrows painted in and out, the clear drive into LEVEL 1 and the LED header over each floor; the ramps run up beside the closed back wall.

In v1.45 the **capper** and the **labeler** followed. The capper is a capping station on four posts over the lane. Its head hangs from the deck on the air cylinder: a spindle motor, a torque clutch, and a knurled chuck that spins as it comes down onto the bottle. The caps come from a centrifugal sorter on its own stand behind the lane: its pocket disc turns, and the caps run down a twin-rail chute to the pick-off under the chuck. The labeler, rebuilt in v1.47, is a side labeler. Its head is a deck plate at label height on a housing and pedestal beside the conveyor. Follow the web off the roll, which lies flat on its spindle, over the dancer arm and a guide roller to the peel blade. The blade's nose stands right at the bottle: there the label leaves the backing, and the backing turns round the nose and comes back to the rubber drive roller, its nip roller and the rewind. The gap-sensor fork on the blade is how a real dispenser stops on each label, and the applicator (a tamp pad, a wrap-around roller, a brush or an air grid) works at the nose.

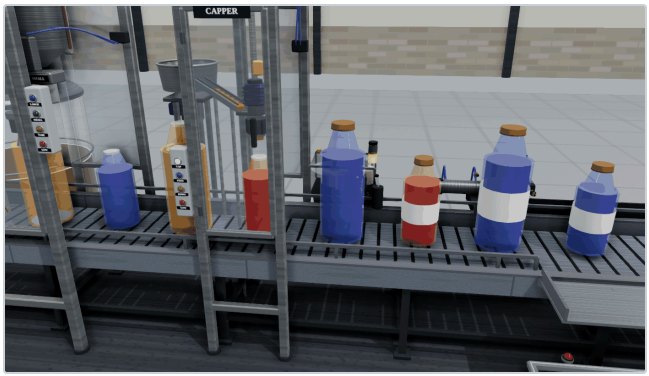


Figure 49. The capper: the head on its cylinder, the chute from the cap sorter, the CAP / FEED / DOWN / LOW panel on the post, and the labeler beside it.

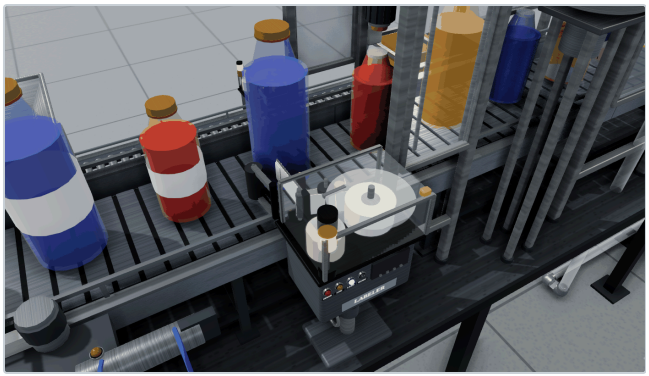


Figure 50. The labeler from behind: the housing with its READY / LABEL / LOW / ERROR row, the roll lying flat and the rewind under the clear cover, the web out along the blade to the bottle.

Each machine shows its own state on its own lamps: CAP, FEED, DOWN and LOW on the capper's panel, READY, LABEL, LOW and ERROR on the labeler's row. The two-tier stack light on each one is different: those are `LampGreen` and `LampAmber`, the outputs your program drives.

TRY IT: RUN THE CAP SORTER DOWN

In 3D, run the bottle line with the Feeder output forced off (or stop the rung that drives it). Each cap the head presses turns the sorter disc one pocket, and the chute empties from the top. At the CapLow count LOW lights on the capper's panel; when the last cap goes, CAP goes out and the next bottle leaves open. Now let your program see CapLow: run the Feeder and show it on LampAmber, and watch the stack light go amber and the sorter refill.

TRY IT: READ THE HEAD TANK OFF THE MACHINE

Run the bottle line in 3D and look at the fill station from the front. Watch the sight glass: every large bottle takes a step out of it, every refill from Supply_Tank puts it back. Now let Supply_Tank run dry (stop the mix kitchen that feeds it) and keep running: the level walks down, LOW lights on the panel at 15 %, and the stack light goes from green to amber, before the program has said a word. Start the kitchen again and watch both come back.

In v1.47 the **tanks** got their liquid. The product is deep and glossy, darker toward the bottom, with a mirror surface that rides on the level, and the glass is clear except toward its edges, so you read the level straight off the tank. The clear pipes carry the same product in the same colour as the tank they come from: soda out of the supply tank and the carbonator, water and dark cola syrup out of the totes. The mix tank shows its batch: pale while the ingredients go in, the product's own colour once it is mixed, the same colour as in the

supply tank. The Supply_Tank's inlet comes in through a nozzle on the dome, a ball valve with an air actuator whose indicator is green when it is open and red when it is shut, and an elbow to the supply line; the Tank_Light stands on its own nozzle beside it.

Figure 51. The Supply_Tank: soda with its surface at the level, the inlet valve and elbow on the dome, the Tank_Light on its own nozzle.

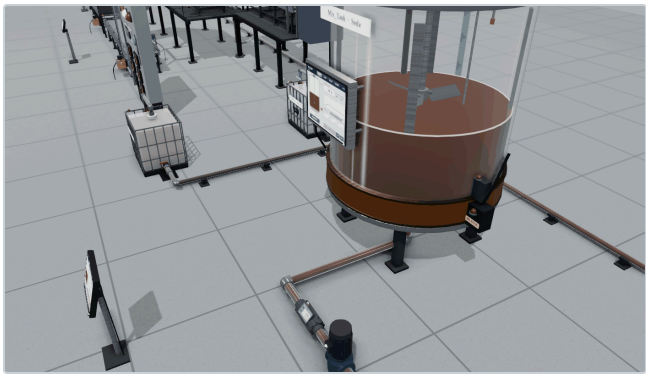


Figure 52. The Mix_Tank after a batch: the cola syrup line from its tote, the mixed soda in the tank and in the outlet line through the pump.

TRY IT: WATCH A BATCH GO THROUGH

Run the bottle line in 3D and look at the Supply_Tank. When the level reaches the low probe your rung opens the inlet: the indicator on the actuator turns green, the pipe from the carbonator runs, and the surface climbs to the high probe. Then turn to the Mix_Tank for a batch: the water and syrup lines run, the tank fills pale, and as the mixer turns the batch goes to the soda brown of the supply tank.

WATCH FOR

A clear pipe that shows nothing has not carried liquid yet: it fills the first time something runs through it and stays full after that. A full pipe that is standing is still; a soft shimmer moving along it means it is flowing. So a pipe from a tank that stays still while your inlet is commanded open is worth a look before the valve is.

WATCH FOR

The stack light on the fill station is the station's own, driven by its condition, not by your program: it is there so you can see the machine's state from across the floor. The lamps your program drives are the ones you wire, such as the operator console's, and LampGreen and LampAmber, the two-tier stack lights on the capper and the labeler. If amber shows on the filler and nothing in your program noticed, that is a missing alarm, not a bug in the lamp.

Rules of the simulated world

- Parts are bottles (small, 60 units, and large, 100 units, some of them broken when the feeder is set to break some), cartons and totes (S / M / L, which a grinder fills with glass), and metal blocks and raw parts for sorting lines.
- A conveyor moves anything within its lane at its speed; parts queue behind each other; a part that reaches the end of a belt with nothing under it falls and is **lost**. Removers and bins take parts off the line and count them as processed. A Weighing Conveyor reports a weight.
- A pivot arm or pusher knocks a part sideways; it slides up to 200 px and lands on the first conveyor it crosses, into a bin, or into a grinder hopper. A raised conveyor (Height = Raised deck) stands 1.7 m up on a mezzanine; a gate on it drops bottles into a grinder below or down a catch incline onto a floor-level conveyor beside it. The incline, the rail opening and the latch door appear by themselves from the layout.
- Sensors see what is physically there, within their range and sensitivity and according to what they are set to see (any part, bottles only, broken bottles, filled bottles, capped bottles, boxes only…). A retro beam at a height only sees parts taller than that height.

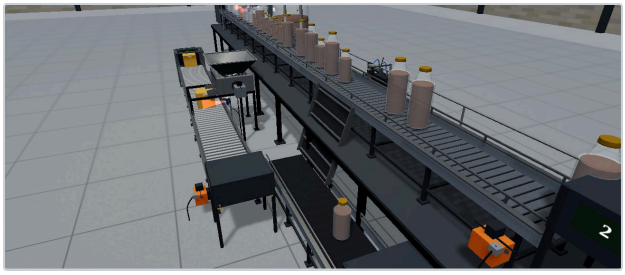


Figure 53. A gate on the deck is a latch door; a small bottle rides the catch incline down to the divert conveyor.

Figure 54. The scrap gate drops a broken bottle into the shredder hopper; the chute fills a box below.

- Fillers draw product from a reservoir through a pipe (the pipe draws itself) into a head tank; a dry tank stops the pour. Cappers take caps from a chute fed by a rotary magazine; an empty chute leaves the bottle open, and a chuck that comes down on a broken bottle crushes it. Grinders need a box under the chute or the glass spills.

13 Parts catalogue

Every part in the palette, in palette order, with its settings and its signals. A signal marked *Digital input* is something the machine tells the controller (it arrives at a `Local:n:I` point); *Digital output* is something the controller tells the machine (a `Local:n:O` point). Some signals only exist when a setting is on (an encoder on a conveyor, a drain valve on a reservoir); the tables show the signals a part has with its default settings. The entries marked *preset of* are the same part with settings already made, listed as parts of their own so you can see every option in the list.

Conveyors & parts

Belt / Roller / Chain Conveyor `belt`

Tick **Incremental encoder** for channels A and B in quadrature (00 → 01 → 11 → 10 running forward, reversed when the belt reverses) plus a Pulses count, one pulse per resolution step of belt travel. Count pulses to position a part without a second sensor.

Figure 55. A slat-chain conveyor on the raised deck (Height = Raised deck): the mezzanine, rails and encoder draw themselves.

Configuration

key	Setting	Default	Range / choices
`length`	Length (m), 0.5 to 40 in half-metre steps	4	0.5 – 40, step 0.5
`speed`	Speed (mm/s)	120	20 – 400, step 10
`reversible`	Reversible (adds Rev output)	False	on / off
`vfd`	VFD speed reference (adds analog output, 0-100 %)	False	on / off
`drive`	Driven by drive module (its name; blank = contactor)		text
`style`	Conveyor type	belt	belt, cleated belt, roller, gravity roller (no motor), modular plastic belt, slat chain, twin chain, wire mesh
`beltColor`	Belt color	Black	Black, Green, Blue, White, Grey
`frame`	Frame	Aluminium	Aluminium, Stainless steel, Painted blue, Painted grey, Safety yellow, Black
`elev`	Height (3D)	Floor level	Floor level, Raised deck (+1.7 m)
`encoder`	Incremental encoder (adds A, B and Pulses)	False	on / off
`encRes`	Encoder resolution (px per pulse)	10	2 – 60, step 1

Signals (with the default settings)

Signal	Kind	Meaning
`Fwd`	Digital output (from the controller)	Motor forward
`Running`	Digital input (to the controller)	Motor contactor aux contact

Emitter (part source) `emitter`

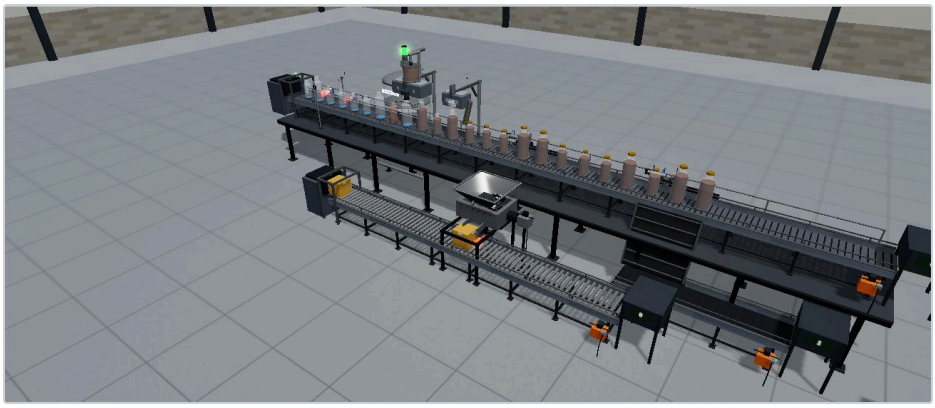


Figure 56. An Emitter at the head of a belt (left) makes parts: bottles, cartons, totes, blocks.

Configuration

key	Setting	Default	Range / choices
`parts`	Part to Emit	cartonS, cartonM, cartonL, metalM	Carton (S), Carton (M), Carton (L), Shipping Crate (XL), Tote, Metal Block (S), Metal Block (M), Metal Block (L), Raw Stock, Blue, Raw Stock, Green, Raw Stock, Metal, Housing Base, Blue, Housing Base, Green, Housing Base, Metal, Housing Lid, Blue, Housing Lid, Green, Housing Lid, Metal, Bottle (S), Can, Bottle (M), Bottle (L)
`color`	Carton / tote / bottle colour	random	random, random (R/B/G), R, B, G, Y
`mode`	Mode	edge	edge, continuous, automatic (no Emit input)
`minTime`	Min Time (s, continuous / automatic)	3	0.5 – 60, step 0.5
`maxTime`	Max Time (s, continuous / automatic)	3	0.5 – 60, step 0.5
`upTo`	Up to (parts, 0 = no limit)	0	0 – 999, step 1
`clearTime`	Automatic: count the time only while the exit is clear (keeps a gap after a stop)	False	on / off
`hazard`	Cartons: scrap boxes, banded with hazard tape (sharp contents)	False	on / off
`broken`	Broken bottles (%)	0	0 – 100, step 5
`randPos`	Random Part Position	False	on / off
`randRot`	Random Part Orientation	False	on / off
`finish`	Bottles leave as	empty	empty, finished (filled, capped, labeled)

key	Setting	Default	Range / choices
`defects`	Finished bottles: defective (%) (no cap, no label or a short fill)	0	0 – 100, step 5
`batches`	Finished bottles: products in batches, PRODUCT:count, … (Level 9; blank = all Soda)		text
`stray`	Finished bottles: one stray of the last product in each new batch	False	on / off

Signals (with the default settings)

Signal	Kind	Meaning
`Emit`	Digital output (from the controller)	Release a part (rising edge, or continuous while on)
`PartReady`	Digital input (to the controller)	Exit clear, ready to emit

Revolver Feeder (rotary magazine: spins and feeds) `revolver`

A rotary magazine like a revolver cylinder: *Pockets* bores round a drum beside the lane, loaded with *Parts loaded at start*. A rising edge on **Index** turns the drum one pocket (*Index time*) and the part that reaches the drop pocket lands on the lane; **InPosition** is 1 while the drum stands still, **PartReady** while a part sits in the drop pocket with the lane clear, **Empty** when the drum has run out. **Reload** (or *Reload by itself*) refills every pocket in about a second. Pair it with a sensor on the lane: XIC(PartReady) XIO(Lane_Eye) OSR → Index is the classic one-at-a-time feed.

Configuration

key	Setting	Default	Range / choices
`parts`	Part to feed	bottle, bottleL	Carton (S), Carton (M), Carton (L), Shipping Crate (XL), Tote, Metal Block (S), Metal Block (M), Metal Block (L), Raw Stock, Blue, Raw Stock, Green, Raw Stock, Metal, Housing Base, Blue, Housing Base, Green, Housing Base, Metal, Housing Lid, Blue, Housing Lid, Green, Housing Lid, Metal, Bottle (S), Can, Bottle (M), Bottle (L)
`color`	Carton / tote / bottle colour	random	random, random (R/B/G), R, B, G, Y
`pockets`	Pockets on the drum	8	4 – 16, step 1
`loaded`	Parts loaded at start	8	0 – 200, step 1
`indexTime`	Index time (s, one pocket)	0.5	0.1 – 3, step 0.1
`autoReload`	Reload by itself when empty	True	on / off
`broken`	Broken bottles (%)	0	0 – 100, step 5

Signals (with the default settings)

Signal	Kind	Meaning
`Index`	Digital output (from the controller)	Turn the drum one pocket (rising edge); the part in the drop pocket lands on the lane
`Reload`	Digital output (from the controller)	Fill the drum (hold on)
`InPosition`	Digital input (to the controller)	Drum at rest on a pocket
`PartReady`	Digital input (to the controller)	A part sits in the drop pocket and the lane under it is clear

Signal	Kind	Meaning
`Empty`	Digital input (to the controller)	No parts left on the drum

Remover (part sink) `remover`

No settings.

Signals (with the default settings)

Signal	Kind	Meaning
`Detect`	Digital input (to the controller)	Part entering the remover

Bin / chute `bin`

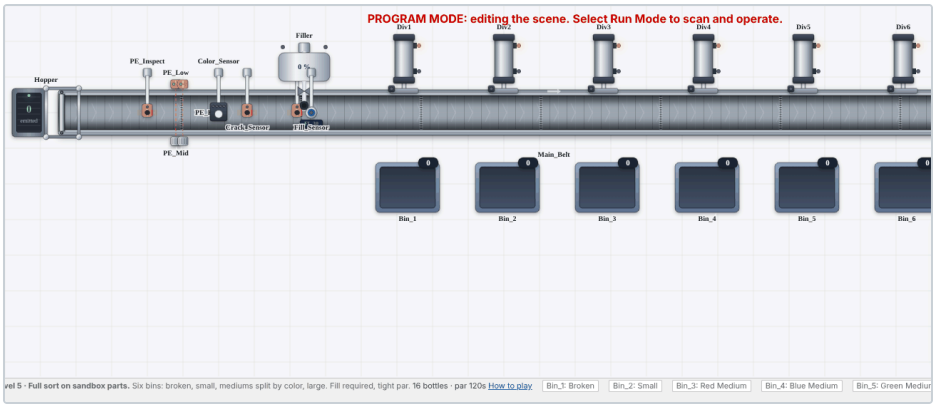


Figure 57. Bins with grading rules under the six pushers of Level 5; the rule shows in the Specs chips.

Configuration

key	Setting	Default	Range / choices
`width`	Width (px)	90	50 – 200, step 10
`capacity`	Full at (parts)	10	1 – 100, step 1
`gKind`	Grading: bottles this bin should get	not graded	not graded, whole bottles, broken bottles, good finished bottles, defective bottles
`gSize`	Grading: size	any	any, S, M, L
`gColor`	Grading: colour	any	any, R, B, G, Y
`gFill`	Grading: must be filled	no	no, yes

Signals (with the default settings)

Signal	Kind	Meaning
`Full`	Digital input (to the controller)	Bin holds its capacity
`Count`	Analog input	Parts in the bin

Raised Conveyor (mezzanine deck, +1.7 m) `preset of belt`

A conveyor on a raised deck. Everything placed in its lane stands on the deck; a gate on it drops bottles into a Grinder hopper or down a catch incline onto a floor-level conveyor beside it

Places a Belt / Roller / Chain Conveyor with `elev` = Raised deck (+1.7 m).

Divert Conveyor (floor level, catch incline) `preset of belt`

A floor-level roller conveyor. Lay it alongside a raised conveyor and a cleated catch incline appears between them where a gate kicks bottles down to it

Places a Belt / Roller / Chain Conveyor with `length` = 4, `speed` = 120, `style` = roller.

Box Conveyor (roller, for scrap boxes) `preset of belt`

A roller conveyor for the scrap boxes: run it under the Grinder chute with a photoeye set to boxes only

Places a Belt / Roller / Chain Conveyor with `length` = 10, `speed` = 80, `style` = roller, `beltColor` = Black.

Box Feeder (cartons, automatic) `preset of emitter`

Drops a carton every 4 s at the head of a box conveyor

Places a Emitter (part source) with `parts` = ['cartonM'], `color` = Y, `mode` = automatic (no Emit input), `minTime` = 4, `maxTime` = 4, `clearTime` = True, `hazard` = True.

Actuators

Motor (AC, DC, stepper, servo) `motor`

Configuration

key	Setting	Default	Range / choices
`kind`	Motor type	Three-phase AC	Three-phase AC, Single-phase AC, DC (PWM drive), Stepper, Servo
`color`	Housing color	Grey	Grey, Blue, Green, Black, Red, Orange, Yellow, White, Stainless steel
`size`	Size	Medium	Small, Medium, Large
`rpm`	Rated speed (rpm)	1750	100 – 6000, step 50
`reversible`	Reversible (adds Rev output)	False	on / off
`vfd`	VFD speed reference (adds analog output, 0-100 %)	False	on / off
`drive`	Driven by drive module (its name; blank = contactor)		text
`brake`	Spring-applied brake (adds Brake output, 1 = release)	False	on / off
`gearbox`	Gearbox on the shaft	False	on / off
`ratio`	Gearbox ratio	10:1	5:1, 10:1, 20:1, 50:1
`tach`	Speed feedback (adds Speed analog input, rpm)	False	on / off
`encoder`	Incremental encoder (adds A, B and Pulses)	False	on / off
`encPPR`	Encoder pulses per revolution	24	1 – 1024, step 1
`control`	Stepper control	Step and direction pulses	Step and direction pulses, Pulse rate (analog) and direction
`stepsRev`	Steps per revolution	200	20 – 6400, step 20
`servoMode`	Servo mode	Position	Position, Velocity
`servoSpeed`	Servo maximum speed (degrees/s)	720	30 – 7200, step 30
`servoAccel`	Servo acceleration (degrees/s²)	3600	90 – 36000, step 90

Signals (with the default settings)

Signal	Kind	Meaning
`Run`	Digital output (from the controller)	Contactor forward
`Running`	Digital input (to the controller)	Aux contact

Signal	Kind	Meaning
`Overload`	Digital input (to the controller)	Overload relay OK (NC, 1 = healthy; click the motor in Run mode to trip or reset it)

Pusher (pneumatic sorter) `pusher`

Configuration

key	Setting	Default	Range / choices
`stroke`	Stroke time (s)	0.15	0.05 – 1, step 0.05

Signals (with the default settings)

Signal	Kind	Meaning
`Extend`	Digital output (from the controller)	Solenoid: extend cylinder
`Extended`	Digital input (to the controller)	Reed switch: cylinder extended
`Retracted`	Digital input (to the controller)	Reed switch: cylinder retracted
`Detect`	Digital input (to the controller)	Photoeye in front of the pusher

Pivot Arm Sorter `pivot`

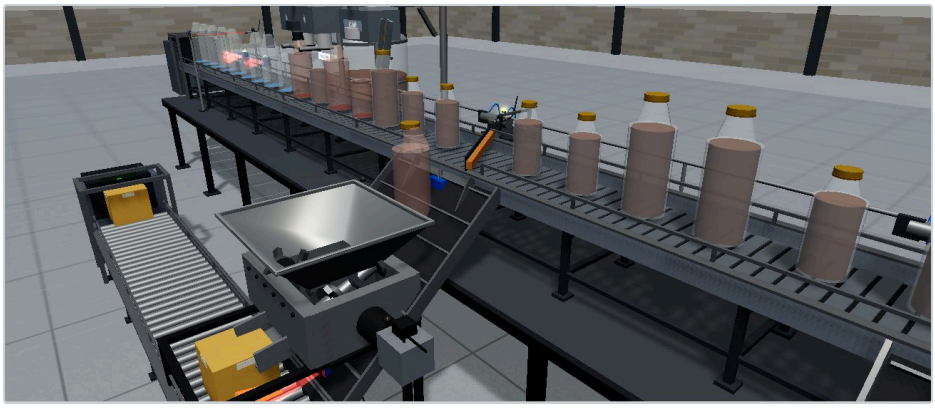


Figure 58. A Pivot Arm Sorter on a raised belt becomes a latch door: the arm swings and the bottle drops through.

Configuration

key	Setting	Default	Range / choices
`stroke`	Swing time (s)	0.12	0.05 – 1, step 0.05

Signals (with the default settings)

Signal	Kind	Meaning
`Turn`	Digital output (from the controller)	Swing the arm into the lane
`Turned`	Digital input (to the controller)	Arm at end of travel
`Detect`	Digital input (to the controller)	Photoeye ahead of the arm

Stop Gate (pneumatic stopper) `gate`

No settings.

Signals (with the default settings)

Signal	Kind	Meaning
`Raise`	Digital output (from the controller)	Raise the stopper into the lane
`Up`	Digital input (to the controller)	Stopper up
`Down`	Digital input (to the controller)	Stopper down

Drives & Axes

Linear Axis (axis and servo motor; its drive is the Festo FHPP module) `axis`

Three things make a servo axis: the **axis** (a toothed-belt or spindle profile with a carriage: the part), the **servo motor** on its end (drawn with it) and the **drive**, the servo controller in the cabinet, which is the module in I/O Configuration. The PLC talks to the drive over EtherNet/IP with the **FHPP** profile: eight bytes each way. Out: **CCON** (Enable, Stop, Brake, Reset, Lock, OPM1/2), **CPOS** (Halt, Start, Home, Jog_Pos, Jog_Neg, Teach, Clear), Record_No, Setpoint_Vel, Setpoint_Pos. In: **SCON** (Enabled, Op_Enabled, Warning, Fault …), **SPOS** (Halt_Ack, Start_Ack, Motion_Complete, Teach_Ack, Moving, Drag_Error, Standstill, Referenced), Record_No, Actual_Vel, Actual_Pos (0.01 mm). The part has no wired signals: type its module's name into *Axis module*, or press **Add Axis Module**. The handshake is the real one: **Enable + Stop** give operation enabled; **Home** is the only move before the axis is referenced; **Start** runs the record (or the Direct-mode setpoint, OPM1 = 1) on its rising edge, and Motion_Complete drops until the target is reached; **Halt** low pauses a task and Start resumes it, **Clear** drops it; a Start while moving is ignored with Warning; a Start before homing, an unset record or a target past the stroke is a fault only Reset clears. Jog moves while held and stops at the ends; Teach writes the actual position into the record. Stroke, speeds and records are the part's properties; the axis is drawn to scale (60 px per metre). **Gripper** (v1.21) puts a vacuum gripper on the carriage: *Grip* rising lifts the part under the carriage and it rides along, unseen by eyes, belts and bins; *Grip* falling sets it down where the carriage is (a bin, a belt, or the floor); *Gripped* and *Part_Below* report. **Axis size** picks the profile (Festo builds them in sizes 45 to 185 mm wide, toothed belt or ball-screw spindle): it sets the drawn width, the stroke range and the velocity and acceleration ceiling, and a setting past the ceiling is held at it. The figures are typical of a catalogue, not of one drive: check the axis you buy. On a live module the part follows the real drive's Actual_Pos.

Configuration

key	Setting	Default	Range / choices
`module`	Axis module (its name in I/O Configuration: a Festo FHPP drive; blank = no drive, the axis stands still)		text
`size`	Axis size (profile width: sets the drawn size, the stroke range and the speed ceiling)	Size 70 (toothed belt)	Size 45 (compact toothed belt), Size 70 (toothed belt), Size 120 (toothed belt), Size 185 (heavy-duty toothed belt), Size 70 spindle (ball screw: slow, strong, precise)
`stroke`	Stroke (mm; drawn to scale, 60 px per metre)	2000	100 – 8000, step 50
`vmax`	Maximum velocity (mm/s; 100 % in a record)	500	10 – 5000, step 10
`accel`	Acceleration (mm/s²)	2000	100 – 50000, step 100
`homeVel`	Homing velocity (mm/s; to the home switch at 0)	50	1 – 500, step 1

key	Setting	Default	Range / choices
`jogVel`	Jog velocity (mm/s)	100	1 – 2000, step 1
`records`	Records 1, 2, 3 …: position mm @ velocity %, comma separated (Teach writes here)	200@50, 1000@80, 1800@100	text
`mount`	Mounting	Floor stand	Floor stand, On the deck
`gripper`	Gripper on the carriage (v1.21: Grip picks the part under the carriage, Gripped and Part_Below report)	None	None, Vacuum gripper

No signals.

VFD (variable frequency drive, hardwired: 0-10 V speed reference) `vfd`

A variable frequency drive wired to the controller the way most are: two-wire **Run** (and **Rev**) from digital outputs, a **0-10 V speed reference** from an analog output channel, and **Ready**, **Running**, **AtSpeed** and **Fault** back to digital inputs, plus its analog output (**OutputFreq**) for the actual frequency. The analog channels are in engineering units, 0-10000 = 0-10.000 V, and 10 V is the *Frequency at 10 V* setting, so a reference of 5000 asks for 30 Hz on a 60 Hz drive: scale it in the program (SCP or CPT). The drive ramps with its *Accel* and *Decel* times, so a conveyor told to stop from full speed coasts on for a while: slow it down first if it has to stop on a mark. A trip (Tools → Inject Machine Fault) switches the output off and latches until the cause is gone *and* FaultReset rises. To put it on a conveyor, type its name in the belt's *Driven by drive module*: the belt then runs at its *Speed* × output frequency ÷ the 10 V frequency, and its own Fwd output is not used.

Configuration

key	Setting	Default	Range / choices
`maxHz`	Frequency at 10 V (Hz)	60	10 – 120, step 1
`accel`	Accel time, 0 to max (s)	1	0.1 – 30, step 0.1
`decel`	Decel time, max to 0 (s)	0.5	0.1 – 30, step 0.1
`atBand`	At Speed band (± Hz)	0.5	0.1 – 5, step 0.1
`fbk`	Analog output: output frequency (adds OutputFreq)	True	on / off

Signals (with the default settings)

Signal	Kind	Meaning
`Run`	Digital output (from the controller)	Run while 1 (two-wire control)
`Rev`	Digital output (from the controller)	Reverse while 1, with Run
`FaultReset`	Digital output (from the controller)	Clears a trip on the rising edge, once its cause has gone
`SpeedRef`	Analog output	Speed reference: 0-10000 = 0-10.000 V = 0 to the 10 V frequency
`Ready`	Digital input (to the controller)	Powered and not tripped
`Running`	Digital input (to the controller)	Output frequency above zero (also while decelerating)
`AtSpeed`	Digital input (to the controller)	Running and within the At Speed band of the reference
`Fault`	Digital input (to the controller)	Tripped: output off until FaultReset
`OutputFreq`	Analog input	Output frequency: 0-10000 = 0-10 V = 0 to the 10 V frequency

Sensors

Weighing Conveyor `scale`

Configuration

key	Setting	Default	Range / choices
`speed`	Speed (mm/s)	120	20 – 400, step 10

Signals (with the default settings)

Signal	Kind	Meaning
`Fwd`	Digital output (from the controller)	Motor forward
`Weight`	Analog input	Weight on the scale, kg
`Running`	Digital input (to the controller)	Motor aux contact

Fill Level Sensor (liquid in the bottle) `level`

Configuration

key	Setting	Default	Range / choices
`height`	Fill line (px above the belt; a small bottle is 40 tall, a large 56; the liquid tops out at about 80 % of that)	30	2 – 80, step 1
`type`	Sensor	Capacitive (through the glass)	Capacitive (through the glass), Photoelectric (clear-liquid eye)
`analog`	Analog output (adds a Percent analog input: fill of the bottle under the sensor, 0-100 % of its label)	False	on / off

Signals (with the default settings)

Signal	Kind	Meaning
`Detect`	Digital input (to the controller)	Liquid at or above the fill line in the bottle under the sensor

Diffuse Sensor (photoelectric) `pe`

Diffuse photoelectric sensor: its own light reflects off the part, so no reflector is needed. The shaded band on the lane shows the sensing range.

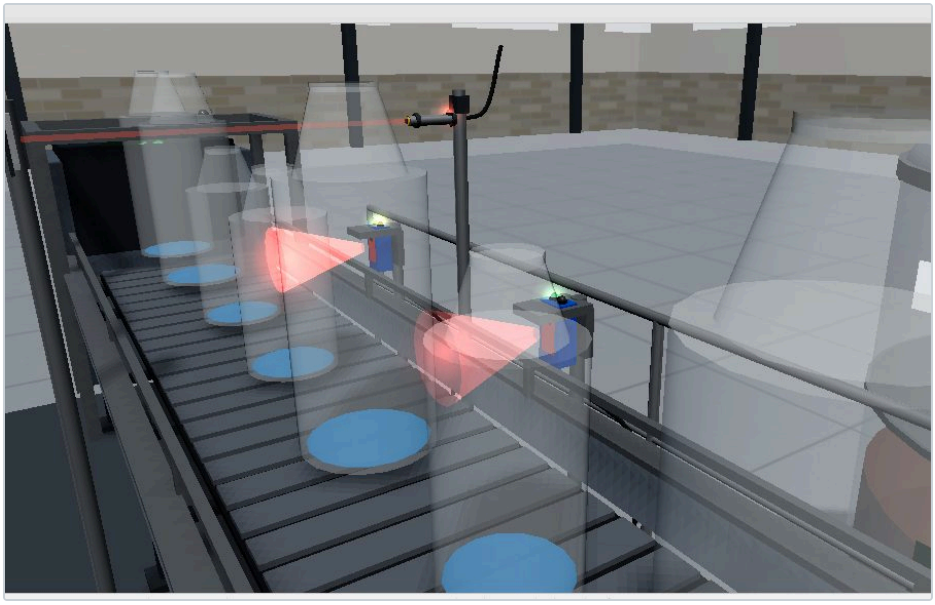


Figure 59. Sensors carry a real indicator LED: lens, bezel and a halo when lit.

Configuration

key	Setting	Default	Range / choices
`range`	Sensing range (px; 60 px = 1 m)	20	2 – 100, step 1
`sensitivity`	Sensitivity (%): the gain. 100 sees every part at full range; lower it and the range shrinks and dull targets (glass bottles, broken glass, dark cartons) drop out before bright ones (metal, cans, white and yellow cartons)	100	10 – 100, step 5
`beam`	Beam	Wide (diffuse)	Wide (diffuse), Narrow (focused spot), Laser (pencil)
`sees`	Sees	any part	any part, bottles only, broken bottles, filled bottles, capped bottles, labeled bottles, empty bottles, boxes only

Signals (with the default settings)

Signal	Kind	Meaning
`Detect`	Digital input (to the controller)	Detecting: a solid part is within range

Retroreflective Sensor and Reflector `retro`

The sensor and its reflector face each other across the conveyor. **Detect** is 1 while the beam is interrupted. Use the beam height to ignore low parts: a lid (8 px) passes under a 12 px beam, a carton does not.

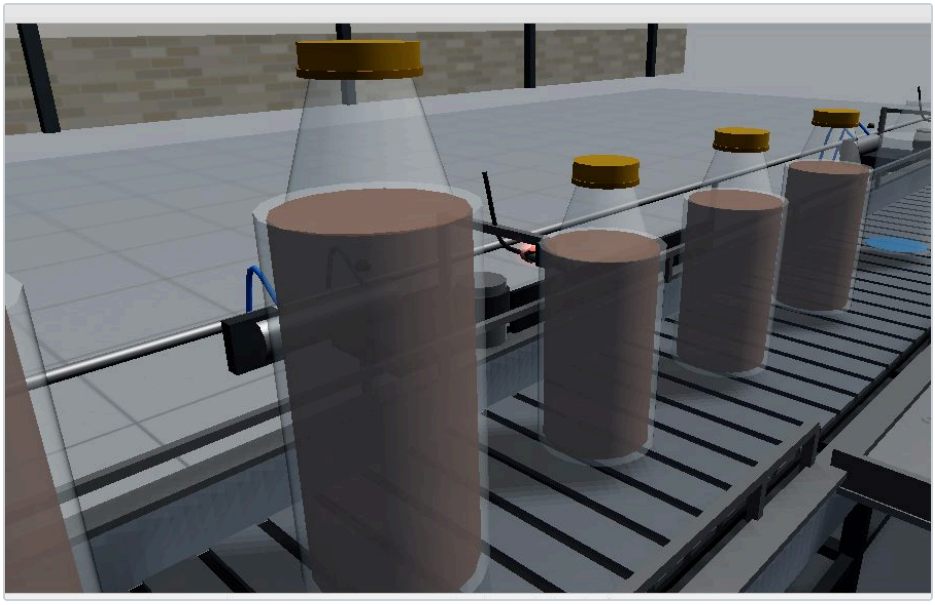


Figure 60. A retroreflective beam at height: only a bottle taller than the beam breaks it.

Configuration

key	Setting	Default	Range / choices
`height`	Beam height (px above the belt)	12	2 – 80, step 1

Signals (with the default settings)

Signal	Kind	Meaning
`Detect`	Digital input (to the controller)	Light beam interrupted

Light Array (8 beams) `array`

Eight parallel beams form a light curtain. Beam 1 is lowest. **Digital** gives one BOOL per beam, **Numerical** packs the beams into one INT (bit 0 = beam 1), **Analog** gives 10 V × interrupted beams ÷ 8 as hundredths of a volt. With the default 6 px spacing a small carton (22 px) breaks beams 1-4, a medium one 5, a large one 7, a crate all 8, a lid only beam 1.

Configuration

key	Setting	Default	Range / choices
`mode`	Output	digital	digital, numerical, analog
`pitch`	Beam spacing (px of part height)	6	2 – 20, step 1

Signals (with the default settings)

Signal	Kind	Meaning
`Beam1`	Digital input (to the controller)	Beam 1 interrupted (3 px above the belt)
`Beam2`	Digital input (to the controller)	Beam 2 interrupted (9 px above the belt)
`Beam3`	Digital input (to the controller)	Beam 3 interrupted (15 px above the belt)
`Beam4`	Digital input (to the controller)	Beam 4 interrupted (21 px above the belt)
`Beam5`	Digital input (to the controller)	Beam 5 interrupted (27 px above the belt)
`Beam6`	Digital input (to the controller)	Beam 6 interrupted (33 px above the belt)
`Beam7`	Digital input (to the controller)	Beam 7 interrupted (39 px above the belt)
`Beam8`	Digital input (to the controller)	Beam 8 interrupted (45 px above the belt)

Inductive Sensor (metal) `inductive`

Proximity sensor for metal. **Digital** gives Detect. **Analog** gives Volts: 0 V with the part at the sensor, rising to 10 V (1000) at the edge of the range, and 1000 when nothing metal is in range.

Configuration

key	Setting	Default	Range / choices
`output`	Output	digital	digital, analog
`range`	Sensing range (px; 60 px = 1 m)	10	2 – 60, step 1

Signals (with the default settings)

Signal	Kind	Meaning
`Detect`	Digital input (to the controller)	Detecting: a conductive (metal) part is within range

Capacitive Sensor (any material) `capacitive`

Short-range proximity sensor for anything. **Digital** gives Detect. **Analog** gives Volts exactly like the inductive sensor, without the material filter.

Configuration

key	Setting	Default	Range / choices
`output`	Output	digital	digital, analog
`range`	Sensing range (px; 60 px = 1 m)	12	2 – 60, step 1

Signals (with the default settings)

Signal	Kind	Meaning
`Detect`	Digital input (to the controller)	Detecting: any solid part is within range

Color Sensor (R/B/G/Y banks) `color`

Four colour banks, one BOOL each.

No settings.

Signals (with the default settings)

Signal	Kind	Meaning
`Red`	Digital input (to the controller)	Bank 1: red
`Blue`	Digital input (to the controller)	Bank 2: blue
`Green`	Digital input (to the controller)	Bank 3: green
`Yellow`	Digital input (to the controller)	Bank 4: yellow

Vision Sensor `vision`

Recognises the part under it. **Digital**: one BOOL per ticked part. **Bits**: the part code as five binary inputs Bit0-Bit4 (Factory I/O's "All Digital"). **Numerical**: one INT with the part code (the number next to each part in the list; 0 = none or not ticked). **ID**: the serial number of the part in view plus its code, usable like a barcode. **Size**: the older Small/Medium/Large/Metal outputs. Tick **Triggered** for a Trigger output with Busy and Complete, like a real smart camera: the result then holds until the next trigger. Set **Inspection job** to **Bottle inspection** and it checks a bottle's cap, label and fill instead: Pass is a bottle with all three, and on a camera module the result code is bit 0 no cap, bit 1 no label, bit 2 short fill, bit 3 no part in view, with the fill in % of the label in byte 9 of Inspection_Results. On a camera module, **Buffer_Results_Enable** queues results up to the buffer depth until the PLC acknowledges each one (Results_Buffer_Overrun when one is lost). A Process HMI set to show the sensor draws its camera screen. **Card mode** (bottle job): the bottle cards SN-4801 to SN-4804 decide each bottle. The hopper sends plain good bottles; when a result comes back, the bottle under the camera takes the card's verdict (no cap, no label, short fill; no card in view makes it a reject) and moves on. Each bottle waits at the camera until its result is presented, so there is time to swap the card. With the simulated camera, pick the card in **Card under the lens** or on the camera screen's CARDS page; with the camera module live on a real In-Sight, the real card decides: its result code, the fill in byte 9 and the card ID in byte 10 of Inspection_Results.

Configuration

key	Setting	Default	Range / choices
`mode`	Output	digital	digital, bits, numerical, id, size, camera tags only
`camera`	Camera module (its name in I/O Configuration; blank = wired signals only)		text
`housing`	Housing	Charcoal	Charcoal, Yellow, Blue, Silver
`detect`	Parts to Detect	cartonS, cartonM, cartonL, metalM	1 · Carton (S), 2 · Carton (M), 3 · Carton (L), 4 · Shipping Crate (XL), 5 · Tote, 6 · Metal Block (S), 7 · Metal Block (M), 8 · Metal Block (L), 9 · Raw Stock, Blue, 10 · Raw Stock, Green, 11 · Raw Stock, Metal, 12 · Housing Base, Blue, 13 · Housing Base, Green, 14 · Housing Base, Metal, 15 · Housing Lid, Blue, 16 · Housing Lid, Green, 17 · Housing Lid, Metal, 18 · Bottle (S), 19 · Can, 20 · Bottle (M), 21 · Bottle (L)
`trigger`	Triggered (adds Trigger input, Busy and Complete)	False	on / off
`time`	Inspection time (ms)	150	0 – 2000, step 10
`job`	Inspection job	Part recognition	Part recognition, Bottle inspection (cap, label, fill), Bottle code verify (cap, label, fill, lot code)

key	Setting	Default	Range / choices
`minFill`	Bottle job: the fill passes at (% of the label)	95	50 – 100, step 1
`bufDepth`	Results buffer depth (camera module, with Buffer_Results_Enable on)	4	1 – 16, step 1
`cards`	Card mode (bottle job): the card under the camera decides what is wrong with each bottle, and each bottle waits at the camera for its result	False	on / off
`jobs`	Camera jobs, in job ID order: PRODUCT:min fill %, … (Level 9; blank = the one job above)		text
`card`	Card under the lens (simulated camera; a live camera reads the real card)	SN-4801 SODA good	SN-4801 SODA good, SN-4802 SODA no cap, SN-4803 SODA no label, SN-4804 SODA short fill, SN-4811 WATER good, SN-4812 WATER no cap, SN-4813 WATER no label, SN-4814 WATER short fill, SN-4821 MILK good, SN-4822 MILK no cap, SN-4823 MILK no label, SN-4824 MILK short fill, SN-4831 JUICE good, SN-4832 JUICE no cap, SN-4833 JUICE no label, SN-4834 JUICE short fill, SN-4841 OIL good, SN-4842 OIL no cap, SN-4843 OIL no label, SN-4844 OIL short fill, No card
`ftpImg`	Live image: after each inspection of a live camera, fetch the image it acquired (image.bmp, over FTP through the bridge) and show it on the camera screen	True	on / off
`ftpUser`	Live image: FTP user on the camera (a user with FTP Read)	admin	text
`ftpPass`	Live image: FTP password (blank for the In-Sight default admin)		text
`ftpMinS`	Live image: at most one image every (s) — each image is work for the camera	1	0 – 30, step 0.5
`ftpW`	Live image: width to fetch (px; the bridge scales the image down to this)	800	128 – 4096, step 16
`liveLayout`	Live camera: where the inspection ID and result code are read	Module fields	Module fields, Results buffer (ID bytes 0-3, code bytes 4-5)

Signals (with the default settings)

Signal	Kind	Meaning
`Detect`	Digital input (to the controller)	A part is in the field of view
`Pass`	Digital input (to the controller)	The part is one of the selected parts
`CartonS`	Digital input (to the controller)	Carton (S) in view
`CartonM`	Digital input (to the controller)	Carton (M) in view
`CartonL`	Digital input (to the controller)	Carton (L) in view
`MetalM`	Digital input (to the controller)	Metal Block (M) in view

RFID Reader `rfid`

Single-trigger command interface: set **Command**, pulse **Execute**; when **CommandID** increments the command has finished and **Status** says how it went (0 = success). Command 0 only checks for a tag, 1 reads the serial into ReadData, 2 reads the word at MemoryIndex, 3 writes WriteData there, 4 clears the memory. Two tags in range give status 2. Write a route code into a part at one station and sort on it at the next.

Configuration

key	Setting	Default	Range / choices
`range`	Reading range (px)	16	4 – 60, step 1
`time`	Command time (ms)	100	0 – 2000, step 10

Signals (with the default settings)

Signal	Kind	Meaning
`Command`	Analog output	Command to execute: 0 = is a tag available?, 1 = read serial number, 2 = read data, 3 = write data, 4 = clear all data
`Execute`	Digital output (from the controller)	Rising edge executes the command
`MemoryIndex`	Analog output	Tag memory index for read/write (0-15)
`WriteData`	Analog output	Data written by command 3
`CommandID`	Analog input	Number of commands executed so far; changes when the current one has finished
`Status`	Analog input	0 = success, 1 = no tag, 2 = too many tags, 3 = invalid index, 4 = invalid command
`ReadData`	Analog input	Result of command 1 (serial) or 2 (data)
`TagPresent`	Digital input (to the controller)	Exactly one tag is within range

Box Photoeye (LS10, boxes only) `preset of pe`

A diffuse eye that only reports boxes: LogixPro LS10 under the grinder chute

Places a Diffuse Sensor (photoelectric) with `range` = 12, `sees` = boxes only.

Broken-bottle Eye (LS3) `preset of pe`

A diffuse eye that only reports broken bottles

Places a Diffuse Sensor (photoelectric) with `range` = 14, `sees` = broken bottles.

Bottling

Fill Station (tube, large and small fill) `filler`

Fill station as on the LogixPro bottle line, built as a real one: a stainless frame over the conveyor, the head tank as the product bowl on top (its sight glass shows the level; it refills from the reservoir it is fed from), the two fill valves on pneumatic cylinders, the nozzle on a lift, a guard with an interlocked door, a LARGE / SMALL / TUBE / LOW lamp panel and a stack light (green fed and filling, amber head tank low, red spilling). **FillTube** lowers the tube (LS4 **TubeDown** when it arrives); with the tube down and a bottle under it, **LargeFill** or **SmallFill** pours at the fill rate. **LargeCharge** (LS5) and **SmallCharge** (LS6) come on when that valve has delivered its charge since it opened, so close the valve on the charge sensor. A small bottle holds 60 units and a large 100; pouring on past the brim overflows, a valve open with no bottle spills, and a broken bottle leaks instead of filling.

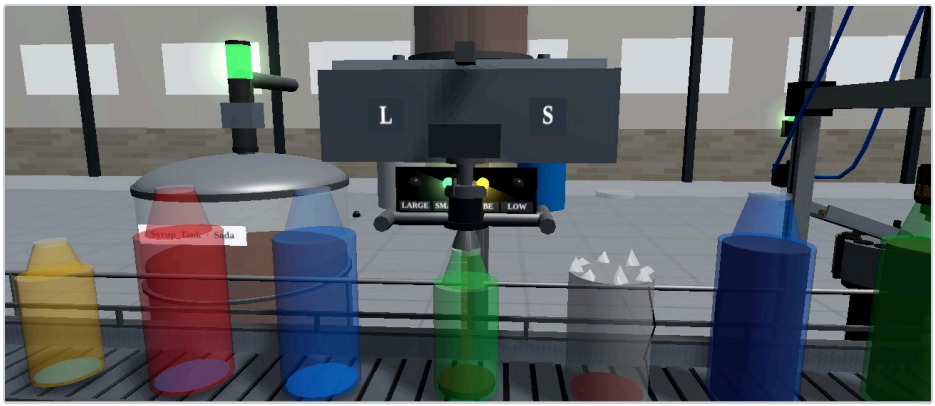


Figure 61. The Fill Station: tube, head tank with level, and the four-lamp indicator panel (L / S charge, tube down, done).

Configuration

key	Setting	Default	Range / choices
`rate`	Fill rate (units/s; a small bottle holds 60, a large 100)	80	10 – 400, step 10
`largeCharge`	Large charge (units)	100	10 – 400, step 10
`smallCharge`	Small charge (units)	60	10 – 400, step 10
`tubeTime`	Tube travel time (s)	0.4	0.1 – 2, step 0.1
`feed`	Fed from		text
`tank`	Head tank (units; refilled from the reservoir, pours from here)	300	50 – 2000, step 50

Signals (with the default settings)

Signal	Kind	Meaning
`FillTube`	Digital output (from the controller)	Lower the fill tube into the bottle
`LargeFill`	Digital output (from the controller)	Large fill valve

Signal	Kind	Meaning
`SmallFill`	Digital output (from the controller)	Small fill valve
`TubeDown`	Digital input (to the controller)	Fill tube down (LS4)
`LargeCharge`	Digital input (to the controller)	Large charge delivered since the large valve opened (LS5)
`SmallCharge`	Digital input (to the controller)	Small charge delivered since the small valve opened (LS6)
`TankLevel`	Analog input	Head tank level 0-100 %
`TankLow`	Digital input (to the controller)	Head tank below 15 %: the reservoir is not keeping up

Inline Cap Feeder (pneumatic capper + cap bowl) `capper`

A pneumatic capping machine as on the LogixPro line, built as a real capping station: four posts and a deck straddling the lane, the head (spindle motor, clutch, knurled chuck) hung from the deck on its air cylinder, guards front and back with an interlocked door, a centrifugal cap sorter on its stand behind the lane feeding a twin-rail chute, a CAP / FEED / DOWN / LOW panel on its own signals, and LampGreen and LampAmber as a two-tier stack light. **CapRam** drives the head down, **RamDown** (LS7) reports the bottom of the stroke, and the bottle under the chuck gets a cap from the chute on the way down. The chute holds *Chute holds* caps and starts with *Caps at start*; **CapPresent** says a cap is waiting, **CapLow** comes on at the *CapLow below* count, and **Feeder** runs the rotary magazine that refills the chute at *Cap feeder rate*. A cycle with an empty chute leaves the bottle open and is logged. **LampGreen** and **LampAmber** are the two indicator lights on the column: yours to program (ready / cycling, caps low…).

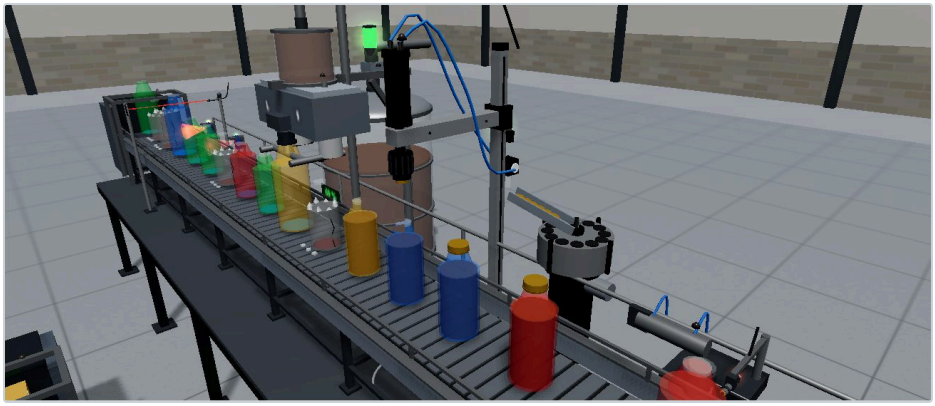


Figure 62. The Inline Cap Feeder: pneumatic ram, the cap chute with its reed switch, the rotary magazine that refills it, and the two column lamps.

Configuration

key	Setting	Default	Range / choices
`stroke`	Ram travel time (s)	0.3	0.1 – 2, step 0.1
`capsStart`	Caps in the chute at start	30	0 – 200, step 1
`chuteCap`	Chute holds (caps)	40	5 – 200, step 1
`lowAt`	CapLow below (caps)	8	1 – 100, step 1
`feedRate`	Cap feeder rate (caps/s)	3	0.5 – 20, step 0.5

Signals (with the default settings)

Signal	Kind	Meaning
`CapRam`	Digital output (from the controller)	Drive the capping head down
`Feeder`	Digital output (from the controller)	Run the rotary cap magazine (refills the chute)

Signal	Kind	Meaning
`LampGreen`	Digital output (from the controller)	Green tier of the stack light on the deck
`LampAmber`	Digital output (from the controller)	Amber tier of the stack light on the deck
`RamDown`	Digital input (to the controller)	Head at the bottom of its stroke (LS7)
`CapLow`	Digital input (to the controller)	Chute at or below the CapLow count
`CapPresent`	Digital input (to the controller)	A cap is waiting at the chuck

Date Coder (laser marker or inkjet: marks the lot code on the bottle under the head)

`coder`

A date coder, as the industry calls it: a **CO_2 laser marker** (Keyence ML / Videojet 3000 class) or a small-character inkjet, over the lane. It writes its message, `L` + the four-digit lot + a running count, on the bottle under the head on the rising edge of **Print**. The laser marks only while **Laser_On** arms it (its interlock). The message is loaded through a handshake, as on the real machines: put the lot in **Lot_No**, pulse **Load_Msg**, wait the load time with **Ready** off, and **Msg_Ready** says the message carries Lot_No. A mark is missed now and then (the bottle rocked, a nozzle skip); after *Marks before service* the protective window has fumed up (**Window_Dirty**; **Ink_Low** on the inkjet) and every other mark comes out faint, which a camera cannot read, until the head is clicked (the window cleaned, a cartridge fitted). **Detect** is the head's own eye. The Vision Sensor's *Bottle code verify* job reads the code back: 32 wrong lot code against User_Data bytes 0-1, 64 no code.

Configuration

key	Setting	Default	Range / choices
`type`	Type	Laser marker (CO2)	Laser marker (CO2), Inkjet (CIJ)
`miss`	Missed marks (%: the bottle rocked, a nozzle skip)	8	0 – 50, step 1
`service`	Marks before service (the window needs cleaning / the cartridge is low; then every other mark is faint)	14	1 – 500, step 1
`load`	Message load time (s)	0.5	0.1 – 3, step 0.1

Signals (with the default settings)

Signal	Kind	Meaning
`Print`	Digital output (from the controller)	Mark the message on the bottle under the head (rising edge)
`Laser_On`	Digital output (from the controller)	Arm the laser (the interlock): no mark while 0
`Load_Msg`	Digital output (from the controller)	Load Lot_No into the message (rising edge; takes the load time)
`Lot_No`	Analog output	The lot number for the message, 1-9999
`Ready`	Digital input (to the controller)	Powered, armed and able to mark (not loading a message)
`Marking`	Digital input (to the controller)	A mark is in progress
`Msg_Ready`	Digital input (to the controller)	The message in the coder carries Lot_No (loaded and not changing)
`Window_Dirty`	Digital input (to the controller)	The protective window has fumed up: marks come out faint until it is cleaned

Signal	Kind	Meaning
`Detect`	Digital input (to the controller)	The head's eye: a bottle is under the head

Inline Labeler (FLS-style dispenser, RH / LH, any applicator) `labeler`

A pressure-sensitive label dispenser in the FLS 204 mould, built as a side labeler: a deck plate at label height on its housing and pedestal, the roll lying flat on its spindle, the web over the dancer arm and a guide to the peel blade whose nose stands at the product, back round the nose to the drive and nip rollers and the rewind, a label-gap fork on the blade, the applicator at the nose and a clear interlocked cover over the reels, a READY / LABEL / LOW / ERROR row on its own signals, and LampGreen and LampAmber as a two-tier stack light: modular, *RH* or *LH* (which side of the lane it stands on, looking downstream), with an *Applicator* of your choice (tamp pad, wipe-on dispensing edge with a brush, blow-on non-contact, wrap-around roller), a label width of 5-233 mm and a web that follows the conveyor or runs at a fixed speed. **Ready** says the web is loaded and the head is home; **Error** is end of roll or a cycle with no label. The next station after the capper: **Apply** drives the pad in, **PadIn** reports the end of the stroke, and the bottle under the pad gets a label from the roll. The roll holds *Roll holds* labels and starts with *Labels at start*; **LabelPresent** says a label is peeled and waiting, **LabelLow** comes on at the *LabelLow below* count, and **Labeled** says the bottle standing under the pad already carries a label. With *Only label capped bottles* on, an open bottle is skipped and logged. **LampGreen** and **LampAmber** are yours to program. A photoeye set to *labeled bottles* checks the work downstream.

Figure 63. The Inline Labeler one stop past the capper: the roll on its spindle, the web down to the pad, and labeled bottles leaving it.

Configuration

key	Setting	Default	Range / choices
`hand`	Hand (side of the lane the dispenser stands on, looking downstream)	RH	RH, LH
`label`	What it applies (v1.24)	Product label (bottles)	Product label (bottles), Hazard tape (scrap boxes)
`applicator`	Applicator	Tamp pad	Tamp pad, Wipe-on (dispensing edge + brush), Blow-on (non-contact), Wrap-around (roller)
`labelW`	Label width (mm, 5-233)	60	5 – 233, step 1

key	Setting	Default	Range / choices
`web`	Web speed	Follows the conveyor	Follows the conveyor, Fixed 20 m/min, Fixed 40 m/min, Fixed 70 m/min
`stroke`	Apply time (s)	0.4	0.1 – 2, step 0.1
`labelsStart`	Labels on the roll at start	200	0 – 2000, step 10
`rollCap`	Roll holds (labels)	500	10 – 5000, step 10
`lowAt`	LabelLow below (labels)	25	1 – 500, step 1
`labelColor`	Label colour	White	White, Cream, Blue, Red, Black
`capOnly`	Only label capped bottles (skips an open one and logs it)	True	on / off

Signals (with the default settings)

Signal	Kind	Meaning
`Apply`	Digital output (from the controller)	Start: drive the applicator in against the bottle
`LampGreen`	Digital output (from the controller)	Green tier of the stack light on the dispenser
`LampAmber`	Digital output (from the controller)	Amber tier of the stack light on the dispenser
`PadIn`	Digital input (to the controller)	Applicator at the end of its stroke (label pressed on)
`Ready`	Digital input (to the controller)	Dispenser ready: web loaded, no fault, not cycling
`Error`	Digital input (to the controller)	Dispenser fault: end of roll (empty) or a cycle with no label
`LabelLow`	Digital input (to the controller)	Roll at or below the LabelLow count
`LabelPresent`	Digital input (to the controller)	A label is peeled and waiting at the dispensing edge
`Labeled`	Digital input (to the controller)	The bottle under the applicator carries a label

Case Packer (top-load: infeed buffer, pick head, case lane beside the line; an optional second pick lane on a conveyor below) `packer`

A top-load case packer at the end of a line. Bottles that reach the end of the belt are taken into its **infeed** buffer (up to *Infeed buffer*); when it is full they wait on the belt at the backstop. A rising edge on **Drop** picks *Bottles per case* from the infeed (only when **GroupReady**) and places them in the case standing under the head on the case lane, *Case lane offset* px across the line. **CaseInPlace** is a case centred under the head within ±6 px; **CaseFull** is that case once packed. Drop with no case under the head, or onto a full case, and the bottles are lost. The case lane is an ordinary conveyor carrying cartons from an emitter: position the case with the program (a slow-down eye and a creep speed on a VFD is the usual way), stop it, drop, then run it out. The head goes home empty after every pack: the bottles stay in the case. Tick *Second pick lane below* and the packer also picks from a conveyor on the floor, *Lower lane offset* px across the line: the head runs out over it and drops the full 1.7 m to the bottle tops. That lane has its own stop and buffer, with **LowerReady**, **LowerFull** and **LowerCount**; the output **PickLower**, read on the rising edge of Drop, chooses the lane, so each case holds one size. **CaseSmall** says the packed case under the head came from the lower lane. A Process HMI set to show the packer draws a packer faceplate.

Configuration

key	Setting	Default	Range / choices
`perCase`	Bottles per case	4	1 – 12, step 1
`infeed`	Infeed buffer (bottles)	8	1 – 24, step 1
`cycle`	Pack cycle (s)	1.6	0.5 – 6, step 0.1
`lane`	Case lane offset across the line (px; negative = far side)	-108	-240 – 240, step 2
`lower`	Second pick lane below: the head drops to a conveyor on the floor (small bottles)	False	on / off
`lowerY`	Lower lane offset across the line (px; positive = near side)	166	40 – 320, step 2

Signals (with the default settings)

Signal	Kind	Meaning
`Drop`	Digital output (from the controller)	Pack: the rising edge picks a case worth from the infeed and places it in the case under the head
`Ready`	Digital input (to the controller)	Head home, not cycling
`GroupReady`	Digital input (to the controller)	The infeed holds at least a case worth
`InfeedFull`	Digital input (to the controller)	Infeed buffer full: bottles arriving now wait on the belt
`InfeedCount`	Analog input	Bottles in the infeed buffer

Signal	Kind	Meaning
`CaseInPlace`	Digital input (to the controller)	A case stands under the head (centre within ±6 px)
`CaseFull`	Digital input (to the controller)	The case under the head has been packed

Grinder (shredder under a raised belt, over a box conveyor) `grinder`

The shredder. Put it beside a **Raised Conveyor** (Height = Raised deck) with a pivot gate on the belt above it: the gate swings a broken bottle off the edge and it drops into the hopper, where it waits (**Detect**) until **Grinder** runs; each bottle is chewed between the shafts and the glass falls down the chute into the box on the conveyor 76 px below the hopper (a **Box Conveyor** with a **Box Feeder**). A box holds *Bottles per box*; **BoxFull** reports a full box under the chute and the level sensor beside the chute mouth gives **BoxLevel**, the number of bottles ground into the box so far (an analog input; the IN BOX display on the template panel shows it). Grinding with no box or a full box under the chute spills the glass and is logged. Put a **Box Photoeye** (boxes only) on the box conveyor under the chute for LS10.

Figure 64. The Grinder under a raised belt: hopper, shredder, chute, and the box conveyor below with its level sensor.

Configuration

key	Setting	Default	Range / choices
`perBox`	Bottles per box	4	1 – 30, step 1
`speed`	Grind time per bottle (s)	0.6	0.1 – 3, step 0.1

Signals (with the default settings)

Signal	Kind	Meaning
`Grinder`	Digital output (from the controller)	Run the grinder
`Detect`	Digital input (to the controller)	A bottle is waiting in the hopper
`BoxFull`	Digital input (to the controller)	The box under the chute holds its share
`BoxLevel`	Analog input	Level sensor on the chute: bottles ground into the box under it (0 with no box)

Dispenser (tank, valve, nozzle) `dispenser`

Configuration

key	Setting	Default	Range / choices
`rate`	Flow (units/s; a small bottle holds 60, a large 100)	60	10 – 400, step 10
`capacity`	Tank holds (units)	1000	100 – 10000, step 100
`start`	Starts at (%)	100	0 – 100, step 5
`refill`	Refill valve (adds Refill output; fills the tank at 4× the flow)	True	on / off
`empty`	Empty below (%)	5	0 – 50, step 1
`feed`	Fed from		text

Signals (with the default settings)

Signal	Kind	Meaning
`Valve`	Digital output (from the controller)	Open the valve: liquid runs out of the nozzle
`Refill`	Digital output (from the controller)	Refill the tank
`Level`	Analog input	Tank level 0-100 %
`Empty`	Digital input (to the controller)	Tank below the Empty level
`Flowing`	Digital input (to the controller)	Liquid is leaving the nozzle

Thumbwheel Switch (digits + Enter) `thumbwheel`

Digit wheels and an Enter button, like the BCD thumbwheel on the LogixPro panel. In Run mode tap the top half of a digit to turn it up and the bottom half to turn it down; **Value** reads the digits as one number and **Enter** is 1 while the button is held.

Configuration

key	Setting	Default	Range / choices
`digits`	Digits	4	1 – 6, step 1
`enter`	Enter button	True	on / off
`text`	Label	PRESET	text

Signals (with the default settings)

Signal	Kind	Meaning
`Value`	Analog input	The digits read as one number
`Enter`	Digital input (to the controller)	Enter button (momentary)

Bottle Feeder (LS1 edge, 15 % broken) `preset of emitter`

LogixPro’s bottle source: small and large bottles on each Emit edge, some of them broken

Places a Emitter (part source) with `parts` = ['bottle', 'bottleL'], `mode` = edge, `minTime` = 2.5, `maxTime` = 4, `broken` = 15.

Process

Supply Tote (an IBC of raw ingredient that tanks pump from) `supply`

A supply tote: an IBC of raw ingredient on its pallet, in its cage, with an outlet valve low on one side. Pipe it to a tank by naming it in that tank's *fed from* setting (a Mix Tank's *Inlet A* or *Inlet B*, a Tank's or Reservoir's *Inlet*, a filler's *Fed from*): the pump or valve on the other end draws from it, and when the tote runs empty the pump runs dry and nothing more goes in. **LowLevel** is the low switch; tick *Level transmitter* for a 0-100 % analog input and *Refill valve* to top it up from the plant header with the **Refill** output. In Run mode, click an empty tote to swap in a full one, as a forklift driver would.

Configuration

key	Setting	Default	Range / choices
`product`	Ingredient	Water	Water, Cola syrup, Lemon-lime syrup, Orange concentrate, Apple concentrate, Milk, Oil
`capacity`	Holds (units)	10000	500 – 100000, step 500
`start`	Starts at (%)	100	0 – 100, step 5
`low`	Low level switch (%)	10	0 – 50, step 1
`locked`	Locked out (LOTO): its valve is locked shut, it gives nothing	False	on / off
`levelAi`	Level transmitter (adds Level analog input, 0-100 %)	False	on / off
`refill`	Refill valve from the plant header (adds Refill output)	False	on / off
`refillRate`	Refill flow (units/s)	200	10 – 5000, step 10

Signals (with the default settings)

Signal	Kind	Meaning
`LowLevel`	Digital input (to the controller)	Low level switch: 1 while the tote is at or below its low level

Reservoir (product supply for fillers) `reservoir`

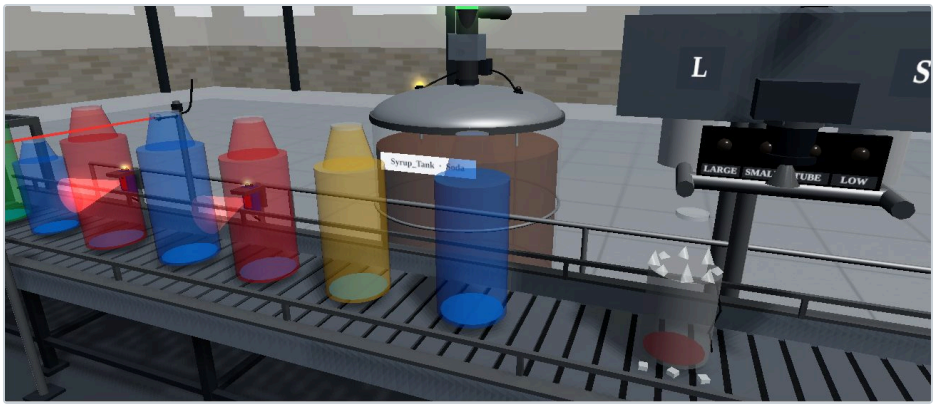


Figure 65. The Reservoir: dome, inlet valve, level transmitter and the high and low float probes; the pipe to the filler draws itself.

Configuration

key	Setting	Default	Range / choices
`product`	Product	Water	Soda, Lemon-lime, Juice, Apple juice, Water, Milk, Oil, Cola syrup, Lemon-lime syrup, Orange concentrate, Apple concentrate
`capacity`	Holds (units; a small bottle takes 60, a large 100)	5000	500 – 100000, step 500
`start`	Starts at (%)	100	0 – 100, step 5
`inflow`	Inlet flow (units/s)	200	10 – 5000, step 10
`high`	High float (%)	90	50 – 100, step 1
`low`	Low float (%)	15	0 – 50, step 1
`drain`	Drain valve (adds Drain output)	False	on / off
`flowAi`	Flow meter (adds Flow analog input; the sandbox has four analog inputs)	False	on / off
`levelSig`	Level transmitter signal	0-100 % (INT)	0-100 % (INT), Raw counts 0-32767 (1769-IF4 style, 4-20 mA), Engineering units 4000-20000 (4-20 mA)
`source`	Inlet fed from		text

Signals (with the default settings)

Signal	Kind	Meaning
`Inlet`	Digital output (from the controller)	Inlet valve: fills the reservoir at the inlet flow
`Level`	Analog input	Level 0-100 % (bind it to a REAL tag for the unrounded value)
`HighFloat`	Digital input (to the controller)	High level float switch

Signal	Kind	Meaning
`LowFloat`	Digital input (to the controller)	Low level float switch

Mix Tank (batch: pumps, flowmeters, heater, thermostat, mixer) `mixtank`

Heel (v1.23): the outlet sits above the bottom, so the outlet pump cannot draw below this share of the tank; the level transmitter reads it, the state shows HEEL, and a purge drains it. The heel carries the last batch into the next one: the tank keeps its contents by ingredient and a changeover is judged on what it carries over. A batch mixing vessel in the mould of the LogixPro Batch Mix. Ingredients come in through **PumpA** (and **PumpB** with two inlets), each through a **flowmeter** that gives one pulse per *Units per pulse* as a square wave, so a CTU on its input counts the dose. **HiLevel** and **LoLevel** are probes, 1 while the liquid covers them; **Level** is the transmitter. The **Heater** under the tank heats while its element is covered (below LO it scorches the batch); the tank loses heat to the room, and cold ingredient pulls the temperature down. **Temp** is °F; the **Thermostat** is a high-limit switch that closes at its setpoint. The **Mixer** counts mixing time only while the liquid covers the impeller (above LO). **PumpOut** empties the tank through the outlet flowmeter; type the tank's name in a filler's *Feed* and the filler draws its product through that pump instead, so the filler only refills while PumpOut runs. **V112:** the product is picked on the RECIPE page of its HMI (or the Operator View), not in Properties: picking one loads its recipe (A share, temperature, mixing time) into the device and the HMI's recipe tags. While a batch is in the tank and the process runs, the recipe is locked; STOP, PURGE and the inlet LOTO still work, and the program can still write the recipe tags.

Configuration

key	Setting	Default	Range / choices
`product`	Product (picked on the RECIPE page of its HMI)	Juice	Soda, Lemon-lime, Juice, Apple juice, Water, Milk, Oil
`capacity`	Holds (units)	2000	500 – 20000, step 100
`start`	Starts at (%)	0	0 – 100, step 5
`inlets`	Ingredient inlets	Two (A and B)	One (A), Two (A and B)
`rateA`	Pump A flow (units/s)	100	10 – 1000, step 10
`rateB`	Pump B flow (units/s)	100	10 – 1000, step 10
`rateOut`	Outlet pump flow (units/s)	150	10 – 1000, step 10
`pulse`	Flowmeters: units per pulse	20	1 – 500, step 1
`high`	HI level probe (%)	90	50 – 100, step 1
`low`	LO level probe (%)	10	0 – 50, step 1
`heat`	Heater: °F per second	3	0.5 – 20, step 0.5
`ambient`	Ambient and ingredient temperature (°F)	70	32 – 120, step 1
`setpoint`	Thermostat setpoint, high limit (°F)	180	80 – 250, step 1

key	Setting	Default	Range / choices
`levelSig`	Level transmitter signal	0-100 % (INT)	0-100 % (INT), Raw counts 0-32767 (1769-IF4 style, 4-20 mA), Engineering units 4000-20000 (4-20 mA)
`spTag`	Panel and HMI: heater setpoint tag (a REAL the operator sets on the TEMP page)		text
`mode`	Inlet mode (interlocks the pumps)	Auto (follow the product)	Auto (follow the product), Mix A + B, A only, B only, Same liquid (B fills from A), Water only (rinse)
`lockA`	Inlet A locked out (LOTO)	False	on / off
`lockB`	Inlet B locked out (LOTO)	False	on / off
`rcpRatio`	Recipe: A share the HMI aims for (%)	50	0 – 100, step 1
`rcpMix`	Recipe: mixing time (s)	10	0 – 600, step 1
`purgeRate`	Purge drain flow (units/s)	250	20 – 2000, step 10
`heel`	Heel (%): what the outlet pump cannot draw below (the outlet sits above the bottom); a purge drains it all. The heel carries the last batch into the next one (v1.23)	0	0 – 10, step 0.5
`startMixed`	Starts holding a finished batch: the start level made to the recipe A share and already mixed	False	on / off
`srcA`	Inlet A fed from		text
`srcB`	Inlet B fed from		text
`rateAi`	Flowmeters also report their rate on analog inputs (units/s; a REAL when bound to a REAL tag)	False	on / off

Signals (with the default settings)

Signal	Kind	Meaning
`PumpA`	Digital output (from the controller)	Ingredient A pump
`PumpB`	Digital output (from the controller)	Ingredient B pump
`PumpOut`	Digital output (from the controller)	Outlet pump: empties the tank, or feeds a filler piped to it
`Heater`	Digital output (from the controller)	Heater element under the tank
`Mixer`	Digital output (from the controller)	Mixer motor
`FlowA`	Digital input (to the controller)	Flowmeter A: one pulse per Units-per-pulse pumped in (square wave)
`FlowB`	Digital input (to the controller)	Flowmeter B: one pulse per Units-per-pulse

Signal	Kind	Meaning
`FlowOut`	Digital input (to the controller)	Outlet flowmeter: one pulse per Units-per-pulse pumped out
`HiLevel`	Digital input (to the controller)	HI probe: 1 while the liquid covers it
`LoLevel`	Digital input (to the controller)	LO probe: 1 while the liquid covers it
`Thermostat`	Digital input (to the controller)	High-limit thermostat: closes at its setpoint
`Level`	Analog input	Level 0-100 % (bind it to a REAL tag for the unrounded value)
`Temp`	Analog input	Temperature, °F

Carbonator (carbo-cooler: chills the blend and dissolves CO2) `carbonator`

A carbo-cooler, the machine that makes a blend into soda. Pipe it after the mix tank (its *Fed from*) and pipe the next tank from it. What goes through is chilled to *Product out* (38 °F) and runs through a carbonating tank held under CO2: the colder the product and the higher the pressure, the more CO2 it takes up (Henry's law: about 1.5 volumes per atmosphere at 38 °F, so 19-22 psi gives 3.5-3.8 volumes). Each 100 units going through takes some of the pressure with it, so the program holds the pressure in a band with the **CO2** valve, reading **Pressure** (psi). **Volumes** reports the CO2 in the product going out, in hundredths (350 is 3.50 volumes). A soda or lemon-lime bottle filled more than 0.3 volumes under its recipe is logged as flat. Leave the valve open and the **Relief** valve lifts at its setting.

Configuration

key	Setting	Default	Range / choices
`source`	Fed from		text
`outTemp`	Product out, chilled (°F)	38	33 – 70, step 1
`co2Rate`	CO2 valve open: pressure rise (psi/s)	6	1 – 40, step 1
`absorb`	CO2 taken up by 100 units of product (psi)	3	0.1 – 20, step 0.1
`relief`	Relief valve (psi)	60	20 – 120, step 5
`startP`	Starts at (psi)	0	0 – 60, step 1

Signals (with the default settings)

Signal	Kind	Meaning
`CO2`	Digital output (from the controller)	CO2 inlet valve: CO2 into the carbonating tank
`Pressure`	Analog input	Carbonating tank pressure, psi
`Volumes`	Analog input	CO2 in the product going out, in hundredths of a volume (350 = 3.50 volumes)
`Relief`	Digital input (to the controller)	Relief valve lifting: the tank is over pressure

Tank with valves & level transmitter `tank`

Configuration

key	Setting	Default	Range / choices
`inflow`	Inlet flow (%/s)	10	1 – 50, step 1
`outflow`	Outlet flow (%/s)	8	1 – 50, step 1
`mixer`	Mixer motor (adds output)	False	on / off
`high`	High float (%)	90	50 – 100, step 1
`low`	Low float (%)	10	0 – 50, step 1
`levelSig`	Level transmitter signal	0-100 % (INT)	0-100 % (INT), Raw counts 0-32767 (1769-IF4 style, 4-20 mA), Engineering units 4000-20000 (4-20 mA)
`product`	Product	Water	Soda, Lemon-lime, Juice, Apple juice, Water, Milk, Oil, Cola syrup, Lemon-lime syrup, Orange concentrate, Apple concentrate
`capacity`	Holds (units: what it gives a filler or another tank)	1000	100 – 20000, step 100
`source`	Inlet fed from		text

Signals (with the default settings)

Signal	Kind	Meaning
`Inlet`	Digital output (from the controller)	Inlet valve
`Outlet`	Digital output (from the controller)	Outlet valve
`Level`	Analog input	Level 0-100 % (bind it to a REAL tag for the unrounded value)
`HighFloat`	Digital input (to the controller)	High level float switch
`LowFloat`	Digital input (to the controller)	Low level float switch

Operator

Push Button `pb`

Configuration

key	Setting	Default	Range / choices
`color`	Color	green	green, red, amber, yellow, orange, blue, white, black
`style`	Style	Flush	Flush, Extended, Mushroom, Square, Illuminated ring, Legend button
`size`	Size	Medium	Small, Medium, Large
`action`	Action	Momentary	Momentary, Maintained (push on, push off)
`text`	Legend text (Legend button style)	START	text
`nc`	Normally closed contact	False	on / off
`lamp`	Illuminated (adds Lamp output)	False	on / off

Signals (with the default settings)

Signal	Kind	Meaning
`Pressed`	Digital input (to the controller)	NO contact: 1 while pressed

Emergency Stop `estop`

Configuration

key	Setting	Default	Range / choices
`style`	Style	Round yellow plate	Round yellow plate, Square yellow plate, No plate
`size`	Size	Medium	Small, Medium, Large
`lamp`	Illuminated (adds Lamp output)	False	on / off

Signals (with the default settings)

Signal	Kind	Meaning
`OK`	Digital input (to the controller)	NC contact: 1 while healthy, 0 when pressed

Selector Switch (2-position) `sel2`

Configuration

key	Setting	Default	Range / choices
`handle`	Handle	Lever	Lever, Round knob, Key
`color`	Handle color	black	black, red, green, blue, yellow, white
`size`	Size	Medium	Small, Medium, Large

Signals (with the default settings)

Signal	Kind	Meaning
`Pos2`	Digital input (to the controller)	1 in the right-hand position

Selector Switch (3-position) `sel3`

Configuration

key	Setting	Default	Range / choices
`handle`	Handle	Lever	Lever, Round knob, Key
`color`	Handle color	black	black, red, green, blue, yellow, white
`size`	Size	Medium	Small, Medium, Large

Signals (with the default settings)

Signal	Kind	Meaning
`Left`	Digital input (to the controller)	1 in the left position
`Right`	Digital input (to the controller)	1 in the right position

Potentiometer (analog) `pot`

No settings.

Signals (with the default settings)

Signal	Kind	Meaning
`Value`	Analog input	0-100

Indicator Light / LED `light`

Configuration

key	Setting	Default	Range / choices
`color`	Color	green	green, red, amber, yellow, orange, blue, white
`style`	Style	Pilot light	Pilot light, Panel LED, Square indicator, Dome beacon, Light bar, Legend plate
`size`	Size	Medium	Small, Medium, Large
`effect`	When on	Steady	Steady, Flashing, Fast flash, Pulsing
`text`	Legend text (Legend plate style)	RUN	text

Signals (with the default settings)

Signal	Kind	Meaning
`On`	Digital output (from the controller)	Lamp

Stack Light `stack`

Configuration

key	Setting	Default	Range / choices
`tiers`	Segments	Red / Amber / Green	Red / Amber / Green, Red / Amber / Green / Blue, Red / Amber / Green / Blue / White, Red / Green, Red / Amber
`lens`	Lens	Ribbed	Ribbed, Smooth, Frosted, LED ring
`mount`	Mount	Pole	Pole, Tall pole, Direct
`size`	Size	Medium	Small, Medium, Large
`flash`	Built-in flasher	None	None, Red flashes, Red and amber flash, All flash, Red strobes
`buzzer`	Buzzer (adds output)	False	on / off

Signals (with the default settings)

Signal	Kind	Meaning
`Red`	Digital output (from the controller)	Red segment
`Amber`	Digital output (from the controller)	Amber segment
`Green`	Digital output (from the controller)	Green segment

Alarm Horn `horn`

No settings.

Signals (with the default settings)

Signal	Kind	Meaning
`On`	Digital output (from the controller)	Horn

Process HMI (operator terminal: a live faceplate of a tank, a case packer or a camera) `hmi`

A Process HMI: the operator terminal a tank would have beside it, showing a live faceplate of the vessel you pick in *Shows* (or the nearest one). Pick a case packer in *Shows* and it draws a packer faceplate instead: a top view of the lanes and the head, the head state and Z, what waits in each lane, cases by size (from the two count tags when you name them), the case lane drive, START and STOP, and COUNTS and ALARMS pages. The faceplate is drawn the way plant HMIs are: grey, dark text, colour only for what is running or wrong. It shows the vessel with its level, HI and LO marks, the mixer and heater, the level as a number, the temperature and the high limit, a one-minute trend of temperature and level, the A:B ratio, the mixing time, the flows, the pumps, heater and mixer (green running, orange heating), the state, and an alarm line. It reads the tank directly and has no I/O of its own; to show a tag of your own, use a Digital Display. In 3D it stands on a pedestal or hangs on a wall bracket.

Configuration

key	Setting	Default	Range / choices
`shows`	Shows		text
`spTag`	Heater setpoint tag (REAL; blank = the tank's own)		text
`startTag`	START button tag (BOOL, pulsed while touched)		text
`stopTag`	STOP button tag (BOOL, pulsed while touched)		text
`runTag`	Running indicator tag (BOOL: lights START)		text
`ratioTag`	Recipe A share tag (REAL, %)		text
`mixTag`	Recipe mixing time tag (REAL, s)		text
`lgTag`	Packer: large cases count tag (DINT)		text
`smTag`	Packer: small cases count tag (DINT)		text
`size`	Screen	15 inch	7 inch, 10 inch, 15 inch, 21 inch
`mount`	Mounting (3D)	Pedestal	Pedestal, Wall bracket

No signals.

Digital Display (any tag) `display`

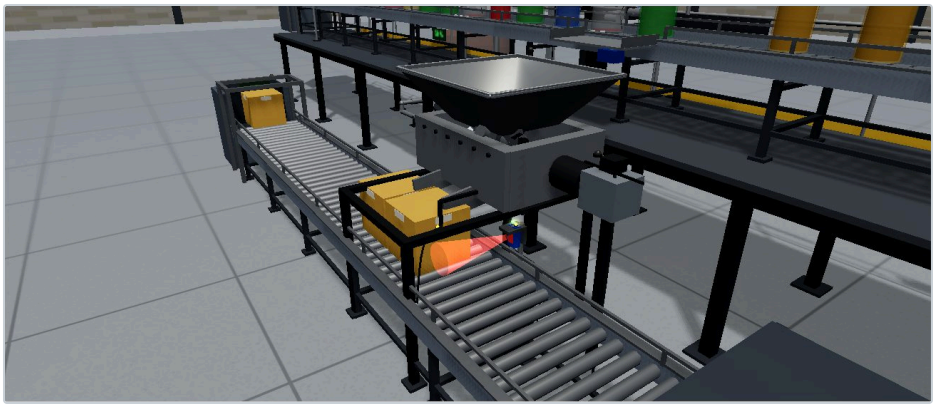


Figure 66. A Digital Display on the console shows any tag; here the IN BOX count from the grinder's box level sensor.

Configuration

key	Setting	Default	Range / choices
`tag`	Tag to show (blank = the wired analog output)		text
`format`	Show as	Auto	Auto, Seconds (from ms), mm:ss (from ms), On / Off, Hex
`decimals`	Decimal places (REAL and seconds)	1	0 – 4, step 1
`units`	Units shown after the value		text
`chars`	Width (characters)	6	4 – 16, step 1
`type`	Display type	LED 7-segment	LED 7-segment, LCD backlit, VFD, Plain text
`color`	Digit color	green	green, red, amber, yellow, orange, blue, cyan, white
`housing`	Housing	Black	Black, Grey, Stainless steel, White, Blue
`size`	Size	Medium	Small, Medium, Large
`caption`	Caption on the bezel (optional)		text
`hi`	Alarm color above this value (blank = never)		text
`lo`	Alarm color below this value (blank = never)		text
`alarm`	Alarm color	red	red, amber, yellow, white
`alarmFlash`	Flash the digits while in alarm	False	on / off
`bands`	Colour bands: value:colour at or above it, <value:colour below it (e.g. 1500:yellow, 3000:red, <100:blue)		text

Signals (with the default settings)

Signal	Kind	Meaning
`Value`	Analog output	Number to show

Communications

TCP Client Socket (via bridge) `socket`

Configuration

key	Setting	Default	Range / choices
`host`	Target IP / host	192.168.1.50	text
`port`	Target port	5000	1 – 65535, step 1
`term`	Terminator	CRLF	none, CR, LF, CRLF
`message`	Message on Send ({v} = SendValue)	T1	text
`m1`	Match 1 (sets Match1 when received text contains it)	PASS	text
`m2`	Match 2	FAIL	text
`m3`	Match 3		text
`m4`	Match 4		text
`latch`	Latch matches until the next message	False	on / off

Signals (with the default settings)

Signal	Kind	Meaning
`Connected`	Digital input (to the controller)	TCP connection is up
`Error`	Digital input (to the controller)	Bridge or connection fault
`Match1`	Digital input (to the controller)	Received text matched string 1
`Match2`	Digital input (to the controller)	Received text matched string 2
`Match3`	Digital input (to the controller)	Received text matched string 3
`Match4`	Digital input (to the controller)	Received text matched string 4
`RxValue`	Analog input	First number in the last received message
`RxCount`	Analog input	Messages received
`Send`	Digital output (from the controller)	Rising edge sends the message
`SendValue`	Analog output	Number substituted for {v}

Classic scenes

Elevator (shaft, cab, doors, hall calls, cab buttons, passengers) `elevator`

The classic elevator scene (v1.25), a panoramic lift. The cab rides between two and six floors on its hoist: **Up** and **Down** move it a floor per *Travel time*; both together do nothing, the contactors are interlocked. **At_N** is the floor switch, made while the cab is level; **Encoder** is the shaft encoder, 1000 counts a floor (floor 1 = 1000). A stop inside the door zone levels the cab. The door: **Open** and **Close** run it in *Door travel time*; **Door_Open** and **Door_Closed** are its end switches; the landing doors part with it. **Door_Eye** is blocked while a passenger stands in the doorway, and a Close then does not close. **Call_N** and **Cab_N** are the buttons, 1 while pressed (a passenger presses for 0.3 s; click the call plates and cab buttons yourself). **Lamp_N** lights the call plate and the cab button of a floor, **Ind_N** the displays. Passengers arrive on their own at the rate set, board at their floor with the door fully open, press a cab button, ride and leave; one who waits 45 s is lost. **Overload** is five riders. Moving with the door open is allowed and counted as *unsafe*: on a real elevator the door contact is in the safety chain and the hoist cannot run; here that is your program's job, so you can see what the chain is for.

Configuration

key	Setting	Default	Range / choices
`floors`	Floors	4	2 – 6, step 1
`travel`	Travel time, floor to floor (s)	2.5	0.5 – 10, step 0.1
`doorT`	Door travel time (s)	1.2	0.3 – 5, step 0.1
`passengers`	Passengers call by themselves	True	on / off
`callRate`	Seconds between passengers (about)	9	2 – 60, step 1
`safety`	Safety switches (adds Door_Eye and Overload)	True	on / off

Signals (with the default settings)

Signal	Kind	Meaning
`Call_1`	Digital input (to the controller)	Hall call button on floor 1 (1 while pressed)
`Call_2`	Digital input (to the controller)	Hall call button on floor 2 (1 while pressed)
`Call_3`	Digital input (to the controller)	Hall call button on floor 3 (1 while pressed)
`Call_4`	Digital input (to the controller)	Hall call button on floor 4 (1 while pressed)
`Cab_1`	Digital input (to the controller)	Cab button for floor 1 (1 while pressed)
`Cab_2`	Digital input (to the controller)	Cab button for floor 2 (1 while pressed)

Signal	Kind	Meaning
`Cab_3`	Digital input (to the controller)	Cab button for floor 3 (1 while pressed)
`Cab_4`	Digital input (to the controller)	Cab button for floor 4 (1 while pressed)
`At_1`	Digital input (to the controller)	Floor switch: the cab is level with floor 1
`At_2`	Digital input (to the controller)	Floor switch: the cab is level with floor 2
`At_3`	Digital input (to the controller)	Floor switch: the cab is level with floor 3
`At_4`	Digital input (to the controller)	Floor switch: the cab is level with floor 4
`Encoder`	Analog input	Shaft encoder on the hoist: the cab position in counts, 1000 per floor (floor 1 = 1000, floor 2 = 2000 …)
`Door_Open`	Digital input (to the controller)	Door fully open (end switch)
`Door_Closed`	Digital input (to the controller)	Door fully closed (end switch)
`Door_Eye`	Digital input (to the controller)	Door photo eye: blocked while someone is in the doorway
`Overload`	Digital input (to the controller)	Cab overload switch (too many passengers)
`Up`	Digital output (from the controller)	Hoist motor up
`Down`	Digital output (from the controller)	Hoist motor down (never with Up: the contactors are interlocked)
`Open`	Digital output (from the controller)	Door motor: open
`Close`	Digital output (from the controller)	Door motor: close
`Lamp_1`	Digital output (from the controller)	Built-in lamp of the floor 1 hall switch and cab button: a call is registered
`Lamp_2`	Digital output (from the controller)	Built-in lamp of the floor 2 hall switch and cab button: a call is registered
`Lamp_3`	Digital output (from the controller)	Built-in lamp of the floor 3 hall switch and cab button: a call is registered
`Lamp_4`	Digital output (from the controller)	Built-in lamp of the floor 4 hall switch and cab button: a call is registered

Signal	Kind	Meaning
`Ind_1`	Digital output (from the controller)	Floor indicator 1: the lamp above every door and in the cab that says the cab is at floor 1
`Ind_2`	Digital output (from the controller)	Floor indicator 2: the lamp above every door and in the cab that says the cab is at floor 2
`Ind_3`	Digital output (from the controller)	Floor indicator 3: the lamp above every door and in the cab that says the cab is at floor 3
`Ind_4`	Digital output (from the controller)	Floor indicator 4: the lamp above every door and in the cab that says the cab is at floor 4
`Buzzer`	Digital output (from the controller)	Alarm buzzer

Traffic Intersection (two roads, signal heads, walk signs, ped buttons, loops, cars, conflict monitor) `traffic`

The classic traffic intersection (v1.26), a real street. Two two-lane roads cross; cars arrive on their own at the rate set, queue at the stop line, go on **green**, decide once on **amber** (a car close to the line at speed goes through, the rest stop) and stop on **red**. **NS_Car** / **EW_Car** are the inductive loops in the approach lanes, 1 while a car stands or passes on one (click a loop to park a car there). **NS_Ped_PB** / **EW_Ped_PB** are the pedestrian buttons at the corners (a pedestrian presses once, 0.4 s, and again after 20 s; click them yourself); a pedestrian crosses when the **WALK** for their crossing is lit, six seconds kerb to kerb. NS means *along* N/S: the N/S walk crosses the E/W road and needs the E/W road red. The **conflict monitor** (MMU) in the cabinet watches the outputs: green or amber on both roads, or a WALK against the green that crosses it, for 0.3 s puts the intersection into red flash in hardware, whatever the program says, and **MMU_Fault** goes 1 until you click the cabinet. Two cars in the box from both roads, or a car through a crosswalk with a pedestrian on it, are logged and counted as conflicts; the program is the only thing that can cause them.

Configuration

key	Setting	Default	Range / choices
`carRate`	Cars per minute, each approach (about)	12	0 – 40, step 1
`peds`	Pedestrians arrive by themselves	True	on / off
`pedRate`	Seconds between pedestrians (about)	25	5 – 120, step 1
`mmu`	Conflict monitor in the cabinet (MMU)	True	on / off
`style`	Signal heads	Mast arm, 12 inch	Mast arm, 12 inch, Mast arm, 8 inch, Span wire
`long`	Long roads out to the next blocks (off: the classic short approaches)	True	on / off
`dayMin`	Day length: a whole day (24 h) lasts this many minutes of run time (12 h of day, then 12 h of night)	24	2 – 1440, step 1
`startHour`	The run starts at this hour (0 to 23)	7	0 – 23, step 1

Signals (with the default settings)

Signal	Kind	Meaning
`NS_Car`	Digital input (to the controller)	A car on either N/S approach loop (northbound or southbound)
`EW_Car`	Digital input (to the controller)	A car on either E/W approach loop
`NS_Ped_PB`	Digital input (to the controller)	Pedestrian button to cross the E/W road (walk along N/S), 1 while pressed

Signal	Kind	Meaning
`EW_Ped_PB`	Digital input (to the controller)	Pedestrian button to cross the N/S road (walk along E/W), 1 while pressed
`MMU_Fault`	Digital input (to the controller)	Conflict monitor tripped: the cabinet is in red flash until the MMU is reset (click the cabinet)
`NS_Red`	Digital output (from the controller)	N/S heads: red
`NS_Yellow`	Digital output (from the controller)	N/S heads: amber
`NS_Green`	Digital output (from the controller)	N/S heads: green
`EW_Red`	Digital output (from the controller)	E/W heads: red
`EW_Yellow`	Digital output (from the controller)	E/W heads: amber
`EW_Green`	Digital output (from the controller)	E/W heads: green
`NS_Walk`	Digital output (from the controller)	WALK for the crossings along N/S (over the E/W road)
`NS_DontWalk`	Digital output (from the controller)	DON'T WALK for those crossings
`EW_Walk`	Digital output (from the controller)	WALK for the crossings along E/W (over the N/S road)
`EW_DontWalk`	Digital output (from the controller)	DON'T WALK for those crossings

Parking Entry Gate (ticket kiosk, barrier boom, loops, photo-eye, lot, exit) `parkgate`

Configuration

key	Setting	Default	Range / choices
`carRate`	Cars per minute arriving (about)	4	0 – 20, step 0.5
`leaveEvery`	Seconds between cars leaving the lot (about)	75	5 – 600, step 5
`capacity`	Stalls in the garage (18 on LEVEL 1, 24 on each floor above)	90	4 – 90, step 1
`dayMin`	Day length: a whole day (24 h) lasts this many minutes of run time (12 h of day, then 12 h of night)	24	2 – 1440, step 1
`startHour`	The run starts at this hour (0 to 23)	7	0 – 23, step 1
`tailgate`	Some drivers tailgate (drive under a raised boom without a ticket)	False	on / off

Signals (with the default settings)

Signal	Kind	Meaning
`Arming_Loop`	Digital input (to the controller)	A car stands over the loop at the ticket kiosk
`Ticket_PB`	Digital input (to the controller)	The kiosk's PRESS FOR TICKET button, 1 while pressed
`Ticket_Taken`	Digital input (to the controller)	The driver has pulled the ticket from the slot (a short pulse)
`Arm_Up`	Digital input (to the controller)	Boom up limit switch: the boom is fully raised
`Arm_Down`	Digital input (to the controller)	Boom down limit switch: the boom is fully lowered
`Gate_Eye`	Digital input (to the controller)	Photo-eye across the lane just past the boom: 1 while the beam is blocked
`Closing_Loop`	Digital input (to the controller)	A car stands over the loop past the boom
`Exit_Loop`	Digital input (to the controller)	A car is leaving the lot over the exit lane's loop
`Arm_Raise`	Digital output (from the controller)	Boom motor: raise
`Arm_Lower`	Digital output (from the controller)	Boom motor: lower
`Ticket_Out`	Digital output (from the controller)	Dispense a ticket (on the rising edge)
`Ticket_Lamp`	Digital output (from the controller)	PRESS FOR TICKET lamp on the kiosk
`Boom_Lights`	Digital output (from the controller)	The red lamps along the boom
`Full_Sign`	Digital output (from the controller)	LOT FULL sign at the entrance

Roll-up Door (curtain, limits, photo-eye, safety edge, forklift, floor loops) `rolldoor`

Configuration

key	Setting	Default	Range / choices
`tripEvery`	Seconds between forklift trips (about)	30	5 – 300, step 5
`dwell`	Seconds the forklift spends outside loading	8	2 – 60, step 1
`travel`	Door travel time, seconds	7	3 – 20, step 1
`forklift`	Forklift runs by itself	True	on / off

Signals (with the default settings)

Signal	Kind	Meaning
`Door_Up`	Digital input (to the controller)	Up limit switch: the door is fully open
`Door_Down`	Digital input (to the controller)	Down limit switch: the door is fully closed
`Eye`	Digital input (to the controller)	Photo-eye across the opening, inside face: 1 while the beam is blocked
`Edge`	Digital input (to the controller)	Safety edge on the curtain's bottom bar: 1 while it is pressed against something
`Inside_Loop`	Digital input (to the controller)	A forklift waits on the floor loop inside the door
`Outside_Loop`	Digital input (to the controller)	A forklift waits on the loop outside the door
`Door_Pos`	Analog input	Door position from the encoder on the operator: 0 shut to 100 fully open (%)
`Motor_Up`	Digital output (from the controller)	Door operator: open
`Motor_Down`	Digital output (from the controller)	Door operator: close
`Beacon`	Digital output (from the controller)	Amber warning beacon over the door

Car Wash (pay station, tunnel conveyor, soap / brush / rinse / wax / dry stations, supply tanks) `carwash`

Configuration

key	Setting	Default	Range / choices
`carRate`	Cars per minute arriving (about)	0.8	0 – 10, step 0.1
`deluxe`	Drivers who buy DELUXE (wax), %	40	0 – 100, step 5
`vacuum`	Drivers who stop at the free vacuums after the wash, %	50	0 – 100, step 5
`dayMin`	Day length: a whole day (24 h) lasts this many minutes of run time (12 h of day, then 12 h of night)	24	2 – 1440, step 1
`startHour`	The run starts at this hour (0 to 23)	7	0 – 23, step 1

Signals (with the default settings)

Signal	Kind	Meaning
`Pay_OK`	Digital input (to the controller)	The pay terminal took the money: a short pulse
`Deluxe_Sel`	Digital input (to the controller)	DELUXE (with wax) was bought: on together with Pay_OK
`TS1_Entry`	Digital input (to the controller)	Wheel treadle 1 (ENTRY): the front wheel stands on it
`TS2_Soap`	Digital input (to the controller)	Wheel treadle 2 (SOAP): the front wheel stands on it
`TS3_Brush`	Digital input (to the controller)	Wheel treadle 3 (BRUSH): the front wheel stands on it
`TS4_Rinse`	Digital input (to the controller)	Wheel treadle 4 (RINSE): the front wheel stands on it
`TS5_Wax`	Digital input (to the controller)	Wheel treadle 5 (WAX): the front wheel stands on it
`TS6_Dry`	Digital input (to the controller)	Wheel treadle 6 (DRY): the front wheel stands on it
`TS7_Exit`	Digital input (to the controller)	Wheel treadle 7 (EXIT): the front wheel stands on it
`PE1_Entry`	Digital input (to the controller)	Photo-eye 1 (ENTRY) across the bay at bumper height: 1 while a car blocks it
`PE2_Soap`	Digital input (to the controller)	Photo-eye 2 (SOAP) across the bay at bumper height: 1 while a car blocks it
`PE3_Brush`	Digital input (to the controller)	Photo-eye 3 (BRUSH) across the bay at bumper height: 1 while a car blocks it
`PE4_Rinse`	Digital input (to the controller)	Photo-eye 4 (RINSE) across the bay at bumper height: 1 while a car blocks it
`PE5_Wax`	Digital input (to the controller)	Photo-eye 5 (WAX) across the bay at bumper height: 1 while a car blocks it
`PE6_Dry`	Digital input (to the controller)	Photo-eye 6 (DRY) across the bay at bumper height: 1 while a car blocks it

Signal	Kind	Meaning
`PE7_Exit`	Digital input (to the controller)	Photo-eye 7 (EXIT) across the bay at bumper height: 1 while a car blocks it
`Brush_OL`	Digital input (to the controller)	Brush motor overload relay: 1 when it has tripped
`Water_Low`	Digital input (to the controller)	Water tank low level switch: 1 below 20 %
`Water_High`	Digital input (to the controller)	Water tank high level switch: 1 above 90 %
`Soap_Low`	Digital input (to the controller)	Soap tank low level switch: 1 below 20 %
`Soap_High`	Digital input (to the controller)	Soap tank high level switch: 1 above 90 %
`Gate_Open`	Digital output (from the controller)	Raise the entry boom (it comes down when this drops)
`Pay_Lamp`	Digital output (from the controller)	PAY HERE on the pay terminal: the wash will take the next car
`Enter_Sign`	Digital output (from the controller)	The sign over the entry: DRIVE ON (green) or STOP (red)
`Conveyor`	Digital output (from the controller)	Tunnel conveyor motor: the roller pushes the front wheel
`Soap_Valve`	Digital output (from the controller)	Soap arch valve
`Brushes`	Digital output (from the controller)	Wrap and top brush motors
`Rinse_Valve`	Digital output (from the controller)	Rinse arch valve
`Wax_Valve`	Digital output (from the controller)	Wax arch valve
`Dryer`	Digital output (from the controller)	Dryer blowers
`Exit_Sign`	Digital output (from the controller)	The sign over the exit: DRIVE OUT (green) or WAIT (red)
`Water_Pump`	Digital output (from the controller)	Water pump to the brushes and the rinse arch
`Soap_Pump`	Digital output (from the controller)	Soap pump to the soap arch
`Water_Fill`	Digital output (from the controller)	Water tank fill valve (city water)
`Soap_Fill`	Digital output (from the controller)	Soap tank fill valve (from the bulk drum)
`Water_Level`	Analog input	Water tank level, %

Signal	Kind	Meaning
`Soap_Level`	Analog input	Soap tank level, %

Panels

Panel / Enclosure `panel`

Configuration

key	Setting	Default	Range / choices
`style`	Style	Enclosure door	Enclosure door, Flat plate, Push button station, Operator console
`finish`	Finish	ANSI 61 grey	ANSI 61 grey, Light grey (RAL 7035), Stainless steel, White, Black, Beige, Safety yellow, Blue
`w`	Width (px)	320	60 – 1400, step 10
`h`	Height (px)	220	60 – 900, step 10
`title`	Title plate (blank for none)	CONTROL PANEL	text
`plate`	Title plate	White on black	White on black, Black on white, Black on yellow, White on red, White on blue, White on green, Black on aluminium
`carry`	Devices on the panel move with it	True	on / off
`names`	Show device name tags on this panel	True	on / off

No signals.

Nameplate (engraved legend) `label`

Configuration

key	Setting	Default	Range / choices
`text`	Text	START	text
`text2`	Second line (optional)		text
`plate`	Colors	White on black	White on black, Black on white, Black on yellow, White on red, White on blue, White on green, Black on aluminium
`size`	Size	Medium	Small, Medium, Large
`screws`	Fixing screws	False	on / off

No signals.

14 Operating the Process HMI

A tank on a real plant has a screen beside it. The operator reads the vessel at a glance, sets a temperature, and sees what went wrong last night. Logix Connect has two such screens. The **Mix Tank** carries its own local panel on the shell, and the **Process HMI** is a separate operator terminal you place anywhere and point at any vessel. Both show the same faceplate, both are touch screens (a mouse click is a touch), and both work in the top view and in 3D. This chapter is the operating manual: what every part of each page means, how to change a value, and a worked example where the operator's setpoint drives the heater through a REAL tag.

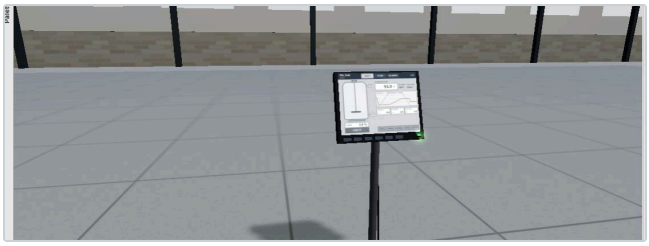


Figure 67. The Process HMI on its pedestal in 3D, a 15-inch terminal.

Figure 68. The mix tank's own panel, on standoffs from the shell.

Placing an HMI and connecting it

1. Program mode. In the palette, *Operator* → **Process HMI**; click the floor to place it.
2. In its properties, **Shows**: pick the vessel (any Mix Tank, Tank, Reservoir or Supply Tote). Left blank, it shows the nearest one.
3. **Heater setpoint tag**: the controller tag the TEMP page writes, normally a REAL. Left blank, the HMI uses the tank's own *Panel and HMI: heater setpoint tag*, so the panel and the HMI set the same tag.
4. **START button tag**, **STOP button tag** and **Running indicator tag**: optional; with them set, START and STOP appear on the MAIN page (below).
5. **Screen**: 7, 10, 15 (the default) or 21 inch. **Mounting**: on a pedestal or a wall bracket (3D).

The HMI has no I/O of its own: it reads the vessel directly and writes only the setpoint tag and the tank's high limit. Its properties end in a **Connections** section, like every part of the liquid family: what it shows and which tag it writes. On the tank, the same section lists every HMI showing it, and on each tote, tank, reservoir, mix tank, filler and dispenser it lists what feeds the part, what it feeds, its product, and whether each link is moving liquid right now. Every name is a link: click it to open that part.

Figure 69. Supply_A: it feeds the mix tank's inlet A, and it holds water.

Figure 70. Mix_Tank: inlets from the two totes, the outlet to the floor drain, its product, shown on Tank_HMI.

Touching the screen

The screen answers in **Run mode**. In the top view, click it; in 3D, click it where it hangs, on the HMI or on the tank's panel. In Program mode a click selects the part instead, so you can move it and edit its settings. The page you are on is remembered for each screen. Across the top are the page tabs, **MAIN**, **TEMP** (on a vessel with a temperature), **RECIPE** (on a mix tank) and **ALARMS**. A red dot on ALARMS means there is an active alarm.

The MAIN page

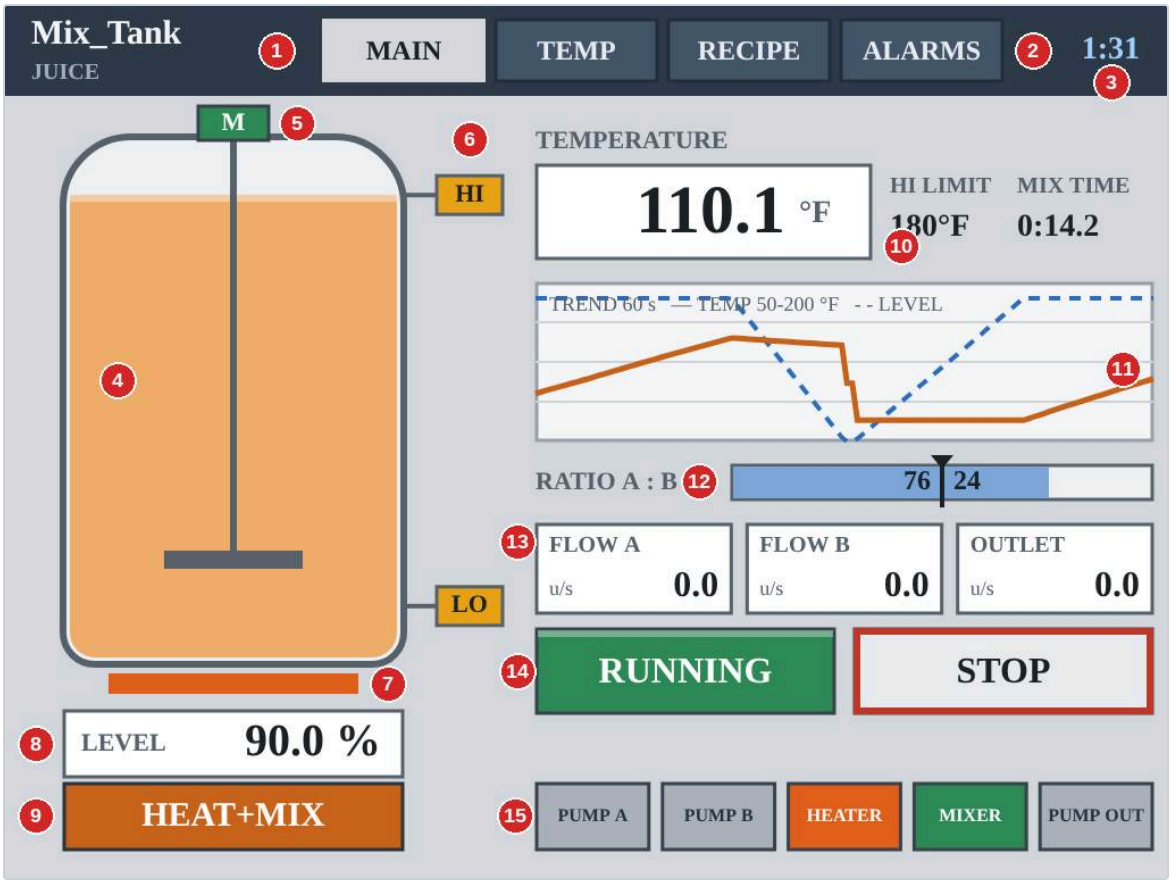


Figure 71. The MAIN page of the Level 7 mix tank during HEAT+MIX, with the batch running from START.

	What it is	What it tells you
1	Name and product	Which vessel this is and what it holds.
2	Page tabs	Touch to flip pages. The one you are on is light. A red dot on ALARMS: an alarm is active.
3	Run clock	Machine time since the run started (m:ss).
4	The vessel	The liquid at its level, in its colour. In the mix tank, clear and pale while the ingredients go in, cloudy as it mixes.
5	Mixer	Green and turning while the mixer runs.
6	HI and LO marks	At the probe heights; amber while the liquid covers them.
7	Heater	Orange while the heater is on.
8	Level	The level in percent.
9	State	FILLING (blue), HEAT+MIX (orange), MIXING, DRAINING (green), HOLDING, EMPTY (grey), PURGING (purple).
10	Temperature, high limit, mix time	The temperature (red once the high limit is made), the thermostat setting, and the mixing time of this batch. Touch the temperature to go to the TEMP page.
11	Trend	The last 60 s: temperature solid orange (50-200 °F), level dashed blue (0-100 %).

	What it is	What it tells you
12	Ratio	The share of A and B in the batch now in the tank. The black marker is the A share set on the RECIPE page.
13	Flows	What each flowmeter reads, units a second, blue while moving.
14	START and STOP	Touch pushbuttons on the START and STOP tags (when they are set). START reads RUNNING, green, while the Running tag is on.
15	Equipment	Each pump, the heater and the mixer: grey stopped, green running, orange heating. A pump on a locked inlet reads A LOCKED or B LOCKED, in red.

The faceplate is drawn the way plant HMIs are (the ISA-101 style): a grey screen and dark text, with colour kept for what is running and what is wrong. A lit heater or a red alarm line stands out because nothing else is bright. When there is an alarm, a red line appears above the equipment buttons; touch it to go to ALARMS.

The TEMP page

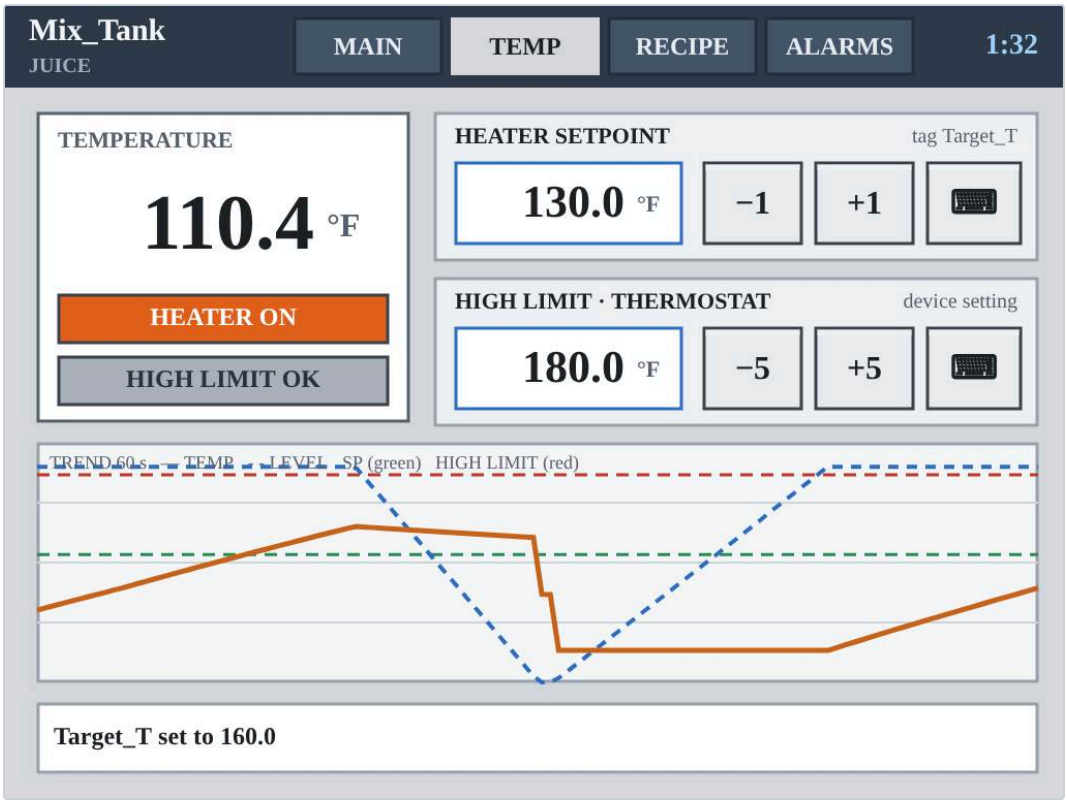


Figure 72. The TEMP page: the temperature, the heater and high-limit status, the two setpoint cards, the trend with the setpoint (green) and the high limit (red), and the message line.

The left card shows the temperature and two status bars: **HEATER ON / OFF** (what the heater output is doing) and **HIGH LIMIT OK / MADE** (the thermostat switch). The two cards on the right are the controls.

- **HEATER SETPOINT** is the controller tag named under the card. Writing it is all the HMI does; what the heater does with it is up to your program, as in the worked example below. **−1** and **+1** step it by one degree. Touching the value, or the keypad button, opens the keypad. The value is kept between 32 and 250 °F.
- **HIGH LIMIT · THERMOSTAT** is the tank's own thermostat setting, the temperature at which its Thermostat input closes. **−5** and **+5** step it, or type it, between 80 and 250 °F. This is the hardware safety, not the control: the program should only ever use it to switch the heater off.

The trend draws both as dashed lines over the temperature, so you can watch the heater hold the setpoint and see how much margin is left below the limit. The message line at the bottom confirms each change (*Heat_SP set to 147.5*), and every change is also written in the Simulator Log with the HMI's name.

Typing a value with the keypad

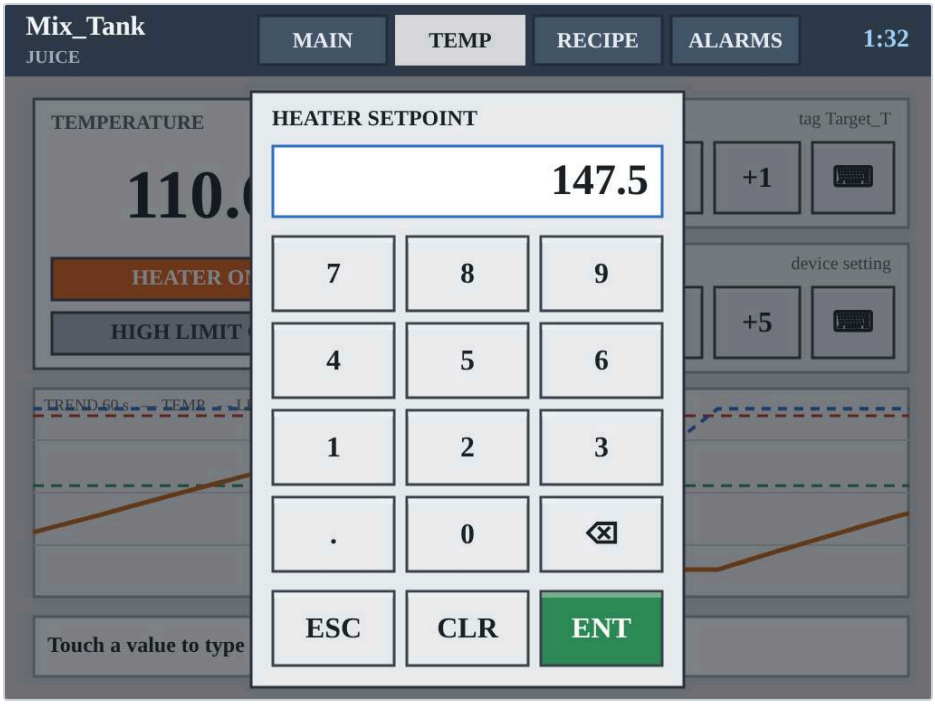


Figure 73. The keypad over the TEMP page, with 147.5 typed.

1. Touch the value box (or the keypad button) of the card you want to change.
2. Touch the digits. **.** adds the decimal point (once), ⌫ deletes the last digit, **CLR** clears them all.
3. **ENT** writes the value and closes the keypad. **ESC** closes it without changing anything.

If the program writes the same tag on every scan, your value lasts one scan and the message line says so in red: *The program wrote Target_T back to 150.0: a rung writes it every scan*. The HMI is not broken: the tag has two writers, and the program always writes last. Give the operator a tag of their own, as the worked example does.

The ALARMS page

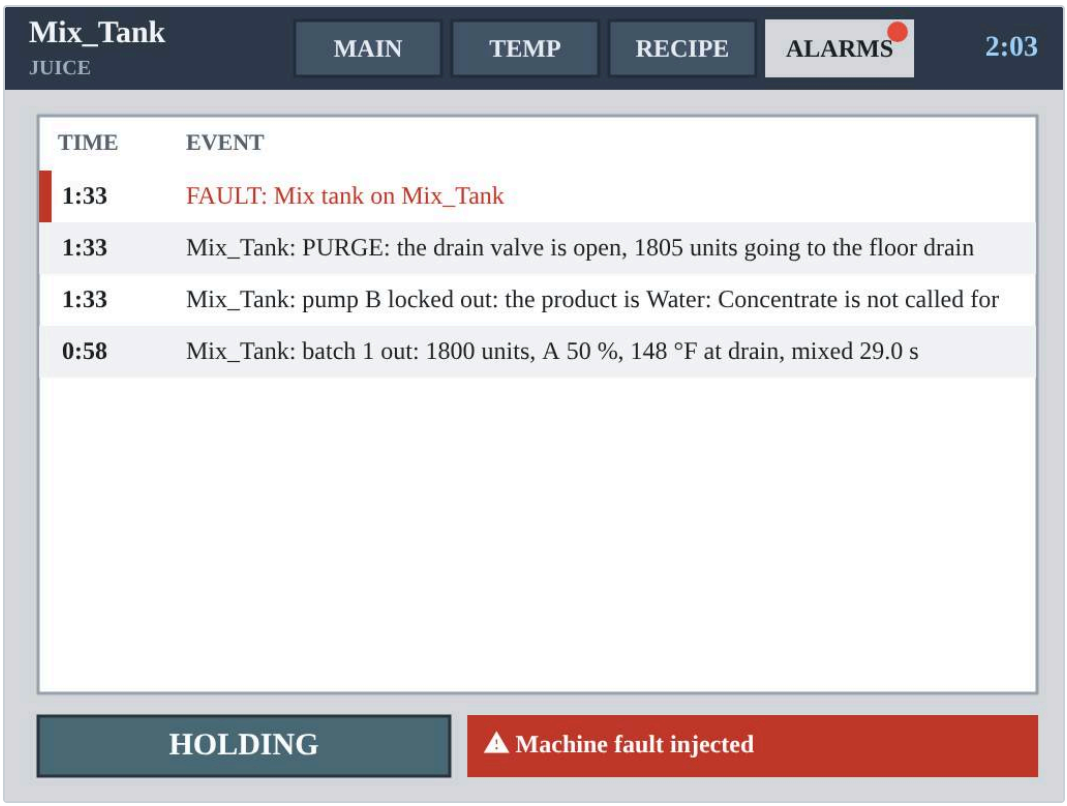


Figure 74. The ALARMS page after an injected pump fault: the newest first, a red bar on anything abnormal, the state and the active alarm at the bottom.

Every Simulator Log line that names this vessel, newest first, with the machine time: batches out, operator changes, pumps running dry, overflows, scorches and injected faults. Anything abnormal has a red bar and red text. The bottom line shows the state and the alarm that is active now.

START and STOP on the screen

Give the HMI a **START button tag** and a **STOP button tag** (BOOLs), and its MAIN page shows two pushbuttons above the alarm line. A touch pulses the tag: on for a quarter of a second of machine time, then off, so the program sees it for several scans, exactly like a momentary pushbutton wired to an input. Give it a **Running indicator tag** too and START lights green and reads RUNNING while that tag is on. Every press goes in the Simulator Log.

The program has to listen for them, and the plant way is in parallel with the hardwired buttons, never instead of them. Level 7 is set up like this: `HMI_Start` and `HMI_Stop` are ordinary BOOL tags (not aliases of inputs, since a real HMI cannot write an input), and the run rung takes either START and either STOP:

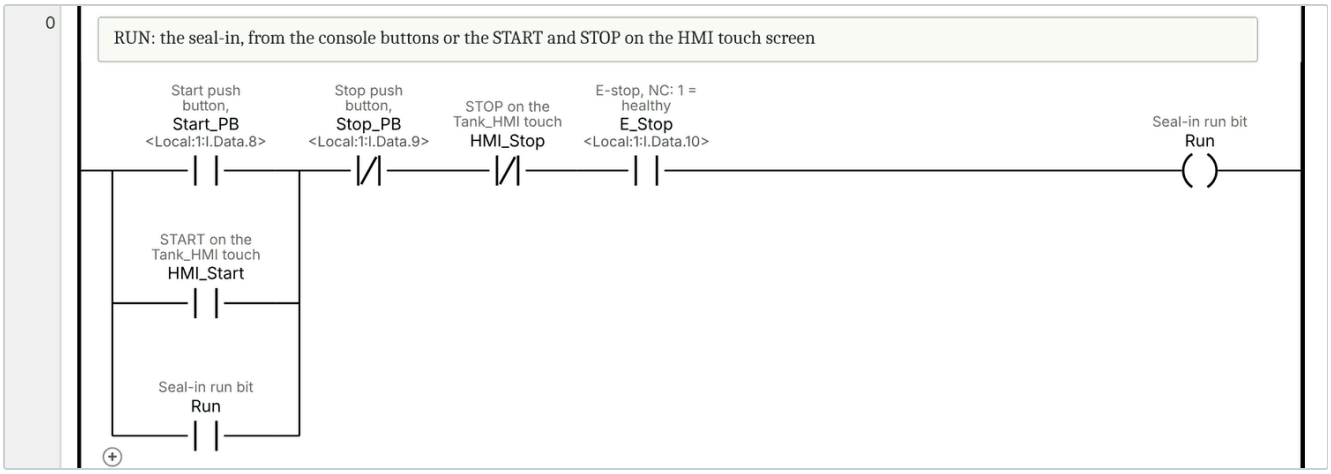


Figure 75. Level 7, rung 0: the seal-in takes the console Start_PB or the HMI's HMI_Start, and drops out on the console Stop_PB or the HMI's HMI_Stop. E_Stop stays hardwired.

The Operator View

The **Operator View** is the same screen as a tab of its own, full size, for running the machine without the scene: **Window → Open Operator View**, the *Operator View* entry under the simulator in the organizer, or **HMI** in the phone's bottom bar. Beside the screen (below it on a phone) is a **control panel** of real-looking pushbuttons:

- **RUNNING** lamp: green while the Running tag is on.
- **ALARM** lamp: red while the tank has an active alarm.
- **START** and **STOP**: pushbuttons that pulse the START and STOP tags, the same as the buttons on the screen. Under each is the tag it writes, in red if there is no such tag.
- A message line: what the last press or change did.

The bar along the top sets what the view shows and its four tags: *Shows* (the mix tank, or the first tank, when blank), *START tag*, *STOP tag*, *Running tag* and *Setpoint tag*. They are kept with the scene and saved with the project; on Level 7 they are already set to Mix_Tank, HMI_Start, HMI_Stop, Run and Target_T. The screen works exactly like the HMI's: touch the tabs, the values and the keypad. The buttons work in Run mode; in Program mode they say so and do nothing.

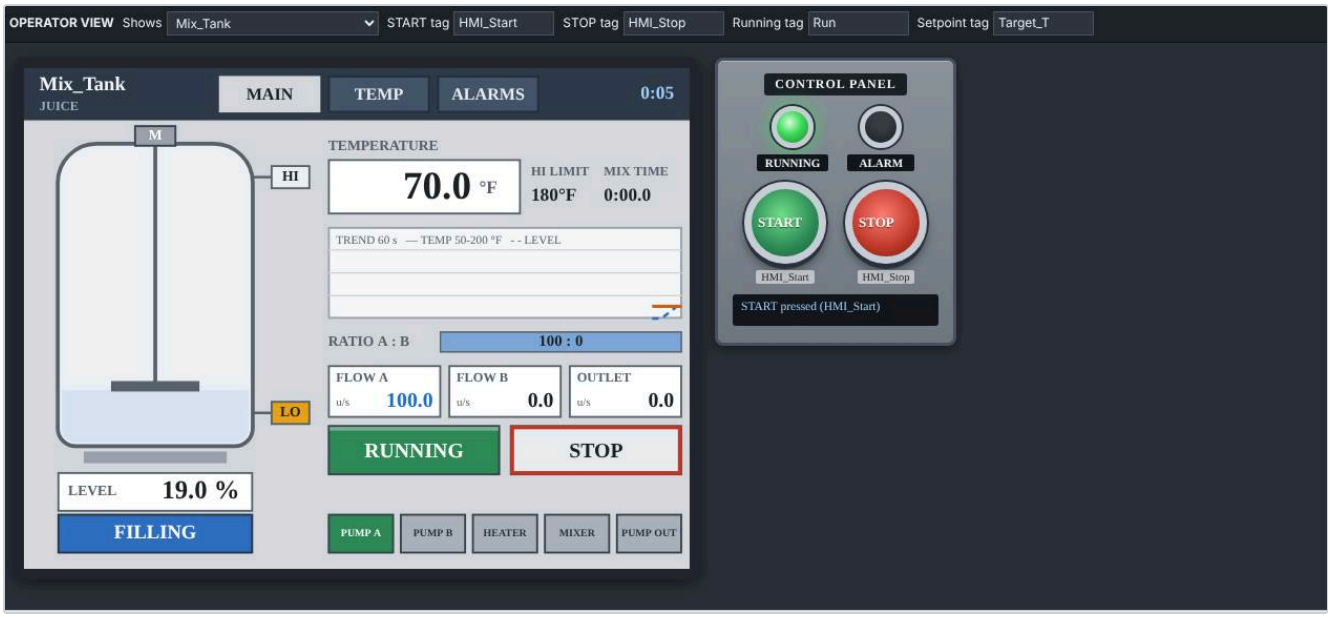


Figure 76. The Operator View on the PC during Level 7, just after START on the control panel: the RUNNING lamp is lit, START on the screen reads RUNNING, pump A is filling the tank, and the message line confirms the press.

Figure 77. The same view on a phone: the screen on top, the control panel below, HMI selected in the bottom bar.

The packer and the camera too

From v1.00.1 the Operator View shows any screen a Process HMI can, not only a tank. *Shows* lists them in groups, **Tanks**, **Case packers** and **Cameras**, and a row of buttons under the bar switches screens in one tap: on the bottle line Mix_Tank, Case_Packer, Supply_Tank and the two totes. Each screen keeps its own tags. The first time you switch to one, the view takes them from the Process HMI in the scene that shows it (Case_Packer from Pack_HMI: Pack_Start, Pack_Stop, Pack_Run and the two case counts), and after that it remembers what you set for each.

The fields follow the screen. The packer has START, STOP, Running and its large and small case count tags, and the control panel's START and STOP pulse Pack_Start and Pack_Stop. A camera has no START or STOP, because it runs when the PLC enables triggering. On a camera the panel keeps only its two lamps, ON LINE and ALARM, and touching the screen turns the camera's pages (MAIN, RESULTS, ALARMS).

Figure 78. The Operator View on a phone, on Case_Packer: the packer faceplate, its tags, and START and STOP on the control panel working the packer.

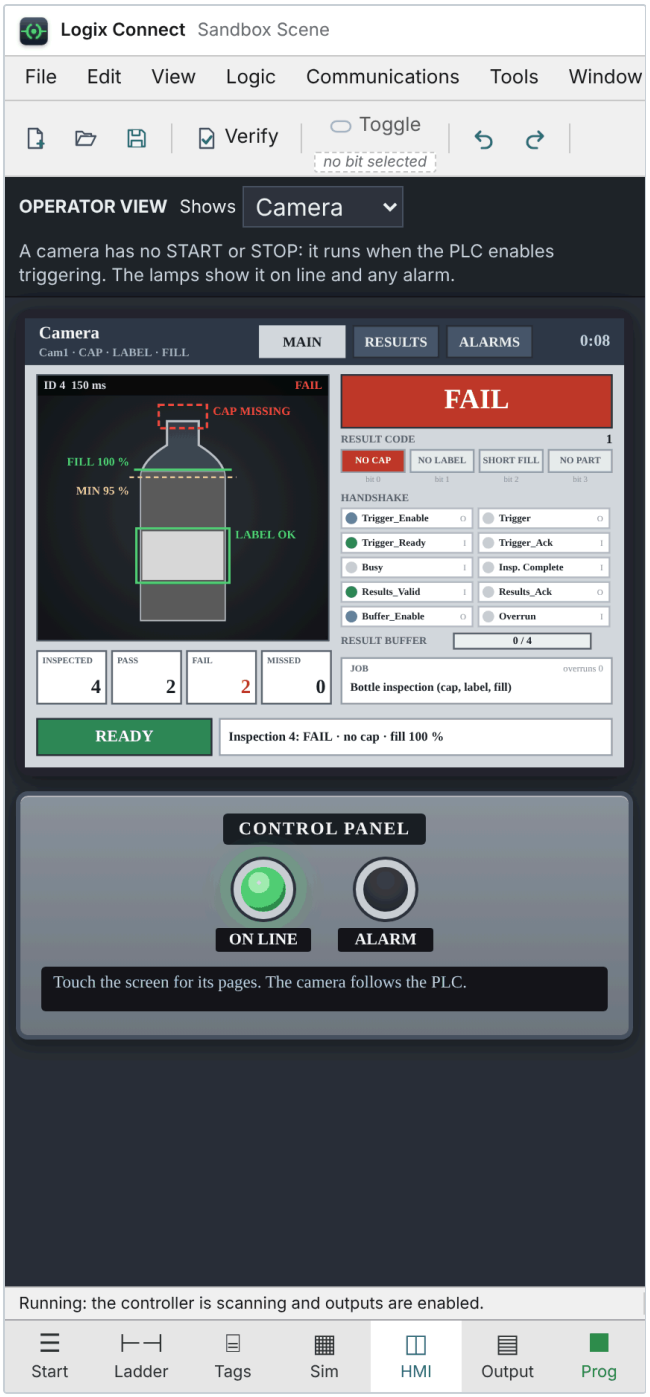


Figure 79. On Level 8 the view opens on the camera: the last image with its tools, the result, the handshake, and ON LINE lit.

TRY IT: THREE WAYS TO START

On Level 7 with the reference program, start the batch from the console Start_PB in the top view, stop it with STOP on Tank_HMI, start it again from the Operator View, and stop it from the console. One seal-in rung, three operator stations. Now delete the HMI_Start branch from rung 0 and try START on the Operator View again: the press is logged, the tag pulses (watch it in Quick Watch), and nothing happens, because no rung reads it. An HMI only asks; the program decides.

The bottle line's own screen

From Version 140 the Operator View has a screen for the whole bottle line, not only for its tank and its packer. On the bottle line it is the first button in the row, **LINE · BOTTLE LINE**, and it belongs to the BOTTLE LINE console. Across the top it tells you the state of the machine in letters you can read from across the room: **PRODUCING** in green while the line runs, **STOPPED** in grey, **ALARM** in red, blinking until somebody presses RESET, with the alarms read out of `Alarm_Word` in words underneath and the bottles a minute beside them. Under that is a live mimic of the line drawn from the scene itself: every conveyor, station and sensor where it stands on the floor, the conveyors green and moving while they run, a station glowing while it works, a sensor lit yellow while it sees a bottle, and the bottles themselves riding through. Then the console's counts and its preset as tiles, and the alarm list. Beside the screen the control panel carries the console's own START, STOP and RESET, labelled with the buttons they are, and its RUN and FAULT lamps: pressing them presses the real buttons, so the program cannot tell the difference.

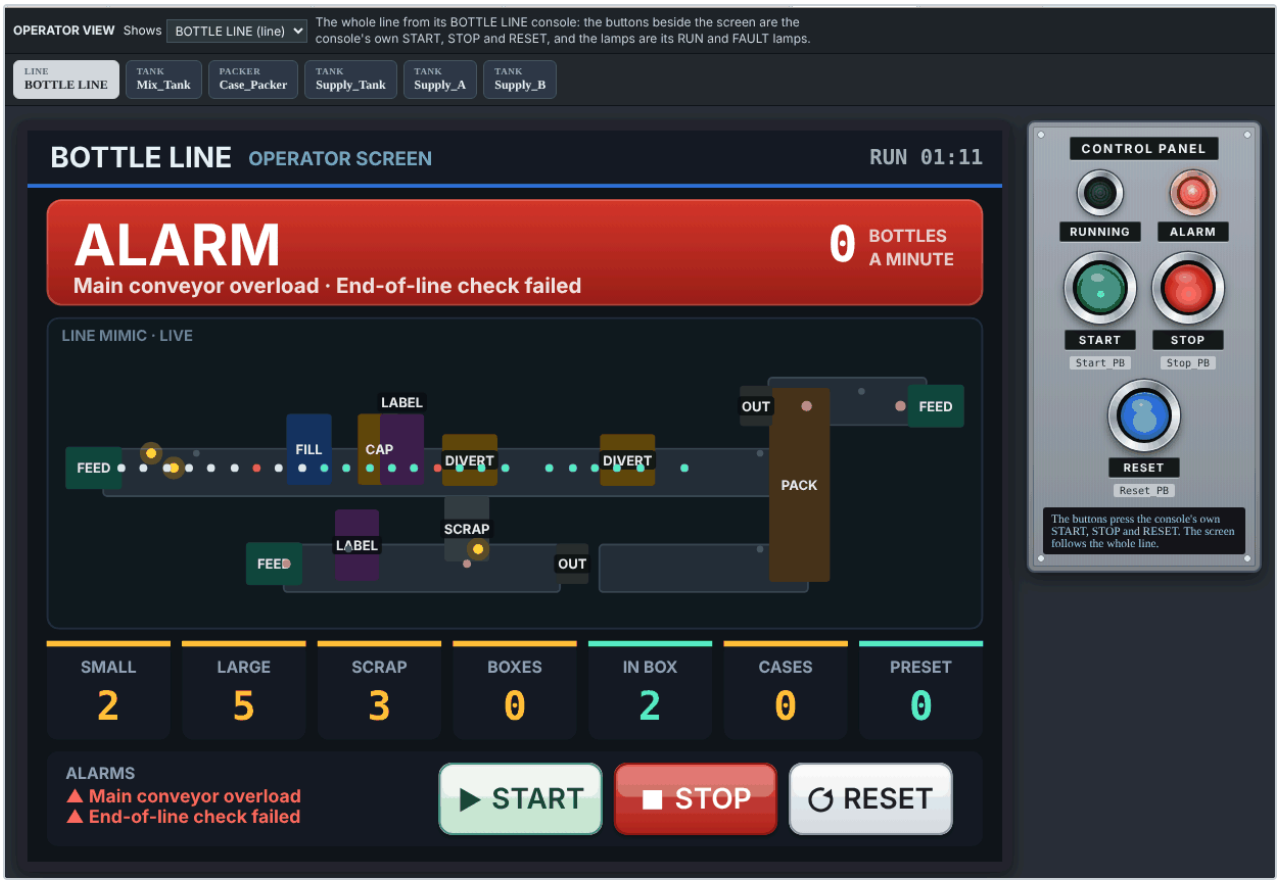


Figure 80. The bottle line operator screen with two alarms active and not yet acknowledged: the banner red and blinking, the alarms read out, the rate, the live mimic of the line, the counts and preset, and the console's START, STOP and RESET beside it.

> **TRY IT: RUN THE LINE FROM ITS SCREEN**
>
> Open the bottle line, go to Run, open the Operator View and pick **LINE · BOTTLE LINE**. Press START beside the screen and watch the banner turn green and the mimic come alive. Then take a sensor out with **Tools → Inject Machine Fault** and watch the screen tell you what is wrong before you look at the ladder.

Recipe mode: what goes in, what stays out, and purge

A real batch tank is never trusted to the program alone to keep the wrong liquid out. When the tank is making water, the concentrate pump must not run, whatever a rung asks for. Oil must never get into milk or water, and water must never get into oil. The mix tank has these interlocks built in, the way a plant puts them in the pump starter circuit. A pump on a **locked** inlet does not run, even with its output on. The first time the program asks for it, the Simulator Log says which pump and why, and the MAIN page shows **A LOCKED** or **B LOCKED** in red.

The **RECIPE** page is where the operator sets the batch up before START: which product, and with it the recipe.

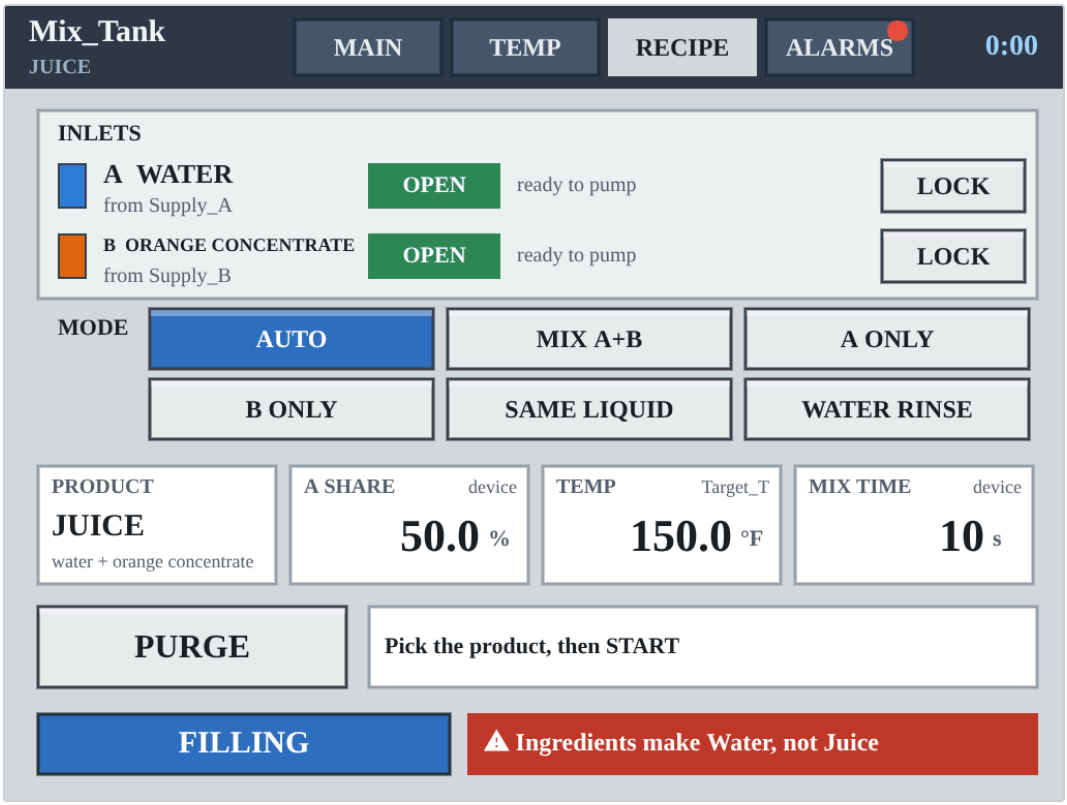


Figure 81. The RECIPE page on Tank_HMI during Level 7: water on inlet A and orange concentrate on inlet B, both open, mode AUTO, the product JUICE and the recipe values for the batch.

Part of the page	What it does
INLETS	One row per inlet: the colour and name of the liquid it carries, where it is piped from, **OPEN** (green) or **LOCKED** (red) with the reason, and a LOCK / UNLOCK button.
MODE	Which inlets this batch uses (the table below). The one in force is blue.
PRODUCT	What the tank makes, with what it is made of. Touch it for the product list: picking one loads its recipe (below) and sets the mode to AUTO. The product is set here, on the HMI, and not in the tank's properties.
A SHARE, TEMP, MIX TIME	Touch a card for the keypad. The temperature is the heater setpoint tag, as on TEMP. The A share and the mixing time are kept on the tank, and written to the Ratio tag and the Mix time tag when those are set on the Process HMI or the Operator View. A card bound to a tag shows the tag, so a value the program writes is the value you see.
PURGE	Drain the tank to the floor drain now (below).
Message line, state, alarm	What the last touch did, what the tank is doing, and the active alarm.

The six modes:

Mode	Inlet A	Inlet B	Use it for
AUTO	runs if the product's recipe calls for what it carries	the same	The default. Pick WATER and the concentrate inlet locks; pick JUICE (what Level 7 makes) and both run.
MIX A+B	runs	runs	Two liquids mixed to the A share.
A ONLY	runs	locked	One liquid, from inlet A.
B ONLY	locked	runs	One liquid, from inlet B.
SAME LIQUID	runs	runs from A's source	Both pumps on the same liquid: inlet B draws through a crossover from whatever feeds inlet A, so the tank fills twice as fast. Inlet B's own tote is not touched.
WATER RINSE	runs if it carries water	runs if it carries water	Running the tank with water only, to rinse it between products.

Ingredients and products. A supply tote holds an *ingredient*: water, cola syrup, lemon-lime syrup, orange concentrate, apple concentrate, milk or oil. A *product*, what a filler puts in a bottle, is made from its ingredients in the tank, to a recipe. Soda is not something a tote holds: it is water and cola syrup at the *throw ratio*, five parts water to one of syrup, carbonated after the blend (below). Juice from concentrate is water and orange concentrate, three to one. Milk, oil and water go through as they come.

Product	Inlet A	Inlet B	A share	Temperature	Mix time	CO2
Soda	water	cola syrup	83 % (5 : 1)	100 °F	8 s	3.5 volumes
Lemon-lime	water	lemon-lime syrup	83 % (5 : 1)	100 °F	8 s	3.8 volumes
Juice	water	orange concentrate	75 % (3 : 1)	160 °F	10 s	—
Apple juice	water	apple concentrate	85 % (5.7 : 1)	160 °F	10 s	—
Water	water	—	100 %	70 °F	0 s	—
Milk	milk	—	100 %	70 °F	0 s	—
Oil	oil	—	100 %	70 °F	0 s	—

Picking a product on the PRODUCT card loads that line: the A share and mixing time into the tank and the HMI's Ratio and Mix time tags, the temperature into the setpoint tag, and the mode to AUTO. It does not change the totes. If Supply_B holds orange concentrate and you pick SODA, inlet B locks in AUTO with *the product is Soda: Orange concentrate is not called for*, and stays locked until someone puts a cola syrup tote there (the tote's *Ingredient* in its properties). That is a changeover on a real line: the recipe on the screen and the ingredient at the tank both have to change.

What is in the tank. The tank holds what actually went into it, so if both totes hold water, mixing makes water whatever product was picked: the tank and the faceplate name it (in red), a filler fed from it gets water, and the alarm line reads *Ingredients make Water, not Soda*.

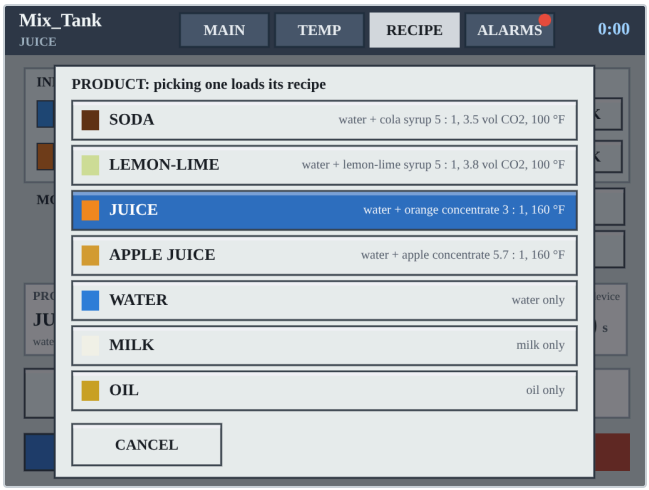


Figure 82. The product list: every product with its ingredients, ratio, temperature and CO2. The one in force is blue.

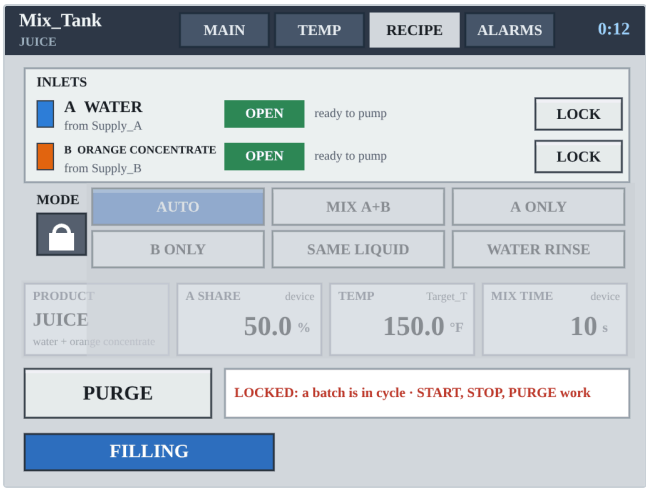


Figure 83. A batch in cycle: the product, the mode and the recipe values are greyed and locked. START, STOP, PURGE and LOCK still work.

The recipe lock. Once a batch is in the tank and the process is running (its Running tag on), the recipe is locked: the product, the mode, the A share, the temperature card and the mixing time. A touch on any of them is refused with *Recipe locked: a batch is in cycle · STOP, PURGE or finish it*, and the padlock shows beside MODE. It opens again when the process is stopped, when the tank is purged, or when the batch is finished and the tank empties. The overrides stay live: START and STOP on MAIN, PURGE, and the inlet LOCK / UNLOCK, because lockout/tagout is a safety function and is never locked out itself. The lock is on the operator, not the program: rungs can still write the recipe tags, and the setpoint on the TEMP page can still be trimmed during a batch.

On top of the mode come three locks that no mode lifts:

- **Contamination.** Oil and the water-based liquids (water, the syrups and concentrates, and everything made from them, and milk) never meet. An inlet carrying oil is locked when the product is water-based, and an inlet carrying a water-based liquid is locked when the product is oil. The tank also remembers what is in it: until it has been emptied or purged, an inlet that would put the other kind on top is locked with *purge it first*.
- **Lockout / tagout.** LOCK on the page (or *Inlet A locked out* and *Inlet B locked out* in the tank's properties) locks that inlet until someone unlocks it. A Supply Tote has *Locked out* too: its valve gives nothing, and every pump drawing from it is locked.
- **Purging.** While the tank purges, both inlets are locked.

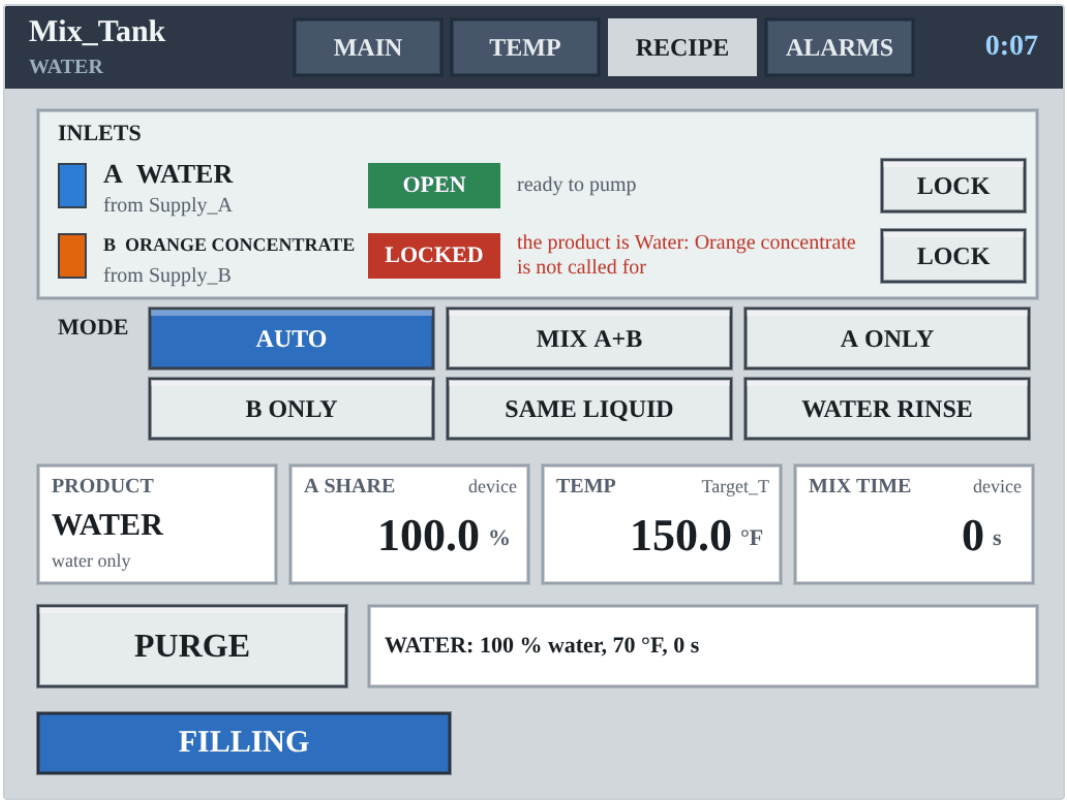


Figure 84. WATER picked, mode AUTO: the orange concentrate inlet is locked and says why. Pump B will not run, even though the program has PumpB on.

PURGE opens the drain valve on the bottom of the tank and empties it to the floor drain on demand. It asks first. It is refused while the process is running (the Running tag is on: stop it first) and when the tank is already empty. While it purges, the state reads **PURGING**, the flows show PURGE, and both inlets are locked. When the tank is empty the valve closes and the batch is discarded: it is not counted or graded.

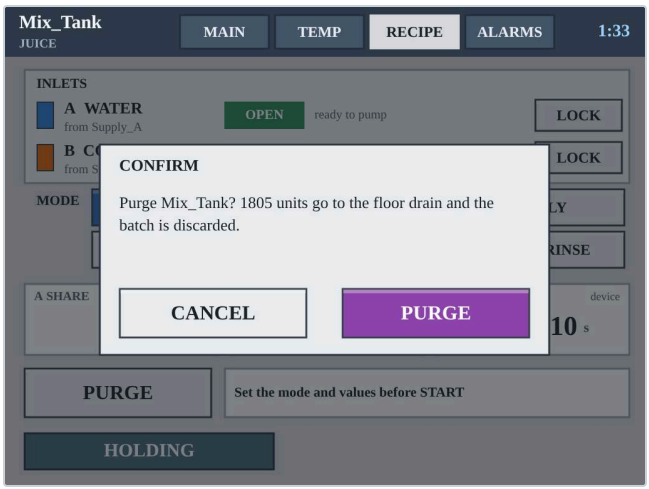


Figure 85. PURGE asks first, with how much will go to the drain.

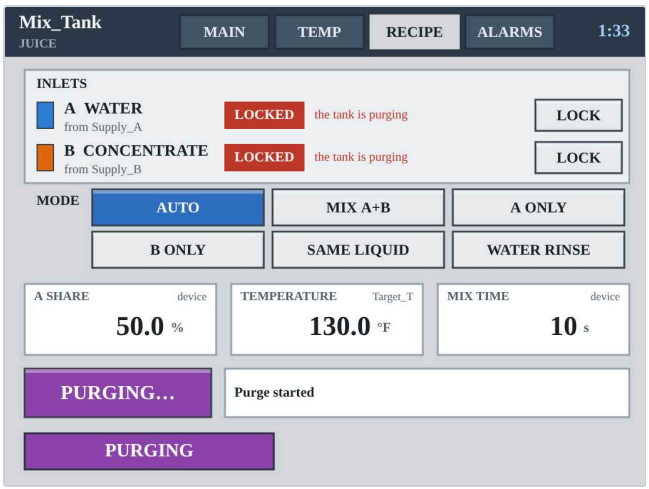


Figure 86. Purging: both inlets locked, the state PURGING.

TRY IT: WATER WHEN WATER IS CALLED FOR

On Level 7 with the reference program, open the Operator View and the RECIPE page. With the batch stopped, touch PRODUCT and pick WATER: inlet B goes LOCKED. Press START, and try to pick JUICE again: refused, the recipe is locked while the batch runs. Step 1 doses water as before, but in step 2 PumpB is on and nothing comes in. Watch FLOW B stay at 0.0, the Simulator Log line, and the tank never reaching HI, so the sequence waits in step 2. The program asked; the interlock said no. Now press STOP, touch **SAME LIQUID** and START again: inlet B runs water from Supply_A and the tank fills. Finish by pressing STOP, then PURGE, then CANCEL; press PURGE again and confirm. Pick JUICE again; it sets the mode back to AUTO.

TRY IT: LOCK OUT A TOTE

Select Supply_B and tick *Locked out*. Run the level: inlet B reads *Supply_B is locked out*, and the RECIPE page cannot unlock it, because the lock is on the tote, not the tank. That is how lockout/tagout works on a real line. Untick it to put the line back.

Purge on the supply vessels. From Version 138 the reservoir that feeds the filler (Supply_Tank on the bottle line), a plain tank and a tote have the same PURGE on their HMI page as the mix tank: refused while the process runs, confirmed, then the drain valve at the bottom opens and the vessel empties to the floor drain at 8 % of its capacity a second. The faceplate shows PURGING with a PURGE flow, the log says when it starts and when it is empty, and the drawing shows each vessel's drain: from the bottom outlet through its valve to a floor grate. The program sees the level fall and the low float make, which is the point of trying it: a step that waits for the reservoir to fill had better not mind it being emptied under it.

Worked example: the operator sets the heater

A mix tank, an HMI and five rungs. The operator types the temperature on the HMI; the program heats to it with a 2 °F band and never past the thermostat.

1. File → New Project… → **Empty sandbox**.
2. Place a **Mix Tank**. Set *Ingredient inlets* to One (A), *Starts at* 80 %, and *Panel and HMI: heater setpoint tag* to `Heat_SP`.
3. Place a **Process HMI** beside it. *Shows* can stay blank (the nearest tank is the mix tank).
4. Make the tags: `Heat_SP` REAL (the operator's setpoint), `Heat_Lo` REAL, `Heat_On` BOOL, and alias tags for the tank's `Temp` (Tank_Temp), `LoLevel` (Lo_Level), `Thermostat`, `Heater` and `Mixer`.
5. Enter the five rungs below, Verify, Run mode.

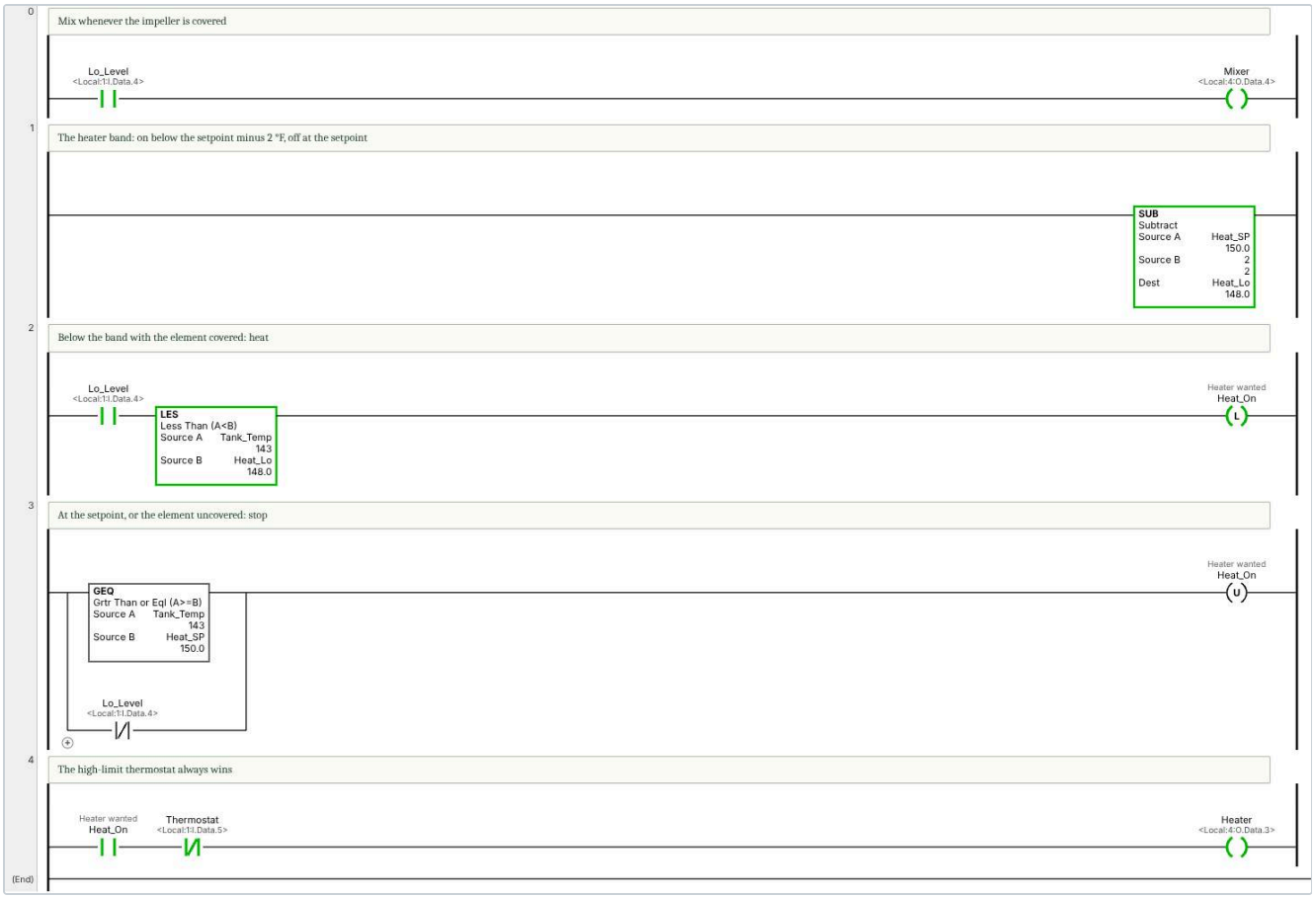


Figure 87. The five rungs, online with Heat_SP at 150.0: the band bottom is 148.0, the tank is at 143 °F, so Heat_On is latched and the heater is on.

- **Rung 0** runs the mixer whenever the impeller is covered.
- **Rung 1** works out the bottom of the band, `Heat_SP − 2`, every scan, so a new setpoint takes effect at once.
- **Rung 2** latches `Heat_On` below the band, but only with the element covered.
- **Rung 3** unlatches it at the setpoint, or as soon as LO uncovers. The two-rung latch is the hysteresis: between 148 and 150 °F the heater keeps doing what it was doing.
- **Rung 4** drives the output, and `XIO(Thermostat)` means the high limit always wins.

Run it. On the TEMP page the setpoint reads 0.0 and the heater stays off: the program does exactly what the operator asked. Touch the value, type 150, ENT. HEATER ON lights, the temperature climbs on the trend toward the green line, and the heater cycles between 148 and 150 °F from then on.

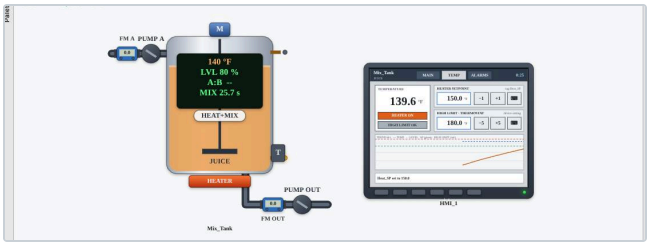


Figure 88. The example in the top view: the mix tank and the HMI on its TEMP page.

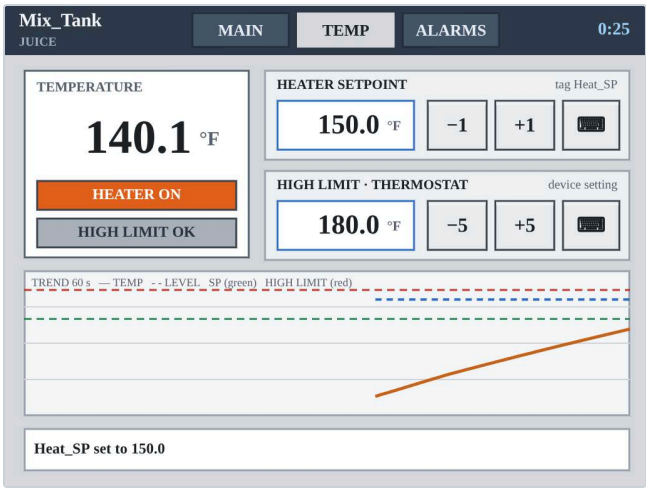


Figure 89. The HMI close up: setpoint 150.0, the heater on, the temperature climbing to the green line.

TRY IT: THE SETPOINT ABOVE THE LIMIT

With the example running, type 190 into HEATER SETPOINT while the high limit is 180. The temperature climbs to 180 and stops there. HIGH LIMIT MADE turns red, the heater drops out, comes back as the tank cools below the limit, and drops out again. The program still wants 190; the thermostat says no. That is what a high limit is for. Now lower the limit to 160 on the same page and watch it take over at 160.

TRY IT: TWO WRITERS, ONE TAG

Open Level 7 and its HMI (Tank_HMI points at the recipe tag Target_T). On the TEMP page, type 160. The message line turns red within a scan: the recipe rungs MOV the recipe temperature into Target_T on every scan, so the operator loses. Fix it the plant way. Add a REAL tag Temp_Trim, point the HMI at it, and change the heater rungs to compare against Target_T plus the trim (ADD them into a third tag). Now the recipe sets the base and the operator adjusts it, and nobody overwrites anybody.

TRY IT: HOW WIDE A BAND

In rung 1, change the 2.0 to 0.2 and watch the HEATER button on MAIN and the trend: the heater switches on and off every second or two, which on a real contactor is wear. Then try 5.0: hardly any switching, but the temperature swings 5 °F. The band is a trade between wear and accuracy, and it is the operator who sees the result on this screen.

WATCH FOR

A click in Program mode selects the HMI rather than pressing the screen: switch to Run mode to operate it. A setpoint tag that does not exist (a typo in the properties) shows "----" on the card; touch it and the message line names the missing tag. The Connections section shows it in red too. The setpoint is written as a number: point it at a REAL (or a DINT if whole degrees will do), not a BOOL or a TIMER.

Not only tanks: a packer faceplate

Set a Process HMI's *Shows* to a **Case Packer** and it draws a different screen in the same style. It has a top view of the case lane and the lanes the head picks from, with the head drawn where it is, and the head state (HOME, PICK DECK, PICK LOWER, PLACE) with its Z and the vacuum. It shows the bottles waiting in each lane, cases packed by size, and the case lane drive in Hz. START and STOP pulse the HMI's START and STOP tags, as on a tank. Below them are an equipment row, the state and a message line, then COUNTS and ALARMS pages. Two settings, *Packer: large cases count tag* and *small cases count tag*, put the program's own counts on the CASES cards; left blank, the cards show the packer's own counts. The bottle line's `Pack_HMI` is one (Lesson 14, chapter 29).

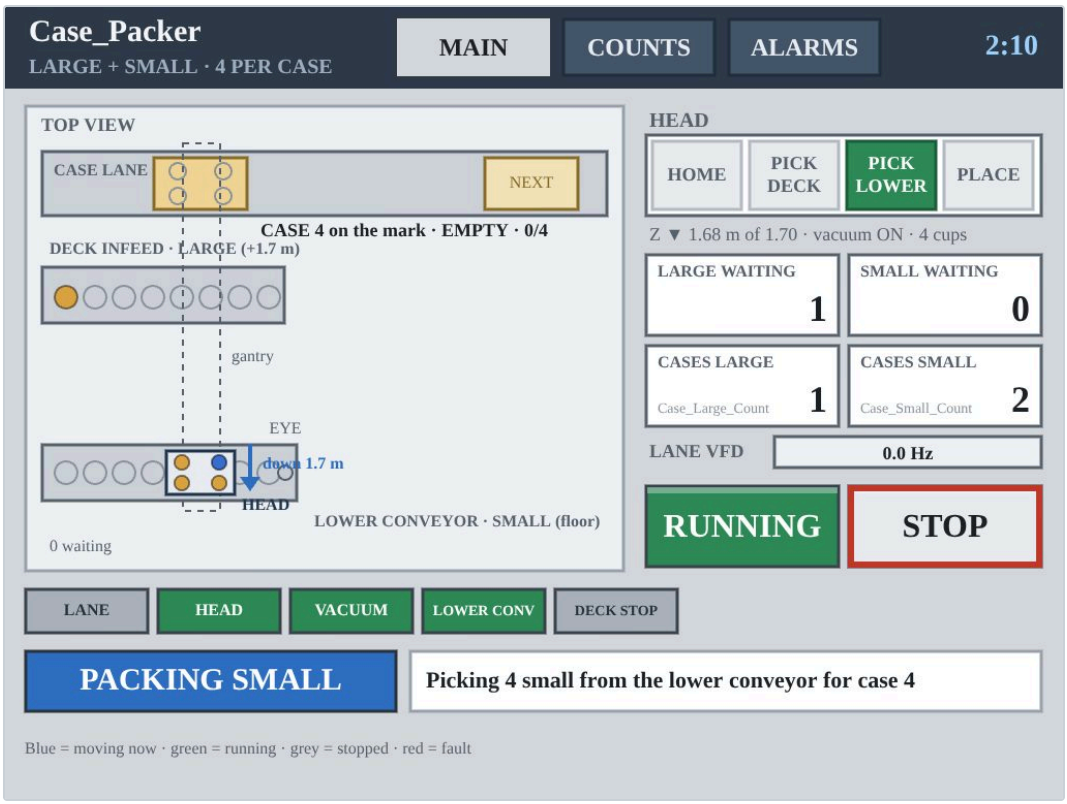


Figure 90. Pack_HMI on the bottle line: PICK LOWER, the head down 1.7 m at the floor conveyor, four small bottles for the empty case on the mark.

And a camera

Set *Shows* to a **Vision Sensor** and the HMI draws an In-Sight style camera screen. MAIN shows the last image with the tool graphics (on the bottle job: CAP, LABEL and FILL against its MIN line, green found, red not), PASS or FAIL, the four result-code bits, and the handshake bits live: the PLC's outputs (Trigger_Enable, Trigger, Buffer_Enable, Results_Ack) in blue, the camera's inputs in green. It also shows the result buffer and the counts. RESULTS lists the last ten inspections and ALARMS has the camera's events. It reads the sensor and its camera module and writes nothing. Level 8's `Camera_HMI` is one (chapter 40).

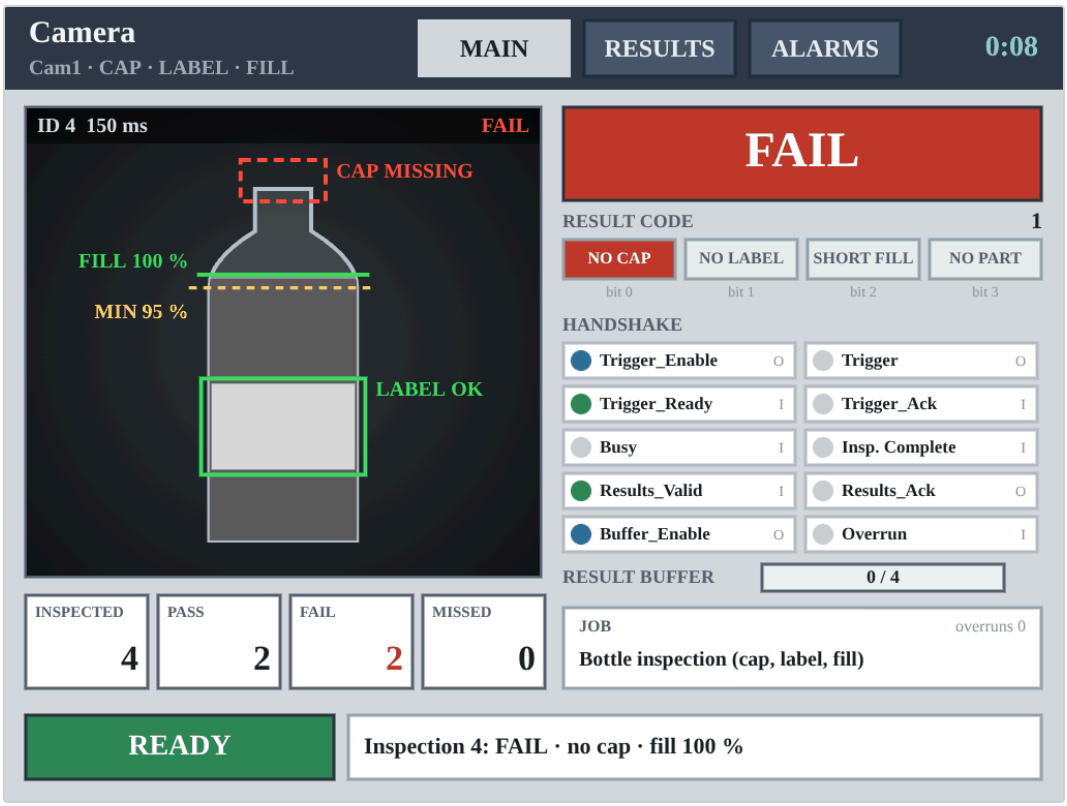


Figure 91. A camera on the HMI: a bottle with no cap. The cap tool is red, the result code is 1, Results_Valid is on and the PLC has not acknowledged it yet.

Where the liquid comes from: the supply under the floor

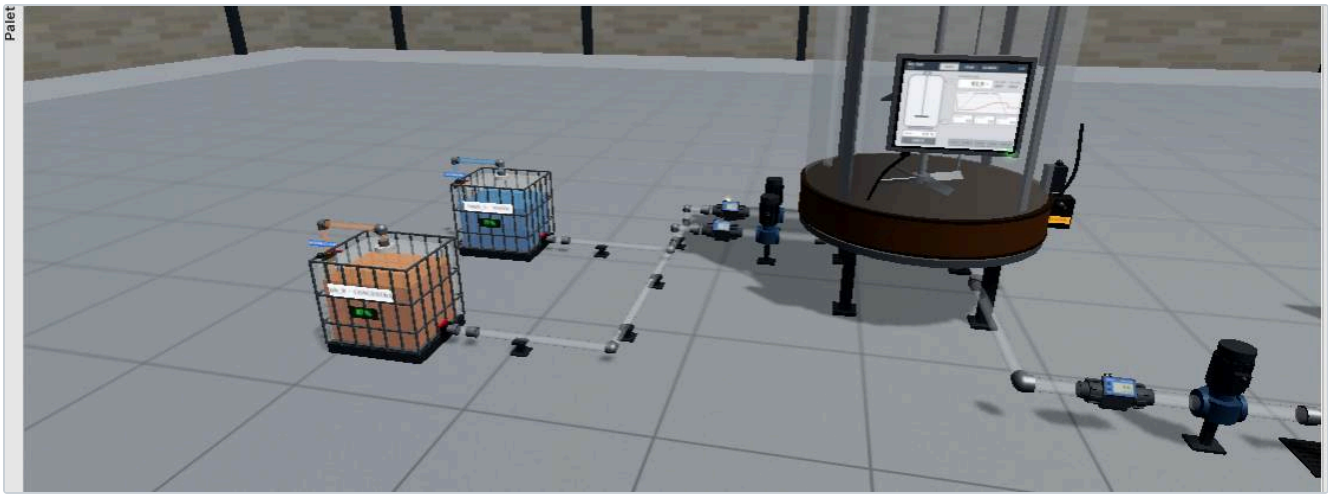


Figure 92. Level 7 in 3D: water and concentrate rise out of the floor beside the two totes and turn down into their fill caps; the totes feed the mix tank's inlets; the HMI stands to the right.

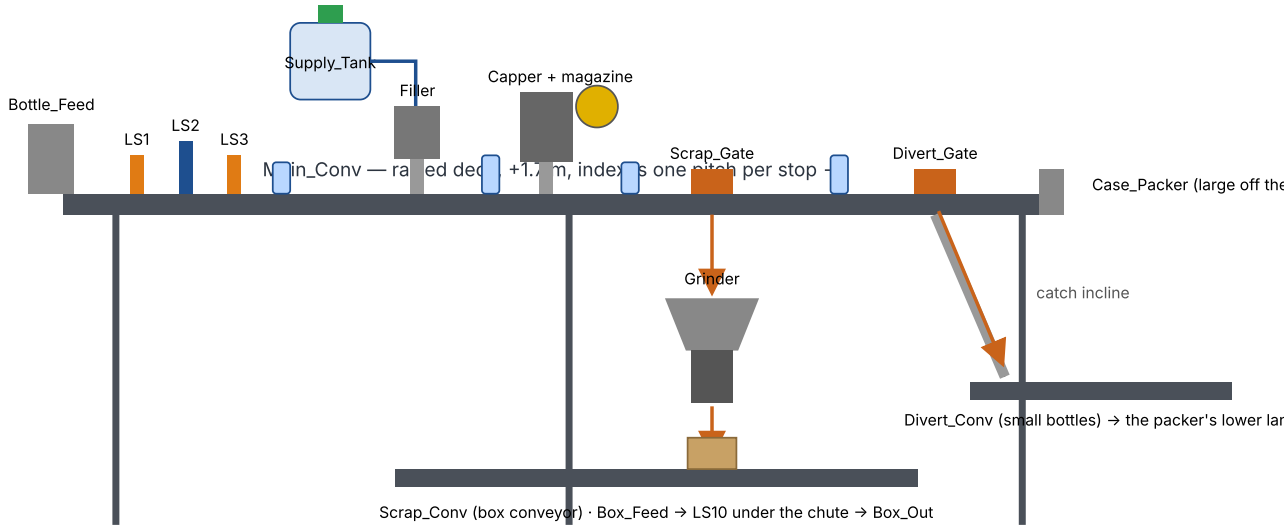
Every supply in the scene now starts somewhere. A supply tote is filled through a clear line that comes up out of a flanged connection in the floor beside it, turns over the cage and drops into the fill cap, with a ball valve at hand height; the line stands full and moves while the tote refills. An inlet on the *plant supply header* (a reservoir, a tank or a mix tank inlet with nothing piped to it) runs down into the floor the same way: the plant's supply mains run under the floor. Once you pipe the inlet to a tote or another tank, that line goes away and the pipe from the other part takes its place.

Part IV — Building the bottle line from ground zero

15 The line we are going to build

The machine is the LogixPro bottle line, the exercise generations of technicians learned shift registers on, rebuilt as a plant. A main conveyor on a raised deck indexes bottles one 40 px pitch at a time past three head sensors, a fill station fed from a soda reservoir, a capper with a rotary cap magazine, a scrap gate and a divert gate. Broken bottles are dropped off the deck into a shredder whose chute fills boxes on a box conveyor below; small bottles go down a catch incline onto a divert conveyor, which runs on under a case packer at the end of the line; large bottles run on along the deck to the same packer. The packer picks large bottles off the deck and small ones off the floor conveyor, 1.7 m below, and packs them four to a case, one size per case, on a case lane driven by a variable frequency drive. It has its own touch-screen HMI. An operator console has Start, Stop, a Run lamp, a red fault lamp and a blue Reset, a thumbwheel preset with Enter, and six displays: SMALL, LARGE, SCRAP, BOXES, IN BOX and CASES.

A name that changed. From Version 140 the reservoir in front of the filler is `Supply_Tank` and its tags are `Supply_Low`, `Supply_High`, `Supply_Inlet`, `Supply_Level` and `Supply_Flow`: it holds whatever the line runs, not syrup. A few pictures made before then still show it as `Syrup_Tank`; it is the same tank.



Side elevation (not to scale). Broken bottles drop into the shredder; small ones ride the incline; large ones run to the end.

Figure 93. The bottle line in one picture. Everything the gates do is a *drop*: the main conveyor is a raised deck, so a gate is a latch door onto the level below.

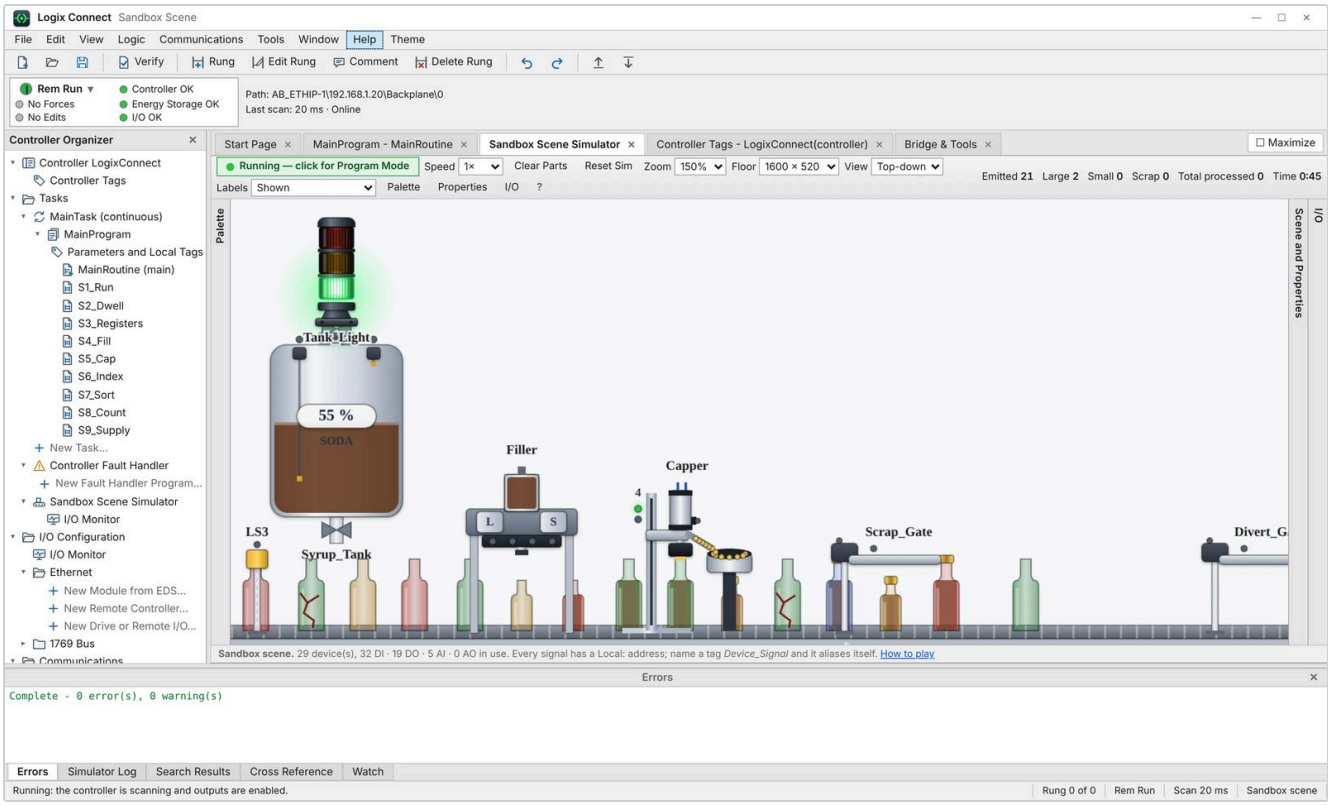


Figure 94. The finished line running in the Side view: bottles standing at the stops, the tank and filler, the capper, the gates over the lower level, the console on the right.

The program is thirteen routines called from the main routine, one per step, and each step is a lesson that leaves you with something that runs:

1. **Run circuit** — Start/Stop seal-in, the run lamp, the conveyors that run with the line.
2. **Dwell and feed** — a one-second timer while the belt stands, and one bottle per stop from the feeder.
3. **Shift registers** — three registers take the three head sensors once per stop.
4. **Fill** — a bottle under the nozzle gets its large or small charge from the head tank.
5. **Cap** — a cap from the chute, the cap feeder, the column lights.
6. **Index** — step exactly one pitch on the encoder when nothing holds the belt.
7. **Sort** — the scrap and divert gates, the box conveyor and the shredder.
8. **Count and preset** — the displays, the thumbwheel batch stop, Enter.
9. **Supply** — the reservoir's inlet on its floats, the head-tank protection, the tank stack light.
10. **Label** — an FLS-style dispenser one stop past the capper: Start / Ready / Error handshake, a latched fault, lamps.
11. **Alarms** — the program watching the machine: conveyor overloads, an end-of-line check of the tracking, a dirty eye, the labeler, the reservoir inlet; a fault lamp and Reset.

12. **Pack** — the case packer and the VFD on its case lane: fast, a creep onto the mark, a speed reference scaled to 0-10 V with SCP, four bottles a case.
13. **Mix** — the mix kitchen beside the line: water and cola syrup batched into soda at the 5 : 1 throw in a mix tank, heated and mixed to the recipe picked on its HMI, and pumped through a carbonator, which chills it and puts the CO2 in, to the reservoir whenever step 9 calls for it.

Guided mode

You can do Part IV with the book open beside the app, or let the app walk you through it. **File → New Project… → Guided: build the bottle line** (or **Help → Guided Mode** on an empty sandbox) opens a panel that follows these chapters one lesson at a time. Each lesson lists what it checks — parts placed at the book's positions, alias tags made, the routine and its JSR in place, the routine verifying — and, for the steps that do something, what it watches for while you run the line: Line_Run coming on when you press Start, a bottle filled and capped, a label going on. A check turns green when the page sees it happen. The reference rungs for the lesson sit under a fold with *Copy* and *Add* buttons; type the rungs yourself and use the buttons only when you are stuck. The panel checks your rungs itself. A reference rung is *logic verified* (one tick) when a rung of yours does the same thing: the page runs both through every combination of the bits they read and write, so the order of the contacts, how you draw a branch, or writing a tag as its alias address make no difference, and a timer or a move must be there with the same operands. A rung that is close but wrong stays open with one line on what differs, and the first one still to write is marked NEXT. In Run mode a verified rung is *proven* (two ticks) the first time it goes true; change it and it has to be proven again. Progress saves with the project; drag the panel by its header out of your way, or press its ▾ to fold it into the **Guided** menu in the top bar (next to Theme), which shows where you are (Level 1 · in progress, or 5 / 16 on the build) and lists the lessons, Next and Back, the reference rungs and Stop; *Show Guide Panel* brings it back.

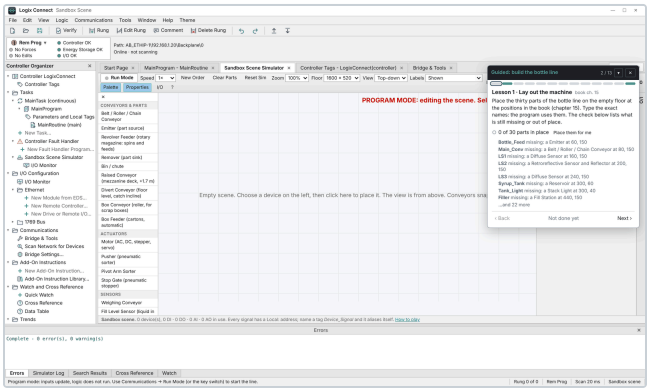


Figure 95. Guided mode on a new project: an empty floor and lesson 1.

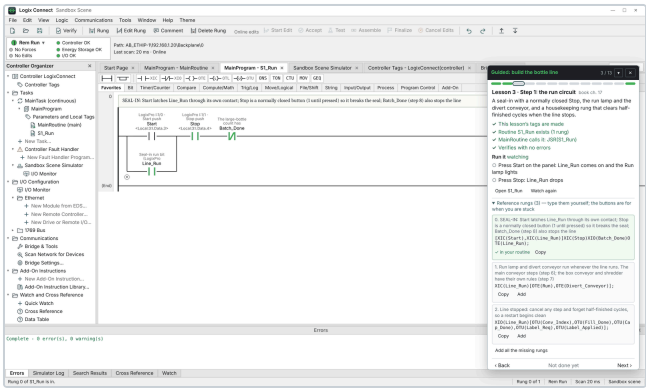


Figure 96. Lesson 3: the routine is in, Start pressed, the run checks watching.

Hints first

A reference rung you can read is a rung you will copy, and a rung you copied teaches you where the Copy button is. From Version 133 the fold does not hand the text over. Each reference rung opens as its **comment** alone: the job, in words, which is how a rung arrives in real life (somebody says 'latch Start on Go, but only if the drive is enabled and not moving') and where the work of programming actually is. Write it from that if you can. If you cannot, **Hint** lists what the rung uses, the contacts, the coils and the instructions, in no order and with no branches, which is the part a tag list and a wiring drawing would give you. **Shape** then shows the rung's skeleton with every tag blanked, `XIC(?)XIC(?)XIO(?)ONS(?)OTL(?)`, which is how many of what and in what order, and only **Show rung** gives the text and the Copy button. The note that tells you a rung of yours is close is blanked the same way until you ask for the text, so it can say 'yours does not have MOV(3,?)' without telling you what the 3 goes into. The routine's heading keeps the score, *n written (m without the text)*, which is the number worth raising. **Guided → Hints First** turns it off for a project; a new guided project starts with it on, and the lesson cards themselves (the tags, the routine, the run checks) are as they were.

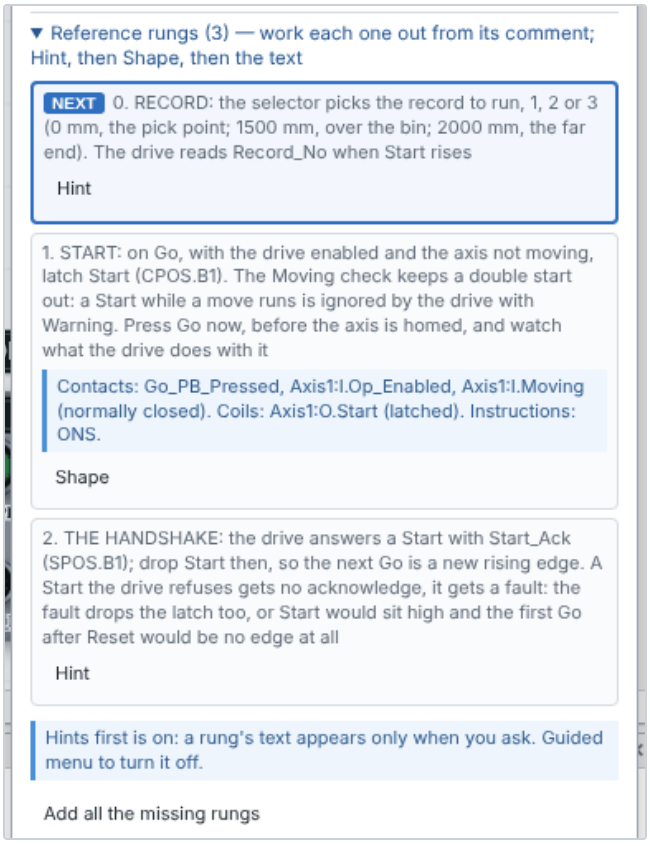


Figure 97. Hints first: the first rung at its comment, the second at Hint: what it uses, no order.

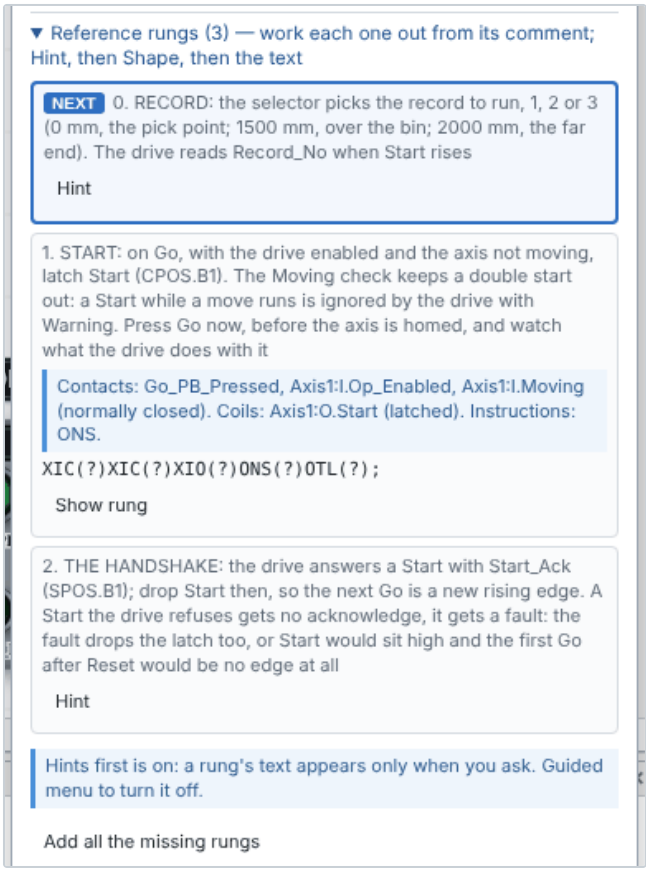


Figure 98. Shape: the skeleton with the tags blanked; Show rung is the last step.

Chapter 16 lays the machine out, chapter 17 makes the tags, chapters 18 to 30 write the thirteen routines, and chapter 31 commissions the line. Every coordinate and setting below is the one the finished template uses, so your machine and the answer key are the same machine.

16 Lesson 1 — Lay out the machine

IN THE APP

Guided mode lesson 1 is this chapter. It counts the parts you have placed against this list — name, type and position within 10 px — and names the ones still missing or out of place. *Place them for me* does the whole list in one click when you would rather get to the programming.

File → New Project… → **empty sandbox**. Set the floor to **1600 × 520** (it is the default) and the view to Top-down. Open the Palette. Place the parts below in order, then set each one's position in the Properties drawer (type the numbers; dragging is for later). Names: type them in the name field at the top of Properties; every name here is used by the program, so use them exactly.

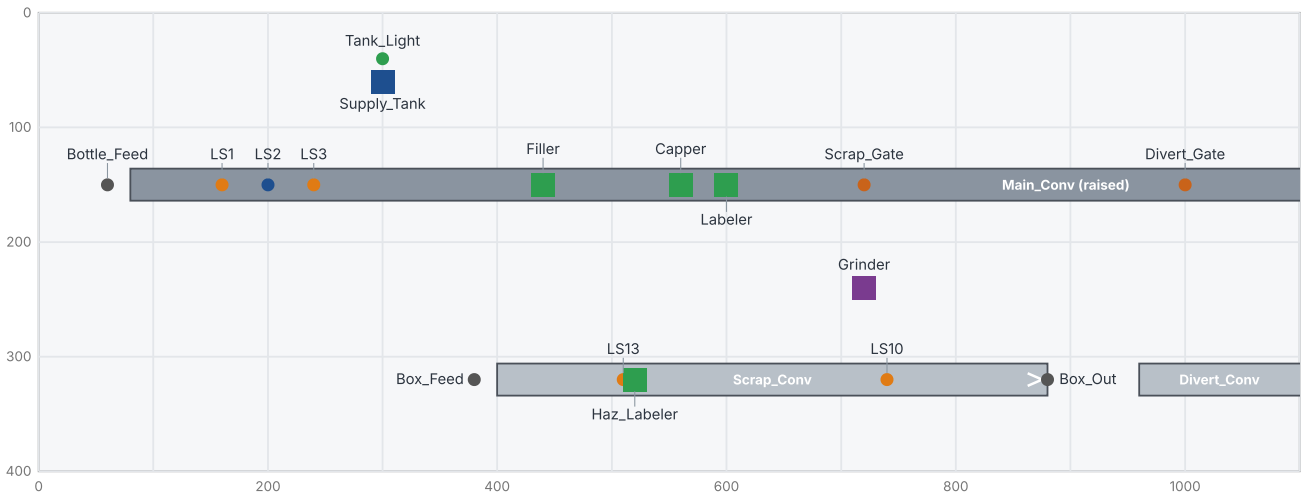


Figure 99. The plan, sheet 1: the main line, x 0 to 1100 and y 0 to 400 (nothing on the line sits lower), drawn from the template's own coordinates (grid in floor pixels; x to the right, y down). Belts are drawn along their length with an arrow for the direction of travel; everything else at its centre.

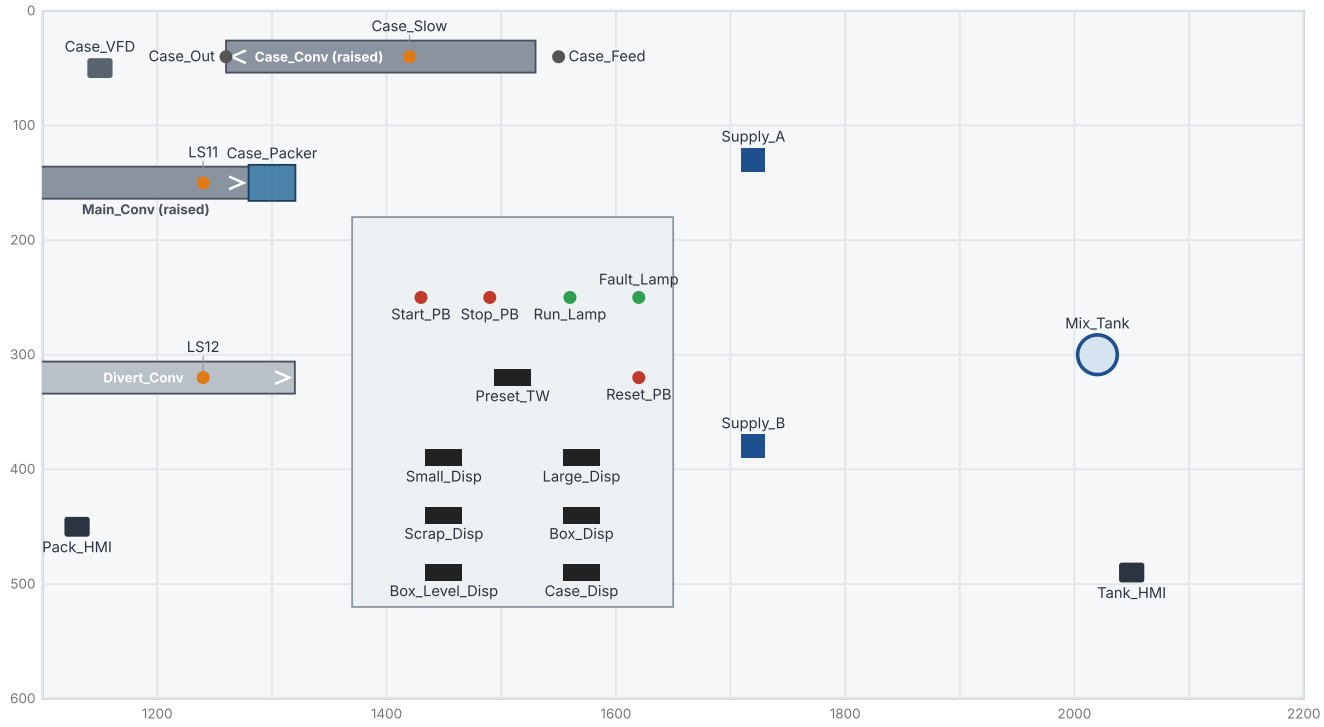


Figure 100. The plan, sheet 2: x 1100 to 2200, the end of the line and the case packer, the console on Panel_1, and the mix kitchen (placed in chapter 30, once the floor is 2400 × 600).

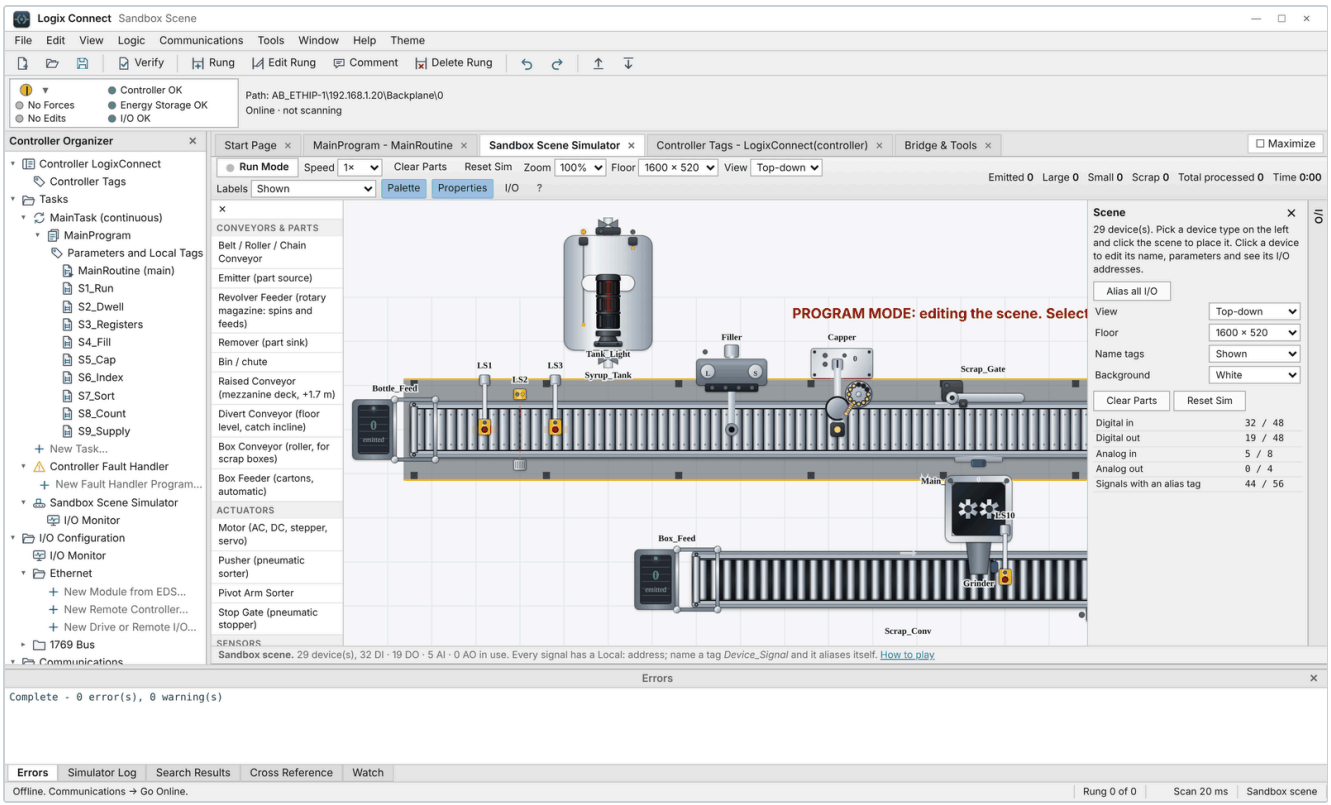


Figure 101. The finished layout in Top-down view with name tags on. Yours should look like this when the lists below are placed.

The main line

- **Emitter (part source)** at **x 60, y 150**, named `Bottle_Feed`: *Part to Emit* = **bottle, bottleL**; *Min Time (s, continuous / automatic)* = **2.5**; *Max Time (s, continuous / automatic)* = **4**; *Broken bottles (%)* = **15**. Edge mode: one bottle per rising edge of Emit. Fifteen per cent of them come out broken; that is the scrap the line has to deal with.
- **Belt / Roller / Chain Conveyor** at **x 80, y 150**, named `Main_Conv`: *Length (m), 0.5 to 40 in half-metre steps* = **20**; *Speed (mm/s)* = **60**; *Conveyor type* = **slat chain**; *Belt color* = **Grey**; *Height (3D)* = **Raised deck (+1.7 m)**; *Incremental encoder (adds A, B and Pulses)* = **True**; *Encoder resolution (px per pulse)* = **40**. Twenty metres of slat chain on a raised deck, with an encoder of 40 px per pulse: one pulse is one stop. The deck is what lets the gates drop bottles to the conveyors below.
- **Diffuse Sensor (photoelectric)** at **x 160, y 150**, named `LS1`: *Sensing range (px; 60 px = 1 m)* = **14**. Bottle present at the head of the line. On the 40 px stop grid, one stop before LS2.
- **Retroreflective Sensor and Reflector** at **x 200, y 150**, named `LS2`: *Beam height (px above the belt)* = **48**. A retro beam 48 px up: only a large bottle (56 px tall) breaks it; a small one (40 px) passes under.
- **Diffuse Sensor (photoelectric)** at **x 240, y 150**, named `LS3`: *Sensing range (px; 60 px = 1 m)* = **14**; *Sees* = **broken bottles**. Set to see broken bottles only.
- **Fill Station (tube, large and small fill)** at **x 440, y 150**, named `Filler`: *Fed from* = **Supply_Tank**. Fed from Supply_Tank; its own 300-unit head tank.
- **Inline Cap Feeder (pneumatic capper + cap bowl)** at **x 560, y 150**, named `Capper` (default settings). Thirty caps in the chute to start; the magazine refills it at 3 caps a second when Feeder runs.
- **Inline Labeler (FLS-style dispenser, RH / LH, any applicator)** at **x 600, y 150**, named `Labeler`: *Applicator* = **Wrap-around (roller)**. One stop past the capper. RH, wrap-around roller: the dispenser stands on the far side of the lane; 200 labels on the roll to start.
- **Pivot Arm Sorter** at **x 720, y 150**, named `Scrap_Gate` (default settings). A pivot arm. Its push side is +y, toward the grinder.
- **Pivot Arm Sorter** at **x 1000, y 150**, named `Divert_Gate` (default settings). Pushes toward the divert conveyor, 166 px below on the lower level.
- **Diffuse Sensor (photoelectric)** at **x 1240, y 150**, named `LS11`: *Sees* = **bottles only**. Set to see bottles only. One stop before the end of the line: the program checks here that the bottle it expects at this stop is really there (step 11).

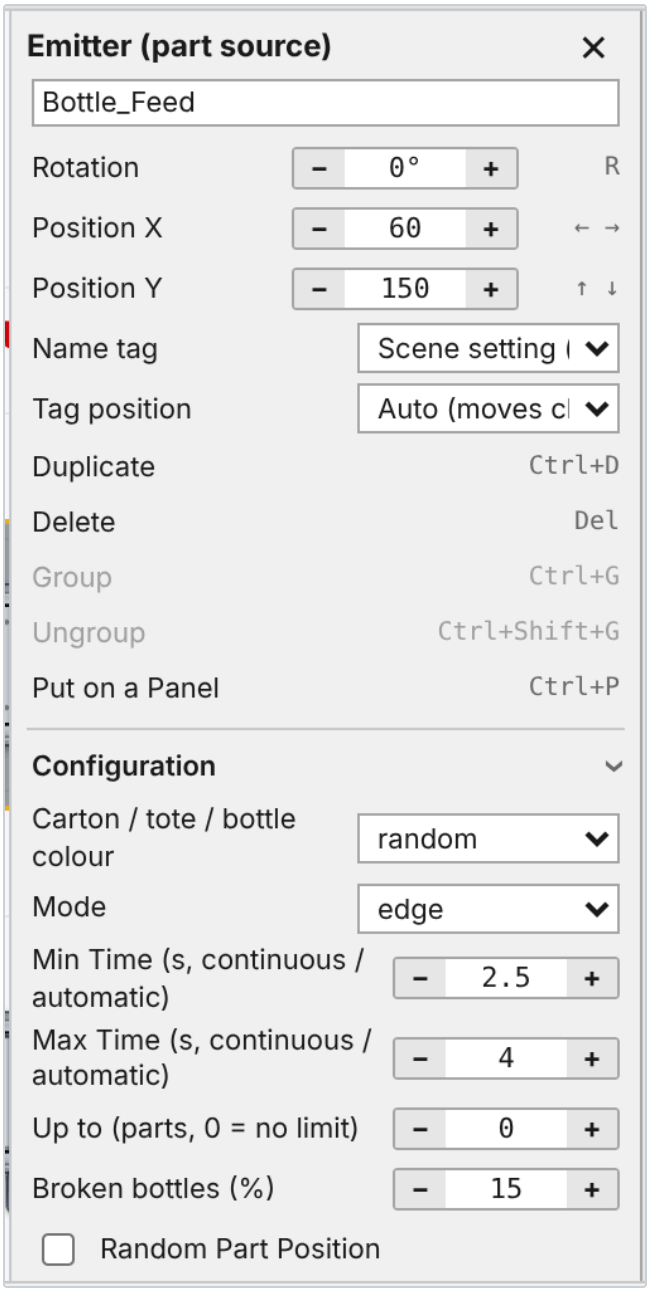


Figure 102. Main_Conv's settings: 20 m slat chain, raised deck, encoder 40 px per pulse.

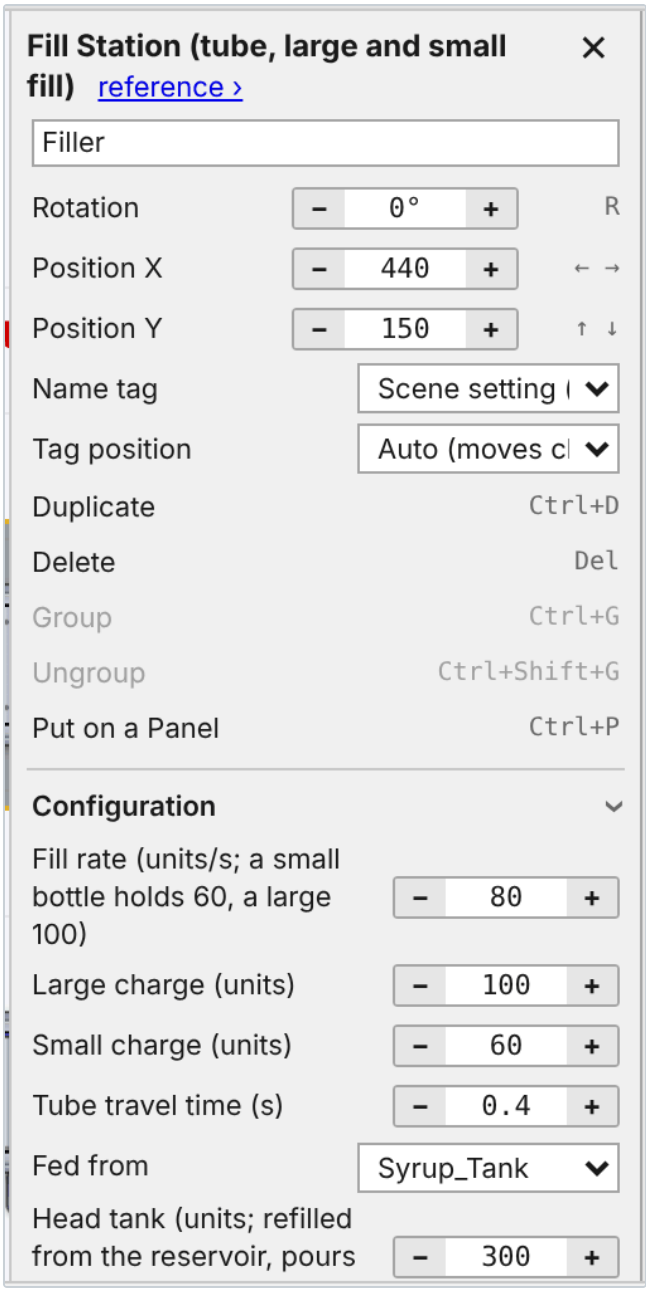


Figure 103. The Capper's settings.

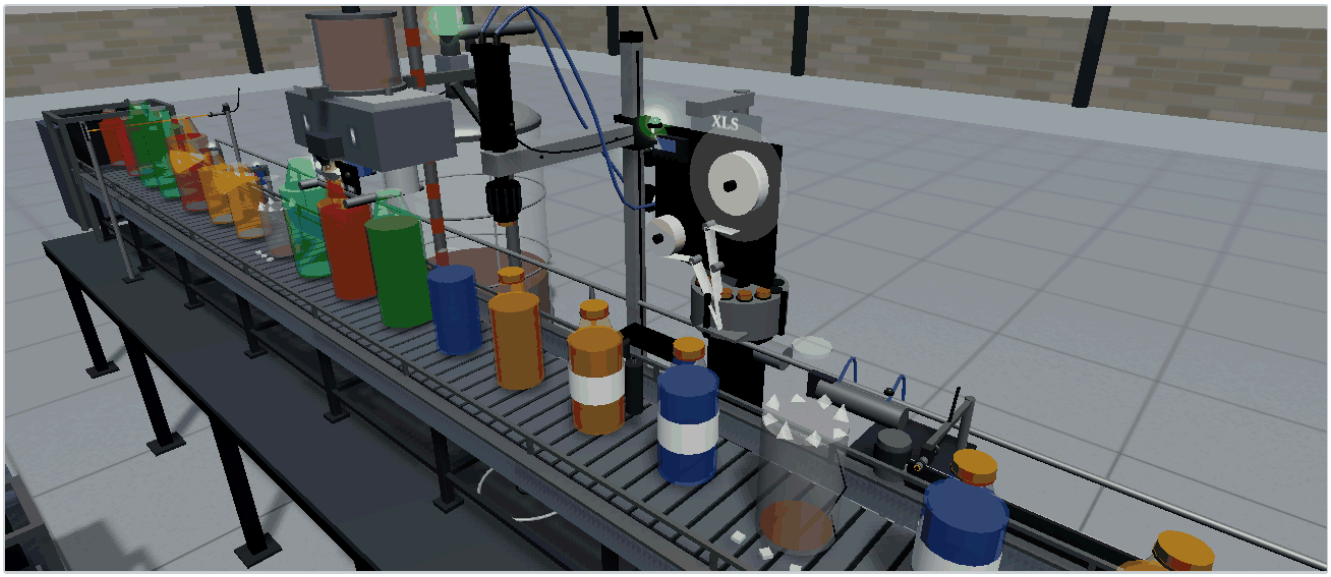


Figure 104. The Labeler on the line, one stop past the capper: the FLS-style dispenser on its column, the unwind reel, the web to the dispensing edge, the wrap-around roller at bottle height.

Packing

- **Case Packer (top-load: infeed buffer, pick head, case lane beside the line; an optional second pick lane on a conveyor below)** at **x 1300, y 150**, named `Case_Packer`: *Second pick lane below: the head drops to a conveyor on the floor (small bottles)* = **True**; *Lower lane offset across the line (px; positive = near side)* = **170**. At the end of the main line: its infeed takes the large bottles off the end of the deck, and its pick head reaches across to the case lane 108 px away on the far side. Tick *Second pick lane below* with the lower lane 166 px away on the near side: the divert conveyor runs on under the packer, and the head drops 1.7 m to pick the small bottles off its end.
- **Process HMI (operator terminal: a live faceplate of a tank, a case packer or a camera)** at **x 1130, y 450**, named `Pack_HMI`: *Shows* = **Case_Packer**; *START button tag (BOOL, pulsed while touched)* = **Pack_Start**; *STOP button tag (BOOL, pulsed while touched)* = **Pack_Stop**; *Running indicator tag (BOOL: lights START)* = **Pack_Run**; *Packer: large cases count tag (DINT)* = **Case_Large_Count**; *Packer: small cases count tag (DINT)* = **Case_Small_Count**; *Screen* = **10 inch**. The packer's own operator terminal, on a pedestal beside it. *Shows* Case_Packer; START, STOP and Running are the packer's own tags (step 12); the two count tags fill its CASES LARGE and CASES SMALL cards.
- **Belt / Roller / Chain Conveyor** at **x 1530, y 40**, rotation 180°, named `Case_Conv`: *Length (m), 0.5 to 40 in half-metre steps* = **4.5**; *Speed (mm/s)* = **150**; *Driven by drive module (its name; blank = contactor)* = **Case_VFD**; *Conveyor type* = **roller**; *Height (3D)* = **Raised deck (+1.7 m)**. The case lane: a raised roller conveyor running right to left (rotation 180°) past the packer head. Its *Driven by drive module* is Case_VFD, so it has no Fwd output of its own: the drive runs it.
- **Emitter (part source)** at **x 1550, y 40**, rotation 180°, named `Case_Feed`: *Part to Emit* = **cartonL**; *Carton / tote / bottle colour* = **Y**; *Mode* = **automatic (no Emit input)**. An empty carton every 3 s at the

upstream (right-hand) end, whenever there is room.

- **Diffuse Sensor (photoelectric)** at **x 1420, y 40**, named `Case_Slow`: *Sensing range (px; 60 px = 1 m)* = **12**; *Sees* = **boxes only**. Boxes only, 100 px before the packer head: the slow-down eye.
- **Remover (part sink)** at **x 1260, y 40**, rotation 180°, named `Case_Out` (default settings). At the downstream end of the lane: packed cases leave here.
- **VFD (variable frequency drive, hardwired: 0-10 V speed reference)** at **x 1150, y 50**, named `Case_VFD` (default settings). The drive for the case lane: 60 Hz at 10 V, 1 s accel, 0.5 s decel.

Figure 105. The end of the line in Top-down view: the packer's infeed table at the end of the deck, the gantry reaching over the case lane on the far side and down to the end of Divert_Conv on the near side, LS12 before the lower stop, the slow-down eye, the case feeder, the VFD, and Pack_HMI on the floor. The head is down on the lower lane.

Supply

- **Reservoir (product supply for fillers)** at **x 300, y 60**, named `Supply_Tank`: *Product* = **Soda**; *Holds (units; a small bottle takes 60, a large 100)* = **2000**; *Starts at (%)* = **35**; *Inlet flow (units/s)* = **150**; *Low float (%)* = **20**. Soda, 2000 units, starting at 35 % so you see the inlet work early. High float 90 %, low float 20 %. Leave *Inlet fed from* on the plant supply header for now: lesson 15 pipes it from the mix kitchen.
- **Stack Light** at **x 300, y 40**, named `Tank_Light`: *Mount* = **Direct**; *Built-in flasher* = **Red flashes**. Mounted directly on the tank dome; its red segment flashes when driven.

The lower level

- **Grinder (shredder under a raised belt, over a box conveyor)** at **x 720, y 240**, named `Grinder` (default settings). Placed 90 px below the scrap gate so a bottle pushed off the deck lands in its hopper; its chute reaches the box conveyor 76 px further down.
- **Emitter (part source)** at **x 380, y 320**, named `Box_Feed`: *Part to Emit* = **cartonM**; *Carton / tote / bottle colour* = **Y**; *Mode* = **automatic (no Emit input)**; *Min Time (s, continuous / automatic)* = **4**; *Max Time (s, continuous / automatic)* = **4**; *Automatic: count the time only while the exit is clear (keeps a gap after a stop)* = **True**. A carton every 4 s, by itself.
- **Belt / Roller / Chain Conveyor** at **x 400, y 320**, named `Scrap_Conv`: *Length (m), 0.5 to 40 in half-metre steps* = **8**; *Speed (mm/s)* = **80**; *Conveyor type* = **roller**. The box conveyor, LogixPro's "scrap conveyor". Eight metres of roller.
- **Diffuse Sensor (photoelectric)** at **x 510, y 320**, named `LS13`: *Sensing range (px; 60 px = 1 m)* = **4**; *Sees* = **boxes only**. The eye of the hazard-tape dispenser, ten pixels before its head: a box is seen in time for the wipe-on cycle to meet it (v1.24).
- **Inline Labeler (FLS-style dispenser, RH / LH, any applicator)** at **x 520, y 320**, named `Haz_Labeler`: *Hand (side of the lane the dispenser stands on, looking downstream)* = **LH**; *What it applies (v1.24)* = **Hazard tape (scrap boxes)**; *Applicator* = **Wipe-on (dispensing edge + brush)**; *Apply time (s)* = **0.3**; *Only label capped bottles (skips an open one and logs it)* = **False**. The second labeler, on the far side of the box conveyor (LH), set to hazard tape with a wipe-on applicator: it bands each scrap box yellow and black on its way to the chute. Boxes come off Box_Feed blank (v1.24).
- **Diffuse Sensor (photoelectric)** at **x 740, y 320**, named `LS10`: *Sensing range (px; 60 px = 1 m)* = **4**; *Sees* = **boxes only**. Boxes only, placed so a carton stops centred under the chute.
- **Remover (part sink)** at **x 880, y 320**, named `Box_Out` (default settings). At the end of the box conveyor.
- **Belt / Roller / Chain Conveyor** at **x 960, y 320**, named `Divert_Conv`: *Length (m), 0.5 to 40 in half-metre steps* = **6**; *Conveyor type* = **roller**. Starts under the divert gate; the catch incline from the deck lands on it. Six metres: it runs on to the end under the case packer, which picks the small bottles off it.

- **Diffuse Sensor (photoelectric)** at **x 1240, y 320**, named `LS12`: *Sensing range (px; 60 px = 1 m)* = **14**; *Sees* = **bottles only**. Bottles only. The stop eye at the end of the divert conveyor, just before the packer's lower lane: when the lane is full, a bottle waiting here stops the conveyor (step 7).

The operator console

- **Panel / Enclosure** at **x 1510, y 350**, named `Panel_1`: *Style* = **Operator console**; *Width (px)* = **280**; *Height (px)* = **340**; *Title plate (blank for none)* = **BOTTLE LINE**. Since 99.1 the console stands 60 px further right than in earlier editions, clear of the packer's lower lane.
- **Push Button** at **x 1430, y 250**, named `Start_PB` (default settings)
- **Push Button** at **x 1490, y 250**, named `Stop_PB`: *Color* = **red**; *Legend text (Legend button style)* = **STOP**; *Normally closed contact* = **True**. Normally closed: its OK signal is 1 until it is pressed.
- **Indicator Light / LED** at **x 1560, y 250**, named `Run_Lamp` (default settings)
- **Indicator Light / LED** at **x 1620, y 250**, named `Fault_Lamp`: *Color* = **red**; *Legend text (Legend plate style)* = **FAULT**. Flashes while an alarm is unacknowledged, steady once acknowledged (step 11).
- **Push Button** at **x 1620, y 320**, named `Reset_PB`: *Color* = **blue**; *Legend text (Legend button style)* = **RESET**. Acknowledges alarms and clears those whose cause has gone (step 11).
- **Thumbwheel Switch (digits + Enter)** at **x 1510, y 320**, named `Preset_TW` (default settings)
- **Digital Display (any tag)** at **x 1450, y 390**, named `Small_Disp`: *Tag to show (blank = the wired analog output)* = **Small_Bottle_Count**; *Width (characters)* = **4**; *Digit color* = **amber**; *Caption on the bezel (optional)* = **SMALL**. Shows the tag named in its settings; the tag does not exist yet, so the display reads dashes until chapter 17.
- **Digital Display (any tag)** at **x 1570, y 390**, named `Large_Disp`: *Tag to show (blank = the wired analog output)* = **Large_Bottle_Count**; *Width (characters)* = **4**; *Digit color* = **amber**; *Caption on the bezel (optional)* = **LARGE**
- **Digital Display (any tag)** at **x 1450, y 440**, named `Scrap_Disp`: *Tag to show (blank = the wired analog output)* = **Scrap_Bottle_Count**; *Width (characters)* = **4**; *Digit color* = **amber**; *Caption on the bezel (optional)* = **SCRAP**
- **Digital Display (any tag)** at **x 1570, y 440**, named `Box_Disp`: *Tag to show (blank = the wired analog output)* = **Scrap_Box_Count**; *Width (characters)* = **4**; *Digit color* = **amber**; *Caption on the bezel (optional)* = **BOXES**
- **Digital Display (any tag)** at **x 1450, y 490**, named `Box_Level_Disp`: *Tag to show (blank = the wired analog output)* = **Box_Level**; *Width (characters)* = **2**; *Caption on the bezel (optional)* = **IN BOX**
- **Digital Display (any tag)** at **x 1570, y 490**, named `Case_Disp`: *Tag to show (blank = the wired analog output)* = **Case_Count**; *Width (characters)* = **4**; *Digit color* = **amber**; *Caption on the bezel (optional)* = **CASES**. CASES: the case count from step 12.

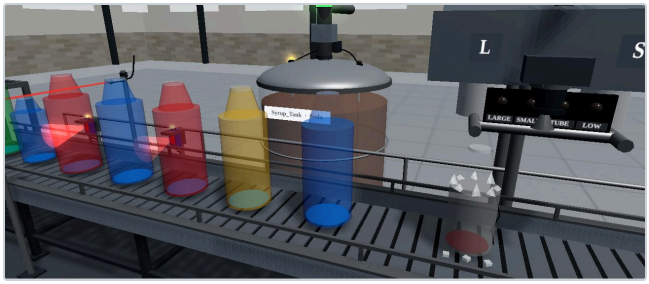


Figure 106. Supply_Tank with Tank_Light on its dome, and the pipe to the Filler.

Figure 107. The Grinder between the deck and the box conveyor, with LS10 and a box under the chute.

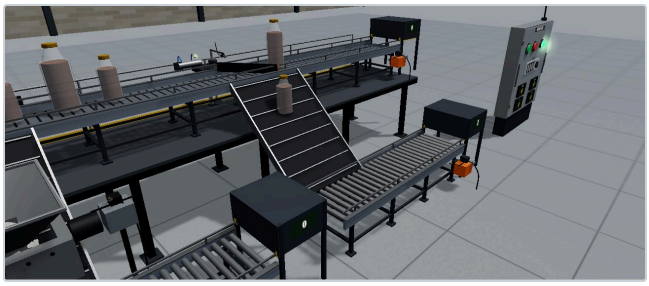


Figure 108. The Divert_Gate, its incline and Divert_Conv.

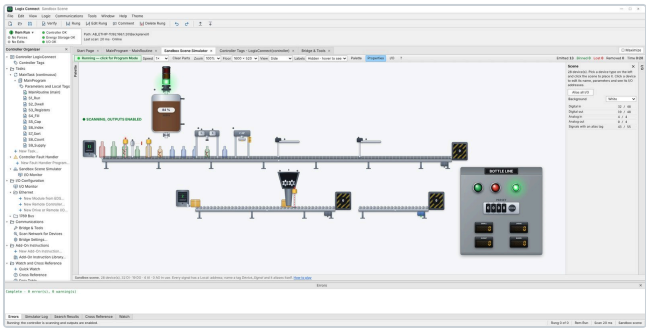


Figure 109. The console at the right-hand end of the floor.

That is 43 devices. Open the Scene sheet (click empty floor, Properties): it should read about 63 digital inputs, 34 digital outputs and 10 analog inputs in use. Switch to Side view and then, on a desktop, 3D, and walk along it: the pipe from the tank to the filler draws itself, the incline from the deck lands on the divert conveyor, the gates have their latch doors.

RUN IT WITH NO PROGRAM

Go to Run mode. Nothing moves — every output is off — but the feeders set to automatic are not outputs: boxes start arriving on the box conveyor and queue at its head, because the conveyor is not running either. The Simulator Log says nothing yet. Back to Program mode.

WATCH FOR

Positions are the centre of a conveyor's start and the centre of every other part. A sensor must be on the conveyor's centreline (y = 150 on the main line) to see bottles; a station must be at a stop position (x = 160 + 40n) to meet a bottle standing still. If you place by dragging, the snap does both for you.

17 Lesson 2 — Name the I/O: the tag list

> **IN THE APP**
>
> Guided mode lesson 2 is this chapter: it lists the alias tags still to make or to fix, and *Make them for me* makes them all.

Every point on the machine now has an address; the program needs names. This chapter is the complete list, so you can see the whole shape of it and look any tag up later — **you do not have to create them all now**. Each lesson from here on begins with *Tags this lesson needs*, a short table of the tags its rungs use for the first time; make those as you go and you have something running at the end of every chapter. Open **Controller Tags → Edit Tags**. The fastest route is **Tools → Create Alias Tags for Wired I/O**, which makes a `Device_Signal` alias for every signal on the floor — and then rename the ones the program calls by LogixPro names (click the name in the grid). Or create them by hand: name, type, *Alias*, and pick the point in *Alias For*. The Properties drawer of each part shows which point each signal has, so you are never guessing.

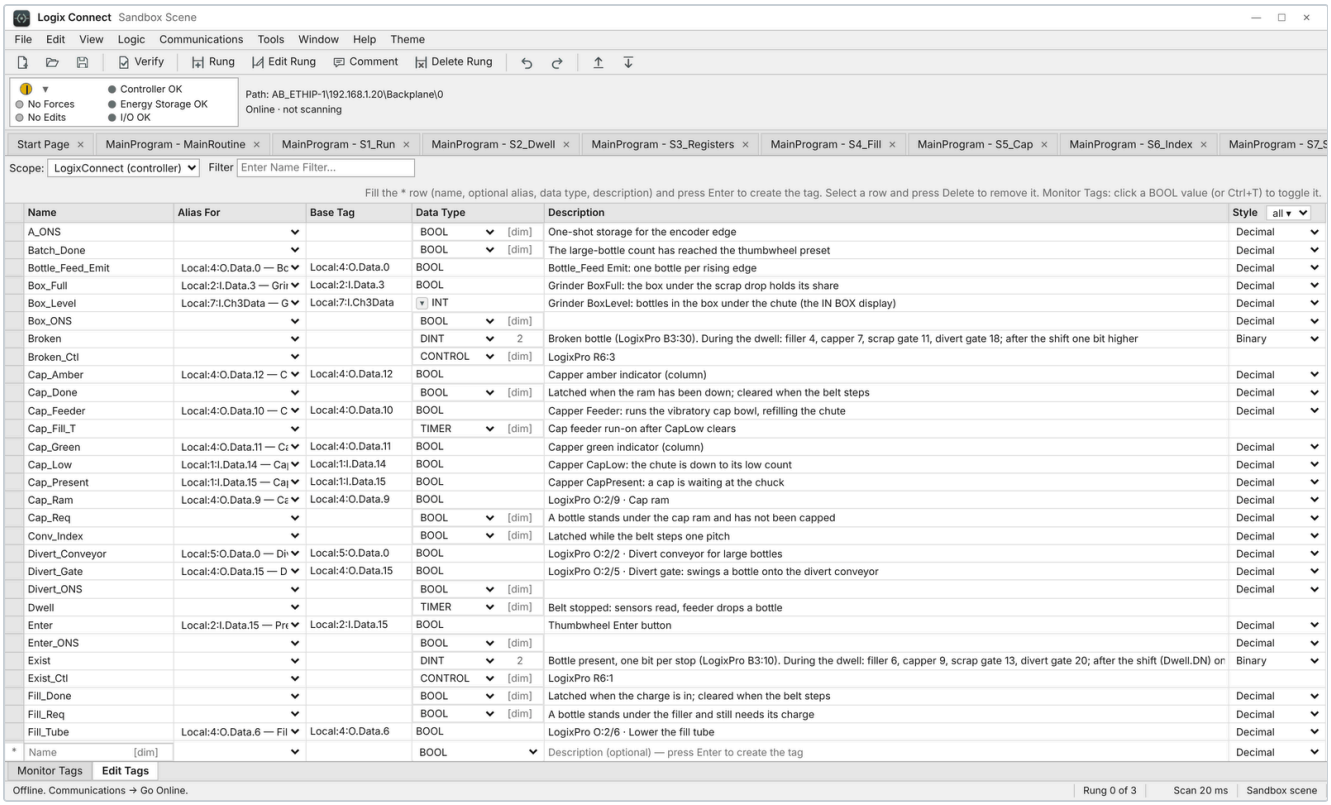


Figure 110. The finished tag list in Edit Tags. Make yours match this table.

Alias tags

Tag	Type	Alias for	Description
`Start`	BOOL	`Local:3:I.Data.9`	LogixPro I:1/0 · Start push button, NO
`Stop`	BOOL	`Local:3:I.Data.10`	LogixPro I:1/1 · Stop push button, NC (1 until pressed)
`LS1`	BOOL	`Local:1:I.Data.4`	LogixPro I:1/6 · Exist sensor: a bottle at the start of the line
`LS2`	BOOL	`Local:1:I.Data.5`	LogixPro I:1/7 · Size check: beam set so only a large bottle breaks it
`LS3`	BOOL	`Local:1:I.Data.6`	LogixPro I:1/8 · Broken check: sees broken bottles only
`LS4`	BOOL	`Local:1:I.Data.9`	LogixPro I:1/9 · Fill tube down
`LS5`	BOOL	`Local:1:I.Data.10`	LogixPro I:1/10 · Large charge delivered
`LS6`	BOOL	`Local:1:I.Data.11`	LogixPro I:1/11 · Small charge delivered
`LS7`	BOOL	`Local:1:I.Data.13`	LogixPro I:1/12 · Cap ram down
`LS8`	BOOL	`Local:2:I.Data.7`	LogixPro I:1/13 · Bottle at the scrap gate
`LS9`	BOOL	`Local:2:I.Data.11`	LogixPro I:1/14 · Bottle at the divert gate
`LS10`	BOOL	`Local:3:I.Data.0`	LogixPro I:1/15 · Scrap box under the grinder chute
`Main_Conveyor`	BOOL	`Local:4:O.Data.1`	LogixPro O:2/0 · Main conveyor motor
`Scrap_Conveyor`	BOOL	`Local:5:O.Data.4`	LogixPro O:2/1 · Box conveyor under the grinder
`Divert_Conveyor`	BOOL	`Local:5:O.Data.3`	LogixPro O:2/2 · Divert conveyor for large bottles
`Grinder`	BOOL	`Local:5:O.Data.1`	LogixPro O:2/3 · Grinder motor
`Scrap_Gate`	BOOL	`Local:5:O.Data.0`	LogixPro O:2/4 · Scrap gate: swings a bottle into the grinder
`Divert_Gate`	BOOL	`Local:5:O.Data.2`	LogixPro O:2/5 · Divert gate: swings a bottle onto the divert conveyor
`Fill_Tube`	BOOL	`Local:4:O.Data.6`	LogixPro O:2/6 · Lower the fill tube
`Large_Fill`	BOOL	`Local:4:O.Data.7`	LogixPro O:2/7 · Large fill valve
`Small_Fill`	BOOL	`Local:4:O.Data.8`	LogixPro O:2/8 · Small fill valve
`Cap_Ram`	BOOL	`Local:4:O.Data.9`	LogixPro O:2/9 · Cap ram
`Run`	BOOL	`Local:5:O.Data.8`	LogixPro O:2/10 · Run lamp
`Cap_Feeder`	BOOL	`Local:4:O.Data.10`	Capper Feeder: runs the vibratory cap bowl, refilling the chute
`Label_Apply`	BOOL	`Local:4:O.Data.13`	Labeler Start: the applicator comes in and presses a label on

Tag	Type	Alias for	Description
`Label_PadIn`	BOOL	`Local:2:I.Data.0`	Labeler applicator at the end of its stroke (label pressed on)
`Label_Ready`	BOOL	`Local:2:I.Data.1`	Labeler ready: web loaded, head home, no fault
`Label_Error`	BOOL	`Local:2:I.Data.2`	Labeler fault: end of roll, or the last cycle found no label
`Label_Low`	BOOL	`Local:2:I.Data.3`	Labeler roll low
`Label_Present`	BOOL	`Local:2:I.Data.4`	A label is peeled and waiting at the dispensing edge
`Label_Done`	BOOL	`Local:2:I.Data.5`	The bottle under the applicator already carries a label
`Label_Green`	BOOL	`Local:4:O.Data.14`	Labeler green indicator
`Label_Amber`	BOOL	`Local:4:O.Data.15`	Labeler amber indicator
`LS13`	BOOL	`Local:3:I.Data.2`	Box at the hazard-tape dispenser
`Haz_Apply`	BOOL	`Local:5:O.Data.5`	Hazard-tape dispenser: start the wipe-on cycle
`Haz_PadIn`	BOOL	`Local:3:I.Data.3`	Hazard-tape dispenser: applicator at the end of its stroke
`Haz_Ready`	BOOL	`Local:3:I.Data.4`	Hazard-tape dispenser ready: web loaded, applicator home, no fault
`Haz_Error`	BOOL	`Local:3:I.Data.5`	Hazard-tape dispenser fault: end of roll, or the last cycle found no tape
`Haz_Low`	BOOL	`Local:3:I.Data.6`	Hazard-tape roll low
`Haz_Present`	BOOL	`Local:3:I.Data.7`	A tape band is peeled and waiting at the dispensing edge
`Haz_Done`	BOOL	`Local:3:I.Data.8`	The box under the applicator already carries its hazard tape
`Haz_Green`	BOOL	`Local:5:O.Data.6`	Hazard-tape dispenser green indicator
`Haz_Amber`	BOOL	`Local:5:O.Data.7`	Hazard-tape dispenser amber indicator
`Cap_Green`	BOOL	`Local:4:O.Data.11`	Capper green indicator (column)
`Cap_Amber`	BOOL	`Local:4:O.Data.12`	Capper amber indicator (column)
`Cap_Low`	BOOL	`Local:1:I.Data.14`	Capper CapLow: the chute is down to its low count
`Cap_Present`	BOOL	`Local:1:I.Data.15`	Capper CapPresent: a cap is waiting at the chuck
`Thumbwheel`	INT	`Local:9:I.Ch0Data`	LogixPro I:3 · Thumbwheel switch, read as a number (BCD on the real one)
`Enter`	BOOL	`Local:3:I.Data.13`	Thumbwheel Enter button

Tag	Type	Alias for	Description
`LS11`	BOOL	`Local:3:I.Data.11`	End-of-line eye at 1240: verifies the tracking (only large, whole bottles reach it)
`Main_Conv_Running`	BOOL	`Local:1:I.Data.1`	Main conveyor running feedback (motor contactor)
`Divert_Conv_Running`	BOOL	`Local:2:I.Data.12`	Divert conveyor running feedback
`Fault_Lamp`	BOOL	`Local:5:O.Data.9`	Red fault lamp on the console: flashing = unacknowledged alarm, steady = alarm still active
`Reset_PB`	BOOL	`Local:3:I.Data.12`	Blue Reset button: acknowledge alarms and clear the ones whose cause is gone
`Case_Run`	BOOL	`Local:5:O.Data.13`	Case lane drive: run (two-wire)
`Case_VFD_Reset`	BOOL	`Local:5:O.Data.15`	Case lane drive: fault reset (rising edge)
`Case_Speed_Ref`	INT	`Local:8:O.Ch0Data`	Case lane drive speed reference: 0-10000 = 0-10 V = 0-60 Hz
`Case_VFD_Ready`	BOOL	`Local:10:I.Data.10`	Case lane drive ready (not tripped)
`Case_VFD_Running`	BOOL	`Local:10:I.Data.11`	Case lane drive output frequency above zero
`Case_At_Speed`	BOOL	`Local:10:I.Data.12`	Case lane drive at the commanded speed
`Case_VFD_Fault`	BOOL	`Local:10:I.Data.13`	Case lane drive tripped
`Case_Hz_Fbk`	INT	`Local:9:I.Ch3Data`	Case lane drive output frequency: 0-10000 = 0-10 V = 0-60 Hz
`Case_Slow`	BOOL	`Local:10:I.Data.9`	Slow-down eye on the case lane, 100 px before the packer head
`Case_In_Place`	BOOL	`Local:10:I.Data.1`	A case is centred under the packer head
`Case_Full`	BOOL	`Local:10:I.Data.2`	The case under the head has been packed
`Group_Ready`	BOOL	`Local:3:I.Data.15`	The packer infeed holds a case worth (4)
`Infeed_Full`	BOOL	`Local:10:I.Data.0`	Packer infeed buffer full: bottles arriving now wait on the belt
`Infeed_Count`	INT	`Local:9:I.Ch1Data`	Bottles in the packer infeed
`Packer_Ready`	BOOL	`Local:3:I.Data.14`	Packer head home, not cycling
`Pack_Drop`	BOOL	`Local:5:O.Data.10`	Packer: pick a case worth from the infeed and place it in the case (rising edge)
`Pick_Lower`	BOOL	`Local:5:O.Data.11`	Packer: the next Drop picks from the lower lane (small bottles) instead of the deck infeed
`Lower_Ready`	BOOL	`Local:10:I.Data.3`	The packer lower lane holds a case worth (4 small bottles)

Tag	Type	Alias for	Description
`Lower_Full`	BOOL	`Local:10:I.Data.4`	Packer lower lane full: small bottles arriving now wait at the stop
`Lower_Count`	INT	`Local:9:I.Ch2Data`	Small bottles waiting in the packer lower lane
`Case_Small`	BOOL	`Local:10:I.Data.5`	The packed case under the head holds small bottles
`LS12`	BOOL	`Local:2:I.Data.13`	Stop eye at the end of the divert conveyor, just before the packer lower lane
`Main_Conv_A`	BOOL	`Local:1:I.Data.2`	Main_Conv encoder channel A: one rising edge per 40 px pitch
`Bottle_Feed_Emit`	BOOL	`Local:4:O.Data.0`	Bottle_Feed Emit: one bottle per rising edge
`Box_Full`	BOOL	`Local:2:I.Data.9`	Grinder BoxFull: the box under the scrap drop holds its share
`Box_Level`	INT	`Local:7:I.Ch3Data`	Grinder BoxLevel: bottles in the box under the chute (the IN BOX display)
`Scrap_Waiting`	BOOL	`Local:2:I.Data.8`	Grinder Detect: a broken bottle is waiting in the hopper
`Tank_Level`	INT	`Local:7:I.Ch2Data`	Filler head tank, 0-100 %
`Tank_Low`	BOOL	`Local:1:I.Data.12`	Filler head tank below 15 %
`Supply_Low`	BOOL	`Local:1:I.Data.8`	Supply_Tank low float
`Supply_High`	BOOL	`Local:1:I.Data.7`	Supply_Tank high float
`Supply_Inlet`	BOOL	`Local:4:O.Data.2`	Supply_Tank inlet valve
`Supply_Level`	INT	`Local:7:I.Ch1Data`	Supply_Tank level, 0-100 %
`Carb_CO2`	BOOL	`Local:6:O.Data.5`	Carbonator CO2 inlet valve
`Carb_Press`	INT	`Local:11:I.Ch2Data`	Carbonator tank pressure, psi
`Carb_Vol`	INT	`Local:11:I.Ch3Data`	CO2 in the soda going to Supply_Tank, hundredths of a volume (350 = 3.50)
`Tank_Red`	BOOL	`Local:4:O.Data.3`	Tank_Light red: reservoir at the low float (flashes)
`Tank_Amber`	BOOL	`Local:4:O.Data.4`	Tank_Light amber: inlet open, refilling
`Tank_Green`	BOOL	`Local:4:O.Data.5`	Tank_Light green: level healthy

Plain tags

These are the program's own memory. The three registers are DINT arrays of two elements (Dim 2 — 64 bits) shown in Binary; the ONS storage bits are BOOLs the one-shots own; the timers and the CONTROL tags belong to their instructions.

Tag	Type	Style	Description
`Haz_Req`	BOOL		Hazard tape requested for the box at LS13 (latched while the applicator cycles)
`Haz_Fault`	BOOL		Hazard-tape dispenser fault, latched until Enter
`Cap_Fill_T`	TIMER		Cap feeder run-on after CapLow clears
`Large_Bottle_Count`	DINT		LogixPro O:4/0 · Large bottle count, shown on the LARGE display
`Small_Bottle_Count`	DINT		LogixPro O:4/1 · Small bottle count, shown on the SMALL display
`Scrap_Bottle_Count`	DINT		LogixPro O:4/2 · Scrap bottle count, shown on the SCRAP display
`Scrap_Box_Count`	DINT		LogixPro O:4/3 · Scrap box count, shown on the BOXES display
`Line_Run`	BOOL		Seal-in run bit (LogixPro B3:0/0)
`Conv_Index`	BOOL		Latched while the belt steps one pitch
`A_ONS`	BOOL		One-shot storage for the encoder edge
`Dwell`	TIMER		Belt stopped: sensors read, feeder drops a bottle
`Exist`	DINT[2]	Binary	Bottle present, one bit per stop (LogixPro B3:10). During the dwell: filler 6, capper 9, scrap gate 13, divert gate 20; after the shift (Dwell.DN) one bit higher
`Large`	DINT[2]	Binary	Large bottle (LogixPro B3:20). During the dwell: filler 5, capper 8, scrap gate 12, divert gate 19; after the shift one bit higher
`Broken`	DINT[2]	Binary	Broken bottle (LogixPro B3:30). During the dwell: filler 4, capper 7, scrap gate 11, divert gate 18; after the shift one bit higher
`Exist_Ctl`	CONTROL		LogixPro R6:1
`Large_Ctl`	CONTROL		LogixPro R6:2
`Broken_Ctl`	CONTROL		LogixPro R6:3
`Fill_Req`	BOOL		A bottle stands under the filler and still needs its charge
`Fill_Done`	BOOL		Latched when the charge is in; cleared when the belt steps
`Cap_Req`	BOOL		A bottle stands under the cap ram and has not been capped
`Cap_Done`	BOOL		Latched when the ram has been down; cleared when the belt steps
`Label_Req`	BOOL		Latched while the labeler is asked to apply; cleared at PadIn
`Label_Applied`	BOOL		Latched when the label is pressed on; cleared when the belt steps
`Label_Fault`	BOOL		Latched on the labeler Error input (end of roll or a missed label); Enter clears it

Tag	Type	Style	Description
`Alarm_Word`	DINT	Binary	Latched alarms, one bit each: 0 main conveyor overload, 1 divert conveyor overload, 2 end-of-line check failed (a jam, a lost bottle, tracking out of step), 3 LS1 dark too long (dirty lens or feeder empty), 4 labeler fault, 5 reservoir inlet open but level not rising, 6 case lane drive tripped, 7 packer infeed full
`Alarm_Unack`	BOOL		New alarm, not yet acknowledged (Reset)
`Main_OL_T`	TIMER		Main conveyor overload timer (no running feedback)
`Div_OL_T`	TIMER		Divert conveyor overload timer (no running feedback)
`EOL_Missing`	BOOL		LS11 dark when the registers said a large, whole bottle was there
`EOL_Extra`	BOOL		LS11 saw a bottle the registers did not expect
`EOL_Inhibit`	BOOL		Tracking cleared by Reset: the end-of-line check waits until the first bottle tracked since then reaches LS11
`EOL_ONS`	BOOL		
`LS1_Dark_T`	TIMER		LS1 dark too long timer
`Inlet_T`	TIMER		Inlet not filling timer
`Flash_T`	TIMER		Fault lamp flasher
`OL_ONS`	BOOL		
`DOL_ONS`	BOOL		
`BLK_ONS`	BOOL		
`Reg_Clear`	BOOL		
`DRK_ONS`	BOOL		
`LBL_ONS`	BOOL		
`INL_ONS`	BOOL		
`RST_ONS`	BOOL		
`VFD_ONS`	BOOL		
`INF_ONS`	BOOL		
`Case_Hz`	REAL		Case lane speed the program wants, in Hz
`Case_Hz_Actual`	REAL		Case lane drive output frequency read back, in Hz
`Case_Count`	DINT		Cases packed, shown on the CASES display
`Case_Creep`	BOOL		Latched from the slow-down eye until an empty case stands under the head
`Drop_ONS`	BOOL		

Tag	Type	Style	Description
`Case_ONS`	BOOL		
`Scrap_ONS`	BOOL		
`Divert_ONS`	BOOL		
`Small_ONS`	BOOL		
`Large_ONS`	BOOL		
`Box_ONS`	BOOL		
`Enter_ONS`	BOOL		
`Batch_Done`	BOOL		The large-bottle count has reached the thumbwheel preset
`Dose_Pre`	DINT		Water dose for this batch, in flowmeter pulses
`Dose_A`	COUNTER		Water dose: flowmeter A pulses
`Case_Large_Count`	DINT		Cases of large bottles packed (CASES LARGE on Pack_HMI)
`Case_Small_Count`	DINT		Cases of small bottles packed (CASES SMALL on Pack_HMI)
`Pack_Start`	BOOL		START on Pack_HMI (pulsed by the touch screen)
`Pack_Stop`	BOOL		STOP on Pack_HMI (pulsed by the touch screen)
`Pack_Run`	BOOL		The packer is running (its own seal-in)
`Pack_Auto_Start`	BOOL		One scan when the line starts: starts the packer too
`Pack_Line_ONS`	BOOL		

Addresses in the alias column are the ones the template's parts get when placed in chapter 16's order; if yours differ (you placed something twice and deleted it, say) use the address in your Properties drawer — the name is what matters to the program.

CHECK IT

Run mode, Controller Tags → Monitor Tags. Push Start_PB on the panel and watch `Start` go to 1. Hold a hand over nothing: `Stop` is already 1 — it is a normally-closed button. Put a bottle in front of LS1 by hand: there is no belt running, so instead toggle `Bottle_Feed_Emit` (Ctrl+T) and watch a bottle appear at the head of the line and LS1 go to 1 a scan later. Back to Program mode and Reset Sim.

18 Lesson 3 — Step 1: the run circuit

A machine has one bit that says it is running, and everything else hangs off it. Make a routine called `S1_Run` (Logic → New Routine…) and give MainRoutine a single rung, `JSR(S1_Run);`. Then write these three rungs in S1_Run:

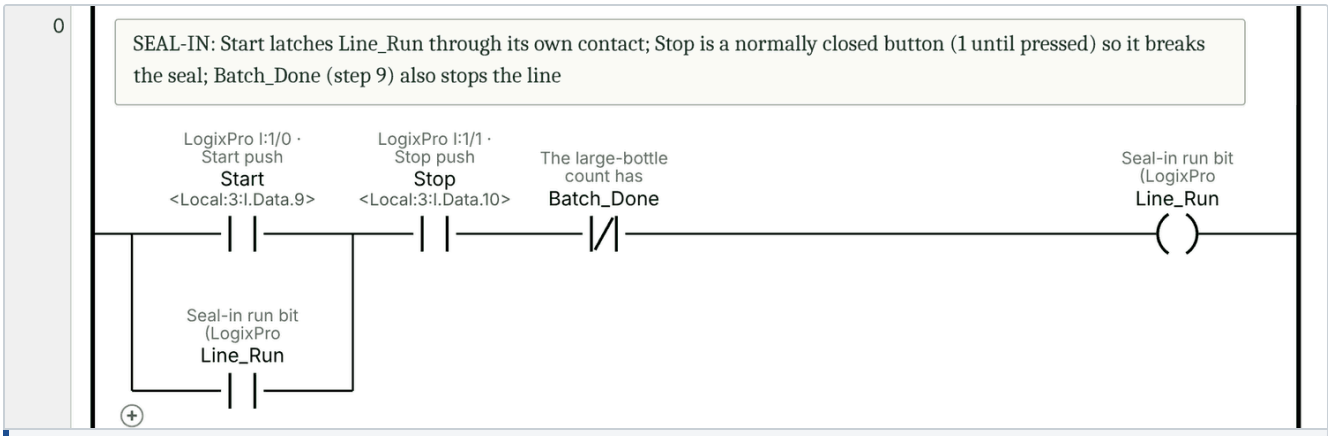
> **IN THE APP**
>
> Guided mode lesson 3 is this chapter: it checks that this lesson's tags are made, that `S1_Run` exists, is called from MainRoutine and verifies, and then watches the line while you run it and ticks each *Run it* result below as it happens. Help → Guided Mode, or start from File → New Project… → Guided: build the bottle line.

Tags this lesson needs

Create these before you type the rungs (Verify reports any you miss as unknown). Tags from earlier lessons are not repeated.

Tag	Type	How to make it	Description
`Start`	BOOL	alias for `Local:3:I.Data.9` — the Properties drawer's *tag* link makes it, then rename it	LogixPro I:1/0 · Start push button, NO
`Line_Run`	BOOL	Edit Tags: type the name, pick the type	Seal-in run bit (LogixPro B3:0/0)
`Stop`	BOOL	alias for `Local:3:I.Data.10` — the Properties drawer's *tag* link makes it, then rename it	LogixPro I:1/1 · Stop push button, NC (1 until pressed)
`Batch_Done`	BOOL	Edit Tags: type the name, pick the type	The large-bottle count has reached the thumbwheel preset
`Run`	BOOL	alias for `Local:5:O.Data.8` — the Properties drawer's *tag* link makes it, then rename it	LogixPro O:2/10 · Run lamp
`Label_Req`	BOOL	Edit Tags: type the name, pick the type	Latched while the labeler is asked to apply; cleared at PadIn

RUNG 0




```
[XIC(Start),XIC(Line_Run)]XIC(Stop)XIO(Batch_Done)OTE(Line_Run);
```

PREDICT IT · 1 `Start` is on, `Stop` is off and `Batch_Done` is off. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When either (`Start` (Start push button, NO) is on) *or* (`Line_Run` (Seal-in run bit (LogixPro B3:0/0)) is on), `Stop` (Stop push button, NC (1 until pressed)) is on and `Batch_Done` (The large-bottle count has reached the thumbwheel preset) is off, the rung turns `Line_Run` on (and off again whenever the rung is false).

Left to right:

1. A branch of 2 parallel paths: power gets through if any one of them is true.
   - **Path 1**
     1. `XIC(Start)` *Examine If Closed* asks: true when `Start` (Start push button, NO) is on.
   - **Path 2**
     1. `XIC(Line_Run)` *Examine If Closed* asks: true when `Line_Run` (Seal-in run bit (LogixPro B3:0/0)) is on.
2. `XIC(Stop)` *Examine If Closed* asks: true when `Stop` (Stop push button, NC (1 until pressed)) is on.
3. `XIO(Batch_Done)` *Examine If Open* asks: true when `Batch_Done` (The large-bottle count has reached the thumbwheel preset) is off.
4. `OTE(Line_Run)` *Output Energize* does: turns `Line_Run` on (and off again whenever the rung is false). Every scan it writes the rung state: true on, false off.

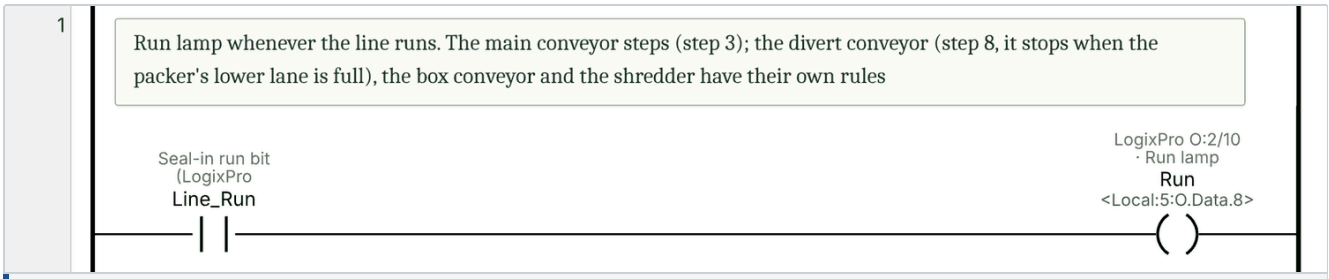
Why it is built this way:

Seal-in. The start condition turns `Line_Run` on, and `Line_Run` 's own contact in the parallel path keeps the rung true after the start goes away. Anything in series outside the branch (a stop, a fault) breaks it, and then it stays off until the start is made again.

> **Think of it like this:** a doorbell button and a hand. The start button is the doorbell, pressed for a moment. The seal-in contact is a hand that grabs the circuit as soon as the output comes on and keeps holding after you let go. The stop button, in series, is the only thing that can knock that hand away.

> **Watch for:** The stop has to be in series outside the branch. Put it inside the start path only and the seal-in path bypasses it: the stop no longer stops.

RUNG 1




```
XIC(Line_Run)OTE(Run);
```

HOW THIS RUNG WORKS

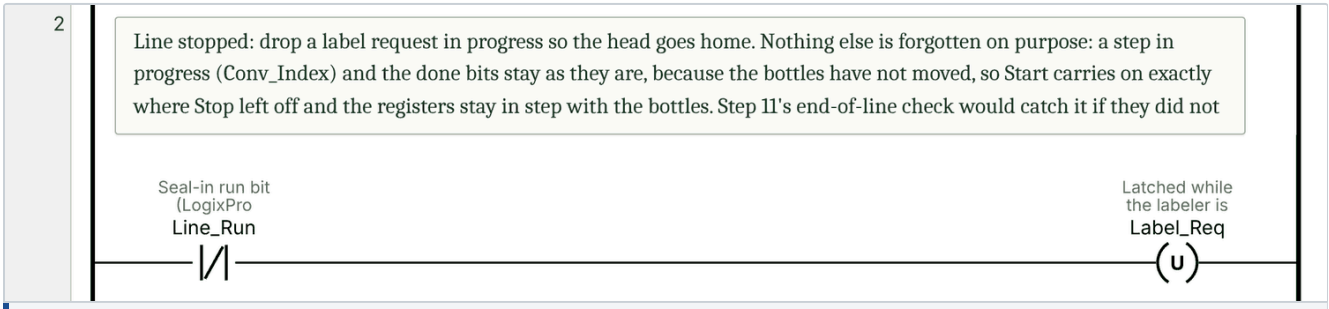
In words. When `Line_Run` (Seal-in run bit (LogixPro B3:0/0)) is on, the rung turns `Run` (Run lamp) on (and off again whenever the rung is false).

Left to right:

1. `XIC(Line_Run)` *Examine If Closed* asks: true when `Line_Run` (Seal-in run bit (LogixPro B3:0/0)) is on.
2. `OTE(Run)` *Output Energize* does: turns `Run` (Run lamp) on (and off again whenever the rung is false).

> **Think of it like this:** a rung is a pipe with valves in it, read left to right. Every contact is a valve: XIC is open while its bit is 1, XIO while its bit is 0. Water reaches the output at the right only if every valve in a row is open. The output does not remember anything: on every scan it is simply wet or dry.

RUNG 2




```
XIO(Line_Run)OTU(Label_Req);
```

HOW THIS RUNG WORKS

In words. When `Line_Run` (Seal-in run bit (LogixPro B3:0/0)) is off, the rung unlatches `Label_Req` (Latched while the labeler is asked to apply; cleared at PadIn) (turns it off).

Left to right:

1. `XIO(Line_Run)` *Examine If Open* asks: true when `Line_Run` (Seal-in run bit (LogixPro B3:0/0)) is off.
2. `OTU(Label_Req)` *Output Unlatch* does: unlatches `Label_Req` (Latched while the labeler is asked to apply; cleared at PadIn) (turns it off). It only ever clears the bit.

Rung 0 is chapter 4's seal-in with two extra conditions: `XIC(Stop)` rather than XIO because the stop button is normally closed — pressing it makes the tag 0 — and `XIO(Batch_Done)`, a bit step 9 will set when the batch is complete. Until then it is 0 and the XIO passes. **Rung 1** drives the run lamp whenever the line runs; the main conveyor is deliberately not here, because it steps (step 3), and the divert conveyor, the box conveyor and the shredder have rules of their own (step 8). **Rung 2** is housekeeping: when the line stops for any reason, drop a label request that is in progress so the labeler's head goes home (step 10 explains the request; make `Label_Req` now, a BOOL, and Verify). Notice what this rung does *not* do. A stopped line keeps the step in progress, the dwell's time and every "done" bit, because the bottles have not moved: Start must carry on exactly where Stop left off, or the shift registers of step 4 and the bottles they describe drift apart. Step 11 has the check that proves it.

Run it

Run mode. Press Start on the panel: the Run lamp lights and `Line_Run` is 1 in Monitor Tags. Press Stop: everything drops. Press Start again and hold Stop while pressing Start: nothing, because the stop contact is open. That is the whole of step 1.

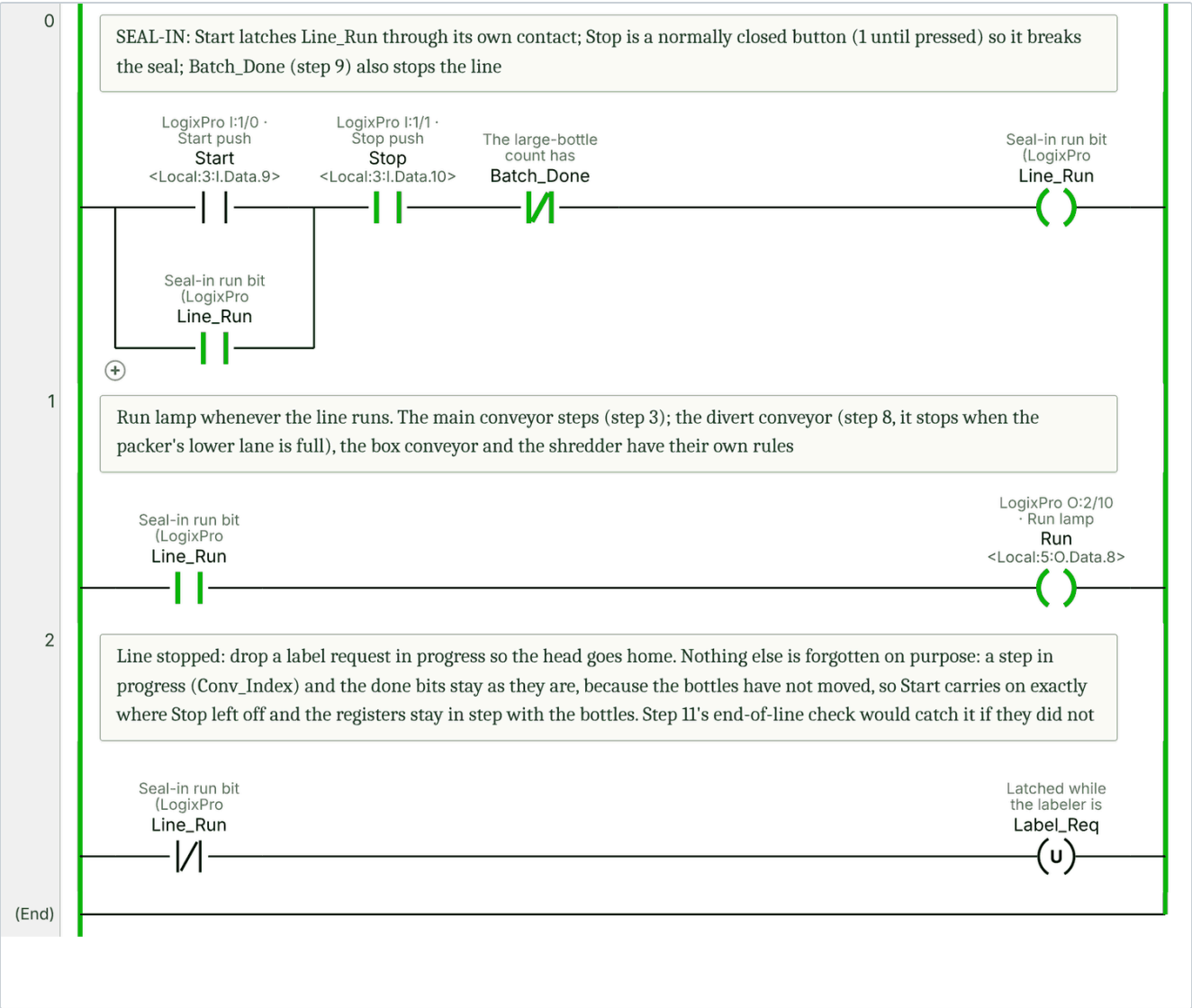


Figure 122. S1_Run online, with the finished line running: this is what your routine should look like once every step is in. Green is power flow.

> **TRY IT**
>
> Give the Run lamp a flash while the line is stopped but the controller is in Run mode: a TON that resets itself (chapter 7) and `XIO(Line_Run)XIC(Flash.TT)OTE(Run)` on a branch. Then take it out again — the lessons assume the lamp means running.

Predict it: the answers

1. The rung is false at `XIC(Stop)`: `Stop` is off, so `Line_Run` is **off** (an OTE writes 0 when its rung is false).

19 Lesson 4 — Step 2: the dwell timer and the feeder

The line indexes: belt stops, sensors read, bottle dropped, belt steps. The dwell is the stop. Add `JSR(S2_Dwell);` to MainRoutine under step 1 and write the rung:

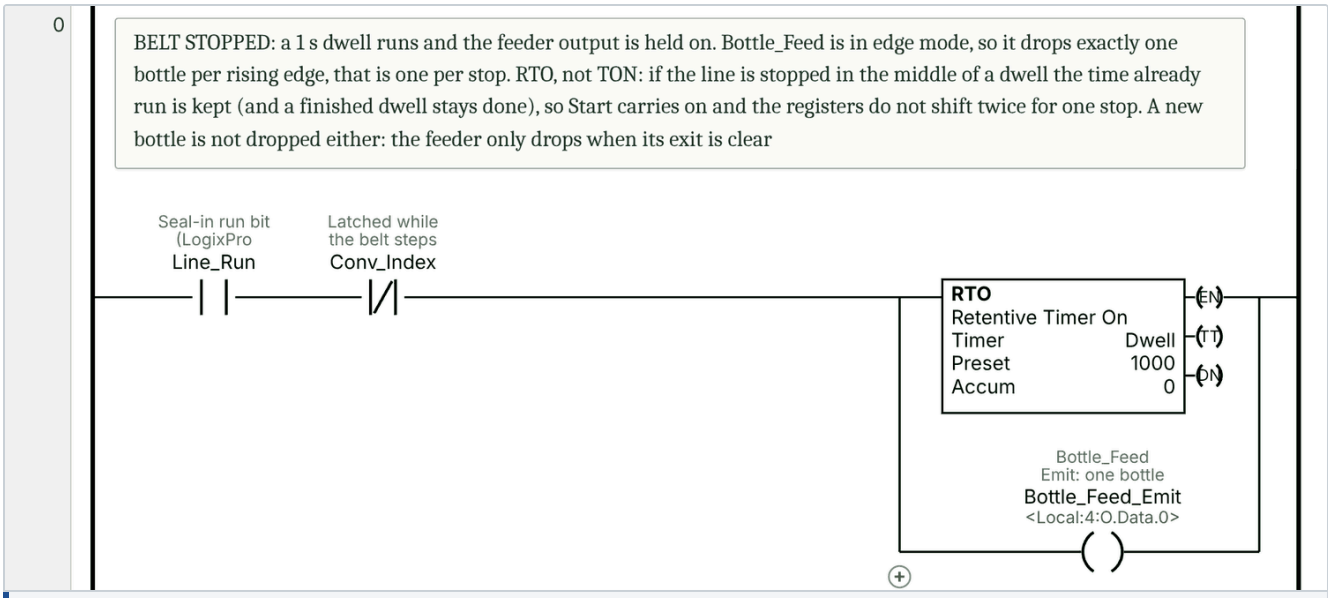
> **IN THE APP**
>
> Guided mode lesson 4 is this chapter: it checks that this lesson's tags are made, that `S2_Dwell` exists, is called from MainRoutine and verifies, and then watches the line while you run it and ticks each *Run it* result below as it happens. Help → Guided Mode, or start from File → New Project… → Guided: build the bottle line.

Tags this lesson needs

Create these before you type the rungs (Verify reports any you miss as unknown). Tags from earlier lessons are not repeated.

Tag	Type	How to make it	Description
`Conv_Index`	BOOL	Edit Tags: type the name, pick the type	Latched while the belt steps one pitch
`Dwell`	TIMER	Edit Tags: type the name, pick the type	Belt stopped: sensors read, feeder drops a bottle
`Bottle_Feed_Emit`	BOOL	alias for `Local:4:O.Data.0` — the Properties drawer's *tag* link makes it, then rename it	Bottle_Feed Emit: one bottle per rising edge

RUNG 0




```
XIC(Line_Run)XIO(Conv_Index)[RTO(Dwell,1000,0),OTE(Bottle_Feed_Emit)];
```

PREDICT IT · 1 `Line_Run` is off and `Conv_Index` is off. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Line_Run` (Seal-in run bit (LogixPro B3:0/0)) is on and `Conv_Index` (Latched while the belt steps one pitch) is off, the rung runs the retentive timer `Dwell` (Belt stopped: sensors read, feeder drops a bottle) toward 1000 ms (1 s) and turns `Bottle_Feed_Emit` (One bottle per rising edge) on (and off again whenever the rung is false).

Left to right:

1. `XIC(Line_Run)` *Examine If Closed* asks: true when `Line_Run` (Seal-in run bit (LogixPro B3:0/0)) is on.
2. `XIO(Conv_Index)` *Examine If Open* asks: true when `Conv_Index` (Latched while the belt steps one pitch) is off.
3. A branch of 2 parallel paths: each path is tested on its own and does its own work.

   Path 1

   1. `RTO(Dwell,1000,0)` *Retentive Timer On* does: runs the retentive timer `Dwell` (Belt stopped: sensors read, feeder drops a bottle) toward 1000 ms (1 s). Like TON but it keeps its time when the rung goes false; only a RES clears it.

   Path 2

   1. `OTE(Bottle_Feed_Emit)` *Output Energize* does: turns `Bottle_Feed_Emit` (One bottle per rising edge) on (and off again whenever the rung is false). Every scan it writes the rung state: true on, false off.

Why it is built this way:

A timer. `Dwell` only times while this rung is true; other rungs use its `.DN` (done) or `.TT` (timing) bits.

> **Think of it like this:** a kitchen timer that only runs while you hold your finger on it. Take your finger off and it snaps back to zero. Hold it long enough and the done light comes on, and stays on as long as your finger is still there.

While the line runs and the belt is not stepping (`Conv_Index`, which step 3 will latch while the belt moves), the `Dwell` timer counts a second, and on the same rung the feeder's `Emit` is energised; the feeder is in edge mode, so one bottle appears at the head of the line on the rising edge, once per stop. When the dwell is done, step 3 steps the belt and resets the timer for the next stop.

Why an **RTO** and not a TON (chapter 7)? A TON resets the moment its rung goes false, and the rung goes false whenever the line stops. Press Stop half-way through a dwell, Start again, and a TON would time the whole second again; press Stop after the dwell was done and the registers (step 4, which shift when `Dwell.DN` comes on) would shift a *second* time for the same stop. The retentive timer keeps what it has: a stopped dwell resumes where it was, and a finished one stays done. A retentive timer only clears with `RES`, and that rung belongs to step 3, which moves the belt: until the belt can step there is nothing to reset the dwell for. The feeder does not drop a second bottle on Start either: an edge-mode feeder only drops when its exit is clear, and the bottle it dropped is still standing there.

Run it

Start the line. Exactly one bottle drops at the head of the belt and sits there, because nothing steps the belt yet; `Dwell.DN` comes on after a second and stays on. Watch `Dwell.ACC` in Monitor Tags count to 1000 and stop. That is correct for now: step 3 is next.

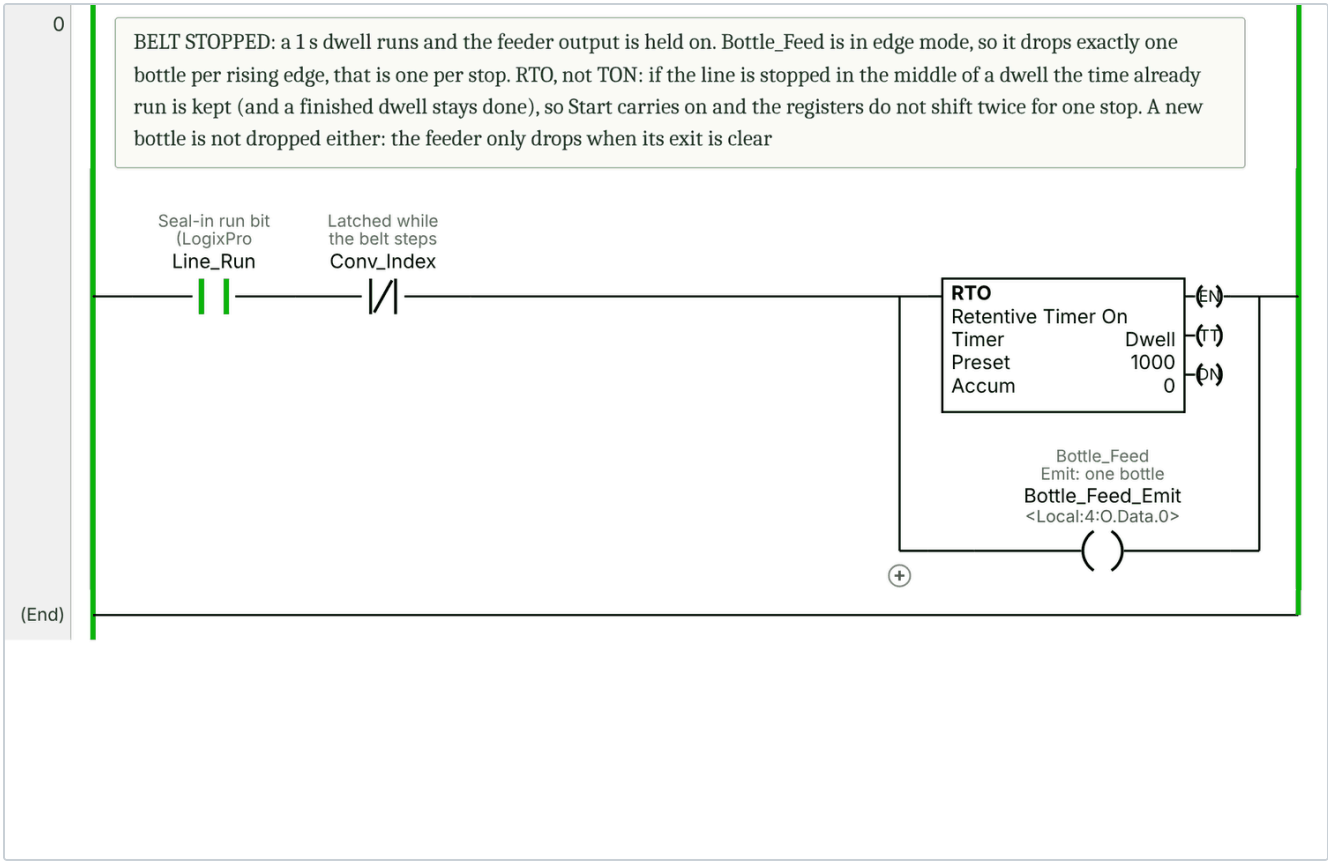


Figure 123. S2_Dwell online, with the finished line running: this is what your routine should look like once every step is in. Green is power flow.

> **TRY IT**
>
> Change the preset to 3000, Start, and press Stop when the accumulator reads about 1500: it holds 1500. Start again and it carries on to 3000. Set it back to 1000. In the finished line the dwell is the one number that sets the line's speed — the belt steps in about 0.7 s, so the line runs a bottle every 1.7 s.

Predict it: the answers

1. The rung is false at `XIC(Line_Run)`: `Line_Run` is off, so `Bottle_Feed_Emit` is **off** (an OTE writes 0 when its rung is false).

20 Lesson 5 — Step 3: index one pitch

This is the step that makes it a line, and it comes third because every station after it needs bottles to arrive. The belt must move exactly one pitch and stop: the encoder on `Main_Conv` gives one rising edge of channel A per 40 px, so the rule is *latch Conv_Index, run the belt, unlatch on the next edge*. Add `JSR(S3_Index);` to MainRoutine under step 2. **From now on it stays the last JSR**: each station you add in the lessons that follow goes above it (the guide's *Add it* puts it there, and says so if one ends up below). Write:

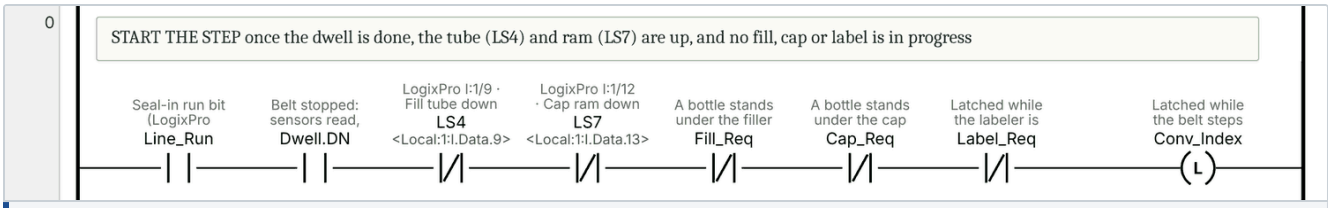
> **IN THE APP**
>
> Guided mode lesson 5 is this chapter: it checks that this lesson's tags are made, that `S3_Index` exists, is called from MainRoutine and verifies, and then watches the line while you run it and ticks each *Run it* result below as it happens. Help → Guided Mode, or start from File → New Project… → Guided: build the bottle line.

Tags this lesson needs

Create these before you type the rungs (Verify reports any you miss as unknown). Tags from earlier lessons are not repeated.

Tag	Type	How to make it	Description
`LS4`	BOOL	alias for `Local:1:I.Data.9` — the Properties drawer's *tag* link makes it, then rename it	LogixPro I:1/9 · Fill tube down
`LS7`	BOOL	alias for `Local:1:I.Data.13` — the Properties drawer's *tag* link makes it, then rename it	LogixPro I:1/12 · Cap ram down
`Fill_Req`	BOOL	Edit Tags: type the name, pick the type	A bottle stands under the filler and still needs its charge
`Cap_Req`	BOOL	Edit Tags: type the name, pick the type	A bottle stands under the cap ram and has not been capped
`Main_Conveyor`	BOOL	alias for `Local:4:O.Data.1` — the Properties drawer's *tag* link makes it, then rename it	LogixPro O:2/0 · Main conveyor motor
`Main_Conv_A`	BOOL	alias for `Local:1:I.Data.2` — the Properties drawer's *tag* link makes it, then rename it	Main_Conv encoder channel A: one rising edge per 40 px pitch
`A_ONS`	BOOL	Edit Tags: type the name, pick the type	One-shot storage for the encoder edge

RUNG 0




```
XIC(Line_Run)XIC(Dwell.DN)XIO(LS4)XIO(LS7)XIO(Fill_Req)XIO(Cap_Req)XIO(Label_Req)OTL(Conv_Index);
```

PREDICT IT · 1 `Line_Run` is off, `Dwell.DN` (the done bit of `Dwell`: Belt stopped: sensors read, feeder drops a bottle) is on, `LS4` is off, `LS7` is off, `Fill_Req` is off, `Cap_Req` is off and `Label_Req` is off. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Line_Run` (Seal-in run bit (LogixPro B3:0/0)) is on, `Dwell.DN` (the done bit of `Dwell`: Belt stopped: sensors read, feeder drops a bottle) is on, `LS4` (Fill tube down) is off, `LS7` (Cap ram down) is off, `Fill_Req` (A bottle stands under the filler and still needs its charge) is off, `Cap_Req` (A bottle stands under the cap ram and has not been capped) is off and `Label_Req` (Latched while the labeler is asked to apply; cleared at PadIn) is off, the rung latches `Conv_Index` (Latched while the belt steps one pitch) on.

Left to right:

1. `XIC(Line_Run)` *Examine If Closed* asks: true when `Line_Run` (Seal-in run bit (LogixPro B3:0/0)) is on.
2. `XIC(Dwell.DN)` *Examine If Closed* asks: true when `Dwell.DN` (the done bit of `Dwell`: Belt stopped: sensors read, feeder drops a bottle) is on.
3. `XIO(LS4)` *Examine If Open* asks: true when `LS4` (Fill tube down) is off.
4. `XIO(LS7)` *Examine If Open* asks: true when `LS7` (Cap ram down) is off.
5. `XIO(Fill_Req)` *Examine If Open* asks: true when `Fill_Req` (A bottle stands under the filler and still needs its charge) is off.
6. `XIO(Cap_Req)` *Examine If Open* asks: true when `Cap_Req` (A bottle stands under the cap ram and has not been capped) is off.
7. `XIO(Label_Req)` *Examine If Open* asks: true when `Label_Req` (Latched while the labeler is asked to apply; cleared at PadIn) is off.
8. `OTL(Conv_Index)` *Output Latch* does: latches `Conv_Index` (Latched while the belt steps one pitch) on. It only ever sets the bit; the bit stays on after the rung goes false.

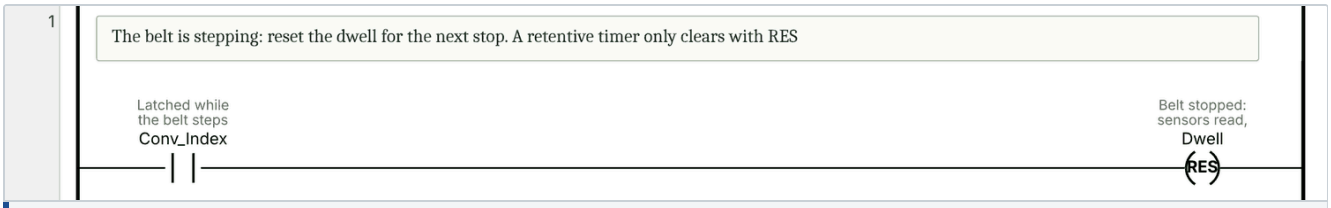
Why it is built this way:

A latch. `Conv_Index` stays on after this rung goes false; an `OTU` elsewhere in the program clears it.

> **Think of it like this:** a light switch instead of a doorbell. OTL flips it on and it stays on when you take your hand away; OTU flips it off. Whatever flipped it on does not have to stay there.

> **Watch for:** A latched bit survives the condition that set it: every way in needs a way out, or it stays on for good.

RUNG 1




```
XIC(Conv_Index)RES(Dwell);
```

HOW THIS RUNG WORKS

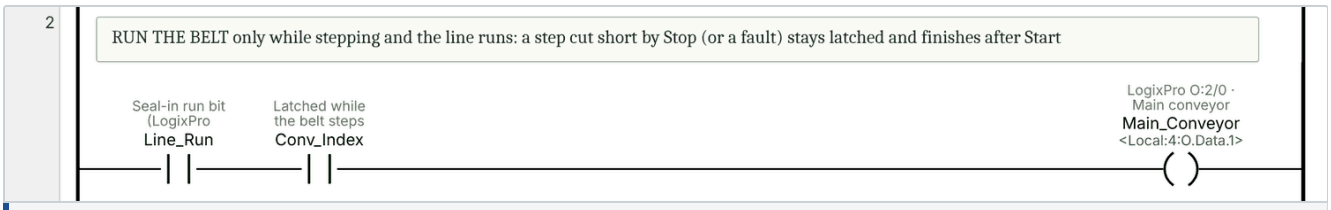
In words. When `Conv_Index` (Latched while the belt steps one pitch) is on, the rung resets `Dwell` (Belt stopped: sensors read, feeder drops a bottle).

Left to right:

1. `XIC(Conv_Index)` *Examine If Closed* asks: true when `Conv_Index` (Latched while the belt steps one pitch) is on.
2. `RES(Dwell)` *Reset* does: resets `Dwell` (Belt stopped: sensors read, feeder drops a bottle). It zeroes a timer or counter (accumulator and status bits), or a control (its position and status bits).

> **Think of it like this:** a rung is a pipe with valves in it, read left to right. Every contact is a valve: XIC is open while its bit is 1, XIO while its bit is 0. Water reaches the output at the right only if every valve in a row is open. The output does not remember anything: on every scan it is simply wet or dry.

RUNG 2




```
XIC(Line_Run)XIC(Conv_Index)OTE(Main_Conveyor);
```

PREDICT IT · 2 `Line_Run` is on and `Conv_Index` is off. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

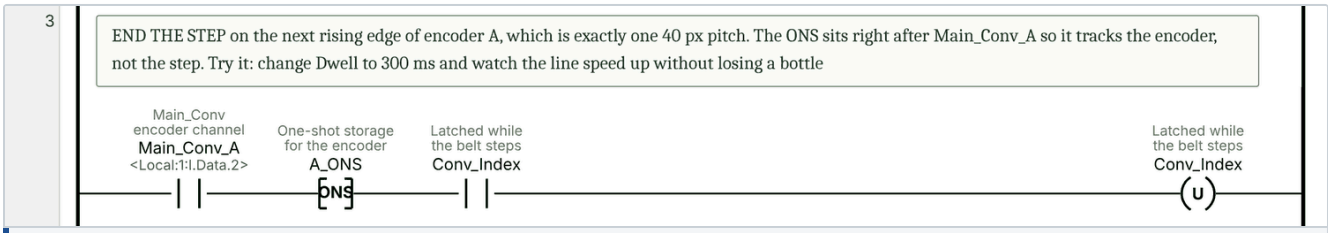
HOW THIS RUNG WORKS

In words. When `Line_Run` (Seal-in run bit (LogixPro B3:0/0)) is on and `Conv_Index` (Latched while the belt steps one pitch) is on, the rung turns `Main_Conveyor` (Main conveyor motor) on (and off again whenever the rung is false).

Left to right:

1. `XIC(Line_Run)` *Examine If Closed* asks: true when `Line_Run` (Seal-in run bit (LogixPro B3:0/0)) is on.
2. `XIC(Conv_Index)` *Examine If Closed* asks: true when `Conv_Index` (Latched while the belt steps one pitch) is on.
3. `OTE(Main_Conveyor)` *Output Energize* does: turns `Main_Conveyor` (Main conveyor motor) on (and off again whenever the rung is false). Every scan it writes the rung state: true on, false off.

RUNG 3




```
XIC(Main_Conv_A)ONS(A_ONS)XIC(Conv_Index)OTU(Conv_Index);
```

HOW THIS RUNG WORKS

In words. When `Main_Conv_A` (One rising edge per 40 px pitch) is on, it is the first scan they are true (the one-shot, remembered in `A_ONS` (One-shot storage for the encoder edge)) and `Conv_Index` (Latched while the belt steps one pitch) is on, the rung unlatches `Conv_Index` (turns it off).

Left to right:

1. `XIC(Main_Conv_A)` *Examine If Closed* asks: true when `Main_Conv_A` (One rising edge per 40 px pitch) is on.
2. `ONS(A_ONS)` *One Shot* asks: true when it is the first scan they are true (the one-shot, remembered in `A_ONS` (One-shot storage for the encoder edge)). It passes power for one scan only, the scan its input goes from false to true; its storage bit remembers last scan.
3. `XIC(Conv_Index)` *Examine If Closed* asks: true when `Conv_Index` (Latched while the belt steps one pitch) is on.
4. `OTU(Conv_Index)` *Output Unlatch* does: unlatches `Conv_Index` (turns it off). It only ever clears the bit.

Why it is built this way:

One-shot. Everything after the ONS happens once per event, not on every scan the condition holds; `A_ONS` is only its memory of the last scan.

> **Think of it like this:** a doorbell that rings once however long you lean on the button. The ONS looks at what the button was doing last scan: only the change from up to down rings it.

> **Watch for:** Each ONS needs its own storage bit. Two ONS sharing one bit make one of them miss its edges.

Rung 0 starts a step: the line runs, the dwell is done, no fill, cap or label is in progress (the request bits are off), and neither the tube (`LS4`) nor the chuck (`LS7`) is down. Those stations do not exist yet, so their bits are 0 and the rung passes; it is written complete now so it never has to be revisited. **Rung 1** resets step 2's dwell while the belt steps. **Rung 2** runs the motor only while stepping and only while the line runs. **Rung 3** ends the step: the next rising edge of encoder A — the ONS sits right after `Main_Conv_A` so it tracks the encoder, not the step — unlatches `Conv_Index`. A Stop in the middle of a step leaves `Conv_Index` latched (step 1 no longer touches it) and stops the motor; Start runs the motor again and the step ends on the same encoder edge it would have, one pitch from where it began. That is what keeps the registers and the bottles together.

Why `JSR(S3_Index)` is the last JSR. Rung 0 looks at the fill, cap and label requests, and those are decided in their own routines, in the same scan, from `Dwell.DN`. If S3_Index ran first it would latch the step in the scan the dwell finished, before the filler had asked to hold the belt, and the bottle would leave

the filler empty. Scanned last, it sees every request first. The rungs inside each routine are written top to bottom and each works the moment it is written; MainRoutine is the one place where the order is about the scan, not the writing.

Run it

Start the line. The belt steps, stops, a bottle drops, a second passes, the belt steps. Bottles walk down the line one pitch at a time and are removed at the end. Nothing fills, caps or sorts them yet: the lessons that follow add one station at a time, and each one works as soon as it is written because the bottles are already arriving.

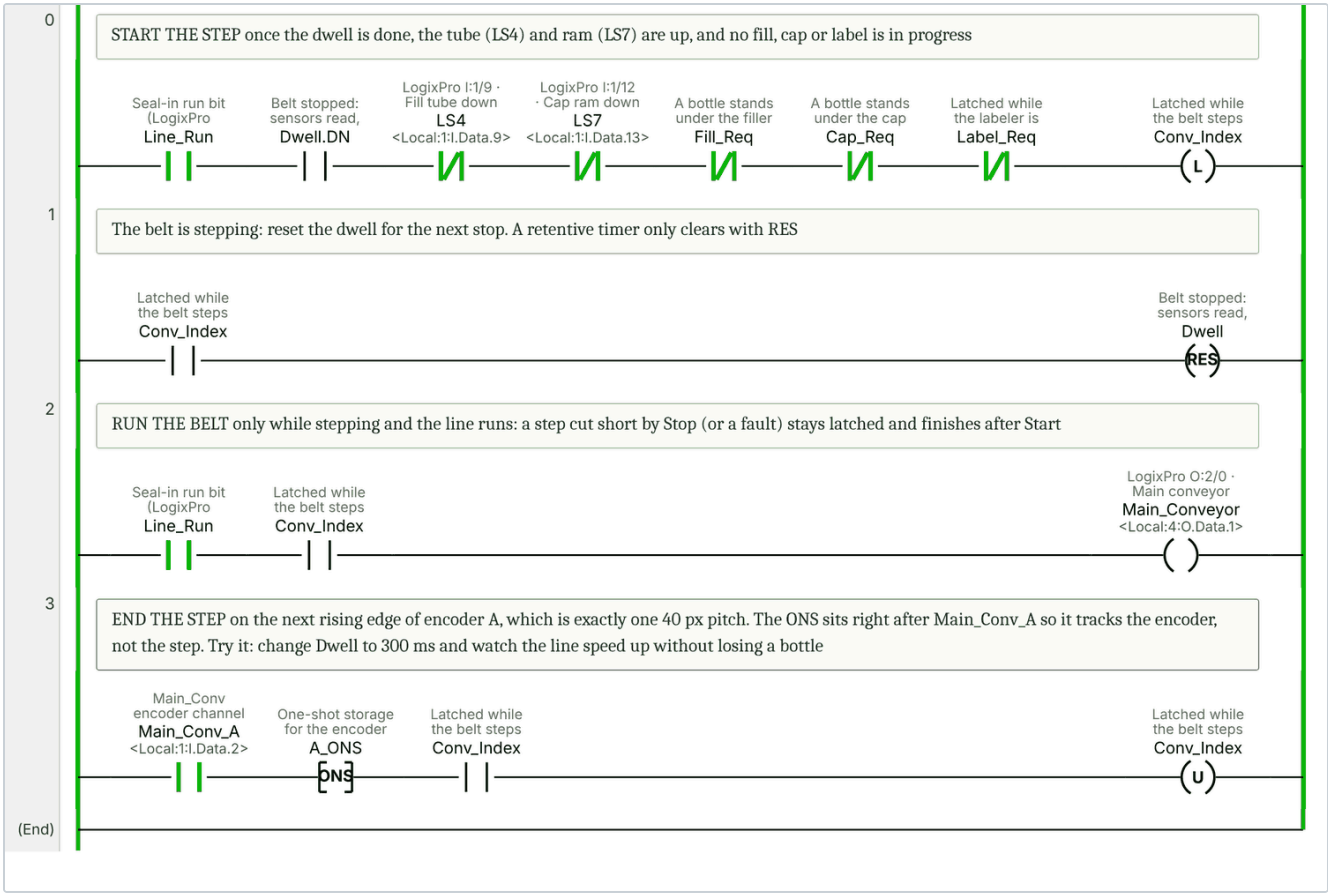


Figure 124. S3_Index online, with the finished line running: this is what your routine should look like once every step is in. Green is power flow.

> **TRY IT**
>
> Change the Dwell preset to 300 ms and watch the line speed up without losing a bottle. Set it back to 1000.

Predict it: the answers

1. The rung is false at `XIC(Line_Run)`: `Line_Run` is off, so the OTL does nothing: `Conv_Index` keeps whatever it had.
2. The rung is false at `XIC(Conv_Index)`: `Conv_Index` is off, so `Main_Conveyor` is **off** (an OTE writes 0 when its rung is false).

21 Lesson 6 — Step 4: the shift registers

Chapter 10 is the theory; this is the rung. Add `JSR(S4_Registers);` to MainRoutine above `JSR(S3_Index)`, and in `S4_Registers`:

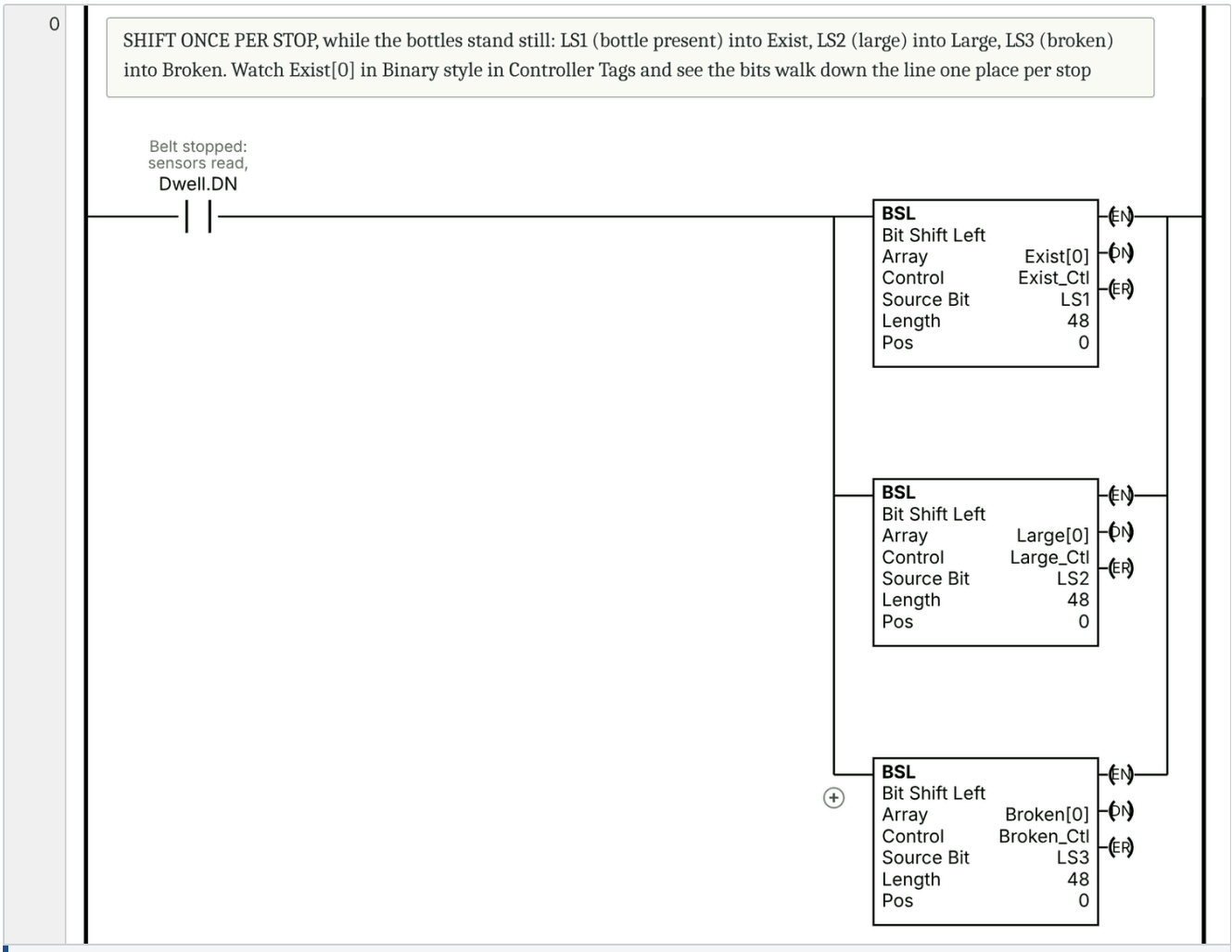
> **IN THE APP**
>
> Guided mode lesson 6 is this chapter: it checks that this lesson's tags are made, that `S4_Registers` exists, is called from MainRoutine and verifies, and then watches the line while you run it and ticks each *Run it* result below as it happens. Help → Guided Mode, or start from File → New Project… → Guided: build the bottle line.

Tags this lesson needs

Create these before you type the rungs (Verify reports any you miss as unknown). Tags from earlier lessons are not repeated.

Tag	Type	How to make it	Description
`Exist`	DINT[2]	Edit Tags: type the name, pick the type, Style Binary, Dim 2	Bottle present, one bit per stop (LogixPro B3:10). During the dwell: filler 6, capper 9, scrap gate 13, divert gate 20; after the shift (Dwell.DN) one bit higher
`Exist_Ctl`	CONTROL	Edit Tags: type the name, pick the type	LogixPro R6:1
`LS1`	BOOL	alias for `Local:1:I.Data.4` — the Properties drawer's *tag* link makes it, then rename it	LogixPro I:1/6 · Exist sensor: a bottle at the start of the line
`Large`	DINT[2]	Edit Tags: type the name, pick the type, Style Binary, Dim 2	Large bottle (LogixPro B3:20). During the dwell: filler 5, capper 8, scrap gate 12, divert gate 19; after the shift one bit higher
`Large_Ctl`	CONTROL	Edit Tags: type the name, pick the type	LogixPro R6:2
`LS2`	BOOL	alias for `Local:1:I.Data.5` — the Properties drawer's *tag* link makes it, then rename it	LogixPro I:1/7 · Size check: beam set so only a large bottle breaks it
`Broken`	DINT[2]	Edit Tags: type the name, pick the type, Style Binary, Dim 2	Broken bottle (LogixPro B3:30). During the dwell: filler 4, capper 7, scrap gate 11, divert gate 18; after the shift one bit higher
`Broken_Ctl`	CONTROL	Edit Tags: type the name, pick the type	LogixPro R6:3
`LS3`	BOOL	alias for `Local:1:I.Data.6` — the Properties drawer's *tag* link makes it, then rename it	LogixPro I:1/8 · Broken check: sees broken bottles only

RUNG 0




```
XIC(Dwell.DN)
[BSL(Exist[0],Exist_Ctl,LS1,48),BSL(Large[0],Large_Ctl,LS2,48),BSL(Broken[0],Br
oken_Ctl,LS3,48)];
```

HOW THIS RUNG WORKS

In words. When `Dwell.DN` (the done bit of `Dwell` : Belt stopped: sensors read, feeder drops a bottle) is on, the rung shifts `Exist[0]` one place left, bringing in `LS1` (Exist sensor: a bottle at the start of the line), shifts `Large[0]` one place left, bringing in `LS2` (Size check: beam set so only a large bottle breaks it) and shifts `Broken[0]` one place left, bringing in `LS3` (Broken check: sees broken bottles only).

Left to right:

1. `XIC(Dwell.DN)` *Examine If Closed* asks: true when `Dwell.DN` (the done bit of `Dwell` : Belt stopped: sensors read, feeder drops a bottle) is on.
2. A branch of 3 parallel paths: each path is tested on its own and does its own work.

   Path 1

   1. `BSL(Exist[0],Exist_Ctl,LS1,48)` *Bit Shift Left* does: shifts `Exist[0]` one place left, bringing in `LS1` (Exist sensor: a bottle at the start of the line). One shift per false-to-true transition of the rung.

Path 2

1. `BSL(Large[0],Large_Ctl,LS2,48)` *Bit Shift Left* does: shifts `Large[0]` one place left, bringing in `LS2` (Size check: beam set so only a large bottle breaks it).

Path 3

1. `BSL(Broken[0],Broken_Ctl,LS3,48)` *Bit Shift Left* does: shifts `Broken[0]` one place left, bringing in `LS3` (Broken check: sees broken bottles only).

Why it is built this way:

A shift register. Every bit of the word is a position on the conveyor; one shift per index moves every part's bit one position along, so a bit at a station says what is standing there.

> **Think of it like this:** a row of pigeonholes riding along with the conveyor. Put a note in the first hole when a part arrives; every index moves every note one hole along, so the note in hole five tells you about the part standing at station five.

> **Watch for:** The shift has to happen exactly once per index. Drive it from a one-shot or a transition, never from a level that stays true.

At the end of every dwell the three registers shift once, each taking its head sensor into bit 0. The three CONTROL tags (`Exist_Ctl`, `Large_Ctl`, `Broken_Ctl`) belong to the BSLs; 48 is the length, more than the 33 stops on the belt, so a bottle is tracked all the way to the end and off.

Run it

Start the line and open Monitor Tags with `Exist`, `Large` and `Broken` expanded (they are Binary). After the first dwell, bit 0 of Exist is 1 — a bottle is at the head. At every stop the registers shift and the 1 walks one bit, in step with the bottle on the belt: the shift happens once per dwell, and step 3 moves the belt exactly one pitch per dwell. A large bottle leaves a 1 in Large, a broken one in Broken, and those walk with it too.

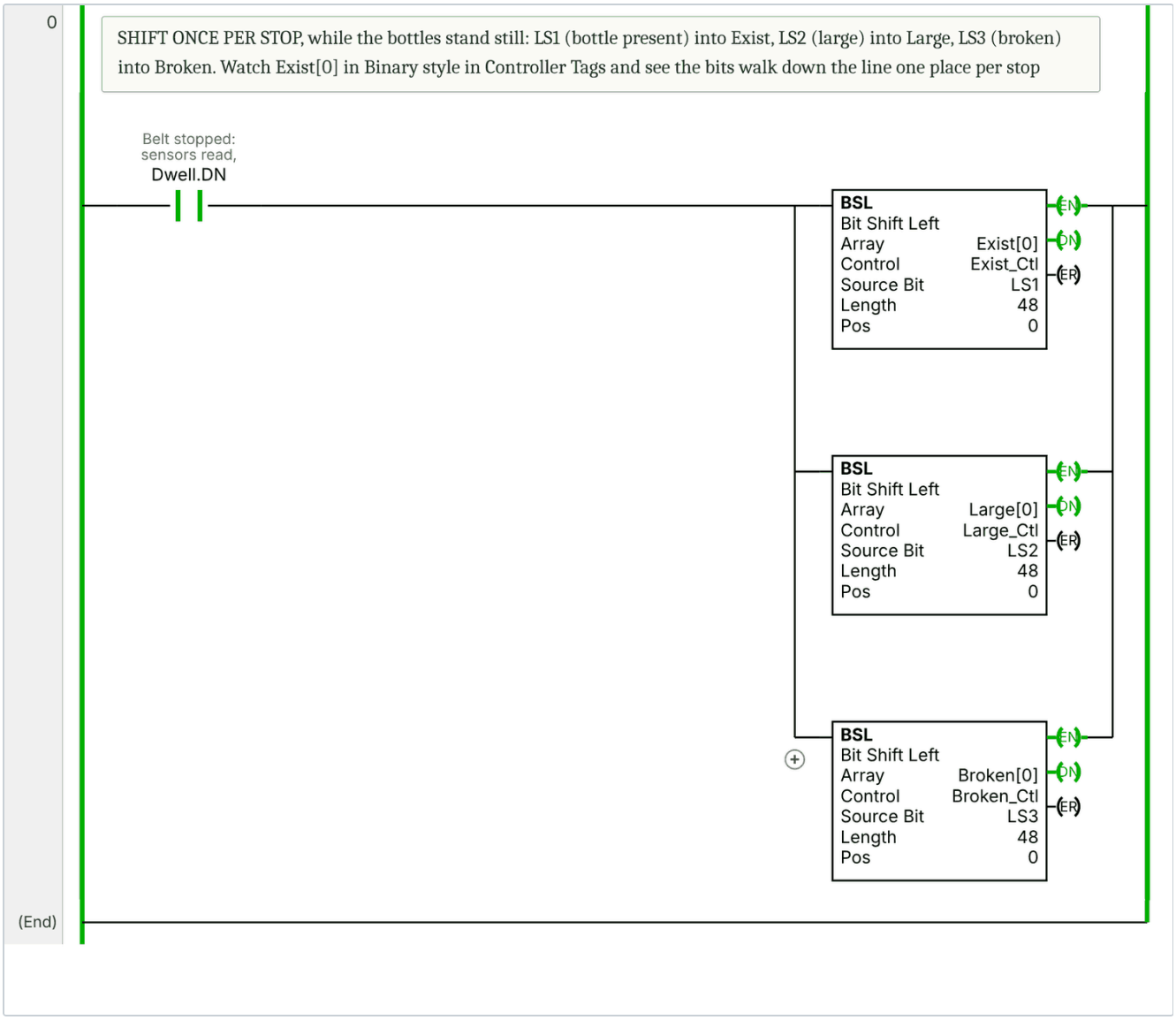


Figure 125. S4_Registers online, with the finished line running: this is what your routine should look like once every step is in. Green is power flow.

TRY IT

Stop the line, then toggle `Conv_Index` on and off by hand (Ctrl+T, twice): the dwell runs again, the registers shift, and every bit moves on although no bottle did. That is a register out of step with the world, and why only step 3 may end a dwell. Reset Sim afterwards. Then set Bottle_Feed's *Broken bottles* to 50 % for a run and watch the Broken register fill; then back to 15.

22 Lesson 7 — Step 5: filling

The fill station needs five rungs: a request, the tube, the valves, a done latch and a clear. The request is written in the *after the shift* frame — it waits on `Dwell.DN`, which means the registers have just shifted and the belt is about to step — so it reads Exist bit 7 and Large bit 6 (chapter 10's table). Add the JSR above `JSR(S3_Index)` and write:

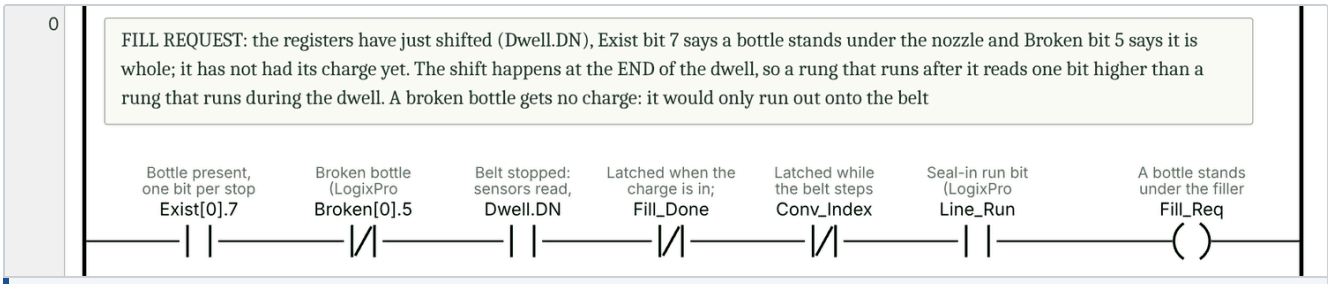
> **IN THE APP**
>
> Guided mode lesson 7 is this chapter: it checks that this lesson's tags are made, that `S5_Fill` exists, is called from MainRoutine and verifies, and then watches the line while you run it and ticks each *Run it* result below as it happens. Help → Guided Mode, or start from File → New Project… → Guided: build the bottle line.

Tags this lesson needs

Create these before you type the rungs (Verify reports any you miss as unknown). Tags from earlier lessons are not repeated.

Tag	Type	How to make it	Description
`Fill_Done`	BOOL	Edit Tags: type the name, pick the type	Latched when the charge is in; cleared when the belt steps
`Fill_Tube`	BOOL	alias for `Local:4:O.Data.6` — the Properties drawer's *tag* link makes it, then rename it	LogixPro O:2/6 · Lower the fill tube
`LS5`	BOOL	alias for `Local:1:I.Data.10` — the Properties drawer's *tag* link makes it, then rename it	LogixPro I:1/10 · Large charge delivered
`Large_Fill`	BOOL	alias for `Local:4:O.Data.7` — the Properties drawer's *tag* link makes it, then rename it	LogixPro O:2/7 · Large fill valve
`LS6`	BOOL	alias for `Local:1:I.Data.11` — the Properties drawer's *tag* link makes it, then rename it	LogixPro I:1/11 · Small charge delivered
`Small_Fill`	BOOL	alias for `Local:4:O.Data.8` — the Properties drawer's *tag* link makes it, then rename it	LogixPro O:2/8 · Small fill valve

RUNG 0




```
XIC(Exist[0].7)XIO(Broken[0].5)XIC(Dwell.DN)XIO(Fill_Done)XIO(Conv_Index)XIC(Line_Run)OTE(Fill_Req);
```

PREDICT IT · 1 `Exist[0].7` is off, `Broken[0].5` is off, `Dwell.DN` (the done bit of `Dwell`: Belt stopped: sensors read, feeder drops a bottle) is on, `Fill_Done` is off, `Conv_Index` is off and `Line_Run` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

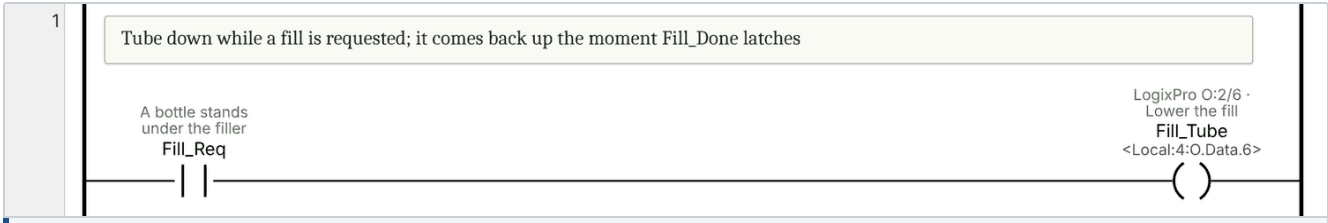
In words. When `Exist[0].7` is on, `Broken[0].5` is off, `Dwell.DN` (the done bit of `Dwell`: Belt stopped: sensors read, feeder drops a bottle) is on, `Fill_Done` (Latched when the charge is in; cleared when the belt steps) is off, `Conv_Index` (Latched while the belt steps one pitch) is off and `Line_Run` (Seal-in run bit (LogixPro B3:0/0)) is on, the rung turns `Fill_Req` (A bottle stands under the filler and still needs its charge) on (and off again whenever the rung is false).

Left to right:

1. `XIC(Exist[0].7)` *Examine If Closed* asks: true when `Exist[0].7` is on.
2. `XIO(Broken[0].5)` *Examine If Open* asks: true when `Broken[0].5` is off.
3. `XIC(Dwell.DN)` *Examine If Closed* asks: true when `Dwell.DN` (the done bit of `Dwell`: Belt stopped: sensors read, feeder drops a bottle) is on.
4. `XIO(Fill_Done)` *Examine If Open* asks: true when `Fill_Done` (Latched when the charge is in; cleared when the belt steps) is off.
5. `XIO(Conv_Index)` *Examine If Open* asks: true when `Conv_Index` (Latched while the belt steps one pitch) is off.
6. `XIC(Line_Run)` *Examine If Closed* asks: true when `Line_Run` (Seal-in run bit (LogixPro B3:0/0)) is on.
7. `OTE(Fill_Req)` *Output Energize* does: turns `Fill_Req` (A bottle stands under the filler and still needs its charge) on (and off again whenever the rung is false). Every scan it writes the rung state: true on, false off.

> **Think of it like this:** a rung is a pipe with valves in it, read left to right. Every contact is a valve: XIC is open while its bit is 1, XIO while its bit is 0. Water reaches the output at the right only if every valve in a row is open. The output does not remember anything: on every scan it is simply wet or dry.

RUNG 1




```
XIC(Fill_Req)OTE(Fill_Tube);
```

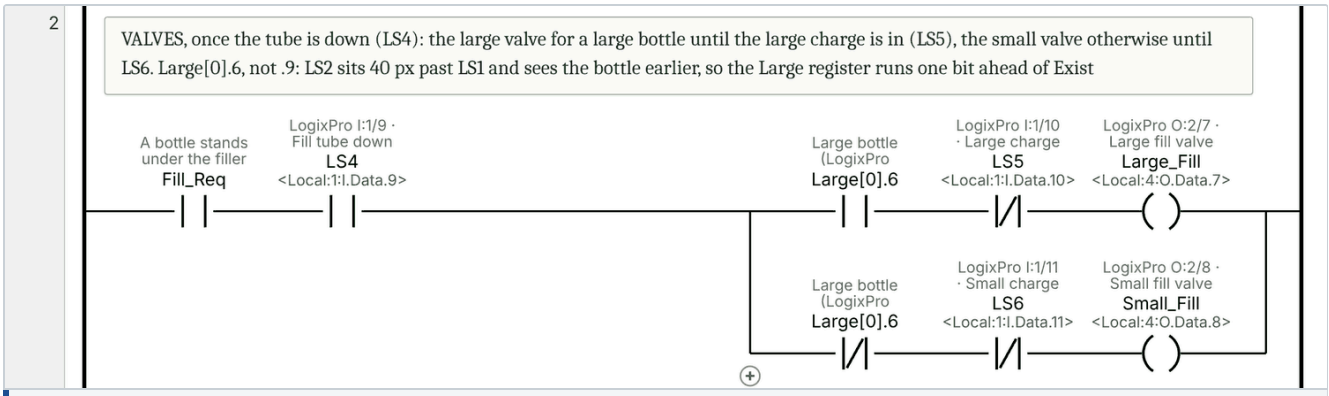
HOW THIS RUNG WORKS

In words. When `Fill_Req` (A bottle stands under the filler and still needs its charge) is on, the rung turns `Fill_Tube` (Lower the fill tube) on (and off again whenever the rung is false).

Left to right:

1. `XIC(Fill_Req)` *Examine If Closed* asks: true when `Fill_Req` (A bottle stands under the filler and still needs its charge) is on.
2. `OTE(Fill_Tube)` *Output Energize* does: turns `Fill_Tube` (Lower the fill tube) on (and off again whenever the rung is false).

RUNG 2




```
XIC(Fill_Req)XIC(LS4)
[XIC(Large[0].6)XIO(LS5)OTE(Large_Fill),XIO(Large[0].6)XIO(LS6)OTE(Small_Fill)]
;
```

PREDICT IT · 2 `Fill_Req` is on, `LS4` is off, `Large[0].6` is off and `LS6` is off. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Fill_Req` (A bottle stands under the filler and still needs its charge) is on and `LS4` (Fill tube down) is on, the rung picks its path: if `Large[0].6` is on and `LS5` (Large charge delivered) is off, it turns `Large_Fill` (Large fill valve) on (and off again whenever the rung is false); if `Large[0].6` is off and `LS6` (Small charge delivered) is off, it turns `Small_Fill` (Small fill valve) on (and off again whenever the rung is false).

Left to right:

1. `XIC(Fill_Req)` *Examine If Closed* asks: true when `Fill_Req` (A bottle stands under the filler and still needs its charge) is on.
2. `XIC(LS4)` *Examine If Closed* asks: true when `LS4` (Fill tube down) is on.
3. A branch of 2 parallel paths: each path is tested on its own and does its own work.

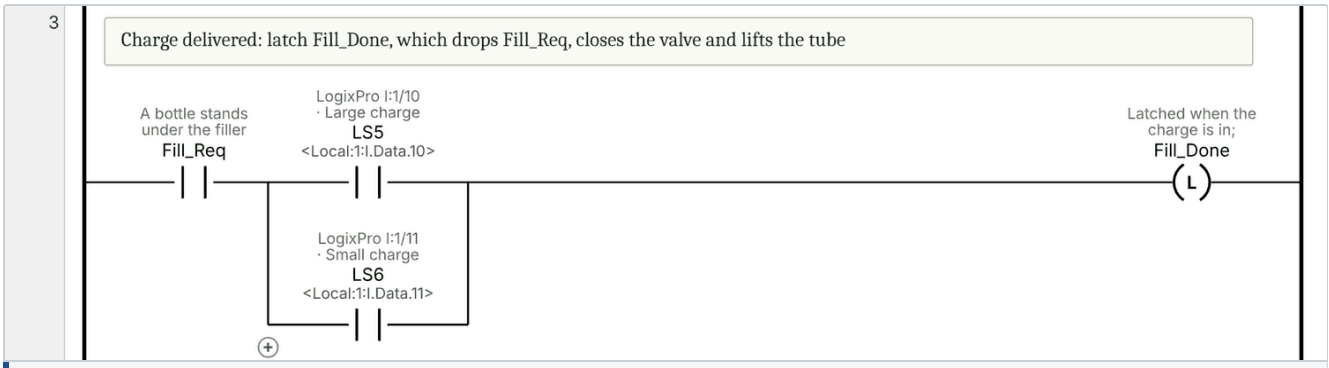
   Path 1

   1. `XIC(Large[0].6)` *Examine If Closed* asks: true when `Large[0].6` is on.
   2. `XIO(LS5)` *Examine If Open* asks: true when `LS5` (Large charge delivered) is off.
   3. `OTE(Large_Fill)` *Output Energize* does: turns `Large_Fill` (Large fill valve) on (and off again whenever the rung is false).

   Path 2

   1. `XIO(Large[0].6)` *Examine If Open* asks: true when `Large[0].6` is off.
   2. `XIO(LS6)` *Examine If Open* asks: true when `LS6` (Small charge delivered) is off.
   3. `OTE(Small_Fill)` *Output Energize* does: turns `Small_Fill` (Small fill valve) on (and off again whenever the rung is false).

RUNG 3




```
XIC(Fill_Req)[XIC(LS5),XIC(LS6)]OTL(Fill_Done);
```

PREDICT IT · 3 `Fill_Req` is on and `LS5` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Fill_Req` (A bottle stands under the filler and still needs its charge) is on and either (`LS5` (Large charge delivered) is on) *or* (`LS6` (Small charge delivered) is on), the rung latches `Fill_Done` (Latched when the charge is in; cleared when the belt steps) on.

Left to right:

1. `XIC(Fill_Req)` *Examine If Closed* asks: true when `Fill_Req` (A bottle stands under the filler and still needs its charge) is on.
2. A branch of 2 parallel paths: power gets through if any one of them is true.
   - **Path 1**
     1. `XIC(LS5)` *Examine If Closed* asks: true when `LS5` (Large charge delivered) is on.
   - **Path 2**
     1. `XIC(LS6)` *Examine If Closed* asks: true when `LS6` (Small charge delivered) is on.
3. `OTL(Fill_Done)` *Output Latch* does: latches `Fill_Done` (Latched when the charge is in; cleared when the belt steps) on. It only ever sets the bit; the bit stays on after the rung goes false.

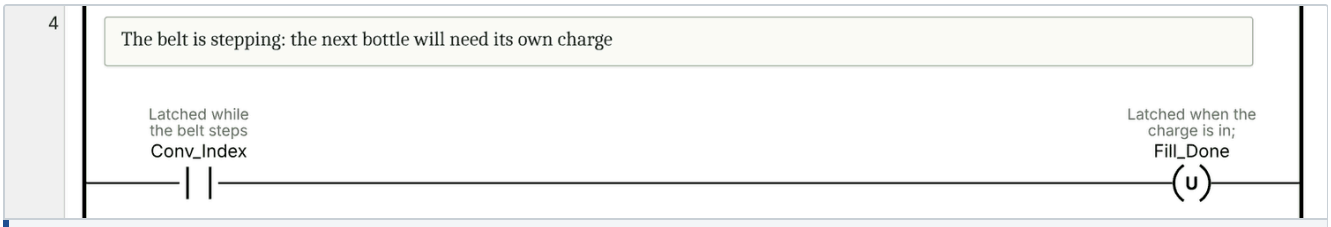
Why it is built this way:

A latch. `Fill_Done` stays on after this rung goes false; an `OTU` elsewhere in the program clears it.

> **Think of it like this:** a light switch instead of a doorbell. OTL flips it on and it stays on when you take your hand away; OTU flips it off. Whatever flipped it on does not have to stay there.

> **Watch for:** A latched bit survives the condition that set it: every way in needs a way out, or it stays on for good.

RUNG 4




```
XIC(Conv_Index)OTU(Fill_Done);
```

HOW THIS RUNG WORKS

In words. When `Conv_Index` (Latched while the belt steps one pitch) is on, the rung unlatches `Fill_Done` (Latched when the charge is in; cleared when the belt steps) (turns it off).

Left to right:

1. `XIC(Conv_Index)` *Examine If Closed* asks: true when `Conv_Index` (Latched while the belt steps one pitch) is on.
2. `OTU(Fill_Done)` *Output Unlatch* does: unlatches `Fill_Done` (Latched when the charge is in; cleared when the belt steps) (turns it off). It only ever clears the bit.

Rung 0: a bottle under the nozzle, not broken, not yet filled, the registers shifted, the belt not stepping, the line running → `Fill_Req`. The request is an OTE, so it drops the moment `Fill_Done` latches. **Rung 1** lowers the tube while requested; the filler reports `LS4` (tube down). **Rung 2** opens a valve once the tube is down: a branch with the large valve on one leg and the small valve on the other, chosen by the Large register, each held until the filler's own float (`LS5` large charge, `LS6` small) says the charge is in. One rung for both sizes means it works on whichever bottle reaches the filler first. **Rung 3** latches `Fill_Done` at that moment, which drops the request and lifts the tube, and **rung 4** unlatches it when the belt steps so the next bottle is treated as new. Step 3 will not let the belt step while the tube is down or the request is on.

Run it

Start the line. At stop 7 the tube comes down and the bottle fills — a large charge for a large bottle, a small one for a small, watch the head-block lamps in the Side view — then the tube lifts and the belt steps. The Simulator Log lists every fill. The filler tops its head tank up from the reservoir, and the reservoir is going down: step 6 keeps it supplied.

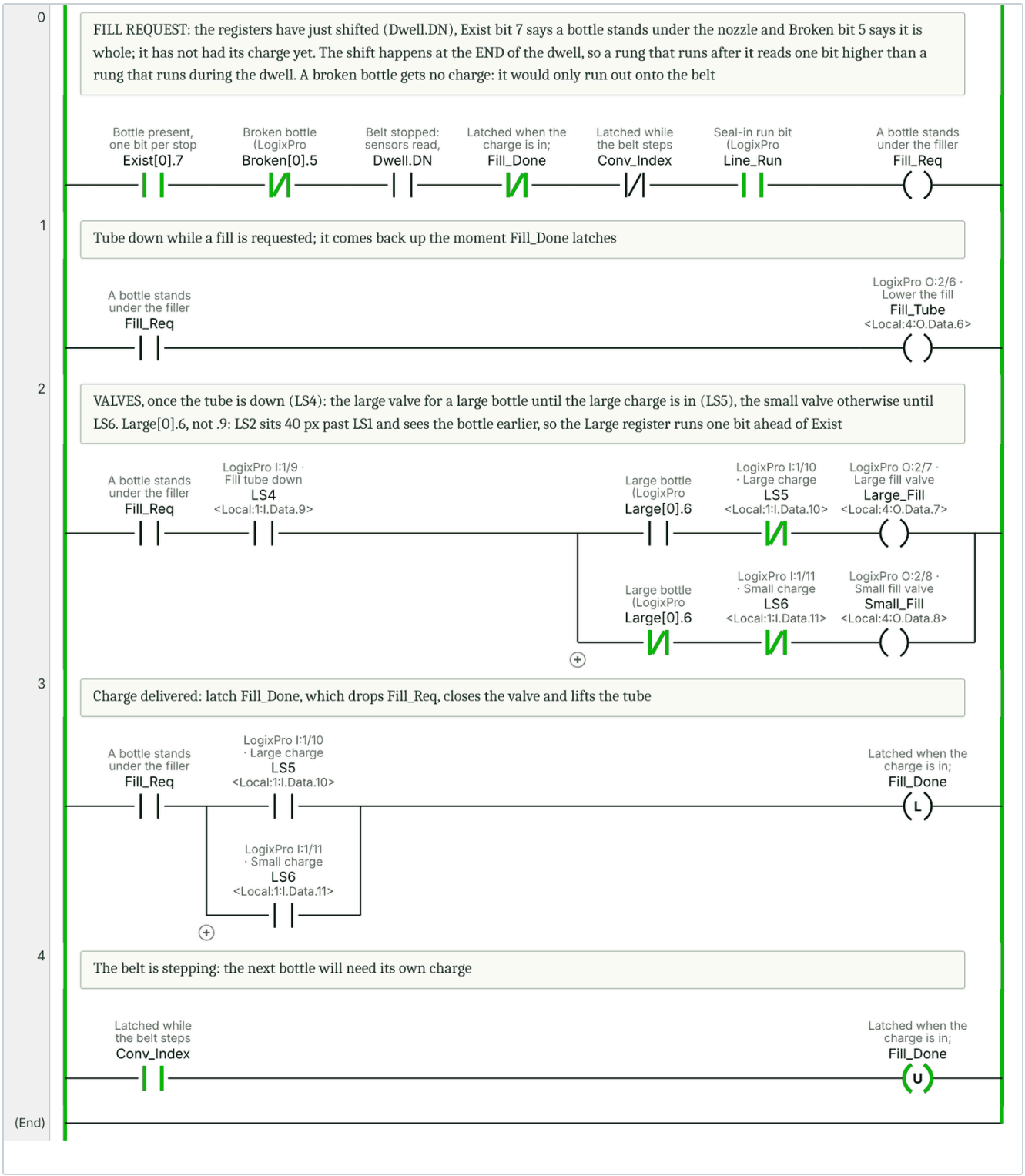


Figure 126. S5_Fill online, with the finished line running: this is what your routine should look like once every step is in. Green is power flow.

TRY IT

Why is the bottle Exist 7 but Large 6? Count the sensors: LS2 is one stop past LS1, so the same bottle is one bit further along in Exist than in Large. Then move `JSR(S3_Index)` above `JSR(S5_Fill)` in MainRoutine and run: the belt steps in the same scan the fill is asked for, and bottles leave the filler empty. Put it back. That is the scan-order lesson in one move.

Predict it: the answers

1. The rung is false at `XIC(Exist[0].7)`: `Exist[0].7` is off, so `Fill_Req` is **off** (an OTE writes 0 when its rung is false).
2. The rung is false at `XIC(LS4)`: `LS4` is off, so `Large_Fill` is **off** (an OTE writes 0 when its rung is false) and `Small_Fill` is **off** (an OTE writes 0 when its rung is false).
3. Every condition on the way is met, so `Fill_Done` latches **on**.

23 Lesson 8 — Step 6: the reservoir and the stack light

The filler draws every charge from its head tank, and the head tank tops itself up from the reservoir, `Supply_Tank`, which nothing refills yet: run step 7 long enough and the line stalls with the tube down, waiting for a charge that never comes. This step keeps the filler fed, so it comes straight after it. Add the JSR above `JSR(S3_Index)` and write:

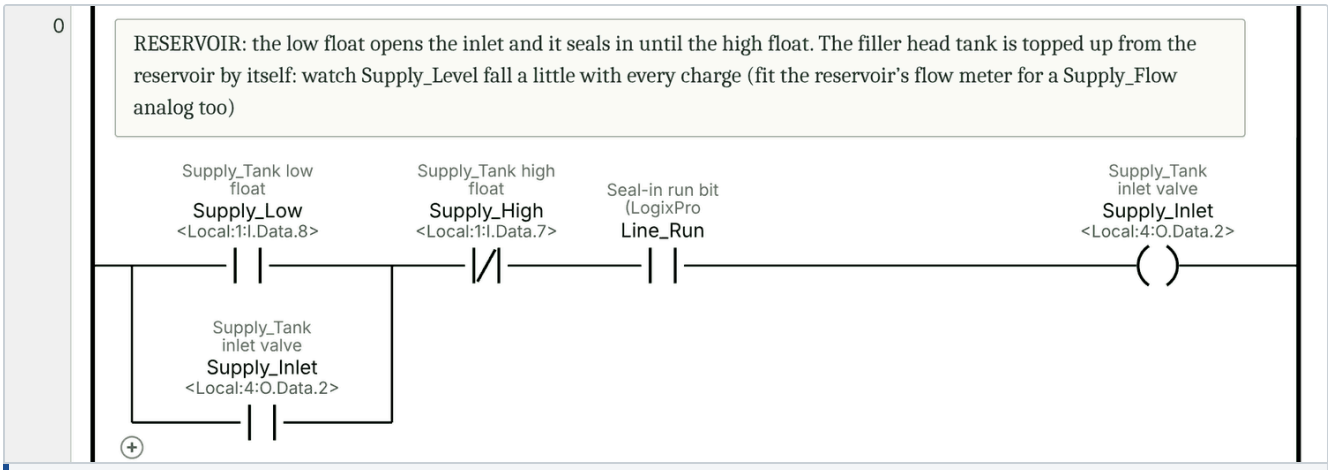
> **IN THE APP**
>
> Guided mode lesson 8 is this chapter: it checks that this lesson's tags are made, that `S6_Supply` exists, is called from MainRoutine and verifies, and then watches the line while you run it and ticks each *Run it* result below as it happens. Help → Guided Mode, or start from File → New Project… → Guided: build the bottle line.

Tags this lesson needs

Create these before you type the rungs (Verify reports any you miss as unknown). Tags from earlier lessons are not repeated.

Tag	Type	How to make it	Description
`Supply_Low`	BOOL	alias for `Local:1:I.Data.8` — the Properties drawer's *tag* link makes it, then rename it	Supply_Tank low float
`Supply_Inlet`	BOOL	alias for `Local:4:O.Data.2` — the Properties drawer's *tag* link makes it, then rename it	Supply_Tank inlet valve
`Supply_High`	BOOL	alias for `Local:1:I.Data.7` — the Properties drawer's *tag* link makes it, then rename it	Supply_Tank high float
`Tank_Low`	BOOL	alias for `Local:1:I.Data.12` — the Properties drawer's *tag* link makes it, then rename it	Filler head tank below 15 %
`Tank_Red`	BOOL	alias for `Local:4:O.Data.3` — the Properties drawer's *tag* link makes it, then rename it	Tank_Light red: reservoir at the low float (flashes)
`Tank_Amber`	BOOL	alias for `Local:4:O.Data.4` — the Properties drawer's *tag* link makes it, then rename it	Tank_Light amber: inlet open, refilling
`Tank_Green`	BOOL	alias for `Local:4:O.Data.5` — the Properties drawer's *tag* link makes it, then rename it	Tank_Light green: level healthy

RUNG 0




```
[XIC(Supply_Low),XIC(Supply_Inlet)]XIO(Supply_High)XIC(Line_Run)OTE(Supply_Inlet);
```

PREDICT IT · 1 `Supply_Low` is on, `Supply_High` is on and `Line_Run` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When either (`Supply_Low` (Supply_Tank low float) is on) *or* (`Supply_Inlet` (Supply_Tank inlet valve) is on), `Supply_High` (Supply_Tank high float) is off and `Line_Run` (Seal-in run bit (LogixPro B3:0/0)) is on, the rung turns `Supply_Inlet` on (and off again whenever the rung is false).

Left to right:

1. A branch of 2 parallel paths: power gets through if any one of them is true.
   - **Path 1**
     1. `XIC(Supply_Low)` *Examine If Closed* asks: true when `Supply_Low` (Supply_Tank low float) is on.
   - **Path 2**
     1. `XIC(Supply_Inlet)` *Examine If Closed* asks: true when `Supply_Inlet` (Supply_Tank inlet valve) is on.
2. `XIO(Supply_High)` *Examine If Open* asks: true when `Supply_High` (Supply_Tank high float) is off.
3. `XIC(Line_Run)` *Examine If Closed* asks: true when `Line_Run` (Seal-in run bit (LogixPro B3:0/0)) is on.
4. `OTE(Supply_Inlet)` *Output Energize* does: turns `Supply_Inlet` on (and off again whenever the rung is false). Every scan it writes the rung state: true on, false off.

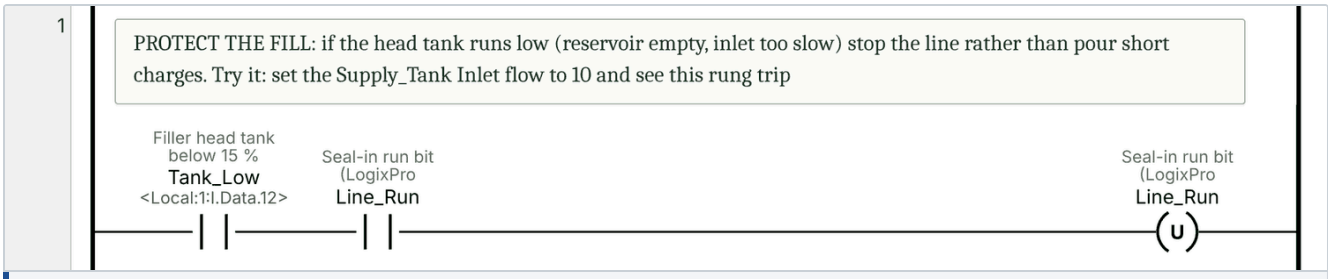
Why it is built this way:

Seal-in. The start condition turns `Supply_Inlet` on, and `Supply_Inlet` 's own contact in the parallel path keeps the rung true after the start goes away. Anything in series outside the branch (a stop, a fault) breaks it, and then it stays off until the start is made again.

> **Think of it like this:** a doorbell button and a hand. The start button is the doorbell, pressed for a moment. The seal-in contact is a hand that grabs the circuit as soon as the output comes on and keeps holding after you let go. The stop button, in series, is the only thing that can knock that hand away.

> **Watch for:** The stop has to be in series outside the branch. Put it inside the start path only and the seal-in path bypasses it: the stop no longer stops.

RUNG 1




```
XIC(Tank_Low)XIC(Line_Run)OTU(Line_Run);
```

PREDICT IT · 2 `Tank_Low` is on and `Line_Run` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

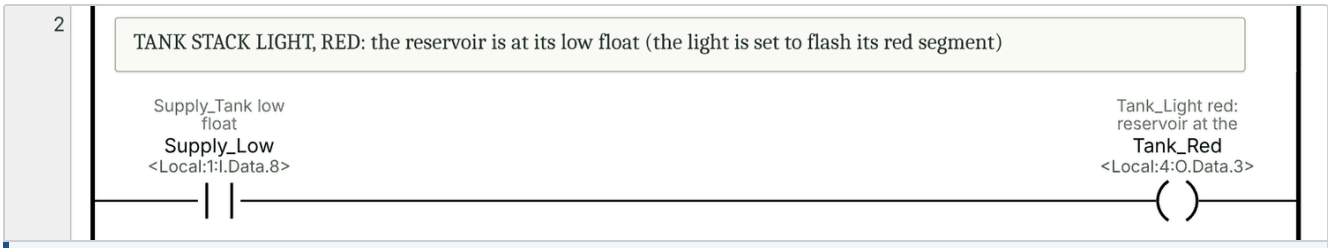
In words. When `Tank_Low` (Filler head tank below 15 %) is on and `Line_Run` (Seal-in run bit (LogixPro B3:0/0)) is on, the rung unlatches `Line_Run` (turns it off).

Left to right:

1. `XIC(Tank_Low)` *Examine If Closed* asks: true when `Tank_Low` (Filler head tank below 15 %) is on.
2. `XIC(Line_Run)` *Examine If Closed* asks: true when `Line_Run` (Seal-in run bit (LogixPro B3:0/0)) is on.
3. `OTU(Line_Run)` *Output Unlatch* does: unlatches `Line_Run` (turns it off). It only ever clears the bit.

> **Think of it like this:** a rung is a pipe with valves in it, read left to right. Every contact is a valve: XIC is open while its bit is 1, XIO while its bit is 0. Water reaches the output at the right only if every valve in a row is open. The output does not remember anything: on every scan it is simply wet or dry.

RUNG 2




```
XIC(Supply_Low)OTE(Tank_Red);
```

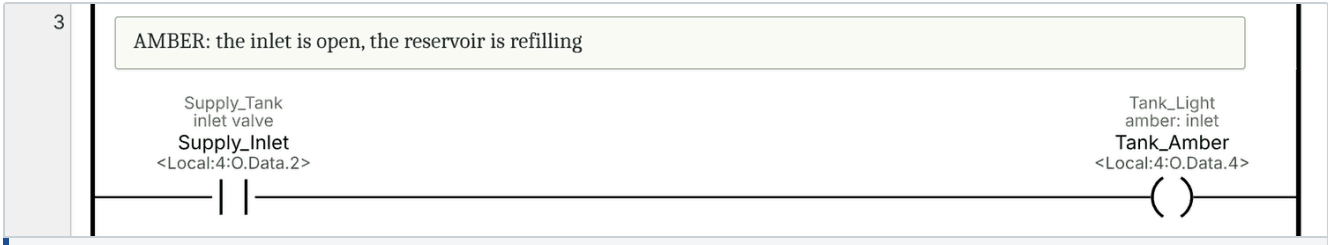
HOW THIS RUNG WORKS

In words. When `Supply_Low` (Supply_Tank low float) is on, the rung turns `Tank_Red` (Tank_Light red: reservoir at the low float (flashes)) on (and off again whenever the rung is false).

Left to right:

1. `XIC(Supply_Low)` *Examine If Closed* asks: true when `Supply_Low` (Supply_Tank low float) is on.
2. `OTE(Tank_Red)` *Output Energize* does: turns `Tank_Red` (Tank_Light red: reservoir at the low float (flashes)) on (and off again whenever the rung is false).

RUNG 3




```
XIC(Supply_Inlet)OTE(Tank_Amber);
```

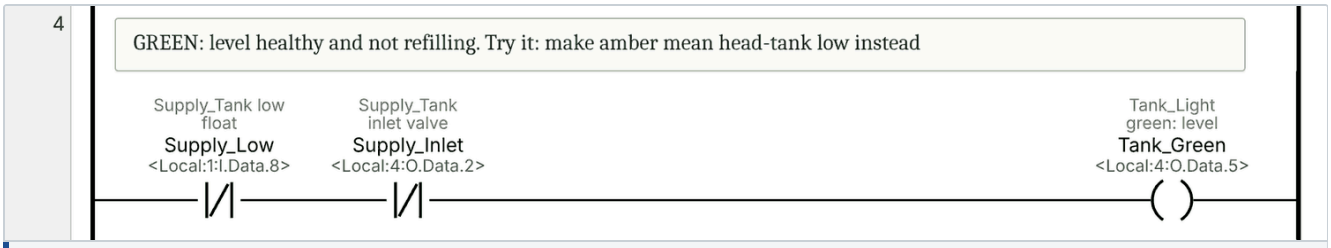
HOW THIS RUNG WORKS

In words. When `Supply_Inlet` (Supply_Tank inlet valve) is on, the rung turns `Tank_Amber` (Tank_Light amber: inlet open, refilling) on (and off again whenever the rung is false).

Left to right:

1. `XIC(Supply_Inlet)` *Examine If Closed* asks: true when `Supply_Inlet` (Supply_Tank inlet valve) is on.
2. `OTE(Tank_Amber)` *Output Energize* does: turns `Tank_Amber` (Tank_Light amber: inlet open, refilling) on (and off again whenever the rung is false).

RUNG 4




```
XIO(Supply_Low)XIO(Supply_Inlet)OTE(Tank_Green);
```

PREDICT IT · 3 `Supply_Low` is on and `Supply_Inlet` is off. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Supply_Low` (Supply_Tank low float) is off and `Supply_Inlet` (Supply_Tank inlet valve) is off, the rung turns `Tank_Green` (Tank_Light green: level healthy) on (and off again whenever the rung is false).

Left to right:

1. `XIO(Supply_Low)` *Examine If Open* asks: true when `Supply_Low` (Supply_Tank low float) is off.
2. `XIO(Supply_Inlet)` *Examine If Open* asks: true when `Supply_Inlet` (Supply_Tank inlet valve) is off.
3. `OTE(Tank_Green)` *Output Energize* does: turns `Tank_Green` (Tank_Light green: level healthy) on (and off again whenever the rung is false).

Rung 0 is a seal-in on the floats: the low probe opens the inlet valve, and the valve stays open until the high probe. **Rung 1** protects the fill: if the filler's head tank runs low (the reservoir empty, the inlet too slow) the line is stopped rather than pouring short charges. **Rungs 2–4** drive the stack light on the tank: red flashing at the low float (the light is set to flash its red segment), amber while refilling, green when healthy and not refilling.

Run it

The tank starts at 35 %: watch it fall a little with every charge (Trends: pen `Supply_Level`), reach 20 %, the red flash, the inlet valve on the tank turn green and the level climb to 90 %. Set the reservoir's *Inlet flow* to 10 and the line will eventually stop on rung 1 — the Simulator Log says why.

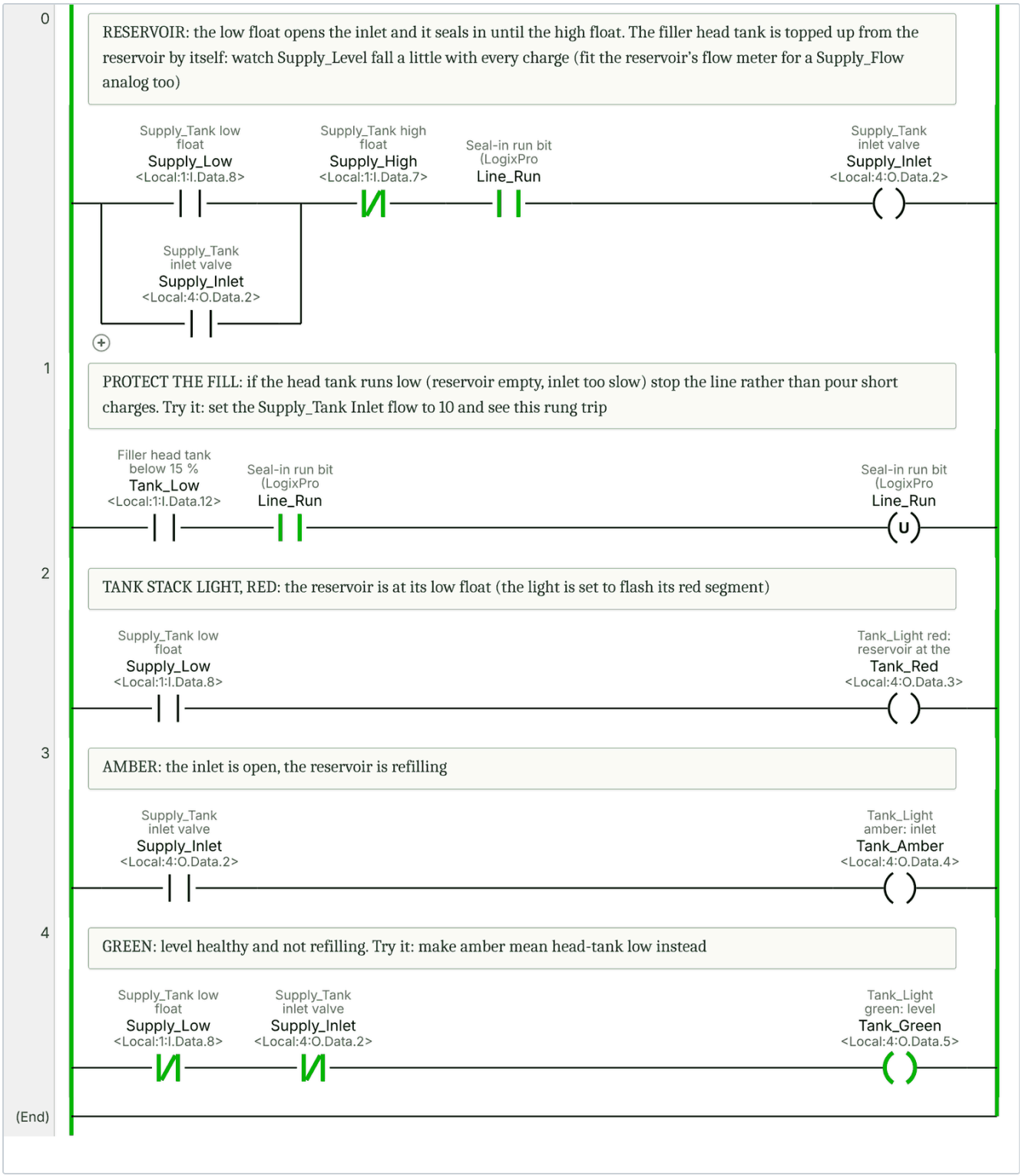


Figure 127. S6_Supply online, with the finished line running: this is what your routine should look like once every step is in. Green is power flow.

> **TRY IT**
>
> Make amber mean head-tank low instead of refilling. Then fit the reservoir's flow meter (a setting; it adds an analog input) and trend `Supply_Flow` against the charges.

Predict it: the answers

1. The rung is false at `XIO(Supply_High)`: `Supply_High` is on, so `Supply_Inlet` is **off** (an OTE writes 0 when its rung is false).
2. Every condition on the way is met, so `Line_Run` is unlatched, **off**.
3. The rung is false at `XIO(Supply_Low)`: `Supply_Low` is on, so `Tank_Green` is **off** (an OTE writes 0 when its rung is false).

24 Lesson 9 — Step 7: capping and the cap feeder

Same shape as the fill, plus the cap supply. Add the JSR above `JSR(S3_Index)` and write:

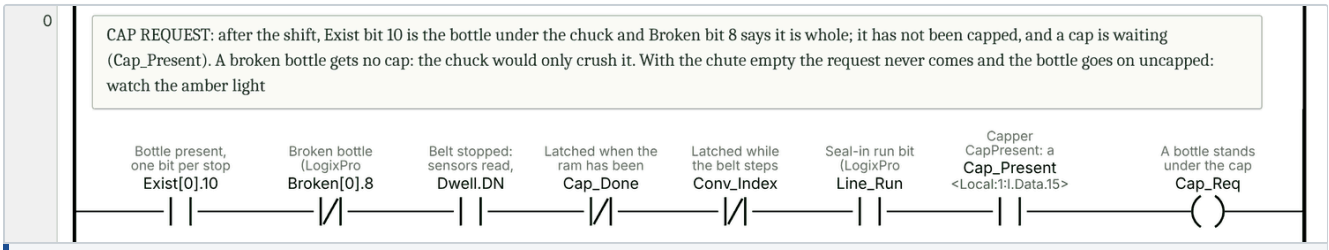
> **IN THE APP**
>
> Guided mode lesson 9 is this chapter: it checks that this lesson's tags are made, that `S7_Cap` exists, is called from MainRoutine and verifies, and then watches the line while you run it and ticks each *Run it* result below as it happens. Help → Guided Mode, or start from File → New Project… → Guided: build the bottle line.

Tags this lesson needs

Create these before you type the rungs (Verify reports any you miss as unknown). Tags from earlier lessons are not repeated.

Tag	Type	How to make it	Description
`Cap_Done`	BOOL	Edit Tags: type the name, pick the type	Latched when the ram has been down; cleared when the belt steps
`Cap_Present`	BOOL	alias for `Local:1:I.Data.15` — the Properties drawer's *tag* link makes it, then rename it	Capper CapPresent: a cap is waiting at the chuck
`Cap_Ram`	BOOL	alias for `Local:4:O.Data.9` — the Properties drawer's *tag* link makes it, then rename it	LogixPro O:2/9 · Cap ram
`Cap_Low`	BOOL	alias for `Local:1:I.Data.14` — the Properties drawer's *tag* link makes it, then rename it	Capper CapLow: the chute is down to its low count
`Cap_Feeder`	BOOL	alias for `Local:4:O.Data.10` — the Properties drawer's *tag* link makes it, then rename it	Capper Feeder: runs the vibratory cap bowl, refilling the chute
`Cap_Fill_T`	TIMER	Edit Tags: type the name, pick the type	Cap feeder run-on after CapLow clears
`Cap_Green`	BOOL	alias for `Local:4:O.Data.11` — the Properties drawer's *tag* link makes it, then rename it	Capper green indicator (column)
`Cap_Amber`	BOOL	alias for `Local:4:O.Data.12` — the Properties drawer's *tag* link makes it, then rename it	Capper amber indicator (column)

RUNG 0




```
XIC(Exist[0].10)XIO(Broken[0].8)XIC(Dwell.DN)XIO(Cap_Done)XIO(Conv_Index)XIC(Line_Run)XIC(Cap_Present)OTE(Cap_Req);
```

PREDICT IT · 1 `Exist[0].10` is off, `Broken[0].8` is off, `Dwell.DN` (the done bit of `Dwell`: Belt stopped: sensors read, feeder drops a bottle) is on, `Cap_Done` is off, `Conv_Index` is off, `Line_Run` is on and `Cap_Present` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

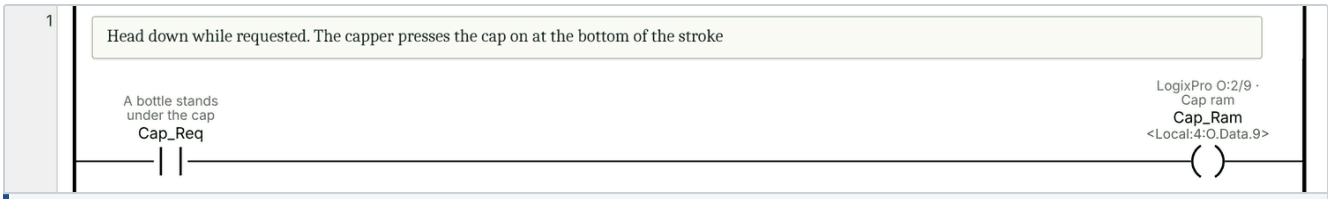
In words. When `Exist[0].10` is on, `Broken[0].8` is off, `Dwell.DN` (the done bit of `Dwell`: Belt stopped: sensors read, feeder drops a bottle) is on, `Cap_Done` (Latched when the ram has been down; cleared when the belt steps) is off, `Conv_Index` (Latched while the belt steps one pitch) is off, `Line_Run` (Seal-in run bit (LogixPro B3:0/0)) is on and `Cap_Present` (A cap is waiting at the chuck) is on, the rung turns `Cap_Req` (A bottle stands under the cap ram and has not been capped) on (and off again whenever the rung is false).

Left to right:

1. `XIC(Exist[0].10)` *Examine If Closed* asks: true when `Exist[0].10` is on.
2. `XIO(Broken[0].8)` *Examine If Open* asks: true when `Broken[0].8` is off.
3. `XIC(Dwell.DN)` *Examine If Closed* asks: true when `Dwell.DN` (the done bit of `Dwell`: Belt stopped: sensors read, feeder drops a bottle) is on.
4. `XIO(Cap_Done)` *Examine If Open* asks: true when `Cap_Done` (Latched when the ram has been down; cleared when the belt steps) is off.
5. `XIO(Conv_Index)` *Examine If Open* asks: true when `Conv_Index` (Latched while the belt steps one pitch) is off.
6. `XIC(Line_Run)` *Examine If Closed* asks: true when `Line_Run` (Seal-in run bit (LogixPro B3:0/0)) is on.
7. `XIC(Cap_Present)` *Examine If Closed* asks: true when `Cap_Present` (A cap is waiting at the chuck) is on.
8. `OTE(Cap_Req)` *Output Energize* does: turns `Cap_Req` (A bottle stands under the cap ram and has not been capped) on (and off again whenever the rung is false). Every scan it writes the rung state: true on, false off.

> **Think of it like this:** a rung is a pipe with valves in it, read left to right. Every contact is a valve: XIC is open while its bit is 1, XIO while its bit is 0. Water reaches the output at the right only if every valve in a row is open. The output does not remember anything: on every scan it is simply wet or dry.

RUNG 1




```
XIC(Cap_Req)OTE(Cap_Ram);
```

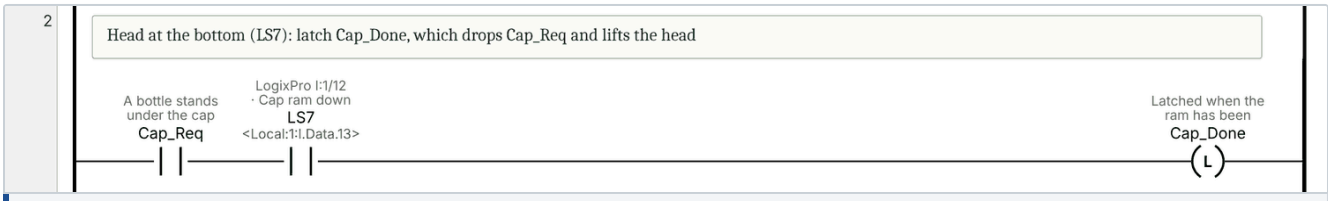
HOW THIS RUNG WORKS

In words. When `Cap_Req` (A bottle stands under the cap ram and has not been capped) is on, the rung turns `Cap_Ram` (Cap ram) on (and off again whenever the rung is false).

Left to right:

1. `XIC(Cap_Req)` *Examine If Closed* asks: true when `Cap_Req` (A bottle stands under the cap ram and has not been capped) is on.
2. `OTE(Cap_Ram)` *Output Energize* does: turns `Cap_Ram` (Cap ram) on (and off again whenever the rung is false).

RUNG 2




```
XIC(Cap_Req)XIC(LS7)OTL(Cap_Done);
```

PREDICT IT · 2 `Cap_Req` is on and `LS7` is off. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Cap_Req` (A bottle stands under the cap ram and has not been capped) is on and `LS7` (Cap ram down) is on, the rung latches `Cap_Done` (Latched when the ram has been down; cleared when the belt steps) on.

Left to right:

1. `XIC(Cap_Req)` *Examine If Closed* asks: true when `Cap_Req` (A bottle stands under the cap ram and has not been capped) is on.
2. `XIC(LS7)` *Examine If Closed* asks: true when `LS7` (Cap ram down) is on.
3. `OTL(Cap_Done)` *Output Latch* does: latches `Cap_Done` (Latched when the ram has been down; cleared when the belt steps) on. It only ever sets the bit; the bit stays on after the rung goes false.

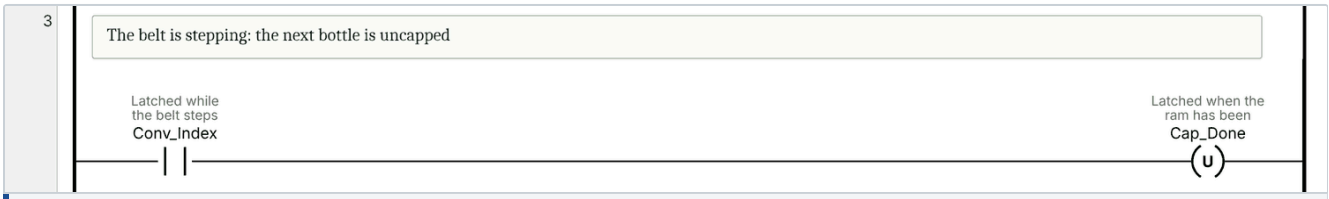
Why it is built this way:

A latch. `Cap_Done` stays on after this rung goes false; an `OTU` elsewhere in the program clears it.

> **Think of it like this:** a light switch instead of a doorbell. OTL flips it on and it stays on when you take your hand away; OTU flips it off. Whatever flipped it on does not have to stay there.

> **Watch for:** A latched bit survives the condition that set it: every way in needs a way out, or it stays on for good.

RUNG 3




```
XIC(Conv_Index)OTU(Cap_Done);
```

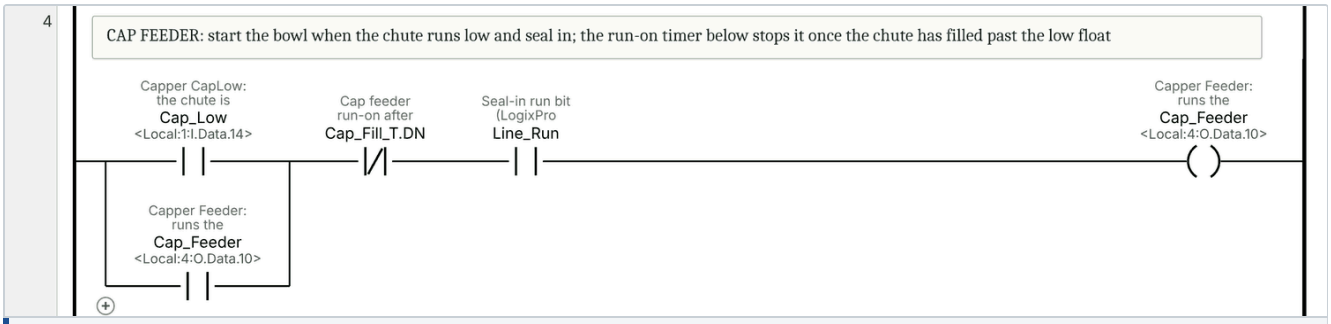
HOW THIS RUNG WORKS

In words. When `Conv_Index` (Latched while the belt steps one pitch) is on, the rung unlatches `Cap_Done` (Latched when the ram has been down; cleared when the belt steps) (turns it off).

Left to right:

1. `XIC(Conv_Index)` *Examine If Closed* asks: true when `Conv_Index` (Latched while the belt steps one pitch) is on.
2. `OTU(Cap_Done)` *Output Unlatch* does: unlatches `Cap_Done` (Latched when the ram has been down; cleared when the belt steps) (turns it off). It only ever clears the bit.

RUNG 4




```
[XIC(Cap_Low),XIC(Cap_Feeder)]XIO(Cap_Fill_T.DN)XIC(Line_Run)OTE(Cap_Feeder);
```

PREDICT IT · 3 `Cap_Low` is on, `Cap_Fill_T.DN` (the done bit of `Cap_Fill_T`: Cap feeder run-on after CapLow clears) is on and `Line_Run` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When either (`Cap_Low` (The chute is down to its low count) is on) *or* (`Cap_Feeder` (Runs the vibratory cap bowl, refilling the chute) is on), `Cap_Fill_T.DN` (the done bit of `Cap_Fill_T`: Cap feeder run-on after CapLow clears) is off and `Line_Run` (Seal-in run bit (LogixPro B3:0/0)) is on, the rung turns `Cap_Feeder` on (and off again whenever the rung is false).

Left to right:

1. A branch of 2 parallel paths: power gets through if any one of them is true.

   Path 1

   1. `XIC(Cap_Low)` *Examine If Closed* asks: true when `Cap_Low` (The chute is down to its low count) is on.

   Path 2

   1. `XIC(Cap_Feeder)` *Examine If Closed* asks: true when `Cap_Feeder` (Runs the vibratory cap bowl, refilling the chute) is on.
2. `XIO(Cap_Fill_T.DN)` *Examine If Open* asks: true when `Cap_Fill_T.DN` (the done bit of `Cap_Fill_T`: Cap feeder run-on after CapLow clears) is off.
3. `XIC(Line_Run)` *Examine If Closed* asks: true when `Line_Run` (Seal-in run bit (LogixPro B3:0/0)) is on.
4. `OTE(Cap_Feeder)` *Output Energize* does: turns `Cap_Feeder` on (and off again whenever the rung is false).

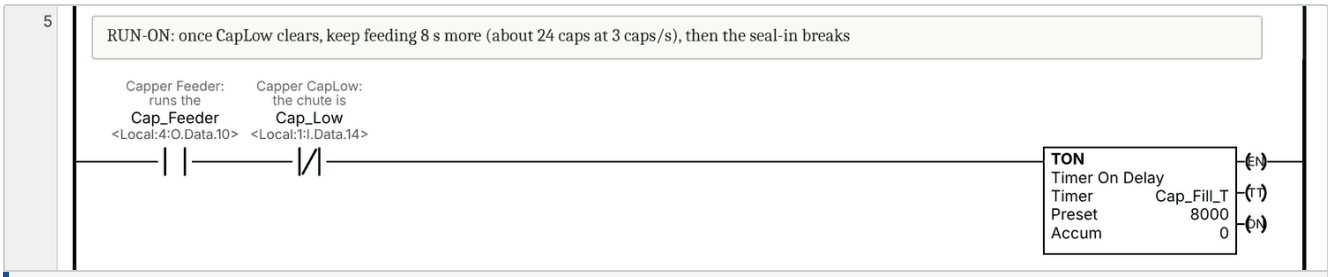
Why it is built this way:

Seal-in. The start condition turns `Cap_Feeder` on, and `Cap_Feeder`'s own contact in the parallel path keeps the rung true after the start goes away. Anything in series outside the branch (a stop, a fault) breaks it, and then it stays off until the start is made again.

> **Think of it like this:** a doorbell button and a hand. The start button is the doorbell, pressed for a moment. The seal-in contact is a hand that grabs the circuit as soon as the output comes on and keeps holding after you let go. The stop button, in series, is the only thing that can knock that hand away.

> **Watch for:** The stop has to be in series outside the branch. Put it inside the start path only and the seal-in path bypasses it: the stop no longer stops.

RUNG 5




```
XIC(Cap_Feeder)XIO(Cap_Low)TON(Cap_Fill_T,8000,0);
```

PREDICT IT · 4 `Cap_Feeder` is on and `Cap_Low` is off. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Cap_Feeder` (Runs the vibratory cap bowl, refilling the chute) is on and `Cap_Low` (The chute is down to its low count) is off, the rung runs the timer `Cap_Fill_T` (Cap feeder run-on after CapLow clears) toward 8000 ms (8 s).

Left to right:

1. `XIC(Cap_Feeder)` *Examine If Closed* asks: true when `Cap_Feeder` (Runs the vibratory cap bowl, refilling the chute) is on.
2. `XIO(Cap_Low)` *Examine If Open* asks: true when `Cap_Low` (The chute is down to its low count) is off.
3. `TON(Cap_Fill_T,8000,0)` *Timer On Delay* does: runs the timer `Cap_Fill_T` (Cap feeder run-on after CapLow clears) toward 8000 ms (8 s). The accumulator counts while the rung is true; .DN comes on at the preset; the moment the rung goes false the timer resets.

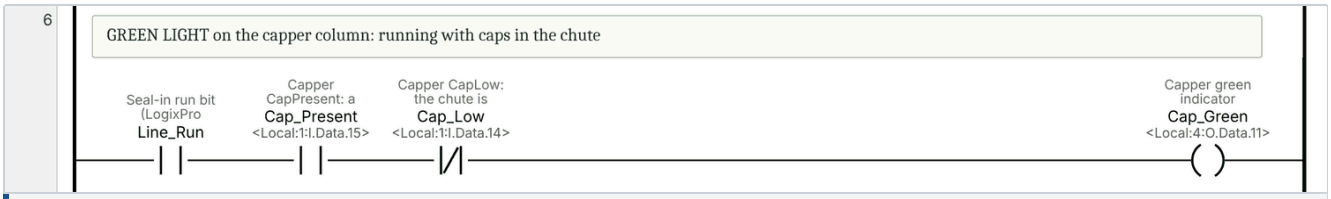
Why it is built this way:

A timer. `Cap_Fill_T` only times while this rung is true; other rungs use its `.DN` (done) or `.TT` (timing) bits. Being a TON, it starts again from zero every time the rung goes false.

> **Think of it like this:** a kitchen timer that only runs while you hold your finger on it. Take your finger off and it snaps back to zero. Hold it long enough and the done light comes on, and stays on as long as your finger is still there.

> **Watch for:** If anything in series flickers false for one scan, a TON starts over from zero. Use RTO when the time must survive an interruption.

RUNG 6




```
XIC(Line_Run)XIC(Cap_Present)XIO(Cap_Low)OTE(Cap_Green);
```

PREDICT IT · 5 `Line_Run` is off, `Cap_Present` is on and `Cap_Low` is off. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

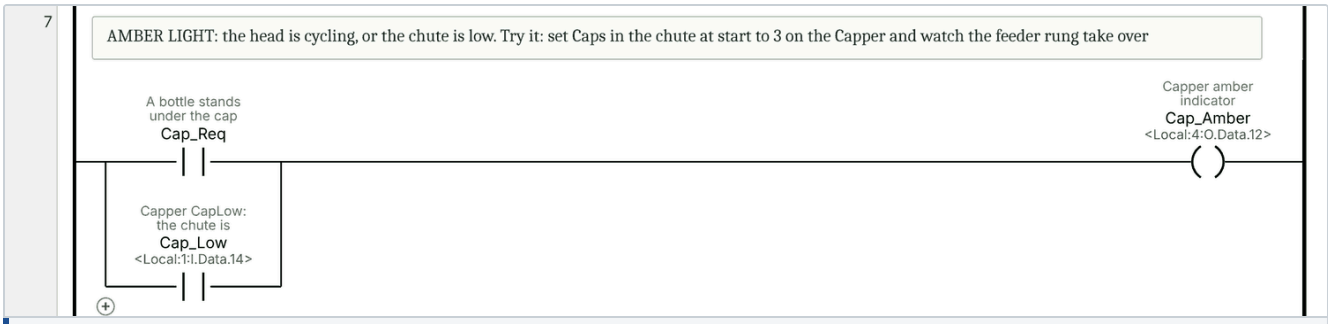
HOW THIS RUNG WORKS

In words. When `Line_Run` (Seal-in run bit (LogixPro B3:0/0)) is on, `Cap_Present` (A cap is waiting at the chuck) is on and `Cap_Low` (The chute is down to its low count) is off, the rung turns `Cap_Green` (Capper green indicator (column)) on (and off again whenever the rung is false).

Left to right:

1. `XIC(Line_Run)` *Examine If Closed* asks: true when `Line_Run` (Seal-in run bit (LogixPro B3:0/0)) is on.
2. `XIC(Cap_Present)` *Examine If Closed* asks: true when `Cap_Present` (A cap is waiting at the chuck) is on.
3. `XIO(Cap_Low)` *Examine If Open* asks: true when `Cap_Low` (The chute is down to its low count) is off.
4. `OTE(Cap_Green)` *Output Energize* does: turns `Cap_Green` (Capper green indicator (column)) on (and off again whenever the rung is false).

RUNG 7




```
[XIC(Cap_Req),XIC(Cap_Low)]OTE(Cap_Amber);
```

PREDICT IT · 6 `Cap_Req` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When either (`Cap_Req` (A bottle stands under the cap ram and has not been capped) is on) *or* (`Cap_Low` (The chute is down to its low count) is on), the rung turns `Cap_Amber` (Capper amber indicator (column)) on (and off again whenever the rung is false).

Left to right:

1. A branch of 2 parallel paths: power gets through if any one of them is true.
   - **Path 1**
     1. `XIC(Cap_Req)` *Examine If Closed* asks: true when `Cap_Req` (A bottle stands under the cap ram and has not been capped) is on.
   - **Path 2**
     1. `XIC(Cap_Low)` *Examine If Closed* asks: true when `Cap_Low` (The chute is down to its low count) is on.
2. `OTE(Cap_Amber)` *Output Energize* does: turns `Cap_Amber` (Capper amber indicator (column)) on (and off again whenever the rung is false).

> **Think of it like this:** parallel paths are two roads to the same town. If either road is open you get there; the rung only cares that power reaches the far end, not which way it went.

Rungs 0–3 mirror the fill station: a whole bottle under the chuck (Exist 10, not Broken 8), a cap waiting (`Cap_Present`), not yet capped → `Cap_Req` ; the chuck comes down; `LS7` at the bottom of the stroke latches `Cap_Done` ; the belt stepping clears it. The Broken bit matters here: a chuck that comes down on a broken bottle crushes it (try it — exercise below). **Rungs 4–5** are the cap feeder: when the chute runs low, `Cap_Feeder` starts the rotary magazine and seals in; once `Cap_Low` clears, the run-on timer gives it eight more seconds (about 24 caps) and then the seal-in breaks. **Rungs 6–7** drive the two lights on the column: green while running with caps in the chute, amber while cycling or low.

Run it

Start the line. At stop 10 the chuck comes down on each filled bottle and presses a cap on, and the magazine indexes one pocket per cap; the green column lamp is on (Side view or 3D, or `Cap_Green` in Monitor Tags). The bottles still leave at the end of the line, broken ones included, which step 8 will fix.

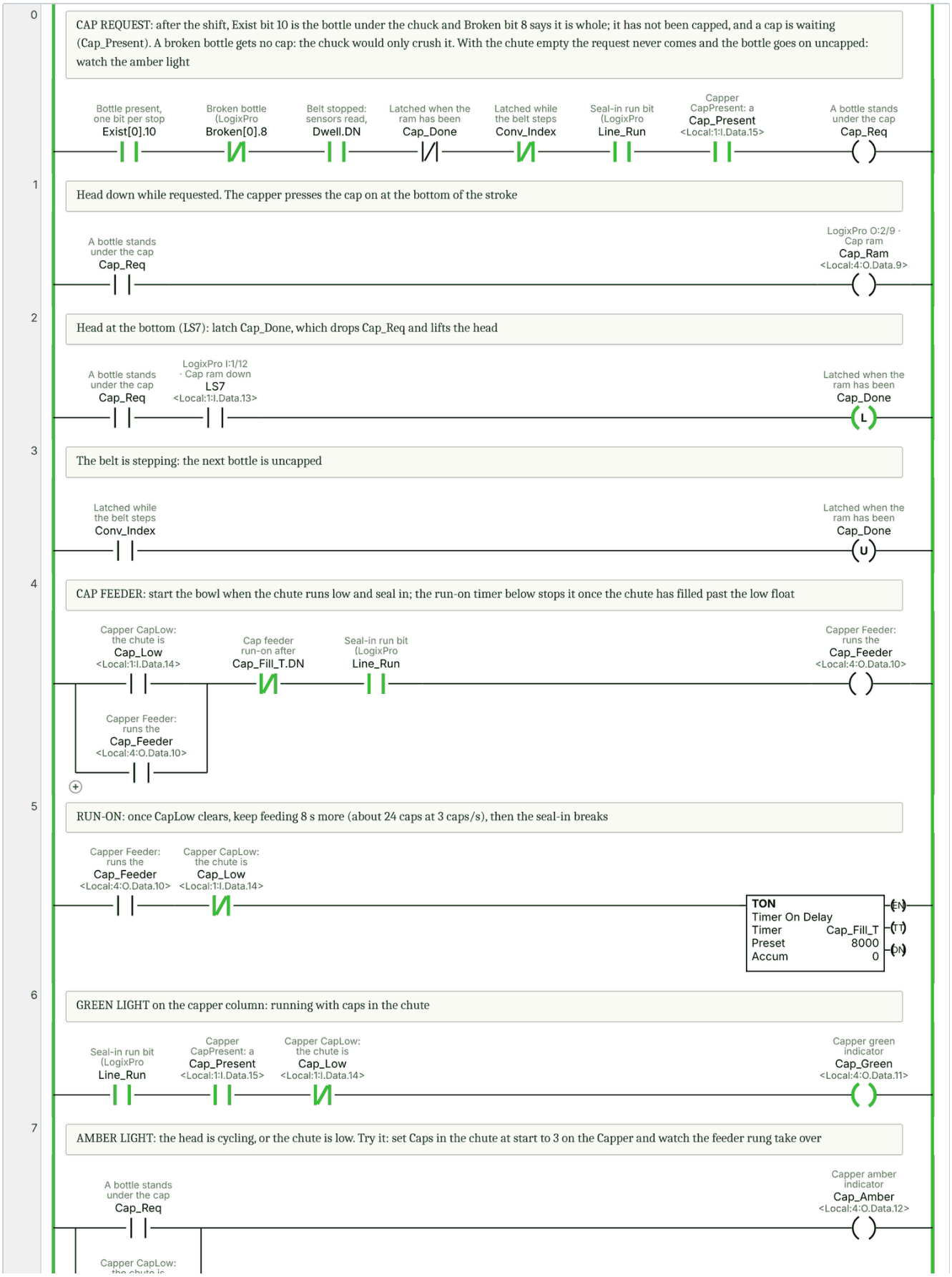

0
CAP REQUEST: after the shift, Exist bit 10 is the bottle under the chuck and Broken bit 8 says it is whole; it has not been capped, and a cap is waiting (Cap_Present). A broken bottle gets no cap: the chuck would only crush it. With the chute empty the request never comes and the bottle goes on uncapped: watch the amber light
Bottle present, one bit per stop
Exist[0].10
Broken bottle (LogixPro
Broken[0].8
Belt stopped: sensors read,
Dwell.DN
Latched when the ram has been
Cap_Done
Latched while the belt steps
Conv_Index
Seal-in run bit (LogixPro
Line_Run
Capper CapPresent: a
Cap_Present
<Local:1:I.Data.15>
A bottle stands under the cap
Cap_Req
1
Head down while requested. The capper presses the cap on at the bottom of the stroke
A bottle stands under the cap
Cap_Req
LogixPro O:2/9 · Cap ram
Cap_Ram
<Local:4:O.Data.9>
2
Head at the bottom (LS7): latch Cap_Done, which drops Cap_Req and lifts the head
A bottle stands under the cap
Cap_Req
LogixPro I:1/12 · Cap ram down
LS7
<Local:1:I.Data.13>
Latched when the ram has been
Cap_Done
L
3
The belt is stepping: the next bottle is uncapped
Latched while the belt steps
Conv_Index
Latched when the ram has been
Cap_Done
U
4
CAP FEEDER: start the bowl when the chute runs low and seal in; the run-on timer below stops it once the chute has filled past the low float
Capper CapLow: the chute is
Cap_Low
<Local:1:I.Data.14>
Cap feeder run-on after
Cap_Fill_T.DN
Seal-in run bit (LogixPro
Line_Run
Capper Feeder: runs the
Cap_Feeder
<Local:4:O.Data.10>
Capper Feeder: runs the
Cap_Feeder
<Local:4:O.Data.10>
5
RUN-ON: once CapLow clears, keep feeding 8 s more (about 24 caps at 3 caps/s), then the seal-in breaks
Capper Feeder: runs the
Cap_Feeder
<Local:4:O.Data.10>
Capper CapLow: the chute is
Cap_Low
<Local:1:I.Data.14>
TON
Timer On Delay
Timer Cap_Fill_T
Preset 8000
Accum 0
EN
TT
DN
6
GREEN LIGHT on the capper column: running with caps in the chute
Seal-in run bit (LogixPro
Line_Run
Capper CapPresent: a
Cap_Present
<Local:1:I.Data.15>
Capper CapLow: the chute is
Cap_Low
<Local:1:I.Data.14>
Capper green indicator
Cap_Green
<Local:4:O.Data.11>
7
AMBER LIGHT: the head is cycling, or the chute is low. Try it: set Caps in the chute at start to 3 on the Capper and watch the feeder rung take over
A bottle stands under the cap
Cap_Req
Capper amber indicator
Cap_Amber
<Local:4:O.Data.12>
Capper CapLow:

Figure 128. S7_Cap online, with the finished line running: this is what your routine should look like once every step is in. Green is power flow.

> **TRY IT**
>
> Set the Capper's *Caps in the chute at start* to 3, run the line, and watch the amber lamp, the feeder rung and the magazine spin as the chute refills. Then take `XIO(Broken[0].8)` out of rung 0 and watch what the chuck does to a broken bottle.

Predict it: the answers

1. The rung is false at `XIC(Exist[0].10)`: `Exist[0].10` is off, so `Cap_Req` is **off** (an OTE writes 0 when its rung is false).
2. The rung is false at `XIC(LS7)`: `LS7` is off, so the OTL does nothing: `Cap_Done` keeps whatever it had.
3. The rung is false at `XIO(Cap_Fill_T.DN)`: `Cap_Fill_T.DN` (the done bit of `Cap_Fill_T`: Cap feeder run-on after CapLow clears) is on, so `Cap_Feeder` is **off** (an OTE writes 0 when its rung is false).
4. Every condition on the way is met, so `Cap_Fill_T` times toward its preset.
5. The rung is false at `XIC(Line_Run)`: `Line_Run` is off, so `Cap_Green` is **off** (an OTE writes 0 when its rung is false).
6. Every condition on the way is met, so `Cap_Amber` is **on**.

25 Lesson 10 — Step 8: the gates, the divert conveyor, the boxes and the shredder

Two gates, and the machinery below them. The gate rungs are written in the *during the dwell* frame — qualified by `Dwell.TT`, the timer timing — so they read the bits for bottles standing on the belt now: Broken 11 at the scrap gate; Exist 20, Large 19, Broken 18 at the divert gate. Add the JSR and write:

> **IN THE APP**
>
> Guided mode lesson 10 is this chapter: it checks that this lesson's tags are made, that `S8_Sort` exists, is called from MainRoutine and verifies, and then watches the line while you run it and ticks each *Run it* result below as it happens. Help → Guided Mode, or start from File → New Project… → Guided: build the bottle line.

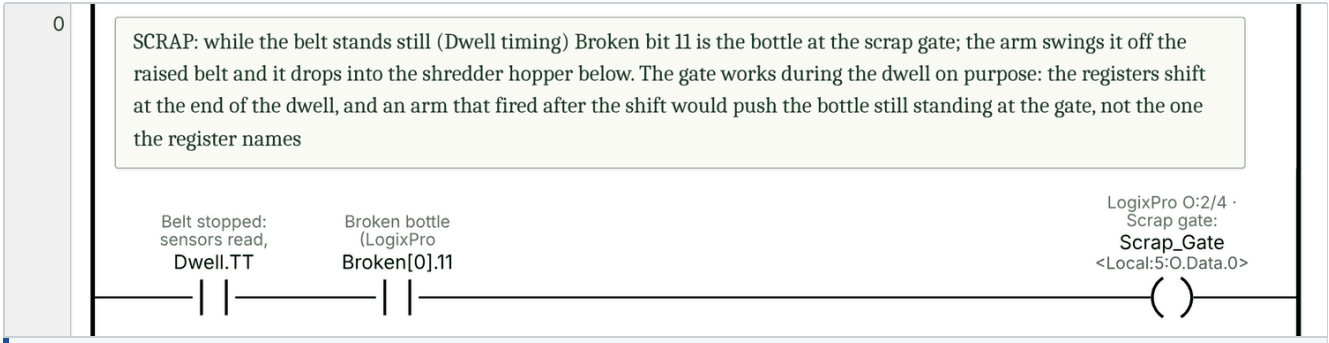
Tags this lesson needs

Create these before you type the rungs (Verify reports any you miss as unknown). Tags from earlier lessons are not repeated.

Tag	Type	How to make it	Description
`Scrap_Gate`	BOOL	alias for `Local:5:O.Data.0` — the Properties drawer's *tag* link makes it, then rename it	LogixPro O:2/4 · Scrap gate: swings a bottle into the grinder
`Divert_Gate`	BOOL	alias for `Local:5:O.Data.2` — the Properties drawer's *tag* link makes it, then rename it	LogixPro O:2/5 · Divert gate: swings a bottle onto the divert conveyor
`Lower_Full`	BOOL	alias for `Local:10:I.Data.4` — the Properties drawer's *tag* link makes it, then rename it	Packer lower lane full: small bottles arriving now wait at the stop
`LS12`	BOOL	alias for `Local:2:I.Data.13` — the Properties drawer's *tag* link makes it, then rename it	Stop eye at the end of the divert conveyor, just before the packer lower lane
`Divert_Conveyor`	BOOL	alias for `Local:5:O.Data.3` — the Properties drawer's *tag* link makes it, then rename it	LogixPro O:2/2 · Divert conveyor for large bottles
`LS10`	BOOL	alias for `Local:3:I.Data.0` — the Properties drawer's *tag* link makes it, then rename it	LogixPro I:1/15 · Scrap box under the grinder chute
`Box_Full`	BOOL	alias for `Local:2:I.Data.9` — the Properties drawer's *tag* link makes it, then rename it	Grinder BoxFull: the box under the scrap drop holds its share
`Scrap_Conveyor`	BOOL	alias for `Local:5:O.Data.4` — the Properties drawer's *tag* link makes it, then rename it	LogixPro O:2/1 · Box conveyor under the grinder
`Scrap_Waiting`	BOOL	alias for `Local:2:I.Data.8` — the Properties drawer's *tag* link makes it, then rename it	Grinder Detect: a broken bottle is waiting in the hopper

Tag	Type	How to make it	Description
`Grinder`	BOOL	alias for `Local:5:O.Data.1` — the Properties drawer's *tag* link makes it, then rename it	LogixPro O:2/3 · Grinder motor

RUNG 0




```
XIC(Dwell.TT)XIC(Broken[0].11)OTE(Scrap_Gate);
```

PREDICT IT · 1 `Dwell.TT` (the timing bit of `Dwell`: Belt stopped: sensors read, feeder drops a bottle) is off and `Broken[0].11` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Dwell.TT` (the timing bit of `Dwell`: Belt stopped: sensors read, feeder drops a bottle) is on and `Broken[0].11` is on, the rung turns `Scrap_Gate` (Scrap gate: swings a bottle into the grinder) on (and off again whenever the rung is false).

Left to right:

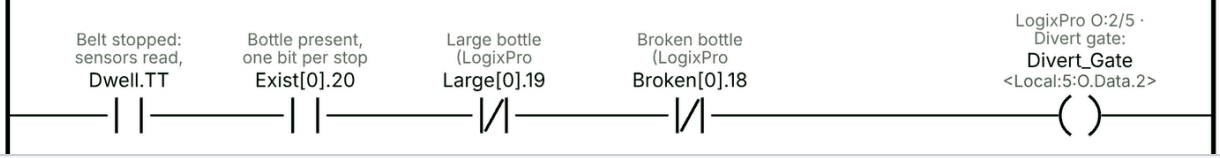
1. `XIC(Dwell.TT)` *Examine If Closed* asks: true when `Dwell.TT` (the timing bit of `Dwell`: Belt stopped: sensors read, feeder drops a bottle) is on.
2. `XIC(Broken[0].11)` *Examine If Closed* asks: true when `Broken[0].11` is on.
3. `OTE(Scrap_Gate)` *Output Energize* does: turns `Scrap_Gate` (Scrap gate: swings a bottle into the grinder) on (and off again whenever the rung is false). Every scan it writes the rung state: true on, false off.

> **Think of it like this:** a rung is a pipe with valves in it, read left to right. Every contact is a valve: XIC is open while its bit is 1, XIO while its bit is 0. Water reaches the output at the right only if every valve in a row is open. The output does not remember anything: on every scan it is simply wet or dry.

RUNG 1

1

DIVERT: a small, unbroken bottle at the divert gate (Exist bit 20, not Large 19, not Broken 18, all in the dwell frame) is swung off the raised belt and rolls down the catch incline onto the divert conveyor. Large bottles carry on to the end of the line. The Broken bit matters: a broken bottle was already scrapped, but its Exist bit is still set, so without it the arm would fire at an empty slot




```
XIC(Dwell.TT)XIC(Exist[0].20)XIO(Large[0].19)XIO(Broken[0].18)OTE(Divert_Gate);
```

PREDICT IT · 2 `Dwell.TT` (the timing bit of `Dwell`: Belt stopped: sensors read, feeder drops a bottle) is on, `Exist[0].20` is on, `Large[0].19` is off and `Broken[0].18` is off. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

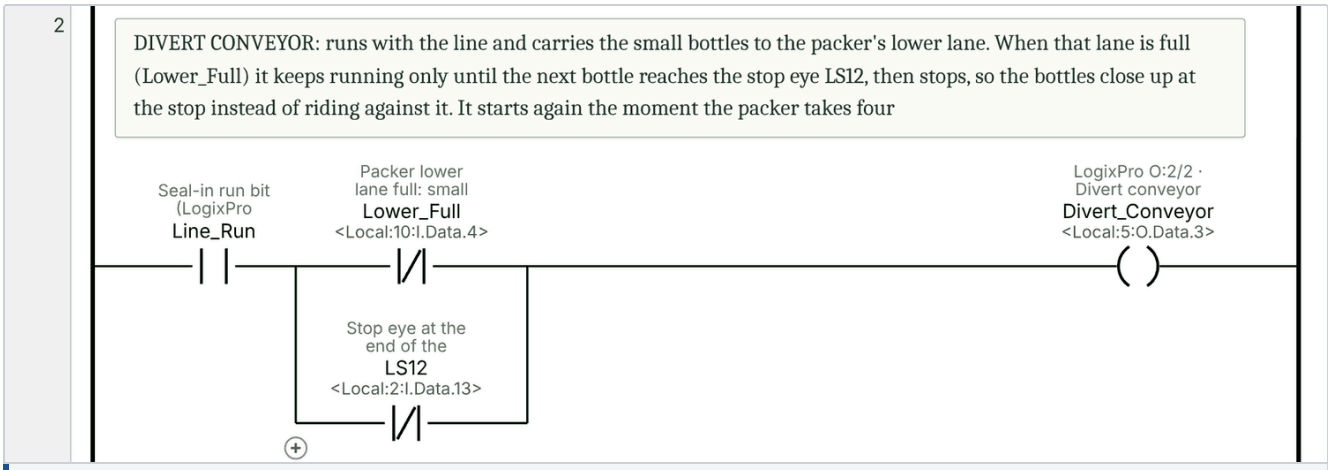
HOW THIS RUNG WORKS

In words. When `Dwell.TT` (the timing bit of `Dwell`: Belt stopped: sensors read, feeder drops a bottle) is on, `Exist[0].20` is on, `Large[0].19` is off and `Broken[0].18` is off, the rung turns `Divert_Gate` (Divert gate: swings a bottle onto the divert conveyor) on (and off again whenever the rung is false).

Left to right:

1. `XIC(Dwell.TT)` *Examine If Closed* asks: true when `Dwell.TT` (the timing bit of `Dwell`: Belt stopped: sensors read, feeder drops a bottle) is on.
2. `XIC(Exist[0].20)` *Examine If Closed* asks: true when `Exist[0].20` is on.
3. `XIO(Large[0].19)` *Examine If Open* asks: true when `Large[0].19` is off.
4. `XIO(Broken[0].18)` *Examine If Open* asks: true when `Broken[0].18` is off.
5. `OTE(Divert_Gate)` *Output Energize* does: turns `Divert_Gate` (Divert gate: swings a bottle onto the divert conveyor) on (and off again whenever the rung is false).

RUNG 2




```
XIC(Line_Run)[XIO(Lower_Full),XIO(LS12)]OTE(Divert_Conveyor);
```

PREDICT IT · 3 `Line_Run` is off and `Lower_Full` is off. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Line_Run` (Seal-in run bit (LogixPro B3:0/0)) is on and either (`Lower_Full` (Packer lower lane full: small bottles arriving now wait at the stop) is off) *or* (`LS12` (Stop eye at the end of the divert conveyor, just before the packer lower lane) is off), the rung turns `Divert_Conveyor` (Divert conveyor for large bottles) on (and off again whenever the rung is false).

Left to right:

1. `XIC(Line_Run)` *Examine If Closed* asks: true when `Line_Run` (Seal-in run bit (LogixPro B3:0/0)) is on.
2. A branch of 2 parallel paths: power gets through if any one of them is true.

   Path 1

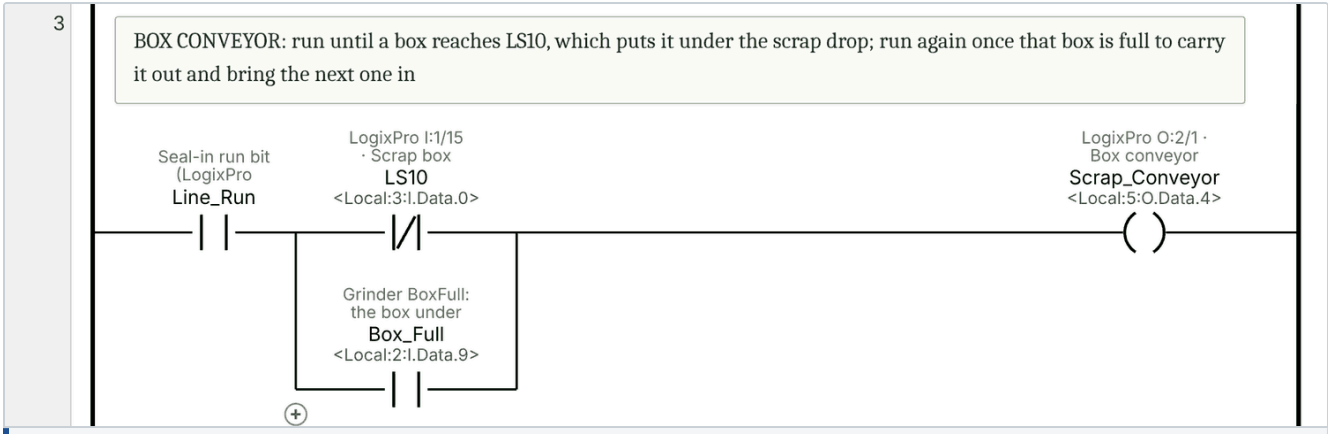
   1. `XIO(Lower_Full)` *Examine If Open* asks: true when `Lower_Full` (Packer lower lane full: small bottles arriving now wait at the stop) is off.

   Path 2

   1. `XIO(LS12)` *Examine If Open* asks: true when `LS12` (Stop eye at the end of the divert conveyor, just before the packer lower lane) is off.
3. `OTE(Divert_Conveyor)` *Output Energize* does: turns `Divert_Conveyor` (Divert conveyor for large bottles) on (and off again whenever the rung is false).

> **Think of it like this:** parallel paths are two roads to the same town. If either road is open you get there; the rung only cares that power reaches the far end, not which way it went.

RUNG 3




```
XIC(Line_Run)[XIO(LS10),XIC(Box_Full)]OTE(Scrap_Conveyor);
```

PREDICT IT · 4 `Line_Run` is on and `LS10` is off. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Line_Run` (Seal-in run bit (LogixPro B3:0/0)) is on and either (`LS10` (Scrap box under the grinder chute) is off) *or* (`Box_Full` (The box under the scrap drop holds its share) is on), the rung turns `Scrap_Conveyor` (Box conveyor under the grinder) on (and off again whenever the rung is false).

Left to right:

1. `XIC(Line_Run)` *Examine If Closed* asks: true when `Line_Run` (Seal-in run bit (LogixPro B3:0/0)) is on.
2. A branch of 2 parallel paths: power gets through if any one of them is true.

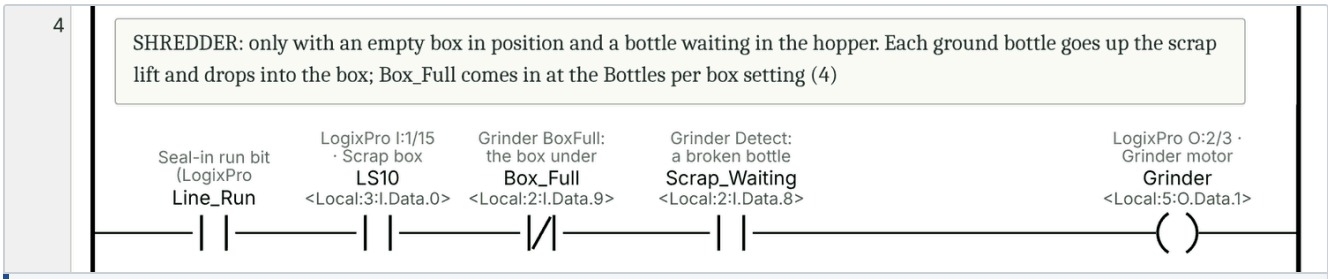
   Path 1

   1. `XIO(LS10)` *Examine If Open* asks: true when `LS10` (Scrap box under the grinder chute) is off.

   Path 2

   1. `XIC(Box_Full)` *Examine If Closed* asks: true when `Box_Full` (The box under the scrap drop holds its share) is on.
3. `OTE(Scrap_Conveyor)` *Output Energize* does: turns `Scrap_Conveyor` (Box conveyor under the grinder) on (and off again whenever the rung is false).

RUNG 4




```
XIC(Line_Run)XIC(LS10)XIO(Box_Full)XIC(Scrap_Waiting)OTE(Grinder);
```

PREDICT IT · 5 `Line_Run` is on, `LS10` is on, `Box_Full` is on and `Scrap_Waiting` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Line_Run` (Seal-in run bit (LogixPro B3:0/0)) is on, `LS10` (Scrap box under the grinder chute) is on, `Box_Full` (The box under the scrap drop holds its share) is off and `Scrap_Waiting` (Grinder Detect: a broken bottle is waiting in the hopper) is on, the rung turns `Grinder` (Grinder motor) on (and off again whenever the rung is false).

Left to right:

1. `XIC(Line_Run)` *Examine If Closed* asks: true when `Line_Run` (Seal-in run bit (LogixPro B3:0/0)) is on.
2. `XIC(LS10)` *Examine If Closed* asks: true when `LS10` (Scrap box under the grinder chute) is on.
3. `XIO(Box_Full)` *Examine If Open* asks: true when `Box_Full` (The box under the scrap drop holds its share) is off.
4. `XIC(Scrap_Waiting)` *Examine If Closed* asks: true when `Scrap_Waiting` (Grinder Detect: a broken bottle is waiting in the hopper) is on.
5. `OTE(Grinder)` *Output Energize* does: turns `Grinder` (Grinder motor) on (and off again whenever the rung is false).

Rung 0: a broken bottle at the scrap gate → the arm swings it off the deck and it drops into the shredder hopper. **Rung 1**: a small, unbroken bottle at the divert gate → down the catch incline to the divert conveyor; large bottles carry on to the end. The Exist bit stops the gate firing at empty slots at start-up; the Broken bit stops it firing at the empty slot a scrapped bottle left behind, because its Exist bit is still set.

Rung 2 is the divert conveyor. It carries the small bottles on under the case packer to its **lower lane**, a stop and a buffer of eight at the end of the conveyor, where the packer's head comes down for them (step 12). While there is room the bottles roll straight into the lane. When the lane is full, `Lower_Full`, the next bottle waits at the stop and the conveyor should not go on running under it: the rung keeps the conveyor running *until a bottle reaches the stop eye* `LS12`, then stops it, so the bottles close up nose to tail behind the stop instead of riding on the rollers against it. It is the accumulation pattern of every conveyor that feeds a machine: `[XIO(Lower_Full),XIO(LS12)]` reads *room in the lane, or nothing at the eye yet*. The moment the packer takes four, `Lower_Full` drops and the conveyor runs again. (Before 99.1 the divert conveyor ran whenever the line ran, from step 1, and the small bottles went out at Divert_Out.)

Rung 3 runs the box conveyor until a box reaches `LS10` under the chute, and again once the box is full to carry it out and bring the next one. **Rung 4** runs the shredder only with an empty box in position and a bottle waiting in the hopper.

Why the dwell frame? A gate arm acts on whatever is standing at the gate. The registers shift at the end of the dwell, before the belt moves; a rung that fired on the shifted bit would swing the arm while the *previous* bottle still stands there and push the wrong one. Qualifying with `Dwell.TT` fires the arm only while the belt is still and the bits describe the bottles in front of it.

Run it

Start the line with the Side view on the grinder. A broken bottle reaches the scrap gate; the door swings, the bottle drops into the hopper; the shredder waits (red lamp on the frame) until a box is under the chute, then runs — glass falls into the box, the IN BOX count rises; at four the box conveyor carries the box out and brings the next. Small bottles go down the incline and ride the divert conveyor to the packer's lower lane; large ones run on to the packer's deck infeed. Until step 12 is written nothing packs them, so both fill up: watch `Lower_Count` climb to 8, the next small bottle stop at LS12 and the divert conveyor stop with it. The Simulator Log should show no *no box* or *pushed off with nothing to land on* lines. (In Part IV's own test the line runs three minutes with nothing lost.)

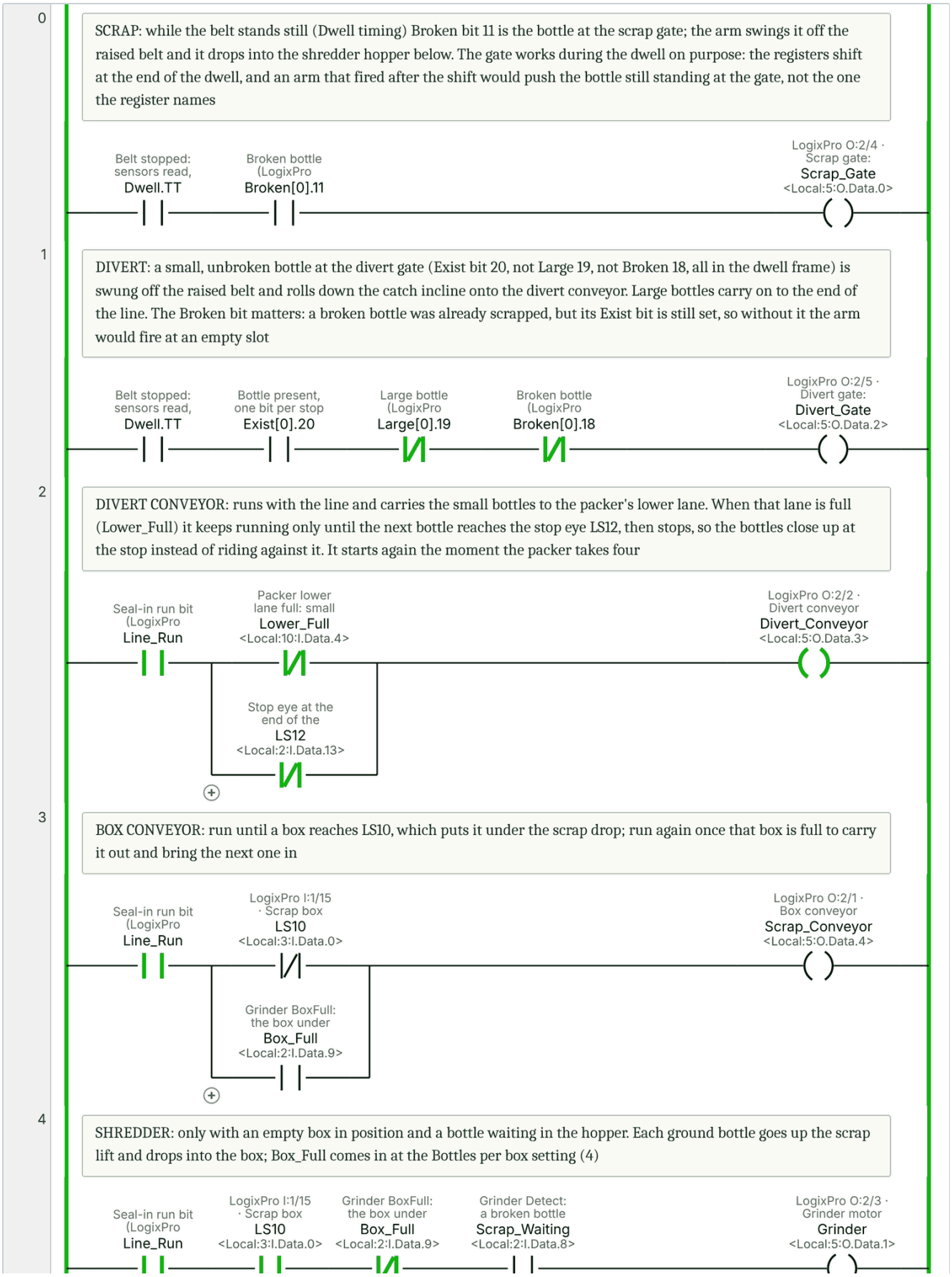

0
SCRAP: while the belt stands still (Dwell timing) Broken bit 11 is the bottle at the scrap gate; the arm swings it off the raised belt and it drops into the shredder hopper below. The gate works during the dwell on purpose: the registers shift at the end of the dwell, and an arm that fired after the shift would push the bottle still standing at the gate, not the one the register names
Belt stopped: sensors read, Dwell.TT
Broken bottle (LogixPro Broken[0].11
LogixPro O:2/4 · Scrap gate: Scrap_Gate <Local:5:O.Data.0>
1
DIVERT: a small, unbroken bottle at the divert gate (Exist bit 20, not Large 19, not Broken 18, all in the dwell frame) is swung off the raised belt and rolls down the catch incline onto the divert conveyor. Large bottles carry on to the end of the line. The Broken bit matters: a broken bottle was already scrapped, but its Exist bit is still set, so without it the arm would fire at an empty slot
Belt stopped: sensors read, Dwell.TT
Bottle present, one bit per stop Exist[0].20
Large bottle (LogixPro Large[0].19
Broken bottle (LogixPro Broken[0].18
LogixPro O:2/5 · Divert gate: Divert_Gate <Local:5:O.Data.2>
2
DIVERT CONVEYOR: runs with the line and carries the small bottles to the packer's lower lane. When that lane is full (Lower_Full) it keeps running only until the next bottle reaches the stop eye LS12, then stops, so the bottles close up at the stop instead of riding against it. It starts again the moment the packer takes four
Seal-in run bit (LogixPro Line_Run
Packer lower lane full: small Lower_Full <Local:10:I.Data.4>
LogixPro O:2/2 · Divert conveyor Divert_Conveyor <Local:5:O.Data.3>
Stop eye at the end of the LS12 <Local:2:I.Data.13>
3
BOX CONVEYOR: run until a box reaches LS10, which puts it under the scrap drop; run again once that box is full to carry it out and bring the next one in
Seal-in run bit (LogixPro Line_Run
LogixPro I:1/15 · Scrap box LS10 <Local:3:I.Data.0>
LogixPro O:2/1 · Box conveyor Scrap_Conveyor <Local:5:O.Data.4>
Grinder BoxFull: the box under Box_Full <Local:2:I.Data.9>
4
SHREDDER: only with an empty box in position and a bottle waiting in the hopper. Each ground bottle goes up the scrap lift and drops into the box; Box_Full comes in at the Bottles per box setting (4)
Seal-in run bit (LogixPro Line_Run
LogixPro I:1/15 · Scrap box LS10 <Local:3:I.Data.0>
Grinder BoxFull: the box under Box_Full <Local:2:I.Data.9>
Grinder Detect: a broken bottle Scrap_Waiting <Local:2:I.Data.8>
LogixPro O:2/3 · Grinder motor Grinder <Local:5:O.Data.1>
Seal-in run bit (LogixPro Line_Run

Figure 129. S8_Sort online, with the finished line running: this is what your routine should look like once every step is in. Green is power flow.

> **TRY IT**
>
> Take `XIC(Dwell.TT)` off rung 1 and watch which bottle the arm pushes. Take `XIO(LS12)` off rung 2 and watch the divert conveyor stop the moment the lane fills, with the last bottles strung out along it. Then change *Bottles per box* on the Grinder to 6 and make the BOXES display (step 9) still count right.

Predict it: the answers

1. The rung is false at `XIC(Dwell.TT)`: `Dwell.TT` (the timing bit of `Dwell`: Belt stopped: sensors read, feeder drops a bottle) is off, so `Scrap_Gate` is **off** (an OTE writes 0 when its rung is false).
2. Every condition on the way is met, so `Divert_Gate` is **on**.
3. The rung is false at `XIC(Line_Run)`: `Line_Run` is off, so `Divert_Conveyor` is **off** (an OTE writes 0 when its rung is false).
4. Every condition on the way is met, so `Scrap_Conveyor` is **on**.
5. The rung is false at `XIO(Box_Full)`: `Box_Full` is on, so `Grinder` is **off** (an OTE writes 0 when its rung is false).

26 Lesson 11 — Step 9: the displays, the preset and Enter

Four counts and a batch stop. Add the JSR and write:

> **IN THE APP**
>
> Guided mode lesson 11 is this chapter: it checks that this lesson's tags are made, that `S9_Count` exists, is called from MainRoutine and verifies, and then watches the line while you run it and ticks each *Run it* result below as it happens. Help → Guided Mode, or start from File → New Project… → Guided: build the bottle line.

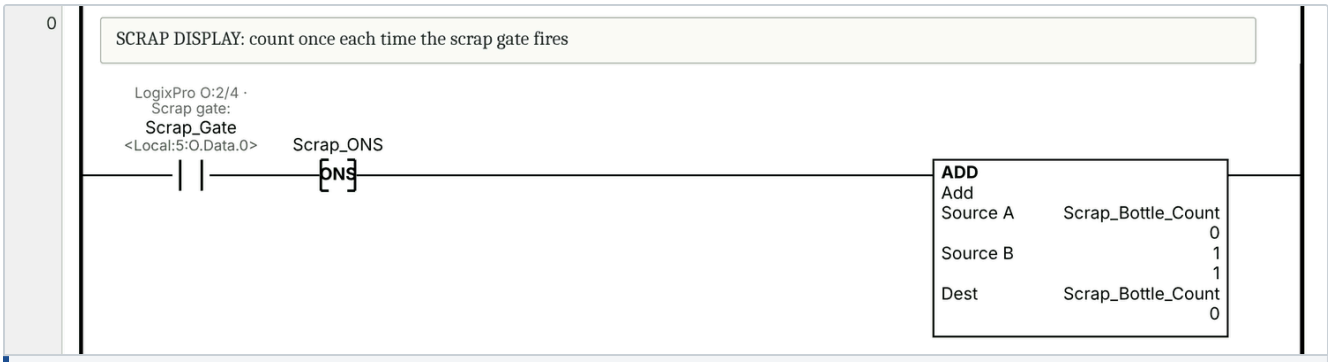
Tags this lesson needs

Create these before you type the rungs (Verify reports any you miss as unknown). Tags from earlier lessons are not repeated.

Tag	Type	How to make it	Description
`Scrap_ONS`	BOOL	Edit Tags: type the name, pick the type	
`Scrap_Bottle_Count`	DINT	Edit Tags: type the name, pick the type	LogixPro O:4/2 · Scrap bottle count, shown on the SCRAP display
`Divert_ONS`	BOOL	Edit Tags: type the name, pick the type	
`Small_Bottle_Count`	DINT	Edit Tags: type the name, pick the type	LogixPro O:4/1 · Small bottle count, shown on the SMALL display
`Large_ONS`	BOOL	Edit Tags: type the name, pick the type	
`Large_Bottle_Count`	DINT	Edit Tags: type the name, pick the type	LogixPro O:4/0 · Large bottle count, shown on the LARGE display
`Box_ONS`	BOOL	Edit Tags: type the name, pick the type	
`Scrap_Box_Count`	DINT	Edit Tags: type the name, pick the type	LogixPro O:4/3 · Scrap box count, shown on the BOXES display
`Thumbwheel`	INT	alias for `Local:9:I.Ch0Data` — the Properties drawer's *tag* link makes it, then rename it	LogixPro I:3 · Thumbwheel switch, read as a number (BCD on the real one)
`Enter`	BOOL	alias for `Local:3:I.Data.13` — the Properties drawer's *tag* link makes it, then rename it	Thumbwheel Enter button

Tag	Type	How to make it	Description
`Enter_ONS`	BOOL	Edit Tags: type the name, pick the type	

RUNG 0




```
XIC(Scrap_Gate)ONS(Scrap_ONS)ADD(Scrap_Bottle_Count,1,Scrap_Bottle_Count);
```

HOW THIS RUNG WORKS

In words. When `Scrap_Gate` (Scrap gate: swings a bottle into the grinder) is on and it is the first scan they are true (the one-shot, remembered in `Scrap_ONS`), the rung makes `Scrap_Bottle_Count` (Scrap bottle count, shown on the SCRAP display) = `Scrap_Bottle_Count` + **1**.

Left to right:

1. `XIC(Scrap_Gate)` *Examine If Closed* asks: true when `Scrap_Gate` (Scrap gate: swings a bottle into the grinder) is on.
2. `ONS(Scrap_ONS)` *One Shot* asks: true when it is the first scan they are true (the one-shot, remembered in `Scrap_ONS`). It passes power for one scan only, the scan its input goes from false to true; its storage bit remembers last scan.
3. `ADD(Scrap_Bottle_Count,1,Scrap_Bottle_Count)` *Add* does: makes `Scrap_Bottle_Count` (Scrap bottle count, shown on the SCRAP display) = `Scrap_Bottle_Count` + **1**. Every scan the rung is true; behind a one-shot it adds once per event.

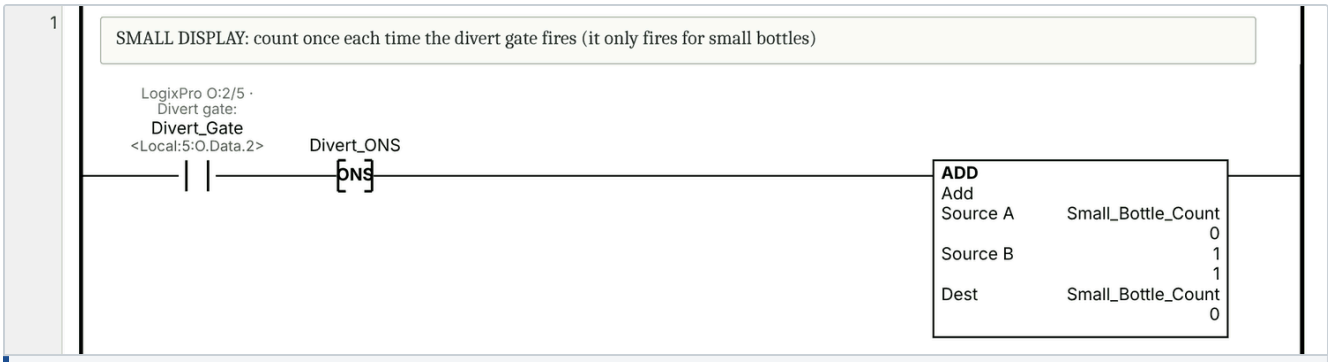
Why it is built this way:

One-shot. Everything after the ONS happens once per event, not on every scan the condition holds; `Scrap_ONS` is only its memory of the last scan.

> **Think of it like this:** a doorbell that rings once however long you lean on the button. The ONS looks at what the button was doing last scan: only the change from up to down rings it.

> **Watch for:** Each ONS needs its own storage bit. Two ONS sharing one bit make one of them miss its edges.

RUNG 1




```
XIC(Divert_Gate)ONS(Divert_ONS)ADD(Small_Bottle_Count,1,Small_Bottle_Count);
```

HOW THIS RUNG WORKS

In words. When `Divert_Gate` (Divert gate: swings a bottle onto the divert conveyor) is on and it is the first scan they are true (the one-shot, remembered in `Divert_ONS`), the rung makes `Small_Bottle_Count` (Small bottle count, shown on the SMALL display) = `Small_Bottle_Count` + **1**.

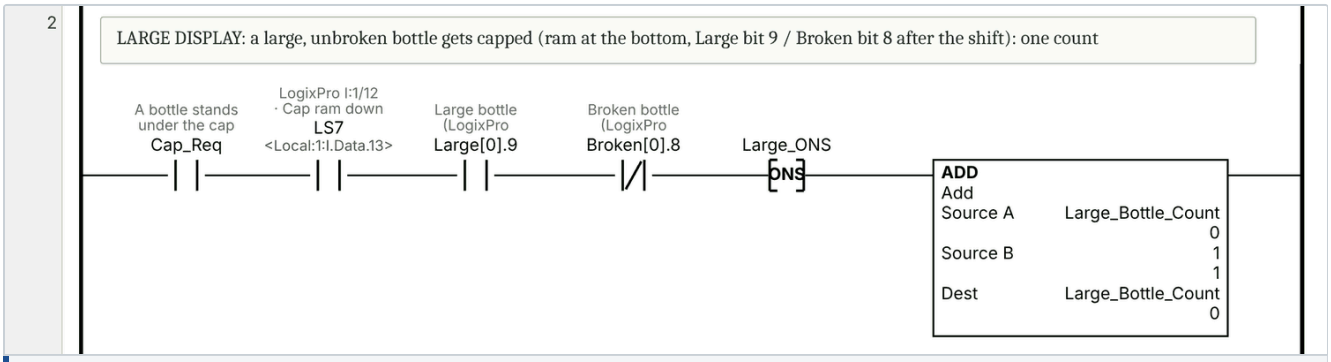
Left to right:

1. `XIC(Divert_Gate)` *Examine If Closed* asks: true when `Divert_Gate` (Divert gate: swings a bottle onto the divert conveyor) is on.
2. `ONS(Divert_ONS)` *One Shot* asks: true when it is the first scan they are true (the one-shot, remembered in `Divert_ONS`).
3. `ADD(Small_Bottle_Count,1,Small_Bottle_Count)` *Add* does: makes `Small_Bottle_Count` (Small bottle count, shown on the SMALL display) = `Small_Bottle_Count` + **1**.

Why it is built this way:

One-shot: once per event, not once per scan.

RUNG 2




```
XIC(Cap_Req)XIC(LS7)XIC(Large[0].9)XIO(Broken[0].8)ONS(Large_ONS)ADD(Large_Bottle_Count,1,Large_Bottle_Count);
```

HOW THIS RUNG WORKS

In words. When `Cap_Req` (A bottle stands under the cap ram and has not been capped) is on, `LS7` (Cap ram down) is on, `Large[0].9` is on, `Broken[0].8` is off and it is the first scan they are true (the one-shot, remembered in `Large_ONS`), the rung makes `Large_Bottle_Count` (Large bottle count, shown on the LARGE display) = `Large_Bottle_Count` + **1**.

Left to right:

1. `XIC(Cap_Req)` *Examine If Closed* asks: true when `Cap_Req` (A bottle stands under the cap ram and has not been capped) is on.
2. `XIC(LS7)` *Examine If Closed* asks: true when `LS7` (Cap ram down) is on.
3. `XIC(Large[0].9)` *Examine If Closed* asks: true when `Large[0].9` is on.
4. `XIO(Broken[0].8)` *Examine If Open* asks: true when `Broken[0].8` is off.
5. `ONS(Large_ONS)` *One Shot* asks: true when it is the first scan they are true (the one-shot, remembered in `Large_ONS`).
6. `ADD(Large_Bottle_Count,1,Large_Bottle_Count)` *Add* does: makes `Large_Bottle_Count` (Large bottle count, shown on the LARGE display) = `Large_Bottle_Count` + **1**.

Why it is built this way:

One-shot: once per event, not once per scan.

RUNG 3

```
XIC(Box_Full)ONS(Box_ONS)ADD(Scrap_Box_Count,1,Scrap_Box_Count);
```

HOW THIS RUNG WORKS

In words. When `Box_Full` (The box under the scrap drop holds its share) is on and it is the first scan they are true (the one-shot, remembered in `Box_ONS`), the rung makes `Scrap_Box_Count` (Scrap box count, shown on the BOXES display) = `Scrap_Box_Count` + **1**.

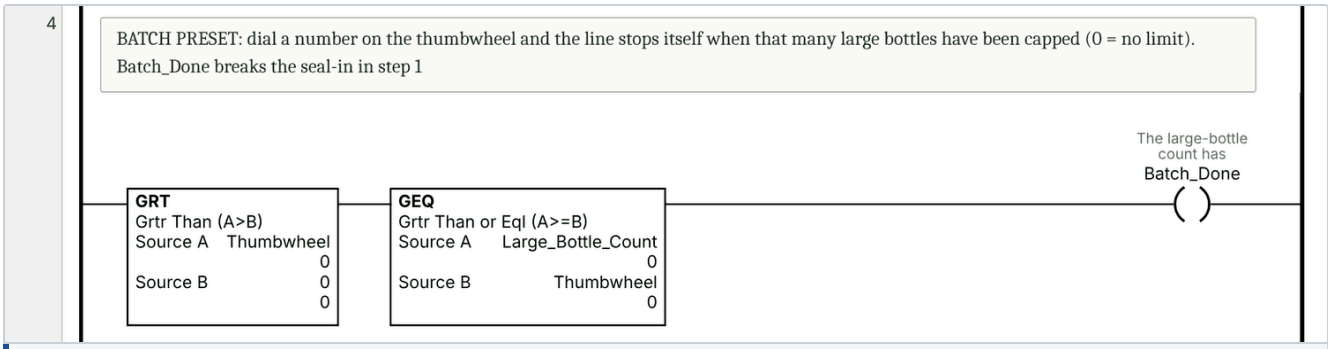
Left to right:

1. `XIC(Box_Full)` *Examine If Closed* asks: true when `Box_Full` (The box under the scrap drop holds its share) is on.
2. `ONS(Box_ONS)` *One Shot* asks: true when it is the first scan they are true (the one-shot, remembered in `Box_ONS`).
3. `ADD(Scrap_Box_Count,1,Scrap_Box_Count)` *Add* does: makes `Scrap_Box_Count` (Scrap box count, shown on the BOXES display) = `Scrap_Box_Count` + **1**.

Why it is built this way:

One-shot: once per event, not once per scan.

RUNG 4




```
GRT(Thumbwheel,0)GEQ(Large_Bottle_Count,Thumbwheel)OTE(Batch_Done);
```

PREDICT IT · 1 `Thumbwheel` is 0 and `Large_Bottle_Count` is 1. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

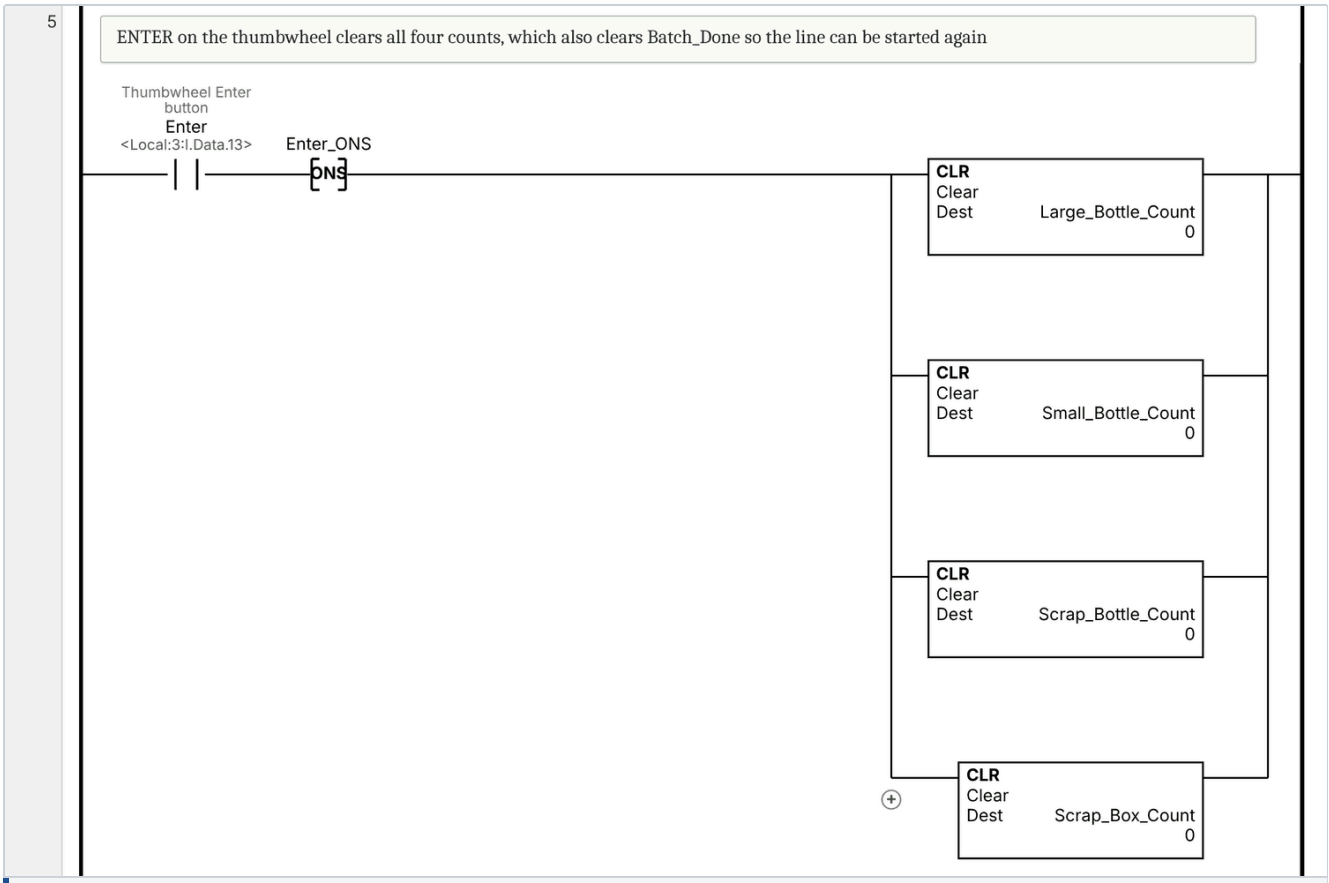
In words. When `Thumbwheel` (Thumbwheel switch, read as a number (BCD on the real one)) is greater than **0** and `Large_Bottle_Count` (Large bottle count, shown on the LARGE display) is at least `Thumbwheel`, the rung turns `Batch_Done` (The large-bottle count has reached the thumbwheel preset) on (and off again whenever the rung is false).

Left to right:

1. `GRT(Thumbwheel,0)` *Greater Than* asks: true when `Thumbwheel` (Thumbwheel switch, read as a number (BCD on the real one)) is greater than **0**.
2. `GEQ(Large_Bottle_Count,Thumbwheel)` *Greater Than or Equal* asks: true when `Large_Bottle_Count` (Large bottle count, shown on the LARGE display) is at least `Thumbwheel`.
3. `OTE(Batch_Done)` *Output Energize* does: turns `Batch_Done` (The large-bottle count has reached the thumbwheel preset) on (and off again whenever the rung is false). Every scan it writes the rung state: true on, false off.

> **Think of it like this:** a rung is a pipe with valves in it, read left to right. Every contact is a valve: XIC is open while its bit is 1, XIO while its bit is 0. Water reaches the output at the right only if every valve in a row is open. The output does not remember anything: on every scan it is simply wet or dry.

RUNG 5




```
XIC(Enter)ONS(Enter_ONS)
[CLR(Large_Bottle_Count),CLR(Small_Bottle_Count),CLR(Scrap_Bottle_Count),CLR(Scrap_Box_Count)];
```

HOW THIS RUNG WORKS

In words. When `Enter` (Thumbwheel Enter button) is on and it is the first scan they are true (the one-shot, remembered in `Enter_ONS`), the rung clears `Large_Bottle_Count` (Large bottle count, shown on the LARGE display) to 0, clears `Small_Bottle_Count` (Small bottle count, shown on the SMALL display) to 0, clears `Scrap_Bottle_Count` (Scrap bottle count, shown on the SCRAP display) to 0 and clears `Scrap_Box_Count` (Scrap box count, shown on the BOXES display) to 0.

Left to right:

1. `XIC(Enter)` *Examine If Closed* asks: true when `Enter` (Thumbwheel Enter button) is on.
2. `ONS(Enter_ONS)` *One Shot* asks: true when it is the first scan they are true (the one-shot, remembered in `Enter_ONS`).
3. A branch of 4 parallel paths: each path is tested on its own and does its own work.

   Path 1

   1. `CLR(Large_Bottle_Count)` *Clear* does: clears `Large_Bottle_Count` (Large bottle count, shown on the LARGE display) to 0.

   Path 2

1. `CLR(Small_Bottle_Count)` *Clear* does: clears `Small_Bottle_Count` (Small bottle count, shown on the SMALL display) to 0.

Path 3

1. `CLR(Scrap_Bottle_Count)` *Clear* does: clears `Scrap_Bottle_Count` (Scrap bottle count, shown on the SCRAP display) to 0.

Path 4

1. `CLR(Scrap_Box_Count)` *Clear* does: clears `Scrap_Box_Count` (Scrap box count, shown on the BOXES display) to 0.

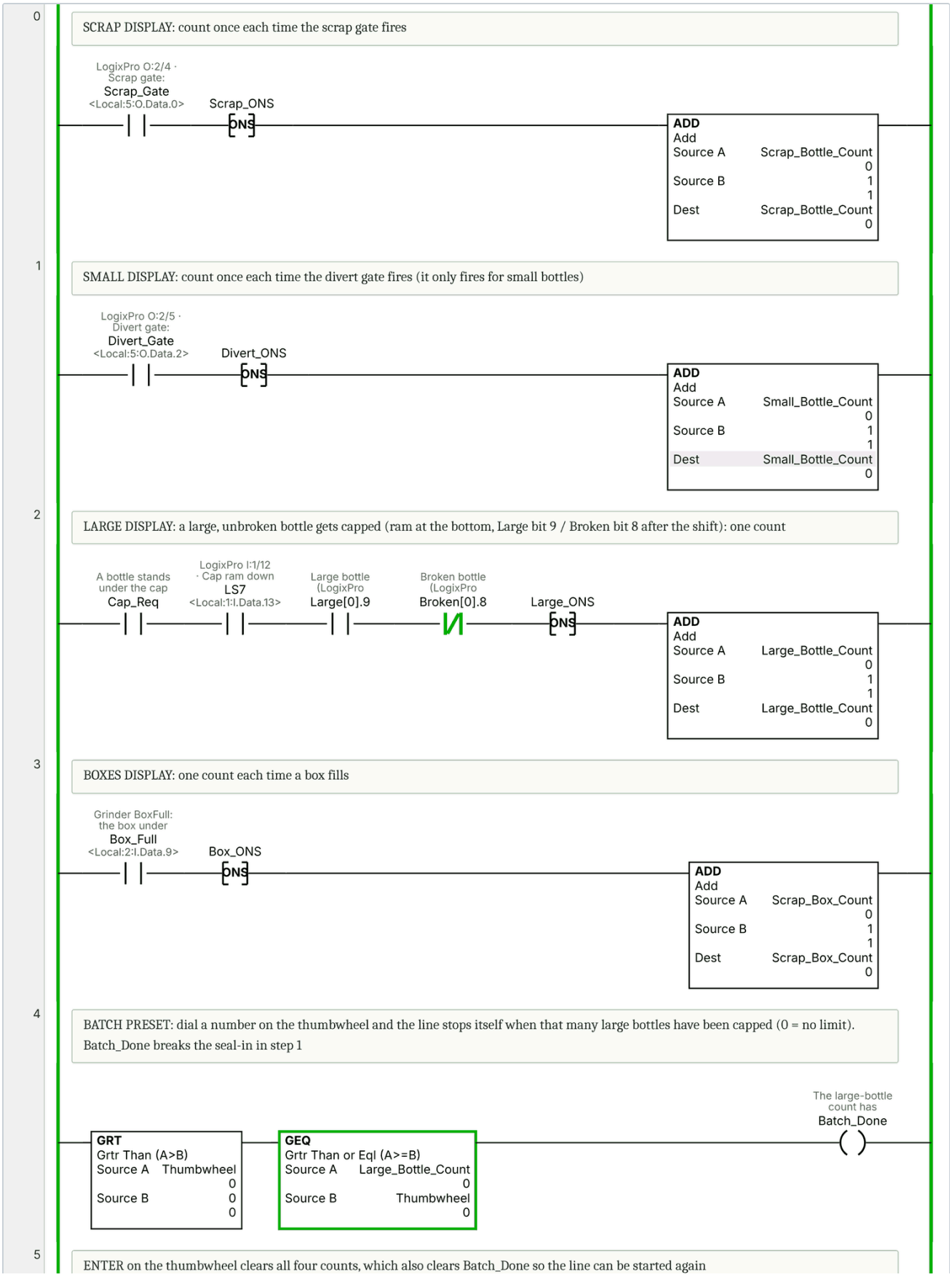
Why it is built this way:

One-shot: once per event, not once per scan.

The four ADD rungs count once per event with a one-shot: the scrap gate firing, the divert gate firing (small bottles), a large unbroken bottle being capped (`Cap_Req` with `LS7`, Large 9, not Broken 8 — after the shift), and a box filling (`Box_Full`). The displays on the panel show those tags, and the toolbar counters read them too. **Rung 4** is the batch preset: a number dialled on the thumbwheel greater than zero, and the large count reaching it, sets `Batch_Done`, which step 1's seal-in breaks on; **rung 5** clears all four counts on Enter, which also clears `Batch_Done` so Start works again.

Run it

Dial 0020 on the thumbwheel and press Start. The line stops itself when the LARGE display reaches 20, the Run lamp goes out. Press Enter: the counts zero. Press Start: it runs again.


0
SCRAP DISPLAY: count once each time the scrap gate fires
LogixPro O:2/4 · Scrap gate:
Scrap_Gate
<Local:5:O.Data.0>
Scrap_ONS
ONS
ADD
Add
Source A Scrap_Bottle_Count
0
Source B 1
1
Dest Scrap_Bottle_Count
0
1
SMALL DISPLAY: count once each time the divert gate fires (it only fires for small bottles)
LogixPro O:2/5 · Divert gate:
Divert_Gate
<Local:5:O.Data.2>
Divert_ONS
ONS
ADD
Add
Source A Small_Bottle_Count
0
Source B 1
1
Dest Small_Bottle_Count
0
2
LARGE DISPLAY: a large, unbroken bottle gets capped (ram at the bottom, Large bit 9 / Broken bit 8 after the shift): one count
A bottle stands under the cap
Cap_Req
LogixPro I:1/12 · Cap ram down
LS7
<Local:1:I.Data.13>
Large bottle (LogixPro
Large[0].9
Broken bottle (LogixPro
Broken[0].8
Large_ONS
ONS
ADD
Add
Source A Large_Bottle_Count
0
Source B 1
1
Dest Large_Bottle_Count
0
3
BOXES DISPLAY: one count each time a box fills
Grinder BoxFull: the box under
Box_Full
<Local:2:I.Data.9>
Box_ONS
ONS
ADD
Add
Source A Scrap_Box_Count
0
Source B 1
1
Dest Scrap_Box_Count
0
4
BATCH PRESET: dial a number on the thumbwheel and the line stops itself when that many large bottles have been capped (0 = no limit). Batch_Done breaks the seal-in in step 1
GRT
Grtr Than (A>B)
Source A Thumbwheel
0
Source B 0
0
GEQ
Grtr Than or Eql (A>=B)
Source A Large_Bottle_Count
0
Source B Thumbwheel
0
The large-bottle count has
Batch_Done
5
ENTER on the thumbwheel clears all four counts, which also clears Batch_Done so the line can be started again

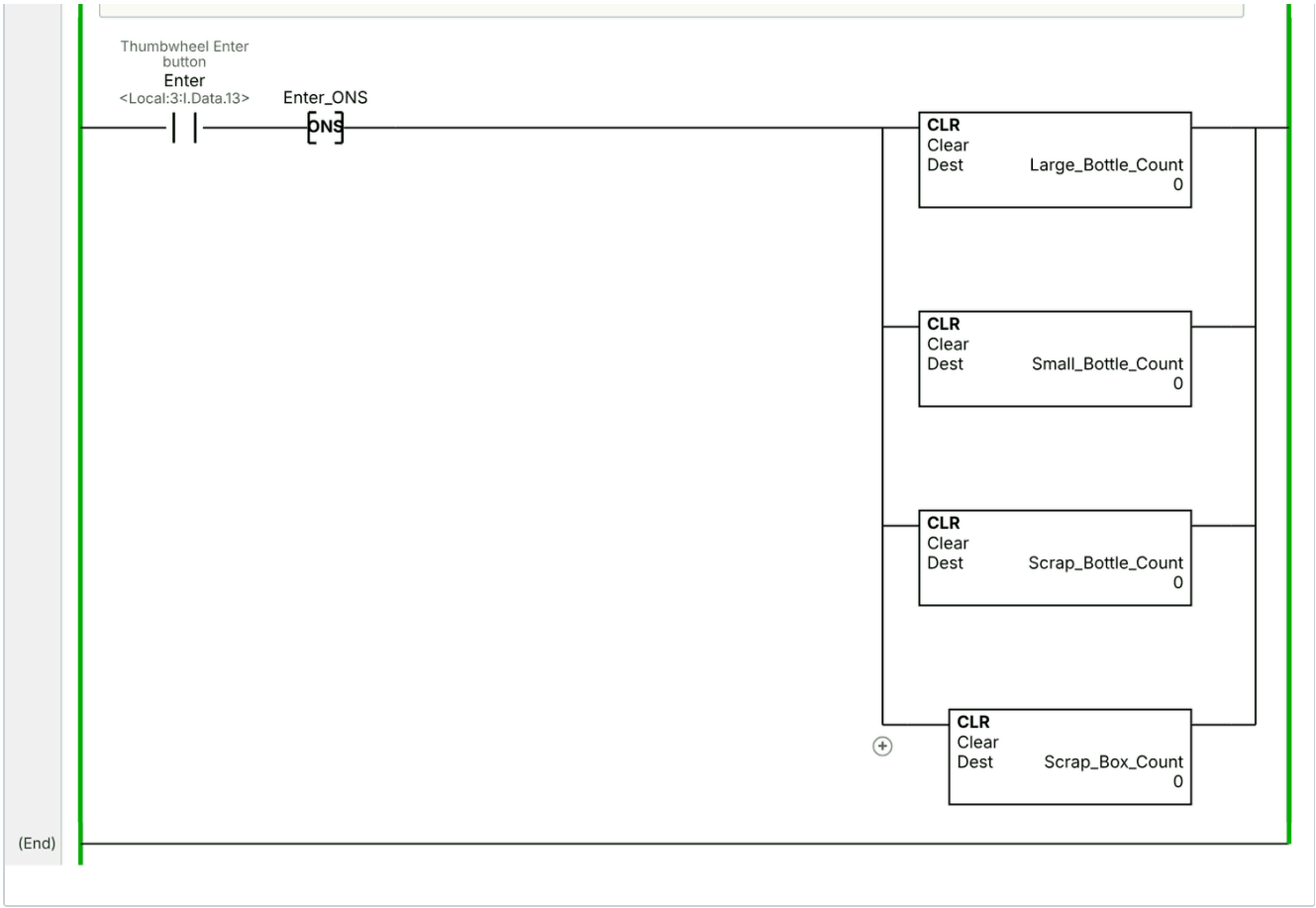


Figure 130. S9_Count online, with the finished line running: this is what your routine should look like once every step is in. Green is power flow.

> **TRY IT**
>
> Count large bottles at the end of the line (LS11's Detect) instead of at the capper, and compare the two counts over a batch.

Predict it: the answers

1. The rung is false at the GRT: `Thumbwheel` is not above **0**, so `Batch_Done` is **off** (an OTE writes 0 when its rung is false).

27 Lesson 12 — Step 10: the labeler, FLS logic

One stop past the capper stands an Inline Labeler, a pressure-sensitive dispenser in the mould of a Novexx FLS 204: a web of labels on a liner runs from the unwind reel over the dispensing edge to the rewind, and an applicator presses the peeled label onto the bottle. Its handshake is the one every dispenser has — **Start** (our `Apply`), **Ready**, **Error**, **Label low** — and the rules are the same as on a real panel: start only when Ready, treat Error as a fault that needs a person, never step the belt mid-cycle. Add the JSR and write:

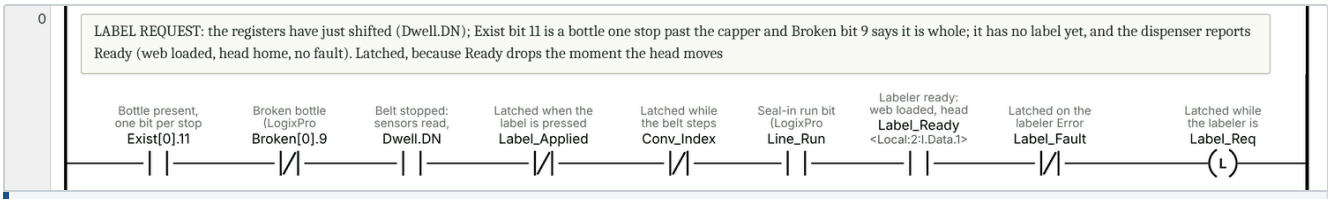
> **IN THE APP**
>
> Guided mode lesson 12 is this chapter: it checks that this lesson's tags are made, that `S10_Label` exists, is called from MainRoutine and verifies, and then watches the line while you run it and ticks each *Run it* result below as it happens. Help → Guided Mode, or start from File → New Project… → Guided: build the bottle line.

Tags this lesson needs

Create these before you type the rungs (Verify reports any you miss as unknown). Tags from earlier lessons are not repeated.

Tag	Type	How to make it	Description
`Label_Applied`	BOOL	Edit Tags: type the name, pick the type	Latched when the label is pressed on; cleared when the belt steps
`Label_Ready`	BOOL	alias for `Local:2:I.Data.1` — the Properties drawer's *tag* link makes it, then rename it	Labeler ready: web loaded, head home, no fault
`Label_Fault`	BOOL	Edit Tags: type the name, pick the type	Latched on the labeler Error input (end of roll or a missed label); Enter clears it
`Label_Apply`	BOOL	alias for `Local:4:O.Data.13` — the Properties drawer's *tag* link makes it, then rename it	Labeler Start: the applicator comes in and presses a label on
`Label_PadIn`	BOOL	alias for `Local:2:I.Data.0` — the Properties drawer's *tag* link makes it, then rename it	Labeler applicator at the end of its stroke (label pressed on)
`Label_Error`	BOOL	alias for `Local:2:I.Data.2` — the Properties drawer's *tag* link makes it, then rename it	Labeler fault: end of roll, or the last cycle found no label
`Label_Low`	BOOL	alias for `Local:2:I.Data.3` — the Properties drawer's *tag* link makes it, then rename it	Labeler roll low
`Label_Green`	BOOL	alias for `Local:4:O.Data.14` — the Properties drawer's *tag* link makes it, then rename it	Labeler green indicator
`Label_Amber`	BOOL	alias for `Local:4:O.Data.15` — the Properties drawer's *tag* link makes it, then rename it	Labeler amber indicator

RUNG 0




```
XIC(Exist[0].11)XIO(Broken[0].9)XIC(Dwell.DN)XIO(Label_Applied)XIO(Conv_Index)X
IC(Line_Run)XIC(Label_Ready)XIO(Label_Fault)OTL(Label_Req);
```

PREDICT IT · 1 `Exist[0].11` is off, `Broken[0].9` is off, `Dwell.DN` (the done bit of `Dwell`: Belt stopped: sensors read, feeder drops a bottle) is on, `Label_Applied` is off, `Conv_Index` is off, `Line_Run` is on, `Label_Ready` is on and `Label_Fault` is off. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Exist[0].11` is on, `Broken[0].9` is off, `Dwell.DN` (the done bit of `Dwell`: Belt stopped: sensors read, feeder drops a bottle) is on, `Label_Applied` (Latched when the label is pressed on; cleared when the belt steps) is off, `Conv_Index` (Latched while the belt steps one pitch) is off, `Line_Run` (Seal-in run bit (LogixPro B3:0/0)) is on, `Label_Ready` (Labeler ready: web loaded, head home, no fault) is on and `Label_Fault` (Latched on the labeler Error input (end of roll or a missed label); Enter clears it) is off, the rung latches `Label_Req` (Latched while the labeler is asked to apply; cleared at PadIn) on.

Left to right:

1. `XIC(Exist[0].11)` *Examine If Closed* asks: true when `Exist[0].11` is on.
2. `XIO(Broken[0].9)` *Examine If Open* asks: true when `Broken[0].9` is off.
3. `XIC(Dwell.DN)` *Examine If Closed* asks: true when `Dwell.DN` (the done bit of `Dwell`: Belt stopped: sensors read, feeder drops a bottle) is on.
4. `XIO(Label_Applied)` *Examine If Open* asks: true when `Label_Applied` (Latched when the label is pressed on; cleared when the belt steps) is off.
5. `XIO(Conv_Index)` *Examine If Open* asks: true when `Conv_Index` (Latched while the belt steps one pitch) is off.
6. `XIC(Line_Run)` *Examine If Closed* asks: true when `Line_Run` (Seal-in run bit (LogixPro B3:0/0)) is on.
7. `XIC(Label_Ready)` *Examine If Closed* asks: true when `Label_Ready` (Labeler ready: web loaded, head home, no fault) is on.
8. `XIO(Label_Fault)` *Examine If Open* asks: true when `Label_Fault` (Latched on the labeler Error input (end of roll or a missed label); Enter clears it) is off.
9. `OTL(Label_Req)` *Output Latch* does: latches `Label_Req` (Latched while the labeler is asked to apply; cleared at PadIn) on. It only ever sets the bit; the bit stays on after the rung goes false.

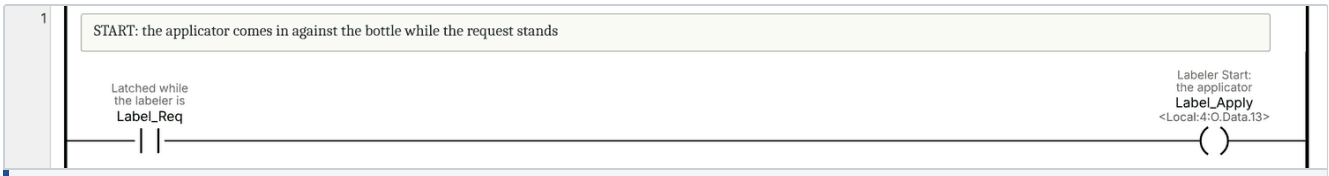
Why it is built this way:

A latch. `Label_Req` stays on after this rung goes false; an `OTU` elsewhere in the program clears it.

> **Think of it like this:** a light switch instead of a doorbell. OTL flips it on and it stays on when you take your hand away; OTU flips it off. Whatever flipped it on does not have to stay there.

> **Watch for:** A latched bit survives the condition that set it: every way in needs a way out, or it stays on for good.

RUNG 1




```
XIC(Label_Req)OTE(Label_Apply);
```

HOW THIS RUNG WORKS

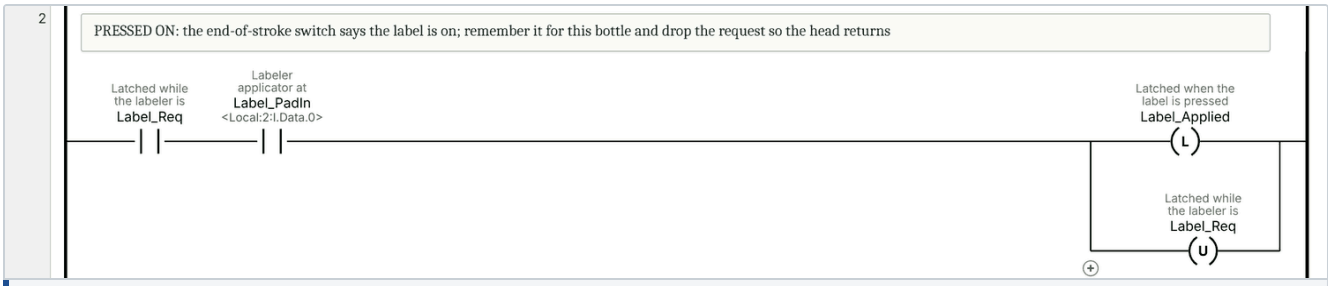
In words. When `Label_Req` (Latched while the labeler is asked to apply; cleared at PadIn) is on, the rung turns `Label_Apply` (Labeler Start: the applicator comes in and presses a label on) on (and off again whenever the rung is false).

Left to right:

1. `XIC(Label_Req)` *Examine If Closed* asks: true when `Label_Req` (Latched while the labeler is asked to apply; cleared at PadIn) is on.
2. `OTE(Label_Apply)` *Output Energize* does: turns `Label_Apply` (Labeler Start: the applicator comes in and presses a label on) on (and off again whenever the rung is false). Every scan it writes the rung state: true on, false off.

> **Think of it like this:** a rung is a pipe with valves in it, read left to right. Every contact is a valve: XIC is open while its bit is 1, XIO while its bit is 0. Water reaches the output at the right only if every valve in a row is open. The output does not remember anything: on every scan it is simply wet or dry.

RUNG 2




```
XIC(Label_Req)XIC(Label_PadIn)[OTL(Label_Applied),OTU(Label_Req)];
```

HOW THIS RUNG WORKS

In words. When `Label_Req` (Latched while the labeler is asked to apply; cleared at PadIn) is on and `Label_PadIn` (Labeler applicator at the end of its stroke (label pressed on)) is on, the rung latches `Label_Applied` (Latched when the label is pressed on; cleared when the belt steps) on and unlatches `Label_Req` (turns it off).

Left to right:

1. `XIC(Label_Req)` *Examine If Closed* asks: true when `Label_Req` (Latched while the labeler is asked to apply; cleared at PadIn) is on.
2. `XIC(Label_PadIn)` *Examine If Closed* asks: true when `Label_PadIn` (Labeler applicator at the end of its stroke (label pressed on)) is on.
3. A branch of 2 parallel paths: each path is tested on its own and does its own work.

   Path 1

   1. `OTL(Label_Applied)` *Output Latch* does: latches `Label_Applied` (Latched when the label is pressed on; cleared when the belt steps) on.

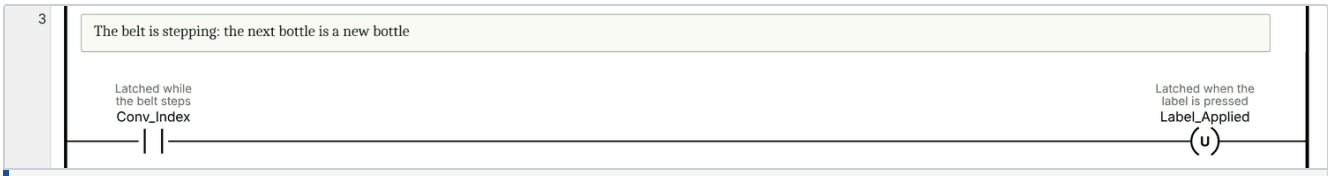
   Path 2

   1. `OTU(Label_Req)` *Output Unlatch* does: unlatches `Label_Req` (turns it off). It only ever clears the bit.

Why it is built this way:

A latch: it stays on until an OTU clears it.

RUNG 3




```
XIC(Conv_Index)OTU(Label_Applied);
```

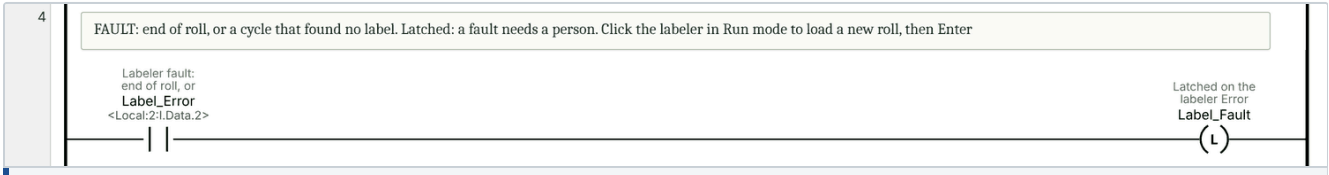
HOW THIS RUNG WORKS

In words. When `Conv_Index` (Latched while the belt steps one pitch) is on, the rung unlatches `Label_Applied` (Latched when the label is pressed on; cleared when the belt steps) (turns it off).

Left to right:

1. `XIC(Conv_Index)` *Examine If Closed* asks: true when `Conv_Index` (Latched while the belt steps one pitch) is on.
2. `OTU(Label_Applied)` *Output Unlatch* does: unlatches `Label_Applied` (Latched when the label is pressed on; cleared when the belt steps) (turns it off).

RUNG 4




```
XIC(Label_Error)OTL(Label_Fault);
```

HOW THIS RUNG WORKS

In words. When `Label_Error` (Labeler fault: end of roll, or the last cycle found no label) is on, the rung latches `Label_Fault` (Latched on the labeler Error input (end of roll or a missed label); Enter clears it) on.

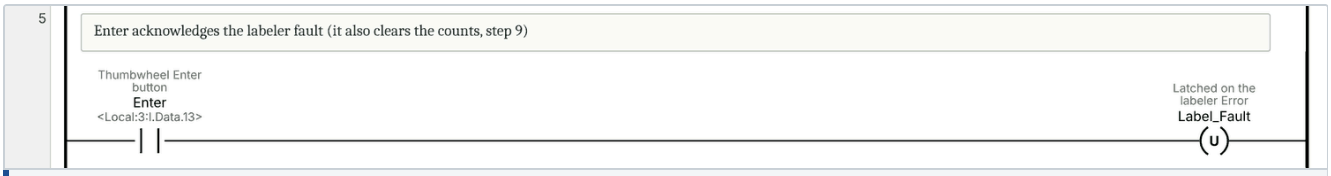
Left to right:

1. `XIC(Label_Error)` *Examine If Closed* asks: true when `Label_Error` (Labeler fault: end of roll, or the last cycle found no label) is on.
2. `OTL(Label_Fault)` *Output Latch* does: latches `Label_Fault` (Latched on the labeler Error input (end of roll or a missed label); Enter clears it) on.

Why it is built this way:

A latch: it stays on until an OTU clears it.

RUNG 5




```
XIC(Enter)OTU(Label_Fault);
```

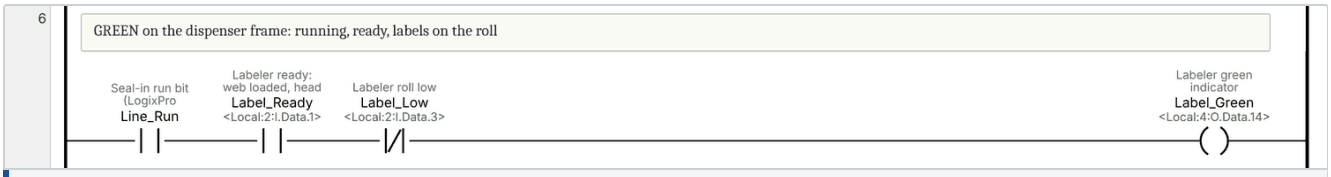
HOW THIS RUNG WORKS

In words. When `Enter` (Thumbwheel Enter button) is on, the rung unlatches `Label_Fault` (Latched on the labeler Error input (end of roll or a missed label); Enter clears it) (turns it off).

Left to right:

1. `XIC(Enter)` *Examine If Closed* asks: true when `Enter` (Thumbwheel Enter button) is on.
2. `OTU(Label_Fault)` *Output Unlatch* does: unlatches `Label_Fault` (Latched on the labeler Error input (end of roll or a missed label); Enter clears it) (turns it off).

RUNG 6




```
XIC(Line_Run)XIC(Label_Ready)XIO(Label_Low)OTE(Label_Green);
```

PREDICT IT · 2 `Line_Run` is off, `Label_Ready` is on and `Label_Low` is off. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Line_Run` (Seal-in run bit (LogixPro B3:0/0)) is on, `Label_Ready` (Labeler ready: web loaded, head home, no fault) is on and `Label_Low` (Labeler roll low) is off, the rung turns `Label_Green` (Labeler green indicator) on (and off again whenever the rung is false).

Left to right:

1. `XIC(Line_Run)` *Examine If Closed* asks: true when `Line_Run` (Seal-in run bit (LogixPro B3:0/0)) is on.
2. `XIC(Label_Ready)` *Examine If Closed* asks: true when `Label_Ready` (Labeler ready: web loaded, head home, no fault) is on.
3. `XIO(Label_Low)` *Examine If Open* asks: true when `Label_Low` (Labeler roll low) is off.
4. `OTE(Label_Green)` *Output Energize* does: turns `Label_Green` (Labeler green indicator) on (and off again whenever the rung is false).

RUNG 7

```
[XIC(Label_Low),XIC(Label_Fault),XIC(Label_Req)]OTE(Label_Amber);
```

PREDICT IT · 3 `Label_Low` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When either (`Label_Low` (Labeler roll low) is on) *or* (`Label_Fault` (Latched on the labeler Error input (end of roll or a missed label); Enter clears it) is on) *or* (`Label_Req` (Latched while the labeler is asked to apply; cleared at PadIn) is on), the rung turns `Label_Amber` (Labeler amber indicator) on (and off again whenever the rung is false).

Left to right:

1. A branch of 3 parallel paths: power gets through if any one of them is true.
   - **Path 1**
     1. `XIC(Label_Low)` *Examine If Closed* asks: true when `Label_Low` (Labeler roll low) is on.
   - **Path 2**
     1. `XIC(Label_Fault)` *Examine If Closed* asks: true when `Label_Fault` (Latched on the labeler Error input (end of roll or a missed label); Enter clears it) is on.
   - **Path 3**
     1. `XIC(Label_Req)` *Examine If Closed* asks: true when `Label_Req` (Latched while the labeler is asked to apply; cleared at PadIn) is on.
2. `OTE(Label_Amber)` *Output Energize* does: turns `Label_Amber` (Labeler amber indicator) on (and off again whenever the rung is false).

> **Think of it like this:** parallel paths are two roads to the same town. If either road is open you get there; the rung only cares that power reaches the far end, not which way it went.

Rung 0 asks for a label in the *after the shift* frame, one bit past the capper (Exist 11, Broken 9), only when the dispenser says Ready — and it is a **latch**, because Ready drops the moment the head leaves home; an OTE here would start the cycle and cancel it in the next scan. **Rung 1** drives Start while the request stands; **rung 2** takes the end-of-stroke switch as "pressed on", remembers it for this bottle and drops the request so the head returns; **rung 3** forgets it when the belt steps. **Rung 4** latches a fault on the dispenser's Error — end of roll, or a cycle that found no label; **rung 5** lets Enter acknowledge it. **Rungs 6–7** are the frame

lights. Step 3's start rung has checked `XIO(Label_Req)` since you wrote it, so the belt never steps with the applicator in, and step 1's housekeeping rung drops the request when the line stops (the head goes home; the bottle is labeled after Start). `Label_Applied` is kept: that bottle has its label.

Run it

Every capped bottle leaves with a label (Side or 3D view: the band on the body; the unwind reel shrinks as the roll goes down). Set the Labeler's *Labels on the roll at start* to 3 and run: three labels, then the Error input, the amber lamp, and `Label_Fault` latched — and the line keeps running, bottles leaving bare, because a labeler fault is not a reason to stop a filler. Click the labeler in Run mode to load a new roll (Ready comes back), press Enter, and labeling resumes.

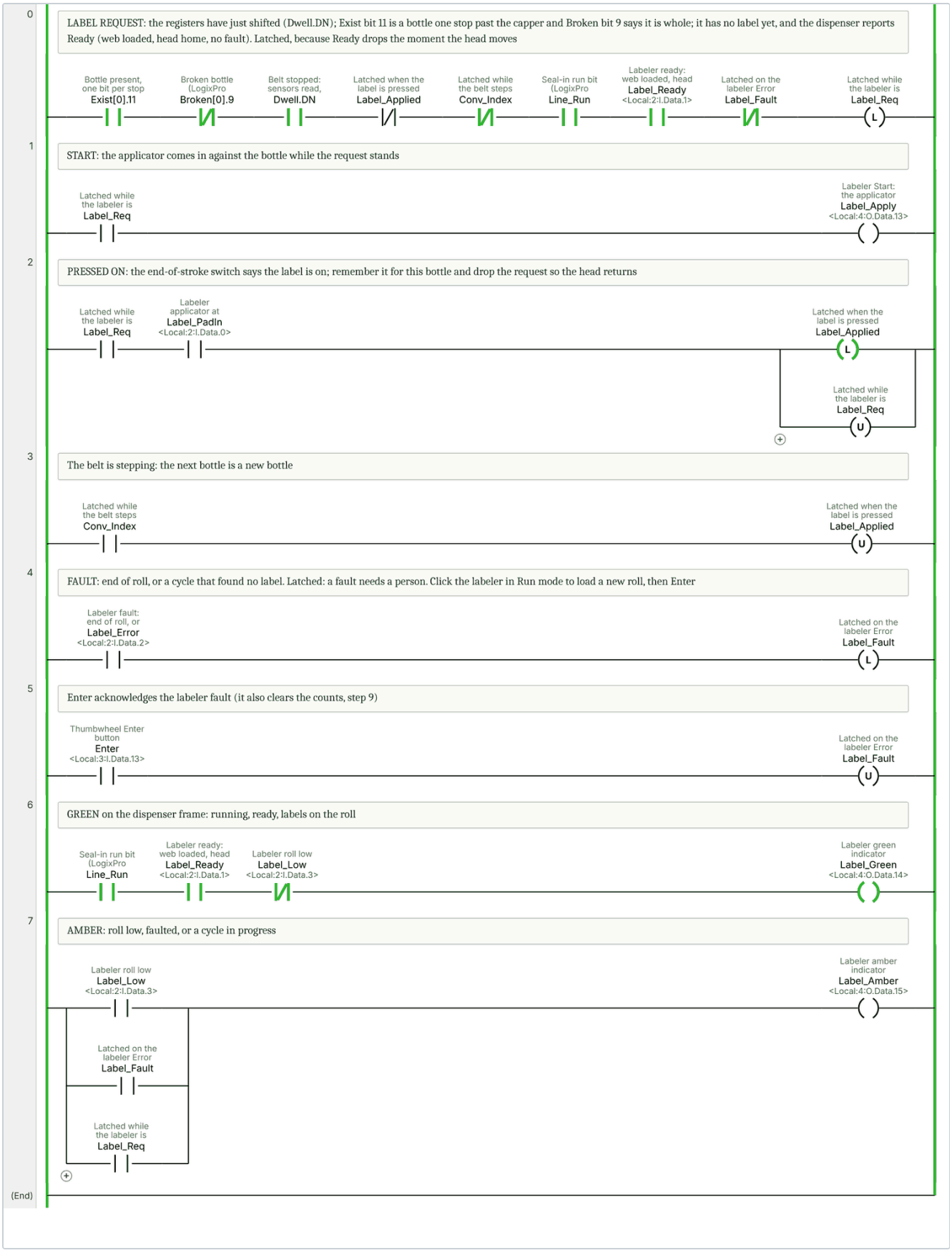

0
LABEL REQUEST: the registers have just shifted (Dwell.DN); Exist bit 11 is a bottle one stop past the capper and Broken bit 9 says it is whole; it has no label yet, and the dispenser reports Ready (web loaded, head home, no fault). Latched, because Ready drops the moment the head moves
Bottle present, one bit per stop
Exist[0].11
Broken bottle (LogixPro
Broken[0].9
Belt stopped: sensors read,
Dwell.DN
Latched when the label is pressed
Label_Applied
Latched while the belt steps
Conv_Index
Seal-in run bit (LogixPro
Line_Run
Labeler ready: web loaded, head
Label_Ready
<Local:2:I.Data.1>
Latched on the labeler Error
Label_Fault
Latched while the labeler is
Label_Req
1
START: the applicator comes in against the bottle while the request stands
Latched while the labeler is
Label_Req
Labeler Start: the applicator
Label_Apply
<Local:4:O.Data.13>
2
PRESSED ON: the end-of-stroke switch says the label is on; remember it for this bottle and drop the request so the head returns
Latched while the labeler is
Label_Req
Labeler applicator at
Label_PadIn
<Local:2:I.Data.0>
Latched when the label is pressed
Label_Applied
Latched while the labeler is
Label_Req
3
The belt is stepping: the next bottle is a new bottle
Latched while the belt steps
Conv_Index
Latched when the label is pressed
Label_Applied
4
FAULT: end of roll, or a cycle that found no label. Latched: a fault needs a person. Click the labeler in Run mode to load a new roll, then Enter
Labeler fault: end of roll, or
Label_Error
<Local:2:I.Data.2>
Latched on the labeler Error
Label_Fault
5
Enter acknowledges the labeler fault (it also clears the counts, step 9)
Thumbwheel Enter button
Enter
<Local:3:I.Data.13>
Latched on the labeler Error
Label_Fault
6
GREEN on the dispenser frame: running, ready, labels on the roll
Seal-in run bit (LogixPro
Line_Run
Labeler ready: web loaded, head
Label_Ready
<Local:2:I.Data.1>
Labeler roll low
Label_Low
<Local:2:I.Data.3>
Labeler green indicator
Label_Green
<Local:4:O.Data.14>
7
AMBER: roll low, faulted, or a cycle in progress
Labeler roll low
Label_Low
<Local:2:I.Data.3>
Labeler amber indicator
Label_Amber
<Local:4:O.Data.15>
Latched on the labeler Error
Label_Fault
Latched while the labeler is
Label_Req
(End)

Figure 131. S10_Label online, with the finished line running: this is what your routine should look like once every step is in. Green is power flow.

> **TRY IT**
>
> Make the fault stop the line instead (add `XIO(Label_Fault)` to step 1's seal-in) and decide which you would want on a real line. Then switch the Labeler's *Hand* to LH and *Applicator* to Tamp pad in Properties and watch the machine rebuild on the other side of the lane — the program does not change.

The second labeler: hazard tape on the scrap boxes

From Version 124 the scrap boxes come off Box_Feed blank, and a second dispenser on the box conveyor, `Haz_Labeler`, bands each one with yellow and black hazard tape on its way to the grinder's chute: a box of broken glass must say so before anyone picks it up. It is the same Inline Labeler part with *What it applies* set to **Hazard tape (scrap boxes)** and a wipe-on applicator: the band is wiped on as the box rolls under the head, half-way through the stroke, so the cycle is started from an eye ten pixels ahead of the head, `LS13`. The handshake is this chapter's, in a routine of its own, `S14_Hazard`, called from MainRoutine under step 12 (step 14 in the JSR list; it has to run before S3_Index like every station). Its `Labeled` input now means *the box under the head carries tape*, so no done bit is kept in the program: the box carries its own.

What changes at the grinder: a box that takes glass with no band on it is logged (*glass into a box with no hazard tape on it*), and a box that leaves at Box_Out with glass and no band is logged as such. Guided mode's run checks for this step want a banded box and glass into a banded box only.

Tags this lesson needs

Create these before you type the rungs (Verify reports any you miss as unknown). Tags from earlier lessons are not repeated.

Tag	Type	How to make it	Description
`LS13`	BOOL	alias for `Local:3:I.Data.2` — the Properties drawer's *tag* link makes it, then rename it	Box at the hazard-tape dispenser
`Haz_Done`	BOOL	alias for `Local:3:I.Data.8` — the Properties drawer's *tag* link makes it, then rename it	The box under the applicator already carries its hazard tape
`Haz_Ready`	BOOL	alias for `Local:3:I.Data.4` — the Properties drawer's *tag* link makes it, then rename it	Hazard-tape dispenser ready: web loaded, applicator home, no fault
`Haz_Fault`	BOOL	Edit Tags: type the name, pick the type	Hazard-tape dispenser fault, latched until Enter
`Haz_Req`	BOOL	Edit Tags: type the name, pick the type	Hazard tape requested for the box at LS13 (latched while the applicator cycles)

Tag	Type	How to make it	Description
`Haz_Apply`	BOOL	alias for `Local:5:O.Data.5` — the Properties drawer's *tag* link makes it, then rename it	Hazard-tape dispenser: start the wipe-on cycle
`Haz_PadIn`	BOOL	alias for `Local:3:I.Data.3` — the Properties drawer's *tag* link makes it, then rename it	Hazard-tape dispenser: applicator at the end of its stroke
`Haz_Error`	BOOL	alias for `Local:3:I.Data.5` — the Properties drawer's *tag* link makes it, then rename it	Hazard-tape dispenser fault: end of roll, or the last cycle found no tape
`Haz_Low`	BOOL	alias for `Local:3:I.Data.6` — the Properties drawer's *tag* link makes it, then rename it	Hazard-tape roll low
`Haz_Green`	BOOL	alias for `Local:5:O.Data.6` — the Properties drawer's *tag* link makes it, then rename it	Hazard-tape dispenser green indicator
`Haz_Amber`	BOOL	alias for `Local:5:O.Data.7` — the Properties drawer's *tag* link makes it, then rename it	Hazard-tape dispenser amber indicator

RUNG 0

```
XIC(LS13)XIO(Haz_Done)XIC(Line_Run)XIC(Haz_Ready)XIO(Haz_Fault)OTL(Haz_Req);
```

PREDICT IT · 4 `LS13` is on, `Haz_Done` is off, `Line_Run` is on, `Haz_Ready` is on and `Haz_Fault` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `LS13` (Box at the hazard-tape dispenser) is on, `Haz_Done` (The box under the applicator already carries its hazard tape) is off, `Line_Run` (Seal-in run bit (LogixPro B3:0/0)) is on, `Haz_Ready` (Hazard-tape dispenser ready: web loaded, applicator home, no fault) is on and `Haz_Fault` (Hazard-tape dispenser fault, latched until Enter) is off, the rung latches `Haz_Req` (Hazard tape requested for the box at LS13 (latched while the applicator cycles)) on.

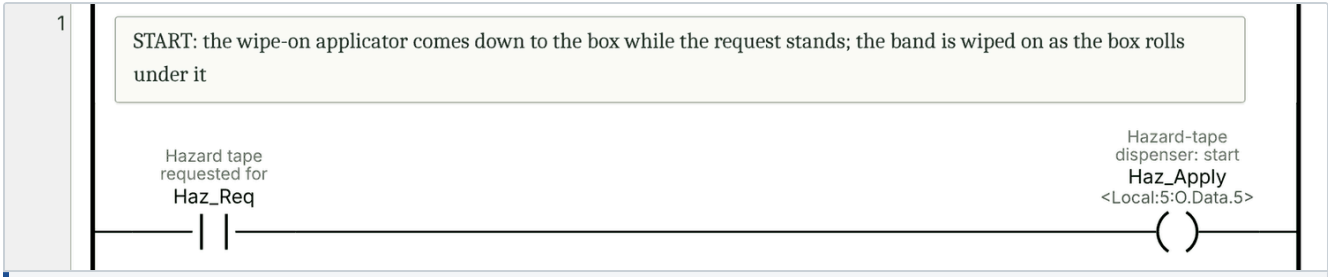
Left to right:

1. `XIC(LS13)` *Examine If Closed* asks: true when `LS13` (Box at the hazard-tape dispenser) is on.
2. `XIO(Haz_Done)` *Examine If Open* asks: true when `Haz_Done` (The box under the applicator already carries its hazard tape) is off.
3. `XIC(Line_Run)` *Examine If Closed* asks: true when `Line_Run` (Seal-in run bit (LogixPro B3:0/0)) is on.
4. `XIC(Haz_Ready)` *Examine If Closed* asks: true when `Haz_Ready` (Hazard-tape dispenser ready: web loaded, applicator home, no fault) is on.
5. `XIO(Haz_Fault)` *Examine If Open* asks: true when `Haz_Fault` (Hazard-tape dispenser fault, latched until Enter) is off.
6. `OTL(Haz_Req)` *Output Latch* does: latches `Haz_Req` (Hazard tape requested for the box at LS13 (latched while the applicator cycles)) on.

Why it is built this way:

A latch: it stays on until an OTU clears it.

RUNG 1




```
XIC(Haz_Req)OTE(Haz_Apply);
```

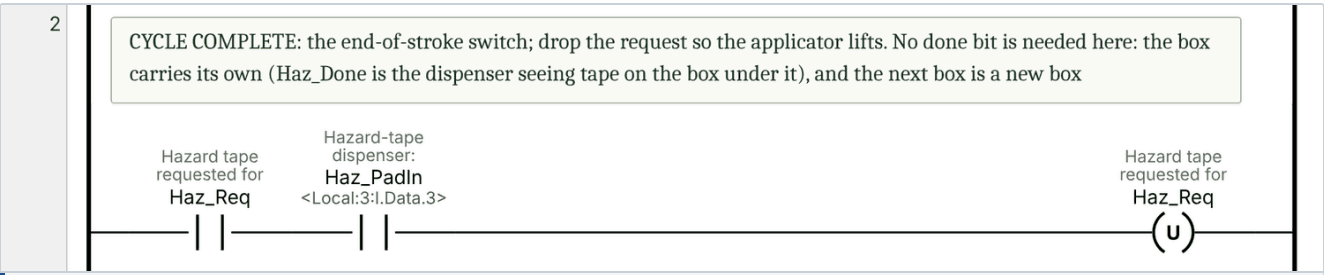
HOW THIS RUNG WORKS

In words. When `Haz_Req` (Hazard tape requested for the box at LS13 (latched while the applicator cycles)) is on, the rung turns `Haz_Apply` (Hazard-tape dispenser: start the wipe-on cycle) on (and off again whenever the rung is false).

Left to right:

1. `XIC(Haz_Req)` *Examine If Closed* asks: true when `Haz_Req` (Hazard tape requested for the box at LS13 (latched while the applicator cycles)) is on.
2. `OTE(Haz_Apply)` *Output Energize* does: turns `Haz_Apply` (Hazard-tape dispenser: start the wipe-on cycle) on (and off again whenever the rung is false).

RUNG 2




```
XIC(Haz_Req)XIC(Haz_PadIn)OTU(Haz_Req);
```

PREDICT IT · 5 `Haz_Req` is on and `Haz_PadIn` is off. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

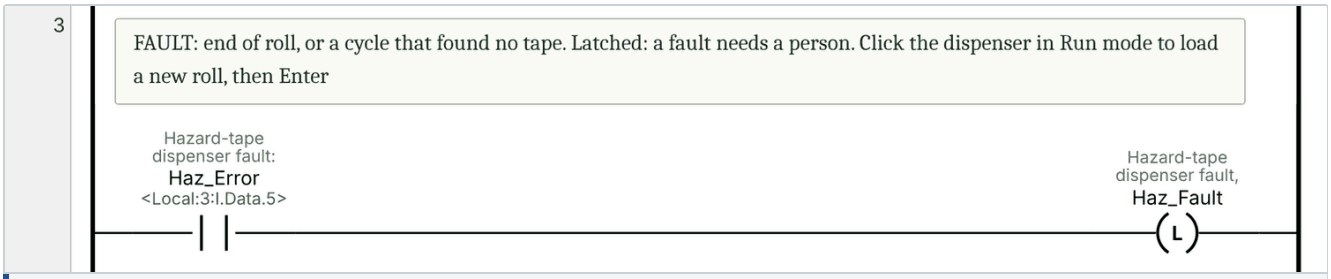
HOW THIS RUNG WORKS

In words. When `Haz_Req` (Hazard tape requested for the box at LS13 (latched while the applicator cycles)) is on and `Haz_PadIn` (Hazard-tape dispenser: applicator at the end of its stroke) is on, the rung unlatches `Haz_Req` (turns it off).

Left to right:

1. `XIC(Haz_Req)` *Examine If Closed* asks: true when `Haz_Req` (Hazard tape requested for the box at LS13 (latched while the applicator cycles)) is on.
2. `XIC(Haz_PadIn)` *Examine If Closed* asks: true when `Haz_PadIn` (Hazard-tape dispenser: applicator at the end of its stroke) is on.
3. `OTU(Haz_Req)` *Output Unlatch* does: unlatches `Haz_Req` (turns it off).

RUNG 3




```
XIC(Haz_Error)OTL(Haz_Fault);
```

HOW THIS RUNG WORKS

In words. When `Haz_Error` (Hazard-tape dispenser fault: end of roll, or the last cycle found no tape) is on, the rung latches `Haz_Fault` (Hazard-tape dispenser fault, latched until Enter) on.

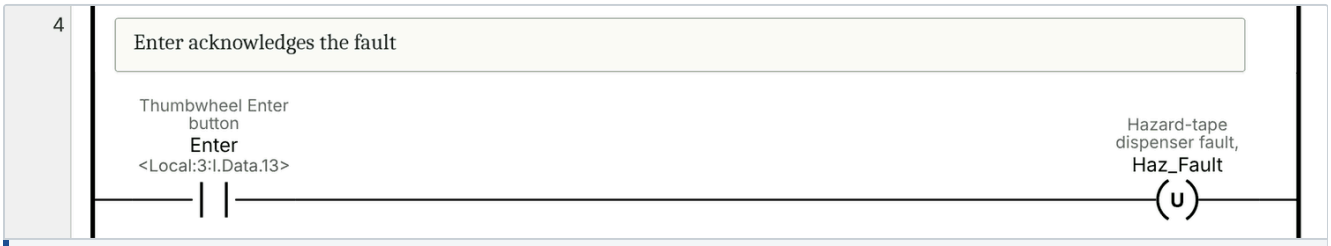
Left to right:

1. `XIC(Haz_Error)` *Examine If Closed* asks: true when `Haz_Error` (Hazard-tape dispenser fault: end of roll, or the last cycle found no tape) is on.
2. `OTL(Haz_Fault)` *Output Latch* does: latches `Haz_Fault` (Hazard-tape dispenser fault, latched until Enter) on.

Why it is built this way:

A latch: it stays on until an OTU clears it.

RUNG 4




```
XIC(Enter)OTU(Haz_Fault);
```

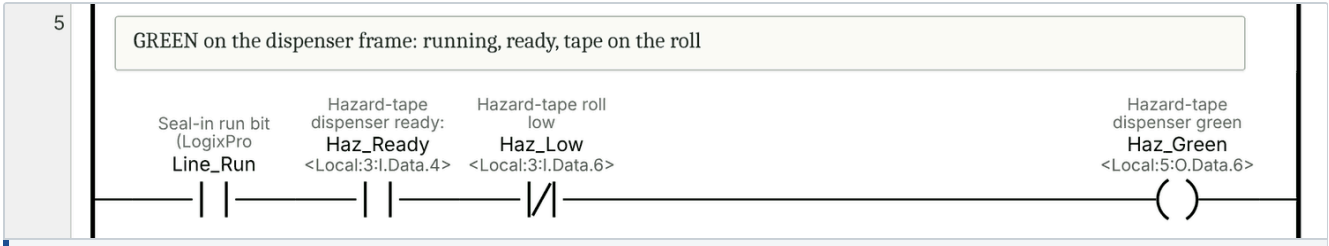
HOW THIS RUNG WORKS

In words. When `Enter` (Thumbwheel Enter button) is on, the rung unlatches `Haz_Fault` (Hazard-tape dispenser fault, latched until Enter) (turns it off).

Left to right:

1. `XIC(Enter)` *Examine If Closed* asks: true when `Enter` (Thumbwheel Enter button) is on.
2. `OTU(Haz_Fault)` *Output Unlatch* does: unlatches `Haz_Fault` (Hazard-tape dispenser fault, latched until Enter) (turns it off).

RUNG 5




```
XIC(Line_Run)XIC(Haz_Ready)XIO(Haz_Low)OTE(Haz_Green);
```

PREDICT IT · 6 `Line_Run` is on, `Haz_Ready` is on and `Haz_Low` is off. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

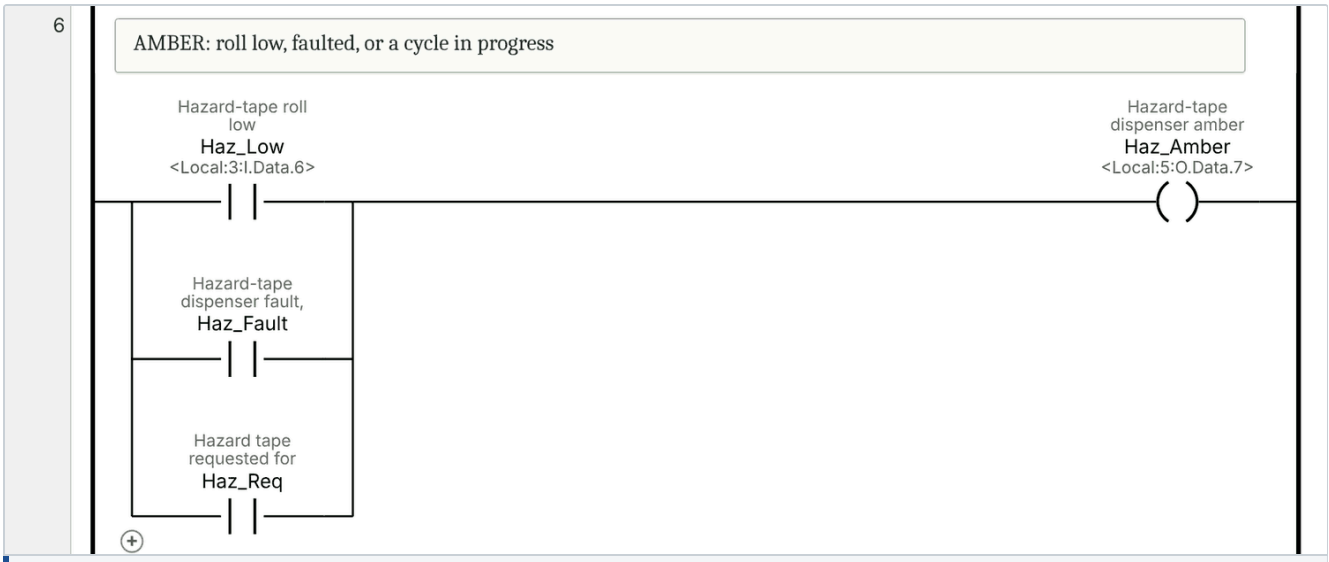
HOW THIS RUNG WORKS

In words. When `Line_Run` (Seal-in run bit (LogixPro B3:0/0)) is on, `Haz_Ready` (Hazard-tape dispenser ready: web loaded, applicator home, no fault) is on and `Haz_Low` (Hazard-tape roll low) is off, the rung turns `Haz_Green` (Hazard-tape dispenser green indicator) on (and off again whenever the rung is false).

Left to right:

1. `XIC(Line_Run)` *Examine If Closed* asks: true when `Line_Run` (Seal-in run bit (LogixPro B3:0/0)) is on.
2. `XIC(Haz_Ready)` *Examine If Closed* asks: true when `Haz_Ready` (Hazard-tape dispenser ready: web loaded, applicator home, no fault) is on.
3. `XIO(Haz_Low)` *Examine If Open* asks: true when `Haz_Low` (Hazard-tape roll low) is off.
4. `OTE(Haz_Green)` *Output Energize* does: turns `Haz_Green` (Hazard-tape dispenser green indicator) on (and off again whenever the rung is false).

RUNG 6




```
[XIC(Haz_Low),XIC(Haz_Fault),XIC(Haz_Req)]OTE(Haz_Amber);
```

PREDICT IT · 7 `Haz_Low` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When either (`Haz_Low` (Hazard-tape roll low) is on) *or* (`Haz_Fault` (Hazard-tape dispenser fault, latched until Enter) is on) *or* (`Haz_Req` (Hazard tape requested for the box at LS13 (latched while the applicator cycles)) is on), the rung turns `Haz_Amber` (Hazard-tape dispenser amber indicator) on (and off again whenever the rung is false).

Left to right:

1. A branch of 3 parallel paths: power gets through if any one of them is true.
   - **Path 1**
     1. `XIC(Haz_Low)` *Examine If Closed* asks: true when `Haz_Low` (Hazard-tape roll low) is on.
   - **Path 2**
     1. `XIC(Haz_Fault)` *Examine If Closed* asks: true when `Haz_Fault` (Hazard-tape dispenser fault, latched until Enter) is on.
   - **Path 3**
     1. `XIC(Haz_Req)` *Examine If Closed* asks: true when `Haz_Req` (Hazard tape requested for the box at LS13 (latched while the applicator cycles)) is on.
2. `OTE(Haz_Amber)` *Output Energize* does: turns `Haz_Amber` (Hazard-tape dispenser amber indicator) on (and off again whenever the rung is false).

Rung 0 latches the request from LS13 for a box with no tape, only while the dispenser is Ready: as in rung 0 above, Ready drops the moment the applicator moves, so a plain OTE would cancel its own cycle. **Rung 1** drives the applicator while the request stands; **rung 2** drops the request at the end-of-stroke switch; there is no done bit to forget, because the next box at LS13 is a new box and `Haz_Done` reads the tape on the box itself. **Rungs 3–4** are the fault and its acknowledge, **rungs 5–6** the frame lamps.

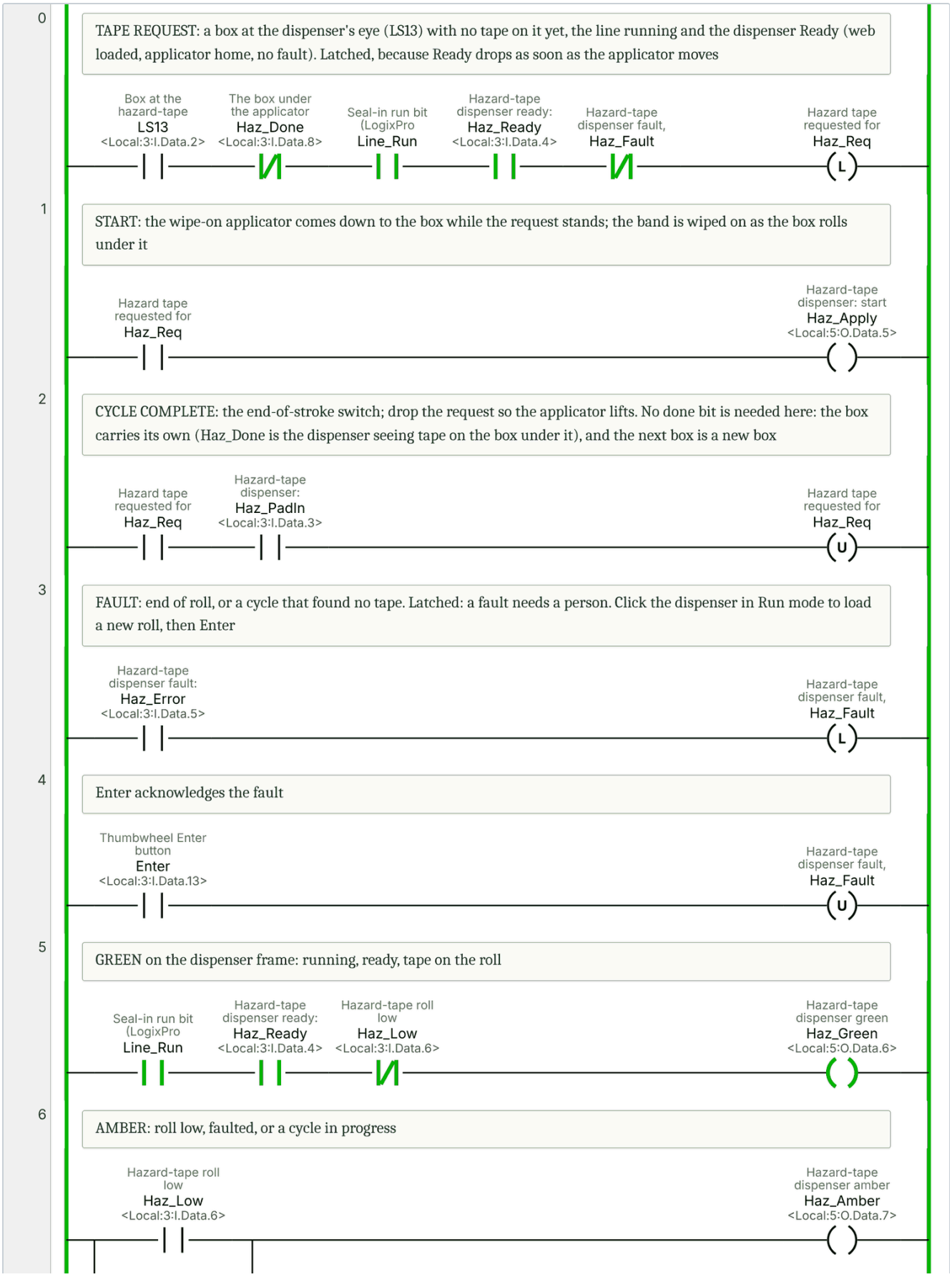

0
TAPE REQUEST: a box at the dispenser's eye (LS13) with no tape on it yet, the line running and the dispenser Ready (web loaded, applicator home, no fault). Latched, because Ready drops as soon as the applicator moves
Box at the hazard-tape
LS13
<Local:3:I.Data.2>
The box under the applicator
Haz_Done
<Local:3:I.Data.8>
Seal-in run bit (LogixPro
Line_Run
Hazard-tape dispenser ready:
Haz_Ready
<Local:3:I.Data.4>
Hazard-tape dispenser fault,
Haz_Fault
Hazard tape requested for
Haz_Req
1
START: the wipe-on applicator comes down to the box while the request stands; the band is wiped on as the box rolls under it
Hazard tape requested for
Haz_Req
Hazard-tape dispenser: start
Haz_Apply
<Local:5:O.Data.5>
2
CYCLE COMPLETE: the end-of-stroke switch; drop the request so the applicator lifts. No done bit is needed here: the box carries its own (Haz_Done is the dispenser seeing tape on the box under it), and the next box is a new box
Hazard tape requested for
Haz_Req
Hazard-tape dispenser:
Haz_PadIn
<Local:3:I.Data.3>
Hazard tape requested for
Haz_Req
3
FAULT: end of roll, or a cycle that found no tape. Latched: a fault needs a person. Click the dispenser in Run mode to load a new roll, then Enter
Hazard-tape dispenser fault:
Haz_Error
<Local:3:I.Data.5>
Hazard-tape dispenser fault,
Haz_Fault
4
Enter acknowledges the fault
Thumbwheel Enter button
Enter
<Local:3:I.Data.13>
Hazard-tape dispenser fault,
Haz_Fault
5
GREEN on the dispenser frame: running, ready, tape on the roll
Seal-in run bit (LogixPro
Line_Run
Hazard-tape dispenser ready:
Haz_Ready
<Local:3:I.Data.4>
Hazard-tape roll low
Haz_Low
<Local:3:I.Data.6>
Hazard-tape dispenser green
Haz_Green
<Local:5:O.Data.6>
6
AMBER: roll low, faulted, or a cycle in progress
Hazard-tape roll low
Haz_Low
<Local:3:I.Data.6>
Hazard-tape dispenser amber
Haz_Amber
<Local:5:O.Data.7>

Figure 132. S14_Hazard online, with the finished line running.

> **TRY IT**
>
> Set Haz_Labeler's *Labels on the roll at start* to 2 and run. Two boxes leave banded, then the Error input, the amber lamp and `Haz_Fault`; the third box reaches the chute blank and the log says *glass into a box with no hazard tape on it*, and again when it leaves at Box_Out. Load a new roll (click the dispenser in Run mode), press Enter. Then move LS13 to x 480 and watch the band miss: the cycle completes before the box is under the head, and the dispenser logs *cycled with no box under it*. An on-the-fly applicator is a timing problem; the eye's place is part of the program.

Predict it: the answers

1. The rung is false at `XIC(Exist[0].11)`: `Exist[0].11` is off, so the OTL does nothing: `Label_Req` keeps whatever it had.
2. The rung is false at `XIC(Line_Run)`: `Line_Run` is off, so `Label_Green` is **off** (an OTE writes 0 when its rung is false).
3. Every condition on the way is met, so `Label_Amber` is **on**.
4. The rung is false at `XIO(Haz_Fault)`: `Haz_Fault` is on, so the OTL does nothing: `Haz_Req` keeps whatever it had.
5. The rung is false at `XIC(Haz_PadIn)`: `Haz_PadIn` is off, so the OTU does nothing: `Haz_Req` keeps whatever it had.
6. Every condition on the way is met, so `Haz_Green` is **on**.
7. Every condition on the way is met, so `Haz_Amber` is **on**.

28 Lesson 13 — Step 11: alarms and machine faults

Everything so far assumes the machine does what it is told. Real machines do not. A bottle jams; a lens gets dirty and the eye stays dark, or stays made; a motor overload trips and the conveyor stops although the output is still on; a label web breaks; a valve sticks shut. The controller is fine through all of this — the scan goes on, no instruction faults — so nothing tells you unless your program checks. This step is the checks.

The template has three parts for it (chapter 16): `LS11`, an eye one stop before the end of the line, a red `Fault_Lamp` and a blue `Reset_PB` on the console. Each belt also reports **Running** — the auxiliary contact on its motor contactor, made when the motor really turns — which the template aliases as `Main_Conv_Running` and `Divert_Conv_Running`. The alarms are the bits of one DINT, `Alarm_Word`, shown in Binary:

Bit	Alarm	Raised when	Stops the line
0	Main conveyor overload	Main_Conveyor commanded, Main_Conv_Running missing for 0.5 s	yes
1	Divert conveyor overload	Divert_Conveyor commanded, Divert_Conv_Running missing for 0.5 s	yes
2	End-of-line check	once per stop, LS11 disagrees with the tracking for that stop: a bottle missing or one extra	yes
3	LS1 dark too long	the line running and LS1 dark for 8 s	yes
4	Labeler fault	Label_Fault (step 10) latched	no
5	Inlet not filling	Supply_Inlet open with Supply_Low still made for 10 s	yes
6	Case lane drive tripped	Case_VFD_Fault (the drive of step 12)	no
7	Packer infeed full	Infeed_Full: eight bottles waiting to be packed	yes

Bits 6 and 7 watch the case packer and its drive, which step 12 programs (chapter 29); their alias tags are already in your list from chapter 17, so type the rungs now and they will make sense two chapters on.

Add the JSR under step 10 and write:

> **IN THE APP**
>
> Guided mode lesson 13 is this chapter: it checks that this lesson's tags are made, that `S11_Alarms` exists, is called from MainRoutine and verifies, and then watches the line while you run it and ticks each *Run it* result below as it happens. Help → Guided Mode, or start from File → New Project… → Guided: build the bottle line.

Tags this lesson needs

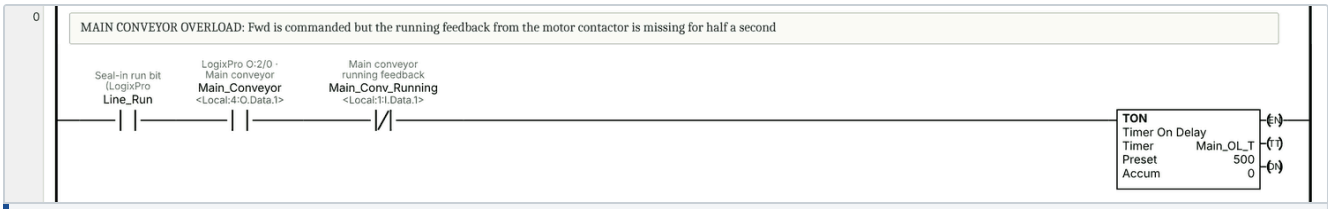
Create these before you type the rungs (Verify reports any you miss as unknown). Tags from earlier lessons are not repeated.

Tag	Type	How to make it	Description
`Main_Conv_Running`	BOOL	alias for `Local:1:I.Data.1` — the Properties drawer's *tag* link makes it, then rename it	Main conveyor running feedback (motor contactor)
`Main_OL_T`	TIMER	Edit Tags: type the name, pick the type	Main conveyor overload timer (no running feedback)
`OL_ONS`	BOOL	Edit Tags: type the name, pick the type	
`Alarm_Word`	DINT	Edit Tags: type the name, pick the type, Style Binary	Latched alarms, one bit each: 0 main conveyor overload, 1 divert conveyor overload, 2 end-of-line check failed (a jam, a lost bottle, tracking out of step), 3 LS1 dark too long (dirty lens or feeder empty), 4 labeler fault, 5 reservoir inlet open but level not rising, 6 case lane drive tripped, 7 packer infeed full
`Alarm_Unack`	BOOL	Edit Tags: type the name, pick the type	New alarm, not yet acknowledged (Reset)
`Divert_Conv_Running`	BOOL	alias for `Local:2:I.Data.12` — the Properties drawer's *tag* link makes it, then rename it	Divert conveyor running feedback
`Div_OL_T`	TIMER	Edit Tags: type the name, pick the type	Divert conveyor overload timer (no running feedback)
`DOL_ONS`	BOOL	Edit Tags: type the name, pick the type	
`EOL_Inhibit`	BOOL	Edit Tags: type the name, pick the type	Tracking cleared by Reset: the end-of-line check waits until the first bottle tracked since then reaches LS11
`EOL_ONS`	BOOL	Edit Tags: type the name, pick the type	

Tag	Type	How to make it	Description
`LS11`	BOOL	alias for `Local:3:I.Data.11` — the Properties drawer's *tag* link makes it, then rename it	End-of-line eye at 1240: verifies the tracking (only large, whole bottles reach it)
`EOL_Missing`	BOOL	Edit Tags: type the name, pick the type	LS11 dark when the registers said a large, whole bottle was there
`EOL_Extra`	BOOL	Edit Tags: type the name, pick the type	LS11 saw a bottle the registers did not expect
`BLK_ONS`	BOOL	Edit Tags: type the name, pick the type	
`LS1_Dark_T`	TIMER	Edit Tags: type the name, pick the type	LS1 dark too long timer
`DRK_ONS`	BOOL	Edit Tags: type the name, pick the type	
`LBL_ONS`	BOOL	Edit Tags: type the name, pick the type	
`Inlet_T`	TIMER	Edit Tags: type the name, pick the type	Inlet not filling timer
`INL_ONS`	BOOL	Edit Tags: type the name, pick the type	
`Case_VFD_Fault`	BOOL	alias for `Local:10:I.Data.13` — the Properties drawer's *tag* link makes it, then rename it	Case lane drive tripped
`VFD_ONS`	BOOL	Edit Tags: type the name, pick the type	
`Infeed_Full`	BOOL	alias for `Local:10:I.Data.0` — the Properties drawer's *tag* link makes it, then rename it	Packer infeed buffer full: bottles arriving now wait on the belt
`INF_ONS`	BOOL	Edit Tags: type the name, pick the type	
`Reset_PB`	BOOL	alias for `Local:3:I.Data.12` — the Properties drawer's *tag* link makes it, then rename it	Blue Reset button: acknowledge alarms and clear the ones whose cause is gone
`RST_ONS`	BOOL	Edit Tags: type the name, pick the type	
`Flash_T`	TIMER	Edit Tags: type the name, pick the type	Fault lamp flasher

Tag	Type	How to make it	Description
`Fault_Lamp`	BOOL	alias for `Local:5:O.Data.9` — the Properties drawer's *tag* link makes it, then rename it	Red fault lamp on the console: flashing = unacknowledged alarm, steady = alarm still active

RUNG 0




```
XIC(Line_Run)XIC(Main_Conveyor)XIO(Main_Conv_Running)TON(Main_OL_T,500,0);
```

PREDICT IT · 1 `Line_Run` is off, `Main_Conveyor` is on and `Main_Conv_Running` is off. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Line_Run` (Seal-in run bit (LogixPro B3:0/0)) is on, `Main_Conveyor` (Main conveyor motor) is on and `Main_Conv_Running` (Main conveyor running feedback (motor contactor)) is off, the rung runs the timer `Main_OL_T` (Main conveyor overload timer (no running feedback)) toward 500 ms (0.5 s).

Left to right:

1. `XIC(Line_Run)` *Examine If Closed* asks: true when `Line_Run` (Seal-in run bit (LogixPro B3:0/0)) is on.
2. `XIC(Main_Conveyor)` *Examine If Closed* asks: true when `Main_Conveyor` (Main conveyor motor) is on.
3. `XIO(Main_Conv_Running)` *Examine If Open* asks: true when `Main_Conv_Running` (Main conveyor running feedback (motor contactor)) is off.
4. `TON(Main_OL_T,500,0)` *Timer On Delay* does: runs the timer `Main_OL_T` (Main conveyor overload timer (no running feedback)) toward 500 ms (0.5 s). The accumulator counts while the rung is true; .DN comes on at the preset; the moment the rung goes false the timer resets.

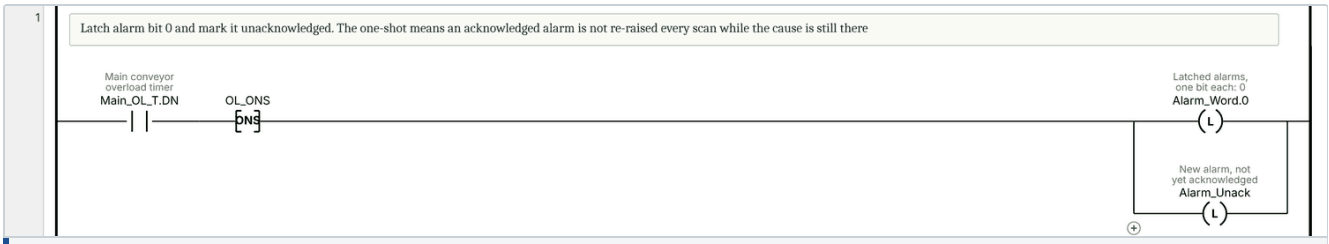
Why it is built this way:

A timer. `Main_OL_T` only times while this rung is true; other rungs use its `.DN` (done) or `.TT` (timing) bits. Being a TON, it starts again from zero every time the rung goes false.

> **Think of it like this:** a kitchen timer that only runs while you hold your finger on it. Take your finger off and it snaps back to zero. Hold it long enough and the done light comes on, and stays on as long as your finger is still there.

> **Watch for:** If anything in series flickers false for one scan, a TON starts over from zero. Use RTO when the time must survive an interruption.

RUNG 1




```
XIC(Main_OL_T.DN)ONS(OL_ONS)[OTL(Alarm_Word.0),OTL(Alarm_Unack)];
```

HOW THIS RUNG WORKS

In words. When `Main_OL_T.DN` (the done bit of `Main_OL_T` : Main conveyor overload timer (no running feedback)) is on and it is the first scan they are true (the one-shot, remembered in `OL_ONS`), the rung latches `Alarm_Word.0` on and latches `Alarm_Unack` (New alarm, not yet acknowledged (Reset)) on.

Left to right:

1. `XIC(Main_OL_T.DN)` *Examine If Closed* asks: true when `Main_OL_T.DN` (the done bit of `Main_OL_T` : Main conveyor overload timer (no running feedback)) is on.
2. `ONS(OL_ONS)` *One Shot* asks: true when it is the first scan they are true (the one-shot, remembered in `OL_ONS`). It passes power for one scan only, the scan its input goes from false to true; its storage bit remembers last scan.
3. A branch of 2 parallel paths: each path is tested on its own and does its own work.

   Path 1

   1. `OTL(Alarm_Word.0)` *Output Latch* does: latches `Alarm_Word.0` on. It only ever sets the bit; the bit stays on after the rung goes false.

   Path 2

   1. `OTL(Alarm_Unack)` *Output Latch* does: latches `Alarm_Unack` (New alarm, not yet acknowledged (Reset)) on.

Why it is built this way:

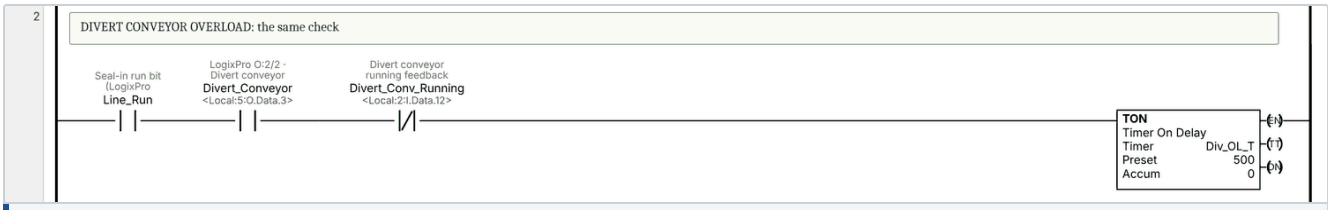
One-shot. Everything after the ONS happens once per event, not on every scan the condition holds; `OL_ONS` is only its memory of the last scan.

A latch. `Alarm_Word.0` stays on after this rung goes false; an `OTU` elsewhere in the program clears it.

> **Think of it like this:** a doorbell that rings once however long you lean on the button. The ONS looks at what the button was doing last scan: only the change from up to down rings it.

> **Watch for:** Each ONS needs its own storage bit. Two ONS sharing one bit make one of them miss its edges.

RUNG 2




```
XIC(Line_Run)XIC(Divert_Conveyor)XIO(Divert_Conv_Running)TON(Div_OL_T,500,0);
```

PREDICT IT · 2 `Line_Run` is on, `Divert_Conveyor` is off and `Divert_Conv_Running` is off. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Line_Run` (Seal-in run bit (LogixPro B3:0/0)) is on, `Divert_Conveyor` (Divert conveyor for large bottles) is on and `Divert_Conv_Running` (Divert conveyor running feedback) is off, the rung runs the timer `Div_OL_T` (Divert conveyor overload timer (no running feedback)) toward 500 ms (0.5 s).

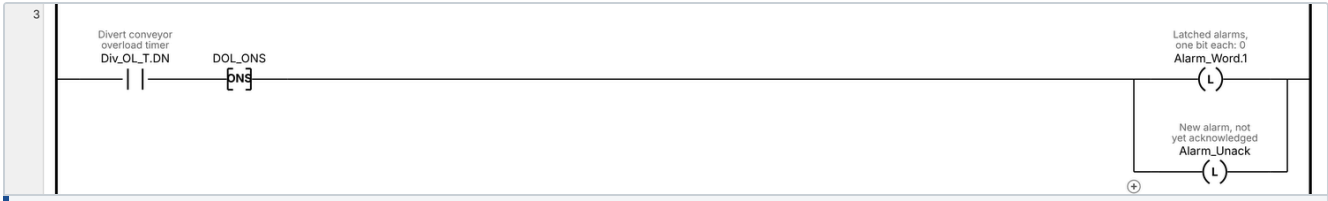
Left to right:

1. `XIC(Line_Run)` *Examine If Closed* asks: true when `Line_Run` (Seal-in run bit (LogixPro B3:0/0)) is on.
2. `XIC(Divert_Conveyor)` *Examine If Closed* asks: true when `Divert_Conveyor` (Divert conveyor for large bottles) is on.
3. `XIO(Divert_Conv_Running)` *Examine If Open* asks: true when `Divert_Conv_Running` (Divert conveyor running feedback) is off.
4. `TON(Div_OL_T,500,0)` *Timer On Delay* does: runs the timer `Div_OL_T` (Divert conveyor overload timer (no running feedback)) toward 500 ms (0.5 s).

Why it is built this way:

A timer: it times only while this rung is true.

RUNG 3




```
XIC(Div_OL_T.DN)ONS(DOL_ONS)[OTL(Alarm_Word.1),OTL(Alarm_Unack)];
```

HOW THIS RUNG WORKS

In words. When `Div_OL_T.DN` (the done bit of `Div_OL_T` : Divert conveyor overload timer (no running feedback)) is on and it is the first scan they are true (the one-shot, remembered in `DOL_ONS`), the rung latches `Alarm_Word.1` on and latches `Alarm_Unack` (New alarm, not yet acknowledged (Reset)) on.

Left to right:

1. `XIC(Div_OL_T.DN)` *Examine If Closed* asks: true when `Div_OL_T.DN` (the done bit of `Div_OL_T` : Divert conveyor overload timer (no running feedback)) is on.
2. `ONS(DOL_ONS)` *One Shot* asks: true when it is the first scan they are true (the one-shot, remembered in `DOL_ONS`).
3. A branch of 2 parallel paths: each path is tested on its own and does its own work.

   Path 1

   1. `OTL(Alarm_Word.1)` *Output Latch* does: latches `Alarm_Word.1` on.

   Path 2

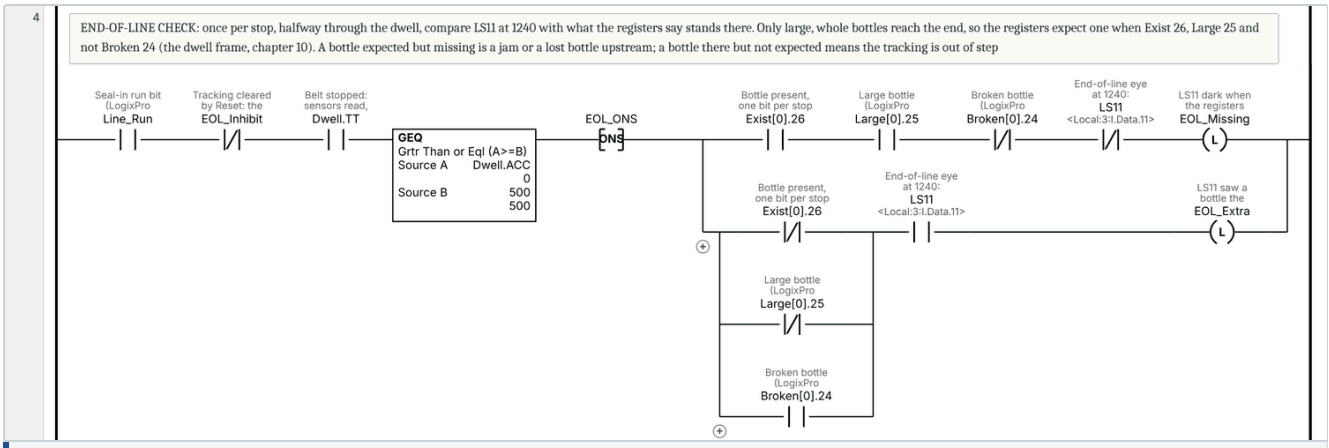
   1. `OTL(Alarm_Unack)` *Output Latch* does: latches `Alarm_Unack` (New alarm, not yet acknowledged (Reset)) on.

Why it is built this way:

One-shot: once per event, not once per scan.

A latch: it stays on until an OTU clears it.

RUNG 4




```
XIC(Line_Run)XIO(EOL_Inhibit)XIC(Dwell.TT)GEQ(Dwell.ACC,500)ONS(EOL_ONS)
[XIC(Exist[0].26)XIC(Large[0].25)XIO(Broken[0].24)XIO(LS11)OTL(EOL_Missing),
[XIO(Exist[0].26),XIO(Large[0].25),XIC(Broken[0].24)]XIC(LS11)OTL(EOL_Extra)];
```

HOW THIS RUNG WORKS

In words. When `Line_Run` (Seal-in run bit (LogixPro B3:0/0)) is on, `EOL_Inhibit` (Tracking cleared by Reset: the end-of-line check waits until the first bottle tracked since then reaches LS11) is off, `Dwell.TT` (the timing bit of `Dwell`: Belt stopped: sensors read, feeder drops a bottle) is on, `Dwell.ACC` (the accumulated value of `Dwell`) is at least **500** and it is the first scan they are true (the one-shot, remembered in `EOL_ONS`), the rung picks its path: if `Exist[0].26` is on, `Large[0].25` is on, `Broken[0].24` is off and `LS11` (End-of-line eye at 1240: verifies the tracking (only large, whole bottles reach it)) is off, it latches `EOL_Missing` (LS11 dark when the registers said a large, whole bottle was there) on; if either (`Exist[0].26` is off) *or* (`Large[0].25` is off) *or* (`Broken[0].24` is on) and `LS11` is on, it latches `EOL_Extra` (LS11 saw a bottle the registers did not expect) on.

Left to right:

1. `XIC(Line_Run)` *Examine If Closed* asks: true when `Line_Run` (Seal-in run bit (LogixPro B3:0/0)) is on.
2. `XIO(EOL_Inhibit)` *Examine If Open* asks: true when `EOL_Inhibit` (Tracking cleared by Reset: the end-of-line check waits until the first bottle tracked since then reaches LS11) is off.
3. `XIC(Dwell.TT)` *Examine If Closed* asks: true when `Dwell.TT` (the timing bit of `Dwell`: Belt stopped: sensors read, feeder drops a bottle) is on.
4. `GEQ(Dwell.ACC,500)` *Greater Than or Equal* asks: true when `Dwell.ACC` (the accumulated value of `Dwell`) is at least **500**.
5. `ONS(EOL_ONS)` *One Shot* asks: true when it is the first scan they are true (the one-shot, remembered in `EOL_ONS`).
6. A branch of 2 parallel paths: each path is tested on its own and does its own work.

   Path 1

   1. `XIC(Exist[0].26)` *Examine If Closed* asks: true when `Exist[0].26` is on.
   2. `XIC(Large[0].25)` *Examine If Closed* asks: true when `Large[0].25` is on.
   3. `XIO(Broken[0].24)` *Examine If Open* asks: true when `Broken[0].24` is off.
   4. `XIO(LS11)` *Examine If Open* asks: true when `LS11` (End-of-line eye at 1240: verifies the tracking (only large, whole bottles reach it)) is off.
   5. `OTL(EOL_Missing)` *Output Latch* does: latches `EOL_Missing` (LS11 dark when the registers said a large, whole bottle was there) on.

Path 2

1. A branch of 3 parallel paths: power gets through if any one of them is true.

   Path 1

   1. `XIO(Exist[0].26)` *Examine If Open* asks: true when `Exist[0].26` is off.

   Path 2

   1. `XIO(Large[0].25)` *Examine If Open* asks: true when `Large[0].25` is off.

   Path 3

   1. `XIC(Broken[0].24)` *Examine If Closed* asks: true when `Broken[0].24` is on.

2. `XIC(LS11)` *Examine If Closed* asks: true when `LS11` is on.
3. `OTL(EOL_Extra)` *Output Latch* does: latches `EOL_Extra` (LS11 saw a bottle the registers did not expect) on.

Why it is built this way:

One-shot: once per event, not once per scan.

A latch: it stays on until an OTU clears it.

RUNG 5

```
[XIC(EOL_Missing),XIC(EOL_Extra)]ONS(BLK_ONS)
[OTL(Alarm_Word.2),OTL(Alarm_Unack)];
```

HOW THIS RUNG WORKS

In words. When either (`EOL_Missing` (LS11 dark when the registers said a large, whole bottle was there) is on) *or* (`EOL_Extra` (LS11 saw a bottle the registers did not expect) is on) and it is the first scan they are true (the one-shot, remembered in `BLK_ONS`), the rung latches `Alarm_Word.2` on and latches `Alarm_Unack` (New alarm, not yet acknowledged (Reset)) on.

Left to right:

1. A branch of 2 parallel paths: power gets through if any one of them is true.

   Path 1

   1. `XIC(EOL_Missing)` *Examine If Closed* asks: true when `EOL_Missing` (LS11 dark when the registers said a large, whole bottle was there) is on.

   Path 2

   1. `XIC(EOL_Extra)` *Examine If Closed* asks: true when `EOL_Extra` (LS11 saw a bottle the registers did not expect) is on.

2. `ONS(BLK_ONS)` *One Shot* asks: true when it is the first scan they are true (the one-shot, remembered in `BLK_ONS`).
3. A branch of 2 parallel paths: each path is tested on its own and does its own work.

   Path 1

   1. `OTL(Alarm_Word.2)` *Output Latch* does: latches `Alarm_Word.2` on.

   Path 2

   1. `OTL(Alarm_Unack)` *Output Latch* does: latches `Alarm_Unack` (New alarm, not yet acknowledged (Reset)) on.

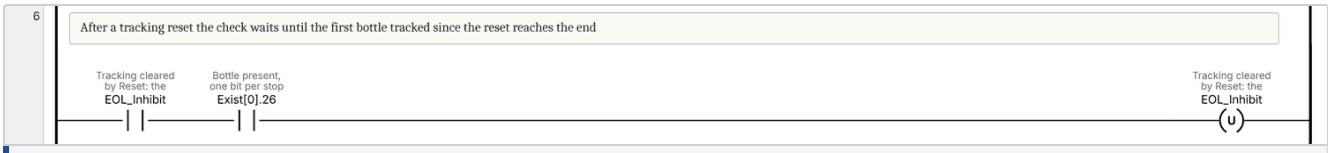
Why it is built this way:

One-shot: once per event, not once per scan.

A latch: it stays on until an OTU clears it.

> **Think of it like this:** parallel paths are two roads to the same town. If either road is open you get there; the rung only cares that power reaches the far end, not which way it went.

RUNG 6




```
XIC(EOL_Inhibit)XIC(Exist[0].26)OTU(EOL_Inhibit);
```

PREDICT IT · 3 `EOL_Inhibit` is on and `Exist[0].26` is off. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

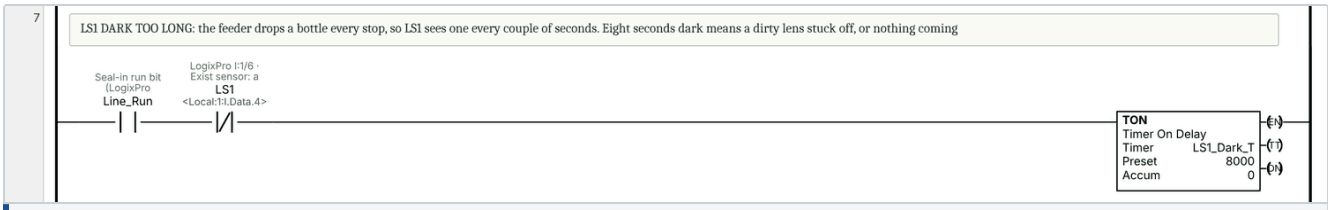
In words. When `EOL_Inhibit` (Tracking cleared by Reset: the end-of-line check waits until the first bottle tracked since then reaches LS11) is on and `Exist[0].26` is on, the rung unlatches `EOL_Inhibit` (turns it off).

Left to right:

1. `XIC(EOL_Inhibit)` *Examine If Closed* asks: true when `EOL_Inhibit` (Tracking cleared by Reset: the end-of-line check waits until the first bottle tracked since then reaches LS11) is on.
2. `XIC(Exist[0].26)` *Examine If Closed* asks: true when `Exist[0].26` is on.
3. `OTU(EOL_Inhibit)` *Output Unlatch* does: unlatches `EOL_Inhibit` (turns it off). It only ever clears the bit.

> **Think of it like this:** a rung is a pipe with valves in it, read left to right. Every contact is a valve: XIC is open while its bit is 1, XIO while its bit is 0. Water reaches the output at the right only if every valve in a row is open. The output does not remember anything: on every scan it is simply wet or dry.

RUNG 7




```
XIC(Line_Run)XIO(LS1)TON(LS1_Dark_T,8000,0);
```

PREDICT IT · 4 `Line_Run` is on and `LS1` is off. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Line_Run` (Seal-in run bit (LogixPro B3:0/0)) is on and `LS1` (Exist sensor: a bottle at the start of the line) is off, the rung runs the timer `LS1_Dark_T` (LS1 dark too long timer) toward 8000 ms (8 s).

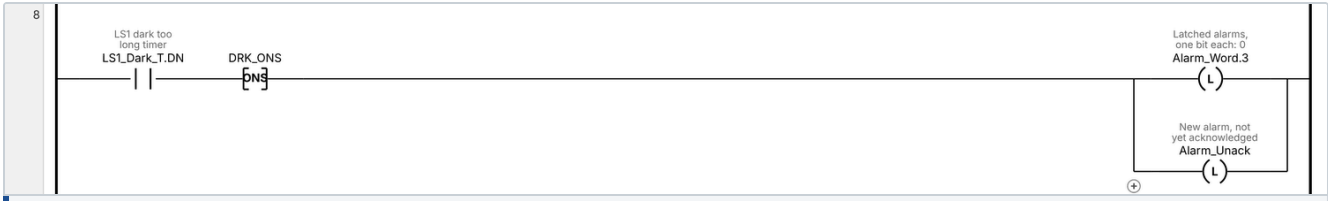
Left to right:

1. `XIC(Line_Run)` *Examine If Closed* asks: true when `Line_Run` (Seal-in run bit (LogixPro B3:0/0)) is on.
2. `XIO(LS1)` *Examine If Open* asks: true when `LS1` (Exist sensor: a bottle at the start of the line) is off.
3. `TON(LS1_Dark_T,8000,0)` *Timer On Delay* does: runs the timer `LS1_Dark_T` (LS1 dark too long timer) toward 8000 ms (8 s).

Why it is built this way:

A timer: it times only while this rung is true.

RUNG 8




```
XIC(LS1_Dark_T.DN)ONS(DRK_ONS)[OTL(Alarm_Word.3),OTL(Alarm_Unack)];
```

HOW THIS RUNG WORKS

In words. When `LS1_Dark_T.DN` (the done bit of `LS1_Dark_T` : LS1 dark too long timer) is on and it is the first scan they are true (the one-shot, remembered in `DRK_ONS`), the rung latches `Alarm_Word.3` on and latches `Alarm_Unack` (New alarm, not yet acknowledged (Reset)) on.

Left to right:

1. `XIC(LS1_Dark_T.DN)` *Examine If Closed* asks: true when `LS1_Dark_T.DN` (the done bit of `LS1_Dark_T` : LS1 dark too long timer) is on.
2. `ONS(DRK_ONS)` *One Shot* asks: true when it is the first scan they are true (the one-shot, remembered in `DRK_ONS`).
3. A branch of 2 parallel paths: each path is tested on its own and does its own work.

   Path 1

   1. `OTL(Alarm_Word.3)` *Output Latch* does: latches `Alarm_Word.3` on.

   Path 2

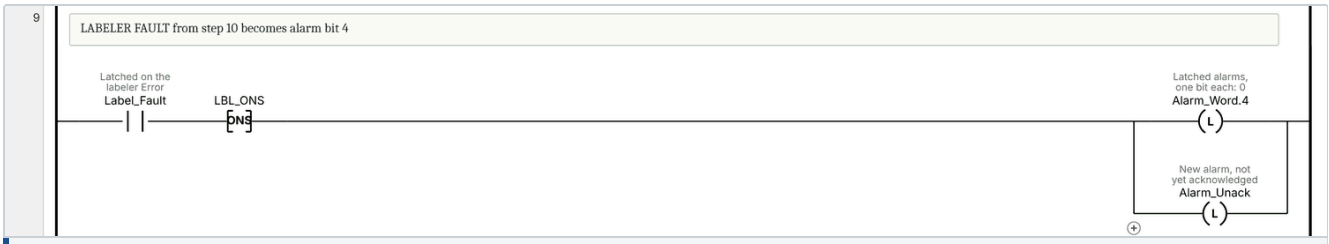
   1. `OTL(Alarm_Unack)` *Output Latch* does: latches `Alarm_Unack` (New alarm, not yet acknowledged (Reset)) on.

Why it is built this way:

One-shot: once per event, not once per scan.

A latch: it stays on until an OTU clears it.

RUNG 9




```
XIC(Label_Fault)ONS(LBL_ONS)[OTL(Alarm_Word.4),OTL(Alarm_Unack)];
```

HOW THIS RUNG WORKS

In words. When `Label_Fault` (Latched on the labeler Error input (end of roll or a missed label); Enter clears it) is on and it is the first scan they are true (the one-shot, remembered in `LBL_ONS`), the rung latches `Alarm_Word.4` on and latches `Alarm_Unack` (New alarm, not yet acknowledged (Reset)) on.

Left to right:

1. `XIC(Label_Fault)` *Examine If Closed* asks: true when `Label_Fault` (Latched on the labeler Error input (end of roll or a missed label); Enter clears it) is on.
2. `ONS(LBL_ONS)` *One Shot* asks: true when it is the first scan they are true (the one-shot, remembered in `LBL_ONS`).
3. A branch of 2 parallel paths: each path is tested on its own and does its own work.

   Path 1

   1. `OTL(Alarm_Word.4)` *Output Latch* does: latches `Alarm_Word.4` on.

   Path 2

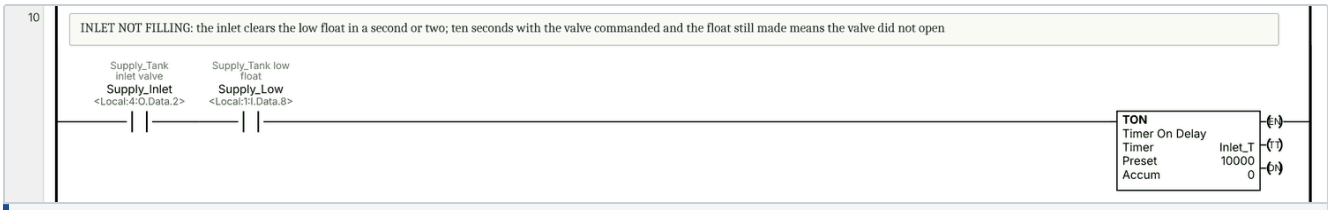
   1. `OTL(Alarm_Unack)` *Output Latch* does: latches `Alarm_Unack` (New alarm, not yet acknowledged (Reset)) on.

Why it is built this way:

One-shot: once per event, not once per scan.

A latch: it stays on until an OTU clears it.

RUNG 10




```
XIC(Supply_Inlet)XIC(Supply_Low)TON(Inlet_T,10000,0);
```

PREDICT IT · 5 `Supply_Inlet` is on and `Supply_Low` is off. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Supply_Inlet` (Supply_Tank inlet valve) is on and `Supply_Low` (Supply_Tank low float) is on, the rung runs the timer `Inlet_T` (Inlet not filling timer) toward 10000 ms (10 s).

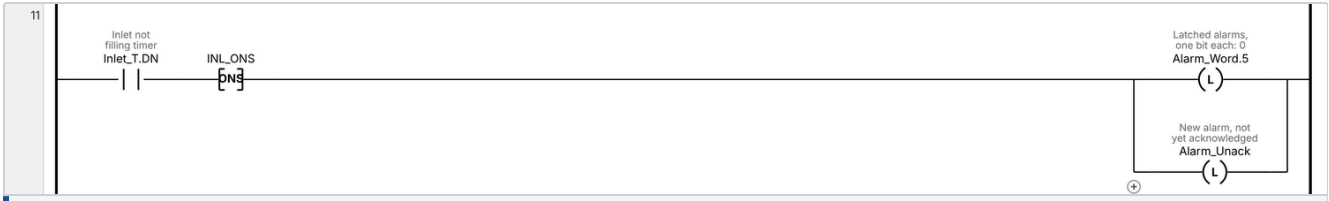
Left to right:

1. `XIC(Supply_Inlet)` *Examine If Closed* asks: true when `Supply_Inlet` (Supply_Tank inlet valve) is on.
2. `XIC(Supply_Low)` *Examine If Closed* asks: true when `Supply_Low` (Supply_Tank low float) is on.
3. `TON(Inlet_T,10000,0)` *Timer On Delay* does: runs the timer `Inlet_T` (Inlet not filling timer) toward 10000 ms (10 s).

Why it is built this way:

A timer: it times only while this rung is true.

RUNG 11




```
XIC(Inlet_T.DN)ONS(INL_ONS)[OTL(Alarm_Word.5),OTL(Alarm_Unack)];
```

HOW THIS RUNG WORKS

In words. When `Inlet_T.DN` (the done bit of `Inlet_T`: Inlet not filling timer) is on and it is the first scan they are true (the one-shot, remembered in `INL_ONS`), the rung latches `Alarm_Word.5` on and latches `Alarm_Unack` (New alarm, not yet acknowledged (Reset)) on.

Left to right:

1. `XIC(Inlet_T.DN)` *Examine If Closed* asks: true when `Inlet_T.DN` (the done bit of `Inlet_T`: Inlet not filling timer) is on.
2. `ONS(INL_ONS)` *One Shot* asks: true when it is the first scan they are true (the one-shot, remembered in `INL_ONS`).
3. A branch of 2 parallel paths: each path is tested on its own and does its own work.

   Path 1

   1. `OTL(Alarm_Word.5)` *Output Latch* does: latches `Alarm_Word.5` on.

   Path 2

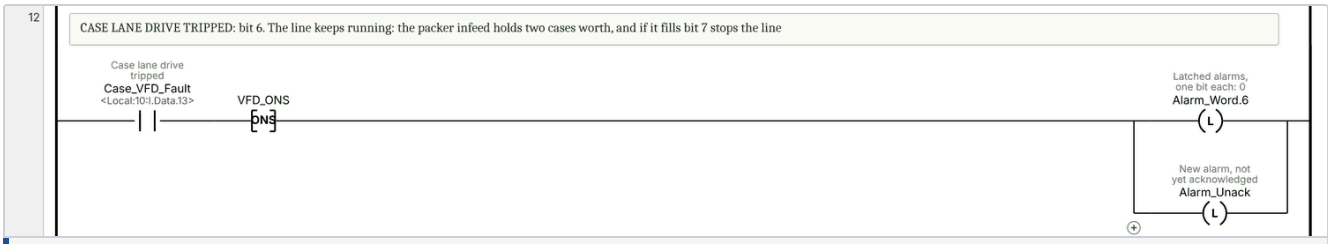
   1. `OTL(Alarm_Unack)` *Output Latch* does: latches `Alarm_Unack` (New alarm, not yet acknowledged (Reset)) on.

Why it is built this way:

One-shot: once per event, not once per scan.

A latch: it stays on until an OTU clears it.

RUNG 12




```
XIC(Case_VFD_Fault)ONS(VFD_ONS)[OTL(Alarm_Word.6),OTL(Alarm_Unack)];
```

HOW THIS RUNG WORKS

In words. When `Case_VFD_Fault` (Case lane drive tripped) is on and it is the first scan they are true (the one-shot, remembered in `VFD_ONS`), the rung latches `Alarm_Word.6` on and latches `Alarm_Unack` (New alarm, not yet acknowledged (Reset)) on.

Left to right:

1. `XIC(Case_VFD_Fault)` *Examine If Closed* asks: true when `Case_VFD_Fault` (Case lane drive tripped) is on.
2. `ONS(VFD_ONS)` *One Shot* asks: true when it is the first scan they are true (the one-shot, remembered in `VFD_ONS`).
3. A branch of 2 parallel paths: each path is tested on its own and does its own work.

   Path 1

   1. `OTL(Alarm_Word.6)` *Output Latch* does: latches `Alarm_Word.6` on.

   Path 2

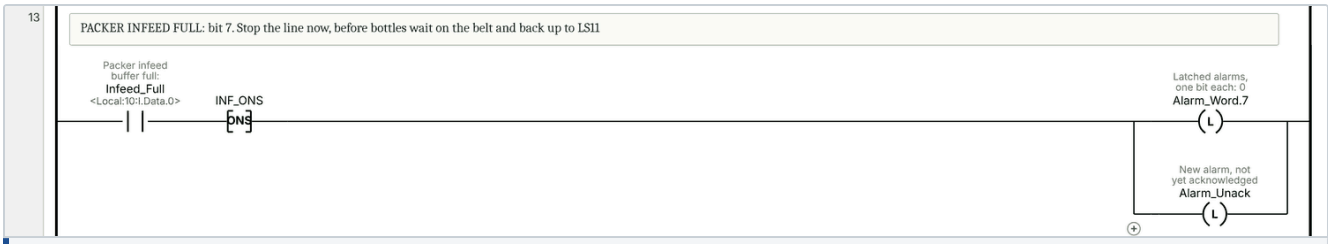
   1. `OTL(Alarm_Unack)` *Output Latch* does: latches `Alarm_Unack` (New alarm, not yet acknowledged (Reset)) on.

Why it is built this way:

One-shot: once per event, not once per scan.

A latch: it stays on until an OTU clears it.

RUNG 13




```
XIC(Infeed_Full)ONS(INF_ONS)[OTL(Alarm_Word.7),OTL(Alarm_Unack)];
```

HOW THIS RUNG WORKS

In words. When `Infeed_Full` (Packer infeed buffer full: bottles arriving now wait on the belt) is on and it is the first scan they are true (the one-shot, remembered in `INF_ONS`), the rung latches `Alarm_Word.7` on and latches `Alarm_Unack` (New alarm, not yet acknowledged (Reset)) on.

Left to right:

1. `XIC(Infeed_Full)` *Examine If Closed* asks: true when `Infeed_Full` (Packer infeed buffer full: bottles arriving now wait on the belt) is on.
2. `ONS(INF_ONS)` *One Shot* asks: true when it is the first scan they are true (the one-shot, remembered in `INF_ONS`).
3. A branch of 2 parallel paths: each path is tested on its own and does its own work.

   Path 1

   1. `OTL(Alarm_Word.7)` *Output Latch* does: latches `Alarm_Word.7` on.

   Path 2

   1. `OTL(Alarm_Unack)` *Output Latch* does: latches `Alarm_Unack` (New alarm, not yet acknowledged (Reset)) on.

Why it is built this way:

One-shot: once per event, not once per scan.

A latch: it stays on until an OTU clears it.

RUNG 14

```
[XIC(Alarm_Word.0),XIC(Alarm_Word.1),XIC(Alarm_Word.2),XIC(Alarm_Word.3),XIC(Alarm_Word.5),XIC(Alarm_Word.7)]OTU(Line_Run);
```

PREDICT IT · 6 `Alarm_Word.0` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

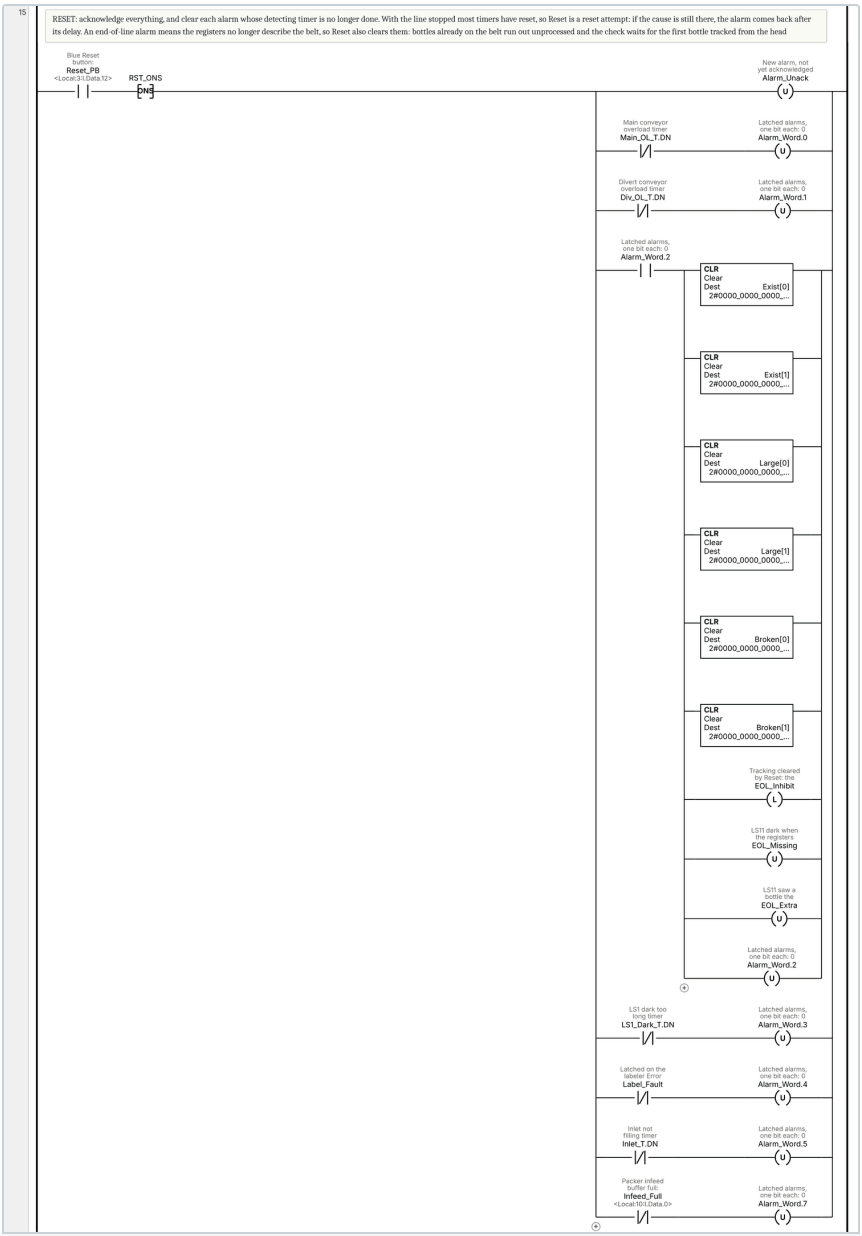
HOW THIS RUNG WORKS

In words. When either (`Alarm_Word.0` is on) *or* (`Alarm_Word.1` is on) *or* (`Alarm_Word.2` is on) *or* (`Alarm_Word.3` is on) *or* (`Alarm_Word.5` is on) *or* (`Alarm_Word.7` is on), the rung unlatches `Line_Run` (Seal-in run bit (LogixPro B3:0/0)) (turns it off).

Left to right:

1. A branch of 6 parallel paths: power gets through if any one of them is true.
   - **Path 1**
     1. `XIC(Alarm_Word.0)` *Examine If Closed* asks: true when `Alarm_Word.0` is on.
   - **Path 2**
     1. `XIC(Alarm_Word.1)` *Examine If Closed* asks: true when `Alarm_Word.1` is on.
   - **Path 3**
     1. `XIC(Alarm_Word.2)` *Examine If Closed* asks: true when `Alarm_Word.2` is on.
   - **Path 4**
     1. `XIC(Alarm_Word.3)` *Examine If Closed* asks: true when `Alarm_Word.3` is on.
   - **Path 5**
     1. `XIC(Alarm_Word.5)` *Examine If Closed* asks: true when `Alarm_Word.5` is on.
   - **Path 6**
     1. `XIC(Alarm_Word.7)` *Examine If Closed* asks: true when `Alarm_Word.7` is on.
2. `OTU(Line_Run)` *Output Unlatch* does: unlatches `Line_Run` (Seal-in run bit (LogixPro B3:0/0)) (turns it off).

RUNG 15




```
XIC(Reset_PB)ONS(RST_ONS)
[OTU(Alarm_Unack),XIO(Main_OL_T.DN)OTU(Alarm_Word.0),XIO(Div_OL_T.DN)OTU(Alarm_
Word.1),XIC(Alarm_Word.2)
[CLR(Exist[0]),CLR(Exist[1]),CLR(Large[0]),CLR(Large[1]),CLR(Broken[0]),CLR(Bro
ken[1]),OTL(EOL_Inhibit),OTU(EOL_Missing),OTU(EOL_Extra),OTU(Alarm_Word.2)],XIO
(LS1_Dark_T.DN)OTU(Alarm_Word.3),XIO(Label_Fault)OTU(Alarm_Word.4),XIO(Inlet_T.
DN)OTU(Alarm_Word.5),XIO(Infeed_Full)OTU(Alarm_Word.7)];
```

HOW THIS RUNG WORKS

In words. When `Reset_PB` (Blue Reset button: acknowledge alarms and clear the ones whose cause is gone) is on and it is the first scan they are true (the one-shot, remembered in `RST_ONS`), the rung picks its path: in any case it unlatches `Alarm_Unack` (New alarm, not yet acknowledged (Reset)) (turns it off); if `Main_OL_T.DN` (the done bit of `Main_OL_T`: Main conveyor overload timer (no running feedback)) is off, it unlatches `Alarm_Word.0` (turns it off); if `Div_OL_T.DN` (the done bit of `Div_OL_T`: Divert conveyor overload timer (no running feedback)) is off, it unlatches `Alarm_Word.1` (turns it off); if `Alarm_Word.2` is on, it clears `Exist[0]` to 0, clears `Exist[1]` to 0, clears `Large[0]` to 0, clears `Large[1]` to 0, clears `Broken[0]` to 0, clears `Broken[1]` to 0, latches `EOL_Inhibit` (Tracking cleared by Reset: the end-of-line check waits until the first bottle tracked since then reaches LS11) on, unlatches `EOL_Missing` (LS11 dark when the registers said a large, whole bottle was there) (turns it off), unlatches `EOL_Extra` (LS11 saw a bottle the registers did not expect) (turns it off) and unlatches `Alarm_Word.2` (turns it off); if `LS1_Dark_T.DN` (the done bit of `LS1_Dark_T`: LS1 dark too long timer) is off, it unlatches `Alarm_Word.3` (turns it off); if `Label_Fault` (Latched on the labeler Error input (end of roll or a missed label); Enter clears it) is off, it unlatches `Alarm_Word.4` (turns it off); if `Inlet_T.DN` (the done bit of `Inlet_T`: Inlet not filling timer) is off, it unlatches `Alarm_Word.5` (turns it off); if `Infeed_Full` (Packer infeed buffer full: bottles arriving now wait on the belt) is off, it unlatches `Alarm_Word.7` (turns it off).

Left to right:

1. `XIC(Reset_PB)` *Examine If Closed* asks: true when `Reset_PB` (Blue Reset button: acknowledge alarms and clear the ones whose cause is gone) is on.
2. `ONS(RST_ONS)` *One Shot* asks: true when it is the first scan they are true (the one-shot, remembered in `RST_ONS`).
3. A branch of 8 parallel paths: each path is tested on its own and does its own work.

   Path 1

   1. `OTU(Alarm_Unack)` *Output Unlatch* does: unlatches `Alarm_Unack` (New alarm, not yet acknowledged (Reset)) (turns it off).

   Path 2

   1. `XIO(Main_OL_T.DN)` *Examine If Open* asks: true when `Main_OL_T.DN` (the done bit of `Main_OL_T`: Main conveyor overload timer (no running feedback)) is off.
   2. `OTU(Alarm_Word.0)` *Output Unlatch* does: unlatches `Alarm_Word.0` (turns it off).

   Path 3

   1. `XIO(Div_OL_T.DN)` *Examine If Open* asks: true when `Div_OL_T.DN` (the done bit of `Div_OL_T`: Divert conveyor overload timer (no running feedback)) is off.
   2. `OTU(Alarm_Word.1)` *Output Unlatch* does: unlatches `Alarm_Word.1` (turns it off).

   Path 4

   1. `XIC(Alarm_Word.2)` *Examine If Closed* asks: true when `Alarm_Word.2` is on.
   2. A branch of 10 parallel paths: each path is tested on its own and does its own work.

      Path 1

      1. `CLR(Exist[0])` *Clear* does: clears `Exist[0]` to 0.

      Path 2

      1. `CLR(Exist[1])` *Clear* does: clears `Exist[1]` to 0.

      Path 3

      1. `CLR(Large[0])` *Clear* does: clears `Large[0]` to 0.

      Path 4

      1. `CLR(Large[1])` *Clear* does: clears `Large[1]` to 0.

      Path 5

      1. `CLR(Broken[0])` *Clear* does: clears `Broken[0]` to 0.

Path 6

1. `CLR(Broken[1])` *Clear* does: clears `Broken[1]` to 0.

Path 7

1. `OTL(EOL_Inhibit)` *Output Latch* does: latches `EOL_Inhibit` (Tracking cleared by Reset: the end-of-line check waits until the first bottle tracked since then reaches LS11) on.

Path 8

1. `OTU(EOL_Missing)` *Output Unlatch* does: unlatches `EOL_Missing` (LS11 dark when the registers said a large, whole bottle was there) (turns it off).

Path 9

1. `OTU(EOL_Extra)` *Output Unlatch* does: unlatches `EOL_Extra` (LS11 saw a bottle the registers did not expect) (turns it off).

Path 10

1. `OTU(Alarm_Word.2)` *Output Unlatch* does: unlatches `Alarm_Word.2` (turns it off).

Path 5

1. `XIO(LS1_Dark_T.DN)` *Examine If Open* asks: true when `LS1_Dark_T.DN` (the done bit of `LS1_Dark_T`: LS1 dark too long timer) is off.
2. `OTU(Alarm_Word.3)` *Output Unlatch* does: unlatches `Alarm_Word.3` (turns it off).

Path 6

1. `XIO(Label_Fault)` *Examine If Open* asks: true when `Label_Fault` (Latched on the labeler Error input (end of roll or a missed label); Enter clears it) is off.
2. `OTU(Alarm_Word.4)` *Output Unlatch* does: unlatches `Alarm_Word.4` (turns it off).

Path 7

1. `XIO(Inlet_T.DN)` *Examine If Open* asks: true when `Inlet_T.DN` (the done bit of `Inlet_T`: Inlet not filling timer) is off.
2. `OTU(Alarm_Word.5)` *Output Unlatch* does: unlatches `Alarm_Word.5` (turns it off).

Path 8

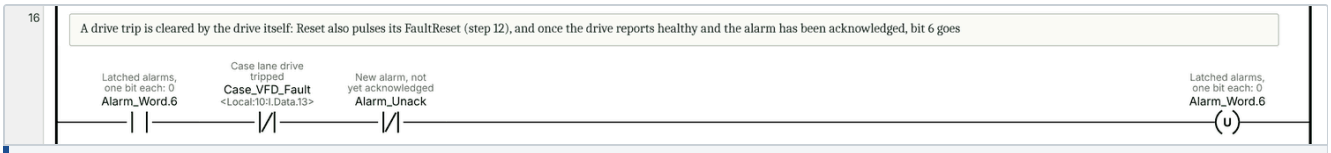
1. `XIO(Infeed_Full)` *Examine If Open* asks: true when `Infeed_Full` (Packer infeed buffer full: bottles arriving now wait on the belt) is off.
2. `OTU(Alarm_Word.7)` *Output Unlatch* does: unlatches `Alarm_Word.7` (turns it off).

Why it is built this way:

One-shot: once per event, not once per scan.

A latch: it stays on until an OTU clears it.

RUNG 16




```
XIC(Alarm_Word.6)XIO(Case_VFD_Fault)XIO(Alarm_Unack)OTU(Alarm_Word.6);
```

PREDICT IT · 7 `Alarm_Word.6` is on, `Case_VFD_Fault` is off and `Alarm_Unack` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

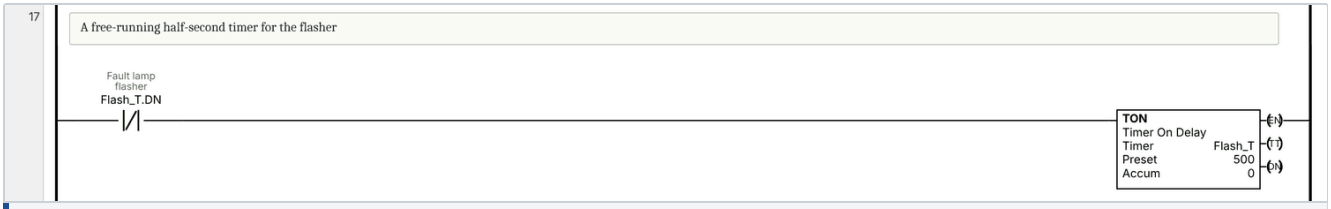
HOW THIS RUNG WORKS

In words. When `Alarm_Word.6` is on, `Case_VFD_Fault` (Case lane drive tripped) is off and `Alarm_Unack` (New alarm, not yet acknowledged (Reset)) is off, the rung unlatches `Alarm_Word.6` (turns it off).

Left to right:

1. `XIC(Alarm_Word.6)` *Examine If Closed* asks: true when `Alarm_Word.6` is on.
2. `XIO(Case_VFD_Fault)` *Examine If Open* asks: true when `Case_VFD_Fault` (Case lane drive tripped) is off.
3. `XIO(Alarm_Unack)` *Examine If Open* asks: true when `Alarm_Unack` (New alarm, not yet acknowledged (Reset)) is off.
4. `OTU(Alarm_Word.6)` *Output Unlatch* does: unlatches `Alarm_Word.6` (turns it off).

RUNG 17




```
XIO(Flash_T.DN)TON(Flash_T,500,0);
```

HOW THIS RUNG WORKS

In words. When `Flash_T.DN` (the done bit of `Flash_T` : Fault lamp flasher) is off, the rung runs the timer `Flash_T` toward 500 ms (0.5 s).

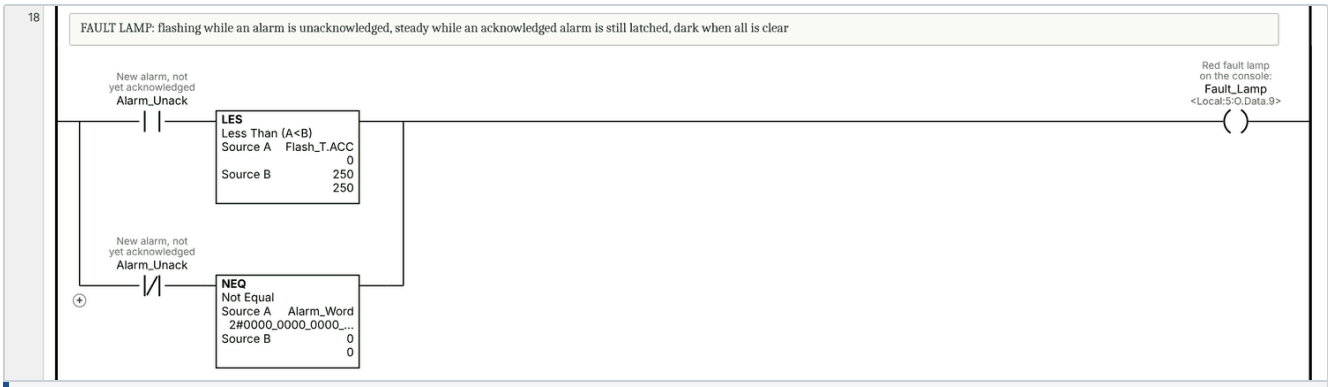
Left to right:

1. `XIO(Flash_T.DN)` *Examine If Open* asks: true when `Flash_T.DN` (the done bit of `Flash_T` : Fault lamp flasher) is off.
2. `TON(Flash_T,500,0)` *Timer On Delay* does: runs the timer `Flash_T` toward 500 ms (0.5 s).

Why it is built this way:

A timer: it times only while this rung is true.

RUNG 18




```
[XIC(Alarm_Unack)LES(Flash_T.ACC,250),XIO(Alarm_Unack)NEQ(Alarm_Word,0)]OTE(Fault_Lamp);
```

PREDICT IT · 8 `Alarm_Unack` is on and `Flash_T.ACC` (the accumulated value of `Flash_T`: Fault lamp flasher) is 249. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When either (`Alarm_Unack` (New alarm, not yet acknowledged (Reset)) is on and `Flash_T.ACC` (the accumulated value of `Flash_T`: Fault lamp flasher) is less than **250**) *or* (`Alarm_Unack` is off and `Alarm_Word` (Latched alarms, one bit each: 0 main conveyor overload, 1 divert conveyor overload, 2 end-of-line check failed (a jam, a lost bottle, tracking out of step), 3 LS1 dark too long (dirty lens or feeder empty), 4 labeler fault, 5 reservoir inlet open but level not rising, 6 case lane drive tripped, 7 packer infeed full) is not **0**), the rung turns `Fault_Lamp` (Red fault lamp on the console: flashing = unacknowledged alarm, steady = alarm still active) on (and off again whenever the rung is false).

Left to right:

1. A branch of 2 parallel paths: power gets through if any one of them is true.

   Path 1

   1. `XIC(Alarm_Unack)` *Examine If Closed* asks: true when `Alarm_Unack` (New alarm, not yet acknowledged (Reset)) is on.
   2. `LES(Flash_T.ACC,250)` *Less Than* asks: true when `Flash_T.ACC` (the accumulated value of `Flash_T`: Fault lamp flasher) is less than **250**.

   Path 2

   1. `XIO(Alarm_Unack)` *Examine If Open* asks: true when `Alarm_Unack` is off.
   2. `NEQ(Alarm_Word,0)` *Not Equal* asks: true when `Alarm_Word` (Latched alarms, one bit each: 0 main conveyor overload, 1 divert conveyor overload, 2 end-of-line check failed (a jam, a lost bottle, tracking out of step), 3 LS1 dark too long (dirty lens or feeder empty), 4 labeler fault, 5 reservoir inlet open but level not rising, 6 case lane drive tripped, 7 packer infeed full) is not **0**.

2. `OTE(Fault_Lamp)` *Output Energize* does: turns `Fault_Lamp` (Red fault lamp on the console: flashing = unacknowledged alarm, steady = alarm still active) on (and off again whenever the rung is false). Every scan it writes the rung state: true on, false off.

Rungs 0–3, overloads. The output says run; the feedback says not running. Half a second allows for the contactor to pull in. When the timer is done, latch the bit and `Alarm_Unack`. The one-shot matters: without it an alarm the operator has acknowledged would be re-raised every scan while the cause is still there.

Rungs 4–6, the end-of-line check. This is the shift register's own proof. Once per stop — halfway through the dwell (`Dwell.TT` and `ACC ≥ 500`, so the belt is standing still and LS11 has settled), once only (`ONS`) — compare LS11 with what the registers say stands in front of it. LS11 at x 1240 is 27 stops after LS1, so in the dwell frame (chapter 10) that is `Exist[0].26`, `Large[0].25`, `Broken[0].24`, and a bottle should be there only if it exists, is large (small ones were diverted at stop 20) and is whole (broken ones were scrapped). Expected and dark is `EOL_Missing`; not expected and made is `EOL_Extra`. Either is bit 2. A jammed bottle shows up here a few stops after it jams: the bottles behind it pile up, the stops they should be at stay empty, the registers walk on regardless. So does a bottle that slipped past LS1 untracked. Rung 6 is for after a reset (below).

Rungs 7–8, LS1 dark too long. The feeder drops a bottle every stop, so LS1 sees one every couple of seconds. Eight seconds dark with the line running means the eye is dirty and stuck off, or nothing is coming.

Rung 9 turns the labeler's fault into bit 4, and **rungs 10–11** watch the reservoir: the inlet clears the low float in a second or two, so ten seconds commanded open with the float still made means the valve did not open.

Rungs 12–13 are the packer: a drive trip is bit 6, a full infeed bit 7. **Rung 14** stops the line on bits 0, 1, 2, 3, 5 and 7 — an `OTU(Line_Run)` after step 1's seal-in, so Start cannot hold while one of them is set. Bits 4 and 6 do not: bottles can leave unlabeled while someone loads a roll (step 10's Try it asks you to decide that for yourself), and the packer's infeed holds two cases worth while the drive is reset — if it fills, bit 7 stops the line before bottles back up to LS11.

Rung 15, Reset. One press, one-shot, does two things. It acknowledges — `Alarm_Unack` off — and it clears every alarm whose cause has gone: an overload bit only once its timer is no longer done, LS1 dark only once LS1 has seen something, the labeler bit only once `Label_Fault` is off. An end-of-line alarm is different: its cause is that the registers no longer describe the belt, so Reset clears all six register words and sets `EOL_Inhibit`, and the check stays off until the first bottle tracked since the reset reaches LS11 (rung 6). That is how a real line recovers after a jam: clear the jam, reset the tracking, purge the line.

Rung 16 lets a drive trip clear itself from the alarm word: Reset also pulses the drive's FaultReset (step 12), and once the drive reports healthy and the alarm has been acknowledged, bit 6 goes. **Rungs 17–18, the lamp.** A free-running half-second timer; the lamp flashes on the first half while an alarm is unacknowledged, and is steady while an acknowledged alarm is still set. Off means healthy.

> **A MACHINE FAULT IS NOT A CONTROLLER FAULT**
>
> Chapter 11's *major fault* is the controller stopping itself: an instruction it cannot execute — an array index out of range, an integer divide by zero. The scan stops, Controller Properties says why, and Communications → Clear Faults clears it. A *machine* fault is everything in this chapter: the controller is healthy, the scan runs, and the only way anyone finds out is a rung like these. The first kind is the PLC protecting itself; the second is your program protecting the machine — and the product, and the people near it.

Inject a fault

The simulator can break the machine on purpose. **Tools → Inject Machine Fault…** lists, for the parts on your floor: a bottle **jammed** on a conveyor (it stops where it is; the bottles behind it pile up), a sensor with a **dirty lens** stuck on or stuck off, a conveyor **overload** (the motor stops and Running drops), a labeler **web break** (Ready drops, Error comes on), a reservoir **inlet stuck shut**. Each is logged in the Simulator Log and drawn with a red dashed outline. **Tools → Clear Machine Faults** or Reset Sim takes them away; click a jammed bottle in Run mode to free it. The *Drill* box injects a random one every few minutes of run time, which is how you practise finding a fault you did not choose.

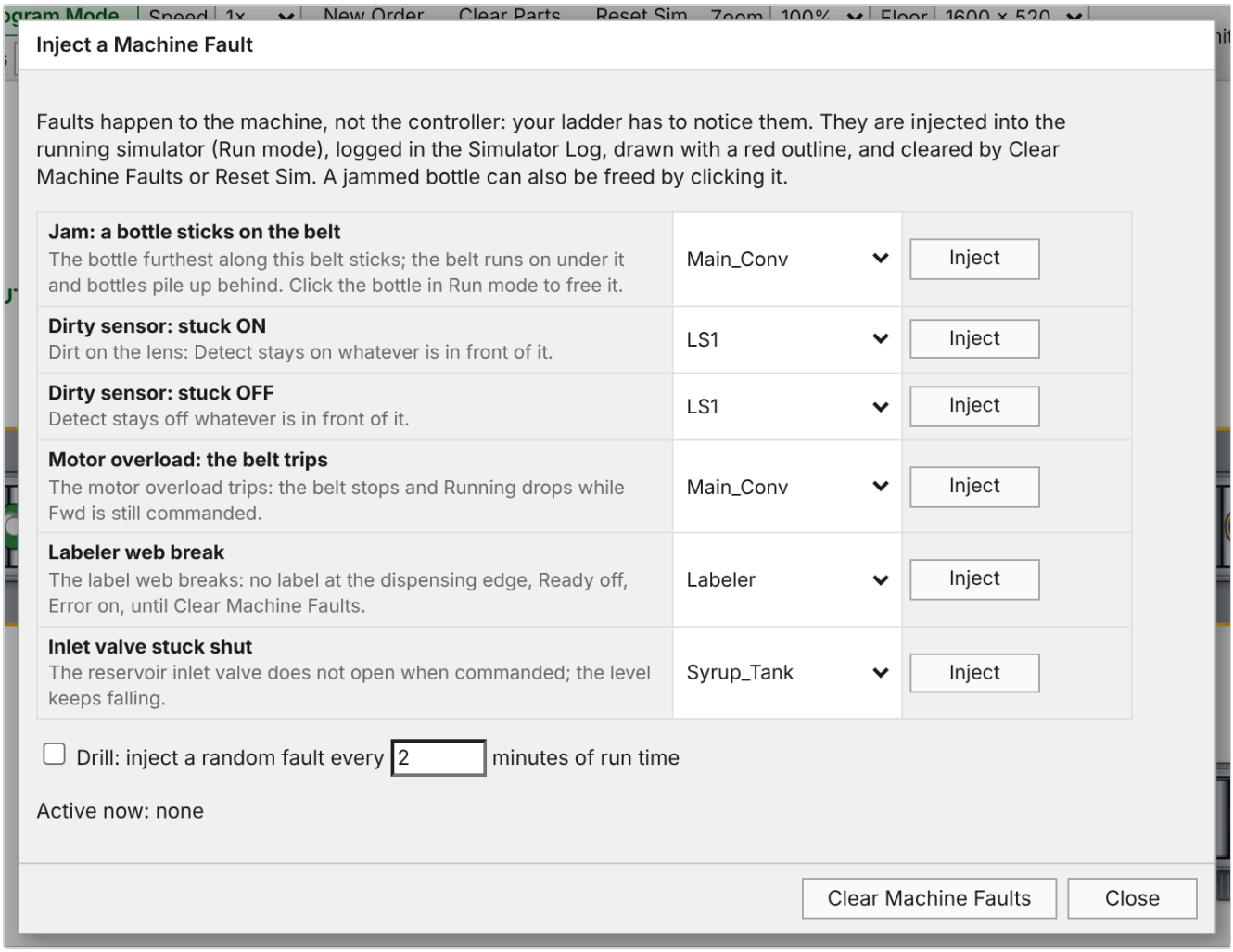


Figure 111. Tools → Inject Machine Fault: one row per kind of fault, the parts on this floor it can be injected into, and the drill.

Run it

Start the line and leave it three minutes: `Alarm_Word` stays 0 and the lamp stays dark. Press Stop and Start at any moment you like, mid-step, mid-dwell, mid-fill — still 0. (Before this step was written the stop rung in step 1 cancelled the step in progress and the dwell started again from zero; the end-of-line check caught the drift on the first try. That is why steps 1, 2 and 6 read the way they do.)

Now break things, one at a time, Clear Machine Faults, Reset and Start between each:

- **Overload on Main_Conv.** At the next step the motor does not turn: bit 0 in under four seconds (the conveyor only runs during a step), the line stops, the lamp flashes. Press Reset without clearing the fault: the lamp goes out (the overload timer is no longer done, because nothing is commanded) — but Start trips it again at the next step. Clear, Reset, Start: the line runs.
- **Jam on Main_Conv.** The furthest bottle along stops. A few stops later LS11 disagrees with the registers: bit 2, the line stops. Free the bottle (click it), Reset — the registers clear and the check waits

— Start, and a minute later the check is armed again with no alarm.

- **Dirty LS1, stuck off.** Eight seconds later, bit 3. Clear, Reset, Start — and about a minute later bit 2: the bottles that passed LS1 while it was blind were never entered in the registers, and they arrive at LS11 as extras. Reset once more.
- **Labeler web break.** Bit 4 at once, the line keeps running, bottles leave bare. Reset acknowledges: the lamp goes steady. Clear the fault (the web is rethreaded), press Enter, Reset: the lamp goes out.
- **Inlet stuck shut.** Nothing happens until the reservoir falls to the low float; ten seconds after that, bit 5.

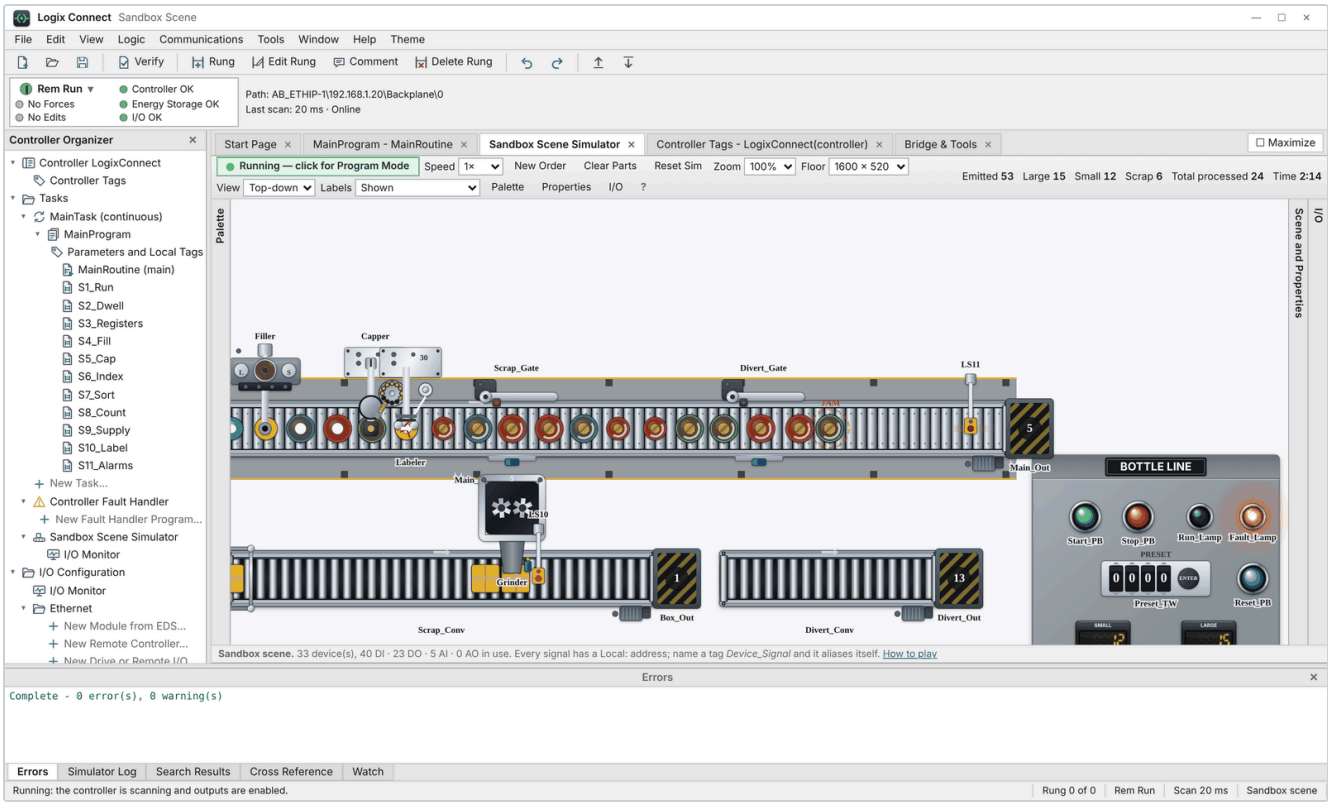
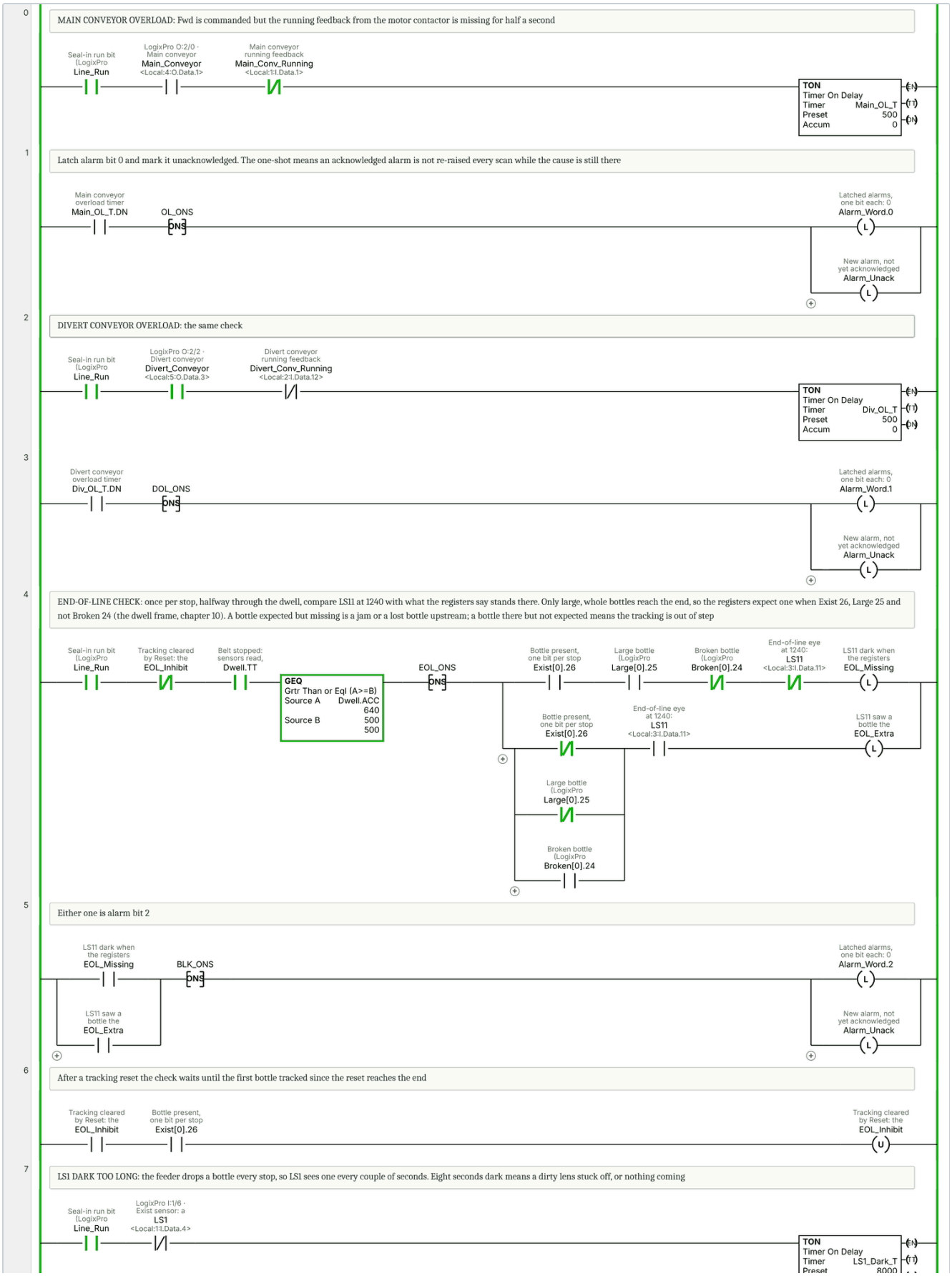
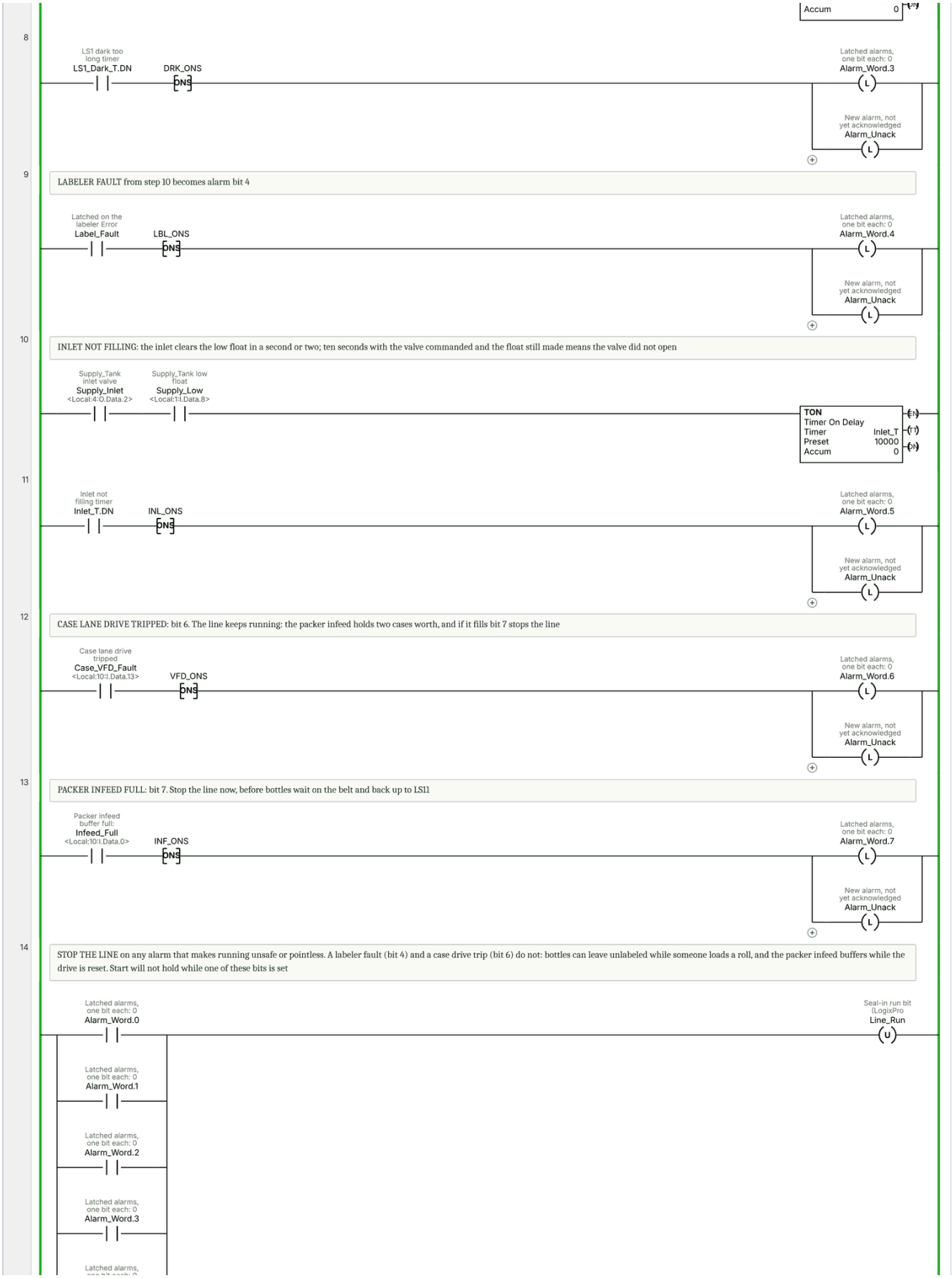
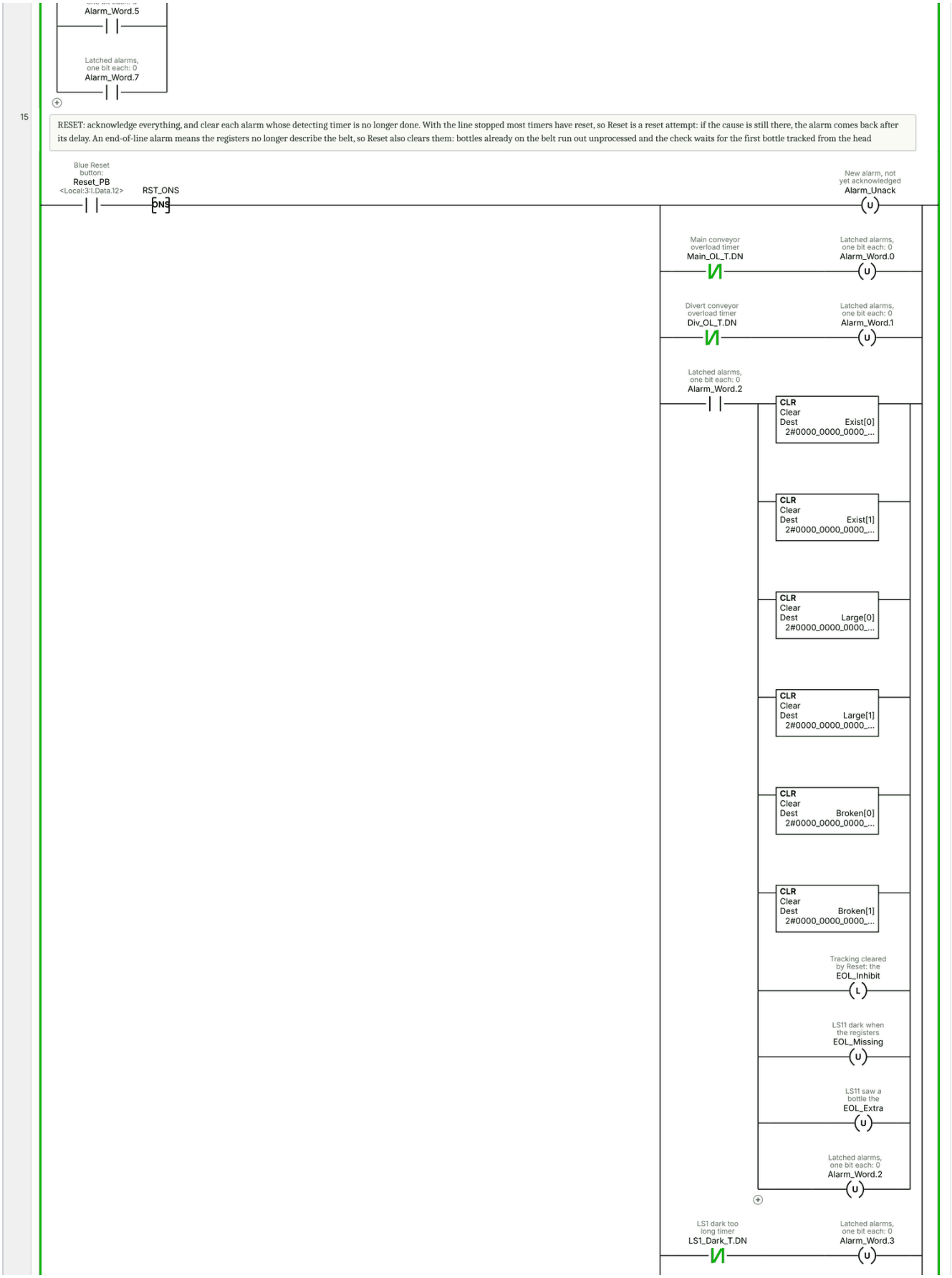


Figure 112. A jam caught. The bottle with the red dashed outline is stuck; the bottles behind it have piled up, so the stops the registers say are full stand empty when they reach LS11. Bit 2 latched, the line stopped, Fault_Lamp lit on the console. In this run it took 13 s from the jam to the alarm.


0
MAIN CONVEYOR OVERLOAD: Fwd is commanded but the running feedback from the motor contactor is missing for half a second
Seal-in run bit (LogixPro Line_Run
LogixPro O:2/0 · Main conveyor Main_Conveyor <Local:4:O.Data.1>
Main conveyor running feedback Main_Conv_Running <Local:1:I.Data.1>
TON Timer On Delay Timer Main_OL_T Preset 500 Accum 0
1
Latch alarm bit 0 and mark it unacknowledged. The one-shot means an acknowledged alarm is not re-raised every scan while the cause is still there
Main conveyor overload timer Main_OL_T.DN
OL_ONS
Latched alarms, one bit each: 0 Alarm_Word.0
New alarm, not yet acknowledged Alarm_Unack
2
DIVERT CONVEYOR OVERLOAD: the same check
Seal-in run bit (LogixPro Line_Run
LogixPro O:2/2 · Divert conveyor Divert_Conveyor <Local:5:O.Data.3>
Divert conveyor running feedback Divert_Conv_Running <Local:2:I.Data.12>
TON Timer On Delay Timer Div_OL_T Preset 500 Accum 0
3
Divert conveyor overload timer Div_OL_T.DN
DOL_ONS
Latched alarms, one bit each: 0 Alarm_Word.1
New alarm, not yet acknowledged Alarm_Unack
4
END-OF-LINE CHECK: once per stop, halfway through the dwell, compare LS11 at 1240 with what the registers say stands there. Only large, whole bottles reach the end, so the registers expect one when Exist 26, Large 25 and not Broken 24 (the dwell frame, chapter 10). A bottle expected but missing is a jam or a lost bottle upstream; a bottle there but not expected means the tracking is out of step
Seal-in run bit (LogixPro Line_Run
Tracking cleared by Reset: the EOL_Inhibit
Belt stopped: sensors read, Dwell.TT
GEQ Grtr Than or Eql (A>=B) Source A Dwell.ACC 640 Source B 500 500
EOL_ONS
Bottle present, one bit per stop Exist[0].26
Large bottle (LogixPro Large[0].25
Broken bottle (LogixPro Broken[0].24
End-of-line eye at 1240: LS11 <Local:3:I.Data.11>
LS11 dark when the registers EOL_Missing
Bottle present, one bit per stop Exist[0].26
End-of-line eye at 1240: LS11 <Local:3:I.Data.11>
LS11 saw a bottle the EOL_Extra
Large bottle (LogixPro Large[0].25
Broken bottle (LogixPro Broken[0].24
5
Either one is alarm bit 2
LS11 dark when the registers EOL_Missing
BLK_ONS
LS11 saw a bottle the EOL_Extra
Latched alarms, one bit each: 0 Alarm_Word.2
New alarm, not yet acknowledged Alarm_Unack
6
After a tracking reset the check waits until the first bottle tracked since the reset reaches the end
Tracking cleared by Reset: the EOL_Inhibit
Bottle present, one bit per stop Exist[0].26
Tracking cleared by Reset: the EOL_Inhibit
7
LS1 DARK TOO LONG: the feeder drops a bottle every stop, so LS1 sees one every couple of seconds. Eight seconds dark means a dirty lens stuck off, or nothing coming
Seal-in run bit (LogixPro Line_Run
LogixPro I:1/6 · Exist sensor: a LS1 <Local:1:I.Data.4>
TON Timer On Delay Timer LS1_Dark_T Preset 8000


Accum 0
8
LS1 dark too long timer
LS1_Dark_T.DN
DRK_ONS
ONS
Latched alarms, one bit each: 0
Alarm_Word.3
L
New alarm, not yet acknowledged
Alarm_Unack
L
9
LABELER FAULT from step 10 becomes alarm bit 4
Latched on the labeler Error
Label_Fault
LBL_ONS
ONS
Latched alarms, one bit each: 0
Alarm_Word.4
L
New alarm, not yet acknowledged
Alarm_Unack
L
10
INLET NOT FILLING: the inlet clears the low float in a second or two; ten seconds with the valve commanded and the float still made means the valve did not open
Supply_Tank inlet valve
Supply_Inlet
<Local:4:O.Data.2>
Supply_Tank low float
Supply_Low
<Local:1:I.Data.8>
TON
Timer On Delay
Timer Inlet_T
Preset 10000
Accum 0
EN
TT
DN
11
Inlet not filling timer
Inlet_T.DN
INL_ONS
ONS
Latched alarms, one bit each: 0
Alarm_Word.5
L
New alarm, not yet acknowledged
Alarm_Unack
L
12
CASE LANE DRIVE TRIPPED: bit 6. The line keeps running: the packer infeed holds two cases worth, and if it fills bit 7 stops the line
Case lane drive tripped
Case_VFD_Fault
<Local:10:I.Data.13>
VFD_ONS
ONS
Latched alarms, one bit each: 0
Alarm_Word.6
L
New alarm, not yet acknowledged
Alarm_Unack
L
13
PACKER INFEED FULL: bit 7. Stop the line now, before bottles wait on the belt and back up to LS11
Packer infeed buffer full:
Infeed_Full
<Local:10:I.Data.0>
INF_ONS
ONS
Latched alarms, one bit each: 0
Alarm_Word.7
L
New alarm, not yet acknowledged
Alarm_Unack
L
14
STOP THE LINE on any alarm that makes running unsafe or pointless. A labeler fault (bit 4) and a case drive trip (bit 6) do not: bottles can leave unlabeled while someone loads a roll, and the packer infeed buffers while the drive is reset. Start will not hold while one of these bits is set
Latched alarms, one bit each: 0
Alarm_Word.0
Seal-in run bit (LogixPro
Line_Run
U
Latched alarms, one bit each: 0
Alarm_Word.1
Latched alarms, one bit each: 0
Alarm_Word.2
Latched alarms, one bit each: 0
Alarm_Word.3
Latched alarms,


Alarm_Word.5
Latched alarms, one bit each: 0
Alarm_Word.7
15
RESET: acknowledge everything, and clear each alarm whose detecting timer is no longer done. With the line stopped most timers have reset, so Reset is a reset attempt: if the cause is still there, the alarm comes back after its delay. An end-of-line alarm means the registers no longer describe the belt, so Reset also clears them: bottles already on the belt run out unprocessed and the check waits for the first bottle tracked from the head
Blue Reset button:
Reset_PB
<Local:3:I.Data.12>
RST_ONS
ONS
New alarm, not yet acknowledged
Alarm_Unack
Main conveyor overload timer
Main_OL_T.DN
Latched alarms, one bit each: 0
Alarm_Word.0
Divert conveyor overload timer
Div_OL_T.DN
Alarm_Word.1
Alarm_Word.2
CLR
Clear
Dest
Exist[0]
2#0000_0000_0000_...
Exist[1]
Large[0]
Large[1]
Broken[0]
Broken[1]
Tracking cleared by Reset: the
EOL_Inhibit
LS11 dark when the registers
EOL_Missing
LS11 saw a bottle the
EOL_Extra
LS1 dark too long timer
LS1_Dark_T.DN
Alarm_Word.3

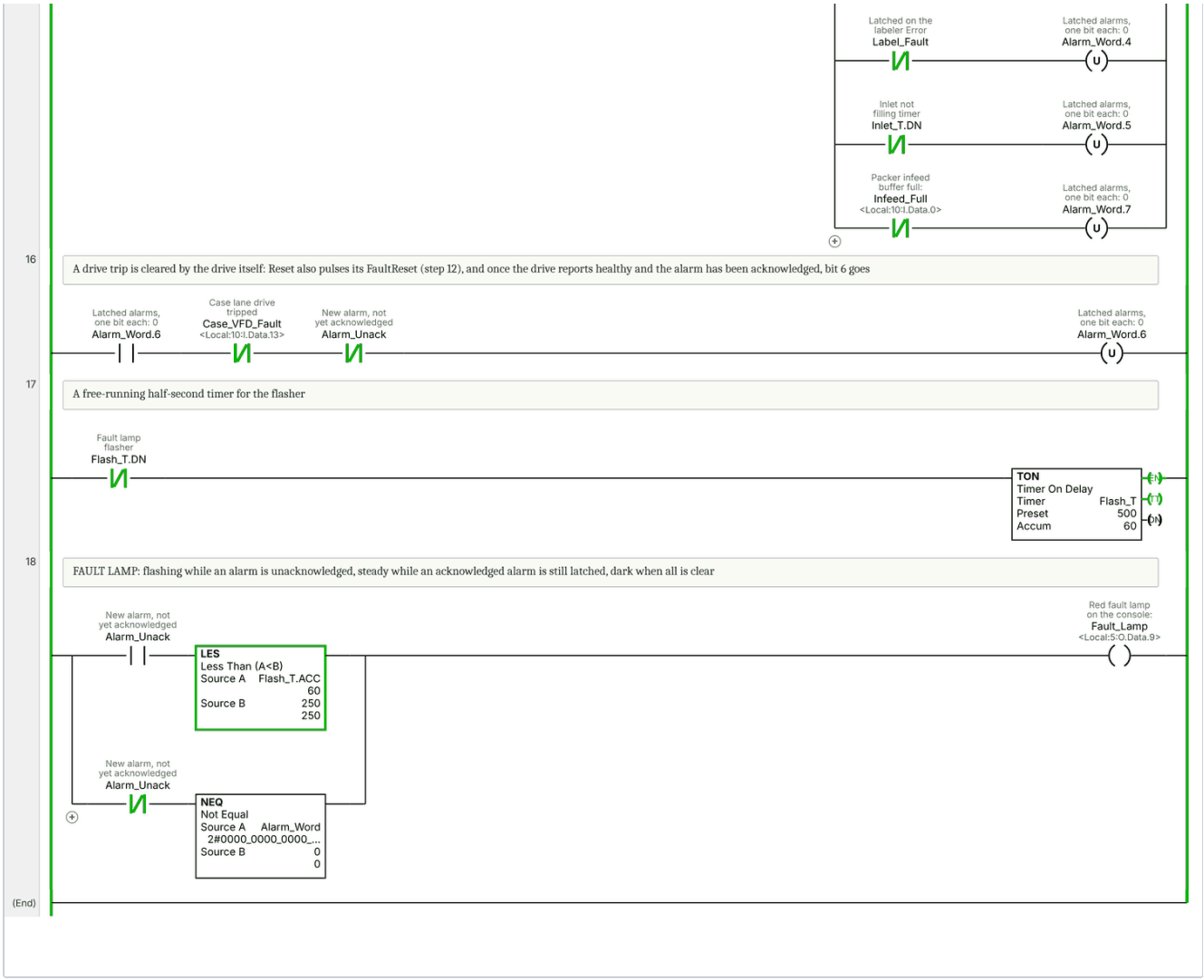


Figure 133. S11_Alarms online, with the finished line running: this is what your routine should look like once every step is in. Green is power flow.

> **TRY IT**
>
> Add a seventh alarm: the head tank (`Tank_Low`) — step 6 already stops the line on it, but nothing tells the operator why. Give it bit 6, latch it like the others, and make Reset clear it once the tank has recovered. Then turn on the drill at one fault every two minutes, run for ten, and keep a log of what you found and how long each took.

Predict it: the answers

1. The rung is false at `XIC(Line_Run)`: `Line_Run` is off, so `Main_OL_T` is reset to zero (a TON resets when its rung goes false).

2. The rung is false at `XIC(Divert_Conveyor)`: `Divert_Conveyor` is off, so `Div_OL_T` is reset to zero (a TON resets when its rung goes false).
3. The rung is false at `XIC(Exist[0].26)`: `Exist[0].26` is off, so the OTU does nothing: `EOL_Inhibit` keeps whatever it had.
4. Every condition on the way is met, so `LS1_Dark_T` times toward its preset.
5. The rung is false at `XIC(Supply_Low)`: `Supply_Low` is off, so `Inlet_T` is reset to zero (a TON resets when its rung goes false).
6. Every condition on the way is met, so `Line_Run` is unlatched, **off**.
7. The rung is false at `XIO(Alarm_Unack)`: `Alarm_Unack` is on, so the OTU does nothing: `Alarm_Word.6` keeps whatever it had.
8. Every condition on the way is met, so `Fault_Lamp` is **on**.

29 Lesson 14 — Step 12: the case packer, both sizes, and the VFD

The last station packs the finished bottles. A **top-load case packer** stands at the end of the main line with two lanes to pick from. The large bottles that reach the end of the deck go into its **deck infeed** (it holds eight). The small bottles come down the divert conveyor on the floor to its **lower lane** (eight more, behind the stop of step 8). On a rising edge of `Pack_Drop` the pick head takes four from one lane, lifts them across to the case lane beside the line and lowers them into the case standing under the head. From the lower lane the head first runs out over the floor conveyor and drops the full 1.7 m to the bottle tops. **One size per case**: the program chooses the lane, and every case is either four large or four small. After the place the head goes home with its cups empty; the four stay in the case. The case lane is the other new thing in this lesson. Its motor is not switched by a contactor like every other conveyor on the line: it runs on a **variable frequency drive**, and the program tells the drive how fast through an **analog output**.

How the drive is wired

This is how most drives are wired to a CompactLogix. Three digital outputs: `Case_Run` (two-wire: the drive runs while it is 1), Rev (not used here) and `Case_VFD_Reset`. One analog output, the **speed reference**: a 0-10 V signal from channel 0 of the 1769-OF4 to the drive's analog input, which the drive is set to read as 0-60 Hz. Four digital inputs back: `Case_VFD_Ready`, `Case_VFD_Running` (output frequency above zero, also while it slows down), `Case_At_Speed` (running and within ½ Hz of the reference) and `Case_VFD_Fault`. And one analog input: the drive's own analog output, set to the output frequency, 0-10 V for 0-60 Hz, into the 1769-IF4 — `Case_Hz_Fbk`.

The analog channels are in **engineering units**: the number in the tag is millivolts, 0-10000 for 0-10.000 V. So the program cannot write "15 Hz" to the drive; it writes the voltage that the drive will read as 15 Hz: 15 ÷ 60 × 10000 = 2500. That conversion is the whole of analog output, and Logix has an instruction for it: **SCP**, Scale with Parameters — *Input*, the input's range (*Input Min*, *Input Max*), the range to map it onto (*Scaled Min*, *Scaled Max*), and the *Output*. It draws a straight line through two points; nothing more.

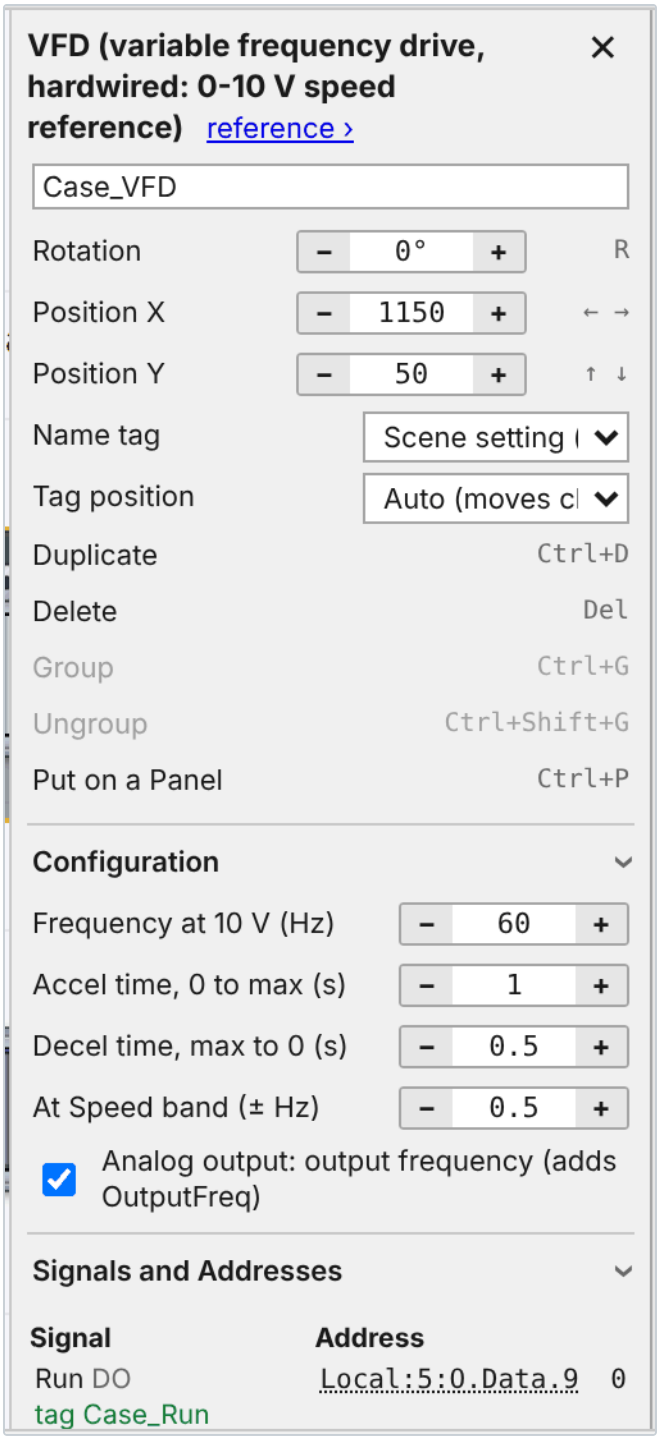


Figure 113. Case_VFD in the Properties drawer: the 10 V frequency, the accel and decel times, the At Speed band, and its signals with their I/O points.

Why a creep speed

A drive does not stop when Run drops: it ramps down on its *decel* time, 0.5 s from 60 Hz to 0 on this one. The case lane runs at 150 px/s at 60 Hz, so a case that is told to stop at full speed travels on for about 37 px — and the packer's `Case_In_Place` window is ±6 px. The cure every packaging line uses: a **slow-down**

eye a little before the mark (`Case_Slow`, 100 px before the head), a **creep speed** from there (15 Hz, 37.5 px/s), and the stop on the mark. From 15 Hz the drive is down in an eighth of a second and the case stops within about 2 px.

The packer's own HMI

`Pack_HMI` is a Process HMI with *Shows* set to Case_Packer, and for a packer it draws a packer faceplate instead of a tank. The MAIN page has a top view of the case lane, the deck infeed and the lower conveyor, with the head drawn where it is now. Beside it are the head state (HOME, PICK DECK, PICK LOWER, PLACE), the head's Z and the vacuum, the bottles waiting in each lane, and the cases packed by size. Below those are the case lane drive in Hz and the packer's START and STOP. At the bottom are an equipment row (LANE, HEAD, VACUUM, LOWER CONV, DECK STOP), the state and a message line. COUNTS and ALARMS are the other two pages. Like Tank_HMI it reads the machine directly, and it writes the PLC only through its two buttons, which pulse `Pack_Start` and `Pack_Stop`. Its CASES cards read the two count tags this step keeps.

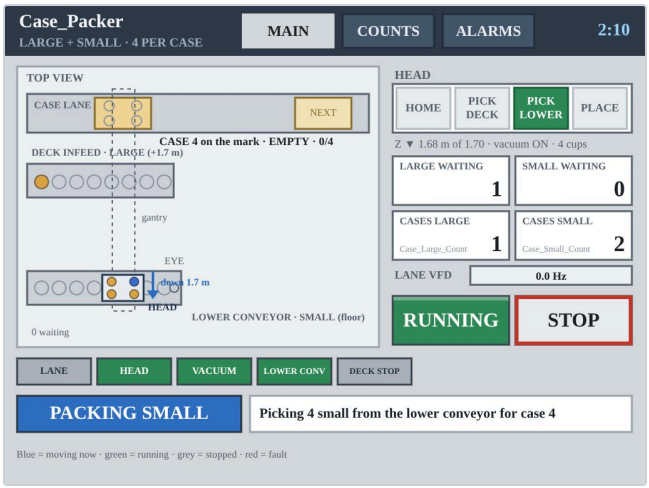


Figure 114. Pack_HMI, MAIN, while the head is down on the lower lane picking four small bottles for the empty case on the mark.

Figure 115. COUNTS: the packer's own counts and the two PLC count tags.

Add the JSR under step 11 and write:

> **IN THE APP**
>
> Guided mode lesson 14 is this chapter: it checks that this lesson's tags are made, that `S12_Pack` exists, is called from MainRoutine and verifies, and then watches the line while you run it and ticks each *Run it* result below as it happens. Help → Guided Mode, or start from File → New Project… → Guided: build the bottle line.

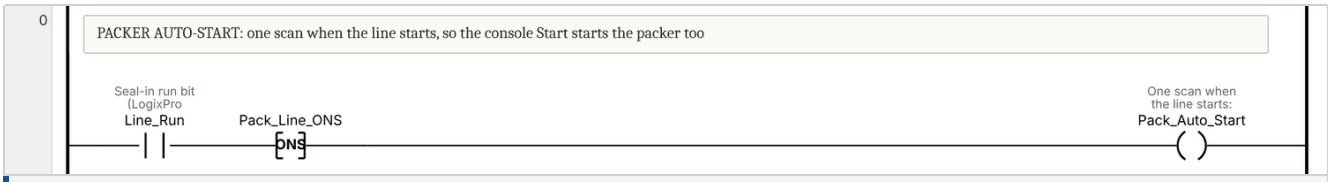
Tags this lesson needs

Create these before you type the rungs (Verify reports any you miss as unknown). Tags from earlier lessons are not repeated.

Tag	Type	How to make it	Description
`Pack_Line_ONS`	BOOL	Edit Tags: type the name, pick the type	
`Pack_Auto_Start`	BOOL	Edit Tags: type the name, pick the type	One scan when the line starts: starts the packer too
`Pack_Start`	BOOL	Edit Tags: type the name, pick the type	START on Pack_HMI (pulsed by the touch screen)
`Pack_Run`	BOOL	Edit Tags: type the name, pick the type	The packer is running (its own seal-in)
`Pack_Stop`	BOOL	Edit Tags: type the name, pick the type	STOP on Pack_HMI (pulsed by the touch screen)
`Case_Creep`	BOOL	Edit Tags: type the name, pick the type	Latched from the slow-down eye until an empty case stands under the head
`Case_Hz`	REAL	Edit Tags: type the name, pick the type	Case lane speed the program wants, in Hz
`Case_Speed_Ref`	INT	alias for `Local:8:O.Ch0Data` — the Properties drawer's *tag* link makes it, then rename it	Case lane drive speed reference: 0-10000 = 0-10 V = 0-60 Hz
`Case_Hz_Fbk`	INT	alias for `Local:9:I.Ch3Data` — the Properties drawer's *tag* link makes it, then rename it	Case lane drive output frequency: 0-10000 = 0-10 V = 0-60 Hz
`Case_Hz_Actual`	REAL	Edit Tags: type the name, pick the type	Case lane drive output frequency read back, in Hz
`Group_Ready`	BOOL	alias for `Local:3:I.Data.15` — the Properties drawer's *tag* link makes it, then rename it	The packer infeed holds a case worth (4)
`Lower_Ready`	BOOL	alias for `Local:10:I.Data.3` — the Properties drawer's *tag* link makes it, then rename it	The packer lower lane holds a case worth (4 small bottles)
`Case_In_Place`	BOOL	alias for `Local:10:I.Data.1` — the Properties drawer's *tag* link makes it, then rename it	A case is centred under the packer head
`Case_Full`	BOOL	alias for `Local:10:I.Data.2` — the Properties drawer's *tag* link makes it, then rename it	The case under the head has been packed
`Case_Run`	BOOL	alias for `Local:5:O.Data.13` — the Properties drawer's *tag* link makes it, then rename it	Case lane drive: run (two-wire)
`Case_Slow`	BOOL	alias for `Local:10:I.Data.9` — the Properties drawer's *tag* link makes it, then rename it	Slow-down eye on the case lane, 100 px before the packer head
`Lower_Count`	INT	alias for `Local:9:I.Ch2Data` — the Properties drawer's *tag* link makes it, then rename it	Small bottles waiting in the packer lower lane

Tag	Type	How to make it	Description
`Infeed_Count`	INT	alias for `Local:9:I.Ch1Data` — the Properties drawer's *tag* link makes it, then rename it	Bottles in the packer infeed
`Pick_Lower`	BOOL	alias for `Local:5:O.Data.11` — the Properties drawer's *tag* link makes it, then rename it	Packer: the next Drop picks from the lower lane (small bottles) instead of the deck infeed
`Case_VFD_Running`	BOOL	alias for `Local:10:I.Data.11` — the Properties drawer's *tag* link makes it, then rename it	Case lane drive output frequency above zero
`Packer_Ready`	BOOL	alias for `Local:3:I.Data.14` — the Properties drawer's *tag* link makes it, then rename it	Packer head home, not cycling
`Drop_ONS`	BOOL	Edit Tags: type the name, pick the type	
`Pack_Drop`	BOOL	alias for `Local:5:O.Data.10` — the Properties drawer's *tag* link makes it, then rename it	Packer: pick a case worth from the infeed and place it in the case (rising edge)
`Case_ONS`	BOOL	Edit Tags: type the name, pick the type	
`Case_Count`	DINT	Edit Tags: type the name, pick the type	Cases packed, shown on the CASES display
`Case_Small`	BOOL	alias for `Local:10:I.Data.5` — the Properties drawer's *tag* link makes it, then rename it	The packed case under the head holds small bottles
`Case_Small_Count`	DINT	Edit Tags: type the name, pick the type	Cases of small bottles packed (CASES SMALL on Pack_HMI)
`Case_Large_Count`	DINT	Edit Tags: type the name, pick the type	Cases of large bottles packed (CASES LARGE on Pack_HMI)
`Case_VFD_Reset`	BOOL	alias for `Local:5:O.Data.15` — the Properties drawer's *tag* link makes it, then rename it	Case lane drive: fault reset (rising edge)

RUNG 0




```
XIC(Line_Run)ONS(Pack_Line_ONS)OTE(Pack_Auto_Start);
```

HOW THIS RUNG WORKS

In words. When `Line_Run` (Seal-in run bit (LogixPro B3:0/0)) is on and it is the first scan they are true (the one-shot, remembered in `Pack_Line_ONS`), the rung turns `Pack_Auto_Start` (One scan when the line starts: starts the packer too) on (and off again whenever the rung is false).

Left to right:

1. `XIC(Line_Run)` *Examine If Closed* asks: true when `Line_Run` (Seal-in run bit (LogixPro B3:0/0)) is on.
2. `ONS(Pack_Line_ONS)` *One Shot* asks: true when it is the first scan they are true (the one-shot, remembered in `Pack_Line_ONS`). It passes power for one scan only, the scan its input goes from false to true; its storage bit remembers last scan.
3. `OTE(Pack_Auto_Start)` *Output Energize* does: turns `Pack_Auto_Start` (One scan when the line starts: starts the packer too) on (and off again whenever the rung is false). Every scan it writes the rung state: true on, false off.

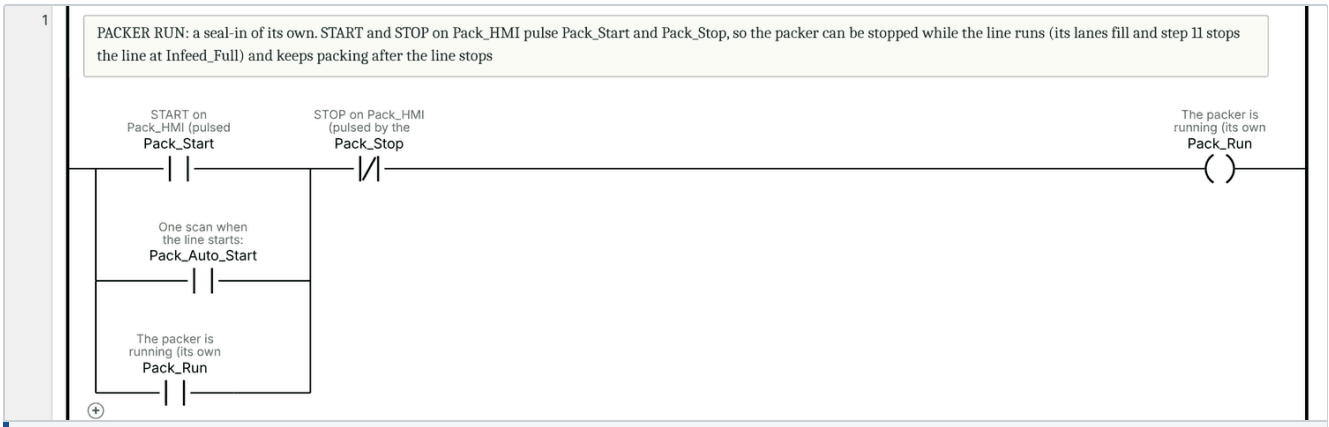
Why it is built this way:

One-shot. Everything after the ONS happens once per event, not on every scan the condition holds; `Pack_Line_ONS` is only its memory of the last scan.

> **Think of it like this:** a doorbell that rings once however long you lean on the button. The ONS looks at what the button was doing last scan: only the change from up to down rings it.

> **Watch for:** Each ONS needs its own storage bit. Two ONS sharing one bit make one of them miss its edges.

RUNG 1




```
[XIC(Pack_Start),XIC(Pack_Auto_Start),XIC(Pack_Run)]XIO(Pack_Stop)OTE(Pack_Run)
;
```

PREDICT IT · 1 `Pack_Start` is on and `Pack_Stop` is off. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When either (`Pack_Start` (START on Pack_HMI (pulsed by the touch screen)) is on) *or* (`Pack_Auto_Start` (One scan when the line starts: starts the packer too) is on) *or* (`Pack_Run` (The packer is running (its own seal-in)) is on) and `Pack_Stop` (STOP on Pack_HMI (pulsed by the touch screen)) is off, the rung turns `Pack_Run` on (and off again whenever the rung is false).

Left to right:

1. A branch of 3 parallel paths: power gets through if any one of them is true.

   Path 1

   1. `XIC(Pack_Start)` *Examine If Closed* asks: true when `Pack_Start` (START on Pack_HMI (pulsed by the touch screen)) is on.

   Path 2

   1. `XIC(Pack_Auto_Start)` *Examine If Closed* asks: true when `Pack_Auto_Start` (One scan when the line starts: starts the packer too) is on.

   Path 3

   1. `XIC(Pack_Run)` *Examine If Closed* asks: true when `Pack_Run` (The packer is running (its own seal-in)) is on.

2. `XIO(Pack_Stop)` *Examine If Open* asks: true when `Pack_Stop` (STOP on Pack_HMI (pulsed by the touch screen)) is off.
3. `OTE(Pack_Run)` *Output Energize* does: turns `Pack_Run` on (and off again whenever the rung is false).

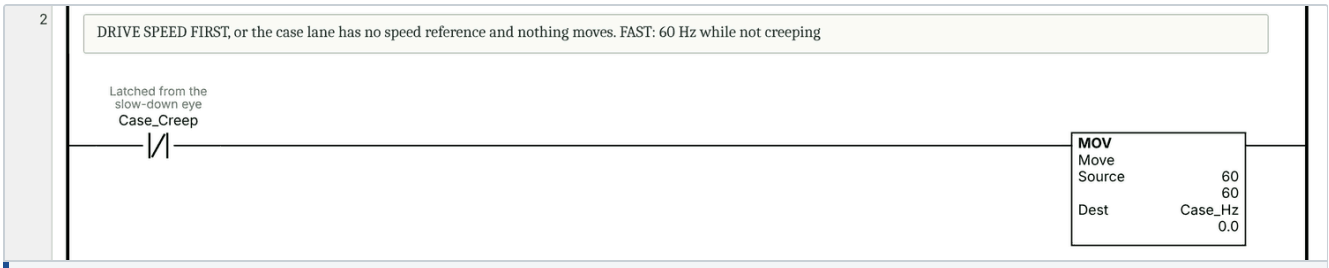
Why it is built this way:

Seal-in. The start condition turns `Pack_Run` on, and `Pack_Run` 's own contact in the parallel path keeps the rung true after the start goes away. Anything in series outside the branch (a stop, a fault) breaks it, and then it stays off until the start is made again.

> **Think of it like this:** a doorbell button and a hand. The start button is the doorbell, pressed for a moment. The seal-in contact is a hand that grabs the circuit as soon as the output comes on and keeps holding after you let go. The stop button, in series, is the only thing that can knock that hand away.

> **Watch for:** The stop has to be in series outside the branch. Put it inside the start path only and the seal-in path bypasses it: the stop no longer stops.

RUNG 2




```
XIO(Case_Creep)MOV(60,Case_Hz);
```

HOW THIS RUNG WORKS

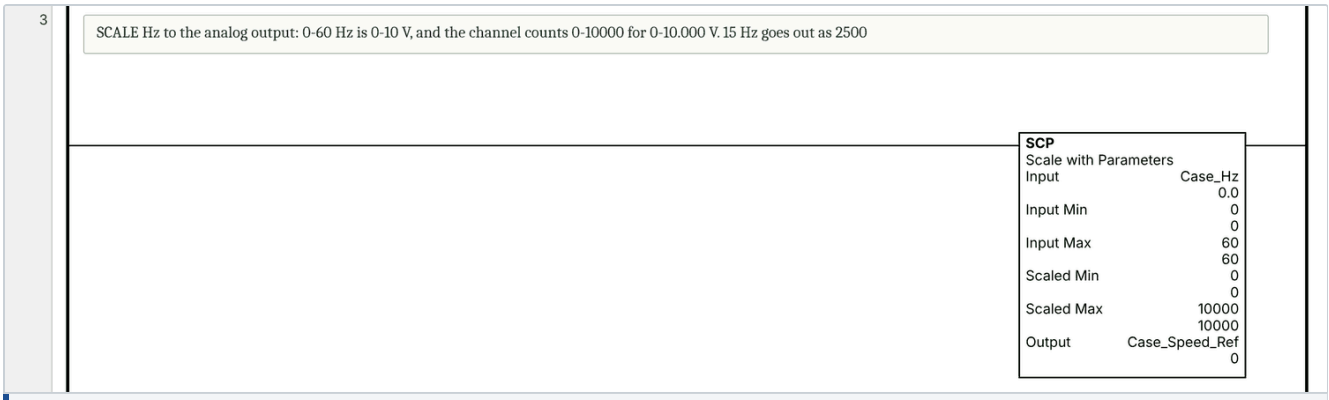
In words. When `Case_Creep` (Latched from the slow-down eye until an empty case stands under the head) is off, the rung puts **60** into `Case_Hz` (Case lane speed the program wants, in Hz).

Left to right:

1. `XIO(Case_Creep)` *Examine If Open* asks: true when `Case_Creep` (Latched from the slow-down eye until an empty case stands under the head) is off.
2. `MOV(60,Case_Hz)` *Move* does: puts **60** into `Case_Hz` (Case lane speed the program wants, in Hz). It writes every scan the rung is true.

> **Think of it like this:** a rung is a pipe with valves in it, read left to right. Every contact is a valve: XIC is open while its bit is 1, XIO while its bit is 0. Water reaches the output at the right only if every valve in a row is open. The output does not remember anything: on every scan it is simply wet or dry.

RUNG 3




```
SCP(Case_Hz,0,60,0,10000,Case_Speed_Ref);
```

HOW THIS RUNG WORKS

In words. Every scan, with nothing to test, the rung scales `Case_Hz` (Case lane speed the program wants, in Hz) from 0…60 to 0…10000 into `Case_Speed_Ref` (Case lane drive speed reference: 0-10000 = 0-10 V = 0-60 Hz).

Left to right:

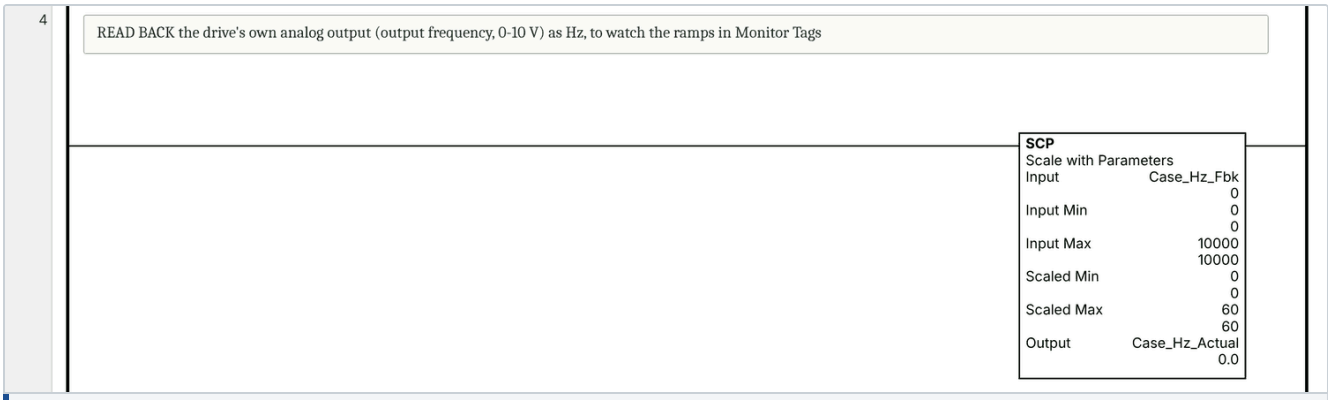
1. `SCP(Case_Hz,0,60,0,10000,Case_Speed_Ref)` *Scale with Parameters* does: scales `Case_Hz` (Case lane speed the program wants, in Hz) from 0…60 to 0…10000 into `Case_Speed_Ref` (Case lane drive speed reference: 0-10000 = 0-10 V = 0-60 Hz).

Why it is built this way:

Scaling. SCP turns the raw number into engineering units with a straight line through the two points you give it.

> **Think of it like this:** reading a thermometer. Two marks you know (freezing and boiling), a straight ruler between them, and any reading on the glass turns into degrees.

RUNG 4




```
SCP(Case_Hz_Fbk,0,10000,0,60,Case_Hz_Actual);
```

HOW THIS RUNG WORKS

In words. Every scan, with nothing to test, the rung scales `Case_Hz_Fbk` (Case lane drive output frequency: 0-10000 = 0-10 V = 0-60 Hz) from 0…10000 to 0…60 into `Case_Hz_Actual` (Case lane drive output frequency read back, in Hz).

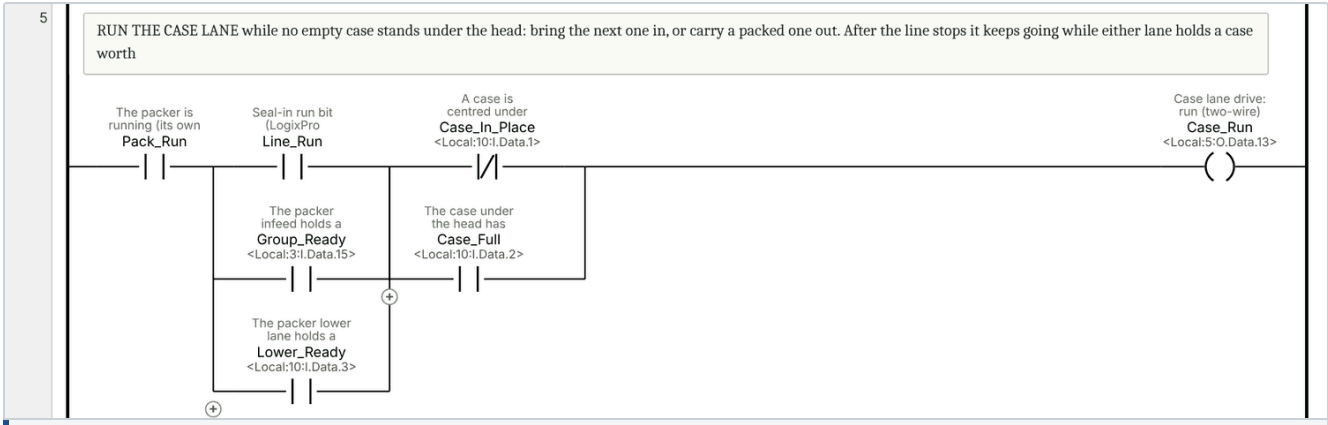
Left to right:

1. `SCP(Case_Hz_Fbk,0,10000,0,60,Case_Hz_Actual)` *Scale with Parameters* does: scales `Case_Hz_Fbk` (Case lane drive output frequency: 0-10000 = 0-10 V = 0-60 Hz) from 0…10000 to 0…60 into `Case_Hz_Actual` (Case lane drive output frequency read back, in Hz).

Why it is built this way:

Scaling raw counts to units.

RUNG 5




```
XIC(Pack_Run)[XIC(Line_Run),XIC(Group_Ready),XIC(Lower_Ready)]
[XIO(Case_In_Place),XIC(Case_Full)]OTE(Case_Run);
```

PREDICT IT · 2 `Pack_Run` is on, `Line_Run` is on and `Case_In_Place` is off. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Pack_Run` (The packer is running (its own seal-in)) is on, either (`Line_Run` (Seal-in run bit (LogixPro B3:0/0)) is on) *or* (`Group_Ready` (The packer infeed holds a case worth (4)) is on) *or* (`Lower_Ready` (The packer lower lane holds a case worth (4 small bottles)) is on) and either (`Case_In_Place` (A case is centred under the packer head) is off) *or* (`Case_Full` (The case under the head has been packed) is on), the rung turns `Case_Run` (Case lane drive: run (two-wire)) on (and off again whenever the rung is false).

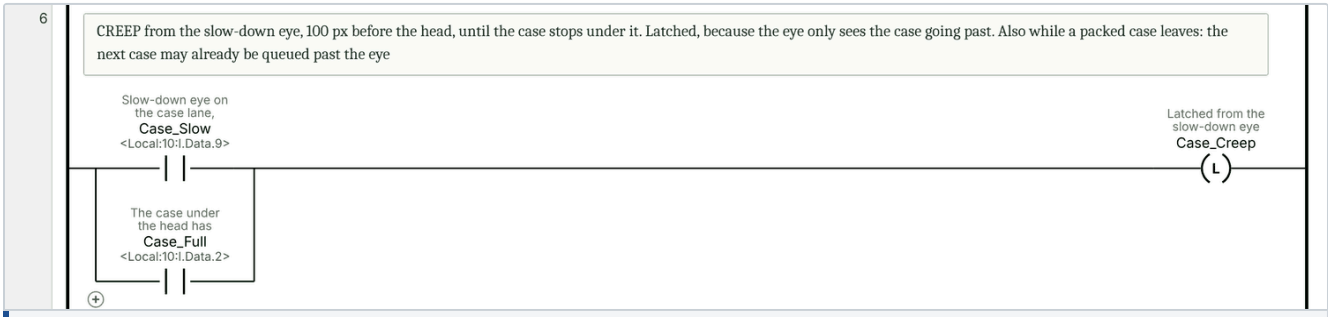
Left to right:

1. `XIC(Pack_Run)` *Examine If Closed* asks: true when `Pack_Run` (The packer is running (its own seal-in)) is on.
2. A branch of 3 parallel paths: power gets through if any one of them is true.
   - **Path 1**
     1. `XIC(Line_Run)` *Examine If Closed* asks: true when `Line_Run` (Seal-in run bit (LogixPro B3:0/0)) is on.
   - **Path 2**
     1. `XIC(Group_Ready)` *Examine If Closed* asks: true when `Group_Ready` (The packer infeed holds a case worth (4)) is on.
   - **Path 3**
     1. `XIC(Lower_Ready)` *Examine If Closed* asks: true when `Lower_Ready` (The packer lower lane holds a case worth (4 small bottles)) is on.
3. A branch of 2 parallel paths: power gets through if any one of them is true.
   - **Path 1**
     1. `XIO(Case_In_Place)` *Examine If Open* asks: true when `Case_In_Place` (A case is centred under the packer head) is off.
   - **Path 2**
     1. `XIC(Case_Full)` *Examine If Closed* asks: true when `Case_Full` (The case under the head has been packed) is on.

4. `OTE(Case_Run)` *Output Energize* does: turns `Case_Run` (Case lane drive: run (two-wire)) on (and off again whenever the rung is false).

> **Think of it like this:** parallel paths are two roads to the same town. If either road is open you get there; the rung only cares that power reaches the far end, not which way it went.

RUNG 6




```
[XIC(Case_Slow),XIC(Case_Full)]OTL(Case_Creep);
```

PREDICT IT · 3 `Case_Slow` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When either (`Case_Slow` (Slow-down eye on the case lane, 100 px before the packer head) is on) *or* (`Case_Full` (The case under the head has been packed) is on), the rung latches `Case_Creep` (Latched from the slow-down eye until an empty case stands under the head) on.

Left to right:

1. A branch of 2 parallel paths: power gets through if any one of them is true.

   Path 1

   1. `XIC(Case_Slow)` *Examine If Closed* asks: true when `Case_Slow` (Slow-down eye on the case lane, 100 px before the packer head) is on.

   Path 2

   1. `XIC(Case_Full)` *Examine If Closed* asks: true when `Case_Full` (The case under the head has been packed) is on.

2. `OTL(Case_Creep)` *Output Latch* does: latches `Case_Creep` (Latched from the slow-down eye until an empty case stands under the head) on. It only ever sets the bit; the bit stays on after the rung goes false.

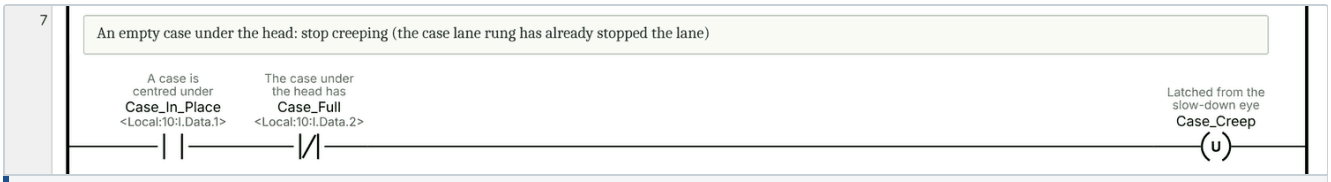
Why it is built this way:

A latch. `Case_Creep` stays on after this rung goes false; an `OTU` elsewhere in the program clears it.

> **Think of it like this:** a light switch instead of a doorbell. OTL flips it on and it stays on when you take your hand away; OTU flips it off. Whatever flipped it on does not have to stay there.

> **Watch for:** A latched bit survives the condition that set it: every way in needs a way out, or it stays on for good.

RUNG 7




```
XIC(Case_In_Place)XIO(Case_Full)OTU(Case_Creep);
```

PREDICT IT · 4 `Case_In_Place` is on and `Case_Full` is off. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

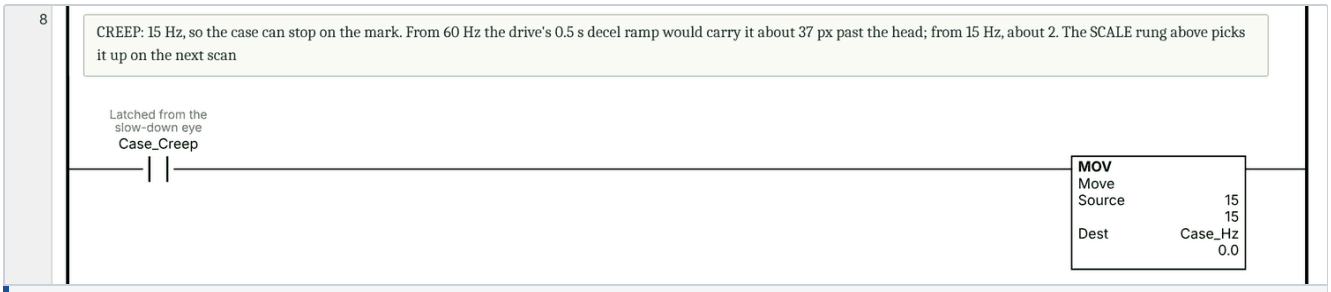
HOW THIS RUNG WORKS

In words. When `Case_In_Place` (A case is centred under the packer head) is on and `Case_Full` (The case under the head has been packed) is off, the rung unlatches `Case_Creep` (Latched from the slow-down eye until an empty case stands under the head) (turns it off).

Left to right:

1. `XIC(Case_In_Place)` *Examine If Closed* asks: true when `Case_In_Place` (A case is centred under the packer head) is on.
2. `XIO(Case_Full)` *Examine If Open* asks: true when `Case_Full` (The case under the head has been packed) is off.
3. `OTU(Case_Creep)` *Output Unlatch* does: unlatches `Case_Creep` (Latched from the slow-down eye until an empty case stands under the head) (turns it off). It only ever clears the bit.

RUNG 8




```
XIC(Case_Creep)MOV(15,Case_Hz);
```

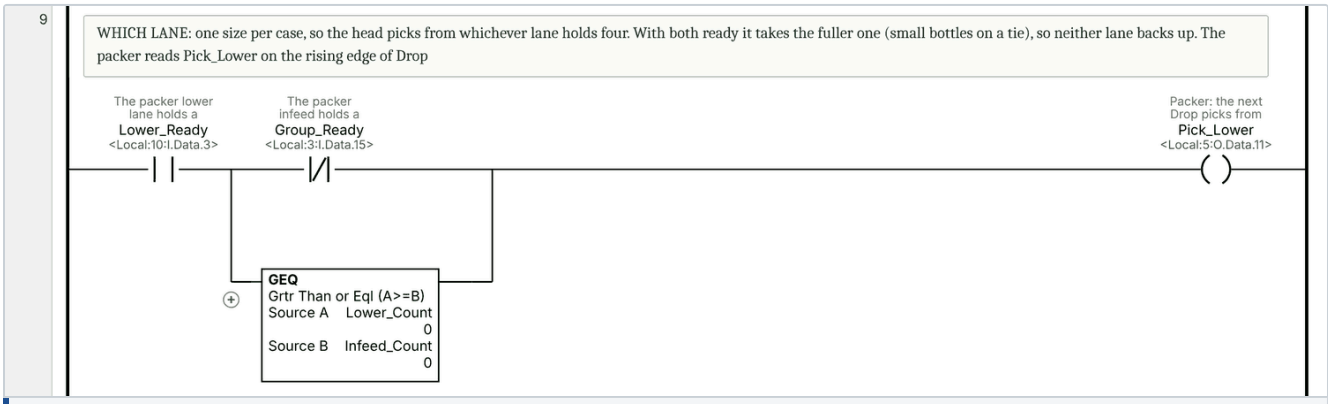
HOW THIS RUNG WORKS

In words. When `Case_Creep` (Latched from the slow-down eye until an empty case stands under the head) is on, the rung puts **15** into `Case_Hz` (Case lane speed the program wants, in Hz).

Left to right:

1. `XIC(Case_Creep)` *Examine If Closed* asks: true when `Case_Creep` (Latched from the slow-down eye until an empty case stands under the head) is on.
2. `MOV(15,Case_Hz)` *Move* does: puts **15** into `Case_Hz` (Case lane speed the program wants, in Hz).

RUNG 9




```
XIC(Lower_Ready)
[XIO(Group_Ready),GEQ(Lower_Count,Infeed_Count)]OTE(Pick_Lower);
```

PREDICT IT · 5 `Lower_Ready` is on and `Group_Ready` is off. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Lower_Ready` (The packer lower lane holds a case worth (4 small bottles)) is on and either (`Group_Ready` (The packer infeed holds a case worth (4)) is off) *or* (`Lower_Count` (Small bottles waiting in the packer lower lane) is at least `Infeed_Count` (Bottles in the packer infeed)), the rung turns `Pick_Lower` (Packer: the next Drop picks from the lower lane (small bottles) instead of the deck infeed) on (and off again whenever the rung is false).

Left to right:

1. `XIC(Lower_Ready)` *Examine If Closed* asks: true when `Lower_Ready` (The packer lower lane holds a case worth (4 small bottles)) is on.
2. A branch of 2 parallel paths: power gets through if any one of them is true.

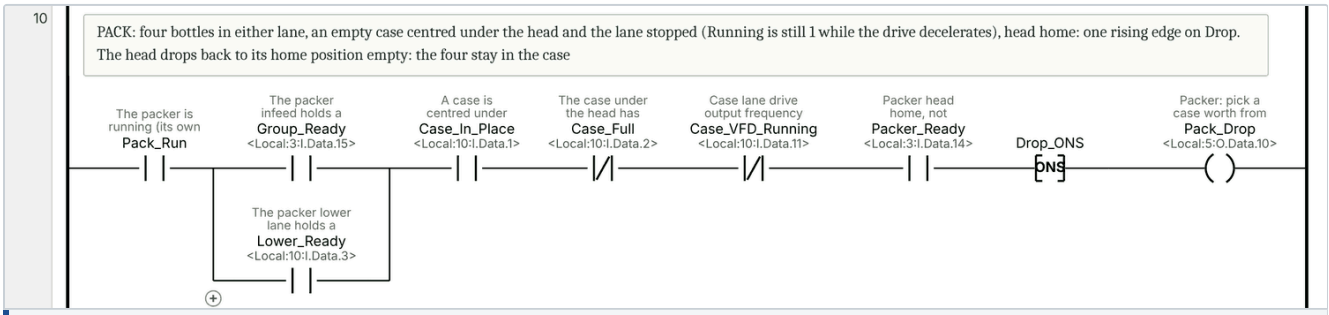
   Path 1

   1. `XIO(Group_Ready)` *Examine If Open* asks: true when `Group_Ready` (The packer infeed holds a case worth (4)) is off.

   Path 2

   1. `GEQ(Lower_Count,Infeed_Count)` *Greater Than or Equal* asks: true when `Lower_Count` (Small bottles waiting in the packer lower lane) is at least `Infeed_Count` (Bottles in the packer infeed).
3. `OTE(Pick_Lower)` *Output Energize* does: turns `Pick_Lower` (Packer: the next Drop picks from the lower lane (small bottles) instead of the deck infeed) on (and off again whenever the rung is false).

RUNG 10




```
XIC(Pack_Run)[XIC(Group_Ready),XIC(Lower_Ready)]XIC(Case_In_Place)XIO(Case_Full)XIO(Case_VFD_Running)XIC(Packer_Ready)ONS(Drop_ONS)OTE(Pack_Drop);
```

HOW THIS RUNG WORKS

In words. When `Pack_Run` (The packer is running (its own seal-in)) is on, either (`Group_Ready` (The packer infeed holds a case worth (4)) is on) *or* (`Lower_Ready` (The packer lower lane holds a case worth (4 small bottles)) is on), `Case_In_Place` (A case is centred under the packer head) is on, `Case_Full` (The case under the head has been packed) is off, `Case_VFD_Running` (Case lane drive output frequency above zero) is off, `Packer_Ready` (Packer head home, not cycling) is on and it is the first scan they are true (the one-shot, remembered in `Drop_ONS`), the rung turns `Pack_Drop` (Packer: pick a case worth from the infeed and place it in the case (rising edge)) on (and off again whenever the rung is false).

Left to right:

1. `XIC(Pack_Run)` *Examine If Closed* asks: true when `Pack_Run` (The packer is running (its own seal-in)) is on.
2. A branch of 2 parallel paths: power gets through if any one of them is true.

   Path 1

   1. `XIC(Group_Ready)` *Examine If Closed* asks: true when `Group_Ready` (The packer infeed holds a case worth (4)) is on.

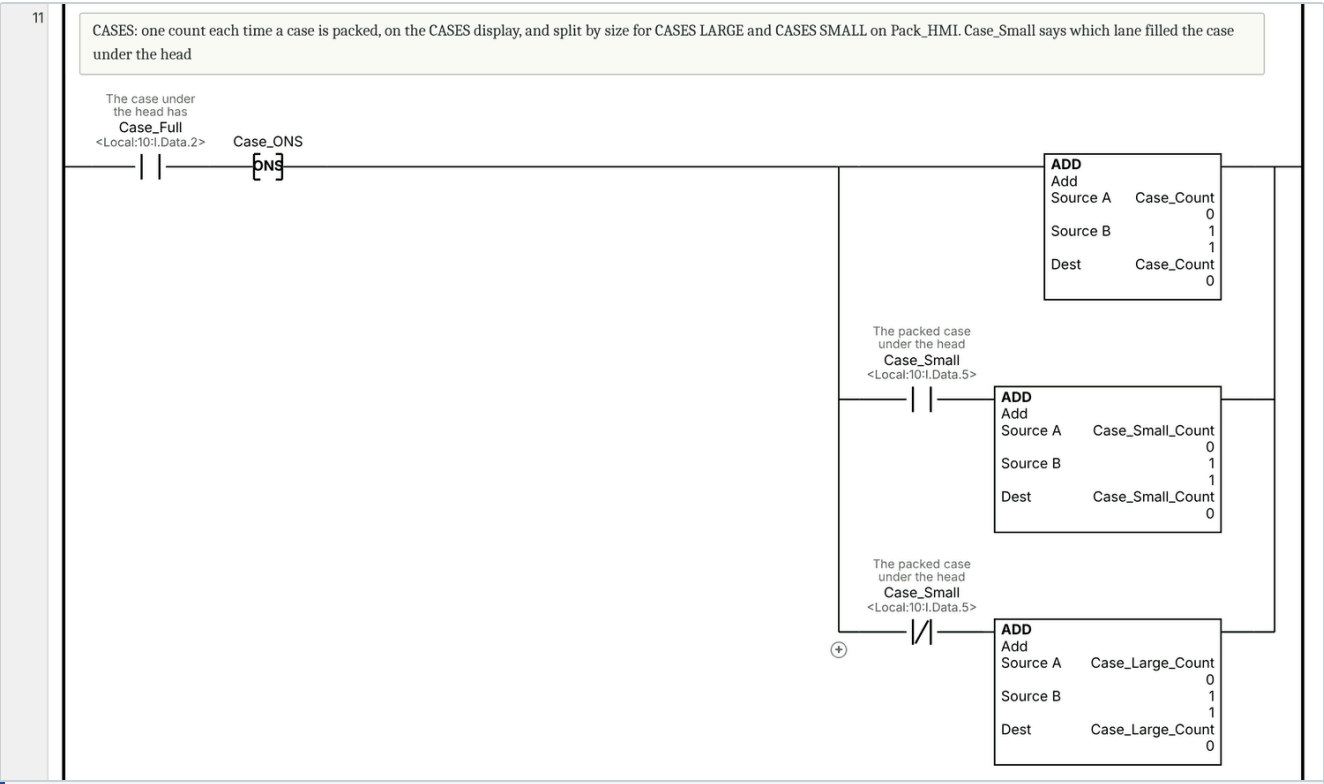
   Path 2

   1. `XIC(Lower_Ready)` *Examine If Closed* asks: true when `Lower_Ready` (The packer lower lane holds a case worth (4 small bottles)) is on.
3. `XIC(Case_In_Place)` *Examine If Closed* asks: true when `Case_In_Place` (A case is centred under the packer head) is on.
4. `XIO(Case_Full)` *Examine If Open* asks: true when `Case_Full` (The case under the head has been packed) is off.
5. `XIO(Case_VFD_Running)` *Examine If Open* asks: true when `Case_VFD_Running` (Case lane drive output frequency above zero) is off.
6. `XIC(Packer_Ready)` *Examine If Closed* asks: true when `Packer_Ready` (Packer head home, not cycling) is on.
7. `ONS(Drop_ONS)` *One Shot* asks: true when it is the first scan they are true (the one-shot, remembered in `Drop_ONS`).
8. `OTE(Pack_Drop)` *Output Energize* does: turns `Pack_Drop` (Packer: pick a case worth from the infeed and place it in the case (rising edge)) on (and off again whenever the rung is false).

Why it is built this way:

One-shot: once per event, not once per scan.

RUNG 11




```
XIC(Case_Full)ONS(Case_ONS)
[ADD(Case_Count,1,Case_Count),XIC(Case_Small)ADD(Case_Small_Count,1,Case_Small_
Count),XIO(Case_Small)ADD(Case_Large_Count,1,Case_Large_Count)];
```

HOW THIS RUNG WORKS

In words. When `Case_Full` (The case under the head has been packed) is on and it is the first scan they are true (the one-shot, remembered in `Case_ONS`), the rung picks its path: in any case it makes `Case_Count` (Cases packed, shown on the CASES display) = `Case_Count` + **1**; if `Case_Small` (The packed case under the head holds small bottles) is on, it makes `Case_Small_Count` (Cases of small bottles packed (CASES SMALL on Pack_HMI)) = `Case_Small_Count` + **1**; if `Case_Small` is off, it makes `Case_Large_Count` (Cases of large bottles packed (CASES LARGE on Pack_HMI)) = `Case_Large_Count` + **1**.

Left to right:

1. `XIC(Case_Full)` *Examine If Closed* asks: true when `Case_Full` (The case under the head has been packed) is on.
2. `ONS(Case_ONS)` *One Shot* asks: true when it is the first scan they are true (the one-shot, remembered in `Case_ONS`).
3. A branch of 3 parallel paths: each path is tested on its own and does its own work.

   Path 1

   1. `ADD(Case_Count,1,Case_Count)` *Add* does: makes `Case_Count` (Cases packed, shown on the CASES display) = `Case_Count` + **1**. Every scan the rung is true; behind a one-shot it adds once per event.

   Path 2

   1. `XIC(Case_Small)` *Examine If Closed* asks: true when `Case_Small` (The packed case under the head holds small bottles) is on.

2. `ADD(Case_Small_Count,1,Case_Small_Count)` *Add* does: makes `Case_Small_Count` (Cases of small bottles packed (CASES SMALL on Pack_HMI)) = `Case_Small_Count` + **1**.

Path 3

1. `XIO(Case_Small)` *Examine If Open* asks: true when `Case_Small` is off.
2. `ADD(Case_Large_Count,1,Case_Large_Count)` *Add* does: makes `Case_Large_Count` (Cases of large bottles packed (CASES LARGE on Pack_HMI)) = `Case_Large_Count` + **1**.

Why it is built this way:

One-shot: once per event, not once per scan.

RUNG 12

```
XIC(Enter)[CLR(Case_Count),CLR(Case_Large_Count),CLR(Case_Small_Count)];
```

HOW THIS RUNG WORKS

In words. When `Enter` (Thumbwheel Enter button) is on, the rung clears `Case_Count` (Cases packed, shown on the CASES display) to 0, clears `Case_Large_Count` (Cases of large bottles packed (CASES LARGE on Pack_HMI)) to 0 and clears `Case_Small_Count` (Cases of small bottles packed (CASES SMALL on Pack_HMI)) to 0.

Left to right:

1. `XIC(Enter)` *Examine If Closed* asks: true when `Enter` (Thumbwheel Enter button) is on.
2. A branch of 3 parallel paths: each path is tested on its own and does its own work.

Path 1

1. `CLR(Case_Count)` *Clear* does: clears `Case_Count` (Cases packed, shown on the CASES display) to 0.

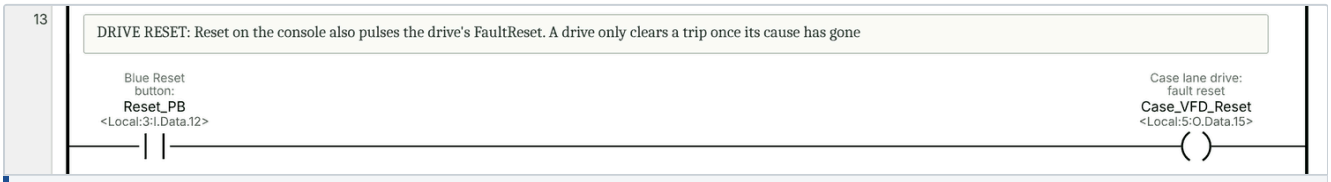
Path 2

1. `CLR(Case_Large_Count)` *Clear* does: clears `Case_Large_Count` (Cases of large bottles packed (CASES LARGE on Pack_HMI)) to 0.

Path 3

1. `CLR(Case_Small_Count)` *Clear* does: clears `Case_Small_Count` (Cases of small bottles packed (CASES SMALL on Pack_HMI)) to 0.

RUNG 13




```
XIC(Reset_PB)OTE(Case_VFD_Reset);
```

HOW THIS RUNG WORKS

In words. When `Reset_PB` (Blue Reset button: acknowledge alarms and clear the ones whose cause is gone) is on, the rung turns `Case_VFD_Reset` (Case lane drive: fault reset (rising edge)) on (and off again whenever the rung is false).

Left to right:

1. `XIC(Reset_PB)` *Examine If Closed* asks: true when `Reset_PB` (Blue Reset button: acknowledge alarms and clear the ones whose cause is gone) is on.
2. `OTE(Case_VFD_Reset)` *Output Energize* does: turns `Case_VFD_Reset` (Case lane drive: fault reset (rising edge)) on (and off again whenever the rung is false).

Rungs 0–1 give the packer a run seal-in of its own, the same pattern as the mix kitchen's in step 13: a one-scan `Pack_Auto_Start` when the line starts, so the console's Start starts the packer too, and a seal-in on `Pack_Run` that START and STOP on Pack_HMI also work. Stop the packer while the line runs and its lanes fill; when the deck infeed is full, step 11's bit 7 stops the line. Stop the line and the packer keeps packing what is waiting.

Rungs 2–4 come first, because without them nothing on the case lane moves: the drive runs at whatever its speed reference says, and that is 0 until the program writes one. **Rung 2** sets the fast speed, 60 Hz, whenever the case is not creeping. **Rung 3** is the analog output: SCP maps 0-60 Hz onto 0-10000, so 60 Hz goes out as 10000 (10 V) and 15 Hz as 2500 (2.5 V). **Rung 4** is the same instruction the other way round: the drive's 0-10 V output frequency back to Hz in `Case_Hz_Actual`, a REAL, so you can watch the ramps in Monitor Tags.

Rung 5 runs the lane whenever no *empty* case stands under the head: either nothing is in place (bring the next case in) or the case in place is full (carry it out). It runs with the line, and also after the line has stopped as long as either lane holds a case worth — or the lanes could never empty. **Rungs 6–7** are the creep, **latched**: the slow-down eye only sees the case going past it, and the case still has 60 px to go when the eye clears. The latch is also set while a packed case leaves, because the next case may already be queued past the eye and must not start for the head at full speed. An empty case under the head ends the creep. **Rung 8** is the creep speed, 15 Hz, while the latch is on. Rung 2 only writes 60 Hz while the case is not creeping, so the two never fight over `Case_Hz`; rung 8 sits below the scale rung, so the 15 Hz reaches the drive on the next scan, which a drive on a 0.5 s decel ramp never notices.

Rung 9 chooses the lane. The packer reads its `Pick_Lower` output on the rising edge of Drop: 1 picks from the lower lane, 0 from the deck. One size per case means the head picks from a lane that holds four, so the rung starts with `XIC(Lower_Ready)`. If only the lower lane is ready, `XIO(Group_Ready)` passes. If both are, `GEQ(Lower_Count,Infeed_Count)` takes the fuller lane (small on a tie), so neither lane backs up while the other is served. Leave `Pick_Lower` at 0 and only large bottles are ever packed; the lower lane fills and step 8 stops the divert conveyor at LS12.

Rung 10 packs: the packer running, a case worth in either lane (`Group_Ready` or `Lower_Ready`), an empty case on the mark, the head home, and the lane *stopped*. `XIO(Case_VFD_Running)`, not `XIO(Case_Run)`: Run drops the moment the case is in place, but the drive is still ramping down for a few scans, and a head that comes down on a moving case misses it. **Rung 11** counts the case on the CASES display and by size for Pack_HMI. The packer's `Case_Small` input says the packed case under the head came from the lower lane, so one branch adds to `Case_Small_Count` and the other to `Case_Large_Count`. **Rung 12** lets Enter clear the three counts with the others, and **rung 13** sends Reset to the drive's FaultReset: a drive clears a trip only on that rising edge, and only once the cause has gone.

Run it

Start the line and put `Case_Hz`, `Case_Speed_Ref`, `Case_Hz_Actual`, `Case_In_Place` and `Infeed_Count` in Monitor Tags (or a Watch). The first carton comes in at 60 Hz — `Case_Speed_Ref` 10000, `Case_Hz_Actual` climbing on the 1 s accel ramp — reaches the slow-down eye, drops to 2500 and creeps, and stops under the head: `Case_In_Place` 1, the drive's keypad reads 0.0. Bottles collect in both lanes. When one lane holds four, the head picks them: from the deck it comes straight down; from the lower lane it runs out over the floor conveyor and drops 1.7 m (Pack_HMI shows PICK LOWER and the Z coming down). Then it lowers them into the case and goes home empty, and the case leaves for Case_Out with four bottle tops showing. CASES counts 1, and CASES LARGE or CASES SMALL on Pack_HMI. In the test run five minutes packed sixteen cases, seven large and nine small, every one a single size, with nothing dropped or lost.

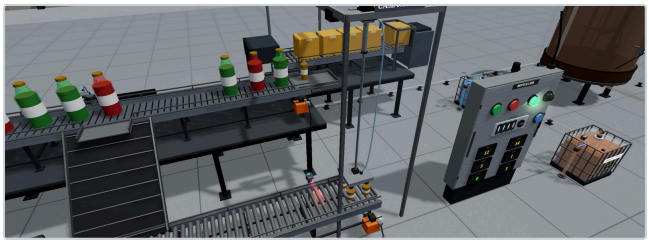


Figure 116. In 3D: the head down at the end of the divert conveyor, its cups on four small bottles at the lower stop, LS12 just upstream.

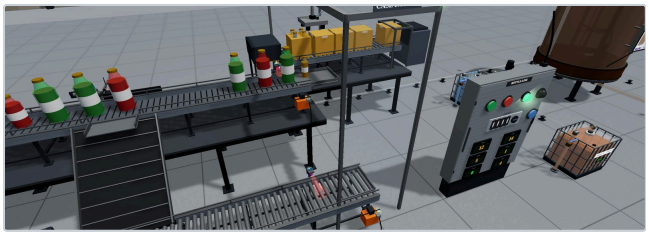


Figure 117. After the place: the head at home with its cups empty, waiting for the next four; the packed case is on its way out.

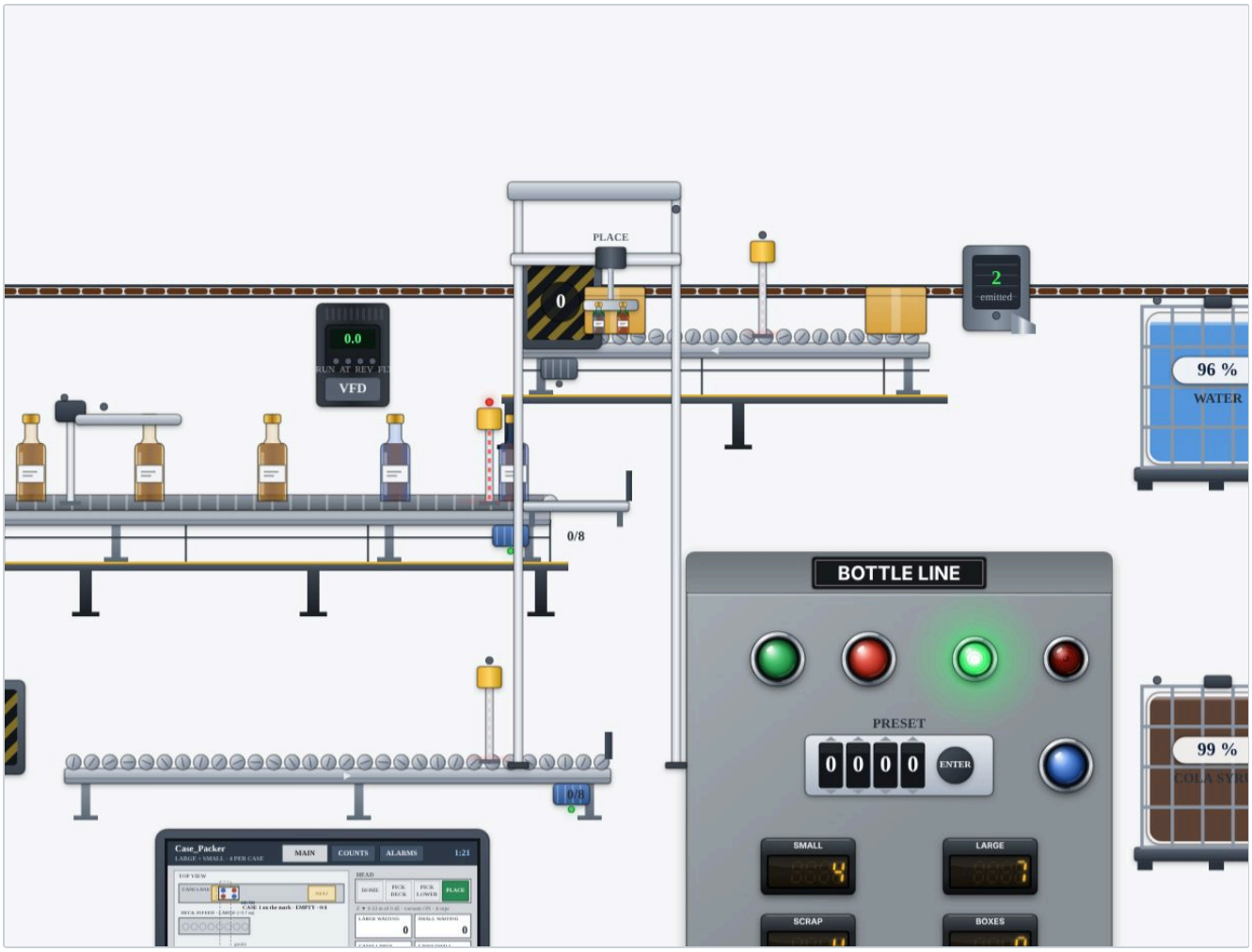
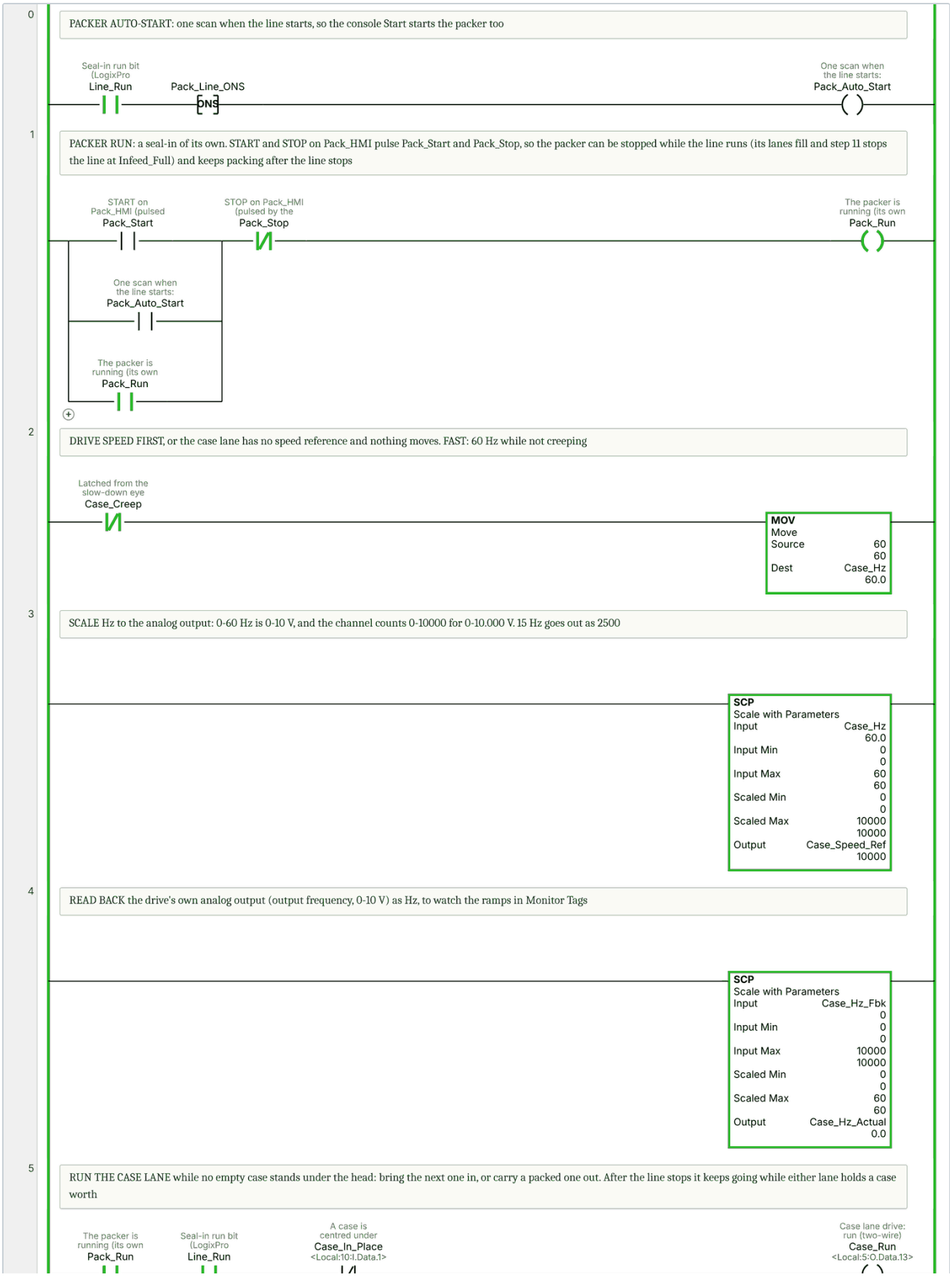
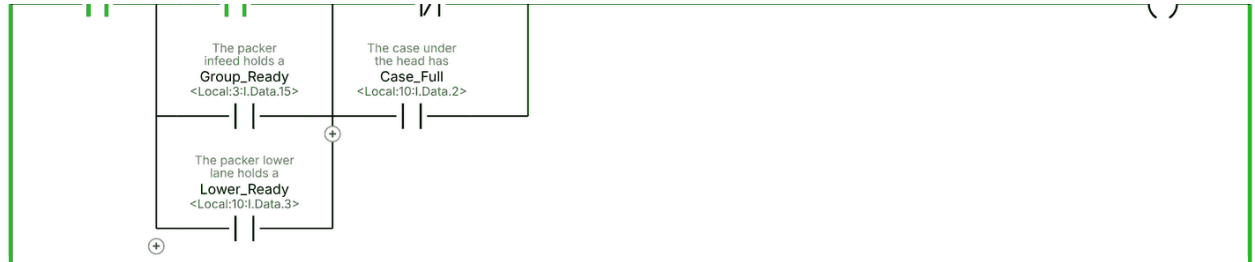


Figure 118. The packer in Side view (from v1.14): each lane on its own row as everything is in Side view, the deck infeed with its table, the case lane above it and the floor conveyor below; the portal posts and beam, the carriage at the row the head is working, and the head lowering four small bottles into the case.

WATCH FOR: THE STOP THAT IS NOT A STOP

Take rung 6 out (or set the creep to 60 Hz) and run: the case reaches the mark, `Case_Run` drops, and the case sails on 30-odd px on the decel ramp, out of the window; `Case_In_Place` goes back to 0, rung 2 runs the lane again, and the case leaves empty. Nothing in the program is wrong except the physics. Every positioning problem with a drive is this one: the stopping distance is speed × decel time ÷ 2.


0
PACKER AUTO-START: one scan when the line starts, so the console Start starts the packer too
Seal-in run bit (LogixPro
Line_Run
Pack_Line_ONS
ONS
One scan when the line starts:
Pack_Auto_Start
1
PACKER RUN: a seal-in of its own. START and STOP on Pack_HMI pulse Pack_Start and Pack_Stop, so the packer can be stopped while the line runs (its lanes fill and step 11 stops the line at Infeed_Full) and keeps packing after the line stops
START on Pack_HMI (pulsed
Pack_Start
STOP on Pack_HMI (pulsed by the
Pack_Stop
The packer is running (its own
Pack_Run
One scan when the line starts:
Pack_Auto_Start
The packer is running (its own
Pack_Run
2
DRIVE SPEED FIRST, or the case lane has no speed reference and nothing moves. FAST: 60 Hz while not creeping
Latched from the slow-down eye
Case_Creep
MOV
Move
Source 60
60
Dest Case_Hz
60.0
3
SCALE Hz to the analog output: 0-60 Hz is 0-10 V, and the channel counts 0-10000 for 0-10.000 V. 15 Hz goes out as 2500
SCP
Scale with Parameters
Input Case_Hz
60.0
Input Min 0
0
Input Max 60
60
Scaled Min 0
0
Scaled Max 10000
10000
Output Case_Speed_Ref
10000
4
READ BACK the drive's own analog output (output frequency, 0-10 V) as Hz, to watch the ramps in Monitor Tags
SCP
Scale with Parameters
Input Case_Hz_Fbk
0
Input Min 0
0
Input Max 10000
10000
Scaled Min 0
0
Scaled Max 60
60
Output Case_Hz_Actual
0.0
5
RUN THE CASE LANE while no empty case stands under the head: bring the next one in, or carry a packed one out. After the line stops it keeps going while either lane holds a case worth
The packer is running (its own
Pack_Run
Seal-in run bit (LogixPro
Line_Run
A case is centred under
Case_In_Place
<Local:10:I.Data.1>
Case lane drive: run (two-wire)
Case_Run
<Local:5:O.Data.13>



6

CREEP from the slow-down eye, 100 px before the head, until the case stops under it. Latched, because the eye only sees the case going past. Also while a packed case leaves: the next case may already be queued past the eye

7

An empty case under the head: stop creeping (the case lane rung has already stopped the lane)

8

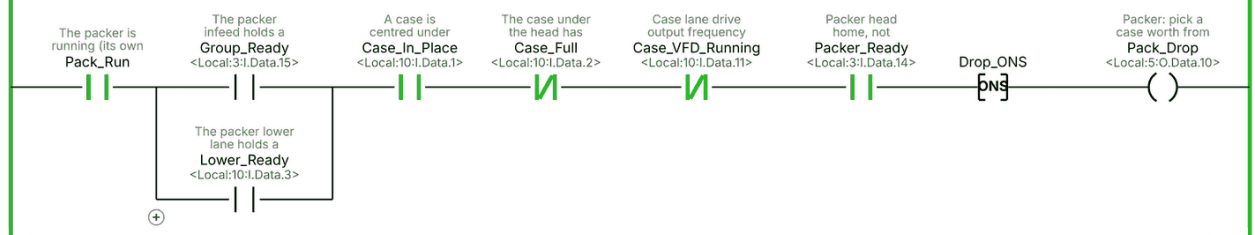
CREEP: 15 Hz, so the case can stop on the mark. From 60 Hz the drive's 0.5 s decel ramp would carry it about 37 px past the head; from 15 Hz, about 2. The SCALE rung above picks it up on the next scan

9

WHICH LANE: one size per case, so the head picks from whichever lane holds four. With both ready it takes the fuller one (small bottles on a tie), so neither lane backs up. The packer reads Pick_Lower on the rising edge of Drop

10

PACK: four bottles in either lane, an empty case centred under the head and the lane stopped (Running is still 1 while the drive decelerates), head home: one rising edge on Drop. The head drops back to its home position empty: the four stay in the case



11

CASES: one count each time a case is packed, on the CASES display, and split by size for CASES LARGE and CASES SMALL on Pack_HMI. Case_Small says which lane filled the case under the head

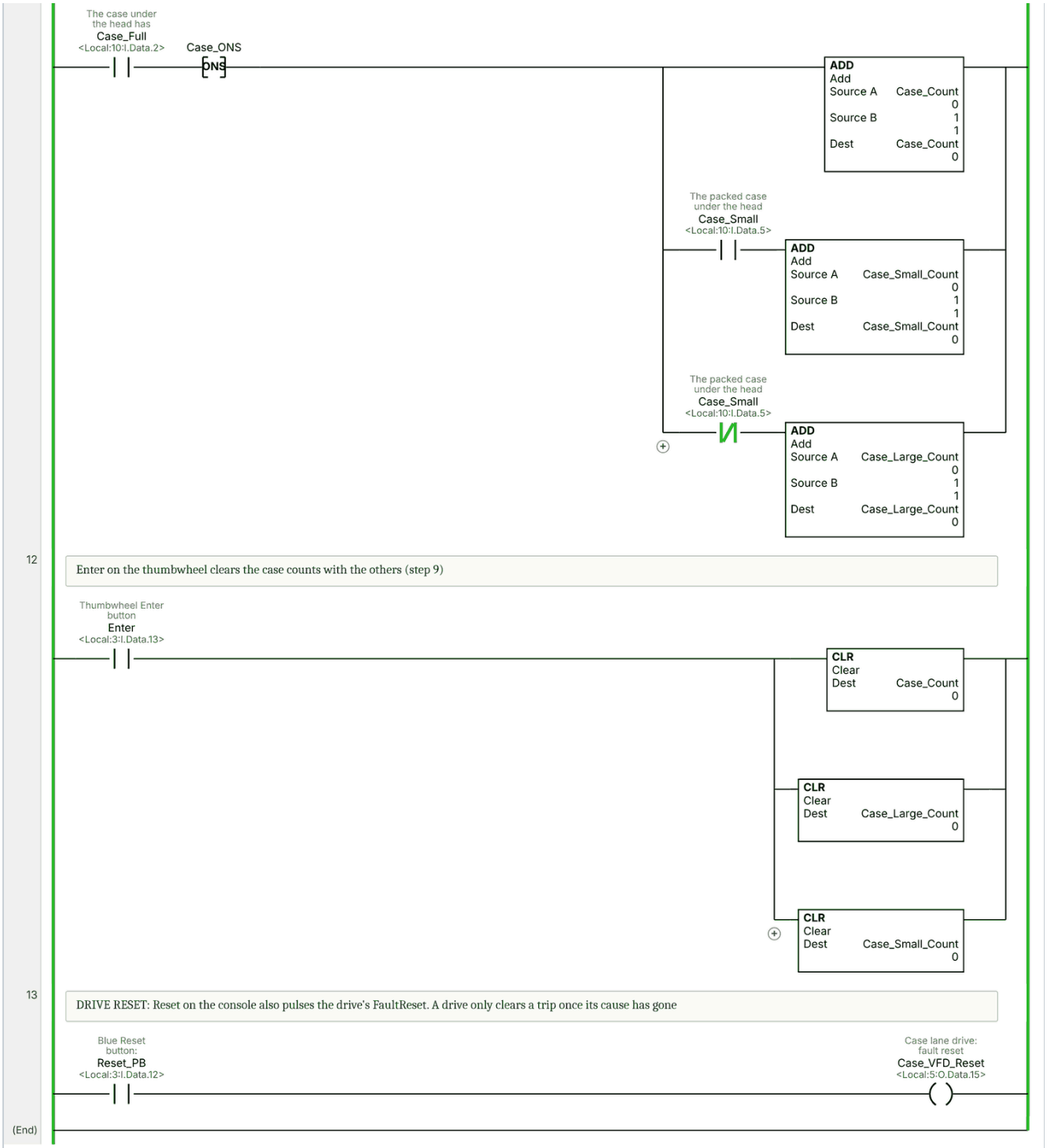


Figure 134. S12_Pack online, with the finished line running: this is what your routine should look like once every step is in. Green is power flow.

TRY IT

Press STOP on Pack_HMI while the line runs: the head finishes its cycle and stops, the lower lane fills and the divert conveyor stops at LS12, the deck infeed fills and bit 7 stops the line. Press START, Reset, Start, and watch both lanes drain. Change rung 9 to `XIC(Lower_Ready)XIO(Group_Ready)OTE(Pick_Lower);` (large first) and see which lane backs up. Set the creep to 30 Hz (5000) and count how often the case still lands in the window. Then set Case_VFD's *Decel* to 2 s: the creep has to come down to make it work again. Last, Tools → Inject Machine Fault → Drive trip on Case_VFD: bit 6 latches, the lane coasts to a stop, the infeed fills — and when it reaches eight, bit 7 stops the line. Clear Machine Faults, press Reset, watch the drive come back and the infeed drain, Reset again, Start.

Predict it: the answers

1. Every condition on the way is met, so `Pack_Run` is **on**.
2. Every condition on the way is met, so `Case_Run` is **on**.
3. Every condition on the way is met, so `Case_Creep` latches **on**.
4. Every condition on the way is met, so `Case_Creep` is unlatched, **off**.
5. Every condition on the way is met, so `Pick_Lower` is **on**.

30 Lesson 15 — Step 13: the mix kitchen

Until now `Supply_Tank` has been filled from the plant supply header: open the inlet and soda arrives, as much as you like. A real filling line rarely works like that. The product is made a batch at a time in a **mix kitchen** beside the line and pumped across to a **surge tank** in front of the filler, which is what Supply_Tank becomes in this lesson. The kitchen is the Mix Tank of Level 7 (chapter 39) with two supply totes, water on inlet A and cola syrup on inlet B, a **carbonator** after it, and a Process HMI to run it (chapter 14). Soda is made here, not bought: water and syrup at the 5 : 1 throw, then chilled and carbonated on the way to the filler.

Steps 1 to 12 do not change. Step 6 still opens the Supply_Tank inlet at the low float and closes it at the high float. What changes is where that inlet draws from: it is piped from `Mix_Tank`, and liquid only moves while the mix tank's outlet pump runs. So step 13 has to have a batch ready whenever step 6 asks, and pump it across while the inlet is open. If it is late, step 11's alarm bit 5 (inlet open ten seconds at the low float) is the first to tell you, and step 6's head-tank protection the second.

Place the kitchen

The kitchen stands to the right of the console, so first set the floor to **2400 × 600** (Floor, on the simulator toolbar). Place:

- **Supply Tote (an IBC of raw ingredient that tanks pump from)** at **x 1720, y 130**, named `Supply_A`: *Holds (units)* = **20000**. Water, a big tote: it lasts a long shift.
- **Supply Tote (an IBC of raw ingredient that tanks pump from)** at **x 1720, y 380**, named `Supply_B`: *Ingredient* = **Cola syrup**; *Holds (units)* = **20000**. Cola syrup: five parts of water go with each part of it, so it lasts five times as long as the water.
- **Mix Tank (batch: pumps, flowmeters, heater, thermostat, mixer)** at **x 2020, y 300**, named `Mix_Tank`: *Product (picked on the RECIPE page of its HMI)* = **Soda**; *Starts at (%)* = **90**; *Pump A flow (units/s)* = **150**; *Pump B flow (units/s)* = **150**; *LO level probe (%)* = **15**; *Panel and HMI: heater setpoint tag (a REAL the operator sets on the TEMP page)* = **Mix_Target_T**; *Recipe: A share the HMI aims for (%)* = **83**; *Recipe: mixing time (s)* = **8**; *Starts holding a finished batch: the start level made to the recipe A share and already mixed* = **True**; *Inlet A fed from* = **Supply_A**; *Inlet B fed from* = **Supply_B**. Inlet A from Supply_A, inlet B from Supply_B; HI 90 %, LO 15 %; the heater setpoint tag is the one Tank_HMI writes; it starts at 90 % holding a finished batch, so the first transfer does not wait for a batch to be made.
- **Carbonator (carbo-cooler: chills the blend and dissolves CO2)** at **x 2300, y 330**, named `Carbonator`: *Fed from* = **Mix_Tank**; *Starts at (psi)* = **20**. To the right of the tank, fed from Mix_Tank. It chills what goes through to 38 °F and dissolves CO2 in it; it starts at 20 psi, pressurised from the last shift.

- **Process HMI (operator terminal: a live faceplate of a tank, a case packer or a camera)** at **x 2050, y 490**, named `Tank_HMI`: *Shows* = **Mix_Tank**; *Heater setpoint tag (REAL; blank = the tank's own)* = **Mix_Target_T**; *START button tag (BOOL, pulsed while touched)* = **Mix_Start**; *STOP button tag (BOOL, pulsed while touched)* = **Mix_Stop**; *Running indicator tag (BOOL: lights START)* = **Mix_Run**; *Recipe A share tag (REAL, %)* = **Mix_Ratio**; *Recipe mixing time tag (REAL, s)* = **Mix_Time**. In front of the tank, where the operator stands. Its START, STOP and Running tags are the kitchen's own; its Ratio and Mix time tags take the RECIPE page's values. The product, SODA, is picked on its RECIPE page.

Then select **Supply_Tank** and set *Inlet fed from* to **Carbonator**. The transfer line runs from the mix tank's outlet pump through the carbonator and along a pipe rack at the back of the floor to the reservoir's inlet. In the app, lesson 15's *Place the kitchen for me* does all of this in one click.

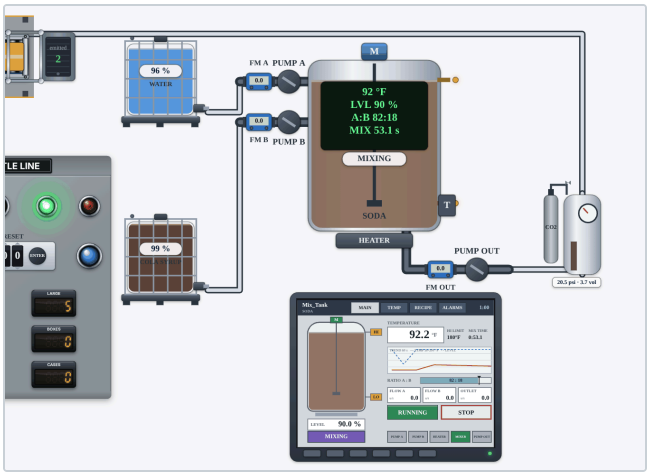


Figure 119. The mix kitchen in Top-down view: the water and cola syrup totes, Mix_Tank with its meters and pumps, the carbonator (CO2 cylinder, gauge, 20.5 psi · 3.7 vol), the transfer line up to the pipe rack, and Tank_HMI in front.

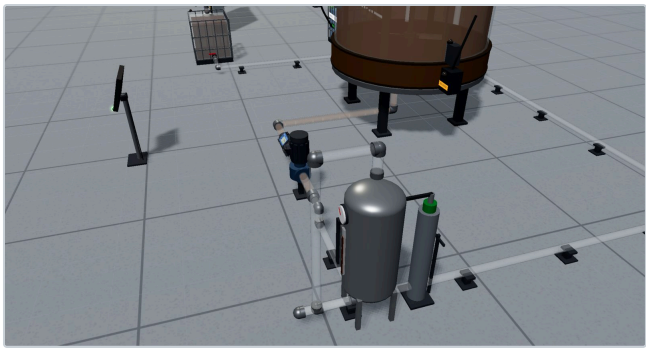


Figure 120. In 3D: the outlet pump, the carbonator with its CO2 cylinder and gauge, and the line on to Supply_Tank.

The kitchen needs nine digital inputs, six outputs and four analog inputs (the tank's level and temperature, the carbonator's pressure and CO2 volumes). From Version 99 the sandbox chassis has a third 1769-IF4, in slot 11; the kitchen fills it.

The batch

One percent of Mix_Tank's level is 20 units, and one pulse of a flowmeter is 20 units, so the room left to the HI probe (90 %) is `90 - Mix_Level` pulses. A batch tops the tank up to HI: the water dose is the recipe's water share of that room, counted on flowmeter A, and the syrup runs until HI. From an empty tank at the soda recipe's 83 % water that is 75 pulses of water, 1500 units, and 300 of syrup: five to one. Then the batch is heated with a 2 °F band and mixed for the mixing time once it is up to temperature, and it is ready.

Why top up, and not wait for the tank to empty the way Level 7 does? Because the transfer is not up to the kitchen. Step 6 closes the inlet at the high float, whatever is left in the mix tank, and opens it again at the low float, ready or not. A kitchen that waits for an empty tank before it makes the next batch will sooner or later be half-way through one when the reservoir calls, and the reservoir only lasts about twenty seconds below its low float. Topping up the moment a transfer ends means a full, mixed batch is standing in the tank every time Supply_Tank asks. What was left from the last batch is already at the recipe, so topping it up at the recipe keeps the whole tank at the recipe.

Add the JSR under step 12 and write:

> **IN THE APP**
>
> Guided mode lesson 15 is this chapter: it checks that this lesson's tags are made, that `S13_Mix` exists, is called from MainRoutine and verifies, and then watches the line while you run it and ticks each *Run it* result below as it happens. Help → Guided Mode, or start from File → New Project… → Guided: build the bottle line.

Tags this lesson needs

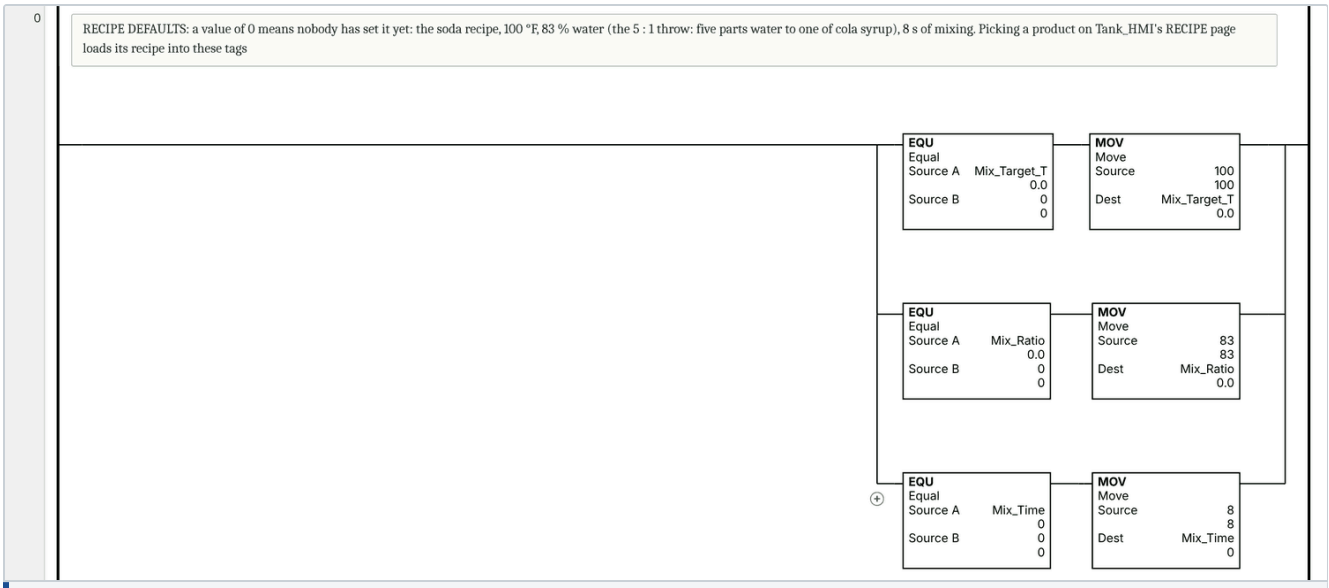
Create these before you type the rungs (Verify reports any you miss as unknown). Tags from earlier lessons are not repeated.

Tag	Type	How to make it	Description
`Mix_Target_T`	REAL	Edit Tags: type the name, pick the type	Batch temperature, °F: the setpoint on Tank_HMI (TEMP page)
`Mix_Ratio`	REAL	Edit Tags: type the name, pick the type	Water share of the batch, %: A SHARE on the RECIPE page
`Mix_Time`	DINT	Edit Tags: type the name, pick the type	Mixing time, s: MIX TIME on the RECIPE page
`Mix_Line_ONS`	BOOL	Edit Tags: type the name, pick the type	
`Mix_Auto_Start`	BOOL	Edit Tags: type the name, pick the type	One scan when the line starts: starts the kitchen too
`Mix_Start`	BOOL	Edit Tags: type the name, pick the type	START on Tank_HMI (pulsed by the touch screen)
`Mix_Run`	BOOL	Edit Tags: type the name, pick the type	The mix kitchen is running (its own seal-in)
`Mix_Stop`	BOOL	Edit Tags: type the name, pick the type	STOP on Tank_HMI (pulsed by the touch screen)
`Mix_Ms`	DINT	Edit Tags: type the name, pick the type	Mixing time in ms (Mix_Time × 1000)

Tag	Type	How to make it	Description
`Mix_Lo_T`	REAL	Edit Tags: type the name, pick the type	Bottom of the heater band: 2 °F under Mix_Target_T
`Mix_Step`	DINT	Edit Tags: type the name, pick the type	Kitchen sequence: 0 empty, 1 dose water, 2 syrup to HI, 3 heat and mix, 4 ready / transfer
`Mix_Hi`	BOOL	alias for `Local:12:I.Data.3` — the Properties drawer's *tag* link makes it, then rename it	Mix_Tank HI probe (90 %)
`Mix_PumpOut`	BOOL	alias for `Local:6:O.Data.2` — the Properties drawer's *tag* link makes it, then rename it	Mix_Tank outlet pump: transfers to Supply_Tank
`Mix_Level`	INT	alias for `Local:11:I.Ch0Data` — the Properties drawer's *tag* link makes it, then rename it	Mix_Tank level, 0-100 %
`Water_Low`	BOOL	alias for `Local:10:I.Data.14` — the Properties drawer's *tag* link makes it, then rename it	Supply_A (water tote) at its low switch
`Conc_Low`	BOOL	alias for `Local:10:I.Data.15` — the Properties drawer's *tag* link makes it, then rename it	Supply_B (syrup tote) at its low switch
`Dose_Pre`	DINT	Edit Tags: type the name, pick the type	Water dose for this batch, in flowmeter pulses
`Dose_A`	COUNTER	Edit Tags: type the name, pick the type	Water dose: flowmeter A pulses
`Mix_PumpA`	BOOL	alias for `Local:6:O.Data.0` — the Properties drawer's *tag* link makes it, then rename it	Mix_Tank pump A: water
`Mix_FlowA`	BOOL	alias for `Local:12:I.Data.0` — the Properties drawer's *tag* link makes it, then rename it	Mix_Tank flowmeter A: one pulse per 20 units
`Mix_PumpB`	BOOL	alias for `Local:6:O.Data.1` — the Properties drawer's *tag* link makes it, then rename it	Mix_Tank pump B: syrup
`Mix_Lo`	BOOL	alias for `Local:12:I.Data.4` — the Properties drawer's *tag* link makes it, then rename it	Mix_Tank LO probe (15 %)
`Mix_Mixer`	BOOL	alias for `Local:6:O.Data.4` — the Properties drawer's *tag* link makes it, then rename it	Mix_Tank mixer

Tag	Type	How to make it	Description
`Mix_Temp`	INT	alias for `Local:11:I.Ch1Data` — the Properties drawer's *tag* link makes it, then rename it	Mix_Tank temperature, °F
`Mix_Heat`	BOOL	Edit Tags: type the name, pick the type	Heater wanted (latched under the band, unlatched at the setpoint)
`Mix_Thermostat`	BOOL	alias for `Local:12:I.Data.5` — the Properties drawer's *tag* link makes it, then rename it	Mix_Tank high-limit thermostat (180 °F)
`Mix_Heater`	BOOL	alias for `Local:6:O.Data.3` — the Properties drawer's *tag* link makes it, then rename it	Mix_Tank heater
`Mix_Timer`	TIMER	Edit Tags: type the name, pick the type	Mixing time at temperature
`Mix_Empty_T`	TIMER	Edit Tags: type the name, pick the type	LO uncovered this long: the tank is empty
`Carb_Press`	INT	alias for `Local:11:I.Ch2Data` — the Properties drawer's *tag* link makes it, then rename it	Carbonator tank pressure, psi
`Carb_CO2`	BOOL	alias for `Local:6:O.Data.5` — the Properties drawer's *tag* link makes it, then rename it	Carbonator CO2 inlet valve

RUNG 0




```
[EQU(Mix_Target_T,0.0)MOV(100.0,Mix_Target_T),EQU(Mix_Ratio,0.0)MOV(83.0,Mix_Ratio),EQU(Mix_Time,0)MOV(8,Mix_Time)];
```

PREDICT IT · 1 `Mix_Target_T` is 1, `Mix_Ratio` is 0 and `Mix_Time` is 1. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. Every scan, with nothing to test, the rung picks its path: if `Mix_Target_T` (Batch temperature, °F: the setpoint on Tank_HMI (TEMP page)) equals **0.0**, it puts **100.0** into `Mix_Target_T` ; if `Mix_Ratio` (Water share of the batch, %: A SHARE on the RECIPE page) equals **0.0**, it puts **83.0** into `Mix_Ratio` ; if `Mix_Time` (Mixing time, s: MIX TIME on the RECIPE page) equals **0**, it puts **8** into `Mix_Time` .

Left to right:

1. A branch of 3 parallel paths: each path is tested on its own and does its own work.

   Path 1

   1. `EQU(Mix_Target_T,0.0)` *Equal* asks: true when `Mix_Target_T` (Batch temperature, °F: the setpoint on Tank_HMI (TEMP page)) equals **0.0**. Both values are compared every scan.
   2. `MOV(100.0,Mix_Target_T)` *Move* does: puts **100.0** into `Mix_Target_T` . It writes every scan the rung is true.

   Path 2

   1. `EQU(Mix_Ratio,0.0)` *Equal* asks: true when `Mix_Ratio` (Water share of the batch, %: A SHARE on the RECIPE page) equals **0.0**.
   2. `MOV(83.0,Mix_Ratio)` *Move* does: puts **83.0** into `Mix_Ratio` .

   Path 3

   1. `EQU(Mix_Time,0)` *Equal* asks: true when `Mix_Time` (Mixing time, s: MIX TIME on the RECIPE page) equals **0**.
   2. `MOV(8,Mix_Time)` *Move* does: puts **8** into `Mix_Time` .

Why it is built this way:

A step of a sequence. The rung only acts while `Mix_Time` is 0; when its conditions are met it writes 8 into `Mix_Time`, and from then on the rungs that test for 8 take over. One step number at a time means one thing happening at a time.

> **Think of it like this:** a recipe card with numbered steps and one finger on it. Each rung only reads the line under the finger; when its line is done it moves the finger to the next number. Because the finger can only be in one place, only one step can ever be "doing".

> **Watch for:** A rung further down that tests for the new step number sees it in this same scan. Rung order decides whether the next step starts now or next scan.

RUNG 1

```
XIC(Line_Run)ONS(Mix_Line_ONS)OTE(Mix_Auto_Start);
```

HOW THIS RUNG WORKS

In words. When `Line_Run` (Seal-in run bit (LogixPro B3:0/0)) is on and it is the first scan they are true (the one-shot, remembered in `Mix_Line_ONS`), the rung turns `Mix_Auto_Start` (One scan when the line starts: starts the kitchen too) on (and off again whenever the rung is false).

Left to right:

1. `XIC(Line_Run)` *Examine If Closed* asks: true when `Line_Run` (Seal-in run bit (LogixPro B3:0/0)) is on.
2. `ONS(Mix_Line_ONS)` *One Shot* asks: true when it is the first scan they are true (the one-shot, remembered in `Mix_Line_ONS`). It passes power for one scan only, the scan its input goes from false to true; its storage bit remembers last scan.
3. `OTE(Mix_Auto_Start)` *Output Energize* does: turns `Mix_Auto_Start` (One scan when the line starts: starts the kitchen too) on (and off again whenever the rung is false). Every scan it writes the rung state: true on, false off.

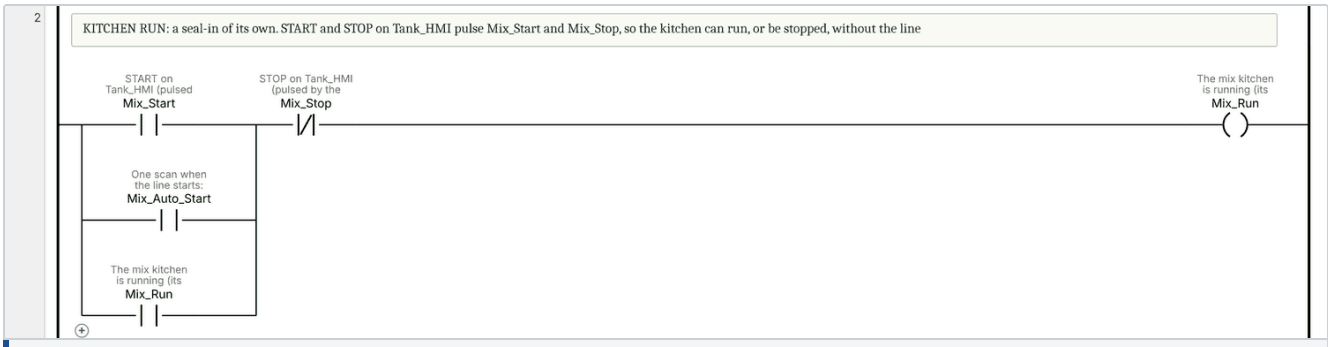
Why it is built this way:

One-shot. Everything after the ONS happens once per event, not on every scan the condition holds; `Mix_Line_ONS` is only its memory of the last scan.

> **Think of it like this:** a doorbell that rings once however long you lean on the button. The ONS looks at what the button was doing last scan: only the change from up to down rings it.

> **Watch for:** Each ONS needs its own storage bit. Two ONS sharing one bit make one of them miss its edges.

RUNG 2




```
[XIC(Mix_Start),XIC(Mix_Auto_Start),XIC(Mix_Run)]XIO(Mix_Stop)OTE(Mix_Run);
```

PREDICT IT · 2 `Mix_Start` is on and `Mix_Stop` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When either (`Mix_Start` (START on Tank_HMI (pulsed by the touch screen)) is on) *or* (`Mix_Auto_Start` (One scan when the line starts: starts the kitchen too) is on) *or* (`Mix_Run` (The mix kitchen is running (its own seal-in)) is on) and `Mix_Stop` (STOP on Tank_HMI (pulsed by the touch screen)) is off, the rung turns `Mix_Run` on (and off again whenever the rung is false).

Left to right:

1. A branch of 3 parallel paths: power gets through if any one of them is true.
   - **Path 1**
     1. `XIC(Mix_Start)` *Examine If Closed* asks: true when `Mix_Start` (START on Tank_HMI (pulsed by the touch screen)) is on.
   - **Path 2**
     1. `XIC(Mix_Auto_Start)` *Examine If Closed* asks: true when `Mix_Auto_Start` (One scan when the line starts: starts the kitchen too) is on.
   - **Path 3**
     1. `XIC(Mix_Run)` *Examine If Closed* asks: true when `Mix_Run` (The mix kitchen is running (its own seal-in)) is on.
2. `XIO(Mix_Stop)` *Examine If Open* asks: true when `Mix_Stop` (STOP on Tank_HMI (pulsed by the touch screen)) is off.
3. `OTE(Mix_Run)` *Output Energize* does: turns `Mix_Run` on (and off again whenever the rung is false).

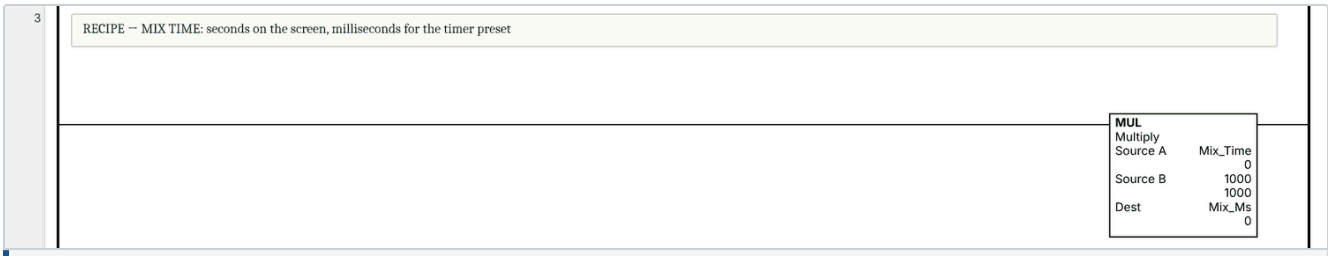
Why it is built this way:

Seal-in. The start condition turns `Mix_Run` on, and `Mix_Run` 's own contact in the parallel path keeps the rung true after the start goes away. Anything in series outside the branch (a stop, a fault) breaks it, and then it stays off until the start is made again.

> **Think of it like this:** a doorbell button and a hand. The start button is the doorbell, pressed for a moment. The seal-in contact is a hand that grabs the circuit as soon as the output comes on and keeps holding after you let go. The stop button, in series, is the only thing that can knock that hand away.

> **Watch for:** The stop has to be in series outside the branch. Put it inside the start path only and the seal-in path bypasses it: the stop no longer stops.

RUNG 3




```
MUL(Mix_Time,1000,Mix_Ms);
```

HOW THIS RUNG WORKS

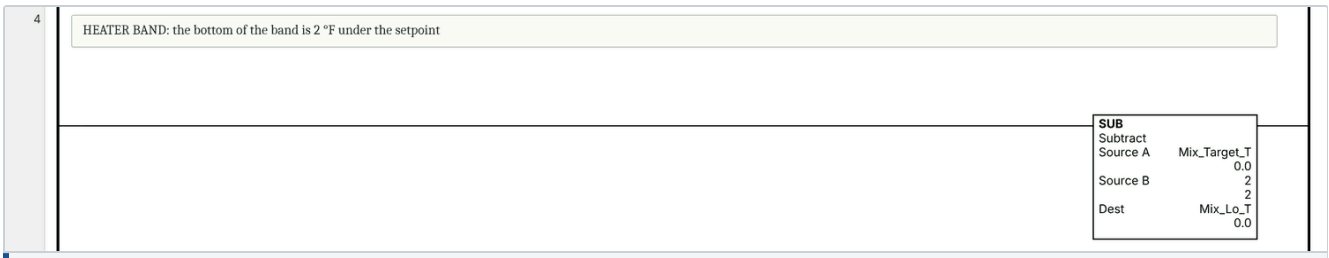
In words. Every scan, with nothing to test, the rung makes `Mix_Ms` (Mixing time in ms (Mix_Time × 1000)) = `Mix_Time` (Mixing time, s: MIX TIME on the RECIPE page) × **1000**.

Left to right:

1. `MUL(Mix_Time,1000,Mix_Ms)` *Multiply* does: makes `Mix_Ms` (Mixing time in ms (Mix_Time × 1000)) = `Mix_Time` (Mixing time, s: MIX TIME on the RECIPE page) × **1000**.

> **Think of it like this:** a rung is a pipe with valves in it, read left to right. Every contact is a valve: XIC is open while its bit is 1, XIO while its bit is 0. Water reaches the output at the right only if every valve in a row is open. The output does not remember anything: on every scan it is simply wet or dry.

RUNG 4




```
SUB(Mix_Target_T,2.0,Mix_Lo_T);
```

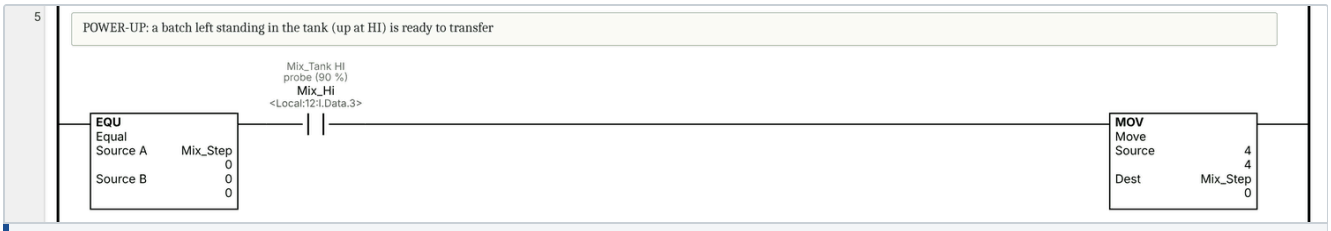
HOW THIS RUNG WORKS

In words. Every scan, with nothing to test, the rung makes `Mix_Lo_T` (Bottom of the heater band: 2 °F under Mix_Target_T) = `Mix_Target_T` (Batch temperature, °F: the setpoint on Tank_HMI (TEMP page)) − **2.0**.

Left to right:

1. `SUB(Mix_Target_T,2.0,Mix_Lo_T)` *Subtract* does: makes `Mix_Lo_T` (Bottom of the heater band: 2 °F under Mix_Target_T) = `Mix_Target_T` (Batch temperature, °F: the setpoint on Tank_HMI (TEMP page)) − **2.0**.

RUNG 5




```
EQU(Mix_Step,0)XIC(Mix_Hi)MOV(4,Mix_Step);
```

PREDICT IT · 3 `Mix_Step` is 0 and `Mix_Hi` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Mix_Step` (Kitchen sequence: 0 empty, 1 dose water, 2 syrup to HI, 3 heat and mix, 4 ready / transfer) equals **0** and `Mix_Hi` (Mix_Tank HI probe (90 %)) is on, the rung puts **4** into `Mix_Step` .

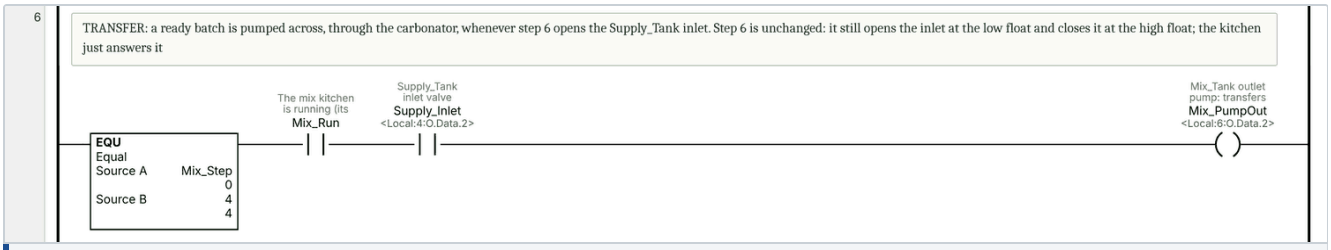
Left to right:

1. `EQU(Mix_Step,0)` *Equal* asks: true when `Mix_Step` (Kitchen sequence: 0 empty, 1 dose water, 2 syrup to HI, 3 heat and mix, 4 ready / transfer) equals **0**.
2. `XIC(Mix_Hi)` *Examine If Closed* asks: true when `Mix_Hi` (Mix_Tank HI probe (90 %)) is on.
3. `MOV(4,Mix_Step)` *Move* does: puts **4** into `Mix_Step` .

Why it is built this way:

A step of the sequence: from Mix_Step = 0 to 4.

RUNG 6




```
EQU(Mix_Step,4)XIC(Mix_Run)XIC(Supply_Inlet)OTE(Mix_PumpOut);
```

PREDICT IT · 4 `Mix_Step` is 5, `Mix_Run` is on and `Supply_Inlet` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

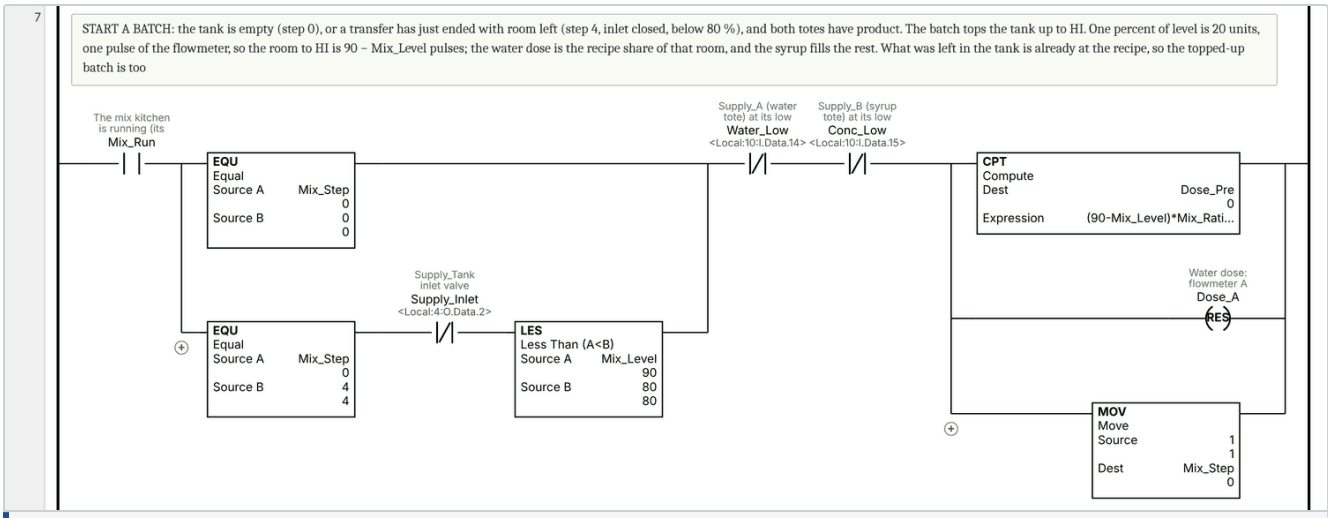
HOW THIS RUNG WORKS

In words. When `Mix_Step` (Kitchen sequence: 0 empty, 1 dose water, 2 syrup to HI, 3 heat and mix, 4 ready / transfer) equals **4**, `Mix_Run` (The mix kitchen is running (its own seal-in)) is on and `Supply_Inlet` (Supply_Tank inlet valve) is on, the rung turns `Mix_PumpOut` (Mix_Tank outlet pump: transfers to Supply_Tank) on (and off again whenever the rung is false).

Left to right:

1. `EQU(Mix_Step,4)` *Equal* asks: true when `Mix_Step` (Kitchen sequence: 0 empty, 1 dose water, 2 syrup to HI, 3 heat and mix, 4 ready / transfer) equals **4**.
2. `XIC(Mix_Run)` *Examine If Closed* asks: true when `Mix_Run` (The mix kitchen is running (its own seal-in)) is on.
3. `XIC(Supply_Inlet)` *Examine If Closed* asks: true when `Supply_Inlet` (Supply_Tank inlet valve) is on.
4. `OTE(Mix_PumpOut)` *Output Energize* does: turns `Mix_PumpOut` (Mix_Tank outlet pump: transfers to Supply_Tank) on (and off again whenever the rung is false).

RUNG 7




```
XIC(Mix_Run)
[EQU(Mix_Step,0),EQU(Mix_Step,4)XIO(Supply_Inlet)LES(Mix_Level,80)]XIO(Water_Low)XIO(Conc_Low)[CPT(Dose_Pre,(90-
Mix_Level)*Mix_Ratio/100),RES(Dose_A),MOV(1,Mix_Step)];
```

PREDICT IT · 5 `Mix_Run` is on, `Mix_Step` is 0, `Water_Low` is off and `Conc_Low` is off. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Mix_Run` (The mix kitchen is running (its own seal-in)) is on, either (`Mix_Step` (Kitchen sequence: 0 empty, 1 dose water, 2 syrup to HI, 3 heat and mix, 4 ready / transfer) equals **0**) *or* (`Mix_Step` equals **4**, `Supply_Inlet` (Supply_Tank inlet valve) is off and `Mix_Level` (Mix_Tank level, 0-100 %) is less than **80**), `Water_Low` (Supply_A (water tote) at its low switch) is off and `Conc_Low` (Supply_B (syrup tote) at its low switch) is off, the rung computes `Dose_Pre` (Water dose for this batch, in flowmeter pulses) = `(90-Mix_Level)*Mix_Ratio/100` , resets `Dose_A` (Water dose: flowmeter A pulses) and puts **1** into `Mix_Step` .

Left to right:

1. `XIC(Mix_Run)` *Examine If Closed* asks: true when `Mix_Run` (The mix kitchen is running (its own seal-in)) is on.
2. A branch of 2 parallel paths: power gets through if any one of them is true.

   Path 1

   1. `EQU(Mix_Step,0)` *Equal* asks: true when `Mix_Step` (Kitchen sequence: 0 empty, 1 dose water, 2 syrup to HI, 3 heat and mix, 4 ready / transfer) equals **0**.

   Path 2

   1. `EQU(Mix_Step,4)` *Equal* asks: true when `Mix_Step` equals **4**.
   2. `XIO(Supply_Inlet)` *Examine If Open* asks: true when `Supply_Inlet` (Supply_Tank inlet valve) is off.
   3. `LES(Mix_Level,80)` *Less Than* asks: true when `Mix_Level` (Mix_Tank level, 0-100 %) is less than **80**.
3. `XIO(Water_Low)` *Examine If Open* asks: true when `Water_Low` (Supply_A (water tote) at its low switch) is off.
4. `XIO(Conc_Low)` *Examine If Open* asks: true when `Conc_Low` (Supply_B (syrup tote) at its low switch) is off.
5. A branch of 3 parallel paths: each path is tested on its own and does its own work.

Path 1

1. `CPT(Dose_Pre,(90-Mix_Level)*Mix_Ratio/100)` *Compute* does: computes `Dose_Pre` (Water dose for this batch, in flowmeter pulses) = `(90-Mix_Level)*Mix_Ratio/100` .

Path 2

1. `RES(Dose_A)` *Reset* does: resets `Dose_A` (Water dose: flowmeter A pulses). It zeroes a timer or counter (accumulator and status bits), or a control (its position and status bits).

Path 3

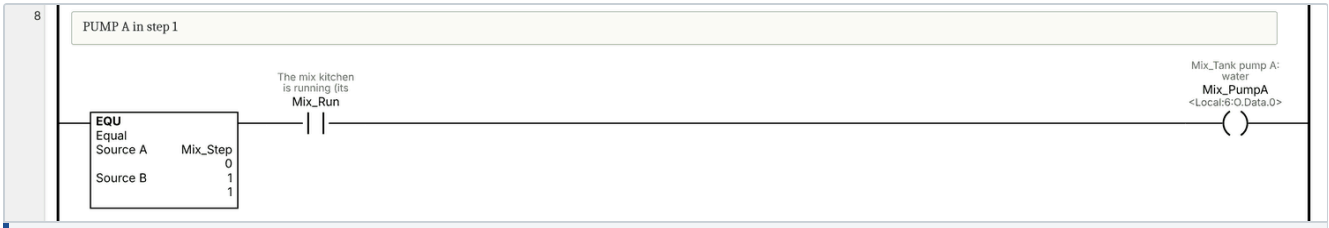
1. `MOV(1,Mix_Step)` *Move* does: puts **1** into `Mix_Step` .

Why it is built this way:

A step of the sequence: from Mix_Step = 0 to 1.

> **Think of it like this:** parallel paths are two roads to the same town. If either road is open you get there; the rung only cares that power reaches the far end, not which way it went.

RUNG 8




```
EQU(Mix_Step,1)XIC(Mix_Run)OTE(Mix_PumpA);
```

PREDICT IT · 6 `Mix_Step` is 2 and `Mix_Run` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

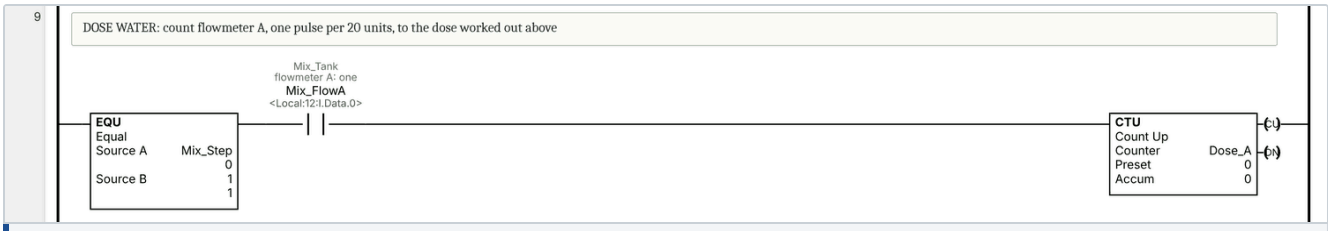
HOW THIS RUNG WORKS

In words. When `Mix_Step` (Kitchen sequence: 0 empty, 1 dose water, 2 syrup to HI, 3 heat and mix, 4 ready / transfer) equals **1** and `Mix_Run` (The mix kitchen is running (its own seal-in)) is on, the rung turns `Mix_PumpA` (Water) on (and off again whenever the rung is false).

Left to right:

1. `EQU(Mix_Step,1)` *Equal* asks: true when `Mix_Step` (Kitchen sequence: 0 empty, 1 dose water, 2 syrup to HI, 3 heat and mix, 4 ready / transfer) equals **1**.
2. `XIC(Mix_Run)` *Examine If Closed* asks: true when `Mix_Run` (The mix kitchen is running (its own seal-in)) is on.
3. `OTE(Mix_PumpA)` *Output Energize* does: turns `Mix_PumpA` (Water) on (and off again whenever the rung is false).

RUNG 9




```
EQU(Mix_Step,1)XIC(Mix_FlowA)CTU(Dose_A,Dose_Pre,0);
```

PREDICT IT · 7 `Mix_Step` is 1 and `Mix_FlowA` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Mix_Step` (Kitchen sequence: 0 empty, 1 dose water, 2 syrup to HI, 3 heat and mix, 4 ready / transfer) equals **1** and `Mix_FlowA` (One pulse per 20 units) is on, the rung counts `Dose_A` (Water dose: flowmeter A pulses) up by one (preset Dose_Pre).

Left to right:

1. `EQU(Mix_Step,1)` *Equal* asks: true when `Mix_Step` (Kitchen sequence: 0 empty, 1 dose water, 2 syrup to HI, 3 heat and mix, 4 ready / transfer) equals **1**.
2. `XIC(Mix_FlowA)` *Examine If Closed* asks: true when `Mix_FlowA` (One pulse per 20 units) is on.
3. `CTU(Dose_A,Dose_Pre,0)` *Count Up* does: counts `Dose_A` (Water dose: flowmeter A pulses) up by one (preset Dose_Pre). It counts false-to-true transitions of the rung, not scans; .DN is on once the count reaches the preset; it keeps counting past it.

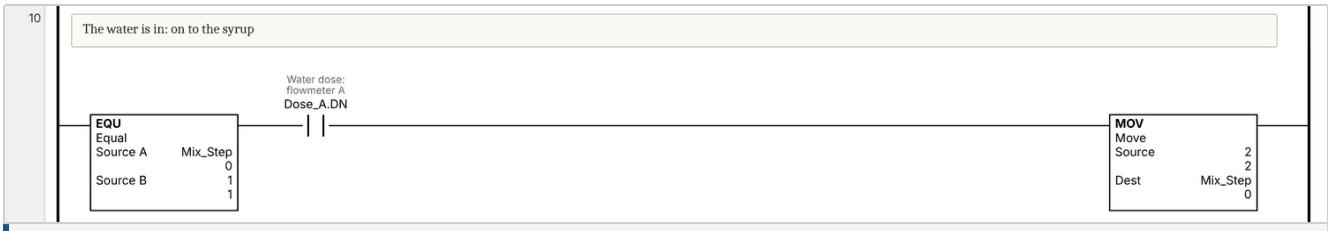
Why it is built this way:

A counter. It counts transitions, not time: each false-to-true of the rung adds one. A RES elsewhere clears it.

> **Think of it like this:** the clicker a doorman holds: one click for every person who walks through the door, however long they stand in it. The number stays on the clicker until somebody presses reset.

> **Watch for:** A counter keeps its count through a stop and a restart. Reset it on purpose where the count should start again.

RUNG 10




```
EQU(Mix_Step,1)XIC(Dose_A.DN)MOV(2,Mix_Step);
```

PREDICT IT · 8 `Mix_Step` is 1 and `Dose_A.DN` (the done bit of `Dose_A`: Water dose: flowmeter A pulses) is off. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Mix_Step` (Kitchen sequence: 0 empty, 1 dose water, 2 syrup to HI, 3 heat and mix, 4 ready / transfer) equals **1** and `Dose_A.DN` (the done bit of `Dose_A`: Water dose: flowmeter A pulses) is on, the rung puts **2** into `Mix_Step`.

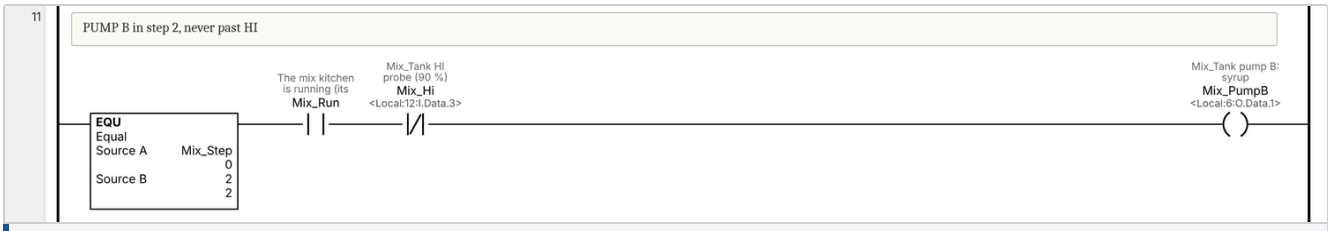
Left to right:

1. `EQU(Mix_Step,1)` *Equal* asks: true when `Mix_Step` (Kitchen sequence: 0 empty, 1 dose water, 2 syrup to HI, 3 heat and mix, 4 ready / transfer) equals **1**.
2. `XIC(Dose_A.DN)` *Examine If Closed* asks: true when `Dose_A.DN` (the done bit of `Dose_A`: Water dose: flowmeter A pulses) is on.
3. `MOV(2,Mix_Step)` *Move* does: puts **2** into `Mix_Step`.

Why it is built this way:

A step of the sequence: from Mix_Step = 1 to 2.

RUNG 11




```
EQU(Mix_Step,2)XIC(Mix_Run)XIO(Mix_Hi)OTE(Mix_PumpB);
```

PREDICT IT · 9 `Mix_Step` is 2, `Mix_Run` is on and `Mix_Hi` is off. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

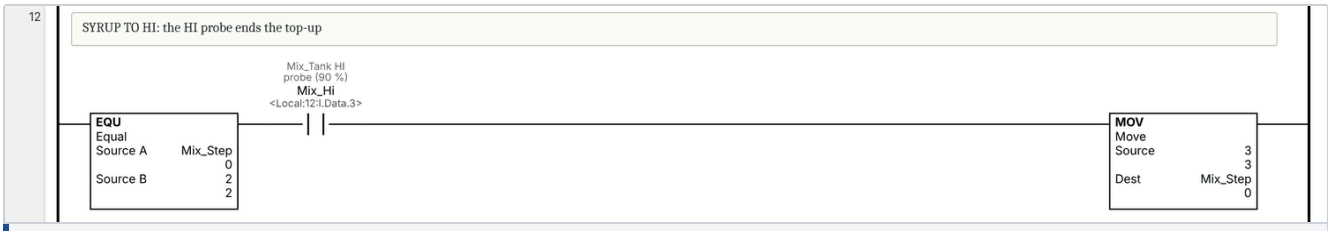
HOW THIS RUNG WORKS

In words. When `Mix_Step` (Kitchen sequence: 0 empty, 1 dose water, 2 syrup to HI, 3 heat and mix, 4 ready / transfer) equals **2**, `Mix_Run` (The mix kitchen is running (its own seal-in)) is on and `Mix_Hi` (Mix_Tank HI probe (90 %)) is off, the rung turns `Mix_PumpB` (Syrup) on (and off again whenever the rung is false).

Left to right:

1. `EQU(Mix_Step,2)` *Equal* asks: true when `Mix_Step` (Kitchen sequence: 0 empty, 1 dose water, 2 syrup to HI, 3 heat and mix, 4 ready / transfer) equals **2**.
2. `XIC(Mix_Run)` *Examine If Closed* asks: true when `Mix_Run` (The mix kitchen is running (its own seal-in)) is on.
3. `XIO(Mix_Hi)` *Examine If Open* asks: true when `Mix_Hi` (Mix_Tank HI probe (90 %)) is off.
4. `OTE(Mix_PumpB)` *Output Energize* does: turns `Mix_PumpB` (Syrup) on (and off again whenever the rung is false).

RUNG 12




```
EQU(Mix_Step,2)XIC(Mix_Hi)MOV(3,Mix_Step);
```

PREDICT IT · 10 `Mix_Step` is 3 and `Mix_Hi` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Mix_Step` (Kitchen sequence: 0 empty, 1 dose water, 2 syrup to HI, 3 heat and mix, 4 ready / transfer) equals **2** and `Mix_Hi` (Mix_Tank HI probe (90 %)) is on, the rung puts **3** into `Mix_Step` .

Left to right:

1. `EQU(Mix_Step,2)` *Equal* asks: true when `Mix_Step` (Kitchen sequence: 0 empty, 1 dose water, 2 syrup to HI, 3 heat and mix, 4 ready / transfer) equals **2**.
2. `XIC(Mix_Hi)` *Examine If Closed* asks: true when `Mix_Hi` (Mix_Tank HI probe (90 %)) is on.
3. `MOV(3,Mix_Step)` *Move* does: puts **3** into `Mix_Step` .

Why it is built this way:

A step of the sequence: from Mix_Step = 2 to 3.

RUNG 13

```
[EQU(Mix_Step,3),EQU(Mix_Step,4)]XIC(Mix_Lo)XIC(Mix_Run)OTE(Mix_Mixer);
```

PREDICT IT · 11 `Mix_Step` is 3, `Mix_Lo` is on and `Mix_Run` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When either (`Mix_Step` (Kitchen sequence: 0 empty, 1 dose water, 2 syrup to HI, 3 heat and mix, 4 ready / transfer) equals **3**) *or* (`Mix_Step` equals **4**), `Mix_Lo` (Mix_Tank LO probe (15 %)) is on and `Mix_Run` (The mix kitchen is running (its own seal-in)) is on, the rung turns `Mix_Mixer` (Mix_Tank mixer) on (and off again whenever the rung is false).

Left to right:

1. A branch of 2 parallel paths: power gets through if any one of them is true.

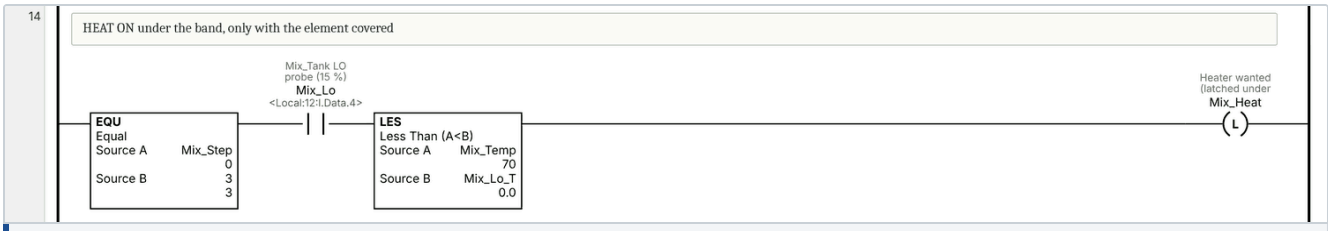
   Path 1

   1. `EQU(Mix_Step,3)` *Equal* asks: true when `Mix_Step` (Kitchen sequence: 0 empty, 1 dose water, 2 syrup to HI, 3 heat and mix, 4 ready / transfer) equals **3**.

   Path 2

   1. `EQU(Mix_Step,4)` *Equal* asks: true when `Mix_Step` equals **4**.
2. `XIC(Mix_Lo)` *Examine If Closed* asks: true when `Mix_Lo` (Mix_Tank LO probe (15 %)) is on.
3. `XIC(Mix_Run)` *Examine If Closed* asks: true when `Mix_Run` (The mix kitchen is running (its own seal-in)) is on.
4. `OTE(Mix_Mixer)` *Output Energize* does: turns `Mix_Mixer` (Mix_Tank mixer) on (and off again whenever the rung is false).

RUNG 14




```
EQU(Mix_Step,3)XIC(Mix_Lo)LES(Mix_Temp,Mix_Lo_T)OTL(Mix_Heat);
```

PREDICT IT · 12 `Mix_Step` is 3, `Mix_Lo` is off, `Mix_Temp` is 499 and `Mix_Lo_T` is 500. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Mix_Step` (Kitchen sequence: 0 empty, 1 dose water, 2 syrup to HI, 3 heat and mix, 4 ready / transfer) equals **3**, `Mix_Lo` (Mix_Tank LO probe (15 %)) is on and `Mix_Temp` (Mix_Tank temperature, °F) is less than `Mix_Lo_T` (Bottom of the heater band: 2 °F under Mix_Target_T), the rung latches `Mix_Heat` (Heater wanted (latched under the band, unlatched at the setpoint)) on.

Left to right:

1. `EQU(Mix_Step,3)` *Equal* asks: true when `Mix_Step` (Kitchen sequence: 0 empty, 1 dose water, 2 syrup to HI, 3 heat and mix, 4 ready / transfer) equals **3**.
2. `XIC(Mix_Lo)` *Examine If Closed* asks: true when `Mix_Lo` (Mix_Tank LO probe (15 %)) is on.
3. `LES(Mix_Temp,Mix_Lo_T)` *Less Than* asks: true when `Mix_Temp` (Mix_Tank temperature, °F) is less than `Mix_Lo_T` (Bottom of the heater band: 2 °F under Mix_Target_T).
4. `OTL(Mix_Heat)` *Output Latch* does: latches `Mix_Heat` (Heater wanted (latched under the band, unlatched at the setpoint)) on. It only ever sets the bit; the bit stays on after the rung goes false.

Why it is built this way:

A latch. `Mix_Heat` stays on after this rung goes false; an `OTU` elsewhere in the program clears it.

> **Think of it like this:** a light switch instead of a doorbell. OTL flips it on and it stays on when you take your hand away; OTU flips it off. Whatever flipped it on does not have to stay there.

> **Watch for:** A latched bit survives the condition that set it: every way in needs a way out, or it stays on for good.

RUNG 15

```
[GEQ(Mix_Temp,Mix_Target_T),XIO(Mix_Lo),NEQ(Mix_Step,3)]OTU(Mix_Heat);
```

PREDICT IT · 13 `Mix_Temp` is 500 and `Mix_Target_T` is 500. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When either (`Mix_Temp` (Mix_Tank temperature, °F) is at least `Mix_Target_T` (Batch temperature, °F: the setpoint on Tank_HMI (TEMP page))) *or* (`Mix_Lo` (Mix_Tank LO probe (15 %)) is off) *or* (`Mix_Step` (Kitchen sequence: 0 empty, 1 dose water, 2 syrup to HI, 3 heat and mix, 4 ready / transfer) is not **3**), the rung unlatches `Mix_Heat` (Heater wanted (latched under the band, unlatched at the setpoint)) (turns it off).

Left to right:

1. A branch of 3 parallel paths: power gets through if any one of them is true.

   Path 1

   1. `GEQ(Mix_Temp,Mix_Target_T)` *Greater Than or Equal* asks: true when `Mix_Temp` (Mix_Tank temperature, °F) is at least `Mix_Target_T` (Batch temperature, °F: the setpoint on Tank_HMI (TEMP page)).

   Path 2

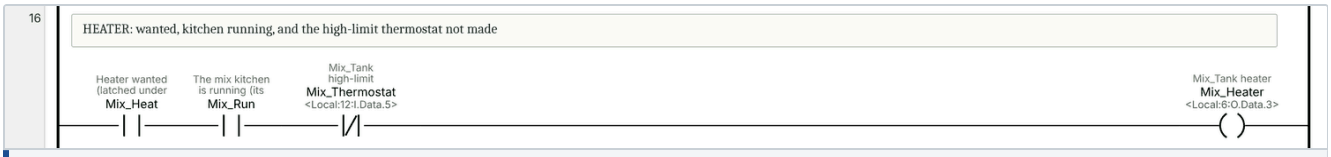
   1. `XIO(Mix_Lo)` *Examine If Open* asks: true when `Mix_Lo` (Mix_Tank LO probe (15 %)) is off.

   Path 3

   1. `NEQ(Mix_Step,3)` *Not Equal* asks: true when `Mix_Step` (Kitchen sequence: 0 empty, 1 dose water, 2 syrup to HI, 3 heat and mix, 4 ready / transfer) is not **3**.

2. `OTU(Mix_Heat)` *Output Unlatch* does: unlatches `Mix_Heat` (Heater wanted (latched under the band, unlatched at the setpoint)) (turns it off). It only ever clears the bit.

RUNG 16




```
XIC(Mix_Heat)XIC(Mix_Run)XIO(Mix_Thermostat)OTE(Mix_Heater);
```

PREDICT IT · 14 `Mix_Heat` is on, `Mix_Run` is on and `Mix_Thermostat` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

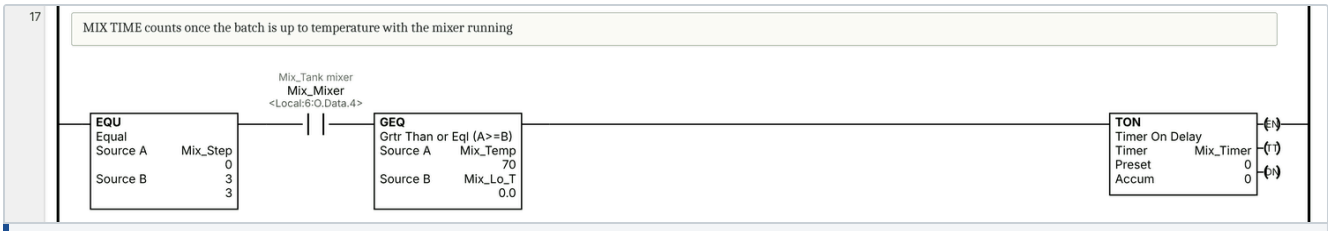
HOW THIS RUNG WORKS

In words. When `Mix_Heat` (Heater wanted (latched under the band, unlatched at the setpoint)) is on, `Mix_Run` (The mix kitchen is running (its own seal-in)) is on and `Mix_Thermostat` (Mix_Tank high-limit thermostat (180 °F)) is off, the rung turns `Mix_Heater` (Mix_Tank heater) on (and off again whenever the rung is false).

Left to right:

1. `XIC(Mix_Heat)` *Examine If Closed* asks: true when `Mix_Heat` (Heater wanted (latched under the band, unlatched at the setpoint)) is on.
2. `XIC(Mix_Run)` *Examine If Closed* asks: true when `Mix_Run` (The mix kitchen is running (its own seal-in)) is on.
3. `XIO(Mix_Thermostat)` *Examine If Open* asks: true when `Mix_Thermostat` (Mix_Tank high-limit thermostat (180 °F)) is off.
4. `OTE(Mix_Heater)` *Output Energize* does: turns `Mix_Heater` (Mix_Tank heater) on (and off again whenever the rung is false).

RUNG 17




```
EQU(Mix_Step,3)XIC(Mix_Mixer)GEQ(Mix_Temp,Mix_Lo_T)TON(Mix_Timer,Mix_Ms,0);
```

PREDICT IT · 15 `Mix_Step` is 3, `Mix_Mixer` is on, `Mix_Temp` is 500 and `Mix_Lo_T` is 500. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Mix_Step` (Kitchen sequence: 0 empty, 1 dose water, 2 syrup to HI, 3 heat and mix, 4 ready / transfer) equals **3**, `Mix_Mixer` (Mix_Tank mixer) is on and `Mix_Temp` (Mix_Tank temperature, °F) is at least `Mix_Lo_T` (Bottom of the heater band: 2 °F under Mix_Target_T), the rung runs the timer `Mix_Timer` (Mixing time at temperature) toward Mix_Ms.

Left to right:

1. `EQU(Mix_Step,3)` *Equal* asks: true when `Mix_Step` (Kitchen sequence: 0 empty, 1 dose water, 2 syrup to HI, 3 heat and mix, 4 ready / transfer) equals **3**.
2. `XIC(Mix_Mixer)` *Examine If Closed* asks: true when `Mix_Mixer` (Mix_Tank mixer) is on.
3. `GEQ(Mix_Temp,Mix_Lo_T)` *Greater Than or Equal* asks: true when `Mix_Temp` (Mix_Tank temperature, °F) is at least `Mix_Lo_T` (Bottom of the heater band: 2 °F under Mix_Target_T).
4. `TON(Mix_Timer,Mix_Ms,0)` *Timer On Delay* does: runs the timer `Mix_Timer` (Mixing time at temperature) toward Mix_Ms. The accumulator counts while the rung is true; .DN comes on at the preset; the moment the rung goes false the timer resets.

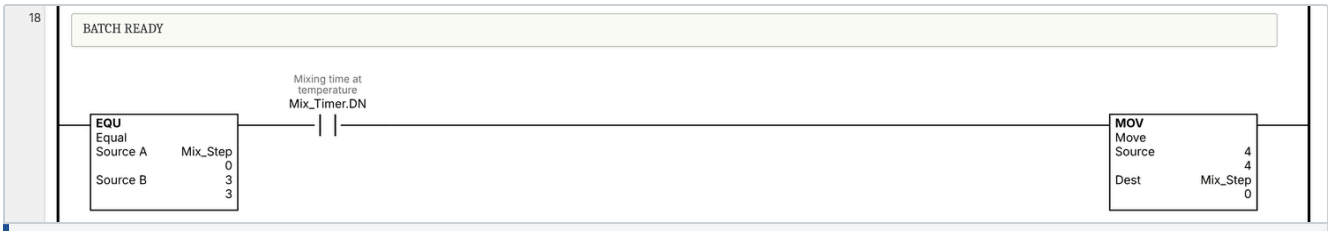
Why it is built this way:

A timer. `Mix_Timer` only times while this rung is true; other rungs use its `.DN` (done) or `.TT` (timing) bits. Being a TON, it starts again from zero every time the rung goes false.

> **Think of it like this:** a kitchen timer that only runs while you hold your finger on it. Take your finger off and it snaps back to zero. Hold it long enough and the done light comes on, and stays on as long as your finger is still there.

> **Watch for:** If anything in series flickers false for one scan, a TON starts over from zero. Use RTO when the time must survive an interruption.

RUNG 18




```
EQU(Mix_Step,3)XIC(Mix_Timer.DN)MOV(4,Mix_Step);
```

PREDICT IT · 16 `Mix_Step` is 3 and `Mix_Timer.DN` (the done bit of `Mix_Timer` : Mixing time at temperature) is off. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Mix_Step` (Kitchen sequence: 0 empty, 1 dose water, 2 syrup to HI, 3 heat and mix, 4 ready / transfer) equals **3** and `Mix_Timer.DN` (the done bit of `Mix_Timer` : Mixing time at temperature) is on, the rung puts **4** into `Mix_Step` .

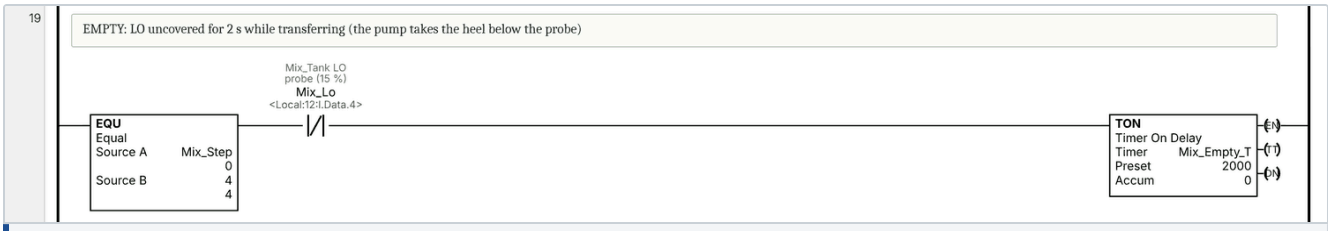
Left to right:

1. `EQU(Mix_Step,3)` *Equal* asks: true when `Mix_Step` (Kitchen sequence: 0 empty, 1 dose water, 2 syrup to HI, 3 heat and mix, 4 ready / transfer) equals **3**.
2. `XIC(Mix_Timer.DN)` *Examine If Closed* asks: true when `Mix_Timer.DN` (the done bit of `Mix_Timer` : Mixing time at temperature) is on.
3. `MOV(4,Mix_Step)` *Move* does: puts **4** into `Mix_Step` .

Why it is built this way:

A step of the sequence: from Mix_Step = 3 to 4.

RUNG 19




```
EQU(Mix_Step,4)XIO(Mix_Lo)TON(Mix_Empty_T,2000,0);
```

PREDICT IT · 17 `Mix_Step` is 4 and `Mix_Lo` is off. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Mix_Step` (Kitchen sequence: 0 empty, 1 dose water, 2 syrup to HI, 3 heat and mix, 4 ready / transfer) equals **4** and `Mix_Lo` (Mix_Tank LO probe (15 %)) is off, the rung runs the timer `Mix_Empty_T` (LO uncovered this long: the tank is empty) toward 2000 ms (2 s).

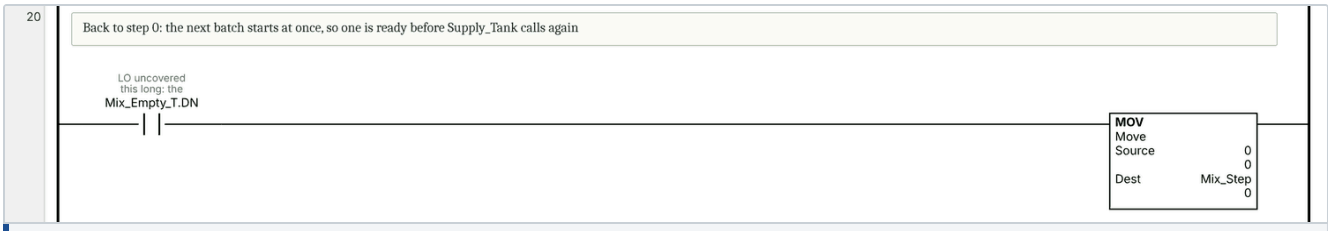
Left to right:

1. `EQU(Mix_Step,4)` *Equal* asks: true when `Mix_Step` (Kitchen sequence: 0 empty, 1 dose water, 2 syrup to HI, 3 heat and mix, 4 ready / transfer) equals **4**.
2. `XIO(Mix_Lo)` *Examine If Open* asks: true when `Mix_Lo` (Mix_Tank LO probe (15 %)) is off.
3. `TON(Mix_Empty_T,2000,0)` *Timer On Delay* does: runs the timer `Mix_Empty_T` (LO uncovered this long: the tank is empty) toward 2000 ms (2 s).

Why it is built this way:

A timer: it times only while this rung is true.

RUNG 20




```
XIC(Mix_Empty_T.DN)MOV(0,Mix_Step);
```

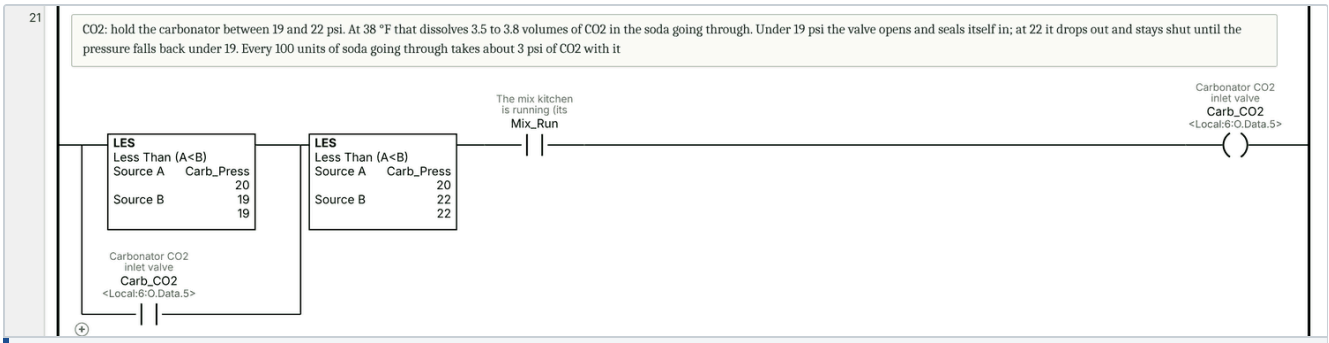
HOW THIS RUNG WORKS

In words. When `Mix_Empty_T.DN` (the done bit of `Mix_Empty_T` : LO uncovered this long: the tank is empty) is on, the rung puts **0** into `Mix_Step` (Kitchen sequence: 0 empty, 1 dose water, 2 syrup to HI, 3 heat and mix, 4 ready / transfer).

Left to right:

1. `XIC(Mix_Empty_T.DN)` *Examine If Closed* asks: true when `Mix_Empty_T.DN` (the done bit of `Mix_Empty_T` : LO uncovered this long: the tank is empty) is on.
2. `MOV(0,Mix_Step)` *Move* does: puts **0** into `Mix_Step` (Kitchen sequence: 0 empty, 1 dose water, 2 syrup to HI, 3 heat and mix, 4 ready / transfer).

RUNG 21




```
[LES(Carb_Press,19),XIC(Carb_CO2)]LES(Carb_Press,22)XIC(Mix_Run)OTE(Carb_CO2);
```

HOW THIS RUNG WORKS

In words. When either (`Carb_Press` (Carbonator tank pressure, psi) is less than **19**) *or* (`Carb_CO2` (Carbonator CO2 inlet valve) is on), `Carb_Press` is less than **22** and `Mix_Run` (The mix kitchen is running (its own seal-in)) is on, the rung turns `Carb_CO2` on (and off again whenever the rung is false).

Left to right:

1. A branch of 2 parallel paths: power gets through if any one of them is true.

   Path 1

   1. `LES(Carb_Press,19)` *Less Than* asks: true when `Carb_Press` (Carbonator tank pressure, psi) is less than **19**.

   Path 2

   1. `XIC(Carb_CO2)` *Examine If Closed* asks: true when `Carb_CO2` (Carbonator CO2 inlet valve) is on.
2. `LES(Carb_Press,22)` *Less Than* asks: true when `Carb_Press` is less than **22**.
3. `XIC(Mix_Run)` *Examine If Closed* asks: true when `Mix_Run` (The mix kitchen is running (its own seal-in)) is on.
4. `OTE(Carb_CO2)` *Output Energize* does: turns `Carb_CO2` on (and off again whenever the rung is false).

Why it is built this way:

Seal-in, as before: the output's own contact holds the rung once the start has been made.

Rung 0 gives the recipe tags their defaults the first time: a REAL that nobody has set is 0, so 0 means "not set yet": the soda recipe, 100 °F, 83 % water, 8 s. Picking a product on Tank_HMI's RECIPE page loads its recipe into the same three tags. **Rungs 1–2** are the kitchen's own run circuit. It is separate from `Line_Run` because a kitchen often runs when the line does not (the first batch is made before the shift starts), and the operator at Tank_HMI must be able to stop it without stopping the line. The console's Start starts both: rung 1 turns the rising edge of Line_Run into a one-scan `Mix_Auto_Start`, which rung 2 treats like any other start button.

Rungs 3–4 turn the screen's units into the program's: seconds into milliseconds for the mixing timer, the setpoint into the bottom of the heater band. **Rung 5** is power-up: a tank standing at HI holds a finished batch, so step 0 goes straight to 4. **Rung 6** is the transfer, right behind it, because that batch goes nowhere until it runs: a ready batch, the kitchen running, and step 6's inlet open. That one rung is the whole interface between the kitchen and the line.

Rung 7 starts a batch, either from empty (step 0) or as a top-up (step 4, the inlet closed, the tank below 80 %), and only while both totes are above their low switches. `CPT` works out the water dose from the room and the recipe in one expression; `RES` clears the counter; `MOV` goes to step 1. From here each output comes straight after the step that runs it, so every rung works the moment it is written: **rung 8** runs pump A in step 1; **rungs 9–10** dose the water, a CTU on flowmeter A whose preset is the tag `Dose_Pre`, done goes to step 2; **rung 11** runs pump B in step 2, never past HI, and **rung 12** ends the syrup at HI. **Rung 13** runs the mixer in steps 3 and 4, only with the impeller covered; **rungs 14–15** are the heater band of chapter 14's worked example and **rung 16** the heater itself, never past the high-limit thermostat; **rungs 17–18** time the mix once the batch is up to temperature and make it ready. Each output still appears in one place only.

Rungs 19–20: if a transfer empties the tank (LO uncovered for two seconds in step 4, which also takes the heel below the probe), the sequence returns to 0 and starts again.

Rung 21 is the carbonator. Everything the transfer pumps across goes through it, chilled to 38 °F, and takes up CO2 according to the tank pressure: at that temperature about 1.5 volumes per atmosphere, so 19 to 22 psi gives 3.5 to 3.8 volumes, the soda recipe. Every 100 units going through takes about 3 psi with it. The rung is an on-off band with a seal-in: under 19 psi the CO2 valve opens, and `XIC(Carb_CO2)` keeps it open until 22, where `LES(Carb_Press,22)` goes false and the valve shuts until the pressure falls under 19 again. It is the heater band of rungs 14–15 done in one rung.

Run it

Run mode, Start. Put `Mix_Step`, `Dose_A.ACC`, `Mix_Level`, `Mix_Temp`, `Carb_Press`, `Carb_Vol` and `Supply_Level` in a Watch. Within a few seconds the filler's head tank pulls Supply_Tank under its low float, step 6 opens the inlet, and with the batch ready `Mix_PumpOut` runs: the transfer line fills, the outlet meter counts, Supply_Level climbs to 90 % and the inlet closes. Mix_Tank is left at about a third. Mix_Step goes to 1, `Dose_A` counts water to its preset, PUMP B tops up to HI, the heater and mixer run to 100 °F and eight seconds, and Mix_Step is 4 again about half a minute after the transfer ended: long before Supply_Tank is back down to 20 %. In the test run, seven minutes of production, the kitchen made five batches, Supply_Tank never went below its low float and `Alarm_Word` stayed 0.

During a transfer `Carb_Press` saws between 19 and 22 and `Carb_Vol` (hundredths of a volume) between about 345 and 380; Supply_Tank and the bottles carry the same CO2. On Tank_HMI, STOP stops the kitchen (every kitchen output drops) while the line keeps running, and START runs it again. The kitchen always has a batch in the tank, so while it runs its recipe is locked (chapter 14): STOP it to change A SHARE or MIX TIME, which write `Mix_Ratio` and `Mix_Time`, and START it again; the next batch uses them. The inlet mode is AUTO and the product SODA, so both inlets are open; pick WATER with the kitchen stopped and inlet B locks out.

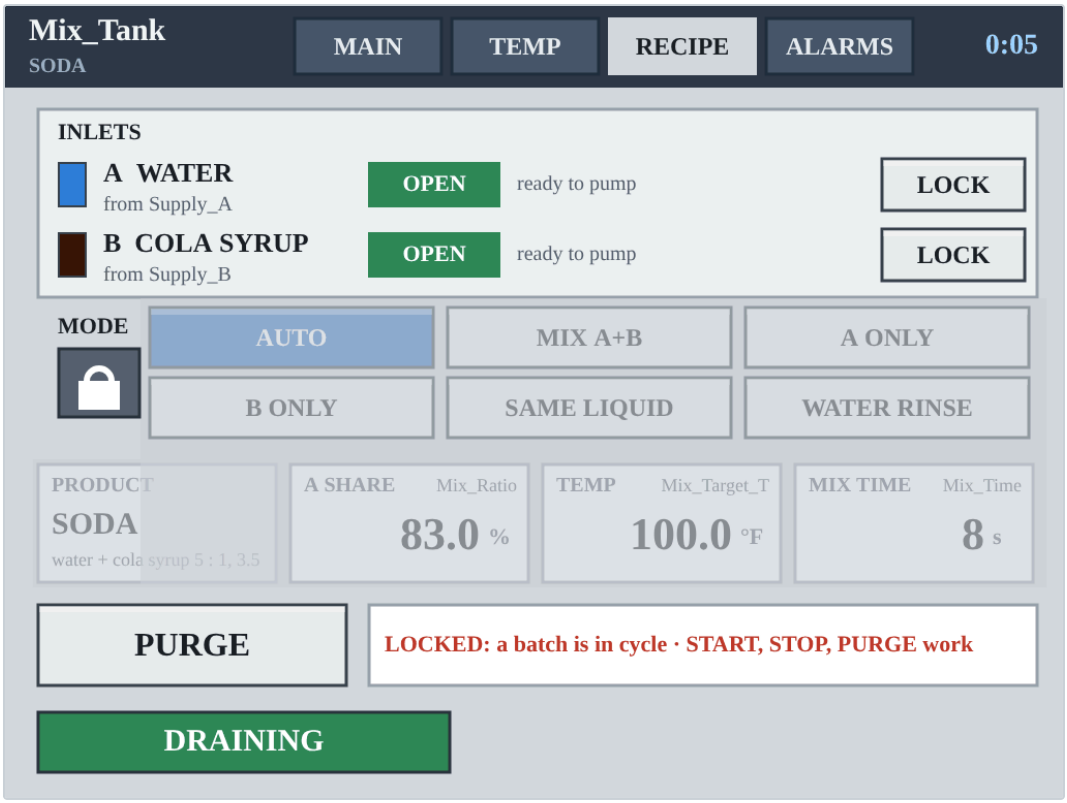
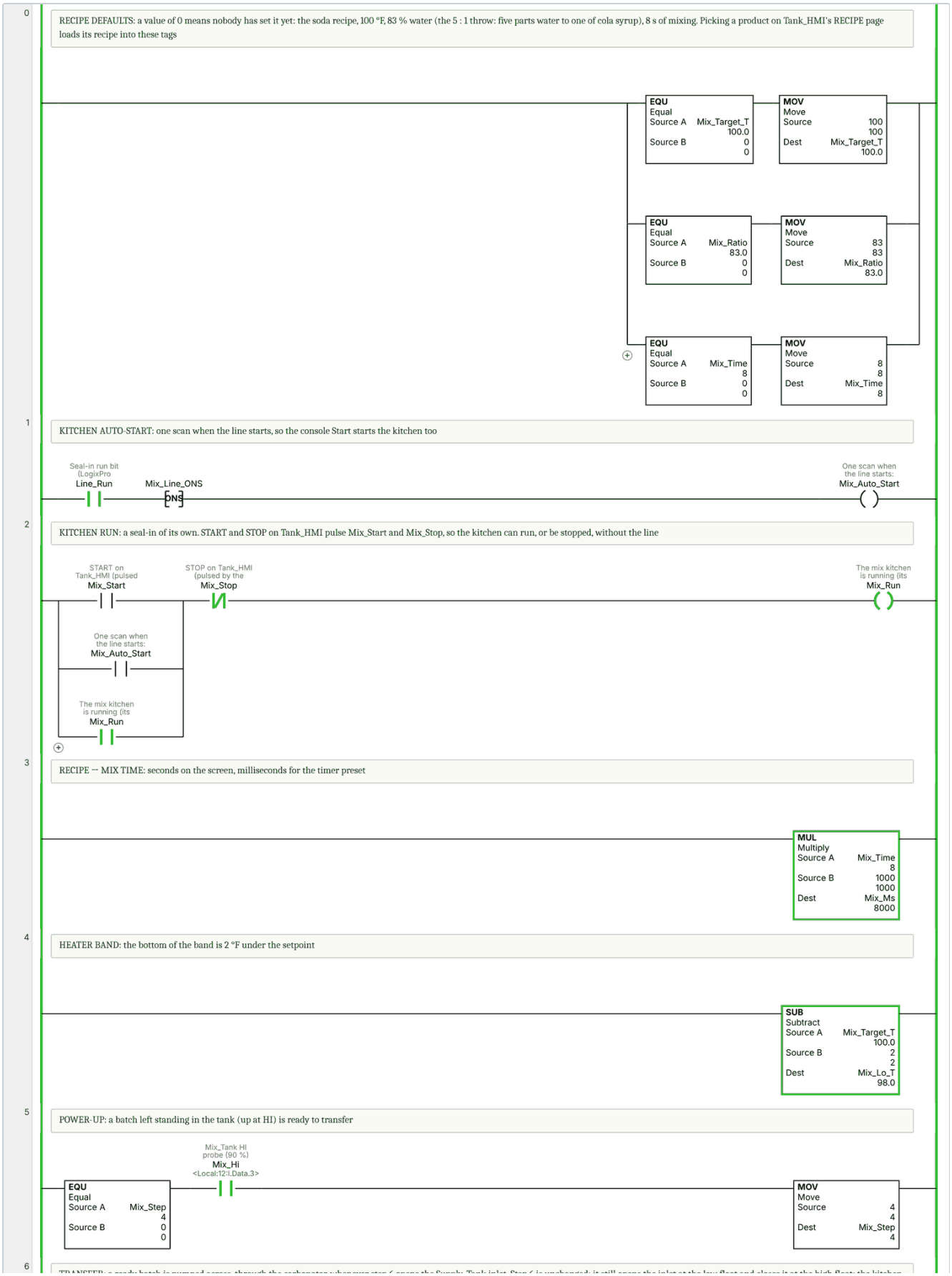
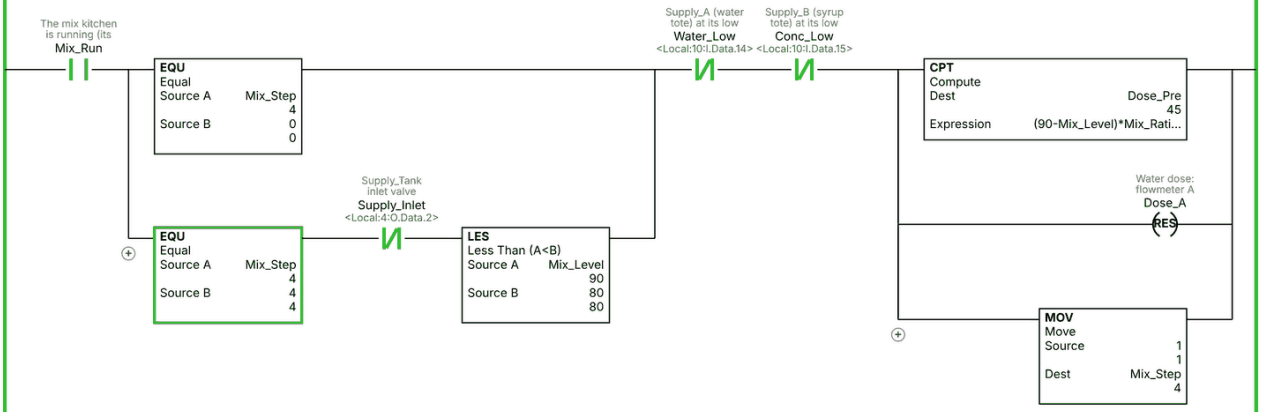


Figure 121. Tank_HMI's RECIPE page during production: water and cola syrup both open, mode AUTO, SODA and its recipe from Mix_Ratio, Mix_Target_T and Mix_Time, and the recipe locked because a batch is in cycle.


0
RECIPE DEFAULTS: a value of 0 means nobody has set it yet: the soda recipe, 100 °F, 83 % water (the 5 : 1 throw: five parts water to one of cola syrup), 8 s of mixing. Picking a product on Tank_HMI's RECIPE page loads its recipe into these tags
EQU Equal Source A Mix_Target_T 100.0 Source B 0 0
MOV Move Source 100 100 Dest Mix_Target_T 100.0
EQU Equal Source A Mix_Ratio 83.0 Source B 0 0
MOV Move Source 83 83 Dest Mix_Ratio 83.0
EQU Equal Source A Mix_Time 8 Source B 0 0
MOV Move Source 8 8 Dest Mix_Time 8
1
KITCHEN AUTO-START: one scan when the line starts, so the console Start starts the kitchen too
Seal-in run bit (LogixPro
Line_Run
Mix_Line_ONS
ONS
One scan when the line starts:
Mix_Auto_Start
2
KITCHEN RUN: a seal-in of its own. START and STOP on Tank_HMI pulse Mix_Start and Mix_Stop, so the kitchen can run, or be stopped, without the line
START on Tank_HMI (pulsed
Mix_Start
STOP on Tank_HMI (pulsed by the
Mix_Stop
The mix kitchen is running (its
Mix_Run
One scan when the line starts:
Mix_Auto_Start
The mix kitchen is running (its
Mix_Run
3
RECIPE -- MIX TIME: seconds on the screen, milliseconds for the timer preset
MUL Multiply Source A Mix_Time 8 Source B 1000 1000 Dest Mix_Ms 8000
4
HEATER BAND: the bottom of the band is 2 °F under the setpoint
SUB Subtract Source A Mix_Target_T 100.0 Source B 2 2 Dest Mix_Lo_T 98.0
5
POWER-UP: a batch left standing in the tank (up at HI) is ready to transfer
Mix_Tank HI probe (90 %)
Mix_Hi
<Local:12:I.Data.3>
EQU Equal Source A Mix_Step 4 Source B 0 0
MOV Move Source 4 4 Dest Mix_Step 4
6

just answers it

7 START A BATCH: the tank is empty (step 0), or a transfer has just ended with room left (step 4, inlet closed, below 80 %), and both totes have product. The batch tops the tank up to HI. One percent of level is 20 units, one pulse of the flowmeter, so the room to HI is 90 − Mix_Level pulses; the water dose is the recipe share of that room, and the syrup fills the rest. What was left in the tank is already at the recipe, so the topped-up batch is too



8 PUMP A in step 1

9 DOSE WATER: count flowmeter A, one pulse per 20 units, to the dose worked out above

10 The water is in: on to the syrup

11 PUMP B in step 2, never past HI

12 SYRUP TO HI: the HI probe ends the top-up

13 MIXER: while the batch heats and mixes, and while it waits and transfers, only with the impeller covered

14 HEAT ON under the band, only with the element covered

15 HEAT OFF at the setpoint, if LO uncovers, or out of step 3

16 HEATER: wanted, kitchen running, and the high-limit thermostat not made

17 MIX TIME counts once the batch is up to temperature with the mixer running

18 BATCH READY

19 EMPTY: LO uncovered for 2 s while transferring (the pump takes the heel below the probe)

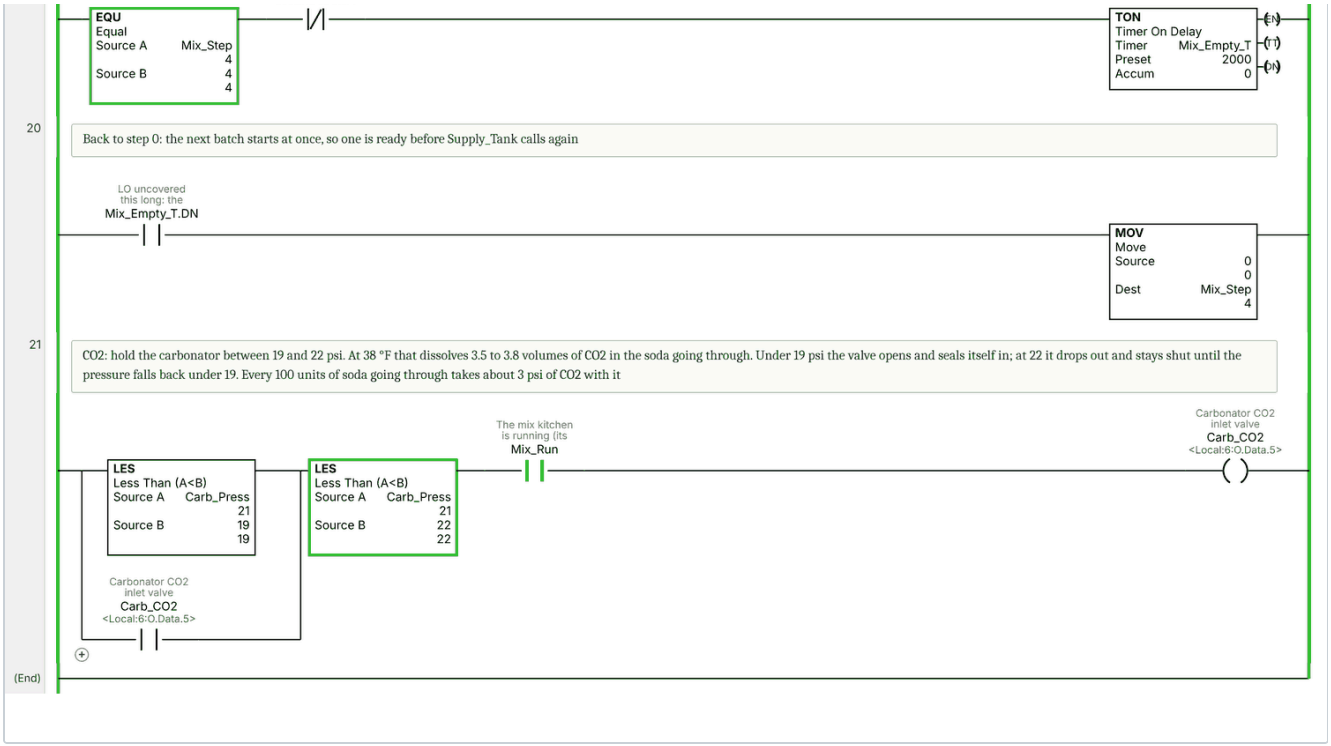


Figure 135. S13_Mix online, with the finished line running: this is what your routine should look like once every step is in. Green is power flow.

TRY IT

Make the kitchen late on purpose. Press STOP on Tank_HMI, set MIX TIME to 60 s on the RECIPE page, press START and run for five minutes. Find the moment Supply_Tank reaches its low float while Mix_Step is still 3, and watch alarm bit 5 latch ten seconds later (chapter 28). Then put MIX TIME back to 8 s, and in rung 6 delete the step-4 branch so the kitchen waits for an empty tank before it makes the next batch. Run again: how long before the same alarm? That is the reason for the top-up.

TRY IT: FLAT SODA

Delete rung 21 and run. The carbonator starts at 20 psi, so the first transfers are fine, but each one takes some of the pressure and nothing puts it back. Watch `Carb_Press` fall to 0 and `Carb_Vol` to about 150, a third of the recipe, and the Simulator Log start to read *filled with flat Soda at Filler: 1.7 volumes of CO2, the recipe calls for 3.5*. Put the rung back and the next transfer is carbonated again; Supply_Tank comes back to the recipe as fresh soda dilutes the flat. Then change the band to 30-34 psi: the soda goes out at about 5 volumes, which would foam out of the bottles at the filler.

TRY IT: A VALVE THAT NEVER SHUTS

Take `LES(Carb_Press,22)` out of rung 21. The valve opens and seals itself in for good; the pressure climbs to 60 psi and the relief valve lifts (the `Relief` input, and a line in the log). That is what the relief valve is for, and why every real band has its top as well as its bottom.

Predict it: the answers

1. Every condition on the way is met, so `Mix_Ratio` becomes **83.0**.
2. The rung is false at `XIO(Mix_Stop)`: `Mix_Stop` is on, so `Mix_Run` is **off** (an OTE writes 0 when its rung is false).
3. Every condition on the way is met, so `Mix_Step` becomes **4**.
4. The rung is false at the EQU: `Mix_Step` is not **4**, so `Mix_PumpOut` is **off** (an OTE writes 0 when its rung is false).
5. Every condition on the way is met, so the CPT writes `Dose_Pre`, `Dose_A` is reset and `Mix_Step` becomes **1**.
6. The rung is false at the EQU: `Mix_Step` is not **1**, so `Mix_PumpA` is **off** (an OTE writes 0 when its rung is false).
7. Every condition on the way is met, so `Dose_A` counts one, if the rung was false last scan.
8. The rung is false at `XIC(Dose_A.DN)`: `Dose_A.DN` (the done bit of `Dose_A`: Water dose: flowmeter A pulses) is off, so the MOV does nothing: `Mix_Step` keeps its value.
9. Every condition on the way is met, so `Mix_PumpB` is **on**.
10. The rung is false at the EQU: `Mix_Step` is not **2**, so the MOV does nothing: `Mix_Step` keeps its value.

11. Every condition on the way is met, so `Mix_Mixer` is **on**.
12. The rung is false at `XIC(Mix_Lo)`: `Mix_Lo` is off, so the OTL does nothing: `Mix_Heat` keeps whatever it had.
13. Every condition on the way is met, so `Mix_Heat` is unlatched, **off**.
14. The rung is false at `XIO(Mix_Thermostat)`: `Mix_Thermostat` is on, so `Mix_Heater` is **off** (an OTE writes 0 when its rung is false).
15. Every condition on the way is met, so `Mix_Timer` times toward its preset.
16. The rung is false at `XIC(Mix_Timer.DN)`: `Mix_Timer.DN` (the done bit of `Mix_Timer`: Mixing time at temperature) is off, so the MOV does nothing: `Mix_Step` keeps its value.
17. Every condition on the way is met, so `Mix_Empty_T` times toward its preset.

31 Commissioning: the whole line, start to finish

With all thirteen routines in place, MainRoutine is thirteen JSRs in step order. Verify Controller (Logic menu): *0 error(s)*. Reset Sim. Then run this checklist, the one the program's own test runs on every release:

1. Run mode. Press **Start**. Run lamp on; divert conveyor turning; Pack_HMI shows RUNNING; the belt steps and a bottle drops every stop.
2. Within a minute bottles are filling at stop 7 (large charge for large bottles — the LARGE lamp on the head block; small — the SMALL lamp) and capping at stop 10 (the magazine indexes a pocket per cap).
3. Broken bottles are dropped into the shredder; the box conveyor positions a box under the chute; IN BOX counts to 4; the box leaves and the next arrives; BOXES counts 1.
4. Small bottles ride the incline to the divert conveyor and are counted on SMALL; large bottles run to the end and are counted on LARGE; SCRAP counts the broken ones. Both sizes are packed, four to a case and one size per case: CASES LARGE and CASES SMALL on Pack_HMI both climb.
5. Nothing is **lost** (the toolbar shows no red number) and the Simulator Log has no *no bottle under it*, *no box* or *nothing to land on* lines.
6. The tank falls to 20 %, the red light flashes, the inlet opens, the level climbs to 90 %. Every time it does, Mix_Tank has a batch ready (Mix_Step 4) and pumps it across; when the inlet closes, the kitchen tops itself up and is at step 4 again within a minute. Supply_Tank never stays below its low float.
7. Dial 0010 on the thumbwheel; the line stops at ten large bottles; Enter clears; Start runs again.
8. Every capped bottle leaves the labeler with a label; `Label_Fault` stays off for the whole run.
9. `Alarm_Word` stays 0 and the fault lamp dark for the whole run, including when you press Stop and Start at random moments.
10. Every few large bottles a case is brought in, stops under the packer head, is packed with four and leaves at Case_Out; CASES counts them, and no empty case is sent out.
11. Each fault of chapter 28's *Run it* (and a drive trip on Case_VFD), injected with Tools → Inject Machine Fault, is caught by the bit the table says, stops the line if it should, and clears with Clear Machine Faults, Reset and Start.

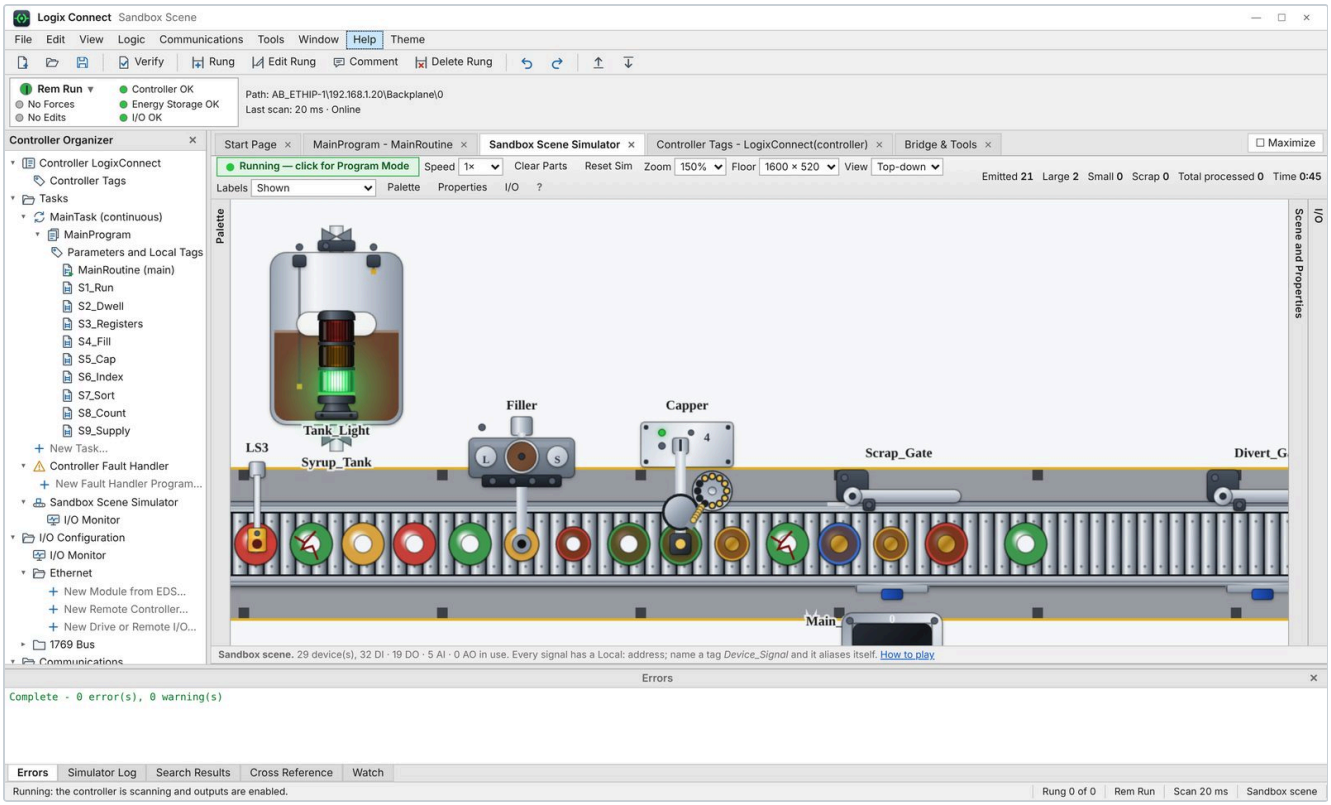


Figure 136. The whole line running in Top-down view: the toolbar counters read the program's own tags.

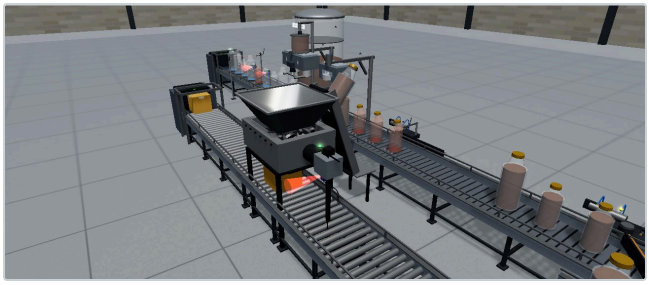


Figure 137. Glass into the box: IN BOX counts up.

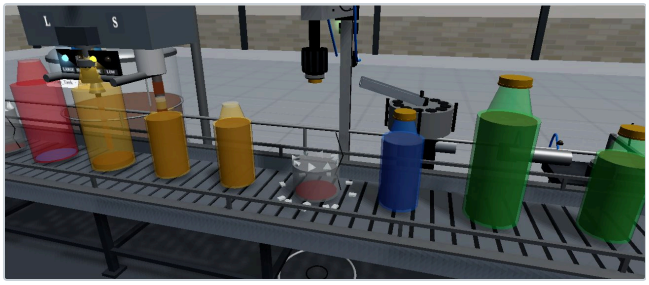


Figure 138. What happens when a chuck comes down on a broken bottle — take the Broken bit out of the cap request and see.

The regression runs every line of that list on every release, plus two dozen random Stop / Start presses. If any line fails for you, the Simulator Log names the device, Monitor Tags shows the bit, and the matching lesson's *Run it* paragraph says what should have happened. The finished template (File → New Project… → Sandbox: LogixPro-style Bottle Line) is the answer key; File → Export / Copy as Text… on both and compare.

Appendix B is the complete listing; Appendix C the bit map. Save the project. You have built a bottling line from an empty floor and written every rung that runs it.

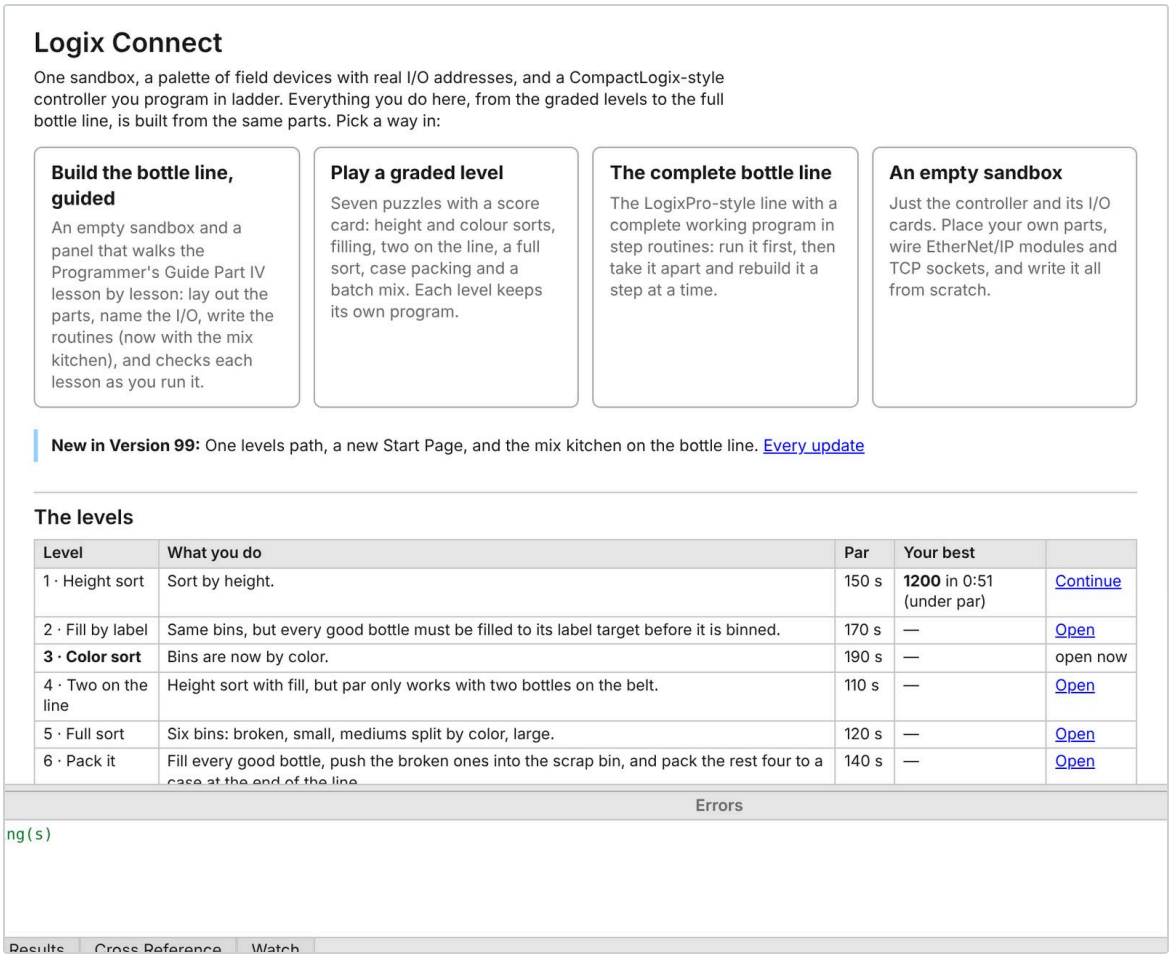
Part V — The graded levels

32 The graded levels: the machine, the bins and the score

Part IV built one machine and one program. Part V is twelve puzzles on a second machine, each a little harder than the last, with a score: the graded levels. They began as a fixed machine of their own; from Version 91 each level could be built from the same palette parts as everything else in this book, from Version 99 that is the only way they come, and from Version 93 every one of them has been solved by the programs printed in the chapters that follow — every program here ran to completion, every bottle in the right bin, before it was printed.

Building a level

File → New Project… → A graded level, choose the level, OK (or Open on the Start Page's table of levels). A level you have not touched opens with an empty program and this machine; one you have worked on opens where you left it. On a level, the simulator toolbar has a **Level** list: pick another level there and the one you were on is kept as it is. Finishing a level records your best score and time, which the Start Page and the New Project list show.



Logix Connect

One sandbox, a palette of field devices with real I/O addresses, and a CompactLogix-style controller you program in ladder. Everything you do here, from the graded levels to the full bottle line, is built from the same parts. Pick a way in:

Build the bottle line, guided
An empty sandbox and a panel that walks the Programmer's Guide Part IV lesson by lesson: lay out the parts, name the I/O, write the routines (now with the mix kitchen), and checks each lesson as you run it.

Play a graded level
Seven puzzles with a score card: height and colour sorts, filling, two on the line, a full sort, case packing and a batch mix. Each level keeps its own program.

The complete bottle line
The LogixPro-style line with a complete working program in step routines: run it first, then take it apart and rebuild it a step at a time.

An empty sandbox
Just the controller and its I/O cards. Place your own parts, wire EtherNet/IP modules and TCP sockets, and write it all from scratch.

New in Version 99: One levels path, a new Start Page, and the mix kitchen on the bottle line. Every update

The levels

Level	What you do	Par	Your best	
1 · Height sort	Sort by height.	150 s	**1200** in 0:51 (under par)	Continue
2 · Fill by label	Same bins, but every good bottle must be filled to its label target before it is binned.	170 s	—	Open
3 · Color sort	Bins are now by color.	190 s	—	open now
4 · Two on the line	Height sort with fill, but par only works with two bottles on the belt.	110 s	—	Open
5 · Full sort	Six bins: broken, small, mediums split by color, large.	120 s	—	Open
6 · Pack it	Fill every good bottle, push the broken ones into the scrap bin, and pack the rest four to a case at the end of the line	140 s	—	Open

Errors

ng(s)

Results | Cross Reference | Watch

Figure 139. The Start Page: four ways in, the levels with par and your best, Open or Continue on each, and what is new in this version.

- **Hopper** — an Emitter in edge mode, limited to the level's number of bottles. `Infeed_Release` drops one bottle on each rising edge; `Infeed_Ready` is 1 while the exit is clear.
- **Main_Belt** — one belt the length of the floor; `Conveyor_Run` runs it.
- **The inspection gantry** — `PE_Inspect` (a bottle is here), three retro beams at 30, 44 and 52 px (`PE_Low`, `PE_Mid`, `PE_High`: a small bottle is 40 px tall and breaks only the low one, a medium is 48 and breaks two, a large is 56 and breaks all three), the colour sensor's three banks (`Color_Red`, `Color_Blue`, `Color_Green`) and the crack eye (`Crack_Sensor`, broken bottles only).
- **The filler** — a dispenser over the belt (`Fill_Valve`), the fill-level sensor beside the nozzle with its analog percent (`Fill_Level`, 0-100 % of the bottle's own label) and the presence eye under the nozzle (`PE_Fill`, short range, placed so a bottle that stops on it stands under the nozzle).
- **One pusher and one bin per lane** — `Div1` … `Div6` extend the pushers; each pusher has its own eye, `PE_Div1` …, just ahead of the arm.
- **The end of the line** — `PE_LineEnd` and a remover that takes anything nobody pushed. On Level 6 the belt is shorter and ends in a **case packer** with a VFD-driven case lane, the same parts and tag names as

step 12 of the bottle line (chapter 29). Level 7 is a different machine altogether: a batch mix tank and its console, no belt at all.

- **The console** — `Start_PB`, `Stop_PB` (a plain button on this machine), `E_Stop` (1 while healthy), `Lamp_Run`, `Lamp_Fault`.

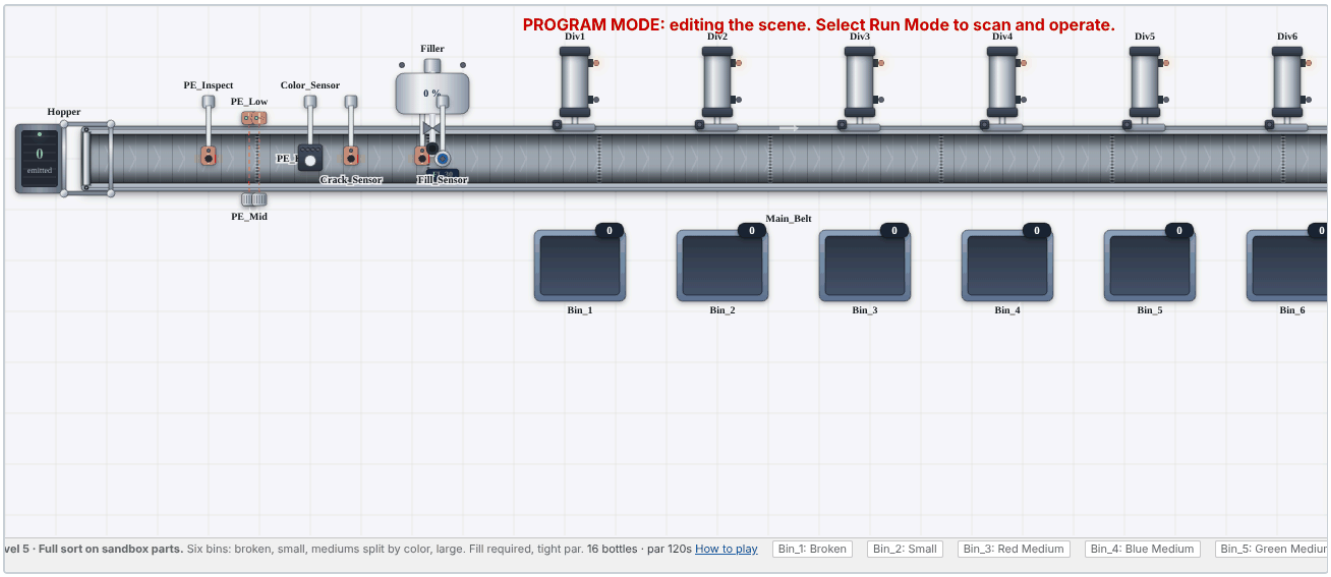


Figure 140. Level 5 on sandbox parts in the Top-down view with Specs on: the hopper, the gantry, the filler, six pushers over six graded bins, the console.

The tags are already made

Every name in the list above is an alias tag the level makes for you, pointing at the part's I/O point. They are the names the levels have always used, so a program written for the old fixed machine runs here unchanged:

Tag	Alias for	Description
`Start_PB`	`Local:3:I.Data.9`	Start push button, momentary
`Stop_PB`	`Local:3:I.Data.10`	Stop push button, momentary
`E_Stop`	`Local:3:I.Data.11`	E-stop, NC: 1 = healthy
`Infeed_Ready`	`Local:1:I.Data.0`	Hopper exit clear: a bottle can be released
`Infeed_Release`	`Local:4:O.Data.0`	Release one bottle onto the belt (rising edge)
`Conveyor_Run`	`Local:4:O.Data.1`	Belt motor
`PE_Inspect`	`Local:1:I.Data.2`	Presence eye at the inspection gantry
`PE_Low`	`Local:1:I.Data.3`	Height beam at 30 px: blocked by every size
`PE_Mid`	`Local:1:I.Data.4`	Height beam at 44 px: blocked by M and L
`PE_High`	`Local:1:I.Data.5`	Height beam at 52 px: blocked by L only

Tag	Alias for	Description
`Color_Red`	`Local:1:I.Data.6`	Colour sensor bank 1: red
`Color_Blue`	`Local:1:I.Data.7`	Colour sensor bank 2: blue
`Color_Green`	`Local:1:I.Data.8`	Colour sensor bank 3: green
`Crack_Sensor`	`Local:1:I.Data.10`	Crack eye: the bottle in front of it is broken
`PE_Fill`	`Local:1:I.Data.14`	Presence eye under the fill nozzle
`Fill_Valve`	`Local:4:O.Data.2`	Fill valve
`Fill_Level`	`Local:7:I.Ch1Data`	Fill of the bottle under the nozzle, 0-100 % of its label
`Fill_Done`	`Local:1:I.Data.13`	Liquid at the fill line of the bottle under the nozzle
`PE_Div1`	`Local:2:I.Data.1`	Presence eye ahead of diverter 1
`Div1`	`Local:4:O.Data.4`	Diverter 1 cylinder
`PE_Div2`	`Local:2:I.Data.5`	Presence eye ahead of diverter 2
`Div2`	`Local:4:O.Data.5`	Diverter 2 cylinder
`PE_Div3`	`Local:2:I.Data.9`	Presence eye ahead of diverter 3
`Div3`	`Local:4:O.Data.6`	Diverter 3 cylinder
`PE_Div4`	`Local:2:I.Data.13`	Presence eye ahead of diverter 4
`Div4`	`Local:4:O.Data.7`	Diverter 4 cylinder
`PE_Div5`	`Local:3:I.Data.1`	Presence eye ahead of diverter 5
`Div5`	`Local:4:O.Data.8`	Diverter 5 cylinder
`PE_Div6`	`Local:3:I.Data.5`	Presence eye ahead of diverter 6
`Div6`	`Local:4:O.Data.9`	Diverter 6 cylinder
`PE_LineEnd`	`Local:3:I.Data.7`	Presence eye at the end of the belt
`Lamp_Run`	`Local:4:O.Data.10`	Run lamp
`Lamp_Fault`	`Local:4:O.Data.11`	Fault lamp

(`Label_Water`, `Label_Soda` and `Label_Juice` are plain tags that stay 0: the label reader of the old machine is not fitted.)

Graded bins and the score

Each bin carries a *Grading* rule — whole or broken bottles, a size, a colour, must be filled — set in its Properties, and it scores what lands in it: **+100** for the right bottle, **−50** for the wrong one, **−40** for the right bin with a wrong fill (or a filled broken bottle). A bottle that falls off the line is −50. When the hopper is

empty and the belt is clear, the level completes: **+200** if you are under par, and a card with your score and stars — three for no mistakes under par, two for no mistakes, one for 70 % right. **Specs** on the simulator toolbar shows each bin's rule. You can give any bin of your own a grading rule, too.

The twelve levels

Level	Bins	Bottles	Fill	Par	The new idea
1 Height sort	broken, S, M, L	10	no	150 s	latching what a sensor saw; one bottle at a time
2 Fill by label	broken, S, M, L	10	yes	170 s	stopping on a sensor and an analog fill
3 Colour sort	broken, R, B, G	12	yes	190 s	a different sensor, the same pattern
4 Two on the line	broken, S, M, L	14	yes	110 s	tracking more than one bottle: FIFO queues
5 Full sort	broken, S, R/B/G medium, L	16	yes	120 s	a class word carrying size and colour
6 Pack it	broken; the rest packed 4 to a case	16	yes	140 s	an analog speed reference, positioning on a drive, the end of a batch
7 Batch Mix	three batches, recipes A, B, C	—	—	240 s	a step sequencer, dosing by counting, temperature with hysteresis
8 Vision check	good, reject	20 finished	done upstream	45 s	a smart camera on EtherNet/IP: the trigger and results handshakes, buffered results, a result FIFO
9 Changeover	good, reject	24 finished, 3 products	done upstream	65 s	changing the camera's job between batches, user data sent with a trigger, a trace record per bottle
10 Servo pick	S, M, L under a servo axis	12	no	80 s	a servo axis on a Festo FHPP drive: home, records, Start and its acknowledge, a vacuum gripper
11 Lot code and verify	good, reject	18 finished, 3 lots	done upstream	45 s	a laser marker's message handshake, the lot carried with the bottle in FIFOs, the camera told what to expect
12 Kitchen changeover	—	4 batches, 2 products	—	270 s	a heel in the tank, a rinse, a tote swap through an operator, both ingredients dosed by flowmeter

Guided mode on a level

A level project opens with the guided-mode panel on, following that level's chapter: the tags to make, every rung of the chapter's program under a fold, a check that MainRoutine verifies, and three results it ticks while you run the level — complete, every bottle right, under par. *Next level* on the score card builds the next level and keeps the panel on. Help → Guided Mode turns it off or back on.

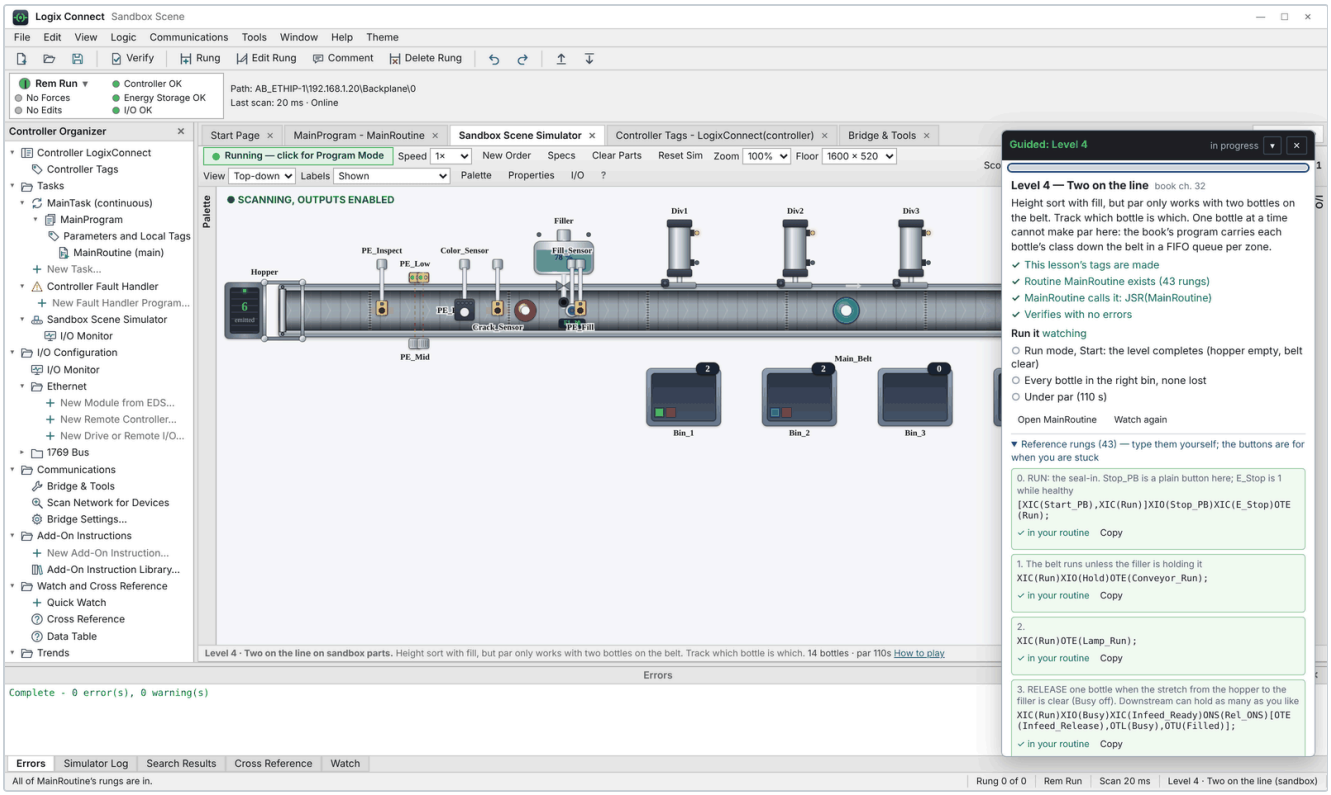


Figure 141. Level 4 with the guided panel: the program in, the line running, the run checks waiting for the level to complete.

Each chapter is built like the lessons of Part IV: what the level asks, the tags, every rung as the editor draws it with the text you type, what the rungs do, *Run it*, the routine online, and the traps. Write each program into MainRoutine. **File → Load Reference Solution: Level N on sandbox parts** installs the Level 4/5 program for whichever level you are in, and works on Levels 1-3 as well; use it as an answer key in a second tab, as chapter 3 describes. **File → Start Level N Over…** puts the level back the way it first opens (parts and their properties, an empty program, the guide); tick *Keep my I/O configuration* to keep your camera module's IP address and connection.

33 Level 1 — Height sort

> **IN THE APP**
>
> A level project opens with guided mode on this chapter: the panel lists the tags the program needs (*Make them*), shows every rung below under its fold with *Copy* and *Add*, checks that MainRoutine verifies, and while you run the level ticks three results — the level completes, every bottle in the right bin, under par.

Ten bottles — small, medium and large, in three colours, some of them broken — come out of the hopper one per release. Sort them by height into four bins: broken bottles to bin 1, small to 2, medium to 3, large to 4. The filler is there but not scored on this level; leave it alone. Par is 150 s.

The plan is the simplest one that works: release a bottle, let it run past the gantry, **remember** what the gantry saw, fire the right pusher when the bottle reaches it, wait for it to land, release the next. One bottle on the belt at a time means one set of memory bits, and that is the whole idea of this level: a sensor tells you something for a fraction of a second, and the program has to keep it until it is needed further down the belt.

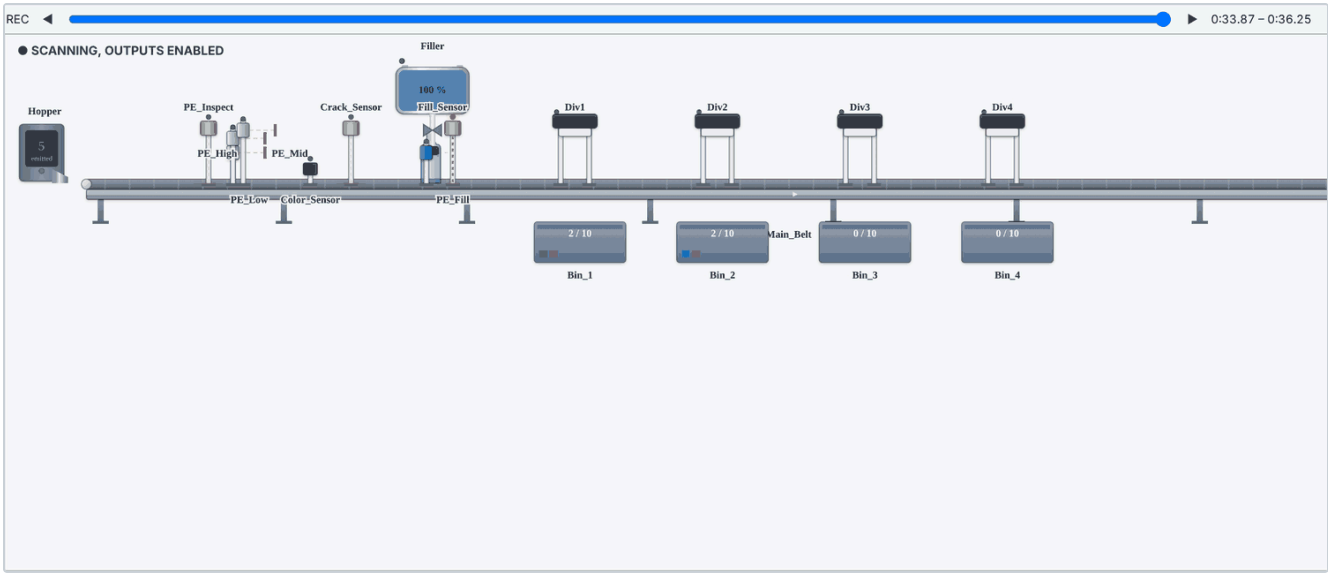


Figure 142. Level 1 running in the Side view: a bottle on its way from the gantry to the pushers.

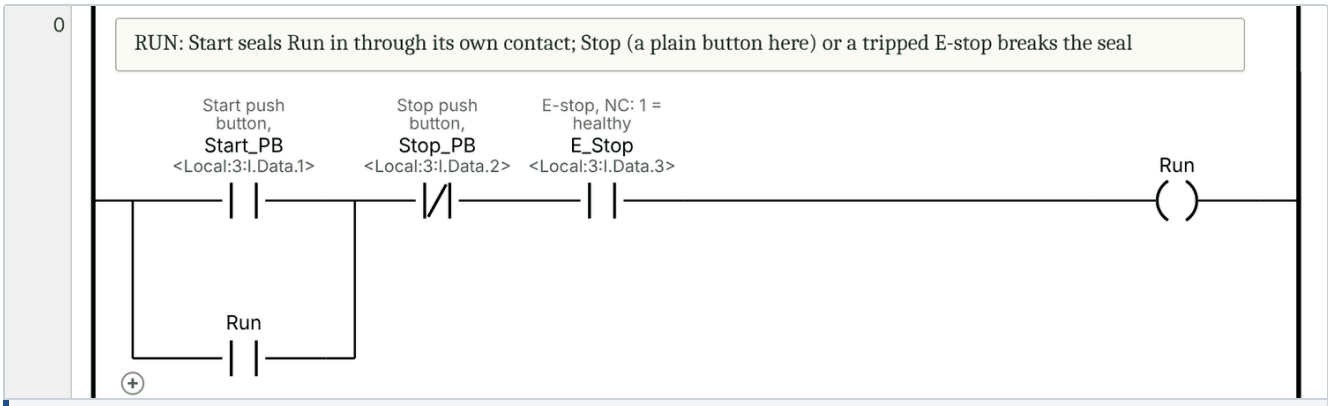
Tags this lesson needs

The level's alias tags are already made for you (chapter 32's table). Create these plain tags before you type the rungs; Verify reports any you miss.

Tag	Type	How to make it	Description
`Run`	BOOL	Edit Tags: name, type	Seal-in run bit
`Busy`	BOOL	Edit Tags: name, type	A bottle is between the hopper and the end of its cycle (Levels 1-3) / between the hopper and the filler (Levels 4-5)
`Rel_ONS`	BOOL	Edit Tags: name, type	One-shot storage for the release
`Is_Low`	BOOL	Edit Tags: name, type	Latched: the low height beam was broken
`Is_Mid`	BOOL	Edit Tags: name, type	Latched: the middle beam was broken
`Is_High`	BOOL	Edit Tags: name, type	Latched: the high beam was broken
`Is_Broken`	BOOL	Edit Tags: name, type	Latched: the crack eye saw a broken bottle
`Trip`	TIMER	Edit Tags: name, type	Watchdog: the bottle ran the whole belt
`DN`	BOOL	Edit Tags: name, type	
`Done`	BOOL	Edit Tags: name, type	A pusher has fired for this bottle
`After`	TIMER	Edit Tags: name, type	Landing time after a push

The run circuit

RUNG 0




```
[XIC(Start_PB),XIC(Run)]XIO(Stop_PB)XIC(E_Stop)OTE(Run);
```

PREDICT IT · 1 `Start_PB` is on, `Stop_PB` is on and `E_Stop` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When either (`Start_PB` (Start push button, momentary) is on) *or* (`Run` (Seal-in run bit) is on), `Stop_PB` (Stop push button, momentary) is off and `E_Stop` (E-stop, NC: 1 = healthy) is on, the rung turns `Run` on (and off again whenever the rung is false).

Left to right:

1. A branch of 2 parallel paths: power gets through if any one of them is true.
   - **Path 1**
     1. `XIC(Start_PB)` *Examine If Closed* asks: true when `Start_PB` (Start push button, momentary) is on.
   - **Path 2**
     1. `XIC(Run)` *Examine If Closed* asks: true when `Run` (Seal-in run bit) is on.
2. `XIO(Stop_PB)` *Examine If Open* asks: true when `Stop_PB` (Stop push button, momentary) is off.
3. `XIC(E_Stop)` *Examine If Closed* asks: true when `E_Stop` (E-stop, NC: 1 = healthy) is on.
4. `OTE(Run)` *Output Energize* does: turns `Run` on (and off again whenever the rung is false). Every scan it writes the rung state: true on, false off.

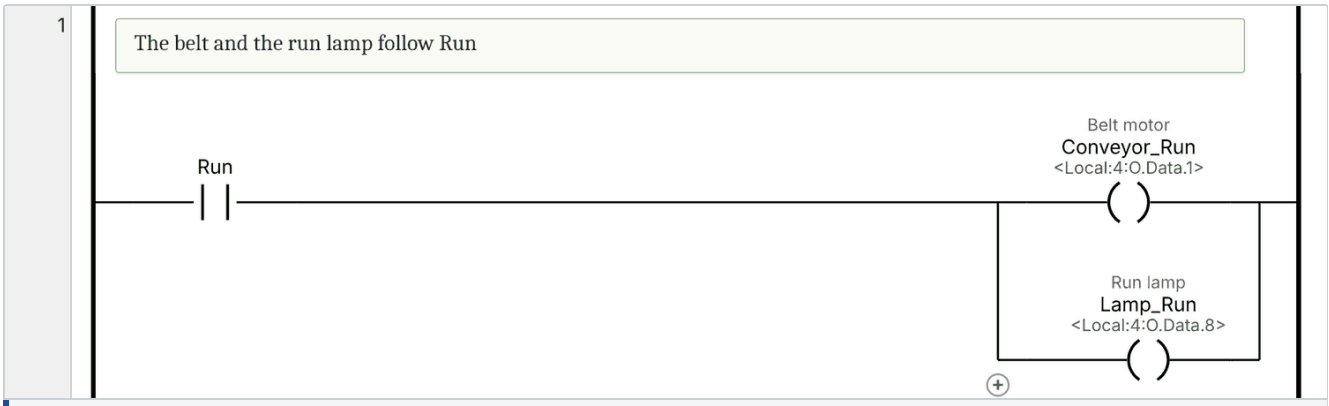
Why it is built this way:

Seal-in. The start condition turns `Run` on, and `Run` 's own contact in the parallel path keeps the rung true after the start goes away. Anything in series outside the branch (a stop, a fault) breaks it, and then it stays off until the start is made again.

> **Think of it like this:** a doorbell button and a hand. The start button is the doorbell, pressed for a moment. The seal-in contact is a hand that grabs the circuit as soon as the output comes on and keeps holding after you let go. The stop button, in series, is the only thing that can knock that hand away.

> **Watch for:** The stop has to be in series outside the branch. Put it inside the start path only and the seal-in path bypasses it: the stop no longer stops.

RUNG 1




```
XIC(Run)[OTE(Conveyor_Run),OTE(Lamp_Run)];
```

HOW THIS RUNG WORKS

In words. When `Run` (Seal-in run bit) is on, the rung turns `Conveyor_Run` (Belt motor) on (and off again whenever the rung is false) and turns `Lamp_Run` (Run lamp) on (and off again whenever the rung is false).

Left to right:

1. `XIC(Run)` *Examine If Closed* asks: true when `Run` (Seal-in run bit) is on.
2. A branch of 2 parallel paths: each path is tested on its own and does its own work.

   Path 1

   1. `OTE(Conveyor_Run)` *Output Energize* does: turns `Conveyor_Run` (Belt motor) on (and off again whenever the rung is false).

   Path 2

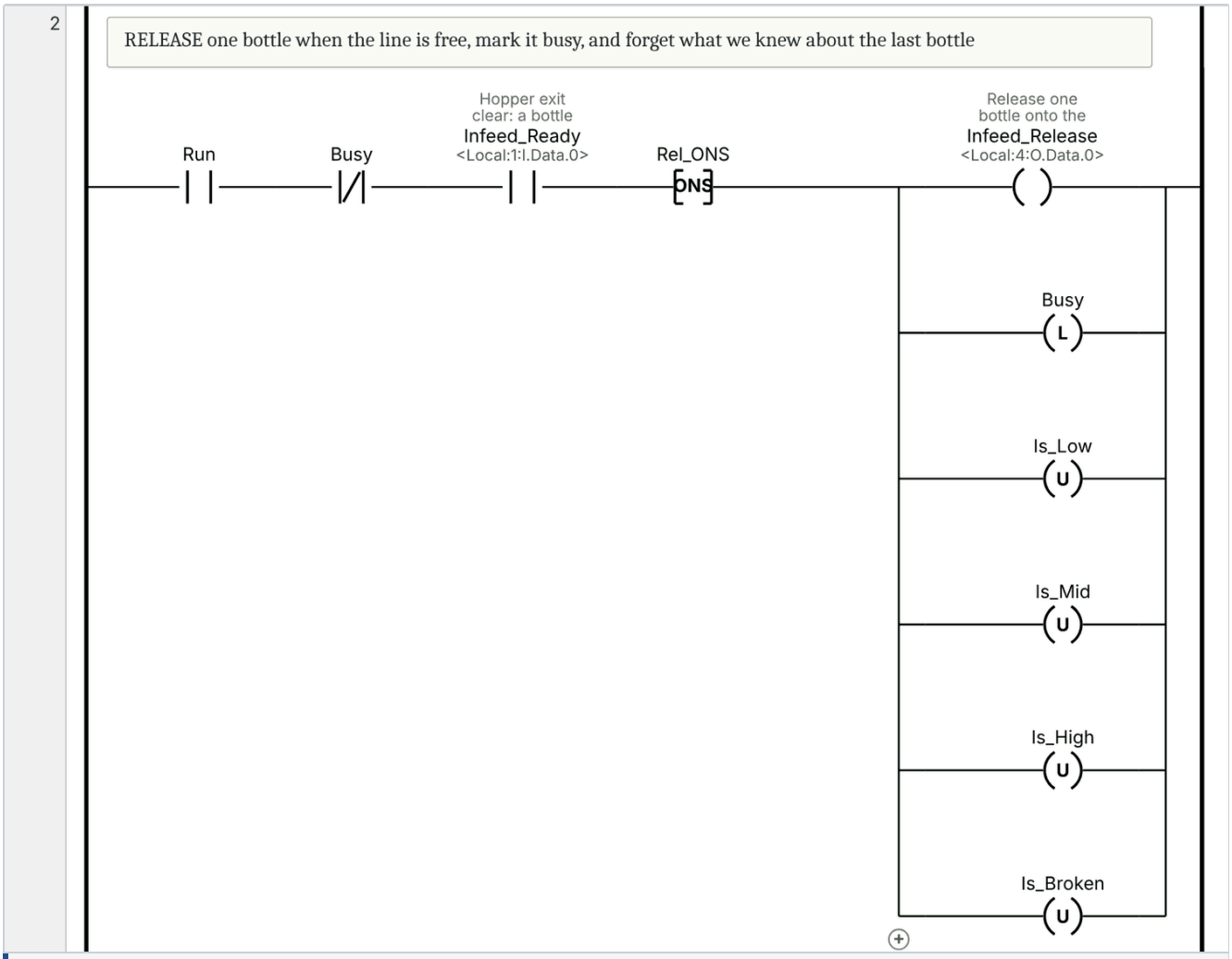
   1. `OTE(Lamp_Run)` *Output Energize* does: turns `Lamp_Run` (Run lamp) on (and off again whenever the rung is false).

> **Think of it like this:** a rung is a pipe with valves in it, read left to right. Every contact is a valve: XIC is open while its bit is 1, XIO while its bit is 0. Water reaches the output at the right only if every valve in a row is open. The output does not remember anything: on every scan it is simply wet or dry.

Rung 0 is chapter 4's seal-in. On this machine Stop is a plain normally-open button, so it is `XIO(Stop_PB)`, not the `XIC(Stop)` of the bottle line, whose stop button is normally closed — read the device's settings, not your habits. `E_Stop` is 1 while the E-stop is healthy. **Rung 1** runs the belt and lights the run lamp whenever the line runs; on this level the belt never stops.

Releasing one bottle

RUNG 2




```
XIC(Run)XIO(Busy)XIC(Infeed_Ready)ONS(Rel_ONS)
[OTE(Infeed_Release),OTL(Busy),OTU(Is_Low),OTU(Is_Mid),OTU(Is_High),OTU(Is_Broken)];
```

HOW THIS RUNG WORKS

In words. When `Run` (Seal-in run bit) is on, `Busy` (A bottle is between the hopper and the end of its cycle (Levels 1-3) / between the hopper and the filler (Levels 4-5)) is off, `Infeed_Ready` (Hopper exit clear: a bottle can be released) is on and it is the first scan they are true (the one-shot, remembered in `Rel_ONS` (One-shot storage for the release)), the rung turns `Infeed_Release` (Release one bottle onto the belt (rising edge)) on (and off again whenever the rung is false), latches `Busy` on, unlatches `Is_Low` (Latched: the low height beam was broken) (turns it off), unlatches `Is_Mid` (Latched: the middle beam was broken) (turns it off), unlatches `Is_High` (Latched: the high beam was broken) (turns it off) and unlatches `Is_Broken` (Latched: the crack eye saw a broken bottle) (turns it off).

Left to right:

1. `XIC(Run)` *Examine If Closed* asks: true when `Run` (Seal-in run bit) is on.
2. `XIO(Busy)` *Examine If Open* asks: true when `Busy` (A bottle is between the hopper and the end of its cycle (Levels 1-3) / between the hopper and the filler (Levels 4-5)) is off.

3. `XIC(Infeed_Ready)` *Examine If Closed* asks: true when `Infeed_Ready` (Hopper exit clear: a bottle can be released) is on.
4. `ONS(Rel_ONS)` *One Shot* asks: true when it is the first scan they are true (the one-shot, remembered in `Rel_ONS` (One-shot storage for the release)). It passes power for one scan only, the scan its input goes from false to true; its storage bit remembers last scan.
5. A branch of 6 parallel paths: each path is tested on its own and does its own work.

   Path 1

   1. `OTE(Infeed_Release)` *Output Energize* does: turns `Infeed_Release` (Release one bottle onto the belt (rising edge)) on (and off again whenever the rung is false).

   Path 2

   1. `OTL(Busy)` *Output Latch* does: latches `Busy` on. It only ever sets the bit; the bit stays on after the rung goes false.

   Path 3

   1. `OTU(Is_Low)` *Output Unlatch* does: unlatches `Is_Low` (Latched: the low height beam was broken) (turns it off). It only ever clears the bit.

   Path 4

   1. `OTU(Is_Mid)` *Output Unlatch* does: unlatches `Is_Mid` (Latched: the middle beam was broken) (turns it off).

   Path 5

   1. `OTU(Is_High)` *Output Unlatch* does: unlatches `Is_High` (Latched: the high beam was broken) (turns it off).

   Path 6

   1. `OTU(Is_Broken)` *Output Unlatch* does: unlatches `Is_Broken` (Latched: the crack eye saw a broken bottle) (turns it off).

Why it is built this way:

One-shot. Everything after the ONS happens once per event, not on every scan the condition holds; `Rel_ONS` is only its memory of the last scan.

A latch. `Busy` stays on after this rung goes false; an `OTU` elsewhere in the program clears it.

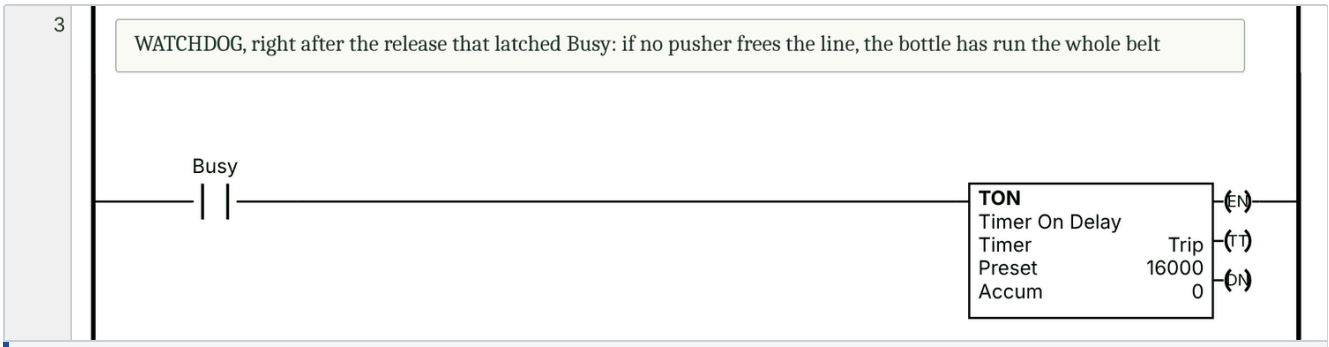
> **Think of it like this:** a doorbell that rings once however long you lean on the button. The ONS looks at what the button was doing last scan: only the change from up to down rings it.

> **Watch for:** Each ONS needs its own storage bit. Two ONS sharing one bit make one of them miss its edges.

The release rung fires once — the one-shot — when the line runs, no bottle is in progress (`XIO(Busy)`) and the hopper's exit is clear. On the same scan it latches `Busy`, so nothing else is released until this bottle is dealt with, and unlatches the four memory bits, so this bottle starts with a clean slate. The hopper is in edge mode: one rising edge of `Infeed_Release`, one bottle.

A way out of Busy: the watchdog

RUNG 3




```
XIC(Busy)TON(Trip,16000,0);
```

HOW THIS RUNG WORKS

In words. When `Busy` (A bottle is between the hopper and the end of its cycle (Levels 1-3) / between the hopper and the filler (Levels 4-5)) is on, the rung runs the timer `Trip` (Watchdog: the bottle ran the whole belt) toward 16000 ms (16 s).

Left to right:

1. `XIC(Busy)` *Examine If Closed* asks: true when `Busy` (A bottle is between the hopper and the end of its cycle (Levels 1-3) / between the hopper and the filler (Levels 4-5)) is on.
2. `TON(Trip,16000,0)` *Timer On Delay* does: runs the timer `Trip` (Watchdog: the bottle ran the whole belt) toward 16000 ms (16 s). The accumulator counts while the rung is true; .DN comes on at the preset; the moment the rung goes false the timer resets.

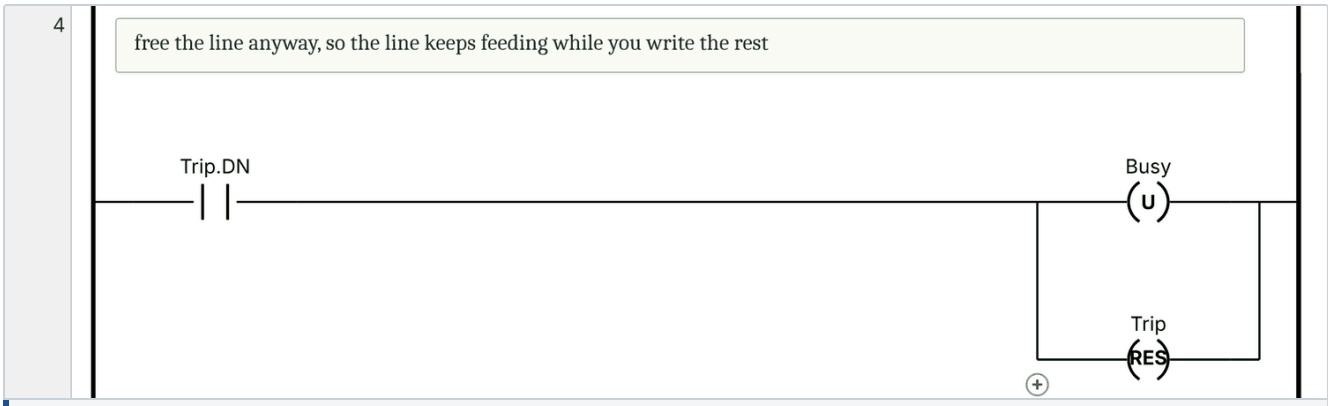
Why it is built this way:

A timer. `Trip` only times while this rung is true; other rungs use its `.DN` (done) or `.TT` (timing) bits. Being a TON, it starts again from zero every time the rung goes false.

> **Think of it like this:** a kitchen timer that only runs while you hold your finger on it. Take your finger off and it snaps back to zero. Hold it long enough and the done light comes on, and stays on as long as your finger is still there.

> **Watch for:** If anything in series flickers false for one scan, a TON starts over from zero. Use RTO when the time must survive an interruption.

RUNG 4




```
XIC(Trip.DN)[OTU(Busy),RES(Trip)];
```

HOW THIS RUNG WORKS

In words. When `Trip.DN` (the done bit of `Trip` : Watchdog: the bottle ran the whole belt) is on, the rung unlatches `Busy` (A bottle is between the hopper and the end of its cycle (Levels 1-3) / between the hopper and the filler (Levels 4-5)) (turns it off) and resets `Trip` .

Left to right:

1. `XIC(Trip.DN)` *Examine If Closed* asks: true when `Trip.DN` (the done bit of `Trip` : Watchdog: the bottle ran the whole belt) is on.
2. A branch of 2 parallel paths: each path is tested on its own and does its own work.

   Path 1

   1. `OTU(Busy)` *Output Unlatch* does: unlatches `Busy` (A bottle is between the hopper and the end of its cycle (Levels 1-3) / between the hopper and the filler (Levels 4-5)) (turns it off).

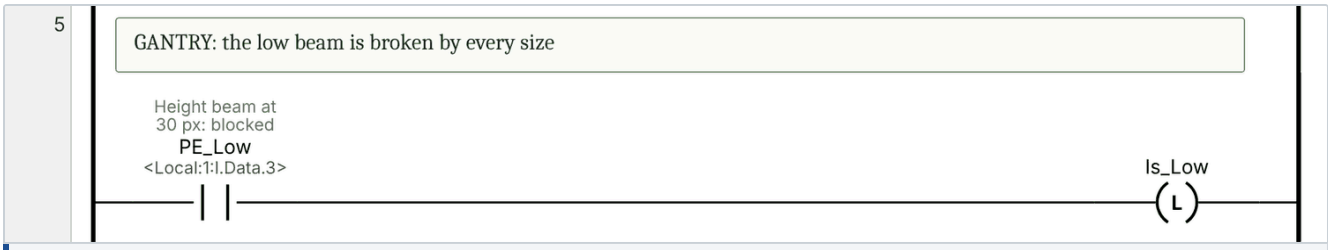
   Path 2

   1. `RES(Trip)` *Reset* does: resets `Trip` . It zeroes a timer or counter (accumulator and status bits), or a control (its position and status bits).

`Busy` is latched, so the next thing to write is a way to free it. **Rungs 3–4** are the watchdog: while a bottle is in progress a 16 s timer runs, and when it is done the line is freed anyway — a bottle nobody wanted, or a bug, runs to the end of the belt and the next one is released. Written straight after the latch, it also keeps the line feeding while you write the rest: every bottle runs to the end and 16 s later the next one drops, so each rung below can be tested in Run the moment it is written. A latch with no way out written next to it is how lines stall.

Remembering what the gantry saw

RUNG 5




```
XIC(PE_Low)OTL(Is_Low);
```

HOW THIS RUNG WORKS

In words. When `PE_Low` (Height beam at 30 px: blocked by every size) is on, the rung latches `Is_Low` (Latched: the low height beam was broken) on.

Left to right:

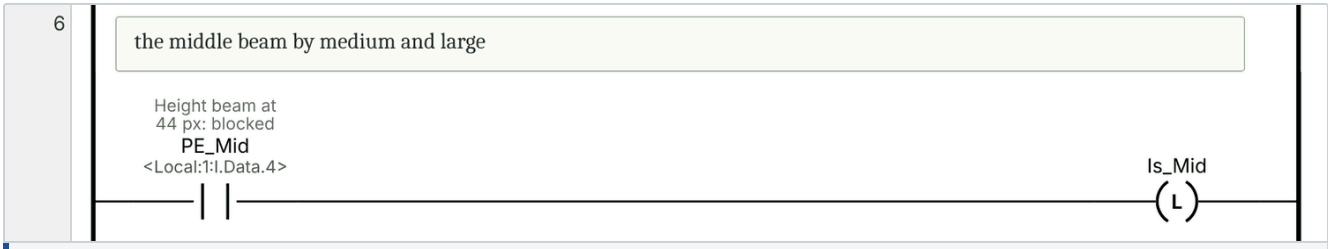
1. `XIC(PE_Low)` *Examine If Closed* asks: true when `PE_Low` (Height beam at 30 px: blocked by every size) is on.
2. `OTL(Is_Low)` *Output Latch* does: latches `Is_Low` (Latched: the low height beam was broken) on.

Why it is built this way:

A latch: it stays on until an OTU clears it.

> **Watch for:** A latched bit survives the condition that set it: every way in needs a way out, or it stays on for good.

RUNG 6




```
XIC(PE_Mid)OTL(Is_Mid);
```

HOW THIS RUNG WORKS

In words. When `PE_Mid` (Height beam at 44 px: blocked by M and L) is on, the rung latches `Is_Mid` (Latched: the middle beam was broken) on.

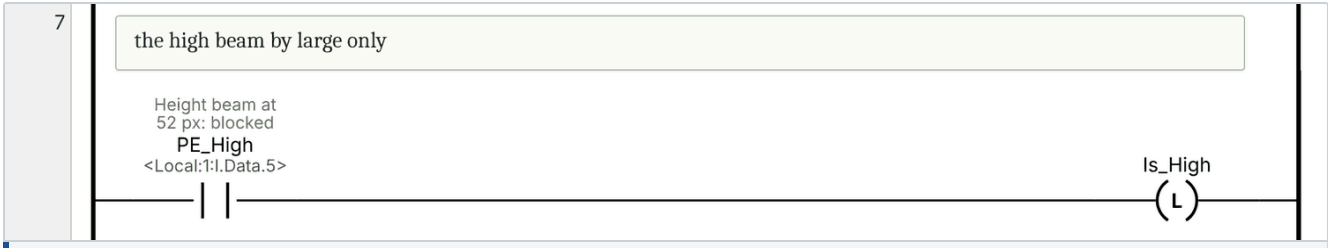
Left to right:

1. `XIC(PE_Mid)` *Examine If Closed* asks: true when `PE_Mid` (Height beam at 44 px: blocked by M and L) is on.
2. `OTL(Is_Mid)` *Output Latch* does: latches `Is_Mid` (Latched: the middle beam was broken) on.

Why it is built this way:

A latch: it stays on until an OTU clears it.

RUNG 7




```
XIC(PE_High)OTL(Is_High);
```

HOW THIS RUNG WORKS

In words. When `PE_High` (Height beam at 52 px: blocked by L only) is on, the rung latches `Is_High` (Latched: the high beam was broken) on.

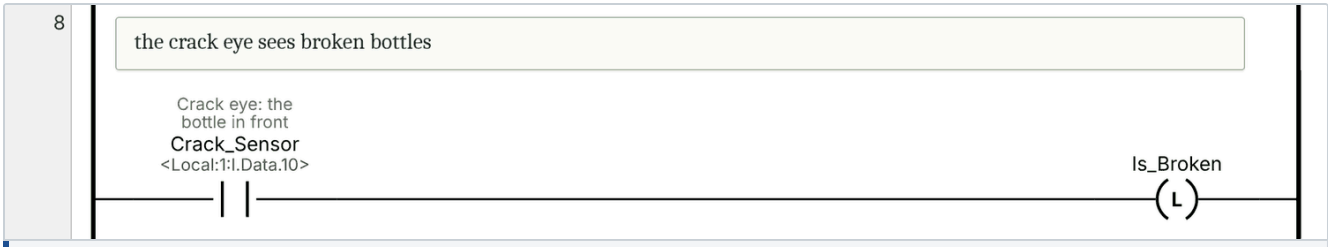
Left to right:

1. `XIC(PE_High)` *Examine If Closed* asks: true when `PE_High` (Height beam at 52 px: blocked by L only) is on.
2. `OTL(Is_High)` *Output Latch* does: latches `Is_High` (Latched: the high beam was broken) on.

Why it is built this way:

A latch: it stays on until an OTU clears it.

RUNG 8




```
XIC(Crack_Sensor)OTL(Is_Broken);
```

HOW THIS RUNG WORKS

In words. When `Crack_Sensor` (Crack eye: the bottle in front of it is broken) is on, the rung latches `Is_Broken` (Latched: the crack eye saw a broken bottle) on.

Left to right:

1. `XIC(Crack_Sensor)` *Examine If Closed* asks: true when `Crack_Sensor` (Crack eye: the bottle in front of it is broken) is on.
2. `OTL(Is_Broken)` *Output Latch* does: latches `Is_Broken` (Latched: the crack eye saw a broken bottle) on.

Why it is built this way:

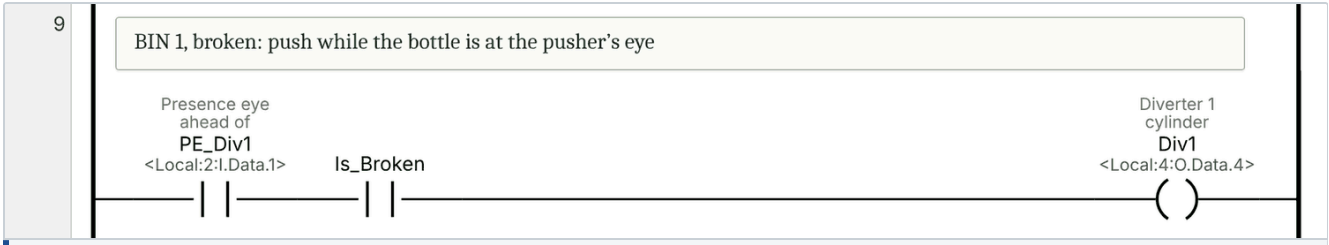
A latch: it stays on until an OTU clears it.

Four latches. Each beam is broken only while the bottle passes it — about half a second — but the pushers are metres downstream, so the program latches what it saw. A small bottle sets only `Is_Low`; a medium sets `Is_Low` and `Is_Mid`; a large sets all three; the crack eye sets `Is_Broken`. Nothing clears them

except the next release.

Firing the right pusher

RUNG 9




```
XIC(PE_Div1)XIC(Is_Broken)OTE(Div1);
```

PREDICT IT · 2 `PE_Div1` is on and `Is_Broken` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

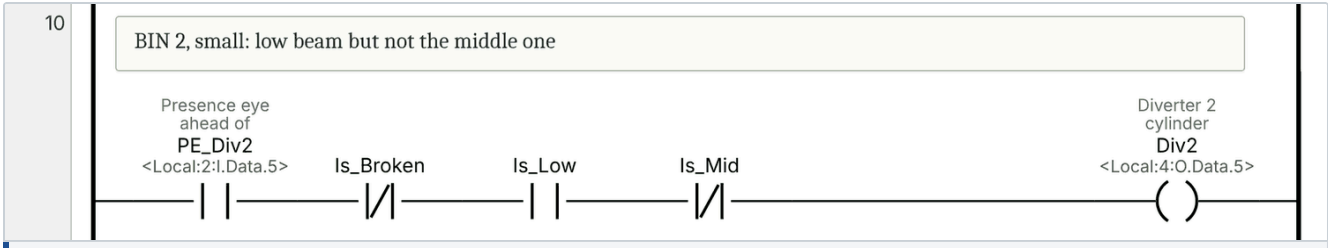
HOW THIS RUNG WORKS

In words. When `PE_Div1` (Presence eye ahead of diverter 1) is on and `Is_Broken` (Latched: the crack eye saw a broken bottle) is on, the rung turns `Div1` (Diverter 1 cylinder) on (and off again whenever the rung is false).

Left to right:

1. `XIC(PE_Div1)` *Examine If Closed* asks: true when `PE_Div1` (Presence eye ahead of diverter 1) is on.
2. `XIC(Is_Broken)` *Examine If Closed* asks: true when `Is_Broken` (Latched: the crack eye saw a broken bottle) is on.
3. `OTE(Div1)` *Output Energize* does: turns `Div1` (Diverter 1 cylinder) on (and off again whenever the rung is false).

RUNG 10




```
XIC(PE_Div2)XIO(Is_Broken)XIC(Is_Low)XIO(Is_Mid)OTE(Div2);
```

PREDICT IT · 3 `PE_Div2` is on, `Is_Broken` is on, `Is_Low` is on and `Is_Mid` is off. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

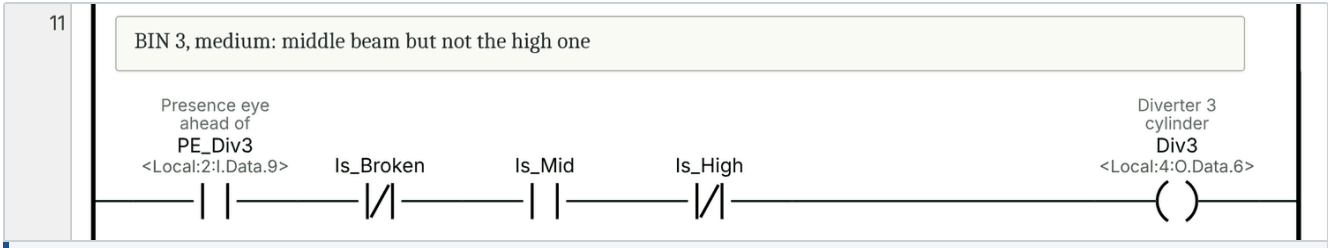
HOW THIS RUNG WORKS

In words. When `PE_Div2` (Presence eye ahead of diverter 2) is on, `Is_Broken` (Latched: the crack eye saw a broken bottle) is off, `Is_Low` (Latched: the low height beam was broken) is on and `Is_Mid` (Latched: the middle beam was broken) is off, the rung turns `Div2` (Diverter 2 cylinder) on (and off again whenever the rung is false).

Left to right:

1. `XIC(PE_Div2)` *Examine If Closed* asks: true when `PE_Div2` (Presence eye ahead of diverter 2) is on.
2. `XIO(Is_Broken)` *Examine If Open* asks: true when `Is_Broken` (Latched: the crack eye saw a broken bottle) is off.
3. `XIC(Is_Low)` *Examine If Closed* asks: true when `Is_Low` (Latched: the low height beam was broken) is on.
4. `XIO(Is_Mid)` *Examine If Open* asks: true when `Is_Mid` (Latched: the middle beam was broken) is off.
5. `OTE(Div2)` *Output Energize* does: turns `Div2` (Diverter 2 cylinder) on (and off again whenever the rung is false).

RUNG 11




```
XIC(PE_Div3)XIO(Is_Broken)XIC(Is_Mid)XIO(Is_High)OTE(Div3);
```

PREDICT IT · 4 `PE_Div3` is on, `Is_Broken` is off, `Is_Mid` is on and `Is_High` is off. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

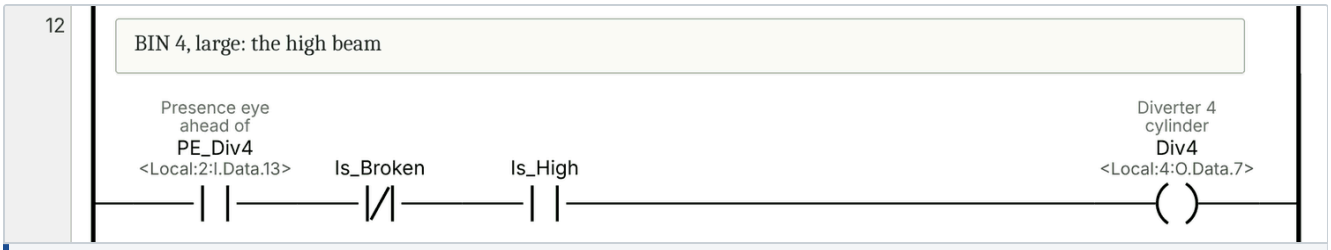
HOW THIS RUNG WORKS

In words. When `PE_Div3` (Presence eye ahead of diverter 3) is on, `Is_Broken` (Latched: the crack eye saw a broken bottle) is off, `Is_Mid` (Latched: the middle beam was broken) is on and `Is_High` (Latched: the high beam was broken) is off, the rung turns `Div3` (Diverter 3 cylinder) on (and off again whenever the rung is false).

Left to right:

1. `XIC(PE_Div3)` *Examine If Closed* asks: true when `PE_Div3` (Presence eye ahead of diverter 3) is on.
2. `XIO(Is_Broken)` *Examine If Open* asks: true when `Is_Broken` (Latched: the crack eye saw a broken bottle) is off.
3. `XIC(Is_Mid)` *Examine If Closed* asks: true when `Is_Mid` (Latched: the middle beam was broken) is on.
4. `XIO(Is_High)` *Examine If Open* asks: true when `Is_High` (Latched: the high beam was broken) is off.
5. `OTE(Div3)` *Output Energize* does: turns `Div3` (Diverter 3 cylinder) on (and off again whenever the rung is false).

RUNG 12




```
XIC(PE_Div4)XIO(Is_Broken)XIC(Is_High)OTE(Div4);
```

PREDICT IT · 5 `PE_Div4` is off, `Is_Broken` is off and `Is_High` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `PE_Div4` (Presence eye ahead of diverter 4) is on, `Is_Broken` (Latched: the crack eye saw a broken bottle) is off and `Is_High` (Latched: the high beam was broken) is on, the rung turns `Div4` (Diverter 4 cylinder) on (and off again whenever the rung is false).

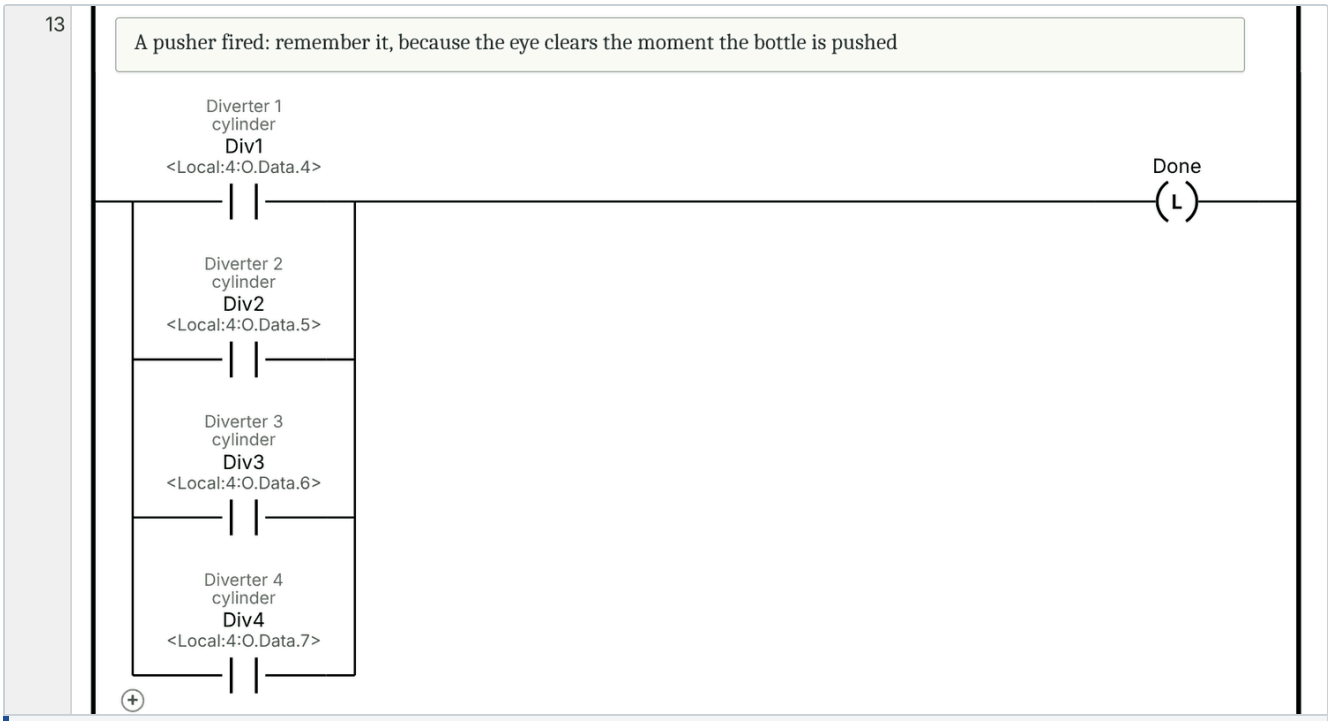
Left to right:

1. `XIC(PE_Div4)` *Examine If Closed* asks: true when `PE_Div4` (Presence eye ahead of diverter 4) is on.
2. `XIO(Is_Broken)` *Examine If Open* asks: true when `Is_Broken` (Latched: the crack eye saw a broken bottle) is off.
3. `XIC(Is_High)` *Examine If Closed* asks: true when `Is_High` (Latched: the high beam was broken) is on.
4. `OTE(Div4)` *Output Energize* does: turns `Div4` (Diverter 4 cylinder) on (and off again whenever the rung is false).

Each pusher has its own eye just ahead of the arm; the rung pushes while that eye sees the bottle and the memory says the bottle belongs in this bin. The size tests are written as *this beam and not the next one up*: small is low-not-mid, medium is mid-not-high, large is high. Every whole-bottle bin tests `XIO(Is_Broken)` first: a broken bottle is broken whatever its size, and bin 1 is upstream, so it has already gone by the time a broken bottle could reach bin 2 — but the test costs nothing and saves you when the bins are reordered.

Freeing the line

RUNG 13




```
[XIC(Div1),XIC(Div2),XIC(Div3),XIC(Div4)]OTL(Done);
```

PREDICT IT · 6 `Div1` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When either (`Div1` (Diverter 1 cylinder) is on) *or* (`Div2` (Diverter 2 cylinder) is on) *or* (`Div3` (Diverter 3 cylinder) is on) *or* (`Div4` (Diverter 4 cylinder) is on), the rung latches `Done` (A pusher has fired for this bottle) on.

Left to right:

1. A branch of 4 parallel paths: power gets through if any one of them is true.
   - **Path 1**
     1. `XIC(Div1)` *Examine If Closed* asks: true when `Div1` (Diverter 1 cylinder) is on.
   - **Path 2**
     1. `XIC(Div2)` *Examine If Closed* asks: true when `Div2` (Diverter 2 cylinder) is on.
   - **Path 3**
     1. `XIC(Div3)` *Examine If Closed* asks: true when `Div3` (Diverter 3 cylinder) is on.
   - **Path 4**
     1. `XIC(Div4)` *Examine If Closed* asks: true when `Div4` (Diverter 4 cylinder) is on.
2. `OTL(Done)` *Output Latch* does: latches `Done` (A pusher has fired for this bottle) on.

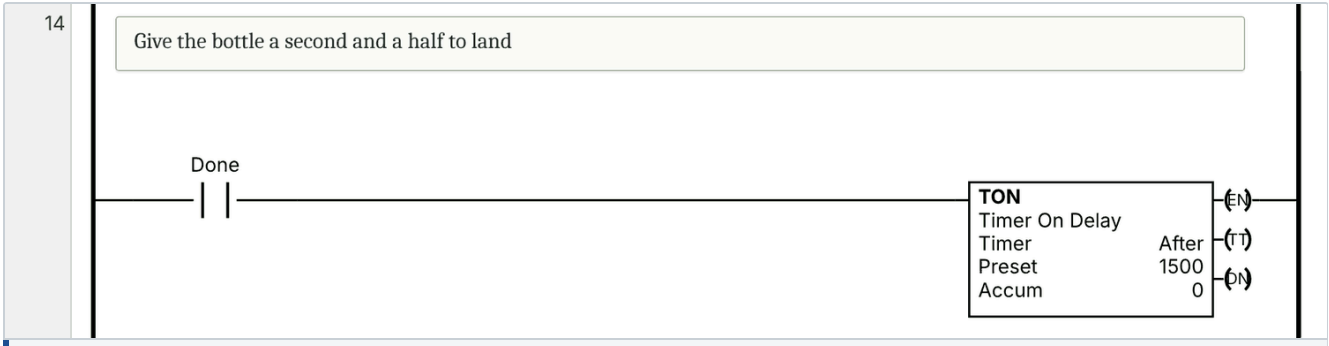
Why it is built this way:

A latch: it stays on until an OTU clears it.

> **Think of it like this:** parallel paths are two roads to the same town. If either road is open you get there; the rung only

> cares that power reaches the far end, not which way it went.

RUNG 14




```
XIC(Done)TON(After,1500,0);
```

HOW THIS RUNG WORKS

In words. When `Done` (A pusher has fired for this bottle) is on, the rung runs the timer `After` (Landing time after a push) toward 1500 ms (1.5 s).

Left to right:

1. `XIC(Done)` *Examine If Closed* asks: true when `Done` (A pusher has fired for this bottle) is on.
2. `TON(After,1500,0)` *Timer On Delay* does: runs the timer `After` (Landing time after a push) toward 1500 ms (1.5 s).

Why it is built this way:

A timer: it times only while this rung is true.

RUNG 15

```
XIC(After.DN)[OTU(Busy),OTU(Done),RES(After),RES(Trip)];
```

HOW THIS RUNG WORKS

In words. When `After.DN` (the done bit of `After` : Landing time after a push) is on, the rung unlatches `Busy` (A bottle is between the hopper and the end of its cycle (Levels 1-3) / between the hopper and the filler (Levels 4-5)) (turns it off), unlatches `Done` (A pusher has fired for this bottle) (turns it off), resets `After` and resets `Trip` (Watchdog: the bottle ran the whole belt).

Left to right:

1. `XIC(After.DN)` *Examine If Closed* asks: true when `After.DN` (the done bit of `After` : Landing time after a push) is on.
2. A branch of 4 parallel paths: each path is tested on its own and does its own work.

   Path 1

   1. `OTU(Busy)` *Output Unlatch* does: unlatches `Busy` (A bottle is between the hopper and the end of its cycle (Levels 1-3) / between the hopper and the filler (Levels 4-5)) (turns it off).

   Path 2

   1. `OTU(Done)` *Output Unlatch* does: unlatches `Done` (A pusher has fired for this bottle) (turns it off).

   Path 3

   1. `RES(After)` *Reset* does: resets `After` .

   Path 4

   1. `RES(Trip)` *Reset* does: resets `Trip` (Watchdog: the bottle ran the whole belt).

Rung 13 latches `Done` when any pusher fires, because the pusher's eye — and so the pusher rung — goes false the moment the bottle is pushed off the belt. A timer fed straight from `Div2` would see half a second, never 1.5. **Rungs 14-15** give the bottle a second and a half to land, then free the line and reset the

watchdog: that bottle was dealt with, so the next one gets its own 16 s. The `RES(Trip)` matters because of where the watchdog sits. Its timer is above the release, and Busy is freed at the bottom of one scan and latched again by the release near the top of the next, so the timer never sees Busy off and never resets by itself; without the RES it would time across bottles and free the line early.

> **WATCH FOR: THE RES**
>
> The first draft of this program had `XIC(After.DN)OTU(Busy);` with no `RES`, and released bottles in pairs. Walk the scan: the timer finishes and rung 15 unlatches Busy. Next scan, rung 2 sees Busy off, releases a bottle and latches Busy again — so rung 14's timer sees its rung *true*, as it was last scan. It never saw the rung go false, so it never reset: it is still done, and rung 15 unlatches Busy again, and the next bottle is released right behind the first. A timer resets only when its rung is false for a whole scan. When the thing that ends a cycle also starts the next one, reset the timer yourself.

Run it

Run mode, press Start on the console. A bottle drops, runs past the gantry (watch the four `Is_` bits in Monitor Tags latch), reaches its pusher and goes into its bin; a moment later the next one drops. The header counts Correct and Wrong; the Simulator Log names every bottle and the bin it landed in. The reference run of this program: 10 of 10 bottles in the right bin, nothing lost, in 94 s against a par of 150 s: 1200 points and three stars.

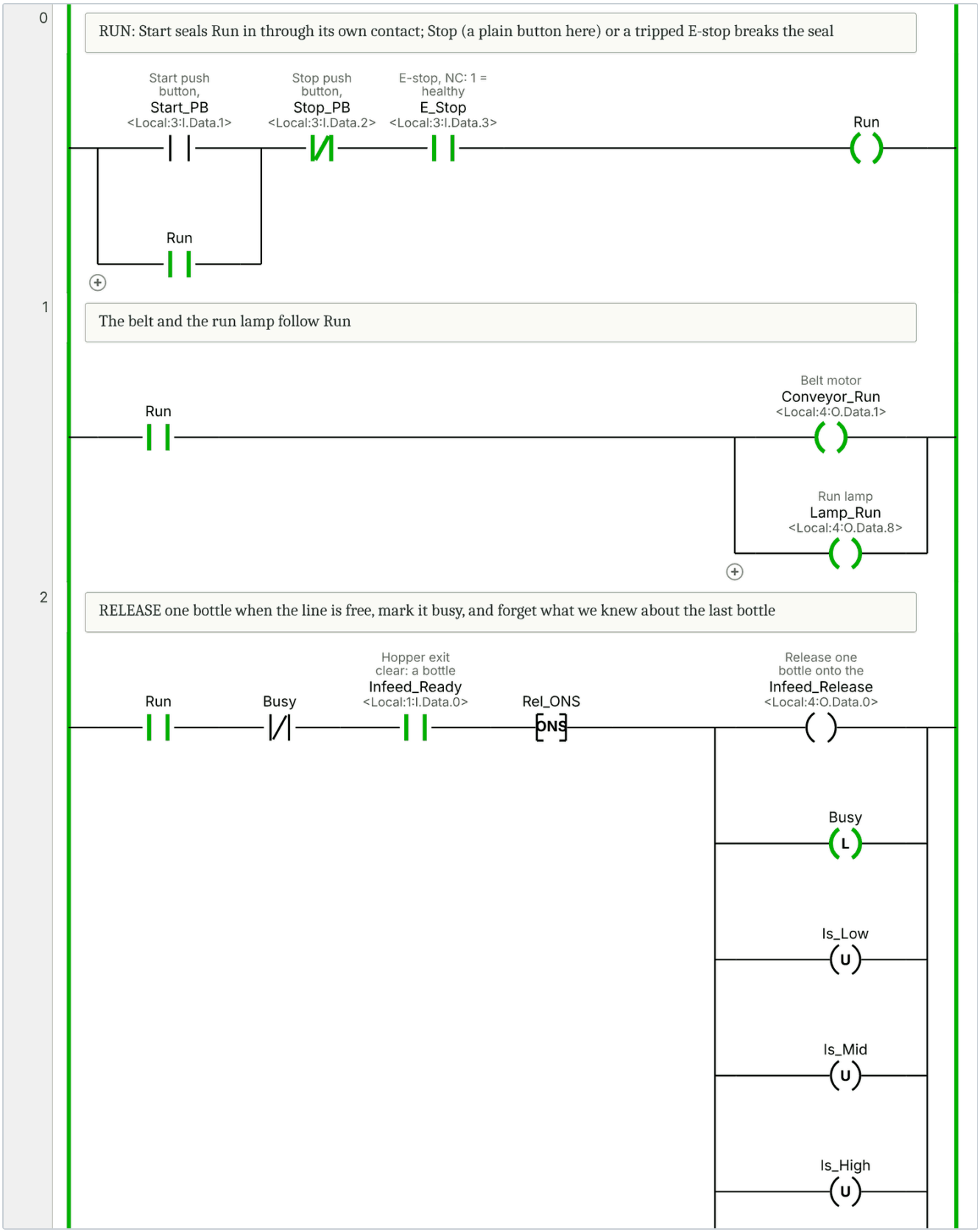


Figure 143. Level 1's program online mid-run: Busy latched, the bottle's height bits latched, the belt running.

TRY IT

Take `XIO(Is_Broken)` out of the small-bin rung and set the hopper's *Broken bottles* to 60 % — then move bin 1 to the far end of the line (swap the pushers' names). A broken small bottle now reaches bin 2 first. Put the test back. Then shorten `After` to 300 ms and see what the next bottle does to a bottle still sliding into its bin.

Predict it: the answers

1. The rung is false at `XIO(Stop_PB)`: `Stop_PB` is on, so `Run` is **off** (an OTE writes 0 when its rung is false).
2. Every condition on the way is met, so `Div1` is **on**.
3. The rung is false at `XIO(Is_Broken)`: `Is_Broken` is on, so `Div2` is **off** (an OTE writes 0 when its rung is false).
4. Every condition on the way is met, so `Div3` is **on**.
5. The rung is false at `XIC(PE_Div4)`: `PE_Div4` is off, so `Div4` is **off** (an OTE writes 0 when its rung is false).
6. Every condition on the way is met, so `Done` latches **on**.

34 Level 2 — Fill by label

> **IN THE APP**
>
> A level project opens with guided mode on this chapter: the panel lists the tags the program needs (*Make them*), shows every rung below under its fold with *Copy* and *Add*, checks that MainRoutine verifies, and while you run the level ticks three results — the level completes, every bottle in the right bin, under par.

The same bins and the same ten bottles, but every whole bottle must be filled before it is binned: a bin graded *must be filled* gives −40 for a bottle that arrives short or overfilled, and a broken bottle that is filled leaks and costs −40 too. Par 170 s. Everything from Level 1 stays; the new problem is **stopping the belt in the right place, filling to a measured level, and going again**.

Three things on the machine do the work: `PE_Fill`, an eye with a short range placed 20 px past the nozzle, so a bottle that stops the belt on it is standing under the nozzle; `Fill_Valve`, the dispenser's valve; and `Fill_Level`, the fill-level sensor's analog input, the bottle's fill as a percent of its *own* label — 60 units for a small bottle, 80 for a medium, 100 for a large all read 100 when full, so one number fills all three sizes.

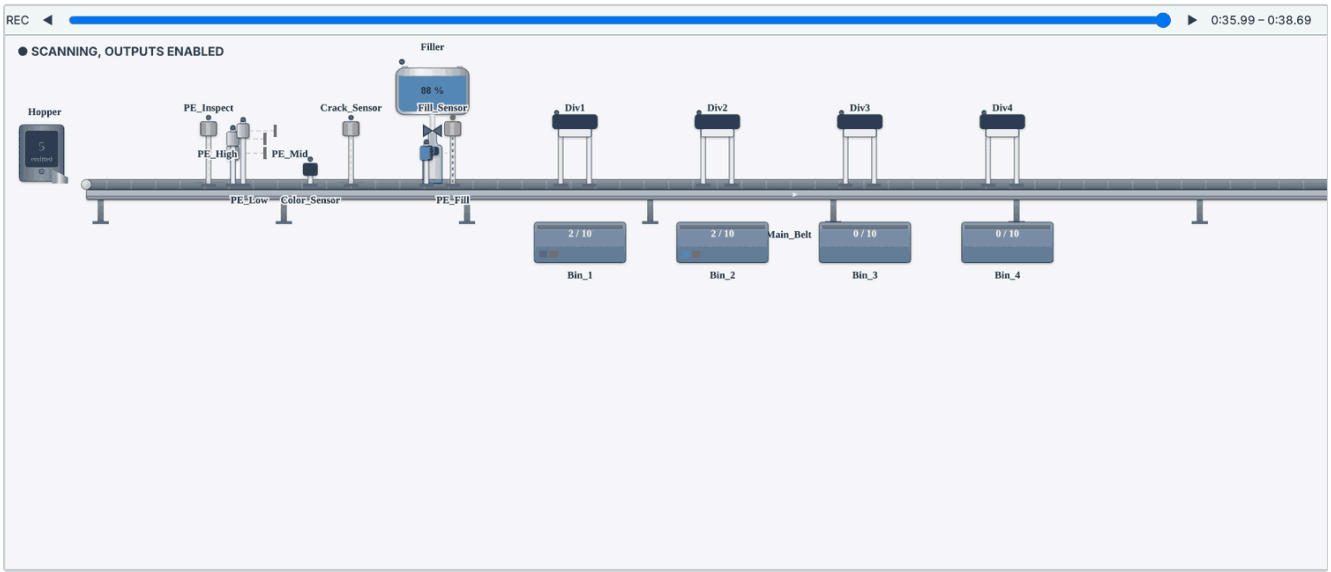


Figure 144. Level 2 in the Side view: a bottle stopped under the nozzle, filling.

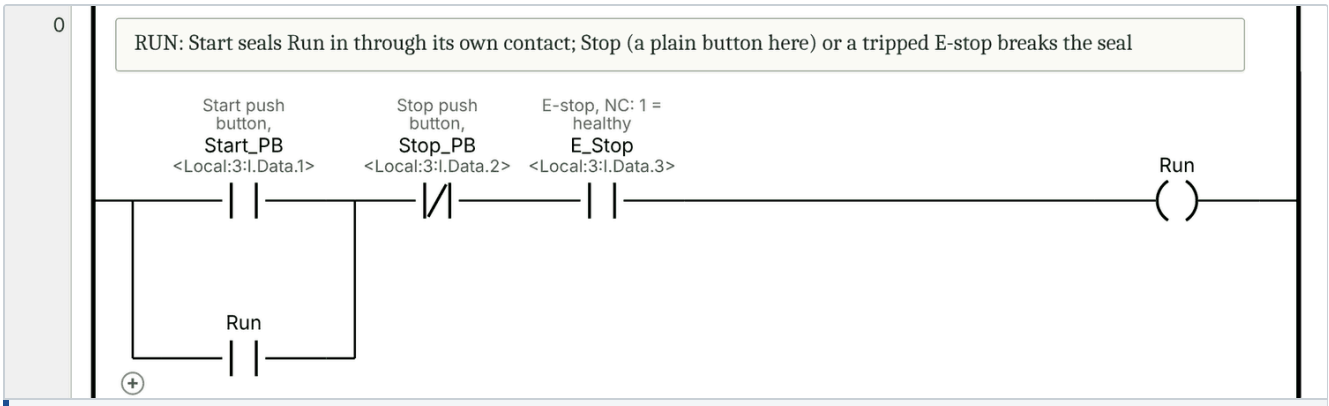
Tags this lesson needs

The level's alias tags are already made for you (chapter 32's table). Create these plain tags before you type the rungs; Verify reports any you miss.

Tag	Type	How to make it	Description
`Run`	BOOL	Edit Tags: name, type	Seal-in run bit
`Hold`	BOOL	Edit Tags: name, type	The filler holds the belt
`Busy`	BOOL	Edit Tags: name, type	A bottle is between the hopper and the end of its cycle (Levels 1-3) / between the hopper and the filler (Levels 4-5)
`Rel_ONS`	BOOL	Edit Tags: name, type	One-shot storage for the release
`Is_Low`	BOOL	Edit Tags: name, type	Latched: the low height beam was broken
`Is_Mid`	BOOL	Edit Tags: name, type	Latched: the middle beam was broken
`Is_High`	BOOL	Edit Tags: name, type	Latched: the high beam was broken
`Is_Broken`	BOOL	Edit Tags: name, type	Latched: the crack eye saw a broken bottle
`Filled`	BOOL	Edit Tags: name, type	This bottle has its charge
`Trip`	TIMER	Edit Tags: name, type	Watchdog: the bottle ran the whole belt
`DN`	BOOL	Edit Tags: name, type	
`Done`	BOOL	Edit Tags: name, type	A pusher has fired for this bottle
`After`	TIMER	Edit Tags: name, type	Landing time after a push

The run circuit, now with a hold

RUNG 0




```
[XIC(Start_PB),XIC(Run)]XIO(Stop_PB)XIC(E_Stop)OTE(Run);
```

PREDICT IT · 1 `Start_PB` is on, `Stop_PB` is on and `E_Stop` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When either (`Start_PB` (Start push button, momentary) is on) *or* (`Run` (Seal-in run bit) is on), `Stop_PB` (Stop push button, momentary) is off and `E_Stop` (E-stop, NC: 1 = healthy) is on, the rung turns `Run` on (and off again whenever the rung is false).

Left to right:

1. A branch of 2 parallel paths: power gets through if any one of them is true.
   - **Path 1**
     1. `XIC(Start_PB)` *Examine If Closed* asks: true when `Start_PB` (Start push button, momentary) is on.
   - **Path 2**
     1. `XIC(Run)` *Examine If Closed* asks: true when `Run` (Seal-in run bit) is on.
2. `XIO(Stop_PB)` *Examine If Open* asks: true when `Stop_PB` (Stop push button, momentary) is off.
3. `XIC(E_Stop)` *Examine If Closed* asks: true when `E_Stop` (E-stop, NC: 1 = healthy) is on.
4. `OTE(Run)` *Output Energize* does: turns `Run` on (and off again whenever the rung is false). Every scan it writes the rung state: true on, false off.

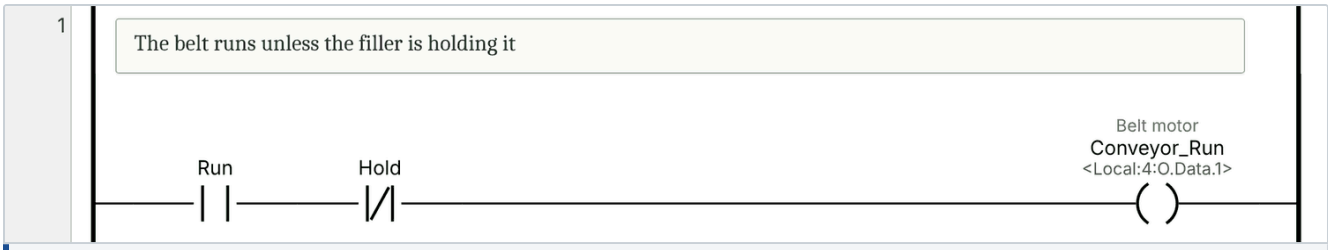
Why it is built this way:

Seal-in. The start condition turns `Run` on, and `Run` 's own contact in the parallel path keeps the rung true after the start goes away. Anything in series outside the branch (a stop, a fault) breaks it, and then it stays off until the start is made again.

> **Think of it like this:** a doorbell button and a hand. The start button is the doorbell, pressed for a moment. The seal-in contact is a hand that grabs the circuit as soon as the output comes on and keeps holding after you let go. The stop button, in series, is the only thing that can knock that hand away.

> **Watch for:** The stop has to be in series outside the branch. Put it inside the start path only and the seal-in path bypasses it: the stop no longer stops.

RUNG 1




```
XIC(Run)XIO(Hold)OTE(Conveyor_Run);
```

PREDICT IT · 2 `Run` is on and `Hold` is off. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

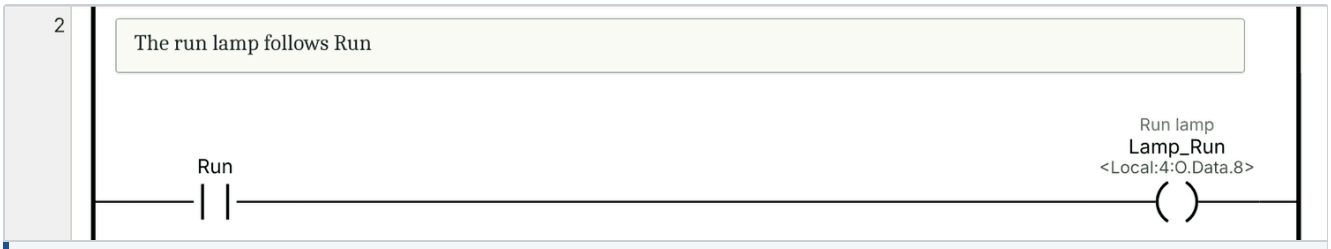
In words. When `Run` (Seal-in run bit) is on and `Hold` (The filler holds the belt) is off, the rung turns `Conveyor_Run` (Belt motor) on (and off again whenever the rung is false).

Left to right:

1. `XIC(Run)` *Examine If Closed* asks: true when `Run` (Seal-in run bit) is on.
2. `XIO(Hold)` *Examine If Open* asks: true when `Hold` (The filler holds the belt) is off.
3. `OTE(Conveyor_Run)` *Output Energize* does: turns `Conveyor_Run` (Belt motor) on (and off again whenever the rung is false).

> **Think of it like this:** a rung is a pipe with valves in it, read left to right. Every contact is a valve: XIC is open while its bit is 1, XIO while its bit is 0. Water reaches the output at the right only if every valve in a row is open. The output does not remember anything: on every scan it is simply wet or dry.

RUNG 2




```
XIC(Run)OTE(Lamp_Run);
```

HOW THIS RUNG WORKS

In words. When `Run` (Seal-in run bit) is on, the rung turns `Lamp_Run` (Run lamp) on (and off again whenever the rung is false).

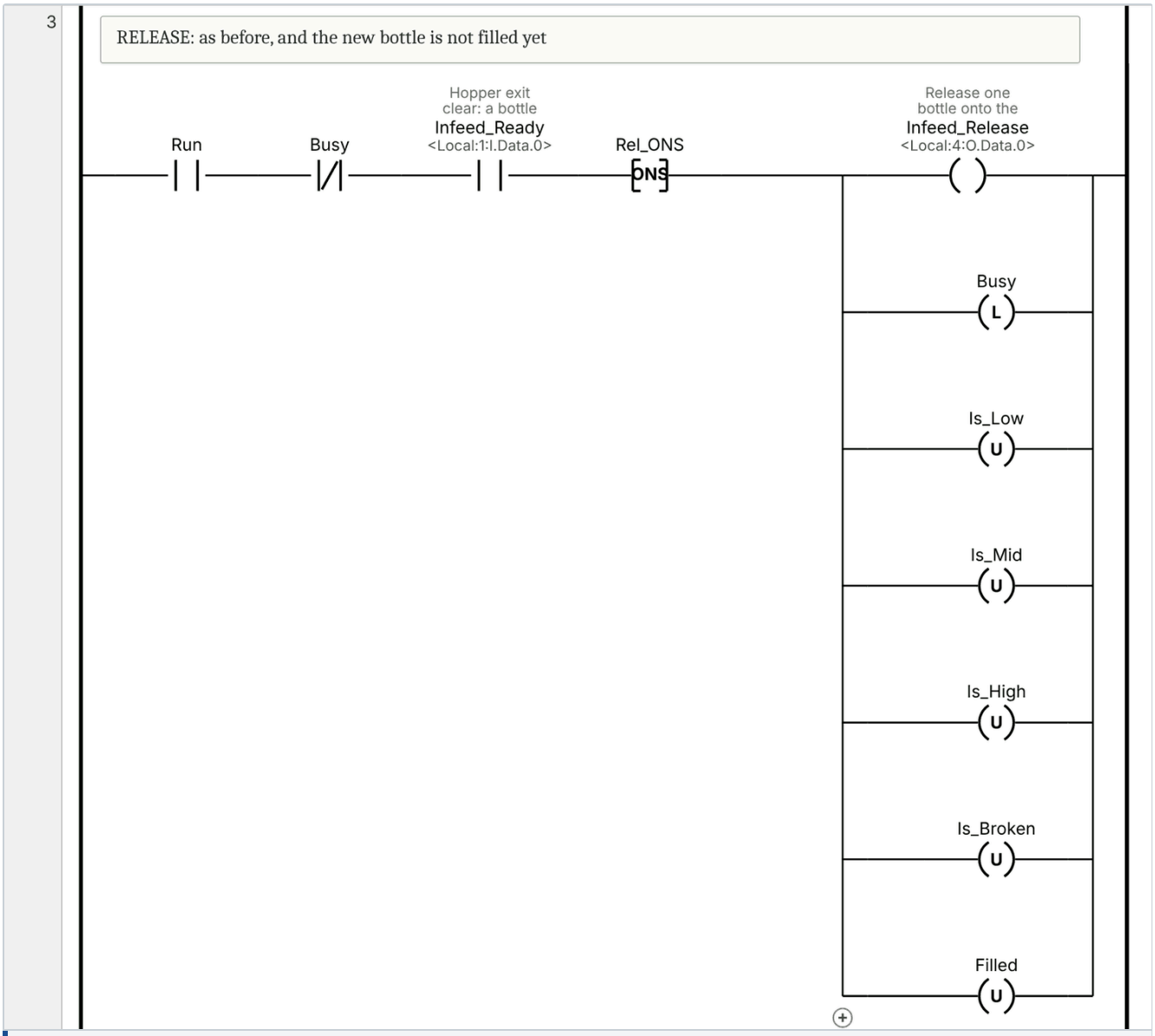
Left to right:

1. `XIC(Run)` *Examine If Closed* asks: true when `Run` (Seal-in run bit) is on.
2. `OTE(Lamp_Run)` *Output Energize* does: turns `Lamp_Run` (Run lamp) on (and off again whenever the rung is false).

Rung 1 changes: the belt runs unless `Hold` is on. The lamp gets a rung of its own, because it should stay lit while the belt is held for a fill — the line is still running.

Release, the watchdog and the gantry

RUNG 3




```
XIC(Run)XIO(Busy)XIC(Infeed_Ready)ONS(Rel_ONS)
[OTE(Infeed_Release),OTL(Busy),OTU(Is_Low),OTU(Is_Mid),OTU(Is_High),OTU(Is_Broken),OTU(Filled)];
```

HOW THIS RUNG WORKS

In words. When `Run` (Seal-in run bit) is on, `Busy` (A bottle is between the hopper and the end of its cycle (Levels 1-3) / between the hopper and the filler (Levels 4-5)) is off, `Infeed_Ready` (Hopper exit clear: a bottle can be released) is on and it is the first scan they are true (the one-shot, remembered in `Rel_ONS` (One-shot storage for the release)), the rung turns `Infeed_Release` (Release one bottle onto the belt (rising edge)) on (and off again whenever the rung is false), latches `Busy` on, unlatches `Is_Low` (Latched: the low height beam was broken) (turns it off), unlatches `Is_Mid` (Latched: the middle beam was broken) (turns it off), unlatches `Is_High` (Latched: the high beam was broken) (turns it off), unlatches `Is_Broken` (Latched: the crack eye saw a broken bottle) (turns it off) and unlatches `Filled` (This bottle has its charge) (turns it off).

Left to right:

1. `XIC(Run)` *Examine If Closed* asks: true when `Run` (Seal-in run bit) is on.
2. `XIO(Busy)` *Examine If Open* asks: true when `Busy` (A bottle is between the hopper and the end of its cycle (Levels 1-3) / between the hopper and the filler (Levels 4-5)) is off.
3. `XIC(Infeed_Ready)` *Examine If Closed* asks: true when `Infeed_Ready` (Hopper exit clear: a bottle can be released) is on.
4. `ONS(Rel_ONS)` *One Shot* asks: true when it is the first scan they are true (the one-shot, remembered in `Rel_ONS` (One-shot storage for the release)). It passes power for one scan only, the scan its input goes from false to true; its storage bit remembers last scan.
5. A branch of 7 parallel paths: each path is tested on its own and does its own work.

   Path 1

   1. `OTE(Infeed_Release)` *Output Energize* does: turns `Infeed_Release` (Release one bottle onto the belt (rising edge)) on (and off again whenever the rung is false).

   Path 2

   1. `OTL(Busy)` *Output Latch* does: latches `Busy` on. It only ever sets the bit; the bit stays on after the rung goes false.

   Path 3

   1. `OTU(Is_Low)` *Output Unlatch* does: unlatches `Is_Low` (Latched: the low height beam was broken) (turns it off). It only ever clears the bit.

   Path 4

   1. `OTU(Is_Mid)` *Output Unlatch* does: unlatches `Is_Mid` (Latched: the middle beam was broken) (turns it off).

   Path 5

   1. `OTU(Is_High)` *Output Unlatch* does: unlatches `Is_High` (Latched: the high beam was broken) (turns it off).

   Path 6

   1. `OTU(Is_Broken)` *Output Unlatch* does: unlatches `Is_Broken` (Latched: the crack eye saw a broken bottle) (turns it off).

   Path 7

   1. `OTU(Filled)` *Output Unlatch* does: unlatches `Filled` (This bottle has its charge) (turns it off).

Why it is built this way:

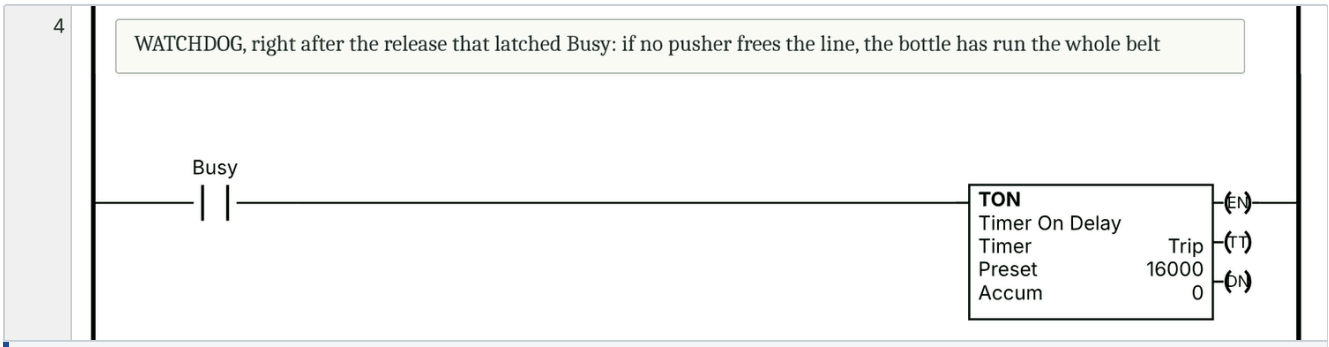
One-shot. Everything after the ONS happens once per event, not on every scan the condition holds; `Rel_ONS` is only its memory of the last scan.

A latch. `Busy` stays on after this rung goes false; an `OTU` elsewhere in the program clears it.

> **Think of it like this:** a doorbell that rings once however long you lean on the button. The ONS looks at what the button was doing last scan: only the change from up to down rings it.

> **Watch for:** Each ONS needs its own storage bit. Two ONS sharing one bit make one of them miss its edges.

RUNG 4




```
XIC(Busy)TON(Trip,16000,0);
```

HOW THIS RUNG WORKS

In words. When `Busy` (A bottle is between the hopper and the end of its cycle (Levels 1-3) / between the hopper and the filler (Levels 4-5)) is on, the rung runs the timer `Trip` (Watchdog: the bottle ran the whole belt) toward 16000 ms (16 s).

Left to right:

1. `XIC(Busy)` *Examine If Closed* asks: true when `Busy` (A bottle is between the hopper and the end of its cycle (Levels 1-3) / between the hopper and the filler (Levels 4-5)) is on.
2. `TON(Trip,16000,0)` *Timer On Delay* does: runs the timer `Trip` (Watchdog: the bottle ran the whole belt) toward 16000 ms (16 s). The accumulator counts while the rung is true; .DN comes on at the preset; the moment the rung goes false the timer resets.

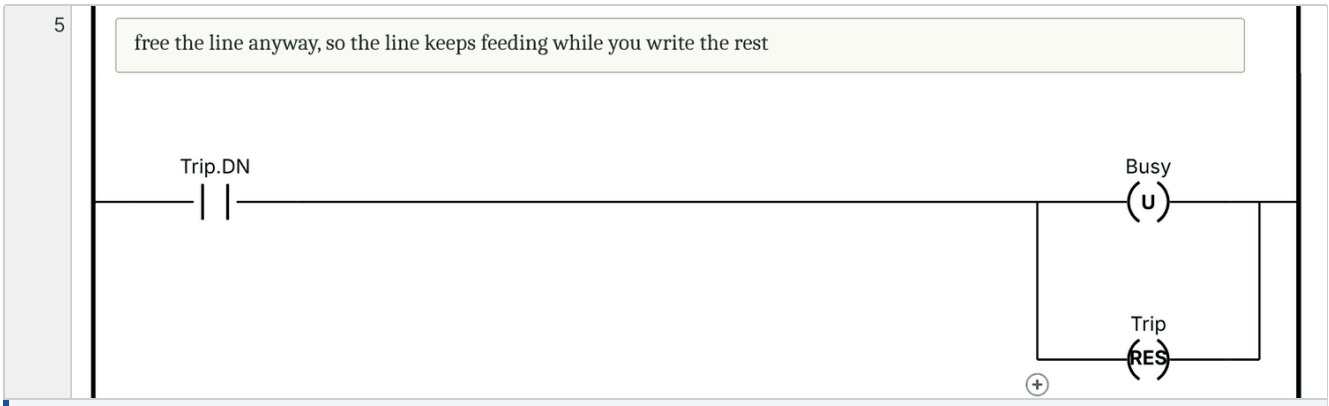
Why it is built this way:

A timer. `Trip` only times while this rung is true; other rungs use its `.DN` (done) or `.TT` (timing) bits. Being a TON, it starts again from zero every time the rung goes false.

> **Think of it like this:** a kitchen timer that only runs while you hold your finger on it. Take your finger off and it snaps back to zero. Hold it long enough and the done light comes on, and stays on as long as your finger is still there.

> **Watch for:** If anything in series flickers false for one scan, a TON starts over from zero. Use RTO when the time must survive an interruption.

RUNG 5




```
XIC(Trip.DN)[OTU(Busy),RES(Trip)];
```

HOW THIS RUNG WORKS

In words. When `Trip.DN` (the done bit of `Trip` : Watchdog: the bottle ran the whole belt) is on, the rung unlatches `Busy` (A bottle is between the hopper and the end of its cycle (Levels 1-3) / between the hopper and the filler (Levels 4-5)) (turns it off) and resets `Trip` .

Left to right:

1. `XIC(Trip.DN)` *Examine If Closed* asks: true when `Trip.DN` (the done bit of `Trip` : Watchdog: the bottle ran the whole belt) is on.
2. A branch of 2 parallel paths: each path is tested on its own and does its own work.

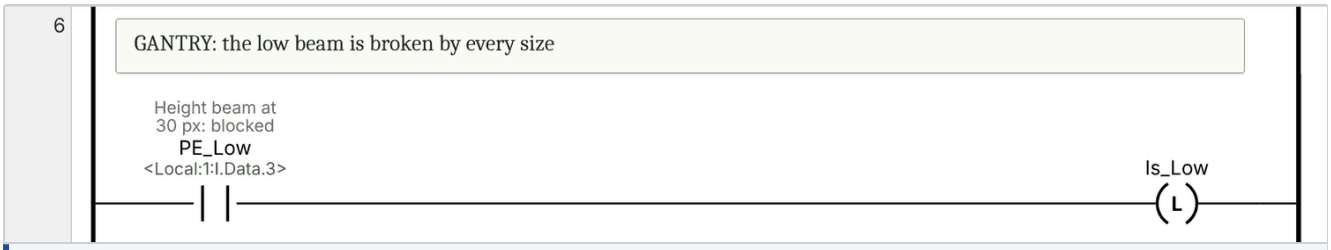
   Path 1

   1. `OTU(Busy)` *Output Unlatch* does: unlatches `Busy` (A bottle is between the hopper and the end of its cycle (Levels 1-3) / between the hopper and the filler (Levels 4-5)) (turns it off).

   Path 2

   1. `RES(Trip)` *Reset* does: resets `Trip` . It zeroes a timer or counter (accumulator and status bits), or a control (its position and status bits).

RUNG 6




```
XIC(PE_Low)OTL(Is_Low);
```

HOW THIS RUNG WORKS

In words. When `PE_Low` (Height beam at 30 px: blocked by every size) is on, the rung latches `Is_Low` (Latched: the low height beam was broken) on.

Left to right:

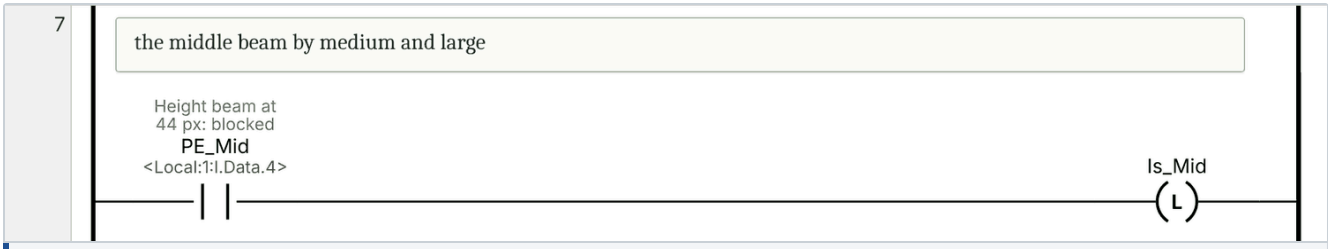
1. `XIC(PE_Low)` *Examine If Closed* asks: true when `PE_Low` (Height beam at 30 px: blocked by every size) is on.
2. `OTL(Is_Low)` *Output Latch* does: latches `Is_Low` (Latched: the low height beam was broken) on.

Why it is built this way:

A latch: it stays on until an OTU clears it.

> **Watch for:** A latched bit survives the condition that set it: every way in needs a way out, or it stays on for good.

RUNG 7




```
XIC(PE_Mid)OTL(Is_Mid);
```

HOW THIS RUNG WORKS

In words. When `PE_Mid` (Height beam at 44 px: blocked by M and L) is on, the rung latches `Is_Mid` (Latched: the middle beam was broken) on.

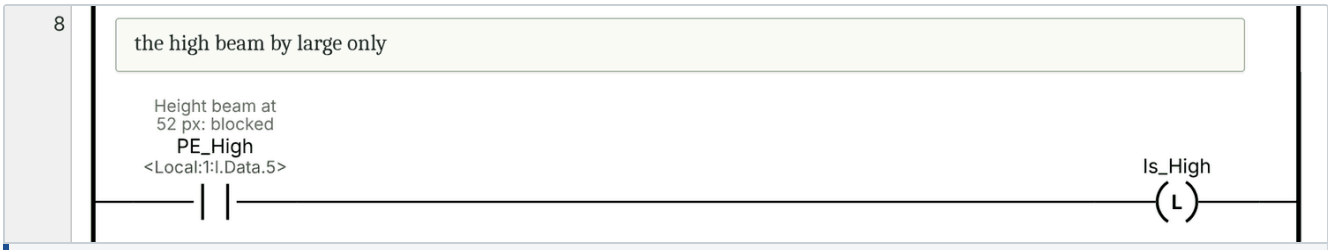
Left to right:

1. `XIC(PE_Mid)` *Examine If Closed* asks: true when `PE_Mid` (Height beam at 44 px: blocked by M and L) is on.
2. `OTL(Is_Mid)` *Output Latch* does: latches `Is_Mid` (Latched: the middle beam was broken) on.

Why it is built this way:

A latch: it stays on until an OTU clears it.

RUNG 8




```
XIC(PE_High)OTL(Is_High);
```

HOW THIS RUNG WORKS

In words. When `PE_High` (Height beam at 52 px: blocked by L only) is on, the rung latches `Is_High` (Latched: the high beam was broken) on.

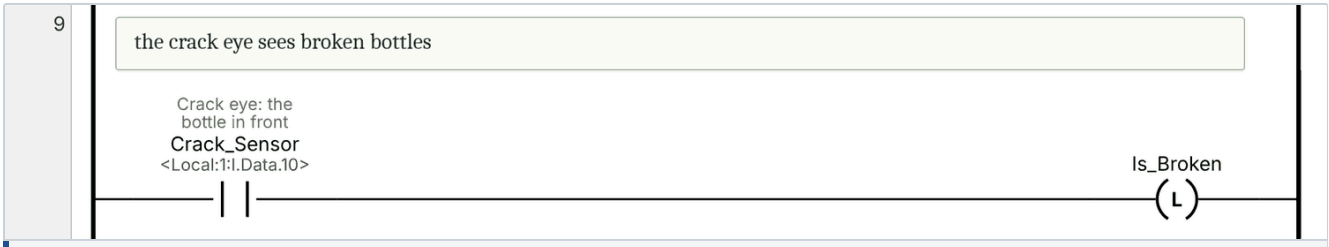
Left to right:

1. `XIC(PE_High)` *Examine If Closed* asks: true when `PE_High` (Height beam at 52 px: blocked by L only) is on.
2. `OTL(Is_High)` *Output Latch* does: latches `Is_High` (Latched: the high beam was broken) on.

Why it is built this way:

A latch: it stays on until an OTU clears it.

RUNG 9




```
XIC(Crack_Sensor)OTL(Is_Broken);
```

HOW THIS RUNG WORKS

In words. When `Crack_Sensor` (Crack eye: the bottle in front of it is broken) is on, the rung latches `Is_Broken` (Latched: the crack eye saw a broken bottle) on.

Left to right:

1. `XIC(Crack_Sensor)` *Examine If Closed* asks: true when `Crack_Sensor` (Crack eye: the bottle in front of it is broken) is on.
2. `OTL(Is_Broken)` *Output Latch* does: latches `Is_Broken` (Latched: the crack eye saw a broken bottle) on.

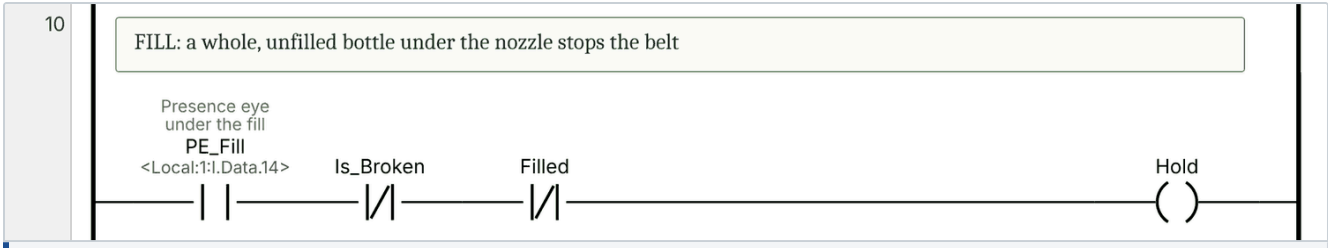
Why it is built this way:

A latch: it stays on until an OTU clears it.

As in Level 1, with one addition to the release rung: `OTU(Filled)`, so the new bottle starts unfilled.

The fill

RUNG 10




```
XIC(PE_Fill)XIO(Is_Broken)XIO(Filled)OTE(Hold);
```

PREDICT IT · 3 `PE_Fill` is on, `Is_Broken` is off and `Filled` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

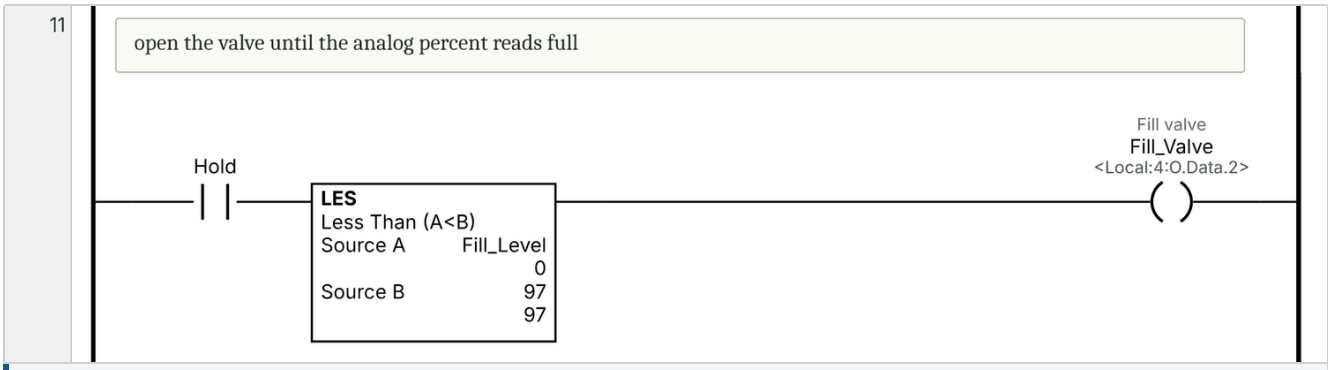
HOW THIS RUNG WORKS

In words. When `PE_Fill` (Presence eye under the fill nozzle) is on, `Is_Broken` (Latched: the crack eye saw a broken bottle) is off and `Filled` (This bottle has its charge) is off, the rung turns `Hold` (The filler holds the belt) on (and off again whenever the rung is false).

Left to right:

1. `XIC(PE_Fill)` *Examine If Closed* asks: true when `PE_Fill` (Presence eye under the fill nozzle) is on.
2. `XIO(Is_Broken)` *Examine If Open* asks: true when `Is_Broken` (Latched: the crack eye saw a broken bottle) is off.
3. `XIO(Filled)` *Examine If Open* asks: true when `Filled` (This bottle has its charge) is off.
4. `OTE(Hold)` *Output Energize* does: turns `Hold` (The filler holds the belt) on (and off again whenever the rung is false).

RUNG 11




```
XIC(Hold)LES(Fill_Level,97)OTE(Fill_Valve);
```

PREDICT IT · 4 `Hold` is on and `Fill_Level` is 96. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

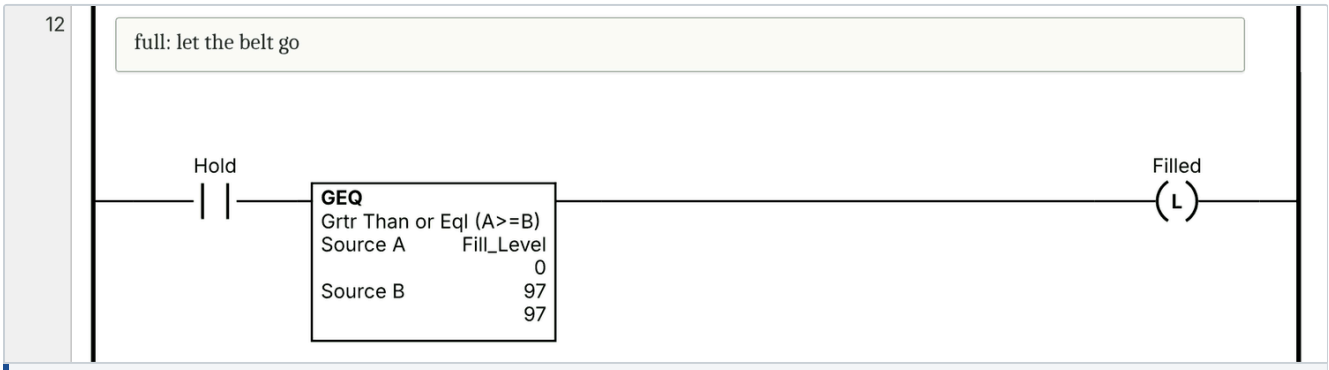
HOW THIS RUNG WORKS

In words. When `Hold` (The filler holds the belt) is on and `Fill_Level` (Fill of the bottle under the nozzle, 0-100 % of its label) is less than **97**, the rung turns `Fill_Valve` (Fill valve) on (and off again whenever the rung is false).

Left to right:

1. `XIC(Hold)` *Examine If Closed* asks: true when `Hold` (The filler holds the belt) is on.
2. `LES(Fill_Level,97)` *Less Than* asks: true when `Fill_Level` (Fill of the bottle under the nozzle, 0-100 % of its label) is less than **97**.
3. `OTE(Fill_Valve)` *Output Energize* does: turns `Fill_Valve` (Fill valve) on (and off again whenever the rung is false).

RUNG 12




```
XIC(Hold)GEQ(Fill_Level,97)OTL(Filled);
```

PREDICT IT · 5 `Hold` is off and `Fill_Level` is 97. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Hold` (The filler holds the belt) is on and `Fill_Level` (Fill of the bottle under the nozzle, 0-100 % of its label) is at least **97**, the rung latches `Filled` (This bottle has its charge) on.

Left to right:

1. `XIC(Hold)` *Examine If Closed* asks: true when `Hold` (The filler holds the belt) is on.
2. `GEQ(Fill_Level,97)` *Greater Than or Equal* asks: true when `Fill_Level` (Fill of the bottle under the nozzle, 0-100 % of its label) is at least **97**.
3. `OTL(Filled)` *Output Latch* does: latches `Filled` (This bottle has its charge) on.

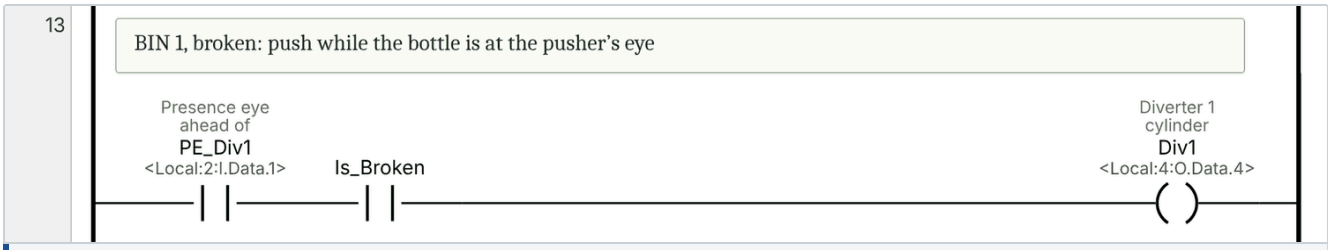
Why it is built this way:

A latch: it stays on until an OTU clears it.

The fill comes next because that is where it is on the belt: a bottle reaches the nozzle before the pushers. **Rung 10** holds the belt when `PE_Fill` sees a bottle that is whole and not yet filled. It is an OTE, so the hold lasts exactly as long as those three things are true. **Rung 11** opens the valve while held and the percent is under 97 — an LES compare on an analog input, chapter 8. **Rung 12** latches `Filled` once it reaches 97, which drops the hold (rung 10's `XIO(Filled)`), which restarts the belt. A broken bottle is never held: `XIO(Is_Broken)` lets it ride through to bin 1 dry.

Pushers and freeing the line

RUNG 13




```
XIC(PE_Div1)XIC(Is_Broken)OTE(Div1);
```

PREDICT IT · 6 `PE_Div1` is on and `Is_Broken` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

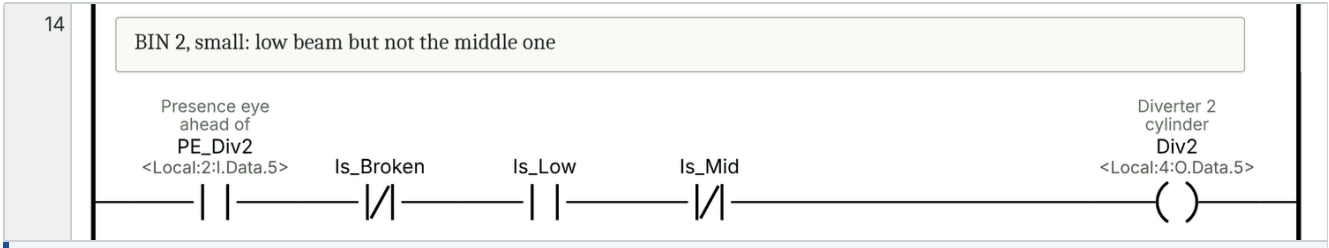
HOW THIS RUNG WORKS

In words. When `PE_Div1` (Presence eye ahead of diverter 1) is on and `Is_Broken` (Latched: the crack eye saw a broken bottle) is on, the rung turns `Div1` (Diverter 1 cylinder) on (and off again whenever the rung is false).

Left to right:

1. `XIC(PE_Div1)` *Examine If Closed* asks: true when `PE_Div1` (Presence eye ahead of diverter 1) is on.
2. `XIC(Is_Broken)` *Examine If Closed* asks: true when `Is_Broken` (Latched: the crack eye saw a broken bottle) is on.
3. `OTE(Div1)` *Output Energize* does: turns `Div1` (Diverter 1 cylinder) on (and off again whenever the rung is false).

RUNG 14




```
XIC(PE_Div2)XIO(Is_Broken)XIC(Is_Low)XIO(Is_Mid)OTE(Div2);
```

PREDICT IT · 7 `PE_Div2` is on, `Is_Broken` is off, `Is_Low` is on and `Is_Mid` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

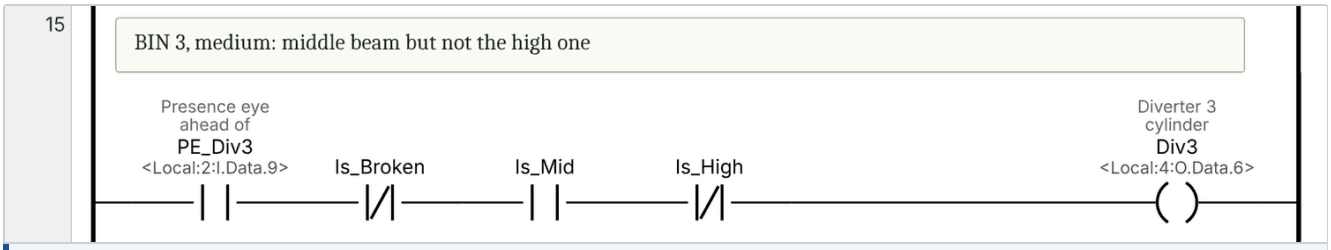
HOW THIS RUNG WORKS

In words. When `PE_Div2` (Presence eye ahead of diverter 2) is on, `Is_Broken` (Latched: the crack eye saw a broken bottle) is off, `Is_Low` (Latched: the low height beam was broken) is on and `Is_Mid` (Latched: the middle beam was broken) is off, the rung turns `Div2` (Diverter 2 cylinder) on (and off again whenever the rung is false).

Left to right:

1. `XIC(PE_Div2)` *Examine If Closed* asks: true when `PE_Div2` (Presence eye ahead of diverter 2) is on.
2. `XIO(Is_Broken)` *Examine If Open* asks: true when `Is_Broken` (Latched: the crack eye saw a broken bottle) is off.
3. `XIC(Is_Low)` *Examine If Closed* asks: true when `Is_Low` (Latched: the low height beam was broken) is on.
4. `XIO(Is_Mid)` *Examine If Open* asks: true when `Is_Mid` (Latched: the middle beam was broken) is off.
5. `OTE(Div2)` *Output Energize* does: turns `Div2` (Diverter 2 cylinder) on (and off again whenever the rung is false).

RUNG 15




```
XIC(PE_Div3)XIO(Is_Broken)XIC(Is_Mid)XIO(Is_High)OTE(Div3);
```

PREDICT IT · 8 `PE_Div3` is on, `Is_Broken` is off, `Is_Mid` is on and `Is_High` is off. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

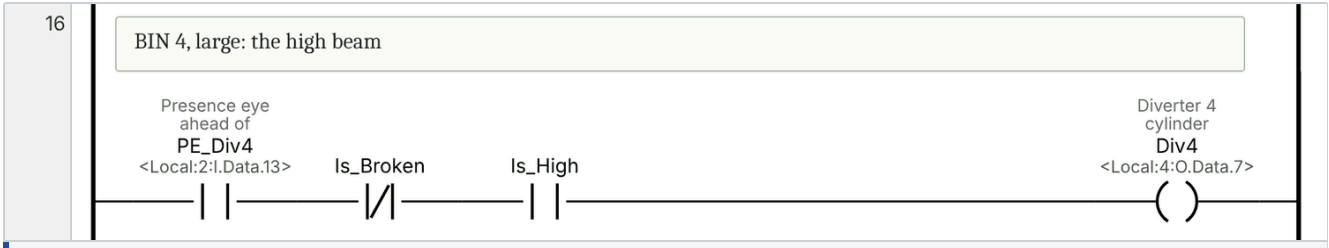
HOW THIS RUNG WORKS

In words. When `PE_Div3` (Presence eye ahead of diverter 3) is on, `Is_Broken` (Latched: the crack eye saw a broken bottle) is off, `Is_Mid` (Latched: the middle beam was broken) is on and `Is_High` (Latched: the high beam was broken) is off, the rung turns `Div3` (Diverter 3 cylinder) on (and off again whenever the rung is false).

Left to right:

1. `XIC(PE_Div3)` *Examine If Closed* asks: true when `PE_Div3` (Presence eye ahead of diverter 3) is on.
2. `XIO(Is_Broken)` *Examine If Open* asks: true when `Is_Broken` (Latched: the crack eye saw a broken bottle) is off.
3. `XIC(Is_Mid)` *Examine If Closed* asks: true when `Is_Mid` (Latched: the middle beam was broken) is on.
4. `XIO(Is_High)` *Examine If Open* asks: true when `Is_High` (Latched: the high beam was broken) is off.
5. `OTE(Div3)` *Output Energize* does: turns `Div3` (Diverter 3 cylinder) on (and off again whenever the rung is false).

RUNG 16




```
XIC(PE_Div4)XIO(Is_Broken)XIC(Is_High)OTE(Div4);
```

PREDICT IT · 9 `PE_Div4` is on, `Is_Broken` is off and `Is_High` is off. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

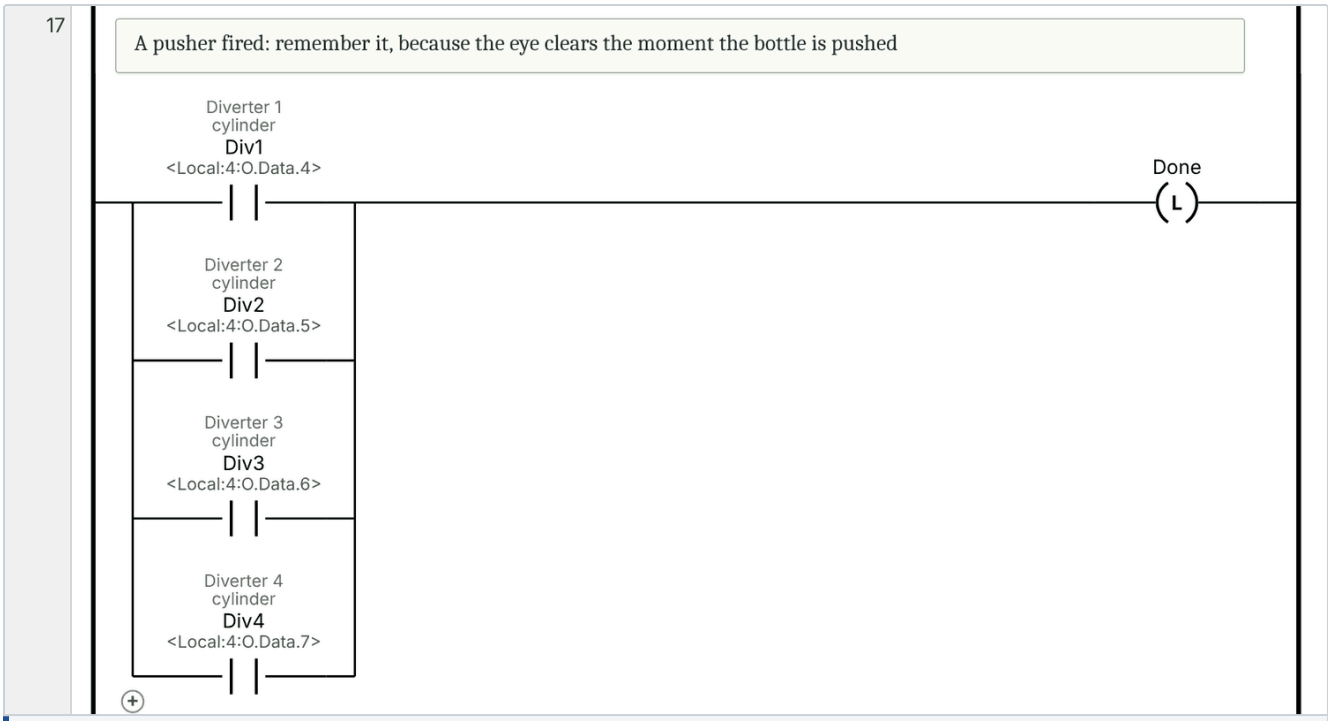
HOW THIS RUNG WORKS

In words. When `PE_Div4` (Presence eye ahead of diverter 4) is on, `Is_Broken` (Latched: the crack eye saw a broken bottle) is off and `Is_High` (Latched: the high beam was broken) is on, the rung turns `Div4` (Diverter 4 cylinder) on (and off again whenever the rung is false).

Left to right:

1. `XIC(PE_Div4)` *Examine If Closed* asks: true when `PE_Div4` (Presence eye ahead of diverter 4) is on.
2. `XIO(Is_Broken)` *Examine If Open* asks: true when `Is_Broken` (Latched: the crack eye saw a broken bottle) is off.
3. `XIC(Is_High)` *Examine If Closed* asks: true when `Is_High` (Latched: the high beam was broken) is on.
4. `OTE(Div4)` *Output Energize* does: turns `Div4` (Diverter 4 cylinder) on (and off again whenever the rung is false).

RUNG 17




```
[XIC(Div1),XIC(Div2),XIC(Div3),XIC(Div4)]OTL(Done);
```

PREDICT IT · 10 `Div1` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When either (`Div1` (Diverter 1 cylinder) is on) *or* (`Div2` (Diverter 2 cylinder) is on) *or* (`Div3` (Diverter 3 cylinder) is on) *or* (`Div4` (Diverter 4 cylinder) is on), the rung latches `Done` (A pusher has fired for this bottle) on.

Left to right:

1. A branch of 4 parallel paths: power gets through if any one of them is true.
   - **Path 1**
     1. `XIC(Div1)` *Examine If Closed* asks: true when `Div1` (Diverter 1 cylinder) is on.
   - **Path 2**
     1. `XIC(Div2)` *Examine If Closed* asks: true when `Div2` (Diverter 2 cylinder) is on.
   - **Path 3**
     1. `XIC(Div3)` *Examine If Closed* asks: true when `Div3` (Diverter 3 cylinder) is on.
   - **Path 4**
     1. `XIC(Div4)` *Examine If Closed* asks: true when `Div4` (Diverter 4 cylinder) is on.
2. `OTL(Done)` *Output Latch* does: latches `Done` (A pusher has fired for this bottle) on.

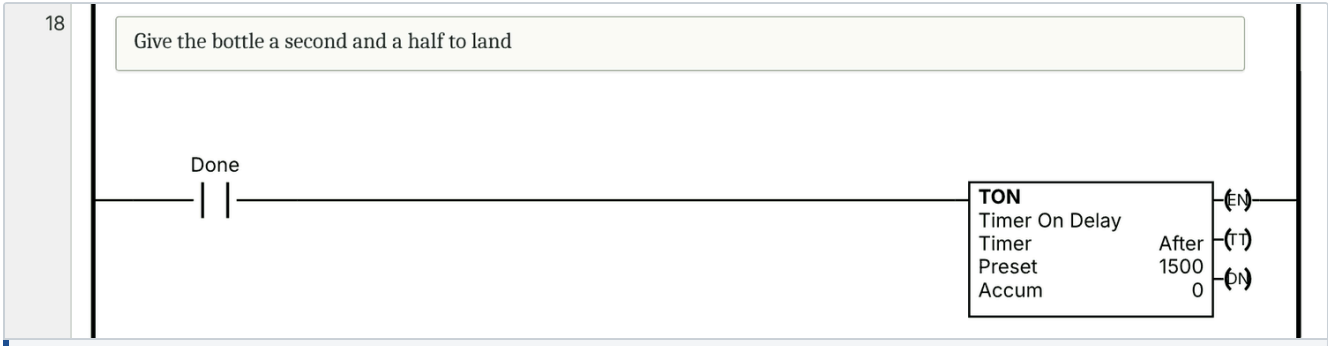
Why it is built this way:

A latch: it stays on until an OTU clears it.

> **Think of it like this:** parallel paths are two roads to the same town. If either road is open you get there; the rung only

> cares that power reaches the far end, not which way it went.

RUNG 18




```
XIC(Done)TON(After,1500,0);
```

HOW THIS RUNG WORKS

In words. When `Done` (A pusher has fired for this bottle) is on, the rung runs the timer `After` (Landing time after a push) toward 1500 ms (1.5 s).

Left to right:

1. `XIC(Done)` *Examine If Closed* asks: true when `Done` (A pusher has fired for this bottle) is on.
2. `TON(After,1500,0)` *Timer On Delay* does: runs the timer `After` (Landing time after a push) toward 1500 ms (1.5 s).

Why it is built this way:

A timer: it times only while this rung is true.

RUNG 19

```
XIC(After.DN)[OTU(Busy),OTU(Done),RES(After),RES(Trip)];
```

HOW THIS RUNG WORKS

In words. When `After.DN` (the done bit of `After` : Landing time after a push) is on, the rung unlatches `Busy` (A bottle is between the hopper and the end of its cycle (Levels 1-3) / between the hopper and the filler (Levels 4-5)) (turns it off), unlatches `Done` (A pusher has fired for this bottle) (turns it off), resets `After` and resets `Trip` (Watchdog: the bottle ran the whole belt).

Left to right:

1. `XIC(After.DN)` *Examine If Closed* asks: true when `After.DN` (the done bit of `After` : Landing time after a push) is on.
2. A branch of 4 parallel paths: each path is tested on its own and does its own work.

   Path 1

   1. `OTU(Busy)` *Output Unlatch* does: unlatches `Busy` (A bottle is between the hopper and the end of its cycle (Levels 1-3) / between the hopper and the filler (Levels 4-5)) (turns it off).

   Path 2

   1. `OTU(Done)` *Output Unlatch* does: unlatches `Done` (A pusher has fired for this bottle) (turns it off).

   Path 3

   1. `RES(After)` *Reset* does: resets `After` .

   Path 4

   1. `RES(Trip)` *Reset* does: resets `Trip` (Watchdog: the bottle ran the whole belt).

As in Level 1.

Why 97 and not 100? The valve closes on the scan *after* the compare goes true, and liquid is still in the air. Stop a fraction short and the last of it brings the bottle to its label; aim at 100 and you overfill. Every real filler has this *in-flight* allowance.

> **WATCH FOR: WHERE THE EYE IS**
>
> The first draft of this program used an eye 8 px before the nozzle with a long range. The belt stopped with the bottle short of the nozzle, the valve opened, and the dispenser's tank emptied onto the floor while `Fill_Level` sat at 0 — the sensor reads the bottle under *it*, and there was none. The level's `PE_Fill` sits past the nozzle with a 6 px range for that reason. Before you trust any stop position, watch it once in the Side view.

Run it

Each whole bottle stops under the nozzle, the liquid rises in the Side view (watch `Fill_Level` climb in Monitor Tags), the belt goes again, and the bottle is sorted as in Level 1. Broken bottles pass the filler without stopping. The reference run: 10 of 10 bottles in the right bin, nothing lost, in 104 s against a par of 170 s: 1200 points and three stars.

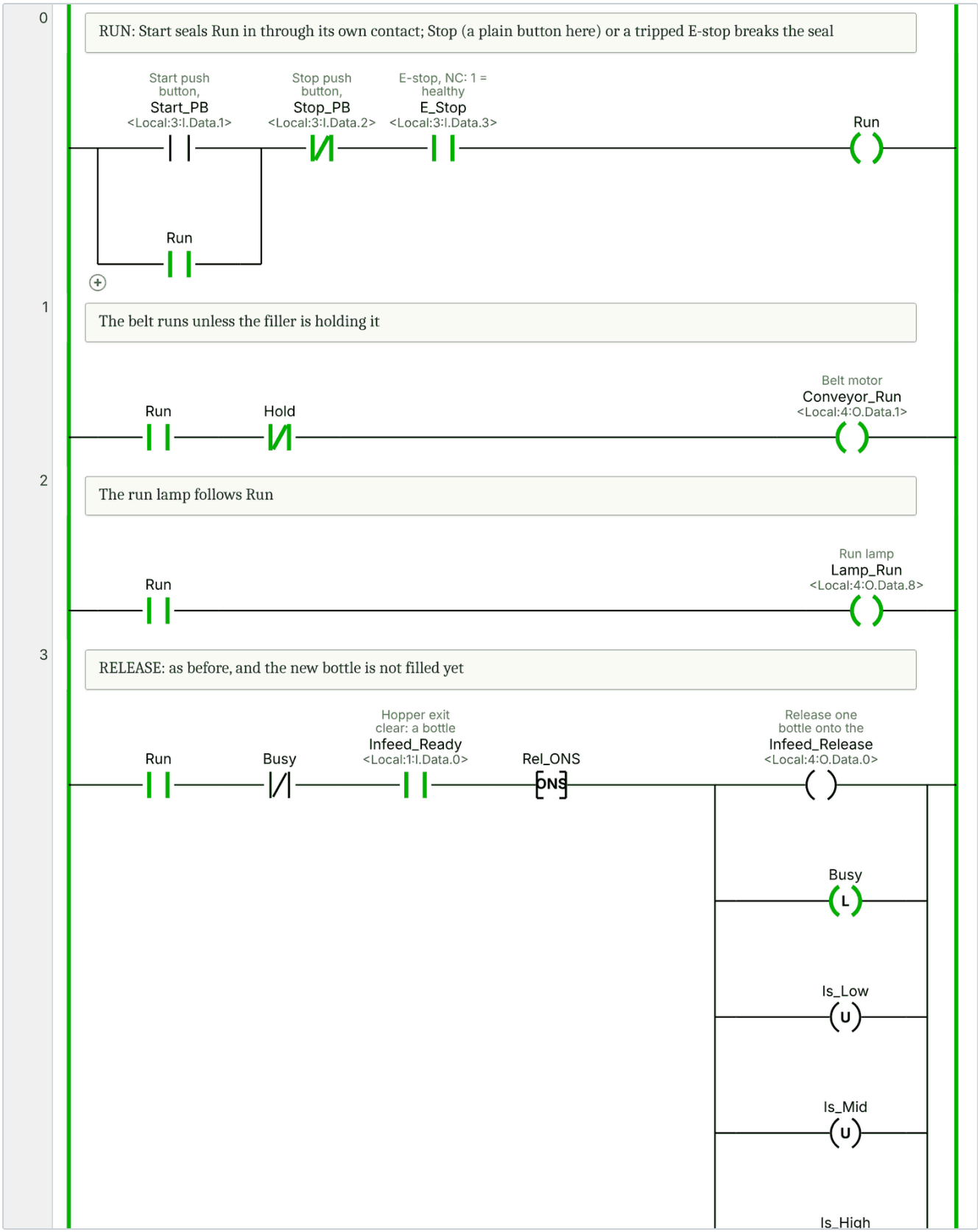


Figure 145. Level 2 online: the hold, the valve and the fill latch are the new rungs, between the gantry and the pushers.

> **TRY IT**
>
> Change 97 to 100 and run: the score card shows −40s for overfilled bottles. Then try 90 and see the short fills. Then take `XIO(Is_Broken)` out of the hold rung and watch a broken bottle filled and leaking into bin 1.

Predict it: the answers

1. The rung is false at `XIO(Stop_PB)`: `Stop_PB` is on, so `Run` is **off** (an OTE writes 0 when its rung is false).
2. Every condition on the way is met, so `Conveyor_Run` is **on**.
3. The rung is false at `XIO(Filled)`: `Filled` is on, so `Hold` is **off** (an OTE writes 0 when its rung is false).
4. Every condition on the way is met, so `Fill_Valve` is **on**.
5. The rung is false at `XIC(Hold)`: `Hold` is off, so the OTL does nothing: `Filled` keeps whatever it had.
6. Every condition on the way is met, so `Div1` is **on**.
7. The rung is false at `XIO(Is_Mid)`: `Is_Mid` is on, so `Div2` is **off** (an OTE writes 0 when its rung is false).
8. Every condition on the way is met, so `Div3` is **on**.
9. The rung is false at `XIC(Is_High)`: `Is_High` is off, so `Div4` is **off** (an OTE writes 0 when its rung is false).
10. Every condition on the way is met, so `Done` latches **on**.

35 Level 3 — Colour sort

> **IN THE APP**
>
> A level project opens with guided mode on this chapter: the panel lists the tags the program needs (*Make them*), shows every rung below under its fold with *Copy* and *Add*, checks that MainRoutine verifies, and while you run the level ticks three results — the level completes, every bottle in the right bin, under par.

Twelve bottles, bins by **colour** now — broken, red, blue, green — and every whole bottle filled. Par 190 s. The machine is the same; only the question the gantry answers changes, and this lesson is about how little of a good program has to change when the question does.

The colour sensor has three banks, one per colour, each on its own input: `Color_Red`, `Color_Blue`, `Color_Green`. Like the height beams, a bank is on only while the bottle is in front of the sensor, so the pattern is the same: latch it, test the latch at the pusher.

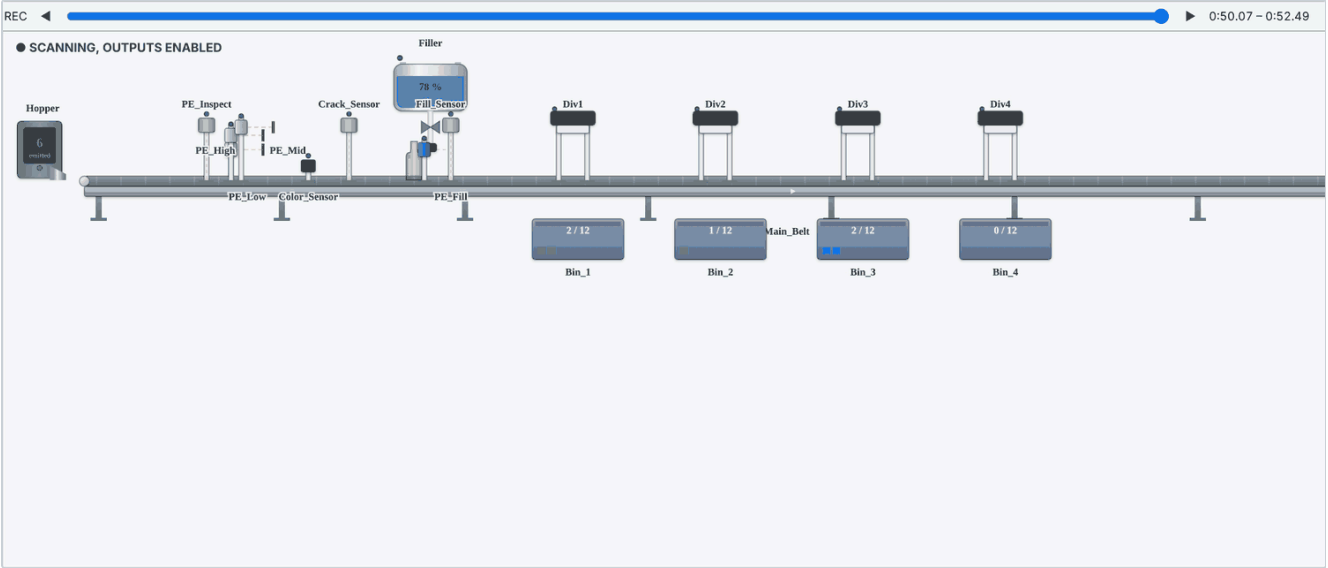


Figure 146. Level 3 running: the colour sensor on the gantry, bins graded by colour.

Tags this lesson needs

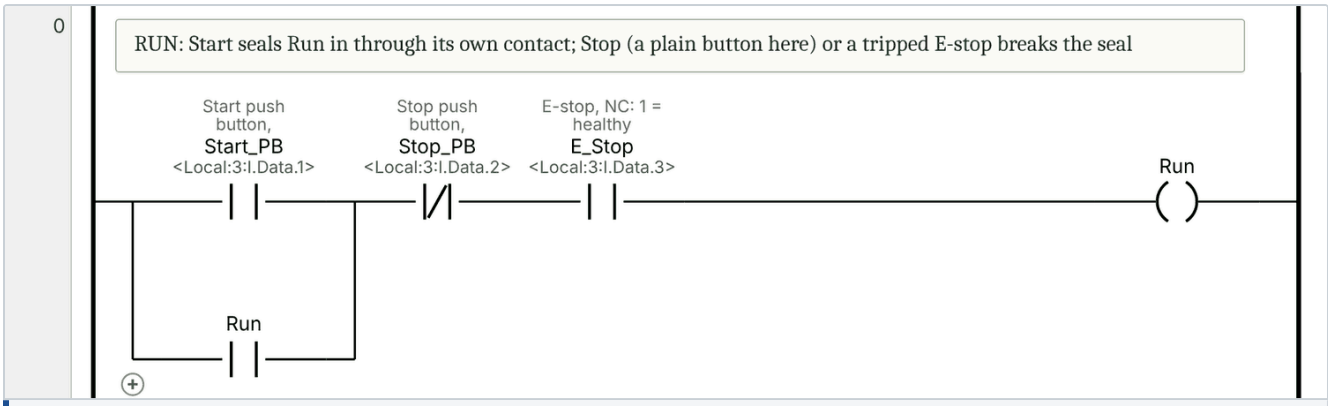
The level's alias tags are already made for you (chapter 32's table). Create these plain tags before you type the rungs; Verify reports any you miss.

Tag	Type	How to make it	Description
`Run`	BOOL	Edit Tags: name, type	Seal-in run bit

Tag	Type	How to make it	Description
`Hold`	BOOL	Edit Tags: name, type	The filler holds the belt
`Busy`	BOOL	Edit Tags: name, type	A bottle is between the hopper and the end of its cycle (Levels 1-3) / between the hopper and the filler (Levels 4-5)
`Rel_ONS`	BOOL	Edit Tags: name, type	One-shot storage for the release
`Is_R`	BOOL	Edit Tags: name, type	Latched: the colour sensor saw red
`Is_B`	BOOL	Edit Tags: name, type	Latched: the colour sensor saw blue
`Is_G`	BOOL	Edit Tags: name, type	Latched: the colour sensor saw green
`Is_Broken`	BOOL	Edit Tags: name, type	Latched: the crack eye saw a broken bottle
`Filled`	BOOL	Edit Tags: name, type	This bottle has its charge
`Trip`	TIMER	Edit Tags: name, type	Watchdog: the bottle ran the whole belt
`DN`	BOOL	Edit Tags: name, type	
`Done`	BOOL	Edit Tags: name, type	A pusher has fired for this bottle
`After`	TIMER	Edit Tags: name, type	Landing time after a push

The run circuit and the release

RUNG 0




```
[XIC(Start_PB),XIC(Run)]XIO(Stop_PB)XIC(E_Stop)OTE(Run);
```

PREDICT IT · 1 `Start_PB` is on, `Stop_PB` is on and `E_Stop` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When either (`Start_PB` (Start push button, momentary) is on) *or* (`Run` (Seal-in run bit) is on), `Stop_PB` (Stop push button, momentary) is off and `E_Stop` (E-stop, NC: 1 = healthy) is on, the rung turns `Run` on (and off again whenever the rung is false).

Left to right:

1. A branch of 2 parallel paths: power gets through if any one of them is true.
   - **Path 1**
     1. `XIC(Start_PB)` *Examine If Closed* asks: true when `Start_PB` (Start push button, momentary) is on.
   - **Path 2**
     1. `XIC(Run)` *Examine If Closed* asks: true when `Run` (Seal-in run bit) is on.
2. `XIO(Stop_PB)` *Examine If Open* asks: true when `Stop_PB` (Stop push button, momentary) is off.
3. `XIC(E_Stop)` *Examine If Closed* asks: true when `E_Stop` (E-stop, NC: 1 = healthy) is on.
4. `OTE(Run)` *Output Energize* does: turns `Run` on (and off again whenever the rung is false). Every scan it writes the rung state: true on, false off.

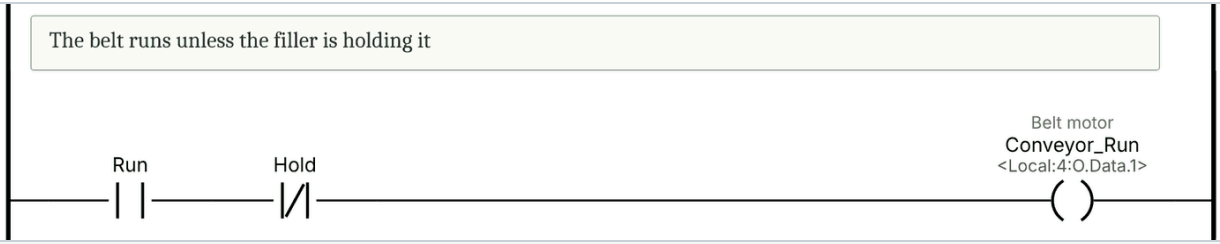
Why it is built this way:

Seal-in. The start condition turns `Run` on, and `Run` 's own contact in the parallel path keeps the rung true after the start goes away. Anything in series outside the branch (a stop, a fault) breaks it, and then it stays off until the start is made again.

> **Think of it like this:** a doorbell button and a hand. The start button is the doorbell, pressed for a moment. The seal-in contact is a hand that grabs the circuit as soon as the output comes on and keeps holding after you let go. The stop button, in series, is the only thing that can knock that hand away.

> **Watch for:** The stop has to be in series outside the branch. Put it inside the start path only and the seal-in path bypasses it: the stop no longer stops.

RUNG 1




```
XIC(Run)XIO(Hold)OTE(Conveyor_Run);
```

PREDICT IT · 2 `Run` is on and `Hold` is off. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

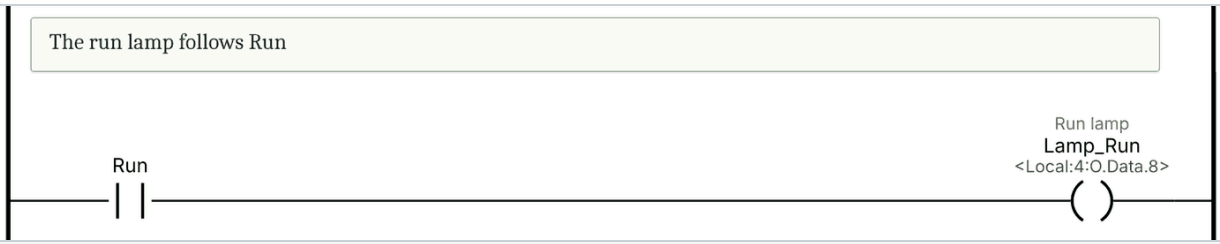
In words. When `Run` (Seal-in run bit) is on and `Hold` (The filler holds the belt) is off, the rung turns `Conveyor_Run` (Belt motor) on (and off again whenever the rung is false).

Left to right:

1. `XIC(Run)` *Examine If Closed* asks: true when `Run` (Seal-in run bit) is on.
2. `XIO(Hold)` *Examine If Open* asks: true when `Hold` (The filler holds the belt) is off.
3. `OTE(Conveyor_Run)` *Output Energize* does: turns `Conveyor_Run` (Belt motor) on (and off again whenever the rung is false).

> **Think of it like this:** a rung is a pipe with valves in it, read left to right. Every contact is a valve: XIC is open while its bit is 1, XIO while its bit is 0. Water reaches the output at the right only if every valve in a row is open. The output does not remember anything: on every scan it is simply wet or dry.

RUNG 2




```
XIC(Run)OTE(Lamp_Run);
```

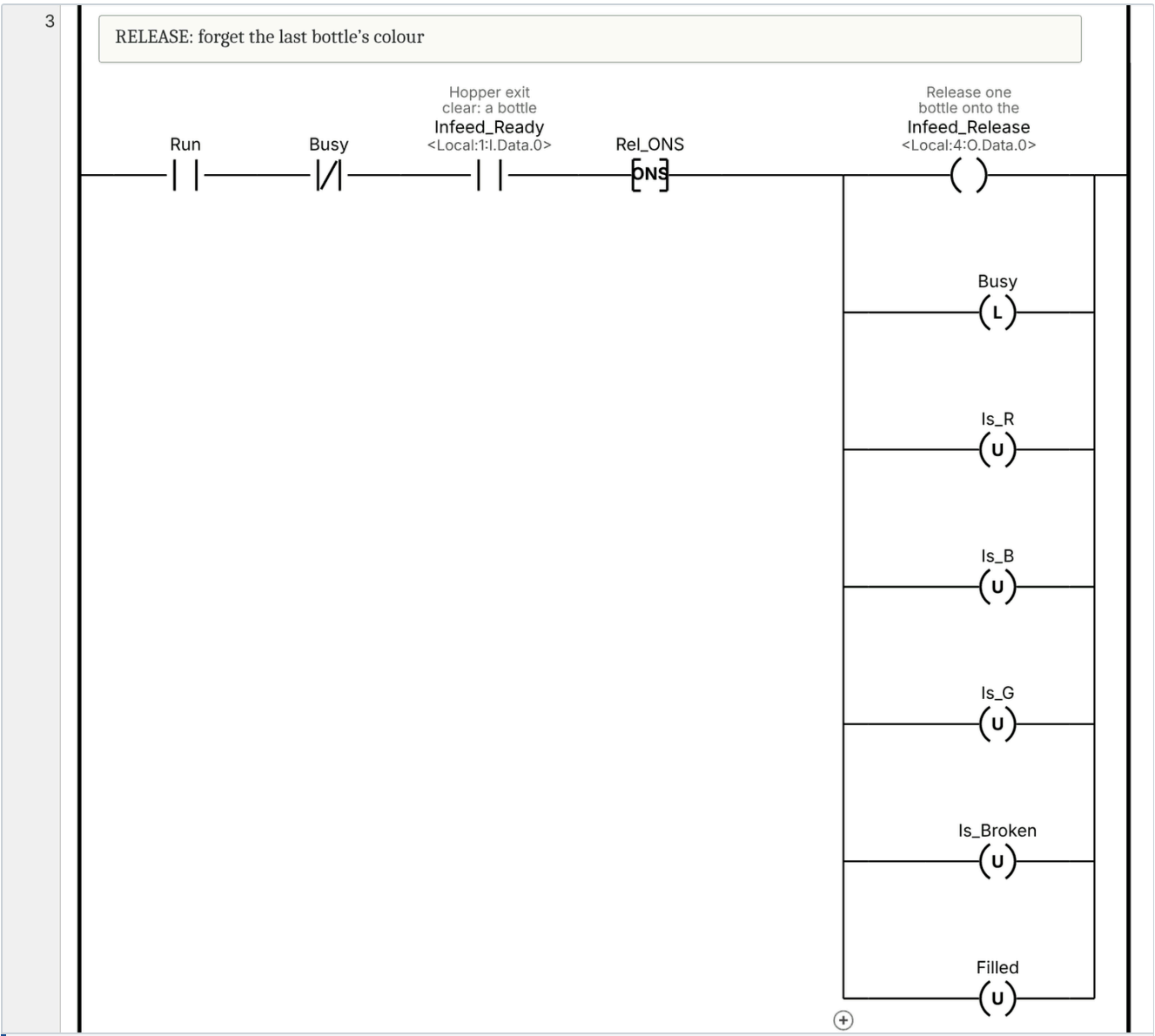
HOW THIS RUNG WORKS

In words. When `Run` (Seal-in run bit) is on, the rung turns `Lamp_Run` (Run lamp) on (and off again whenever the rung is false).

Left to right:

1. `XIC(Run)` *Examine If Closed* asks: true when `Run` (Seal-in run bit) is on.
2. `OTE(Lamp_Run)` *Output Energize* does: turns `Lamp_Run` (Run lamp) on (and off again whenever the rung is false).

RUNG 3




```
XIC(Run)XIO(Busy)XIC(Infeed_Ready)ONS(Rel_ONS)
[OTE(Infeed_Release),OTL(Busy),OTU(Is_R),OTU(Is_B),OTU(Is_G),OTU(Is_Broken),OTU
(Filled)];
```

HOW THIS RUNG WORKS

In words. When `Run` (Seal-in run bit) is on, `Busy` (A bottle is between the hopper and the end of its cycle (Levels 1-3) / between the hopper and the filler (Levels 4-5)) is off, `Infeed_Ready` (Hopper exit clear: a bottle can be released) is on and it is the first scan they are true (the one-shot, remembered in `Rel_ONS` (One-shot storage for the release)), the rung turns `Infeed_Release` (Release one bottle onto the belt (rising edge)) on (and off again whenever the rung is false), latches `Busy` on, unlatches `Is_R` (Latched: the colour sensor saw red) (turns it off), unlatches `Is_B` (Latched: the colour sensor saw blue) (turns it off), unlatches `Is_G` (Latched: the colour sensor saw green) (turns it off), unlatches `Is_Broken` (Latched: the crack eye saw a broken bottle) (turns it off) and unlatches `Filled` (This bottle has its charge) (turns it off).

Left to right:

1. `XIC(Run)` *Examine If Closed* asks: true when `Run` (Seal-in run bit) is on.
2. `XIO(Busy)` *Examine If Open* asks: true when `Busy` (A bottle is between the hopper and the end of its cycle (Levels 1-3) / between the hopper and the filler (Levels 4-5)) is off.
3. `XIC(Infeed_Ready)` *Examine If Closed* asks: true when `Infeed_Ready` (Hopper exit clear: a bottle can be released) is on.
4. `ONS(Rel_ONS)` *One Shot* asks: true when it is the first scan they are true (the one-shot, remembered in `Rel_ONS` (One-shot storage for the release)). It passes power for one scan only, the scan its input goes from false to true; its storage bit remembers last scan.
5. A branch of 7 parallel paths: each path is tested on its own and does its own work.

   Path 1

   1. `OTE(Infeed_Release)` *Output Energize* does: turns `Infeed_Release` (Release one bottle onto the belt (rising edge)) on (and off again whenever the rung is false).

   Path 2

   1. `OTL(Busy)` *Output Latch* does: latches `Busy` on. It only ever sets the bit; the bit stays on after the rung goes false.

   Path 3

   1. `OTU(Is_R)` *Output Unlatch* does: unlatches `Is_R` (Latched: the colour sensor saw red) (turns it off). It only ever clears the bit.

   Path 4

   1. `OTU(Is_B)` *Output Unlatch* does: unlatches `Is_B` (Latched: the colour sensor saw blue) (turns it off).

   Path 5

   1. `OTU(Is_G)` *Output Unlatch* does: unlatches `Is_G` (Latched: the colour sensor saw green) (turns it off).

   Path 6

   1. `OTU(Is_Broken)` *Output Unlatch* does: unlatches `Is_Broken` (Latched: the crack eye saw a broken bottle) (turns it off).

   Path 7

   1. `OTU(Filled)` *Output Unlatch* does: unlatches `Filled` (This bottle has its charge) (turns it off).

Why it is built this way:

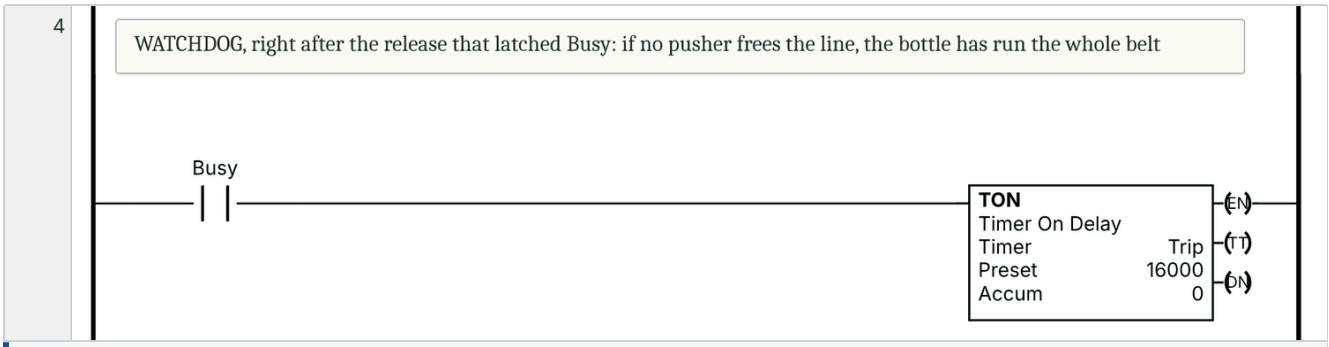
One-shot. Everything after the ONS happens once per event, not on every scan the condition holds; `Rel_ONS` is only its memory of the last scan.

A latch. `Busy` stays on after this rung goes false; an `OTU` elsewhere in the program clears it.

> **Think of it like this:** a doorbell that rings once however long you lean on the button. The ONS looks at what the button was doing last scan: only the change from up to down rings it.

> **Watch for:** Each ONS needs its own storage bit. Two ONS sharing one bit make one of them miss its edges.

RUNG 4




```
XIC(Busy)TON(Trip,16000,0);
```

HOW THIS RUNG WORKS

In words. When `Busy` (A bottle is between the hopper and the end of its cycle (Levels 1-3) / between the hopper and the filler (Levels 4-5)) is on, the rung runs the timer `Trip` (Watchdog: the bottle ran the whole belt) toward 16000 ms (16 s).

Left to right:

1. `XIC(Busy)` *Examine If Closed* asks: true when `Busy` (A bottle is between the hopper and the end of its cycle (Levels 1-3) / between the hopper and the filler (Levels 4-5)) is on.
2. `TON(Trip,16000,0)` *Timer On Delay* does: runs the timer `Trip` (Watchdog: the bottle ran the whole belt) toward 16000 ms (16 s). The accumulator counts while the rung is true; .DN comes on at the preset; the moment the rung goes false the timer resets.

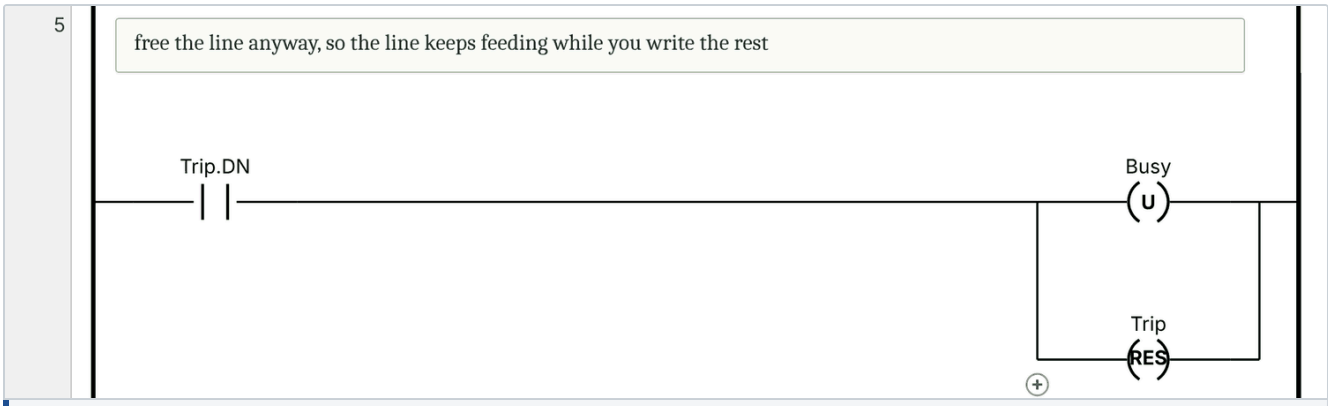
Why it is built this way:

A timer. `Trip` only times while this rung is true; other rungs use its `.DN` (done) or `.TT` (timing) bits. Being a TON, it starts again from zero every time the rung goes false.

> **Think of it like this:** a kitchen timer that only runs while you hold your finger on it. Take your finger off and it snaps back to zero. Hold it long enough and the done light comes on, and stays on as long as your finger is still there.

> **Watch for:** If anything in series flickers false for one scan, a TON starts over from zero. Use RTO when the time must survive an interruption.

RUNG 5




```
XIC(Trip.DN)[OTU(Busy),RES(Trip)];
```

HOW THIS RUNG WORKS

In words. When `Trip.DN` (the done bit of `Trip` : Watchdog: the bottle ran the whole belt) is on, the rung unlatches `Busy` (A bottle is between the hopper and the end of its cycle (Levels 1-3) / between the hopper and the filler (Levels 4-5)) (turns it off) and resets `Trip` .

Left to right:

1. `XIC(Trip.DN)` *Examine If Closed* asks: true when `Trip.DN` (the done bit of `Trip` : Watchdog: the bottle ran the whole belt) is on.
2. A branch of 2 parallel paths: each path is tested on its own and does its own work.

   Path 1

   1. `OTU(Busy)` *Output Unlatch* does: unlatches `Busy` (A bottle is between the hopper and the end of its cycle (Levels 1-3) / between the hopper and the filler (Levels 4-5)) (turns it off).

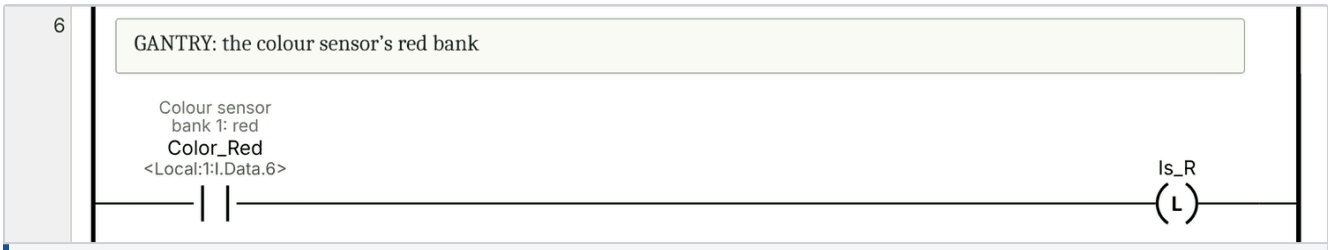
   Path 2

   1. `RES(Trip)` *Reset* does: resets `Trip` . It zeroes a timer or counter (accumulator and status bits), or a control (its position and status bits).

The release now unlatches the colour bits instead of the height bits; the watchdog follows it as in Level 1.

Latching the colour

RUNG 6




```
XIC(Color_Red)OTL(Is_R);
```

HOW THIS RUNG WORKS

In words. When `Color_Red` (Colour sensor bank 1: red) is on, the rung latches `Is_R` (Latched: the colour sensor saw red) on.

Left to right:

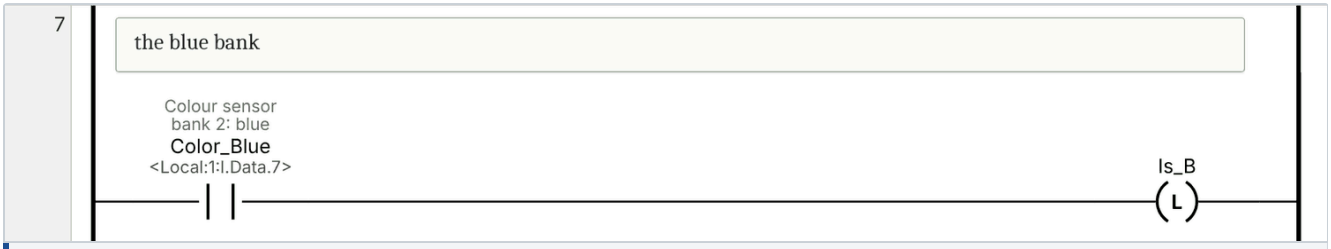
1. `XIC(Color_Red)` *Examine If Closed* asks: true when `Color_Red` (Colour sensor bank 1: red) is on.
2. `OTL(Is_R)` *Output Latch* does: latches `Is_R` (Latched: the colour sensor saw red) on.

Why it is built this way:

A latch: it stays on until an OTU clears it.

> **Watch for:** A latched bit survives the condition that set it: every way in needs a way out, or it stays on for good.

RUNG 7




```
XIC(Color_Blue)OTL(Is_B);
```

HOW THIS RUNG WORKS

In words. When `Color_Blue` (Colour sensor bank 2: blue) is on, the rung latches `Is_B` (Latched: the colour sensor saw blue) on.

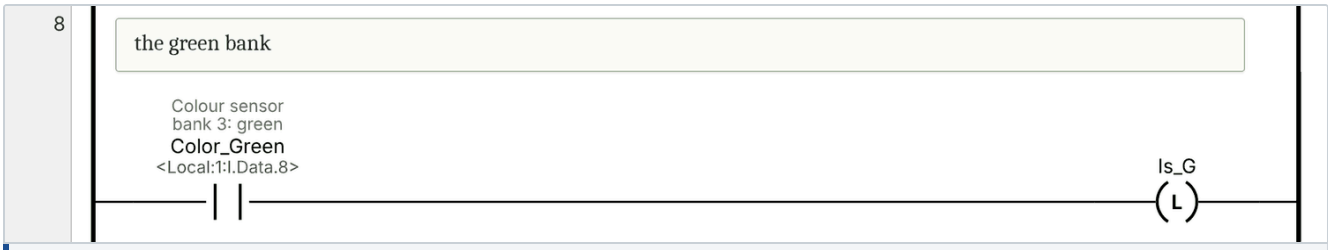
Left to right:

1. `XIC(Color_Blue)` *Examine If Closed* asks: true when `Color_Blue` (Colour sensor bank 2: blue) is on.
2. `OTL(Is_B)` *Output Latch* does: latches `Is_B` (Latched: the colour sensor saw blue) on.

Why it is built this way:

A latch: it stays on until an OTU clears it.

RUNG 8




```
XIC(Color_Green)OTL(Is_G);
```

HOW THIS RUNG WORKS

In words. When `Color_Green` (Colour sensor bank 3: green) is on, the rung latches `Is_G` (Latched: the colour sensor saw green) on.

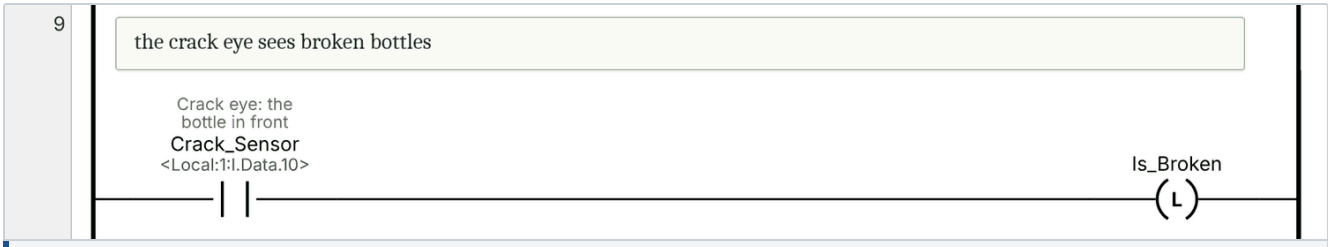
Left to right:

1. `XIC(Color_Green)` *Examine If Closed* asks: true when `Color_Green` (Colour sensor bank 3: green) is on.
2. `OTL(Is_G)` *Output Latch* does: latches `Is_G` (Latched: the colour sensor saw green) on.

Why it is built this way:

A latch: it stays on until an OTU clears it.

RUNG 9




```
XIC(Crack_Sensor)OTL(Is_Broken);
```

HOW THIS RUNG WORKS

In words. When `Crack_Sensor` (Crack eye: the bottle in front of it is broken) is on, the rung latches `Is_Broken` (Latched: the crack eye saw a broken bottle) on.

Left to right:

1. `XIC(Crack_Sensor)` *Examine If Closed* asks: true when `Crack_Sensor` (Crack eye: the bottle in front of it is broken) is on.
2. `OTL(Is_Broken)` *Output Latch* does: latches `Is_Broken` (Latched: the crack eye saw a broken bottle) on.

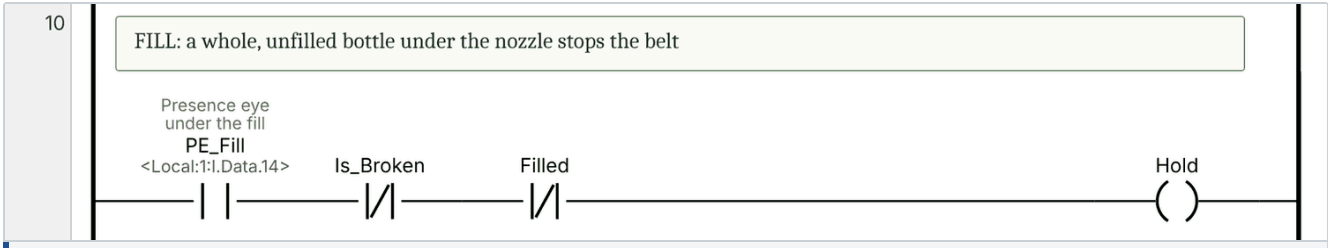
Why it is built this way:

A latch: it stays on until an OTU clears it.

Three latches replace the three height latches; the crack eye stays.

The fill

RUNG 10




```
XIC(PE_Fill)XIO(Is_Broken)XIO(Filled)OTE(Hold);
```

PREDICT IT · 3 `PE_Fill` is on, `Is_Broken` is off and `Filled` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

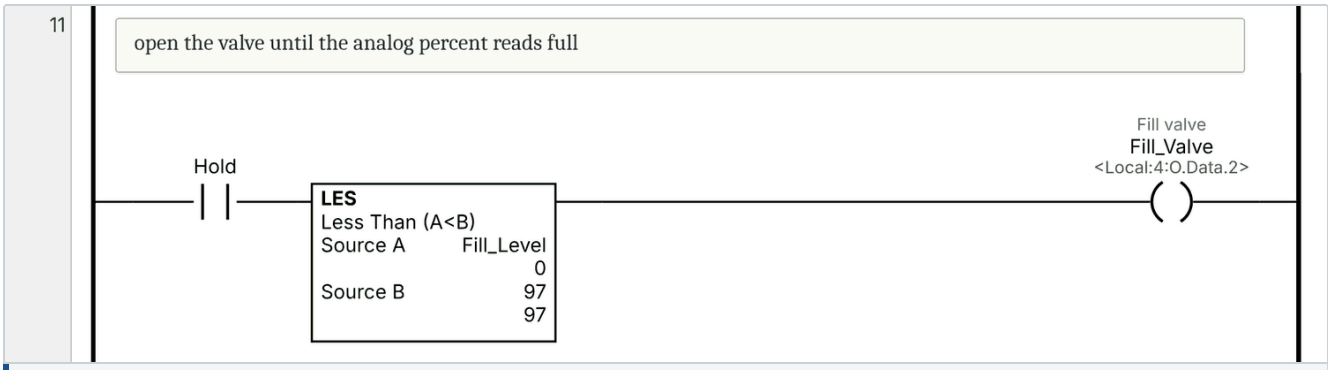
HOW THIS RUNG WORKS

In words. When `PE_Fill` (Presence eye under the fill nozzle) is on, `Is_Broken` (Latched: the crack eye saw a broken bottle) is off and `Filled` (This bottle has its charge) is off, the rung turns `Hold` (The filler holds the belt) on (and off again whenever the rung is false).

Left to right:

1. `XIC(PE_Fill)` *Examine If Closed* asks: true when `PE_Fill` (Presence eye under the fill nozzle) is on.
2. `XIO(Is_Broken)` *Examine If Open* asks: true when `Is_Broken` (Latched: the crack eye saw a broken bottle) is off.
3. `XIO(Filled)` *Examine If Open* asks: true when `Filled` (This bottle has its charge) is off.
4. `OTE(Hold)` *Output Energize* does: turns `Hold` (The filler holds the belt) on (and off again whenever the rung is false).

RUNG 11




```
XIC(Hold)LES(Fill_Level,97)OTE(Fill_Valve);
```

PREDICT IT · 4 `Hold` is on and `Fill_Level` is 96. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

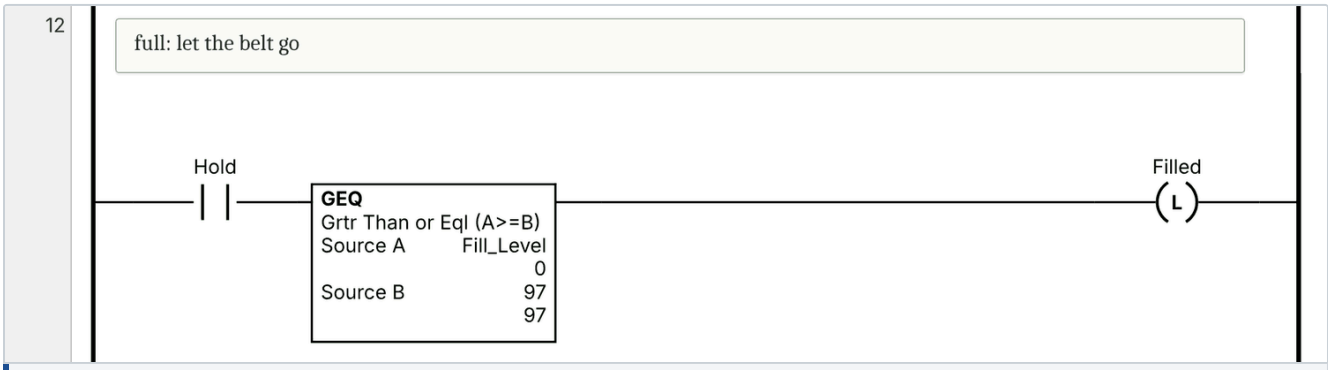
HOW THIS RUNG WORKS

In words. When `Hold` (The filler holds the belt) is on and `Fill_Level` (Fill of the bottle under the nozzle, 0-100 % of its label) is less than **97**, the rung turns `Fill_Valve` (Fill valve) on (and off again whenever the rung is false).

Left to right:

1. `XIC(Hold)` *Examine If Closed* asks: true when `Hold` (The filler holds the belt) is on.
2. `LES(Fill_Level,97)` *Less Than* asks: true when `Fill_Level` (Fill of the bottle under the nozzle, 0-100 % of its label) is less than **97**.
3. `OTE(Fill_Valve)` *Output Energize* does: turns `Fill_Valve` (Fill valve) on (and off again whenever the rung is false).

RUNG 12




```
XIC(Hold)GEQ(Fill_Level,97)OTL(Filled);
```

PREDICT IT · 5 `Hold` is off and `Fill_Level` is 97. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Hold` (The filler holds the belt) is on and `Fill_Level` (Fill of the bottle under the nozzle, 0-100 % of its label) is at least **97**, the rung latches `Filled` (This bottle has its charge) on.

Left to right:

1. `XIC(Hold)` *Examine If Closed* asks: true when `Hold` (The filler holds the belt) is on.
2. `GEQ(Fill_Level,97)` *Greater Than or Equal* asks: true when `Fill_Level` (Fill of the bottle under the nozzle, 0-100 % of its label) is at least **97**.
3. `OTL(Filled)` *Output Latch* does: latches `Filled` (This bottle has its charge) on.

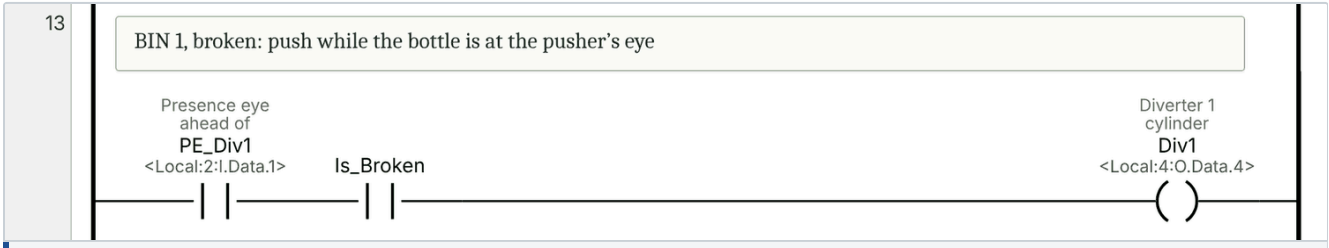
Why it is built this way:

A latch: it stays on until an OTU clears it.

Unchanged from Level 2.

The pushers

RUNG 13




```
XIC(PE_Div1)XIC(Is_Broken)OTE(Div1);
```

PREDICT IT · 6 `PE_Div1` is on and `Is_Broken` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

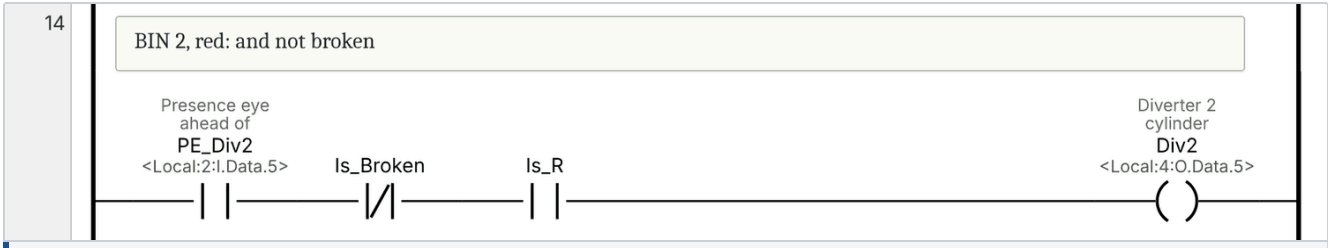
HOW THIS RUNG WORKS

In words. When `PE_Div1` (Presence eye ahead of diverter 1) is on and `Is_Broken` (Latched: the crack eye saw a broken bottle) is on, the rung turns `Div1` (Diverter 1 cylinder) on (and off again whenever the rung is false).

Left to right:

1. `XIC(PE_Div1)` *Examine If Closed* asks: true when `PE_Div1` (Presence eye ahead of diverter 1) is on.
2. `XIC(Is_Broken)` *Examine If Closed* asks: true when `Is_Broken` (Latched: the crack eye saw a broken bottle) is on.
3. `OTE(Div1)` *Output Energize* does: turns `Div1` (Diverter 1 cylinder) on (and off again whenever the rung is false).

RUNG 14




```
XIC(PE_Div2)XIO(Is_Broken)XIC(Is_R)OTE(Div2);
```

PREDICT IT · 7 `PE_Div2` is on, `Is_Broken` is on and `Is_R` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

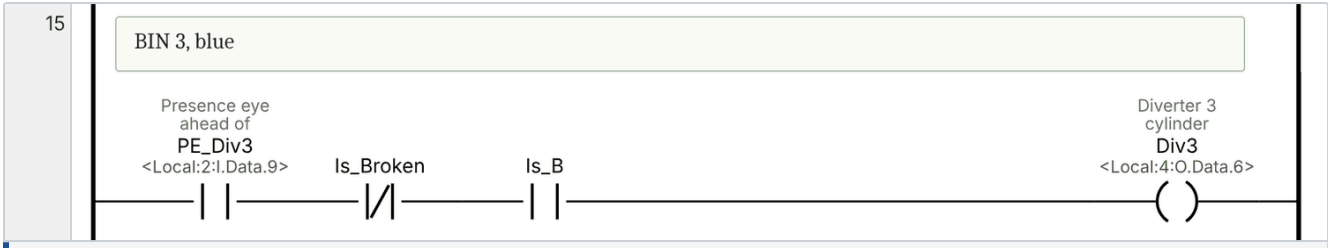
HOW THIS RUNG WORKS

In words. When `PE_Div2` (Presence eye ahead of diverter 2) is on, `Is_Broken` (Latched: the crack eye saw a broken bottle) is off and `Is_R` (Latched: the colour sensor saw red) is on, the rung turns `Div2` (Diverter 2 cylinder) on (and off again whenever the rung is false).

Left to right:

1. `XIC(PE_Div2)` *Examine If Closed* asks: true when `PE_Div2` (Presence eye ahead of diverter 2) is on.
2. `XIO(Is_Broken)` *Examine If Open* asks: true when `Is_Broken` (Latched: the crack eye saw a broken bottle) is off.
3. `XIC(Is_R)` *Examine If Closed* asks: true when `Is_R` (Latched: the colour sensor saw red) is on.
4. `OTE(Div2)` *Output Energize* does: turns `Div2` (Diverter 2 cylinder) on (and off again whenever the rung is false).

RUNG 15




```
XIC(PE_Div3)XIO(Is_Broken)XIC(Is_B)OTE(Div3);
```

PREDICT IT · 8 `PE_Div3` is on, `Is_Broken` is off and `Is_B` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

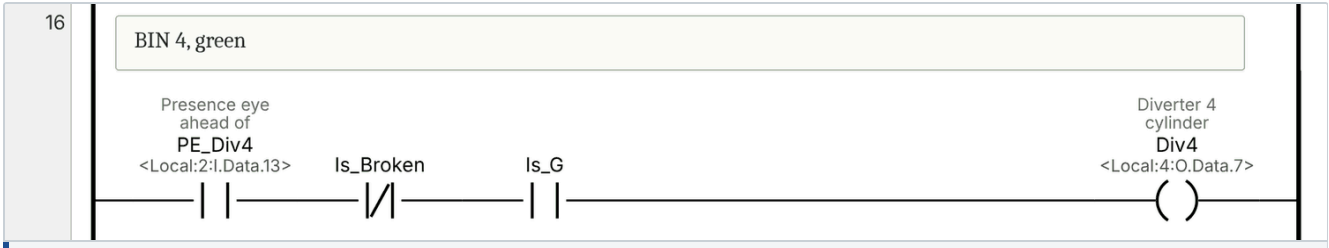
HOW THIS RUNG WORKS

In words. When `PE_Div3` (Presence eye ahead of diverter 3) is on, `Is_Broken` (Latched: the crack eye saw a broken bottle) is off and `Is_B` (Latched: the colour sensor saw blue) is on, the rung turns `Div3` (Diverter 3 cylinder) on (and off again whenever the rung is false).

Left to right:

1. `XIC(PE_Div3)` *Examine If Closed* asks: true when `PE_Div3` (Presence eye ahead of diverter 3) is on.
2. `XIO(Is_Broken)` *Examine If Open* asks: true when `Is_Broken` (Latched: the crack eye saw a broken bottle) is off.
3. `XIC(Is_B)` *Examine If Closed* asks: true when `Is_B` (Latched: the colour sensor saw blue) is on.
4. `OTE(Div3)` *Output Energize* does: turns `Div3` (Diverter 3 cylinder) on (and off again whenever the rung is false).

RUNG 16




```
XIC(PE_Div4)XIO(Is_Broken)XIC(Is_G)OTE(Div4);
```

PREDICT IT · 9 `PE_Div4` is on, `Is_Broken` is off and `Is_G` is off. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `PE_Div4` (Presence eye ahead of diverter 4) is on, `Is_Broken` (Latched: the crack eye saw a broken bottle) is off and `Is_G` (Latched: the colour sensor saw green) is on, the rung turns `Div4` (Diverter 4 cylinder) on (and off again whenever the rung is false).

Left to right:

1. `XIC(PE_Div4)` *Examine If Closed* asks: true when `PE_Div4` (Presence eye ahead of diverter 4) is on.
2. `XIO(Is_Broken)` *Examine If Open* asks: true when `Is_Broken` (Latched: the crack eye saw a broken bottle) is off.
3. `XIC(Is_G)` *Examine If Closed* asks: true when `Is_G` (Latched: the colour sensor saw green) is on.
4. `OTE(Div4)` *Output Energize* does: turns `Div4` (Diverter 4 cylinder) on (and off again whenever the rung is false).

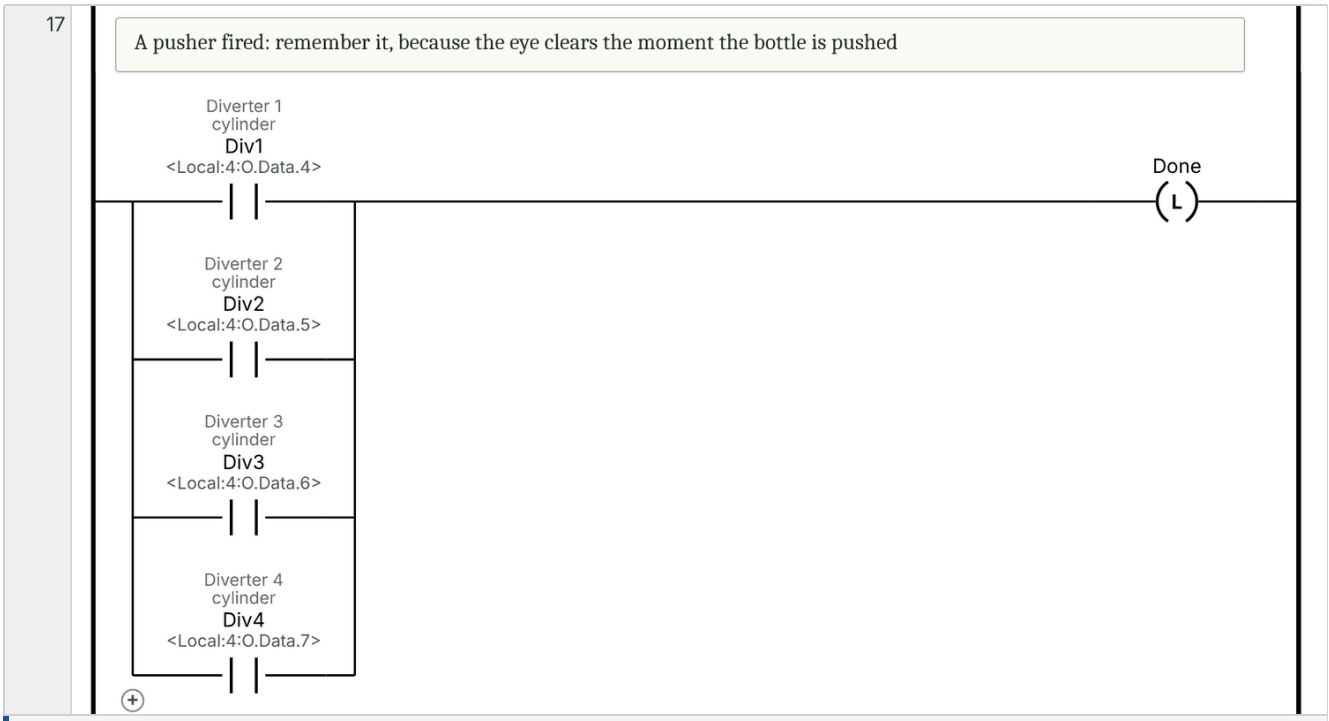
Bin 1 is still broken bottles; bins 2, 3 and 4 test one colour each.

WATCH FOR: A BROKEN BOTTLE HAS A COLOUR TOO

The colour sensor does not know a bottle is broken; a broken red bottle turns `Color_Red` on like any other. If the red bin's rung tested only `Is_R`, a broken red bottle that somehow got past bin 1 would score −50 in bin 2. Every whole-bottle rung keeps `XIO(Is_Broken)`.

Freeing the line

RUNG 17




```
[XIC(Div1),XIC(Div2),XIC(Div3),XIC(Div4)]OTL(Done);
```

PREDICT IT · 10 `Div1` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When either (`Div1` (Diverter 1 cylinder) is on) *or* (`Div2` (Diverter 2 cylinder) is on) *or* (`Div3` (Diverter 3 cylinder) is on) *or* (`Div4` (Diverter 4 cylinder) is on), the rung latches `Done` (A pusher has fired for this bottle) on.

Left to right:

1. A branch of 4 parallel paths: power gets through if any one of them is true.
   - **Path 1**
     1. `XIC(Div1)` *Examine If Closed* asks: true when `Div1` (Diverter 1 cylinder) is on.
   - **Path 2**
     1. `XIC(Div2)` *Examine If Closed* asks: true when `Div2` (Diverter 2 cylinder) is on.
   - **Path 3**
     1. `XIC(Div3)` *Examine If Closed* asks: true when `Div3` (Diverter 3 cylinder) is on.
   - **Path 4**
     1. `XIC(Div4)` *Examine If Closed* asks: true when `Div4` (Diverter 4 cylinder) is on.
2. `OTL(Done)` *Output Latch* does: latches `Done` (A pusher has fired for this bottle) on.

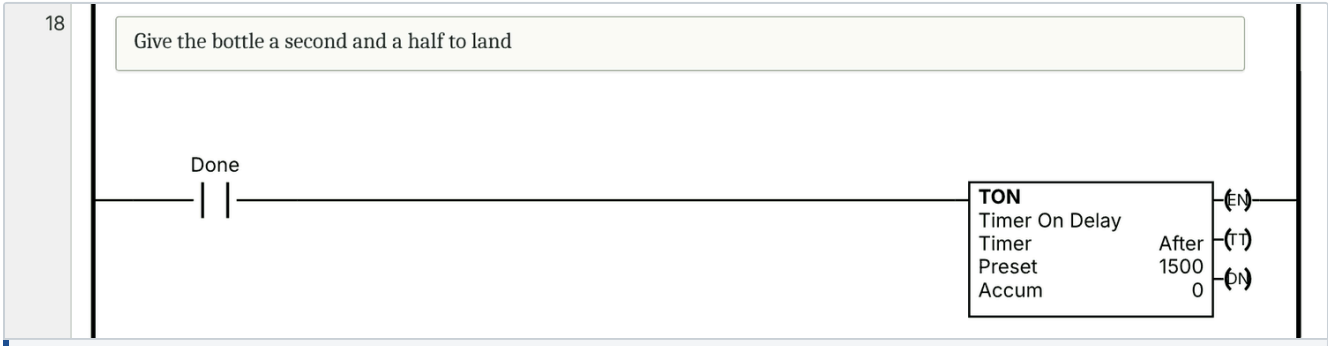
Why it is built this way:

A latch: it stays on until an OTU clears it.

> **Think of it like this:** parallel paths are two roads to the same town. If either road is open you get there; the rung only

> cares that power reaches the far end, not which way it went.

RUNG 18




```
XIC(Done)TON(After,1500,0);
```

HOW THIS RUNG WORKS

In words. When `Done` (A pusher has fired for this bottle) is on, the rung runs the timer `After` (Landing time after a push) toward 1500 ms (1.5 s).

Left to right:

1. `XIC(Done)` *Examine If Closed* asks: true when `Done` (A pusher has fired for this bottle) is on.
2. `TON(After,1500,0)` *Timer On Delay* does: runs the timer `After` (Landing time after a push) toward 1500 ms (1.5 s).

Why it is built this way:

A timer: it times only while this rung is true.

RUNG 19

```
XIC(After.DN)[OTU(Busy),OTU(Done),RES(After),RES(Trip)];
```

HOW THIS RUNG WORKS

In words. When `After.DN` (the done bit of `After` : Landing time after a push) is on, the rung unlatches `Busy` (A bottle is between the hopper and the end of its cycle (Levels 1-3) / between the hopper and the filler (Levels 4-5)) (turns it off), unlatches `Done` (A pusher has fired for this bottle) (turns it off), resets `After` and resets `Trip` (Watchdog: the bottle ran the whole belt).

Left to right:

1. `XIC(After.DN)` *Examine If Closed* asks: true when `After.DN` (the done bit of `After` : Landing time after a push) is on.
2. A branch of 4 parallel paths: each path is tested on its own and does its own work.

   Path 1

   1. `OTU(Busy)` *Output Unlatch* does: unlatches `Busy` (A bottle is between the hopper and the end of its cycle (Levels 1-3) / between the hopper and the filler (Levels 4-5)) (turns it off).

   Path 2

   1. `OTU(Done)` *Output Unlatch* does: unlatches `Done` (A pusher has fired for this bottle) (turns it off).

   Path 3

   1. `RES(After)` *Reset* does: resets `After` .

   Path 4

   1. `RES(Trip)` *Reset* does: resets `Trip` (Watchdog: the bottle ran the whole belt).

Unchanged from Level 2: the landing timer frees the line and resets the watchdog.

Run it

Twelve bottles, sorted red, blue and green, broken ones to bin 1, every whole one filled. The reference run: 12 of 12 bottles in the right bin, nothing lost, in 125 s against a par of 190 s: 1400 points and three stars. Look at the time: one bottle at a time is now taking most of two minutes, and the next level asks for fourteen bottles in 110 seconds.

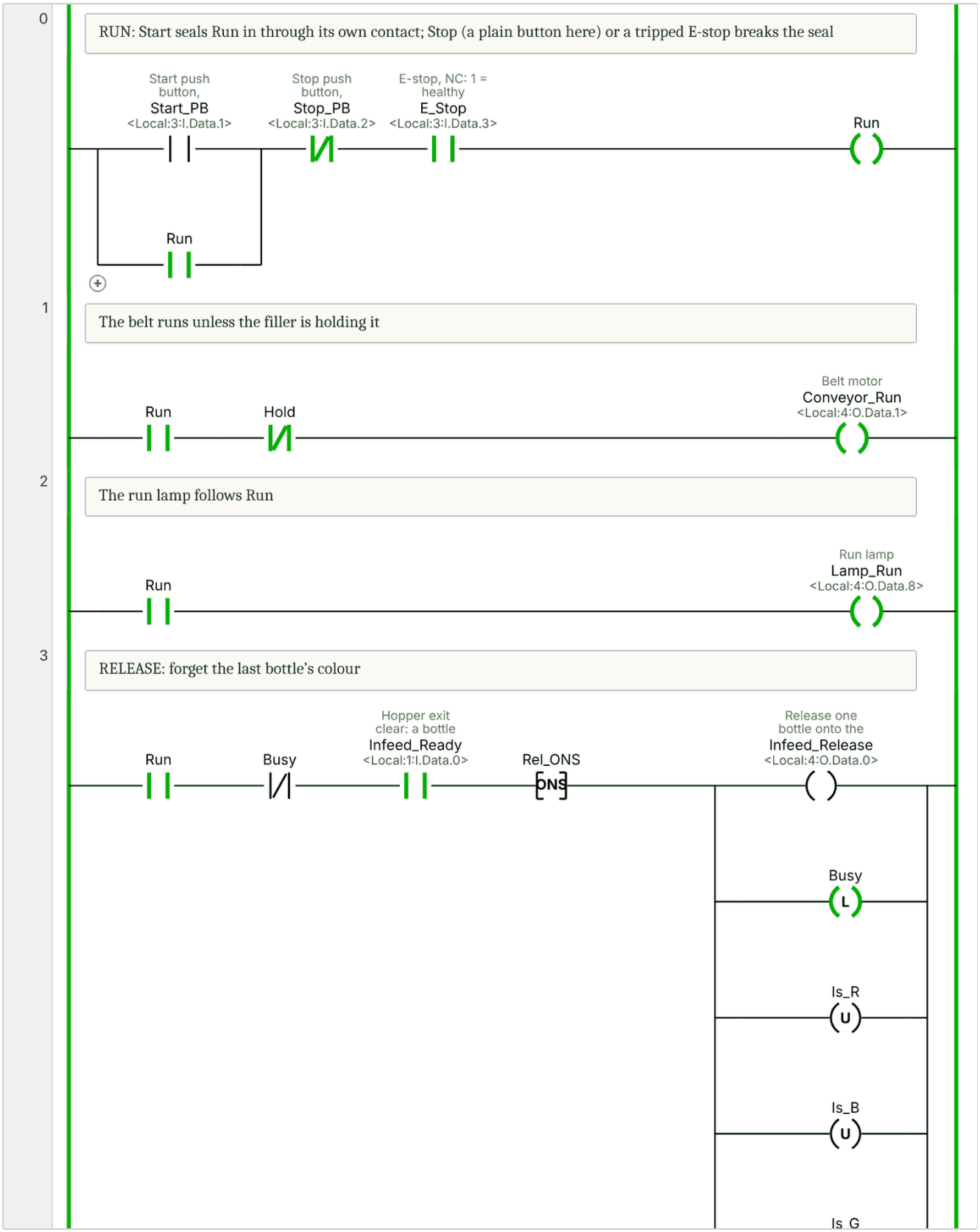


Figure 147. Level 3 online: the colour latches in place of the height latches.

TRY IT

Set the hopper's colour mix to *random* instead of *random (R/B/G)*: some bottles are now yellow. Nobody wants them, so they ride to the end of the belt and the watchdog frees the line after 16 s. Count what that costs against par, then write a rung that frees the line as soon as `PE_LineEnd` sees the bottle.

Predict it: the answers

1. The rung is false at `XIO(Stop_PB)`: `Stop_PB` is on, so `Run` is **off** (an OTE writes 0 when its rung is false).
2. Every condition on the way is met, so `Conveyor_Run` is **on**.
3. The rung is false at `XIO(Filled)`: `Filled` is on, so `Hold` is **off** (an OTE writes 0 when its rung is false).
4. Every condition on the way is met, so `Fill_Valve` is **on**.
5. The rung is false at `XIC(Hold)`: `Hold` is off, so the OTL does nothing: `Filled` keeps whatever it had.
6. Every condition on the way is met, so `Div1` is **on**.
7. The rung is false at `XIO(Is_Broken)`: `Is_Broken` is on, so `Div2` is **off** (an OTE writes 0 when its rung is false).
8. Every condition on the way is met, so `Div3` is **on**.
9. The rung is false at `XIC(Is_G)`: `Is_G` is off, so `Div4` is **off** (an OTE writes 0 when its rung is false).
10. Every condition on the way is met, so `Done` latches **on**.

36 Level 4 — Two on the line

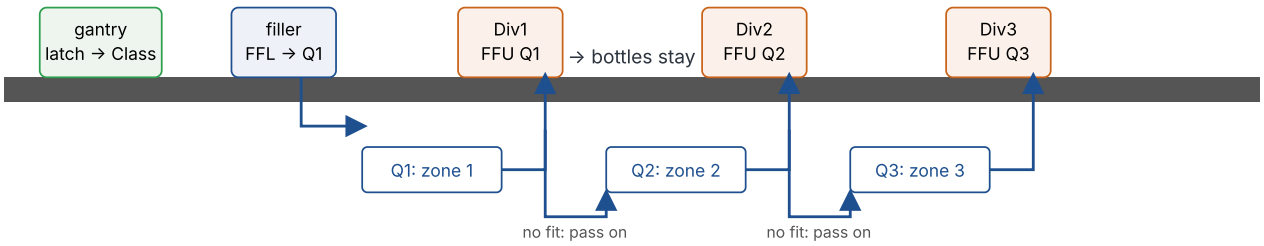
> **IN THE APP**
>
> A level project opens with guided mode on this chapter: the panel lists the tags the program needs (*Make them*), shows every rung below under its fold with *Copy* and *Add*, checks that MainRoutine verifies, and while you run the level ticks three results — the level completes, every bottle in the right bin, under par.

Height sort with fill again — Level 2's bins — but fourteen bottles and a par of 110 seconds. Level 2's program takes about 10 s a bottle; fourteen bottles is over two minutes. No setting fixes that. **The trap is the program's shape**: as long as the next bottle waits for the last one to reach its bin, the belt runs mostly empty. The next bottle has to leave the hopper while the last one is still on the belt, and as soon as there are two bottles on the belt, one set of `Is_` bits is not enough memory: by the time the first bottle reaches its pusher, the gantry has overwritten what it saw.

The idea: a queue per stretch of belt

On the indexing line of Part IV, the memory that moved with the bottles was a shift register, one bit per stop, shifted once per index. This belt does not index — bottles arrive at the pushers whenever they arrive — but it keeps them **in order**: nothing on a belt overtakes anything else. Memory that keeps things in order is a **FIFO**, first in, first out. So:

- Keep one bottle at a time only in the *front* of the line — hopper to filler — where the gantry has to latch what it sees. Release the next bottle the moment the last one leaves the filler.
- Fold what the gantry saw into one number, the **class**: bit 0 small, 1 medium, 2 large, 3 broken (and 4-6 for colour, used in Level 5).
- When a bottle leaves the filler, load its class into **zone 1's queue** — the stretch between the filler and diverter 1.
- When a bottle reaches diverter *n*'s eye, **unload** the head of zone *n*'s queue: that is this bottle's class, because it is the oldest bottle in that stretch. If the class fits the bin, push. If not, the bottle rides on into zone *n*+1, so **load its class into zone *n*+1's queue**.



Each diverter unloads the head of its own zone's queue when a bottle reaches its eye: fit → push; no fit → load the class into the next zone's queue

Figure 148. Zone queues. The class of every bottle travels through FIFOs in step with the bottle itself — a shift register for a belt that does not index.

`FFL(source,array,control,length)` — FIFO Load — copies *source* to the tail of the queue in *array* on a rising edge of its rung; `FFU(array,dest,control,length)` — FIFO Unload — copies the head into *dest* and moves everything up. The CONTROL tag's `.POS` says how many are in the queue (chapter 5). Eight entries per zone is far more than this belt holds.

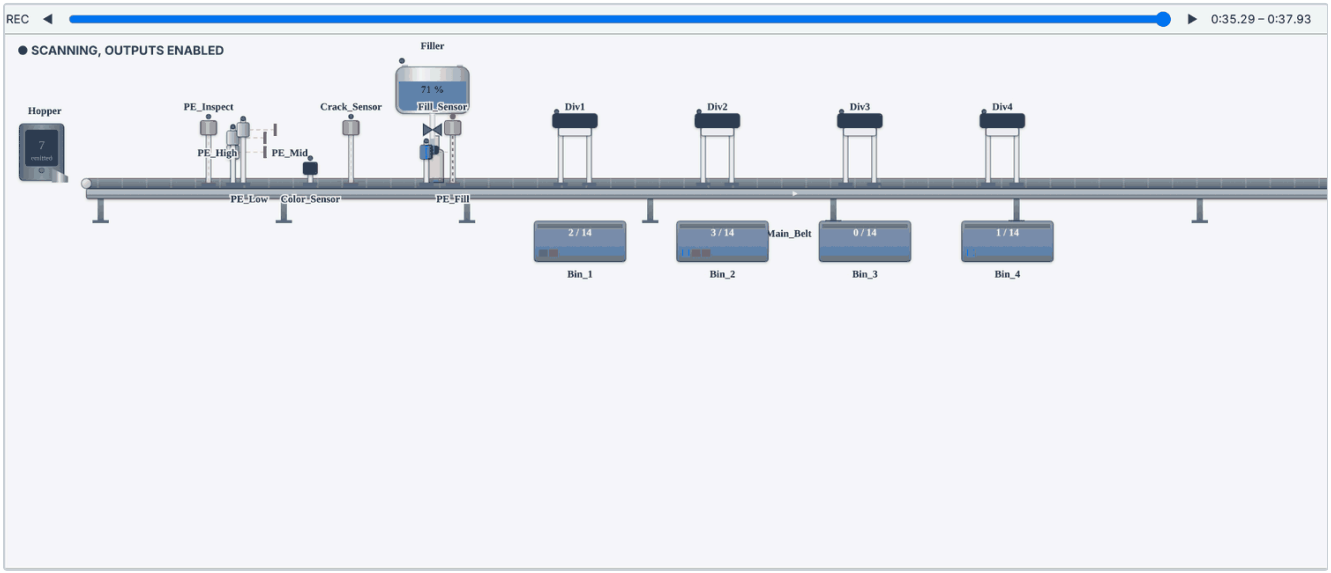


Figure 149. Level 4 running: several bottles on the belt at once, each with its class in a queue.

Tags this lesson needs

The level's alias tags are already made for you (chapter 32's table). Create these plain tags before you type the rungs; Verify reports any you miss.

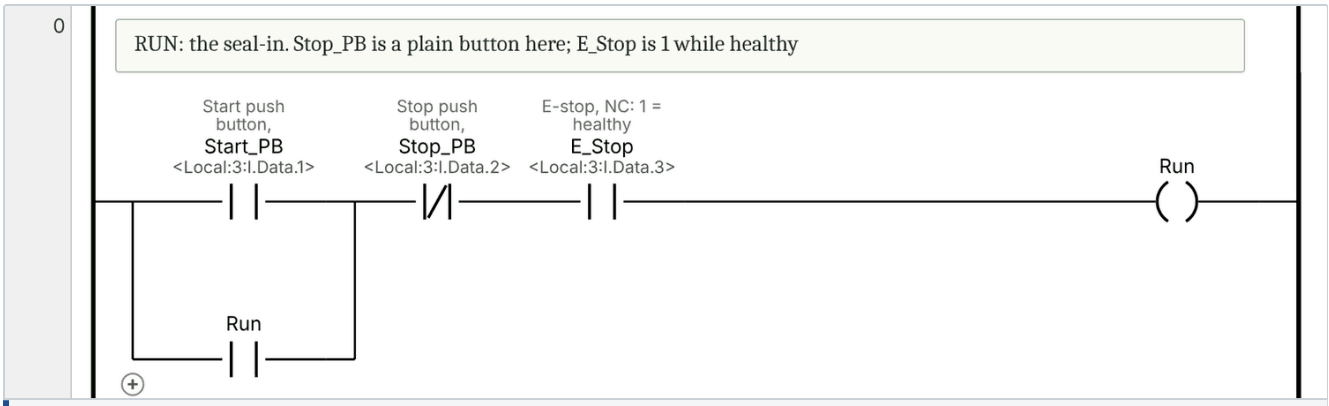
Tag	Type	How to make it	Description
`Run`	BOOL	Edit Tags: name, type	Seal-in run bit

Tag	Type	How to make it	Description
`Busy`	BOOL	Edit Tags: name, type	A bottle is between the hopper and the end of its cycle (Levels 1-3) / between the hopper and the filler (Levels 4-5)
`Rel_ONS`	BOOL	Edit Tags: name, type	One-shot storage for the release
`Hold`	BOOL	Edit Tags: name, type	The filler holds the belt
`Filled`	BOOL	Edit Tags: name, type	This bottle has its charge
`AtFill`	BOOL	Edit Tags: name, type	The eye under the nozzle has seen this bottle
`Leave_ONS`	BOOL	Edit Tags: name, type	One-shot storage: the bottle left the filler
`Is_Low`	BOOL	Edit Tags: name, type	Latched: the low height beam was broken
`Is_Mid`	BOOL	Edit Tags: name, type	Latched: the middle beam was broken
`Is_High`	BOOL	Edit Tags: name, type	Latched: the high beam was broken
`Is_Broken`	BOOL	Edit Tags: name, type	Latched: the crack eye saw a broken bottle
`Is_R`	BOOL	Edit Tags: name, type	Latched: the colour sensor saw red
`Is_B`	BOOL	Edit Tags: name, type	Latched: the colour sensor saw blue
`Is_G`	BOOL	Edit Tags: name, type	Latched: the colour sensor saw green
`Class`	DINT	Edit Tags: name, type, Style Binary	The bottle in the gantry: bit 0 S, 1 M, 2 L, 3 broken, 4 red, 5 blue, 6 green
`Cur`	DINT	Edit Tags: name, type, Style Binary	The class unloaded from a zone queue at a diverter
`Q1`	DINT[8]	Edit Tags: name, type, Dim 8	Zone 1 queue: classes of the bottles between the filler and diverter 1
`Q1_Ctl`	CONTROL	Edit Tags: name, type	Queue control
`D1_ONS`	BOOL	Edit Tags: name, type	One-shot storage for a diverter
`D1_ONS2`	BOOL	Edit Tags: name, type	One-shot storage for a diverter
`D1_ONS3`	BOOL	Edit Tags: name, type	One-shot storage for a diverter
`Fire_1`	BOOL	Edit Tags: name, type	Armed: this diverter will push the bottle at its eye
`Q2`	DINT[8]	Edit Tags: name, type, Dim 8	Zone 2 queue: classes of the bottles between diverter 1 and diverter 2
`Q2_Ctl`	CONTROL	Edit Tags: name, type	Queue control
`D2_ONS`	BOOL	Edit Tags: name, type	One-shot storage for a diverter
`D2_ONS2`	BOOL	Edit Tags: name, type	One-shot storage for a diverter
`D2_ONS3`	BOOL	Edit Tags: name, type	One-shot storage for a diverter

Tag	Type	How to make it	Description
`Fire_2`	BOOL	Edit Tags: name, type	Armed: this diverter will push the bottle at its eye
`Q3`	DINT[8]	Edit Tags: name, type, Dim 8	Zone 3 queue: classes of the bottles between diverter 2 and diverter 3
`Q3_Ctl`	CONTROL	Edit Tags: name, type	Queue control
`D3_ONS`	BOOL	Edit Tags: name, type	One-shot storage for a diverter
`D3_ONS2`	BOOL	Edit Tags: name, type	One-shot storage for a diverter
`D3_ONS3`	BOOL	Edit Tags: name, type	One-shot storage for a diverter
`Fire_3`	BOOL	Edit Tags: name, type	Armed: this diverter will push the bottle at its eye
`Q4`	DINT[8]	Edit Tags: name, type, Dim 8	Zone 4 queue: classes of the bottles between diverter 3 and diverter 4
`Q4_Ctl`	CONTROL	Edit Tags: name, type	Queue control
`D4_ONS`	BOOL	Edit Tags: name, type	One-shot storage for a diverter
`D4_ONS2`	BOOL	Edit Tags: name, type	One-shot storage for a diverter
`D4_ONS3`	BOOL	Edit Tags: name, type	One-shot storage for a diverter
`Fire_4`	BOOL	Edit Tags: name, type	Armed: this diverter will push the bottle at its eye

The run circuit and the release

RUNG 0




```
[XIC(Start_PB),XIC(Run)]XIO(Stop_PB)XIC(E_Stop)OTE(Run);
```

PREDICT IT · 1 `Start_PB` is on, `Stop_PB` is on and `E_Stop` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When either (`Start_PB` (Start push button, momentary) is on) *or* (`Run` (Seal-in run bit) is on), `Stop_PB` (Stop push button, momentary) is off and `E_Stop` (E-stop, NC: 1 = healthy) is on, the rung turns `Run` on (and off again whenever the rung is false).

Left to right:

1. A branch of 2 parallel paths: power gets through if any one of them is true.
   - **Path 1**
     1. `XIC(Start_PB)` *Examine If Closed* asks: true when `Start_PB` (Start push button, momentary) is on.
   - **Path 2**
     1. `XIC(Run)` *Examine If Closed* asks: true when `Run` (Seal-in run bit) is on.
2. `XIO(Stop_PB)` *Examine If Open* asks: true when `Stop_PB` (Stop push button, momentary) is off.
3. `XIC(E_Stop)` *Examine If Closed* asks: true when `E_Stop` (E-stop, NC: 1 = healthy) is on.
4. `OTE(Run)` *Output Energize* does: turns `Run` on (and off again whenever the rung is false). Every scan it writes the rung state: true on, false off.

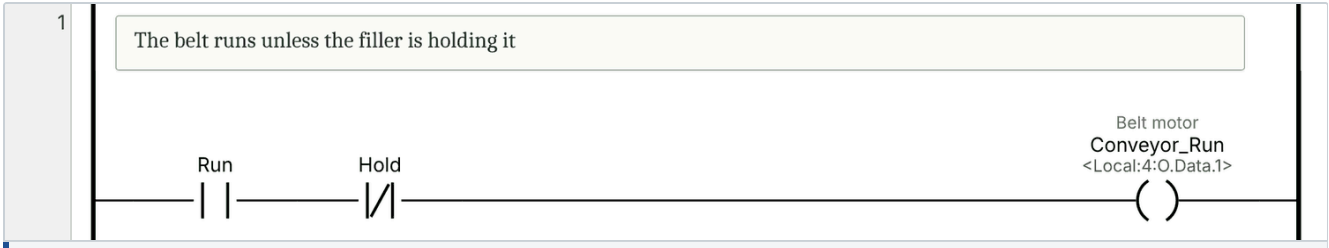
Why it is built this way:

Seal-in. The start condition turns `Run` on, and `Run` 's own contact in the parallel path keeps the rung true after the start goes away. Anything in series outside the branch (a stop, a fault) breaks it, and then it stays off until the start is made again.

> **Think of it like this:** a doorbell button and a hand. The start button is the doorbell, pressed for a moment. The seal-in contact is a hand that grabs the circuit as soon as the output comes on and keeps holding after you let go. The stop button, in series, is the only thing that can knock that hand away.

> **Watch for:** The stop has to be in series outside the branch. Put it inside the start path only and the seal-in path bypasses it: the stop no longer stops.

RUNG 1




```
XIC(Run)XIO(Hold)OTE(Conveyor_Run);
```

PREDICT IT · 2 `Run` is on and `Hold` is off. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Run` (Seal-in run bit) is on and `Hold` (The filler holds the belt) is off, the rung turns `Conveyor_Run` (Belt motor) on (and off again whenever the rung is false).

Left to right:

1. `XIC(Run)` *Examine If Closed* asks: true when `Run` (Seal-in run bit) is on.
2. `XIO(Hold)` *Examine If Open* asks: true when `Hold` (The filler holds the belt) is off.
3. `OTE(Conveyor_Run)` *Output Energize* does: turns `Conveyor_Run` (Belt motor) on (and off again whenever the rung is false).

> **Think of it like this:** a rung is a pipe with valves in it, read left to right. Every contact is a valve: XIC is open while its bit is 1, XIO while its bit is 0. Water reaches the output at the right only if every valve in a row is open. The output does not remember anything: on every scan it is simply wet or dry.

RUNG 2

```
XIC(Run)OTE(Lamp_Run);
```

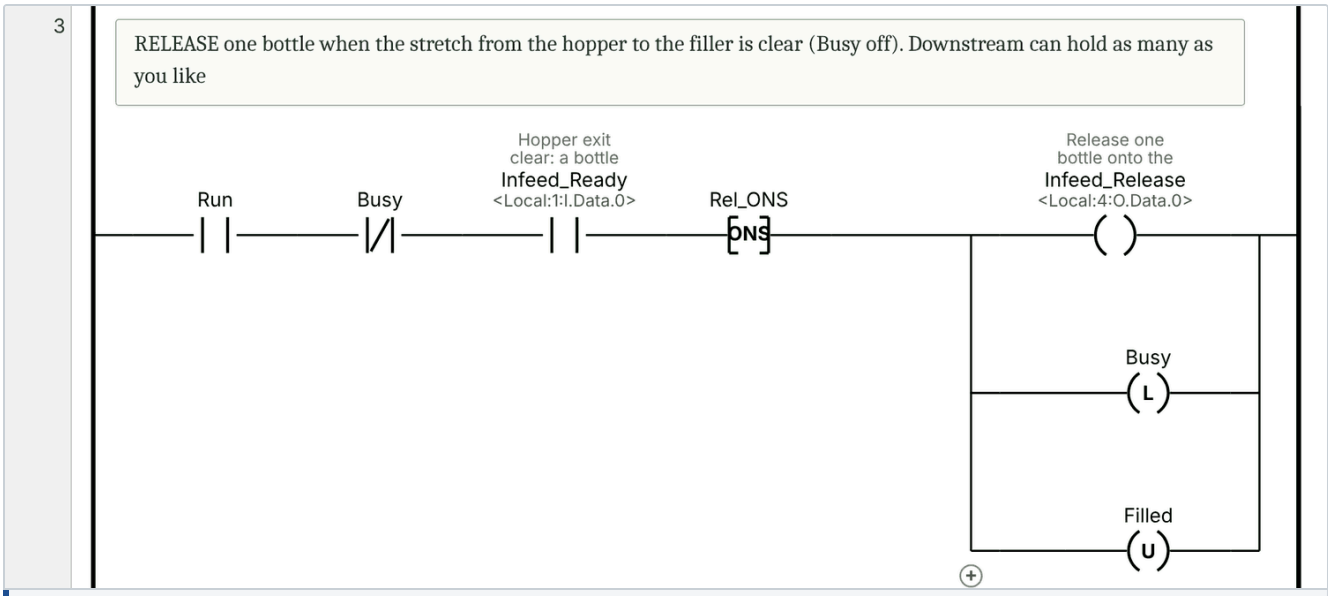
HOW THIS RUNG WORKS

In words. When `Run` (Seal-in run bit) is on, the rung turns `Lamp_Run` (Run lamp) on (and off again whenever the rung is false).

Left to right:

1. `XIC(Run)` *Examine If Closed* asks: true when `Run` (Seal-in run bit) is on.
2. `OTE(Lamp_Run)` *Output Energize* does: turns `Lamp_Run` (Run lamp) on (and off again whenever the rung is false).

RUNG 3




```
XIC(Run)XIO(Busy)XIC(Infeed_Ready)ONS(Rel_ONS)
[OTE(Infeed_Release),OTL(Busy),OTU(Filled)];
```

HOW THIS RUNG WORKS

In words. When `Run` (Seal-in run bit) is on, `Busy` (A bottle is between the hopper and the end of its cycle (Levels 1-3) / between the hopper and the filler (Levels 4-5)) is off, `Infeed_Ready` (Hopper exit clear: a bottle can be released) is on and it is the first scan they are true (the one-shot, remembered in `Rel_ONS` (One-shot storage for the release)), the rung turns `Infeed_Release` (Release one bottle onto the belt (rising edge)) on (and off again whenever the rung is false), latches `Busy` on and unlatches `Filled` (This bottle has its charge) (turns it off).

Left to right:

1. `XIC(Run)` *Examine If Closed* asks: true when `Run` (Seal-in run bit) is on.
2. `XIO(Busy)` *Examine If Open* asks: true when `Busy` (A bottle is between the hopper and the end of its cycle (Levels 1-3) / between the hopper and the filler (Levels 4-5)) is off.
3. `XIC(Infeed_Ready)` *Examine If Closed* asks: true when `Infeed_Ready` (Hopper exit clear: a bottle can be released) is on.
4. `ONS(Rel_ONS)` *One Shot* asks: true when it is the first scan they are true (the one-shot, remembered in `Rel_ONS` (One-shot storage for the release)). It passes power for one scan only, the scan its input goes from false to true; its storage bit remembers last scan.
5. A branch of 3 parallel paths: each path is tested on its own and does its own work.

   Path 1

   1. `OTE(Infeed_Release)` *Output Energize* does: turns `Infeed_Release` (Release one bottle onto the belt (rising edge)) on (and off again whenever the rung is false).

   Path 2

   1. `OTL(Busy)` *Output Latch* does: latches `Busy` on. It only ever sets the bit; the bit stays on after the rung goes false.

   Path 3

   1. `OTU(Filled)` *Output Unlatch* does: unlatches `Filled` (This bottle has its charge) (turns it off). It only ever clears the bit.

Why it is built this way:

One-shot. Everything after the ONS happens once per event, not on every scan the condition holds; `Rel_ONS` is only its memory of the last scan.

A latch. `Busy` stays on after this rung goes false; an `OTU` elsewhere in the program clears it.

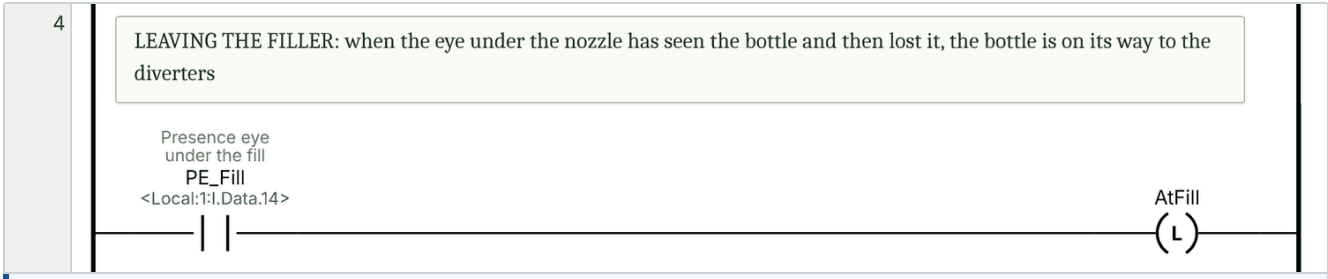
> **Think of it like this:** a doorbell that rings once however long you lean on the button. The ONS looks at what the button was doing last scan: only the change from up to down rings it.

> **Watch for:** Each ONS needs its own storage bit. Two ONS sharing one bit make one of them miss its edges.

As in Level 2, with one change in meaning: `Busy` now means *a bottle is between the hopper and the filler*, not *a bottle is somewhere on the line*.

Leaving the filler: the way out of Busy

RUNG 4




```
XIC(PE_Fill)OTL(AtFill);
```

HOW THIS RUNG WORKS

In words. When `PE_Fill` (Presence eye under the fill nozzle) is on, the rung latches `AtFill` (The eye under the nozzle has seen this bottle) on.

Left to right:

1. `XIC(PE_Fill)` *Examine If Closed* asks: true when `PE_Fill` (Presence eye under the fill nozzle) is on.
2. `OTL(AtFill)` *Output Latch* does: latches `AtFill` (The eye under the nozzle has seen this bottle) on.

Why it is built this way:

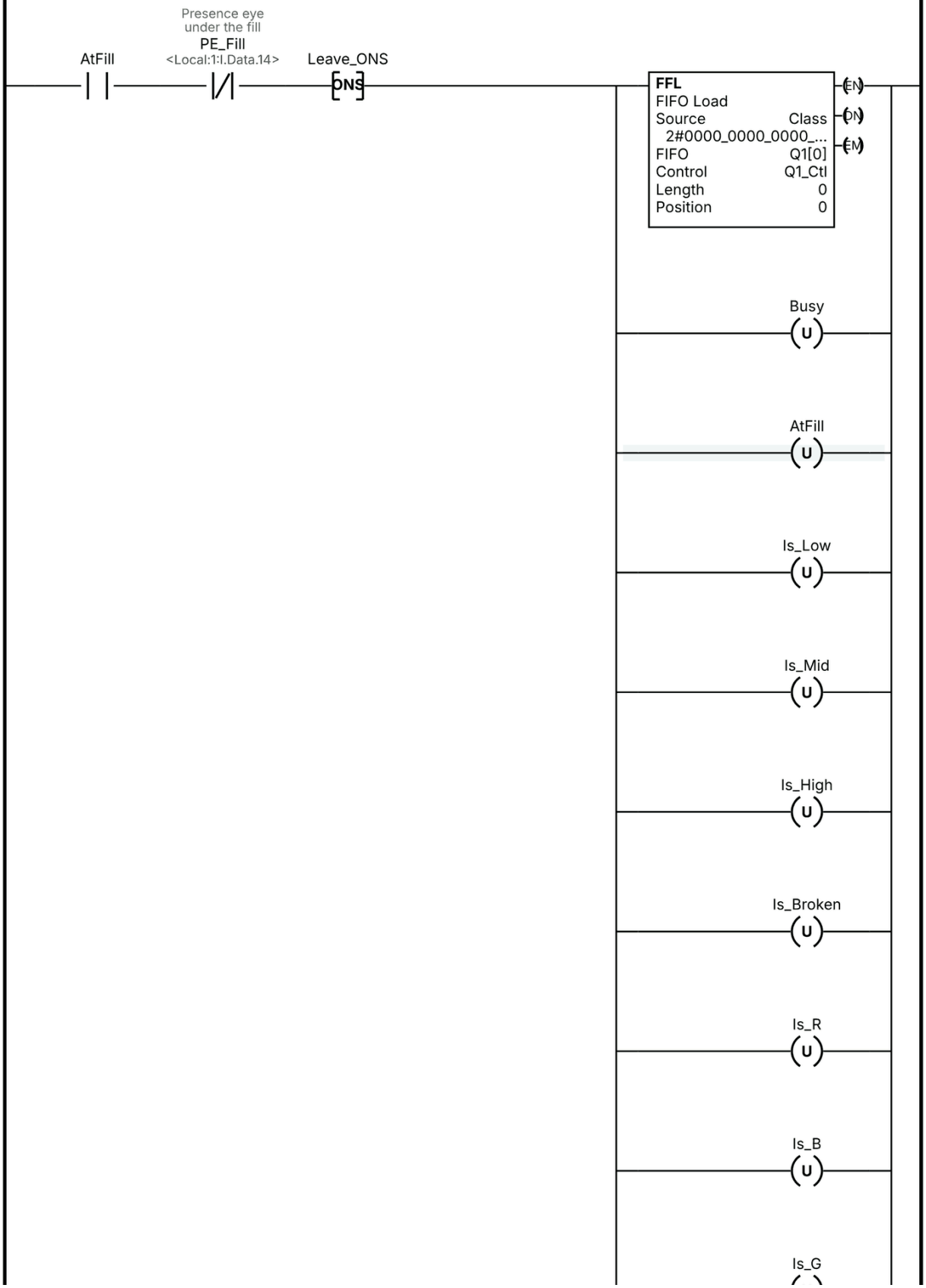
A latch: it stays on until an OTU clears it.

> **Watch for:** A latched bit survives the condition that set it: every way in needs a way out, or it stays on for good.

RUNG 5

5

Load its class into zone 1's queue, free the front of the line for the next bottle, and forget what the gantry saw



```
XIC(AtFill)XIO(PE_Fill)ONS(Leave_ONS)
[FFL(Class,Q1[0],Q1_Ctl,8),OTU(Busy),OTU(AtFill),OTU(Is_Low),OTU(Is_Mid),OTU(Is
_High),OTU(Is_Broken),OTU(Is_R),OTU(Is_B),OTU(Is_G)];
```

HOW THIS RUNG WORKS

In words. When `AtFill` (The eye under the nozzle has seen this bottle) is on, `PE_Fill` (Presence eye under the fill nozzle) is off and it is the first scan they are true (the one-shot, remembered in `Leave_ONS` (One-shot storage: the bottle left the filler)), the rung loads `Class` (The bottle in the gantry: bit 0 S, 1 M, 2 L, 3 broken, 4 red, 5 blue, 6 green) into the queue `Q1[0]`, unlatches `Busy` (A bottle is between the hopper and the end of its cycle (Levels 1-3) / between the hopper and the filler (Levels 4-5)) (turns it off), unlatches `AtFill` (turns it off), unlatches `Is_Low` (Latched: the low height beam was broken) (turns it off), unlatches `Is_Mid` (Latched: the middle beam was broken) (turns it off), unlatches `Is_High` (Latched: the high beam was broken) (turns it off), unlatches `Is_Broken` (Latched: the crack eye saw a broken bottle) (turns it off), unlatches `Is_R` (Latched: the colour sensor saw red) (turns it off), unlatches `Is_B` (Latched: the colour sensor saw blue) (turns it off) and unlatches `Is_G` (Latched: the colour sensor saw green) (turns it off).

Left to right:

1. `XIC(AtFill)` *Examine If Closed* asks: true when `AtFill` (The eye under the nozzle has seen this bottle) is on.
2. `XIO(PE_Fill)` *Examine If Open* asks: true when `PE_Fill` (Presence eye under the fill nozzle) is off.
3. `ONS(Leave_ONS)` *One Shot* asks: true when it is the first scan they are true (the one-shot, remembered in `Leave_ONS` (One-shot storage: the bottle left the filler)).
4. A branch of 10 parallel paths: each path is tested on its own and does its own work.

   Path 1

   1. `FFL(Class,Q1[0],Q1_Ctl,8)` *FIFO Load* does: loads `Class` (The bottle in the gantry: bit 0 S, 1 M, 2 L, 3 broken, 4 red, 5 blue, 6 green) into the queue `Q1[0]`. One entry per false-to-true transition.

   Path 2

   1. `OTU(Busy)` *Output Unlatch* does: unlatches `Busy` (A bottle is between the hopper and the end of its cycle (Levels 1-3) / between the hopper and the filler (Levels 4-5)) (turns it off).

   Path 3

   1. `OTU(AtFill)` *Output Unlatch* does: unlatches `AtFill` (turns it off).

   Path 4

   1. `OTU(Is_Low)` *Output Unlatch* does: unlatches `Is_Low` (Latched: the low height beam was broken) (turns it off).

   Path 5

   1. `OTU(Is_Mid)` *Output Unlatch* does: unlatches `Is_Mid` (Latched: the middle beam was broken) (turns it off).

   Path 6

   1. `OTU(Is_High)` *Output Unlatch* does: unlatches `Is_High` (Latched: the high beam was broken) (turns it off).

   Path 7

   1. `OTU(Is_Broken)` *Output Unlatch* does: unlatches `Is_Broken` (Latched: the crack eye saw a broken bottle) (turns it off).

   Path 8

   1. `OTU(Is_R)` *Output Unlatch* does: unlatches `Is_R` (Latched: the colour sensor saw red) (turns it off).

Path 9

1. `OTU(Is_B)` *Output Unlatch* does: unlatches `Is_B` (Latched: the colour sensor saw blue) (turns it off).

Path 10

1. `OTU(Is_G)` *Output Unlatch* does: unlatches `Is_G` (Latched: the colour sensor saw green) (turns it off).

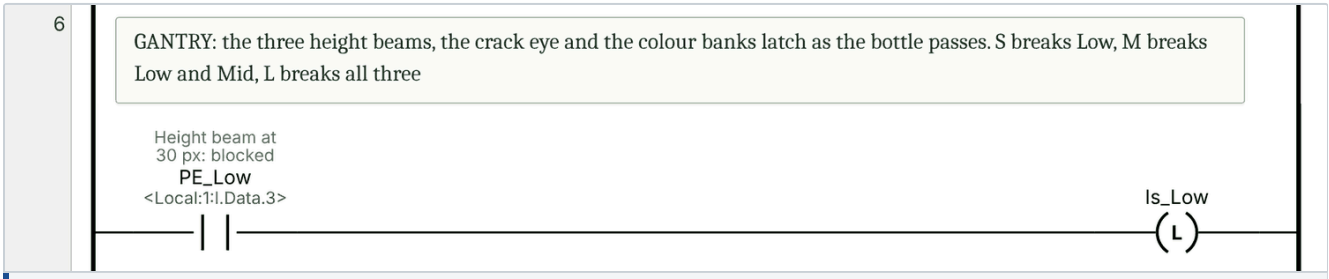
Why it is built this way:

One-shot: once per event, not once per scan.

Rung 4 latches `AtFill` when the eye under the nozzle sees the bottle. **Rung 5** fires once when that eye then loses it — the bottle has left — and does four things in one scan: loads `Class` into zone 1's queue, frees the front of the line (`OTU(Busy)`, so rung 3 releases the next bottle), clears `AtFill`, and forgets the gantry's latches. They come straight after the release because they are its way out: Busy is latched in rung 3 and freed in rung 5, so from here on the line feeds one bottle at a time and every rung you write below can be tested in Run as you write it. The order inside the branch does not matter: the FFL copies `Class` as it was at the end of the last scan, and the latches feed `Class` through rungs 13-19, which run after it. Until the diverters are in, nothing unloads the queue and it fills up; use Tools → Clear Tag Values before you run with the diverters, so they start on an empty queue.

The gantry and the class word

RUNG 6




```
XIC(PE_Low)OTL(Is_Low);
```

HOW THIS RUNG WORKS

In words. When `PE_Low` (Height beam at 30 px: blocked by every size) is on, the rung latches `Is_Low` (Latched: the low height beam was broken) on.

Left to right:

1. `XIC(PE_Low)` *Examine If Closed* asks: true when `PE_Low` (Height beam at 30 px: blocked by every size) is on.
2. `OTL(Is_Low)` *Output Latch* does: latches `Is_Low` (Latched: the low height beam was broken) on.

Why it is built this way:

A latch: it stays on until an OTU clears it.

RUNG 7

```
XIC(PE_Mid)OTL(Is_Mid);
```

HOW THIS RUNG WORKS

In words. When `PE_Mid` (Height beam at 44 px: blocked by M and L) is on, the rung latches `Is_Mid` (Latched: the middle beam was broken) on.

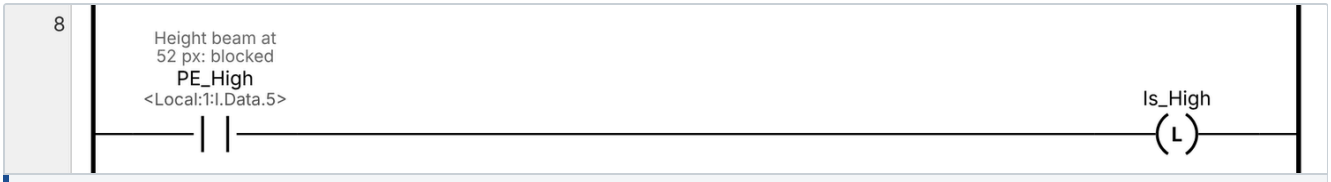
Left to right:

1. `XIC(PE_Mid)` *Examine If Closed* asks: true when `PE_Mid` (Height beam at 44 px: blocked by M and L) is on.
2. `OTL(Is_Mid)` *Output Latch* does: latches `Is_Mid` (Latched: the middle beam was broken) on.

Why it is built this way:

A latch: it stays on until an OTU clears it.

RUNG 8




```
XIC(PE_High)OTL(Is_High);
```

HOW THIS RUNG WORKS

In words. When `PE_High` (Height beam at 52 px: blocked by L only) is on, the rung latches `Is_High` (Latched: the high beam was broken) on.

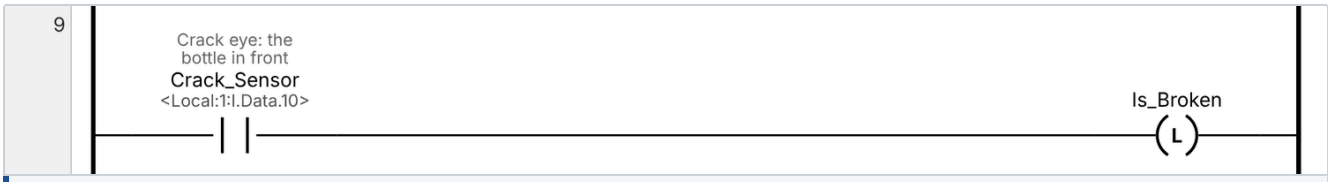
Left to right:

1. `XIC(PE_High)` *Examine If Closed* asks: true when `PE_High` (Height beam at 52 px: blocked by L only) is on.
2. `OTL(Is_High)` *Output Latch* does: latches `Is_High` (Latched: the high beam was broken) on.

Why it is built this way:

A latch: it stays on until an OTU clears it.

RUNG 9




```
XIC(Crack_Sensor)OTL(Is_Broken);
```

HOW THIS RUNG WORKS

In words. When `Crack_Sensor` (Crack eye: the bottle in front of it is broken) is on, the rung latches `Is_Broken` (Latched: the crack eye saw a broken bottle) on.

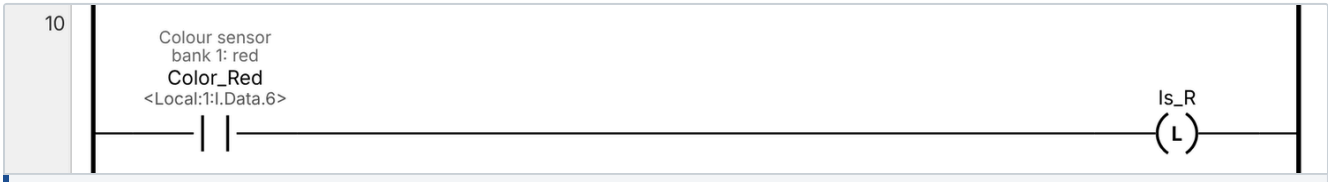
Left to right:

1. `XIC(Crack_Sensor)` *Examine If Closed* asks: true when `Crack_Sensor` (Crack eye: the bottle in front of it is broken) is on.
2. `OTL(Is_Broken)` *Output Latch* does: latches `Is_Broken` (Latched: the crack eye saw a broken bottle) on.

Why it is built this way:

A latch: it stays on until an OTU clears it.

RUNG 10




```
XIC(Color_Red)OTL(Is_R);
```

HOW THIS RUNG WORKS

In words. When `Color_Red` (Colour sensor bank 1: red) is on, the rung latches `Is_R` (Latched: the colour sensor saw red) on.

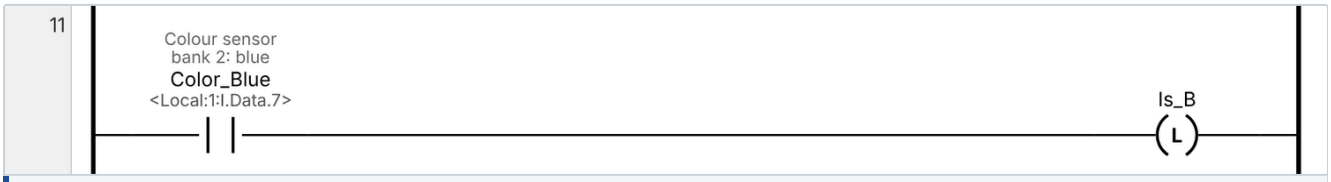
Left to right:

1. `XIC(Color_Red)` *Examine If Closed* asks: true when `Color_Red` (Colour sensor bank 1: red) is on.
2. `OTL(Is_R)` *Output Latch* does: latches `Is_R` (Latched: the colour sensor saw red) on.

Why it is built this way:

A latch: it stays on until an OTU clears it.

RUNG 11




```
XIC(Color_Blue)OTL(Is_B);
```

HOW THIS RUNG WORKS

In words. When `Color_Blue` (Colour sensor bank 2: blue) is on, the rung latches `Is_B` (Latched: the colour sensor saw blue) on.

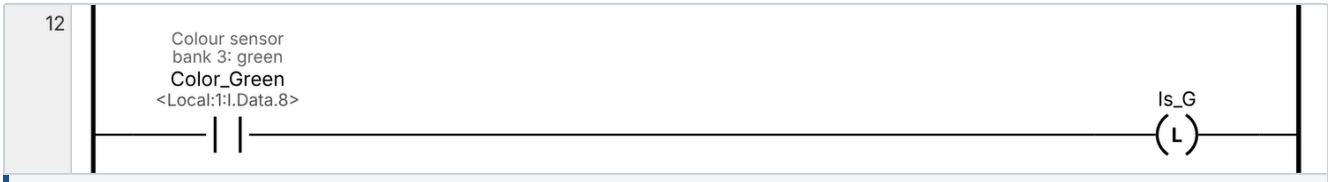
Left to right:

1. `XIC(Color_Blue)` *Examine If Closed* asks: true when `Color_Blue` (Colour sensor bank 2: blue) is on.
2. `OTL(Is_B)` *Output Latch* does: latches `Is_B` (Latched: the colour sensor saw blue) on.

Why it is built this way:

A latch: it stays on until an OTU clears it.

RUNG 12




```
XIC(Color_Green)OTL(Is_G);
```

HOW THIS RUNG WORKS

In words. When `Color_Green` (Colour sensor bank 3: green) is on, the rung latches `Is_G` (Latched: the colour sensor saw green) on.

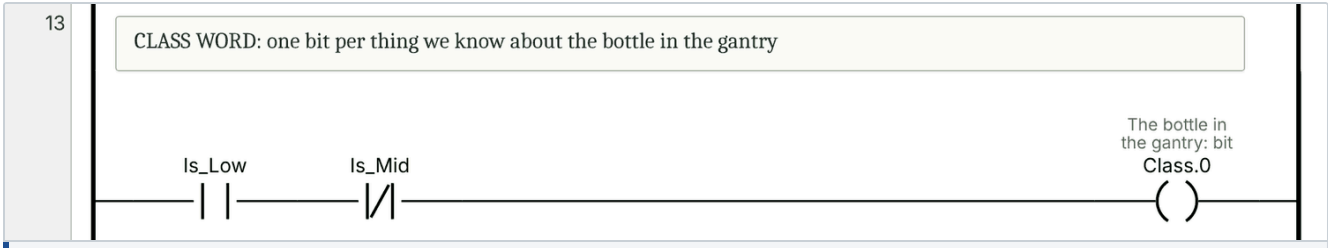
Left to right:

1. `XIC(Color_Green)` *Examine If Closed* asks: true when `Color_Green` (Colour sensor bank 3: green) is on.
2. `OTL(Is_G)` *Output Latch* does: latches `Is_G` (Latched: the colour sensor saw green) on.

Why it is built this way:

A latch: it stays on until an OTU clears it.

RUNG 13




```
XIC(Is_Low)XIO(Is_Mid)OTE(Class.0);
```

PREDICT IT · 3 `Is_Low` is on and `Is_Mid` is off. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Is_Low` (Latched: the low height beam was broken) is on and `Is_Mid` (Latched: the middle beam was broken) is off, the rung turns `Class.0` on (and off again whenever the rung is false).

Left to right:

1. `XIC(Is_Low)` *Examine If Closed* asks: true when `Is_Low` (Latched: the low height beam was broken) is on.
2. `XIO(Is_Mid)` *Examine If Open* asks: true when `Is_Mid` (Latched: the middle beam was broken) is off.
3. `OTE(Class.0)` *Output Energize* does: turns `Class.0` on (and off again whenever the rung is false).

RUNG 14

```
XIC(Is_Mid)XIO(Is_High)OTE(Class.1);
```

PREDICT IT · 4 `Is_Mid` is on and `Is_High` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

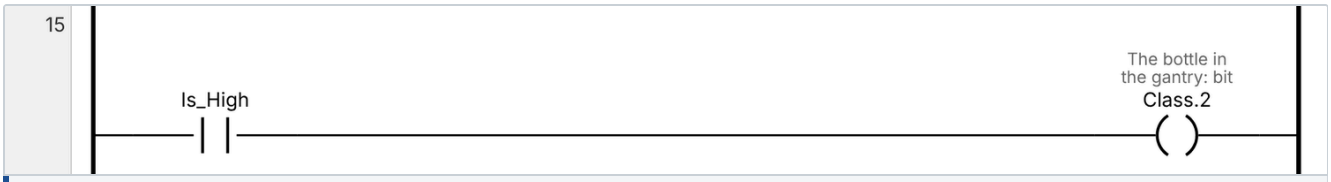
HOW THIS RUNG WORKS

In words. When `Is_Mid` (Latched: the middle beam was broken) is on and `Is_High` (Latched: the high beam was broken) is off, the rung turns `Class.1` on (and off again whenever the rung is false).

Left to right:

1. `XIC(Is_Mid)` *Examine If Closed* asks: true when `Is_Mid` (Latched: the middle beam was broken) is on.
2. `XIO(Is_High)` *Examine If Open* asks: true when `Is_High` (Latched: the high beam was broken) is off.
3. `OTE(Class.1)` *Output Energize* does: turns `Class.1` on (and off again whenever the rung is false).

RUNG 15




```
XIC(Is_High)OTE(Class.2);
```

HOW THIS RUNG WORKS

In words. When `Is_High` (Latched: the high beam was broken) is on, the rung turns `Class.2` on (and off again whenever the rung is false).

Left to right:

1. `XIC(Is_High)` *Examine If Closed* asks: true when `Is_High` (Latched: the high beam was broken) is on.
2. `OTE(Class.2)` *Output Energize* does: turns `Class.2` on (and off again whenever the rung is false).

RUNG 16

```
XIC(Is_Broken)OTE(Class.3);
```

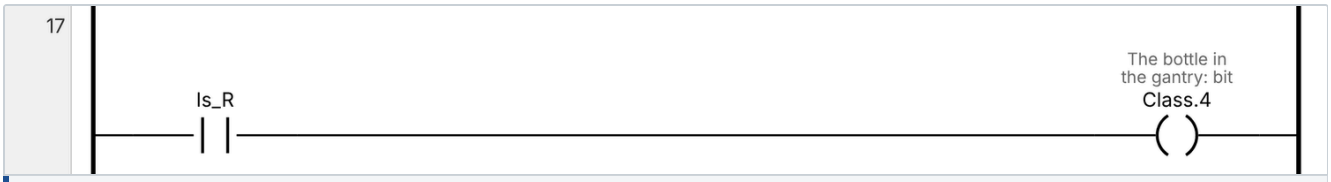
HOW THIS RUNG WORKS

In words. When `Is_Broken` (Latched: the crack eye saw a broken bottle) is on, the rung turns `Class.3` on (and off again whenever the rung is false).

Left to right:

1. `XIC(Is_Broken)` *Examine If Closed* asks: true when `Is_Broken` (Latched: the crack eye saw a broken bottle) is on.
2. `OTE(Class.3)` *Output Energize* does: turns `Class.3` on (and off again whenever the rung is false).

RUNG 17




```
XIC(Is_R)OTE(Class.4);
```

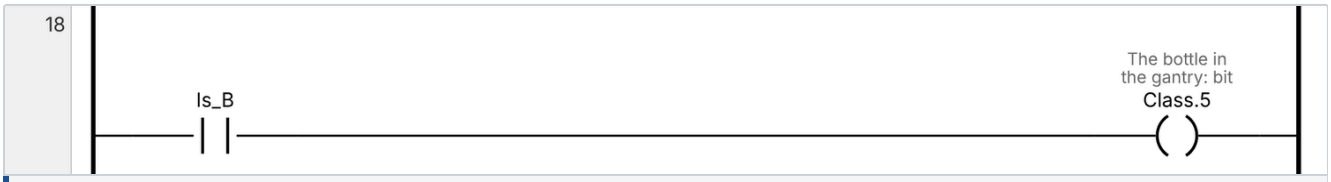
HOW THIS RUNG WORKS

In words. When `Is_R` (Latched: the colour sensor saw red) is on, the rung turns `Class.4` on (and off again whenever the rung is false).

Left to right:

1. `XIC(Is_R)` *Examine If Closed* asks: true when `Is_R` (Latched: the colour sensor saw red) is on.
2. `OTE(Class.4)` *Output Energize* does: turns `Class.4` on (and off again whenever the rung is false).

RUNG 18




```
XIC(Is_B)OTE(Class.5);
```

HOW THIS RUNG WORKS

In words. When `Is_B` (Latched: the colour sensor saw blue) is on, the rung turns `Class.5` on (and off again whenever the rung is false).

Left to right:

1. `XIC(Is_B)` *Examine If Closed* asks: true when `Is_B` (Latched: the colour sensor saw blue) is on.
2. `OTE(Class.5)` *Output Energize* does: turns `Class.5` on (and off again whenever the rung is false).

RUNG 19

```
XIC(Is_G)OTE(Class.6);
```

HOW THIS RUNG WORKS

In words. When `Is_G` (Latched: the colour sensor saw green) is on, the rung turns `Class.6` on (and off again whenever the rung is false).

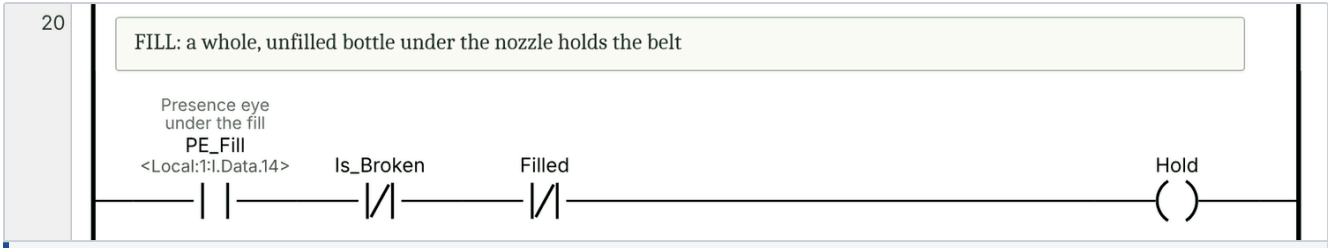
Left to right:

1. `XIC(Is_G)` *Examine If Closed* asks: true when `Is_G` (Latched: the colour sensor saw green) is on.
2. `OTE(Class.6)` *Output Energize* does: turns `Class.6` on (and off again whenever the rung is false).

The seven latches are Level 1's and Level 3's together (the colour bits are not needed on this level; they are there because this is the one program for all five, and they cost nothing). **Rungs 13-19** fold them into the bits of the DINT `Class` with OTEs — a tag used a bit at a time, chapter 5. `Class` is Binary style, so Monitor Tags shows the bits.

The fill

RUNG 20




```
XIC(PE_Fill)XIO(Is_Broken)XIO(Filled)OTE(Hold);
```

PREDICT IT · 5 `PE_Fill` is on, `Is_Broken` is on and `Filled` is off. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

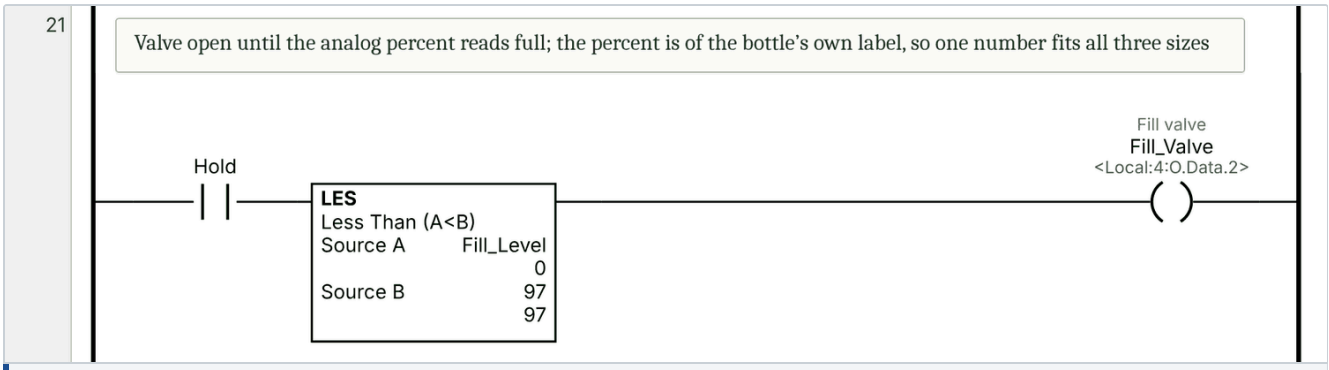
HOW THIS RUNG WORKS

In words. When `PE_Fill` (Presence eye under the fill nozzle) is on, `Is_Broken` (Latched: the crack eye saw a broken bottle) is off and `Filled` (This bottle has its charge) is off, the rung turns `Hold` (The filler holds the belt) on (and off again whenever the rung is false).

Left to right:

1. `XIC(PE_Fill)` *Examine If Closed* asks: true when `PE_Fill` (Presence eye under the fill nozzle) is on.
2. `XIO(Is_Broken)` *Examine If Open* asks: true when `Is_Broken` (Latched: the crack eye saw a broken bottle) is off.
3. `XIO(Filled)` *Examine If Open* asks: true when `Filled` (This bottle has its charge) is off.
4. `OTE(Hold)` *Output Energize* does: turns `Hold` (The filler holds the belt) on (and off again whenever the rung is false).

RUNG 21




```
XIC(Hold)LES(Fill_Level,97)OTE(Fill_Valve);
```

PREDICT IT · 6 `Hold` is on and `Fill_Level` is 96. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

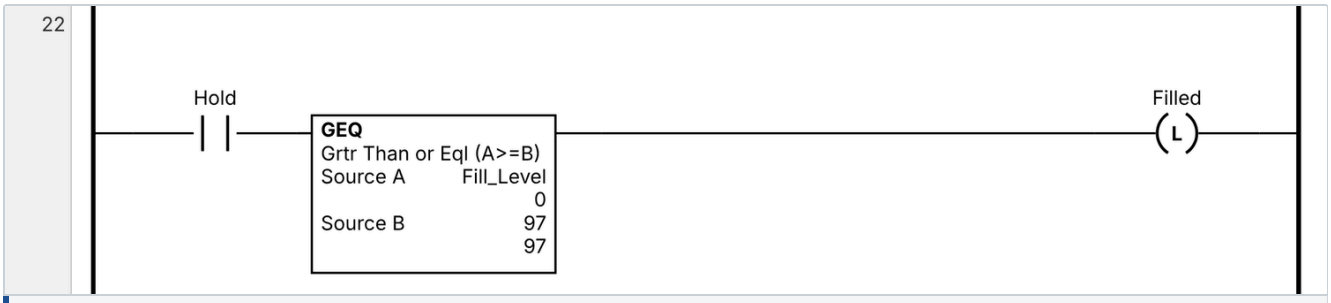
HOW THIS RUNG WORKS

In words. When `Hold` (The filler holds the belt) is on and `Fill_Level` (Fill of the bottle under the nozzle, 0-100 % of its label) is less than **97**, the rung turns `Fill_Valve` (Fill valve) on (and off again whenever the rung is false).

Left to right:

1. `XIC(Hold)` *Examine If Closed* asks: true when `Hold` (The filler holds the belt) is on.
2. `LES(Fill_Level,97)` *Less Than* asks: true when `Fill_Level` (Fill of the bottle under the nozzle, 0-100 % of its label) is less than **97**.
3. `OTE(Fill_Valve)` *Output Energize* does: turns `Fill_Valve` (Fill valve) on (and off again whenever the rung is false).

RUNG 22




```
XIC(Hold)GEQ(Fill_Level,97)OTL(Filled);
```

PREDICT IT · 7 `Hold` is on and `Fill_Level` is 96. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Hold` (The filler holds the belt) is on and `Fill_Level` (Fill of the bottle under the nozzle, 0-100 % of its label) is at least **97**, the rung latches `Filled` (This bottle has its charge) on.

Left to right:

1. `XIC(Hold)` *Examine If Closed* asks: true when `Hold` (The filler holds the belt) is on.
2. `GEQ(Fill_Level,97)` *Greater Than or Equal* asks: true when `Fill_Level` (Fill of the bottle under the nozzle, 0-100 % of its label) is at least **97**.
3. `OTL(Filled)` *Output Latch* does: latches `Filled` (This bottle has its charge) on.

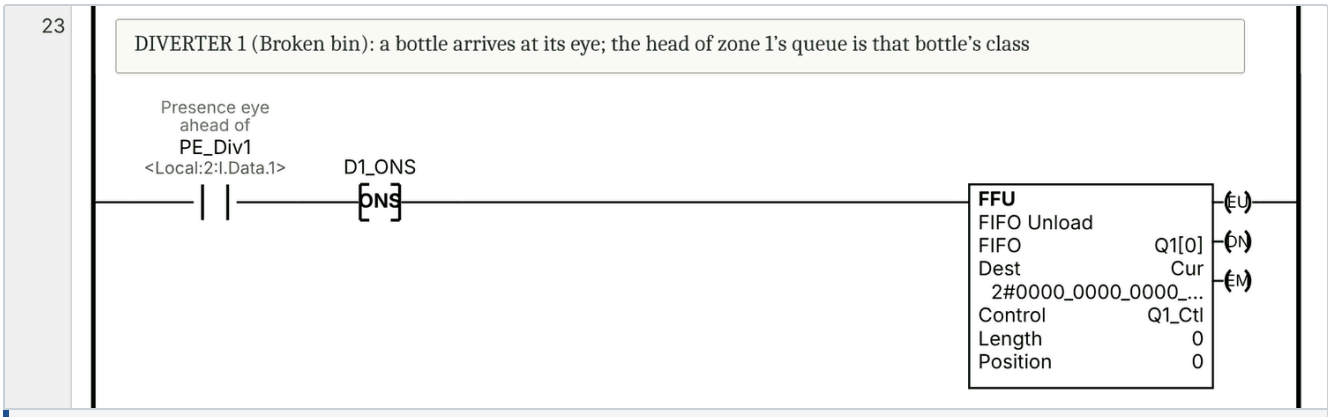
Why it is built this way:

A latch: it stays on until an OTU clears it.

Level 2's fill, unchanged.

Diverter 1

RUNG 23




```
XIC(PE_Div1)ONS(D1_ONS)FFU(Q1[0],Cur,Q1_Ctl,8);
```

HOW THIS RUNG WORKS

In words. When `PE_Div1` (Presence eye ahead of diverter 1) is on and it is the first scan they are true (the one-shot, remembered in `D1_ONS`), the rung unloads the oldest entry of `Q1[0]` into `Cur` (The class unloaded from a zone queue at a diverter).

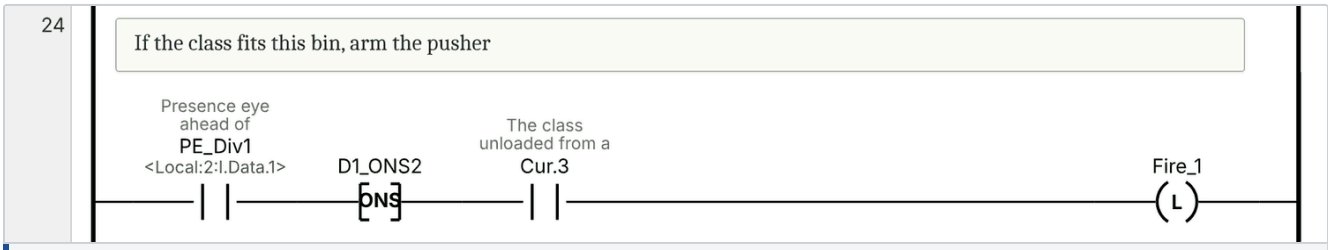
Left to right:

1. `XIC(PE_Div1)` *Examine If Closed* asks: true when `PE_Div1` (Presence eye ahead of diverter 1) is on.
2. `ONS(D1_ONS)` *One Shot* asks: true when it is the first scan they are true (the one-shot, remembered in `D1_ONS`).
3. `FFU(Q1[0],Cur,Q1_Ctl,8)` *FIFO Unload* does: unloads the oldest entry of `Q1[0]` into `Cur` (The class unloaded from a zone queue at a diverter). One entry per false-to-true transition.

Why it is built this way:

One-shot: once per event, not once per scan.

RUNG 24




```
XIC(PE_Div1)ONS(D1_ONS2)XIC(Cur.3)OTL(Fire_1);
```

HOW THIS RUNG WORKS

In words. When `PE_Div1` (Presence eye ahead of diverter 1) is on, it is the first scan they are true (the one-shot, remembered in `D1_ONS2`) and `Cur.3` is on, the rung latches `Fire_1` on.

Left to right:

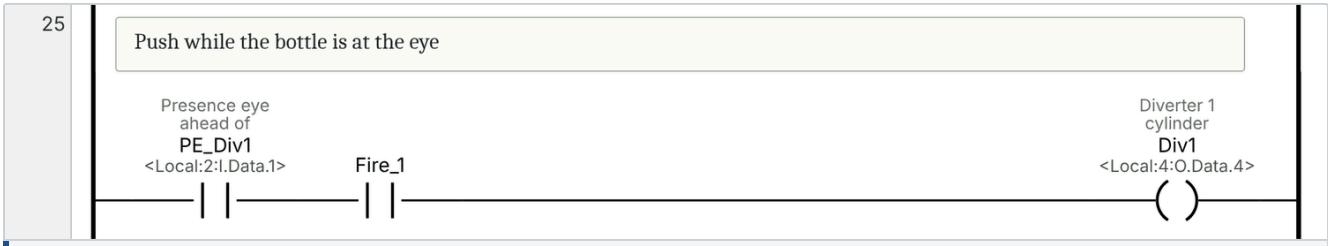
1. `XIC(PE_Div1)` *Examine If Closed* asks: true when `PE_Div1` (Presence eye ahead of diverter 1) is on.
2. `ONS(D1_ONS2)` *One Shot* asks: true when it is the first scan they are true (the one-shot, remembered in `D1_ONS2`).
3. `XIC(Cur.3)` *Examine If Closed* asks: true when `Cur.3` is on.
4. `OTL(Fire_1)` *Output Latch* does: latches `Fire_1` on.

Why it is built this way:

One-shot: once per event, not once per scan.

A latch: it stays on until an OTU clears it.

RUNG 25




```
XIC(PE_Div1)XIC(Fire_1)OTE(Div1);
```

PREDICT IT · 8 `PE_Div1` is on and `Fire_1` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

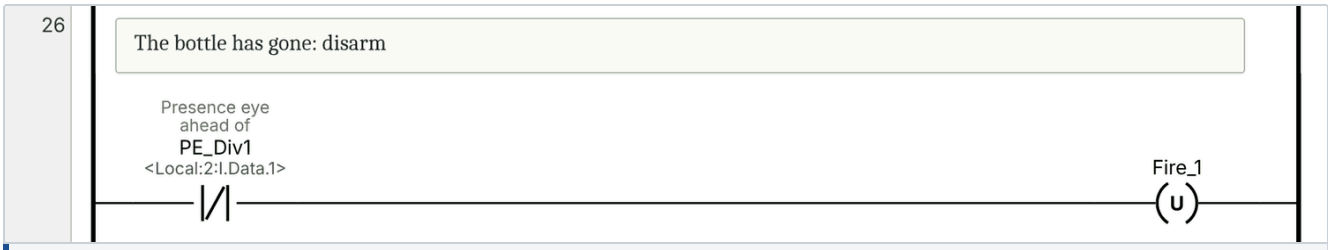
HOW THIS RUNG WORKS

In words. When `PE_Div1` (Presence eye ahead of diverter 1) is on and `Fire_1` is on, the rung turns `Div1` (Diverter 1 cylinder) on (and off again whenever the rung is false).

Left to right:

1. `XIC(PE_Div1)` *Examine If Closed* asks: true when `PE_Div1` (Presence eye ahead of diverter 1) is on.
2. `XIC(Fire_1)` *Examine If Closed* asks: true when `Fire_1` is on.
3. `OTE(Div1)` *Output Energize* does: turns `Div1` (Diverter 1 cylinder) on (and off again whenever the rung is false).

RUNG 26




```
XIO(PE_Div1)OTU(Fire_1);
```

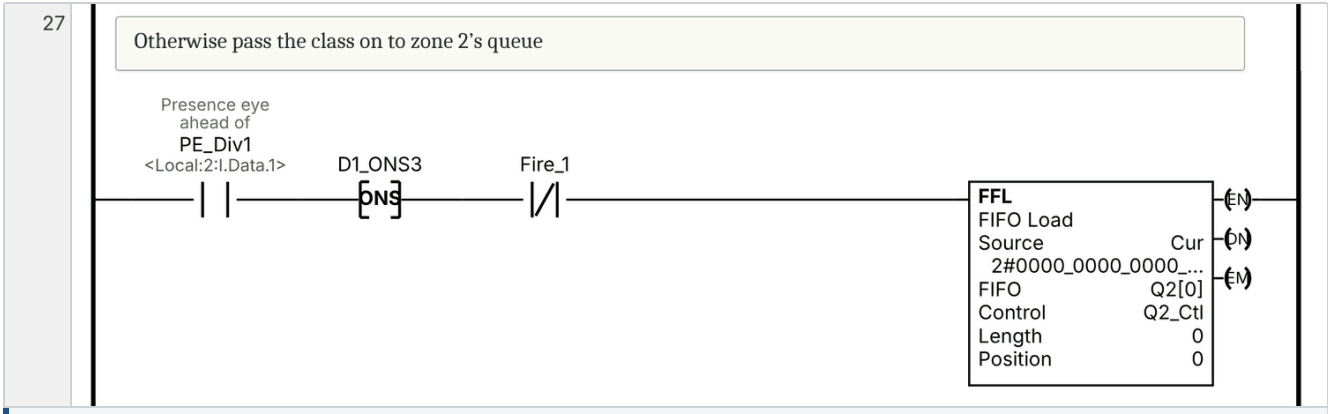
HOW THIS RUNG WORKS

In words. When `PE_Div1` (Presence eye ahead of diverter 1) is off, the rung unlatches `Fire_1` (turns it off).

Left to right:

1. `XIO(PE_Div1)` *Examine If Open* asks: true when `PE_Div1` (Presence eye ahead of diverter 1) is off.
2. `OTU(Fire_1)` *Output Unlatch* does: unlatches `Fire_1` (turns it off).

RUNG 27




```
XIC(PE_Div1)ONS(D1_ONS3)XIO(Fire_1)FFL(Cur,Q2[0],Q2_Ctl,8);
```

HOW THIS RUNG WORKS

In words. When `PE_Div1` (Presence eye ahead of diverter 1) is on, it is the first scan they are true (the one-shot, remembered in `D1_ONS3`) and `Fire_1` is off, the rung loads `Cur` (The class unloaded from a zone queue at a diverter) into the queue `Q2[0]`.

Left to right:

1. `XIC(PE_Div1)` *Examine If Closed* asks: true when `PE_Div1` (Presence eye ahead of diverter 1) is on.
2. `ONS(D1_ONS3)` *One Shot* asks: true when it is the first scan they are true (the one-shot, remembered in `D1_ONS3`).
3. `XIO(Fire_1)` *Examine If Open* asks: true when `Fire_1` is off.
4. `FFL(Cur,Q2[0],Q2_Ctl,8)` *FIFO Load* does: loads `Cur` (The class unloaded from a zone queue at a diverter) into the queue `Q2[0]`.

Why it is built this way:

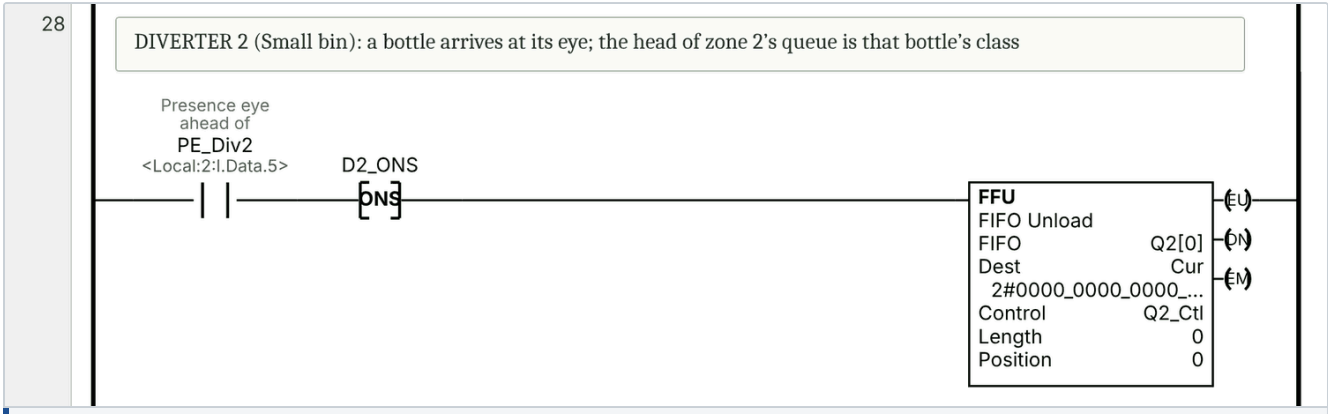
One-shot: once per event, not once per scan.

Five rungs per diverter, and the same five for every lane. **Rung 23**: a bottle arrives at the eye — unload the head of zone 1's queue into `Cur`. **Rung 24**: if `Cur` fits this bin (here: bit 3, broken), arm the pusher. **Rung 25**: push while the bottle is at the eye and the pusher is armed. **Rung 26**: the bottle has gone, disarm. **Rung 27**: if it did not fit, pass the class on to zone 2's queue.

The three one-shots on the same eye (`D1_ONS`, `D1_ONS2`, `D1_ONS3`) all fire on the same scan, in rung order — that is the scan of chapter 4 doing the work. Rung 23 unloads, so `Cur` is fresh when rung 24 tests it; rung 24 arms, so when rung 27 asks `XIO(Fire_1)` it already knows the answer (rung 26 cannot disarm in between: it waits for the eye to clear, and the eye is still made). The pass-on rung comes last so that it works the moment it is written: written before the push and the disarm, a broken bottle would leave `Fire_1` armed for good and the pass-on rung would never see it off. That is how *otherwise* is written without repeating the match in reverse. Each needs its own storage bit: a one-shot remembers last scan's rung state, and three rungs sharing one bit would see each other's.

Diverters 2, 3 and 4

RUNG 28




```
XIC(PE_Div2)ONS(D2_ONS)FFU(Q2[0],Cur,Q2_Ctl,8);
```

HOW THIS RUNG WORKS

In words. When `PE_Div2` (Presence eye ahead of diverter 2) is on and it is the first scan they are true (the one-shot, remembered in `D2_ONS`), the rung unloads the oldest entry of `Q2[0]` into `Cur` (The class unloaded from a zone queue at a diverter).

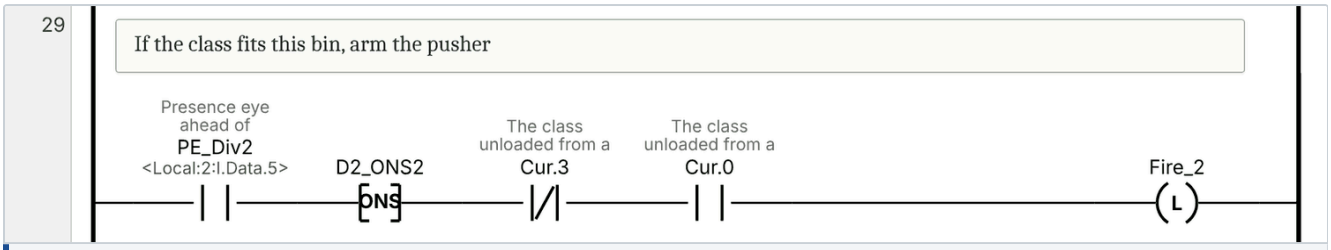
Left to right:

1. `XIC(PE_Div2)` *Examine If Closed* asks: true when `PE_Div2` (Presence eye ahead of diverter 2) is on.
2. `ONS(D2_ONS)` *One Shot* asks: true when it is the first scan they are true (the one-shot, remembered in `D2_ONS`).
3. `FFU(Q2[0],Cur,Q2_Ctl,8)` *FIFO Unload* does: unloads the oldest entry of `Q2[0]` into `Cur` (The class unloaded from a zone queue at a diverter).

Why it is built this way:

One-shot: once per event, not once per scan.

RUNG 29




```
XIC(PE_Div2)ONS(D2_ONS2)XIO(Cur.3)XIC(Cur.0)OTL(Fire_2);
```

HOW THIS RUNG WORKS

In words. When `PE_Div2` (Presence eye ahead of diverter 2) is on, it is the first scan they are true (the one-shot, remembered in `D2_ONS2`), `Cur.3` is off and `Cur.0` is on, the rung latches `Fire_2` on.

Left to right:

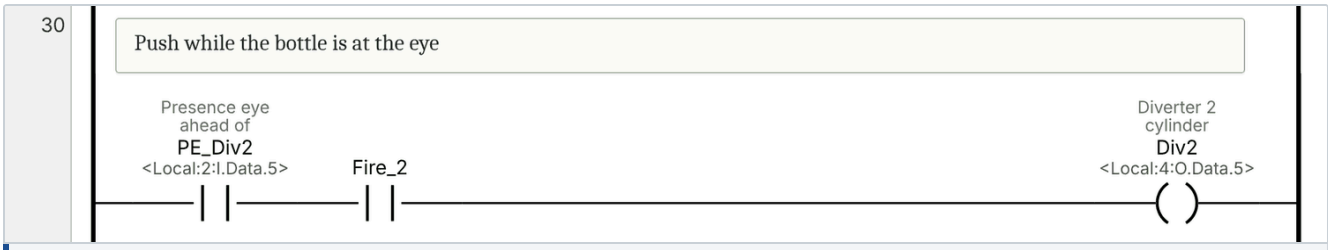
1. `XIC(PE_Div2)` *Examine If Closed* asks: true when `PE_Div2` (Presence eye ahead of diverter 2) is on.
2. `ONS(D2_ONS2)` *One Shot* asks: true when it is the first scan they are true (the one-shot, remembered in `D2_ONS2`).
3. `XIO(Cur.3)` *Examine If Open* asks: true when `Cur.3` is off.
4. `XIC(Cur.0)` *Examine If Closed* asks: true when `Cur.0` is on.
5. `OTL(Fire_2)` *Output Latch* does: latches `Fire_2` on.

Why it is built this way:

One-shot: once per event, not once per scan.

A latch: it stays on until an OTU clears it.

RUNG 30




```
XIC(PE_Div2)XIC(Fire_2)OTE(Div2);
```

PREDICT IT · 9 `PE_Div2` is on and `Fire_2` is off. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

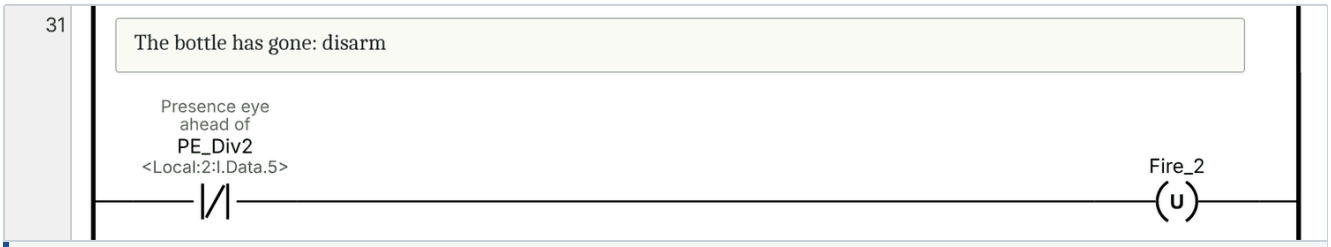
HOW THIS RUNG WORKS

In words. When `PE_Div2` (Presence eye ahead of diverter 2) is on and `Fire_2` is on, the rung turns `Div2` (Diverter 2 cylinder) on (and off again whenever the rung is false).

Left to right:

1. `XIC(PE_Div2)` *Examine If Closed* asks: true when `PE_Div2` (Presence eye ahead of diverter 2) is on.
2. `XIC(Fire_2)` *Examine If Closed* asks: true when `Fire_2` is on.
3. `OTE(Div2)` *Output Energize* does: turns `Div2` (Diverter 2 cylinder) on (and off again whenever the rung is false).

RUNG 31




```
XIO(PE_Div2)OTU(Fire_2);
```

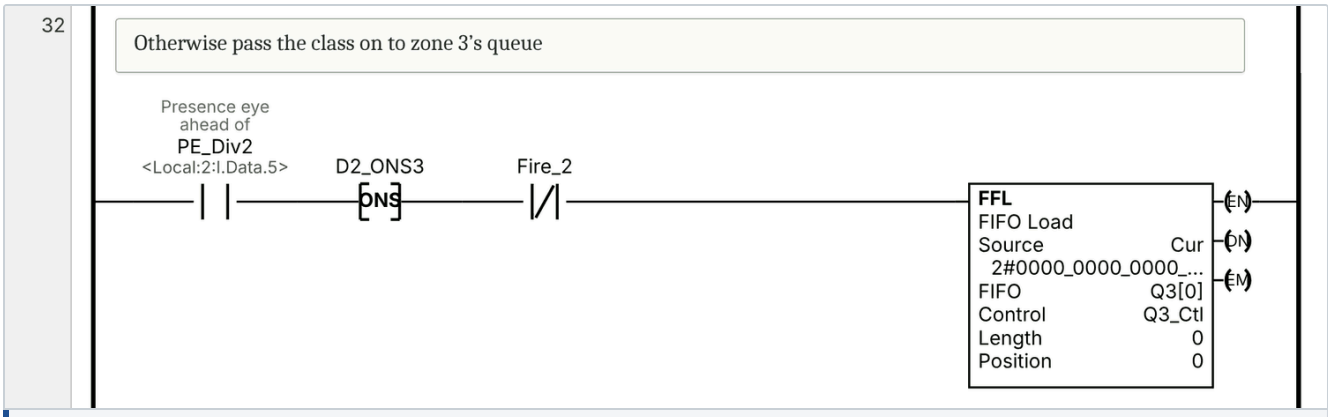
HOW THIS RUNG WORKS

In words. When `PE_Div2` (Presence eye ahead of diverter 2) is off, the rung unlatches `Fire_2` (turns it off).

Left to right:

1. `XIO(PE_Div2)` *Examine If Open* asks: true when `PE_Div2` (Presence eye ahead of diverter 2) is off.
2. `OTU(Fire_2)` *Output Unlatch* does: unlatches `Fire_2` (turns it off).

RUNG 32




```
XIC(PE_Div2)ONS(D2_ONS3)XIO(Fire_2)FFL(Cur,Q3[0],Q3_Ctl,8);
```

HOW THIS RUNG WORKS

In words. When `PE_Div2` (Presence eye ahead of diverter 2) is on, it is the first scan they are true (the one-shot, remembered in `D2_ONS3`) and `Fire_2` is off, the rung loads `Cur` (The class unloaded from a zone queue at a diverter) into the queue `Q3[0]`.

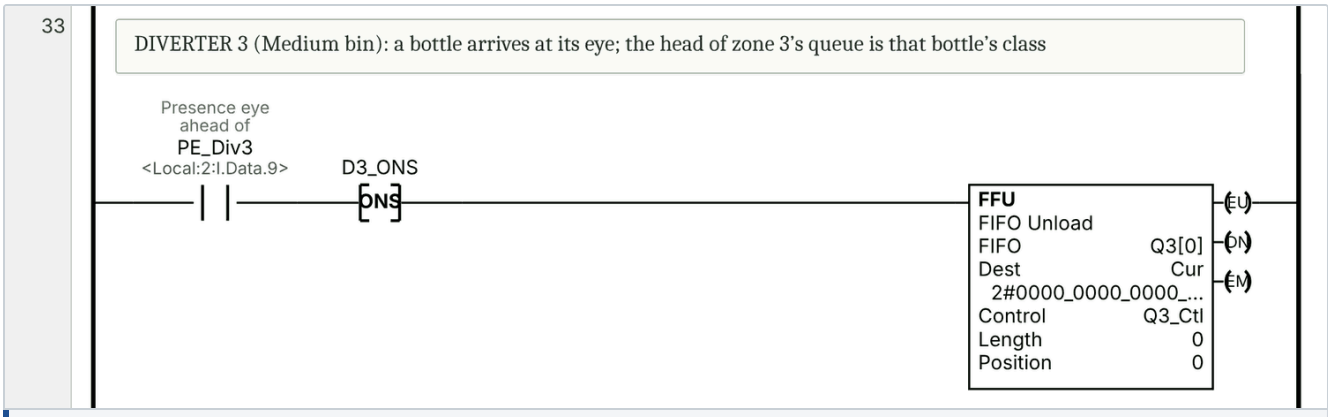
Left to right:

1. `XIC(PE_Div2)` *Examine If Closed* asks: true when `PE_Div2` (Presence eye ahead of diverter 2) is on.
2. `ONS(D2_ONS3)` *One Shot* asks: true when it is the first scan they are true (the one-shot, remembered in `D2_ONS3`).
3. `XIO(Fire_2)` *Examine If Open* asks: true when `Fire_2` is off.
4. `FFL(Cur,Q3[0],Q3_Ctl,8)` *FIFO Load* does: loads `Cur` (The class unloaded from a zone queue at a diverter) into the queue `Q3[0]`.

Why it is built this way:

One-shot: once per event, not once per scan.

RUNG 33




```
XIC(PE_Div3)ONS(D3_ONS)FFU(Q3[0],Cur,Q3_Ctl,8);
```

HOW THIS RUNG WORKS

In words. When `PE_Div3` (Presence eye ahead of diverter 3) is on and it is the first scan they are true (the one-shot, remembered in `D3_ONS`), the rung unloads the oldest entry of `Q3[0]` into `Cur` (The class unloaded from a zone queue at a diverter).

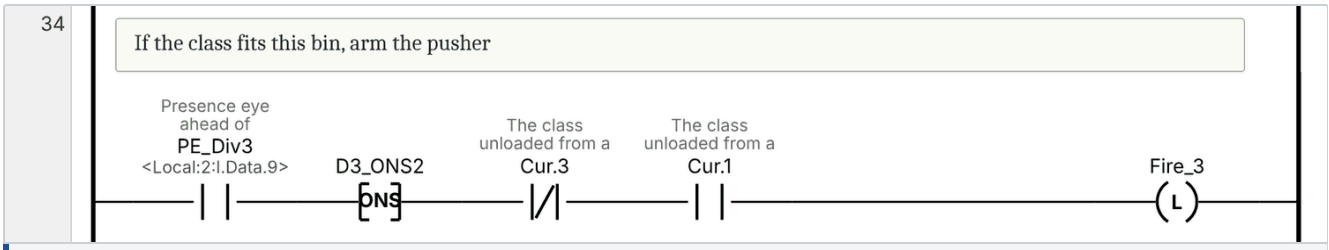
Left to right:

1. `XIC(PE_Div3)` *Examine If Closed* asks: true when `PE_Div3` (Presence eye ahead of diverter 3) is on.
2. `ONS(D3_ONS)` *One Shot* asks: true when it is the first scan they are true (the one-shot, remembered in `D3_ONS`).
3. `FFU(Q3[0],Cur,Q3_Ctl,8)` *FIFO Unload* does: unloads the oldest entry of `Q3[0]` into `Cur` (The class unloaded from a zone queue at a diverter).

Why it is built this way:

One-shot: once per event, not once per scan.

RUNG 34




```
XIC(PE_Div3)ONS(D3_ONS2)XIO(Cur.3)XIC(Cur.1)OTL(Fire_3);
```

HOW THIS RUNG WORKS

In words. When `PE_Div3` (Presence eye ahead of diverter 3) is on, it is the first scan they are true (the one-shot, remembered in `D3_ONS2`), `Cur.3` is off and `Cur.1` is on, the rung latches `Fire_3` on.

Left to right:

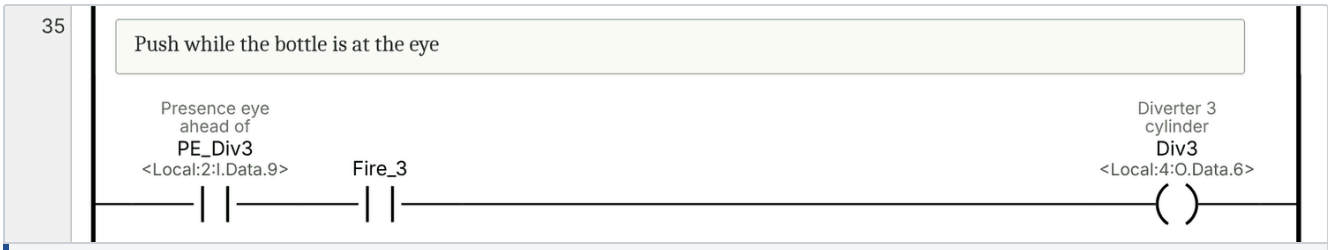
1. `XIC(PE_Div3)` *Examine If Closed* asks: true when `PE_Div3` (Presence eye ahead of diverter 3) is on.
2. `ONS(D3_ONS2)` *One Shot* asks: true when it is the first scan they are true (the one-shot, remembered in `D3_ONS2`).
3. `XIO(Cur.3)` *Examine If Open* asks: true when `Cur.3` is off.
4. `XIC(Cur.1)` *Examine If Closed* asks: true when `Cur.1` is on.
5. `OTL(Fire_3)` *Output Latch* does: latches `Fire_3` on.

Why it is built this way:

One-shot: once per event, not once per scan.

A latch: it stays on until an OTU clears it.

RUNG 35




```
XIC(PE_Div3)XIC(Fire_3)OTE(Div3);
```

PREDICT IT · 10 `PE_Div3` is on and `Fire_3` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

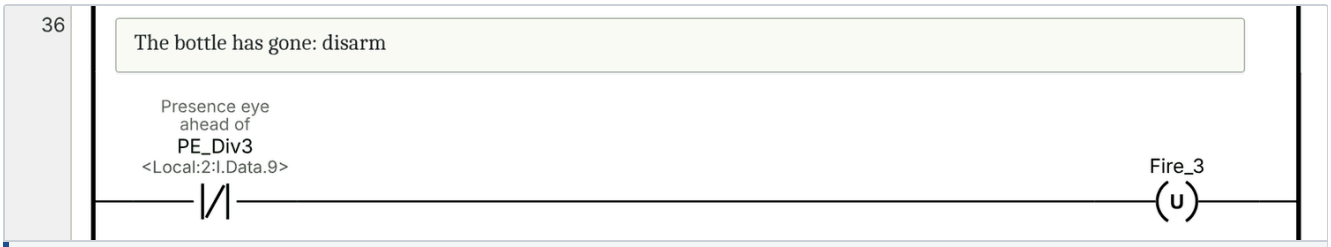
HOW THIS RUNG WORKS

In words. When `PE_Div3` (Presence eye ahead of diverter 3) is on and `Fire_3` is on, the rung turns `Div3` (Diverter 3 cylinder) on (and off again whenever the rung is false).

Left to right:

1. `XIC(PE_Div3)` *Examine If Closed* asks: true when `PE_Div3` (Presence eye ahead of diverter 3) is on.
2. `XIC(Fire_3)` *Examine If Closed* asks: true when `Fire_3` is on.
3. `OTE(Div3)` *Output Energize* does: turns `Div3` (Diverter 3 cylinder) on (and off again whenever the rung is false).

RUNG 36




```
XIO(PE_Div3)OTU(Fire_3);
```

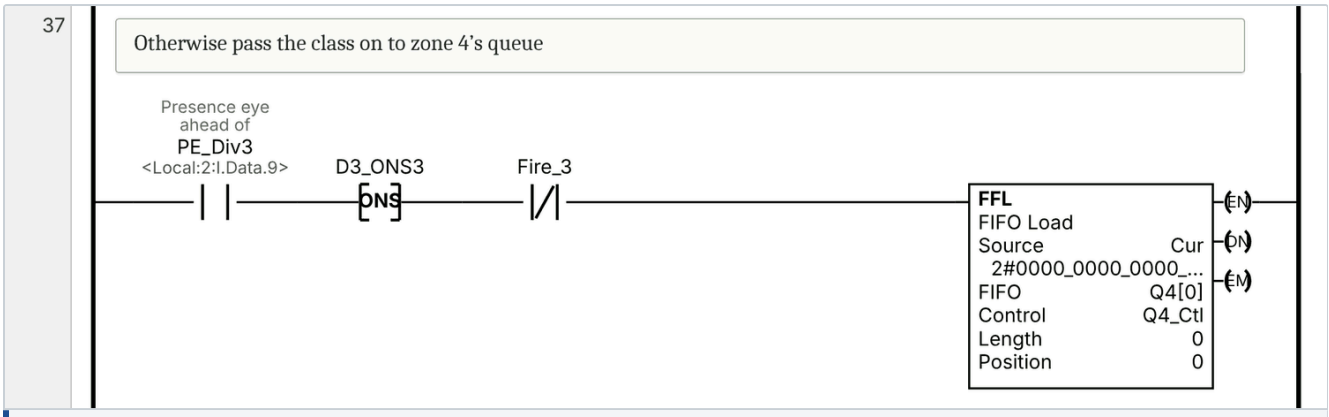
HOW THIS RUNG WORKS

In words. When `PE_Div3` (Presence eye ahead of diverter 3) is off, the rung unlatches `Fire_3` (turns it off).

Left to right:

1. `XIO(PE_Div3)` *Examine If Open* asks: true when `PE_Div3` (Presence eye ahead of diverter 3) is off.
2. `OTU(Fire_3)` *Output Unlatch* does: unlatches `Fire_3` (turns it off).

RUNG 37




```
XIC(PE_Div3)ONS(D3_ONS3)XIO(Fire_3)FFL(Cur,Q4[0],Q4_Ctl,8);
```

HOW THIS RUNG WORKS

In words. When `PE_Div3` (Presence eye ahead of diverter 3) is on, it is the first scan they are true (the one-shot, remembered in `D3_ONS3`) and `Fire_3` is off, the rung loads `Cur` (The class unloaded from a zone queue at a diverter) into the queue `Q4[0]`.

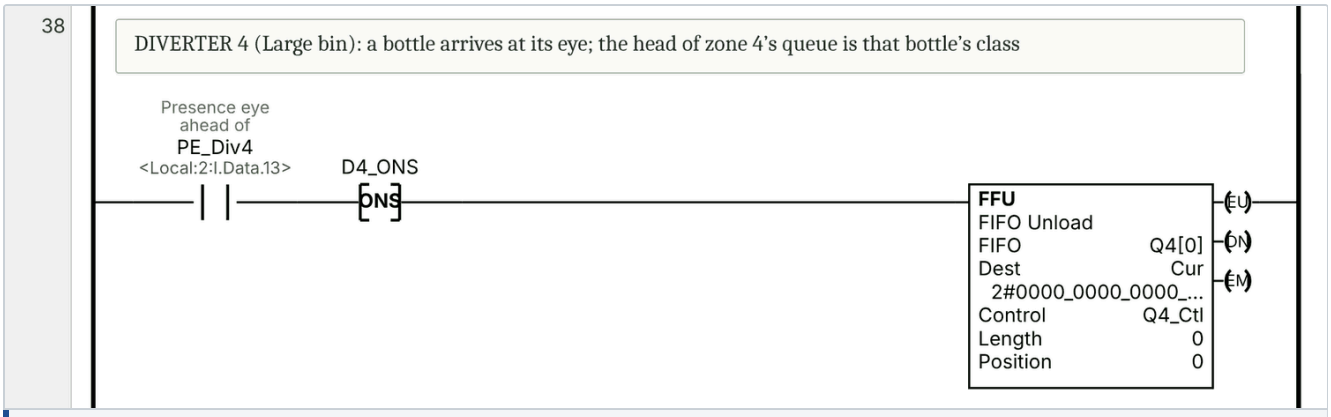
Left to right:

1. `XIC(PE_Div3)` *Examine If Closed* asks: true when `PE_Div3` (Presence eye ahead of diverter 3) is on.
2. `ONS(D3_ONS3)` *One Shot* asks: true when it is the first scan they are true (the one-shot, remembered in `D3_ONS3`).
3. `XIO(Fire_3)` *Examine If Open* asks: true when `Fire_3` is off.
4. `FFL(Cur,Q4[0],Q4_Ctl,8)` *FIFO Load* does: loads `Cur` (The class unloaded from a zone queue at a diverter) into the queue `Q4[0]`.

Why it is built this way:

One-shot: once per event, not once per scan.

RUNG 38




```
XIC(PE_Div4)ONS(D4_ONS)FFU(Q4[0],Cur,Q4_Ctl,8);
```

HOW THIS RUNG WORKS

In words. When `PE_Div4` (Presence eye ahead of diverter 4) is on and it is the first scan they are true (the one-shot, remembered in `D4_ONS`), the rung unloads the oldest entry of `Q4[0]` into `Cur` (The class unloaded from a zone queue at a diverter).

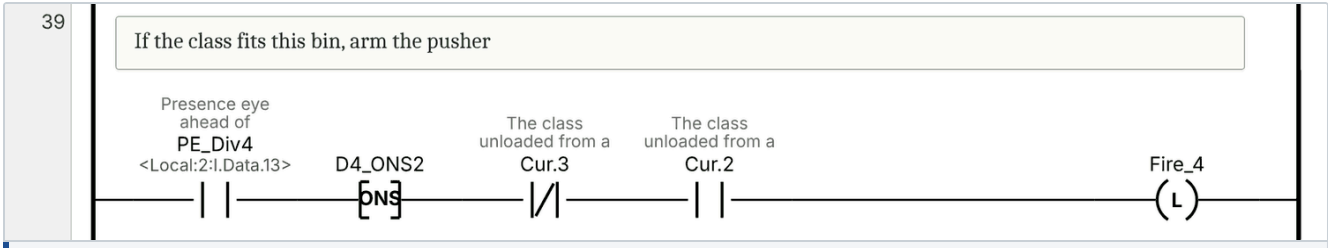
Left to right:

1. `XIC(PE_Div4)` *Examine If Closed* asks: true when `PE_Div4` (Presence eye ahead of diverter 4) is on.
2. `ONS(D4_ONS)` *One Shot* asks: true when it is the first scan they are true (the one-shot, remembered in `D4_ONS`).
3. `FFU(Q4[0],Cur,Q4_Ctl,8)` *FIFO Unload* does: unloads the oldest entry of `Q4[0]` into `Cur` (The class unloaded from a zone queue at a diverter).

Why it is built this way:

One-shot: once per event, not once per scan.

RUNG 39




```
XIC(PE_Div4)ONS(D4_ONS2)XIO(Cur.3)XIC(Cur.2)OTL(Fire_4);
```

HOW THIS RUNG WORKS

In words. When `PE_Div4` (Presence eye ahead of diverter 4) is on, it is the first scan they are true (the one-shot, remembered in `D4_ONS2`), `Cur.3` is off and `Cur.2` is on, the rung latches `Fire_4` on.

Left to right:

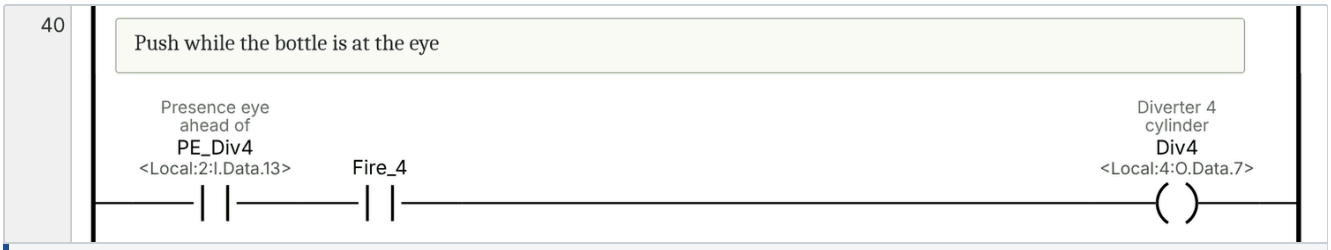
1. `XIC(PE_Div4)` *Examine If Closed* asks: true when `PE_Div4` (Presence eye ahead of diverter 4) is on.
2. `ONS(D4_ONS2)` *One Shot* asks: true when it is the first scan they are true (the one-shot, remembered in `D4_ONS2`).
3. `XIO(Cur.3)` *Examine If Open* asks: true when `Cur.3` is off.
4. `XIC(Cur.2)` *Examine If Closed* asks: true when `Cur.2` is on.
5. `OTL(Fire_4)` *Output Latch* does: latches `Fire_4` on.

Why it is built this way:

One-shot: once per event, not once per scan.

A latch: it stays on until an OTU clears it.

RUNG 40




```
XIC(PE_Div4)XIC(Fire_4)OTE(Div4);
```

PREDICT IT · 11 `PE_Div4` is off and `Fire_4` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

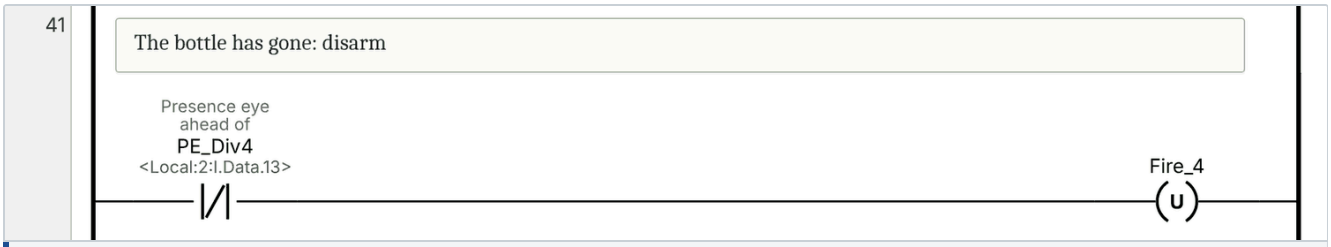
HOW THIS RUNG WORKS

In words. When `PE_Div4` (Presence eye ahead of diverter 4) is on and `Fire_4` is on, the rung turns `Div4` (Diverter 4 cylinder) on (and off again whenever the rung is false).

Left to right:

1. `XIC(PE_Div4)` *Examine If Closed* asks: true when `PE_Div4` (Presence eye ahead of diverter 4) is on.
2. `XIC(Fire_4)` *Examine If Closed* asks: true when `Fire_4` is on.
3. `OTE(Div4)` *Output Energize* does: turns `Div4` (Diverter 4 cylinder) on (and off again whenever the rung is false).

RUNG 41




```
XIO(PE_Div4)OTU(Fire_4);
```

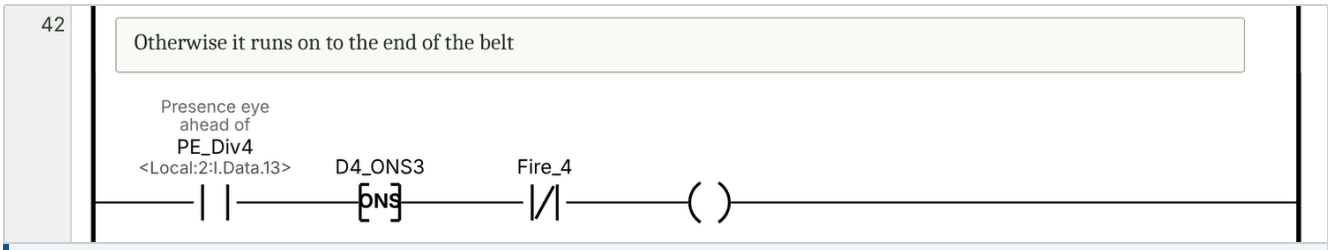
HOW THIS RUNG WORKS

In words. When `PE_Div4` (Presence eye ahead of diverter 4) is off, the rung unlatches `Fire_4` (turns it off).

Left to right:

1. `XIO(PE_Div4)` *Examine If Open* asks: true when `PE_Div4` (Presence eye ahead of diverter 4) is off.
2. `OTU(Fire_4)` *Output Unlatch* does: unlatches `Fire_4` (turns it off).

RUNG 42




```
XIC(PE_Div4)ONS(D4_ONS3)XIO(Fire_4)NOP();
```

HOW THIS RUNG WORKS

In words. When `PE_Div4` (Presence eye ahead of diverter 4) is on, it is the first scan they are true (the one-shot, remembered in `D4_ONS3`) and `Fire_4` is off, the rung does nothing (a placeholder).

Left to right:

1. `XIC(PE_Div4)` *Examine If Closed* asks: true when `PE_Div4` (Presence eye ahead of diverter 4) is on.
2. `ONS(D4_ONS3)` *One Shot* asks: true when it is the first scan they are true (the one-shot, remembered in `D4_ONS3`).
3. `XIO(Fire_4)` *Examine If Open* asks: true when `Fire_4` is off.
4. `NOP()` *No Operation* does: does nothing (a placeholder).

Why it is built this way:

One-shot: once per event, not once per scan.

The same five rungs, with the match changed: small is `XIO(Cur.3)XIC(Cur.0)`, medium `XIO(Cur.3)XIC(Cur.1)`, large `XIO(Cur.3)XIC(Cur.2)`. The last diverter's pass-on rung ends in `NOP()`: a bottle no bin wanted rides to the remover at the end of the belt, and there is no queue after the last zone.

WATCH FOR: A SLIPPED QUEUE

Everything depends on every bottle that enters a zone being loaded into its queue exactly once, and unloaded exactly once. Miss one load — a pass-on rung with the wrong condition — and every class in the next zone is one bottle late: the next diverter sorts the wrong bottle, and so does every diverter after it, until the line is emptied. That is the same failure as a shift register that misses a pulse in Part IV, and the same check finds it: Monitor Tags, `Q1_Ctl.POS` and `Q2_Ctl.POS`, against the bottles you can see in each stretch of belt.

Run it

Bottles leave the hopper every few seconds; several are on the belt at once, and each still goes to the right bin. Expand `Q1` and `Q2` in Monitor Tags and watch the classes move through the queues as the bottles move down the belt. The reference run: 14 of 14 bottles in the right bin, nothing lost, in 85 s against a par of 110 s: 1600 points and three stars.

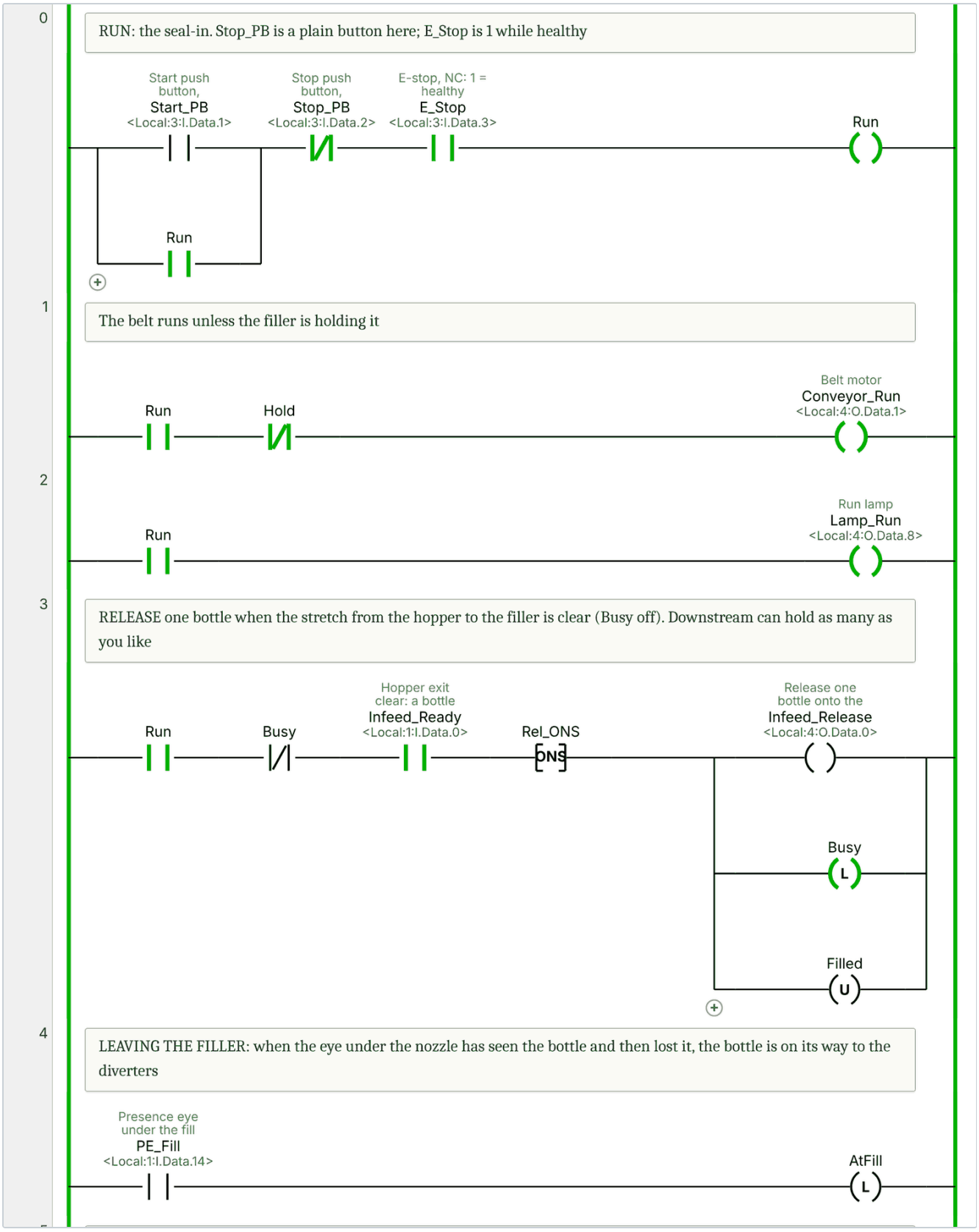


Figure 150. Level 4 online, the top of the routine: release, leaving the filler, gantry and class word.

> **TRY IT**
>
> Delete diverter 1's pass-on rung (rung 27) and run: from the first non-broken bottle on, diverter 2 unloads nothing (its queue is empty) and nothing is sorted. Put it back and delete rung 26 instead: `Fire_1` stays armed and diverter 1 pushes the next bottle too, whatever it is.

Predict it: the answers

1. The rung is false at `XIO(Stop_PB)`: `Stop_PB` is on, so `Run` is **off** (an OTE writes 0 when its rung is false).
2. Every condition on the way is met, so `Conveyor_Run` is **on**.
3. Every condition on the way is met, so `Class.0` is **on**.
4. The rung is false at `XIO(Is_High)`: `Is_High` is on, so `Class.1` is **off** (an OTE writes 0 when its rung is false).
5. The rung is false at `XIO(Is_Broken)`: `Is_Broken` is on, so `Hold` is **off** (an OTE writes 0 when its rung is false).
6. Every condition on the way is met, so `Fill_Valve` is **on**.
7. The rung is false at the GEQ: `Fill_Level` is below **97**, so the OTL does nothing: `Filled` keeps whatever it had.
8. Every condition on the way is met, so `Div1` is **on**.
9. The rung is false at `XIC(Fire_2)`: `Fire_2` is off, so `Div2` is **off** (an OTE writes 0 when its rung is false).
10. Every condition on the way is met, so `Div3` is **on**.
11. The rung is false at `XIC(PE_Div4)`: `PE_Div4` is off, so `Div4` is **off** (an OTE writes 0 when its rung is false).

37 Level 5 — Full sort

> **IN THE APP**
>
> A level project opens with guided mode on this chapter: the panel lists the tags the program needs (*Make them*), shows every rung below under its fold with *Copy* and *Add*, checks that MainRoutine verifies, and while you run the level ticks three results — the level completes, every bottle in the right bin, under par.

Sixteen bottles, six bins: broken; small; mediums split three ways by colour — red, blue, green; and large. Every whole bottle filled. Par 120 s. The method is Level 4's, unchanged; what grows is the question each diverter asks, and this lesson is about writing a match that tests **everything** it depends on.

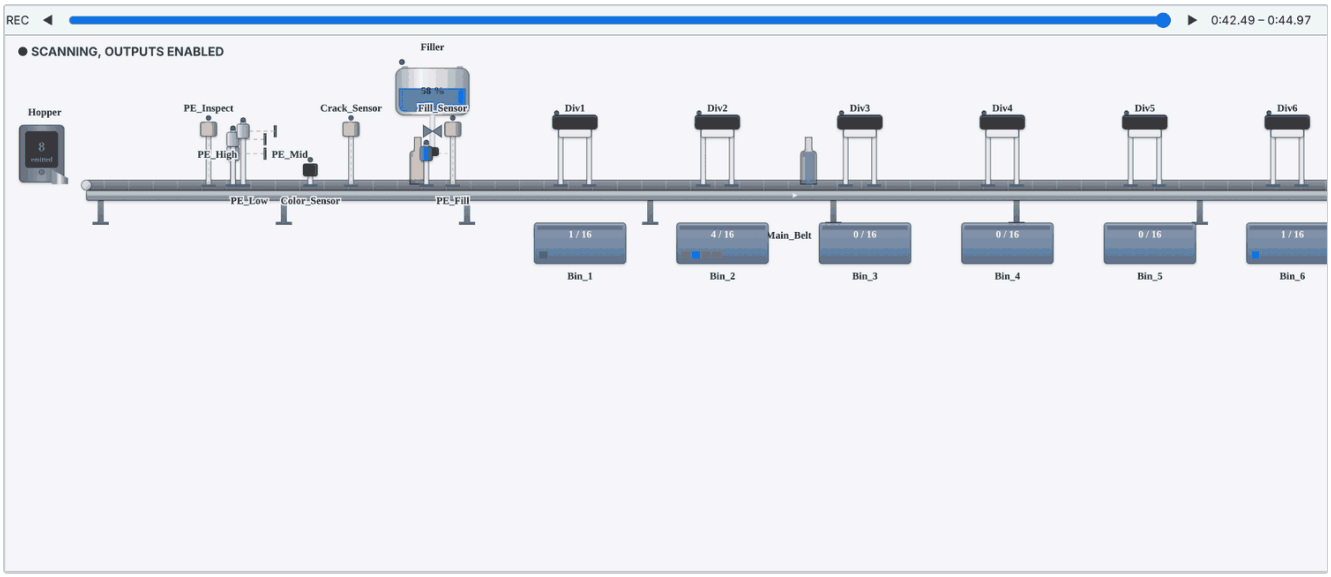


Figure 151. Level 5 running: six pushers, six graded bins.

Tags this lesson needs

The level's alias tags are already made for you (chapter 32's table). Create these plain tags before you type the rungs; Verify reports any you miss.

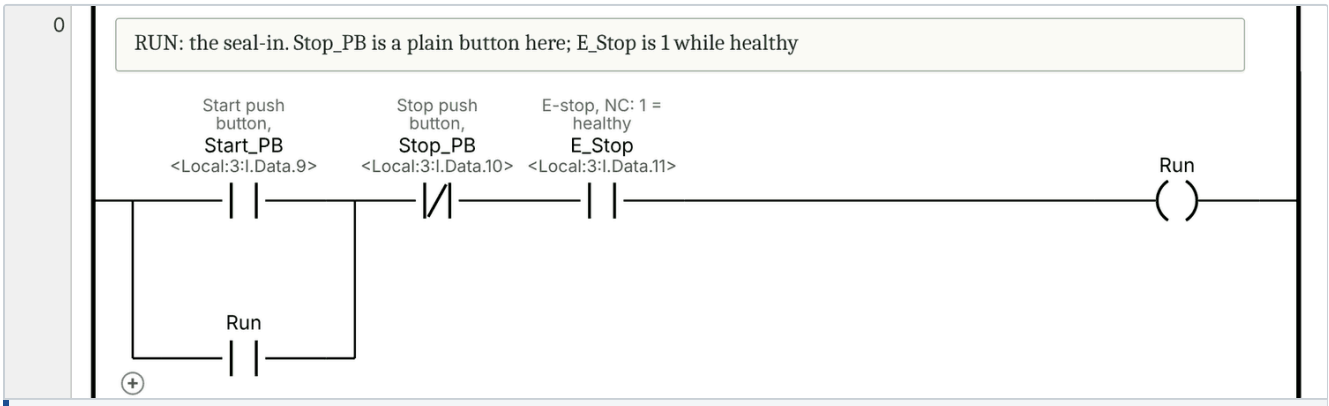
Tag	Type	How to make it	Description
`Run`	BOOL	Edit Tags: name, type	Seal-in run bit
`Busy`	BOOL	Edit Tags: name, type	A bottle is between the hopper and the end of its cycle (Levels 1-3) / between the hopper and the filler (Levels 4-5)
`Rel_ONS`	BOOL	Edit Tags: name, type	One-shot storage for the release
`Hold`	BOOL	Edit Tags: name, type	The filler holds the belt

Tag	Type	How to make it	Description
`Filled`	BOOL	Edit Tags: name, type	This bottle has its charge
`AtFill`	BOOL	Edit Tags: name, type	The eye under the nozzle has seen this bottle
`Leave_ONS`	BOOL	Edit Tags: name, type	One-shot storage: the bottle left the filler
`Is_Low`	BOOL	Edit Tags: name, type	Latched: the low height beam was broken
`Is_Mid`	BOOL	Edit Tags: name, type	Latched: the middle beam was broken
`Is_High`	BOOL	Edit Tags: name, type	Latched: the high beam was broken
`Is_Broken`	BOOL	Edit Tags: name, type	Latched: the crack eye saw a broken bottle
`Is_R`	BOOL	Edit Tags: name, type	Latched: the colour sensor saw red
`Is_B`	BOOL	Edit Tags: name, type	Latched: the colour sensor saw blue
`Is_G`	BOOL	Edit Tags: name, type	Latched: the colour sensor saw green
`Class`	DINT	Edit Tags: name, type, Style Binary	The bottle in the gantry: bit 0 S, 1 M, 2 L, 3 broken, 4 red, 5 blue, 6 green
`Cur`	DINT	Edit Tags: name, type, Style Binary	The class unloaded from a zone queue at a diverter
`Q1`	DINT[8]	Edit Tags: name, type, Dim 8	Zone 1 queue: classes of the bottles between the filler and diverter 1
`Q1_Ctl`	CONTROL	Edit Tags: name, type	Queue control
`D1_ONS`	BOOL	Edit Tags: name, type	One-shot storage for a diverter
`D1_ONS2`	BOOL	Edit Tags: name, type	One-shot storage for a diverter
`D1_ONS3`	BOOL	Edit Tags: name, type	One-shot storage for a diverter
`Fire_1`	BOOL	Edit Tags: name, type	Armed: this diverter will push the bottle at its eye
`Q2`	DINT[8]	Edit Tags: name, type, Dim 8	Zone 2 queue: classes of the bottles between diverter 1 and diverter 2
`Q2_Ctl`	CONTROL	Edit Tags: name, type	Queue control
`D2_ONS`	BOOL	Edit Tags: name, type	One-shot storage for a diverter
`D2_ONS2`	BOOL	Edit Tags: name, type	One-shot storage for a diverter
`D2_ONS3`	BOOL	Edit Tags: name, type	One-shot storage for a diverter
`Fire_2`	BOOL	Edit Tags: name, type	Armed: this diverter will push the bottle at its eye
`Q3`	DINT[8]	Edit Tags: name, type, Dim 8	Zone 3 queue: classes of the bottles between diverter 2 and diverter 3
`Q3_Ctl`	CONTROL	Edit Tags: name, type	Queue control

Tag	Type	How to make it	Description
`D3_ONS`	BOOL	Edit Tags: name, type	One-shot storage for a diverter
`D3_ONS2`	BOOL	Edit Tags: name, type	One-shot storage for a diverter
`D3_ONS3`	BOOL	Edit Tags: name, type	One-shot storage for a diverter
`Fire_3`	BOOL	Edit Tags: name, type	Armed: this diverter will push the bottle at its eye
`Q4`	DINT[8]	Edit Tags: name, type, Dim 8	Zone 4 queue: classes of the bottles between diverter 3 and diverter 4
`Q4_Ctl`	CONTROL	Edit Tags: name, type	Queue control
`D4_ONS`	BOOL	Edit Tags: name, type	One-shot storage for a diverter
`D4_ONS2`	BOOL	Edit Tags: name, type	One-shot storage for a diverter
`D4_ONS3`	BOOL	Edit Tags: name, type	One-shot storage for a diverter
`Fire_4`	BOOL	Edit Tags: name, type	Armed: this diverter will push the bottle at its eye
`Q5`	DINT[8]	Edit Tags: name, type, Dim 8	Zone 5 queue: classes of the bottles between diverter 4 and diverter 5
`Q5_Ctl`	CONTROL	Edit Tags: name, type	Queue control
`D5_ONS`	BOOL	Edit Tags: name, type	One-shot storage for a diverter
`D5_ONS2`	BOOL	Edit Tags: name, type	One-shot storage for a diverter
`D5_ONS3`	BOOL	Edit Tags: name, type	One-shot storage for a diverter
`Fire_5`	BOOL	Edit Tags: name, type	Armed: this diverter will push the bottle at its eye
`Q6`	DINT[8]	Edit Tags: name, type, Dim 8	Zone 6 queue: classes of the bottles between diverter 5 and diverter 6
`Q6_Ctl`	CONTROL	Edit Tags: name, type	Queue control
`D6_ONS`	BOOL	Edit Tags: name, type	One-shot storage for a diverter
`D6_ONS2`	BOOL	Edit Tags: name, type	One-shot storage for a diverter
`D6_ONS3`	BOOL	Edit Tags: name, type	One-shot storage for a diverter
`Fire_6`	BOOL	Edit Tags: name, type	Armed: this diverter will push the bottle at its eye

Run circuit, release, leaving the filler, gantry, class word and fill

RUNG 0




```
[XIC(Start_PB),XIC(Run)]XIO(Stop_PB)XIC(E_Stop)OTE(Run);
```

PREDICT IT · 1 `Start_PB` is on, `Stop_PB` is on and `E_Stop` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When either (`Start_PB` (Start push button, momentary) is on) *or* (`Run` (Seal-in run bit) is on), `Stop_PB` (Stop push button, momentary) is off and `E_Stop` (E-stop, NC: 1 = healthy) is on, the rung turns `Run` on (and off again whenever the rung is false).

Left to right:

1. A branch of 2 parallel paths: power gets through if any one of them is true.
   - **Path 1**
     1. `XIC(Start_PB)` *Examine If Closed* asks: true when `Start_PB` (Start push button, momentary) is on.
   - **Path 2**
     1. `XIC(Run)` *Examine If Closed* asks: true when `Run` (Seal-in run bit) is on.
2. `XIO(Stop_PB)` *Examine If Open* asks: true when `Stop_PB` (Stop push button, momentary) is off.
3. `XIC(E_Stop)` *Examine If Closed* asks: true when `E_Stop` (E-stop, NC: 1 = healthy) is on.
4. `OTE(Run)` *Output Energize* does: turns `Run` on (and off again whenever the rung is false). Every scan it writes the rung state: true on, false off.

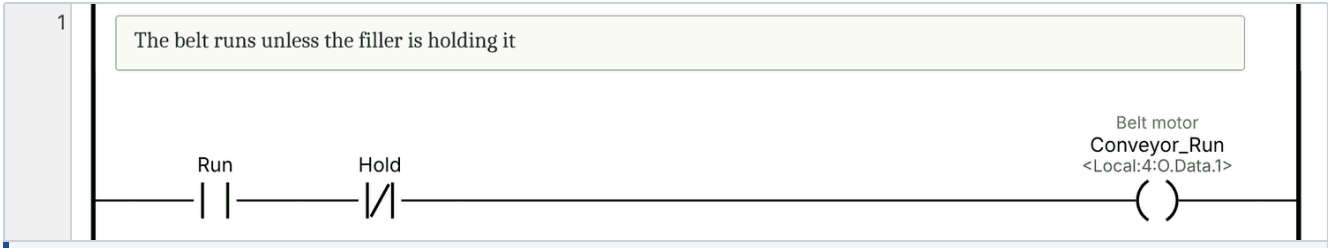
Why it is built this way:

Seal-in. The start condition turns `Run` on, and `Run` 's own contact in the parallel path keeps the rung true after the start goes away. Anything in series outside the branch (a stop, a fault) breaks it, and then it stays off until the start is made again.

> **Think of it like this:** a doorbell button and a hand. The start button is the doorbell, pressed for a moment. The seal-in contact is a hand that grabs the circuit as soon as the output comes on and keeps holding after you let go. The stop button, in series, is the only thing that can knock that hand away.

> **Watch for:** The stop has to be in series outside the branch. Put it inside the start path only and the seal-in path bypasses it: the stop no longer stops.

RUNG 1




```
XIC(Run)XIO(Hold)OTE(Conveyor_Run);
```

PREDICT IT · 2 `Run` is on and `Hold` is off. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Run` (Seal-in run bit) is on and `Hold` (The filler holds the belt) is off, the rung turns `Conveyor_Run` (Belt motor) on (and off again whenever the rung is false).

Left to right:

1. `XIC(Run)` *Examine If Closed* asks: true when `Run` (Seal-in run bit) is on.
2. `XIO(Hold)` *Examine If Open* asks: true when `Hold` (The filler holds the belt) is off.
3. `OTE(Conveyor_Run)` *Output Energize* does: turns `Conveyor_Run` (Belt motor) on (and off again whenever the rung is false).

> **Think of it like this:** a rung is a pipe with valves in it, read left to right. Every contact is a valve: XIC is open while its bit is 1, XIO while its bit is 0. Water reaches the output at the right only if every valve in a row is open. The output does not remember anything: on every scan it is simply wet or dry.

RUNG 2

```
XIC(Run)OTE(Lamp_Run);
```

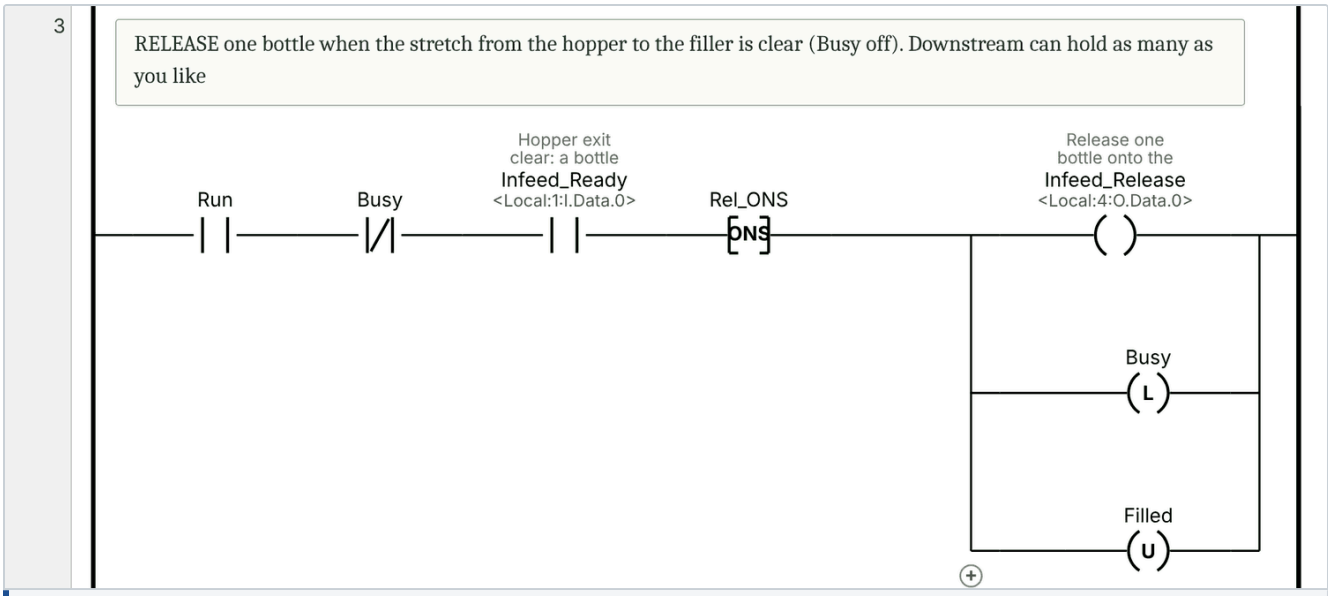
HOW THIS RUNG WORKS

In words. When `Run` (Seal-in run bit) is on, the rung turns `Lamp_Run` (Run lamp) on (and off again whenever the rung is false).

Left to right:

1. `XIC(Run)` *Examine If Closed* asks: true when `Run` (Seal-in run bit) is on.
2. `OTE(Lamp_Run)` *Output Energize* does: turns `Lamp_Run` (Run lamp) on (and off again whenever the rung is false).

RUNG 3




```
XIC(Run)XIO(Busy)XIC(Infeed_Ready)ONS(Rel_ONS)
[OTE(Infeed_Release),OTL(Busy),OTU(Filled)];
```

HOW THIS RUNG WORKS

In words. When `Run` (Seal-in run bit) is on, `Busy` (A bottle is between the hopper and the end of its cycle (Levels 1-3) / between the hopper and the filler (Levels 4-5)) is off, `Infeed_Ready` (Hopper exit clear: a bottle can be released) is on and it is the first scan they are true (the one-shot, remembered in `Rel_ONS` (One-shot storage for the release)), the rung turns `Infeed_Release` (Release one bottle onto the belt (rising edge)) on (and off again whenever the rung is false), latches `Busy` on and unlatches `Filled` (This bottle has its charge) (turns it off).

Left to right:

1. `XIC(Run)` *Examine If Closed* asks: true when `Run` (Seal-in run bit) is on.
2. `XIO(Busy)` *Examine If Open* asks: true when `Busy` (A bottle is between the hopper and the end of its cycle (Levels 1-3) / between the hopper and the filler (Levels 4-5)) is off.
3. `XIC(Infeed_Ready)` *Examine If Closed* asks: true when `Infeed_Ready` (Hopper exit clear: a bottle can be released) is on.
4. `ONS(Rel_ONS)` *One Shot* asks: true when it is the first scan they are true (the one-shot, remembered in `Rel_ONS` (One-shot storage for the release)). It passes power for one scan only, the scan its input goes from false to true; its storage bit remembers last scan.
5. A branch of 3 parallel paths: each path is tested on its own and does its own work.

   Path 1

   1. `OTE(Infeed_Release)` *Output Energize* does: turns `Infeed_Release` (Release one bottle onto the belt (rising edge)) on (and off again whenever the rung is false).

   Path 2

   1. `OTL(Busy)` *Output Latch* does: latches `Busy` on. It only ever sets the bit; the bit stays on after the rung goes false.

   Path 3

   1. `OTU(Filled)` *Output Unlatch* does: unlatches `Filled` (This bottle has its charge) (turns it off). It only ever clears the bit.

Why it is built this way:

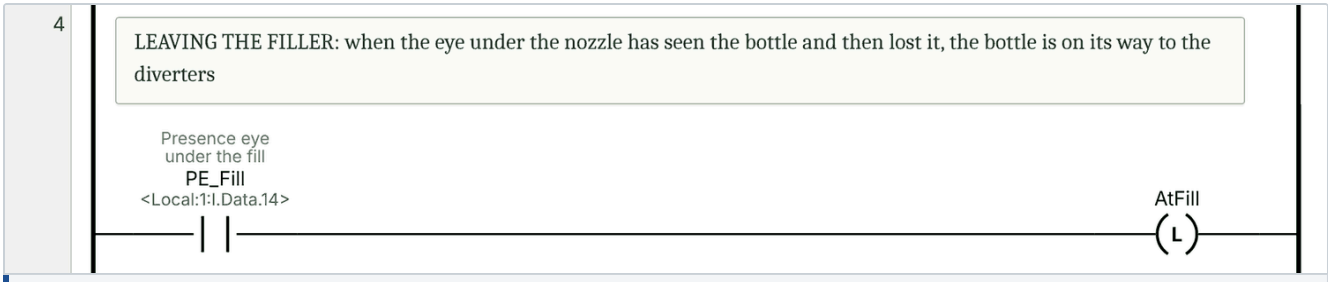
One-shot. Everything after the ONS happens once per event, not on every scan the condition holds; `Rel_ONS` is only its memory of the last scan.

A latch. `Busy` stays on after this rung goes false; an `OTU` elsewhere in the program clears it.

> **Think of it like this:** a doorbell that rings once however long you lean on the button. The ONS looks at what the button was doing last scan: only the change from up to down rings it.

> **Watch for:** Each ONS needs its own storage bit. Two ONS sharing one bit make one of them miss its edges.

RUNG 4




```
XIC(PE_Fill)OTL(AtFill);
```

HOW THIS RUNG WORKS

In words. When `PE_Fill` (Presence eye under the fill nozzle) is on, the rung latches `AtFill` (The eye under the nozzle has seen this bottle) on.

Left to right:

1. `XIC(PE_Fill)` *Examine If Closed* asks: true when `PE_Fill` (Presence eye under the fill nozzle) is on.
2. `OTL(AtFill)` *Output Latch* does: latches `AtFill` (The eye under the nozzle has seen this bottle) on.

Why it is built this way:

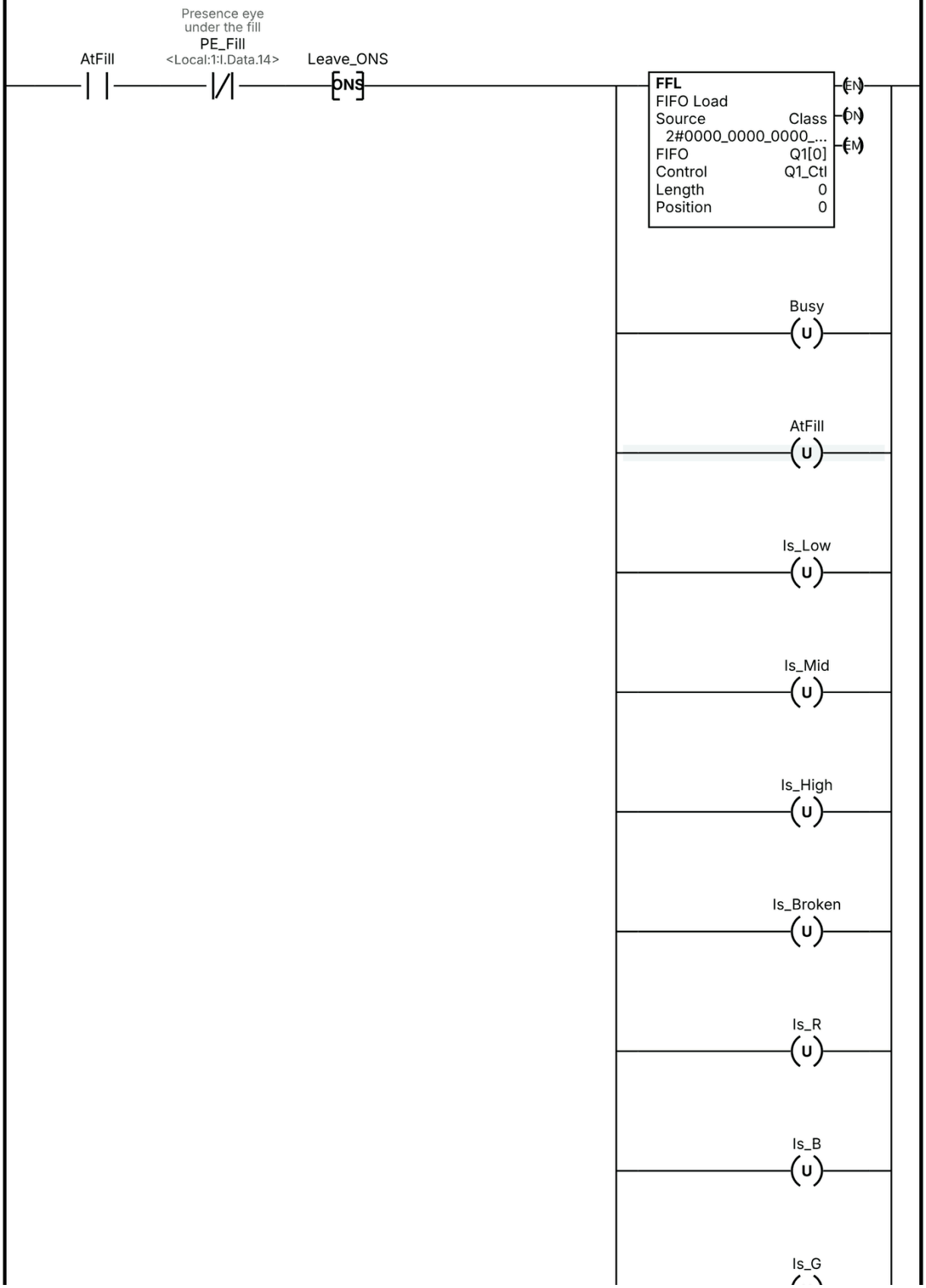
A latch: it stays on until an OTU clears it.

> **Watch for:** A latched bit survives the condition that set it: every way in needs a way out, or it stays on for good.

RUNG 5

5

Load its class into zone 1's queue, free the front of the line for the next bottle, and forget what the gantry saw



```
XIC(AtFill)XIO(PE_Fill)ONS(Leave_ONS)
[FFL(Class,Q1[0],Q1_Ctl,8),OTU(Busy),OTU(AtFill),OTU(Is_Low),OTU(Is_Mid),OTU(Is
_High),OTU(Is_Broken),OTU(Is_R),OTU(Is_B),OTU(Is_G)];
```

HOW THIS RUNG WORKS

In words. When `AtFill` (The eye under the nozzle has seen this bottle) is on, `PE_Fill` (Presence eye under the fill nozzle) is off and it is the first scan they are true (the one-shot, remembered in `Leave_ONS` (One-shot storage: the bottle left the filler)), the rung loads `Class` (The bottle in the gantry: bit 0 S, 1 M, 2 L, 3 broken, 4 red, 5 blue, 6 green) into the queue `Q1[0]`, unlatches `Busy` (A bottle is between the hopper and the end of its cycle (Levels 1-3) / between the hopper and the filler (Levels 4-5)) (turns it off), unlatches `AtFill` (turns it off), unlatches `Is_Low` (Latched: the low height beam was broken) (turns it off), unlatches `Is_Mid` (Latched: the middle beam was broken) (turns it off), unlatches `Is_High` (Latched: the high beam was broken) (turns it off), unlatches `Is_Broken` (Latched: the crack eye saw a broken bottle) (turns it off), unlatches `Is_R` (Latched: the colour sensor saw red) (turns it off), unlatches `Is_B` (Latched: the colour sensor saw blue) (turns it off) and unlatches `Is_G` (Latched: the colour sensor saw green) (turns it off).

Left to right:

1. `XIC(AtFill)` *Examine If Closed* asks: true when `AtFill` (The eye under the nozzle has seen this bottle) is on.
2. `XIO(PE_Fill)` *Examine If Open* asks: true when `PE_Fill` (Presence eye under the fill nozzle) is off.
3. `ONS(Leave_ONS)` *One Shot* asks: true when it is the first scan they are true (the one-shot, remembered in `Leave_ONS` (One-shot storage: the bottle left the filler)).
4. A branch of 10 parallel paths: each path is tested on its own and does its own work.

   Path 1

   1. `FFL(Class,Q1[0],Q1_Ctl,8)` *FIFO Load* does: loads `Class` (The bottle in the gantry: bit 0 S, 1 M, 2 L, 3 broken, 4 red, 5 blue, 6 green) into the queue `Q1[0]`. One entry per false-to-true transition.

   Path 2

   1. `OTU(Busy)` *Output Unlatch* does: unlatches `Busy` (A bottle is between the hopper and the end of its cycle (Levels 1-3) / between the hopper and the filler (Levels 4-5)) (turns it off).

   Path 3

   1. `OTU(AtFill)` *Output Unlatch* does: unlatches `AtFill` (turns it off).

   Path 4

   1. `OTU(Is_Low)` *Output Unlatch* does: unlatches `Is_Low` (Latched: the low height beam was broken) (turns it off).

   Path 5

   1. `OTU(Is_Mid)` *Output Unlatch* does: unlatches `Is_Mid` (Latched: the middle beam was broken) (turns it off).

   Path 6

   1. `OTU(Is_High)` *Output Unlatch* does: unlatches `Is_High` (Latched: the high beam was broken) (turns it off).

   Path 7

   1. `OTU(Is_Broken)` *Output Unlatch* does: unlatches `Is_Broken` (Latched: the crack eye saw a broken bottle) (turns it off).

   Path 8

   1. `OTU(Is_R)` *Output Unlatch* does: unlatches `Is_R` (Latched: the colour sensor saw red) (turns it off).

Path 9

1. `OTU(Is_B)` *Output Unlatch* does: unlatches `Is_B` (Latched: the colour sensor saw blue) (turns it off).

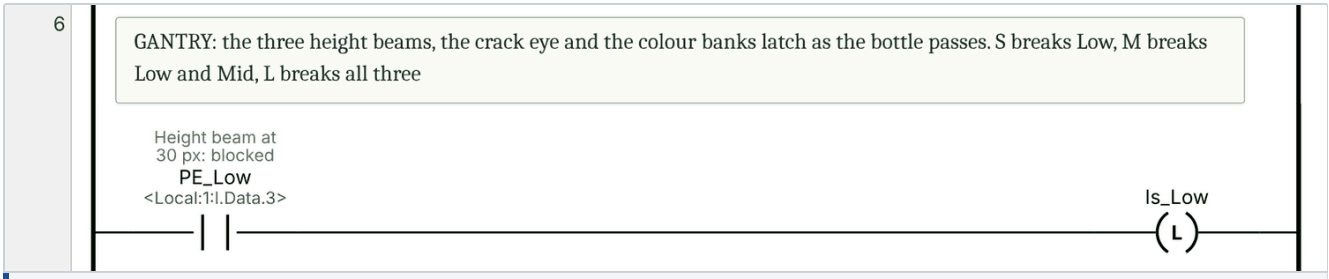
Path 10

1. `OTU(Is_G)` *Output Unlatch* does: unlatches `Is_G` (Latched: the colour sensor saw green) (turns it off).

Why it is built this way:

One-shot: once per event, not once per scan.

RUNG 6




```
XIC(PE_Low)OTL(Is_Low);
```

HOW THIS RUNG WORKS

In words. When `PE_Low` (Height beam at 30 px: blocked by every size) is on, the rung latches `Is_Low` (Latched: the low height beam was broken) on.

Left to right:

1. `XIC(PE_Low)` *Examine If Closed* asks: true when `PE_Low` (Height beam at 30 px: blocked by every size) is on.
2. `OTL(Is_Low)` *Output Latch* does: latches `Is_Low` (Latched: the low height beam was broken) on.

Why it is built this way:

A latch: it stays on until an OTU clears it.

RUNG 7

```
XIC(PE_Mid)OTL(Is_Mid);
```

HOW THIS RUNG WORKS

In words. When `PE_Mid` (Height beam at 44 px: blocked by M and L) is on, the rung latches `Is_Mid` (Latched: the middle beam was broken) on.

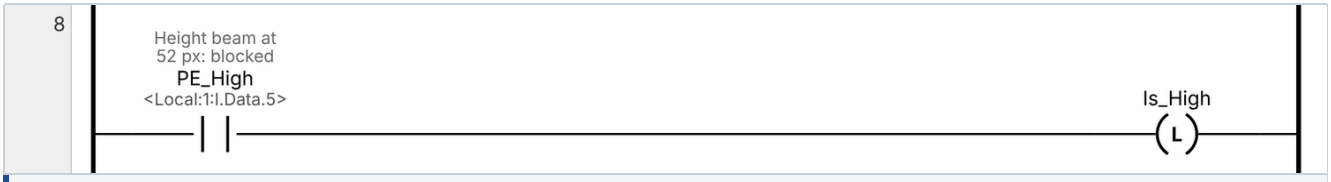
Left to right:

1. `XIC(PE_Mid)` *Examine If Closed* asks: true when `PE_Mid` (Height beam at 44 px: blocked by M and L) is on.
2. `OTL(Is_Mid)` *Output Latch* does: latches `Is_Mid` (Latched: the middle beam was broken) on.

Why it is built this way:

A latch: it stays on until an OTU clears it.

RUNG 8




```
XIC(PE_High)OTL(Is_High);
```

HOW THIS RUNG WORKS

In words. When `PE_High` (Height beam at 52 px: blocked by L only) is on, the rung latches `Is_High` (Latched: the high beam was broken) on.

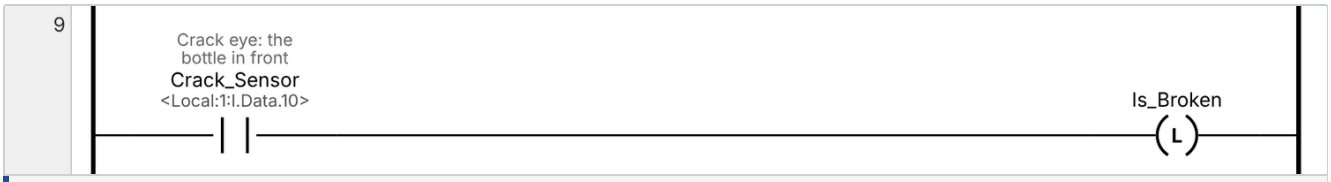
Left to right:

1. `XIC(PE_High)` *Examine If Closed* asks: true when `PE_High` (Height beam at 52 px: blocked by L only) is on.
2. `OTL(Is_High)` *Output Latch* does: latches `Is_High` (Latched: the high beam was broken) on.

Why it is built this way:

A latch: it stays on until an OTU clears it.

RUNG 9




```
XIC(Crack_Sensor)OTL(Is_Broken);
```

HOW THIS RUNG WORKS

In words. When `Crack_Sensor` (Crack eye: the bottle in front of it is broken) is on, the rung latches `Is_Broken` (Latched: the crack eye saw a broken bottle) on.

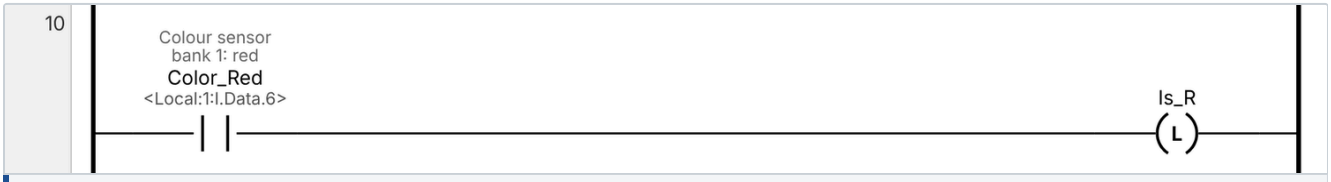
Left to right:

1. `XIC(Crack_Sensor)` *Examine If Closed* asks: true when `Crack_Sensor` (Crack eye: the bottle in front of it is broken) is on.
2. `OTL(Is_Broken)` *Output Latch* does: latches `Is_Broken` (Latched: the crack eye saw a broken bottle) on.

Why it is built this way:

A latch: it stays on until an OTU clears it.

RUNG 10




```
XIC(Color_Red)OTL(Is_R);
```

HOW THIS RUNG WORKS

In words. When `Color_Red` (Colour sensor bank 1: red) is on, the rung latches `Is_R` (Latched: the colour sensor saw red) on.

Left to right:

1. `XIC(Color_Red)` *Examine If Closed* asks: true when `Color_Red` (Colour sensor bank 1: red) is on.
2. `OTL(Is_R)` *Output Latch* does: latches `Is_R` (Latched: the colour sensor saw red) on.

Why it is built this way:

A latch: it stays on until an OTU clears it.

RUNG 11

```
XIC(Color_Blue)OTL(Is_B);
```

HOW THIS RUNG WORKS

In words. When `Color_Blue` (Colour sensor bank 2: blue) is on, the rung latches `Is_B` (Latched: the colour sensor saw blue) on.

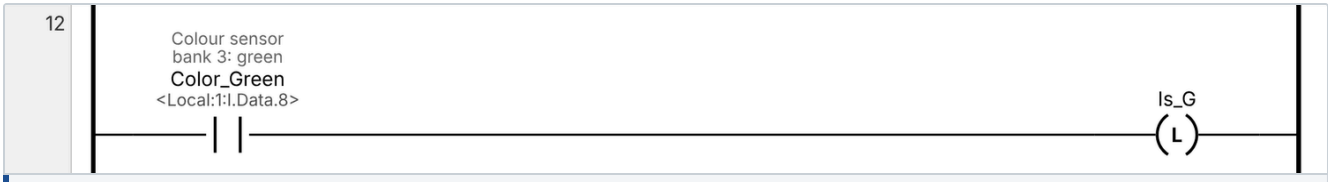
Left to right:

1. `XIC(Color_Blue)` *Examine If Closed* asks: true when `Color_Blue` (Colour sensor bank 2: blue) is on.
2. `OTL(Is_B)` *Output Latch* does: latches `Is_B` (Latched: the colour sensor saw blue) on.

Why it is built this way:

A latch: it stays on until an OTU clears it.

RUNG 12




```
XIC(Color_Green)OTL(Is_G);
```

HOW THIS RUNG WORKS

In words. When `Color_Green` (Colour sensor bank 3: green) is on, the rung latches `Is_G` (Latched: the colour sensor saw green) on.

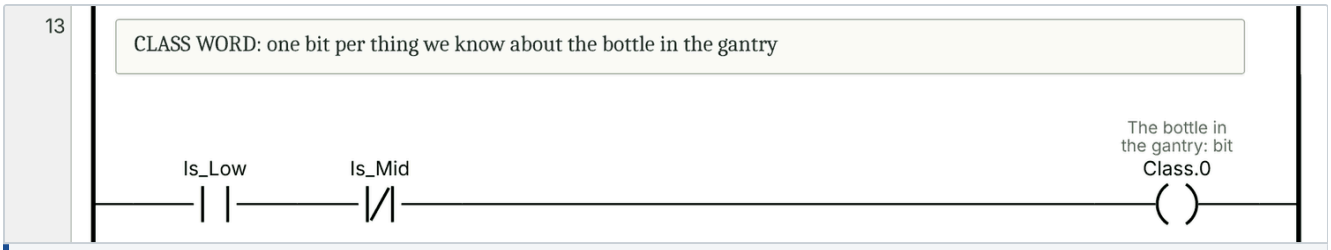
Left to right:

1. `XIC(Color_Green)` *Examine If Closed* asks: true when `Color_Green` (Colour sensor bank 3: green) is on.
2. `OTL(Is_G)` *Output Latch* does: latches `Is_G` (Latched: the colour sensor saw green) on.

Why it is built this way:

A latch: it stays on until an OTU clears it.

RUNG 13




```
XIC(Is_Low)XIO(Is_Mid)OTE(Class.0);
```

PREDICT IT · 3 `Is_Low` is on and `Is_Mid` is off. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

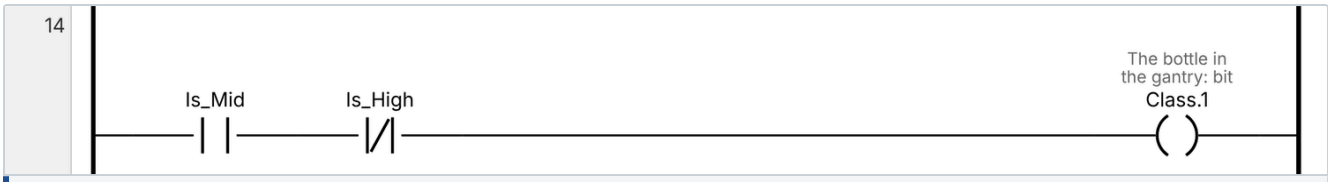
HOW THIS RUNG WORKS

In words. When `Is_Low` (Latched: the low height beam was broken) is on and `Is_Mid` (Latched: the middle beam was broken) is off, the rung turns `Class.0` on (and off again whenever the rung is false).

Left to right:

1. `XIC(Is_Low)` *Examine If Closed* asks: true when `Is_Low` (Latched: the low height beam was broken) is on.
2. `XIO(Is_Mid)` *Examine If Open* asks: true when `Is_Mid` (Latched: the middle beam was broken) is off.
3. `OTE(Class.0)` *Output Energize* does: turns `Class.0` on (and off again whenever the rung is false).

RUNG 14




```
XIC(Is_Mid)XIO(Is_High)OTE(Class.1);
```

PREDICT IT · 4 `Is_Mid` is on and `Is_High` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

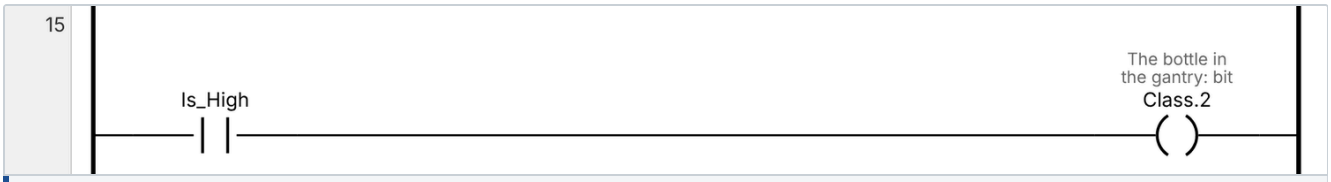
HOW THIS RUNG WORKS

In words. When `Is_Mid` (Latched: the middle beam was broken) is on and `Is_High` (Latched: the high beam was broken) is off, the rung turns `Class.1` on (and off again whenever the rung is false).

Left to right:

1. `XIC(Is_Mid)` *Examine If Closed* asks: true when `Is_Mid` (Latched: the middle beam was broken) is on.
2. `XIO(Is_High)` *Examine If Open* asks: true when `Is_High` (Latched: the high beam was broken) is off.
3. `OTE(Class.1)` *Output Energize* does: turns `Class.1` on (and off again whenever the rung is false).

RUNG 15




```
XIC(Is_High)OTE(Class.2);
```

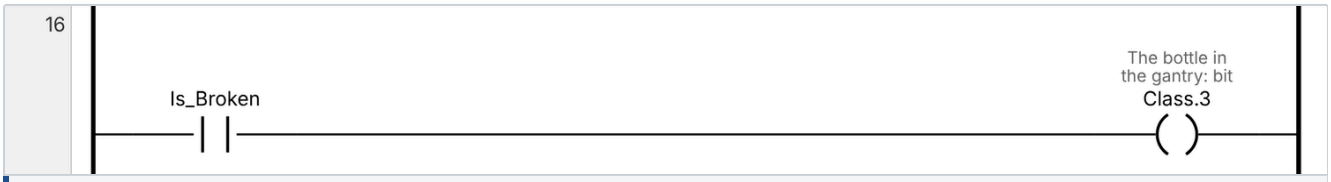
HOW THIS RUNG WORKS

In words. When `Is_High` (Latched: the high beam was broken) is on, the rung turns `Class.2` on (and off again whenever the rung is false).

Left to right:

1. `XIC(Is_High)` *Examine If Closed* asks: true when `Is_High` (Latched: the high beam was broken) is on.
2. `OTE(Class.2)` *Output Energize* does: turns `Class.2` on (and off again whenever the rung is false).

RUNG 16




```
XIC(Is_Broken)OTE(Class.3);
```

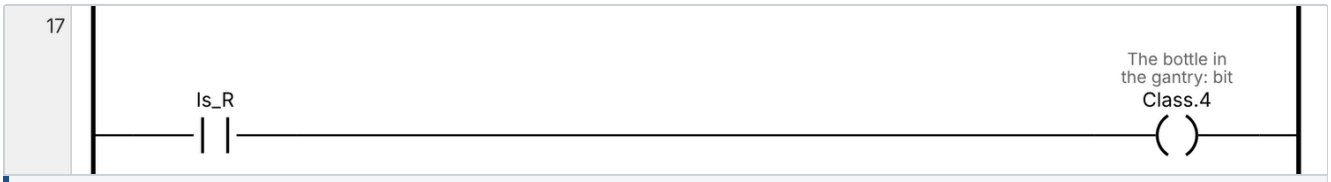
HOW THIS RUNG WORKS

In words. When `Is_Broken` (Latched: the crack eye saw a broken bottle) is on, the rung turns `Class.3` on (and off again whenever the rung is false).

Left to right:

1. `XIC(Is_Broken)` *Examine If Closed* asks: true when `Is_Broken` (Latched: the crack eye saw a broken bottle) is on.
2. `OTE(Class.3)` *Output Energize* does: turns `Class.3` on (and off again whenever the rung is false).

RUNG 17




```
XIC(Is_R)OTE(Class.4);
```

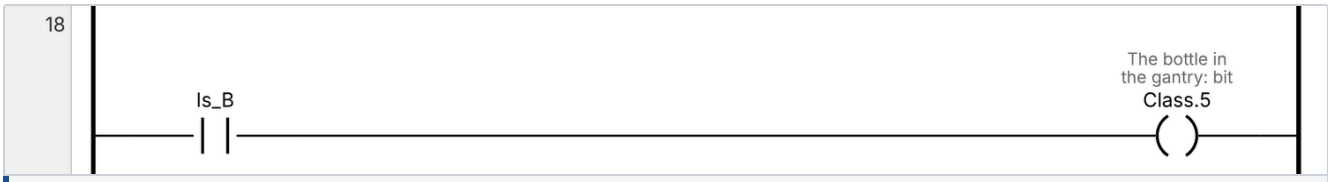
HOW THIS RUNG WORKS

In words. When `Is_R` (Latched: the colour sensor saw red) is on, the rung turns `Class.4` on (and off again whenever the rung is false).

Left to right:

1. `XIC(Is_R)` *Examine If Closed* asks: true when `Is_R` (Latched: the colour sensor saw red) is on.
2. `OTE(Class.4)` *Output Energize* does: turns `Class.4` on (and off again whenever the rung is false).

RUNG 18




```
XIC(Is_B)OTE(Class.5);
```

HOW THIS RUNG WORKS

In words. When `Is_B` (Latched: the colour sensor saw blue) is on, the rung turns `Class.5` on (and off again whenever the rung is false).

Left to right:

1. `XIC(Is_B)` *Examine If Closed* asks: true when `Is_B` (Latched: the colour sensor saw blue) is on.
2. `OTE(Class.5)` *Output Energize* does: turns `Class.5` on (and off again whenever the rung is false).

RUNG 19

```
XIC(Is_G)OTE(Class.6);
```

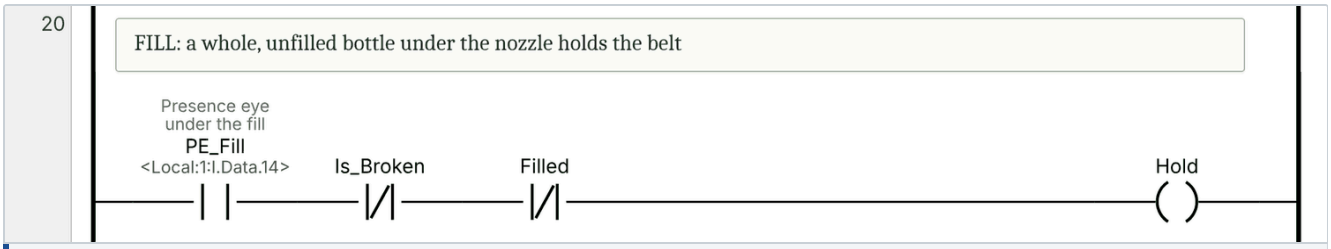
HOW THIS RUNG WORKS

In words. When `Is_G` (Latched: the colour sensor saw green) is on, the rung turns `Class.6` on (and off again whenever the rung is false).

Left to right:

1. `XIC(Is_G)` *Examine If Closed* asks: true when `Is_G` (Latched: the colour sensor saw green) is on.
2. `OTE(Class.6)` *Output Energize* does: turns `Class.6` on (and off again whenever the rung is false).

RUNG 20




```
XIC(PE_Fill)XIO(Is_Broken)XIO(Filled)OTE(Hold);
```

PREDICT IT · 5 `PE_Fill` is on, `Is_Broken` is on and `Filled` is off. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

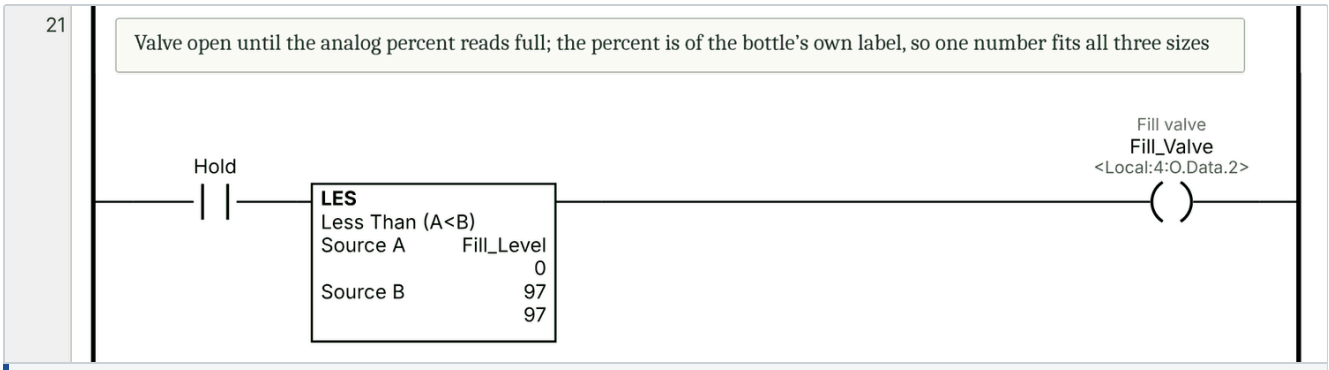
HOW THIS RUNG WORKS

In words. When `PE_Fill` (Presence eye under the fill nozzle) is on, `Is_Broken` (Latched: the crack eye saw a broken bottle) is off and `Filled` (This bottle has its charge) is off, the rung turns `Hold` (The filler holds the belt) on (and off again whenever the rung is false).

Left to right:

1. `XIC(PE_Fill)` *Examine If Closed* asks: true when `PE_Fill` (Presence eye under the fill nozzle) is on.
2. `XIO(Is_Broken)` *Examine If Open* asks: true when `Is_Broken` (Latched: the crack eye saw a broken bottle) is off.
3. `XIO(Filled)` *Examine If Open* asks: true when `Filled` (This bottle has its charge) is off.
4. `OTE(Hold)` *Output Energize* does: turns `Hold` (The filler holds the belt) on (and off again whenever the rung is false).

RUNG 21




```
XIC(Hold)LES(Fill_Level,97)OTE(Fill_Valve);
```

PREDICT IT · 6 `Hold` is on and `Fill_Level` is 96. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

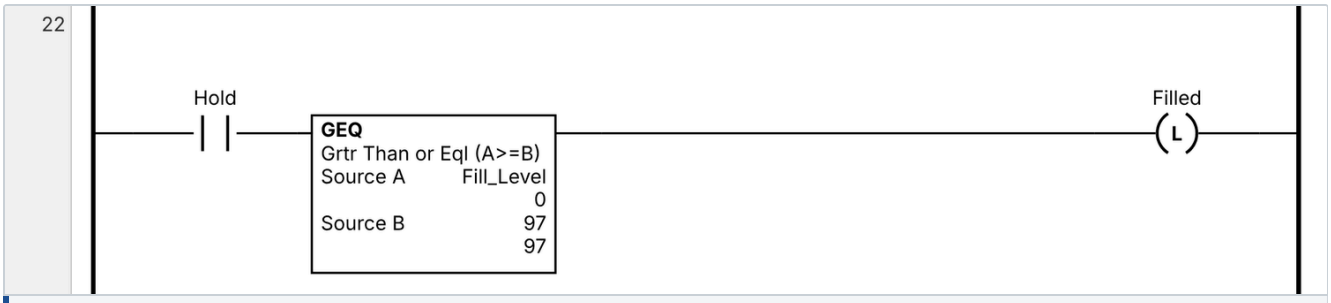
HOW THIS RUNG WORKS

In words. When `Hold` (The filler holds the belt) is on and `Fill_Level` (Fill of the bottle under the nozzle, 0-100 % of its label) is less than **97**, the rung turns `Fill_Valve` (Fill valve) on (and off again whenever the rung is false).

Left to right:

1. `XIC(Hold)` *Examine If Closed* asks: true when `Hold` (The filler holds the belt) is on.
2. `LES(Fill_Level,97)` *Less Than* asks: true when `Fill_Level` (Fill of the bottle under the nozzle, 0-100 % of its label) is less than **97**.
3. `OTE(Fill_Valve)` *Output Energize* does: turns `Fill_Valve` (Fill valve) on (and off again whenever the rung is false).

RUNG 22




```
XIC(Hold)GEQ(Fill_Level,97)OTL(Filled);
```

PREDICT IT · 7 `Hold` is on and `Fill_Level` is 96. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Hold` (The filler holds the belt) is on and `Fill_Level` (Fill of the bottle under the nozzle, 0-100 % of its label) is at least **97**, the rung latches `Filled` (This bottle has its charge) on.

Left to right:

1. `XIC(Hold)` *Examine If Closed* asks: true when `Hold` (The filler holds the belt) is on.
2. `GEQ(Fill_Level,97)` *Greater Than or Equal* asks: true when `Fill_Level` (Fill of the bottle under the nozzle, 0-100 % of its label) is at least **97**.
3. `OTL(Filled)` *Output Latch* does: latches `Filled` (This bottle has its charge) on.

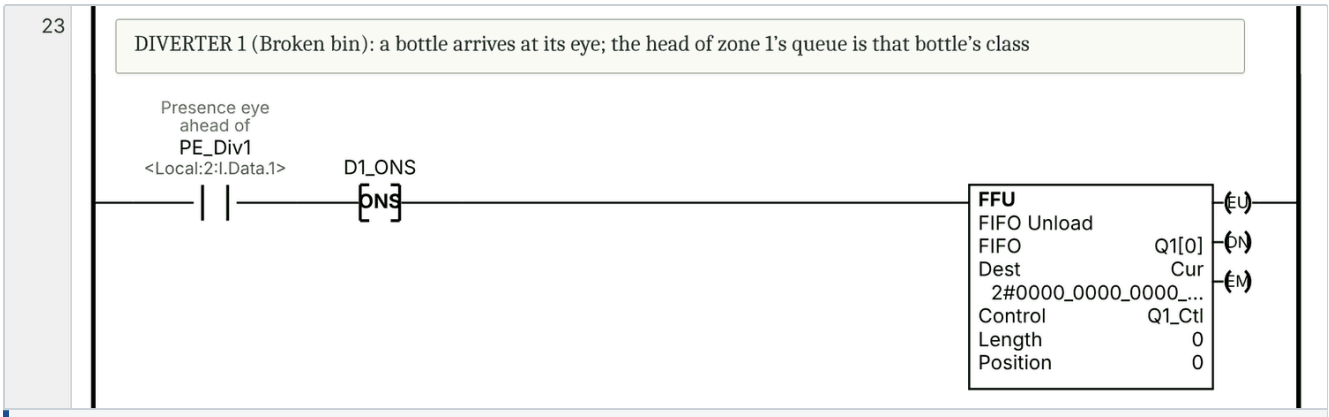
Why it is built this way:

A latch: it stays on until an OTU clears it.

Identical to Level 4. Now the colour bits of `Class` — 4 red, 5 blue, 6 green — are needed.

Diverter 1: broken

RUNG 23




```
XIC(PE_Div1)ONS(D1_ONS)FFU(Q1[0],Cur,Q1_Ctl,8);
```

HOW THIS RUNG WORKS

In words. When `PE_Div1` (Presence eye ahead of diverter 1) is on and it is the first scan they are true (the one-shot, remembered in `D1_ONS`), the rung unloads the oldest entry of `Q1[0]` into `Cur` (The class unloaded from a zone queue at a diverter).

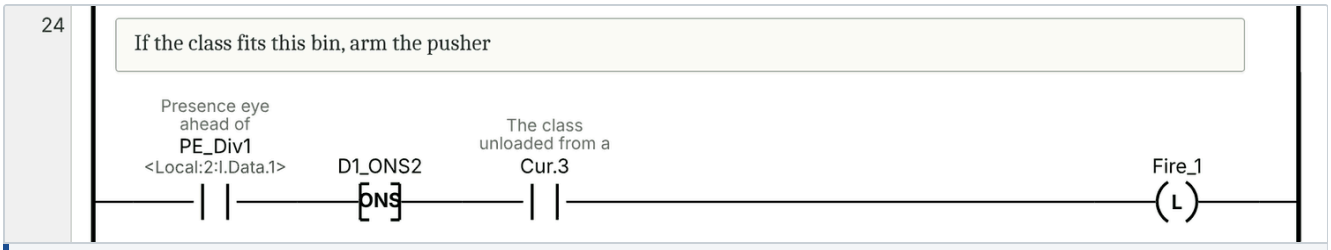
Left to right:

1. `XIC(PE_Div1)` *Examine If Closed* asks: true when `PE_Div1` (Presence eye ahead of diverter 1) is on.
2. `ONS(D1_ONS)` *One Shot* asks: true when it is the first scan they are true (the one-shot, remembered in `D1_ONS`).
3. `FFU(Q1[0],Cur,Q1_Ctl,8)` *FIFO Unload* does: unloads the oldest entry of `Q1[0]` into `Cur` (The class unloaded from a zone queue at a diverter). One entry per false-to-true transition.

Why it is built this way:

One-shot: once per event, not once per scan.

RUNG 24




```
XIC(PE_Div1)ONS(D1_ONS2)XIC(Cur.3)OTL(Fire_1);
```

HOW THIS RUNG WORKS

In words. When `PE_Div1` (Presence eye ahead of diverter 1) is on, it is the first scan they are true (the one-shot, remembered in `D1_ONS2`) and `Cur.3` is on, the rung latches `Fire_1` on.

Left to right:

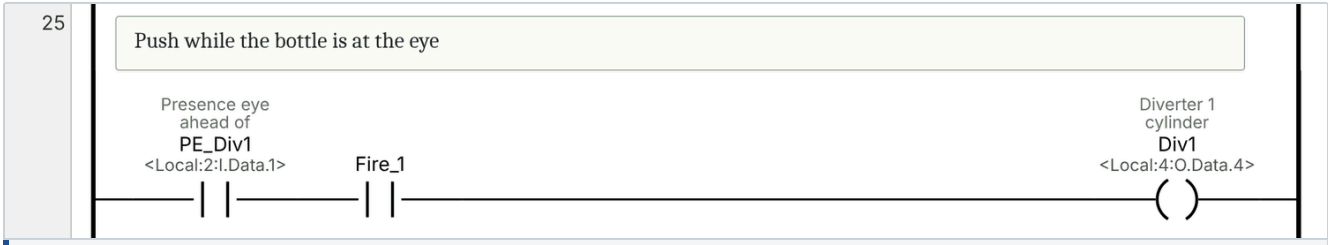
1. `XIC(PE_Div1)` *Examine If Closed* asks: true when `PE_Div1` (Presence eye ahead of diverter 1) is on.
2. `ONS(D1_ONS2)` *One Shot* asks: true when it is the first scan they are true (the one-shot, remembered in `D1_ONS2`).
3. `XIC(Cur.3)` *Examine If Closed* asks: true when `Cur.3` is on.
4. `OTL(Fire_1)` *Output Latch* does: latches `Fire_1` on.

Why it is built this way:

One-shot: once per event, not once per scan.

A latch: it stays on until an OTU clears it.

RUNG 25




```
XIC(PE_Div1)XIC(Fire_1)OTE(Div1);
```

PREDICT IT · 8 `PE_Div1` is on and `Fire_1` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

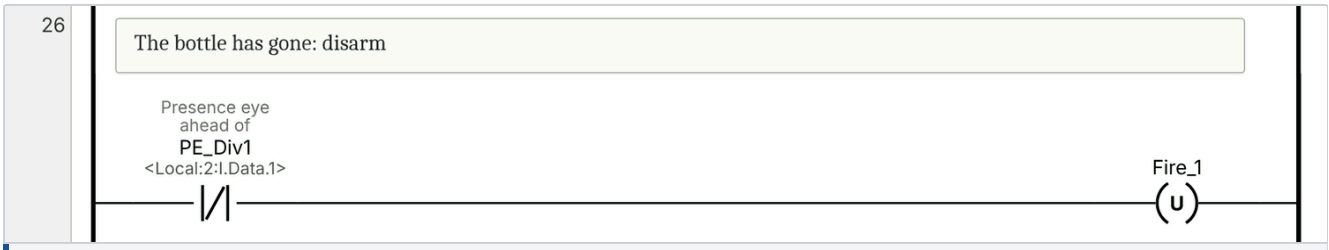
HOW THIS RUNG WORKS

In words. When `PE_Div1` (Presence eye ahead of diverter 1) is on and `Fire_1` is on, the rung turns `Div1` (Diverter 1 cylinder) on (and off again whenever the rung is false).

Left to right:

1. `XIC(PE_Div1)` *Examine If Closed* asks: true when `PE_Div1` (Presence eye ahead of diverter 1) is on.
2. `XIC(Fire_1)` *Examine If Closed* asks: true when `Fire_1` is on.
3. `OTE(Div1)` *Output Energize* does: turns `Div1` (Diverter 1 cylinder) on (and off again whenever the rung is false).

RUNG 26




```
XIO(PE_Div1)OTU(Fire_1);
```

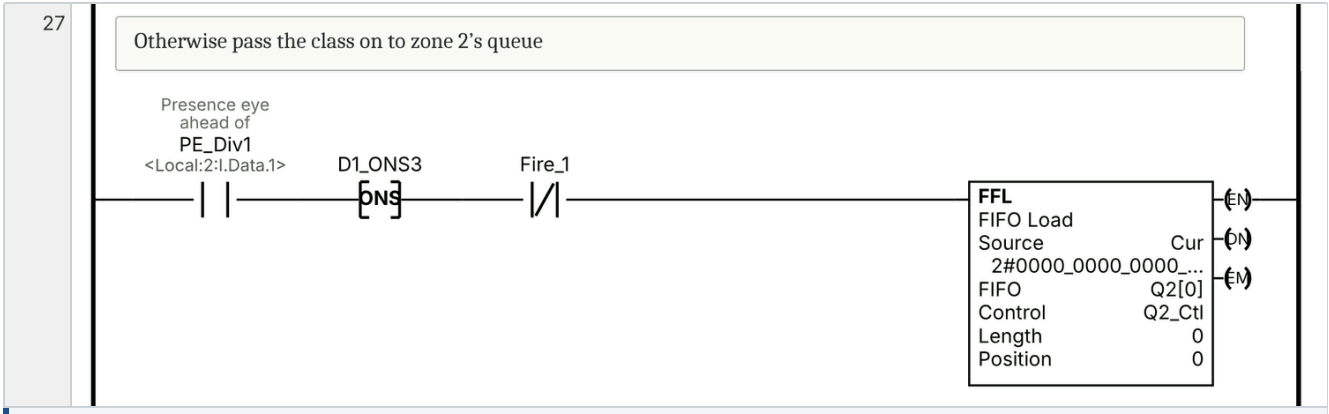
HOW THIS RUNG WORKS

In words. When `PE_Div1` (Presence eye ahead of diverter 1) is off, the rung unlatches `Fire_1` (turns it off).

Left to right:

1. `XIO(PE_Div1)` *Examine If Open* asks: true when `PE_Div1` (Presence eye ahead of diverter 1) is off.
2. `OTU(Fire_1)` *Output Unlatch* does: unlatches `Fire_1` (turns it off).

RUNG 27




```
XIC(PE_Div1)ONS(D1_ONS3)XIO(Fire_1)FFL(Cur,Q2[0],Q2_Ctl,8);
```

HOW THIS RUNG WORKS

In words. When `PE_Div1` (Presence eye ahead of diverter 1) is on, it is the first scan they are true (the one-shot, remembered in `D1_ONS3`) and `Fire_1` is off, the rung loads `Cur` (The class unloaded from a zone queue at a diverter) into the queue `Q2[0]`.

Left to right:

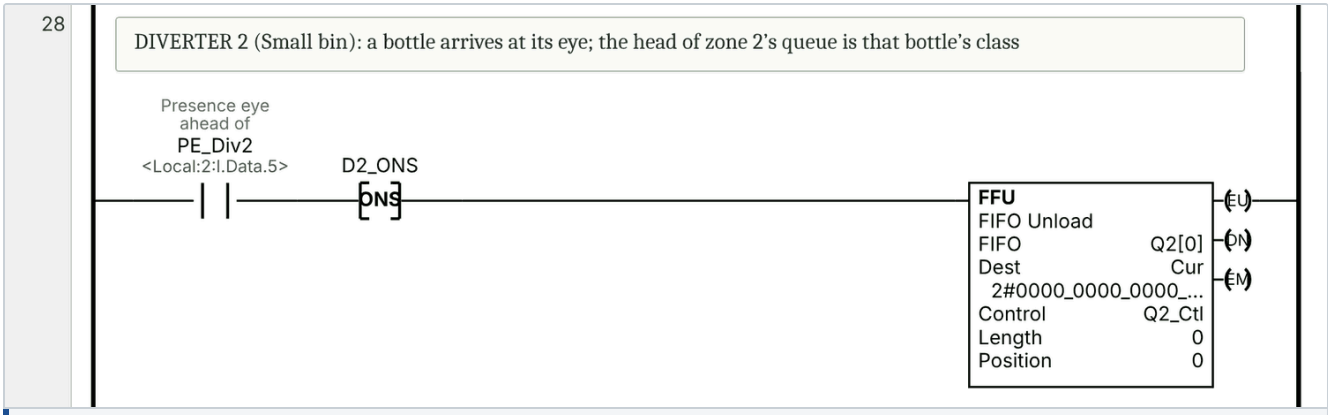
1. `XIC(PE_Div1)` *Examine If Closed* asks: true when `PE_Div1` (Presence eye ahead of diverter 1) is on.
2. `ONS(D1_ONS3)` *One Shot* asks: true when it is the first scan they are true (the one-shot, remembered in `D1_ONS3`).
3. `XIO(Fire_1)` *Examine If Open* asks: true when `Fire_1` is off.
4. `FFL(Cur,Q2[0],Q2_Ctl,8)` *FIFO Load* does: loads `Cur` (The class unloaded from a zone queue at a diverter) into the queue `Q2[0]`.

Why it is built this way:

One-shot: once per event, not once per scan.

Diverter 2: small

RUNG 28




```
XIC(PE_Div2)ONS(D2_ONS)FFU(Q2[0],Cur,Q2_Ctl,8);
```

HOW THIS RUNG WORKS

In words. When `PE_Div2` (Presence eye ahead of diverter 2) is on and it is the first scan they are true (the one-shot, remembered in `D2_ONS`), the rung unloads the oldest entry of `Q2[0]` into `Cur` (The class unloaded from a zone queue at a diverter).

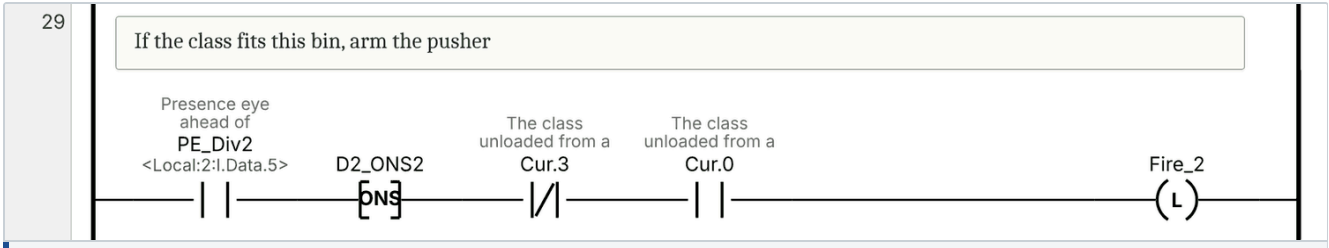
Left to right:

1. `XIC(PE_Div2)` *Examine If Closed* asks: true when `PE_Div2` (Presence eye ahead of diverter 2) is on.
2. `ONS(D2_ONS)` *One Shot* asks: true when it is the first scan they are true (the one-shot, remembered in `D2_ONS`).
3. `FFU(Q2[0],Cur,Q2_Ctl,8)` *FIFO Unload* does: unloads the oldest entry of `Q2[0]` into `Cur` (The class unloaded from a zone queue at a diverter).

Why it is built this way:

One-shot: once per event, not once per scan.

RUNG 29




```
XIC(PE_Div2)ONS(D2_ONS2)XIO(Cur.3)XIC(Cur.0)OTL(Fire_2);
```

HOW THIS RUNG WORKS

In words. When `PE_Div2` (Presence eye ahead of diverter 2) is on, it is the first scan they are true (the one-shot, remembered in `D2_ONS2`), `Cur.3` is off and `Cur.0` is on, the rung latches `Fire_2` on.

Left to right:

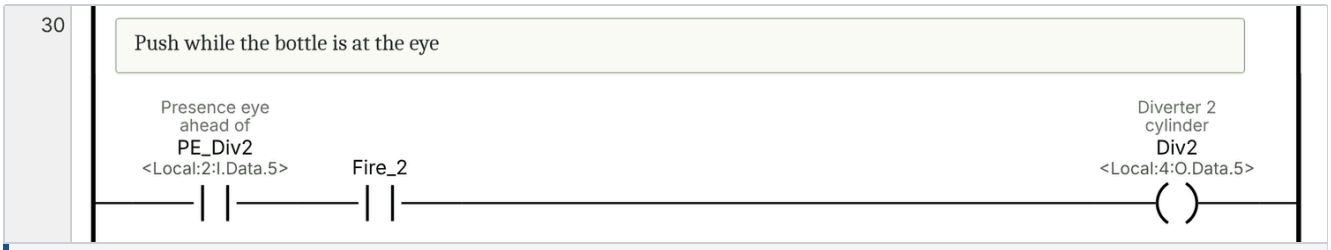
1. `XIC(PE_Div2)` *Examine If Closed* asks: true when `PE_Div2` (Presence eye ahead of diverter 2) is on.
2. `ONS(D2_ONS2)` *One Shot* asks: true when it is the first scan they are true (the one-shot, remembered in `D2_ONS2`).
3. `XIO(Cur.3)` *Examine If Open* asks: true when `Cur.3` is off.
4. `XIC(Cur.0)` *Examine If Closed* asks: true when `Cur.0` is on.
5. `OTL(Fire_2)` *Output Latch* does: latches `Fire_2` on.

Why it is built this way:

One-shot: once per event, not once per scan.

A latch: it stays on until an OTU clears it.

RUNG 30




```
XIC(PE_Div2)XIC(Fire_2)OTE(Div2);
```

PREDICT IT · 9 `PE_Div2` is on and `Fire_2` is off. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

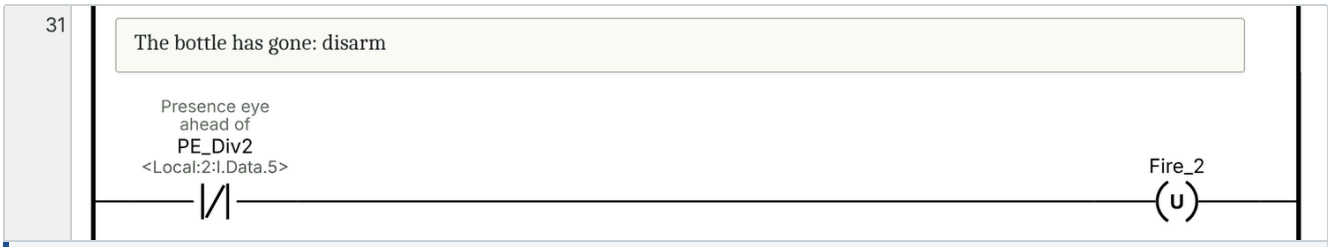
HOW THIS RUNG WORKS

In words. When `PE_Div2` (Presence eye ahead of diverter 2) is on and `Fire_2` is on, the rung turns `Div2` (Diverter 2 cylinder) on (and off again whenever the rung is false).

Left to right:

1. `XIC(PE_Div2)` *Examine If Closed* asks: true when `PE_Div2` (Presence eye ahead of diverter 2) is on.
2. `XIC(Fire_2)` *Examine If Closed* asks: true when `Fire_2` is on.
3. `OTE(Div2)` *Output Energize* does: turns `Div2` (Diverter 2 cylinder) on (and off again whenever the rung is false).

RUNG 31




```
XIO(PE_Div2)OTU(Fire_2);
```

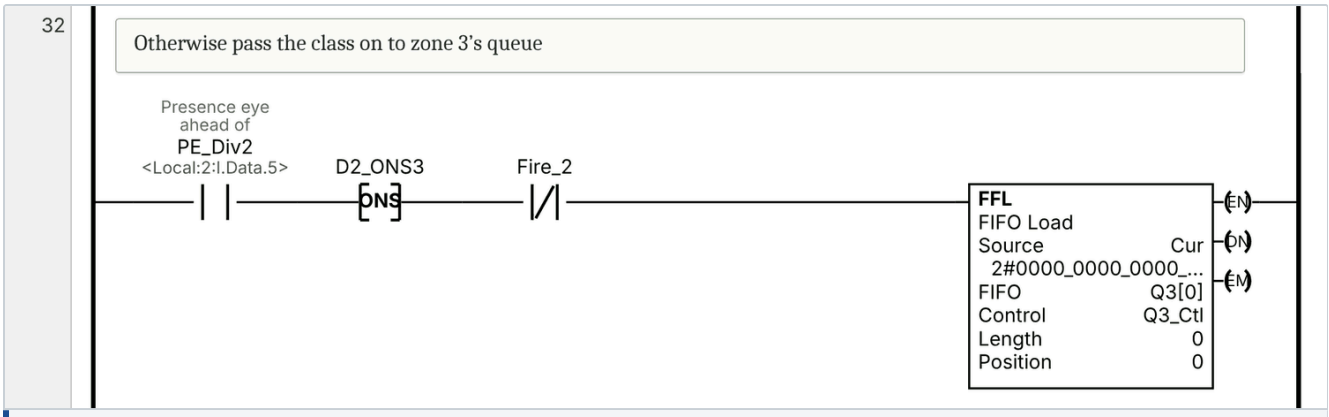
HOW THIS RUNG WORKS

In words. When `PE_Div2` (Presence eye ahead of diverter 2) is off, the rung unlatches `Fire_2` (turns it off).

Left to right:

1. `XIO(PE_Div2)` *Examine If Open* asks: true when `PE_Div2` (Presence eye ahead of diverter 2) is off.
2. `OTU(Fire_2)` *Output Unlatch* does: unlatches `Fire_2` (turns it off).

RUNG 32




```
XIC(PE_Div2)ONS(D2_ONS3)XIO(Fire_2)FFL(Cur,Q3[0],Q3_Ctl,8);
```

HOW THIS RUNG WORKS

In words. When `PE_Div2` (Presence eye ahead of diverter 2) is on, it is the first scan they are true (the one-shot, remembered in `D2_ONS3`) and `Fire_2` is off, the rung loads `Cur` (The class unloaded from a zone queue at a diverter) into the queue `Q3[0]`.

Left to right:

1. `XIC(PE_Div2)` *Examine If Closed* asks: true when `PE_Div2` (Presence eye ahead of diverter 2) is on.
2. `ONS(D2_ONS3)` *One Shot* asks: true when it is the first scan they are true (the one-shot, remembered in `D2_ONS3`).
3. `XIO(Fire_2)` *Examine If Open* asks: true when `Fire_2` is off.
4. `FFL(Cur,Q3[0],Q3_Ctl,8)` *FIFO Load* does: loads `Cur` (The class unloaded from a zone queue at a diverter) into the queue `Q3[0]`.

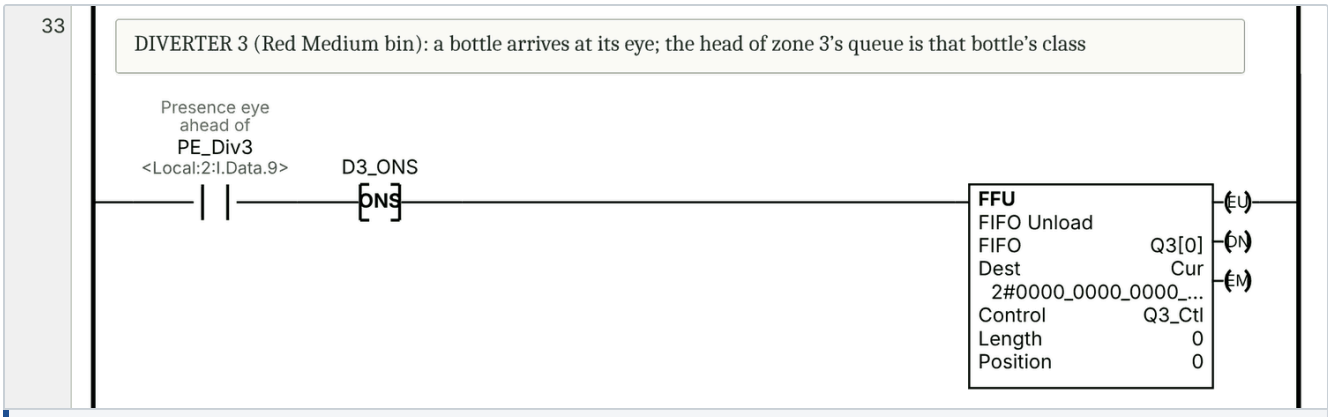
Why it is built this way:

One-shot: once per event, not once per scan.

Any small, whole bottle, any colour: `XIO(Cur.3)XIC(Cur.0)`.

Diverters 3, 4 and 5: mediums by colour

RUNG 33




```
XIC(PE_Div3)ONS(D3_ONS)FFU(Q3[0],Cur,Q3_Ctl,8);
```

HOW THIS RUNG WORKS

In words. When `PE_Div3` (Presence eye ahead of diverter 3) is on and it is the first scan they are true (the one-shot, remembered in `D3_ONS`), the rung unloads the oldest entry of `Q3[0]` into `Cur` (The class unloaded from a zone queue at a diverter).

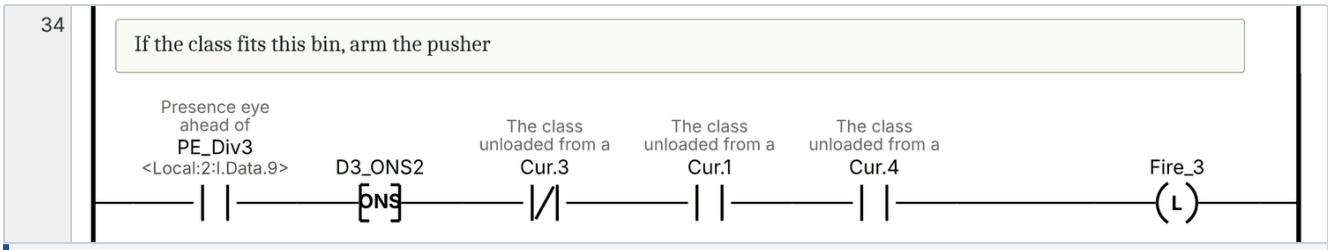
Left to right:

1. `XIC(PE_Div3)` *Examine If Closed* asks: true when `PE_Div3` (Presence eye ahead of diverter 3) is on.
2. `ONS(D3_ONS)` *One Shot* asks: true when it is the first scan they are true (the one-shot, remembered in `D3_ONS`).
3. `FFU(Q3[0],Cur,Q3_Ctl,8)` *FIFO Unload* does: unloads the oldest entry of `Q3[0]` into `Cur` (The class unloaded from a zone queue at a diverter).

Why it is built this way:

One-shot: once per event, not once per scan.

RUNG 34




```
XIC(PE_Div3)ONS(D3_ONS2)XIO(Cur.3)XIC(Cur.1)XIC(Cur.4)OTL(Fire_3);
```

HOW THIS RUNG WORKS

In words. When `PE_Div3` (Presence eye ahead of diverter 3) is on, it is the first scan they are true (the one-shot, remembered in `D3_ONS2`), `Cur.3` is off, `Cur.1` is on and `Cur.4` is on, the rung latches `Fire_3` on.

Left to right:

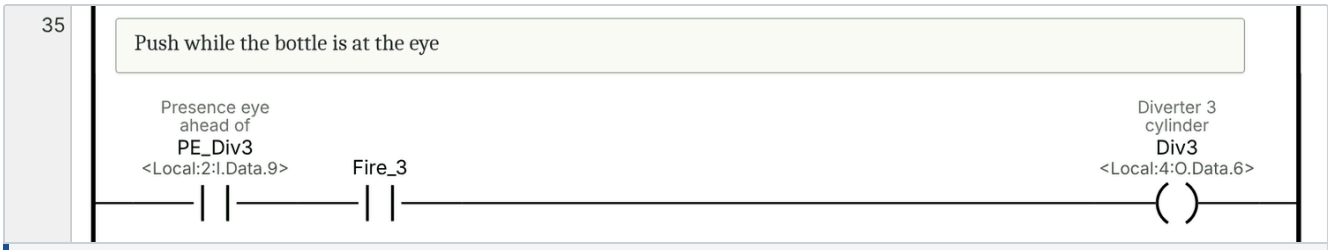
1. `XIC(PE_Div3)` *Examine If Closed* asks: true when `PE_Div3` (Presence eye ahead of diverter 3) is on.
2. `ONS(D3_ONS2)` *One Shot* asks: true when it is the first scan they are true (the one-shot, remembered in `D3_ONS2`).
3. `XIO(Cur.3)` *Examine If Open* asks: true when `Cur.3` is off.
4. `XIC(Cur.1)` *Examine If Closed* asks: true when `Cur.1` is on.
5. `XIC(Cur.4)` *Examine If Closed* asks: true when `Cur.4` is on.
6. `OTL(Fire_3)` *Output Latch* does: latches `Fire_3` on.

Why it is built this way:

One-shot: once per event, not once per scan.

A latch: it stays on until an OTU clears it.

RUNG 35




```
XIC(PE_Div3)XIC(Fire_3)OTE(Div3);
```

PREDICT IT · 10 `PE_Div3` is on and `Fire_3` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

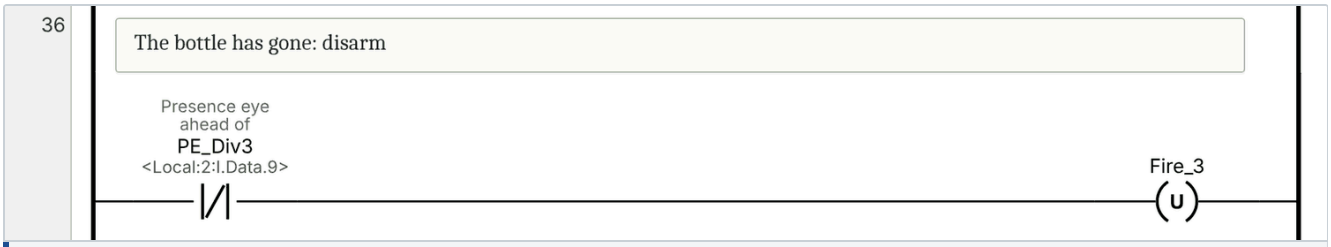
HOW THIS RUNG WORKS

In words. When `PE_Div3` (Presence eye ahead of diverter 3) is on and `Fire_3` is on, the rung turns `Div3` (Diverter 3 cylinder) on (and off again whenever the rung is false).

Left to right:

1. `XIC(PE_Div3)` *Examine If Closed* asks: true when `PE_Div3` (Presence eye ahead of diverter 3) is on.
2. `XIC(Fire_3)` *Examine If Closed* asks: true when `Fire_3` is on.
3. `OTE(Div3)` *Output Energize* does: turns `Div3` (Diverter 3 cylinder) on (and off again whenever the rung is false).

RUNG 36




```
XIO(PE_Div3)OTU(Fire_3);
```

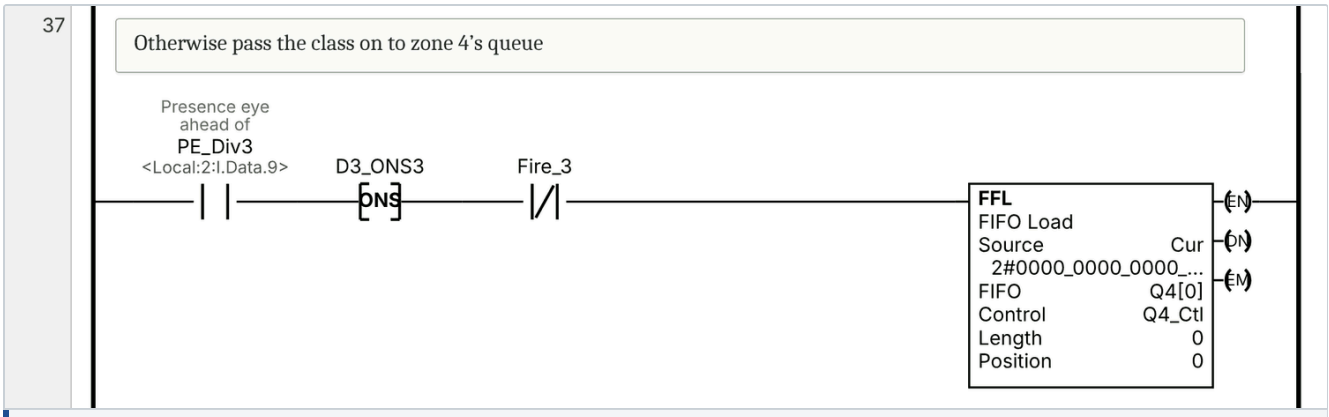
HOW THIS RUNG WORKS

In words. When `PE_Div3` (Presence eye ahead of diverter 3) is off, the rung unlatches `Fire_3` (turns it off).

Left to right:

1. `XIO(PE_Div3)` *Examine If Open* asks: true when `PE_Div3` (Presence eye ahead of diverter 3) is off.
2. `OTU(Fire_3)` *Output Unlatch* does: unlatches `Fire_3` (turns it off).

RUNG 37




```
XIC(PE_Div3)ONS(D3_ONS3)XIO(Fire_3)FFL(Cur,Q4[0],Q4_Ctl,8);
```

HOW THIS RUNG WORKS

In words. When `PE_Div3` (Presence eye ahead of diverter 3) is on, it is the first scan they are true (the one-shot, remembered in `D3_ONS3`) and `Fire_3` is off, the rung loads `Cur` (The class unloaded from a zone queue at a diverter) into the queue `Q4[0]`.

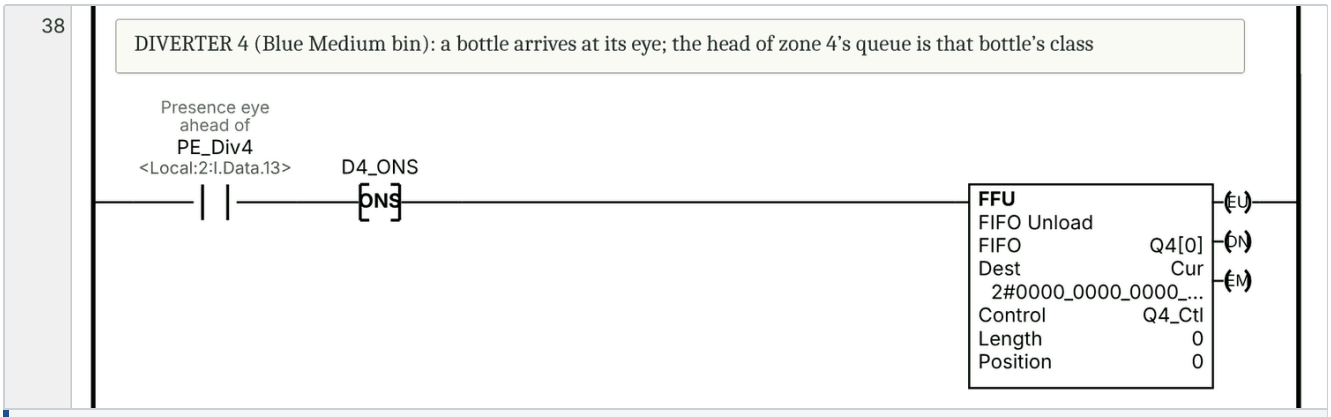
Left to right:

1. `XIC(PE_Div3)` *Examine If Closed* asks: true when `PE_Div3` (Presence eye ahead of diverter 3) is on.
2. `ONS(D3_ONS3)` *One Shot* asks: true when it is the first scan they are true (the one-shot, remembered in `D3_ONS3`).
3. `XIO(Fire_3)` *Examine If Open* asks: true when `Fire_3` is off.
4. `FFL(Cur,Q4[0],Q4_Ctl,8)` *FIFO Load* does: loads `Cur` (The class unloaded from a zone queue at a diverter) into the queue `Q4[0]`.

Why it is built this way:

One-shot: once per event, not once per scan.

RUNG 38




```
XIC(PE_Div4)ONS(D4_ONS)FFU(Q4[0],Cur,Q4_Ctl,8);
```

HOW THIS RUNG WORKS

In words. When `PE_Div4` (Presence eye ahead of diverter 4) is on and it is the first scan they are true (the one-shot, remembered in `D4_ONS`), the rung unloads the oldest entry of `Q4[0]` into `Cur` (The class unloaded from a zone queue at a diverter).

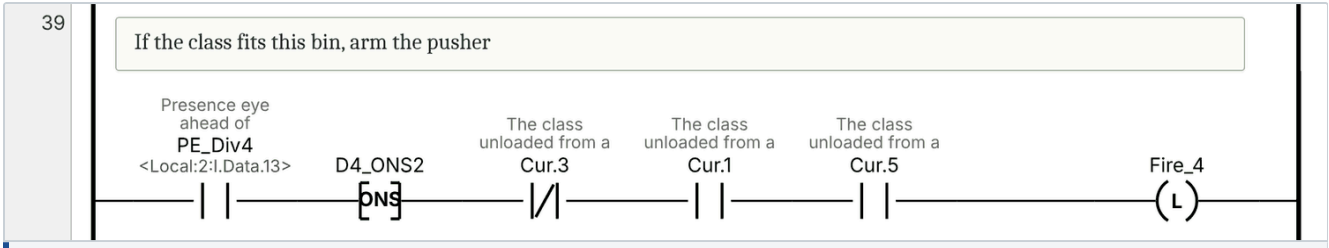
Left to right:

1. `XIC(PE_Div4)` *Examine If Closed* asks: true when `PE_Div4` (Presence eye ahead of diverter 4) is on.
2. `ONS(D4_ONS)` *One Shot* asks: true when it is the first scan they are true (the one-shot, remembered in `D4_ONS`).
3. `FFU(Q4[0],Cur,Q4_Ctl,8)` *FIFO Unload* does: unloads the oldest entry of `Q4[0]` into `Cur` (The class unloaded from a zone queue at a diverter).

Why it is built this way:

One-shot: once per event, not once per scan.

RUNG 39




```
XIC(PE_Div4)ONS(D4_ONS2)XIO(Cur.3)XIC(Cur.1)XIC(Cur.5)OTL(Fire_4);
```

HOW THIS RUNG WORKS

In words. When `PE_Div4` (Presence eye ahead of diverter 4) is on, it is the first scan they are true (the one-shot, remembered in `D4_ONS2`), `Cur.3` is off, `Cur.1` is on and `Cur.5` is on, the rung latches `Fire_4` on.

Left to right:

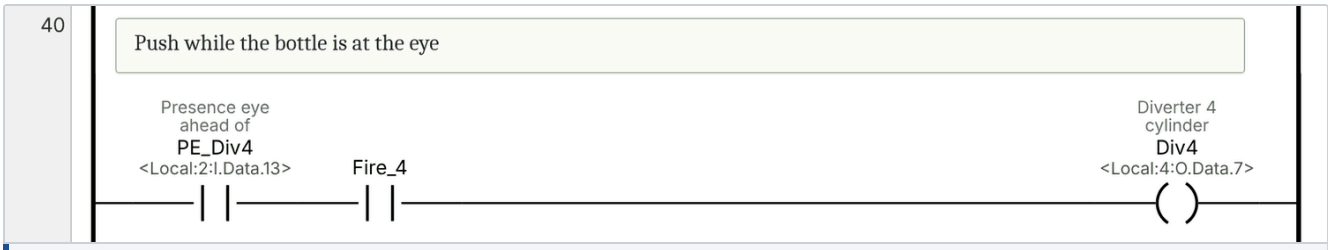
1. `XIC(PE_Div4)` *Examine If Closed* asks: true when `PE_Div4` (Presence eye ahead of diverter 4) is on.
2. `ONS(D4_ONS2)` *One Shot* asks: true when it is the first scan they are true (the one-shot, remembered in `D4_ONS2`).
3. `XIO(Cur.3)` *Examine If Open* asks: true when `Cur.3` is off.
4. `XIC(Cur.1)` *Examine If Closed* asks: true when `Cur.1` is on.
5. `XIC(Cur.5)` *Examine If Closed* asks: true when `Cur.5` is on.
6. `OTL(Fire_4)` *Output Latch* does: latches `Fire_4` on.

Why it is built this way:

One-shot: once per event, not once per scan.

A latch: it stays on until an OTU clears it.

RUNG 40




```
XIC(PE_Div4)XIC(Fire_4)OTE(Div4);
```

PREDICT IT · 11 `PE_Div4` is off and `Fire_4` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

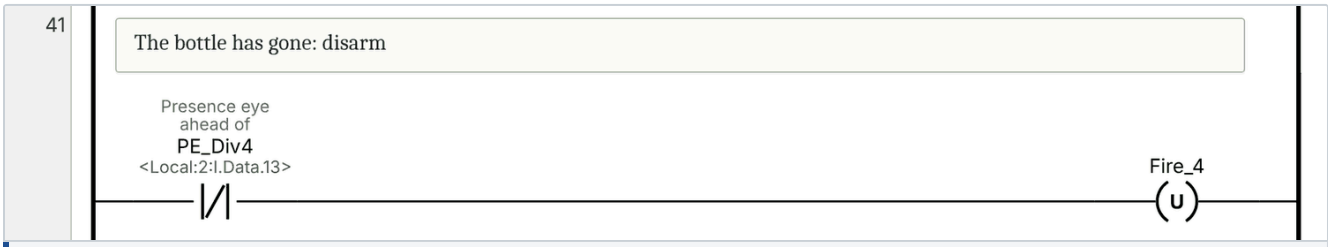
HOW THIS RUNG WORKS

In words. When `PE_Div4` (Presence eye ahead of diverter 4) is on and `Fire_4` is on, the rung turns `Div4` (Diverter 4 cylinder) on (and off again whenever the rung is false).

Left to right:

1. `XIC(PE_Div4)` *Examine If Closed* asks: true when `PE_Div4` (Presence eye ahead of diverter 4) is on.
2. `XIC(Fire_4)` *Examine If Closed* asks: true when `Fire_4` is on.
3. `OTE(Div4)` *Output Energize* does: turns `Div4` (Diverter 4 cylinder) on (and off again whenever the rung is false).

RUNG 41




```
XIO(PE_Div4)OTU(Fire_4);
```

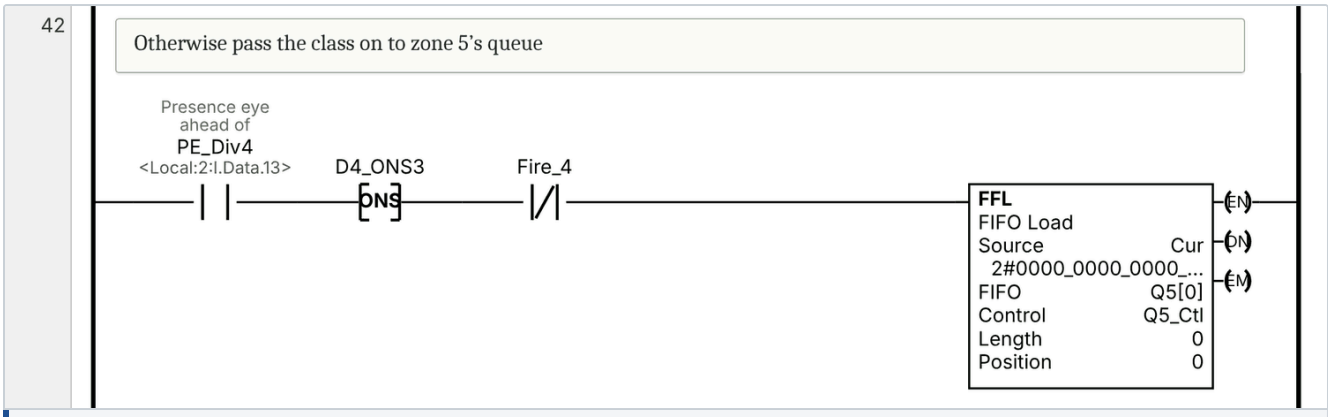
HOW THIS RUNG WORKS

In words. When `PE_Div4` (Presence eye ahead of diverter 4) is off, the rung unlatches `Fire_4` (turns it off).

Left to right:

1. `XIO(PE_Div4)` *Examine If Open* asks: true when `PE_Div4` (Presence eye ahead of diverter 4) is off.
2. `OTU(Fire_4)` *Output Unlatch* does: unlatches `Fire_4` (turns it off).

RUNG 42




```
XIC(PE_Div4)ONS(D4_ONS3)XIO(Fire_4)FFL(Cur,Q5[0],Q5_Ctl,8);
```

HOW THIS RUNG WORKS

In words. When `PE_Div4` (Presence eye ahead of diverter 4) is on, it is the first scan they are true (the one-shot, remembered in `D4_ONS3`) and `Fire_4` is off, the rung loads `Cur` (The class unloaded from a zone queue at a diverter) into the queue `Q5[0]`.

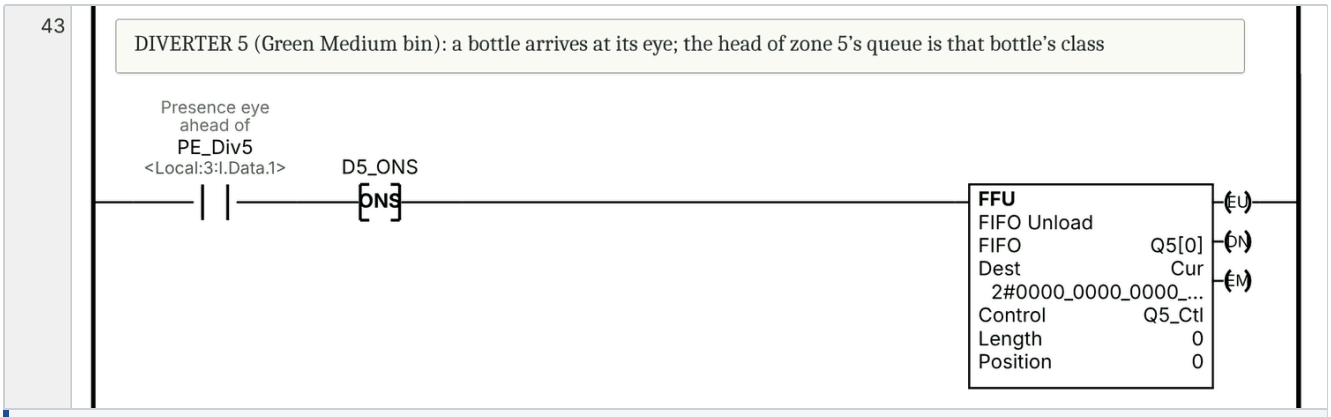
Left to right:

1. `XIC(PE_Div4)` *Examine If Closed* asks: true when `PE_Div4` (Presence eye ahead of diverter 4) is on.
2. `ONS(D4_ONS3)` *One Shot* asks: true when it is the first scan they are true (the one-shot, remembered in `D4_ONS3`).
3. `XIO(Fire_4)` *Examine If Open* asks: true when `Fire_4` is off.
4. `FFL(Cur,Q5[0],Q5_Ctl,8)` *FIFO Load* does: loads `Cur` (The class unloaded from a zone queue at a diverter) into the queue `Q5[0]`.

Why it is built this way:

One-shot: once per event, not once per scan.

RUNG 43




```
XIC(PE_Div5)ONS(D5_ONS)FFU(Q5[0],Cur,Q5_Ctl,8);
```

HOW THIS RUNG WORKS

In words. When `PE_Div5` (Presence eye ahead of diverter 5) is on and it is the first scan they are true (the one-shot, remembered in `D5_ONS`), the rung unloads the oldest entry of `Q5[0]` into `Cur` (The class unloaded from a zone queue at a diverter).

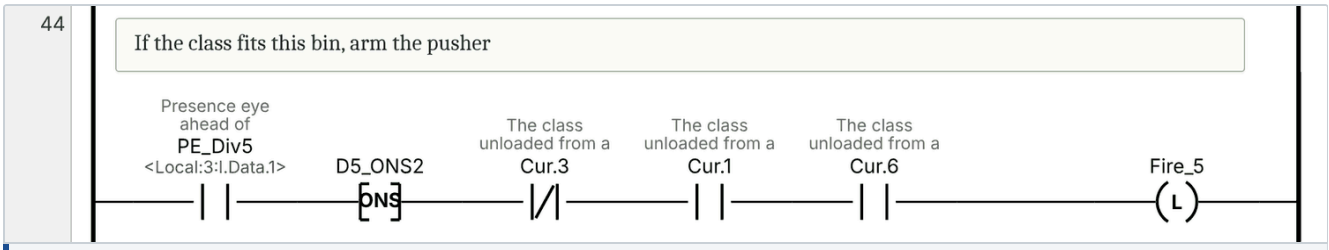
Left to right:

1. `XIC(PE_Div5)` *Examine If Closed* asks: true when `PE_Div5` (Presence eye ahead of diverter 5) is on.
2. `ONS(D5_ONS)` *One Shot* asks: true when it is the first scan they are true (the one-shot, remembered in `D5_ONS`).
3. `FFU(Q5[0],Cur,Q5_Ctl,8)` *FIFO Unload* does: unloads the oldest entry of `Q5[0]` into `Cur` (The class unloaded from a zone queue at a diverter).

Why it is built this way:

One-shot: once per event, not once per scan.

RUNG 44




```
XIC(PE_Div5)ONS(D5_ONS2)XIO(Cur.3)XIC(Cur.1)XIC(Cur.6)OTL(Fire_5);
```

HOW THIS RUNG WORKS

In words. When `PE_Div5` (Presence eye ahead of diverter 5) is on, it is the first scan they are true (the one-shot, remembered in `D5_ONS2`), `Cur.3` is off, `Cur.1` is on and `Cur.6` is on, the rung latches `Fire_5` on.

Left to right:

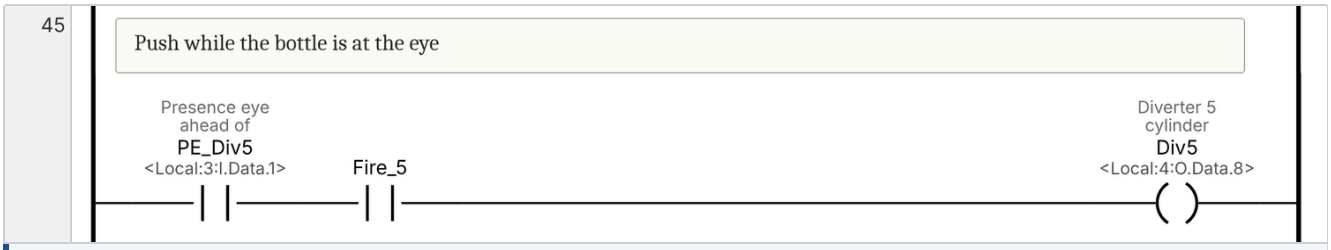
1. `XIC(PE_Div5)` *Examine If Closed* asks: true when `PE_Div5` (Presence eye ahead of diverter 5) is on.
2. `ONS(D5_ONS2)` *One Shot* asks: true when it is the first scan they are true (the one-shot, remembered in `D5_ONS2`).
3. `XIO(Cur.3)` *Examine If Open* asks: true when `Cur.3` is off.
4. `XIC(Cur.1)` *Examine If Closed* asks: true when `Cur.1` is on.
5. `XIC(Cur.6)` *Examine If Closed* asks: true when `Cur.6` is on.
6. `OTL(Fire_5)` *Output Latch* does: latches `Fire_5` on.

Why it is built this way:

One-shot: once per event, not once per scan.

A latch: it stays on until an OTU clears it.

RUNG 45




```
XIC(PE_Div5)XIC(Fire_5)OTE(Div5);
```

PREDICT IT · 12 `PE_Div5` is on and `Fire_5` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

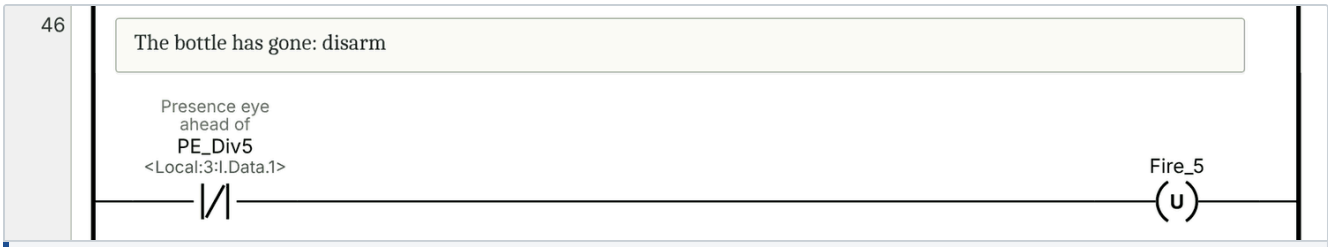
HOW THIS RUNG WORKS

In words. When `PE_Div5` (Presence eye ahead of diverter 5) is on and `Fire_5` is on, the rung turns `Div5` (Diverter 5 cylinder) on (and off again whenever the rung is false).

Left to right:

1. `XIC(PE_Div5)` *Examine If Closed* asks: true when `PE_Div5` (Presence eye ahead of diverter 5) is on.
2. `XIC(Fire_5)` *Examine If Closed* asks: true when `Fire_5` is on.
3. `OTE(Div5)` *Output Energize* does: turns `Div5` (Diverter 5 cylinder) on (and off again whenever the rung is false).

RUNG 46




```
XIO(PE_Div5)OTU(Fire_5);
```

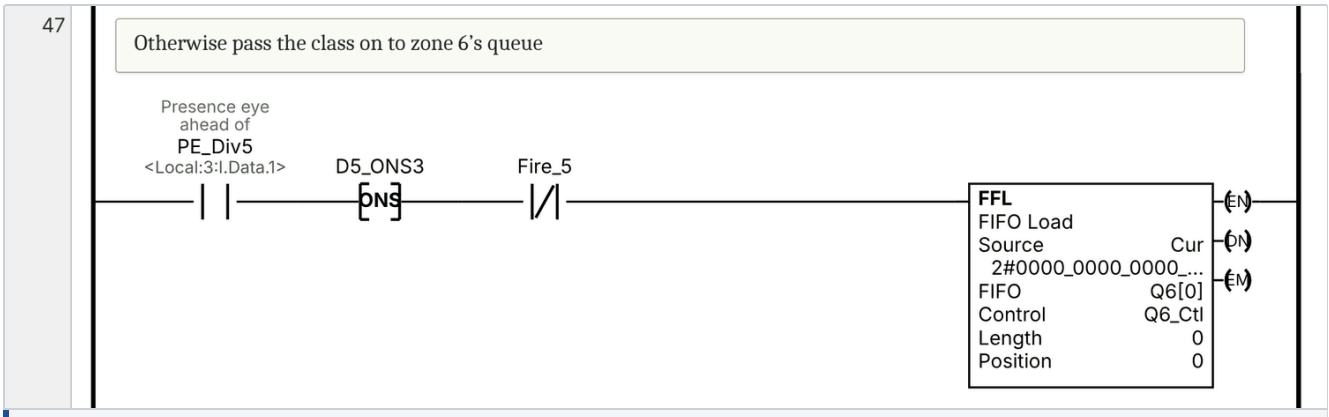
HOW THIS RUNG WORKS

In words. When `PE_Div5` (Presence eye ahead of diverter 5) is off, the rung unlatches `Fire_5` (turns it off).

Left to right:

1. `XIO(PE_Div5)` *Examine If Open* asks: true when `PE_Div5` (Presence eye ahead of diverter 5) is off.
2. `OTU(Fire_5)` *Output Unlatch* does: unlatches `Fire_5` (turns it off).

RUNG 47




```
XIC(PE_Div5)ONS(D5_ONS3)XIO(Fire_5)FFL(Cur,Q6[0],Q6_Ctl,8);
```

HOW THIS RUNG WORKS

In words. When `PE_Div5` (Presence eye ahead of diverter 5) is on, it is the first scan they are true (the one-shot, remembered in `D5_ONS3`) and `Fire_5` is off, the rung loads `Cur` (The class unloaded from a zone queue at a diverter) into the queue `Q6[0]`.

Left to right:

1. `XIC(PE_Div5)` *Examine If Closed* asks: true when `PE_Div5` (Presence eye ahead of diverter 5) is on.
2. `ONS(D5_ONS3)` *One Shot* asks: true when it is the first scan they are true (the one-shot, remembered in `D5_ONS3`).
3. `XIO(Fire_5)` *Examine If Open* asks: true when `Fire_5` is off.
4. `FFL(Cur,Q6[0],Q6_Ctl,8)` *FIFO Load* does: loads `Cur` (The class unloaded from a zone queue at a diverter) into the queue `Q6[0]`.

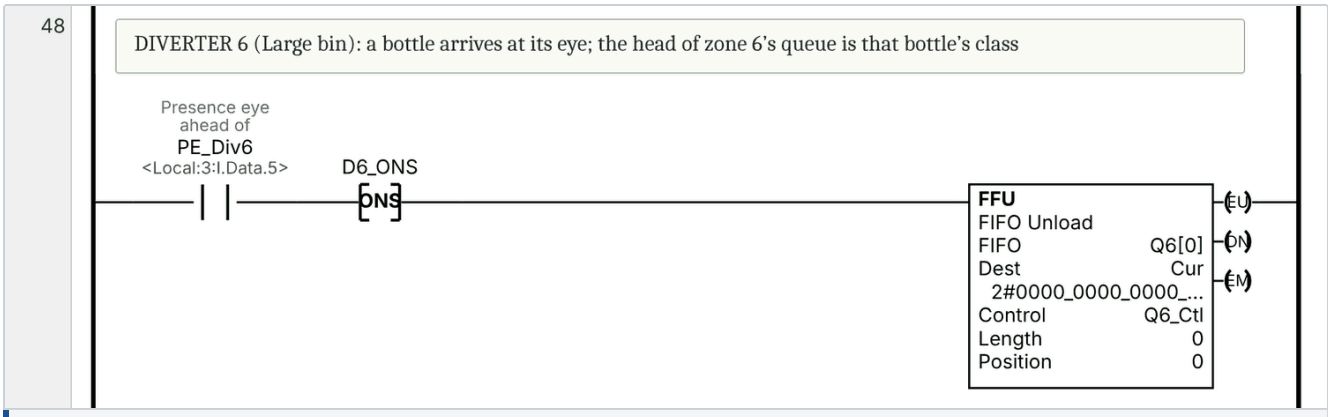
Why it is built this way:

One-shot: once per event, not once per scan.

Each tests three bits: not broken, medium, and its colour — `XIO(Cur.3)XIC(Cur.1)XIC(Cur.4)` for red, `…XIC(Cur.5)` for blue, `…XIC(Cur.6)` for green. A medium bottle that is not the colour of bin 3 is passed on to zone 4, and so on, until the bin that wants it.

Diverter 6: large

RUNG 48




```
XIC(PE_Div6)ONS(D6_ONS)FFU(Q6[0],Cur,Q6_Ctl,8);
```

HOW THIS RUNG WORKS

In words. When `PE_Div6` (Presence eye ahead of diverter 6) is on and it is the first scan they are true (the one-shot, remembered in `D6_ONS`), the rung unloads the oldest entry of `Q6[0]` into `Cur` (The class unloaded from a zone queue at a diverter).

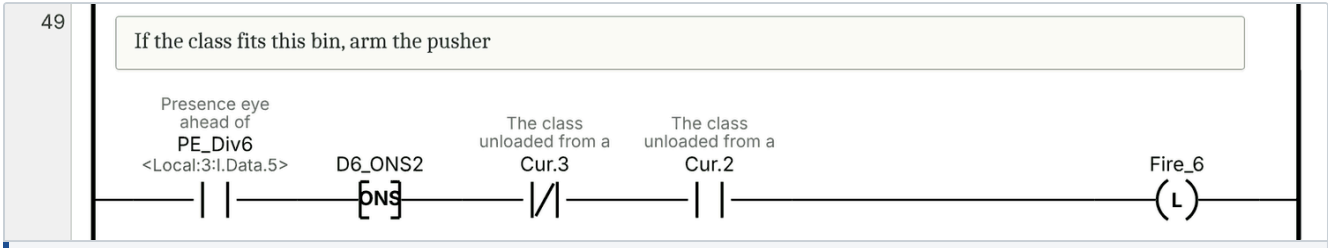
Left to right:

1. `XIC(PE_Div6)` *Examine If Closed* asks: true when `PE_Div6` (Presence eye ahead of diverter 6) is on.
2. `ONS(D6_ONS)` *One Shot* asks: true when it is the first scan they are true (the one-shot, remembered in `D6_ONS`).
3. `FFU(Q6[0],Cur,Q6_Ctl,8)` *FIFO Unload* does: unloads the oldest entry of `Q6[0]` into `Cur` (The class unloaded from a zone queue at a diverter).

Why it is built this way:

One-shot: once per event, not once per scan.

RUNG 49




```
XIC(PE_Div6)ONS(D6_ONS2)XIO(Cur.3)XIC(Cur.2)OTL(Fire_6);
```

HOW THIS RUNG WORKS

In words. When `PE_Div6` (Presence eye ahead of diverter 6) is on, it is the first scan they are true (the one-shot, remembered in `D6_ONS2`), `Cur.3` is off and `Cur.2` is on, the rung latches `Fire_6` on.

Left to right:

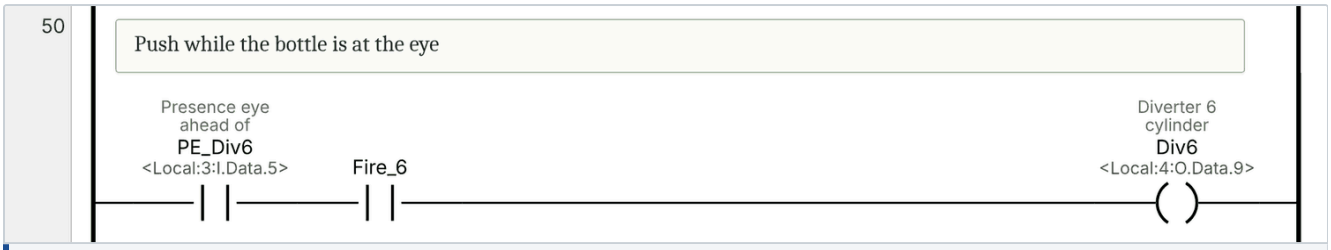
1. `XIC(PE_Div6)` *Examine If Closed* asks: true when `PE_Div6` (Presence eye ahead of diverter 6) is on.
2. `ONS(D6_ONS2)` *One Shot* asks: true when it is the first scan they are true (the one-shot, remembered in `D6_ONS2`).
3. `XIO(Cur.3)` *Examine If Open* asks: true when `Cur.3` is off.
4. `XIC(Cur.2)` *Examine If Closed* asks: true when `Cur.2` is on.
5. `OTL(Fire_6)` *Output Latch* does: latches `Fire_6` on.

Why it is built this way:

One-shot: once per event, not once per scan.

A latch: it stays on until an OTU clears it.

RUNG 50




```
XIC(PE_Div6)XIC(Fire_6)OTE(Div6);
```

PREDICT IT · 13 `PE_Div6` is on and `Fire_6` is off. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

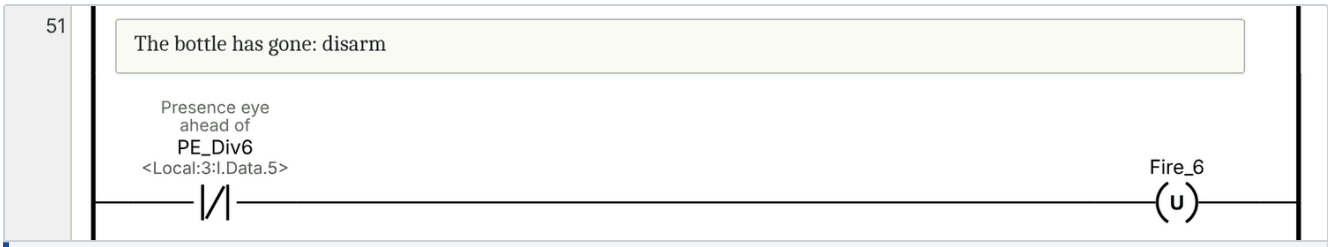
HOW THIS RUNG WORKS

In words. When `PE_Div6` (Presence eye ahead of diverter 6) is on and `Fire_6` is on, the rung turns `Div6` (Diverter 6 cylinder) on (and off again whenever the rung is false).

Left to right:

1. `XIC(PE_Div6)` *Examine If Closed* asks: true when `PE_Div6` (Presence eye ahead of diverter 6) is on.
2. `XIC(Fire_6)` *Examine If Closed* asks: true when `Fire_6` is on.
3. `OTE(Div6)` *Output Energize* does: turns `Div6` (Diverter 6 cylinder) on (and off again whenever the rung is false).

RUNG 51




```
XIO(PE_Div6)OTU(Fire_6);
```

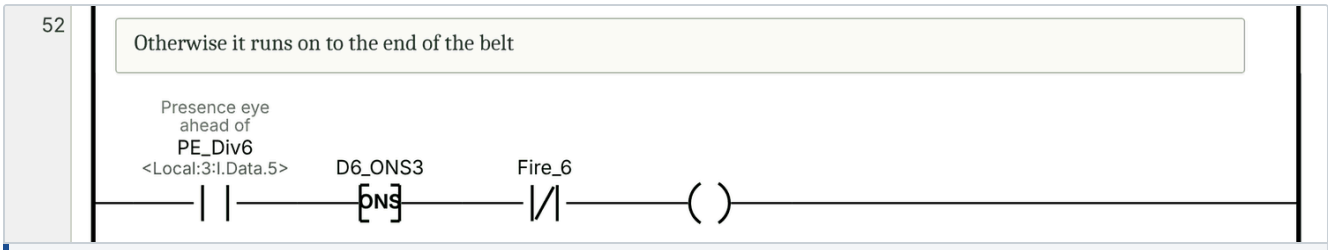
HOW THIS RUNG WORKS

In words. When `PE_Div6` (Presence eye ahead of diverter 6) is off, the rung unlatches `Fire_6` (turns it off).

Left to right:

1. `XIO(PE_Div6)` *Examine If Open* asks: true when `PE_Div6` (Presence eye ahead of diverter 6) is off.
2. `OTU(Fire_6)` *Output Unlatch* does: unlatches `Fire_6` (turns it off).

RUNG 52




```
XIC(PE_Div6)ONS(D6_ONS3)XIO(Fire_6)NOP();
```

HOW THIS RUNG WORKS

In words. When `PE_Div6` (Presence eye ahead of diverter 6) is on, it is the first scan they are true (the one-shot, remembered in `D6_ONS3`) and `Fire_6` is off, the rung does nothing (a placeholder).

Left to right:

1. `XIC(PE_Div6)` *Examine If Closed* asks: true when `PE_Div6` (Presence eye ahead of diverter 6) is on.
2. `ONS(D6_ONS3)` *One Shot* asks: true when it is the first scan they are true (the one-shot, remembered in `D6_ONS3`).
3. `XIO(Fire_6)` *Examine If Open* asks: true when `Fire_6` is off.
4. `NOP()` *No Operation* does: does nothing (a placeholder).

Why it is built this way:

One-shot: once per event, not once per scan.

WATCH FOR: A MATCH THAT TESTS TOO LITTLE

Write the red-medium rung as `XIC(Cur.4)` alone and it looks right in a test with only medium bottles. Then a *large* red bottle comes along — and bin 3 is upstream of bin 6, so it gets there first and takes it: −50. Bin 2 already took every small bottle, so a small red bottle would never reach bin 3 in this order — but reorder the bins and that hides a second bug. Write every match with every bit it depends on, and the order of the bins stops mattering.

Run it

Sixteen bottles into six bins. Turn on Specs to see each bin's rule, and keep the Simulator Log open: every line names the bottle, its colour and size, and the bin. The reference run: 16 of 16 bottles in the right bin, nothing lost, in 95 s against a par of 120 s: 1800 points and three stars.

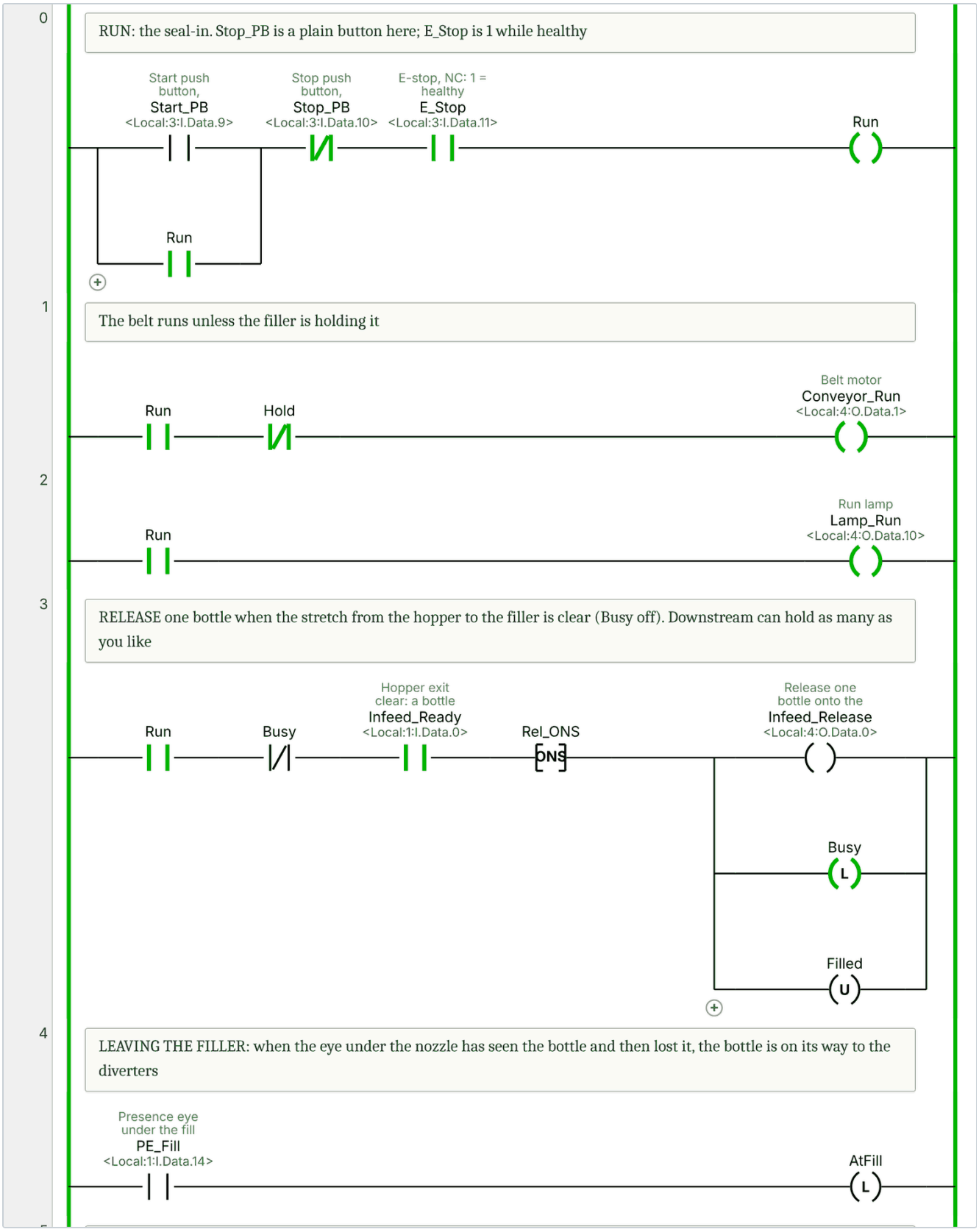


Figure 152. Level 5 online, the top of the routine.

> **TRY IT**
>
> Give a bin of your own a grading rule: drop a seventh pusher and bin at the end of the line for *yellow* bottles, set the hopper's colour mix to *random*, and extend the program — a colour bank for yellow, a bit 7 in the class word, five rungs for the new diverter. Then the result table below has a row that is yours.

Predict it: the answers

1. The rung is false at `XIO(Stop_PB)`: `Stop_PB` is on, so `Run` is **off** (an OTE writes 0 when its rung is false).
2. Every condition on the way is met, so `Conveyor_Run` is **on**.
3. Every condition on the way is met, so `Class.0` is **on**.
4. The rung is false at `XIO(Is_High)`: `Is_High` is on, so `Class.1` is **off** (an OTE writes 0 when its rung is false).
5. The rung is false at `XIO(Is_Broken)`: `Is_Broken` is on, so `Hold` is **off** (an OTE writes 0 when its rung is false).
6. Every condition on the way is met, so `Fill_Valve` is **on**.
7. The rung is false at the GEQ: `Fill_Level` is below **97**, so the OTL does nothing: `Filled` keeps whatever it had.
8. Every condition on the way is met, so `Div1` is **on**.
9. The rung is false at `XIC(Fire_2)`: `Fire_2` is off, so `Div2` is **off** (an OTE writes 0 when its rung is false).
10. Every condition on the way is met, so `Div3` is **on**.
11. The rung is false at `XIC(PE_Div4)`: `PE_Div4` is off, so `Div4` is **off** (an OTE writes 0 when its rung is false).
12. Every condition on the way is met, so `Div5` is **on**.
13. The rung is false at `XIC(Fire_6)`: `Fire_6` is off, so `Div6` is **off** (an OTE writes 0 when its rung is false).

38 Level 6 — Pack it

> **IN THE APP**
>
> File → New Project… → A graded level → **6**. The level opens with guided mode on this chapter: the tags to make, every rung below under its fold, a Verify check, and five results it ticks while you run — a case stops on the mark, a case is packed, the level completes, every bottle right, under par.

Sixteen bottles. The broken ones go into the scrap bin as before; every other bottle is filled and then **packed**: the belt ends in the case packer of step 12 (chapter 29), and a case lane on a VFD runs beside it. Four bottles to a case, and when the hopper is empty whatever is left in the infeed goes out as a part case. Par 140 s.

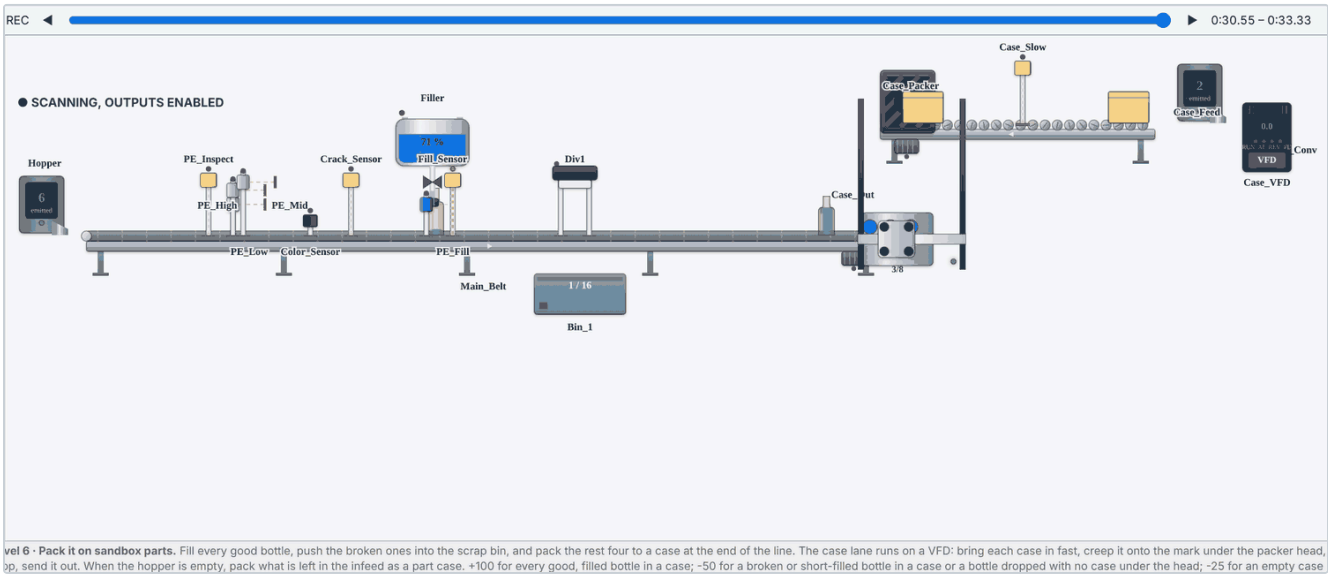


Figure 153. Level 6: the hopper, the gantry, the filler, one scrap pusher and bin, the packer at the end of the belt, its case lane on the far side with the slow-down eye and the case feeder, the VFD, the console.

How the level is scored

What happens	Points
A good, filled bottle packed into a case	+100
A broken bottle in the scrap bin	+100
A broken or short-filled bottle packed into a case	−50
A whole bottle in the scrap bin	−50
Bottles dropped by the packer with no case under the head (each)	−50
An empty case shipped at Case_Out	−25

What happens	Points
Complete under par (140 s)	+200

The level is complete when the hopper is empty, no bottle is left on the belt or in the packer's infeed, and every packed case has reached Case_Out. Three stars: no mistakes and under par.

How to pass it, step by step

1. **Start from Level 5.** Run circuit, release one bottle at a time to the filler, latch the gantry into a class word, fill to 97 %, and load the class into zone 1's queue as the bottle leaves the filler. Copy those 23 rungs; they are unchanged (the colour and size bits are not needed here, but they do no harm).
2. **One diverter only.** Diverter 1 takes broken bottles into the scrap bin. A whole bottle is not pushed: it runs on to the end of the belt, where the packer's infeed takes it. Nothing else is needed for the infeed — it fills itself.
3. **Run the case lane.** `Case_Run` while no empty case is under the head: `[XIO(Case_In_Place),XIC(Case_Full)]`. Let it run while the line runs *or* while a full group waits, so the last cases still come in after you stop.
4. **Two speeds and a latch.** 60 Hz normally; 15 Hz from the slow-down eye until an empty case is on the mark, latched, and also while a packed case leaves. Without the creep the case overshoots on the drive's decel ramp and leaves empty: −25 every time.
5. **Scale to the analog output.** `SCP(Case_Hz,0,60,0,10000,Case_Speed_Ref)`. Writing 60 or 15 straight to Case_Speed_Ref gives the drive 0.06 V or 0.015 V — it barely moves.
6. **Drop only on a stopped, empty case.** Group ready, case in place, case not full, `XIO(Case_VFD_Running)`, packer ready, one-shot. Dropping while the case is still ramping down misses it: −50 per bottle.
7. **Finish the batch.** Count the releases with a CTU preset 16; when it is done and the last bottle has left the filler, wait 15 s for it to reach the packer, then drop whatever is in the infeed as a part case. Without this the last two or three bottles sit in the infeed and the level never completes.

Tags this lesson needs

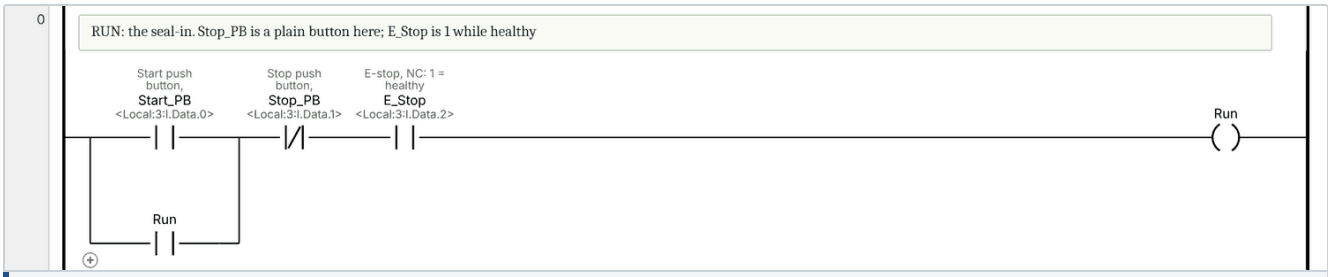
The level's alias tags are already made for you (chapter 32's table). Create these plain tags before you type the rungs; Verify reports any you miss.

Tag	Type	How to make it	Description
`Run`	BOOL	Edit Tags: name, type	Seal-in run bit
`Busy`	BOOL	Edit Tags: name, type	A bottle is between the hopper and the end of its cycle (Levels 1-3) / between the hopper and the filler (Levels 4-5)
`Rel_ONS`	BOOL	Edit Tags: name, type	One-shot storage for the release

Tag	Type	How to make it	Description
`Hold`	BOOL	Edit Tags: name, type	The filler holds the belt
`Filled`	BOOL	Edit Tags: name, type	This bottle has its charge
`AtFill`	BOOL	Edit Tags: name, type	The eye under the nozzle has seen this bottle
`Leave_ONS`	BOOL	Edit Tags: name, type	One-shot storage: the bottle left the filler
`Is_Low`	BOOL	Edit Tags: name, type	Latched: the low height beam was broken
`Is_Mid`	BOOL	Edit Tags: name, type	Latched: the middle beam was broken
`Is_High`	BOOL	Edit Tags: name, type	Latched: the high beam was broken
`Is_Broken`	BOOL	Edit Tags: name, type	Latched: the crack eye saw a broken bottle
`Is_R`	BOOL	Edit Tags: name, type	Latched: the colour sensor saw red
`Is_B`	BOOL	Edit Tags: name, type	Latched: the colour sensor saw blue
`Is_G`	BOOL	Edit Tags: name, type	Latched: the colour sensor saw green
`Class`	DINT	Edit Tags: name, type, Style Binary	The bottle in the gantry: bit 0 S, 1 M, 2 L, 3 broken, 4 red, 5 blue, 6 green
`Cur`	DINT	Edit Tags: name, type, Style Binary	The class unloaded from a zone queue at a diverter
`Q1`	DINT[8]	Edit Tags: name, type, Dim 8	Zone 1 queue: classes of the bottles between the filler and diverter 1
`Q1_Ctl`	CONTROL	Edit Tags: name, type	Queue control
`D1_ONS`	BOOL	Edit Tags: name, type	One-shot storage for a diverter
`D1_ONS2`	BOOL	Edit Tags: name, type	One-shot storage for a diverter
`D1_ONS3`	BOOL	Edit Tags: name, type	One-shot storage for a diverter
`Fire_1`	BOOL	Edit Tags: name, type	Armed: this diverter will push the bottle at its eye
`Case_Creep`	BOOL	Edit Tags: name, type	Latched from the slow-down eye until an empty case stands under the head
`Case_Hz`	REAL	Edit Tags: name, type	Case lane speed wanted, Hz
`Case_Hz_Actual`	REAL	Edit Tags: name, type	Case lane drive output frequency read back, Hz
`Drop_ONS`	BOOL	Edit Tags: name, type	
`Case_ONS`	BOOL	Edit Tags: name, type	
`Released`	COUNTER	Edit Tags: name, type	Bottles released from the hopper
`Tail_T`	TIMER	Edit Tags: name, type	After the last release: time for the last bottle to reach the packer

Run, release, leaving the filler, gantry, class word and fill

RUNG 0




```
[XIC(Start_PB),XIC(Run)]XIO(Stop_PB)XIC(E_Stop)OTE(Run);
```

PREDICT IT · 1 `Start_PB` is on, `Stop_PB` is on and `E_Stop` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When either (`Start_PB` (Start push button, momentary) is on) *or* (`Run` (Seal-in run bit) is on), `Stop_PB` (Stop push button, momentary) is off and `E_Stop` (E-stop, NC: 1 = healthy) is on, the rung turns `Run` on (and off again whenever the rung is false).

Left to right:

1. A branch of 2 parallel paths: power gets through if any one of them is true.
   - **Path 1**
     1. `XIC(Start_PB)` *Examine If Closed* asks: true when `Start_PB` (Start push button, momentary) is on.
   - **Path 2**
     1. `XIC(Run)` *Examine If Closed* asks: true when `Run` (Seal-in run bit) is on.
2. `XIO(Stop_PB)` *Examine If Open* asks: true when `Stop_PB` (Stop push button, momentary) is off.
3. `XIC(E_Stop)` *Examine If Closed* asks: true when `E_Stop` (E-stop, NC: 1 = healthy) is on.
4. `OTE(Run)` *Output Energize* does: turns `Run` on (and off again whenever the rung is false). Every scan it writes the rung state: true on, false off.

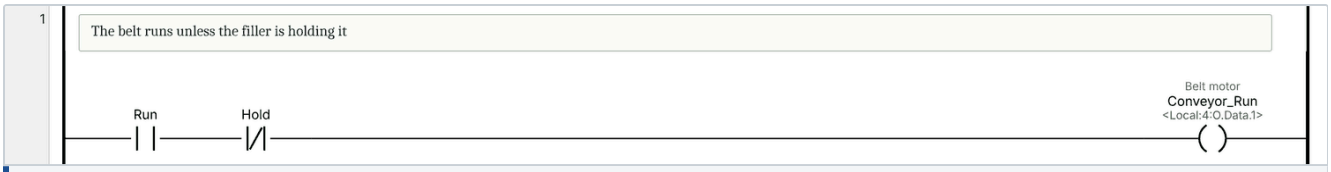
Why it is built this way:

Seal-in. The start condition turns `Run` on, and `Run` 's own contact in the parallel path keeps the rung true after the start goes away. Anything in series outside the branch (a stop, a fault) breaks it, and then it stays off until the start is made again.

> **Think of it like this:** a doorbell button and a hand. The start button is the doorbell, pressed for a moment. The seal-in contact is a hand that grabs the circuit as soon as the output comes on and keeps holding after you let go. The stop button, in series, is the only thing that can knock that hand away.

> **Watch for:** The stop has to be in series outside the branch. Put it inside the start path only and the seal-in path bypasses it: the stop no longer stops.

RUNG 1




```
XIC(Run)XIO(Hold)OTE(Conveyor_Run);
```

PREDICT IT · 2 `Run` is on and `Hold` is off. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Run` (Seal-in run bit) is on and `Hold` (The filler holds the belt) is off, the rung turns `Conveyor_Run` (Belt motor) on (and off again whenever the rung is false).

Left to right:

1. `XIC(Run)` *Examine If Closed* asks: true when `Run` (Seal-in run bit) is on.
2. `XIO(Hold)` *Examine If Open* asks: true when `Hold` (The filler holds the belt) is off.
3. `OTE(Conveyor_Run)` *Output Energize* does: turns `Conveyor_Run` (Belt motor) on (and off again whenever the rung is false).

> **Think of it like this:** a rung is a pipe with valves in it, read left to right. Every contact is a valve: XIC is open while its bit is 1, XIO while its bit is 0. Water reaches the output at the right only if every valve in a row is open. The output does not remember anything: on every scan it is simply wet or dry.

RUNG 2

```
XIC(Run)OTE(Lamp_Run);
```

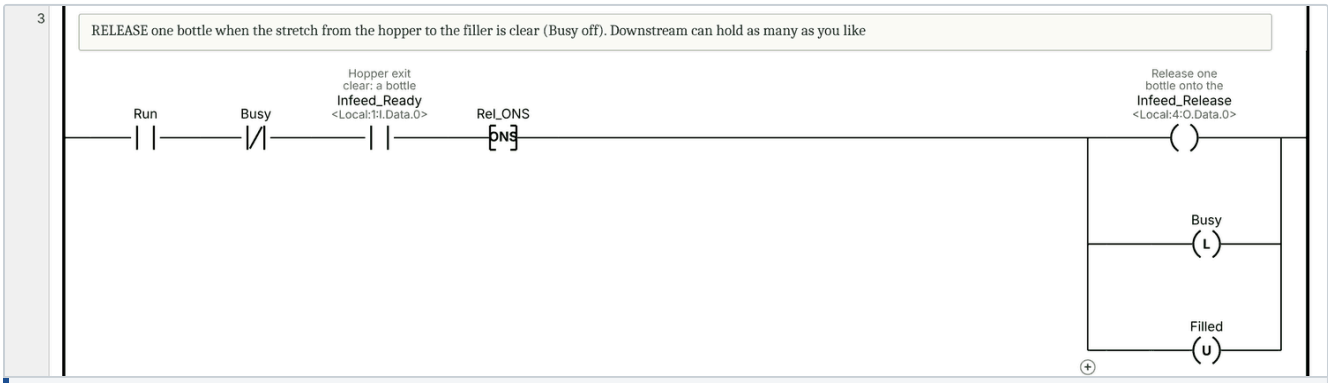
HOW THIS RUNG WORKS

In words. When `Run` (Seal-in run bit) is on, the rung turns `Lamp_Run` (Run lamp) on (and off again whenever the rung is false).

Left to right:

1. `XIC(Run)` *Examine If Closed* asks: true when `Run` (Seal-in run bit) is on.
2. `OTE(Lamp_Run)` *Output Energize* does: turns `Lamp_Run` (Run lamp) on (and off again whenever the rung is false).

RUNG 3




```
XIC(Run)XIO(Busy)XIC(Infeed_Ready)ONS(Rel_ONS)
[OTE(Infeed_Release),OTL(Busy),OTU(Filled)];
```

HOW THIS RUNG WORKS

In words. When `Run` (Seal-in run bit) is on, `Busy` (A bottle is between the hopper and the end of its cycle (Levels 1-3) / between the hopper and the filler (Levels 4-5)) is off, `Infeed_Ready` (Hopper exit clear: a bottle can be released) is on and it is the first scan they are true (the one-shot, remembered in `Rel_ONS` (One-shot storage for the release)), the rung turns `Infeed_Release` (Release one bottle onto the belt (rising edge)) on (and off again whenever the rung is false), latches `Busy` on and unlatches `Filled` (This bottle has its charge) (turns it off).

Left to right:

1. `XIC(Run)` *Examine If Closed* asks: true when `Run` (Seal-in run bit) is on.
2. `XIO(Busy)` *Examine If Open* asks: true when `Busy` (A bottle is between the hopper and the end of its cycle (Levels 1-3) / between the hopper and the filler (Levels 4-5)) is off.
3. `XIC(Infeed_Ready)` *Examine If Closed* asks: true when `Infeed_Ready` (Hopper exit clear: a bottle can be released) is on.
4. `ONS(Rel_ONS)` *One Shot* asks: true when it is the first scan they are true (the one-shot, remembered in `Rel_ONS` (One-shot storage for the release)). It passes power for one scan only, the scan its input goes from false to true; its storage bit remembers last scan.
5. A branch of 3 parallel paths: each path is tested on its own and does its own work.

   #### Path 1

   1. `OTE(Infeed_Release)` *Output Energize* does: turns `Infeed_Release` (Release one bottle onto the belt (rising edge)) on (and off again whenever the rung is false).

   #### Path 2

   1. `OTL(Busy)` *Output Latch* does: latches `Busy` on. It only ever sets the bit; the bit stays on after the rung goes false.

   #### Path 3

   1. `OTU(Filled)` *Output Unlatch* does: unlatches `Filled` (This bottle has its charge) (turns it off). It only ever clears the bit.

Why it is built this way:

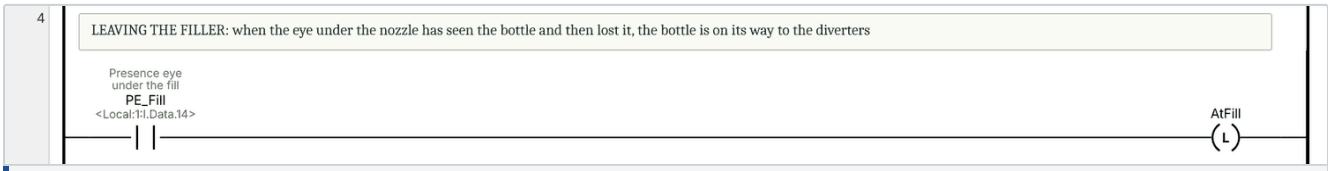
One-shot. Everything after the ONS happens once per event, not on every scan the condition holds; `Rel_ONS` is only its memory of the last scan.

A latch. `Busy` stays on after this rung goes false; an `OTU` elsewhere in the program clears it.

> **Think of it like this:** a doorbell that rings once however long you lean on the button. The ONS looks at what the button was doing last scan: only the change from up to down rings it.

> **Watch for:** Each ONS needs its own storage bit. Two ONS sharing one bit make one of them miss its edges.

RUNG 4




```
XIC(PE_Fill)OTL(AtFill);
```

HOW THIS RUNG WORKS

In words. When `PE_Fill` (Presence eye under the fill nozzle) is on, the rung latches `AtFill` (The eye under the nozzle has seen this bottle) on.

Left to right:

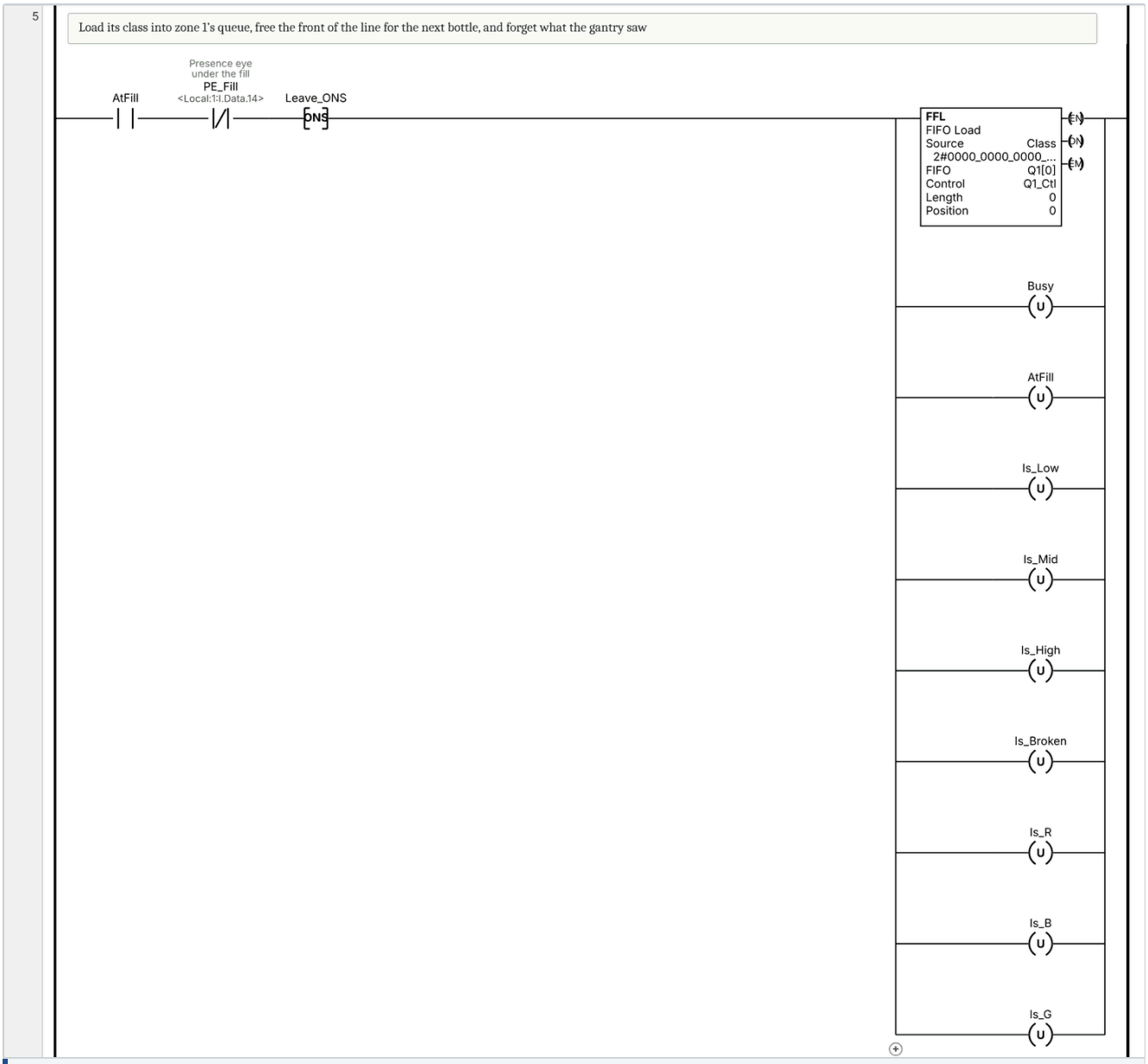
1. `XIC(PE_Fill)` *Examine If Closed* asks: true when `PE_Fill` (Presence eye under the fill nozzle) is on.
2. `OTL(AtFill)` *Output Latch* does: latches `AtFill` (The eye under the nozzle has seen this bottle) on.

Why it is built this way:

A latch: it stays on until an OTU clears it.

> **Watch for:** A latched bit survives the condition that set it: every way in needs a way out, or it stays on for good.

RUNG 5




```
XIC(AtFill)XIO(PE_Fill)ONS(Leave_ONS)
[FFL(Class,Q1[0],Q1_Ctl,8),OTU(Busy),OTU(AtFill),OTU(Is_Low),OTU(Is_Mid),OTU(Is
_High),OTU(Is_Broken),OTU(Is_R),OTU(Is_B),OTU(Is_G)];
```

HOW THIS RUNG WORKS

In words. When `AtFill` (The eye under the nozzle has seen this bottle) is on, `PE_Fill` (Presence eye under the fill nozzle) is off and it is the first scan they are true (the one-shot, remembered in `Leave_ONS` (One-shot storage: the bottle left the filler)), the rung loads `Class` (The bottle in the gantry: bit 0 S, 1 M, 2 L, 3 broken, 4 red, 5 blue, 6 green) into the queue `Q1[0]`, unlatches `Busy` (A bottle is between the hopper and the end of its cycle (Levels 1-3) / between the hopper and the filler (Levels 4-5)) (turns it off), unlatches `AtFill` (turns it off), unlatches `Is_Low` (Latched: the low height beam was broken) (turns it off), unlatches `Is_Mid` (Latched: the middle beam was broken) (turns it off), unlatches

`Is_High` (Latched: the high beam was broken) (turns it off), unlatches `Is_Broken` (Latched: the crack eye saw a broken bottle) (turns it off), unlatches `Is_R` (Latched: the colour sensor saw red) (turns it off), unlatches `Is_B` (Latched: the colour sensor saw blue) (turns it off) and unlatches `Is_G` (Latched: the colour sensor saw green) (turns it off).

Left to right:

1. `XIC(AtFill)` *Examine If Closed* asks: true when `AtFill` (The eye under the nozzle has seen this bottle) is on.
2. `XIO(PE_Fill)` *Examine If Open* asks: true when `PE_Fill` (Presence eye under the fill nozzle) is off.
3. `ONS(Leave_ONS)` *One Shot* asks: true when it is the first scan they are true (the one-shot, remembered in `Leave_ONS` (One-shot storage: the bottle left the filler)).
4. A branch of 10 parallel paths: each path is tested on its own and does its own work.

   Path 1

   1. `FFL(Class,Q1[0],Q1_Ctl,8)` *FIFO Load* does: loads `Class` (The bottle in the gantry: bit 0 S, 1 M, 2 L, 3 broken, 4 red, 5 blue, 6 green) into the queue `Q1[0]` . One entry per false-to-true transition.

   Path 2

   1. `OTU(Busy)` *Output Unlatch* does: unlatches `Busy` (A bottle is between the hopper and the end of its cycle (Levels 1-3) / between the hopper and the filler (Levels 4-5)) (turns it off).

   Path 3

   1. `OTU(AtFill)` *Output Unlatch* does: unlatches `AtFill` (turns it off).

   Path 4

   1. `OTU(Is_Low)` *Output Unlatch* does: unlatches `Is_Low` (Latched: the low height beam was broken) (turns it off).

   Path 5

   1. `OTU(Is_Mid)` *Output Unlatch* does: unlatches `Is_Mid` (Latched: the middle beam was broken) (turns it off).

   Path 6

   1. `OTU(Is_High)` *Output Unlatch* does: unlatches `Is_High` (Latched: the high beam was broken) (turns it off).

   Path 7

   1. `OTU(Is_Broken)` *Output Unlatch* does: unlatches `Is_Broken` (Latched: the crack eye saw a broken bottle) (turns it off).

   Path 8

   1. `OTU(Is_R)` *Output Unlatch* does: unlatches `Is_R` (Latched: the colour sensor saw red) (turns it off).

   Path 9

   1. `OTU(Is_B)` *Output Unlatch* does: unlatches `Is_B` (Latched: the colour sensor saw blue) (turns it off).

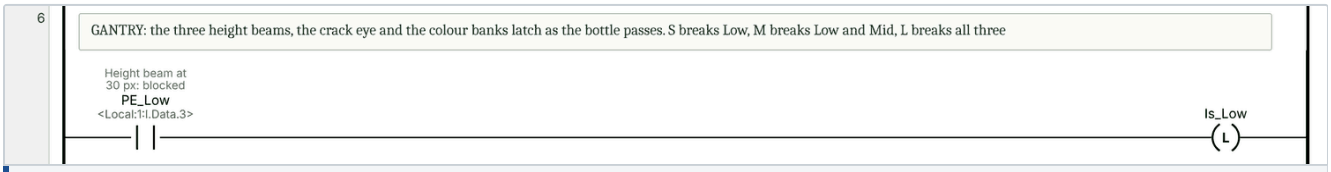
   Path 10

   1. `OTU(Is_G)` *Output Unlatch* does: unlatches `Is_G` (Latched: the colour sensor saw green) (turns it off).

Why it is built this way:

One-shot: once per event, not once per scan.

RUNG 6




```
XIC(PE_Low)OTL(Is_Low);
```

HOW THIS RUNG WORKS

In words. When `PE_Low` (Height beam at 30 px: blocked by every size) is on, the rung latches `Is_Low` (Latched: the low height beam was broken) on.

Left to right:

1. `XIC(PE_Low)` *Examine If Closed* asks: true when `PE_Low` (Height beam at 30 px: blocked by every size) is on.
2. `OTL(Is_Low)` *Output Latch* does: latches `Is_Low` (Latched: the low height beam was broken) on.

Why it is built this way:

A latch: it stays on until an OTU clears it.

RUNG 7

```
XIC(PE_Mid)OTL(Is_Mid);
```

HOW THIS RUNG WORKS

In words. When `PE_Mid` (Height beam at 44 px: blocked by M and L) is on, the rung latches `Is_Mid` (Latched: the middle beam was broken) on.

Left to right:

1. `XIC(PE_Mid)` *Examine If Closed* asks: true when `PE_Mid` (Height beam at 44 px: blocked by M and L) is on.
2. `OTL(Is_Mid)` *Output Latch* does: latches `Is_Mid` (Latched: the middle beam was broken) on.

Why it is built this way:

A latch: it stays on until an OTU clears it.

RUNG 8

```
XIC(PE_High)OTL(Is_High);
```

HOW THIS RUNG WORKS

In words. When `PE_High` (Height beam at 52 px: blocked by L only) is on, the rung latches `Is_High` (Latched: the high beam was broken) on.

Left to right:

1. `XIC(PE_High)` *Examine If Closed* asks: true when `PE_High` (Height beam at 52 px: blocked by L only) is on.
2. `OTL(Is_High)` *Output Latch* does: latches `Is_High` (Latched: the high beam was broken) on.

Why it is built this way:

A latch: it stays on until an OTU clears it.

RUNG 9

```
XIC(Crack_Sensor)OTL(Is_Broken);
```

HOW THIS RUNG WORKS

In words. When `Crack_Sensor` (Crack eye: the bottle in front of it is broken) is on, the rung latches `Is_Broken` (Latched: the crack eye saw a broken bottle) on.

Left to right:

1. `XIC(Crack_Sensor)` *Examine If Closed* asks: true when `Crack_Sensor` (Crack eye: the bottle in front of it is broken) is on.
2. `OTL(Is_Broken)` *Output Latch* does: latches `Is_Broken` (Latched: the crack eye saw a broken bottle) on.

Why it is built this way:

A latch: it stays on until an OTU clears it.

RUNG 10

```
XIC(Color_Red)OTL(Is_R);
```

HOW THIS RUNG WORKS

In words. When `Color_Red` (Colour sensor bank 1: red) is on, the rung latches `Is_R` (Latched: the colour sensor saw red) on.

Left to right:

1. `XIC(Color_Red)` *Examine If Closed* asks: true when `Color_Red` (Colour sensor bank 1: red) is on.
2. `OTL(Is_R)` *Output Latch* does: latches `Is_R` (Latched: the colour sensor saw red) on.

Why it is built this way:

A latch: it stays on until an OTU clears it.

RUNG 11

```
XIC(Color_Blue)OTL(Is_B);
```

HOW THIS RUNG WORKS

In words. When `Color_Blue` (Colour sensor bank 2: blue) is on, the rung latches `Is_B` (Latched: the colour sensor saw blue) on.

Left to right:

1. `XIC(Color_Blue)` *Examine If Closed* asks: true when `Color_Blue` (Colour sensor bank 2: blue) is on.
2. `OTL(Is_B)` *Output Latch* does: latches `Is_B` (Latched: the colour sensor saw blue) on.

Why it is built this way:

A latch: it stays on until an OTU clears it.

RUNG 12

```
XIC(Color_Green)OTL(Is_G);
```

HOW THIS RUNG WORKS

In words. When `Color_Green` (Colour sensor bank 3: green) is on, the rung latches `Is_G` (Latched: the colour sensor saw green) on.

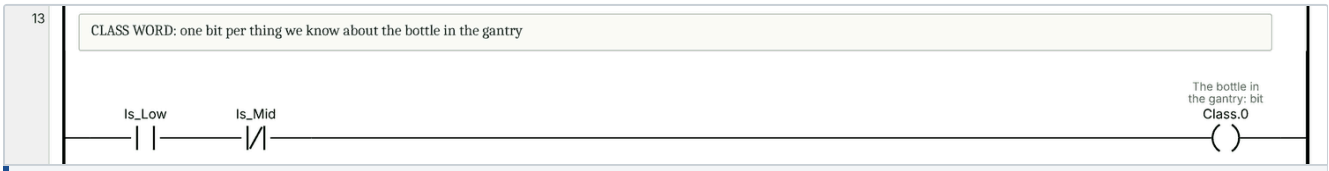
Left to right:

1. `XIC(Color_Green)` *Examine If Closed* asks: true when `Color_Green` (Colour sensor bank 3: green) is on.
2. `OTL(Is_G)` *Output Latch* does: latches `Is_G` (Latched: the colour sensor saw green) on.

Why it is built this way:

A latch: it stays on until an OTU clears it.

RUNG 13




```
XIC(Is_Low)XIO(Is_Mid)OTE(Class.0);
```

PREDICT IT · 3 `Is_Low` is on and `Is_Mid` is off. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Is_Low` (Latched: the low height beam was broken) is on and `Is_Mid` (Latched: the middle beam was broken) is off, the rung turns `Class.0` on (and off again whenever the rung is false).

Left to right:

1. `XIC(Is_Low)` *Examine If Closed* asks: true when `Is_Low` (Latched: the low height beam was broken) is on.
2. `XIO(Is_Mid)` *Examine If Open* asks: true when `Is_Mid` (Latched: the middle beam was broken) is off.
3. `OTE(Class.0)` *Output Energize* does: turns `Class.0` on (and off again whenever the rung is false).

RUNG 14

```
XIC(Is_Mid)XIO(Is_High)OTE(Class.1);
```

PREDICT IT · 4 `Is_Mid` is on and `Is_High` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Is_Mid` (Latched: the middle beam was broken) is on and `Is_High` (Latched: the high beam was broken) is off, the rung turns `Class.1` on (and off again whenever the rung is false).

Left to right:

1. `XIC(Is_Mid)` *Examine If Closed* asks: true when `Is_Mid` (Latched: the middle beam was broken) is on.
2. `XIO(Is_High)` *Examine If Open* asks: true when `Is_High` (Latched: the high beam was broken) is off.
3. `OTE(Class.1)` *Output Energize* does: turns `Class.1` on (and off again whenever the rung is false).

RUNG 15

```
XIC(Is_High)OTE(Class.2);
```

HOW THIS RUNG WORKS

In words. When `Is_High` (Latched: the high beam was broken) is on, the rung turns `Class.2` on (and off again whenever the rung is false).

Left to right:

1. `XIC(Is_High)` *Examine If Closed* asks: true when `Is_High` (Latched: the high beam was broken) is on.
2. `OTE(Class.2)` *Output Energize* does: turns `Class.2` on (and off again whenever the rung is false).

RUNG 16

```
XIC(Is_Broken)OTE(Class.3);
```

HOW THIS RUNG WORKS

In words. When `Is_Broken` (Latched: the crack eye saw a broken bottle) is on, the rung turns `Class.3` on (and off again whenever the rung is false).

Left to right:

1. `XIC(Is_Broken)` *Examine If Closed* asks: true when `Is_Broken` (Latched: the crack eye saw a broken bottle) is on.
2. `OTE(Class.3)` *Output Energize* does: turns `Class.3` on (and off again whenever the rung is false).

RUNG 17

```
XIC(Is_R)OTE(Class.4);
```

HOW THIS RUNG WORKS

In words. When `Is_R` (Latched: the colour sensor saw red) is on, the rung turns `Class.4` on (and off again whenever the rung is false).

Left to right:

1. `XIC(Is_R)` *Examine If Closed* asks: true when `Is_R` (Latched: the colour sensor saw red) is on.
2. `OTE(Class.4)` *Output Energize* does: turns `Class.4` on (and off again whenever the rung is false).

RUNG 18

```
XIC(Is_B)OTE(Class.5);
```

HOW THIS RUNG WORKS

In words. When `Is_B` (Latched: the colour sensor saw blue) is on, the rung turns `Class.5` on (and off again whenever the rung is false).

Left to right:

1. `XIC(Is_B)` *Examine If Closed* asks: true when `Is_B` (Latched: the colour sensor saw blue) is on.
2. `OTE(Class.5)` *Output Energize* does: turns `Class.5` on (and off again whenever the rung is false).

RUNG 19

```
XIC(Is_G)OTE(Class.6);
```

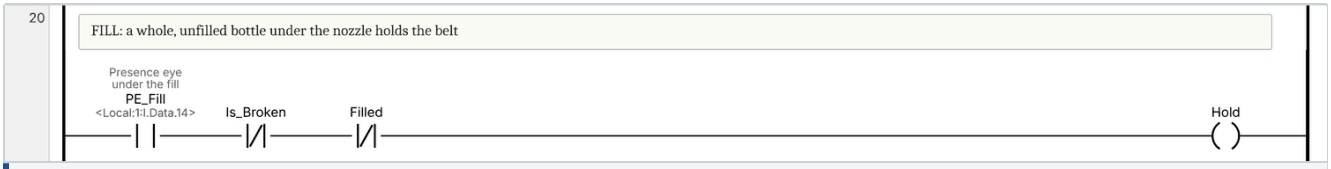
HOW THIS RUNG WORKS

In words. When `Is_G` (Latched: the colour sensor saw green) is on, the rung turns `Class.6` on (and off again whenever the rung is false).

Left to right:

1. `XIC(Is_G)` *Examine If Closed* asks: true when `Is_G` (Latched: the colour sensor saw green) is on.
2. `OTE(Class.6)` *Output Energize* does: turns `Class.6` on (and off again whenever the rung is false).

RUNG 20




```
XIC(PE_Fill)XIO(Is_Broken)XIO(Filled)OTE(Hold);
```

PREDICT IT · 5 `PE_Fill` is on, `Is_Broken` is on and `Filled` is off. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

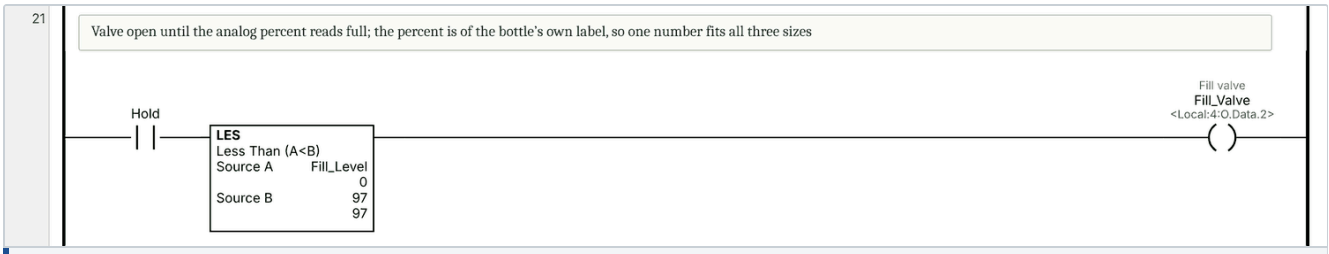
HOW THIS RUNG WORKS

In words. When `PE_Fill` (Presence eye under the fill nozzle) is on, `Is_Broken` (Latched: the crack eye saw a broken bottle) is off and `Filled` (This bottle has its charge) is off, the rung turns `Hold` (The filler holds the belt) on (and off again whenever the rung is false).

Left to right:

1. `XIC(PE_Fill)` *Examine If Closed* asks: true when `PE_Fill` (Presence eye under the fill nozzle) is on.
2. `XIO(Is_Broken)` *Examine If Open* asks: true when `Is_Broken` (Latched: the crack eye saw a broken bottle) is off.
3. `XIO(Filled)` *Examine If Open* asks: true when `Filled` (This bottle has its charge) is off.
4. `OTE(Hold)` *Output Energize* does: turns `Hold` (The filler holds the belt) on (and off again whenever the rung is false).

RUNG 21




```
XIC(Hold)LES(Fill_Level,97)OTE(Fill_Valve);
```

PREDICT IT · 6 `Hold` is on and `Fill_Level` is 96. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

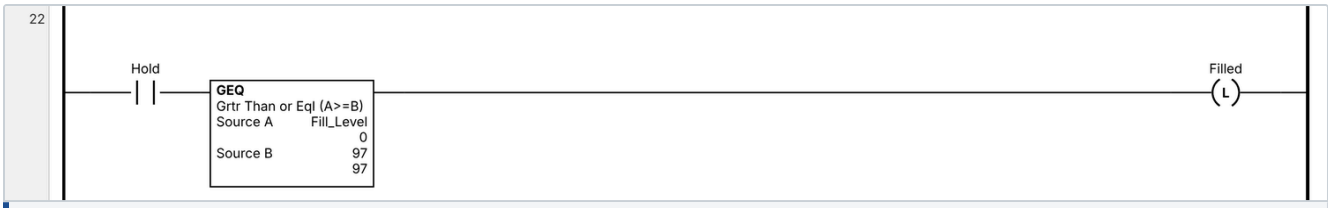
HOW THIS RUNG WORKS

In words. When `Hold` (The filler holds the belt) is on and `Fill_Level` (Fill of the bottle under the nozzle, 0-100 % of its label) is less than **97**, the rung turns `Fill_Valve` (Fill valve) on (and off again whenever the rung is false).

Left to right:

1. `XIC(Hold)` *Examine If Closed* asks: true when `Hold` (The filler holds the belt) is on.
2. `LES(Fill_Level,97)` *Less Than* asks: true when `Fill_Level` (Fill of the bottle under the nozzle, 0-100 % of its label) is less than **97**.
3. `OTE(Fill_Valve)` *Output Energize* does: turns `Fill_Valve` (Fill valve) on (and off again whenever the rung is false).

RUNG 22




```
XIC(Hold)GEQ(Fill_Level,97)OTL(Filled);
```

PREDICT IT · 7 `Hold` is on and `Fill_Level` is 96. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Hold` (The filler holds the belt) is on and `Fill_Level` (Fill of the bottle under the nozzle, 0-100 % of its label) is at least **97**, the rung latches `Filled` (This bottle has its charge) on.

Left to right:

1. `XIC(Hold)` *Examine If Closed* asks: true when `Hold` (The filler holds the belt) is on.
2. `GEQ(Fill_Level,97)` *Greater Than or Equal* asks: true when `Fill_Level` (Fill of the bottle under the nozzle, 0-100 % of its label) is at least **97**.
3. `OTL(Filled)` *Output Latch* does: latches `Filled` (This bottle has its charge) on.

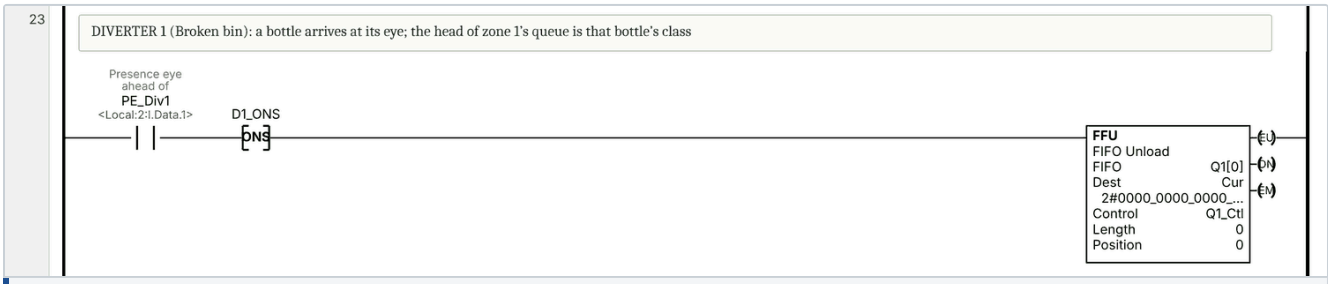
Why it is built this way:

A latch: it stays on until an OTU clears it.

Level 5's first 23 rungs, word for word.

Diverter 1: broken bottles to the scrap bin

RUNG 23




```
XIC(PE_Div1)ONS(D1_ONS)FFU(Q1[0],Cur,Q1_Ctl,8);
```

HOW THIS RUNG WORKS

In words. When `PE_Div1` (Presence eye ahead of diverter 1) is on and it is the first scan they are true (the one-shot, remembered in `D1_ONS`), the rung unloads the oldest entry of `Q1[0]` into `Cur` (The class unloaded from a zone queue at a diverter).

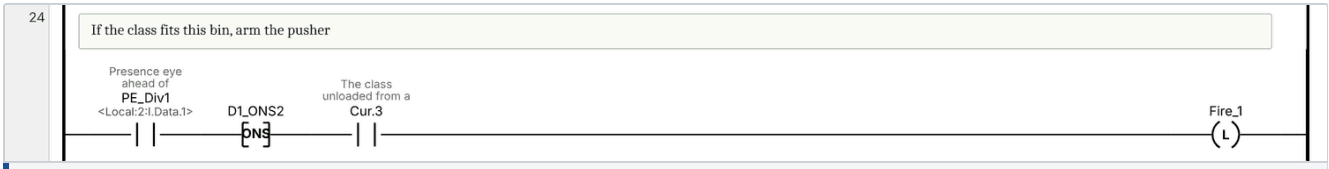
Left to right:

1. `XIC(PE_Div1)` *Examine If Closed* asks: true when `PE_Div1` (Presence eye ahead of diverter 1) is on.
2. `ONS(D1_ONS)` *One Shot* asks: true when it is the first scan they are true (the one-shot, remembered in `D1_ONS`).
3. `FFU(Q1[0],Cur,Q1_Ctl,8)` *FIFO Unload* does: unloads the oldest entry of `Q1[0]` into `Cur` (The class unloaded from a zone queue at a diverter). One entry per false-to-true transition.

Why it is built this way:

One-shot: once per event, not once per scan.

RUNG 24




```
XIC(PE_Div1)ONS(D1_ONS2)XIC(Cur.3)OTL(Fire_1);
```

HOW THIS RUNG WORKS

In words. When `PE_Div1` (Presence eye ahead of diverter 1) is on, it is the first scan they are true (the one-shot, remembered in `D1_ONS2`) and `Cur.3` is on, the rung latches `Fire_1` on.

Left to right:

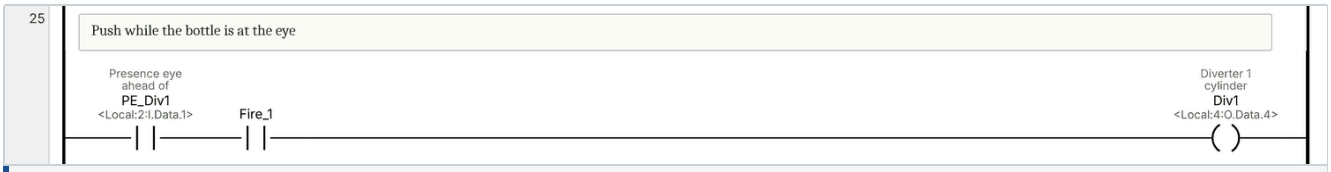
1. `XIC(PE_Div1)` *Examine If Closed* asks: true when `PE_Div1` (Presence eye ahead of diverter 1) is on.
2. `ONS(D1_ONS2)` *One Shot* asks: true when it is the first scan they are true (the one-shot, remembered in `D1_ONS2`).
3. `XIC(Cur.3)` *Examine If Closed* asks: true when `Cur.3` is on.
4. `OTL(Fire_1)` *Output Latch* does: latches `Fire_1` on.

Why it is built this way:

One-shot: once per event, not once per scan.

A latch: it stays on until an OTU clears it.

RUNG 25




```
XIC(PE_Div1)XIC(Fire_1)OTE(Div1);
```

PREDICT IT · 8 `PE_Div1` is on and `Fire_1` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

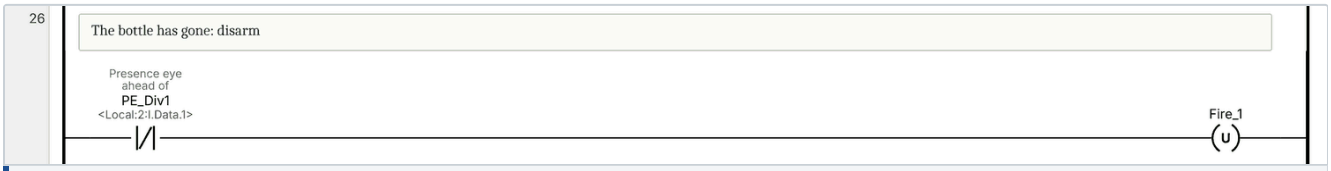
HOW THIS RUNG WORKS

In words. When `PE_Div1` (Presence eye ahead of diverter 1) is on and `Fire_1` is on, the rung turns `Div1` (Diverter 1 cylinder) on (and off again whenever the rung is false).

Left to right:

1. `XIC(PE_Div1)` *Examine If Closed* asks: true when `PE_Div1` (Presence eye ahead of diverter 1) is on.
2. `XIC(Fire_1)` *Examine If Closed* asks: true when `Fire_1` is on.
3. `OTE(Div1)` *Output Energize* does: turns `Div1` (Diverter 1 cylinder) on (and off again whenever the rung is false).

RUNG 26




```
XIO(PE_Div1)OTU(Fire_1);
```

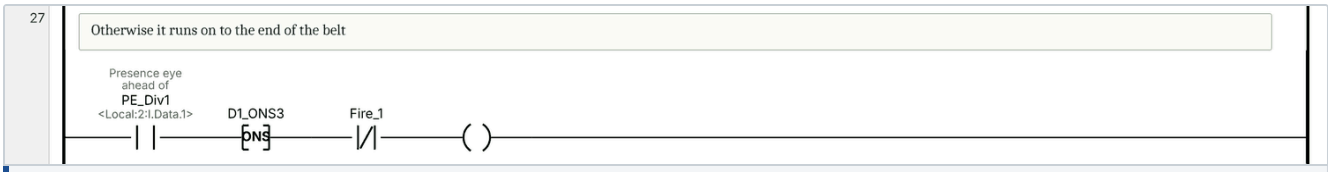
HOW THIS RUNG WORKS

In words. When `PE_Div1` (Presence eye ahead of diverter 1) is off, the rung unlatches `Fire_1` (turns it off).

Left to right:

1. `XIO(PE_Div1)` *Examine If Open* asks: true when `PE_Div1` (Presence eye ahead of diverter 1) is off.
2. `OTU(Fire_1)` *Output Unlatch* does: unlatches `Fire_1` (turns it off).

RUNG 27




```
XIC(PE_Div1)ONS(D1_ONS3)XIO(Fire_1)NOP();
```

HOW THIS RUNG WORKS

In words. When `PE_Div1` (Presence eye ahead of diverter 1) is on, it is the first scan they are true (the one-shot, remembered in `D1_ONS3`) and `Fire_1` is off, the rung does nothing (a placeholder).

Left to right:

1. `XIC(PE_Div1)` *Examine If Closed* asks: true when `PE_Div1` (Presence eye ahead of diverter 1) is on.
2. `ONS(D1_ONS3)` *One Shot* asks: true when it is the first scan they are true (the one-shot, remembered in `D1_ONS3`).
3. `XIO(Fire_1)` *Examine If Open* asks: true when `Fire_1` is off.
4. `NOP()` *No Operation* does: does nothing (a placeholder).

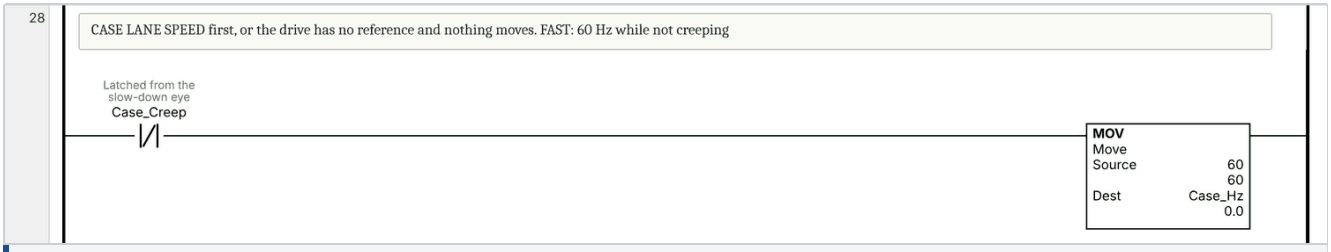
Why it is built this way:

One-shot: once per event, not once per scan.

The last diverter's "otherwise" rung passes nothing on: a whole bottle simply runs to the end of the belt.

The case lane: run, creep, speed and scaling

RUNG 28




```
XIO(Case_Creep)MOV(60,Case_Hz);
```

HOW THIS RUNG WORKS

In words. When `Case_Creep` (Latched from the slow-down eye until an empty case stands under the head) is off, the rung puts **60** into `Case_Hz` (Case lane speed wanted, Hz).

Left to right:

1. `XIO(Case_Creep)` *Examine If Open* asks: true when `Case_Creep` (Latched from the slow-down eye until an empty case stands under the head) is off.
2. `MOV(60,Case_Hz)` *Move* does: puts **60** into `Case_Hz` (Case lane speed wanted, Hz). It writes every scan the rung is true.

RUNG 29

```
SCP(Case_Hz,0,60,0,10000,Case_Speed_Ref);
```

HOW THIS RUNG WORKS

In words. Every scan, with nothing to test, the rung scales `Case_Hz` (Case lane speed wanted, Hz) from 0…60 to 0…10000 into `Case_Speed_Ref` (Case lane drive speed reference: 0-10000 = 0-10 V = 0-60 Hz).

Left to right:

1. `SCP(Case_Hz,0,60,0,10000,Case_Speed_Ref)` *Scale with Parameters* does: scales `Case_Hz` (Case lane speed wanted, Hz) from 0…60 to 0…10000 into `Case_Speed_Ref` (Case lane drive speed reference: 0-10000 = 0-10 V = 0-60 Hz).

Why it is built this way:

Scaling. SCP turns the raw number into engineering units with a straight line through the two points you give it.

> **Think of it like this:** reading a thermometer. Two marks you know (freezing and boiling), a straight ruler between them, and any reading on the glass turns into degrees.

RUNG 30

```
SCP(Case_Hz_Fbk,0,10000,0,60,Case_Hz_Actual);
```

HOW THIS RUNG WORKS

In words. Every scan, with nothing to test, the rung scales `Case_Hz_Fbk` (Case lane drive output frequency: 0-10000 = 0-60 Hz) from 0…10000 to 0…60 into `Case_Hz_Actual` (Case lane drive output frequency read back, Hz).

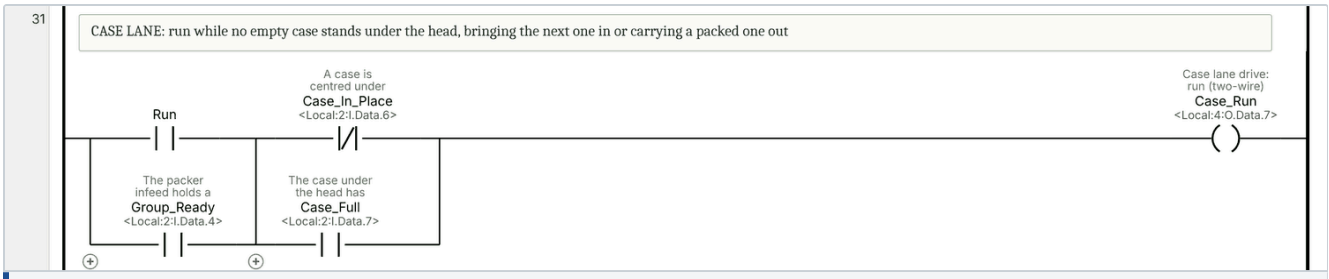
Left to right:

1. `SCP(Case_Hz_Fbk,0,10000,0,60,Case_Hz_Actual)` *Scale with Parameters* does: scales `Case_Hz_Fbk` (Case lane drive output frequency: 0-10000 = 0-60 Hz) from 0…10000 to 0…60 into `Case_Hz_Actual` (Case lane drive output frequency read back, Hz).

Why it is built this way:

Scaling raw counts to units.

RUNG 31




```
[XIC(Run),XIC(Group_Ready)][XIO(Case_In_Place),XIC(Case_Full)]OTE(Case_Run);
```

PREDICT IT · 9 `Run` is on and `Case_In_Place` is off. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

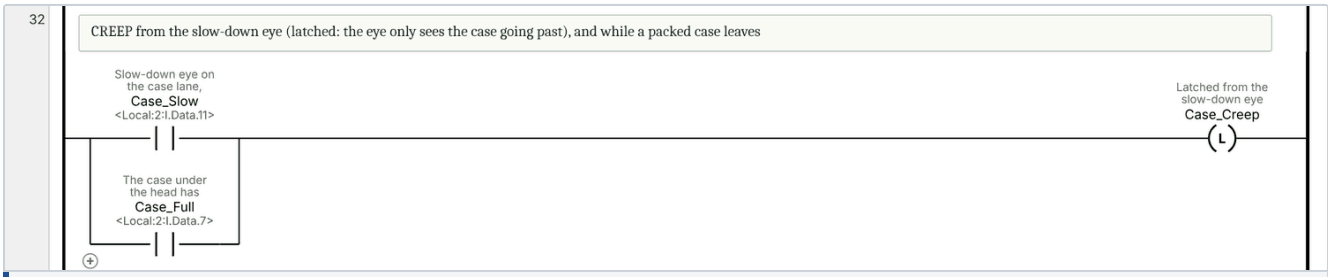
In words. When either (`Run` (Seal-in run bit) is on) *or* (`Group_Ready` (The packer infeed holds a case worth) is on) and either (`Case_In_Place` (A case is centred under the packer head) is off) *or* (`Case_Full` (The case under the head has been packed) is on), the rung turns `Case_Run` (Case lane drive: run (two-wire)) on (and off again whenever the rung is false).

Left to right:

1. A branch of 2 parallel paths: power gets through if any one of them is true.
   - **Path 1**
     1. `XIC(Run)` *Examine If Closed* asks: true when `Run` (Seal-in run bit) is on.
   - **Path 2**
     1. `XIC(Group_Ready)` *Examine If Closed* asks: true when `Group_Ready` (The packer infeed holds a case worth) is on.
2. A branch of 2 parallel paths: power gets through if any one of them is true.
   - **Path 1**
     1. `XIO(Case_In_Place)` *Examine If Open* asks: true when `Case_In_Place` (A case is centred under the packer head) is off.
   - **Path 2**
     1. `XIC(Case_Full)` *Examine If Closed* asks: true when `Case_Full` (The case under the head has been packed) is on.
3. `OTE(Case_Run)` *Output Energize* does: turns `Case_Run` (Case lane drive: run (two-wire)) on (and off again whenever the rung is false).

> **Think of it like this:** parallel paths are two roads to the same town. If either road is open you get there; the rung only cares that power reaches the far end, not which way it went.

RUNG 32




```
[XIC(Case_Slow),XIC(Case_Full)]OTL(Case_Creep);
```

PREDICT IT · 10 `Case_Slow` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When either (`Case_Slow` (Slow-down eye on the case lane, 100 px before the packer head) is on) *or* (`Case_Full` (The case under the head has been packed) is on), the rung latches `Case_Creep` (Latched from the slow-down eye until an empty case stands under the head) on.

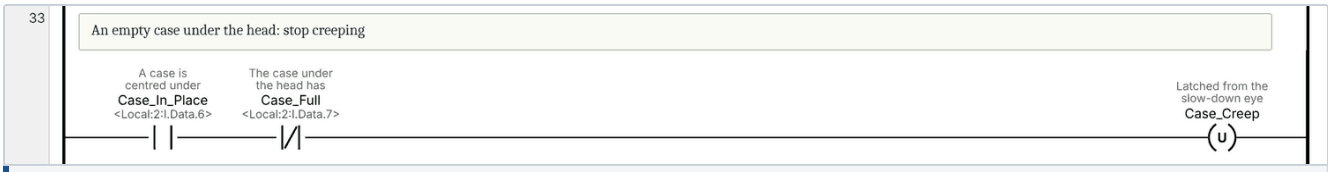
Left to right:

1. A branch of 2 parallel paths: power gets through if any one of them is true.
   - **Path 1**
     1. `XIC(Case_Slow)` *Examine If Closed* asks: true when `Case_Slow` (Slow-down eye on the case lane, 100 px before the packer head) is on.
   - **Path 2**
     1. `XIC(Case_Full)` *Examine If Closed* asks: true when `Case_Full` (The case under the head has been packed) is on.
2. `OTL(Case_Creep)` *Output Latch* does: latches `Case_Creep` (Latched from the slow-down eye until an empty case stands under the head) on.

Why it is built this way:

A latch: it stays on until an OTU clears it.

RUNG 33




```
XIC(Case_In_Place)XIO(Case_Full)OTU(Case_Creep);
```

PREDICT IT · 11 `Case_In_Place` is on and `Case_Full` is off. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

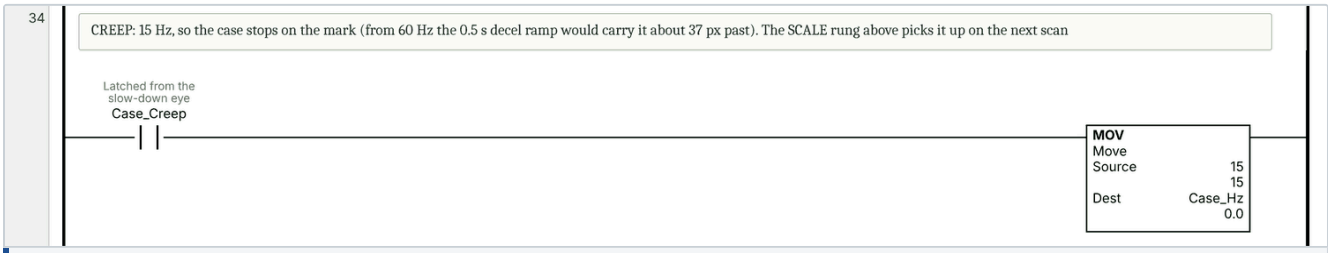
HOW THIS RUNG WORKS

In words. When `Case_In_Place` (A case is centred under the packer head) is on and `Case_Full` (The case under the head has been packed) is off, the rung unlatches `Case_Creep` (Latched from the slow-down eye until an empty case stands under the head) (turns it off).

Left to right:

1. `XIC(Case_In_Place)` *Examine If Closed* asks: true when `Case_In_Place` (A case is centred under the packer head) is on.
2. `XIO(Case_Full)` *Examine If Open* asks: true when `Case_Full` (The case under the head has been packed) is off.
3. `OTU(Case_Creep)` *Output Unlatch* does: unlatches `Case_Creep` (Latched from the slow-down eye until an empty case stands under the head) (turns it off).

RUNG 34




```
XIC(Case_Creep)MOV(15,Case_Hz);
```

HOW THIS RUNG WORKS

In words. When `Case_Creep` (Latched from the slow-down eye until an empty case stands under the head) is on, the rung puts **15** into `Case_Hz` (Case lane speed wanted, Hz).

Left to right:

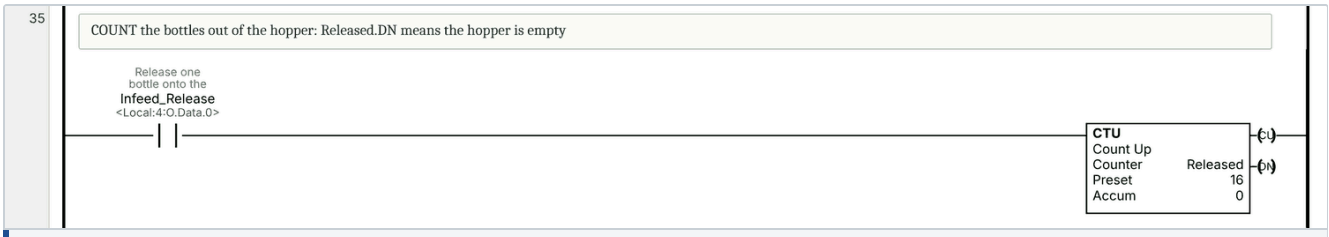
1. `XIC(Case_Creep)` *Examine If Closed* asks: true when `Case_Creep` (Latched from the slow-down eye until an empty case stands under the head) is on.
2. `MOV(15,Case_Hz)` *Move* does: puts **15** into `Case_Hz` (Case lane speed wanted, Hz).

The same seven rungs as the bottle line's step 12, with `Run` in place of `Line_Run`. The speed comes first, or the case cannot move: 60 Hz while not creeping, SCP out to the 0-10 V reference and back from the drive's output frequency. Then the lane runs while no empty case is on the mark, a latched creep from the

slow-down eye, and 15 Hz while creeping.

Counting the batch and packing

RUNG 35




```
XIC(Infeed_Release)CTU(Released,16,0);
```

HOW THIS RUNG WORKS

In words. When `Infeed_Release` (Release one bottle onto the belt (rising edge)) is on, the rung counts `Released` (Bottles released from the hopper) up by one (preset 16).

Left to right:

1. `XIC(Infeed_Release)` *Examine If Closed* asks: true when `Infeed_Release` (Release one bottle onto the belt (rising edge)) is on.
2. `CTU(Released,16,0)` *Count Up* does: counts `Released` (Bottles released from the hopper) up by one (preset 16). It counts false-to-true transitions of the rung, not scans; .DN is on once the count reaches the preset; it keeps counting past it.

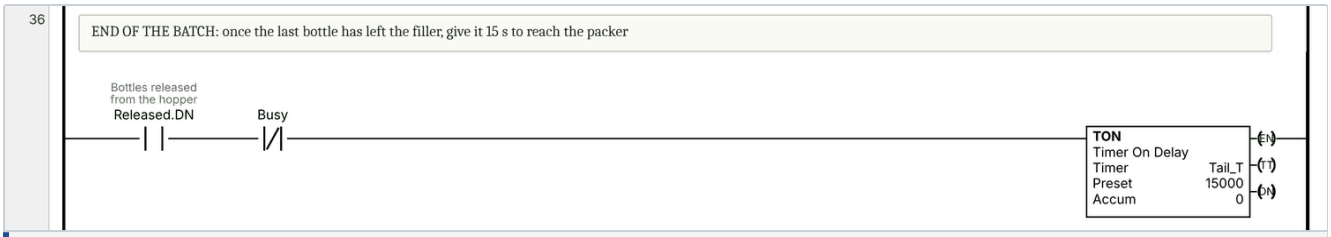
Why it is built this way:

A counter. It counts transitions, not time: each false-to-true of the rung adds one. A RES elsewhere clears it.

> **Think of it like this:** the clicker a doorman holds: one click for every person who walks through the door, however long they stand in it. The number stays on the clicker until somebody presses reset.

> **Watch for:** A counter keeps its count through a stop and a restart. Reset it on purpose where the count should start again.

RUNG 36




```
XIC(Released.DN)XIO(Busy)TON(Tail_T,15000,0);
```

PREDICT IT · 12 `Released.DN` (the done bit of `Released`: Bottles released from the hopper) is off and `Busy` is off. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Released.DN` (the done bit of `Released`: Bottles released from the hopper) is on and `Busy` (A bottle is between the hopper and the end of its cycle (Levels 1-3) / between the hopper and the filler (Levels 4-5)) is off, the rung runs the timer `Tail_T` (After the last release: time for the last bottle to reach the packer) toward 15000 ms (15 s).

Left to right:

1. `XIC(Released.DN)` *Examine If Closed* asks: true when `Released.DN` (the done bit of `Released`: Bottles released from the hopper) is on.
2. `XIO(Busy)` *Examine If Open* asks: true when `Busy` (A bottle is between the hopper and the end of its cycle (Levels 1-3) / between the hopper and the filler (Levels 4-5)) is off.
3. `TON(Tail_T,15000,0)` *Timer On Delay* does: runs the timer `Tail_T` (After the last release: time for the last bottle to reach the packer) toward 15000 ms (15 s). The accumulator counts while the rung is true; .DN comes on at the preset; the moment the rung goes false the timer resets.

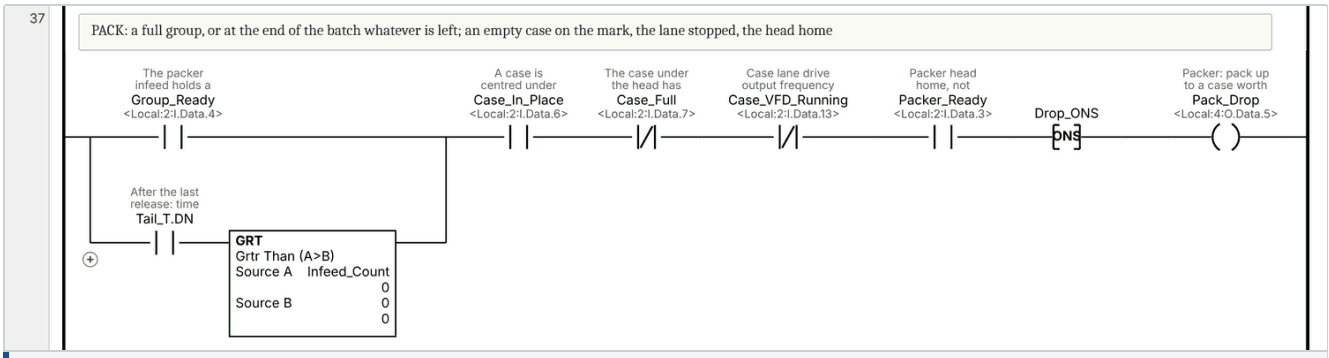
Why it is built this way:

A timer. `Tail_T` only times while this rung is true; other rungs use its `.DN` (done) or `.TT` (timing) bits. Being a TON, it starts again from zero every time the rung goes false.

> **Think of it like this:** a kitchen timer that only runs while you hold your finger on it. Take your finger off and it snaps back to zero. Hold it long enough and the done light comes on, and stays on as long as your finger is still there.

> **Watch for:** If anything in series flickers false for one scan, a TON starts over from zero. Use RTO when the time must survive an interruption.

RUNG 37




```
[XIC(Group_Ready),XIC(Tail_T.DN)GRT(Infeed_Count,0)]XIC(Case_In_Place)XIO(Case_Full)XIO(Case_VFD_Running)XIC(Packer_Ready)ONS(Drop_ONS)OTE(Pack_Drop);
```

HOW THIS RUNG WORKS

In words. When either (`Group_Ready` (The packer infeed holds a case worth) is on) *or* (`Tail_T.DN` (the done bit of `Tail_T` : After the last release: time for the last bottle to reach the packer) is on and `Infeed_Count` (Bottles in the packer infeed) is greater than **0**), `Case_In_Place` (A case is centred under the packer head) is on, `Case_Full` (The case under the head has been packed) is off, `Case_VFD_Running` (Case lane drive output frequency above zero) is off, `Packer_Ready` (Packer head home, not cycling) is on and it is the first scan they are true (the one-shot, remembered in `Drop_ONS`), the rung turns `Pack_Drop` (Packer: pack up to a case worth from the infeed into the case under the head (rising edge)) on (and off again whenever the rung is false).

Left to right:

1. A branch of 2 parallel paths: power gets through if any one of them is true.

   Path 1

   1. `XIC(Group_Ready)` *Examine If Closed* asks: true when `Group_Ready` (The packer infeed holds a case worth) is on.

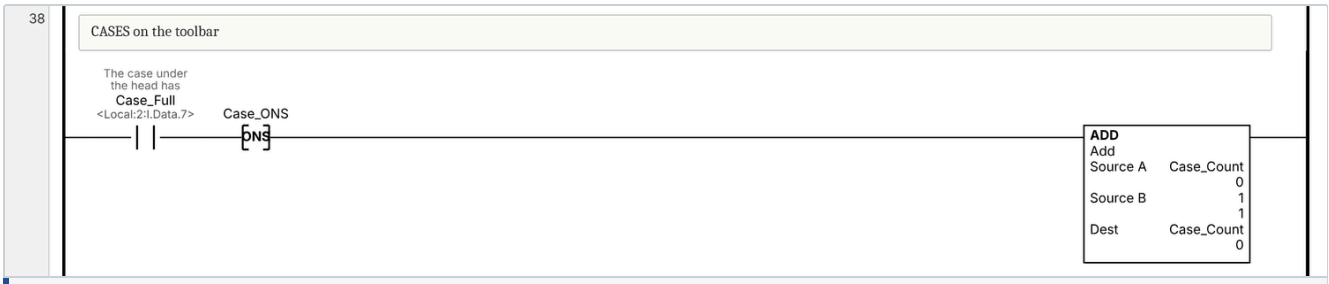
   Path 2

   1. `XIC(Tail_T.DN)` *Examine If Closed* asks: true when `Tail_T.DN` (the done bit of `Tail_T` : After the last release: time for the last bottle to reach the packer) is on.
   2. `GRT(Infeed_Count,0)` *Greater Than* asks: true when `Infeed_Count` (Bottles in the packer infeed) is greater than **0**.
2. `XIC(Case_In_Place)` *Examine If Closed* asks: true when `Case_In_Place` (A case is centred under the packer head) is on.
3. `XIO(Case_Full)` *Examine If Open* asks: true when `Case_Full` (The case under the head has been packed) is off.
4. `XIO(Case_VFD_Running)` *Examine If Open* asks: true when `Case_VFD_Running` (Case lane drive output frequency above zero) is off.
5. `XIC(Packer_Ready)` *Examine If Closed* asks: true when `Packer_Ready` (Packer head home, not cycling) is on.
6. `ONS(Drop_ONS)` *One Shot* asks: true when it is the first scan they are true (the one-shot, remembered in `Drop_ONS`).
7. `OTE(Pack_Drop)` *Output Energize* does: turns `Pack_Drop` (Packer: pack up to a case worth from the infeed into the case under the head (rising edge)) on (and off again whenever the rung is false).

Why it is built this way:

One-shot: once per event, not once per scan.

RUNG 38




```
XIC(Case_Full)ONS(Case_ONS)ADD(Case_Count,1,Case_Count);
```

HOW THIS RUNG WORKS

In words. When `Case_Full` (The case under the head has been packed) is on and it is the first scan they are true (the one-shot, remembered in `Case_ONS`), the rung makes `Case_Count = Case_Count + 1`.

Left to right:

1. `XIC(Case_Full)` *Examine If Closed* asks: true when `Case_Full` (The case under the head has been packed) is on.
2. `ONS(Case_ONS)` *One Shot* asks: true when it is the first scan they are true (the one-shot, remembered in `Case_ONS`).
3. `ADD(Case_Count,1,Case_Count)` *Add* does: makes `Case_Count = Case_Count + 1`. Every scan the rung is true; behind a one-shot it adds once per event.

Why it is built this way:

One-shot: once per event, not once per scan.

The CTU counts rising edges of `Infeed_Release`; at 16 the hopper is empty. **The TON** starts when the last bottle has left the filler (`Busy` off) and gives it 15 s to run down the belt — the belt is 780 px long at 80 px/s, so about 5 s is needed; 15 s is generous and costs nothing, because the timer only starts at the very end. **The PACK rung** drops on a full group as usual, or, once the tail timer is done, on whatever is left in the infeed (`GRT(Infeed_Count,0)`): the packer takes up to four, so the part case gets the last one, two or three.

WATCH FOR: THE LEVEL THAT NEVER ENDS

Sixteen bottles minus the broken ones is rarely a multiple of four. Pack only on `Group_Ready` and the last bottles wait in the infeed for a group that never comes: the hopper is empty, the belt is clear, and the level does not complete. A real line has the same problem at the end of every batch — the last case is a part case, and someone has to decide to send it.

Run it

Load your program, Run mode, Start. Watch the first case come in fast, slow at the eye, stop on the mark; bottles collect in the infeed (the count on the packer's table), and the fourth starts the head. The Simulator Log shows every pack and every scrap-bin push, with points. The reference run: 16 of 16 bottles in the right

bin, nothing lost, in 111 s against a par of 140 s: 1800 points and three stars.

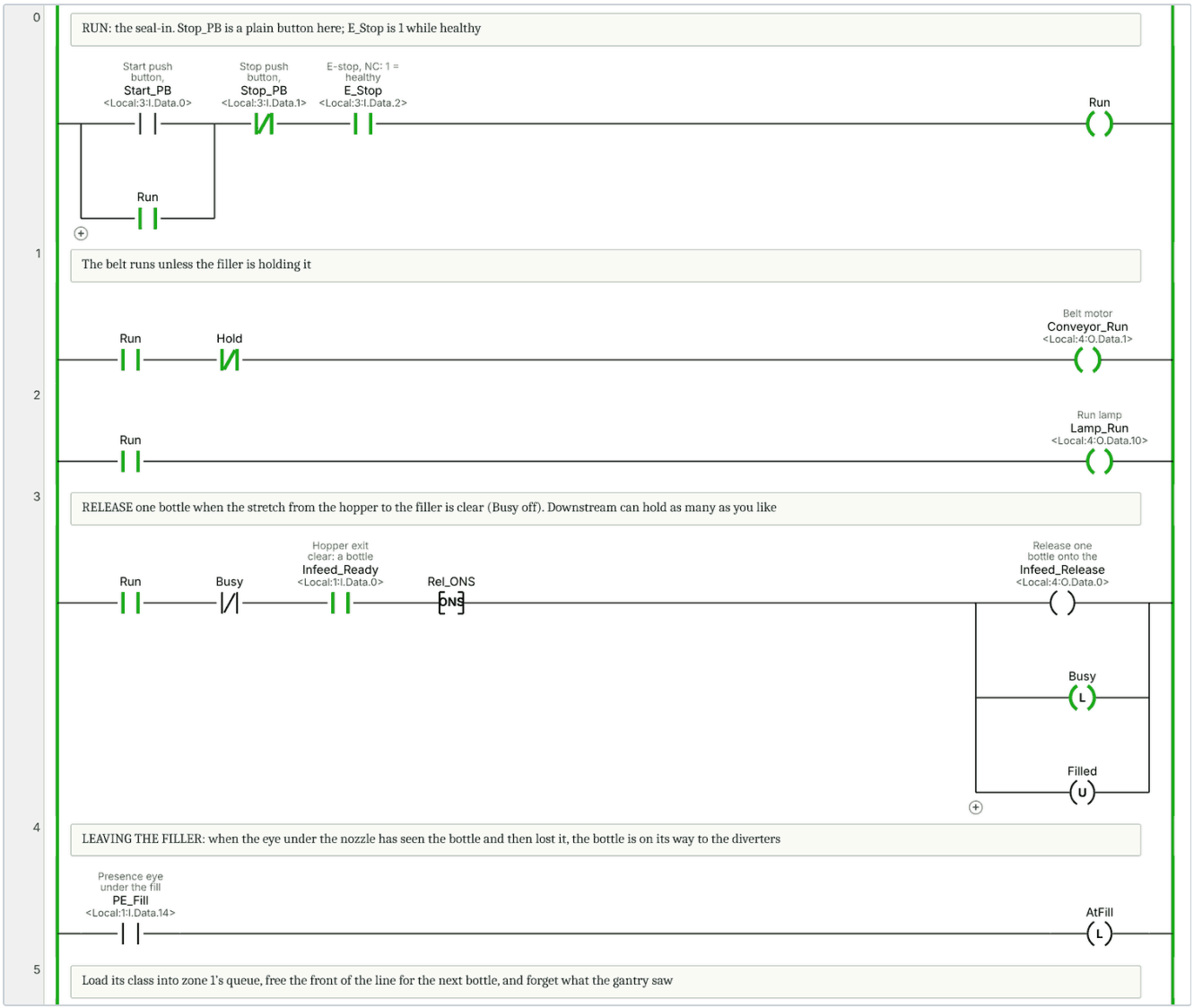


Figure 154. Level 6 online, the top of the routine.

> **TRY IT**
>
> Go for a faster time: the line stops at the filler for every fill, so the packer is never the bottleneck — or is it? Set the case feeder's interval to 8 s and find out. Then pack six to a case (the packer's *Bottles per case*, and the CTU and part-case logic still work unchanged).

Predict it: the answers

1. The rung is false at `XIO(Stop_PB)`: `Stop_PB` is on, so `Run` is **off** (an OTE writes 0 when its rung is false).
2. Every condition on the way is met, so `Conveyor_Run` is **on**.

3. Every condition on the way is met, so `Class.0` is **on**.
4. The rung is false at `XIO(Is_High)`: `Is_High` is on, so `Class.1` is **off** (an OTE writes 0 when its rung is false).
5. The rung is false at `XIO(Is_Broken)`: `Is_Broken` is on, so `Hold` is **off** (an OTE writes 0 when its rung is false).
6. Every condition on the way is met, so `Fill_Valve` is **on**.
7. The rung is false at the GEQ: `Fill_Level` is below **97**, so the OTL does nothing: `Filled` keeps whatever it had.
8. Every condition on the way is met, so `Div1` is **on**.
9. Every condition on the way is met, so `Case_Run` is **on**.
10. Every condition on the way is met, so `Case_Creep` latches **on**.
11. Every condition on the way is met, so `Case_Creep` is unlatched, **off**.
12. The rung is false at `XIC(Released.DN)`: `Released.DN` (the done bit of `Released`: Bottles released from the hopper) is off, so `Tail_T` is reset to zero (a TON resets when its rung goes false).

39 Level 7 — Batch Mix

> **IN THE APP**
>
> File → New Project… → A graded level → **7**. Guided mode opens on this chapter: the tags, every rung below under its fold, a Verify check, and five results — pump A stops on its count and B tops up to HI, the batch heats with the mixer running, all three batches made, every batch on spec, under par.

No belt, no bottles. One **Mix Tank** — the new part of Version 97, built after the LogixPro Batch Mix — and a console. Make three batches, one of each recipe, in the order the **recipe selector** shows: the level turns the selector to the next recipe each time a batch drains. Par 240 s.

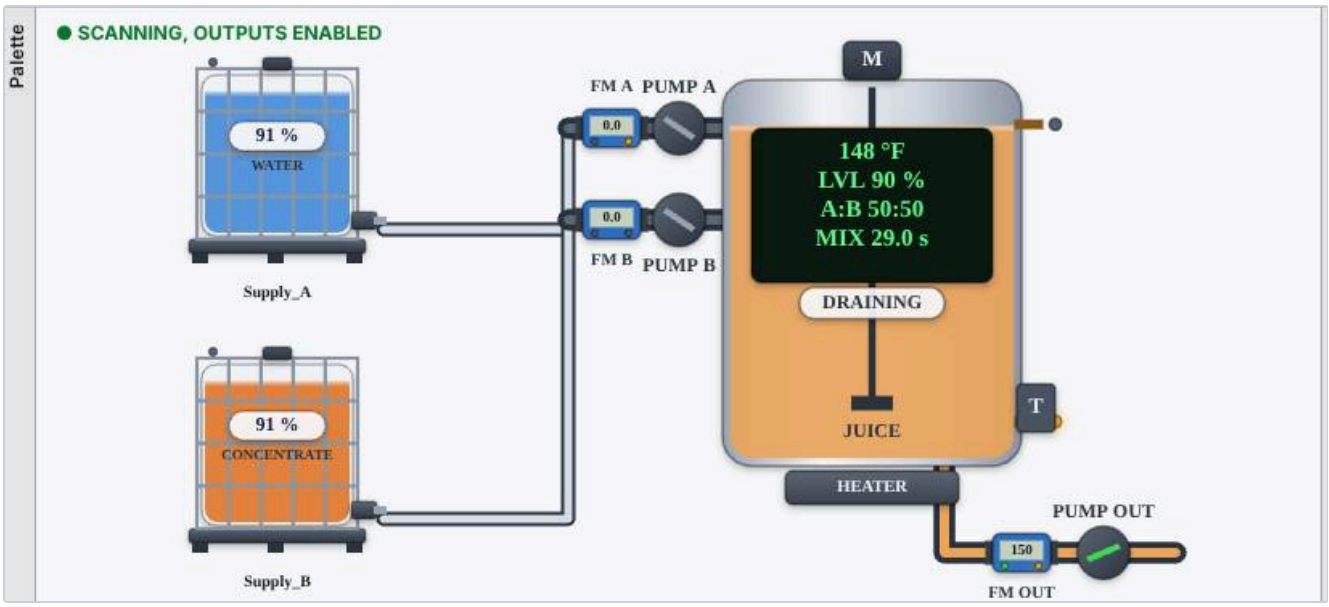


Figure 155. Level 7: the two supply totes piped to the mix tank's inlet pumps and flowmeters (the pipes light in the liquid's colour while it moves), the heater under the tank, the mixer on the lid, HI and LO probes and the thermostat on the wall, the outlet pump and flowmeter.

The tank

- **Pump_A**, **Pump_B** — the two ingredients, 100 units/s each. Each runs through a flowmeter: **Flow_A**, **Flow_B** give one pulse per 20 units, a square wave, so a CTU on the input counts the dose.
- **Hi_Level**, **Lo_Level** — probes, 1 while the liquid covers them (90 % and 10 %). **Tank_Level** is the transmitter, 0-100 %; the tank holds 2000 units, so HI is 1800.
- **Heater** — under the tank, about 3 °F a second while the element is covered. The tank loses heat to the room all the time, faster the hotter it is. Run it below LO and the element is uncovered: the batch is scorched.

- **Tank_Temp** — °F, an analog input. **Thermostat** is a separate high-limit switch that closes at 180 °F: a safety, not the control.
- **Mixer** — mixing time counts only while the impeller is covered.
- **Pump_Out** — empties the tank at 150 units/s through its own flowmeter. (Pipe the tank to a filler and the filler draws through this pump instead — see Try it.)
- The operator stations: the console and **Tank_HMI**, whose START and STOP pulse **HMI_Start** and **HMI_Stop**, also on the Operator View tab (chapter 14).
- The console: **Start_PB**, **Stop_PB**, **E_Stop**, **Recipe_A** and **Recipe_C** from the three-position selector (neither means B), **Lamp_Run**, **Lamp_Idle**, **Lamp_Full**, **Lamp_Fault**, and the BATCHES and °F displays.

Where the ingredients come from

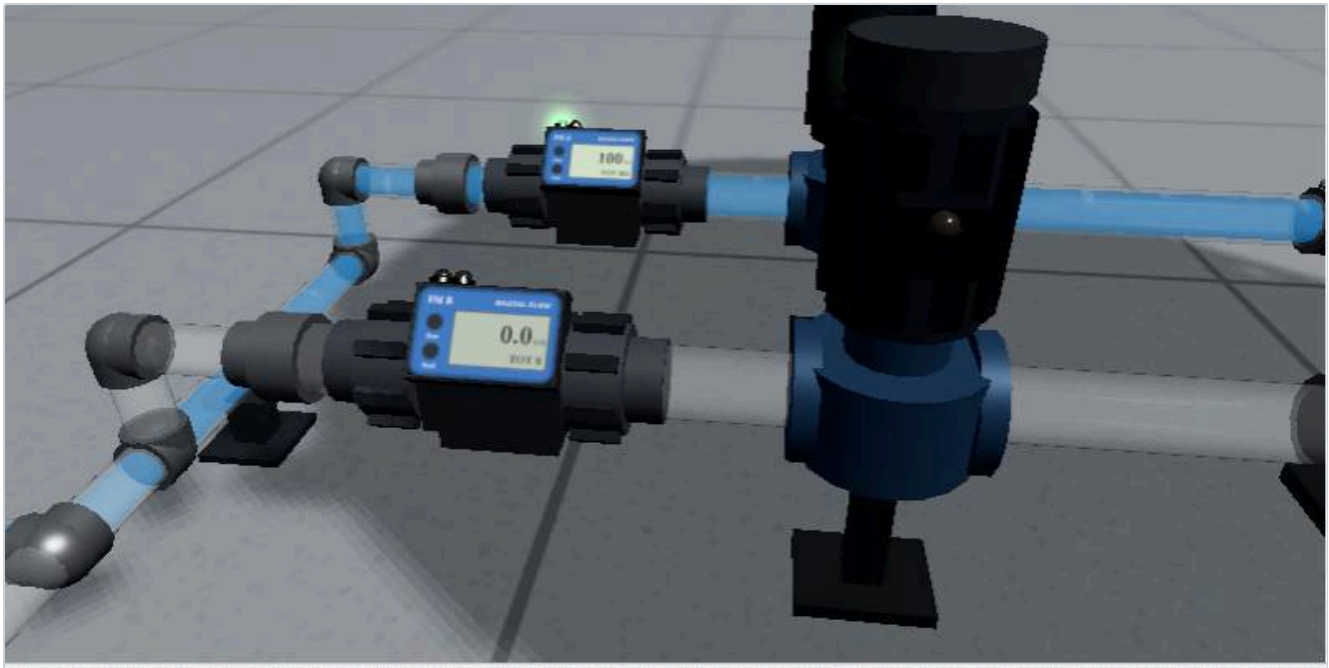
Each inlet is piped to a **Supply Tote**: **Supply_A** holds water, **Supply_B** orange concentrate, 10,000 units each. The pipes are part of the scene. The Mix Tank's *Inlet A fed from* and *Inlet B fed from* settings name the totes, and the pumps take what they move out of them. An empty tote runs its pump dry: no flow, no flowmeter pulses, and a line in the Simulator Log. An inlet fed from nothing is on the *plant supply header*, drawn in 3D as a riser out of the floor with a ball valve. The outlet, with nothing piped to it, runs to a floor drain.

The flowmeters are inline digital meters. The LCD shows the rate in units a second and the running total; the green lamp is on while liquid moves and the amber one blinks with each pulse the meter sends the controller. Tick *Flowmeters also report their rate* and each meter adds an analog input, `FlowA_Rate` and so on. Bind that signal to a REAL tag in its properties and the program reads 87.5, not 87. While the ingredients go in, the tank's contents look clear and pale; mixing, faster when warm, turns them cloudy, and the next batch starts clear again.

The tank has a local panel on its shell, and the console side of the floor has a **Process HMI** on a pedestal, **Tank_HMI**. Both show the same live faceplate: the vessel with its level and probes, the temperature against the high limit, the mixing time, a one-minute trend, the ratio, the flows, which pumps and heater are on, the state and an alarm line. It is drawn the way plant HMIs are, grey with colour only for what is running or wrong, so a lit heater or an alarm stands out at a glance. The HMI reads the tank directly; it is a window for you, not an input to the program.

vel 7 · Batch Mix on sandbox parts. Make three batches in the mix tank, one of each recipe, in the order the recipe selector shows (the level turns it after
tch). A: 50 % ingredient A, 150 °F, mix 10 s. B: 75 % A, 130 °F, mix 15 s. C: 25 % A, 170 °F, mix 8 s. Dose A by counting flowmeter A, top up with B to the HI
at and mix, drain. Each batch is graded as it drains: +100 each for the ratio (±5 %), the temperature at drain (−3 to +8 °F) and the mixing time; −50 for eac
ssed. −50 for a short batch. −100 for an overflow or a scorched batch (heater on with the element uncovered). −25 if the outlet pump runs dry more than 3

Figure 156. The mix tank's local panel during HEAT+MIX: 125 °F against a 180 °F high limit, 90 % level with both probes covered, the trend of temperature (solid) and level (dashed), the 50:50 ratio, and the heater and mixer lit.



vel 7 · Batch Mix on sandbox parts. Make three batches in the mix tank, one of each recipe, in the order the recipe selector shows (the level turns it after
tch). A: 50 % ingredient A, 150 °F, mix 10 s. B: 75 % A, 130 °F, mix 15 s. C: 25 % A, 170 °F, mix 8 s. Dose A by counting flowmeter A, top up with B to the HI
at and mix, drain. Each batch is graded as it drains: +100 each for the ratio (±5 %), the temperature at drain (−3 to +8 °F) and the mixing time; −50 for eac
ssed. −50 for a short batch. −100 for an overflow or a scorched batch (heater on with the element uncovered). −25 if the outlet pump runs dry more than 3

Figure 157. The inlet runs in 3D: the clear pipe from Supply_A carries water to flowmeter A, whose LCD reads 100 units/s with its green lamp lit, then through pump A; flowmeter B reads 0.0 with pump B stopped.

How each batch is scored

Recipe	Ingredient A	Temperature	Mix at least
A	50 %	150 °F	10 s
B	75 %	130 °F	15 s
C	25 %	170 °F	8 s

Graded when the batch drains	Points
A share within ±5 % of the recipe	+100 (−50 if not)
Temperature when the outlet pump started: recipe −3 to +8 °F	+100 (−50 if not)
Mixed at least the recipe time	+100 (−50 if not)
Less than 80 % of the tank (a short batch)	−50
Overflowed past the top	−100
Scorched (heater on with the element uncovered)	−100
Outlet pump ran dry more than 3 s	−25
All three batches under par (240 s)	+200

How to pass it, step by step

1. **Think in steps, not in rungs.** One DINT, `Step`, says what the tank is doing: 0 waiting for an empty tank, 1 dosing A, 2 topping up with B, 3 heating and mixing, 4 draining. Every output rung is `XIC(Run)EQU(Step,n)OTE(…)`, and every transition is a rung that moves Step on.
2. **Read the recipe into tags.** Three rungs: the selector position sets `Target_A` (pulses of A), `Target_T` (°F) and `Mix_Pre` (ms). A full tank at HI is 1800 units, so 50 % A is 900 units — 45 pulses; 75 % is 68; 25 % is 23.
3. **Dose A by counting.** Pump A runs in step 1; a CTU counts Flow_A; at `Target_A` move to step 2. Reset the counter when a batch starts.
4. **Top up with B to HI.** Pump B runs in step 2 until Hi_Level, and the HI probe ends it — B is "whatever is left", so the ratio is set by the count of A alone.
5. **Heat with hysteresis.** Latch `Heat_On` below the target minus 2 °F, unlatch it at the target. Gate the Heater output with `XIC(Lo_Level)` (never an uncovered element) and `XIO(Thermostat)` (the high limit). Heat on `LES(Tank_Temp,Target_T)` alone and the heater switches on and off every scan at the setpoint.
6. **Mix on a timer with a preset tag.** `TON(Mix_T,Mix_Pre,0)` in step 3 — the preset operand can be a tag, so one timer serves all three recipes. Leave step 3 when the timer is done *and* the temperature is in the band.

7. **Drain to empty.** Pump_Out in step 4. The level reads 0 % with a few units still in the tank, so run the pump on for a second (`Empty_T`) and then go back to step 0 and count the batch. Any longer and the pump runs dry.

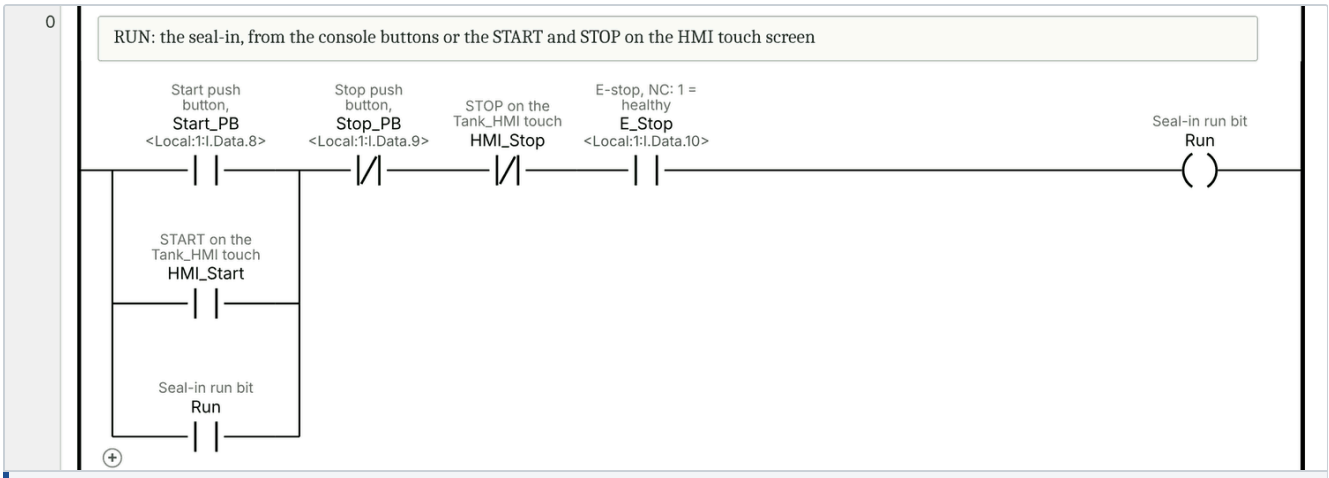
Tags this lesson needs

The level's alias tags are already made for you (chapter 32's table). Create these plain tags before you type the rungs; Verify reports any you miss.

Tag	Type	How to make it	Description
`Run`	BOOL	Edit Tags: name, type	Seal-in run bit
`HMI_Start`	BOOL	Edit Tags: name, type	START on the Tank_HMI touch screen (pulsed by the HMI)
`HMI_Stop`	BOOL	Edit Tags: name, type	STOP on the Tank_HMI touch screen (pulsed by the HMI)
`Step`	DINT	Edit Tags: name, type	Batch step: 0 wait for an empty tank, 1 dose A, 2 top up with B, 3 heat and mix, 4 drain
`A_Count`	COUNTER	Edit Tags: name, type	Flowmeter A pulses this batch
`Target_A`	DINT	Edit Tags: name, type	Pulses of A for the selected recipe
`Target_T`	DINT	Edit Tags: name, type	Temperature for the selected recipe, °F
`Target_Lo`	DINT	Edit Tags: name, type	Heater comes back on below this, °F
`Mix_T`	TIMER	Edit Tags: name, type	Mixing time this batch
`Mix_Pre`	DINT	Edit Tags: name, type	Mixing time for the selected recipe, ms
`Heat_On`	BOOL	Edit Tags: name, type	Heater wanted (with hysteresis)
`Empty_T`	TIMER	Edit Tags: name, type	Run the outlet pump on a moment after the level reads 0
`Done_ONS`	BOOL	Edit Tags: name, type	

Run, recipe A and the start of a batch

RUNG 0




```
[XIC(Start_PB),XIC(HMI_Start),XIC(Run)]XIO(Stop_PB)XIO(HMI_Stop)XIC(E_Stop)OTE(Run);
```

PREDICT IT · 1 `Start_PB` is on, `Stop_PB` is on, `HMI_Stop` is off and `E_Stop` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When either (`Start_PB` (Start push button, momentary) is on) *or* (`HMI_Start` (START on the Tank_HMI touch screen (pulsed by the HMI)) is on) *or* (`Run` (Seal-in run bit) is on), `Stop_PB` (Stop push button, momentary) is off, `HMI_Stop` (STOP on the Tank_HMI touch screen (pulsed by the HMI)) is off and `E_Stop` (E-stop, NC: 1 = healthy) is on, the rung turns `Run` on (and off again whenever the rung is false).

Left to right:

1. A branch of 3 parallel paths: power gets through if any one of them is true.
   - **Path 1**
     1. `XIC(Start_PB)` *Examine If Closed* asks: true when `Start_PB` (Start push button, momentary) is on.
   - **Path 2**
     1. `XIC(HMI_Start)` *Examine If Closed* asks: true when `HMI_Start` (START on the Tank_HMI touch screen (pulsed by the HMI)) is on.
   - **Path 3**
     1. `XIC(Run)` *Examine If Closed* asks: true when `Run` (Seal-in run bit) is on.
2. `XIO(Stop_PB)` *Examine If Open* asks: true when `Stop_PB` (Stop push button, momentary) is off.
3. `XIO(HMI_Stop)` *Examine If Open* asks: true when `HMI_Stop` (STOP on the Tank_HMI touch screen (pulsed by the HMI)) is off.
4. `XIC(E_Stop)` *Examine If Closed* asks: true when `E_Stop` (E-stop, NC: 1 = healthy) is on.
5. `OTE(Run)` *Output Energize* does: turns `Run` on (and off again whenever the rung is false). Every scan it writes the rung state: true on, false off.

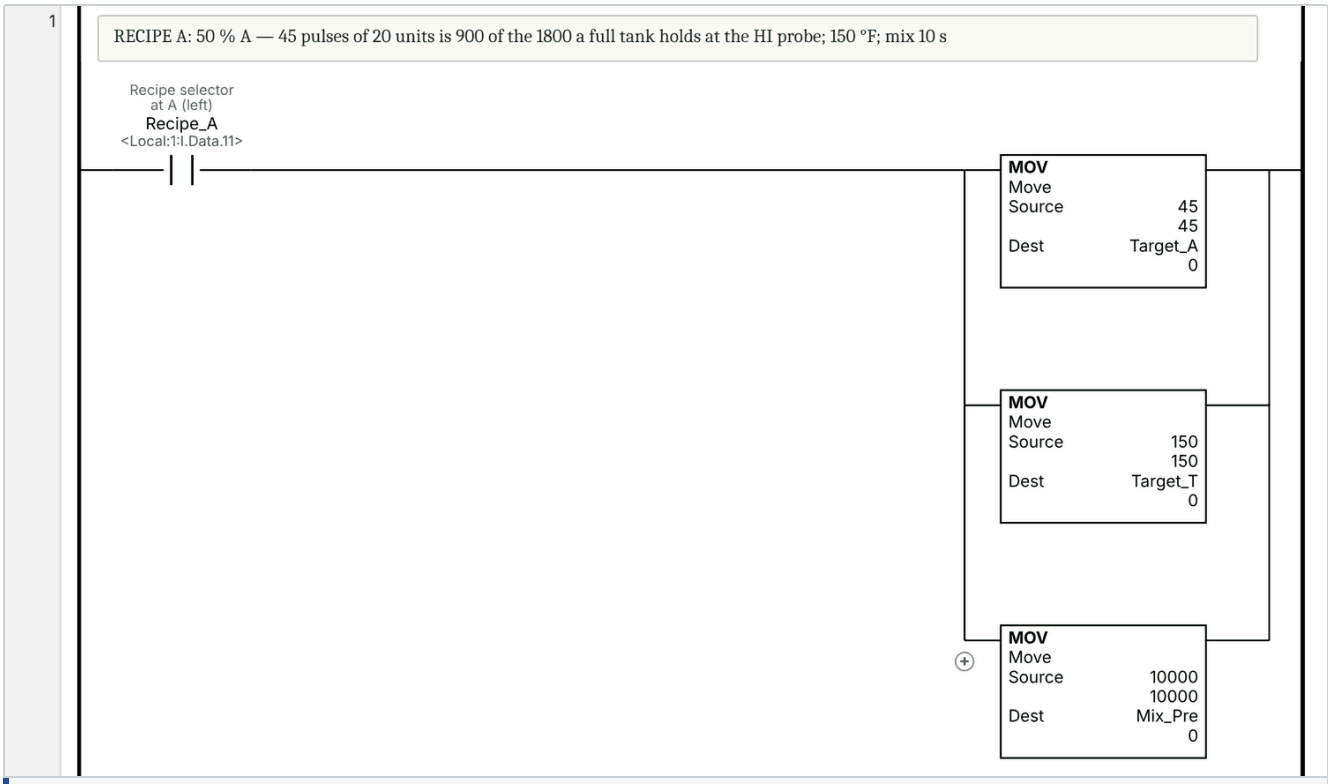
Why it is built this way:

Seal-in. The start condition turns `Run` on, and `Run` 's own contact in the parallel path keeps the rung true after the start goes away. Anything in series outside the branch (a stop, a fault) breaks it, and then it stays off until the start is made again.

> **Think of it like this:** a doorbell button and a hand. The start button is the doorbell, pressed for a moment. The seal-in contact is a hand that grabs the circuit as soon as the output comes on and keeps holding after you let go. The stop button, in series, is the only thing that can knock that hand away.

> **Watch for:** The stop has to be in series outside the branch. Put it inside the start path only and the seal-in path bypasses it: the stop no longer stops.

RUNG 1




```
XIC(Recipe_A)[MOV(45,Target_A),MOV(150,Target_T),MOV(10000,Mix_Pre)];
```

HOW THIS RUNG WORKS

In words. When `Recipe_A` (Recipe selector at A (left)) is on, the rung puts **45** into `Target_A` (Pulses of A for the selected recipe), puts **150** into `Target_T` (Temperature for the selected recipe, °F) and puts **10000** into `Mix_Pre` (Mixing time for the selected recipe, ms).

Left to right:

1. `XIC(Recipe_A)` *Examine If Closed* asks: true when `Recipe_A` (Recipe selector at A (left)) is on.
2. A branch of 3 parallel paths: each path is tested on its own and does its own work.

Path 1

1. `MOV(45,Target_A)` *Move* does: puts **45** into `Target_A` (Pulses of A for the selected recipe). It writes every scan the rung is true.

Path 2

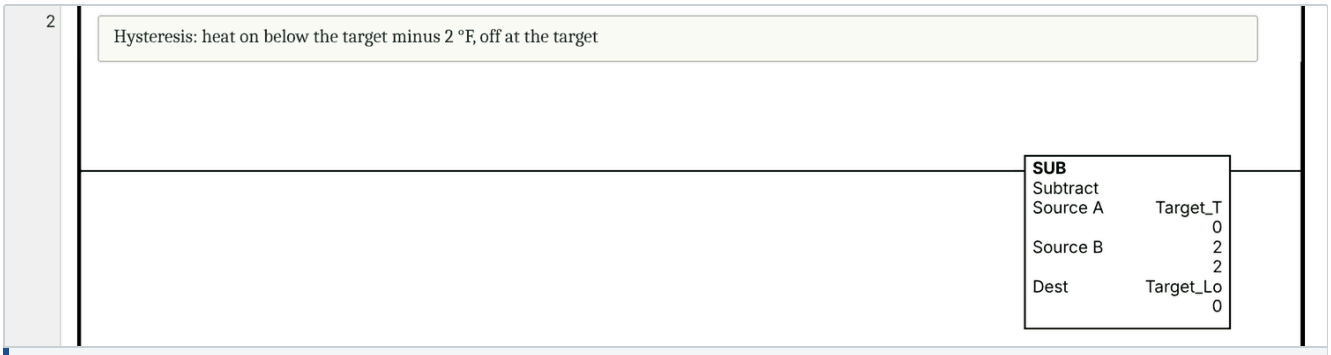
1. `MOV(150,Target_T)` *Move* does: puts **150** into `Target_T` (Temperature for the selected recipe, °F).

Path 3

1. `MOV(10000,Mix_Pre)` *Move* does: puts **10000** into `Mix_Pre` (Mixing time for the selected recipe, ms).

> **Think of it like this:** a rung is a pipe with valves in it, read left to right. Every contact is a valve: XIC is open while its bit is 1, XIO while its bit is 0. Water reaches the output at the right only if every valve in a row is open. The output does not remember anything: on every scan it is simply wet or dry.

RUNG 2




```
SUB(Target_T,2,Target_Lo);
```

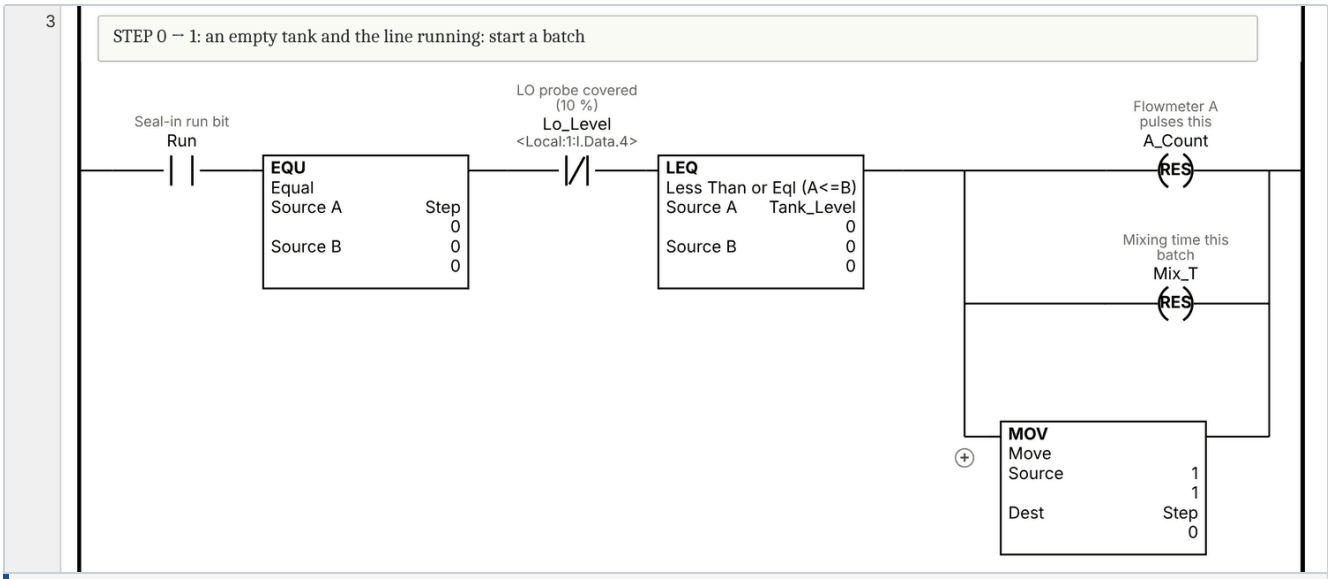
HOW THIS RUNG WORKS

In words. Every scan, with nothing to test, the rung makes `Target_Lo` (Heater comes back on below this, °F) = `Target_T` (Temperature for the selected recipe, °F) − **2**.

Left to right:

1. `SUB(Target_T,2,Target_Lo)` *Subtract* does: makes `Target_Lo` (Heater comes back on below this, °F) = `Target_T` (Temperature for the selected recipe, °F) − **2**.

RUNG 3




```
XIC(Run)EQU(Step,0)XIO(Lo_Level)LEQ(Tank_Level,0)
[RES(A_Count),RES(Mix_T),MOV(1,Step)];
```

PREDICT IT · 2 `Run` is on, `Step` is 0, `Lo_Level` is off and `Tank_Level` is 0. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Run` (Seal-in run bit) is on, `Step` (Batch step: 0 wait for an empty tank, 1 dose A, 2 top up with B, 3 heat and mix, 4 drain) equals **0**, `Lo_Level` (LO probe covered (10 %)) is off and `Tank_Level` (Tank level 0-100 %) is at most **0**, the rung resets `A_Count` (Flowmeter A pulses this batch), resets `Mix_T` (Mixing time this batch) and puts **1** into `Step`.

Left to right:

1. `XIC(Run)` *Examine If Closed* asks: true when `Run` (Seal-in run bit) is on.
2. `EQU(Step,0)` *Equal* asks: true when `Step` (Batch step: 0 wait for an empty tank, 1 dose A, 2 top up with B, 3 heat and mix, 4 drain) equals **0**. Both values are compared every scan.
3. `XIO(Lo_Level)` *Examine If Open* asks: true when `Lo_Level` (LO probe covered (10 %)) is off.
4. `LEQ(Tank_Level,0)` *Less Than or Equal* asks: true when `Tank_Level` (Tank level 0-100 %) is at most **0**.
5. A branch of 3 parallel paths: each path is tested on its own and does its own work.

   Path 1

   1. `RES(A_Count)` *Reset* does: resets `A_Count` (Flowmeter A pulses this batch). It zeroes a timer or counter (accumulator and status bits), or a control (its position and status bits).

   Path 2

   1. `RES(Mix_T)` *Reset* does: resets `Mix_T` (Mixing time this batch).

   Path 3

   1. `MOV(1,Step)` *Move* does: puts **1** into `Step`.

Why it is built this way:

A step of a sequence. The rung only acts while `Step` is 0; when its conditions are met it writes 1 into `Step`, and from then on the rungs that test for 1 take over. One step number at a time means one thing happening at a time.

> **Think of it like this:** a recipe card with numbered steps and one finger on it. Each rung only reads the line under the finger; when its line is done it moves the finger to the next number. Because the finger can only be in one place, only one step can ever be "doing".

> **Watch for:** A rung further down that tests for the new step number sees it in this same scan. Rung order decides whether the next step starts now or next scan.

The recipe rungs run every scan; the level only turns the selector when a batch has drained, so the values never change in the middle of one. Only recipe A is here: the first batch is always A, and the rungs for B and C come after the batch-out rung, where they can be seen working the moment they are written. `SUB(Target_T,2,Target_Lo)` is the bottom of the heater's band. A batch starts only on an empty tank: `XIO(Lo_Level)` and `LEQ(Tank_Level,0)`.

Dosing: A by count, B to HI

RUNG 4

```
XIC(Run)EQU(Step,1)OTE(Pump_A);
```

PREDICT IT · 3 `Run` is off and `Step` is 1. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

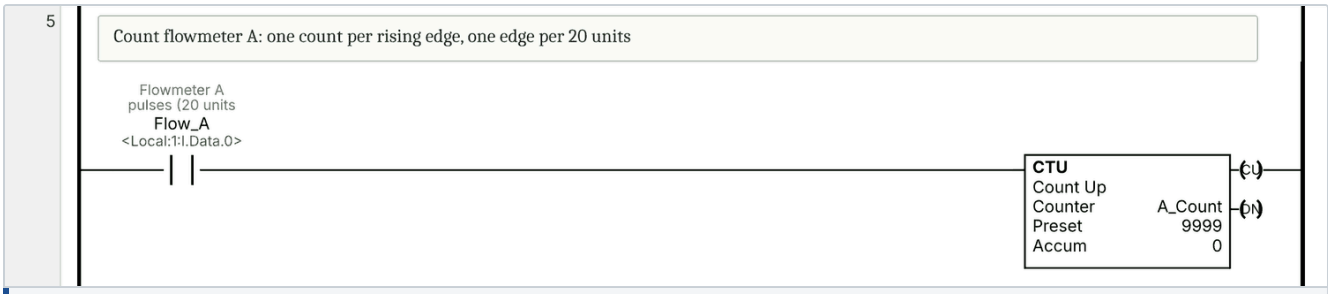
HOW THIS RUNG WORKS

In words. When `Run` (Seal-in run bit) is on and `Step` (Batch step: 0 wait for an empty tank, 1 dose A, 2 top up with B, 3 heat and mix, 4 drain) equals **1**, the rung turns `Pump_A` (Ingredient A pump) on (and off again whenever the rung is false).

Left to right:

1. `XIC(Run)` *Examine If Closed* asks: true when `Run` (Seal-in run bit) is on.
2. `EQU(Step,1)` *Equal* asks: true when `Step` (Batch step: 0 wait for an empty tank, 1 dose A, 2 top up with B, 3 heat and mix, 4 drain) equals **1**.
3. `OTE(Pump_A)` *Output Energize* does: turns `Pump_A` (Ingredient A pump) on (and off again whenever the rung is false).

RUNG 5




```
XIC(Flow_A)CTU(A_Count,9999,0);
```

HOW THIS RUNG WORKS

In words. When `Flow_A` (Flowmeter A pulses (20 units each)) is on, the rung counts `A_Count` (Flowmeter A pulses this batch) up by one (preset 9999).

Left to right:

1. `XIC(Flow_A)` *Examine If Closed* asks: true when `Flow_A` (Flowmeter A pulses (20 units each)) is on.
2. `CTU(A_Count,9999,0)` *Count Up* does: counts `A_Count` (Flowmeter A pulses this batch) up by one (preset 9999). It counts false-to-true transitions of the rung, not scans; .DN is on once the count reaches the preset; it keeps counting past it.

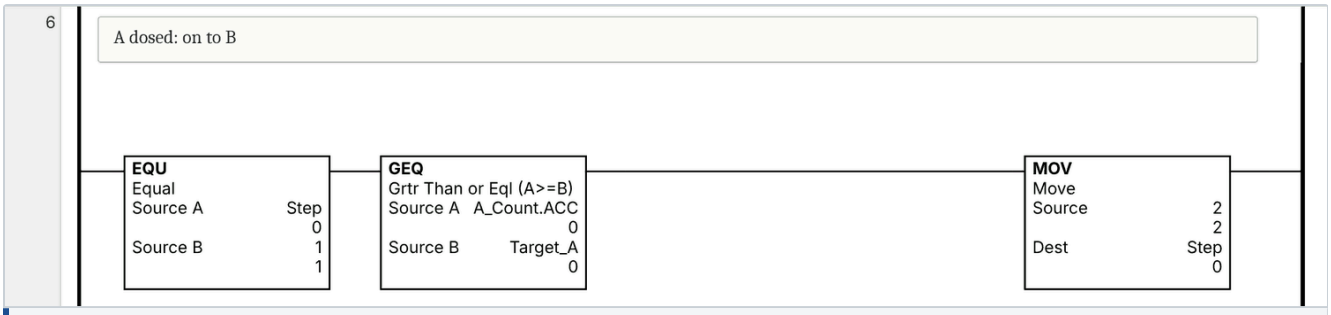
Why it is built this way:

A counter. It counts transitions, not time: each false-to-true of the rung adds one. A RES elsewhere clears it.

> **Think of it like this:** the clicker a doorman holds: one click for every person who walks through the door, however long they stand in it. The number stays on the clicker until somebody presses reset.

> **Watch for:** A counter keeps its count through a stop and a restart. Reset it on purpose where the count should start again.

RUNG 6




```
EQU(Step,1)GEQ(A_Count.ACC,Target_A)MOV(2,Step);
```

PREDICT IT · 4 `Step` is 1, `A_Count.ACC` (the accumulated value of `A_Count` : Flowmeter A pulses this batch) is 499 and `Target_A` is 500. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Step` (Batch step: 0 wait for an empty tank, 1 dose A, 2 top up with B, 3 heat and mix, 4 drain) equals **1** and `A_Count.ACC` (the accumulated value of `A_Count` : Flowmeter A pulses this batch) is at least `Target_A` (Pulses of A for the selected recipe), the rung puts **2** into `Step` .

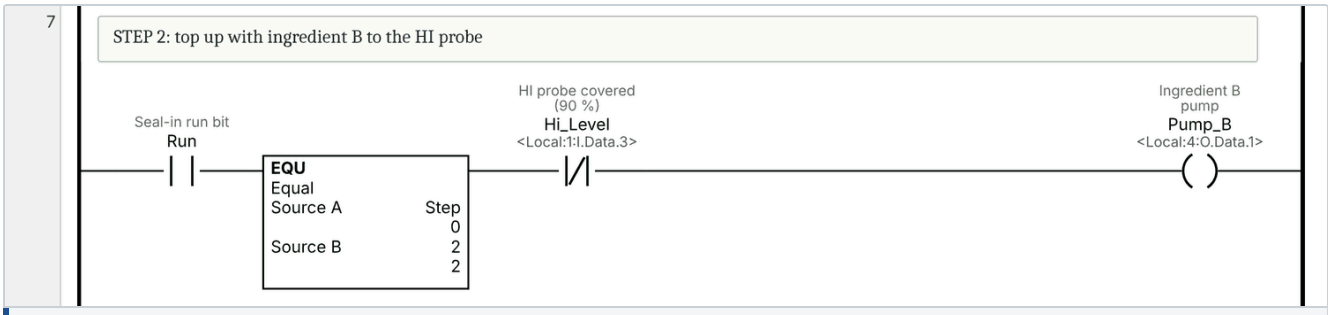
Left to right:

1. `EQU(Step,1)` *Equal* asks: true when `Step` (Batch step: 0 wait for an empty tank, 1 dose A, 2 top up with B, 3 heat and mix, 4 drain) equals **1**.
2. `GEQ(A_Count.ACC,Target_A)` *Greater Than or Equal* asks: true when `A_Count.ACC` (the accumulated value of `A_Count` : Flowmeter A pulses this batch) is at least `Target_A` (Pulses of A for the selected recipe).
3. `MOV(2,Step)` *Move* does: puts **2** into `Step` .

Why it is built this way:

A step of the sequence: from Step = 1 to 2.

RUNG 7




```
XIC(Run)EQU(Step,2)XIO(Hi_Level)OTE(Pump_B);
```

PREDICT IT · 5 `Run` is on, `Step` is 2 and `Hi_Level` is off. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Run` (Seal-in run bit) is on, `Step` (Batch step: 0 wait for an empty tank, 1 dose A, 2 top up with B, 3 heat and mix, 4 drain) equals **2** and `Hi_Level` (HI probe covered (90 %)) is off, the rung turns `Pump_B` (Ingredient B pump) on (and off again whenever the rung is false).

Left to right:

1. `XIC(Run)` *Examine If Closed* asks: true when `Run` (Seal-in run bit) is on.
2. `EQU(Step,2)` *Equal* asks: true when `Step` (Batch step: 0 wait for an empty tank, 1 dose A, 2 top up with B, 3 heat and mix, 4 drain) equals **2**.
3. `XIO(Hi_Level)` *Examine If Open* asks: true when `Hi_Level` (HI probe covered (90 %)) is off.
4. `OTE(Pump_B)` *Output Energize* does: turns `Pump_B` (Ingredient B pump) on (and off again whenever the rung is false).

RUNG 8

```
EQU(Step,2)XIC(Hi_Level)MOV(3,Step);
```

PREDICT IT · 6 `Step` is 3 and `Hi_Level` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Step` (Batch step: 0 wait for an empty tank, 1 dose A, 2 top up with B, 3 heat and mix, 4 drain) equals **2** and `Hi_Level` (HI probe covered (90 %)) is on, the rung puts **3** into `Step`.

Left to right:

1. `EQU(Step,2)` *Equal* asks: true when `Step` (Batch step: 0 wait for an empty tank, 1 dose A, 2 top up with B, 3 heat and mix, 4 drain) equals **2**.
2. `XIC(Hi_Level)` *Examine If Closed* asks: true when `Hi_Level` (HI probe covered (90 %)) is on.
3. `MOV(3,Step)` *Move* does: puts **3** into `Step`.

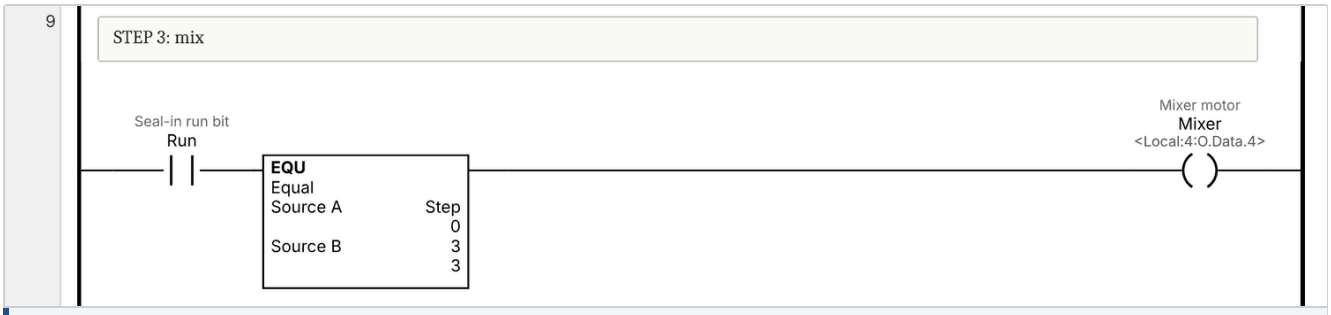
Why it is built this way:

A step of the sequence: from Step = 2 to 3.

The CTU counts every rising edge of Flow_A, whatever step it is in — fine, because only step 1 runs pump A, and the batch start resets it.

Heating and mixing

RUNG 9




```
XIC(Run)EQU(Step,3)OTE(Mixer);
```

PREDICT IT · 7 `Run` is on and `Step` is 3. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Run` (Seal-in run bit) is on and `Step` (Batch step: 0 wait for an empty tank, 1 dose A, 2 top up with B, 3 heat and mix, 4 drain) equals **3**, the rung turns `Mixer` (Mixer motor) on (and off again whenever the rung is false).

Left to right:

1. `XIC(Run)` *Examine If Closed* asks: true when `Run` (Seal-in run bit) is on.
2. `EQU(Step,3)` *Equal* asks: true when `Step` (Batch step: 0 wait for an empty tank, 1 dose A, 2 top up with B, 3 heat and mix, 4 drain) equals **3**.
3. `OTE(Mixer)` *Output Energize* does: turns `Mixer` (Mixer motor) on (and off again whenever the rung is false).

RUNG 10

```
XIC(Run)EQU(Step,3)TON(Mix_T,Mix_Pre,0);
```

PREDICT IT · 8 `Run` is on and `Step` is 4. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Run` (Seal-in run bit) is on and `Step` (Batch step: 0 wait for an empty tank, 1 dose A, 2 top up with B, 3 heat and mix, 4 drain) equals **3**, the rung runs the timer `Mix_T` (Mixing time this batch) toward Mix_Pre.

Left to right:

1. `XIC(Run)` *Examine If Closed* asks: true when `Run` (Seal-in run bit) is on.
2. `EQU(Step,3)` *Equal* asks: true when `Step` (Batch step: 0 wait for an empty tank, 1 dose A, 2 top up with B, 3 heat and mix, 4 drain) equals **3**.
3. `TON(Mix_T,Mix_Pre,0)` *Timer On Delay* does: runs the timer `Mix_T` (Mixing time this batch) toward Mix_Pre. The accumulator counts while the rung is true; .DN comes on at the preset; the moment the rung goes false the timer resets.

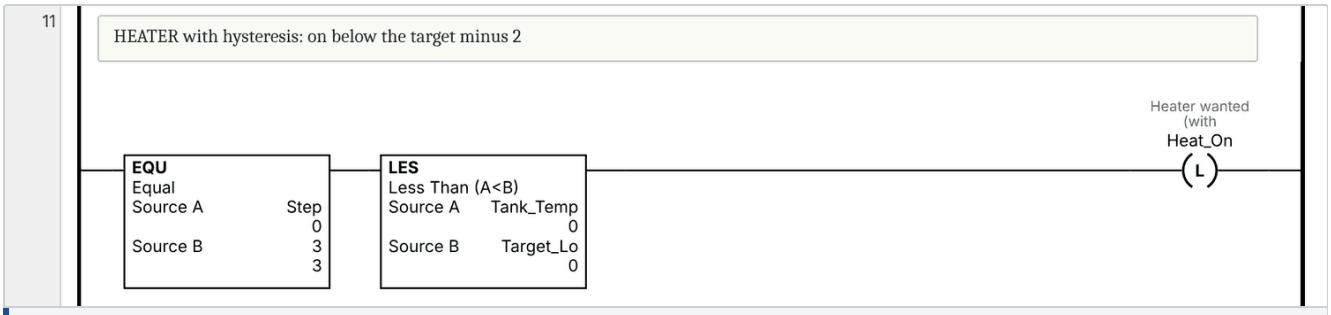
Why it is built this way:

A timer. `Mix_T` only times while this rung is true; other rungs use its `.DN` (done) or `.TT` (timing) bits. Being a TON, it starts again from zero every time the rung goes false.

> **Think of it like this:** a kitchen timer that only runs while you hold your finger on it. Take your finger off and it snaps back to zero. Hold it long enough and the done light comes on, and stays on as long as your finger is still there.

> **Watch for:** If anything in series flickers false for one scan, a TON starts over from zero. Use RTO when the time must survive an interruption.

RUNG 11




```
EQU(Step,3)LES(Tank_Temp,Target_Lo)OTL(Heat_On);
```

PREDICT IT · 9 `Step` is 3, `Tank_Temp` is 499 and `Target_Lo` is 500. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Step` (Batch step: 0 wait for an empty tank, 1 dose A, 2 top up with B, 3 heat and mix, 4 drain) equals **3** and `Tank_Temp` (Tank temperature, °F) is less than `Target_Lo` (Heater comes back on below this, °F), the rung latches `Heat_On` (Heater wanted (with hysteresis)) on.

Left to right:

1. `EQU(Step,3)` *Equal* asks: true when `Step` (Batch step: 0 wait for an empty tank, 1 dose A, 2 top up with B, 3 heat and mix, 4 drain) equals **3**.
2. `LES(Tank_Temp,Target_Lo)` *Less Than* asks: true when `Tank_Temp` (Tank temperature, °F) is less than `Target_Lo` (Heater comes back on below this, °F).
3. `OTL(Heat_On)` *Output Latch* does: latches `Heat_On` (Heater wanted (with hysteresis)) on. It only ever sets the bit; the bit stays on after the rung goes false.

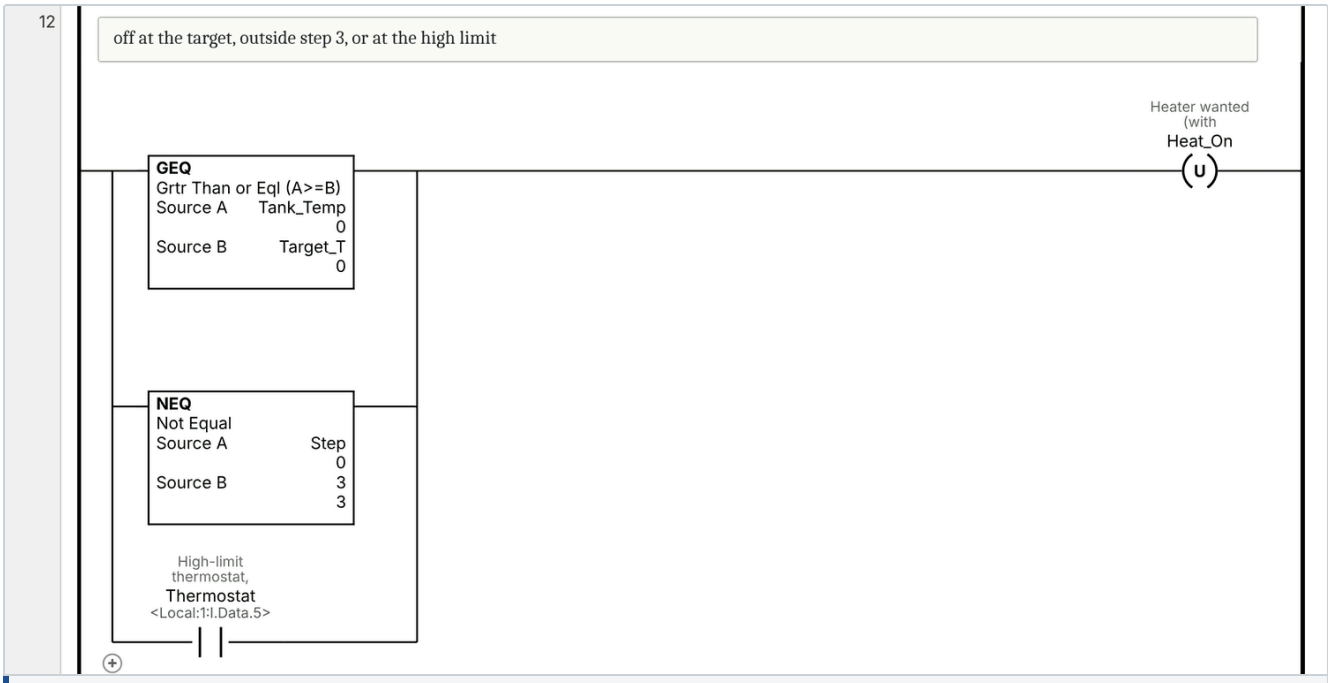
Why it is built this way:

A latch. `Heat_On` stays on after this rung goes false; an `OTU` elsewhere in the program clears it.

> **Think of it like this:** a light switch instead of a doorbell. OTL flips it on and it stays on when you take your hand away; OTU flips it off. Whatever flipped it on does not have to stay there.

> **Watch for:** A latched bit survives the condition that set it: every way in needs a way out, or it stays on for good.

RUNG 12




```
[GEQ(Tank_Temp,Target_T),NEQ(Step,3),XIC(Thermostat)]OTU(Heat_On);
```

PREDICT IT · 10 `Tank_Temp` is 500 and `Target_T` is 500. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When either (`Tank_Temp` (Tank temperature, °F) is at least `Target_T` (Temperature for the selected recipe, °F)) *or* (`Step` (Batch step: 0 wait for an empty tank, 1 dose A, 2 top up with B, 3 heat and mix, 4 drain) is not **3**) *or* (`Thermostat` (High-limit thermostat, closes at 180 °F) is on), the rung unlatches `Heat_On` (Heater wanted (with hysteresis)) (turns it off).

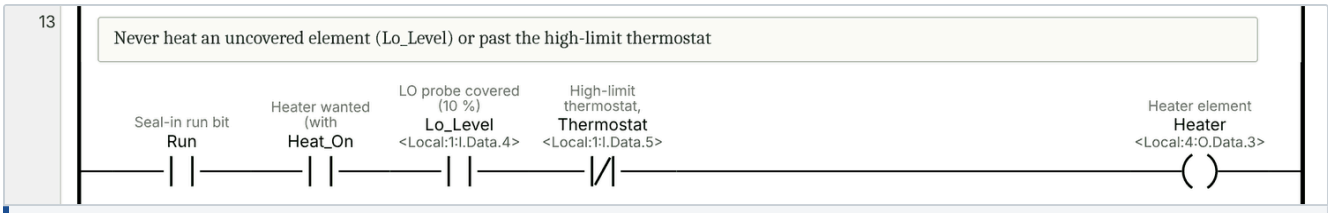
Left to right:

1. A branch of 3 parallel paths: power gets through if any one of them is true.
   - **Path 1**
     1. `GEQ(Tank_Temp,Target_T)` *Greater Than or Equal* asks: true when `Tank_Temp` (Tank temperature, °F) is at least `Target_T` (Temperature for the selected recipe, °F).
   - **Path 2**
     1. `NEQ(Step,3)` *Not Equal* asks: true when `Step` (Batch step: 0 wait for an empty tank, 1 dose A, 2 top up with B, 3 heat and mix, 4 drain) is not **3**.
   - **Path 3**
     1. `XIC(Thermostat)` *Examine If Closed* asks: true when `Thermostat` (High-limit thermostat, closes at 180 °F) is on.
2. `OTU(Heat_On)` *Output Unlatch* does: unlatches `Heat_On` (Heater wanted (with hysteresis)) (turns it off). It only ever clears the bit.

> **Think of it like this:** parallel paths are two roads to the same town. If either road is open you get there; the rung only

> cares that power reaches the far end, not which way it went.

RUNG 13




```
XIC(Run)XIC(Heat_On)XIC(Lo_Level)XIO(Thermostat)OTE(Heater);
```

PREDICT IT · 11 `Run` is on, `Heat_On` is on, `Lo_Level` is on and `Thermostat` is off. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

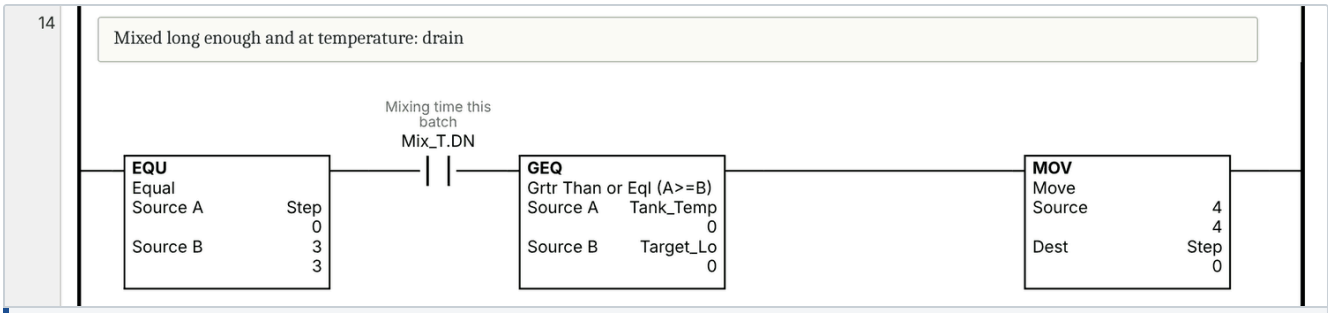
HOW THIS RUNG WORKS

In words. When `Run` (Seal-in run bit) is on, `Heat_On` (Heater wanted (with hysteresis)) is on, `Lo_Level` (LO probe covered (10 %)) is on and `Thermostat` (High-limit thermostat, closes at 180 °F) is off, the rung turns `Heater` (Heater element) on (and off again whenever the rung is false).

Left to right:

1. `XIC(Run)` *Examine If Closed* asks: true when `Run` (Seal-in run bit) is on.
2. `XIC(Heat_On)` *Examine If Closed* asks: true when `Heat_On` (Heater wanted (with hysteresis)) is on.
3. `XIC(Lo_Level)` *Examine If Closed* asks: true when `Lo_Level` (LO probe covered (10 %)) is on.
4. `XIO(Thermostat)` *Examine If Open* asks: true when `Thermostat` (High-limit thermostat, closes at 180 °F) is off.
5. `OTE(Heater)` *Output Energize* does: turns `Heater` (Heater element) on (and off again whenever the rung is false).

RUNG 14




```
EQU(Step,3)XIC(Mix_T.DN)GEQ(Tank_Temp,Target_Lo)MOV(4,Step);
```

PREDICT IT · 12 `Step` is 3, `Mix_T.DN` (the done bit of `Mix_T` : Mixing time this batch) is off, `Tank_Temp` is 500 and `Target_Lo` is 500. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Step` (Batch step: 0 wait for an empty tank, 1 dose A, 2 top up with B, 3 heat and mix, 4 drain) equals **3**, `Mix_T.DN` (the done bit of `Mix_T` : Mixing time this batch) is on and `Tank_Temp` (Tank temperature, °F) is at least `Target_Lo` (Heater comes back on below this, °F), the rung puts **4** into `Step` .

Left to right:

1. `EQU(Step,3)` *Equal* asks: true when `Step` (Batch step: 0 wait for an empty tank, 1 dose A, 2 top up with B, 3 heat and mix, 4 drain) equals **3**.
2. `XIC(Mix_T.DN)` *Examine If Closed* asks: true when `Mix_T.DN` (the done bit of `Mix_T` : Mixing time this batch) is on.
3. `GEQ(Tank_Temp,Target_Lo)` *Greater Than or Equal* asks: true when `Tank_Temp` (Tank temperature, °F) is at least `Target_Lo` (Heater comes back on below this, °F).
4. `MOV(4,Step)` *Move* does: puts **4** into `Step` .

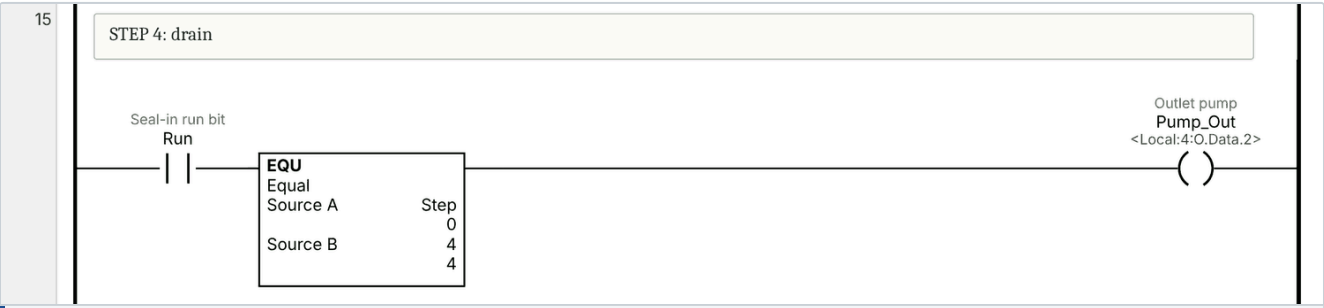
Why it is built this way:

A step of the sequence: from Step = 3 to 4.

Rung 12 unlatches `Heat_On` at the target, outside step 3, or at the high limit; rung 13 is the only rung that drives the heater, and it refuses an uncovered element. Rung 14 leaves step 3 when the mixing timer is done and the batch is at least at the bottom of its band.

Draining and counting

RUNG 15




```
XIC(Run)EQU(Step,4)OTE(Pump_Out);
```

PREDICT IT · 13 `Run` is on and `Step` is 4. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

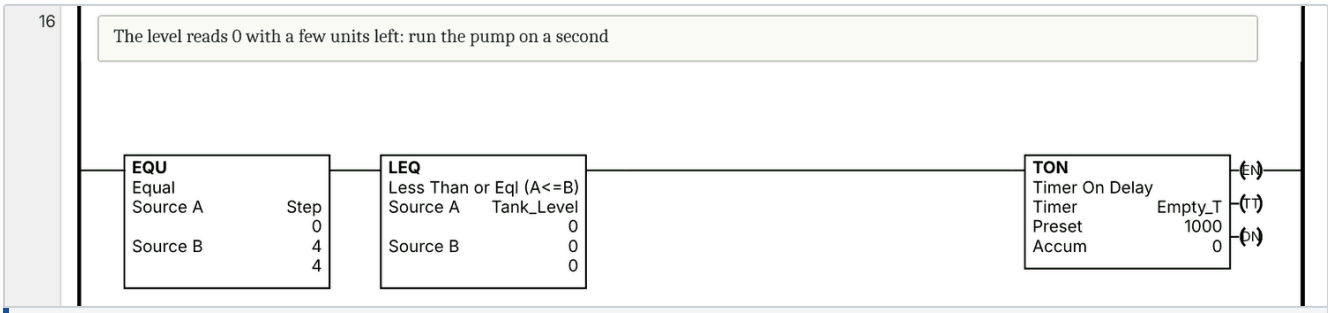
HOW THIS RUNG WORKS

In words. When `Run` (Seal-in run bit) is on and `Step` (Batch step: 0 wait for an empty tank, 1 dose A, 2 top up with B, 3 heat and mix, 4 drain) equals **4**, the rung turns `Pump_Out` (Outlet pump) on (and off again whenever the rung is false).

Left to right:

1. `XIC(Run)` *Examine If Closed* asks: true when `Run` (Seal-in run bit) is on.
2. `EQU(Step,4)` *Equal* asks: true when `Step` (Batch step: 0 wait for an empty tank, 1 dose A, 2 top up with B, 3 heat and mix, 4 drain) equals **4**.
3. `OTE(Pump_Out)` *Output Energize* does: turns `Pump_Out` (Outlet pump) on (and off again whenever the rung is false).

RUNG 16




```
EQU(Step,4)LEQ(Tank_Level,0)TON(Empty_T,1000,0);
```

PREDICT IT · 14 `Step` is 5 and `Tank_Level` is 0. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Step` (Batch step: 0 wait for an empty tank, 1 dose A, 2 top up with B, 3 heat and mix, 4 drain) equals **4** and `Tank_Level` (Tank level 0-100 %) is at most **0**, the rung runs the timer `Empty_T` (Run the outlet pump on a moment after the level reads 0) toward 1000 ms (1 s).

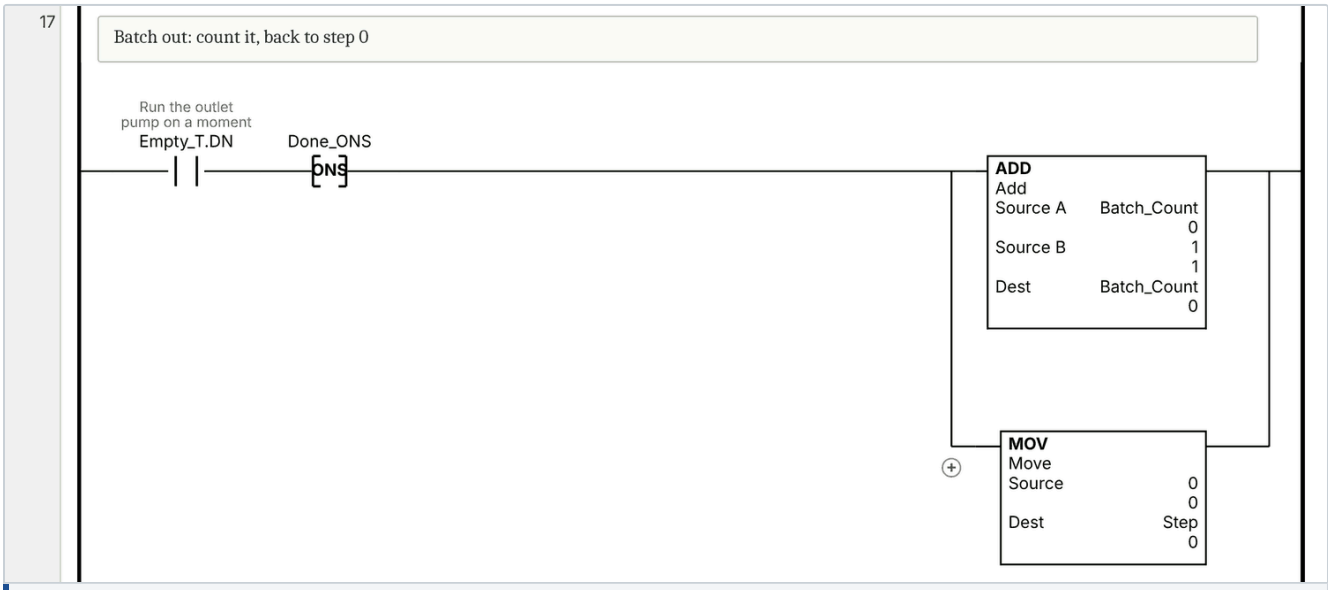
Left to right:

1. `EQU(Step,4)` *Equal* asks: true when `Step` (Batch step: 0 wait for an empty tank, 1 dose A, 2 top up with B, 3 heat and mix, 4 drain) equals **4**.
2. `LEQ(Tank_Level,0)` *Less Than or Equal* asks: true when `Tank_Level` (Tank level 0-100 %) is at most **0**.
3. `TON(Empty_T,1000,0)` *Timer On Delay* does: runs the timer `Empty_T` (Run the outlet pump on a moment after the level reads 0) toward 1000 ms (1 s).

Why it is built this way:

A timer: it times only while this rung is true.

RUNG 17




```
XIC(Empty_T.DN)ONS(Done_ONS)[ADD(Batch_Count,1,Batch_Count),MOV(0,Step)];
```

HOW THIS RUNG WORKS

In words. When `Empty_T.DN` (the done bit of `Empty_T` : Run the outlet pump on a moment after the level reads 0) is on and it is the first scan they are true (the one-shot, remembered in `Done_ONS`), the rung makes `Batch_Count` = `Batch_Count` + **1** and puts **0** into `Step` (Batch step: 0 wait for an empty tank, 1 dose A, 2 top up with B, 3 heat and mix, 4 drain).

Left to right:

1. `XIC(Empty_T.DN)` *Examine If Closed* asks: true when `Empty_T.DN` (the done bit of `Empty_T` : Run the outlet pump on a moment after the level reads 0) is on.
2. `ONS(Done_ONS)` *One Shot* asks: true when it is the first scan they are true (the one-shot, remembered in `Done_ONS`). It passes power for one scan only, the scan its input goes from false to true; its storage bit remembers last scan.
3. A branch of 2 parallel paths: each path is tested on its own and does its own work.

   #### Path 1

   1. `ADD(Batch_Count,1,Batch_Count)` *Add* does: makes `Batch_Count` = `Batch_Count` + **1**. Every scan the rung is true; behind a one-shot it adds once per event.

   #### Path 2

   1. `MOV(0,Step)` *Move* does: puts **0** into `Step` (Batch step: 0 wait for an empty tank, 1 dose A, 2 top up with B, 3 heat and mix, 4 drain).

Why it is built this way:

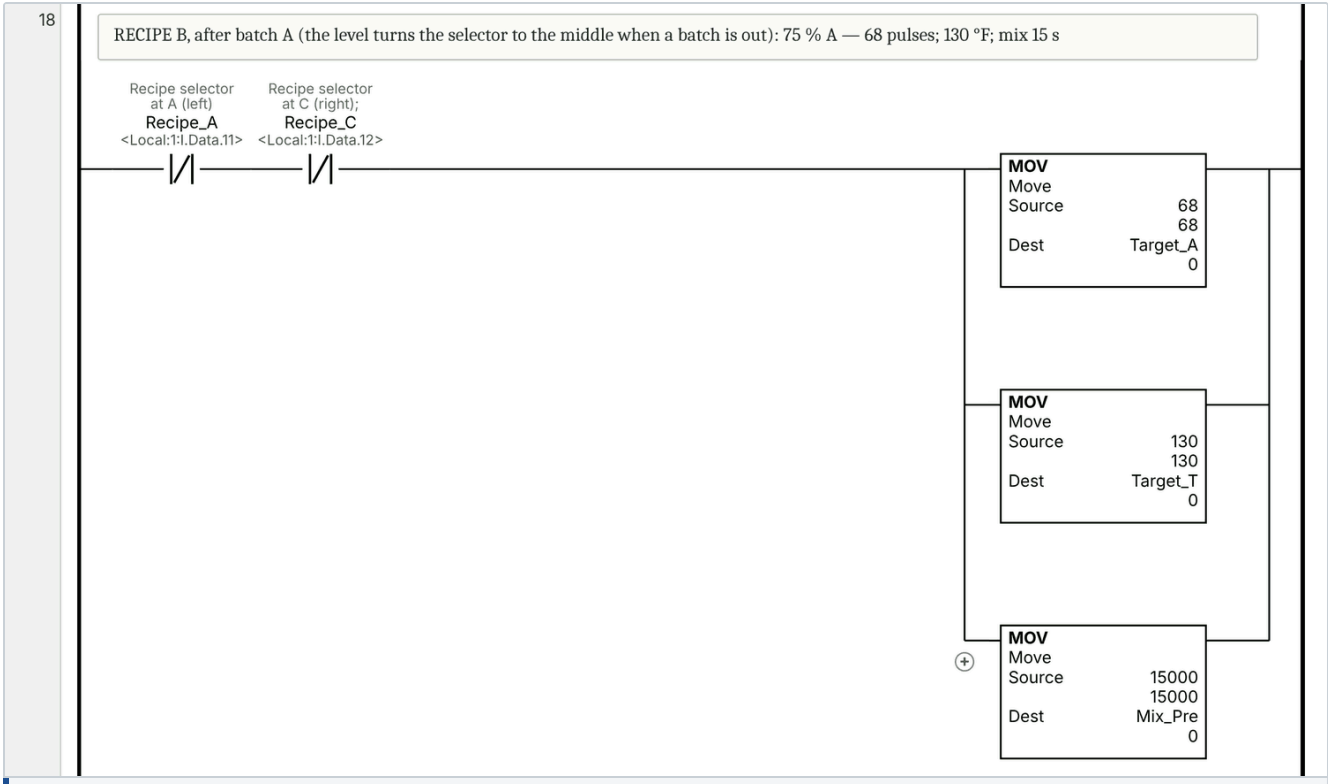
One-shot. Everything after the ONS happens once per event, not on every scan the condition holds; `Done_ONS` is only its memory of the last scan.

> **Think of it like this:** a doorbell that rings once however long you lean on the button. The ONS looks at what the button was doing last scan: only the change from up to down rings it.

> **Watch for:** Each ONS needs its own storage bit. Two ONS sharing one bit make one of them miss its edges.

Recipes B and C

RUNG 18




```
XIO(Recipe_A)XIO(Recipe_C)
[MOV(68,Target_A),MOV(130,Target_T),MOV(15000,Mix_Pre)];
```

HOW THIS RUNG WORKS

In words. When `Recipe_A` (Recipe selector at A (left)) is off and `Recipe_C` (Recipe selector at C (right); neither = B) is off, the rung puts **68** into `Target_A` (Pulses of A for the selected recipe), puts **130** into `Target_T` (Temperature for the selected recipe, °F) and puts **15000** into `Mix_Pre` (Mixing time for the selected recipe, ms).

Left to right:

1. `XIO(Recipe_A)` *Examine If Open* asks: true when `Recipe_A` (Recipe selector at A (left)) is off.
2. `XIO(Recipe_C)` *Examine If Open* asks: true when `Recipe_C` (Recipe selector at C (right); neither = B) is off.
3. A branch of 3 parallel paths: each path is tested on its own and does its own work.

   Path 1

   1. `MOV(68,Target_A)` *Move* does: puts **68** into `Target_A` (Pulses of A for the selected recipe).

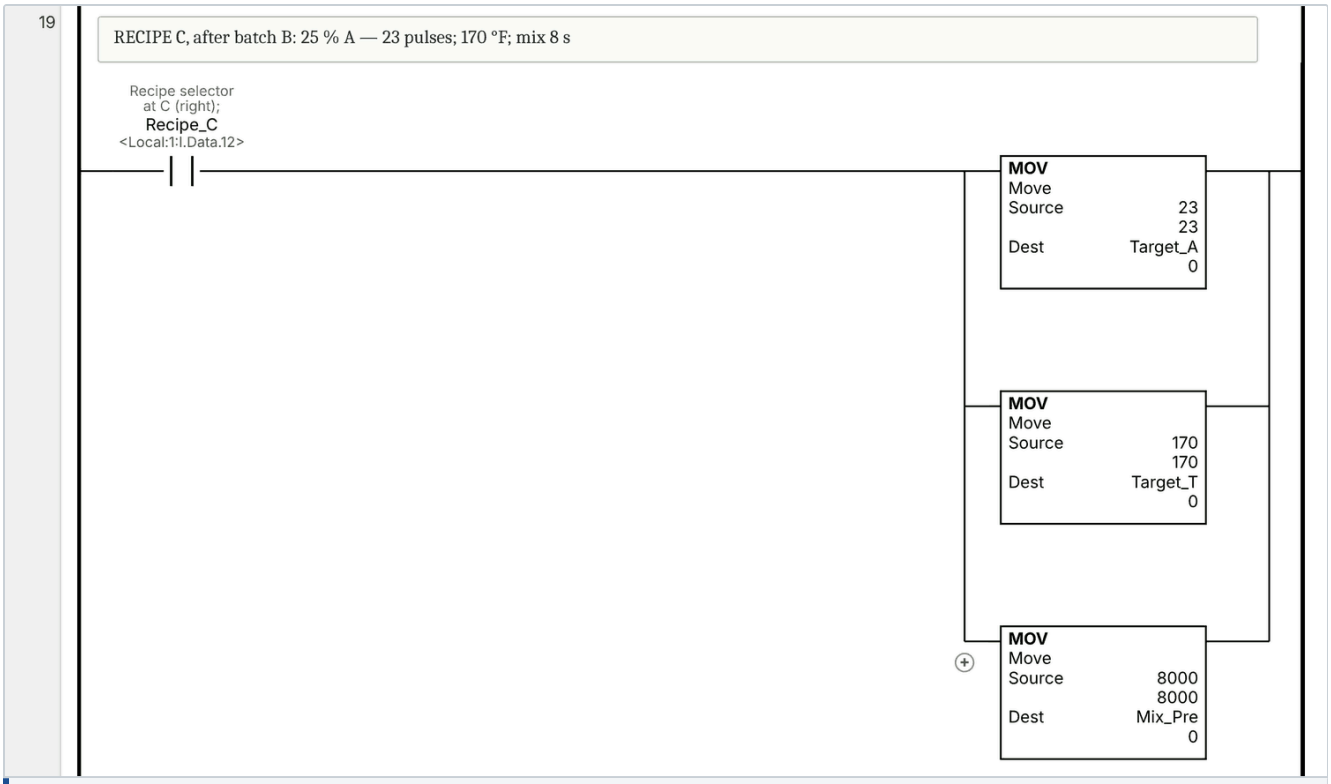
   Path 2

   1. `MOV(130,Target_T)` *Move* does: puts **130** into `Target_T` (Temperature for the selected recipe, °F).

   Path 3

   1. `MOV(15000,Mix_Pre)` *Move* does: puts **15000** into `Mix_Pre` (Mixing time for the selected recipe, ms).

RUNG 19




```
XIC(Recipe_C)[MOV(23,Target_A),MOV(170,Target_T),MOV(8000,Mix_Pre)];
```

HOW THIS RUNG WORKS

In words. When `Recipe_C` (Recipe selector at C (right); neither = B) is on, the rung puts **23** into `Target_A` (Pulses of A for the selected recipe), puts **170** into `Target_T` (Temperature for the selected recipe, °F) and puts **8000** into `Mix_Pre` (Mixing time for the selected recipe, ms).

Left to right:

1. `XIC(Recipe_C)` *Examine If Closed* asks: true when `Recipe_C` (Recipe selector at C (right); neither = B) is on.
2. A branch of 3 parallel paths: each path is tested on its own and does its own work.

   Path 1

   1. `MOV(23,Target_A)` *Move* does: puts **23** into `Target_A` (Pulses of A for the selected recipe).

   Path 2

   1. `MOV(170,Target_T)` *Move* does: puts **170** into `Target_T` (Temperature for the selected recipe, °F).

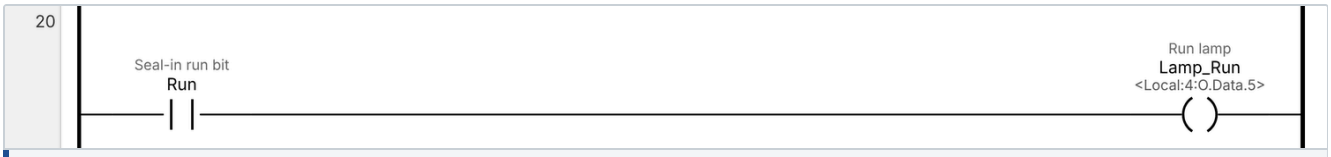
   Path 3

   1. `MOV(8000,Mix_Pre)` *Move* does: puts **8000** into `Mix_Pre` (Mixing time for the selected recipe, ms).

Written after batch A has drained, these go true as soon as the level turns the selector to B, and to C after the next batch.

Lamps

RUNG 20




```
XIC(Run)OTE(Lamp_Run);
```

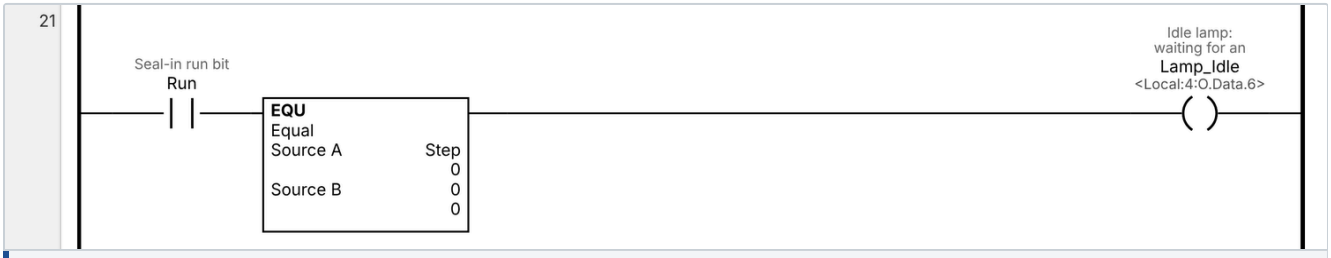
HOW THIS RUNG WORKS

In words. When `Run` (Seal-in run bit) is on, the rung turns `Lamp_Run` (Run lamp) on (and off again whenever the rung is false).

Left to right:

1. `XIC(Run)` *Examine If Closed* asks: true when `Run` (Seal-in run bit) is on.
2. `OTE(Lamp_Run)` *Output Energize* does: turns `Lamp_Run` (Run lamp) on (and off again whenever the rung is false).

RUNG 21




```
XIC(Run)EQU(Step,0)OTE(Lamp_Idle);
```

PREDICT IT · 15 `Run` is on and `Step` is 0. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

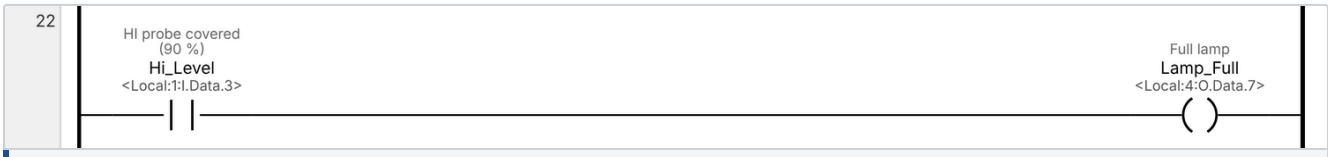
HOW THIS RUNG WORKS

In words. When `Run` (Seal-in run bit) is on and `Step` (Batch step: 0 wait for an empty tank, 1 dose A, 2 top up with B, 3 heat and mix, 4 drain) equals **0**, the rung turns `Lamp_Idle` (Idle lamp: waiting for an empty tank or a Start) on (and off again whenever the rung is false).

Left to right:

1. `XIC(Run)` *Examine If Closed* asks: true when `Run` (Seal-in run bit) is on.
2. `EQU(Step,0)` *Equal* asks: true when `Step` (Batch step: 0 wait for an empty tank, 1 dose A, 2 top up with B, 3 heat and mix, 4 drain) equals **0**.
3. `OTE(Lamp_Idle)` *Output Energize* does: turns `Lamp_Idle` (Idle lamp: waiting for an empty tank or a Start) on (and off again whenever the rung is false).

RUNG 22




```
XIC(Hi_Level)OTE(Lamp_Full);
```

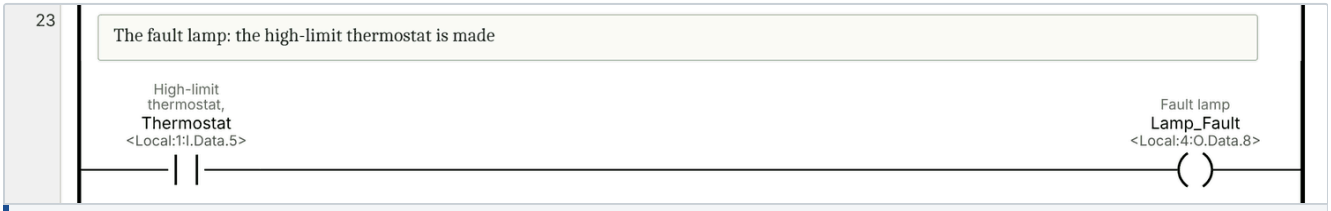
HOW THIS RUNG WORKS

In words. When `Hi_Level` (HI probe covered (90 %)) is on, the rung turns `Lamp_Full` (Full lamp) on (and off again whenever the rung is false).

Left to right:

1. `XIC(Hi_Level)` *Examine If Closed* asks: true when `Hi_Level` (HI probe covered (90 %)) is on.
2. `OTE(Lamp_Full)` *Output Energize* does: turns `Lamp_Full` (Full lamp) on (and off again whenever the rung is false).

RUNG 23




```
XIC(Thermostat)OTE(Lamp_Fault);
```

HOW THIS RUNG WORKS

In words. When `Thermostat` (High-limit thermostat, closes at 180 °F) is on, the rung turns `Lamp_Fault` (Fault lamp) on (and off again whenever the rung is false).

Left to right:

1. `XIC(Thermostat)` *Examine If Closed* asks: true when `Thermostat` (High-limit thermostat, closes at 180 °F) is on.
2. `OTE(Lamp_Fault)` *Output Energize* does: turns `Lamp_Fault` (Fault lamp) on (and off again whenever the rung is false).

WATCH FOR: COUNTING THE WRONG THING

Count `Flow_A` with an XIC and an ADD instead of a CTU and the count races: the pulse is high for a whole scan or more, and the ADD adds on every one of them. A CTU (or an ONS in front of the ADD) counts edges. Then the opposite trap: count `Flow_B` by mistake and pump A never stops — exactly what a dead flowmeter does (Tools → Inject Machine Fault → flowmeter A dead): the tank fills to the top and overflows. A dose that ends on a count needs a second way to end — a HI probe, a time limit — or a dead meter floods the floor.

Run it

Load your program, Run mode, Start. Watch the display: FILLING with the A share climbing, then B pulling it down to the recipe; HEAT+MIX with the temperature climbing at a little over 2 °F a second; DRAINING; EMPTY; and the selector turns. The Simulator Log grades each batch as it drains. The reference run: 3 of 3 bottles in the right bin, nothing lost, in 179 s against a par of 240 s: 1100 points and three stars.

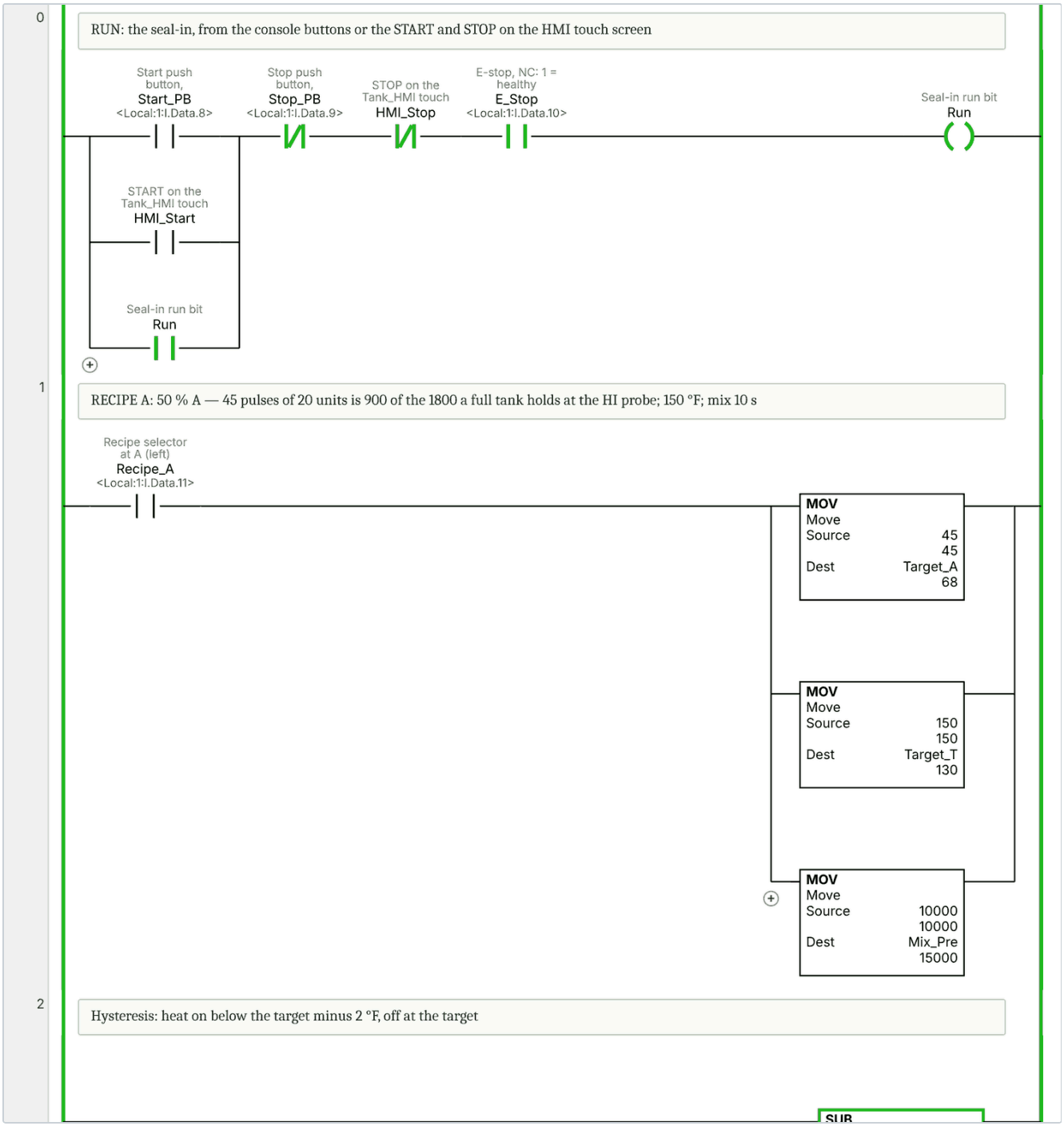


Figure 158. Level 7 online, the top of the routine.

Figure 159. The mix tank in 3D while it heats and mixes: the product has gone from clear to cloudy, the heating band glows, the impellers turn behind the displays; Supply_A and Supply_B stand behind on the left.

TRY IT

The reference program wastes most of its time heating after the tank is full. Start the heater as soon as Lo_Level is covered — in step 1 — and watch two things happen: the batch is ready much sooner, and every slug of cold ingredient pulls the temperature down (the tank mixes it in), so the heater works against the fill. Beat 150 s. Then take `XIC(Lo_Level)` out of the heater rung and start a batch: the element comes on before the liquid covers it, and the batch is scorched. Last, give the heater no band at all — `LES(Tank_Temp,Target_T)OTE(Heater)` — and watch it chatter on and off at the setpoint in Monitor Tags.

TRY IT — RUN A TOTE DRY

Set Supply_B's *Starts at* to 15 % (1,500 units) and run the reference program. Batches 1 and 2 take about 1,350 units of B between them, so in batch 3 the tote empties while pump B is topping up. Pump B runs dry, the HI probe never makes, and the program sits in step 2 forever with the Run lamp on. A real line would be exactly as stuck. Make two fixes. First, a watchdog: `XIC(Run)EQU(Step,2)TON(B_Fill_T,20000,0)`, and when it is done, light Lamp_Fault and stop. Second, use the tote's own **LowLevel** switch, a digital input like any other (alias it as `Tote_B_Low`): do not start a batch while it is on. The watchdog catches the failure; the low switch stops you walking into it. In Run mode, clicking an empty tote swaps in a full one, the way a forklift would.

A mix tank on the bottle line

The Mix Tank pipes to a filler the way the Reservoir does: put one in place of Supply_Tank and pick it in the Filler's *Fed from*, which lists every tote, tank, reservoir and mix tank in the scene. A Tank can also be fed from the Mix Tank (its *Inlet fed from*); it then fills only while the mix tank's outlet pump runs and its own inlet valve is open. The filler then only gets product while the tank's outlet pump runs, so the program has to make a batch first and then hold Pump_Out on while the filler draws — a batch-to-line handoff that a supply tank never asked for.

Predict it: the answers

1. The rung is false at `XIO(Stop_PB)`: `Stop_PB` is on, so `Run` is **off** (an OTE writes 0 when its rung is false).
2. Every condition on the way is met, so `A_Count` is reset, `Mix_T` is reset and `Step` becomes **1**.
3. The rung is false at `XIC(Run)`: `Run` is off, so `Pump_A` is **off** (an OTE writes 0 when its rung is false).
4. The rung is false at the GEQ: `A_Count.ACC` (the accumulated value of `A_Count`: Flowmeter A pulses this batch) is below `Target_A`, so the MOV does nothing: `Step` keeps its value.
5. Every condition on the way is met, so `Pump_B` is **on**.
6. The rung is false at the EQU: `Step` is not **2**, so the MOV does nothing: `Step` keeps its value.
7. Every condition on the way is met, so `Mixer` is **on**.
8. The rung is false at the EQU: `Step` is not **3**, so `Mix_T` is reset to zero (a TON resets when its rung goes false).
9. Every condition on the way is met, so `Heat_On` latches **on**.
10. Every condition on the way is met, so `Heat_On` is unlatched, **off**.
11. Every condition on the way is met, so `Heater` is **on**.
12. The rung is false at `XIC(Mix_T.DN)`: `Mix_T.DN` (the done bit of `Mix_T`: Mixing time this batch) is off, so the MOV does nothing: `Step` keeps its value.
13. Every condition on the way is met, so `Pump_Out` is **on**.
14. The rung is false at the EQU: `Step` is not **4**, so `Empty_T` is reset to zero (a TON resets when its rung goes false).

15. Every condition on the way is met, so `Lamp_Idle` is **on**.

40 Level 8 — Vision check

IN THE APP

File → New Project… → A graded level → **8**. Guided mode opens on this chapter: the tags, every rung below under its fold, a Verify check, and six results: the camera inspects a bottle, a result is acknowledged, a defective bottle is rejected, the level completes, every bottle in the right bin, under par.

This level is the one most plants would recognise first: a camera on the line. Finished bottles arrive from an upstream line, filled, capped and labeled, but about a third of them have one defect: no cap, no label, or a short fill. An In-Sight smart camera on a stand over the belt looks at each bottle as it passes. Your program triggers it, takes each result, carries it down the belt to the reject pusher, and pushes every defective bottle into `Reject_Bin`. The good ones run on into `Good_Bin` at the end of the belt. Twenty bottles, par 45 s.

The camera is not a made-up part. It is a **camera module** in I/O Configuration, `Cam1`, built from Cognex's own device file for the In-Sight 5700 series, at 192.168.1.18. Every tag is named the way the real camera names it, so the rungs in this chapter are the rungs you would write for a real 5705 on a CompactLogix. The camera's job here is **Bottle inspection**: three tools, one for the cap, one for the label, one for the fill level.

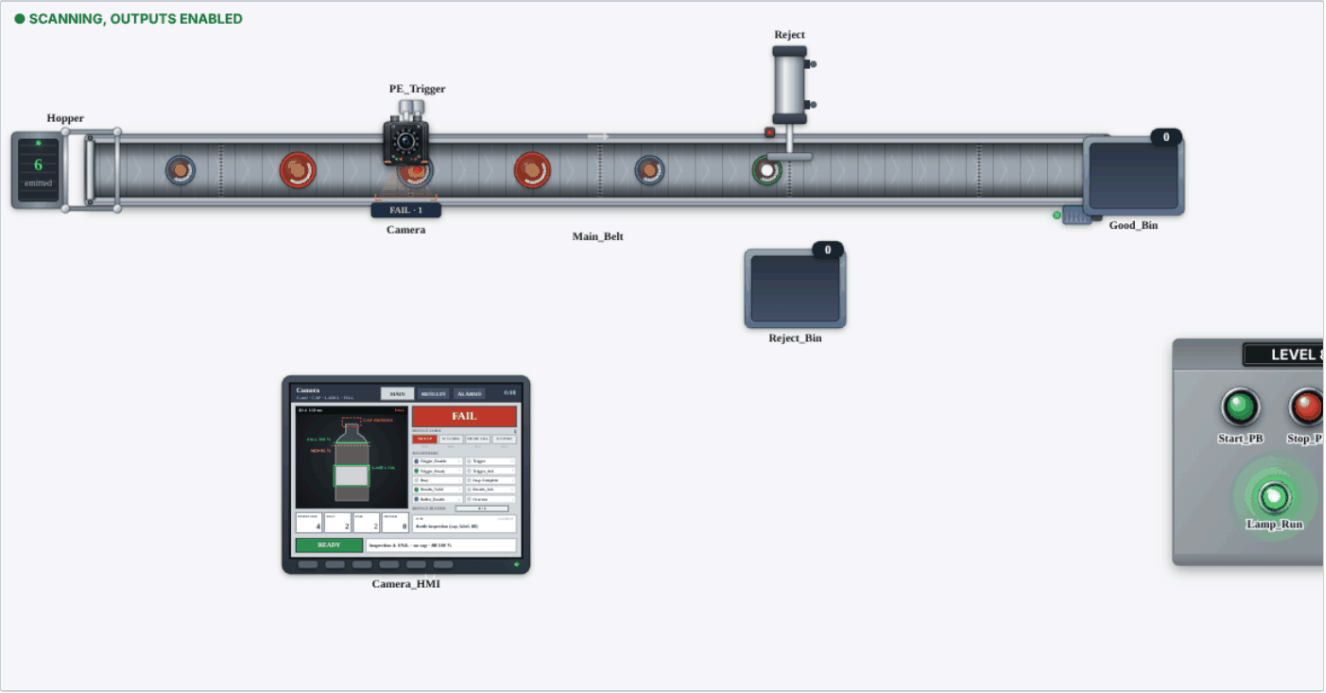


Figure 160. Level 8 in the top view: the hopper, PE_Trigger and the camera, the reject pusher over Reject_Bin, Good_Bin at the end of the belt, Camera_HMI on the floor and the console. The camera's label shows its last result, FAIL with code 1 (no cap).

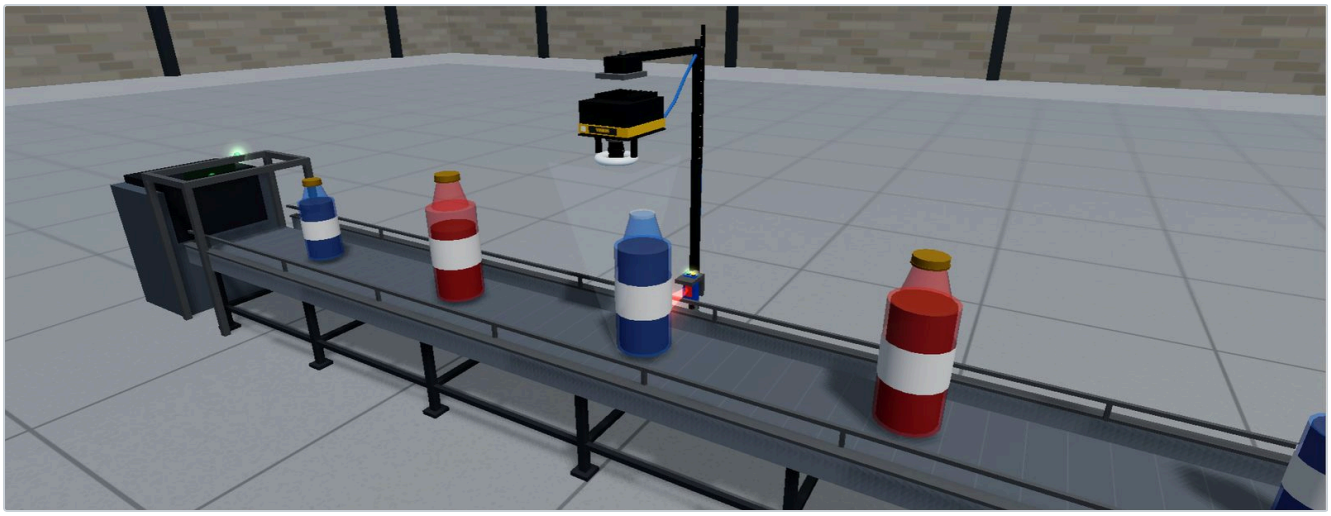


Figure 161. In 3D: the camera on its stand over the belt, with a ring light round the lens, looking down on a bottle with no cap. PE_Trigger is the eye on the near side, just past the camera.

The machine

- **Hopper** — the Emitter of every level, set to *finished* bottles with 35 % defective. `Infeed_Release` drops one bottle on each rising edge; `Infeed_Ready` is 1 while the exit is clear.
- **Main_Belt** — 15 m at 80 px/s. It never stops for the camera: the camera images a moving bottle, which is what a 150 ms inspection on a real line is for.
- **PE_Trigger** — a narrow-beam eye just past the camera's centre, set so that when it first sees a bottle, that bottle is under the lens.
- **Camera** — a Vision Sensor with *Inspection job* Bottle inspection, *Camera module* Cam1, *Inspection time* 150 ms, *Fill passes at* 95 % of the label, *Results buffer depth* 4.
- **Reject** — a pusher 340 px downstream of the camera, with its own eye (`PE_Reject`) and cylinder (`Reject`); **Reject_Bin** under it is graded *defective bottles*. **Good_Bin** catches whatever reaches the end of the belt, graded *good finished bottles*.
- **Camera_HMI** — a Process HMI showing the camera (chapter 14).
- The console: `Start_PB`, `Stop_PB`, `E_Stop`, `Lamp_Run`, `Lamp_Fault`.

The camera's tags

The module's tags are `Cam1:O.…` (the PLC writes them) and `Cam1:I.…` (the camera writes them). The ones this level uses:

Tag	Meaning
`Cam1:O.Trigger_Enable`	Allows triggers at all. Off, the camera ignores Trigger and Trigger_Ready stays off.
`Cam1:O.Trigger`	Rising edge: acquire an image now.

Tag	Meaning
`Cam1:O.Buffer_Results_Enable`	Queue results until each one is acknowledged (below).
`Cam1:O.Inspection_Results_Ack`	The PLC has read the result.
`Cam1:I.Trigger_Ready`	The camera can take a trigger: on line, enabled, not acquiring or inspecting.
`Cam1:I.Trigger_Ack`	The trigger was taken; stays on until Trigger goes off.
`Cam1:I.Missed_Acq`	A trigger arrived while the camera was busy: that bottle has no image.
`Cam1:I.System_Busy`	Acquiring or inspecting.
`Cam1:I.Inspection_Completed`	Changes state at the end of every inspection: it toggles, so look for a change, not for a 1.
`Cam1:I.Results_Valid`	A result is presented in the tags below.
`Cam1:I.Results_Buffer_Overrun`	A result was thrown away because the buffer was full.
`Cam1:I.Job_Pass`	The presented result passed.
`Cam1:I.Inspection_ID`	The number of the inspection the presented result belongs to.
`Cam1:I.Inspection_Result_Code`	Bit 0 no cap, bit 1 no label, bit 2 short fill, bit 3 no part in view; 0 is a good bottle.
`Cam1:I.Inspection_Results[…]`	The result data: bytes 0-3 the part's serial number, 4-5 the result code, 6 the size (1 S, 2 M, 3 L), 7 metal, 8 pass, 9 the fill in % of the label.

The two handshakes

The trigger. Wait for `Trigger_Ready`, raise `Trigger`, wait for `Trigger_Ack`, drop `Trigger`; the camera drops `Trigger_Ack`. `System_Busy` covers the 30 ms exposure and the 150 ms inspection, and `Trigger_Ready` comes back when both are over. Trigger while it is busy and nothing is acquired: `Missed_Acq` comes on and that bottle goes by with no image.

The result. When an inspection ends, the camera presents its result and raises `Results_Valid`. Read the result, then raise `Inspection_Results_Ack`. The camera drops `Results_Valid`, and then the PLC drops the ack. That makes four edges, always in that order: Valid up, Ack up, Valid down, Ack down.

Buffered results. With `Buffer_Results_Enable` on, a result that finishes while the last one is still unacknowledged is not thrown away. It waits in a queue, up to the camera's buffer depth (4 here), and the camera presents it once the PLC has finished the handshake for the one before. With buffering off, each new result simply overwrites the last, whether the PLC has read it or not. Buffering is what lets the program look away for a scan or two without losing a bottle. It has a limit, though: a result that finds the queue full is lost, and `Results_Buffer_Overrun` comes on. Lose one result and every bottle after it gets the wrong verdict, because the FIFO is one short.

How the level is scored

What happens	Points
A defective bottle in Reject_Bin	+100
A good bottle in Good_Bin	+100
A defective bottle in Good_Bin (shipped)	−100
A good bottle in Reject_Bin	−50
A bottle that passes the camera without being inspected	−25
Complete under par	+200

How to pass it, step by step

1. **Set the camera up once.** `Trigger_Enable` and `Buffer_Results_Enable` on whenever the machine is healthy. Do not tie them to Run: a Stop must not throw away the results of the bottles still on the belt.
2. **Space the bottles.** A timer that restarts itself releases one every 0.9 s. At 80 px/s that puts four or five bottles between the camera and the pusher, so remembering one bottle is not enough: the results have to be queued.
3. **Trigger once per bottle.** An ONS on PE_Trigger latches a request; the trigger goes out only when the camera is ready; Trigger_Ack clears both.
4. **Take each result into a FIFO.** Results_Valid and not yet acknowledged: FFL the result code into `Res_Q` and latch the ack in the same rung. Results_Valid gone: unlatch the ack.
5. **Unload at the pusher.** When a bottle reaches PE_Reject, FFU the oldest code. Bottles on a belt stay in order, so the oldest result is this bottle's. Non-zero arms the pusher.
6. **Light the fault lamp** on anything that breaks the one-to-one match between bottles and results: an overrun, a missed acquisition, the camera off line.

Tags this lesson needs

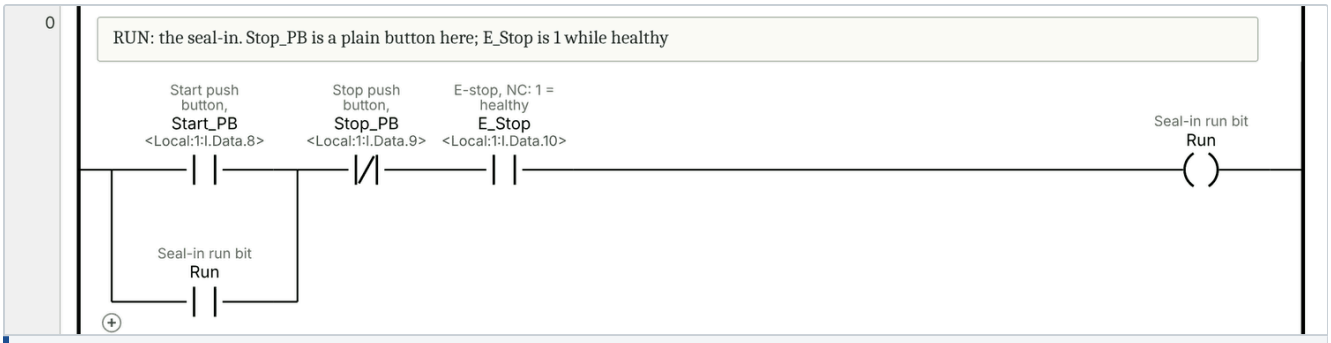
The level's alias tags are already made for you (chapter 32's table). Create these plain tags before you type the rungs; Verify reports any you miss.

Tag	Type	How to make it	Description
`Run`	BOOL	Edit Tags: name, type	Seal-in run bit
`Rel_T`	TIMER	Edit Tags: name, type	Spaces the releases: one bottle every 0.9 s
`Trig_Req`	BOOL	Edit Tags: name, type	A bottle has reached PE_Trigger and wants an inspection
`Trig_ONS`	BOOL	Edit Tags: name, type	
`Res_Q`	DINT[8]	Edit Tags: name, type, Dim 8	The result codes of the inspected bottles between the camera and the reject pusher, oldest first

Tag	Type	How to make it	Description
`Res_Q_Ctl`	CONTROL	Edit Tags: name, type	FIFO control for Res_Q
`Cur`	DINT	Edit Tags: name, type	The result code of the bottle at the reject pusher
`Fire`	BOOL	Edit Tags: name, type	Armed: the bottle at the pusher is to be rejected
`Rej_ONS`	BOOL	Edit Tags: name, type	
`Rej_ONS2`	BOOL	Edit Tags: name, type	
`Cnt_ONS`	BOOL	Edit Tags: name, type	

Run, setup and release

RUNG 0




```
[XIC(Start_PB),XIC(Run)]XIO(Stop_PB)XIC(E_Stop)OTE(Run);
```

PREDICT IT · 1 `Start_PB` is on, `Stop_PB` is on and `E_Stop` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When either (`Start_PB` (Start push button, momentary) is on) *or* (`Run` (Seal-in run bit) is on), `Stop_PB` (Stop push button, momentary) is off and `E_Stop` (E-stop, NC: 1 = healthy) is on, the rung turns `Run` on (and off again whenever the rung is false).

Left to right:

1. A branch of 2 parallel paths: power gets through if any one of them is true.
   - **Path 1**
     1. `XIC(Start_PB)` *Examine If Closed* asks: true when `Start_PB` (Start push button, momentary) is on.
   - **Path 2**
     1. `XIC(Run)` *Examine If Closed* asks: true when `Run` (Seal-in run bit) is on.
2. `XIO(Stop_PB)` *Examine If Open* asks: true when `Stop_PB` (Stop push button, momentary) is off.
3. `XIC(E_Stop)` *Examine If Closed* asks: true when `E_Stop` (E-stop, NC: 1 = healthy) is on.
4. `OTE(Run)` *Output Energize* does: turns `Run` on (and off again whenever the rung is false). Every scan it writes the rung state: true on, false off.

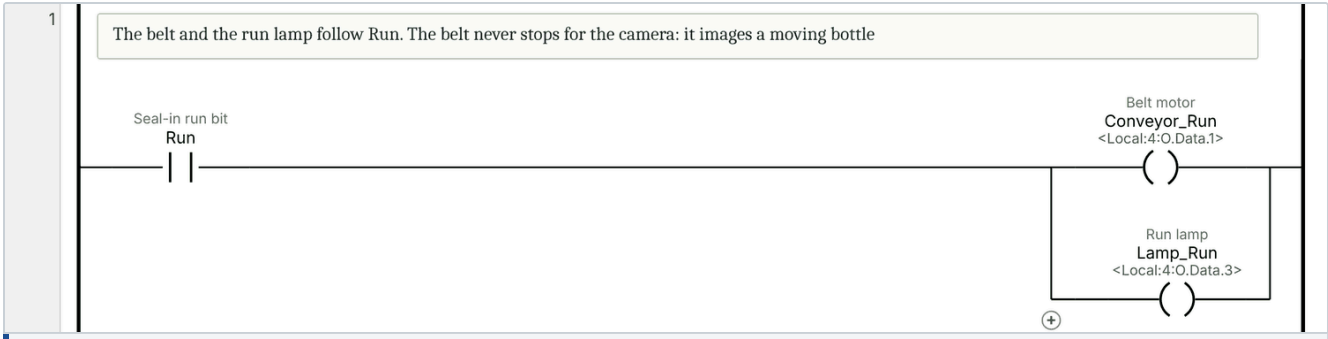
Why it is built this way:

Seal-in. The start condition turns `Run` on, and `Run` 's own contact in the parallel path keeps the rung true after the start goes away. Anything in series outside the branch (a stop, a fault) breaks it, and then it stays off until the start is made again.

> **Think of it like this:** a doorbell button and a hand. The start button is the doorbell, pressed for a moment. The seal-in contact is a hand that grabs the circuit as soon as the output comes on and keeps holding after you let go. The stop button, in series, is the only thing that can knock that hand away.

> **Watch for:** The stop has to be in series outside the branch. Put it inside the start path only and the seal-in path bypasses it: the stop no longer stops.

RUNG 1




```
XIC(Run)[OTE(Conveyor_Run),OTE(Lamp_Run)];
```

HOW THIS RUNG WORKS

In words. When `Run` (Seal-in run bit) is on, the rung turns `Conveyor_Run` (Belt motor) on (and off again whenever the rung is false) and turns `Lamp_Run` (Run lamp) on (and off again whenever the rung is false).

Left to right:

1. `XIC(Run)` *Examine If Closed* asks: true when `Run` (Seal-in run bit) is on.
2. A branch of 2 parallel paths: each path is tested on its own and does its own work.

   Path 1

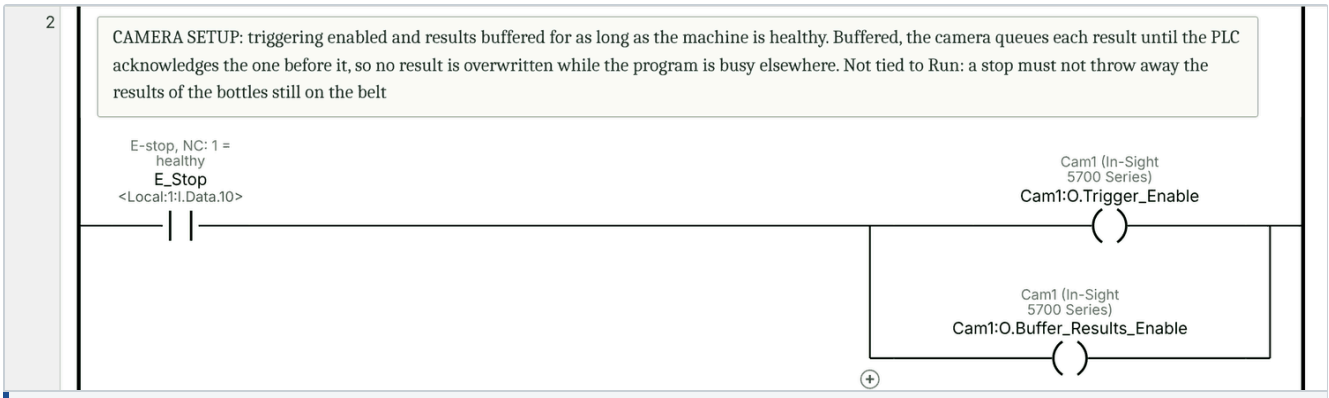
   1. `OTE(Conveyor_Run)` *Output Energize* does: turns `Conveyor_Run` (Belt motor) on (and off again whenever the rung is false).

   Path 2

   1. `OTE(Lamp_Run)` *Output Energize* does: turns `Lamp_Run` (Run lamp) on (and off again whenever the rung is false).

> **Think of it like this:** a rung is a pipe with valves in it, read left to right. Every contact is a valve: XIC is open while its bit is 1, XIO while its bit is 0. Water reaches the output at the right only if every valve in a row is open. The output does not remember anything: on every scan it is simply wet or dry.

RUNG 2




```
XIC(E_Stop)[OTE(Cam1:O.Trigger_Enable),OTE(Cam1:O.Buffer_Results_Enable)];
```

HOW THIS RUNG WORKS

In words. When `E_Stop` (E-stop, NC: 1 = healthy) is on, the rung turns `Cam1:O.Trigger_Enable` on (and off again whenever the rung is false) and turns `Cam1:O.Buffer_Results_Enable` on (and off again whenever the rung is false).

Left to right:

1. `XIC(E_Stop)` *Examine If Closed* asks: true when `E_Stop` (E-stop, NC: 1 = healthy) is on.
2. A branch of 2 parallel paths: each path is tested on its own and does its own work.

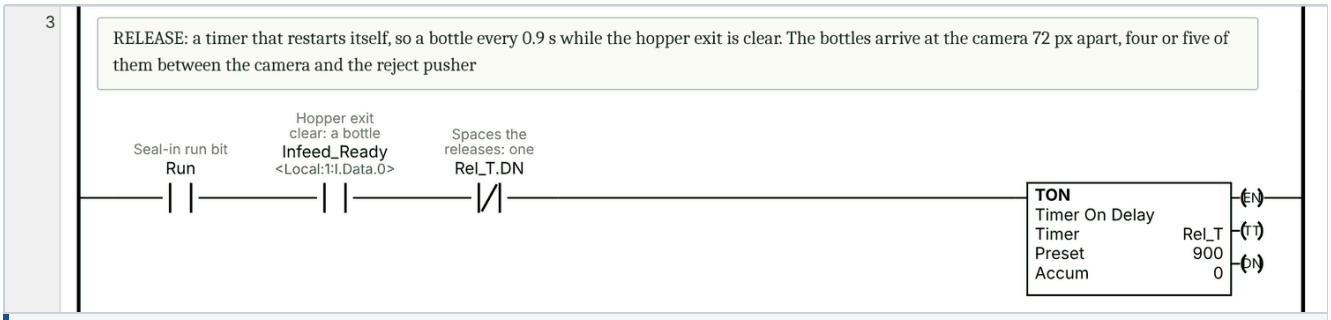
   Path 1

   1. `OTE(Cam1:O.Trigger_Enable)` *Output Energize* does: turns `Cam1:O.Trigger_Enable` on (and off again whenever the rung is false).

   Path 2

   1. `OTE(Cam1:O.Buffer_Results_Enable)` *Output Energize* does: turns `Cam1:O.Buffer_Results_Enable` on (and off again whenever the rung is false).

RUNG 3




```
XIC(Run)XIC(Infeed_Ready)XIO(Rel_T.DN)TON(Rel_T,900,0);
```

PREDICT IT · 2 `Run` is on, `Infeed_Ready` is on and `Rel_T.DN` (the done bit of `Rel_T` : Spaces the releases: one bottle every 0.9 s) is off. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Run` (Seal-in run bit) is on, `Infeed_Ready` (Hopper exit clear: a bottle can be released) is on and `Rel_T.DN` (the done bit of `Rel_T` : Spaces the releases: one bottle every 0.9 s) is off, the rung runs the timer `Rel_T` toward 900 ms (0.9 s).

Left to right:

1. `XIC(Run)` *Examine If Closed* asks: true when `Run` (Seal-in run bit) is on.
2. `XIC(Infeed_Ready)` *Examine If Closed* asks: true when `Infeed_Ready` (Hopper exit clear: a bottle can be released) is on.
3. `XIO(Rel_T.DN)` *Examine If Open* asks: true when `Rel_T.DN` (the done bit of `Rel_T` : Spaces the releases: one bottle every 0.9 s) is off.
4. `TON(Rel_T,900,0)` *Timer On Delay* does: runs the timer `Rel_T` toward 900 ms (0.9 s). The accumulator counts while the rung is true; .DN comes on at the preset; the moment the rung goes false the timer resets.

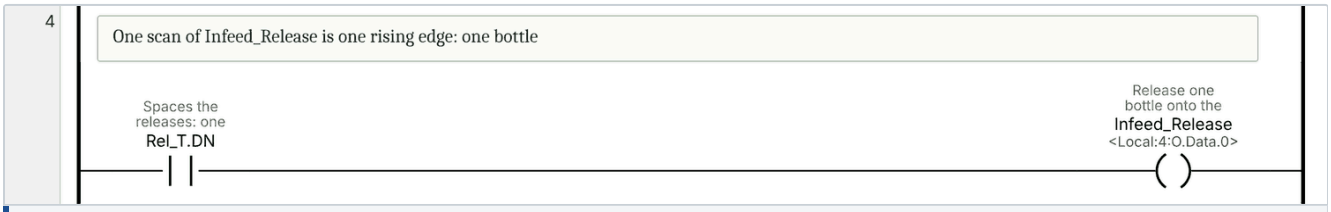
Why it is built this way:

A timer. `Rel_T` only times while this rung is true; other rungs use its `.DN` (done) or `.TT` (timing) bits. Being a TON, it starts again from zero every time the rung goes false.

> **Think of it like this:** a kitchen timer that only runs while you hold your finger on it. Take your finger off and it snaps back to zero. Hold it long enough and the done light comes on, and stays on as long as your finger is still there.

> **Watch for:** If anything in series flickers false for one scan, a TON starts over from zero. Use RTO when the time must survive an interruption.

RUNG 4




```
XIC(Rel_T.DN)OTE(Infeed_Release);
```

HOW THIS RUNG WORKS

In words. When `Rel_T.DN` (the done bit of `Rel_T` : Spaces the releases: one bottle every 0.9 s) is on, the rung turns `Infeed_Release` (Release one bottle onto the belt (rising edge)) on (and off again whenever the rung is false).

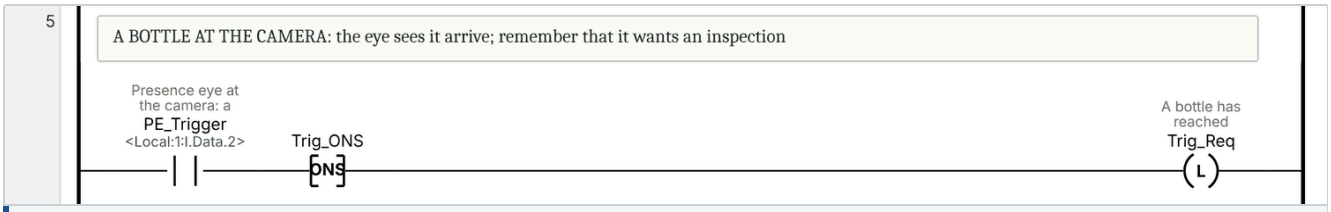
Left to right:

1. `XIC(Rel_T.DN)` *Examine If Closed* asks: true when `Rel_T.DN` (the done bit of `Rel_T` : Spaces the releases: one bottle every 0.9 s) is on.
2. `OTE(Infeed_Release)` *Output Energize* does: turns `Infeed_Release` (Release one bottle onto the belt (rising edge)) on (and off again whenever the rung is false).

Rung 2 has no Run in it on purpose. Rung 3 is the self-restarting timer of chapter 7: `XIO(Rel_T.DN)` resets it the scan after it is done, so `Rel_T.DN` is on for exactly one scan, which rung 4 turns into one rising edge on `Infeed_Release`.

The trigger handshake

RUNG 5




```
XIC(PE_Trigger)ONS(Trig_ONS)OTL(Trig_Req);
```

HOW THIS RUNG WORKS

In words. When `PE_Trigger` (Presence eye at the camera: a bottle is in the field of view) is on and it is the first scan they are true (the one-shot, remembered in `Trig_ONS`), the rung latches `Trig_Req` (A bottle has reached PE_Trigger and wants an inspection) on.

Left to right:

1. `XIC(PE_Trigger)` *Examine If Closed* asks: true when `PE_Trigger` (Presence eye at the camera: a bottle is in the field of view) is on.
2. `ONS(Trig_ONS)` *One Shot* asks: true when it is the first scan they are true (the one-shot, remembered in `Trig_ONS`). It passes power for one scan only, the scan its input goes from false to true; its storage bit remembers last scan.
3. `OTL(Trig_Req)` *Output Latch* does: latches `Trig_Req` (A bottle has reached PE_Trigger and wants an inspection) on. It only ever sets the bit; the bit stays on after the rung goes false.

Why it is built this way:

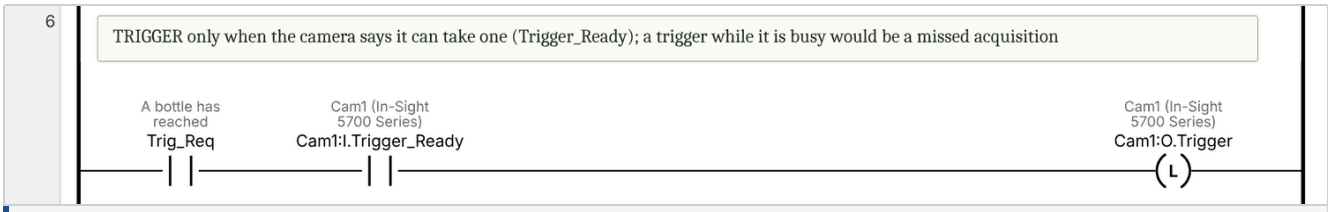
One-shot. Everything after the ONS happens once per event, not on every scan the condition holds; `Trig_ONS` is only its memory of the last scan.

A latch. `Trig_Req` stays on after this rung goes false; an `OTU` elsewhere in the program clears it.

> **Think of it like this:** a doorbell that rings once however long you lean on the button. The ONS looks at what the button was doing last scan: only the change from up to down rings it.

> **Watch for:** Each ONS needs its own storage bit. Two ONS sharing one bit make one of them miss its edges.

RUNG 6




```
XIC(Trig_Req)XIC(Cam1:I.Trigger_Ready)OTL(Cam1:O.Trigger);
```

PREDICT IT · 3 `Trig_Req` is on and `Cam1:I.Trigger_Ready` is off. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Trig_Req` (A bottle has reached PE_Trigger and wants an inspection) is on and `Cam1:I.Trigger_Ready` is on, the rung latches `Cam1:O.Trigger` on.

Left to right:

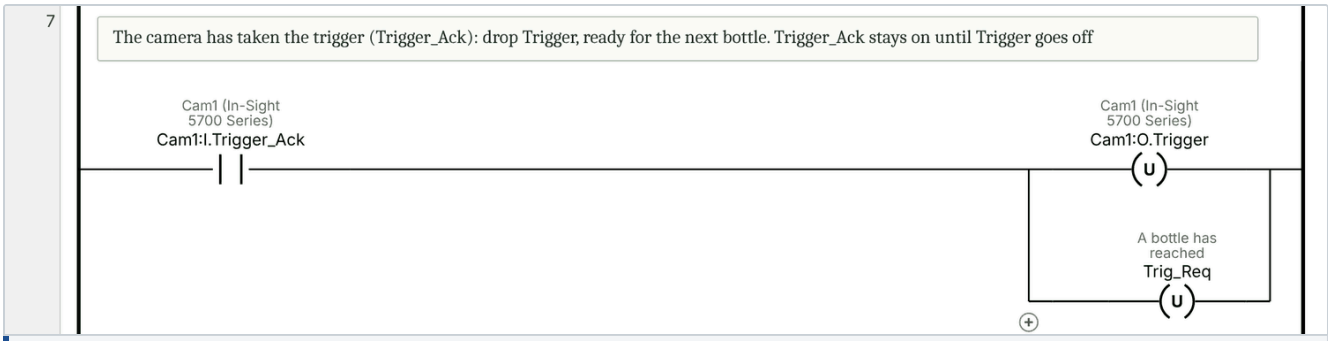
1. `XIC(Trig_Req)` *Examine If Closed* asks: true when `Trig_Req` (A bottle has reached PE_Trigger and wants an inspection) is on.
2. `XIC(Cam1:I.Trigger_Ready)` *Examine If Closed* asks: true when `Cam1:I.Trigger_Ready` is on.
3. `OTL(Cam1:O.Trigger)` *Output Latch* does: latches `Cam1:O.Trigger` on.

Why it is built this way:

A latch: it stays on until an OTU clears it.

> **Watch for:** A latched bit survives the condition that set it: every way in needs a way out, or it stays on for good.

RUNG 7




```
XIC(Cam1:I.Trigger_Ack)[OTU(Cam1:O.Trigger),OTU(Trig_Req)];
```

HOW THIS RUNG WORKS

In words. When `Cam1:I.Trigger_Ack` is on, the rung unlatches `Cam1:O.Trigger` (turns it off) and unlatches `Trig_Req` (A bottle has reached PE_Trigger and wants an inspection) (turns it off).

Left to right:

1. `XIC(Cam1:I.Trigger_Ack)` *Examine If Closed* asks: true when `Cam1:I.Trigger_Ack` is on.
2. A branch of 2 parallel paths: each path is tested on its own and does its own work.

   Path 1

   1. `OTU(Cam1:O.Trigger)` *Output Unlatch* does: unlatches `Cam1:O.Trigger` (turns it off). It only ever clears the bit.

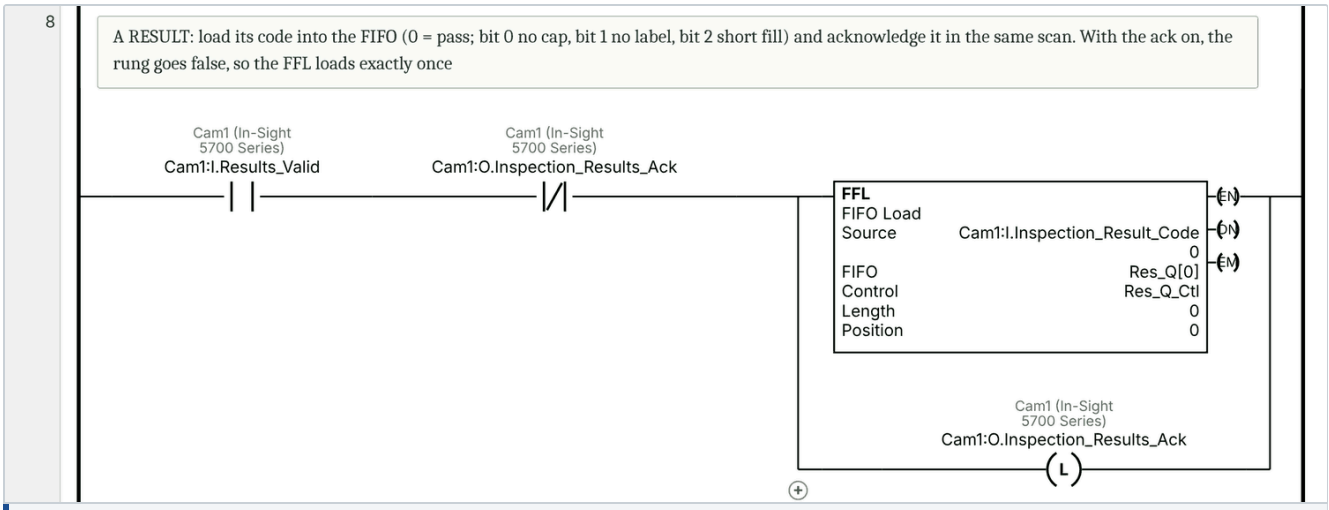
   Path 2

   1. `OTU(Trig_Req)` *Output Unlatch* does: unlatches `Trig_Req` (A bottle has reached PE_Trigger and wants an inspection) (turns it off).

The request is latched because the eye's edge and the camera being ready do not have to happen in the same scan. With 0.9 s between bottles and 180 ms of camera time they always do here, but the program does not rely on it. Rung 7 drops Trigger the moment the camera acknowledges it; a Trigger held on is not a second trigger, and the camera waits for it to drop before it can be acknowledged again.

The results handshake

RUNG 8




```
XIC(Cam1:I.Results_Valid)XIO(Cam1:O.Inspection_Results_Ack)
[FFL(Cam1:I.Inspection_Result_Code,Res_Q[0],Res_Q_Ctl,8),OTL(Cam1:O.Inspection_
Results_Ack)];
```

HOW THIS RUNG WORKS

In words. When `Cam1:I.Results_Valid` is on and `Cam1:O.Inspection_Results_Ack` is off, the rung loads `Cam1:I.Inspection_Result_Code` into the queue `Res_Q[0]` and latches `Cam1:O.Inspection_Results_Ack` on.

Left to right:

1. `XIC(Cam1:I.Results_Valid)` *Examine If Closed* asks: true when `Cam1:I.Results_Valid` is on.
2. `XIO(Cam1:O.Inspection_Results_Ack)` *Examine If Open* asks: true when `Cam1:O.Inspection_Results_Ack` is off.
3. A branch of 2 parallel paths: each path is tested on its own and does its own work.

   Path 1

   1. `FFL(Cam1:I.Inspection_Result_Code,Res_Q[0],Res_Q_Ctl,8)` *FIFO Load* does: loads `Cam1:I.Inspection_Result_Code` into the queue `Res_Q[0]`. One entry per false-to-true transition.

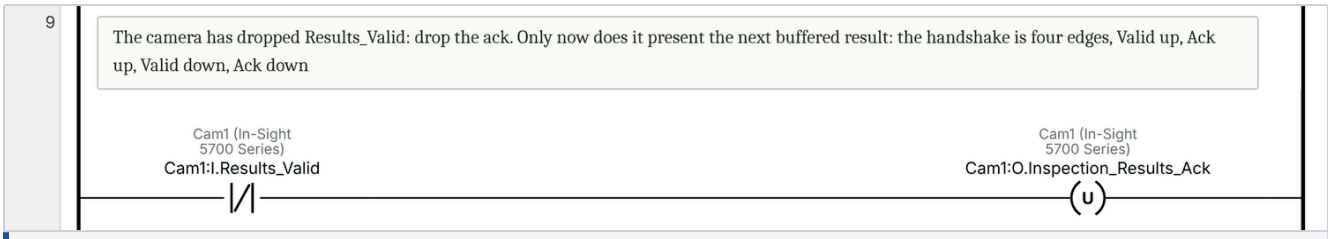
   Path 2

   1. `OTL(Cam1:O.Inspection_Results_Ack)` *Output Latch* does: latches `Cam1:O.Inspection_Results_Ack` on.

Why it is built this way:

A latch: it stays on until an OTU clears it.

RUNG 9




```
XIO(Cam1:I.Results_Valid)OTU(Cam1:O.Inspection_Results_Ack);
```

HOW THIS RUNG WORKS

In words. When `Cam1:I.Results_Valid` is off, the rung unlatches `Cam1:O.Inspection_Results_Ack` (turns it off).

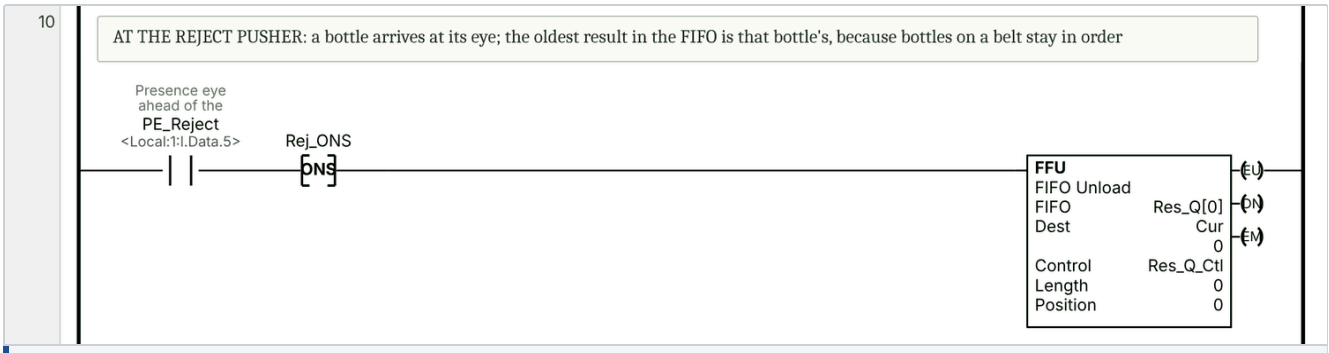
Left to right:

1. `XIO(Cam1:I.Results_Valid)` *Examine If Open* asks: true when `Cam1:I.Results_Valid` is off.
2. `OTU(Cam1:O.Inspection_Results_Ack)` *Output Unlatch* does: unlatches `Cam1:O.Inspection_Results_Ack` (turns it off).

FFL loads on the rung going true. Latching the ack on the same rung makes the rung go false on the next scan, so each result is loaded exactly once, even though Results_Valid stays on until the camera sees the ack. Rung 9 is the other half: the ack comes off when Valid has gone, and only then does the camera present the next queued result.

The reject pusher and the counts

RUNG 10




```
XIC(PE_Reject)ONS(Rej_ONS)FFU(Res_Q[0],Cur,Res_Q_Ctl,8);
```

HOW THIS RUNG WORKS

In words. When `PE_Reject` (Presence eye ahead of the reject pusher) is on and it is the first scan they are true (the one-shot, remembered in `Rej_ONS`), the rung unloads the oldest entry of `Res_Q[0]` into `Cur` (The result code of the bottle at the reject pusher).

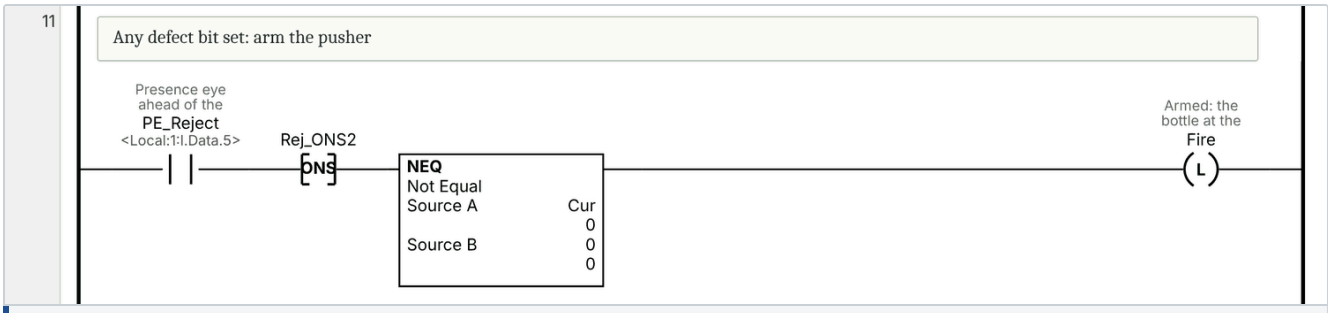
Left to right:

1. `XIC(PE_Reject)` *Examine If Closed* asks: true when `PE_Reject` (Presence eye ahead of the reject pusher) is on.
2. `ONS(Rej_ONS)` *One Shot* asks: true when it is the first scan they are true (the one-shot, remembered in `Rej_ONS`).
3. `FFU(Res_Q[0],Cur,Res_Q_Ctl,8)` *FIFO Unload* does: unloads the oldest entry of `Res_Q[0]` into `Cur` (The result code of the bottle at the reject pusher). One entry per false-to-true transition.

Why it is built this way:

One-shot: once per event, not once per scan.

RUNG 11




```
XIC(PE_Reject)ONS(Rej_ONS2)NEQ(Cur,0)OTL(Fire);
```

HOW THIS RUNG WORKS

In words. When `PE_Reject` (Presence eye ahead of the reject pusher) is on, it is the first scan they are true (the one-shot, remembered in `Rej_ONS2`) and `Cur` (The result code of the bottle at the reject pusher) is not **0**, the rung latches `Fire` (Armed: the bottle at the pusher is to be rejected) on.

Left to right:

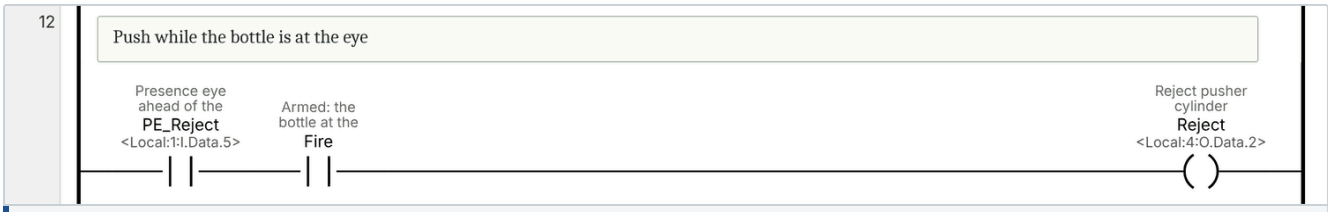
1. `XIC(PE_Reject)` *Examine If Closed* asks: true when `PE_Reject` (Presence eye ahead of the reject pusher) is on.
2. `ONS(Rej_ONS2)` *One Shot* asks: true when it is the first scan they are true (the one-shot, remembered in `Rej_ONS2`).
3. `NEQ(Cur,0)` *Not Equal* asks: true when `Cur` (The result code of the bottle at the reject pusher) is not **0**.
4. `OTL(Fire)` *Output Latch* does: latches `Fire` (Armed: the bottle at the pusher is to be rejected) on.

Why it is built this way:

One-shot: once per event, not once per scan.

A latch: it stays on until an OTU clears it.

RUNG 12




```
XIC(PE_Reject)XIC(Fire)OTE(Reject);
```

PREDICT IT · 4 `PE_Reject` is off and `Fire` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

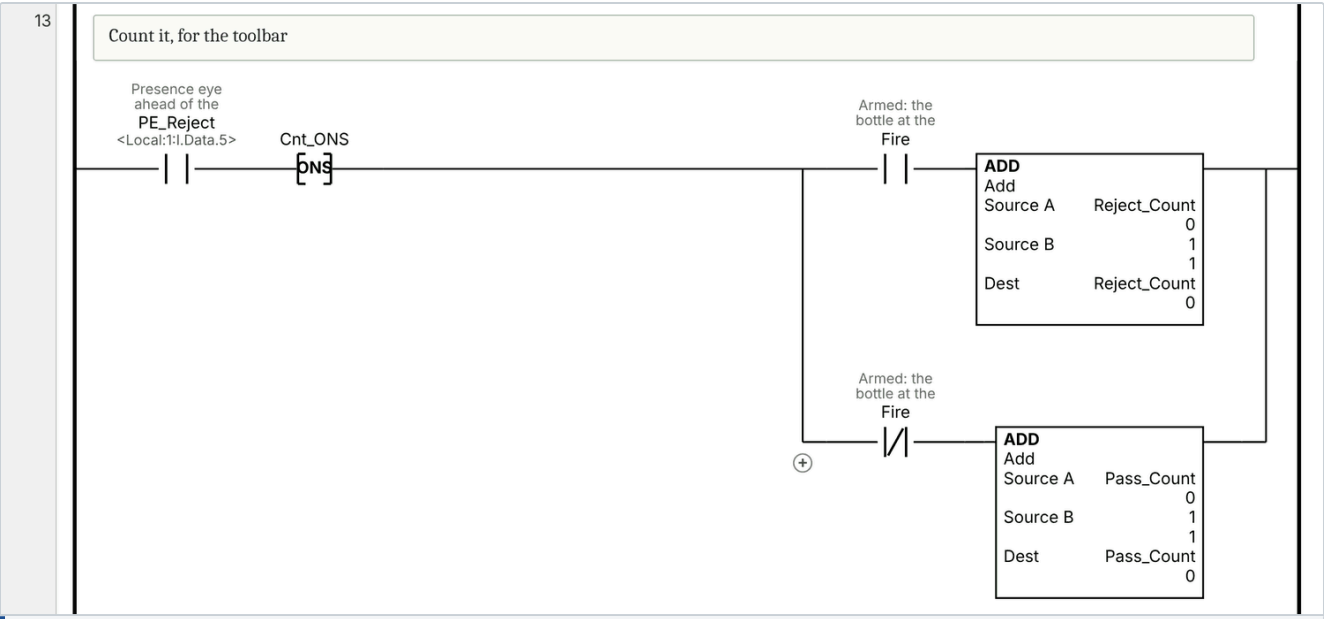
HOW THIS RUNG WORKS

In words. When `PE_Reject` (Presence eye ahead of the reject pusher) is on and `Fire` (Armed: the bottle at the pusher is to be rejected) is on, the rung turns `Reject` (Reject pusher cylinder) on (and off again whenever the rung is false).

Left to right:

1. `XIC(PE_Reject)` *Examine If Closed* asks: true when `PE_Reject` (Presence eye ahead of the reject pusher) is on.
2. `XIC(Fire)` *Examine If Closed* asks: true when `Fire` (Armed: the bottle at the pusher is to be rejected) is on.
3. `OTE(Reject)` *Output Energize* does: turns `Reject` (Reject pusher cylinder) on (and off again whenever the rung is false).

RUNG 13




```
XIC(PE_Reject)ONS(Cnt_ONS)
[XIC(Fire)ADD(Reject_Count,1,Reject_Count),XIO(Fire)ADD(Pass_Count,1,Pass_Count
)];
```

HOW THIS RUNG WORKS

In words. When `PE_Reject` (Presence eye ahead of the reject pusher) is on and it is the first scan they are true (the one-shot, remembered in `Cnt_ONS`), the rung picks its path: if `Fire` (Armed: the bottle at the pusher is to be rejected) is on, it makes `Reject_Count = Reject_Count + 1`; if `Fire` is off, it makes `Pass_Count = Pass_Count + 1`.

Left to right:

1. `XIC(PE_Reject)` *Examine If Closed* asks: true when `PE_Reject` (Presence eye ahead of the reject pusher) is on.
2. `ONS(Cnt_ONS)` *One Shot* asks: true when it is the first scan they are true (the one-shot, remembered in `Cnt_ONS`).
3. A branch of 2 parallel paths: each path is tested on its own and does its own work.

   Path 1

   1. `XIC(Fire)` *Examine If Closed* asks: true when `Fire` (Armed: the bottle at the pusher is to be rejected) is on.
   2. `ADD(Reject_Count,1,Reject_Count)` *Add* does: makes `Reject_Count = Reject_Count + 1`. Every scan the rung is true; behind a one-shot it adds once per event.

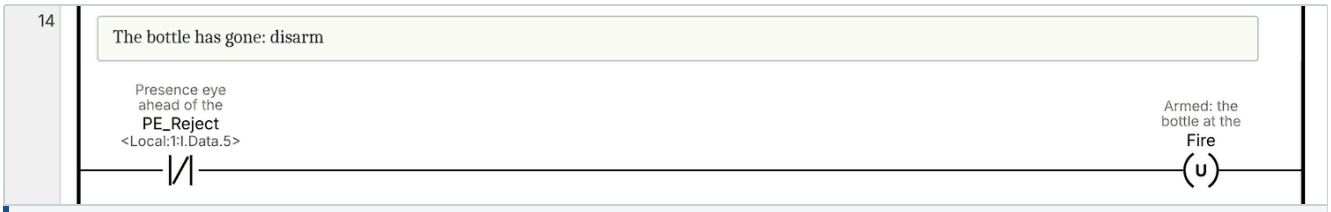
   Path 2

   1. `XIO(Fire)` *Examine If Open* asks: true when `Fire` is off.
   2. `ADD(Pass_Count,1,Pass_Count)` *Add* does: makes `Pass_Count = Pass_Count + 1`.

Why it is built this way:

One-shot: once per event, not once per scan.

RUNG 14




```
XIO(PE_Reject)OTU(Fire);
```

HOW THIS RUNG WORKS

In words. When `PE_Reject` (Presence eye ahead of the reject pusher) is off, the rung unlatches `Fire` (Armed: the bottle at the pusher is to be rejected) (turns it off).

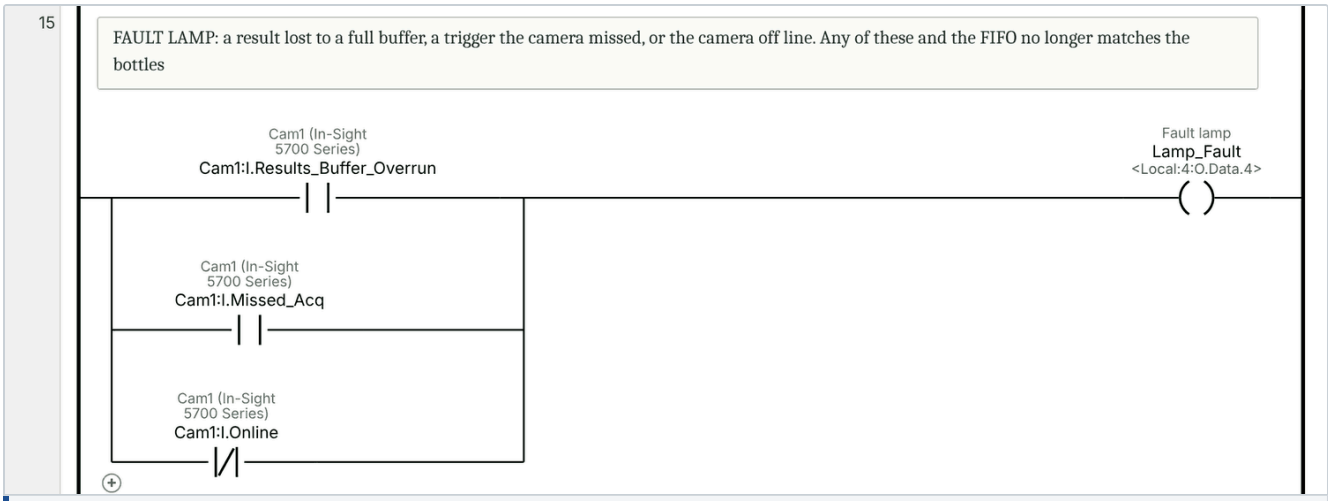
Left to right:

1. `XIO(PE_Reject)` *Examine If Open* asks: true when `PE_Reject` (Presence eye ahead of the reject pusher) is off.
2. `OTU(Fire)` *Output Unlatch* does: unlatches `Fire` (Armed: the bottle at the pusher is to be rejected) (turns it off).

The pattern is Level 4's diverter, with one FIFO instead of one per zone: unload on the eye's rising edge, arm, push while the bottle is at the eye, disarm when it has gone. The counts are for the toolbar; the score card keeps the official ones.

The fault lamp

RUNG 15




```
[XIC(Cam1:I.Results_Buffer_Overrun),XIC(Cam1:I.Missed_Acq),XIO(Cam1:I.Online)]OTE(Lamp_Fault);
```

PREDICT IT · 5 `Cam1:I.Results_Buffer_Overrun` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When either (`Cam1:I.Results_Buffer_Overrun` is on) *or* (`Cam1:I.Missed_Acq` is on) *or* (`Cam1:I.Online` is off), the rung turns `Lamp_Fault` (Fault lamp) on (and off again whenever the rung is false).

Left to right:

1. A branch of 3 parallel paths: power gets through if any one of them is true.
   - **Path 1**
     1. `XIC(Cam1:I.Results_Buffer_Overrun)` *Examine If Closed* asks: true when `Cam1:I.Results_Buffer_Overrun` is on.
   - **Path 2**
     1. `XIC(Cam1:I.Missed_Acq)` *Examine If Closed* asks: true when `Cam1:I.Missed_Acq` is on.
   - **Path 3**
     1. `XIO(Cam1:I.Online)` *Examine If Open* asks: true when `Cam1:I.Online` is off.
2. `OTE(Lamp_Fault)` *Output Energize* does: turns `Lamp_Fault` (Fault lamp) on (and off again whenever the rung is false).

> **Think of it like this:** parallel paths are two roads to the same town. If either road is open you get there; the rung only cares that power reaches the far end, not which way it went.

Run it

Load your program, Run mode, Start. Camera_HMI shows each image as it is taken, with the three tools drawn on it in green or red. In the reference run, 20 of 20 bottles in the right bin, nothing lost, in 36 s against a par of 45 s: 2200 points and three stars.

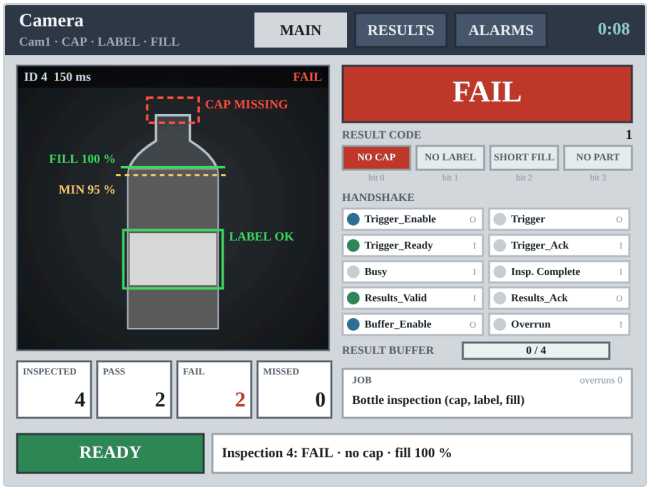


Figure 162. Camera_HMI on a reject: the cap tool finds no cap (red, dashed), the label and fill tools pass; result code 1; Results_Valid on, the ack not yet raised.

Figure 163. A pass: all three tools green, code 0. Trigger_Ready is back on and the result buffer is empty.

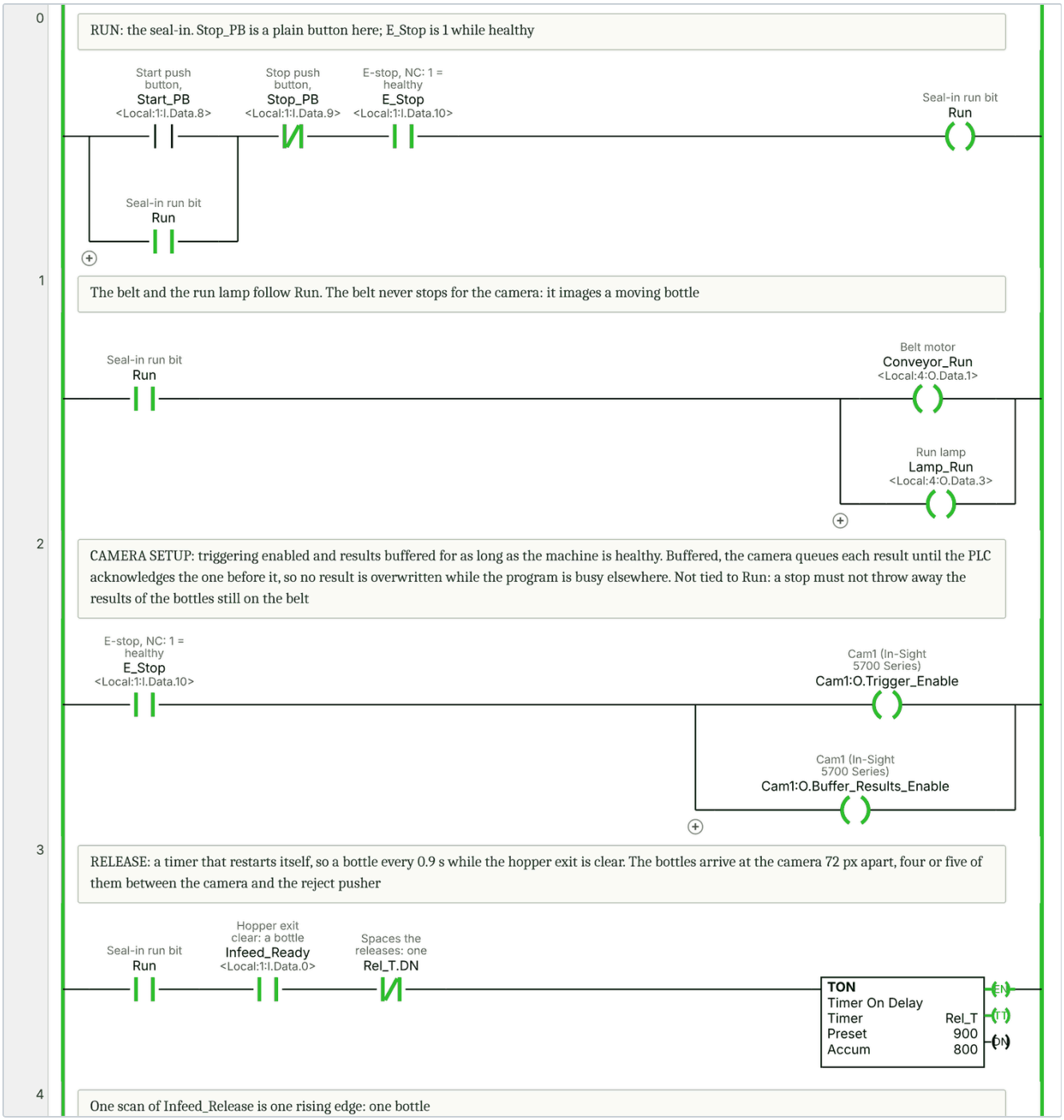


Figure 164. Level 8 online, the top of the routine.

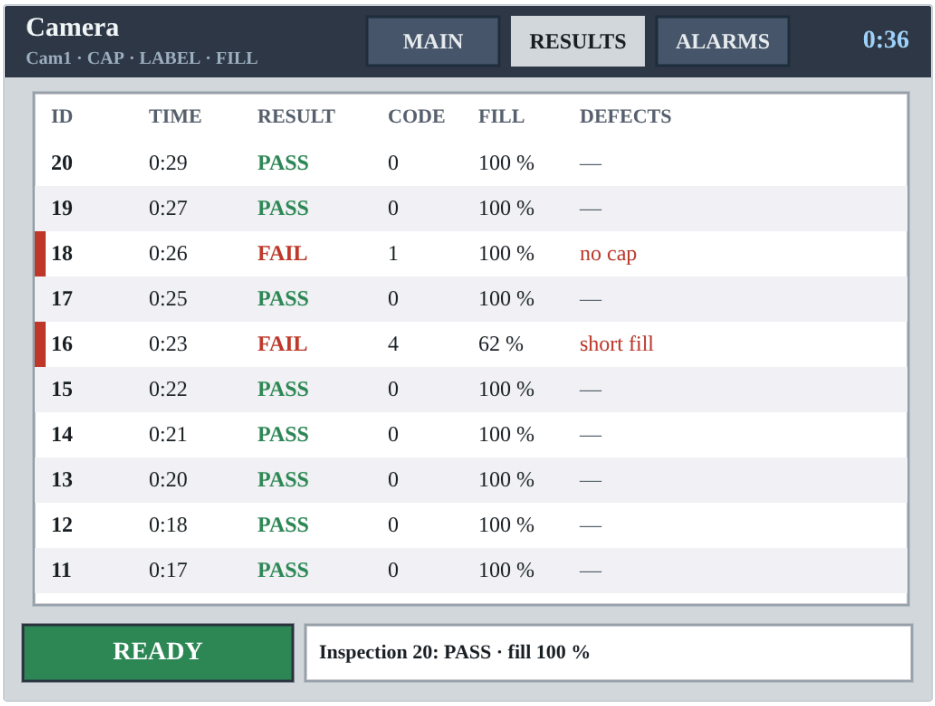


Figure 165. RESULTS after the run: the last ten inspections, the failed ones flagged with their defects.

WATCH FOR: THE TOGGLE AND THE LEVEL

`Inspection_Completed` toggles. A rung with `XIC(Cam1:I.Inspection_Completed)` on it is true for every other inspection, not every one. If you want to use it, remember its last state in a BOOL and compare. `Results_Valid` with the ack handshake is simpler, and it is the signal that means the result is ready to read.

WATCH FOR: ACKING WITHOUT DROPPING

Raise the ack and never drop it (leave out rung 9) and you get exactly one result. With buffering on, the camera queues the next four and throws the fifth away, and from then on every verdict belongs to a different bottle. The fault lamp lights on the overrun, but by then the FIFO is already wrong.

TRY IT: SEE AN OVERRUN

Take rung 9 out and run. Watch the RESULT BUFFER bar on Camera_HMI climb to 4 / 4 and the overrun count start. Then put rung 9 back, take out `Cam1:O.Buffer_Results_Enable` in rung 2, and slow the ack by putting `TON(Ack_T,500,0)` before it: the results overwrite each other instead, and nothing tells you, which is why plants buffer.

> **TRY IT: A MISSED ACQUISITION**
>
> Set Rel_T to 300 ms and take `XIC(Cam1:I.Trigger_Ready)` out of rung 6. Some bottles now arrive while the camera is still inspecting the last: Missed_Acq, the fault lamp, and −25 for every bottle that passes uninspected.

On a real camera

The module's *Mode* in I/O Configuration is *Simulated*, and the Vision Sensor then plays the camera. In the desktop app, switch Cam1 to *implicit I/O* at the camera's address and the same program drives a real In-Sight. The real camera takes the trigger, runs its own job on whatever is under its lens, and sends back its own Job_Pass, result code and Results_Valid. The sim only mirrors its bits: the ring light flashes when the real camera acquires. That mode is for working the handshake on real hardware and is not scored, because the score is kept against the sim's bottles. For the program to work unchanged, the job on the camera has to report the same result layout: the defect bits in the result code and the fill in the result data. Which spreadsheet cells feed those is set in In-Sight Explorer's EtherNet/IP settings.

Card mode: the card decides

On a live camera the verdict comes from whatever is under the lens, and the sim's bottles have their own defects, so the two do not match and the run is not scored. Card mode fixes that. Tick **Card mode** in the Camera's properties and the hopper sends only good bottles; each bottle waits at the camera until its result is presented, then takes that verdict and moves on. The verdict comes from a bottle card under the camera:

Card	Bottle	Result code	Fill (byte 9)	Card ID (byte 10)
SN-4801	good	0	100	1
SN-4802	no cap	1	100	2
SN-4803	no label	2	100	3
SN-4804	short fill	4	70	4
no card	no part in view: a reject	8	0	0

Level 8's rungs do not change: they trigger, read the code, carry it in the FIFO and reject on it, exactly as before. What changes is the belt. A bottle that waits at the camera holds the bottles behind it, so the line runs at the pace of the camera's answers, and with a real camera that is the pace at which you swap cards.

With the simulated camera, choose the card in **Card under the lens**, or on Camera_HMI's **CARDS** page touch the card you want under the lens. The page draws the four cards as the camera sees them through the red filter and keeps a tally per card: what it should read, how many times it was seen, how many it read right and its last code. MAIN names the card on the image, reads HOLDING while a bottle waits for its result, and turns the card line red when a card reads the wrong verdict.

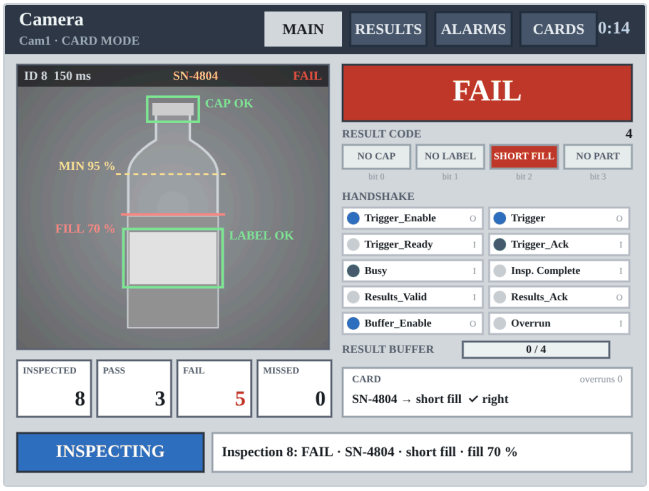


Figure 166. Card mode, MAIN: SN-4804 under the lens, the short-fill verdict at 70 % and the card line confirming it.

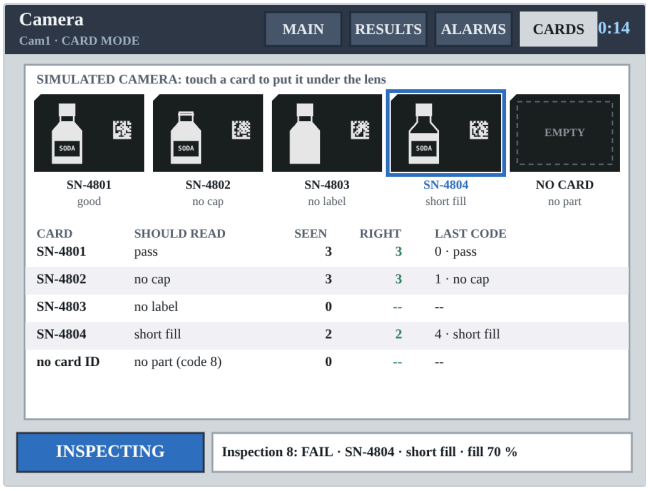


Figure 167. The CARDS page: touch a card to put it under the lens; the tally shows what each card read.

With a real camera (Cam1 on implicit I/O in the desktop app), the 5705 decides. The sim reads three things from it when Results_Valid rises: `Inspection_Result_Code` for the verdict, byte 9 of `Inspection_Results` for the fill and byte 10 for the card ID. The job on the camera writes those, so the card ID is something the job reads off the card, not something typed in. A result whose code does not match its card ID shows up on the CARDS page and in the event log.

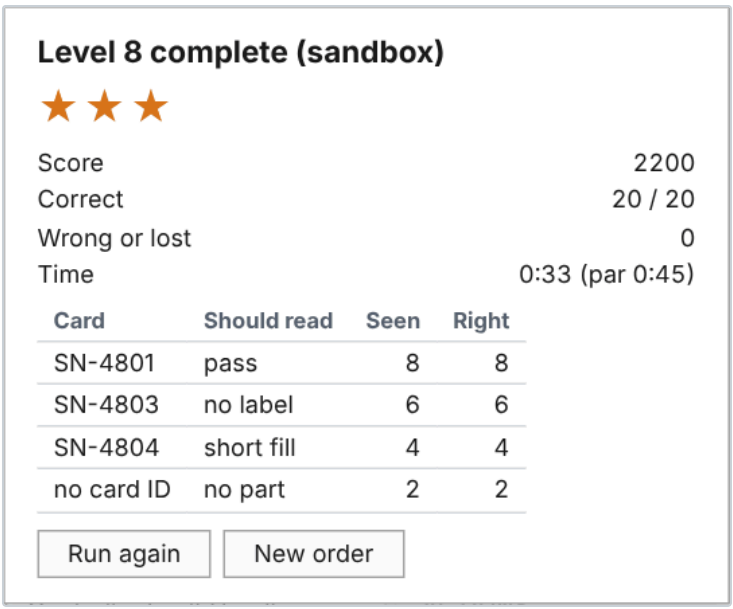


Figure 168. The score card in card mode lists the cards seen and how many each read right.

TRY IT: SWAP CARDS MID-RUN

Tick Card mode, start with SN-4801 under the lens and run. After five bottles touch SN-4804 on the CARDS page, then the empty card, then SN-4803. Watch each bottle stop under the camera, take the card, and go to the bin its verdict sends it to. The score card at the end lists every card you used, and if the rungs are right every count in the Right column matches Seen.

TRY IT: BREAK THE ACK

In card mode, take rung 9 out (the ack never drops). The first bottle gets its verdict, the second waits at the camera and never moves: its result is in the buffer but never presented, and the line stays held. Put the rung back and the line runs again. That is the same fault as the overrun Try it, made visible on the belt.

The bottle cards

The cards are printed on the bench printer from STEP files in the repository's `cards` folder: 100 × 70 × 4 mm, a chamfered corner so a card only sits one way in its nest, and two bodies in each file, a dark base and red features inlaid 0.8 mm and standing 0.6 mm proud. Through the red band-pass filter the red reads bright and the base dark. Everything on a card is something the job uses: the bottle's outline (to locate it), the cap, the liquid (its top edge is the fill line), a label window with the product name for OCR, and a DataMatrix that encodes the card's number. There is no text for people on a card.

The number is `SN-48RK`: R is the product (0 SODA, 1 WATER, 2 MILK, 3 JUICE, 4 OIL) and K the defect (1 good, 2 no cap, 3 no label, 4 short fill at 70 %). The job puts R × 10 + K in byte 10, so the SODA set reads 1 to 4, the IDs in the table above. Level 8 uses the SODA set; Level 9 uses the whole set (see its chapter, *Card mode on Level 9*). On Camera_HMI's CARDS page a tab per product picks the row of four cards.

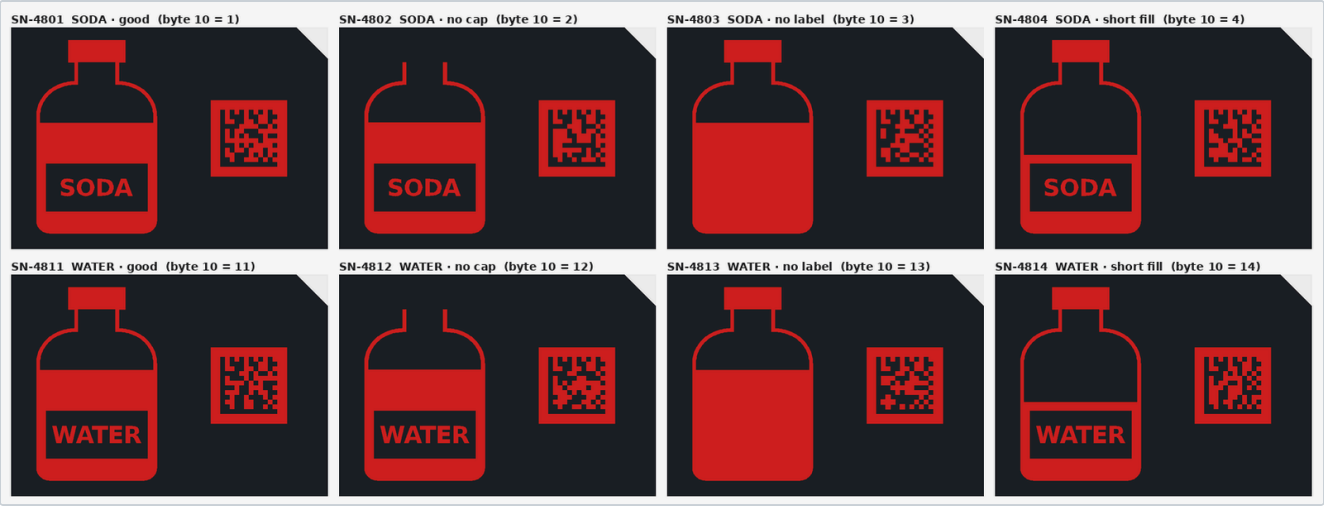


Figure 169. The SODA and WATER cards as printed: good, no cap, no label, short fill.

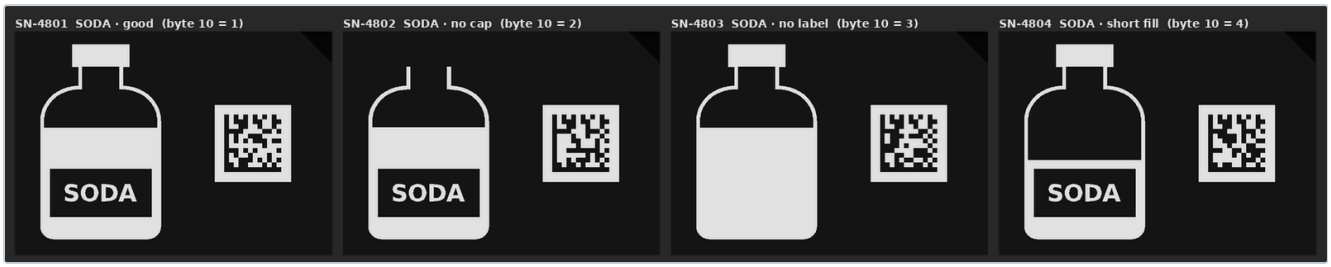


Figure 170. The SODA set as the camera sees it through the red filter. Each card here was decoded (DataMatrix) and read (OCR) from this view before the files were released.

The camera's own image

With Cam1 live, the camera screen shows the picture the 5705 actually took. Every time `Inspection_Complete` toggles, the page asks the bridge for it; the bridge logs in to the camera's FTP server, reads `image.bmp` (an In-Sight always keeps its last acquired image there), scales it down to the width set in the Camera's properties and hands it back. The screen fits it into the image pane and writes under it what came back from the camera: the result code, the fill (byte 9) and the card (byte 10). The camera's own graphics are not in `image.bmp`; it is the raw image.

Nothing is added to the camera job for this. The bridge pulls the file, so there are no spreadsheet cells, no Event and no FTP server on the PC. The login is `admin` with a blank password unless you set another user (one with FTP Read under Sensor → User Access Settings) in the Camera's properties: **Live image**, **FTP user**, **FTP password** and **width to fetch**. One fetch runs at a time; inspections that finish while an image is on its way ask for one more when it lands, so at a fast trigger rate the screen shows the latest image, not every one. A refused login or a camera that does not answer is written in the pane.

Touch the image and it fills the screen: the IMAGE view shows it as large as Camera_HMI allows, with the inspection ID, the image size and how long the fetch took above it, and the code, fill and card below. Touch it again to go back. **Live image: width to fetch** sets the size the bridge sends: 800 is plenty for the screen; set it to the camera's own width (1600 on a 5603) to get every pixel on the screen. **FULL SIZE** on the IMAGE view goes further: it fetches the last image unscaled and opens it over the whole window, with Fit, 100, 200 and 400 %, scrolling to pan, and the pixel under the pointer with its grey value, which is handy for checking a threshold or a DataMatrix module against what the job sees. Esc closes it. The screen keeps its smaller copy, so the full image is only fetched when you ask for it.

Each image is work for the camera, on top of inspecting and keeping the I/O connection going, so **Live image: at most one image every** (1 s by default) keeps a fast trigger from queueing one fetch after another. If a busy camera goes quiet for a moment, the I/O connection is held, not dropped: the module log says *stalled* and then *resumed after the stall*, and only ten seconds of silence closes and reopens it. On an older camera such as the 5603, an RPI of 50–100 ms and In-Sight Explorer offline (no live video) keep the stalls away.

A real In-Sight 4.x job

On In-Sight 4.x firmware (a 5603, say) a spreadsheet job sends its results to the PLC with two cells: `FormatOutputBuffer`, which packs values into bytes, and `WriteEIPBuffer(A0, …)`, which puts them in `Inspection_Results` after each inspection. Lay them out the way the simulated camera does, and Logix Connect reads them where it expects them:

Bytes	Value	Format
0–3	inspection count	32-bit
4–5	result code (bit 0 no cap, 1 no label, 2 short fill, 3 no card, 4 wrong product)	16-bit
6, 7	0	8-bit
8	pass (1/0)	8-bit
9	fill %	8-bit
10	card ID, R × 10 + K	8-bit

Then set the Camera's **Live camera: where the inspection ID and result code are read** to *Results buffer*: the screen, the RESULTS page and card mode take the ID from bytes 0–3 and the code from bytes 4–5. On 4.x the module's own `Inspection_Result_Code` field is not set by the job, so Level 8's results rung has to read the code from the buffer too: `FFL(Cam1:I.Inspection_Results[4],Res_Q[0],Res_Q_Ctl,8)` in place of the code field, the change you would make on a real line with that camera.

Two more things a 4.x camera does its own way. With `Buffer_Results_Enable` on, it raises `Results_Valid` only when the job writes a results buffer; a job without `WriteEIPBuffer` never raises it, Level 8's results rung never runs, and nothing is rejected. (The screen still counts such a camera's inspections on `Inspection_Complete` until it has seen `Results_Valid` once.) And PASS / FAIL on the screen is the camera's `Job_Pass`, which follows the job's pass/fail cell (Sensor → Job Pass/Fail Cell Setup): a cell such as `And($F$6==1,$F$13==1)` that is 1 when the label read and the code decoded.

TRY IT: SN-4801 TO SN-4804 ON A REAL CAMERA

With the two buffer cells in the job and *Results buffer* selected, put each SODA card under the lens in turn and trigger. SN-4801 should read code 0, fill 100 and card 1; SN-4802 code 1 (no cap); SN-4803 code 2 (no label: the OCR fails); SN-4804 code 4 with fill 70 and card 4. The CARDS page tallies every one, and any card whose code does not match what it should read shows red: that is the job to fix, not the program.

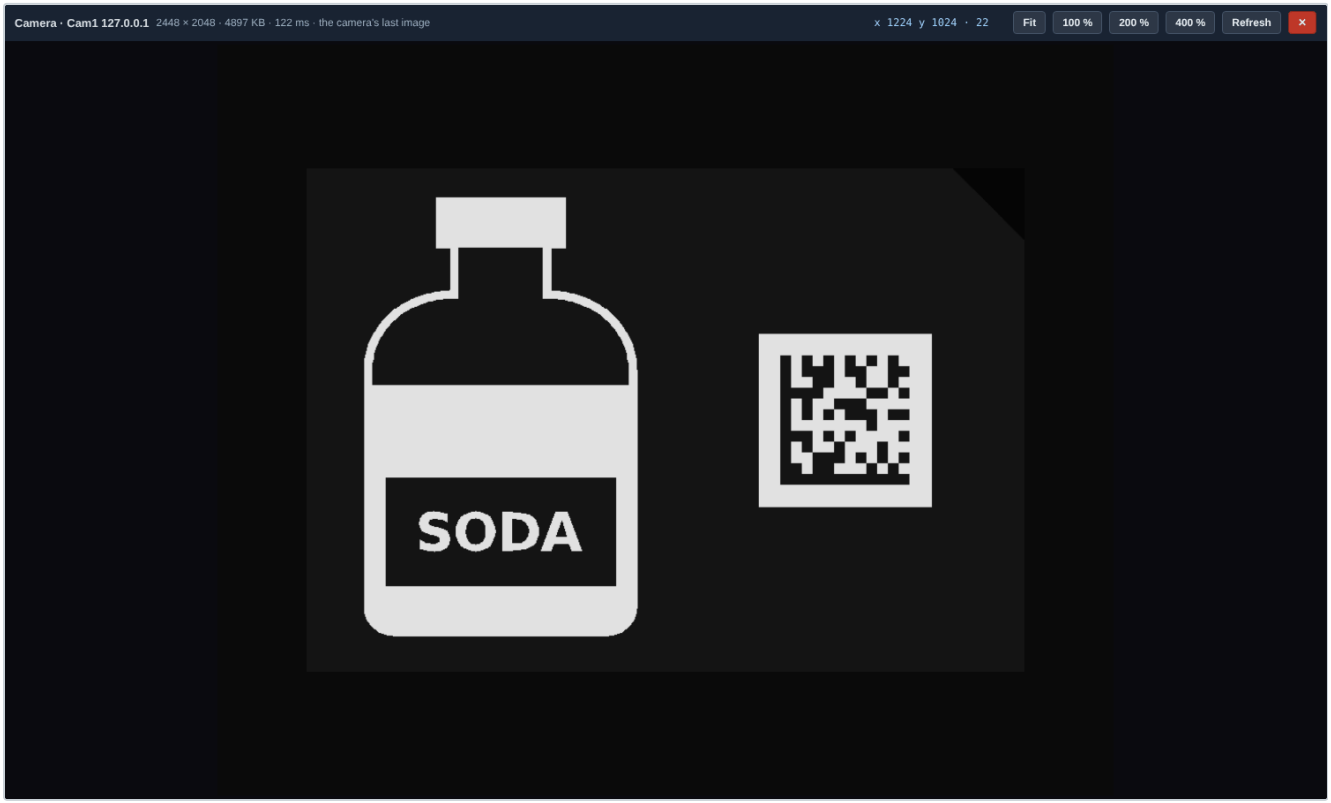


Figure 171. FULL SIZE: the camera's last image at its own resolution, with the pixel under the pointer and its grey value at the top right.

Without card mode, a live camera's inspections count too. Each time `Inspection_Complete` toggles (or `Results_Valid` rises, with the results buffer on) the screen takes the camera's `Inspection_ID`, its result code, `Job_Pass` and, on a bottle job, bytes 9 and 10, and shows them as the simulated camera's would:

PASS or FAIL, the counts, the RESULTS page and the banner. Those results do not move the sim's bottles; that is what card mode is for.

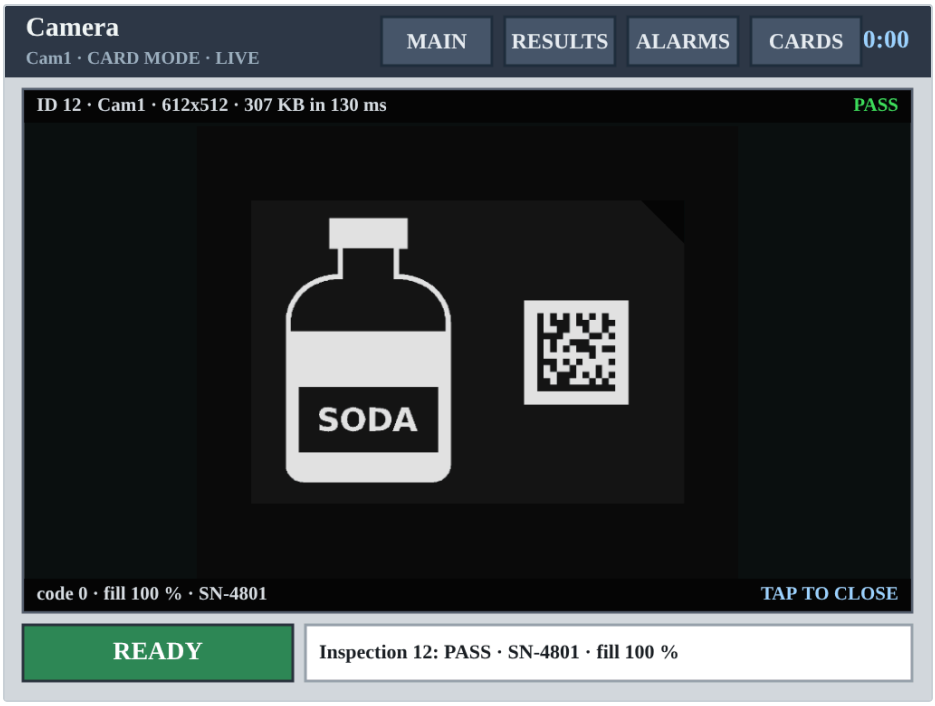


Figure 172. The IMAGE view: touch the live image on MAIN and it fills the screen.

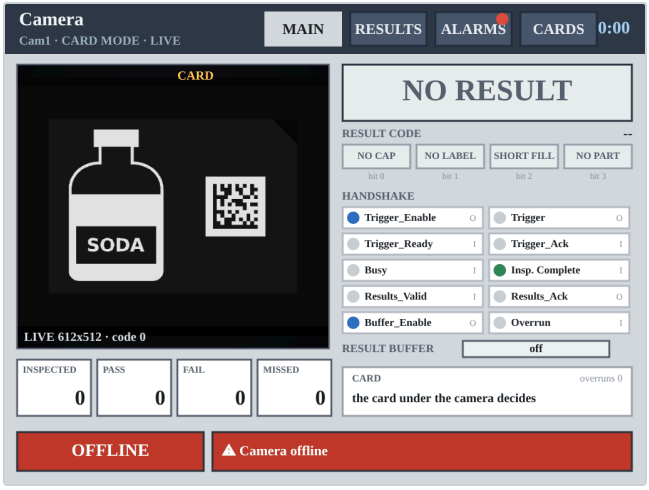


Figure 173. Card mode with a live camera: SN-4801 as the camera took it, fetched over FTP and scaled to 612 × 512.

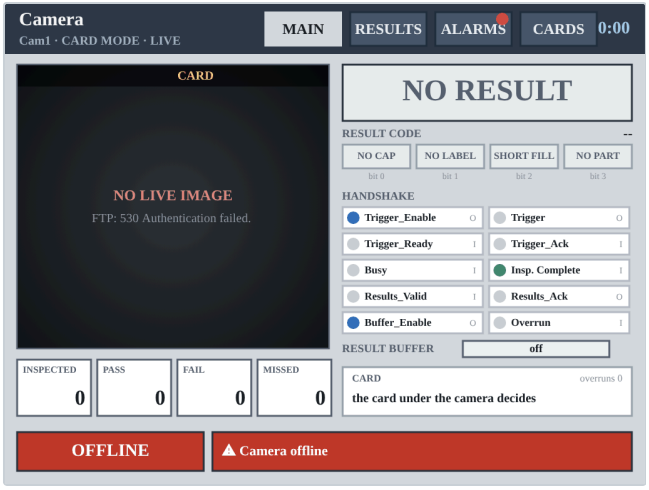


Figure 174. A wrong FTP password: NO LIVE IMAGE and the server's reply.

> **TRY IT: SEE WHAT THE CAMERA SAW**
>
> With Cam1 live in the desktop app, tick Card mode and run Level 8 with SN-4804 under the lens. Each bottle that stops at the camera brings the card's picture to Camera_HMI with *code 4 · fill 70 %* under it. Now cover half the card with your hand and trigger again: the picture shows your hand and the code shows what the job made of it. Then set **FTP password** to anything and watch the pane report the camera's refusal.

Predict it: the answers

1. The rung is false at `XIO(Stop_PB)`: `Stop_PB` is on, so `Run` is **off** (an OTE writes 0 when its rung is false).
2. Every condition on the way is met, so `Rel_T` times toward its preset.
3. The rung is false at `XIC(Cam1:I.Trigger_Ready)`: `Cam1:I.Trigger_Ready` is off, so the OTL does nothing: `Cam1:O.Trigger` keeps whatever it had.
4. The rung is false at `XIC(PE_Reject)`: `PE_Reject` is off, so `Reject` is **off** (an OTE writes 0 when its rung is false).
5. Every condition on the way is met, so `Lamp_Fault` is **on**.

41 Level 9 — Changeover

> **IN THE APP**
>
> File → New Project… → A graded level → **9**. Guided mode opens on this chapter: the tags, every rung below under its fold, a Verify check, and the run results: the camera changes job to WATER and then JUICE, the stray bottle of the old product is rejected, the first trace record reads `SODA,01,000001`, the level completes, every bottle in the right bin, under par.

Level 8 had one product. This line runs three, one batch after another: eight SODA, eight WATER, eight JUICE. Each product has its own camera job, because each has its own bottle, label and fill line: SODA passes at 95 % of the label, WATER at 90 %, JUICE at 97 %. Between batches the program has to do what an operator does at a real changeover: hold the infeed, let the last bottle of the old product clear the camera, load the next job and wait for the camera to confirm it, then start feeding again.

Two more things come with it. Every changeover leaves a bottle of the old product behind somewhere upstream, and in this level it turns up as the third bottle of the next batch: a SODA bottle among the WATER, a WATER bottle among the JUICE. The camera has to catch it, and it does, because the job it is running knows what product it expects. And every bottle gets a **trace record**: the PLC tells the camera which bottle is coming (its position in the batch and its number on the line), the camera puts that into its answer, and the PLC files the answer in a STRING array. Twenty-four bottles, par 65 s.

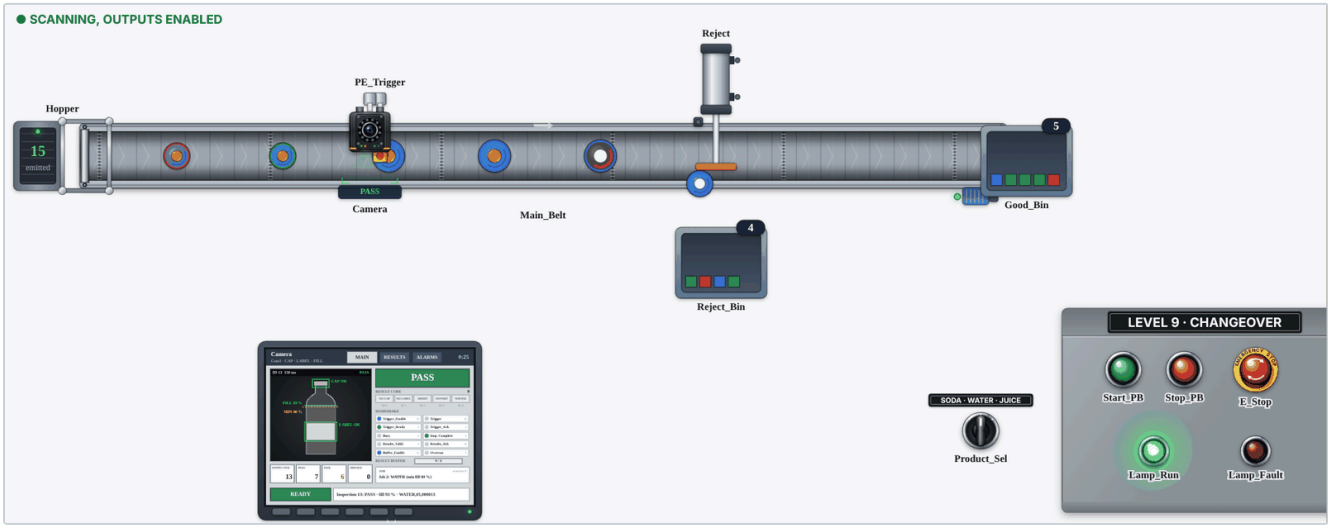


Figure 175. Level 9 in the top view. The machine is Level 8's: hopper, PE_Trigger and the camera, the reject pusher over Reject_Bin, Good_Bin at the end. New: Product_Sel beside the console, which shows the batch the next bottle belongs to. Camera_HMI is showing Job 2, WATER.

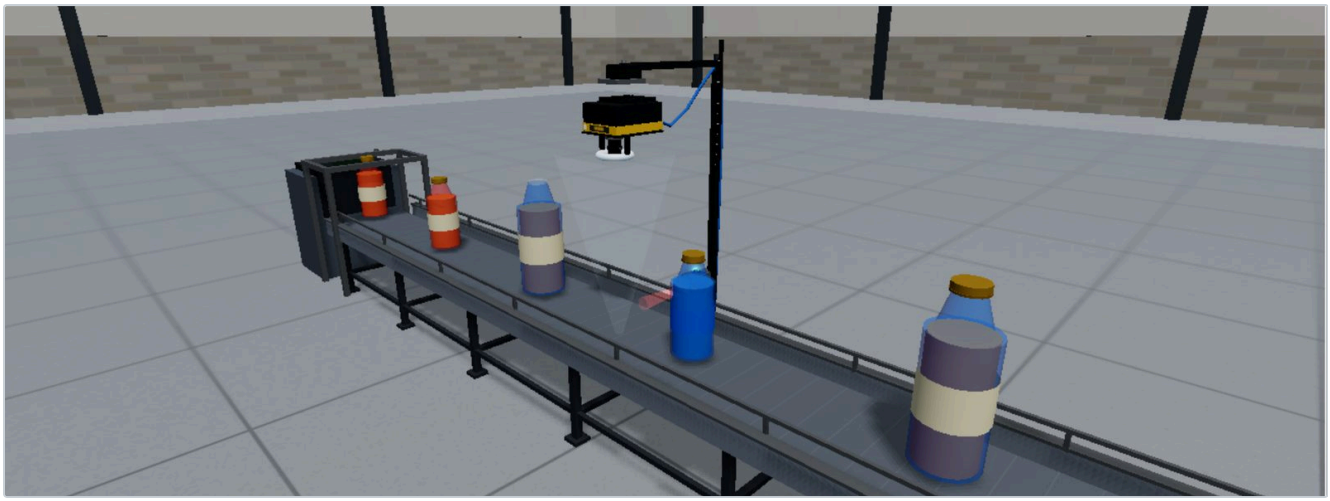


Figure 176. In 3D: the batches are told apart by their labels: red for SODA, blue for WATER, cream for JUICE.

The machine

- **Hopper**: the Emitter of Level 8 with *Batches* `SODA:8,WATER:8,JUICE:8` and *Stray bottle* ticked. Each bottle carries its product, its batch and its serial number. About one in five has a defect (no cap, no label, short fill), and about one in seven is filled to 91-94 %, which WATER passes and SODA and JUICE do not.
- **Product_Sel**: a three-position selector driven by the hopper, not by you: left for SODA, centre for WATER, right for JUICE. It shows the batch of the *next* bottle to leave the hopper, so it changes the moment the last bottle of a batch has gone. In a plant this signal would come from the scheduling system or a filler recipe; here it is two inputs, `Prod_SODA` and `Prod_JUICE`, with neither on meaning WATER.
- **Camera**: the Vision Sensor of Level 8, with three jobs in *Jobs*: `SODA:95,WATER:90,JUICE:97`. Job 1 is SODA, job 2 WATER, job 3 JUICE. It starts on job 1.
- Main_Belt, PE_Trigger, Reject with PE_Reject, Reject_Bin, Good_Bin, Camera_HMI and the console are Level 8's.

The camera's new tags

Everything from Level 8 still applies. These are the tags this level adds. They are the In-Sight 5700 series' own names.

Tag	Meaning
`Cam1:I.Current_Job_ID`	The job the camera is running now (1, 2 or 3).
`Cam1:O.Command_ID`	The job to load when Execute_Command rises.
`Cam1:O.Execute_Command`	Rising edge: run the command (here, load job Command_ID). Hold it until Command_Complete.

Tag	Meaning
`Cam1:I.Command_Complete`	The command has finished; stays on until Execute_Command drops.
`Cam1:I.Command_Failed`	The command could not run, for example a job number the camera does not have.
`Cam1:O.User_Data[…]`	Bytes the PLC sends with an inspection. The job reads bytes 0-1 as the bottle's position in its batch and bytes 4-7 as its number on the line.
`Cam1:O.Set_User_Data`	Rising edge: the camera latches User_Data for the next trigger.
`Cam1:I.Set_User_Data_Ack`	The camera has the data; stays on until Set_User_Data drops.
`Cam1:I.Inspection_Results[12…27]`	The trace: 16 ASCII characters, `PRODUCT,PP,NNNNNN`. PRODUCT is the job's product, PP the position and NNNNNN the number the PLC sent.
Result code bit 4	Wrong product: the bottle is not the product the running job expects. Any job, any bottle.

Three handshakes, in order

The job change. Put the job number in `Command_ID` and raise `Execute_Command`. The camera loads the job (about half a second, with `Trigger_Ready` off while it does), sets `Current_Job_ID` and raises `Command_Complete`. Drop `Execute_Command` and the camera drops `Command_Complete`. A trigger during the change is a missed acquisition. A bottle inspected under the wrong job gets the wrong verdict: a SODA bottle inspected under the WATER job is a wrong-product reject.

The user data. Before each trigger, write User_Data and raise `Set_User_Data`. Wait for `Set_User_Data_Ack`, then trigger. The camera ties the data it latched to the next image it takes. Trigger before the ack and the inspection carries the last bottle's data.

The trigger and the result are Level 8's, unchanged.

How the level is scored

What happens	Points
A defective or wrong-product bottle in Reject_Bin	+100
A good bottle in Good_Bin	+100
A defective bottle in Good_Bin (shipped)	−100
A good bottle in Reject_Bin	−50
A bottle that passes the camera without being inspected	−25
Each `Trace[n]` that holds bottle n+1's record exactly, checked at the end	+25
Complete under par	+200

The trace check compares `Trace[0]` to `SODA,01,000001`, `Trace[1]` to `SODA,02,000002` and so on to `Trace[23]`, `JUICE,08,000024`. The stray bottle's record names the batch it was in, not the product it was: the 11th bottle is a SODA bottle in the WATER batch, and its record is `WATER,03,000011`. That is what the camera knows: the job it ran and the data it was sent.

How to pass it, step by step

1. **Start from Level 8.** Run, belt, camera setup, trigger, results FIFO, reject: all of it stays.
2. **Work out the wanted job** from the selector, every scan.
3. **Gate the release on the job.** A bottle leaves the hopper only while the camera is running the wanted job and no change is under way.
4. **Wait for the line to clear** once the hopper is held: 5 s is enough for the last bottle of the old batch to pass the camera and its result to come back.
5. **Change the job**, and drop Execute_Command on Command_Complete.
6. **Send the user data** with each trigger, and trigger only on the ack.
7. **File the trace** when each result is read.

Tags this lesson needs

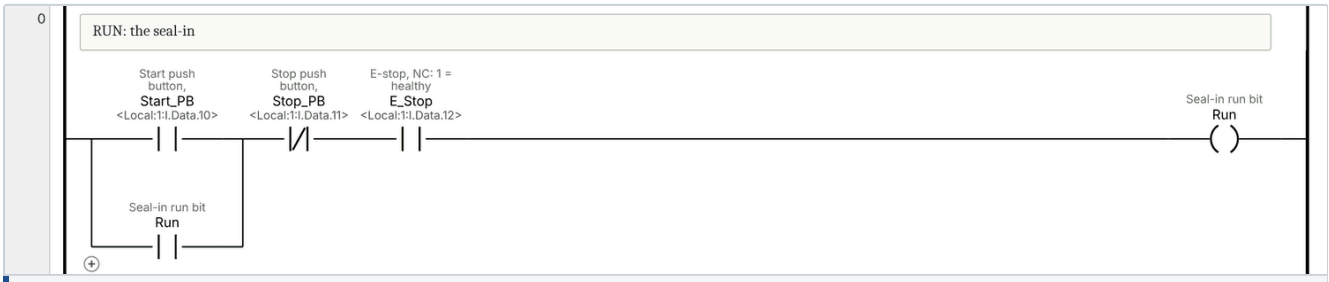
The level's alias tags are already made for you (chapter 32's table). Create these plain tags before you type the rungs; Verify reports any you miss.

Tag	Type	How to make it	Description
`Run`	BOOL	Edit Tags: name, type	Seal-in run bit
`Want_Job`	DINT	Edit Tags: name, type	The camera job the next batch needs: 1 SODA, 2 WATER, 3 JUICE
`Job_OK`	BOOL	Edit Tags: name, type	The camera has that job loaded and no change is running
`Rel_T`	TIMER	Edit Tags: name, type	Release interval
`Clear_T`	TIMER	Edit Tags: name, type	Line clear: time for the last bottle of a batch to pass the camera
`Bottle_No`	DINT	Edit Tags: name, type	Bottles that have reached the camera, counted at the trigger eye
`Pos`	DINT	Edit Tags: name, type	This bottle's position in its batch, 1-8
`Trig_Req`	BOOL	Edit Tags: name, type	
`Trig_ONS`	BOOL	Edit Tags: name, type	
`Fire`	BOOL	Edit Tags: name, type	
`Rej_ONS`	BOOL	Edit Tags: name, type	
`Rej_ONS2`	BOOL	Edit Tags: name, type	

Tag	Type	How to make it	Description
`Cnt_ONS`	BOOL	Edit Tags: name, type	
`Res_Q`	DINT[8]	Edit Tags: name, type, Dim 8	Result codes waiting for the reject eye
`Res_Q_Ctl`	CONTROL	Edit Tags: name, type	Queue control
`Cur`	DINT	Edit Tags: name, type	The result code of the bottle at the reject eye
`Trace`	STRING[24]	Edit Tags: name, type, Dim 24	One trace record per bottle, PRODUCT,PP,NNNNNN
`Trace_Idx`	DINT	Edit Tags: name, type	Next free record in Trace

Run, setup and the wanted job

RUNG 0




```
[XIC(Start_PB),XIC(Run)]XIO(Stop_PB)XIC(E_Stop)OTE(Run);
```

PREDICT IT · 1 `Start_PB` is on, `Stop_PB` is on and `E_Stop` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When either (`Start_PB` (Start push button, momentary) is on) *or* (`Run` (Seal-in run bit) is on), `Stop_PB` (Stop push button, momentary) is off and `E_Stop` (E-stop, NC: 1 = healthy) is on, the rung turns `Run` on (and off again whenever the rung is false).

Left to right:

1. A branch of 2 parallel paths: power gets through if any one of them is true.

   Path 1

   1. `XIC(Start_PB)` *Examine If Closed* asks: true when `Start_PB` (Start push button, momentary) is on.

   Path 2

   1. `XIC(Run)` *Examine If Closed* asks: true when `Run` (Seal-in run bit) is on.
2. `XIO(Stop_PB)` *Examine If Open* asks: true when `Stop_PB` (Stop push button, momentary) is off.
3. `XIC(E_Stop)` *Examine If Closed* asks: true when `E_Stop` (E-stop, NC: 1 = healthy) is on.
4. `OTE(Run)` *Output Energize* does: turns `Run` on (and off again whenever the rung is false). Every scan it writes the rung state: true on, false off.

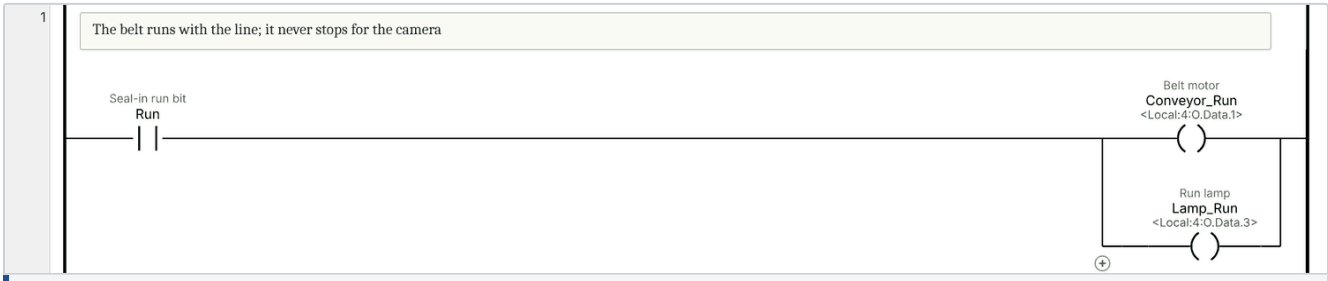
Why it is built this way:

Seal-in. The start condition turns `Run` on, and `Run` 's own contact in the parallel path keeps the rung true after the start goes away. Anything in series outside the branch (a stop, a fault) breaks it, and then it stays off until the start is made again.

> **Think of it like this:** a doorbell button and a hand. The start button is the doorbell, pressed for a moment. The seal-in contact is a hand that grabs the circuit as soon as the output comes on and keeps holding after you let go. The stop button, in series, is the only thing that can knock that hand away.

> **Watch for:** The stop has to be in series outside the branch. Put it inside the start path only and the seal-in path bypasses it: the stop no longer stops.

RUNG 1




```
XIC(Run)[OTE(Conveyor_Run),OTE(Lamp_Run)];
```

HOW THIS RUNG WORKS

In words. When `Run` (Seal-in run bit) is on, the rung turns `Conveyor_Run` (Belt motor) on (and off again whenever the rung is false) and turns `Lamp_Run` (Run lamp) on (and off again whenever the rung is false).

Left to right:

1. `XIC(Run)` *Examine If Closed* asks: true when `Run` (Seal-in run bit) is on.
2. A branch of 2 parallel paths: each path is tested on its own and does its own work.

   Path 1

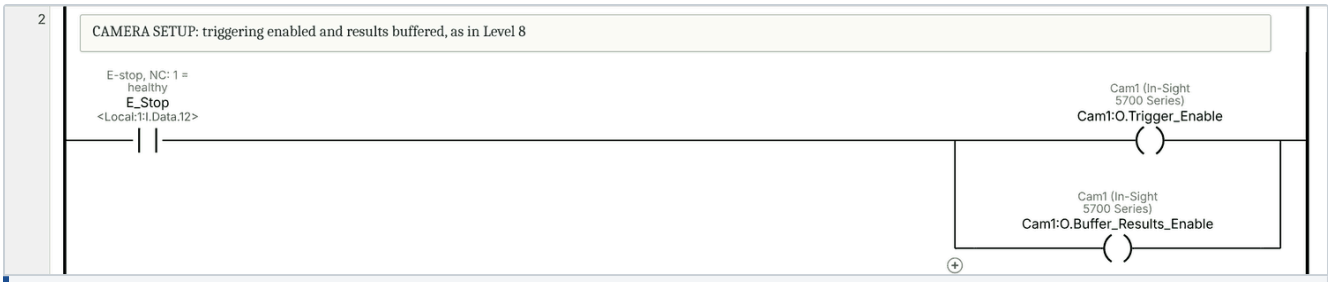
   1. `OTE(Conveyor_Run)` *Output Energize* does: turns `Conveyor_Run` (Belt motor) on (and off again whenever the rung is false).

   Path 2

   1. `OTE(Lamp_Run)` *Output Energize* does: turns `Lamp_Run` (Run lamp) on (and off again whenever the rung is false).

> **Think of it like this:** a rung is a pipe with valves in it, read left to right. Every contact is a valve: XIC is open while its bit is 1, XIO while its bit is 0. Water reaches the output at the right only if every valve in a row is open. The output does not remember anything: on every scan it is simply wet or dry.

RUNG 2




```
XIC(E_Stop)[OTE(Cam1:O.Trigger_Enable),OTE(Cam1:O.Buffer_Results_Enable)];
```

HOW THIS RUNG WORKS

In words. When `E_Stop` (E-stop, NC: 1 = healthy) is on, the rung turns `Cam1:O.Trigger_Enable` on (and off again whenever the rung is false) and turns `Cam1:O.Buffer_Results_Enable` on (and off again whenever the rung is false).

Left to right:

1. `XIC(E_Stop)` *Examine If Closed* asks: true when `E_Stop` (E-stop, NC: 1 = healthy) is on.
2. A branch of 2 parallel paths: each path is tested on its own and does its own work.

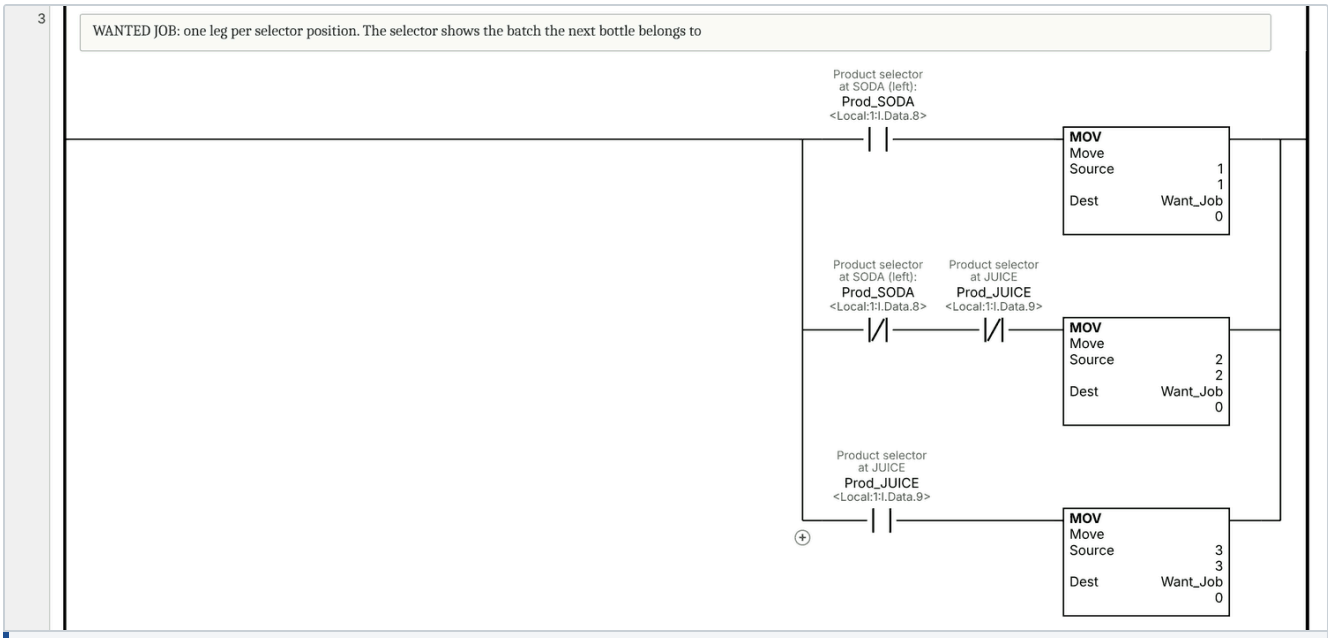
   Path 1

   1. `OTE(Cam1:O.Trigger_Enable)` *Output Energize* does: turns `Cam1:O.Trigger_Enable` on (and off again whenever the rung is false).

   Path 2

   1. `OTE(Cam1:O.Buffer_Results_Enable)` *Output Energize* does: turns `Cam1:O.Buffer_Results_Enable` on (and off again whenever the rung is false).

RUNG 3




```
[XIC(Prod_SODA)MOV(1,Want_Job),XIO(Prod_SODA)XIO(Prod_JUICE)MOV(2,Want_Job),XIC(Prod_JUICE)MOV(3,Want_Job)];
```

PREDICT IT · 2 `Prod_SODA` is off and `Prod_JUICE` is off. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. Every scan, with nothing to test, the rung picks its path: if `Prod_SODA` (Product selector at SODA (left): the next batch is SODA) is on, it puts **1** into `Want_Job` (The camera job the next batch needs: 1 SODA, 2 WATER, 3 JUICE); if `Prod_SODA` is off and `Prod_JUICE` (Product selector at JUICE (right); neither = WATER) is off, it puts **2** into `Want_Job`; if `Prod_JUICE` is on, it puts **3** into `Want_Job`.

Left to right:

1. A branch of 3 parallel paths: each path is tested on its own and does its own work.

Path 1

1. `XIC(Prod_SODA)` *Examine If Closed* asks: true when `Prod_SODA` (Product selector at SODA (left): the next batch is SODA) is on.
2. `MOV(1,Want_Job)` *Move* does: puts **1** into `Want_Job` (The camera job the next batch needs: 1 SODA, 2 WATER, 3 JUICE). It writes every scan the rung is true.

Path 2

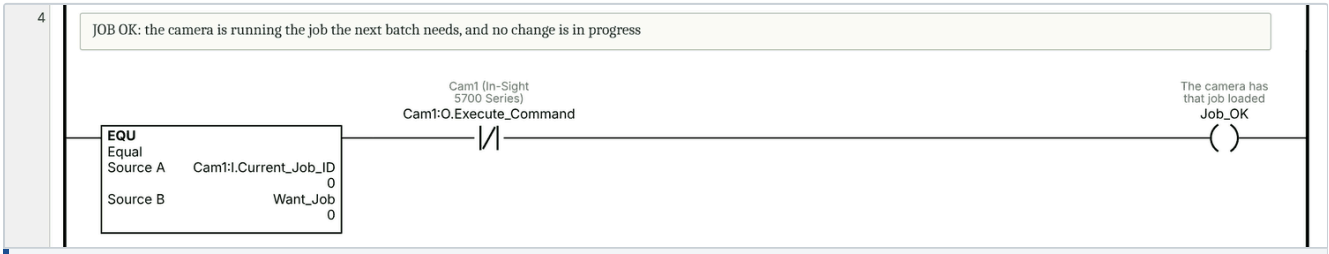
1. `XIO(Prod_SODA)` *Examine If Open* asks: true when `Prod_SODA` is off.
2. `XIO(Prod_JUICE)` *Examine If Open* asks: true when `Prod_JUICE` (Product selector at JUICE (right); neither = WATER) is off.
3. `MOV(2,Want_Job)` *Move* does: puts **2** into `Want_Job`.

Path 3

1. `XIC(Prod_JUICE)` *Examine If Closed* asks: true when `Prod_JUICE` is on.

2. `MOV(3,Want_Job)` *Move* does: puts **3** into `Want_Job` .

RUNG 4




```
EQU(Cam1:I.Current_Job_ID,Want_Job)XIO(Cam1:O.Execute_Command)OTE(Job_OK);
```

PREDICT IT · 3 `Cam1:I.Current_Job_ID` is 501, `Want_Job` is 500 and `Cam1:O.Execute_Command` is off. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Cam1:I.Current_Job_ID` equals `Want_Job` (The camera job the next batch needs: 1 SODA, 2 WATER, 3 JUICE) and `Cam1:O.Execute_Command` is off, the rung turns `Job_OK` (The camera has that job loaded and no change is running) on (and off again whenever the rung is false).

Left to right:

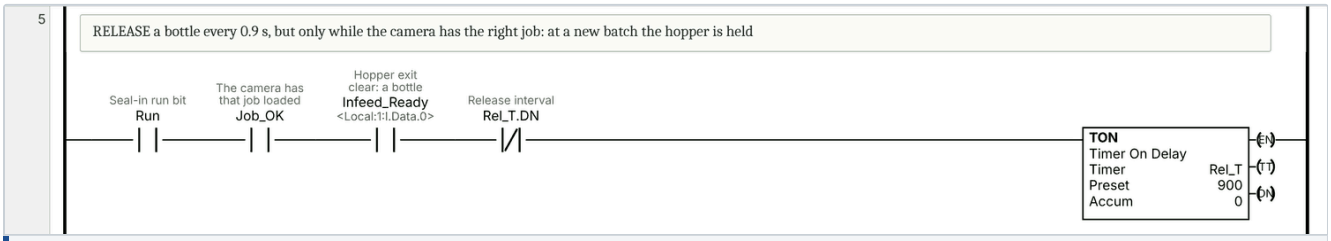
1. `EQU(Cam1:I.Current_Job_ID,Want_Job)` *Equal* asks: true when `Cam1:I.Current_Job_ID` equals `Want_Job` (The camera job the next batch needs: 1 SODA, 2 WATER, 3 JUICE). Both values are compared every scan.
2. `XIO(Cam1:O.Execute_Command)` *Examine If Open* asks: true when `Cam1:O.Execute_Command` is off.
3. `OTE(Job_OK)` *Output Energize* does: turns `Job_OK` (The camera has that job loaded and no change is running) on (and off again whenever the rung is false).

Rung 3 is one rung with three legs, one per selector position. Exactly one leg is true at a time. With neither `Prod_SODA` nor `Prod_JUICE` on, the middle leg writes 2. Rung 4 is the gate for everything below it: `Job_OK` is on only when the camera's job matches and no change is in progress. Because rung 3 is written before rung 4, the job is worked out and then tested in the same scan.

Release, only on the right job

Rung 5 is Level 8's release timer with `Job_OK` in it. When the last SODA bottle leaves the hopper, the selector moves to WATER, Want_Job becomes 2, Job_OK drops and the timer stops: nothing more is released until the camera has job 2.

RUNG 5




```
XIC(Run)XIC(Job_OK)XIC(Infeed_Ready)XIO(Rel_T.DN)TON(Rel_T,900,0);
```

PREDICT IT · 4 `Run` is on, `Job_OK` is on, `Infeed_Ready` is on and `Rel_T.DN` (the done bit of `Rel_T`: Release interval) is off. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Run` (Seal-in run bit) is on, `Job_OK` (The camera has that job loaded and no change is running) is on, `Infeed_Ready` (Hopper exit clear: a bottle can be released) is on and `Rel_T.DN` (the done bit of `Rel_T`: Release interval) is off, the rung runs the timer `Rel_T` toward 900 ms (0.9 s).

Left to right:

1. `XIC(Run)` *Examine If Closed* asks: true when `Run` (Seal-in run bit) is on.
2. `XIC(Job_OK)` *Examine If Closed* asks: true when `Job_OK` (The camera has that job loaded and no change is running) is on.
3. `XIC(Infeed_Ready)` *Examine If Closed* asks: true when `Infeed_Ready` (Hopper exit clear: a bottle can be released) is on.
4. `XIO(Rel_T.DN)` *Examine If Open* asks: true when `Rel_T.DN` (the done bit of `Rel_T`: Release interval) is off.
5. `TON(Rel_T,900,0)` *Timer On Delay* does: runs the timer `Rel_T` toward 900 ms (0.9 s). The accumulator counts while the rung is true; .DN comes on at the preset; the moment the rung goes false the timer resets.

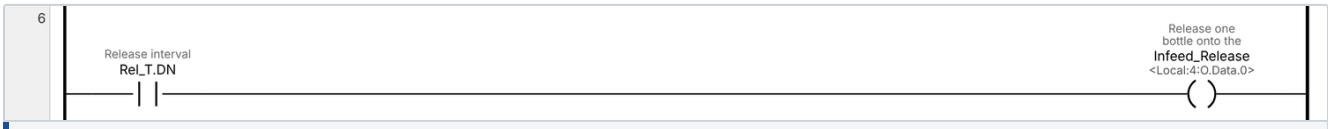
Why it is built this way:

A timer. `Rel_T` only times while this rung is true; other rungs use its `.DN` (done) or `.TT` (timing) bits. Being a TON, it starts again from zero every time the rung goes false.

> **Think of it like this:** a kitchen timer that only runs while you hold your finger on it. Take your finger off and it snaps back to zero. Hold it long enough and the done light comes on, and stays on as long as your finger is still there.

> **Watch for:** If anything in series flickers false for one scan, a TON starts over from zero. Use RTO when the time must survive an interruption.

RUNG 6




```
XIC(Rel_T.DN)OTE(Infeed_Release);
```

HOW THIS RUNG WORKS

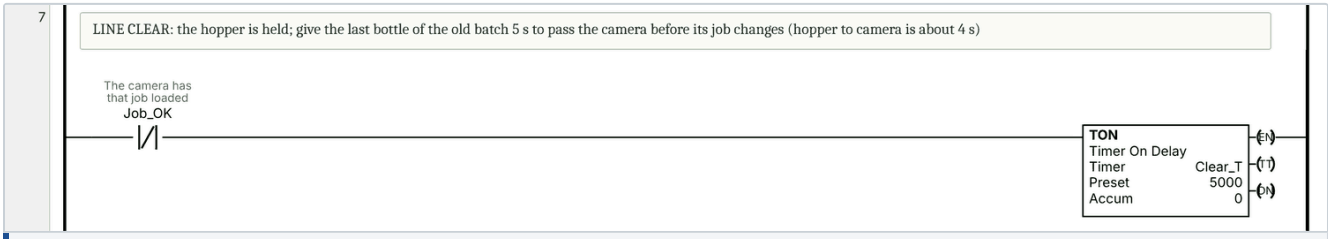
In words. When `Rel_T.DN` (the done bit of `Rel_T` : Release interval) is on, the rung turns `Infeed_Release` (Release one bottle onto the belt (rising edge)) on (and off again whenever the rung is false).

Left to right:

1. `XIC(Rel_T.DN)` *Examine If Closed* asks: true when `Rel_T.DN` (the done bit of `Rel_T` : Release interval) is on.
2. `OTE(Infeed_Release)` *Output Energize* does: turns `Infeed_Release` (Release one bottle onto the belt (rising edge)) on (and off again whenever the rung is false).

The changeover

RUNG 7




```
XIO(Job_OK)TON(Clear_T,5000,0);
```

HOW THIS RUNG WORKS

In words. When `Job_OK` (The camera has that job loaded and no change is running) is off, the rung runs the timer `Clear_T` (Line clear: time for the last bottle of a batch to pass the camera) toward 5000 ms (5 s).

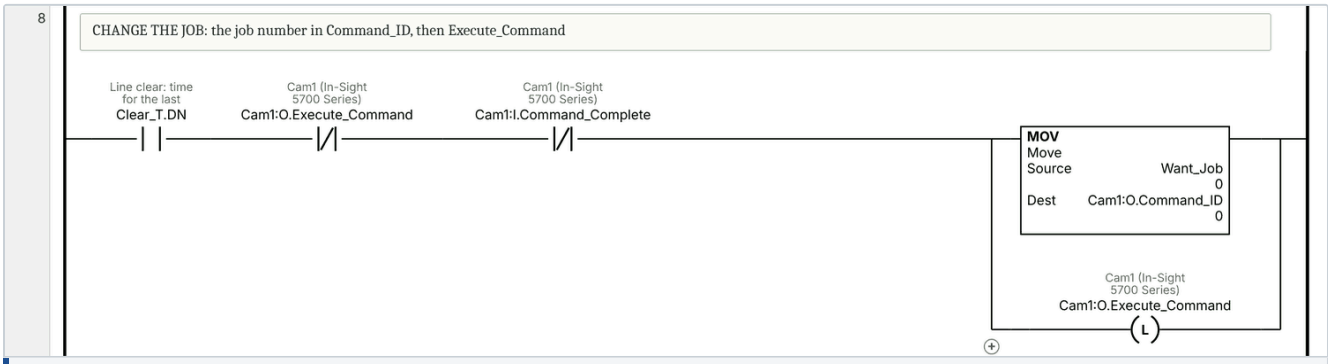
Left to right:

1. `XIO(Job_OK)` *Examine If Open* asks: true when `Job_OK` (The camera has that job loaded and no change is running) is off.
2. `TON(Clear_T,5000,0)` *Timer On Delay* does: runs the timer `Clear_T` (Line clear: time for the last bottle of a batch to pass the camera) toward 5000 ms (5 s).

Why it is built this way:

A timer: it times only while this rung is true.

RUNG 8




```
XIC(Clear_T.DN)XIO(Cam1:O.Execute_Command)XIO(Cam1:I.Command_Complete)
[MOV(Want_Job,Cam1:O.Command_ID),OTL(Cam1:O.Execute_Command)];
```

HOW THIS RUNG WORKS

In words. When `Clear_T.DN` (the done bit of `Clear_T` : Line clear: time for the last bottle of a batch to pass the camera) is on, `Cam1:O.Execute_Command` is off and `Cam1:I.Command_Complete` is off, the rung puts `Want_Job` (The camera job the next batch needs: 1 SODA, 2 WATER, 3 JUICE) into `Cam1:O.Command_ID` and latches `Cam1:O.Execute_Command` on.

Left to right:

1. `XIC(Clear_T.DN)` *Examine If Closed* asks: true when `Clear_T.DN` (the done bit of `Clear_T` : Line clear: time for the last bottle of a batch to pass the camera) is on.
2. `XIO(Cam1:O.Execute_Command)` *Examine If Open* asks: true when `Cam1:O.Execute_Command` is off.
3. `XIO(Cam1:I.Command_Complete)` *Examine If Open* asks: true when `Cam1:I.Command_Complete` is off.
4. A branch of 2 parallel paths: each path is tested on its own and does its own work.

   Path 1

   1. `MOV(Want_Job,Cam1:O.Command_ID)` *Move* does: puts `Want_Job` (The camera job the next batch needs: 1 SODA, 2 WATER, 3 JUICE) into `Cam1:O.Command_ID` .

   Path 2

   1. `OTL(Cam1:O.Execute_Command)` *Output Latch* does: latches `Cam1:O.Execute_Command` on. It only ever sets the bit; the bit stays on after the rung goes false.

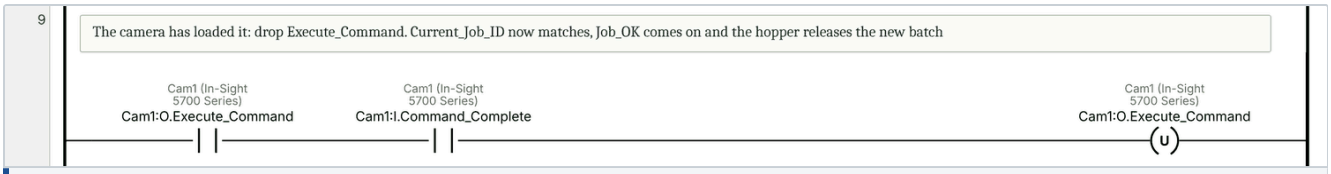
Why it is built this way:

A latch. `Cam1:O.Execute_Command` stays on after this rung goes false; an `OTU` elsewhere in the program clears it.

> **Think of it like this:** a light switch instead of a doorbell. OTL flips it on and it stays on when you take your hand away; OTU flips it off. Whatever flipped it on does not have to stay there.

> **Watch for:** A latched bit survives the condition that set it: every way in needs a way out, or it stays on for good.

RUNG 9




```
XIC(Cam1:O.Execute_Command)XIC(Cam1:I.Command_Complete)OTU(Cam1:O.Execute_Command);
```

PREDICT IT · 5 `Cam1:O.Execute_Command` is on and `Cam1:I.Command_Complete` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Cam1:O.Execute_Command` is on and `Cam1:I.Command_Complete` is on, the rung unlatches `Cam1:O.Execute_Command` (turns it off).

Left to right:

1. `XIC(Cam1:O.Execute_Command)` *Examine If Closed* asks: true when `Cam1:O.Execute_Command` is on.
2. `XIC(Cam1:I.Command_Complete)` *Examine If Closed* asks: true when `Cam1:I.Command_Complete` is on.
3. `OTU(Cam1:O.Execute_Command)` *Output Unlatch* does: unlatches `Cam1:O.Execute_Command` (turns it off). It only ever clears the bit.

Rung 7 counts the line-clear time from the moment the hopper is held. The last SODA bottle is somewhere between the hopper and the camera. At 80 px/s it needs about 4 s to reach the camera and pass it, and its result comes back 180 ms after that. Five seconds covers it. Rung 8 is the command: it fires once, because `XIO(Cam1:O.Execute_Command)` is in it and it latches Execute_Command. Rung 9 drops the command when the camera answers. On the next scan Current_Job_ID equals Want_Job and Execute_Command is off, so rung 4 sets Job_OK, rung 7 resets Clear_T and rung 5 starts releasing the WATER batch.

WATCH FOR: CHANGING JOB TOO SOON

Take the 5 s out (Clear_T at 0) and the camera loads the WATER job while the last SODA bottle is still on its way. The last SODA bottles are inspected under WATER and rejected as the wrong product, −50 each for good bottles in Reject_Bin, and their trace records name the wrong batch. In the reference program with Clear_T at 0, six bottles go wrong and the score drops from 3200 to 2150. A bottle that reaches the camera during the half-second load is worse off still: its trigger is missed and it goes by with no image at all.

User data and the trigger

RUNG 10

```
XIC(PE_Trigger)ONS(Trig_ONS)[OTL(Trig_Req),ADD(Bottle_No,1,Bottle_No)];
```

HOW THIS RUNG WORKS

In words. When `PE_Trigger` (Presence eye at the camera: a bottle is in the field of view) is on and it is the first scan they are true (the one-shot, remembered in `Trig_ONS`), the rung latches `Trig_Req` on and makes `Bottle_No` (Bottles that have reached the camera, counted at the trigger eye) = `Bottle_No` + **1**.

Left to right:

1. `XIC(PE_Trigger)` *Examine If Closed* asks: true when `PE_Trigger` (Presence eye at the camera: a bottle is in the field of view) is on.
2. `ONS(Trig_ONS)` *One Shot* asks: true when it is the first scan they are true (the one-shot, remembered in `Trig_ONS`). It passes power for one scan only, the scan its input goes from false to true; its storage bit remembers last scan.
3. A branch of 2 parallel paths: each path is tested on its own and does its own work.

   #### Path 1

   1. `OTL(Trig_Req)` *Output Latch* does: latches `Trig_Req` on.

   #### Path 2

   1. `ADD(Bottle_No,1,Bottle_No)` *Add* does: makes `Bottle_No` (Bottles that have reached the camera, counted at the trigger eye) = `Bottle_No` + **1**. Every scan the rung is true; behind a one-shot it adds once per event.

Why it is built this way:

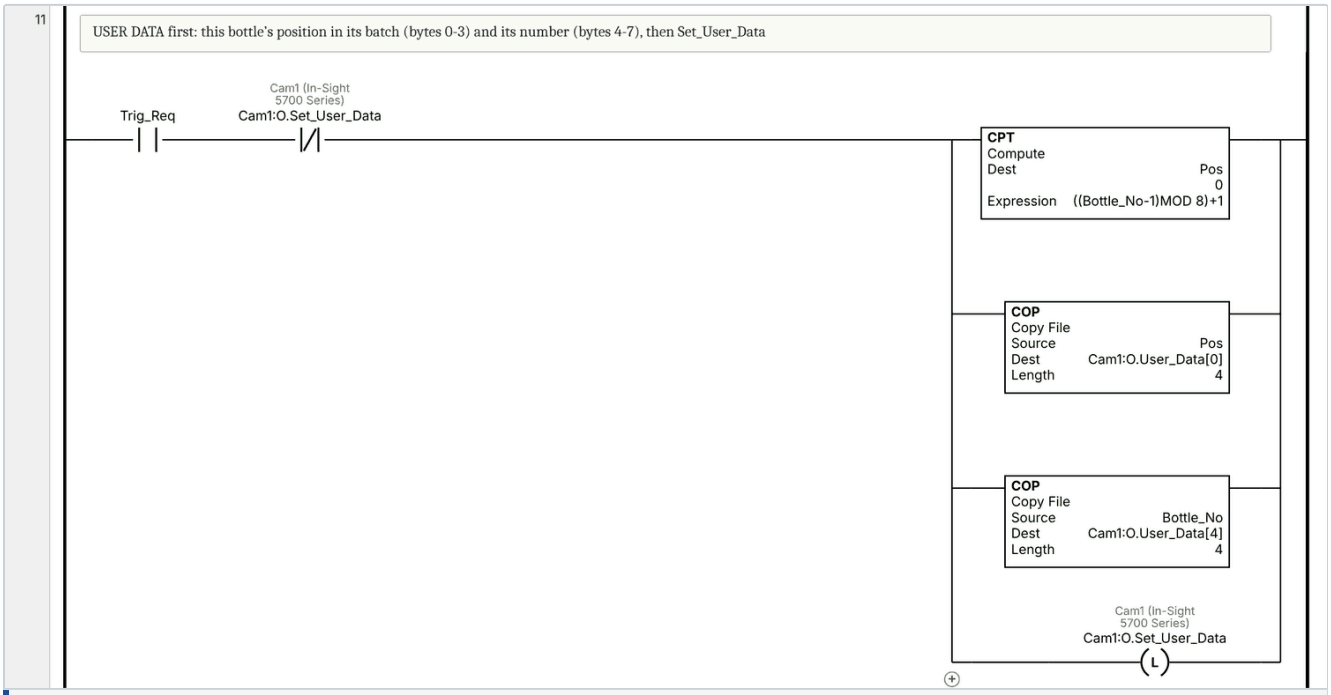
One-shot. Everything after the ONS happens once per event, not on every scan the condition holds; `Trig_ONS` is only its memory of the last scan.

A latch: it stays on until an OTU clears it.

> **Think of it like this:** a doorbell that rings once however long you lean on the button. The ONS looks at what the button was doing last scan: only the change from up to down rings it.

> **Watch for:** Each ONS needs its own storage bit. Two ONS sharing one bit make one of them miss its edges.

RUNG 11




```
XIC(Trig_Req)XIO(Cam1:O.Set_User_Data)[CPT(Pos,((Bottle_No-1) MOD 8)+1),COP(Pos,Cam1:O.User_Data[0],4),COP(Bottle_No,Cam1:O.User_Data[4],4),OTL(Cam1:O.Set_User_Data)];
```

PREDICT IT · 6 `Trig_Req` is on and `Cam1:O.Set_User_Data` is off. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Trig_Req` is on and `Cam1:O.Set_User_Data` is off, the rung computes `Pos` (This bottle's position in its batch, 1-8) = `((Bottle_No-1) MOD 8)+1`, copies 4 elements from `Pos` to `Cam1:O.User_Data[0]`, copies 4 elements from `Bottle_No` (Bottles that have reached the camera, counted at the trigger eye) to `Cam1:O.User_Data[4]` and latches `Cam1:O.Set_User_Data` on.

Left to right:

1. `XIC(Trig_Req)` *Examine If Closed* asks: true when `Trig_Req` is on.
2. `XIO(Cam1:O.Set_User_Data)` *Examine If Open* asks: true when `Cam1:O.Set_User_Data` is off.
3. A branch of 4 parallel paths: each path is tested on its own and does its own work.

Path 1

1. `CPT(Pos,((Bottle_No-1) MOD 8)+1)` *Compute* does: computes `Pos` (This bottle's position in its batch, 1-8) = `((Bottle_No-1) MOD 8)+1`.

Path 2

1. `COP(Pos,Cam1:O.User_Data[0],4)` *Copy File* does: copies 4 elements from `Pos` to `Cam1:O.User_Data[0]`.

Path 3

1. `COP(Bottle_No,Cam1:O.User_Data[4],4)` *Copy File* does: copies 4 elements from `Bottle_No` (Bottles that have reached the camera, counted at the trigger eye) to `Cam1:O.User_Data[4]`.

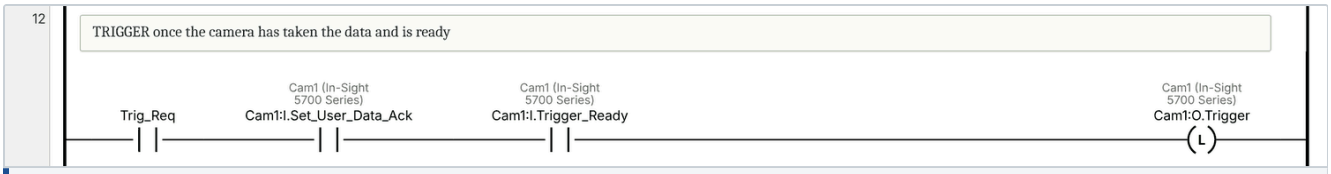
Path 4

1. `OTL(Cam1:O.Set_User_Data)` *Output Latch* does: latches `Cam1:O.Set_User_Data` on.

Why it is built this way:

A latch: it stays on until an OTU clears it.

RUNG 12




```
XIC(Trig_Req)XIC(Cam1:I.Set_User_Data_Ack)XIC(Cam1:I.Trigger_Ready)OTL(Cam1:O.Trigger);
```

PREDICT IT · 7 `Trig_Req` is off, `Cam1:I.Set_User_Data_Ack` is on and `Cam1:I.Trigger_Ready` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Trig_Req` is on, `Cam1:I.Set_User_Data_Ack` is on and `Cam1:I.Trigger_Ready` is on, the rung latches `Cam1:O.Trigger` on.

Left to right:

1. `XIC(Trig_Req)` *Examine If Closed* asks: true when `Trig_Req` is on.
2. `XIC(Cam1:I.Set_User_Data_Ack)` *Examine If Closed* asks: true when `Cam1:I.Set_User_Data_Ack` is on.
3. `XIC(Cam1:I.Trigger_Ready)` *Examine If Closed* asks: true when `Cam1:I.Trigger_Ready` is on.
4. `OTL(Cam1:O.Trigger)` *Output Latch* does: latches `Cam1:O.Trigger` on.

Why it is built this way:

A latch: it stays on until an OTU clears it.

RUNG 13

```
XIC(Cam1:I.Trigger_Ack)
[OTU(Cam1:O.Trigger),OTU(Trig_Req),OTU(Cam1:O.Set_User_Data)];
```

HOW THIS RUNG WORKS

In words. When `Cam1:I.Trigger_Ack` is on, the rung unlatches `Cam1:O.Trigger` (turns it off), unlatches `Trig_Req` (turns it off) and unlatches `Cam1:O.Set_User_Data` (turns it off).

Left to right:

1. `XIC(Cam1:I.Trigger_Ack)` *Examine If Closed* asks: true when `Cam1:I.Trigger_Ack` is on.
2. A branch of 3 parallel paths: each path is tested on its own and does its own work.

Path 1

1. `OTU(Cam1:O.Trigger)` *Output Unlatch* does: unlatches `Cam1:O.Trigger` (turns it off).

Path 2

1. `OTU(Trig_Req)` *Output Unlatch* does: unlatches `Trig_Req` (turns it off).

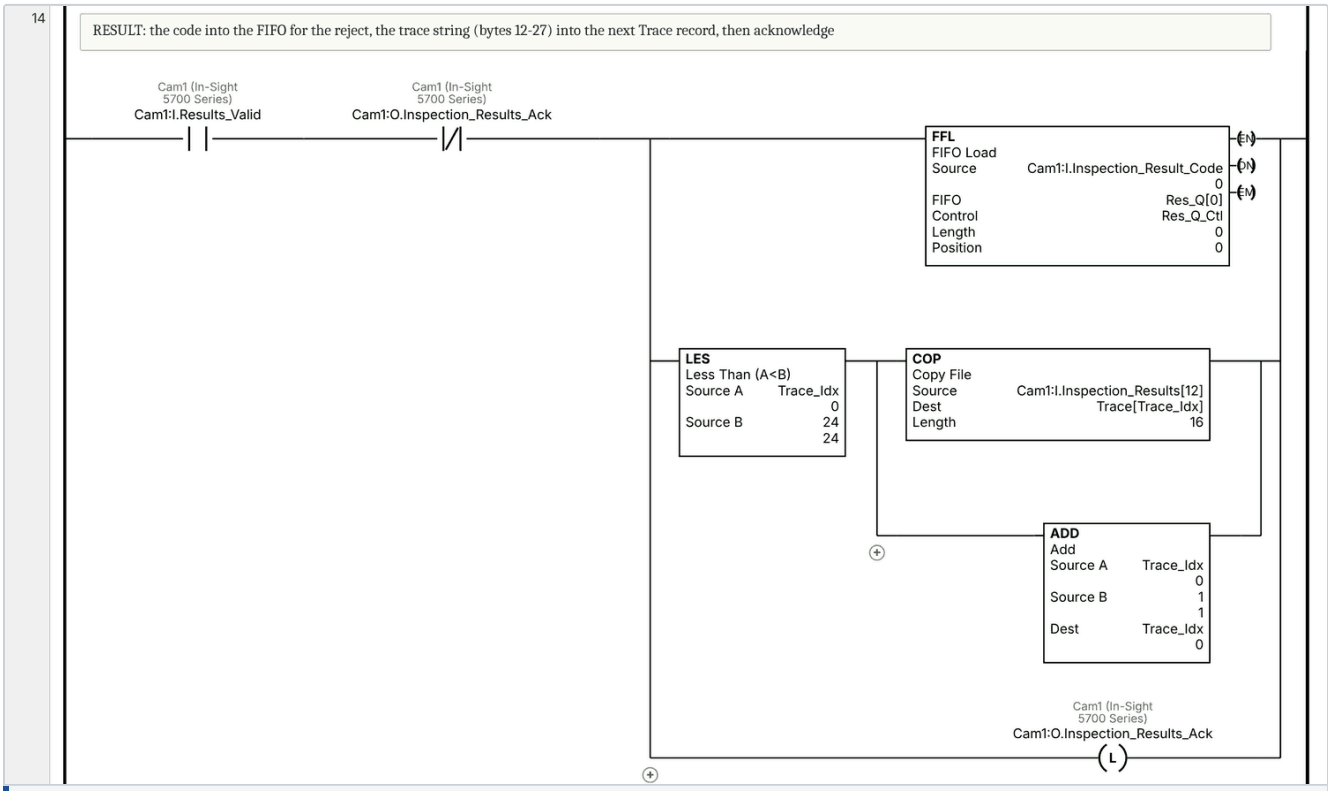
Path 3

1. `OTU(Cam1:O.Set_User_Data)` *Output Unlatch* does: unlatches `Cam1:O.Set_User_Data` (turns it off).

Rung 10 counts the bottle as it reaches the camera, so `Bottle_No` is its number on the line: 1 for the first SODA bottle, 24 for the last JUICE bottle. Rung 11 works out its position in the batch, `((Bottle_No-1) MOD 8)+1`, and copies both numbers into User_Data. Pos is a DINT, so COP with a length of 4 copies its four bytes into User_Data[0] to [3]. Bottle_No goes into [4] to [7]. Rung 12 triggers only after the camera has acknowledged the data, and rung 13 drops both requests when the trigger is acknowledged. The next bottle gets fresh data.

The result and the trace

RUNG 14




```
XIC(Cam1:I.Results_Valid)XIO(Cam1:O.Inspection_Results_Ack)
[FFL(Cam1:I.Inspection_Result_Code,Res_Q[0],Res_Q_Ctl,8),LES(Trace_Idx,24)
[COP(Cam1:I.Inspection_Results[12],Trace[Trace_Idx],16),ADD(Trace_Idx,1,Trace_I
dx)],OTL(Cam1:O.Inspection_Results_Ack)];
```

HOW THIS RUNG WORKS

In words. When `Cam1:I.Results_Valid` is on and `Cam1:O.Inspection_Results_Ack` is off, the rung picks its path: in any case it loads `Cam1:I.Inspection_Result_Code` into the queue `Res_Q[0]`; if `Trace_Idx` (Next free record in Trace) is less than **24**, it copies 16 elements from `Cam1:I.Inspection_Results[12]` to `Trace[Trace_Idx]` and makes `Trace_Idx = Trace_Idx + 1`; in any case it latches `Cam1:O.Inspection_Results_Ack` on.

Left to right:

1. `XIC(Cam1:I.Results_Valid)` *Examine If Closed* asks: true when `Cam1:I.Results_Valid` is on.
2. `XIO(Cam1:O.Inspection_Results_Ack)` *Examine If Open* asks: true when `Cam1:O.Inspection_Results_Ack` is off.
3. A branch of 3 parallel paths: each path is tested on its own and does its own work.

   Path 1

   1. `FFL(Cam1:I.Inspection_Result_Code,Res_Q[0],Res_Q_Ctl,8)` *FIFO Load* does: loads `Cam1:I.Inspection_Result_Code` into the queue `Res_Q[0]`. One entry per false-to-true transition.

   Path 2

   1. `LES(Trace_Idx,24)` *Less Than* asks: true when `Trace_Idx` (Next free record in Trace) is less than **24**.
   2. A branch of 2 parallel paths: each path is tested on its own and does its own work.

Path 1

1. `COP(Cam1:I.Inspection_Results[12],Trace[Trace_Idx],16)` *Copy File* does: copies 16 elements from `Cam1:I.Inspection_Results[12]` to `Trace[Trace_Idx]`.

Path 2

1. `ADD(Trace_Idx,1,Trace_Idx)` *Add* does: makes `Trace_Idx` = `Trace_Idx` + **1**.

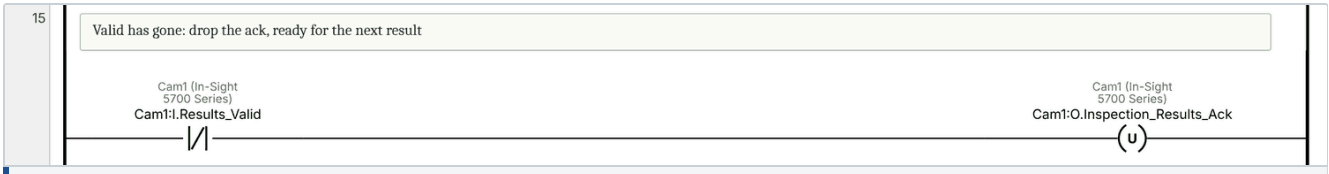
Path 3

1. `OTL(Cam1:O.Inspection_Results_Ack)` *Output Latch* does: latches `Cam1:O.Inspection_Results_Ack` on.

Why it is built this way:

A latch: it stays on until an OTU clears it.

RUNG 15




```
XIO(Cam1:I.Results_Valid)OTU(Cam1:O.Inspection_Results_Ack);
```

HOW THIS RUNG WORKS

In words. When `Cam1:I.Results_Valid` is off, the rung unlatches `Cam1:O.Inspection_Results_Ack` (turns it off).

Left to right:

1. `XIO(Cam1:I.Results_Valid)` *Examine If Open* asks: true when `Cam1:I.Results_Valid` is off.
2. `OTU(Cam1:O.Inspection_Results_Ack)` *Output Unlatch* does: unlatches `Cam1:O.Inspection_Results_Ack` (turns it off).

Rung 14 is Level 8's result rung with one more branch. While `Trace_Idx` is under 24, COP copies the 16 trace bytes into `Trace[Trace_Idx]`, then ADD moves the index on. The index addresses the array at run time, so one rung fills all 24 records in the order the results arrive, which is the order of the bottles. The LES stops it at the end of the array: `Trace[24]` does not exist, and writing to it would be a major fault.

ON A REAL CONTROLLER: COP INTO A STRING

Here, COP into a STRING copies characters and sets LEN, and its length counts characters. A real ControlLogix counts the length in destination *elements*, and an element of a STRING array is a whole 88-byte STRING. `COP(…,Trace[Trace_Idx],16)` on a real controller would copy 16 strings' worth of bytes over the array and fault at the end of it. The real rung copies into the STRING's DATA array and sets its length itself:
`COP(Cam1:I.Inspection_Results[12],Trace[Trace_Idx].DATA[0],16)`
`MOV(16,Trace[Trace_Idx].LEN)`.

The reject pusher, the counts and the fault lamp

RUNG 16

```
XIC(PE_Reject)ONS(Rej_ONS)FFU(Res_Q[0],Cur,Res_Q_Ctl,8);
```

HOW THIS RUNG WORKS

In words. When `PE_Reject` (Presence eye ahead of the reject pusher) is on and it is the first scan they are true (the one-shot, remembered in `Rej_ONS`), the rung unloads the oldest entry of `Res_Q[0]` into `Cur` (The result code of the bottle at the reject eye).

Left to right:

1. `XIC(PE_Reject)` *Examine If Closed* asks: true when `PE_Reject` (Presence eye ahead of the reject pusher) is on.
2. `ONS(Rej_ONS)` *One Shot* asks: true when it is the first scan they are true (the one-shot, remembered in `Rej_ONS`).
3. `FFU(Res_Q[0],Cur,Res_Q_Ctl,8)` *FIFO Unload* does: unloads the oldest entry of `Res_Q[0]` into `Cur` (The result code of the bottle at the reject eye). One entry per false-to-true transition.

Why it is built this way:

One-shot: once per event, not once per scan.

RUNG 17

```
XIC(PE_Reject)ONS(Rej_ONS2)NEQ(Cur,0)OTL(Fire);
```

HOW THIS RUNG WORKS

In words. When `PE_Reject` (Presence eye ahead of the reject pusher) is on, it is the first scan they are true (the one-shot, remembered in `Rej_ONS2`) and `Cur` (The result code of the bottle at the reject eye) is not **0**, the rung latches `Fire` on.

Left to right:

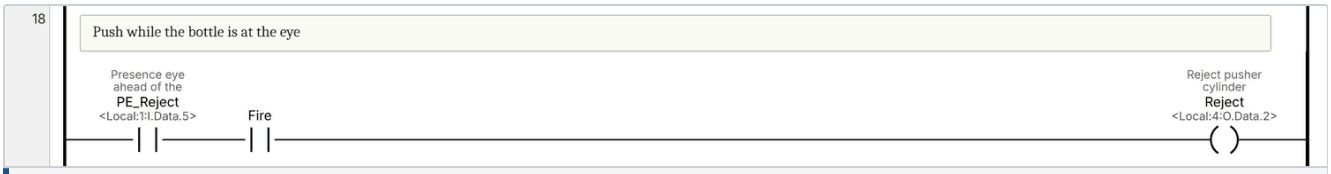
1. `XIC(PE_Reject)` *Examine If Closed* asks: true when `PE_Reject` (Presence eye ahead of the reject pusher) is on.
2. `ONS(Rej_ONS2)` *One Shot* asks: true when it is the first scan they are true (the one-shot, remembered in `Rej_ONS2`).
3. `NEQ(Cur,0)` *Not Equal* asks: true when `Cur` (The result code of the bottle at the reject eye) is not **0**.
4. `OTL(Fire)` *Output Latch* does: latches `Fire` on.

Why it is built this way:

One-shot: once per event, not once per scan.

A latch: it stays on until an OTU clears it.

RUNG 18




```
XIC(PE_Reject)XIC(Fire)OTE(Reject);
```

PREDICT IT · 8 `PE_Reject` is on and `Fire` is off. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `PE_Reject` (Presence eye ahead of the reject pusher) is on and `Fire` is on, the rung turns `Reject` (Reject pusher cylinder) on (and off again whenever the rung is false).

Left to right:

1. `XIC(PE_Reject)` *Examine If Closed* asks: true when `PE_Reject` (Presence eye ahead of the reject pusher) is on.
2. `XIC(Fire)` *Examine If Closed* asks: true when `Fire` is on.
3. `OTE(Reject)` *Output Energize* does: turns `Reject` (Reject pusher cylinder) on (and off again whenever the rung is false).

RUNG 19

```
XIC(PE_Reject)ONS(Cnt_ONS)
[XIC(Fire)ADD(Reject_Count,1,Reject_Count),XIO(Fire)ADD(Pass_Count,1,Pass_Count
)];
```

HOW THIS RUNG WORKS

In words. When `PE_Reject` (Presence eye ahead of the reject pusher) is on and it is the first scan they are true (the one-shot, remembered in `Cnt_ONS`), the rung picks its path: if `Fire` is on, it makes `Reject_Count = Reject_Count + 1`; if `Fire` is off, it makes `Pass_Count = Pass_Count + 1`.

Left to right:

1. `XIC(PE_Reject)` *Examine If Closed* asks: true when `PE_Reject` (Presence eye ahead of the reject pusher) is on.
2. `ONS(Cnt_ONS)` *One Shot* asks: true when it is the first scan they are true (the one-shot, remembered in `Cnt_ONS`).
3. A branch of 2 parallel paths: each path is tested on its own and does its own work.

Path 1

1. `XIC(Fire)` *Examine If Closed* asks: true when `Fire` is on.
2. `ADD(Reject_Count,1,Reject_Count)` *Add* does: makes `Reject_Count = Reject_Count + 1`.

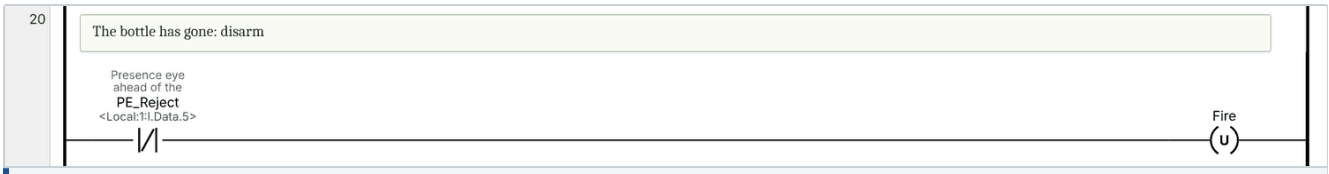
Path 2

1. `XIO(Fire)` *Examine If Open* asks: true when `Fire` is off.
2. `ADD(Pass_Count,1,Pass_Count)` *Add* does: makes `Pass_Count = Pass_Count + 1`.

Why it is built this way:

One-shot: once per event, not once per scan.

RUNG 20




```
XIO(PE_Reject)OTU(Fire);
```

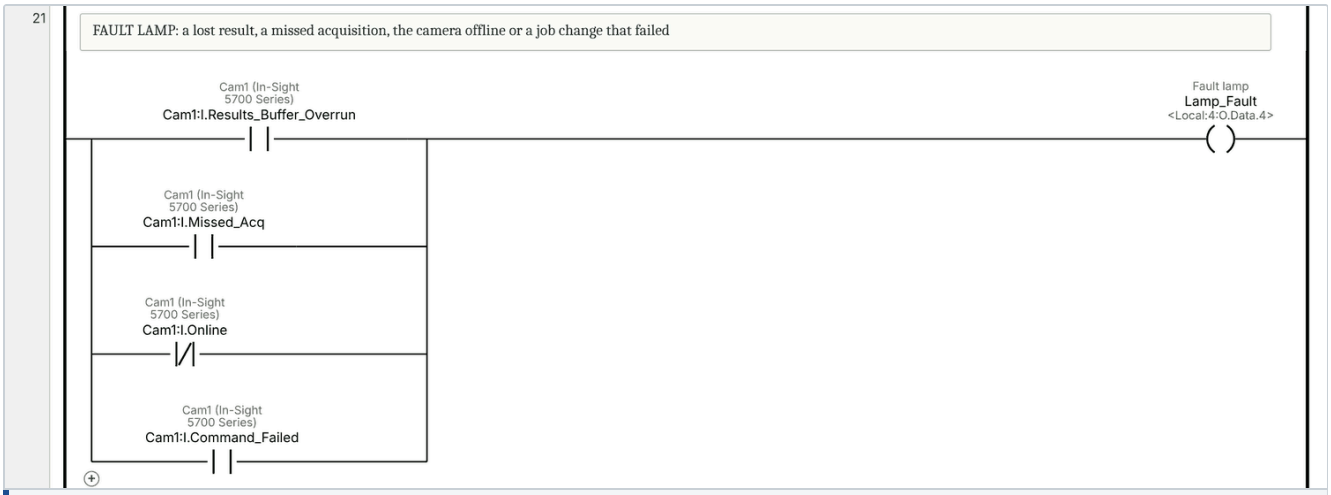
HOW THIS RUNG WORKS

In words. When `PE_Reject` (Presence eye ahead of the reject pusher) is off, the rung unlatches `Fire` (turns it off).

Left to right:

1. `XIO(PE_Reject)` *Examine If Open* asks: true when `PE_Reject` (Presence eye ahead of the reject pusher) is off.
2. `OTU(Fire)` *Output Unlatch* does: unlatches `Fire` (turns it off).

RUNG 21




```
[XIC(Cam1:I.Results_Buffer_Overrun),XIC(Cam1:I.Missed_Acq),XIO(Cam1:I.Online),XIC(Cam1:I.Command_Failed)]OTE(Lamp_Fault);
```

PREDICT IT · 9 `Cam1:I.Results_Buffer_Overrun` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When either (`Cam1:I.Results_Buffer_Overrun` is on) *or* (`Cam1:I.Missed_Acq` is on) *or* (`Cam1:I.Online` is off) *or* (`Cam1:I.Command_Failed` is on), the rung turns `Lamp_Fault` (Fault lamp) on (and off again whenever the rung is false).

Left to right:

1. A branch of 4 parallel paths: power gets through if any one of them is true.
   - **Path 1**
     1. `XIC(Cam1:I.Results_Buffer_Overrun)` *Examine If Closed* asks: true when `Cam1:I.Results_Buffer_Overrun` is on.
   - **Path 2**
     1. `XIC(Cam1:I.Missed_Acq)` *Examine If Closed* asks: true when `Cam1:I.Missed_Acq` is on.
   - **Path 3**
     1. `XIO(Cam1:I.Online)` *Examine If Open* asks: true when `Cam1:I.Online` is off.
   - **Path 4**
     1. `XIC(Cam1:I.Command_Failed)` *Examine If Closed* asks: true when `Cam1:I.Command_Failed` is on.
2. `OTE(Lamp_Fault)` *Output Energize* does: turns `Lamp_Fault` (Fault lamp) on (and off again whenever the rung is false).

> **Think of it like this:** parallel paths are two roads to the same town. If either road is open you get there; the rung only cares that power reaches the far end, not which way it went.

These are Level 8's, with one more fault on the lamp: `Command_Failed`. Ask for job 4, which this camera does not have, and the camera keeps job 1, raises Command_Failed instead of Command_Complete, and the line stays held. The lamp tells the operator why.

Run it

Load your program, Run mode, Start. Watch Product_Sel jump to WATER as the eighth SODA bottle leaves, the hopper hold, Camera_HMI's JOB box change to *Job 2: WATER (min fill 90 %)*, and the WATER batch start. The third WATER bottle has a red label: the camera fails it with code 16, WRONG, and the pusher takes it. In the reference run, 24 of 24 bottles in the right bin, nothing lost, in 52 s against a par of 65 s: 3200 points and three stars, and all 24 trace records right.

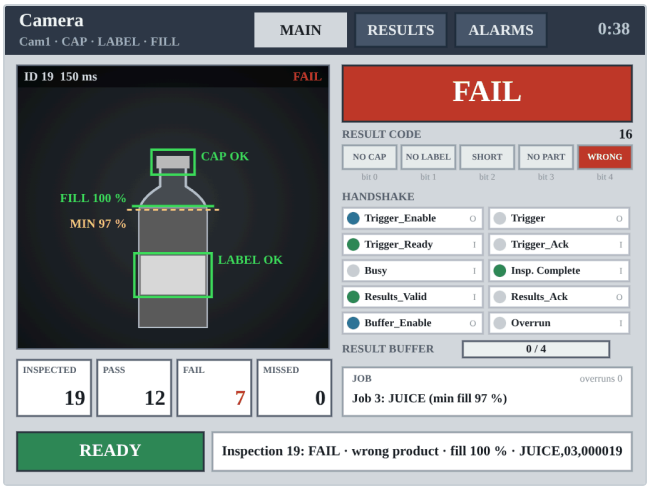


Figure 177. The stray in the JUICE batch: a WATER bottle under the JUICE job. Cap, label and fill all pass. It fails on bit 4, WRONG, and the trace line reads JUICE,03,000019.

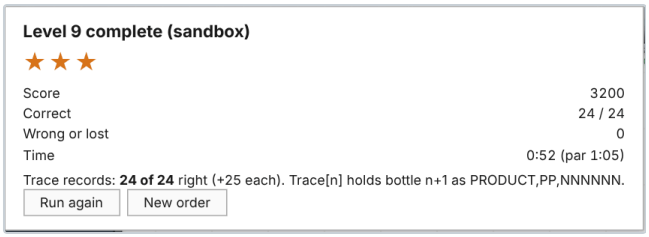


Figure 178. The score card: every bottle right, under par, and the trace records counted.

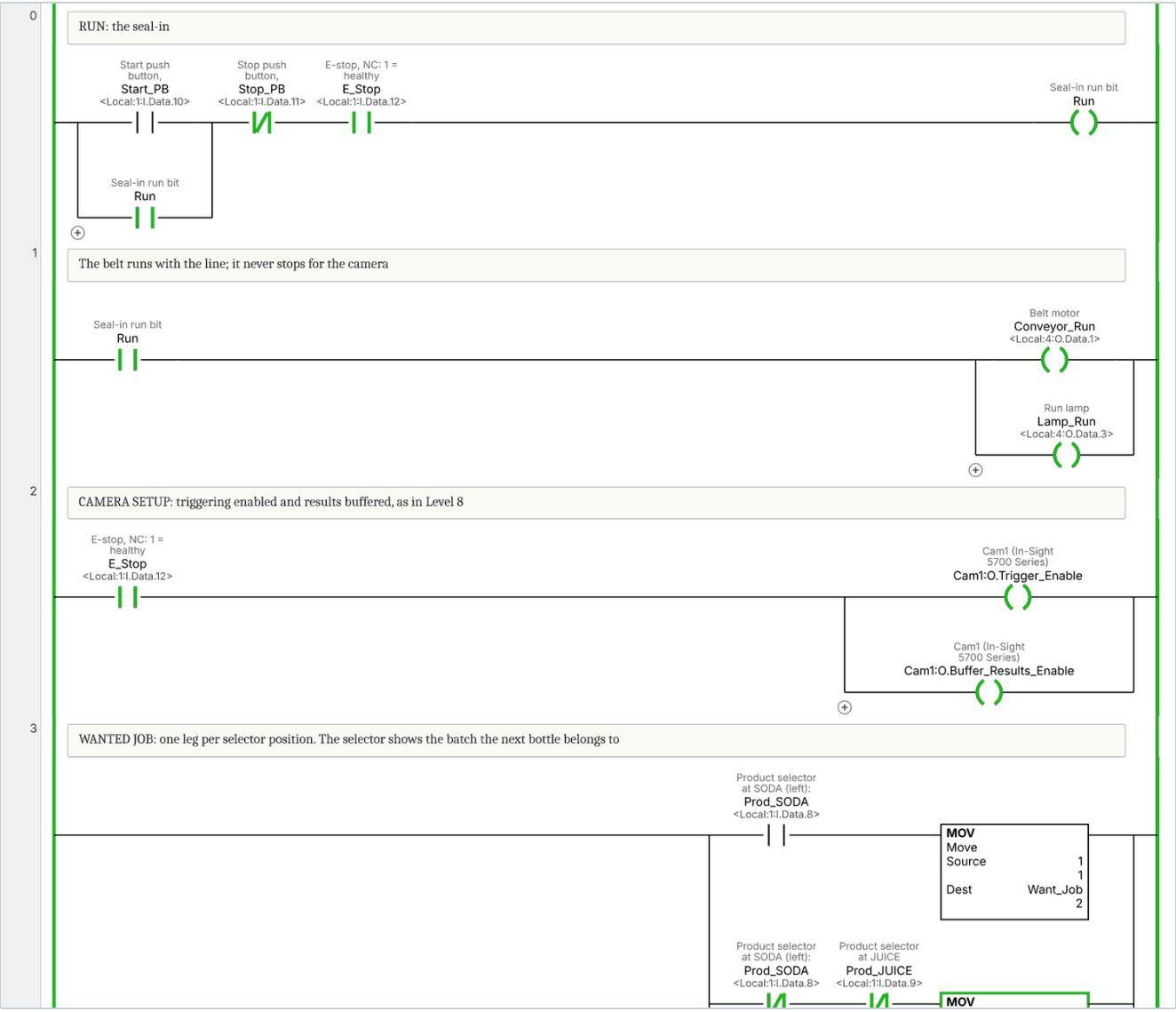


Figure 179. Level 9 online, the top of the routine.

TRY IT: THE WRONG JOB

In rung 3, swap the 1 and the 3. The camera now runs JUICE on the SODA batch and SODA on the JUICE batch, and bit 4 fails every bottle in both, because none of them is the product the job expects. The reject bin fills with good bottles: 19 of the 24 end up there, and the score falls to about 1150. That is the changeover mistake a plant fears most, and the reason a camera job carries the product it expects.

TRY IT: TRIGGER WITHOUT THE DATA

Take `XIC(Cam1:I.Set_User_Data_Ack)` out of rung 12. Rungs 11 and 12 now go true in the same scan, so Set_User_Data and Trigger reach the camera in the same I/O update, and the camera takes the image before it has latched the data. Every record carries the previous bottle's data: `Trace[0]` reads `SODA,00,000000` and `Trace[1]` reads `SODA,01,000001`. The trace score falls to 0 while every bottle still lands in the right bin. A trace can be wrong while the product is right, so check the records as well as the bins.

Card mode on Level 9

Card mode works on Level 9 the same way it does on Level 8 (tick **Card mode** in the Camera's properties): the hopper's bottles wait at the camera and each takes the verdict of the card under the lens. What Level 9 adds is the product. A card is `SN-48RK`, R the product and K the defect, and the camera answers **WRONG** (bit 4) when the card's product is not the loaded job's. The bottle becomes what the card shows: under the SODA job a WATER card is a WATER bottle in the SODA batch, and it is rejected as a stray.

So the cards are swapped at the changeover, as an operator would swap product: when the camera loads WATER, put a WATER card under the lens. To give the camera its stray, show it the old product's card for a bottle or two after the change; the level's stray check is met when one of those is rejected. The CARDS page opens on the loaded job's product, marks it on its tab (JOB: at the top right) and lists what each card of the row should read under that job.

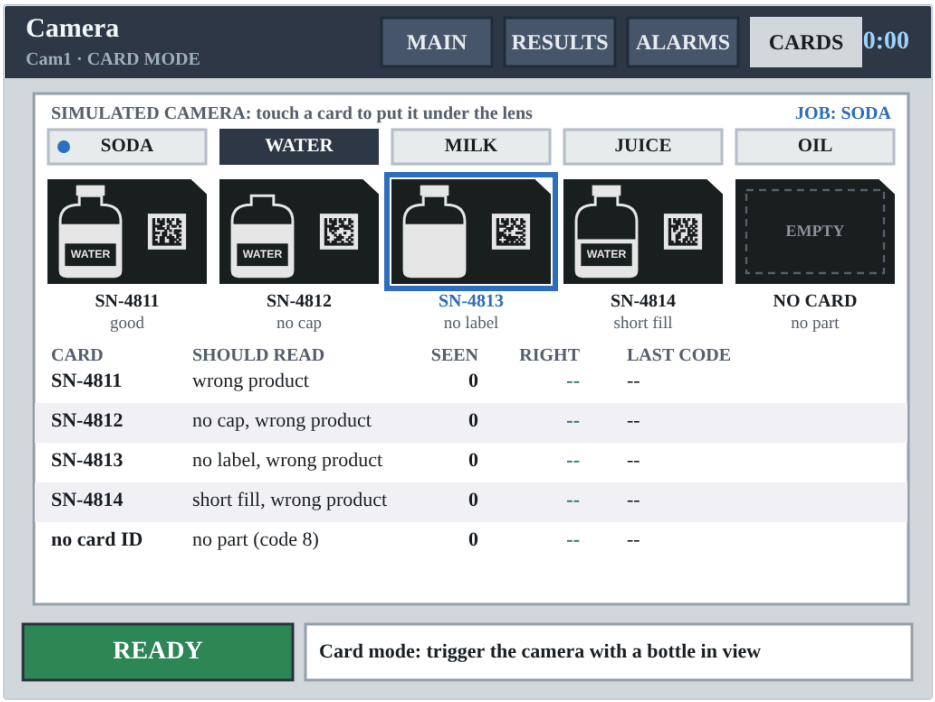


Figure 180. CARDS on Level 9: the SODA job is loaded, so every WATER card should read WRONG on top of its own defect.

With the camera live, the real job decides. The page reads the card from byte 10 (R × 10 + K) and the loaded job from `Current_Job_ID`, and counts a result right when the code matches the card's defect plus WRONG for a card of another product. Your In-Sight job does the same: decode the DataMatrix for R and K, read the label with OCR, and set bit 4 when the text is not the job's product.

> **TRY IT: RUN LEVEL 9 ON CARDS**
>
> Tick Card mode with SN-4801 under the lens and run your Level 9 program. When Product_Sel turns to WATER and the camera loads job 2, leave SN-4801 there for two more bottles: both read WRONG and go to the reject bin. Then touch the WATER tab and SN-4811, and at the JUICE change SN-4831. The level completes, and the score card lists SN-4801 with its WRONG reads counted as right.

The twelve levels, solved

Level	Program	Bottles	Par	Time	Score
1 Height sort	latch, one at a time, 16 rungs	10	150 s	94 s	1200
2 Fill by label	latch, one at a time, 20 rungs	10	170 s	104 s	1200
3 Color sort	latch, one at a time, 20 rungs	12	190 s	125 s	1400
4 Two on the line	zone queues, 43 rungs	14	110 s	85 s	1600
5 Full sort	zone queues, 53 rungs	16	120 s	95 s	1800
6 Pack it	zone queues and a case lane, 39 rungs	16	140 s	111 s	1800
7 Batch Mix	step sequencer, 24 rungs	3 batches	240 s	179 s	1100
8 Vision check	camera handshake and a result FIFO, 16 rungs	20	45 s	36 s	2200
9 Changeover	camera job changes, user data and a trace, 22 rungs	24 in 3 batches	65 s	52 s	3200
10 Servo pick	FHPP handshake and a gripper, 15 rungs	12	80 s	62 s	1400
11 Date code	marker message, lot FIFOs and code verify, 24 rungs	18 in 3 lots	45 s	34 s	2000
12 Kitchen changeover	step sequencer with a changeover branch, 31 rungs	4 batches	270 s	218 s	1400

Every row is a run of the program printed in its chapter, made by the page's own test suite on this version. The zone-queue program solves Levels 1-3 as well, faster; File → Load Reference Solution installs the right one for any level.

Predict it: the answers

1. The rung is false at `XIO(Stop_PB)`: `Stop_PB` is on, so `Run` is **off** (an OTE writes 0 when its rung is false).
2. Every condition on the way is met, so `Want_Job` becomes **2**.

3. The rung is false at the EQU: `Cam1:I.Current_Job_ID` is not `Want_Job`, so `Job_OK` is **off** (an OTE writes 0 when its rung is false).
4. Every condition on the way is met, so `Rel_T` times toward its preset.
5. Every condition on the way is met, so `Cam1:O.Execute_Command` is unlatched, **off**.
6. Every condition on the way is met, so the CPT writes `Pos`, the COP writes **4** and `Cam1:O.Set_User_Data` latches **on**.
7. The rung is false at `XIC(Trig_Req)`: `Trig_Req` is off, so the OTL does nothing: `Cam1:O.Trigger` keeps whatever it had.
8. The rung is false at `XIC(Fire)`: `Fire` is off, so `Reject` is **off** (an OTE writes 0 when its rung is false).
9. Every condition on the way is met, so `Lamp_Fault` is **on**.

42 Level 10 — Servo pick

> **IN THE APP**
>
> File → New Project… → A graded level → **10**. Guided mode opens on this chapter: the tags, every rung below under its fold, a Verify check, and the run results: the axis is homed, a bottle is picked up, a bottle is set down in a bin, then complete, every bottle right and under par. **File → Load Reference Solution** loads the program printed here.

The axis chapter in Part VI (chapter 46) taught the FHPP handshake on a panel. This level puts it to work. Twelve bottles, small, medium and large, come down a short infeed one at a time. The belt ends under a servo axis standing across it: a Size 120 toothed-belt axis with a 4.5 m stroke and a **vacuum gripper** on its carriage. Three bins sit under the axis at 1500, 3000 and 4500 mm, one per size. Pick each bottle off the end of the belt, carry it to its bin, set it down, come back. The drive's own rules are the level's traps: no move before the axis is homed, no second Start while a move is running, and a bottle set down anywhere but over a bin is on the floor.

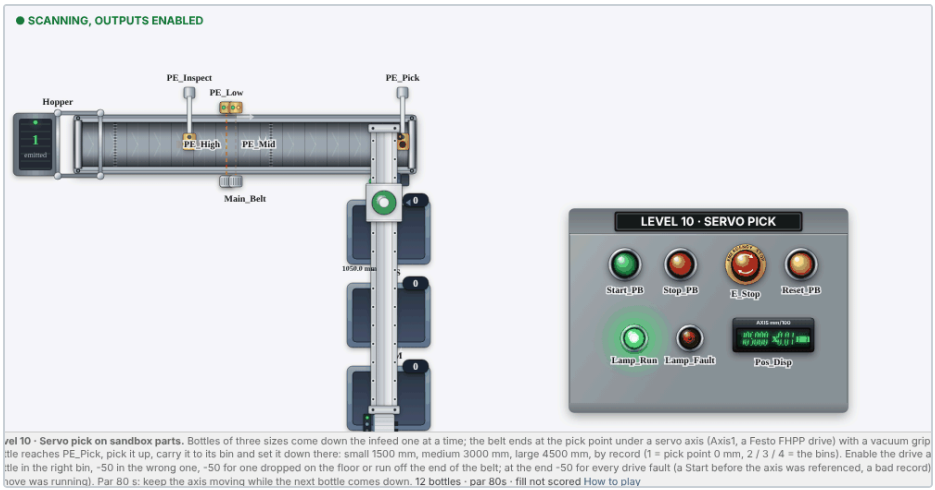


Figure 181. Level 10 in the top view: the infeed with its height gantry, PE_Pick at the end of the belt, the axis across the belt end with a bottle in its gripper on the way to Bin_S, the three bins under the axis, and the console.

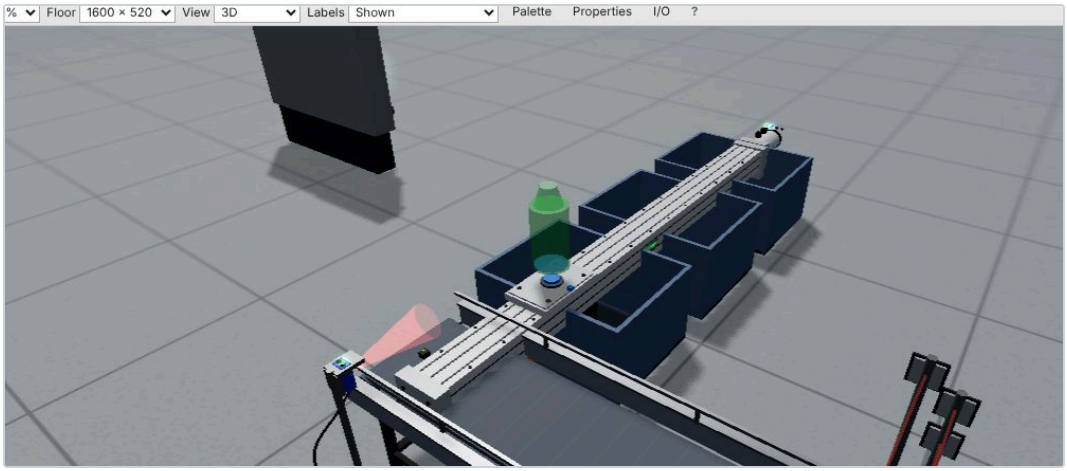


Figure 182. In 3D: the axis stands over the belt end on its floor stand; the carriage carries the bottle on the gripper pad above the bins.

The machine

- **Hopper** and **Main_Belt** (6 m): one bottle per rising edge of `Infeed_Release`, when `Infeed_Ready` says the exit is clear. The belt runs on `Conveyor_Run` and ends 30 px past the axis: a bottle that reaches the end on a running belt falls off and is lost.
- **PE_Low, PE_Mid, PE_High**: Level 1's height gantry. Every bottle breaks the low beam, medium and large the middle one, large alone the high one.
- **PE_Pick**: a short-range eye at the end of the belt. A bottle it sees is standing under the carriage at 0 mm.
- **Axis**: a Linear Axis, Size 120, stroke 4500 mm, 3000 mm/s, 20 m/s², tied to the module `Axis1` (a Festo FHPP drive, simulated). Its records are `0@100, 1500@100, 3000@100, 4500@100`: record 1 is the pick point, 2, 3 and 4 the bins. Its gripper gives three wired signals: `Grip` out, `Gripped` and `Part_Below` back.
- **Bin_S, Bin_M, Bin_L**: graded bins under the axis at 1500, 3000 and 4500 mm. A small bottle in Bin_S is +100; a medium one there is −50.
- **Console**: Start, Stop, E-stop, **Reset** for the drive, the run and fault lamps, and the axis position in hundredths of a millimetre.

How the level is scored

What happens	Points
A bottle set down in its own bin	+100
A bottle in the wrong bin	−50
A bottle set down off the bins, or run off the end of the belt	−50
Every drive fault at the end (a Start before Referenced, a record that is not set, a target past the stroke)	−50 each
Every double start at the end (a Start while a move was running: the drive ignores it and raises Warning)	−25 each

What happens	Points
All twelve bottles done under par (80 s)	+200

The drive keeps the count: the axis properties show faults and double starts as you run, and the simulator log prints the *Drive record* line when the level completes.

The gripper

`Grip` is the vacuum. On its rising edge the gripper takes whatever bottle is under the carriage (`Part_Below` says there is one) and `Gripped` comes on; the bottle rides with the carriage and no eye, belt or bin sees it on the way. Drop `Grip` and the bottle is set down where the carriage is: into the bin under it, or onto the floor. Grip with nothing under the carriage does nothing but a line in the log. There is no Z axis here: the pick and the set-down are the vacuum alone, which is how a short-stroke vacuum pick on a gantry often is.

How to pass it, step by step

1. Power the drive with Run: Enable, Stop and Halt high, as in the axis chapter.
2. Home it before anything else, and only release bottles once `Axis1:I.Referenced` is on.
3. Run the belt until PE_Pick sees a bottle, then stop it, or the bottle runs off the end.
4. Latch the gantry beams as the bottle passes, as Level 1 did.
5. Grip when a bottle is at the pick point and the carriage is at rest at 0 mm. Pick the record from the height. Start once.
6. Over the bin, at rest: Grip off. Then record 1 and Start again, back to the pick point.
7. Drop Start when Start_Ack answers, every time.

Tags this lesson needs

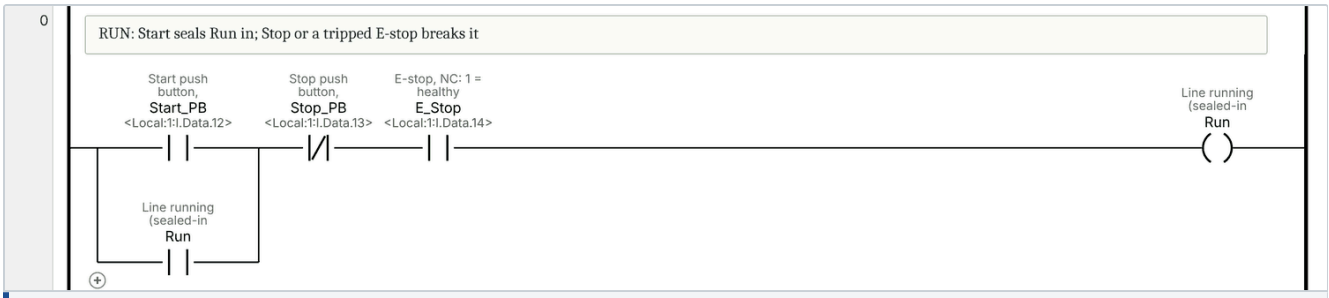
The level's alias tags are already made for you (chapter 32's table). Create these plain tags before you type the rungs; Verify reports any you miss.

Tag	Type	How to make it	Description
`Run`	BOOL	Edit Tags: name, type	Line running (sealed-in Start)
`Busy`	BOOL	Edit Tags: name, type	A bottle is on the belt or in the gripper
`Is_Mid`	BOOL	Edit Tags: name, type	The bottle broke the middle beam (medium or large)
`Is_High`	BOOL	Edit Tags: name, type	The bottle broke the high beam (large)
`Rel_ONS`	BOOL	Edit Tags: name, type	One-shot storage for the release
`Out_ONS`	BOOL	Edit Tags: name, type	One-shot storage for the move to the bin

Tag	Type	How to make it	Description
`Back_ONS`	BOOL	Edit Tags: name, type	One-shot storage for the move back

Run, power and home

RUNG 0




```
[XIC(Start_PB),XIC(Run)]XIO(Stop_PB)XIC(E_Stop)OTE(Run);
```

PREDICT IT · 1 `Start_PB` is on, `Stop_PB` is on and `E_Stop` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When either (`Start_PB` (Start push button, momentary) is on) *or* (`Run` (Line running (sealed-in Start)) is on), `Stop_PB` (Stop push button, momentary) is off and `E_Stop` (E-stop, NC: 1 = healthy) is on, the rung turns `Run` on (and off again whenever the rung is false).

Left to right:

1. A branch of 2 parallel paths: power gets through if any one of them is true.
   - **Path 1**
     1. `XIC(Start_PB)` *Examine If Closed* asks: true when `Start_PB` (Start push button, momentary) is on.
   - **Path 2**
     1. `XIC(Run)` *Examine If Closed* asks: true when `Run` (Line running (sealed-in Start)) is on.
2. `XIO(Stop_PB)` *Examine If Open* asks: true when `Stop_PB` (Stop push button, momentary) is off.
3. `XIC(E_Stop)` *Examine If Closed* asks: true when `E_Stop` (E-stop, NC: 1 = healthy) is on.
4. `OTE(Run)` *Output Energize* does: turns `Run` on (and off again whenever the rung is false). Every scan it writes the rung state: true on, false off.

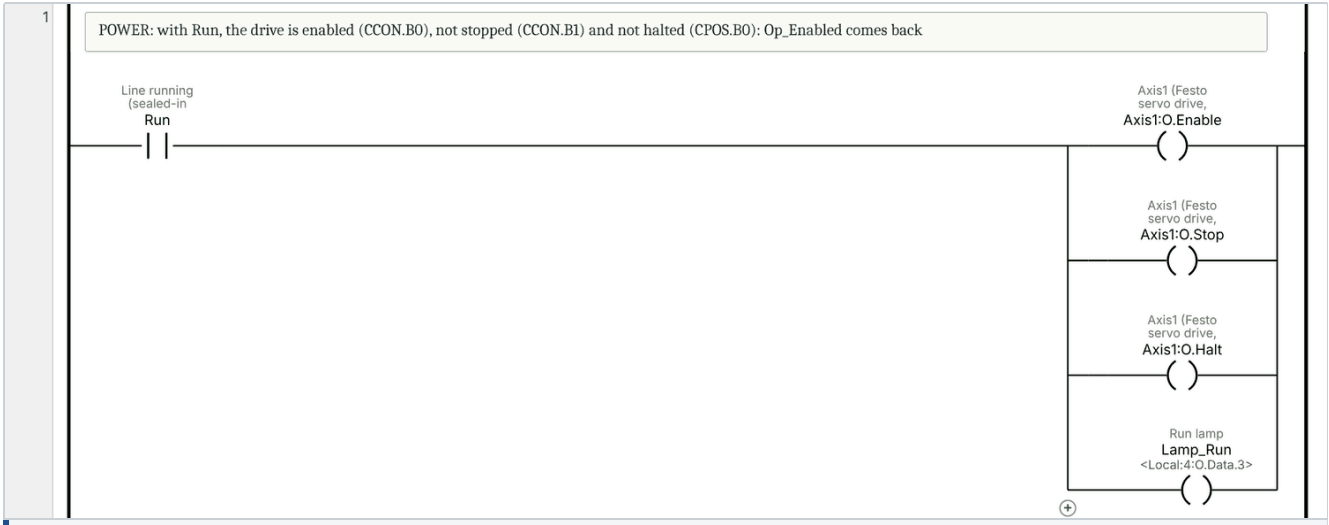
Why it is built this way:

Seal-in. The start condition turns `Run` on, and `Run` 's own contact in the parallel path keeps the rung true after the start goes away. Anything in series outside the branch (a stop, a fault) breaks it, and then it stays off until the start is made again.

> **Think of it like this:** a doorbell button and a hand. The start button is the doorbell, pressed for a moment. The seal-in contact is a hand that grabs the circuit as soon as the output comes on and keeps holding after you let go. The stop button, in series, is the only thing that can knock that hand away.

> **Watch for:** The stop has to be in series outside the branch. Put it inside the start path only and the seal-in path bypasses it: the stop no longer stops.

RUNG 1




```
XIC(Run)
[OTE(Axis1:O.Enable),OTE(Axis1:O.Stop),OTE(Axis1:O.Halt),OTE(Lamp_Run)];
```

HOW THIS RUNG WORKS

In words. When `Run` (Line running (sealed-in Start)) is on, the rung turns `Axis1:O.Enable` on (and off again whenever the rung is false), turns `Axis1:O.Stop` on (and off again whenever the rung is false), turns `Axis1:O.Halt` on (and off again whenever the rung is false) and turns `Lamp_Run` (Run lamp) on (and off again whenever the rung is false).

Left to right:

1. `XIC(Run)` *Examine If Closed* asks: true when `Run` (Line running (sealed-in Start)) is on.
2. A branch of 4 parallel paths: each path is tested on its own and does its own work.

Path 1

1. `OTE(Axis1:O.Enable)` *Output Energize* does: turns `Axis1:O.Enable` on (and off again whenever the rung is false).

Path 2

1. `OTE(Axis1:O.Stop)` *Output Energize* does: turns `Axis1:O.Stop` on (and off again whenever the rung is false).

Path 3

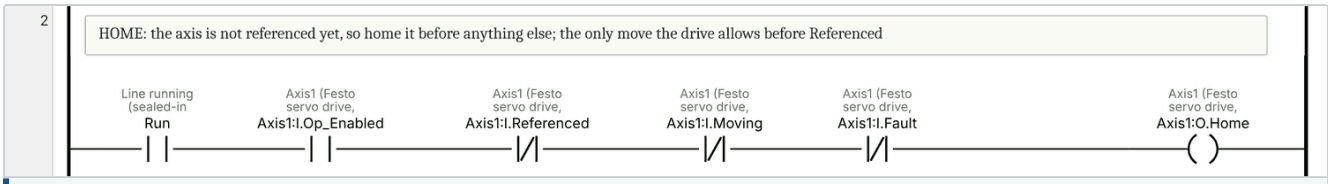
1. `OTE(Axis1:O.Halt)` *Output Energize* does: turns `Axis1:O.Halt` on (and off again whenever the rung is false).

Path 4

1. `OTE(Lamp_Run)` *Output Energize* does: turns `Lamp_Run` (Run lamp) on (and off again whenever the rung is false).

> **Think of it like this:** a rung is a pipe with valves in it, read left to right. Every contact is a valve: XIC is open while its bit is 1, XIO while its bit is 0. Water reaches the output at the right only if every valve in a row is open. The output does not remember anything: on every scan it is simply wet or dry.

RUNG 2




```
XIC(Run)XIC(Axis1:I.Op_Enabled)XIO(Axis1:I.Referenced)XIO(Axis1:I.Moving)XIO(Axis1:I.Fault)OTE(Axis1:O.Home);
```

PREDICT IT · 2 `Run` is on, `Axis1:I.Op_Enabled` is off, `Axis1:I.Referenced` is off, `Axis1:I.Moving` is off and `Axis1:I.Fault` is off. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Run` (Line running (sealed-in Start)) is on, `Axis1:I.Op_Enabled` is on, `Axis1:I.Referenced` is off, `Axis1:I.Moving` is off and `Axis1:I.Fault` is off, the rung turns `Axis1:O.Home` on (and off again whenever the rung is false).

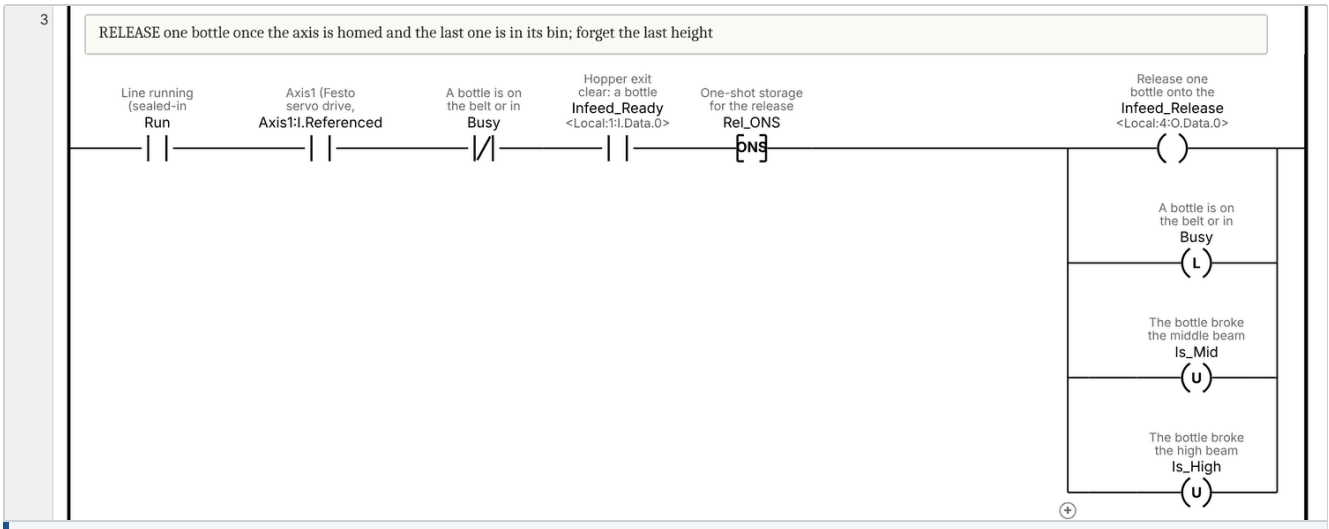
Left to right:

1. `XIC(Run)` *Examine If Closed* asks: true when `Run` (Line running (sealed-in Start)) is on.
2. `XIC(Axis1:I.Op_Enabled)` *Examine If Closed* asks: true when `Axis1:I.Op_Enabled` is on.
3. `XIO(Axis1:I.Referenced)` *Examine If Open* asks: true when `Axis1:I.Referenced` is off.
4. `XIO(Axis1:I.Moving)` *Examine If Open* asks: true when `Axis1:I.Moving` is off.
5. `XIO(Axis1:I.Fault)` *Examine If Open* asks: true when `Axis1:I.Fault` is off.
6. `OTE(Axis1:O.Home)` *Output Energize* does: turns `Axis1:O.Home` on (and off again whenever the rung is false).

Rung 2 is the automatic home: while Run is on and the axis is not referenced, Home is held. The drive homes on Home's rising edge and the rung drops out when Referenced comes back. XIO(Moving) and XIO(Fault) keep it from pulsing Home during a run or into a faulted drive.

Release, belt and gantry

RUNG 3




```
XIC(Run)XIC(Axis1:I.Referenced)XIO(Busy)XIC(Infeed_Ready)ONS(Rel_ONS)
[OTE(Infeed_Release),OTL(Busy),OTU(Is_Mid),OTU(Is_High)];
```

HOW THIS RUNG WORKS

In words. When `Run` (Line running (sealed-in Start)) is on, `Axis1:I.Referenced` is on, `Busy` (A bottle is on the belt or in the gripper) is off, `Infeed_Ready` (Hopper exit clear: a bottle can be released) is on and it is the first scan they are true (the one-shot, remembered in `Rel_ONS` (One-shot storage for the release)), the rung turns `Infeed_Release` (Release one bottle onto the belt (rising edge)) on (and off again whenever the rung is false), latches `Busy` on, unlatches `Is_Mid` (The bottle broke the middle beam (medium or large)) (turns it off) and unlatches `Is_High` (The bottle broke the high beam (large)) (turns it off).

Left to right:

1. `XIC(Run)` *Examine If Closed* asks: true when `Run` (Line running (sealed-in Start)) is on.
2. `XIC(Axis1:I.Referenced)` *Examine If Closed* asks: true when `Axis1:I.Referenced` is on.
3. `XIO(Busy)` *Examine If Open* asks: true when `Busy` (A bottle is on the belt or in the gripper) is off.
4. `XIC(Infeed_Ready)` *Examine If Closed* asks: true when `Infeed_Ready` (Hopper exit clear: a bottle can be released) is on.
5. `ONS(Rel_ONS)` *One Shot* asks: true when it is the first scan they are true (the one-shot, remembered in `Rel_ONS` (One-shot storage for the release)). It passes power for one scan only, the scan its input goes from false to true; its storage bit remembers last scan.
6. A branch of 4 parallel paths: each path is tested on its own and does its own work.

   Path 1

   1. `OTE(Infeed_Release)` *Output Energize* does: turns `Infeed_Release` (Release one bottle onto the belt (rising edge)) on (and off again whenever the rung is false).

   Path 2

   1. `OTL(Busy)` *Output Latch* does: latches `Busy` on. It only ever sets the bit; the bit stays on after the rung goes false.

   Path 3

   1. `OTU(Is_Mid)` *Output Unlatch* does: unlatches `Is_Mid` (The bottle broke the middle beam (medium or large)) (turns it off). It only ever clears the bit.

Path 4

1. `OTU(Is_High)` *Output Unlatch* does: unlatches `Is_High` (The bottle broke the high beam (large)) (turns it off).

Why it is built this way:

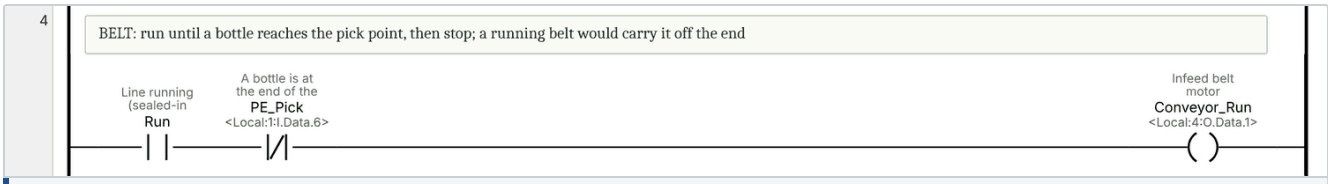
One-shot. Everything after the ONS happens once per event, not on every scan the condition holds; `Rel_ONS` is only its memory of the last scan.

A latch. `Busy` stays on after this rung goes false; an `OTU` elsewhere in the program clears it.

> **Think of it like this:** a doorbell that rings once however long you lean on the button. The ONS looks at what the button was doing last scan: only the change from up to down rings it.

> **Watch for:** Each ONS needs its own storage bit. Two ONS sharing one bit make one of them miss its edges.

RUNG 4




```
XIC(Run)XIO(PE_Pick)OTE(Conveyor_Run);
```

PREDICT IT · 3 `Run` is off and `PE_Pick` is off. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

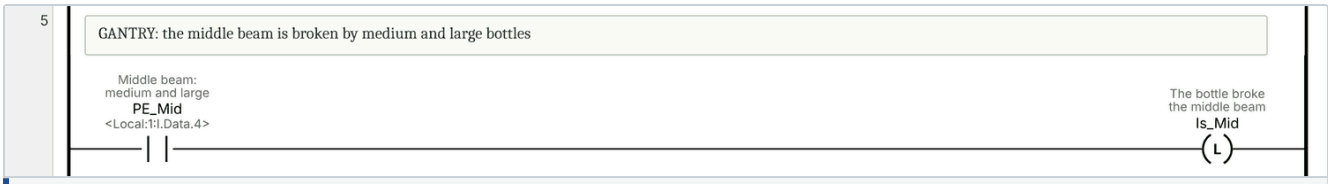
HOW THIS RUNG WORKS

In words. When `Run` (Line running (sealed-in Start)) is on and `PE_Pick` (A bottle is at the end of the belt, under the axis at 0 mm) is off, the rung turns `Conveyor_Run` (Infeed belt motor) on (and off again whenever the rung is false).

Left to right:

1. `XIC(Run)` *Examine If Closed* asks: true when `Run` (Line running (sealed-in Start)) is on.
2. `XIO(PE_Pick)` *Examine If Open* asks: true when `PE_Pick` (A bottle is at the end of the belt, under the axis at 0 mm) is off.
3. `OTE(Conveyor_Run)` *Output Energize* does: turns `Conveyor_Run` (Infeed belt motor) on (and off again whenever the rung is false).

RUNG 5




```
XIC(PE_Mid)OTL(Is_Mid);
```

HOW THIS RUNG WORKS

In words. When `PE_Mid` (Middle beam: medium and large) is on, the rung latches `Is_Mid` (The bottle broke the middle beam (medium or large)) on.

Left to right:

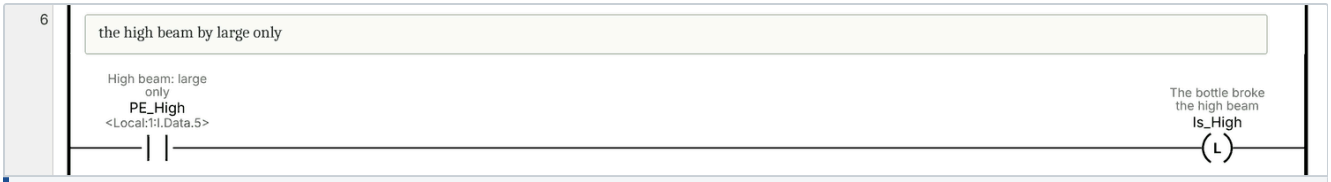
1. `XIC(PE_Mid)` *Examine If Closed* asks: true when `PE_Mid` (Middle beam: medium and large) is on.
2. `OTL(Is_Mid)` *Output Latch* does: latches `Is_Mid` (The bottle broke the middle beam (medium or large)) on.

Why it is built this way:

A latch: it stays on until an OTU clears it.

> **Watch for:** A latched bit survives the condition that set it: every way in needs a way out, or it stays on for good.

RUNG 6




```
XIC(PE_High)OTL(Is_High);
```

HOW THIS RUNG WORKS

In words. When `PE_High` (High beam: large only) is on, the rung latches `Is_High` (The bottle broke the high beam (large)) on.

Left to right:

1. `XIC(PE_High)` *Examine If Closed* asks: true when `PE_High` (High beam: large only) is on.
2. `OTL(Is_High)` *Output Latch* does: latches `Is_High` (The bottle broke the high beam (large)) on.

Why it is built this way:

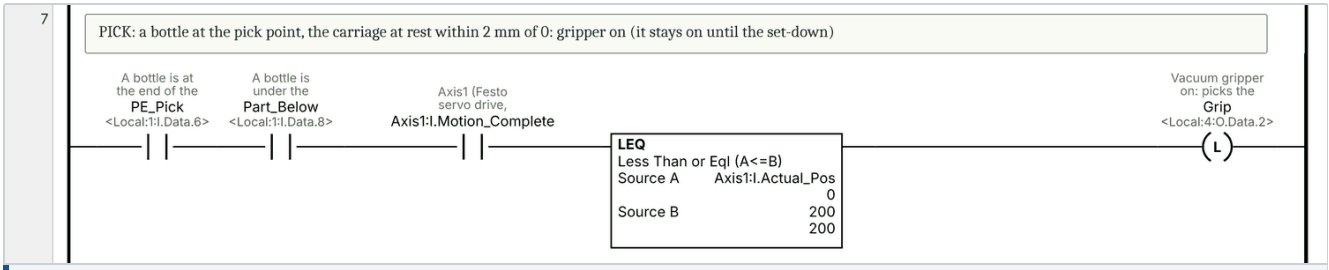
A latch: it stays on until an OTU clears it.

WATCH FOR: THE BELT THAT NEVER STOPS

Leave `XIO(PE_Pick)` out of rung 4 and the first bottle runs straight off the end of the belt: −50 and a lost bottle. Rung 3's XIC(Axis1:I.Referenced) matters as much: release before the home run is done and the bottle waits at PE_Pick while the axis finds its switch, which is harmless here, but on a real machine the home run would sweep a gripper over a bottle that should not be there.

Pick, record, out

RUNG 7




```
XIC(PE_Pick)XIC(Part_Below)XIC(Axis1:I.Motion_Complete)LEQ(Axis1:I.Actual_Pos,200)OTL(Grip);
```

PREDICT IT · 4 `PE_Pick` is on, `Part_Below` is on, `Axis1:I.Motion_Complete` is on and `Axis1:I.Actual_Pos` is 200. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `PE_Pick` (A bottle is at the end of the belt, under the axis at 0 mm) is on, `Part_Below` (A bottle is under the carriage, within the gripper's reach) is on, `Axis1:I.Motion_Complete` is on and `Axis1:I.Actual_Pos` is at most **200**, the rung latches `Grip` (Vacuum gripper on: picks the bottle under the carriage and holds it while 1) on.

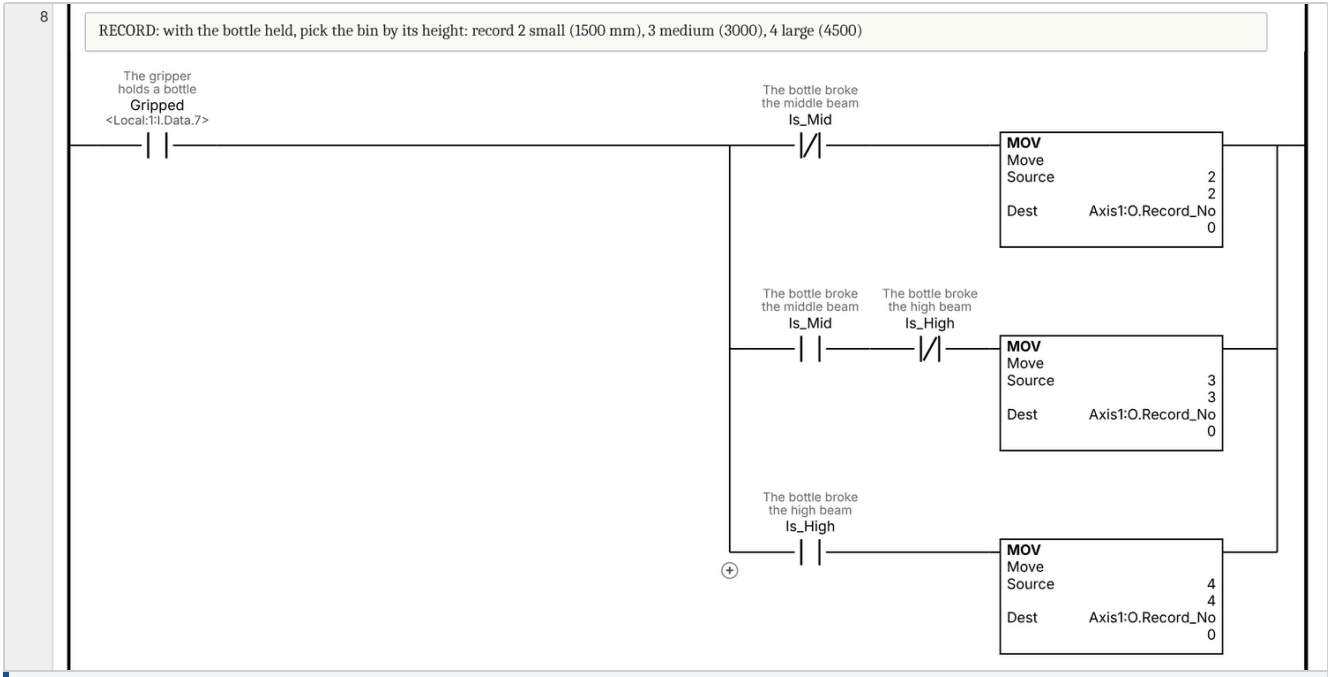
Left to right:

1. `XIC(PE_Pick)` *Examine If Closed* asks: true when `PE_Pick` (A bottle is at the end of the belt, under the axis at 0 mm) is on.
2. `XIC(Part_Below)` *Examine If Closed* asks: true when `Part_Below` (A bottle is under the carriage, within the gripper's reach) is on.
3. `XIC(Axis1:I.Motion_Complete)` *Examine If Closed* asks: true when `Axis1:I.Motion_Complete` is on.
4. `LEQ(Axis1:I.Actual_Pos,200)` *Less Than or Equal* asks: true when `Axis1:I.Actual_Pos` is at most **200**.
5. `OTL(Grip)` *Output Latch* does: latches `Grip` (Vacuum gripper on: picks the bottle under the carriage and holds it while 1) on.

Why it is built this way:

A latch: it stays on until an OTU clears it.

RUNG 8




```
XIC(Gripped)
[XIO(Is_Mid)MOV(2,Axis1:O.Record_No),XIC(Is_Mid)XIO(Is_High)MOV(3,Axis1:O.Record_No),XIC(Is_High)MOV(4,Axis1:O.Record_No)];
```

HOW THIS RUNG WORKS

In words. When `Gripped` (The gripper holds a bottle) is on, the rung picks its path: if `Is_Mid` (The bottle broke the middle beam (medium or large)) is off, it puts **2** into `Axis1:O.Record_No`; if `Is_Mid` is on and `Is_High` (The bottle broke the high beam (large)) is off, it puts **3** into `Axis1:O.Record_No`; if `Is_High` is on, it puts **4** into `Axis1:O.Record_No`.

Left to right:

1. `XIC(Gripped)` *Examine If Closed* asks: true when `Gripped` (The gripper holds a bottle) is on.
2. A branch of 3 parallel paths: each path is tested on its own and does its own work.

   Path 1

   1. `XIO(Is_Mid)` *Examine If Open* asks: true when `Is_Mid` (The bottle broke the middle beam (medium or large)) is off.
   2. `MOV(2,Axis1:O.Record_No)` *Move* does: puts **2** into `Axis1:O.Record_No`. It writes every scan the rung is true.

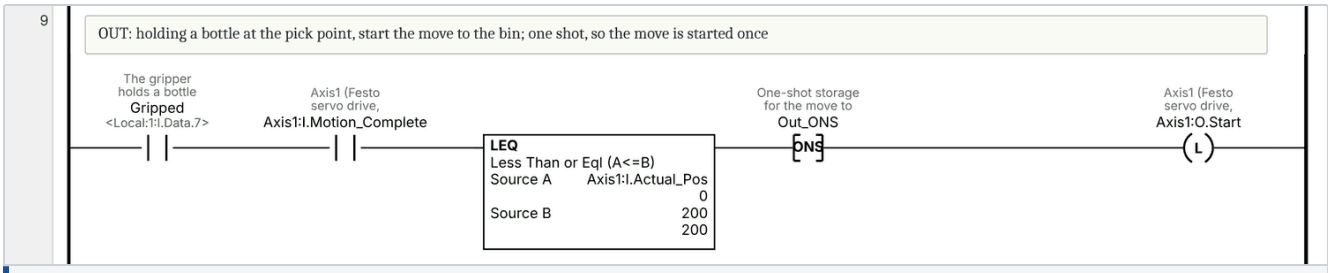
   Path 2

   1. `XIC(Is_Mid)` *Examine If Closed* asks: true when `Is_Mid` is on.
   2. `XIO(Is_High)` *Examine If Open* asks: true when `Is_High` (The bottle broke the high beam (large)) is off.
   3. `MOV(3,Axis1:O.Record_No)` *Move* does: puts **3** into `Axis1:O.Record_No`.

   Path 3

   1. `XIC(Is_High)` *Examine If Closed* asks: true when `Is_High` is on.
   2. `MOV(4,Axis1:O.Record_No)` *Move* does: puts **4** into `Axis1:O.Record_No`.

RUNG 9




```
XIC(Gripped)XIC(Axis1:I.Motion_Complete)LEQ(Axis1:I.Actual_Pos,200)ONS(Out_ONS)
OTL(Axis1:O.Start);
```

HOW THIS RUNG WORKS

In words. When `Gripped` (The gripper holds a bottle) is on, `Axis1:I.Motion_Complete` is on, `Axis1:I.Actual_Pos` is at most **200** and it is the first scan they are true (the one-shot, remembered in `Out_ONS` (One-shot storage for the move to the bin)), the rung latches `Axis1:O.Start` on.

Left to right:

1. `XIC(Gripped)` *Examine If Closed* asks: true when `Gripped` (The gripper holds a bottle) is on.
2. `XIC(Axis1:I.Motion_Complete)` *Examine If Closed* asks: true when `Axis1:I.Motion_Complete` is on.
3. `LEQ(Axis1:I.Actual_Pos,200)` *Less Than or Equal* asks: true when `Axis1:I.Actual_Pos` is at most **200**.
4. `ONS(Out_ONS)` *One Shot* asks: true when it is the first scan they are true (the one-shot, remembered in `Out_ONS` (One-shot storage for the move to the bin)).
5. `OTL(Axis1:O.Start)` *Output Latch* does: latches `Axis1:O.Start` on.

Why it is built this way:

One-shot: once per event, not once per scan.

A latch: it stays on until an OTU clears it.

The pick rung wants four things at once: a bottle at PE_Pick, Part_Below from the gripper, the carriage at rest (Motion_Complete) and at the pick point (Actual_Pos within 2 mm of 0: the drive reports hundredths of a millimetre, so 200). It latches Grip. Rung 8 turns the two latched beams into a record number while the bottle is held. Rung 9 starts the move, once, on the one-shot: Start stays latched until rung 12 drops it on the acknowledge.

TRY IT: A DOUBLE START

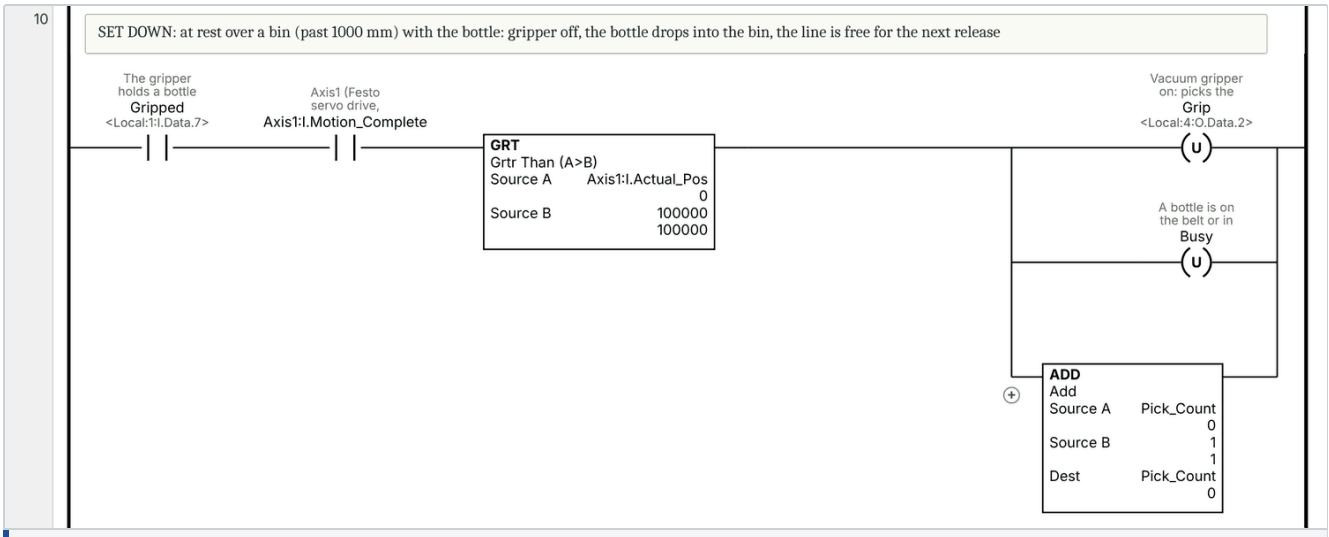
Take the `ONS(Out_ONS)` out of rung 9. Now Start is latched every scan the rung is true, and the drive sees a second Start the moment Motion_Complete comes back over the bin, before rung 10 has let go of the bottle. The log says *Start while moving: ignored (a double start)*, Warning comes on, and the drive record at the end costs 25 for each one.

TRY IT: START BEFORE HOMING

Take `XIC(Axis1:I.Referenced)` out of rung 3 and change rung 2 so Home never fires (delete it). The first bottle arrives, the gripper takes it, rung 9 starts the move, and the drive faults: *positioning before homing*. Op_Enabled drops, the fault lamp lights, nothing moves until Reset, and the bottle hangs in the gripper. Put rung 2 back, press Reset, and the home run takes the bottle with it to 0 mm, where the pick rung is already satisfied.

Set down, back, acknowledge

RUNG 10




```
XIC(Gripped)XIC(Axis1:I.Motion_Complete)GRT(Axis1:I.Actual_Pos,100000)
[OTU(Grip),OTU(Busy),ADD(Pick_Count,1,Pick_Count)];
```

HOW THIS RUNG WORKS

In words. When `Gripped` (The gripper holds a bottle) is on, `Axis1:I.Motion_Complete` is on and `Axis1:I.Actual_Pos` is greater than **100000**, the rung unlatches `Grip` (Vacuum gripper on: picks the bottle under the carriage and holds it while 1) (turns it off), unlatches `Busy` (A bottle is on the belt or in the gripper) (turns it off) and makes `Pick_Count = Pick_Count + 1`.

Left to right:

1. `XIC(Gripped)` *Examine If Closed* asks: true when `Gripped` (The gripper holds a bottle) is on.
2. `XIC(Axis1:I.Motion_Complete)` *Examine If Closed* asks: true when `Axis1:I.Motion_Complete` is on.
3. `GRT(Axis1:I.Actual_Pos,100000)` *Greater Than* asks: true when `Axis1:I.Actual_Pos` is greater than **100000**.
4. A branch of 3 parallel paths: each path is tested on its own and does its own work.

   Path 1

   1. `OTU(Grip)` *Output Unlatch* does: unlatches `Grip` (Vacuum gripper on: picks the bottle under the carriage and holds it while 1) (turns it off).

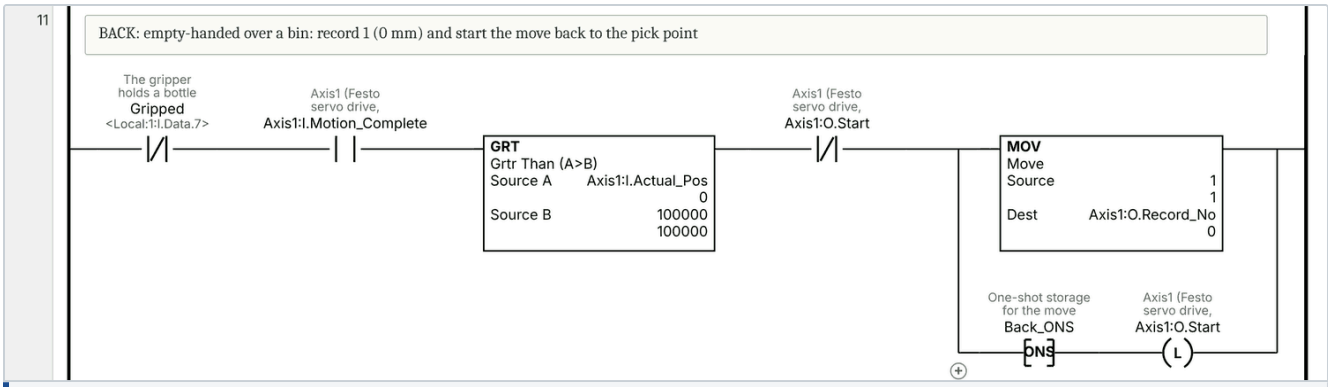
   Path 2

   1. `OTU(Busy)` *Output Unlatch* does: unlatches `Busy` (A bottle is on the belt or in the gripper) (turns it off).

   Path 3

   1. `ADD(Pick_Count,1,Pick_Count)` *Add* does: makes `Pick_Count = Pick_Count + 1`. Every scan the rung is true; behind a one-shot it adds once per event.

RUNG 11




```
XIO(Gripped)XIC(Axis1:I.Motion_Complete)GRT(Axis1:I.Actual_Pos,100000)XIO(Axis1
:O.Start)[MOV(1,Axis1:O.Record_No),ONS(Back_ONS)OTL(Axis1:O.Start)];
```

HOW THIS RUNG WORKS

In words. When `Gripped` (The gripper holds a bottle) is off, `Axis1:I.Motion_Complete` is on, `Axis1:I.Actual_Pos` is greater than **100000** and `Axis1:O.Start` is off, the rung picks its path: in any case it puts **1** into `Axis1:O.Record_No`; if it is the first scan they are true (the one-shot, remembered in `Back_ONS` (One-shot storage for the move back)), it latches `Axis1:O.Start` on.

Left to right:

1. `XIO(Gripped)` *Examine If Open* asks: true when `Gripped` (The gripper holds a bottle) is off.
2. `XIC(Axis1:I.Motion_Complete)` *Examine If Closed* asks: true when `Axis1:I.Motion_Complete` is on.
3. `GRT(Axis1:I.Actual_Pos,100000)` *Greater Than* asks: true when `Axis1:I.Actual_Pos` is greater than **100000**.
4. `XIO(Axis1:O.Start)` *Examine If Open* asks: true when `Axis1:O.Start` is off.
5. A branch of 2 parallel paths: each path is tested on its own and does its own work.

   Path 1

   1. `MOV(1,Axis1:O.Record_No)` *Move* does: puts **1** into `Axis1:O.Record_No`.

   Path 2

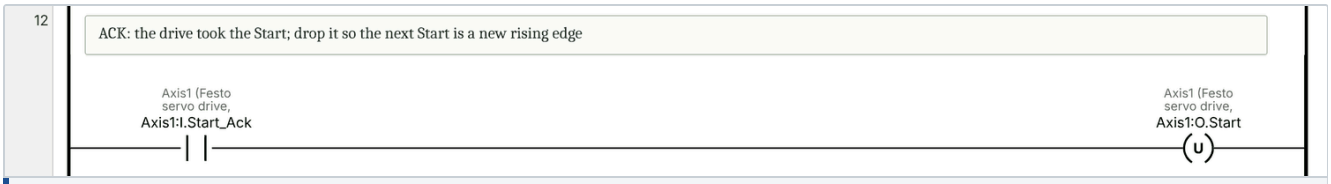
   1. `ONS(Back_ONS)` *One Shot* asks: true when it is the first scan they are true (the one-shot, remembered in `Back_ONS` (One-shot storage for the move back)).
   2. `OTL(Axis1:O.Start)` *Output Latch* does: latches `Axis1:O.Start` on.

Why it is built this way:

One-shot: once per event, not once per scan.

A latch: it stays on until an OTU clears it.

RUNG 12




```
XIC(Axis1:I.Start_Ack)OTU(Axis1:O.Start);
```

HOW THIS RUNG WORKS

In words. When `Axis1:I.Start_Ack` is on, the rung unlatches `Axis1:O.Start` (turns it off).

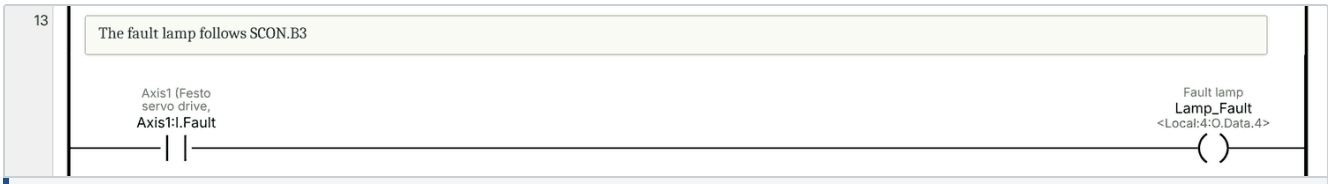
Left to right:

1. `XIC(Axis1:I.Start_Ack)` *Examine If Closed* asks: true when `Axis1:I.Start_Ack` is on.
2. `OTU(Axis1:O.Start)` *Output Unlatch* does: unlatches `Axis1:O.Start` (turns it off).

Rung 10 is the only place Grip is unlatched: at rest, past 1000 mm, holding a bottle. The bottle drops into the bin under the carriage on the next tick and Busy is unlatched in the same rung, so the hopper releases the next bottle while the axis is still on its way back. That overlap is where the par comes from: the book's program does the twelve in 62 s against a par of 80. Rung 11 sends the empty carriage back on record 1; XIO(Axis1:O.Start) keeps it from re-latching Start in the scan Start_Ack is dropping it.

The fault lamp and Reset

RUNG 13




```
XIC(Axis1:I.Fault)OTE(Lamp_Fault);
```

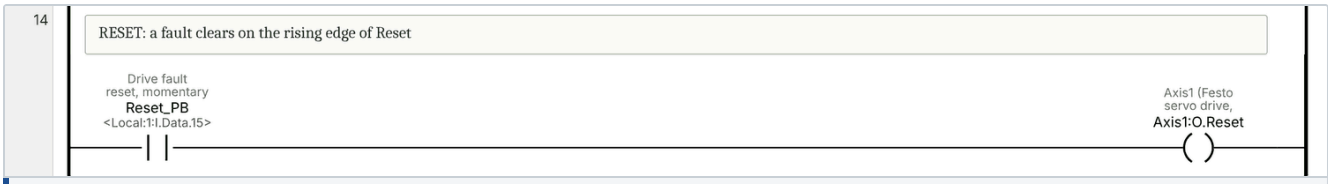
HOW THIS RUNG WORKS

In words. When `Axis1:I.Fault` is on, the rung turns `Lamp_Fault` (Fault lamp) on (and off again whenever the rung is false).

Left to right:

1. `XIC(Axis1:I.Fault)` *Examine If Closed* asks: true when `Axis1:I.Fault` is on.
2. `OTE(Lamp_Fault)` *Output Energize* does: turns `Lamp_Fault` (Fault lamp) on (and off again whenever the rung is false).

RUNG 14




```
XIC(Reset_PB)OTE(Axis1:O.Reset);
```

HOW THIS RUNG WORKS

In words. When `Reset_PB` (Drive fault reset, momentary) is on, the rung turns `Axis1:O.Reset` on (and off again whenever the rung is false).

Left to right:

1. `XIC(Reset_PB)` *Examine If Closed* asks: true when `Reset_PB` (Drive fault reset, momentary) is on.
2. `OTE(Axis1:O.Reset)` *Output Energize* does: turns `Axis1:O.Reset` on (and off again whenever the rung is false).

Run it

Load the program, Run mode, Start. The axis runs to its home switch at the pick point, the blue LED at the motor lights, the first bottle comes down, stops on PE_Pick, the gripper ring turns blue, and the carriage takes the bottle to its bin and comes back. 12 of 12 bottles in the right bin, nothing lost, in 62 s against a par of 80 s: 1400 points and three stars.

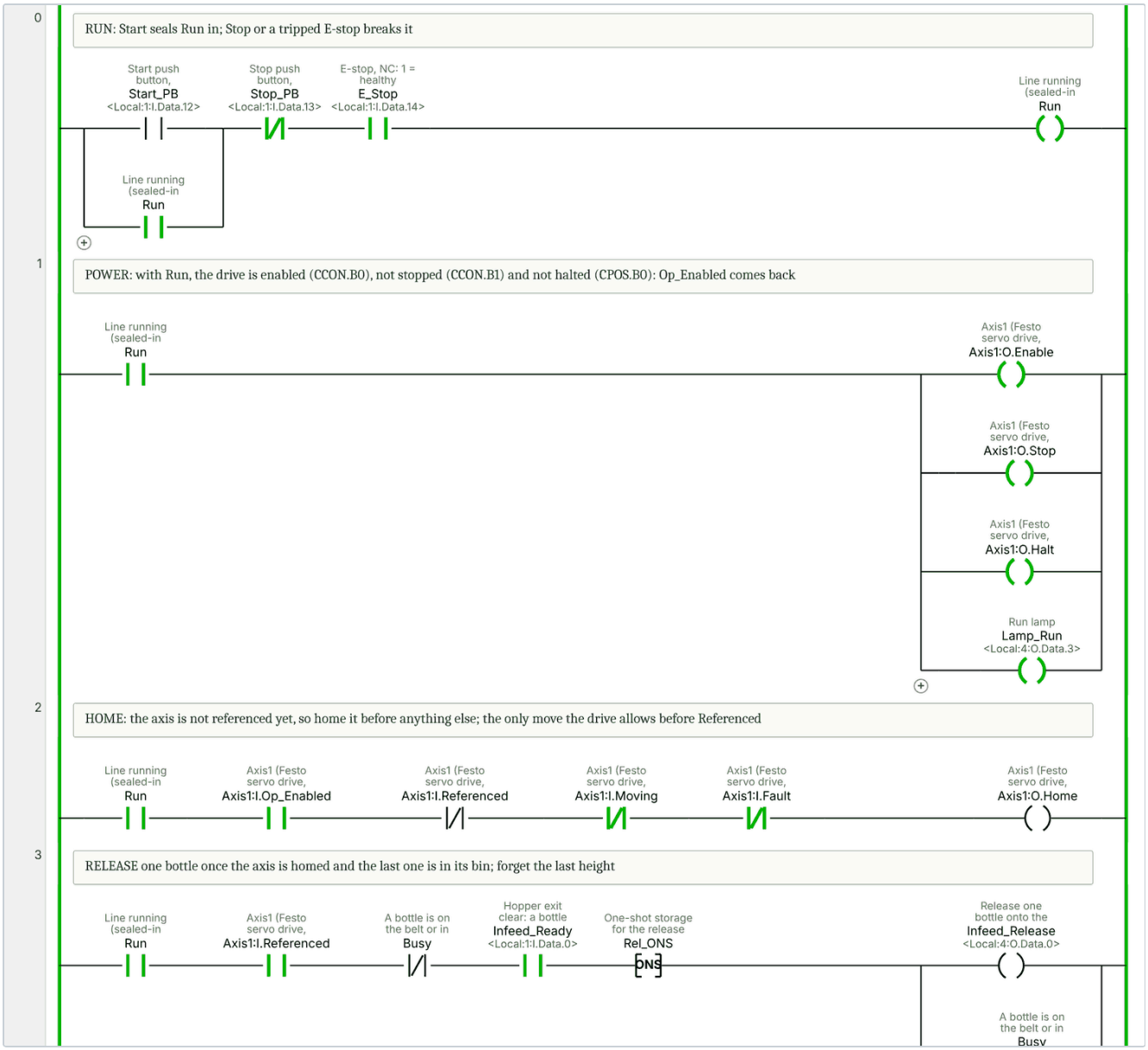


Figure 183. Level 10 online, the top of the routine.

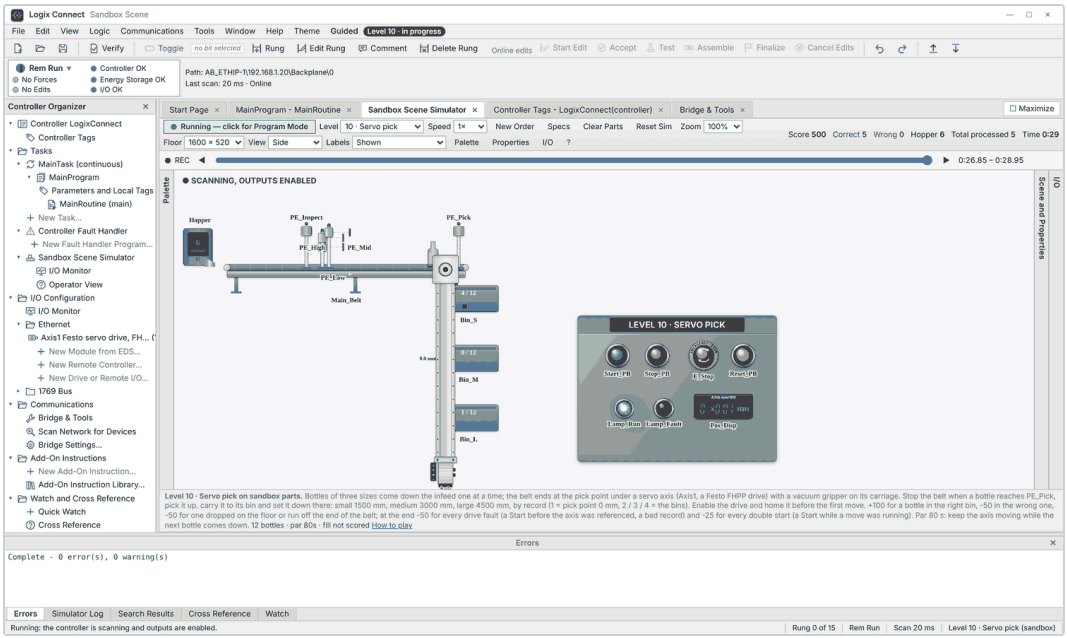


Figure 184. The machine halfway through the run.

Predict it: the answers

1. The rung is false at `XIO(Stop_PB)`: `Stop_PB` is on, so `Run` is **off** (an OTE writes 0 when its rung is false).
2. The rung is false at `XIC(Axis1:I.Op_Enabled)`: `Axis1:I.Op_Enabled` is off, so `Axis1:O.Home` is **off** (an OTE writes 0 when its rung is false).
3. The rung is false at `XIC(Run)`: `Run` is off, so `Conveyor_Run` is **off** (an OTE writes 0 when its rung is false).
4. Every condition on the way is met, so `Grip` latches **on**.

43 Level 11 — Lot code and verify

> **IN THE APP**
>
> File → New Project… → A graded level → **11**. Guided mode opens on this chapter: the tags, every rung below under its fold, a Verify check, and the run results: the marker carries its message, a bottle is marked, a bottle of the second lot carries L0102, then complete, every bottle right and under par. **File → Load Reference Solution** loads the program printed here.

Every bottle that leaves a plant carries a code: the lot it came from, so a complaint can be traced back to a tank and a day. Printing it is one machine's job and reading it back is another's, because a printer that has stopped printing does not know it has. Level 11 is Level 8's line with a **laser marker** in front of the camera. The marker writes the lot code on each bottle as it passes; the camera, on a code-verify job, reads it back and tells you whether it is there and whether it is the right lot. Three lots of six bottles go through, and the lot changes while bottles are still on the belt. The traps are all in the handshakes: the marker's message, the camera's expectation, and keeping each bottle's lot with the bottle.

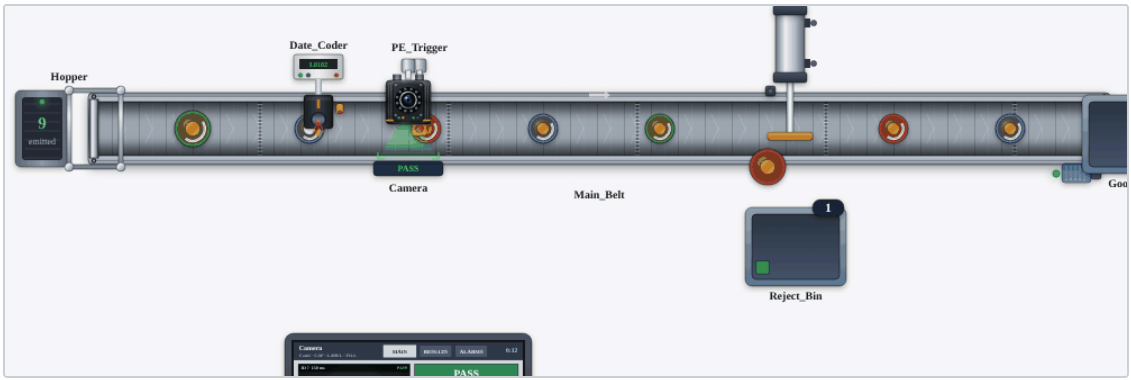


Figure 185. Level 11 in the top view: the marker head over the lane ahead of the camera, its controller showing the loaded message, a bottle under the beam. Lot_Sel at the console shows the lot of the next bottle out of the hopper.

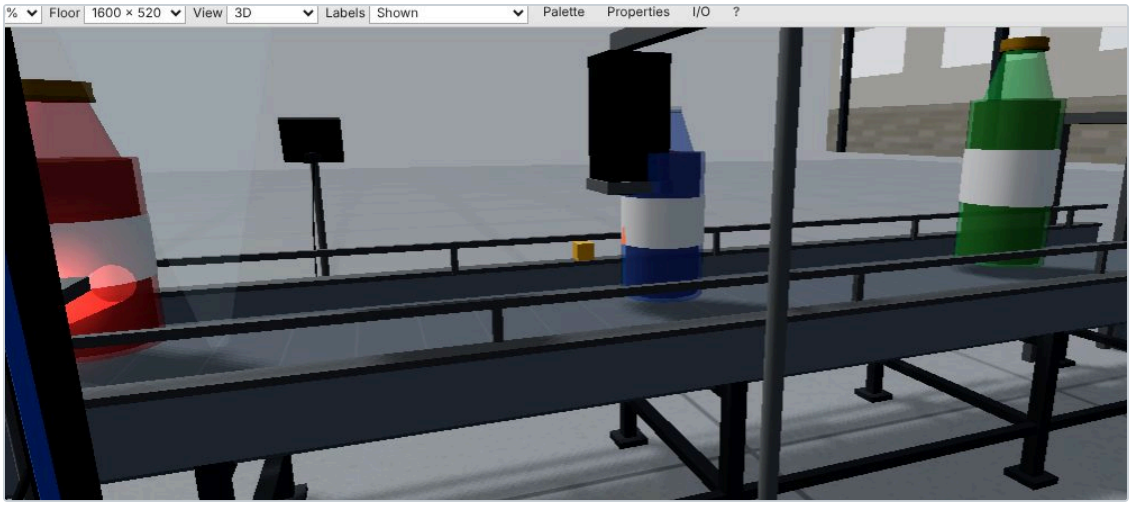


Figure 186. In 3D: the laser head on its bracket over the lane, the orange LASER ON bar lit, the camera and the HMI behind.

The marker

The *Date Coder* part under Bottling is a CO_2 laser marker by default (an inkjet is the other option; its I/O differs only in the service signal). It marks the bottle under its head when `Print` rises, and only while `Laser_On` arms it: that is the marker's interlock, and a Mark with the laser disarmed does nothing but a line in the log. Its message is `L`, the four-digit lot, a dash and a running count: `L0101-001`. The lot comes from `Lot_No`, but writing Lot_No changes nothing by itself: the message is loaded on the rising edge of `Load_Msg`, the load takes half a second with `Ready` off, and `Msg_Ready` comes on when the message carries the lot Lot_No holds. A mark is missed now and then (the bottle rocked), and after ten marks the protective window has fumed up: `Window_Dirty` comes on and every other mark comes out faint until the window is cleaned. The head's own eye, `Detect`, says a bottle is under it.

The camera's code-verify job

The camera runs *Bottle code verify*: cap, label and fill as before, and the code read back by OCR. Two result bits are new: **32**, wrong lot code: the camera read a code, but not the lot it was told to expect; **64**, no code: nothing readable, a missed mark, a faint one or a bottle that was never marked. The lot the camera should expect comes from the PLC in `User_Data` bytes 0-1, latched with `Set_User_Data` ahead of each trigger, the way Level 9 passed its batch data. Tell it nothing and it checks nothing: every code reads as right, and a bottle with the wrong lot ships. The string it read comes back in `Inspection_Results` bytes 12-27 (`NOREAD` when there was nothing), and Camera_HMI shows it.

How the level is scored

What happens	Points
A bottle in Good_Bin that is good and carries its own lot code	+100

What happens	Points
A defective bottle in Reject_Bin (cap, label, fill, no code, wrong lot code)	+100
A bottle shipped with a defect, no code or the wrong lot code	−100
A good bottle rejected	−50
All eighteen bottles done under par (45 s)	+200

The lot a bottle should carry is the lot the selector showed when it left the hopper. A bottle marked with the lot of the batch before or after it is wrong even if the camera, told the wrong thing, let it through. The *Coder record* line in the simulator log at the end counts marks, misses, faint marks, marks with no bottle and marks with the laser disarmed.

Three handshakes and two FIFOs

The selector tells you the lot of the *next* bottle out of the hopper. The bottle under the marker left the hopper 2.5 s ago, and the bottle under the camera a second before that: at a lot change there are bottles of the old lot on the belt when the selector already shows the new one. So the lot has to travel with the bottle, and the program does that the way Level 8 carried result codes: a FIFO from the hopper to the marker (`Lot_Q`), loaded with the selector's lot at each release and unloaded as each bottle reaches the marker's eye, and a second FIFO from the marker to the camera (`Cam_Q`). Then three handshakes: the marker's message load, the camera's user data, and the camera's trigger and results, which are Level 8's.

One more thing the FIFO gives you: its head, `Lot_Q[0]`, is the lot of the next bottle to reach the marker. The program hands the marker that lot as soon as it changes, while the bottle is still on its way, so the half-second load is over before the bottle arrives. Load it on arrival instead and the bottle is past the head before Msg_Ready returns.

Tags this lesson needs

The level's alias tags are already made for you (chapter 32's table). Create these plain tags before you type the rungs; Verify reports any you miss.

Tag	Type	How to make it	Description
`Run`	BOOL	Edit Tags: name, type	Seal-in run bit
`Lot_Want`	INT	Edit Tags: name, type	The lot the selector shows: the lot of the next bottle to be released
`Load_ONS`	BOOL	Edit Tags: name, type	One-shot storage for the message load
`Rel_T`	TIMER	Edit Tags: name, type	Spaces the releases: one bottle every 0.9 s
`Lot_Q`	INT[8]	Edit Tags: name, type, Dim 8	The lots of the bottles between the hopper and the marker, oldest first
`Lot_Q_Ctl`	CONTROL	Edit Tags: name, type	FIFO control for Lot_Q

Tag	Type	How to make it	Description
`Mark_Lot`	INT	Edit Tags: name, type	The lot of the bottle at the marker: what its message must carry
`Mark_ONS`	BOOL	Edit Tags: name, type	
`Cam_Q`	INT[8]	Edit Tags: name, type, Dim 8	The lots of the bottles between the marker and the camera, oldest first
`Cam_Q_Ctl`	CONTROL	Edit Tags: name, type	FIFO control for Cam_Q
`Cam_Lot`	INT	Edit Tags: name, type	The lot the camera should read on the bottle it is about to inspect
`Cam_ONS`	BOOL	Edit Tags: name, type	
`Trig_Req`	BOOL	Edit Tags: name, type	A bottle has reached PE_Trigger and wants an inspection
`Trig_ONS`	BOOL	Edit Tags: name, type	
`Res_Q`	DINT[8]	Edit Tags: name, type, Dim 8	The result codes between the camera and the reject pusher, oldest first
`Res_Q_Ctl`	CONTROL	Edit Tags: name, type	FIFO control for Res_Q
`Cur`	DINT	Edit Tags: name, type	The result code of the bottle at the reject pusher
`Fire`	BOOL	Edit Tags: name, type	Armed: the bottle at the pusher is to be rejected
`Rej_ONS`	BOOL	Edit Tags: name, type	
`Rej_ONS2`	BOOL	Edit Tags: name, type	
`Cnt_ONS`	BOOL	Edit Tags: name, type	

Run, camera, laser, the lot

RUNG 0

```
[XIC(Start_PB),XIC(Run)]XIO(Stop_PB)XIC(E_Stop)OTE(Run);
```

PREDICT IT · 1 `Start_PB` is on, `Stop_PB` is on and `E_Stop` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When either (`Start_PB` (Start push button, momentary) is on) *or* (`Run` (Seal-in run bit) is on), `Stop_PB` (Stop push button, momentary) is off and `E_Stop` (E-stop, NC: 1 = healthy) is on, the rung turns `Run` on (and off again whenever the rung is false).

Left to right:

1. A branch of 2 parallel paths: power gets through if any one of them is true.
   - **Path 1**
     1. `XIC(Start_PB)` *Examine If Closed* asks: true when `Start_PB` (Start push button, momentary) is on.
   - **Path 2**
     1. `XIC(Run)` *Examine If Closed* asks: true when `Run` (Seal-in run bit) is on.
2. `XIO(Stop_PB)` *Examine If Open* asks: true when `Stop_PB` (Stop push button, momentary) is off.
3. `XIC(E_Stop)` *Examine If Closed* asks: true when `E_Stop` (E-stop, NC: 1 = healthy) is on.
4. `OTE(Run)` *Output Energize* does: turns `Run` on (and off again whenever the rung is false). Every scan it writes the rung state: true on, false off.

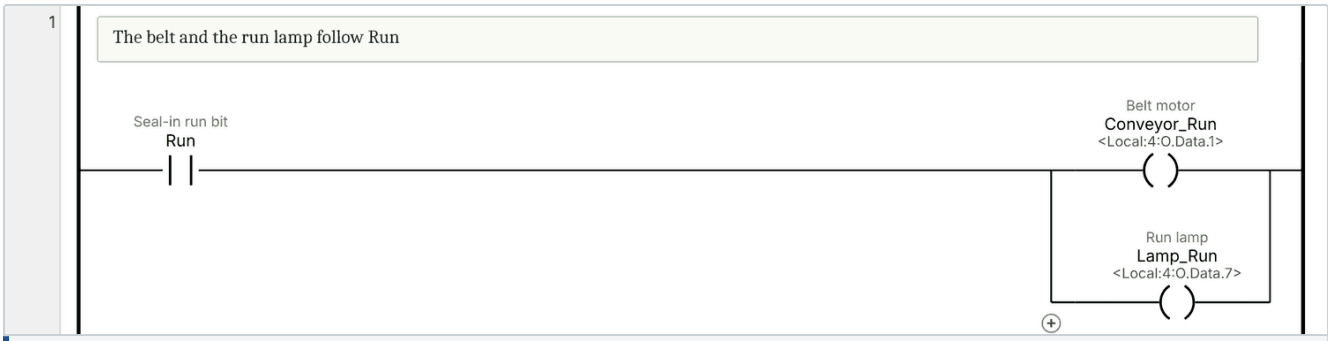
Why it is built this way:

Seal-in. The start condition turns `Run` on, and `Run` 's own contact in the parallel path keeps the rung true after the start goes away. Anything in series outside the branch (a stop, a fault) breaks it, and then it stays off until the start is made again.

> **Think of it like this:** a doorbell button and a hand. The start button is the doorbell, pressed for a moment. The seal-in contact is a hand that grabs the circuit as soon as the output comes on and keeps holding after you let go. The stop button, in series, is the only thing that can knock that hand away.

> **Watch for:** The stop has to be in series outside the branch. Put it inside the start path only and the seal-in path bypasses it: the stop no longer stops.

RUNG 1




```
XIC(Run)[OTE(Conveyor_Run),OTE(Lamp_Run)];
```

HOW THIS RUNG WORKS

In words. When `Run` (Seal-in run bit) is on, the rung turns `Conveyor_Run` (Belt motor) on (and off again whenever the rung is false) and turns `Lamp_Run` (Run lamp) on (and off again whenever the rung is false).

Left to right:

1. `XIC(Run)` *Examine If Closed* asks: true when `Run` (Seal-in run bit) is on.
2. A branch of 2 parallel paths: each path is tested on its own and does its own work.

   Path 1

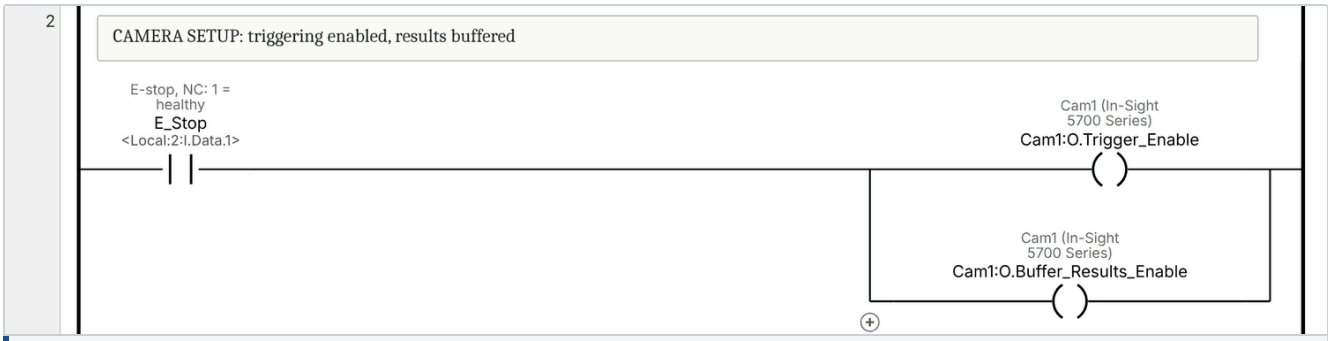
   1. `OTE(Conveyor_Run)` *Output Energize* does: turns `Conveyor_Run` (Belt motor) on (and off again whenever the rung is false).

   Path 2

   1. `OTE(Lamp_Run)` *Output Energize* does: turns `Lamp_Run` (Run lamp) on (and off again whenever the rung is false).

> **Think of it like this:** a rung is a pipe with valves in it, read left to right. Every contact is a valve: XIC is open while its bit is 1, XIO while its bit is 0. Water reaches the output at the right only if every valve in a row is open. The output does not remember anything: on every scan it is simply wet or dry.

RUNG 2




```
XIC(E_Stop)[OTE(Cam1:O.Trigger_Enable),OTE(Cam1:O.Buffer_Results_Enable)];
```

HOW THIS RUNG WORKS

In words. When `E_Stop` (E-stop, NC: 1 = healthy) is on, the rung turns `Cam1:O.Trigger_Enable` on (and off again whenever the rung is false) and turns `Cam1:O.Buffer_Results_Enable` on (and off again whenever the rung is false).

Left to right:

1. `XIC(E_Stop)` *Examine If Closed* asks: true when `E_Stop` (E-stop, NC: 1 = healthy) is on.
2. A branch of 2 parallel paths: each path is tested on its own and does its own work.

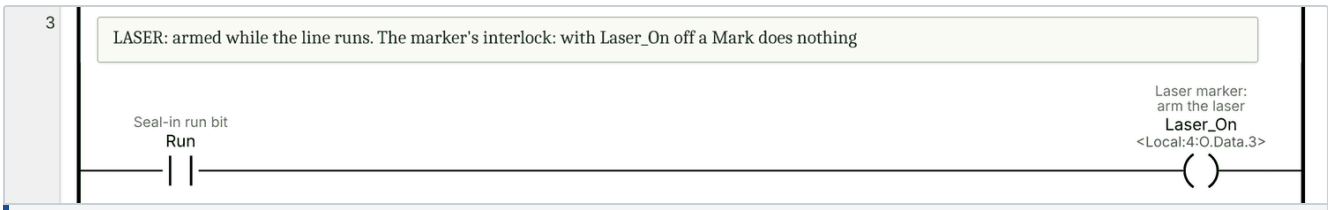
   Path 1

   1. `OTE(Cam1:O.Trigger_Enable)` *Output Energize* does: turns `Cam1:O.Trigger_Enable` on (and off again whenever the rung is false).

   Path 2

   1. `OTE(Cam1:O.Buffer_Results_Enable)` *Output Energize* does: turns `Cam1:O.Buffer_Results_Enable` on (and off again whenever the rung is false).

RUNG 3




```
XIC(Run)OTE(Laser_On);
```

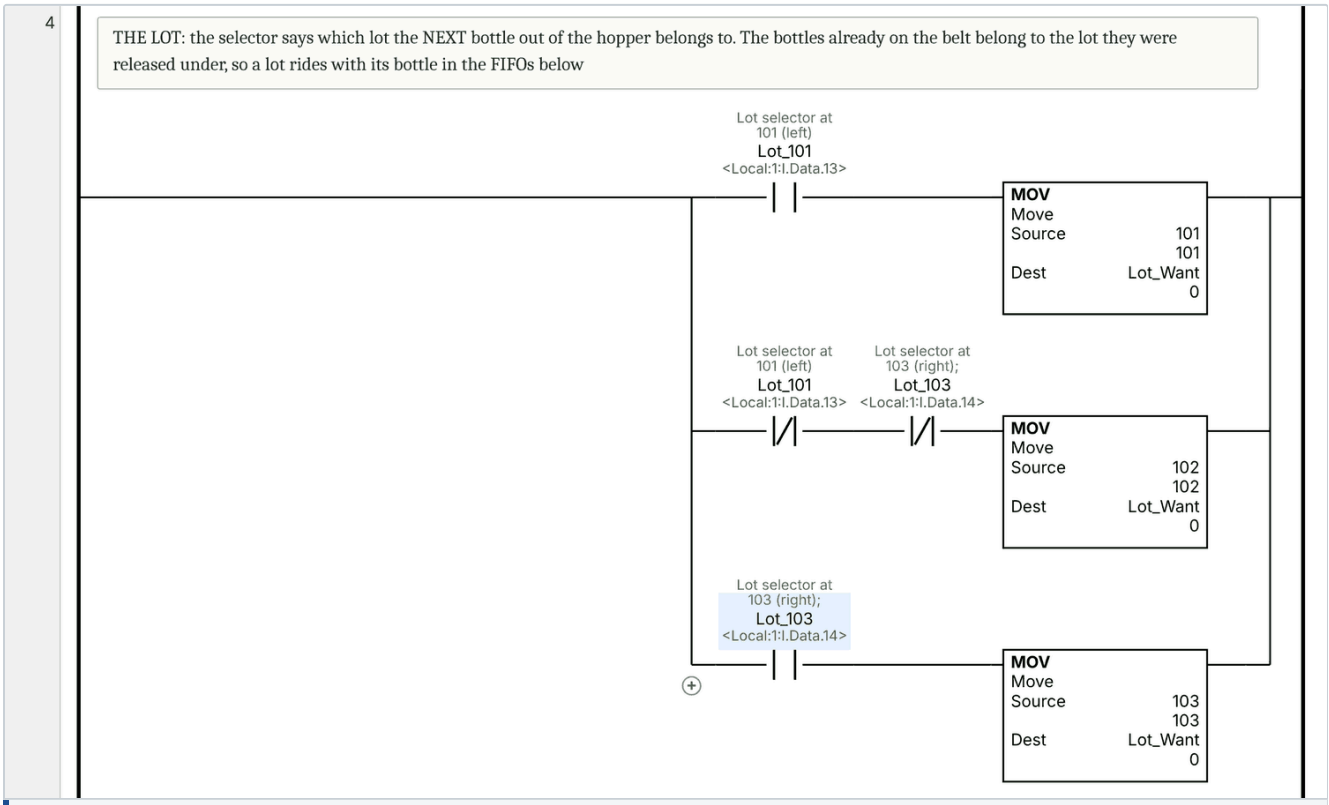
HOW THIS RUNG WORKS

In words. When `Run` (Seal-in run bit) is on, the rung turns `Laser_On` (Laser marker: arm the laser (its interlock); no mark while 0) on (and off again whenever the rung is false).

Left to right:

1. `XIC(Run)` *Examine If Closed* asks: true when `Run` (Seal-in run bit) is on.
2. `OTE(Laser_On)` *Output Energize* does: turns `Laser_On` (Laser marker: arm the laser (its interlock); no mark while 0) on (and off again whenever the rung is false).

RUNG 4




```
[XIC(Lot_101)MOV(101,Lot_Want),XIO(Lot_101)XIO(Lot_103)MOV(102,Lot_Want),XIC(Lot_103)MOV(103,Lot_Want)];
```

PREDICT IT · 2 `Lot_103` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. Every scan, with nothing to test, the rung picks its path: if `Lot_101` (Lot selector at 101 (left)) is on, it puts **101** into `Lot_Want` (The lot the selector shows: the lot of the next bottle to be released); if `Lot_101` is off and `Lot_103` (Lot selector at 103 (right); neither = 102) is off, it puts **102** into `Lot_Want` ; if `Lot_103` is on, it puts **103** into `Lot_Want` .

Left to right:

1. A branch of 3 parallel paths: each path is tested on its own and does its own work.

   Path 1

   1. `XIC(Lot_101)` *Examine If Closed* asks: true when `Lot_101` (Lot selector at 101 (left)) is on.
   2. `MOV(101,Lot_Want)` *Move* does: puts **101** into `Lot_Want` (The lot the selector shows: the lot of the next bottle to be released). It writes every scan the rung is true.

   Path 2

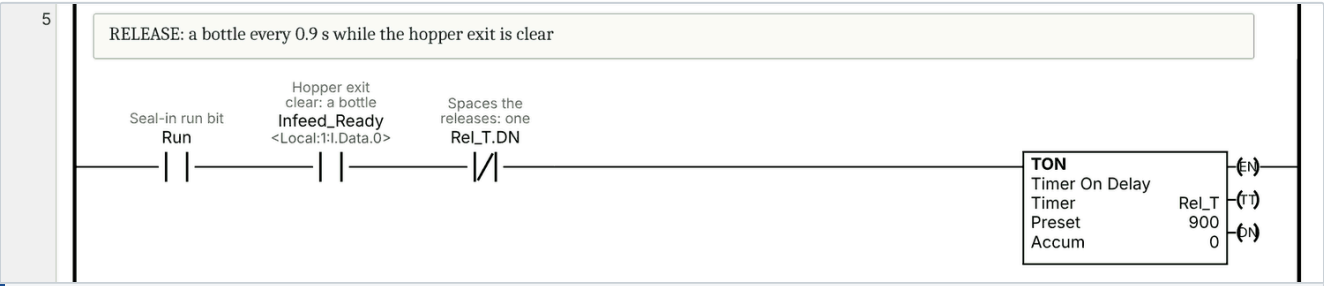
   1. `XIO(Lot_101)` *Examine If Open* asks: true when `Lot_101` is off.
   2. `XIO(Lot_103)` *Examine If Open* asks: true when `Lot_103` (Lot selector at 103 (right); neither = 102) is off.
   3. `MOV(102,Lot_Want)` *Move* does: puts **102** into `Lot_Want` .

   Path 3

1. `XIC(Lot_103)` *Examine If Closed* asks: true when `Lot_103` is on.
2. `MOV(103,Lot_Want)` *Move* does: puts **103** into `Lot_Want`.

Release and the hopper-to-marker FIFO

RUNG 5




```
XIC(Run)XIC(Infeed_Ready)XIO(Rel_T.DN)TON(Rel_T,900,0);
```

PREDICT IT · 3 `Run` is on, `Infeed_Ready` is on and `Rel_T.DN` (the done bit of `Rel_T`: Spaces the releases: one bottle every 0.9 s) is off. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Run` (Seal-in run bit) is on, `Infeed_Ready` (Hopper exit clear: a bottle can be released) is on and `Rel_T.DN` (the done bit of `Rel_T`: Spaces the releases: one bottle every 0.9 s) is off, the rung runs the timer `Rel_T` toward 900 ms (0.9 s).

Left to right:

1. `XIC(Run)` *Examine If Closed* asks: true when `Run` (Seal-in run bit) is on.
2. `XIC(Infeed_Ready)` *Examine If Closed* asks: true when `Infeed_Ready` (Hopper exit clear: a bottle can be released) is on.
3. `XIO(Rel_T.DN)` *Examine If Open* asks: true when `Rel_T.DN` (the done bit of `Rel_T`: Spaces the releases: one bottle every 0.9 s) is off.
4. `TON(Rel_T,900,0)` *Timer On Delay* does: runs the timer `Rel_T` toward 900 ms (0.9 s). The accumulator counts while the rung is true; .DN comes on at the preset; the moment the rung goes false the timer resets.

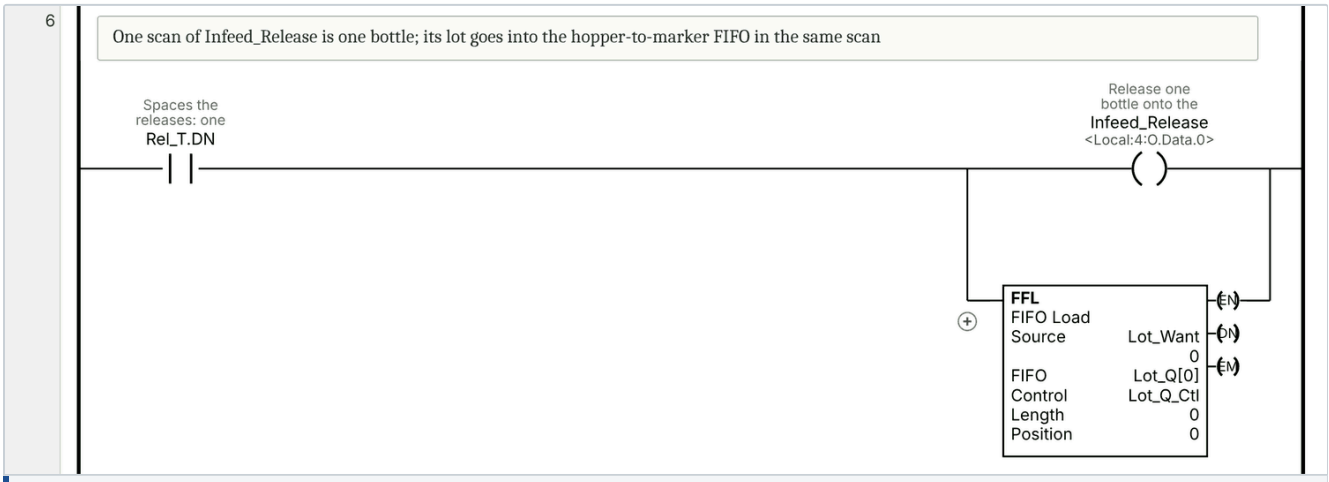
Why it is built this way:

A timer. `Rel_T` only times while this rung is true; other rungs use its `.DN` (done) or `.TT` (timing) bits. Being a TON, it starts again from zero every time the rung goes false.

> **Think of it like this:** a kitchen timer that only runs while you hold your finger on it. Take your finger off and it snaps back to zero. Hold it long enough and the done light comes on, and stays on as long as your finger is still there.

> **Watch for:** If anything in series flickers false for one scan, a TON starts over from zero. Use RTO when the time must survive an interruption.

RUNG 6




```
XIC(Rel_T.DN)[OTE(Infeed_Release),FFL(Lot_Want,Lot_Q[0],Lot_Q_Ctl,8)];
```

HOW THIS RUNG WORKS

In words. When `Rel_T.DN` (the done bit of `Rel_T` : Spaces the releases: one bottle every 0.9 s) is on, the rung turns `Infeed_Release` (Release one bottle onto the belt (rising edge)) on (and off again whenever the rung is false) and loads `Lot_Want` (The lot the selector shows: the lot of the next bottle to be released) into the queue `Lot_Q[0]` .

Left to right:

1. `XIC(Rel_T.DN)` *Examine If Closed* asks: true when `Rel_T.DN` (the done bit of `Rel_T` : Spaces the releases: one bottle every 0.9 s) is on.
2. A branch of 2 parallel paths: each path is tested on its own and does its own work.

   Path 1

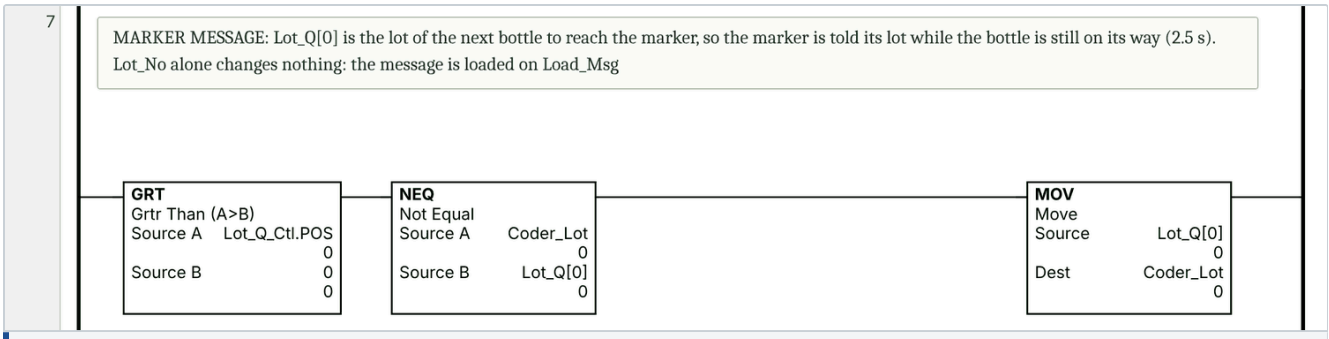
   1. `OTE(Infeed_Release)` *Output Energize* does: turns `Infeed_Release` (Release one bottle onto the belt (rising edge)) on (and off again whenever the rung is false).

   Path 2

   1. `FFL(Lot_Want,Lot_Q[0],Lot_Q_Ctl,8)` *FIFO Load* does: loads `Lot_Want` (The lot the selector shows: the lot of the next bottle to be released) into the queue `Lot_Q[0]` . One entry per false-to-true transition.

The marker's message and the mark

RUNG 7




```
GRT(Lot_Q_Ctl.POS,0)NEQ(Coder_Lot,Lot_Q[0])MOV(Lot_Q[0],Coder_Lot);
```

PREDICT IT · 4 `Lot_Q_Ctl.POS` (the position of `Lot_Q_Ctl`: FIFO control for Lot_Q) is 1, `Coder_Lot` is 501 and `Lot_Q[0]` is 500. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

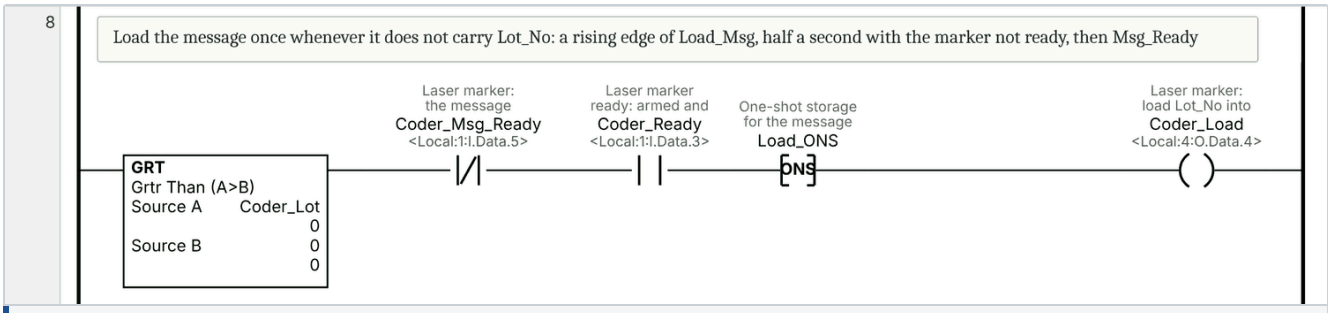
HOW THIS RUNG WORKS

In words. When `Lot_Q_Ctl.POS` (the position of `Lot_Q_Ctl`: FIFO control for Lot_Q) is greater than **0** and `Coder_Lot` (Laser marker: the lot number for the message) is not `Lot_Q[0]`, the rung puts `Lot_Q[0]` into `Coder_Lot`.

Left to right:

1. `GRT(Lot_Q_Ctl.POS,0)` *Greater Than* asks: true when `Lot_Q_Ctl.POS` (the position of `Lot_Q_Ctl`: FIFO control for Lot_Q) is greater than **0**.
2. `NEQ(Coder_Lot,Lot_Q[0])` *Not Equal* asks: true when `Coder_Lot` (Laser marker: the lot number for the message) is not `Lot_Q[0]`.
3. `MOV(Lot_Q[0],Coder_Lot)` *Move* does: puts `Lot_Q[0]` into `Coder_Lot`.

RUNG 8




```
GRT(Coder_Lot,0)XIO(Coder_Msg_Ready)XIC(Coder_Ready)ONS(Load_ONS)OTE(Coder_Load
);
```

HOW THIS RUNG WORKS

In words. When `Coder_Lot` (Laser marker: the lot number for the message) is greater than **0**, `Coder_Msg_Ready` (Laser marker: the message carries Lot_No) is off, `Coder_Ready` (Laser marker ready: armed and not loading) is on and it is the first scan they are true (the one-shot, remembered in `Load_ONS` (One-shot storage for the message load)), the rung turns `Coder_Load` (Laser marker: load Lot_No into the message (rising edge, 0.5 s)) on (and off again whenever the rung is false).

Left to right:

1. `GRT(Coder_Lot,0)` *Greater Than* asks: true when `Coder_Lot` (Laser marker: the lot number for the message) is greater than **0**.
2. `XIO(Coder_Msg_Ready)` *Examine If Open* asks: true when `Coder_Msg_Ready` (Laser marker: the message carries Lot_No) is off.
3. `XIC(Coder_Ready)` *Examine If Closed* asks: true when `Coder_Ready` (Laser marker ready: armed and not loading) is on.
4. `ONS(Load_ONS)` *One Shot* asks: true when it is the first scan they are true (the one-shot, remembered in `Load_ONS` (One-shot storage for the message load)). It passes power for one scan only, the scan its input goes from false to true; its storage bit remembers last scan.
5. `OTE(Coder_Load)` *Output Energize* does: turns `Coder_Load` (Laser marker: load Lot_No into the message (rising edge, 0.5 s)) on (and off again whenever the rung is false).

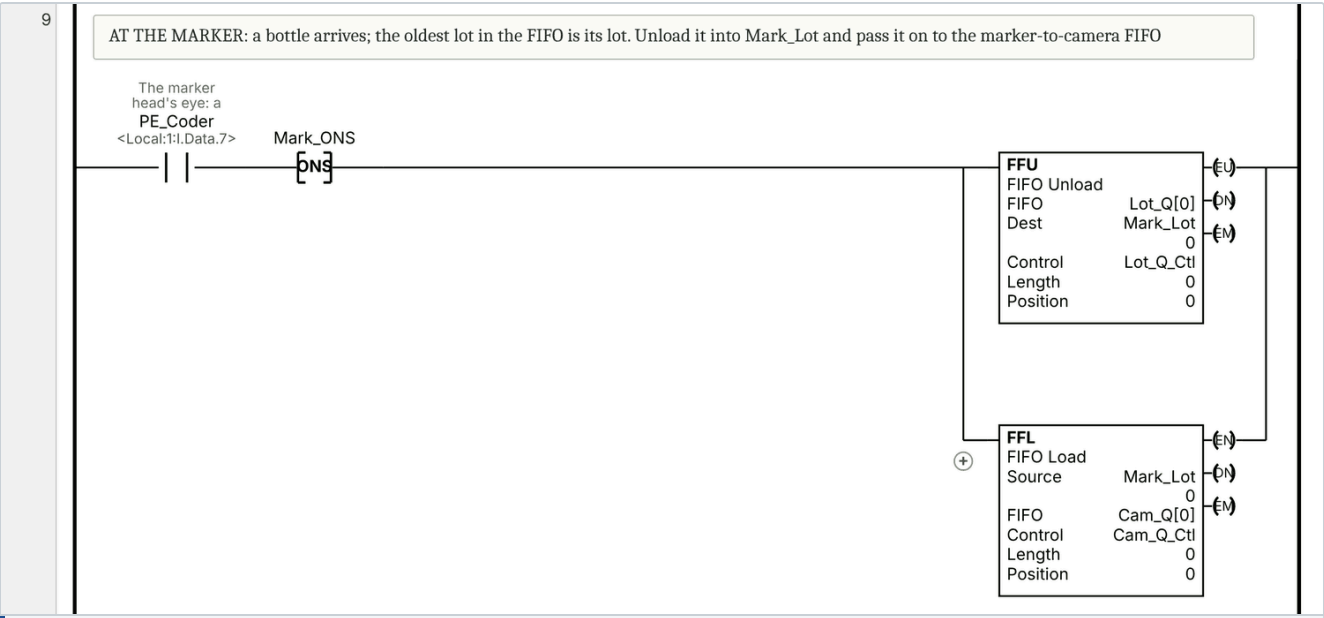
Why it is built this way:

One-shot. Everything after the ONS happens once per event, not on every scan the condition holds; `Load_ONS` is only its memory of the last scan.

> **Think of it like this:** a doorbell that rings once however long you lean on the button. The ONS looks at what the button was doing last scan: only the change from up to down rings it.

> **Watch for:** Each ONS needs its own storage bit. Two ONS sharing one bit make one of them miss its edges.

RUNG 9




```
XIC(PE_Coder)ONS(Mark_ONS)
[FFU(Lot_Q[0],Mark_Lot,Lot_Q_Ctl,8),FFL(Mark_Lot,Cam_Q[0],Cam_Q_Ctl,8)];
```

HOW THIS RUNG WORKS

In words. When `PE_Coder` (The marker head's eye: a bottle is under the head) is on and it is the first scan they are true (the one-shot, remembered in `Mark_ONS`), the rung unloads the oldest entry of `Lot_Q[0]` into `Mark_Lot` (The lot of the bottle at the marker: what its message must carry) and loads `Mark_Lot` into the queue `Cam_Q[0]`.

Left to right:

1. `XIC(PE_Coder)` *Examine If Closed* asks: true when `PE_Coder` (The marker head's eye: a bottle is under the head) is on.
2. `ONS(Mark_ONS)` *One Shot* asks: true when it is the first scan they are true (the one-shot, remembered in `Mark_ONS`).
3. A branch of 2 parallel paths: each path is tested on its own and does its own work.

   Path 1

   1. `FFU(Lot_Q[0],Mark_Lot,Lot_Q_Ctl,8)` *FIFO Unload* does: unloads the oldest entry of `Lot_Q[0]` into `Mark_Lot` (The lot of the bottle at the marker: what its message must carry). One entry per false-to-true transition.

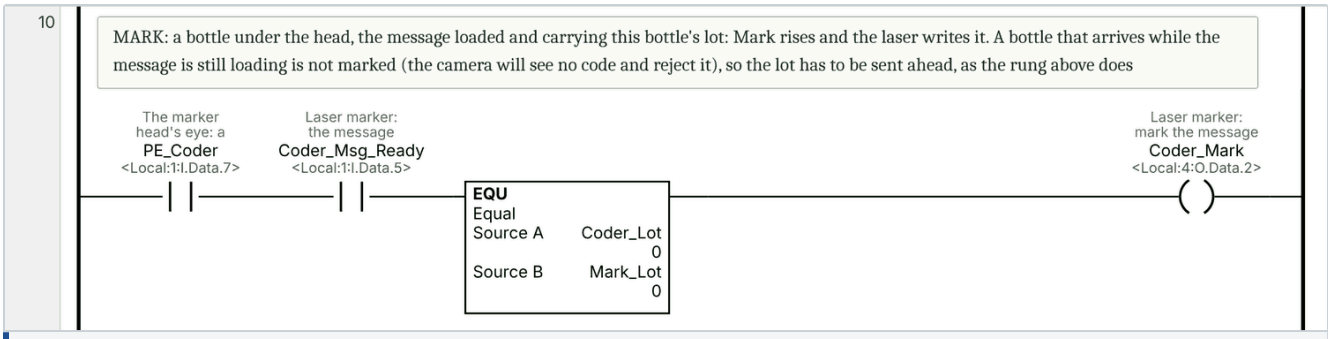
   Path 2

   1. `FFL(Mark_Lot,Cam_Q[0],Cam_Q_Ctl,8)` *FIFO Load* does: loads `Mark_Lot` into the queue `Cam_Q[0]`.

Why it is built this way:

One-shot: once per event, not once per scan.

RUNG 10




```
XIC(PE_Coder)XIC(Coder_Msg_Ready)EQU(Coder_Lot,Mark_Lot)OTE(Coder_Mark);
```

PREDICT IT · 5 `PE_Coder` is on, `Coder_Msg_Ready` is on, `Coder_Lot` is 501 and `Mark_Lot` is 500. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `PE_Coder` (The marker head's eye: a bottle is under the head) is on, `Coder_Msg_Ready` (Laser marker: the message carries Lot_No) is on and `Coder_Lot` (Laser marker: the lot number for the message) equals `Mark_Lot` (The lot of the bottle at the marker: what its message must carry), the rung turns `Coder_Mark` (Laser marker: mark the message on the bottle under the head (rising edge)) on (and off again whenever the rung is false).

Left to right:

1. `XIC(PE_Coder)` *Examine If Closed* asks: true when `PE_Coder` (The marker head's eye: a bottle is under the head) is on.
2. `XIC(Coder_Msg_Ready)` *Examine If Closed* asks: true when `Coder_Msg_Ready` (Laser marker: the message carries Lot_No) is on.
3. `EQU(Coder_Lot,Mark_Lot)` *Equal* asks: true when `Coder_Lot` (Laser marker: the lot number for the message) equals `Mark_Lot` (The lot of the bottle at the marker: what its message must carry). Both values are compared every scan.
4. `OTE(Coder_Mark)` *Output Energize* does: turns `Coder_Mark` (Laser marker: mark the message on the bottle under the head (rising edge)) on (and off again whenever the rung is false).

WATCH FOR: MARKING WITH THE LIVE SELECTOR

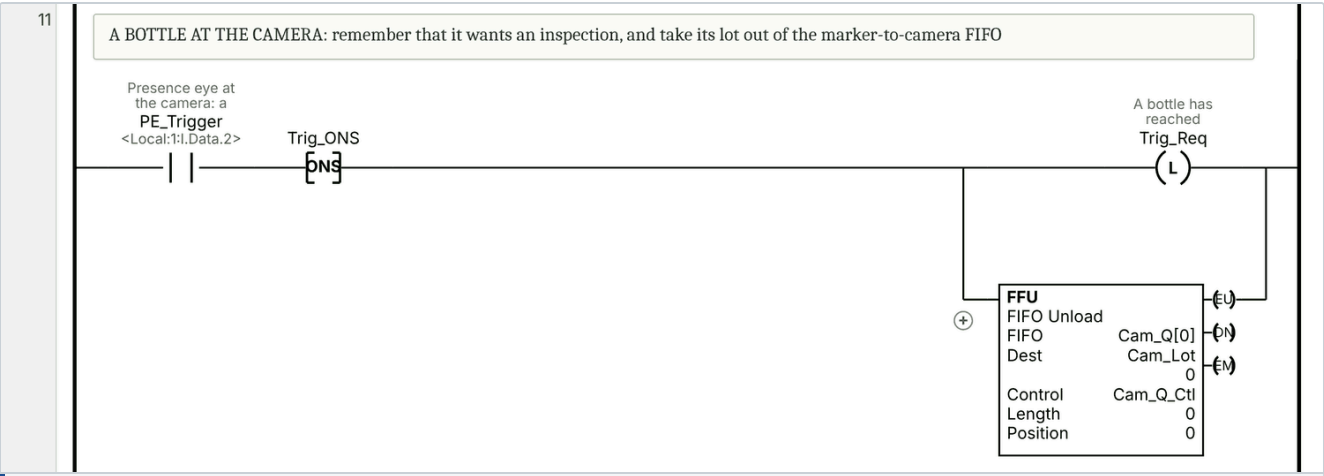
Replace `Lot_Q[0]` in rung 7 and `Mark_Lot` in rung 10 with `Lot_Want`, the live selector, and run. The first bottle of each lot change is marked with the old lot or the new one depending on where it was when the selector turned, and the camera, if you leave rung 12 alone, catches it as 32 and rejects a good bottle: −50 each. Leave rung 12 on `Lot_Want` too and the camera agrees with the marker and the bottle ships with the wrong lot: −100. Both are the same mistake, the belt treated as if it were empty.

TRY IT: THE LOAD THAT COMES TOO LATE

Change rung 7 to load from `Mark_Lot` instead of `Lot_Q[0]`: the marker learns the lot when the bottle is already under the head. The log shows *loading message* and, half a second later, *message loaded*; by then the first bottle of the new lot has gone by unmarked, reads 64 at the camera and is rejected. A right reject, but a lost bottle, three times a run.

User data, trigger, results, reject

RUNG 11




```
XIC(PE_Trigger)ONS(Trig_ONS)[OTL(Trig_Req),FFU(Cam_Q[0],Cam_Lot,Cam_Q_Ctl,8)];
```

HOW THIS RUNG WORKS

In words. When `PE_Trigger` (Presence eye at the camera: a bottle is in the field of view) is on and it is the first scan they are true (the one-shot, remembered in `Trig_ONS`), the rung latches `Trig_Req` (A bottle has reached PE_Trigger and wants an inspection) on and unloads the oldest entry of `Cam_Q[0]` into `Cam_Lot` (The lot the camera should read on the bottle it is about to inspect).

Left to right:

1. `XIC(PE_Trigger)` *Examine If Closed* asks: true when `PE_Trigger` (Presence eye at the camera: a bottle is in the field of view) is on.
2. `ONS(Trig_ONS)` *One Shot* asks: true when it is the first scan they are true (the one-shot, remembered in `Trig_ONS`).
3. A branch of 2 parallel paths: each path is tested on its own and does its own work.

   #### Path 1

   1. `OTL(Trig_Req)` *Output Latch* does: latches `Trig_Req` (A bottle has reached PE_Trigger and wants an inspection) on. It only ever sets the bit; the bit stays on after the rung goes false.

   #### Path 2

   1. `FFU(Cam_Q[0],Cam_Lot,Cam_Q_Ctl,8)` *FIFO Unload* does: unloads the oldest entry of `Cam_Q[0]` into `Cam_Lot` (The lot the camera should read on the bottle it is about to inspect).

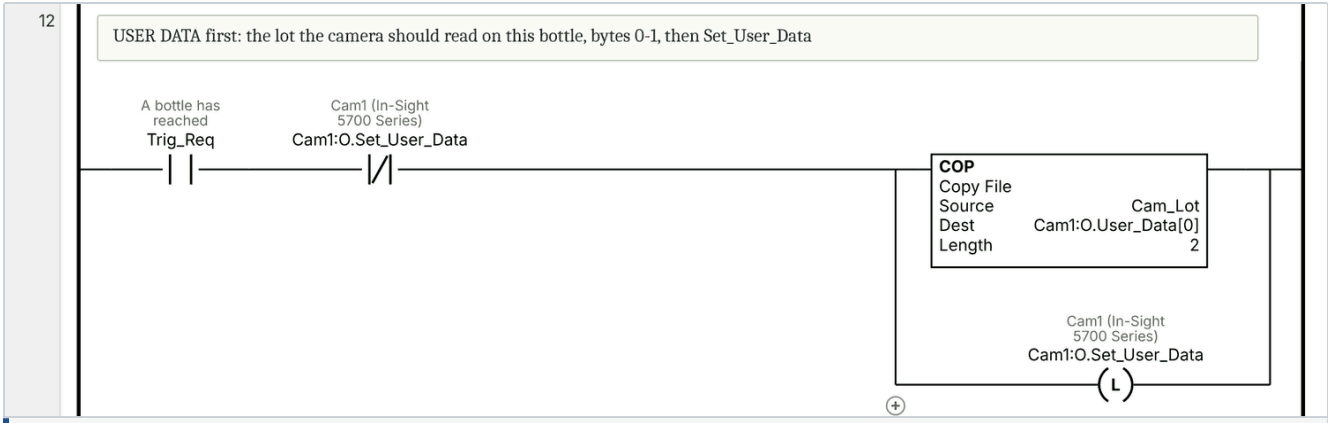
Why it is built this way:

One-shot: once per event, not once per scan.

A latch. `Trig_Req` stays on after this rung goes false; an `OTU` elsewhere in the program clears it.

> **Think of it like this:** a light switch instead of a doorbell. OTL flips it on and it stays on when you take your hand away; OTU flips it off. Whatever flipped it on does not have to stay there.

RUNG 12




```
XIC(Trig_Req)XIO(Cam1:O.Set_User_Data)
[COP(Cam_Lot,Cam1:O.User_Data[0],2),OTL(Cam1:O.Set_User_Data)];
```

HOW THIS RUNG WORKS

In words. When `Trig_Req` (A bottle has reached PE_Trigger and wants an inspection) is on and `Cam1:O.Set_User_Data` is off, the rung copies 2 elements from `Cam_Lot` (The lot the camera should read on the bottle it is about to inspect) to `Cam1:O.User_Data[0]` and latches `Cam1:O.Set_User_Data` on.

Left to right:

1. `XIC(Trig_Req)` *Examine If Closed* asks: true when `Trig_Req` (A bottle has reached PE_Trigger and wants an inspection) is on.
2. `XIO(Cam1:O.Set_User_Data)` *Examine If Open* asks: true when `Cam1:O.Set_User_Data` is off.
3. A branch of 2 parallel paths: each path is tested on its own and does its own work.

   Path 1

   1. `COP(Cam_Lot,Cam1:O.User_Data[0],2)` *Copy File* does: copies 2 elements from `Cam_Lot` (The lot the camera should read on the bottle it is about to inspect) to `Cam1:O.User_Data[0]`.

   Path 2

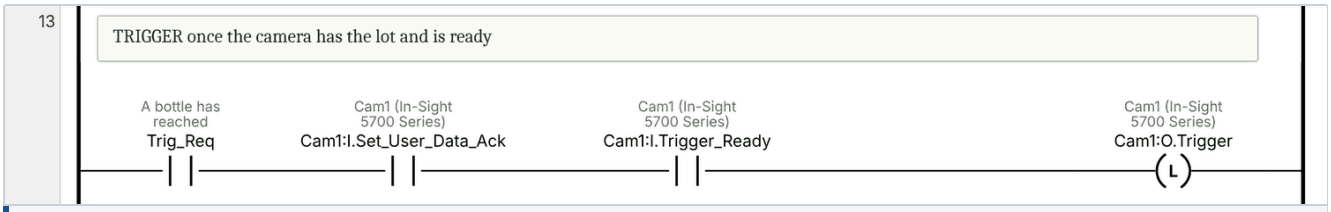
   1. `OTL(Cam1:O.Set_User_Data)` *Output Latch* does: latches `Cam1:O.Set_User_Data` on.

Why it is built this way:

A latch: it stays on until an OTU clears it.

> **Watch for:** A latched bit survives the condition that set it: every way in needs a way out, or it stays on for good.

RUNG 13




```
XIC(Trig_Req)XIC(Cam1:I.Set_User_Data_Ack)XIC(Cam1:I.Trigger_Ready)OTL(Cam1:O.Trigger);
```

PREDICT IT · 6 `Trig_Req` is on, `Cam1:I.Set_User_Data_Ack` is on and `Cam1:I.Trigger_Ready` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Trig_Req` (A bottle has reached PE_Trigger and wants an inspection) is on, `Cam1:I.Set_User_Data_Ack` is on and `Cam1:I.Trigger_Ready` is on, the rung latches `Cam1:O.Trigger` on.

Left to right:

1. `XIC(Trig_Req)` *Examine If Closed* asks: true when `Trig_Req` (A bottle has reached PE_Trigger and wants an inspection) is on.
2. `XIC(Cam1:I.Set_User_Data_Ack)` *Examine If Closed* asks: true when `Cam1:I.Set_User_Data_Ack` is on.
3. `XIC(Cam1:I.Trigger_Ready)` *Examine If Closed* asks: true when `Cam1:I.Trigger_Ready` is on.
4. `OTL(Cam1:O.Trigger)` *Output Latch* does: latches `Cam1:O.Trigger` on.

Why it is built this way:

A latch: it stays on until an OTU clears it.

RUNG 14

```
XIC(Cam1:I.Trigger_Ack)
[OTU(Cam1:O.Trigger),OTU(Trig_Req),OTU(Cam1:O.Set_User_Data)];
```

HOW THIS RUNG WORKS

In words. When `Cam1:I.Trigger_Ack` is on, the rung unlatches `Cam1:O.Trigger` (turns it off), unlatches `Trig_Req` (A bottle has reached PE_Trigger and wants an inspection) (turns it off) and unlatches `Cam1:O.Set_User_Data` (turns it off).

Left to right:

1. `XIC(Cam1:I.Trigger_Ack)` *Examine If Closed* asks: true when `Cam1:I.Trigger_Ack` is on.
2. A branch of 3 parallel paths: each path is tested on its own and does its own work.

   Path 1

   1. `OTU(Cam1:O.Trigger)` *Output Unlatch* does: unlatches `Cam1:O.Trigger` (turns it off). It only ever clears the bit.

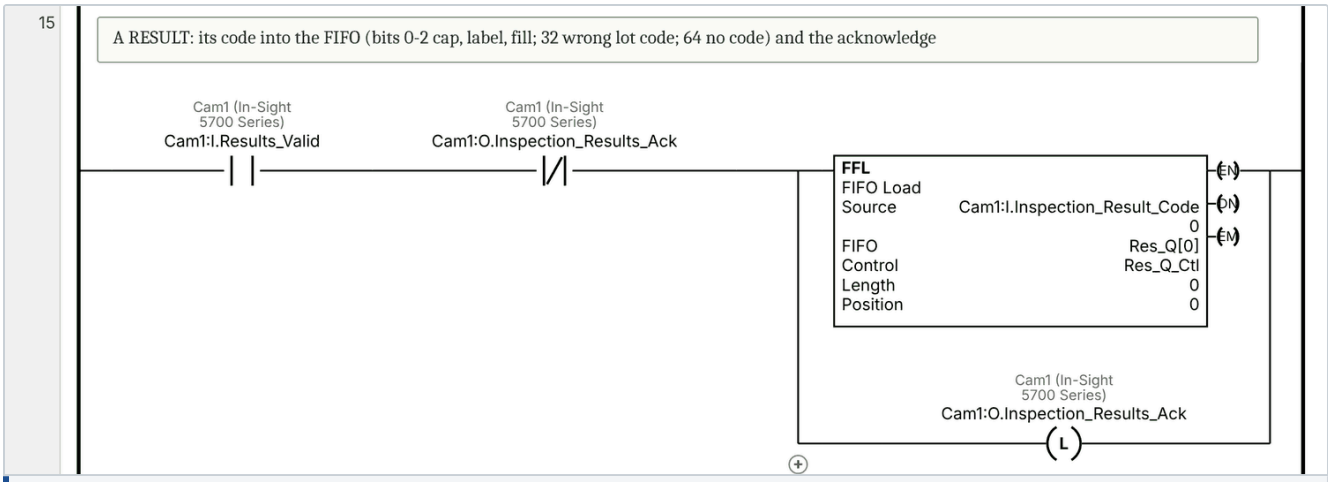
   Path 2

   1. `OTU(Trig_Req)` *Output Unlatch* does: unlatches `Trig_Req` (A bottle has reached PE_Trigger and wants an inspection) (turns it off).

   Path 3

   1. `OTU(Cam1:O.Set_User_Data)` *Output Unlatch* does: unlatches `Cam1:O.Set_User_Data` (turns it off).

RUNG 15




```
XIC(Cam1:I.Results_Valid)XIO(Cam1:O.Inspection_Results_Ack)
[FFL(Cam1:I.Inspection_Result_Code,Res_Q[0],Res_Q_Ctl,8),OTL(Cam1:O.Inspection_
Results_Ack)];
```

HOW THIS RUNG WORKS

In words. When `Cam1:I.Results_Valid` is on and `Cam1:O.Inspection_Results_Ack` is off, the rung loads `Cam1:I.Inspection_Result_Code` into the queue `Res_Q[0]` and latches `Cam1:O.Inspection_Results_Ack` on.

Left to right:

1. `XIC(Cam1:I.Results_Valid)` *Examine If Closed* asks: true when `Cam1:I.Results_Valid` is on.
2. `XIO(Cam1:O.Inspection_Results_Ack)` *Examine If Open* asks: true when `Cam1:O.Inspection_Results_Ack` is off.
3. A branch of 2 parallel paths: each path is tested on its own and does its own work.

   Path 1

   1. `FFL(Cam1:I.Inspection_Result_Code,Res_Q[0],Res_Q_Ctl,8)` *FIFO Load* does: loads `Cam1:I.Inspection_Result_Code` into the queue `Res_Q[0]`.

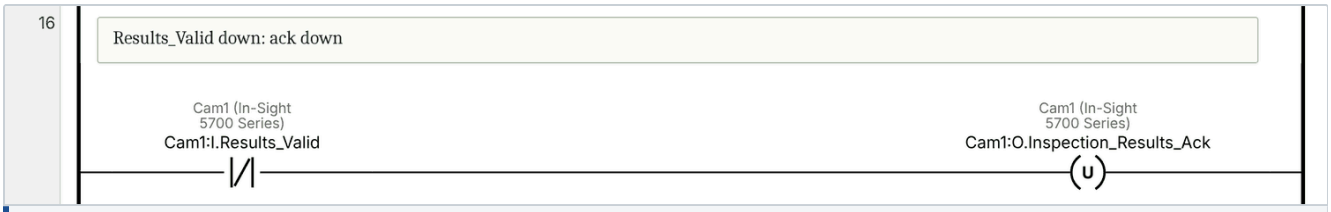
   Path 2

   1. `OTL(Cam1:O.Inspection_Results_Ack)` *Output Latch* does: latches `Cam1:O.Inspection_Results_Ack` on.

Why it is built this way:

A latch: it stays on until an OTU clears it.

RUNG 16




```
XIO(Cam1:I.Results_Valid)OTU(Cam1:O.Inspection_Results_Ack);
```

HOW THIS RUNG WORKS

In words. When `Cam1:I.Results_Valid` is off, the rung unlatches `Cam1:O.Inspection_Results_Ack` (turns it off).

Left to right:

1. `XIO(Cam1:I.Results_Valid)` *Examine If Open* asks: true when `Cam1:I.Results_Valid` is off.
2. `OTU(Cam1:O.Inspection_Results_Ack)` *Output Unlatch* does: unlatches `Cam1:O.Inspection_Results_Ack` (turns it off).

RUNG 17

```
XIC(PE_Reject)ONS(Rej_ONS)FFU(Res_Q[0],Cur,Res_Q_Ctl,8);
```

HOW THIS RUNG WORKS

In words. When `PE_Reject` (Presence eye ahead of the reject pusher) is on and it is the first scan they are true (the one-shot, remembered in `Rej_ONS`), the rung unloads the oldest entry of `Res_Q[0]` into `Cur` (The result code of the bottle at the reject pusher).

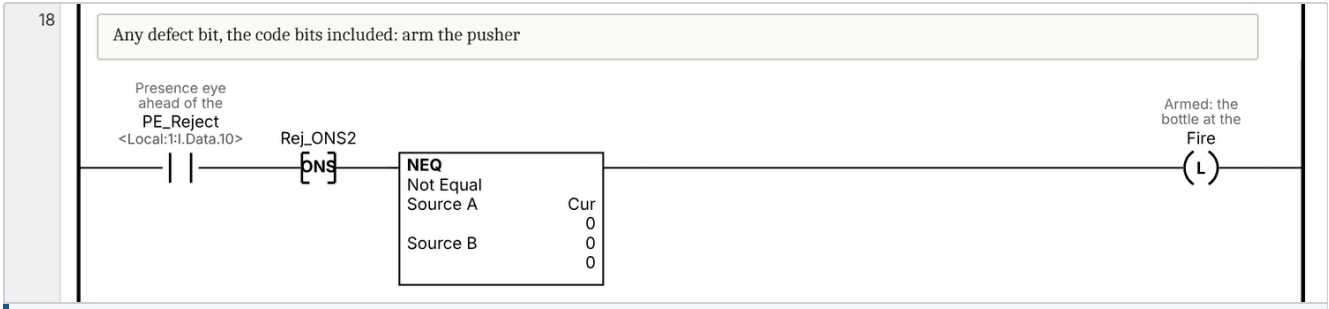
Left to right:

1. `XIC(PE_Reject)` *Examine If Closed* asks: true when `PE_Reject` (Presence eye ahead of the reject pusher) is on.
2. `ONS(Rej_ONS)` *One Shot* asks: true when it is the first scan they are true (the one-shot, remembered in `Rej_ONS`).
3. `FFU(Res_Q[0],Cur,Res_Q_Ctl,8)` *FIFO Unload* does: unloads the oldest entry of `Res_Q[0]` into `Cur` (The result code of the bottle at the reject pusher).

Why it is built this way:

One-shot: once per event, not once per scan.

RUNG 18




```
XIC(PE_Reject)ONS(Rej_ONS2)NEQ(Cur,0)OTL(Fire);
```

HOW THIS RUNG WORKS

In words. When `PE_Reject` (Presence eye ahead of the reject pusher) is on, it is the first scan they are true (the one-shot, remembered in `Rej_ONS2`) and `Cur` (The result code of the bottle at the reject pusher) is not **0**, the rung latches `Fire` (Armed: the bottle at the pusher is to be rejected) on.

Left to right:

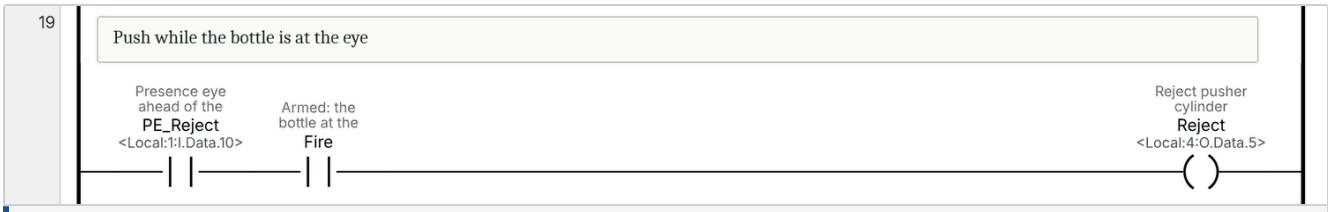
1. `XIC(PE_Reject)` *Examine If Closed* asks: true when `PE_Reject` (Presence eye ahead of the reject pusher) is on.
2. `ONS(Rej_ONS2)` *One Shot* asks: true when it is the first scan they are true (the one-shot, remembered in `Rej_ONS2`).
3. `NEQ(Cur,0)` *Not Equal* asks: true when `Cur` (The result code of the bottle at the reject pusher) is not **0**.
4. `OTL(Fire)` *Output Latch* does: latches `Fire` (Armed: the bottle at the pusher is to be rejected) on.

Why it is built this way:

One-shot: once per event, not once per scan.

A latch: it stays on until an OTU clears it.

RUNG 19




```
XIC(PE_Reject)XIC(Fire)OTE(Reject);
```

PREDICT IT · 7 `PE_Reject` is on and `Fire` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

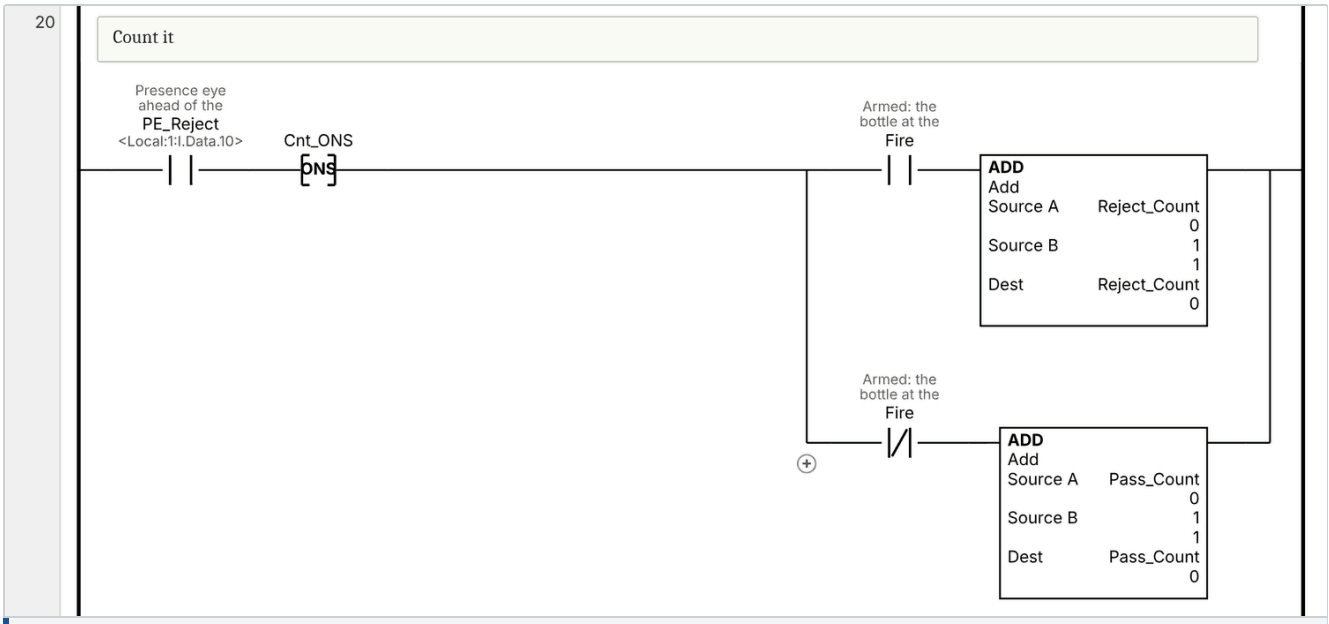
HOW THIS RUNG WORKS

In words. When `PE_Reject` (Presence eye ahead of the reject pusher) is on and `Fire` (Armed: the bottle at the pusher is to be rejected) is on, the rung turns `Reject` (Reject pusher cylinder) on (and off again whenever the rung is false).

Left to right:

1. `XIC(PE_Reject)` *Examine If Closed* asks: true when `PE_Reject` (Presence eye ahead of the reject pusher) is on.
2. `XIC(Fire)` *Examine If Closed* asks: true when `Fire` (Armed: the bottle at the pusher is to be rejected) is on.
3. `OTE(Reject)` *Output Energize* does: turns `Reject` (Reject pusher cylinder) on (and off again whenever the rung is false).

RUNG 20




```
XIC(PE_Reject)ONS(Cnt_ONS)
[XIC(Fire)ADD(Reject_Count,1,Reject_Count),XIO(Fire)ADD(Pass_Count,1,Pass_Count
)];
```

HOW THIS RUNG WORKS

In words. When `PE_Reject` (Presence eye ahead of the reject pusher) is on and it is the first scan they are true (the one-shot, remembered in `Cnt_ONS`), the rung picks its path: if `Fire` (Armed: the bottle at the pusher is to be rejected) is on, it makes `Reject_Count = Reject_Count + 1`; if `Fire` is off, it makes `Pass_Count = Pass_Count + 1`.

Left to right:

1. `XIC(PE_Reject)` *Examine If Closed* asks: true when `PE_Reject` (Presence eye ahead of the reject pusher) is on.
2. `ONS(Cnt_ONS)` *One Shot* asks: true when it is the first scan they are true (the one-shot, remembered in `Cnt_ONS`).
3. A branch of 2 parallel paths: each path is tested on its own and does its own work.

   Path 1

   1. `XIC(Fire)` *Examine If Closed* asks: true when `Fire` (Armed: the bottle at the pusher is to be rejected) is on.
   2. `ADD(Reject_Count,1,Reject_Count)` *Add* does: makes `Reject_Count = Reject_Count + 1`. Every scan the rung is true; behind a one-shot it adds once per event.

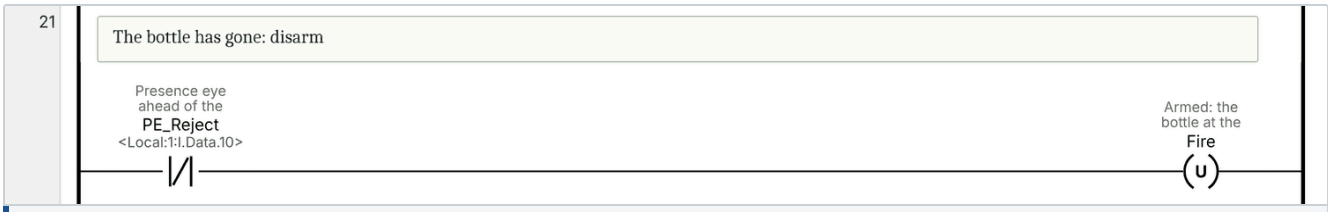
   Path 2

   1. `XIO(Fire)` *Examine If Open* asks: true when `Fire` is off.
   2. `ADD(Pass_Count,1,Pass_Count)` *Add* does: makes `Pass_Count = Pass_Count + 1`.

Why it is built this way:

One-shot: once per event, not once per scan.

RUNG 21




```
XIO(PE_Reject)OTU(Fire);
```

HOW THIS RUNG WORKS

In words. When `PE_Reject` (Presence eye ahead of the reject pusher) is off, the rung unlatches `Fire` (Armed: the bottle at the pusher is to be rejected) (turns it off).

Left to right:

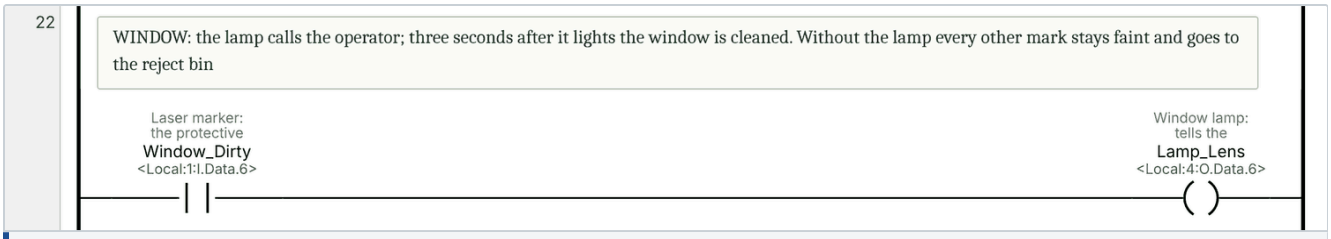
1. `XIO(PE_Reject)` *Examine If Open* asks: true when `PE_Reject` (Presence eye ahead of the reject pusher) is off.
2. `OTU(Fire)` *Output Unlatch* does: unlatches `Fire` (Armed: the bottle at the pusher is to be rejected) (turns it off).

TRY IT: A CAMERA THAT IS TOLD NOTHING

Delete rung 12 (the user data) and take `XIC(Cam1:I.Set_User_Data_Ack)` out of rung 13. The camera still reads every code, but with no lot to compare it against bit 32 never comes on. Now break the marker on purpose as in the box above and watch bottles with the wrong lot land in Good_Bin at −100 each. The verify is only as good as what you tell the camera to expect.

The window and the fault lamp

RUNG 22




```
XIC(Window_Dirty)OTE(Lamp_Lens);
```

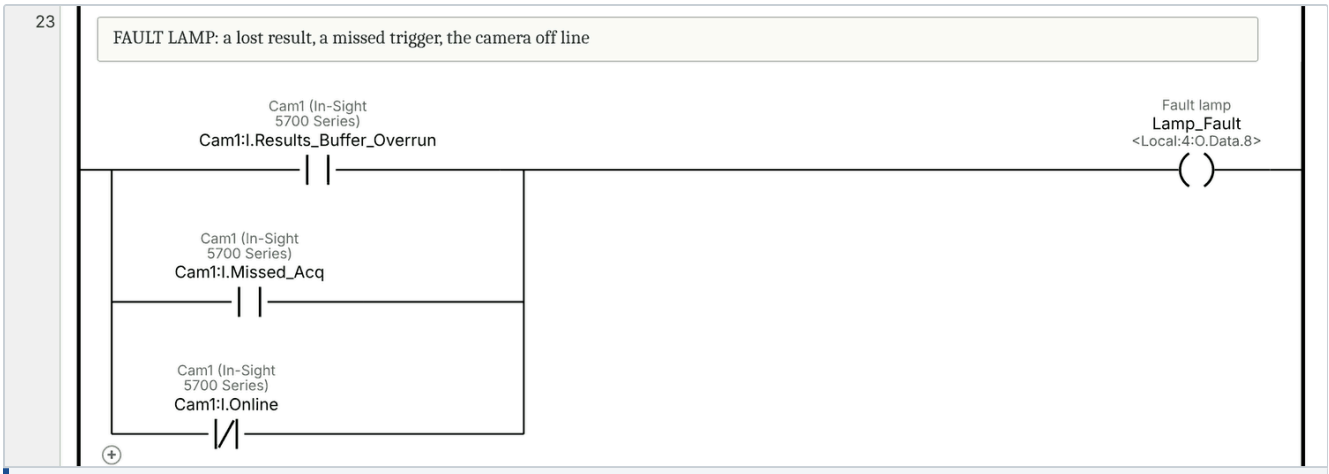
HOW THIS RUNG WORKS

In words. When `Window_Dirty` (Laser marker: the protective window has fumed up, marks come out faint) is on, the rung turns `Lamp_Lens` (Window lamp: tells the operator to clean the laser window) on (and off again whenever the rung is false).

Left to right:

1. `XIC(Window_Dirty)` *Examine If Closed* asks: true when `Window_Dirty` (Laser marker: the protective window has fumed up, marks come out faint) is on.
2. `OTE(Lamp_Lens)` *Output Energize* does: turns `Lamp_Lens` (Window lamp: tells the operator to clean the laser window) on (and off again whenever the rung is false).

RUNG 23




```
[XIC(Cam1:I.Results_Buffer_Overrun),XIC(Cam1:I.Missed_Acq),XIO(Cam1:I.Online)]OTE(Lamp_Fault);
```

PREDICT IT · 8 `Cam1:I.Results_Buffer_Overrun` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When either (`Cam1:I.Results_Buffer_Overrun` is on) *or* (`Cam1:I.Missed_Acq` is on) *or* (`Cam1:I.Online` is off), the rung turns `Lamp_Fault` (Fault lamp) on (and off again whenever the rung is false).

Left to right:

1. A branch of 3 parallel paths: power gets through if any one of them is true.
   - **Path 1**
     1. `XIC(Cam1:I.Results_Buffer_Overrun)` *Examine If Closed* asks: true when `Cam1:I.Results_Buffer_Overrun` is on.
   - **Path 2**
     1. `XIC(Cam1:I.Missed_Acq)` *Examine If Closed* asks: true when `Cam1:I.Missed_Acq` is on.
   - **Path 3**
     1. `XIO(Cam1:I.Online)` *Examine If Open* asks: true when `Cam1:I.Online` is off.
2. `OTE(Lamp_Fault)` *Output Energize* does: turns `Lamp_Fault` (Fault lamp) on (and off again whenever the rung is false).

> **Think of it like this:** parallel paths are two roads to the same town. If either road is open you get there; the rung only cares that power reaches the far end, not which way it went.

> **TRY IT: THE LASER INTERLOCK AND THE WINDOW**
>
> Delete rung 3 (Laser_On). The marker is never Ready, so the message never loads, nothing is marked, and all eighteen bottles go to Reject_Bin for no code: eighteen right rejects and nothing shipped. Put it back and delete rung 22 instead: after ten marks Window_Dirty comes on and stays on, every other mark is faint, the camera reads 64 on each and rejects it. With the lamp, the operator cleans the window three seconds after it lights and the faint marks stop.

Run it

Load the program, Run mode, Start. The marker loads L0101 before the first bottle reaches it (the FIFO head is set at the first release), marks the first six, and the moment the sixth has been marked the FIFO head reads 102 and the log shows *loading message L0102*. 18 of 18 bottles in the right bin, nothing lost, in 34 s against a par of 45 s: 2000 points and three stars.

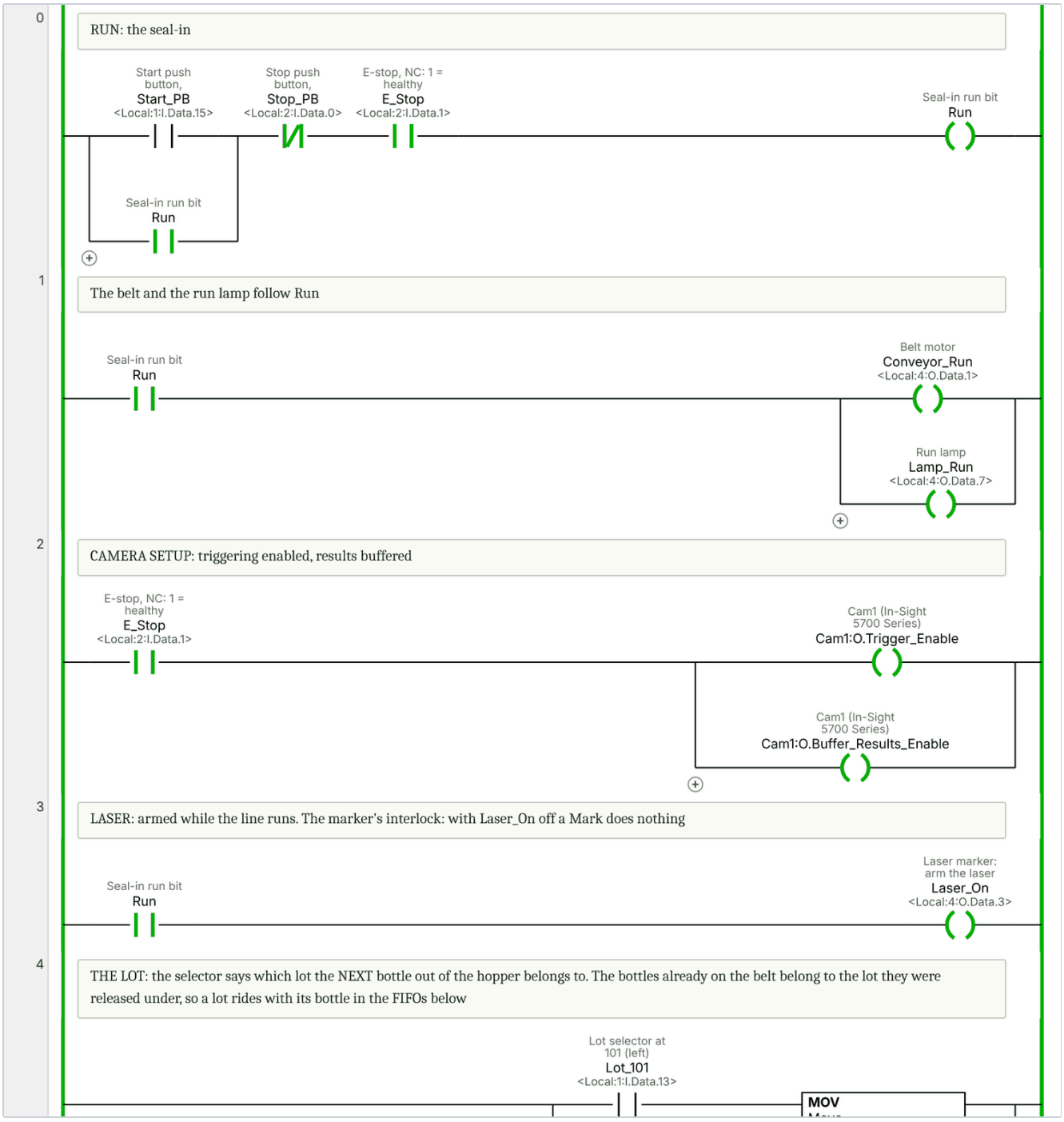


Figure 187. Level 11 online, the top of the routine.

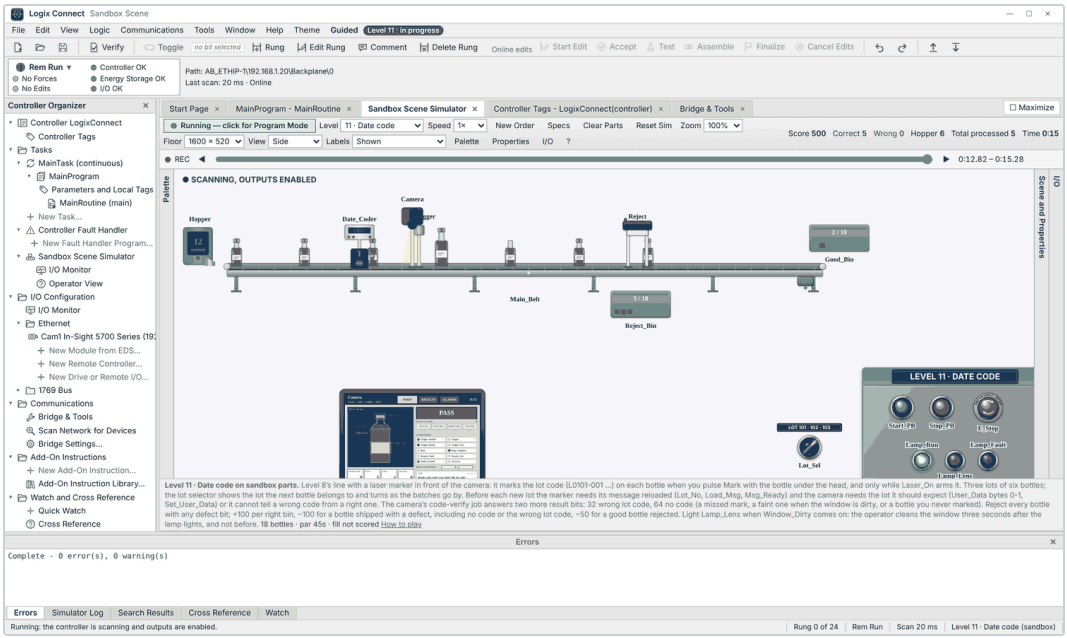


Figure 188. The machine halfway through the run.

Predict it: the answers

1. The rung is false at `XIO(Stop_PB)`: `Stop_PB` is on, so `Run` is **off** (an OTE writes 0 when its rung is false).
2. Every condition on the way is met, so `Lot_Want` becomes **103**.
3. Every condition on the way is met, so `Rel_T` times toward its preset.
4. Every condition on the way is met, so `Coder_Lot` gets the value of `Lot_Q[0]`.
5. The rung is false at the EQU: `Coder_Lot` is not `Mark_Lot`, so `Coder_Mark` is **off** (an OTE writes 0 when its rung is false).
6. Every condition on the way is met, so `Cam1:O.Trigger` latches **on**.
7. Every condition on the way is met, so `Reject` is **on**.
8. Every condition on the way is met, so `Lamp_Fault` is **on**.

44 Level 12 — Kitchen changeover

> **IN THE APP**
>
> File → New Project… → A graded level → **12**. Guided mode opens on this chapter: the tags, every rung below under its fold, a Verify check, and the run results. **File → Load Reference Solution** loads the program printed here.

Level 7 made three batches of one family of recipes in one tank. A real kitchen changes product, and the change is where the trouble is. Level 12 is Level 7's kitchen making two batches of **Soda** (water and cola syrup, 5:1) and then two of **Juice** (water and orange concentrate, 3:1). Between them comes the changeover, and three things about it are new. The tank's outlet sits above the bottom, so the outlet pump leaves a **heel**, five per cent of the tank, that carries the last batch into the next one unless it is rinsed out. The syrup tote has to be swapped for the concentrate tote, by an operator, and only when the tank is down to its heel. And the recipe changes: not just the amounts, but how you dose them, because a heel in the tank means topping up to the HI probe no longer gives you the ratio.

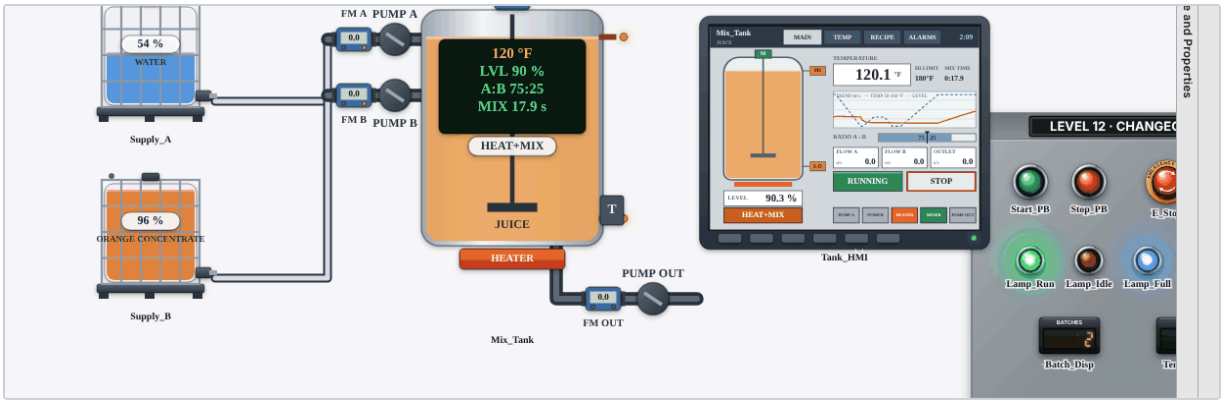


Figure 189. Level 12 after the changeover: Supply_B now holds orange concentrate, the first Juice batch is heating and mixing (120 °F on its way to 160, A:B 75:25), the HMI on the Juice recipe. Product_Sel, Lamp_Swap and TOTE OK sit on the console to the right of Level 7's lamps.

The machine

- **Mix_Tank**: Level 7's, with *Heel* at 5 %: the outlet pump stops drawing at 100 units, and the level transmitter reads 5 there, not 0. A purge from the HMI drains the heel too, to the floor. The tank now keeps what is in it by ingredient (water, cola syrup, orange concentrate …), and so does every batch record.
- **Supply_A**: water. **Supply_B**: the cola syrup tote at the start. The level's operator swaps it for a full tote of orange concentrate when you ask, and picks Juice on the HMI's RECIPE page at the same time (the inlet interlock of chapter 15 refuses an ingredient the recipe does not call for).

- **Product_Sel**: SODA or JUICE, turned by the level the moment the second Soda batch is out. It says what the *next* batch is to be.
- **Lamp_Swap** and **Tote_OK_PB**: light the lamp and, if the tank is down to its heel, three seconds later the tote is swapped and TOTE OK is pressed for half a second. Light it with a batch in the tank and nothing happens but a line in the log: no operator changes a tote on a tank with product in it.
- Tank_HMI, the console, Batch_Disp and Temp_Disp are Level 7's.

How the level is scored

What happens	Points
Each batch: the ratio within 5 % of the recipe	+100 / −50
Each batch: the temperature at drain within −3 / +8 °F	+100 / −50
Each batch: mixed at least the recipe time	+100 / −50
A batch with more than 0.5 % of the other product in it	−200
A short batch (under 80 % of the tank), an overflow, a scorched batch	−50, −100, −100
Four batches out under par (270 s)	+200

The recipes are the plant's: Soda 83 % water at 100 °F, mixed 8 s; Juice 75 % water at 160 °F, mixed 10 s. Without a rinse the Juice carries about 1 % cola out of the heel, twice the limit.

Why the heel changes the dosing

Level 7 dosed A by counting its flowmeter and topped up with B to the HI probe: the ratio came out of the probe's height. With 100 units of the last batch already in the tank, the probe is reached 100 units sooner, and whatever B is short, the ratio is off. So Level 12 doses **both** ingredients by their flowmeters, to counts the recipe rungs set, and the HI probe is left to the Full lamp. That is also the honest way: a recipe is amounts, not levels.

Tags this lesson needs

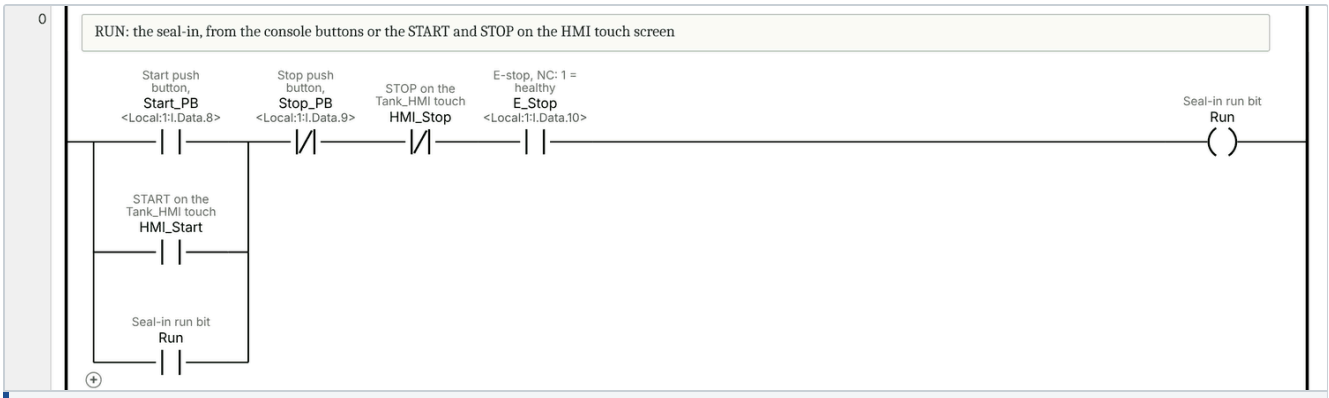
The level's alias tags are already made for you (chapter 32's table). Create these plain tags before you type the rungs; Verify reports any you miss.

Tag	Type	How to make it	Description
`Run`	BOOL	Edit Tags: name, type	Seal-in run bit
`HMI_Start`	BOOL	Edit Tags: name, type	START on the Tank_HMI touch screen (pulsed by the HMI)

Tag	Type	How to make it	Description
`HMI_Stop`	BOOL	Edit Tags: name, type	STOP on the Tank_HMI touch screen (pulsed by the HMI)
`Step`	DINT	Edit Tags: name, type	Step: 0 wait at the heel, 1 dose A, 2 dose B, 3 heat and mix, 4 drain; 5 rinse fill, 6 rinse mix, 7 rinse drain, 8 tote swap
`Cur_Juice`	BOOL	Edit Tags: name, type	The tank and the tote are set up for Juice (0: Soda)
`A_Count`	COUNTER	Edit Tags: name, type	Flowmeter A pulses this batch
`B_Count`	COUNTER	Edit Tags: name, type	Flowmeter B pulses this batch
`Target_A`	DINT	Edit Tags: name, type	Pulses of A for the selected product
`Target_B`	DINT	Edit Tags: name, type	Pulses of B for the selected product
`Target_T`	DINT	Edit Tags: name, type	Temperature for the selected product, °F
`Target_Lo`	DINT	Edit Tags: name, type	Heater comes back on below this, °F
`Mix_T`	TIMER	Edit Tags: name, type	Mixing time this batch
`Mix_Pre`	DINT	Edit Tags: name, type	Mixing time for the selected product, ms
`Heat_On`	BOOL	Edit Tags: name, type	Heater wanted (with hysteresis)
`Empty_T`	TIMER	Edit Tags: name, type	Run the outlet pump on a moment once the level reads the heel
`Rinse_T`	TIMER	Edit Tags: name, type	Rinse mixing time
`Done_ONS`	BOOL	Edit Tags: name, type	
`Rinse_ONS`	BOOL	Edit Tags: name, type	

Run and the two recipes

RUNG 0




```
[XIC(Start_PB),XIC(HMI_Start),XIC(Run)]XIO(Stop_PB)XIO(HMI_Stop)XIC(E_Stop)OTE(Run);
```

PREDICT IT · 1 `Start_PB` is on, `Stop_PB` is on, `HMI_Stop` is off and `E_Stop` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When either (`Start_PB` (Start push button, momentary) is on) *or* (`HMI_Start` (START on the Tank_HMI touch screen (pulsed by the HMI)) is on) *or* (`Run` (Seal-in run bit) is on), `Stop_PB` (Stop push button, momentary) is off, `HMI_Stop` (STOP on the Tank_HMI touch screen (pulsed by the HMI)) is off and `E_Stop` (E-stop, NC: 1 = healthy) is on, the rung turns `Run` on (and off again whenever the rung is false).

Left to right:

1. A branch of 3 parallel paths: power gets through if any one of them is true.
   - **Path 1**
     1. `XIC(Start_PB)` *Examine If Closed* asks: true when `Start_PB` (Start push button, momentary) is on.
   - **Path 2**
     1. `XIC(HMI_Start)` *Examine If Closed* asks: true when `HMI_Start` (START on the Tank_HMI touch screen (pulsed by the HMI)) is on.
   - **Path 3**
     1. `XIC(Run)` *Examine If Closed* asks: true when `Run` (Seal-in run bit) is on.
2. `XIO(Stop_PB)` *Examine If Open* asks: true when `Stop_PB` (Stop push button, momentary) is off.
3. `XIO(HMI_Stop)` *Examine If Open* asks: true when `HMI_Stop` (STOP on the Tank_HMI touch screen (pulsed by the HMI)) is off.
4. `XIC(E_Stop)` *Examine If Closed* asks: true when `E_Stop` (E-stop, NC: 1 = healthy) is on.
5. `OTE(Run)` *Output Energize* does: turns `Run` on (and off again whenever the rung is false). Every scan it writes the rung state: true on, false off.

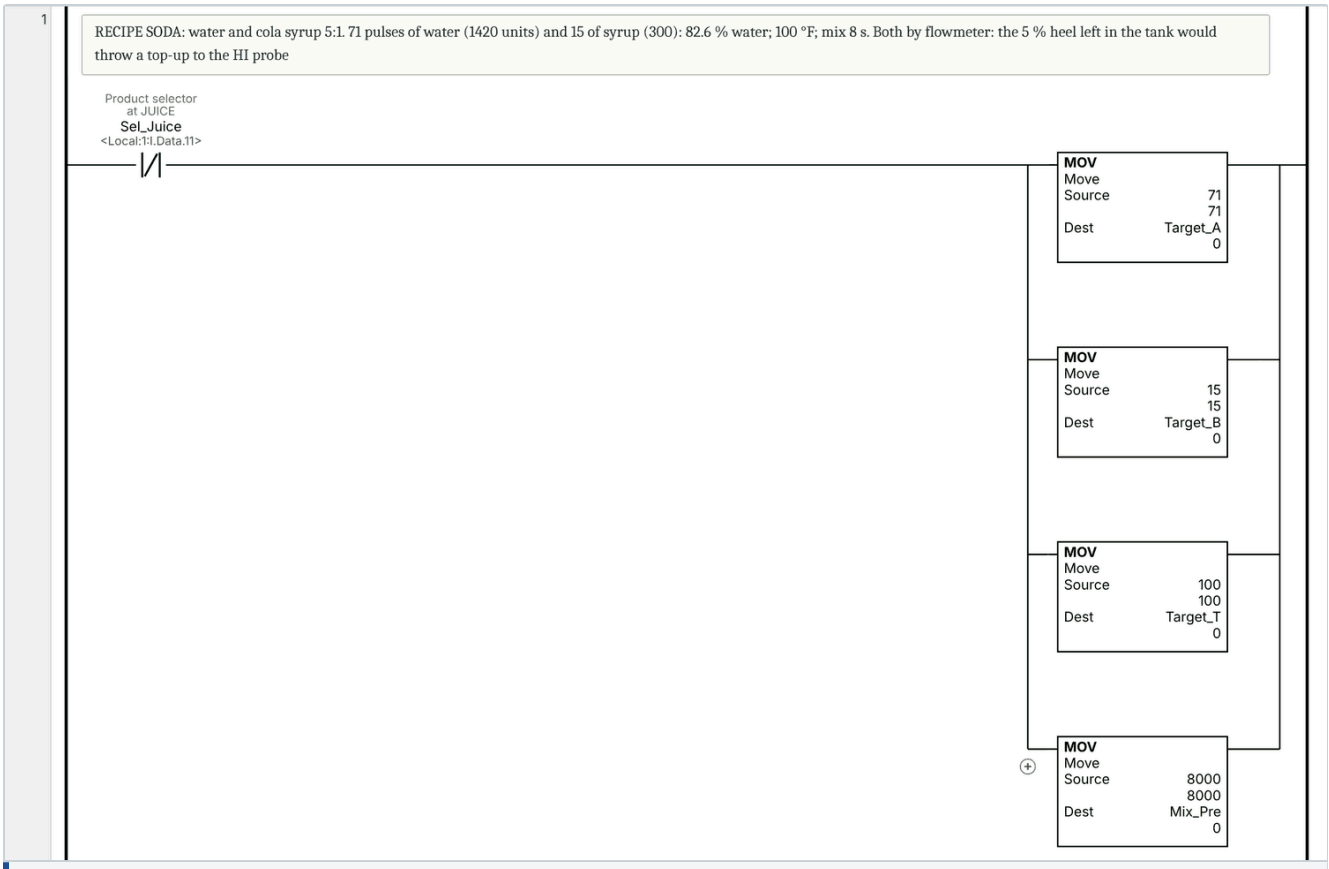
Why it is built this way:

Seal-in. The start condition turns `Run` on, and `Run` 's own contact in the parallel path keeps the rung true after the start goes away. Anything in series outside the branch (a stop, a fault) breaks it, and then it stays off until the start is made again.

> **Think of it like this:** a doorbell button and a hand. The start button is the doorbell, pressed for a moment. The seal-in contact is a hand that grabs the circuit as soon as the output comes on and keeps holding after you let go. The stop button, in series, is the only thing that can knock that hand away.

> **Watch for:** The stop has to be in series outside the branch. Put it inside the start path only and the seal-in path bypasses it: the stop no longer stops.

RUNG 1




```
XIO(Sel_Juice)
[MOV(71,Target_A),MOV(15,Target_B),MOV(100,Target_T),MOV(8000,Mix_Pre)];
```

HOW THIS RUNG WORKS

In words. When `Sel_Juice` (Product selector at JUICE (right); 0 = SODA) is off, the rung puts **71** into `Target_A` (Pulses of A for the selected product), puts **15** into `Target_B` (Pulses of B for the selected product), puts **100** into `Target_T` (Temperature for the selected product, °F) and puts **8000** into `Mix_Pre` (Mixing time for the selected product, ms).

Left to right:

1. `XIO(Sel_Juice)` *Examine If Open* asks: true when `Sel_Juice` (Product selector at JUICE (right); 0 = SODA) is off.
2. A branch of 4 parallel paths: each path is tested on its own and does its own work.

Path 1

1. `MOV(71,Target_A)` *Move* does: puts **71** into `Target_A` (Pulses of A for the selected product). It writes every scan the rung is true.

Path 2

1. `MOV(15,Target_B)` *Move* does: puts **15** into `Target_B` (Pulses of B for the selected product).

Path 3

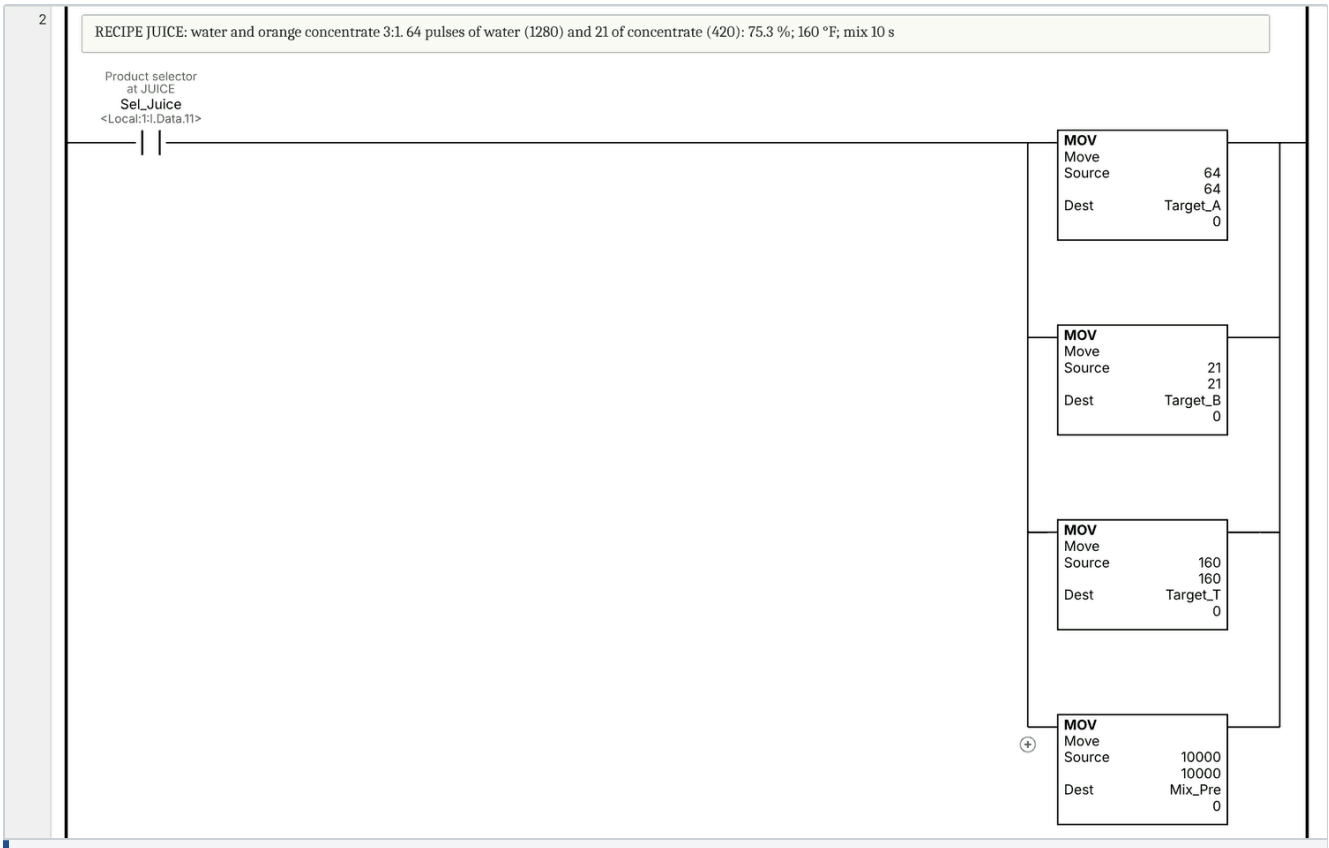
1. `MOV(100,Target_T)` *Move* does: puts **100** into `Target_T` (Temperature for the selected product, °F).

Path 4

1. `MOV(8000,Mix_Pre)` *Move* does: puts **8000** into `Mix_Pre` (Mixing time for the selected product, ms).

> **Think of it like this:** a rung is a pipe with valves in it, read left to right. Every contact is a valve: XIC is open while its bit is 1, XIO while its bit is 0. Water reaches the output at the right only if every valve in a row is open. The output does not remember anything: on every scan it is simply wet or dry.

RUNG 2




```
XIC(Sel_Juice)
[MOV(64,Target_A),MOV(21,Target_B),MOV(160,Target_T),MOV(10000,Mix_Pre)];
```

HOW THIS RUNG WORKS

In words. When `Sel_Juice` (Product selector at JUICE (right); 0 = SODA) is on, the rung puts **64** into `Target_A` (Pulses of A for the selected product), puts **21** into `Target_B` (Pulses of B for the selected product), puts **160** into `Target_T` (Temperature for the selected product, °F) and puts **10000** into `Mix_Pre` (Mixing time for the selected product, ms).

Left to right:

1. `XIC(Sel_Juice)` *Examine If Closed* asks: true when `Sel_Juice` (Product selector at JUICE (right); 0 = SODA) is on.
2. A branch of 4 parallel paths: each path is tested on its own and does its own work.

   Path 1

   1. `MOV(64,Target_A)` *Move* does: puts **64** into `Target_A` (Pulses of A for the selected product).

   Path 2

   1. `MOV(21,Target_B)` *Move* does: puts **21** into `Target_B` (Pulses of B for the selected product).

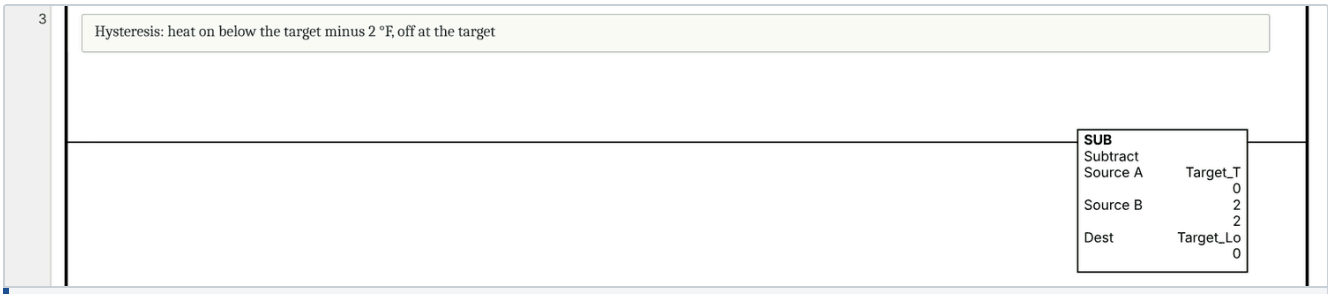
   Path 3

   1. `MOV(160,Target_T)` *Move* does: puts **160** into `Target_T` (Temperature for the selected product, °F).

   Path 4

   1. `MOV(10000,Mix_Pre)` *Move* does: puts **10000** into `Mix_Pre` (Mixing time for the selected product, ms).

RUNG 3




```
SUB(Target_T,2,Target_Lo);
```

HOW THIS RUNG WORKS

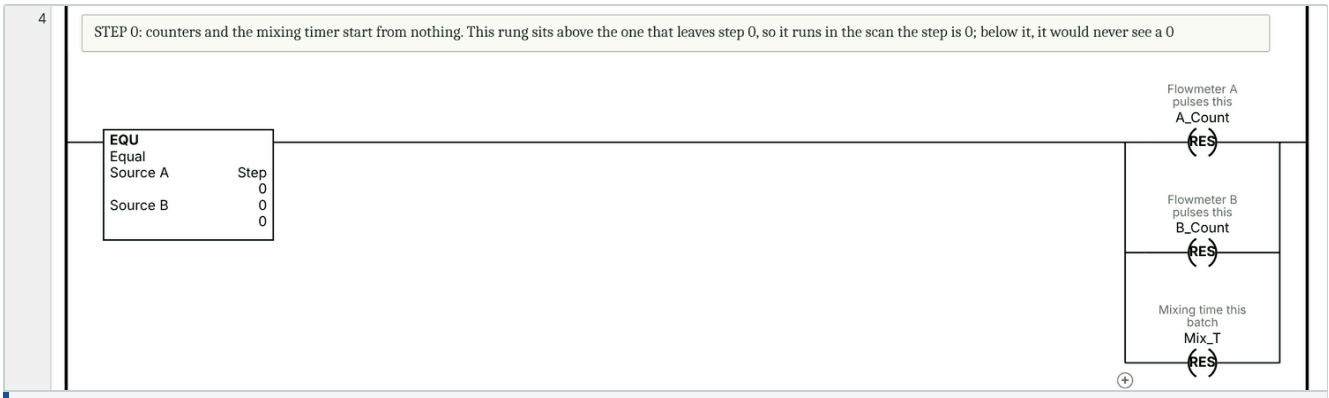
In words. Every scan, with nothing to test, the rung makes `Target_Lo` (Heater comes back on below this, °F) = `Target_T` (Temperature for the selected product, °F) − **2**.

Left to right:

1. `SUB(Target_T,2,Target_Lo)` *Subtract* does: makes `Target_Lo` (Heater comes back on below this, °F) = `Target_T` (Temperature for the selected product, °F) − **2**.

Step 0: at the heel, batch or changeover

RUNG 4




```
EQU(Step,0)[RES(A_Count),RES(B_Count),RES(Mix_T)];
```

HOW THIS RUNG WORKS

In words. When `Step` (Step: 0 wait at the heel, 1 dose A, 2 dose B, 3 heat and mix, 4 drain; 5 rinse fill, 6 rinse mix, 7 rinse drain, 8 tote swap) equals **0**, the rung resets `A_Count` (Flowmeter A pulses this batch), resets `B_Count` (Flowmeter B pulses this batch) and resets `Mix_T` (Mixing time this batch).

Left to right:

1. `EQU(Step,0)` *Equal* asks: true when `Step` (Step: 0 wait at the heel, 1 dose A, 2 dose B, 3 heat and mix, 4 drain; 5 rinse fill, 6 rinse mix, 7 rinse drain, 8 tote swap) equals **0**. Both values are compared every scan.
2. A branch of 3 parallel paths: each path is tested on its own and does its own work.

   Path 1

   1. `RES(A_Count)` *Reset* does: resets `A_Count` (Flowmeter A pulses this batch). It zeroes a timer or counter (accumulator and status bits), or a control (its position and status bits).

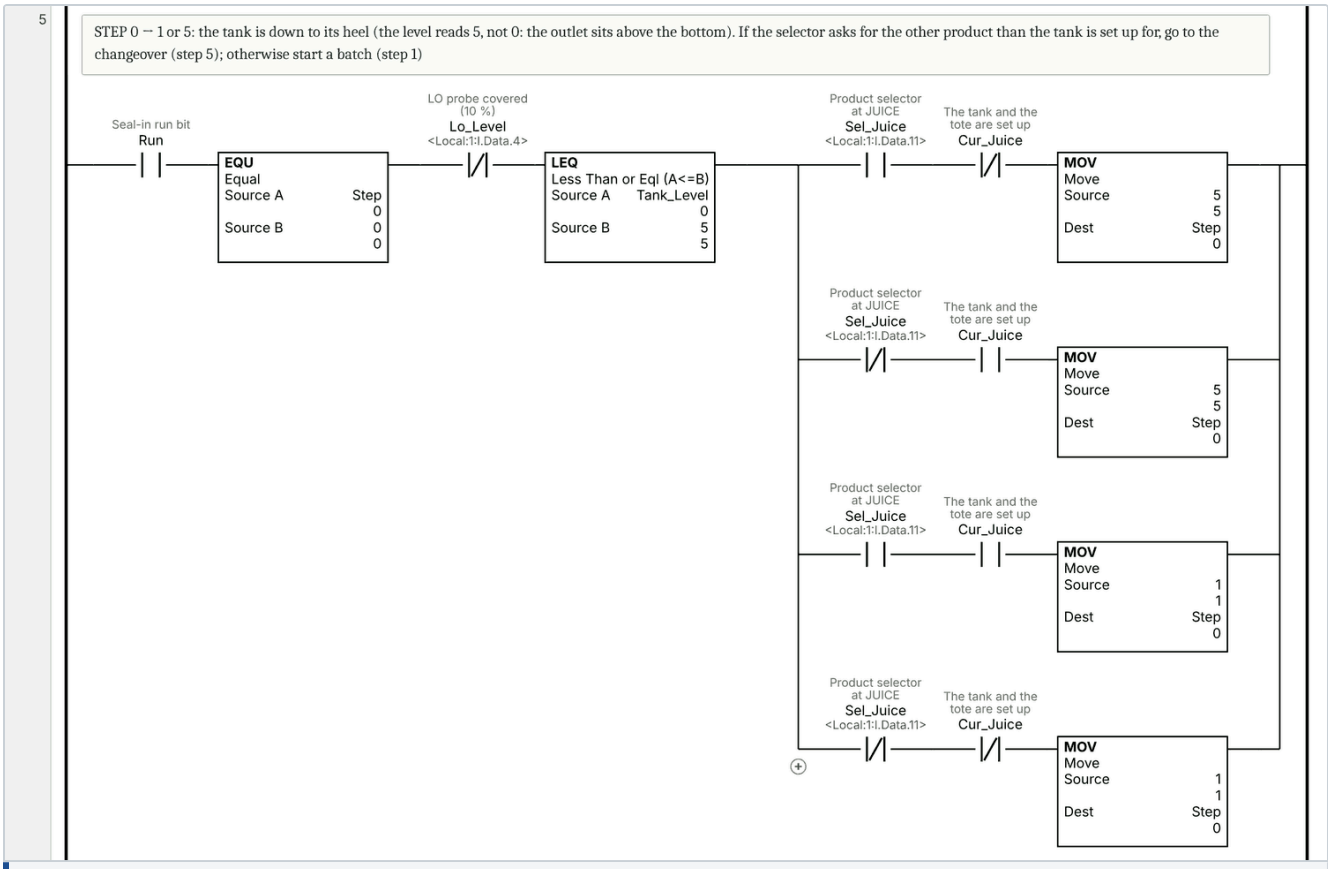
   Path 2

   1. `RES(B_Count)` *Reset* does: resets `B_Count` (Flowmeter B pulses this batch).

   Path 3

   1. `RES(Mix_T)` *Reset* does: resets `Mix_T` (Mixing time this batch).

RUNG 5




```
XIC(Run)EQU(Step,0)XIO(Lo_Level)LEQ(Tank_Level,5)
[XIC(Sel_Juice)XIO(Cur_Juice)MOV(5,Step),XIO(Sel_Juice)XIC(Cur_Juice)MOV(5,Step
),XIC(Sel_Juice)XIC(Cur_Juice)MOV(1,Step),XIO(Sel_Juice)XIO(Cur_Juice)MOV(1,Ste
p)];
```

PREDICT IT · 2 `Run` is on, `Step` is 0, `Lo_Level` is off, `Tank_Level` is 5, `Sel_Juice` is on and `Cur_Juice` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Run` (Seal-in run bit) is on, `Step` (Step: 0 wait at the heel, 1 dose A, 2 dose B, 3 heat and mix, 4 drain; 5 rinse fill, 6 rinse mix, 7 rinse drain, 8 tote swap) equals **0**, `Lo_Level` (LO probe covered (10 %)) is off and `Tank_Level` (Tank level 0-100 %) is at most **5**, the rung picks its path: if (`Sel_Juice` (Product selector at JUICE (right); 0 = SODA) is on and `Cur_Juice` (The tank and the tote are set up for Juice (0: Soda)) is off) *or* (`Sel_Juice` is off and `Cur_Juice` is on) (that is, when `Sel_Juice` and `Cur_Juice` disagree), it puts **5** into `Step` ; if (`Sel_Juice` is on and `Cur_Juice` is on) *or* (`Sel_Juice` is off and `Cur_Juice` is off) (that is, when `Sel_Juice` and `Cur_Juice` agree), it puts **1** into `Step` .

Left to right:

1. `XIC(Run)` *Examine If Closed* asks: true when `Run` (Seal-in run bit) is on.
2. `EQU(Step,0)` *Equal* asks: true when `Step` (Step: 0 wait at the heel, 1 dose A, 2 dose B, 3 heat and mix, 4 drain; 5 rinse fill, 6 rinse mix, 7 rinse drain, 8 tote swap) equals **0**.

3. `XIO(Lo_Level)` *Examine If Open* asks: true when `Lo_Level` (LO probe covered (10 %)) is off.
4. `LEQ(Tank_Level,5)` *Less Than or Equal* asks: true when `Tank_Level` (Tank level 0-100 %) is at most **5**.
5. A branch of 4 parallel paths: each path is tested on its own and does its own work.

Path 1

1. `XIC(Sel_Juice)` *Examine If Closed* asks: true when `Sel_Juice` (Product selector at JUICE (right); 0 = SODA) is on.
2. `XIO(Cur_Juice)` *Examine If Open* asks: true when `Cur_Juice` (The tank and the tote are set up for Juice (0: Soda)) is off.
3. `MOV(5,Step)` *Move* does: puts **5** into `Step` .

Path 2

1. `XIO(Sel_Juice)` *Examine If Open* asks: true when `Sel_Juice` is off.
2. `XIC(Cur_Juice)` *Examine If Closed* asks: true when `Cur_Juice` is on.
3. `MOV(5,Step)` *Move* does: puts **5** into `Step` .

Path 3

1. `XIC(Sel_Juice)` *Examine If Closed* asks: true when `Sel_Juice` is on.
2. `XIC(Cur_Juice)` *Examine If Closed* asks: true when `Cur_Juice` is on.
3. `MOV(1,Step)` *Move* does: puts **1** into `Step` .

Path 4

1. `XIO(Sel_Juice)` *Examine If Open* asks: true when `Sel_Juice` is off.
2. `XIO(Cur_Juice)` *Examine If Open* asks: true when `Cur_Juice` is off.
3. `MOV(1,Step)` *Move* does: puts **1** into `Step` .

Why it is built this way:

A step of a sequence. The rung only acts while `Step` is 0; when its conditions are met it writes 1 or 5 into `Step` , and from then on the rungs that test for 1 or 5 take over. One step number at a time means one thing happening at a time.

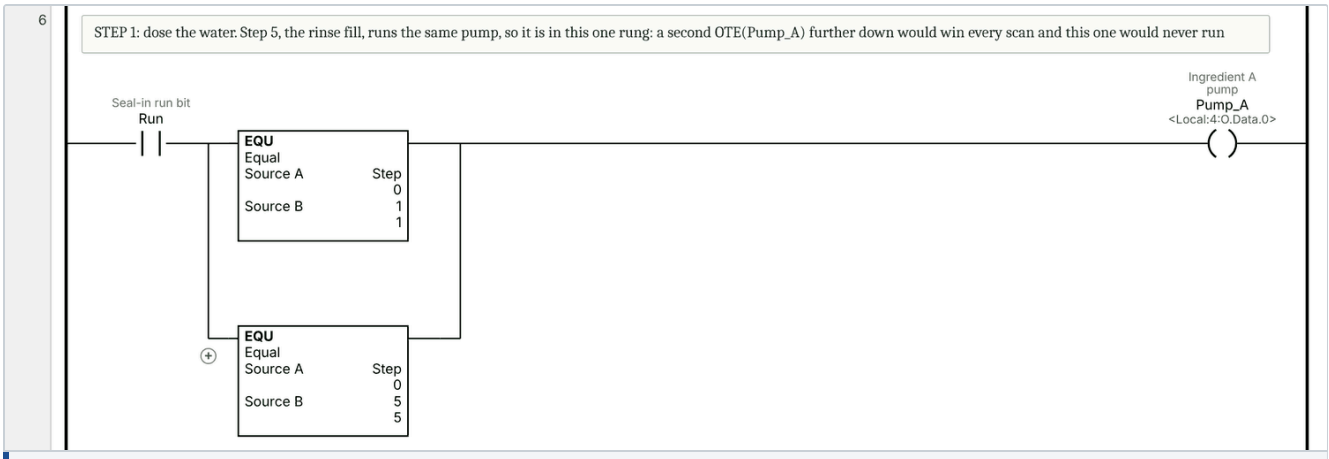
> **Think of it like this:** a recipe card with numbered steps and one finger on it. Each rung only reads the line under the finger; when its line is done it moves the finger to the next number. Because the finger can only be in one place, only one step can ever be "doing".

> **Watch for:** A rung further down that tests for the new step number sees it in this same scan. Rung order decides whether the next step starts now or next scan.

> **WATCH FOR: A RESET THAT NEVER RUNS**
>
> Rung 4 resets the counters while Step is 0, and it sits above rung 5, which leaves step 0. Swap the two and rung 4 never sees a 0: the step is already 1 by the time it is scanned. The counters keep last batch's counts, both dosing steps pass in a scan, and a batch of nothing goes to heat and mix. Rung order is part of the logic.

Dose, heat, mix, drain

RUNG 6




```
XIC(Run)[EQU(Step,1),EQU(Step,5)]OTE(Pump_A);
```

PREDICT IT · 3 `Run` is off and `Step` is 1. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Run` (Seal-in run bit) is on and either (`Step` (Step: 0 wait at the heel, 1 dose A, 2 dose B, 3 heat and mix, 4 drain; 5 rinse fill, 6 rinse mix, 7 rinse drain, 8 tote swap) equals **1**) *or* (`Step` equals **5**), the rung turns `Pump_A` (Ingredient A pump) on (and off again whenever the rung is false).

Left to right:

1. `XIC(Run)` *Examine If Closed* asks: true when `Run` (Seal-in run bit) is on.
2. A branch of 2 parallel paths: power gets through if any one of them is true.

   Path 1

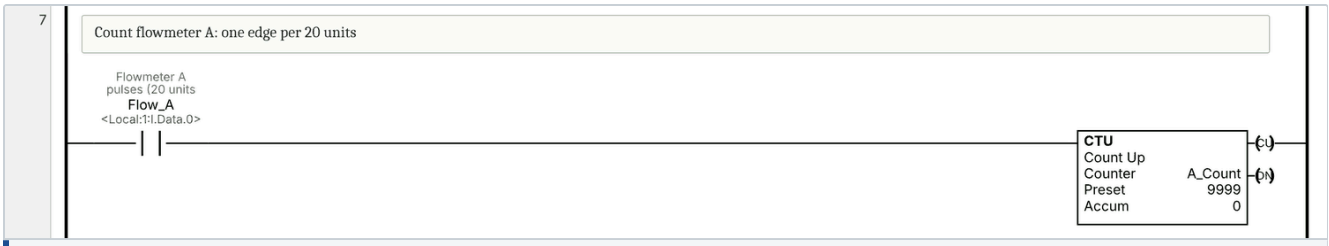
   1. `EQU(Step,1)` *Equal* asks: true when `Step` (Step: 0 wait at the heel, 1 dose A, 2 dose B, 3 heat and mix, 4 drain; 5 rinse fill, 6 rinse mix, 7 rinse drain, 8 tote swap) equals **1**.

   Path 2

   1. `EQU(Step,5)` *Equal* asks: true when `Step` equals **5**.
3. `OTE(Pump_A)` *Output Energize* does: turns `Pump_A` (Ingredient A pump) on (and off again whenever the rung is false).

> **Think of it like this:** parallel paths are two roads to the same town. If either road is open you get there; the rung only cares that power reaches the far end, not which way it went.

RUNG 7




```
XIC(Flow_A)CTU(A_Count,9999,0);
```

HOW THIS RUNG WORKS

In words. When `Flow_A` (Flowmeter A pulses (20 units each)) is on, the rung counts `A_Count` (Flowmeter A pulses this batch) up by one (preset 9999).

Left to right:

1. `XIC(Flow_A)` *Examine If Closed* asks: true when `Flow_A` (Flowmeter A pulses (20 units each)) is on.
2. `CTU(A_Count,9999,0)` *Count Up* does: counts `A_Count` (Flowmeter A pulses this batch) up by one (preset 9999). It counts false-to-true transitions of the rung, not scans; .DN is on once the count reaches the preset; it keeps counting past it.

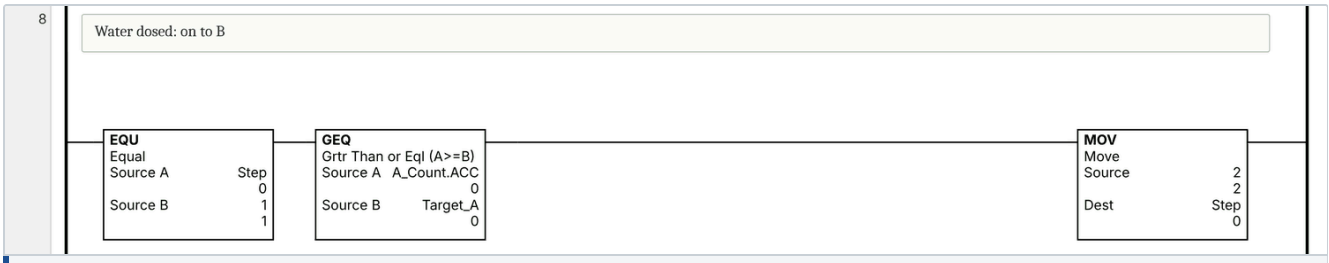
Why it is built this way:

A counter. It counts transitions, not time: each false-to-true of the rung adds one. A RES elsewhere clears it.

> **Think of it like this:** the clicker a doorman holds: one click for every person who walks through the door, however long they stand in it. The number stays on the clicker until somebody presses reset.

> **Watch for:** A counter keeps its count through a stop and a restart. Reset it on purpose where the count should start again.

RUNG 8




```
EQU(Step,1)GEQ(A_Count.ACC,Target_A)MOV(2,Step);
```

PREDICT IT · 4 `Step` is 2, `A_Count.ACC` (the accumulated value of `A_Count` : Flowmeter A pulses this batch) is 500 and `Target_A` is 500. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Step` (Step: 0 wait at the heel, 1 dose A, 2 dose B, 3 heat and mix, 4 drain; 5 rinse fill, 6 rinse mix, 7 rinse drain, 8 tote swap) equals **1** and `A_Count.ACC` (the accumulated value of `A_Count` : Flowmeter A pulses this batch) is at least `Target_A` (Pulses of A for the selected product), the rung puts **2** into `Step` .

Left to right:

1. `EQU(Step,1)` *Equal* asks: true when `Step` (Step: 0 wait at the heel, 1 dose A, 2 dose B, 3 heat and mix, 4 drain; 5 rinse fill, 6 rinse mix, 7 rinse drain, 8 tote swap) equals **1**.
2. `GEQ(A_Count.ACC,Target_A)` *Greater Than or Equal* asks: true when `A_Count.ACC` (the accumulated value of `A_Count` : Flowmeter A pulses this batch) is at least `Target_A` (Pulses of A for the selected product).
3. `MOV(2,Step)` *Move* does: puts **2** into `Step` .

Why it is built this way:

A step of the sequence: from Step = 1 to 2.

RUNG 9

```
XIC(Run)EQU(Step,2)OTE(Pump_B);
```

PREDICT IT · 5 `Run` is on and `Step` is 2. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

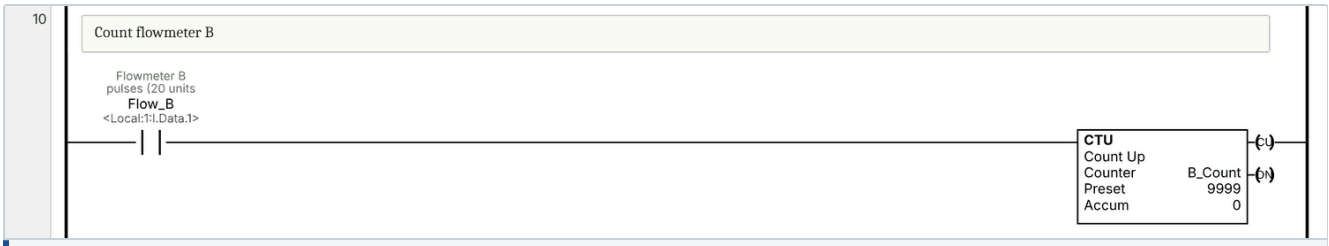
HOW THIS RUNG WORKS

In words. When `Run` (Seal-in run bit) is on and `Step` (Step: 0 wait at the heel, 1 dose A, 2 dose B, 3 heat and mix, 4 drain; 5 rinse fill, 6 rinse mix, 7 rinse drain, 8 tote swap) equals **2**, the rung turns `Pump_B` (Ingredient B pump) on (and off again whenever the rung is false).

Left to right:

1. `XIC(Run)` *Examine If Closed* asks: true when `Run` (Seal-in run bit) is on.
2. `EQU(Step,2)` *Equal* asks: true when `Step` (Step: 0 wait at the heel, 1 dose A, 2 dose B, 3 heat and mix, 4 drain; 5 rinse fill, 6 rinse mix, 7 rinse drain, 8 tote swap) equals **2**.
3. `OTE(Pump_B)` *Output Energize* does: turns `Pump_B` (Ingredient B pump) on (and off again whenever the rung is false).

RUNG 10




```
XIC(Flow_B)CTU(B_Count,9999,0);
```

HOW THIS RUNG WORKS

In words. When `Flow_B` (Flowmeter B pulses (20 units each)) is on, the rung counts `B_Count` (Flowmeter B pulses this batch) up by one (preset 9999).

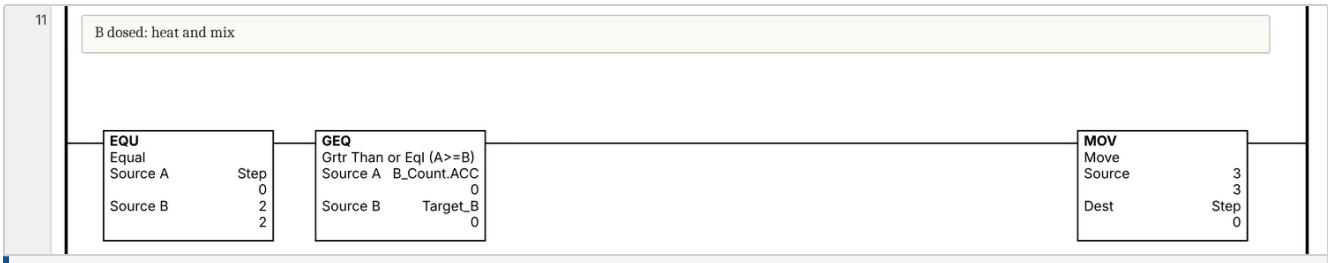
Left to right:

1. `XIC(Flow_B)` *Examine If Closed* asks: true when `Flow_B` (Flowmeter B pulses (20 units each)) is on.
2. `CTU(B_Count,9999,0)` *Count Up* does: counts `B_Count` (Flowmeter B pulses this batch) up by one (preset 9999).

Why it is built this way:

A counter: one count per false-to-true.

RUNG 11




```
EQU(Step,2)GEQ(B_Count.ACC,Target_B)MOV(3,Step);
```

PREDICT IT · 6 `Step` is 2, `B_Count.ACC` (the accumulated value of `B_Count` : Flowmeter B pulses this batch) is 500 and `Target_B` is 500. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Step` (Step: 0 wait at the heel, 1 dose A, 2 dose B, 3 heat and mix, 4 drain; 5 rinse fill, 6 rinse mix, 7 rinse drain, 8 tote swap) equals **2** and `B_Count.ACC` (the accumulated value of `B_Count` : Flowmeter B pulses this batch) is at least `Target_B` (Pulses of B for the selected product), the rung puts **3** into `Step` .

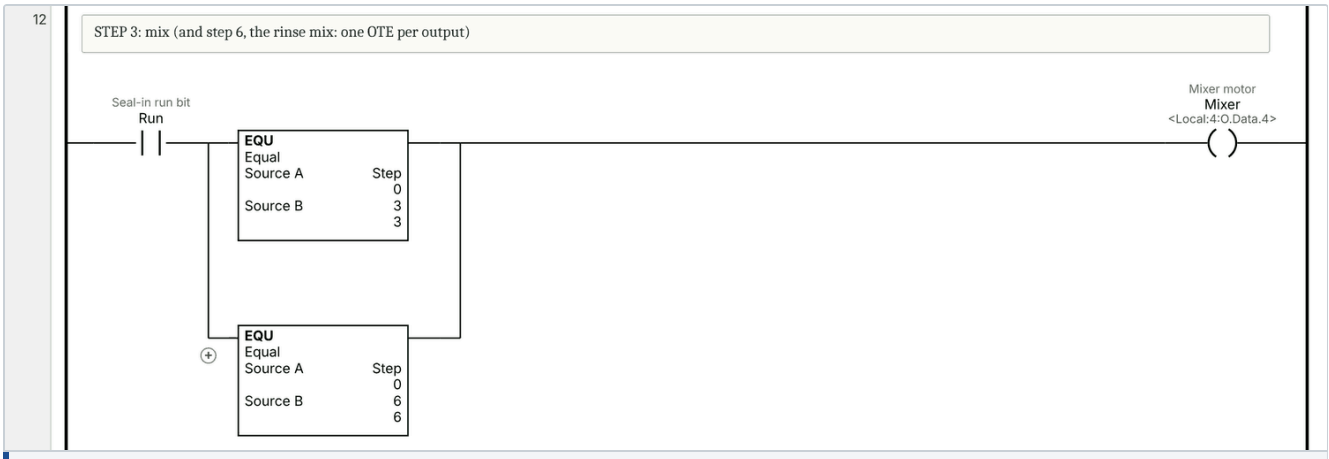
Left to right:

1. `EQU(Step,2)` *Equal* asks: true when `Step` (Step: 0 wait at the heel, 1 dose A, 2 dose B, 3 heat and mix, 4 drain; 5 rinse fill, 6 rinse mix, 7 rinse drain, 8 tote swap) equals **2**.
2. `GEQ(B_Count.ACC,Target_B)` *Greater Than or Equal* asks: true when `B_Count.ACC` (the accumulated value of `B_Count` : Flowmeter B pulses this batch) is at least `Target_B` (Pulses of B for the selected product).
3. `MOV(3,Step)` *Move* does: puts **3** into `Step` .

Why it is built this way:

A step of the sequence: from Step = 2 to 3.

RUNG 12




```
XIC(Run)[EQU(Step,3),EQU(Step,6)]OTE(Mixer);
```

PREDICT IT · 7 `Run` is off and `Step` is 3. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Run` (Seal-in run bit) is on and either (`Step` (Step: 0 wait at the heel, 1 dose A, 2 dose B, 3 heat and mix, 4 drain; 5 rinse fill, 6 rinse mix, 7 rinse drain, 8 tote swap) equals **3**) *or* (`Step` equals **6**), the rung turns `Mixer` (Mixer motor) on (and off again whenever the rung is false).

Left to right:

1. `XIC(Run)` *Examine If Closed* asks: true when `Run` (Seal-in run bit) is on.
2. A branch of 2 parallel paths: power gets through if any one of them is true.

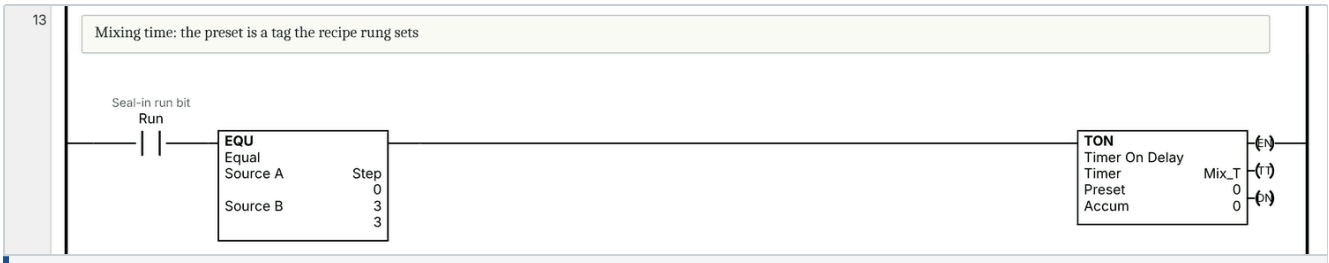
   Path 1

   1. `EQU(Step,3)` *Equal* asks: true when `Step` (Step: 0 wait at the heel, 1 dose A, 2 dose B, 3 heat and mix, 4 drain; 5 rinse fill, 6 rinse mix, 7 rinse drain, 8 tote swap) equals **3**.

   Path 2

   1. `EQU(Step,6)` *Equal* asks: true when `Step` equals **6**.
3. `OTE(Mixer)` *Output Energize* does: turns `Mixer` (Mixer motor) on (and off again whenever the rung is false).

RUNG 13




```
XIC(Run)EQU(Step,3)TON(Mix_T,Mix_Pre,0);
```

PREDICT IT · 8 `Run` is on and `Step` is 3. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Run` (Seal-in run bit) is on and `Step` (Step: 0 wait at the heel, 1 dose A, 2 dose B, 3 heat and mix, 4 drain; 5 rinse fill, 6 rinse mix, 7 rinse drain, 8 tote swap) equals **3**, the rung runs the timer `Mix_T` (Mixing time this batch) toward Mix_Pre.

Left to right:

1. `XIC(Run)` *Examine If Closed* asks: true when `Run` (Seal-in run bit) is on.
2. `EQU(Step,3)` *Equal* asks: true when `Step` (Step: 0 wait at the heel, 1 dose A, 2 dose B, 3 heat and mix, 4 drain; 5 rinse fill, 6 rinse mix, 7 rinse drain, 8 tote swap) equals **3**.
3. `TON(Mix_T,Mix_Pre,0)` *Timer On Delay* does: runs the timer `Mix_T` (Mixing time this batch) toward Mix_Pre. The accumulator counts while the rung is true; .DN comes on at the preset; the moment the rung goes false the timer resets.

Why it is built this way:

A timer. `Mix_T` only times while this rung is true; other rungs use its `.DN` (done) or `.TT` (timing) bits. Being a TON, it starts again from zero every time the rung goes false.

> **Think of it like this:** a kitchen timer that only runs while you hold your finger on it. Take your finger off and it snaps back to zero. Hold it long enough and the done light comes on, and stays on as long as your finger is still there.

> **Watch for:** If anything in series flickers false for one scan, a TON starts over from zero. Use RTO when the time must survive an interruption.

RUNG 14

```
EQU(Step,3)LES(Tank_Temp,Target_Lo)OTL(Heat_On);
```

PREDICT IT · 9 `Step` is 3, `Tank_Temp` is 500 and `Target_Lo` is 500. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Step` (Step: 0 wait at the heel, 1 dose A, 2 dose B, 3 heat and mix, 4 drain; 5 rinse fill, 6 rinse mix, 7 rinse drain, 8 tote swap) equals **3** and `Tank_Temp` (Tank temperature, °F) is less than `Target_Lo` (Heater comes back on below this, °F), the rung latches `Heat_On` (Heater wanted (with hysteresis)) on.

Left to right:

1. `EQU(Step,3)` *Equal* asks: true when `Step` (Step: 0 wait at the heel, 1 dose A, 2 dose B, 3 heat and mix, 4 drain; 5 rinse fill, 6 rinse mix, 7 rinse drain, 8 tote swap) equals **3**.
2. `LES(Tank_Temp,Target_Lo)` *Less Than* asks: true when `Tank_Temp` (Tank temperature, °F) is less than `Target_Lo` (Heater comes back on below this, °F).
3. `OTL(Heat_On)` *Output Latch* does: latches `Heat_On` (Heater wanted (with hysteresis)) on. It only ever sets the bit; the bit stays on after the rung goes false.

Why it is built this way:

A latch. `Heat_On` stays on after this rung goes false; an `OTU` elsewhere in the program clears it.

> **Think of it like this:** a light switch instead of a doorbell. OTL flips it on and it stays on when you take your hand away; OTU flips it off. Whatever flipped it on does not have to stay there.

> **Watch for:** A latched bit survives the condition that set it: every way in needs a way out, or it stays on for good.

RUNG 15

```
[GEQ(Tank_Temp,Target_T),NEQ(Step,3),XIC(Thermostat)]OTU(Heat_On);
```

PREDICT IT · 10 `Tank_Temp` is 500 and `Target_T` is 500. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When either (`Tank_Temp` (Tank temperature, °F) is at least `Target_T` (Temperature for the selected product, °F)) *or* (`Step` (Step: 0 wait at the heel, 1 dose A, 2 dose B, 3 heat and mix, 4 drain; 5 rinse fill, 6 rinse mix, 7 rinse drain, 8 tote swap) is not **3**) *or* (`Thermostat` (High-limit thermostat, closes at 180 °F) is on), the rung unlatches `Heat_On` (Heater wanted (with hysteresis)) (turns it off).

Left to right:

1. A branch of 3 parallel paths: power gets through if any one of them is true.

   Path 1

   1. `GEQ(Tank_Temp,Target_T)` *Greater Than or Equal* asks: true when `Tank_Temp` (Tank temperature, °F) is at least `Target_T` (Temperature for the selected product, °F).

   Path 2

   1. `NEQ(Step,3)` *Not Equal* asks: true when `Step` (Step: 0 wait at the heel, 1 dose A, 2 dose B, 3 heat and mix, 4 drain; 5 rinse fill, 6 rinse mix, 7 rinse drain, 8 tote swap) is not **3**.

   Path 3

   1. `XIC(Thermostat)` *Examine If Closed* asks: true when `Thermostat` (High-limit thermostat, closes at 180 °F) is on.

2. `OTU(Heat_On)` *Output Unlatch* does: unlatches `Heat_On` (Heater wanted (with hysteresis)) (turns it off). It only ever clears the bit.

RUNG 16

```
XIC(Run)XIC(Heat_On)XIC(Lo_Level)XIO(Thermostat)OTE(Heater);
```

PREDICT IT · 11 `Run` is off, `Heat_On` is on, `Lo_Level` is on and `Thermostat` is off. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

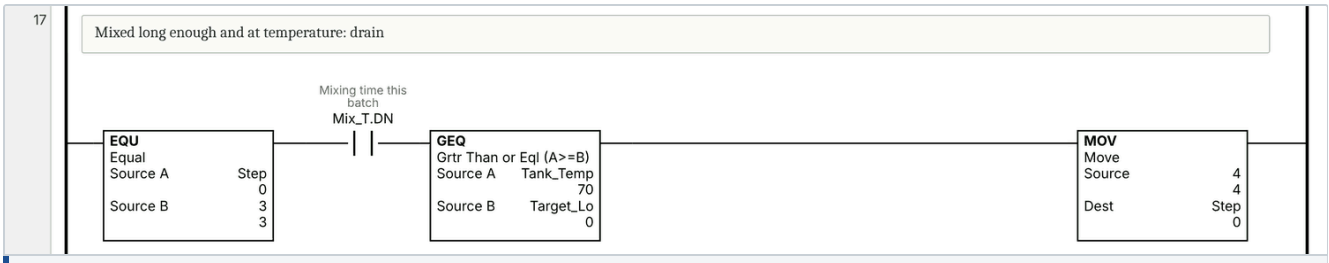
HOW THIS RUNG WORKS

In words. When `Run` (Seal-in run bit) is on, `Heat_On` (Heater wanted (with hysteresis)) is on, `Lo_Level` (LO probe covered (10 %)) is on and `Thermostat` (High-limit thermostat, closes at 180 °F) is off, the rung turns `Heater` (Heater element) on (and off again whenever the rung is false).

Left to right:

1. `XIC(Run)` *Examine If Closed* asks: true when `Run` (Seal-in run bit) is on.
2. `XIC(Heat_On)` *Examine If Closed* asks: true when `Heat_On` (Heater wanted (with hysteresis)) is on.
3. `XIC(Lo_Level)` *Examine If Closed* asks: true when `Lo_Level` (LO probe covered (10 %)) is on.
4. `XIO(Thermostat)` *Examine If Open* asks: true when `Thermostat` (High-limit thermostat, closes at 180 °F) is off.
5. `OTE(Heater)` *Output Energize* does: turns `Heater` (Heater element) on (and off again whenever the rung is false).

RUNG 17




```
EQU(Step,3)XIC(Mix_T.DN)GEQ(Tank_Temp,Target_Lo)MOV(4,Step);
```

PREDICT IT · 12 `Step` is 3, `Mix_T.DN` (the done bit of `Mix_T` : Mixing time this batch) is on, `Tank_Temp` is 500 and `Target_Lo` is 500. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Step` (Step: 0 wait at the heel, 1 dose A, 2 dose B, 3 heat and mix, 4 drain; 5 rinse fill, 6 rinse mix, 7 rinse drain, 8 tote swap) equals **3**, `Mix_T.DN` (the done bit of `Mix_T` : Mixing time this batch) is on and `Tank_Temp` (Tank temperature, °F) is at least `Target_Lo` (Heater comes back on below this, °F), the rung puts **4** into `Step` .

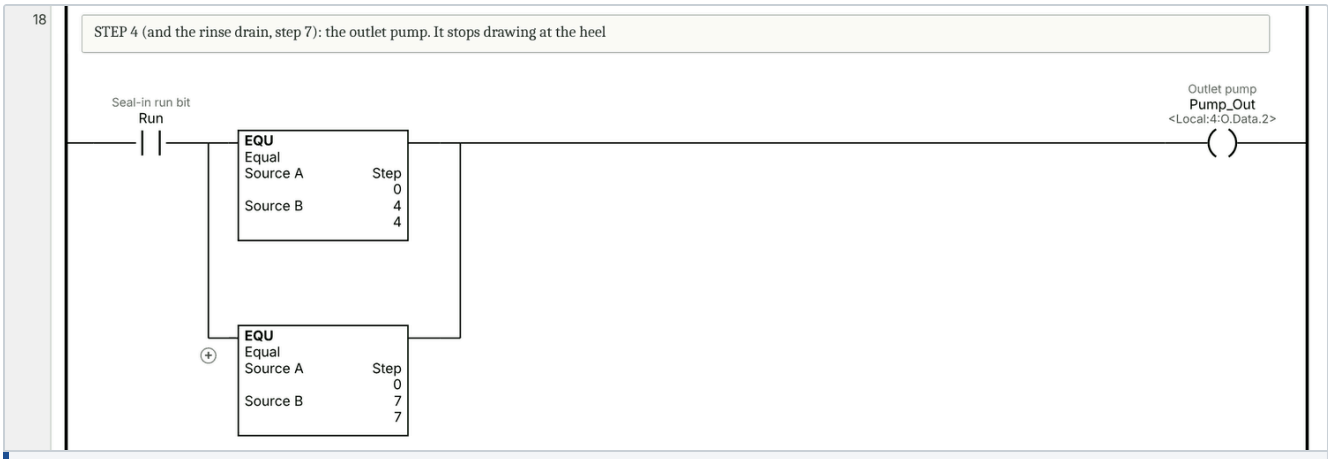
Left to right:

1. `EQU(Step,3)` *Equal* asks: true when `Step` (Step: 0 wait at the heel, 1 dose A, 2 dose B, 3 heat and mix, 4 drain; 5 rinse fill, 6 rinse mix, 7 rinse drain, 8 tote swap) equals **3**.
2. `XIC(Mix_T.DN)` *Examine If Closed* asks: true when `Mix_T.DN` (the done bit of `Mix_T` : Mixing time this batch) is on.
3. `GEQ(Tank_Temp,Target_Lo)` *Greater Than or Equal* asks: true when `Tank_Temp` (Tank temperature, °F) is at least `Target_Lo` (Heater comes back on below this, °F).
4. `MOV(4,Step)` *Move* does: puts **4** into `Step` .

Why it is built this way:

A step of the sequence: from Step = 3 to 4.

RUNG 18




```
XIC(Run)[EQU(Step,4),EQU(Step,7)]OTE(Pump_Out);
```

PREDICT IT · 13 `Run` is off and `Step` is 4. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Run` (Seal-in run bit) is on and either (`Step` (Step: 0 wait at the heel, 1 dose A, 2 dose B, 3 heat and mix, 4 drain; 5 rinse fill, 6 rinse mix, 7 rinse drain, 8 tote swap) equals **4**) *or* (`Step` equals **7**), the rung turns `Pump_Out` (Outlet pump) on (and off again whenever the rung is false).

Left to right:

1. `XIC(Run)` *Examine If Closed* asks: true when `Run` (Seal-in run bit) is on.
2. A branch of 2 parallel paths: power gets through if any one of them is true.

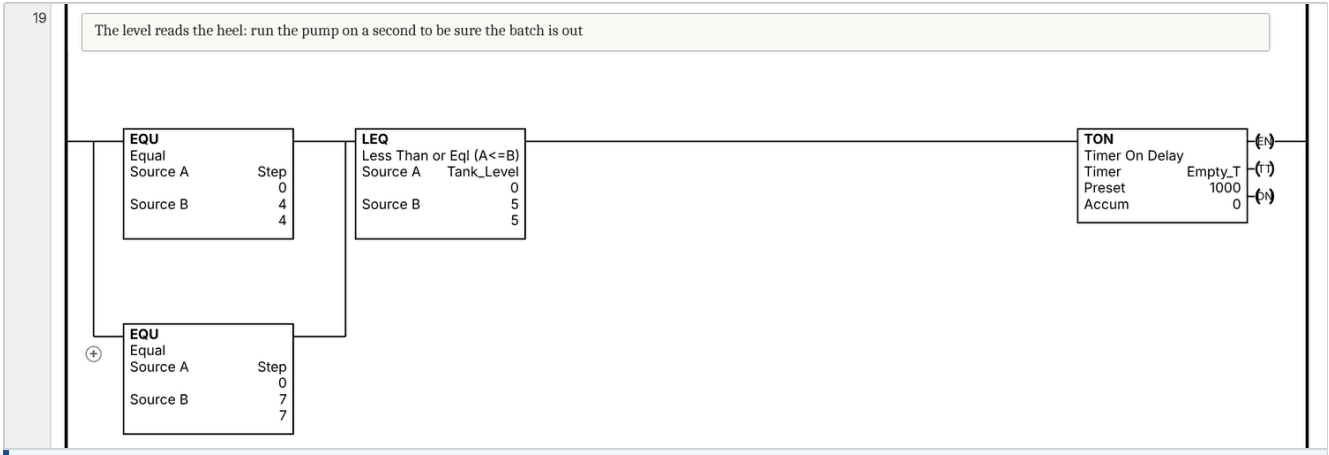
   Path 1

   1. `EQU(Step,4)` *Equal* asks: true when `Step` (Step: 0 wait at the heel, 1 dose A, 2 dose B, 3 heat and mix, 4 drain; 5 rinse fill, 6 rinse mix, 7 rinse drain, 8 tote swap) equals **4**.

   Path 2

   1. `EQU(Step,7)` *Equal* asks: true when `Step` equals **7**.
3. `OTE(Pump_Out)` *Output Energize* does: turns `Pump_Out` (Outlet pump) on (and off again whenever the rung is false).

RUNG 19




```
[EQU(Step,4),EQU(Step,7)]LEQ(Tank_Level,5)TON(Empty_T,1000,0);
```

PREDICT IT · 14 `Step` is 4 and `Tank_Level` is 5. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When either (`Step` (Step: 0 wait at the heel, 1 dose A, 2 dose B, 3 heat and mix, 4 drain; 5 rinse fill, 6 rinse mix, 7 rinse drain, 8 tote swap) equals **4**) *or* (`Step` equals **7**) and `Tank_Level` (Tank level 0-100 %) is at most **5**, the rung runs the timer `Empty_T` (Run the outlet pump on a moment once the level reads the heel) toward 1000 ms (1 s).

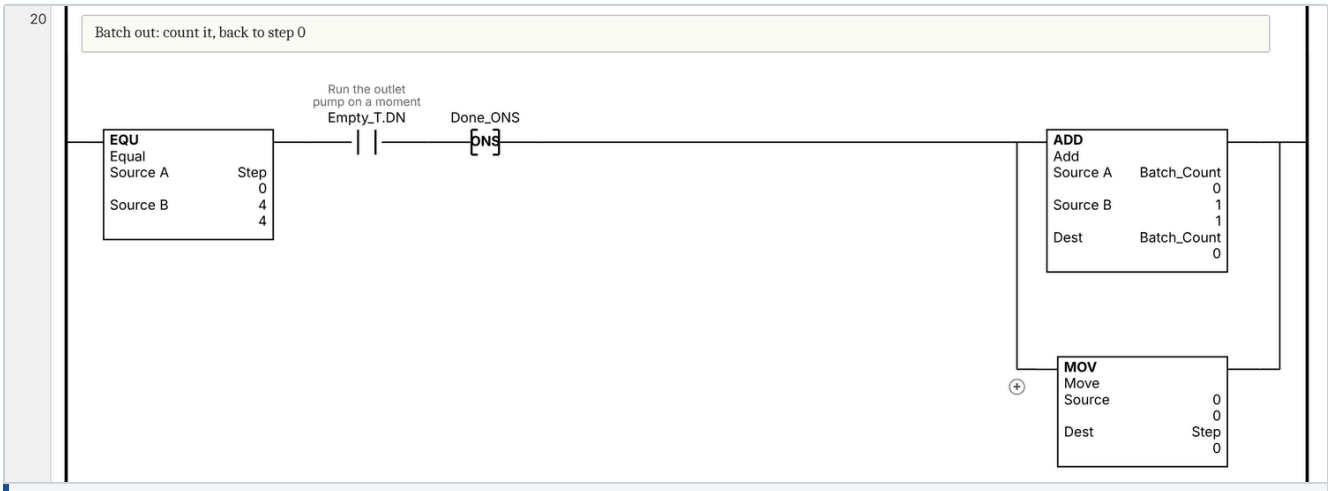
Left to right:

1. A branch of 2 parallel paths: power gets through if any one of them is true.
   - **Path 1**
     1. `EQU(Step,4)` *Equal* asks: true when `Step` (Step: 0 wait at the heel, 1 dose A, 2 dose B, 3 heat and mix, 4 drain; 5 rinse fill, 6 rinse mix, 7 rinse drain, 8 tote swap) equals **4**.
   - **Path 2**
     1. `EQU(Step,7)` *Equal* asks: true when `Step` equals **7**.
2. `LEQ(Tank_Level,5)` *Less Than or Equal* asks: true when `Tank_Level` (Tank level 0-100 %) is at most **5**.
3. `TON(Empty_T,1000,0)` *Timer On Delay* does: runs the timer `Empty_T` (Run the outlet pump on a moment once the level reads the heel) toward 1000 ms (1 s).

Why it is built this way:

A timer: it times only while this rung is true.

RUNG 20




```
EQU(Step,4)XIC(Empty_T.DN)ONS(Done_ONS)
[ADD(Batch_Count,1,Batch_Count),MOV(0,Step)];
```

HOW THIS RUNG WORKS

In words. When `Step` (Step: 0 wait at the heel, 1 dose A, 2 dose B, 3 heat and mix, 4 drain; 5 rinse fill, 6 rinse mix, 7 rinse drain, 8 tote swap) equals **4**, `Empty_T.DN` (the done bit of `Empty_T`: Run the outlet pump on a moment once the level reads the heel) is on and it is the first scan they are true (the one-shot, remembered in `Done_ONS`), the rung makes `Batch_Count` = `Batch_Count` + **1** and puts **0** into `Step`.

Left to right:

1. `EQU(Step,4)` *Equal* asks: true when `Step` (Step: 0 wait at the heel, 1 dose A, 2 dose B, 3 heat and mix, 4 drain; 5 rinse fill, 6 rinse mix, 7 rinse drain, 8 tote swap) equals **4**.
2. `XIC(Empty_T.DN)` *Examine If Closed* asks: true when `Empty_T.DN` (the done bit of `Empty_T`: Run the outlet pump on a moment once the level reads the heel) is on.
3. `ONS(Done_ONS)` *One Shot* asks: true when it is the first scan they are true (the one-shot, remembered in `Done_ONS`). It passes power for one scan only, the scan its input goes from false to true; its storage bit remembers last scan.
4. A branch of 2 parallel paths: each path is tested on its own and does its own work.

   #### Path 1

   1. `ADD(Batch_Count,1,Batch_Count)` *Add* does: makes `Batch_Count` = `Batch_Count` + **1**. Every scan the rung is true; behind a one-shot it adds once per event.

   #### Path 2

   1. `MOV(0,Step)` *Move* does: puts **0** into `Step`.

Why it is built this way:

A step of the sequence: from Step = 4 to 0.

One-shot. Everything after the ONS happens once per event, not on every scan the condition holds; `Done_ONS` is only its memory of the last scan.

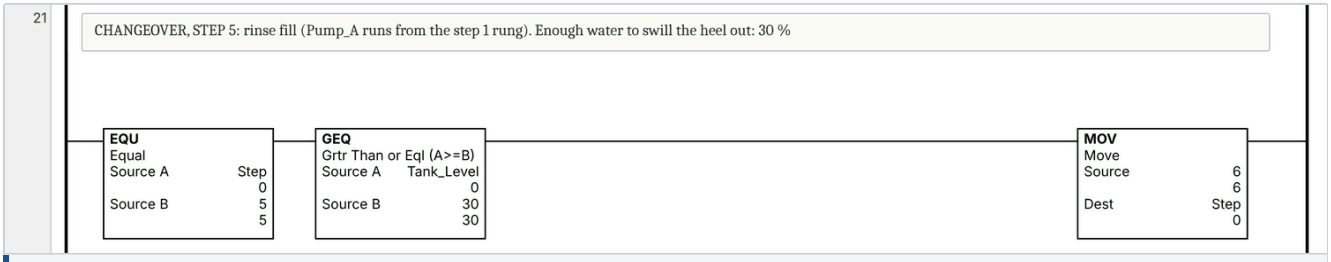
> **Think of it like this:** a doorbell that rings once however long you lean on the button. The ONS looks at what the button was doing last scan: only the change from up to down rings it.

WATCH FOR: TWO COILS ON ONE OUTPUT

The rinse (steps 5 and 6) uses the water pump and the mixer again. Write `XIC(Run)EQU(Step,5)OTE(Pump_A)` as its own rung further down and step 1 stops working: the last OTE scanned decides the output, and in step 1 that last one is false. One output, one rung: rungs 6, 12 and 18 each carry both steps.

The changeover: rinse, swap, new set-up

RUNG 21




```
EQU(Step,5)GEQ(Tank_Level,30)MOV(6,Step);
```

PREDICT IT · 15 `Step` is 5 and `Tank_Level` is 30. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Step` (Step: 0 wait at the heel, 1 dose A, 2 dose B, 3 heat and mix, 4 drain; 5 rinse fill, 6 rinse mix, 7 rinse drain, 8 tote swap) equals **5** and `Tank_Level` (Tank level 0-100 %) is at least **30**, the rung puts **6** into `Step`.

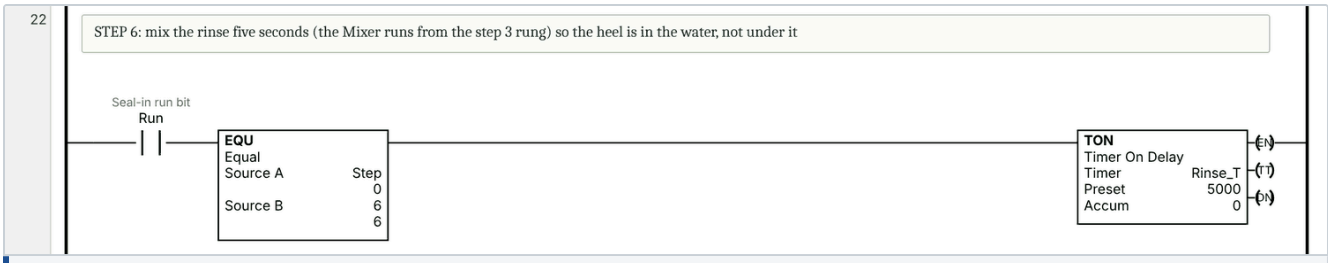
Left to right:

1. `EQU(Step,5)` *Equal* asks: true when `Step` (Step: 0 wait at the heel, 1 dose A, 2 dose B, 3 heat and mix, 4 drain; 5 rinse fill, 6 rinse mix, 7 rinse drain, 8 tote swap) equals **5**.
2. `GEQ(Tank_Level,30)` *Greater Than or Equal* asks: true when `Tank_Level` (Tank level 0-100 %) is at least **30**.
3. `MOV(6,Step)` *Move* does: puts **6** into `Step`.

Why it is built this way:

A step of the sequence: from Step = 5 to 6.

RUNG 22




```
XIC(Run)EQU(Step,6)TON(Rinse_T,5000,0);
```

PREDICT IT · 16 `Run` is on and `Step` is 7. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Run` (Seal-in run bit) is on and `Step` (Step: 0 wait at the heel, 1 dose A, 2 dose B, 3 heat and mix, 4 drain; 5 rinse fill, 6 rinse mix, 7 rinse drain, 8 tote swap) equals **6**, the rung runs the timer `Rinse_T` (Rinse mixing time) toward 5000 ms (5 s).

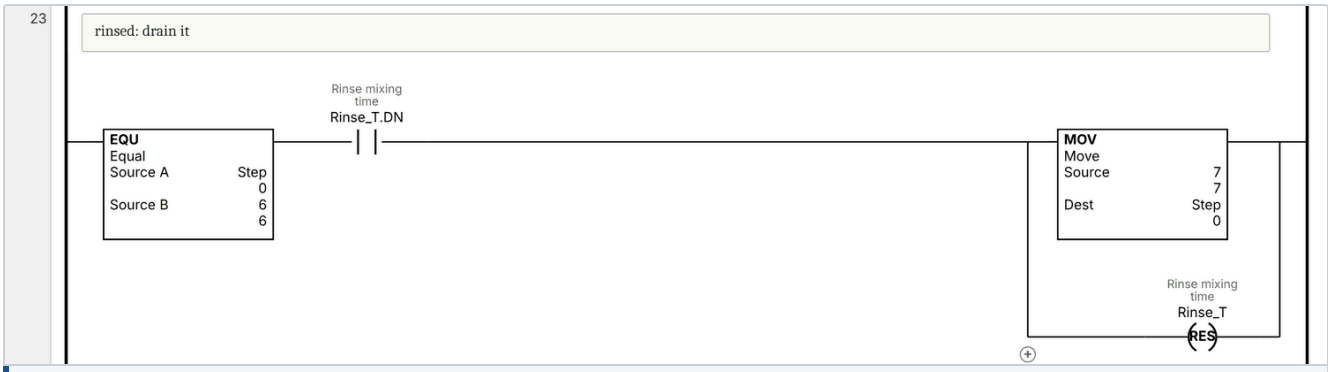
Left to right:

1. `XIC(Run)` *Examine If Closed* asks: true when `Run` (Seal-in run bit) is on.
2. `EQU(Step,6)` *Equal* asks: true when `Step` (Step: 0 wait at the heel, 1 dose A, 2 dose B, 3 heat and mix, 4 drain; 5 rinse fill, 6 rinse mix, 7 rinse drain, 8 tote swap) equals **6**.
3. `TON(Rinse_T,5000,0)` *Timer On Delay* does: runs the timer `Rinse_T` (Rinse mixing time) toward 5000 ms (5 s).

Why it is built this way:

A timer: it times only while this rung is true.

RUNG 23




```
EQU(Step,6)XIC(Rinse_T.DN)[MOV(7,Step),RES(Rinse_T)];
```

PREDICT IT · 17 `Step` is 6 and `Rinse_T.DN` (the done bit of `Rinse_T` : Rinse mixing time) is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Step` (Step: 0 wait at the heel, 1 dose A, 2 dose B, 3 heat and mix, 4 drain; 5 rinse fill, 6 rinse mix, 7 rinse drain, 8 tote swap) equals **6** and `Rinse_T.DN` (the done bit of `Rinse_T` : Rinse mixing time) is on, the rung puts **7** into `Step` and resets `Rinse_T` .

Left to right:

1. `EQU(Step,6)` *Equal* asks: true when `Step` (Step: 0 wait at the heel, 1 dose A, 2 dose B, 3 heat and mix, 4 drain; 5 rinse fill, 6 rinse mix, 7 rinse drain, 8 tote swap) equals **6**.
2. `XIC(Rinse_T.DN)` *Examine If Closed* asks: true when `Rinse_T.DN` (the done bit of `Rinse_T` : Rinse mixing time) is on.
3. A branch of 2 parallel paths: each path is tested on its own and does its own work.

   Path 1

   1. `MOV(7,Step)` *Move* does: puts **7** into `Step` .

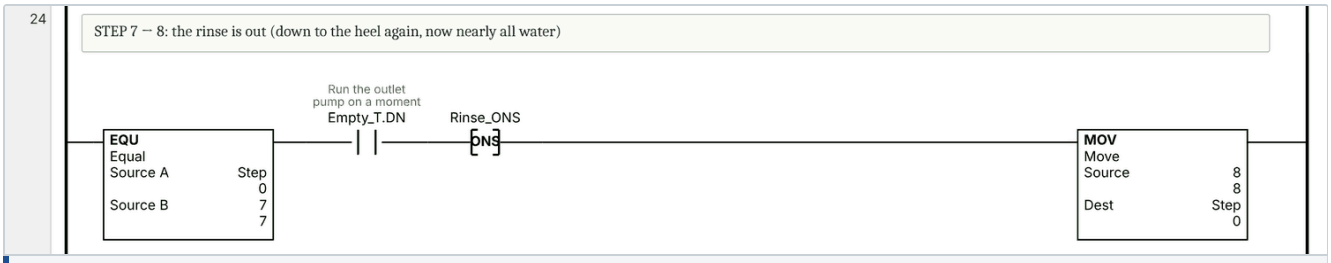
   Path 2

   1. `RES(Rinse_T)` *Reset* does: resets `Rinse_T` .

Why it is built this way:

A step of the sequence: from Step = 6 to 7.

RUNG 24




```
EQU(Step,7)XIC(Empty_T.DN)ONS(Rinse_ONS)MOV(8,Step);
```

HOW THIS RUNG WORKS

In words. When `Step` (Step: 0 wait at the heel, 1 dose A, 2 dose B, 3 heat and mix, 4 drain; 5 rinse fill, 6 rinse mix, 7 rinse drain, 8 tote swap) equals **7**, `Empty_T.DN` (the done bit of `Empty_T` : Run the outlet pump on a moment once the level reads the heel) is on and it is the first scan they are true (the one-shot, remembered in `Rinse_ONS`), the rung puts **8** into `Step` .

Left to right:

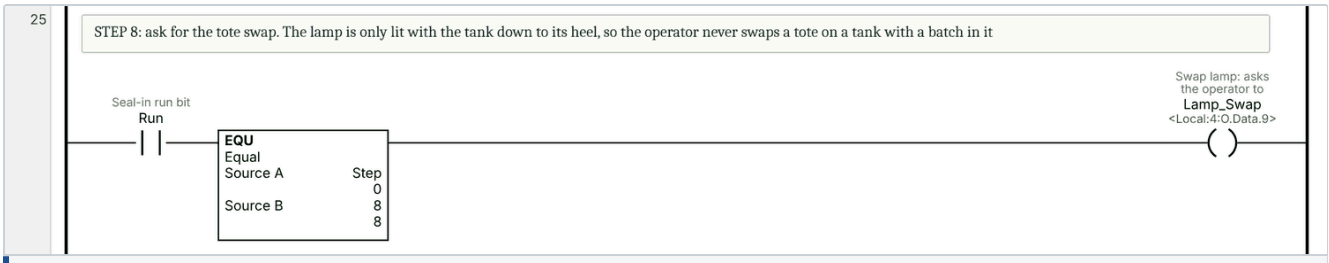
1. `EQU(Step,7)` *Equal* asks: true when `Step` (Step: 0 wait at the heel, 1 dose A, 2 dose B, 3 heat and mix, 4 drain; 5 rinse fill, 6 rinse mix, 7 rinse drain, 8 tote swap) equals **7**.
2. `XIC(Empty_T.DN)` *Examine If Closed* asks: true when `Empty_T.DN` (the done bit of `Empty_T` : Run the outlet pump on a moment once the level reads the heel) is on.
3. `ONS(Rinse_ONS)` *One Shot* asks: true when it is the first scan they are true (the one-shot, remembered in `Rinse_ONS`).
4. `MOV(8,Step)` *Move* does: puts **8** into `Step` .

Why it is built this way:

A step of the sequence: from Step = 7 to 8.

One-shot: once per event, not once per scan.

RUNG 25




```
XIC(Run)EQU(Step,8)OTE(Lamp_Swap);
```

PREDICT IT · 18 `Run` is on and `Step` is 8. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Run` (Seal-in run bit) is on and `Step` (Step: 0 wait at the heel, 1 dose A, 2 dose B, 3 heat and mix, 4 drain; 5 rinse fill, 6 rinse mix, 7 rinse drain, 8 tote swap) equals **8**, the rung turns `Lamp_Swap` (Swap lamp: asks the operator to swap the syrup tote (with the tank down to its heel)) on (and off again whenever the rung is false).

Left to right:

1. `XIC(Run)` *Examine If Closed* asks: true when `Run` (Seal-in run bit) is on.
2. `EQU(Step,8)` *Equal* asks: true when `Step` (Step: 0 wait at the heel, 1 dose A, 2 dose B, 3 heat and mix, 4 drain; 5 rinse fill, 6 rinse mix, 7 rinse drain, 8 tote swap) equals **8**.
3. `OTE(Lamp_Swap)` *Output Energize* does: turns `Lamp_Swap` (Swap lamp: asks the operator to swap the syrup tote (with the tank down to its heel)) on (and off again whenever the rung is false).

RUNG 26

```
EQU(Step,8)XIC(Tote_OK)
[XIC(Sel_Juice)OTL(Cur_Juice),XIO(Sel_Juice)OTU(Cur_Juice),MOV(0,Step)];
```

HOW THIS RUNG WORKS

In words. When `Step` (Step: 0 wait at the heel, 1 dose A, 2 dose B, 3 heat and mix, 4 drain; 5 rinse fill, 6 rinse mix, 7 rinse drain, 8 tote swap) equals **8** and `Tote_OK` (The operator has swapped the tote and pressed TOTE OK) is on, the rung picks its path: if `Sel_Juice` (Product selector at JUICE (right); 0 = SODA) is on, it latches `Cur_Juice` (The tank and the tote are set up for Juice (0: Soda)) on; if `Sel_Juice` is off, it unlatches `Cur_Juice` (turns it off); in any case it puts **0** into `Step`.

Left to right:

1. `EQU(Step,8)` *Equal* asks: true when `Step` (Step: 0 wait at the heel, 1 dose A, 2 dose B, 3 heat and mix, 4 drain; 5 rinse fill, 6 rinse mix, 7 rinse drain, 8 tote swap) equals **8**.
2. `XIC(Tote_OK)` *Examine If Closed* asks: true when `Tote_OK` (The operator has swapped the tote and pressed TOTE OK) is on.
3. A branch of 3 parallel paths: each path is tested on its own and does its own work.

Path 1

1. `XIC(Sel_Juice)` *Examine If Closed* asks: true when `Sel_Juice` (Product selector at JUICE (right); 0 = SODA) is on.
2. `OTL(Cur_Juice)` *Output Latch* does: latches `Cur_Juice` (The tank and the tote are set up for Juice (0: Soda)) on.

Path 2

1. `XIO(Sel_Juice)` *Examine If Open* asks: true when `Sel_Juice` is off.
2. `OTU(Cur_Juice)` *Output Unlatch* does: unlatches `Cur_Juice` (turns it off).

Path 3

1. `MOV(0,Step)` *Move* does: puts **0** into `Step`.

Why it is built this way:

A step of the sequence: from Step = 8 to 0.

A latch: it stays on until an OTU clears it.

TRY IT: NO RINSE

In rung 5 change the two `MOV(5,Step)` to `MOV(8,Step)`: straight from the heel to the tote swap. The operator swaps the tote, the first Juice batch is made on a cola heel, and the log says *cross-contaminated: 1.0 % Cola syrup in the Juice*, −200. The second Juice is clean: by then the heel is Juice.

TRY IT: THE LAMP WITH A BATCH IN THE TANK

Change rung 25 to light Lamp_Swap whenever Sel_Juice is on and Cur_Juice is not. It lights while the second Soda batch is still mixing; the log says the operator will not swap a tote on a tank with a batch in it, and nothing happens until the tank is at its heel. The interlock is in the operator, not the program; a real one should be in both.

The lamps

RUNG 27

```
XIC(Run)OTE(Lamp_Run);
```

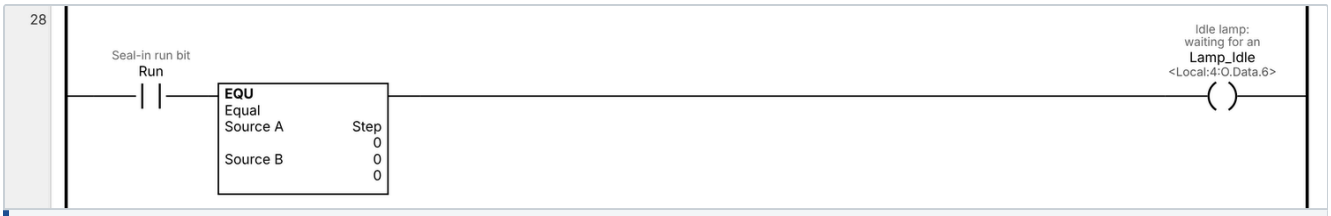
HOW THIS RUNG WORKS

In words. When `Run` (Seal-in run bit) is on, the rung turns `Lamp_Run` (Run lamp) on (and off again whenever the rung is false).

Left to right:

1. `XIC(Run)` *Examine If Closed* asks: true when `Run` (Seal-in run bit) is on.
2. `OTE(Lamp_Run)` *Output Energize* does: turns `Lamp_Run` (Run lamp) on (and off again whenever the rung is false).

RUNG 28




```
XIC(Run)EQU(Step,0)OTE(Lamp_Idle);
```

PREDICT IT · 19 `Run` is off and `Step` is 0. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

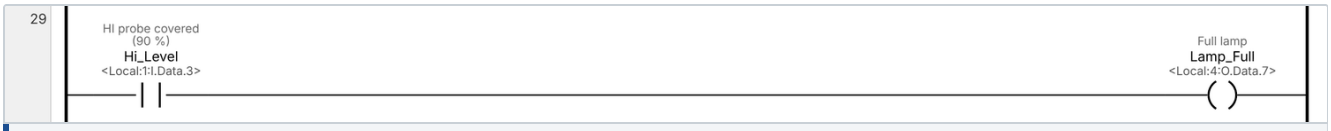
HOW THIS RUNG WORKS

In words. When `Run` (Seal-in run bit) is on and `Step` (Step: 0 wait at the heel, 1 dose A, 2 dose B, 3 heat and mix, 4 drain; 5 rinse fill, 6 rinse mix, 7 rinse drain, 8 tote swap) equals **0**, the rung turns `Lamp_Idle` (Idle lamp: waiting for an empty tank or a Start) on (and off again whenever the rung is false).

Left to right:

1. `XIC(Run)` *Examine If Closed* asks: true when `Run` (Seal-in run bit) is on.
2. `EQU(Step,0)` *Equal* asks: true when `Step` (Step: 0 wait at the heel, 1 dose A, 2 dose B, 3 heat and mix, 4 drain; 5 rinse fill, 6 rinse mix, 7 rinse drain, 8 tote swap) equals **0**.
3. `OTE(Lamp_Idle)` *Output Energize* does: turns `Lamp_Idle` (Idle lamp: waiting for an empty tank or a Start) on (and off again whenever the rung is false).

RUNG 29




```
XIC(Hi_Level)OTE(Lamp_Full);
```

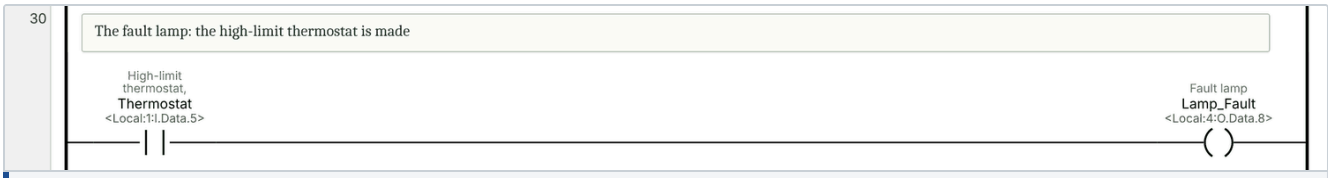
HOW THIS RUNG WORKS

In words. When `Hi_Level` (HI probe covered (90 %)) is on, the rung turns `Lamp_Full` (Full lamp) on (and off again whenever the rung is false).

Left to right:

1. `XIC(Hi_Level)` *Examine If Closed* asks: true when `Hi_Level` (HI probe covered (90 %)) is on.
2. `OTE(Lamp_Full)` *Output Energize* does: turns `Lamp_Full` (Full lamp) on (and off again whenever the rung is false).

RUNG 30




```
XIC(Thermostat)OTE(Lamp_Fault);
```

HOW THIS RUNG WORKS

In words. When `Thermostat` (High-limit thermostat, closes at 180 °F) is on, the rung turns `Lamp_Fault` (Fault lamp) on (and off again whenever the rung is false).

Left to right:

1. `XIC(Thermostat)` *Examine If Closed* asks: true when `Thermostat` (High-limit thermostat, closes at 180 °F) is on.
2. `OTE(Lamp_Fault)` *Output Energize* does: turns `Lamp_Fault` (Fault lamp) on (and off again whenever the rung is false).

Run it

Load the program, Run mode, Start. Two Soda batches, each about 37 s. The selector turns to JUICE as the second drains; the tank fills to 30 % with water, mixes five seconds, drains, the lamp lights, the orange tote comes in and TOTE OK is pressed; two Juice batches at 160 °F, each a minute, because the heater has twice as far to go. 4 of 4 bottles in the right bin, nothing lost, in 218 s against a par of 270 s: 1400 points and three stars.

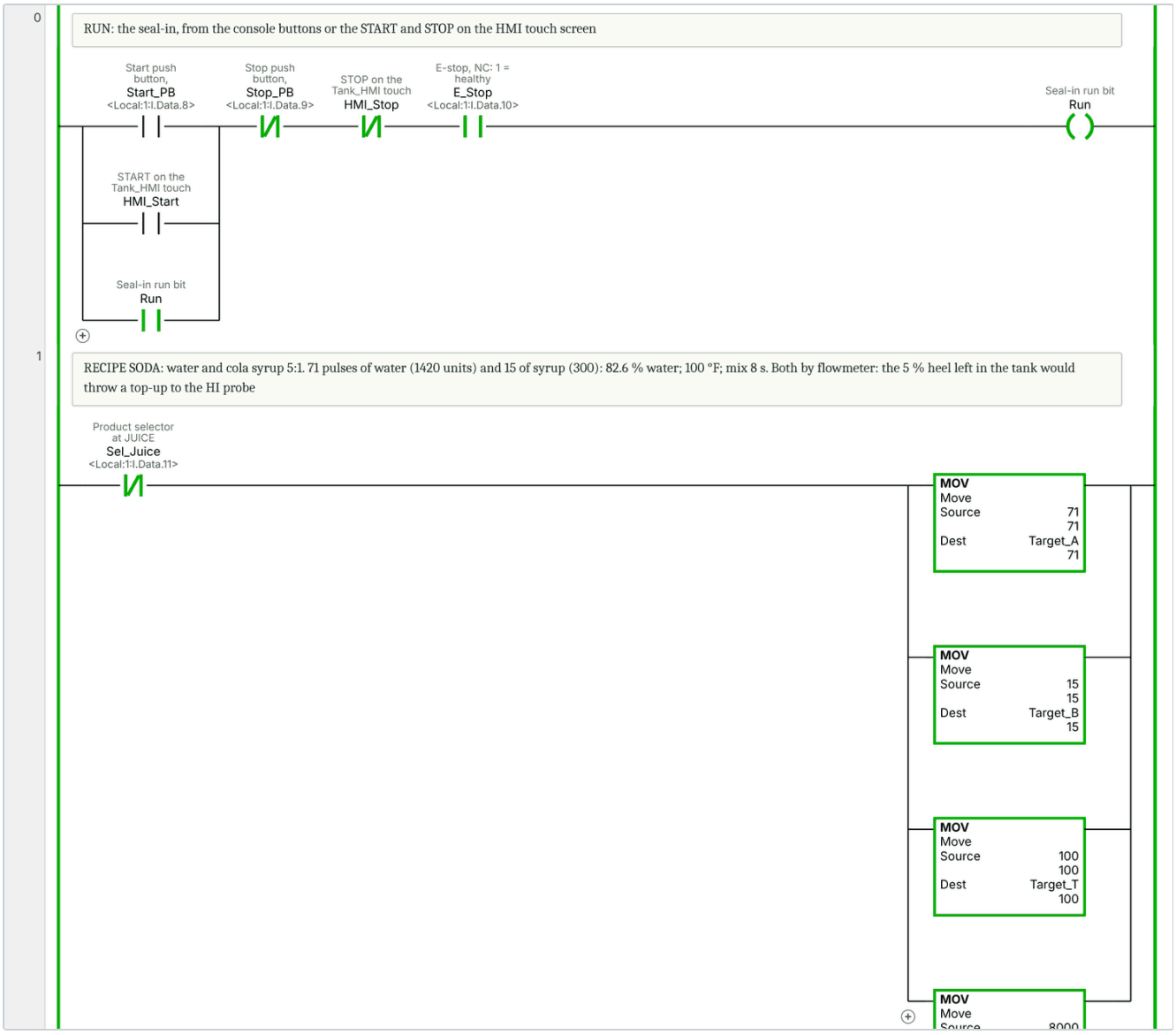


Figure 190. Level 12 online, the top of the routine.

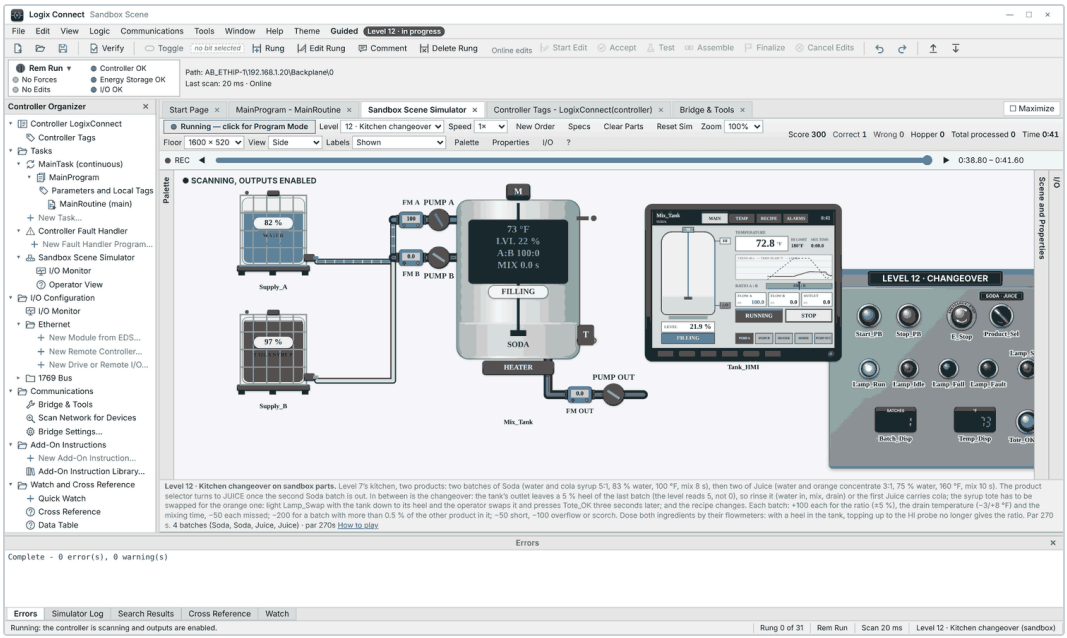


Figure 191. The machine halfway through the run.

Predict it: the answers

1. The rung is false at `XIO(Stop_PB)`: `Stop_PB` is on, so `Run` is **off** (an OTE writes 0 when its rung is false).
2. Every condition on the way is met, so `Step` becomes **1**.
3. The rung is false at `XIC(Run)`: `Run` is off, so `Pump_A` is **off** (an OTE writes 0 when its rung is false).
4. The rung is false at the EQU: `Step` is not **1**, so the MOV does nothing: `Step` keeps its value.
5. Every condition on the way is met, so `Pump_B` is **on**.
6. Every condition on the way is met, so `Step` becomes **3**.
7. The rung is false at `XIC(Run)`: `Run` is off, so `Mixer` is **off** (an OTE writes 0 when its rung is false).
8. Every condition on the way is met, so `Mix_T` times toward its preset.
9. The rung is false at the LES: `Tank_Temp` is not below `Target_Lo`, so the OTL does nothing: `Heat_On` keeps whatever it had.
10. Every condition on the way is met, so `Heat_On` is unlatched, **off**.
11. The rung is false at `XIC(Run)`: `Run` is off, so `Heater` is **off** (an OTE writes 0 when its rung is false).
12. Every condition on the way is met, so `Step` becomes **4**.
13. The rung is false at `XIC(Run)`: `Run` is off, so `Pump_Out` is **off** (an OTE writes 0 when its rung is false).
14. Every condition on the way is met, so `Empty_T` times toward its preset.
15. Every condition on the way is met, so `Step` becomes **6**.
16. The rung is false at the EQU: `Step` is not **6**, so `Rinse_T` is reset to zero (a TON resets when its rung goes false).
17. Every condition on the way is met, so `Step` becomes **7** and `Rinse_T` is reset.
18. Every condition on the way is met, so `Lamp_Swap` is **on**.

19. The rung is false at `XIC(Run)`: `Run` is off, so `Lamp_Idle` is **off** (an OTE writes 0 when its rung is false).

Part VI — Beyond the bottle line

45 Save, share and replay

Three things outlive a run: the project file, a link that carries the project to someone else, and the recording of the last two minutes of the machine. This chapter is about all three, and about using the recording to debug. Being able to wind a machine back and watch the scan where it went wrong is something no real line gives you.

The project file

File → Save (Ctrl+S) writes one `.json` file holding everything: the scene and every device's settings, the tags, the routines, the I/O configuration and, if you tick it, the network settings. **File → Open** reads it back. The page also keeps your work by itself between visits, so the file is for versions and for moving a project to another PC.

A share link

File → Share Link… packs the same project into the address itself. The JSON is deflated and written after `#lc=`, so nothing is uploaded anywhere: whoever has the link has the project. The bottle line with its thirteen routines makes a link of about 20 KB, a third of the file.

- **Opens at** is the address of the web copy of Logix Connect, your GitHub Pages site. On the site it fills itself in. In the desktop app, type it once and it is remembered. The desktop app runs from your own PC, so a link pointing at it would open nowhere else.
- **Include network settings** adds module IP addresses, connections and the bridge address. Leave it off when you send a project to someone on another network.
- **Include the score card** is offered once a level is finished. The link then carries the level, the score, the correct and wrong counts, the time and whether it beat par, and it shows when the link is opened.

Opening a link asks first, with a summary of what is in it, because it replaces the project you have open. The project you have open is put aside first (a level keeps its program, the sandbox project comes back as My sandbox project), and your other levels are kept. In the desktop app, **File → Open Link…** takes a pasted link, or the whole message it came in.

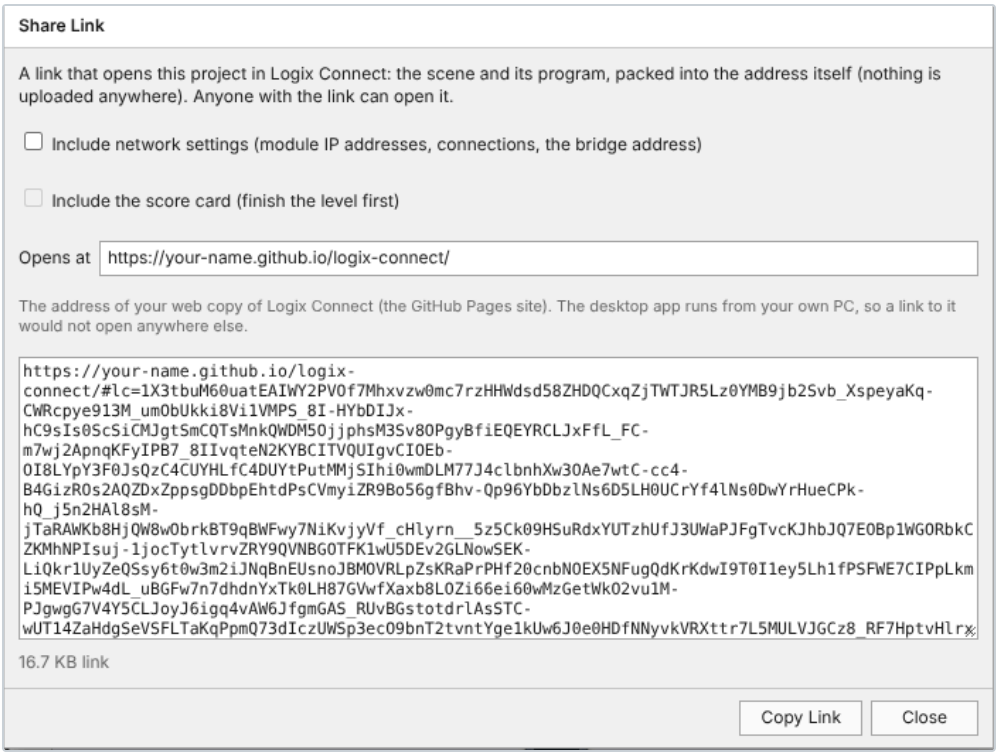


Figure 192. File → Share Link… on the bottle line: the web address it opens at, the two options and the link itself, 16.7 KB.

> **WATCH FOR**
>
> Some mail and chat apps cut very long links short. A link cut short is refused with a clear message rather than opening half a project. For a big scene (over about 60 KB of link, which the dialog warns about) send the file instead.

The scene recorder

While a sandbox scene runs, the recorder keeps a frame every 0.25 s of machine time for the last two minutes (one minute on a phone). A frame holds every part on the floor, every device's state (a timer half run, a cylinder half out, a tank's level and temperature, a case on the packer's lane), the simulator's counters and log, the machine faults you injected, the random generator that picks the part mix, and every tag in the controller.

The bar under the simulator toolbar shows the span recorded, **REC 0:00.05 – 0:40.38**. Drag the slider back, or press ◀, and the scene goes into **REPLAY**:

- All three views, the I/O panel, Quick Watch and the data table show that moment, read-only. Nothing steps.
- The ladder shows the power flow of that moment. The page runs one scan of no time over the recorded tags to light the rungs, then puts the tags back exactly as they were recorded, so looking changes

nothing.

- ▶ steps forward a frame. **Live** puts the run back exactly as you left it. The Run Mode button does the same.
- **Run from here** keeps that moment and runs on from it, with the program as it is *now*. The frames after it are dropped, and the log notes the rewind.

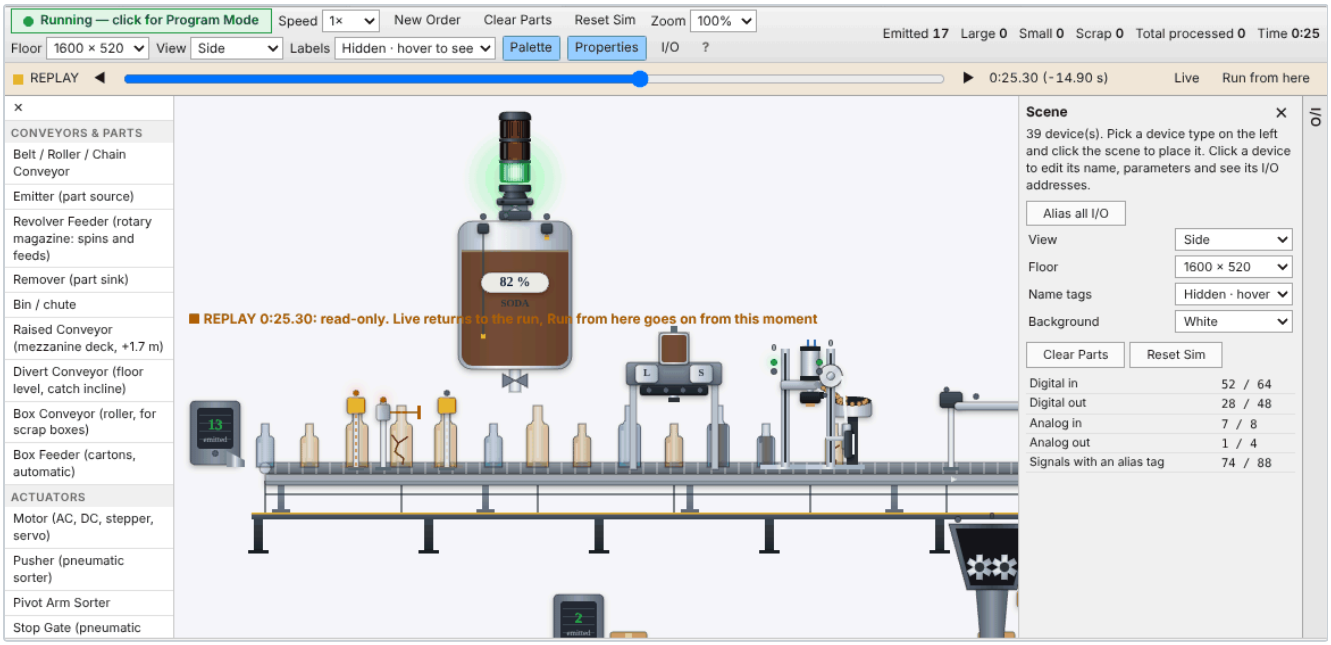


Figure 193. Replay on the bottle line, 14.9 s back: the bar is amber, the scene says REPLAY and is frozen at 0:25.30, and Live and Run from here are offered.

Because the random generator is rewound with everything else, running on from a frame with the same program gives the same run, bottle for bottle. Change a rung and run on from the same frame, and the only difference between the two runs is your change. That is the most useful thing the recorder does.

TRY IT — CATCH A JAM AND READ THE SCAN

Run the bottle line. After a few bottles, use Tools → Inject Machine Fault… → *Jam*. The jammed bottle stops, LS11 does not see it where the registers say it should be, Alarm_Word bit 2 sets and the line stops. Switch to Program mode, open `S11_Alarms` and drag the slider back a second at a time to the frame where `EOL_Missing` first goes true. The rung above it shows why: `LS11` dark while the Exist and Large bits said a bottle was due. Now drag back to before you injected the jam and press **Run from here**. The jam is gone, because the fault was injected after that moment and the recorder rewinds faults with the rest. The line runs as if it had never happened.

TRY IT — PROVE A CHANGE ON THE SAME RUN

On Level 6, run until the first case creeps onto the mark under the packer head, then scrub back to the frame where `Case_Slow` makes. Note where the case stops: on the mark. Change the creep rung from `MOV(15,Case_Hz)` to `MOV(30,Case_Hz)`, accept the edit and press **Run from here**. The same case, on the same frame, now runs past the head before `Case_In_Place` can stop it, because at 30 Hz the drive's 0.5 s decel ramp carries it beyond the ±6 px window. With no case under the head nothing is packed, and the case goes out empty for −25. Put 15 back, scrub to the same frame, run from here again, and it lands. Two runs that differ only in one number.

What is and is not rewound

Rewound: the parts, the devices, the controller's tags (timers, counters, ONS bits, arrays, UDTs), its periodic task schedule, the machine faults and the random part mix. Not rewound: your program and the scene layout (they are what you are testing), the trend charts, and anything outside the page: a real device on the bridge, or an EtherNet/IP module. Reset Sim, New Order or opening another project starts a new recording.

46 A servo axis on a Festo FHPP drive

Everything so far moved on a contactor or a VFD: a belt runs or it does not, faster or slower. A **servo axis** is the other kind of motion. The PLC names a position and the drive puts the carriage there, to a hundredth of a millimetre, and tells you when it has arrived. From Version 120 the palette has a **Drives & Axes** category (the VFD of the case packer chapter moved there too) with a *Linear Axis* part. Three things make a servo axis, and it pays to keep them apart: the **axis** is the mechanics, a toothed-belt or spindle profile with a carriage; the **servo motor** is bolted to its end; the **drive** is the servo controller, a box in the cabinet. The part on the floor is the axis with its motor. The drive is the module in I/O Configuration, and the PLC talks to it over EtherNet/IP with Festo's **FHPP** profile, the Festo Handling and Positioning Profile. This chapter builds the small panel that drives it and walks the handshake rung by rung. Level 10 (chapter 42) puts the axis to work.

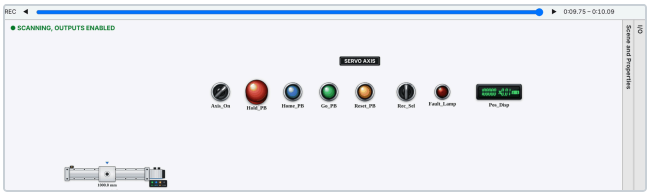


Figure 195. The servo-axis sandbox: the axis, drawn to scale (60 px per metre, like the belts), and its panel. The carriage sits at record 2, 1000 mm.

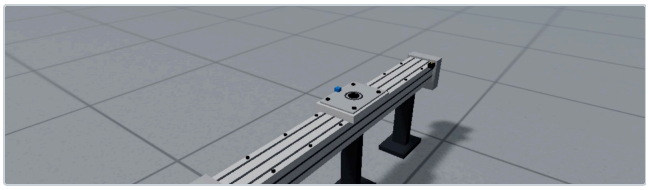


Figure 196. The same axis in 3D: an anodised profile with its T-slots, the carriage plate, the motor at the right-hand end and the four status LEDs.

What FHPP is

A Festo drive on EtherNet/IP exchanges eight bytes each way on the implicit connection, and FHPP says what the bytes mean. The PLC writes two **control bytes**, CCON and CPOS, a record number, a velocity and a position; the drive answers with two **status bytes**, SCON and SPOS, the record it ran, its actual velocity and its actual position in hundredths of a millimetre. Logix Connect gives every bit a name under the module, so the rungs read `Axis1:O.Enable` and `Axis1:I.Motion_Complete` rather than `Axis1:O.Data[0].0`; the bit map underneath is the profile's.

Output (PLC → drive)	Type	Where	Meaning
`Enable`	BOOL	byte 1 bit 0	CCON.B0 ENABLE: enable the drive
`Stop`	BOOL	byte 1 bit 1	CCON.B1 STOP: 1 = operation enabled, 0 = stop (a controlled stop, the task is dropped)
`Brake`	BOOL	byte 1 bit 2	CCON.B2 BRAKE: release the holding brake

Output (PLC → drive)	Type	Where	Meaning
`Reset`	BOOL	byte 1 bit 3	CCON.B3 RESET: clear a fault on the rising edge
`Lock`	BOOL	byte 1 bit 5	CCON.B5 LOCK: block FCT/MMI access
`OPM1`	BOOL	byte 1 bit 6	CCON.B6: operating mode bit 1 (0 Record Select, 1 Direct mode)
`OPM2`	BOOL	byte 1 bit 7	CCON.B7: operating mode bit 2 (0)
`Halt`	BOOL	byte 2 bit 0	CPOS.B0 HALT: 1 = run, 0 = halt (stop and keep the task; Start resumes it)
`Start`	BOOL	byte 2 bit 1	CPOS.B1 START: start the task on the rising edge
`Home`	BOOL	byte 2 bit 2	CPOS.B2 HOM: start homing on the rising edge
`Jog_Pos`	BOOL	byte 2 bit 3	CPOS.B3 JOGP: jog positive while 1
`Jog_Neg`	BOOL	byte 2 bit 4	CPOS.B4 JOGN: jog negative while 1
`Teach`	BOOL	byte 2 bit 5	CPOS.B5 TEACH: store the actual position in the record (rising edge)
`Clear`	BOOL	byte 2 bit 6	CPOS.B6 CLEAR: drop the task held by a halt (rising edge)
`Record_No`	SINT	byte 3	Byte 3: the record to run, 1 … n (Record Select) / CDIR (Direct mode: bit 0 = relative)
`Setpoint_Vel`	SINT	byte 4	Byte 4 (Direct mode): velocity, % of the maximum
`Setpoint_Pos`	DINT	bytes 5-8	Bytes 5-8 (Direct mode): target position, 0.01 mm

Input (drive → PLC)	Type	Where	Meaning
`Enabled`	BOOL	byte 1 bit 0	SCON.B0 ENABLED: the drive is enabled
`Op_Enabled`	BOOL	byte 1 bit 1	SCON.B1 OPEN: operation enabled, motion allowed
`Warning`	BOOL	byte 1 bit 2	SCON.B2 WARN: a warning is pending (a start that was ignored)
`Fault`	BOOL	byte 1 bit 3	SCON.B3 FAULT: faulted, no motion until Reset
`Load_Voltage`	BOOL	byte 1 bit 4	SCON.B4 VLOAD: load voltage present
`FCT_Control`	BOOL	byte 1 bit 5	SCON.B5 FCT/MMI has device control

Input (drive → PLC)	Type	Where	Meaning
`OPM1_Ack`	BOOL	byte 1 bit 6	SCON.B6: operating mode bit 1 as the drive runs it
`OPM2_Ack`	BOOL	byte 1 bit 7	SCON.B7: operating mode bit 2
`Halt_Ack`	BOOL	byte 2 bit 0	SPOS.B0 HALT: 1 = halt not active
`Start_Ack`	BOOL	byte 2 bit 1	SPOS.B1 ACK: the start was taken (follows Start)
`Motion_Complete`	BOOL	byte 2 bit 2	SPOS.B2 MC: the positioning task is complete (1 at rest)
`Teach_Ack`	BOOL	byte 2 bit 3	SPOS.B3 TEACH: the position was stored
`Moving`	BOOL	byte 2 bit 4	SPOS.B4 MOV: the axis is moving
`Drag_Error`	BOOL	byte 2 bit 5	SPOS.B5 DEV: following error
`Standstill`	BOOL	byte 2 bit 6	SPOS.B6 STILL: at rest with operation enabled
`Referenced`	BOOL	byte 2 bit 7	SPOS.B7 REF: the axis is homed
`Record_No`	SINT	byte 3	Byte 3: the record that ran (Record Select) / SDIR (Direct mode)
`Actual_Vel`	SINT	byte 4	Byte 4: actual velocity, % of the maximum
`Actual_Pos`	DINT	bytes 5-8	Bytes 5-8: actual position, 0.01 mm

> **CHECK IT AGAINST THE MANUAL**
>
> The bit map was written from the profile as I know it. A CMMT or CMMP drive manual has the FHPP control and status bytes on one page: compare them before you run a real axis, and change the module's member names in *I/O Configuration* if yours differ. The simulated drive follows the names, not the bit numbers, so the sandbox runs either way.

The part and its module

Place a Linear Axis and press **Add Axis Module** in its properties: a module `Axis1` appears in I/O Configuration with the FHPP members, and the axis is tied to it. The part has no wired signals at all; it moves on the module's tags, which is exactly how the real one works. Its properties are the axis you ordered: **Axis size** (Festo builds them in profile sizes, 45 to 185 mm wide, toothed belt or ball-screw spindle; the size sets the drawn width, the stroke range and the drive's velocity and acceleration ceiling, and a setting past the ceiling is held at it), **Stroke** in millimetres, the **maximum velocity** that 100 % in a record means, the **acceleration**, the homing and jog speeds, and the **records**: a position and a velocity percentage each, `200@50, 1000@80, 1800@100` here. On a live module the part follows the real drive's Actual_Pos instead.

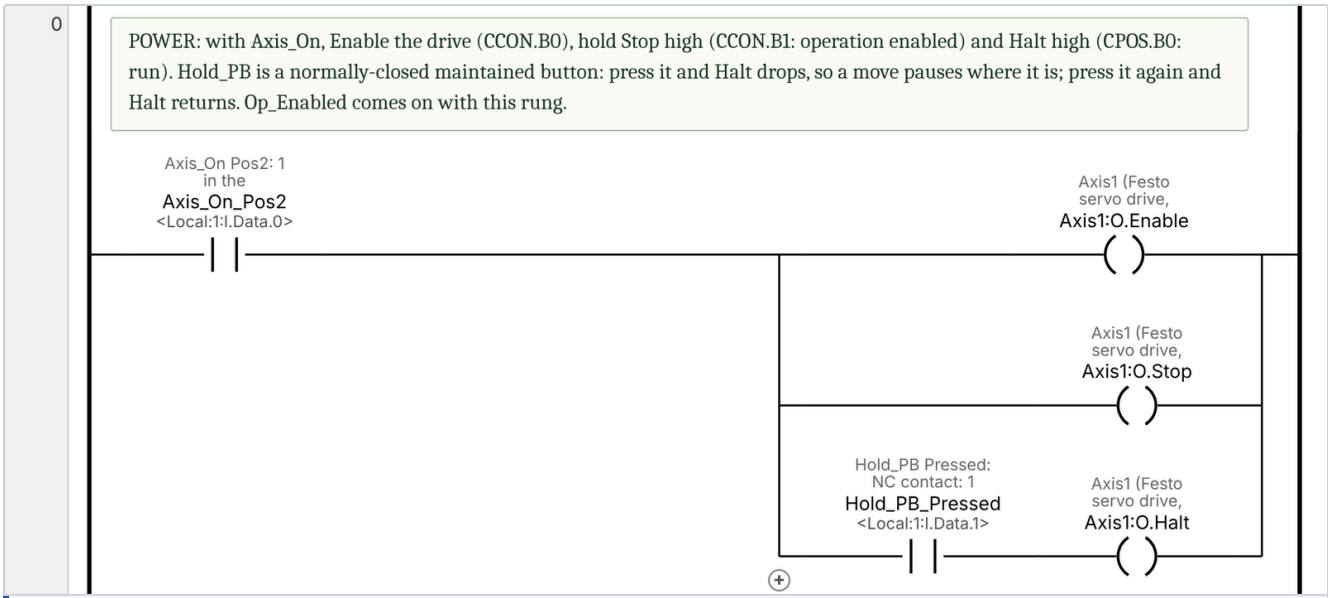
FOUR THINGS THE DRIVE INSISTS ON

- **Enable and Stop** must both be 1 before anything moves: Enable powers the stage, Stop high means *not stopped*. Op_Enabled comes back when both are there.
- **Home first.** A Start before the axis is referenced is a fault, not a move. Home is the one move allowed before Referenced.
- **A record must exist** and its target must be inside the stroke, or Start is a fault.
- **One Start at a time.** A second Start while a move runs is ignored with Warning. The drive counts them; so does Level 10.

The program, top down

File → New Project → Sandbox: servo axis opens the panel with these seven rungs. Read them in order: power, fault handling, homing, the record, the start, the handshake.

RUNG 0




```
XIC(Axis_On_Pos2)
[OTE(Axis1:O.Enable),OTE(Axis1:O.Stop),XIC(Hold_PB_Pressed)OTE(Axis1:O.Halt)];
```

POWER: with Axis_On, Enable the drive (CCON.B0), hold Stop high (CCON.B1: operation enabled) and Halt high (CPOS.B0: run). Hold_PB is a normally-closed maintained button: press it and Halt drops, so a move pauses where it is; press it again and Halt returns. Op_Enabled comes on with this rung.

PREDICT IT · 1 `Axis_On_Pos2` is off. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Axis_On_Pos2` (1 in the right-hand position) is on, the rung picks its path: in any case it turns `Axis1:O.Enable` on (and off again whenever the rung is false); in any case it turns `Axis1:O.Stop` on (and off again whenever the rung is false); if `Hold_PB_Pressed` (NC contact: 1 until pressed) is on, it turns `Axis1:O.Halt` on (and off again whenever the rung is false).

Left to right:

1. `XIC(Axis_On_Pos2)` *Examine If Closed* asks: true when `Axis_On_Pos2` (1 in the right-hand position) is on.
2. A branch of 3 parallel paths: each path is tested on its own and does its own work.

   Path 1

   1. `OTE(Axis1:O.Enable)` *Output Energize* does: turns `Axis1:O.Enable` on (and off again whenever the rung is false). Every scan it writes the rung state: true on, false off.

   Path 2

   1. `OTE(Axis1:O.Stop)` *Output Energize* does: turns `Axis1:O.Stop` on (and off again whenever the rung is false).

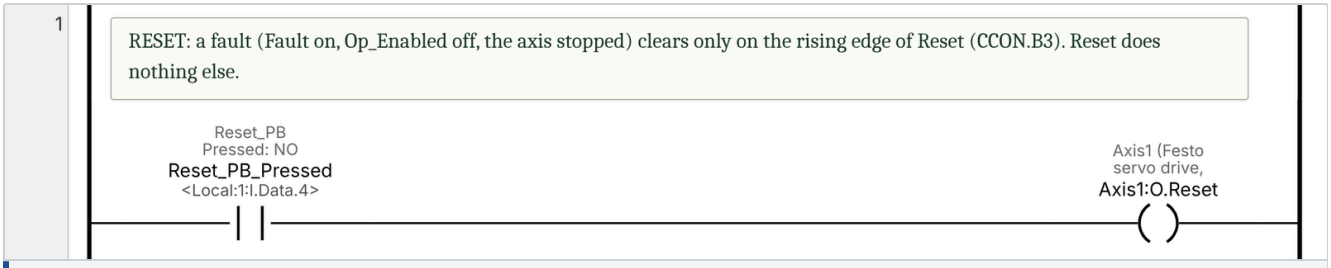
   Path 3

   1. `XIC(Hold_PB_Pressed)` *Examine If Closed* asks: true when `Hold_PB_Pressed` (NC contact: 1 until pressed) is on.

2. `OTE(Axis1:O.Halt)` *Output Energize* does: turns `Axis1:O.Halt` on (and off again whenever the rung is false).

> **Think of it like this:** a rung is a pipe with valves in it, read left to right. Every contact is a valve: XIC is open while its bit is 1, XIO while its bit is 0. Water reaches the output at the right only if every valve in a row is open. The output does not remember anything: on every scan it is simply wet or dry.

RUNG 1




```
XIC(Reset_PB_Pressed)OTE(Axis1:O.Reset);
```

RESET: a fault (Fault on, Op_Enabled off, the axis stopped) clears only on the rising edge of Reset (CCON.B3). Reset does nothing else.

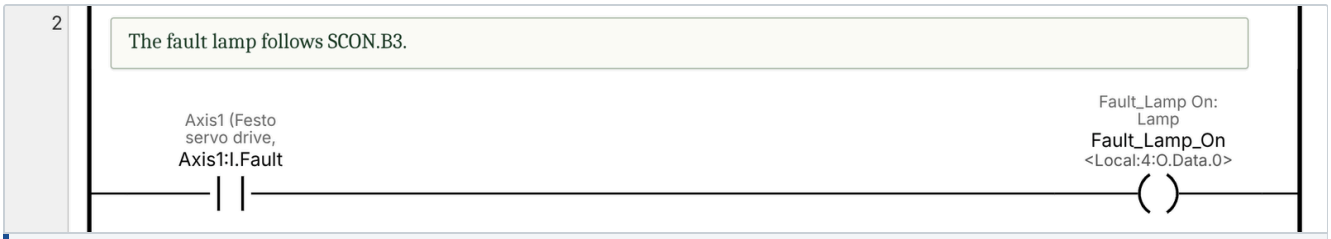
HOW THIS RUNG WORKS

In words. When `Reset_PB_Pressed` (NO contact: 1 while pressed) is on, the rung turns `Axis1:O.Reset` on (and off again whenever the rung is false).

Left to right:

1. `XIC(Reset_PB_Pressed)` *Examine If Closed* asks: true when `Reset_PB_Pressed` (NO contact: 1 while pressed) is on.
2. `OTE(Axis1:O.Reset)` *Output Energize* does: turns `Axis1:O.Reset` on (and off again whenever the rung is false).

RUNG 2




```
XIC(Axis1:I.Fault)OTE(Fault_Lamp_On);
```

The fault lamp follows SCON.B3.

HOW THIS RUNG WORKS

In words. When `Axis1:I.Fault` is on, the rung turns `Fault_Lamp_On` (Lamp) on (and off again whenever the rung is false).

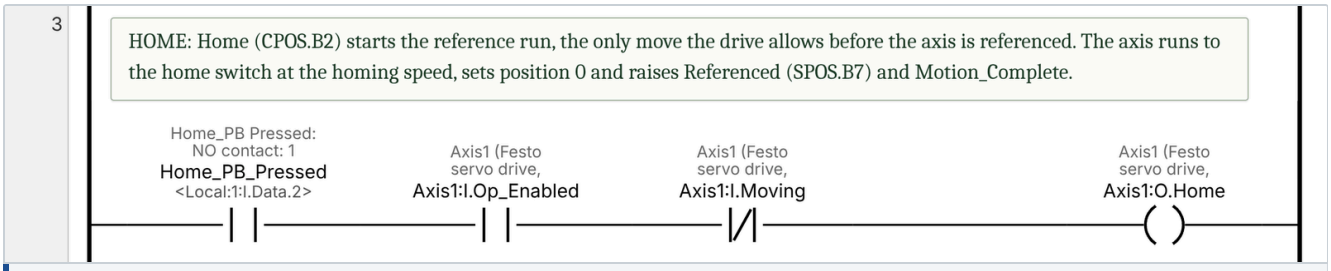
Left to right:

1. `XIC(Axis1:I.Fault)` *Examine If Closed* asks: true when `Axis1:I.Fault` is on.
2. `OTE(Fault_Lamp_On)` *Output Energize* does: turns `Fault_Lamp_On` (Lamp) on (and off again whenever the rung is false).

TRY IT: A FAULT AND ITS RESET

Run it, turn *Axis_On* and watch Op_Enabled come on (green LED at the motor). Now, before homing, give the drive a Start by hand: in the tag monitor set `Axis1:O.Start` to 1 for a moment. The red LED lights, the lamp lights, Op_Enabled drops, and the axis properties say why: *not referenced*. Press *Reset*: the fault clears on the rising edge and nothing else happens. The START rung below never lets this happen, because it checks Referenced first; the drive protects itself anyway.

RUNG 3




```
XIC(Home_PB_Pressed)XIC(Axis1:I.Op_Enabled)XIO(Axis1:I.Moving)OTE(Axis1:O.Home)
;
```

HOME: Home (CPOS.B2) starts the reference run, the only move the drive allows before the axis is referenced. The axis runs to the home switch at the homing speed, sets position 0 and raises Referenced (SPOS.B7) and Motion_Complete.

PREDICT IT · 2 `Home_PB_Pressed` is on, `Axis1:I.Op_Enabled` is on and `Axis1:I.Moving` is off. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Home_PB_Pressed` (NO contact: 1 while pressed) is on, `Axis1:I.Op_Enabled` is on and `Axis1:I.Moving` is off, the rung turns `Axis1:O.Home` on (and off again whenever the rung is false).

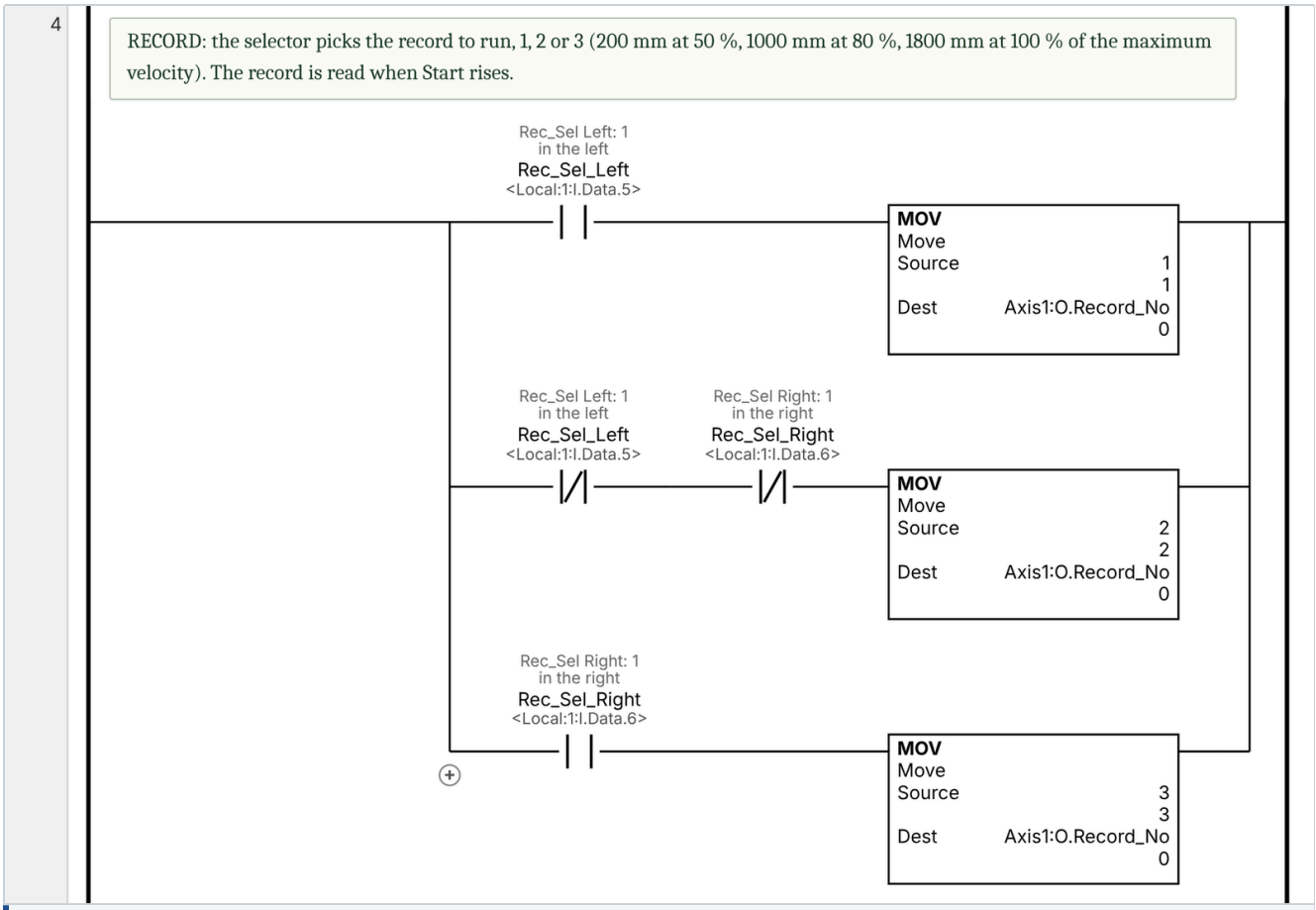
Left to right:

1. `XIC(Home_PB_Pressed)` *Examine If Closed* asks: true when `Home_PB_Pressed` (NO contact: 1 while pressed) is on.
2. `XIC(Axis1:I.Op_Enabled)` *Examine If Closed* asks: true when `Axis1:I.Op_Enabled` is on.
3. `XIO(Axis1:I.Moving)` *Examine If Open* asks: true when `Axis1:I.Moving` is off.
4. `OTE(Axis1:O.Home)` *Output Energize* does: turns `Axis1:O.Home` on (and off again whenever the rung is false).

TRY IT: HOMING

Press *Home*. The carriage runs to the home switch at the left at the homing speed (50 mm/s), the position readout goes to 0.0 mm and the blue LED (Referenced) lights, then the yellow one (Motion_Complete). Watch Moving while it runs: the HOME rung uses it so a second press during the run does nothing.

RUNG 4




```
[XIC(Rec_Sel_Left)MOV(1,Axis1:O.Record_No),XIO(Rec_Sel_Left)XIO(Rec_Sel_Right)MOV(2,Axis1:O.Record_No),XIC(Rec_Sel_Right)MOV(3,Axis1:O.Record_No)];
```

RECORD: the selector picks the record to run, 1, 2 or 3 (200 mm at 50 %, 1000 mm at 80 %, 1800 mm at 100 % of the maximum velocity). The record is read when Start rises.

PREDICT IT · 3 `Rec_Sel_Right` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. Every scan, with nothing to test, the rung picks its path: if `Rec_Sel_Left` (1 in the left position) is on, it puts **1** into `Axis1:O.Record_No`; if `Rec_Sel_Left` is off and `Rec_Sel_Right` (1 in the right position) is off, it puts **2** into `Axis1:O.Record_No`; if `Rec_Sel_Right` is on, it puts **3** into `Axis1:O.Record_No`.

Left to right:

1. A branch of 3 parallel paths: each path is tested on its own and does its own work.

   Path 1

   1. `XIC(Rec_Sel_Left)` *Examine If Closed* asks: true when `Rec_Sel_Left` (1 in the left position) is on.
   2. `MOV(1,Axis1:O.Record_No)` *Move* does: puts **1** into `Axis1:O.Record_No`. It writes every scan the rung is true.

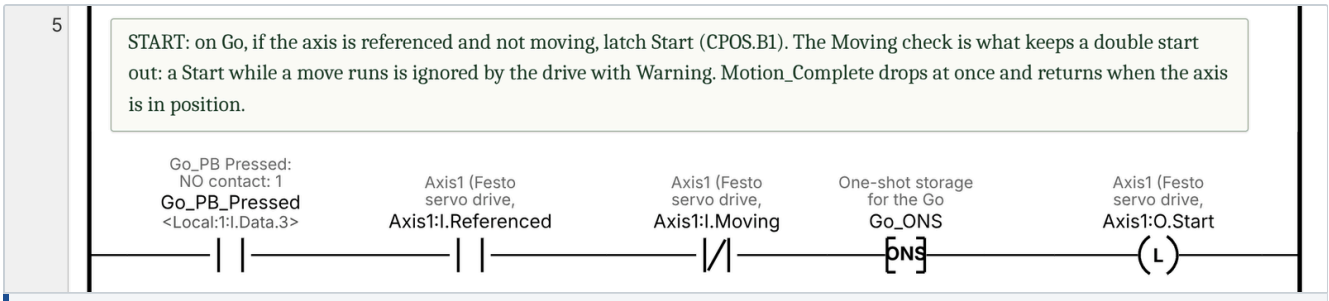
   Path 2

3. `XIO(Rec_Sel_Left)` *Examine If Open* asks: true when `Rec_Sel_Left` is off.
4. `XIO(Rec_Sel_Right)` *Examine If Open* asks: true when `Rec_Sel_Right` (1 in the right position) is off.
5. `MOV(2,Axis1:O.Record_No)` *Move* does: puts **2** into `Axis1:O.Record_No`.

Path 3

1. `XIC(Rec_Sel_Right)` *Examine If Closed* asks: true when `Rec_Sel_Right` is on.
2. `MOV(3,Axis1:O.Record_No)` *Move* does: puts **3** into `Axis1:O.Record_No`.

RUNG 5




```
XIC(Go_PB_Pressed)XIC(Axis1:I.Referenced)XIO(Axis1:I.Moving)ONS(Go_ONS)OTL(Axis1:O.Start);
```

START: on Go, if the axis is referenced and not moving, latch Start (CPOS.B1). The Moving check is what keeps a double start out: a Start while a move runs is ignored by the drive with Warning. Motion_Complete drops at once and returns when the axis is in position.

HOW THIS RUNG WORKS

In words. When `Go_PB_Pressed` (NO contact: 1 while pressed) is on, `Axis1:I.Referenced` is on, `Axis1:I.Moving` is off and it is the first scan they are true (the one-shot, remembered in `Go_ONS` (One-shot storage for the Go button)), the rung latches `Axis1:O.Start` on.

Left to right:

1. `XIC(Go_PB_Pressed)` *Examine If Closed* asks: true when `Go_PB_Pressed` (NO contact: 1 while pressed) is on.
2. `XIC(Axis1:I.Referenced)` *Examine If Closed* asks: true when `Axis1:I.Referenced` is on.
3. `XIO(Axis1:I.Moving)` *Examine If Open* asks: true when `Axis1:I.Moving` is off.
4. `ONS(Go_ONS)` *One Shot* asks: true when it is the first scan they are true (the one-shot, remembered in `Go_ONS` (One-shot storage for the Go button)). It passes power for one scan only, the scan its input goes from false to true; its storage bit remembers last scan.
5. `OTL(Axis1:O.Start)` *Output Latch* does: latches `Axis1:O.Start` on. It only ever sets the bit; the bit stays on after the rung goes false.

Why it is built this way:

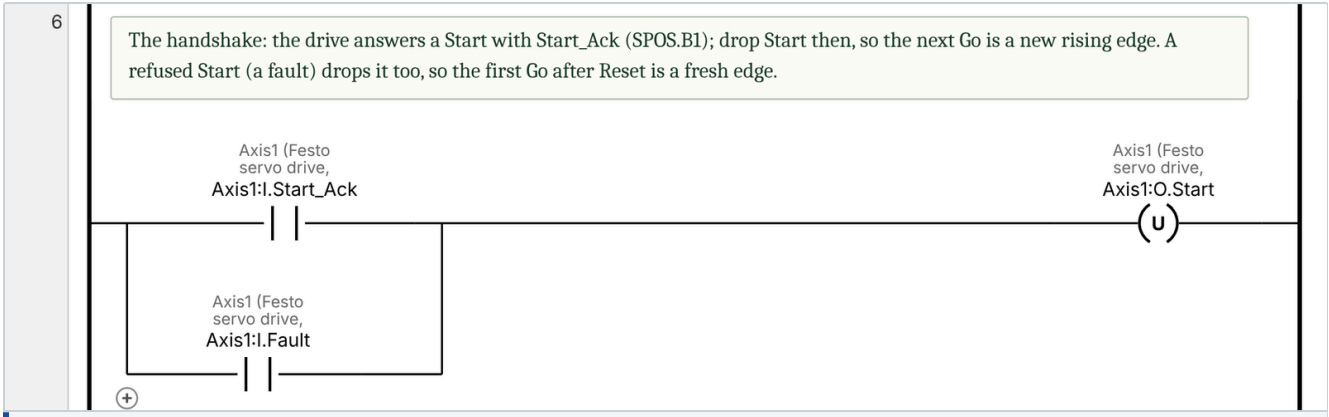
One-shot. Everything after the ONS happens once per event, not on every scan the condition holds; `Go_ONS` is only its memory of the last scan.

A latch. `Axis1:O.Start` stays on after this rung goes false; an `OTU` elsewhere in the program clears it.

> **Think of it like this:** a doorbell that rings once however long you lean on the button. The ONS looks at what the button was doing last scan: only the change from up to down rings it.

> **Watch for:** Each ONS needs its own storage bit. Two ONS sharing one bit make one of them miss its edges.

RUNG 6




```
[XIC(Axis1:I.Start_Ack),XIC(Axis1:I.Fault)]OTU(Axis1:O.Start);
```

The handshake: the drive answers a Start with Start_Ack (SPOS.B1); drop Start then, so the next Go is a new rising edge. A refused Start (a fault) drops it too, so the first Go after Reset is a fresh edge.

PREDICT IT · 4 `Axis1:I.Start_Ack` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When either (`Axis1:I.Start_Ack` is on) *or* (`Axis1:I.Fault` is on), the rung unlatches `Axis1:O.Start` (turns it off).

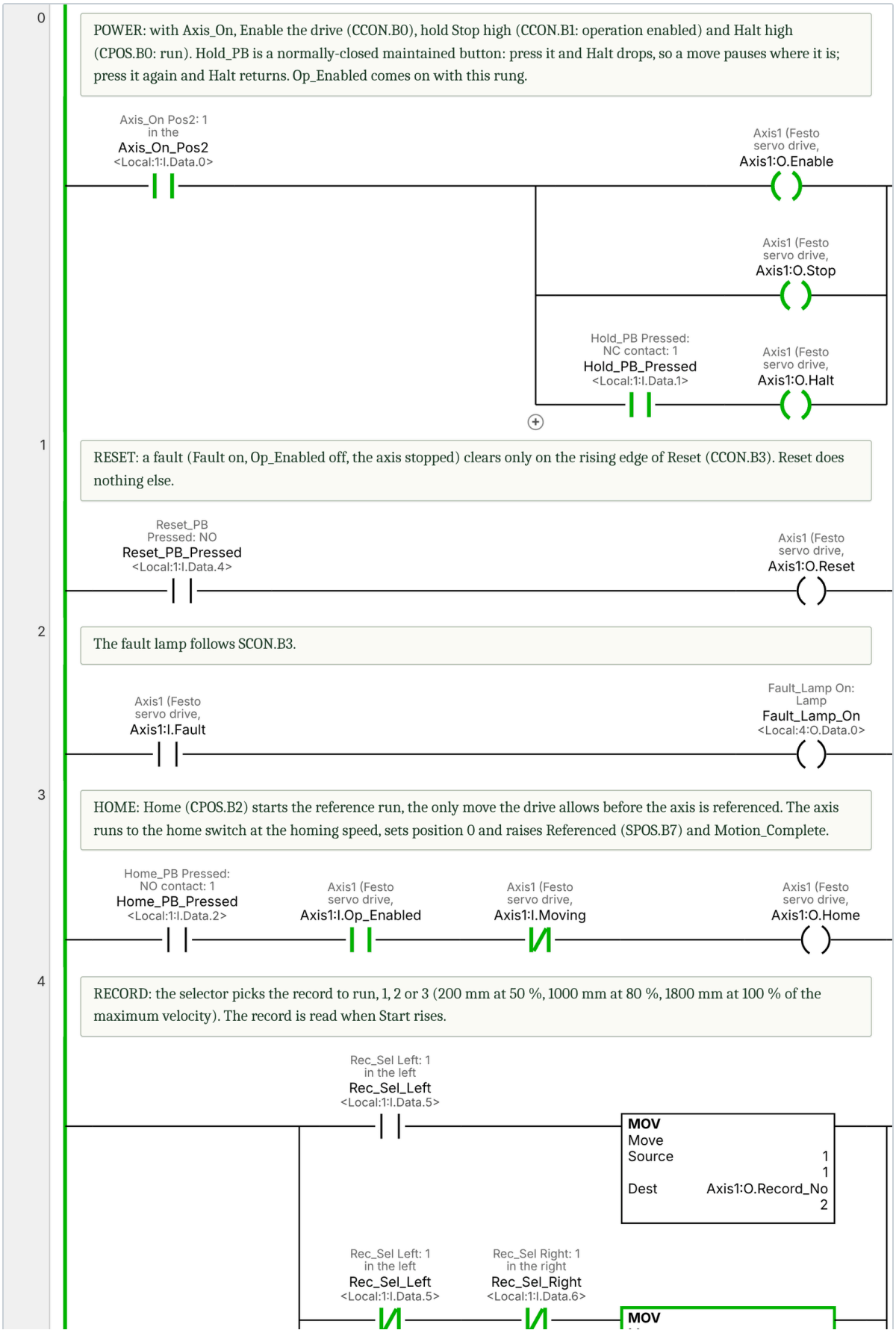
Left to right:

1. A branch of 2 parallel paths: power gets through if any one of them is true.
   - **Path 1**
     1. `XIC(Axis1:I.Start_Ack)` *Examine If Closed* asks: true when `Axis1:I.Start_Ack` is on.
   - **Path 2**
     1. `XIC(Axis1:I.Fault)` *Examine If Closed* asks: true when `Axis1:I.Fault` is on.
2. `OTU(Axis1:O.Start)` *Output Unlatch* does: unlatches `Axis1:O.Start` (turns it off). It only ever clears the bit.

> **Think of it like this:** parallel paths are two roads to the same town. If either road is open you get there; the rung only cares that power reaches the far end, not which way it went.

TRY IT: RECORDS, A DOUBLE START, HALT AND RESUME

Selector left, *Go*: 200 mm at half speed. Middle: 1000 mm. Right: 1800 mm, full speed. Watch `Axis1:O.Start` in the monitor: it goes 1 on the ONS and 0 again a scan later when Start_Ack answers, so Go is a new rising edge every time. Now press *Go* twice quickly during a long move. The second press is blocked by XIO(Moving) in rung 5, so nothing happens. Delete that contact and try again: the drive raises Warning and the axis properties count a *double start*. Put the contact back. Last, press the red *Hold* mushroom during a move: Halt drops, the axis stops where it is, Halt_Ack drops, Motion_Complete stays off. Press it again: Halt returns and the move carries on to its target.


0
POWER: with Axis_On, Enable the drive (CCON.B0), hold Stop high (CCON.B1: operation enabled) and Halt high (CPOS.B0: run). Hold_PB is a normally-closed maintained button: press it and Halt drops, so a move pauses where it is; press it again and Halt returns. Op_Enabled comes on with this rung.
Axis_On Pos2: 1 in the
Axis_On_Pos2
<Local:1:I.Data.0>
Axis1 (Festo servo drive,
Axis1:O.Enable
Axis1 (Festo servo drive,
Axis1:O.Stop
Hold_PB Pressed: NC contact: 1
Hold_PB_Pressed
<Local:1:I.Data.1>
Axis1 (Festo servo drive,
Axis1:O.Halt
1
RESET: a fault (Fault on, Op_Enabled off, the axis stopped) clears only on the rising edge of Reset (CCON.B3). Reset does nothing else.
Reset_PB Pressed: NO
Reset_PB_Pressed
<Local:1:I.Data.4>
Axis1 (Festo servo drive,
Axis1:O.Reset
2
The fault lamp follows SCON.B3.
Axis1 (Festo servo drive,
Axis1:I.Fault
Fault_Lamp On: Lamp
Fault_Lamp_On
<Local:4:O.Data.0>
3
HOME: Home (CPOS.B2) starts the reference run, the only move the drive allows before the axis is referenced. The axis runs to the home switch at the homing speed, sets position 0 and raises Referenced (SPOS.B7) and Motion_Complete.
Home_PB Pressed: NO contact: 1
Home_PB_Pressed
<Local:1:I.Data.2>
Axis1 (Festo servo drive,
Axis1:I.Op_Enabled
Axis1 (Festo servo drive,
Axis1:I.Moving
Axis1 (Festo servo drive,
Axis1:O.Home
4
RECORD: the selector picks the record to run, 1, 2 or 3 (200 mm at 50 %, 1000 mm at 80 %, 1800 mm at 100 % of the maximum velocity). The record is read when Start rises.
Rec_Sel Left: 1 in the left
Rec_Sel_Left
<Local:1:I.Data.5>
MOV
Move
Source
1
1
Dest
Axis1:O.Record_No
2
Rec_Sel Left: 1 in the left
Rec_Sel_Left
<Local:1:I.Data.5>
Rec_Sel Right: 1 in the right
Rec_Sel_Right
<Local:1:I.Data.6>
MOV

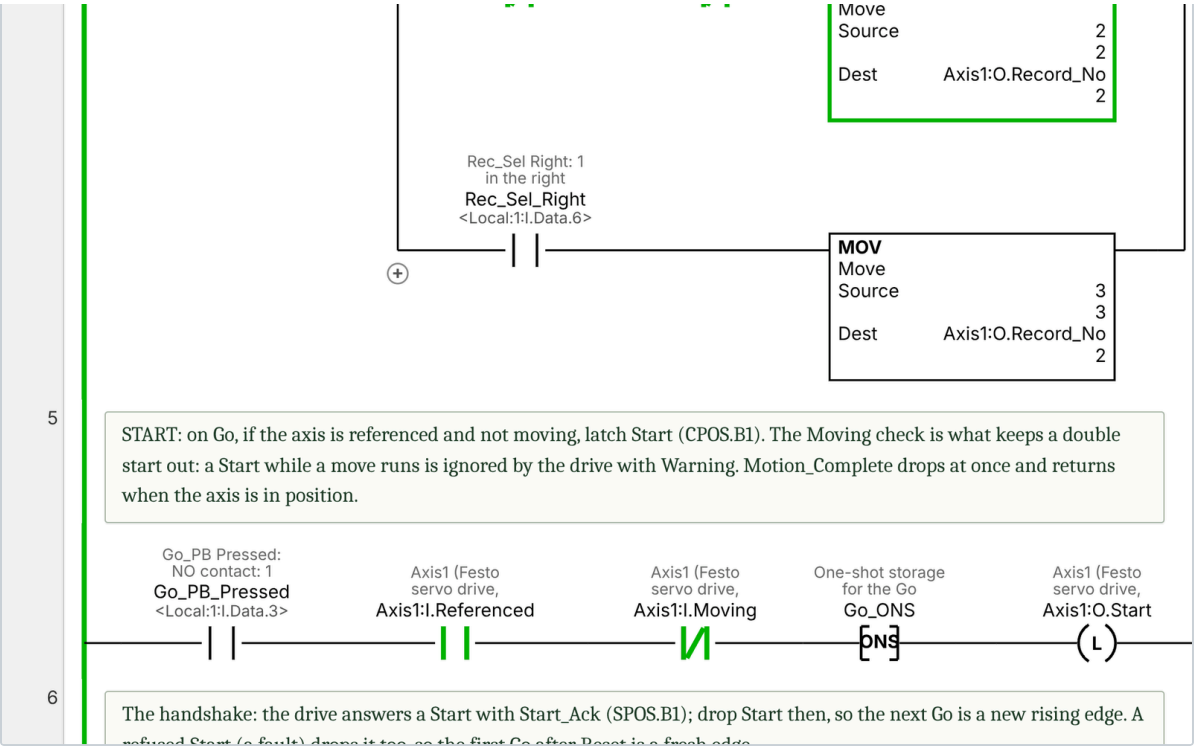


Figure 197. The routine online, the axis sitting at record 2: Enable, Stop and Halt made, Op_Enabled and Referenced true, Record_No 2, Start already dropped by the acknowledge, Motion_Complete back.

Reading the drive: the control and status bits, and what a healthy one looks like

Two things about FHPP trip up everybody the first time, and they are the same thing seen from both ends. On the way out, the bits named **Stop** and **Halt** are *permissions*, not commands: a 1 means *allowed*, a 0 means *stop* or *hold*. On the way back, the drive answers every permission with an acknowledge bit that mirrors it, so a healthy, idle drive shows a row of 1s where you might expect 0s. Learn the idle picture once and every fault afterwards is a bit that is not where it should be.

The Power rung

```
XIC(Run)
[OTE(Axis1:O.Enable),OTE(Axis1:O.Stop),OTE(Axis1:O.Halt),OTE(Lamp_Run)];
```

HOW THIS RUNG WORKS

In words. When `Run` is on, the rung turns `Axis1:O.Enable` on (and off again whenever the rung is false), turns `Axis1:O.Stop` on (and off again whenever the rung is false), turns `Axis1:O.Halt` on (and off again whenever the rung is false) and turns `Lamp_Run` on (and off again whenever the rung is false).

Left to right:

1. `XIC(Run)` *Examine If Closed* asks: true when `Run` is on.
2. A branch of 4 parallel paths: each path is tested on its own and does its own work.

Path 1

1. `OTE(Axis1:O.Enable)` *Output Energize* does: turns `Axis1:O.Enable` on (and off again whenever the rung is false).

Path 2

1. `OTE(Axis1:O.Stop)` *Output Energize* does: turns `Axis1:O.Stop` on (and off again whenever the rung is false).

Path 3

1. `OTE(Axis1:O.Halt)` *Output Energize* does: turns `Axis1:O.Halt` on (and off again whenever the rung is false).

Path 4

1. `OTE(Lamp_Run)` *Output Energize* does: turns `Lamp_Run` on (and off again whenever the rung is false).

When Run is true all three drive bits go to 1 and the drive reads them as:

Output	FHPP bit	1 means	0 means
`Axis1:O.Enable`	CCON.0 ENABLE	Power stage on: the motor is under closed-loop control and holds position	Disabled: no torque, no commands taken; a running move coasts to a controlled stop and the task is dropped
`Axis1:O.Stop`	CCON.1 STOP	**Operation enabled**, despite the name. The drive's "OK to run"	Controlled stop on the stop ramp; the task is cancelled; no Start is taken until the bit is 1 again
`Axis1:O.Halt`	CPOS.0 HALT	**Not halted**: a move runs	The move pauses on its ramp but the task is *kept*; raise Halt and a Start resumes to the same target. The sandbox panel's Hold button works this bit

The drive sequences them itself, Enable → Stop → Halt, so writing all three in one rung is normal; when the last one is in it answers with `Op_Enabled`, and that is the bit the Home and Start rungs wait on. Stop throws a move away; Halt parks it. That is the whole difference, and it decides what you tie each bit to. Tying all three to Run, as above, means a Stop button or an E-stop cancels the move and takes the torque off: fine for a pick-and-place. On a machine that should hold position on a stop, keep Enable on and drop only Stop; the axis stands still under torque and the next Start begins a fresh move.

A healthy drive, idle and homed

This is the axis in the sandbox scene after Axis_On and a Home, standing at 0 with nothing to do. Every bit in both tables is in the state it should be; the two rows of 1s on the status side are the acknowledges.

Name	Value	Force Mask	Style all ▾	Data Type	Description
Axis1:O				_festo:O:0	Festo servo drive, FHPP 192.168.1.61 (simulated)
Axis1:O.Enable	1		Decimal	BOOL	CCON.B0 ENABLE: enable the drive
Axis1:O.Stop	1		Decimal	BOOL	CCON.B1 STOP: 1 = operation enabled, 0 = stop (a controlled stop, the task is dropped)
Axis1:O.Brake	0		Decimal	BOOL	CCON.B2 BRAKE: release the holding brake
Axis1:O.Reset	0		Decimal	BOOL	CCON.B3 RESET: clear a fault on the rising edge
Axis1:O.Lock	0		Decimal	BOOL	CCON.B5 LOCK: block FCT/MMI access
Axis1:O.OPM1	0		Decimal	BOOL	CCON.B6: operating mode bit 1 (0 Record Select, 1 Direct mode)
Axis1:O.OPM2	0		Decimal	BOOL	CCON.B7: operating mode bit 2 (0)
Axis1:O.Halt	1		Decimal	BOOL	CPOS.B0 HALT: 1 = run, 0 = halt (stop and keep the task; Start resumes it)
Axis1:O.Start	0		Decimal	BOOL	CPOS.B1 START: start the task on the rising edge
Axis1:O.Home	0		Decimal	BOOL	CPOS.B2 HOM: start homing on the rising edge
Axis1:O.Jog_Pos	0		Decimal	BOOL	CPOS.B3 JOGP: jog positive while 1
Axis1:O.Jog_Neg	0		Decimal	BOOL	CPOS.B4 JOGN: jog negative while 1
Axis1:O.Teach	0		Decimal	BOOL	CPOS.B5 TEACH: store the actual position in the record (rising edge)
Axis1:O.Clear	0		Decimal	BOOL	CPOS.B6 CLEAR: drop the task held by a halt (rising edge)
Axis1:O.Record_No	2		Decimal	SINT	Byte 3: the record to run, 1 … n (Record Select) / CDIR (Direct mode: bit 0 = relative)
Axis1:O.Setpoint_Vel	0		Decimal	SINT	Byte 4 (Direct mode): velocity, % of the maximum
Axis1:O.Setpoint_Pos	0		Decimal	DINT	Bytes 5-8 (Direct mode): target position, 0.01 mm

Figure 198. Axis1:O, the control bytes the program writes. Three 1s: Enable, Stop and Halt, the permissions. Record_No carries the selector's choice (2) and waits for a Start.

Name	Value	Force Mask	Style all ▾	Data Type	Description
Axis1:I				_festo:I:0	Festo servo drive, FHPP 192.168.1.61 (simulated)
Axis1:I.Enabled	1		Decimal	BOOL	SCON.B0 ENABLED: the drive is enabled
Axis1:I.Op_Enabled	1		Decimal	BOOL	SCON.B1 OPEN: operation enabled, motion allowed
Axis1:I.Warning	0		Decimal	BOOL	SCON.B2 WARN: a warning is pending (a start that was ignored)
Axis1:I.Fault	0		Decimal	BOOL	SCON.B3 FAULT: faulted, no motion until Reset
Axis1:I.Load_Voltage	1		Decimal	BOOL	SCON.B4 VLOAD: load voltage present
Axis1:I.FCT_Control	0		Decimal	BOOL	SCON.B5 FCT/MMI has device control
Axis1:I.OPM1_Ack	0		Decimal	BOOL	SCON.B6: operating mode bit 1 as the drive runs it
Axis1:I.OPM2_Ack	0		Decimal	BOOL	SCON.B7: operating mode bit 2
Axis1:I.Halt_Ack	1		Decimal	BOOL	SPOS.B0 HALT: 1 = halt not active
Axis1:I.Start_Ack	0		Decimal	BOOL	SPOS.B1 ACK: the start was taken (follows Start)
Axis1:I.Motion_Complete	1		Decimal	BOOL	SPOS.B2 MC: the positioning task is complete (1 at rest)
Axis1:I.Teach_Ack	0		Decimal	BOOL	SPOS.B3 TEACH: the position was stored
Axis1:I.Moving	0		Decimal	BOOL	SPOS.B4 MOV: the axis is moving
Axis1:I.Drag_Error	0		Decimal	BOOL	SPOS.B5 DEV: following error
Axis1:I.Standstill	1		Decimal	BOOL	SPOS.B6 STILL: at rest with operation enabled
Axis1:I.Referenced	1		Decimal	BOOL	SPOS.B7 REF: the axis is homed
Axis1:I.Record_No	0		Decimal	SINT	Byte 3: the record that ran (Record Select) / SDIR (Direct mode)
Axis1:I.Actual_Vel	0		Decimal	SINT	Byte 4: actual velocity, % of the maximum
Axis1:I.Actual_Pos	0		Decimal	DINT	Bytes 5-8: actual position, 0.01 mm

Figure 199. Axis1:I, the status bytes the drive writes back. Enabled, Op_Enabled, Load_Voltage, Halt_Ack, Motion_Complete, Standstill and Referenced are 1; everything that means trouble or activity is 0.

Input	FHPP bit	Healthy idle	What it tells you
`Enabled`	SCON.0 ENABLED	1	The power stage is on: your Enable was accepted
`Op_Enabled`	SCON.1 OPEN	1	All three permissions are in; the drive will take Home, Start, Jog. Everything downstream waits on this
`Warning`	SCON.2 WARN	0	A command was ignored (a Start while moving, a Start with Halt low). Not a fault: the drive carries on, and Reset clears it
`Fault`	SCON.3 FAULT	0	Faulted: Op_Enabled drops, the axis stops, nothing runs until the rising edge of Reset

Input	FHPP bit	Healthy idle	What it tells you
`Load_Voltage`	SCON.4 VLOAD	1	The motor supply is present. Separate from the 24 V logic supply: on a real panel this is the bit that goes to 0 when the safety contactor drops the motor power while the drive stays awake
`FCT_Control`	SCON.5 FCT/MMI	0	The PLC has control. When a technician takes device control from the Festo Configuration Tool this goes to 1 and your Starts are ignored; Lock (CCON.5) refuses it
`OPM1_Ack`, `OPM2_Ack`	SCON.6, SCON.7	0, 0	The operating mode the drive is actually in: 0 0 is Record Select, 1 0 is Direct mode. They mirror your OPM1 and OPM2; check them before a Start if you switch modes
`Halt_Ack`	SPOS.0 HALT	1	Mirrors Halt: 1 = not halted, free to move
`Start_Ack`	SPOS.1 ACK	0	1 while your Start is high and the drive has taken it. The edge the unlatch rung uses; a refused Start gets a fault instead, which is why the unlatch rung has a Fault branch too
`Motion_Complete`	SPOS.2 MC	1	No task in progress. Drops the instant a move or a homing starts, returns when the target is reached. Gate your in-position logic on it
`Teach_Ack`	SPOS.3 TEACH	0	1 after a Teach stored the position
`Moving`	SPOS.4 MOV	0	The axis is in motion; use it to keep a second Start out
`Drag_Error`	SPOS.5 DEV	0	Following error: the axis is not where the drive commanded, a mechanical problem or an overload
`Standstill`	SPOS.6 STILL	1	At rest with operation enabled; off while moving
`Referenced`	SPOS.7 REF	1	Homed. 0 after power-up (and after a homing starts) until the reference run completes; a Start with this 0 is the positioning-before-homing fault
`Record_No`, `Actual_Vel`, `Actual_Pos`	bytes 3, 4, 5-8	last record, 0, position	The record that ran, the speed in percent, and the position in 0.01 mm: 150000 is 1500 mm

The LEDs on the motor

The drive's status is also on the machine, where a technician sees it without a laptop: four LEDs on the motor end of the axis, labelled from Version 130. They are the four status bits that matter most, and the healthy idle picture is three lit and one dark.

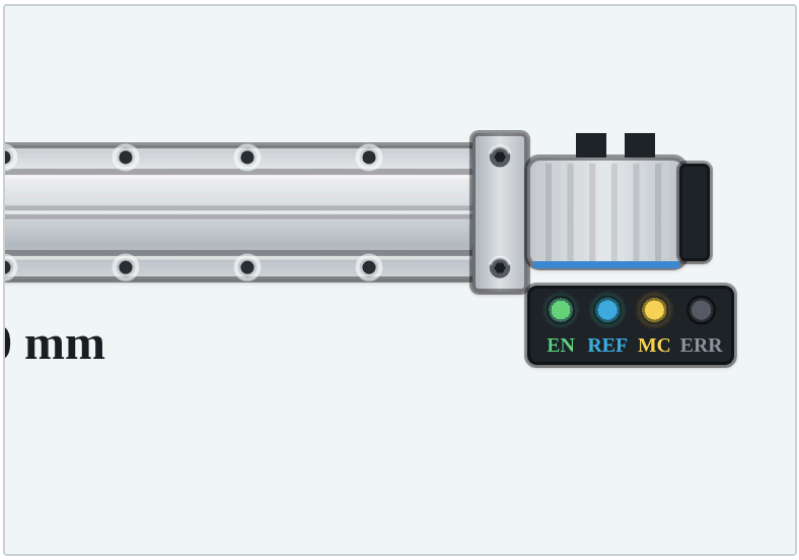


Figure 200. The motor end of the axis after Axis_On and Home: EN, REF and MC lit, ERR dark. The same four LEDs are on the motor in the side view and in 3D.

LED	Colour	Status bit	Lit means
EN	green	`Op_Enabled` (SCON.1)	The drive is enabled and in operation: Enable, Stop and Halt are all 1. Dark with Axis_On on: look at the Power rung's conditions, or ERR is lit
REF	blue	`Referenced` (SPOS.7)	The axis is homed; positioning is allowed. Dark: Home has not run yet (or is running), and a Start will fault
MC	yellow	`Motion_Complete` and not `Moving`	At rest with the last task finished. Dark while a move or a homing runs; back when the target is reached
ERR	red	`Fault` (SCON.3)	Faulted: EN goes dark with it, the axis stops, nothing runs until Reset. The log names the fault

Read them as a pattern. **EN REF MC**: healthy and idle, ready for a Start. **EN** alone: powered but not homed, the state right after power-up. **EN REF** with MC dark: moving. **REF MC** with EN dark: Axis_On is off, or Stop or Halt has dropped: the drive remembers it is homed, it just will not move. **ERR**: Reset, then see which of the others comes back. Nothing lit: no power to the drive, or the module is not connected (Axis1 red in I/O Configuration).

Auto-home

```
XIC(Run)XIC(Axis1:I.Op_Enabled)XIO(Axis1:I.Referenced)XIO(Axis1:I.Moving)XIO(Axis1:I.Fault)OTE(Axis1:O.Home);
```

PREDICT IT · 5 `Run` is on, `Axis1:I.Op_Enabled` is on, `Axis1:I.Referenced` is off, `Axis1:I.Moving` is off and `Axis1:I.Fault` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Run` is on, `Axis1:I.Op_Enabled` is on, `Axis1:I.Referenced` is off, `Axis1:I.Moving` is off and `Axis1:I.Fault` is off, the rung turns `Axis1:O.Home` on (and off again whenever the rung is false).

Left to right:

1. `XIC(Run)` *Examine If Closed* asks: true when `Run` is on.
2. `XIC(Axis1:I.Op_Enabled)` *Examine If Closed* asks: true when `Axis1:I.Op_Enabled` is on.
3. `XIO(Axis1:I.Referenced)` *Examine If Open* asks: true when `Axis1:I.Referenced` is off.
4. `XIO(Axis1:I.Moving)` *Examine If Open* asks: true when `Axis1:I.Moving` is off.
5. `XIO(Axis1:I.Fault)` *Examine If Open* asks: true when `Axis1:I.Fault` is off.
6. `OTE(Axis1:O.Home)` *Output Energize* does: turns `Axis1:O.Home` on (and off again whenever the rung is false).

The lesson programs home from a button so the fault can be shown on purpose; a cell that runs on its own homes itself like this. The instant the drive is in operation and the axis is not referenced, Home goes true; the drive takes the rising edge and runs its reference move; Referenced comes on and the XIO opens. Two details that make it right: `XIO(Moving)` opens as soon as homing starts, so Home is a short pulse rather than held for the whole run, which is fine because HOM is edge-triggered and dropping it does not abort the homing; and `XIO(Fault)` means a faulted drive is never asked to home, but since Referenced is still 0 it homes again by itself the moment Reset clears the fault. **Where the homing goes** is not in the program at all: the method, to a reference switch, to the end stop with a current limit, or switch then index, is configured in the drive. With an absolute multiturn encoder there is no homing run and Referenced is 1 from power-up.

Field notes: the first program, and what went wrong

This section is the log of one afternoon spent writing the Level 10 program and the guided track from scratch, kept because every item in it is a mistake the next person will make too. Each is a symptom, the cause, and the fix. Going through a program this way, one failure at a time, is what makes it bulletproof: a rung you have watched fail is a rung you understand.

Every Go faults, and the fault lamp is on

Where to read it: the axis itself says `FAULT: not referenced, Home first` under the carriage, and the Simulator Log tab (bottom of the window; on the phone, Output → Simulator Log) has the full line: `Axis1: fault: positioning before homing (axis not referenced): pulse Home first`. The simulated drive raises exactly three faults: a Start before the axis is referenced; a Start for a record that is not set (`record 3 is not set (records 1 to 2)`); a target past the stroke. Everything else the drive refuses is a **warning**, not a fault: a Start while moving (a double start), a Start with Halt low. A warning sets the Warning bit and a log line, the axis carries on, Reset clears it. A fault drops Op_Enabled and stops the axis until Reset.

The cause, that afternoon: the Home rung had been typed with the controller in Run. It was a pending online edit, verified and visible and not running. See the Watch-for above; the card now says so, and Program → Run (a download) or the card's Assemble button makes it the program.

The carriage is stranded over a bin

Clear Parts removed the bottle the gripper was holding; the set-down rung, which is what sent the carriage back to the pick point, never fired because there was nothing to set down. Any program whose return move is triggered by an *event* has this hole; a fault reset mid-cycle or a dropped bottle opens it too. Make the return depend on the *state*: empty-handed, standing still, not at the pick point → go home.

```
XIC(Run)XIC(Axis1:I.Op_Enabled)XIC(Axis1:I.Referenced)XIO(Gripped)XIO(Axis1:I.M
oving)XIC(Axis1:I.Motion_Complete)GRT(Axis1:I.Actual_Pos,200)ONS(Park_ONS)
[MOV(1,Axis1:O.Record_No),OTL(Axis1:O.Start)];
```

HOW THIS RUNG WORKS

In words. When `Run` is on, `Axis1:I.Op_Enabled` is on, `Axis1:I.Referenced` is on, `Gripped` is off, `Axis1:I.Moving` is off, `Axis1:I.Motion_Complete` is on, `Axis1:I.Actual_Pos` is greater than **200** and it is the first scan they are true (the one-shot, remembered in `Park_ONS`), the rung puts **1** into `Axis1:O.Record_No` and latches `Axis1:O.Start` on.

Left to right:

1. `XIC(Run)` *Examine If Closed* asks: true when `Run` is on.
2. `XIC(Axis1:I.Op_Enabled)` *Examine If Closed* asks: true when `Axis1:I.Op_Enabled` is on.
3. `XIC(Axis1:I.Referenced)` *Examine If Closed* asks: true when `Axis1:I.Referenced` is on.
4. `XIO(Gripped)` *Examine If Open* asks: true when `Gripped` is off.
5. `XIO(Axis1:I.Moving)` *Examine If Open* asks: true when `Axis1:I.Moving` is off.
6. `XIC(Axis1:I.Motion_Complete)` *Examine If Closed* asks: true when `Axis1:I.Motion_Complete` is on.
7. `GRT(Axis1:I.Actual_Pos,200)` *Greater Than* asks: true when `Axis1:I.Actual_Pos` is greater than **200**.
8. `ONS(Park_ONS)` *One Shot* asks: true when it is the first scan they are true (the one-shot, remembered in `Park_ONS`).
9. A branch of 2 parallel paths: each path is tested on its own and does its own work.

   Path 1

   1. `MOV(1,Axis1:O.Record_No)` *Move* does: puts **1** into `Axis1:O.Record_No`.

   Path 2

   1. `OTL(Axis1:O.Start)` *Output Latch* does: latches `Axis1:O.Start` on.

Why it is built this way:

One-shot: once per event, not once per scan.

A latch: it stays on until an OTU clears it.

Two questions this rung raised, worth answering here. **What unlatches Record_No?** Nothing, and nothing needs to: it is not a bit, it is a number the drive *samples once*, on the rising edge of Start. The value left in it afterwards means nothing until the next Start, and the next Start's own MOV writes what that move wants.

The latch in this rung is `Start`, and the Start_Ack / Fault rung drops it. **Does the ONS fire only once, ever?** Once per *rising edge of the rung condition*. The conditions go true when the carriage is stranded, one pulse, the move starts, Moving makes the rung false, the ONS re-arms; at the pick point GRT is false and nothing fires; the next time something strands the carriage the rung rises again and it fires again. Without the ONS the rung would stay true for the several scans before the drive reports Moving, re-latch Start each scan after Start_Ack dropped it, and the drive would log a double start.

By hand, meanwhile: Toggle on `Axis1:O.Jog_Neg` (Tags tab, pick the tag, Toggle on the toolbar) jogs the carriage back at the jog speed and stops it at the stroke end; toggle it off there. Jog works for this because no rung writes it. Toggling `Home` does not: the auto-home rung is an OTE that writes Home false every scan, so the drive never sees the edge. A tag a rung writes is the rung's, not yours.

The in-position rung is true everywhere

A LIM with its High Limit still 0: Low 149500, High 0. With High below Low, LIM tests *outside* the band, so the rung was true at every position except 0 to 1495 mm, nearly the opposite of the window wanted. Fill both limits before you trust a LIM, and read the rung's power flow against the position display once: it should be true in exactly one place.

The record rung asks for a different box every time

A three-position selector gives the controller two inputs, Left and Right; the middle position is both of them off, which is why the middle branch is two XIOs. The RECORD rung is one rung with three branches, one per position, each a MOV into Record_No; built one branch at a time it is one of three, and the guide's card used to name only the first missing box, so it seemed to want something different each time a branch changed. It now lists everything missing with a count.

The handshake rung that stuck

Covered under the guided track: a Start the drive refuses gets a fault, not a Start_Ack, so an unlatch rung that waits only for Start_Ack leaves Start high, and the first Go after Reset is no edge. The fault branch in `[XIC(Axis1:I.Start_Ack),XIC(Axis1:I.Fault)]OTU(Axis1:O.Start)` is the fix, and a latch needs a way out for every way in.

Strict or inclusive

`LES(Axis1:I.Actual_Pos,200)` against the reference's `LEQ`: on a measured word the two differ by one count of one hundredth of a millimetre, and the guide accepts either. On a counter or a step word it does not, because `GRT(Pick_Count,16)` and `GEQ(Pick_Count,16)` are different programs, and that difference is exactly the kind of bug a compare hides.

WATCH FOR: THE BIT THAT IS NOT WHERE IT SHOULD BE

Keep the healthy table in your head and read the status bytes against it. Op_Enabled 0 with Enabled 1: Stop or Halt is low, look at your Power rung's conditions. Load_Voltage 0: the motor supply is off, the safety circuit or a contactor, not the program. FCT_Control 1: somebody has the configuration tool connected. Referenced 0 with Motion_Complete 1: not homed; the next Start will fault. Warning 1: a Start was ignored, usually a double start, look for a missing Moving or Motion_Complete contact. Fault 1: open the axis in the Device Monitor, the log names the fault, Reset clears it on the rising edge and nothing else does.

Direct mode, jog and teach

Records are the production way: the positions live in the drive and the PLC picks a number. FHPP also has **Direct mode**: set `Axis1:O.OPM1` to 1, write the target into `Setpoint_Pos` (0.01 mm) and the speed into `Setpoint_Vel` (percent), and Start runs to it; `Record_No` then carries the control byte, where bit 0 makes the target relative to where the axis is. **Jog_Pos** and **Jog_Neg** move while held, at the jog speed, and stop at the ends of the stroke. **Teach** writes the actual position into the selected record, Teach_Ack answers, and the record list in the part's properties changes: that is how positions are set up on a real machine, by jogging to the spot and teaching it.

TRY IT: JOG AND TEACH

Add two push buttons, `Jog_P` and `Jog_N`, and a `Teach_PB`; add rungs `XIC(Jog_P_Pressed)OTE(Axis1:O.Jog_Pos);`, `XIC(Jog_N_Pressed)OTE(Axis1:O.Jog_Neg);` and `XIC(Teach_PB_Pressed)OTE(Axis1:O.Teach);`. Jog to 1500 mm with the selector in the middle, press Teach, and look at the axis properties: record 2 now reads 1500@80. Go runs there from now on.

Level 10 puts a vacuum gripper on this carriage (the part's *Gripper* option): pick from a belt, set down at one of three bins by record, and the score watches for moves without a reference, double starts and the wrong bin.

Guided: servo axis

From Version 127 the handshake above is also a guided track, the run-up to Level 10: **Guided → New Guided Project: Servo Axis**. The scene is a small pick cell, the axis with its gripper standing over the end of a short belt, a hopper, the pick eye, a bin at 1500 mm, and a panel with the power selector, Hold, Home, Go, Reset, a record selector, Grip, the fault lamp, two in-position lamps and the position display. The guide

panel takes it one routine at a time and checks each lesson while you run it, the way the bottle-line track does: the rungs you write are verified against the reference by their logic, and proven the first time they go true in Run.

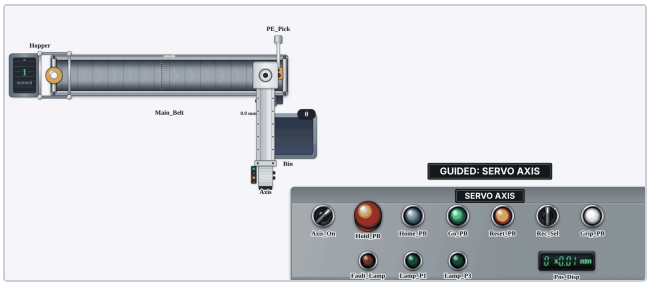


Figure 201. The pick cell: the belt brings a bottle to the pick eye under the carriage at 0 mm; the bin is at 1500 mm; the panel is the sandbox servo panel plus Grip and two in-position lamps.

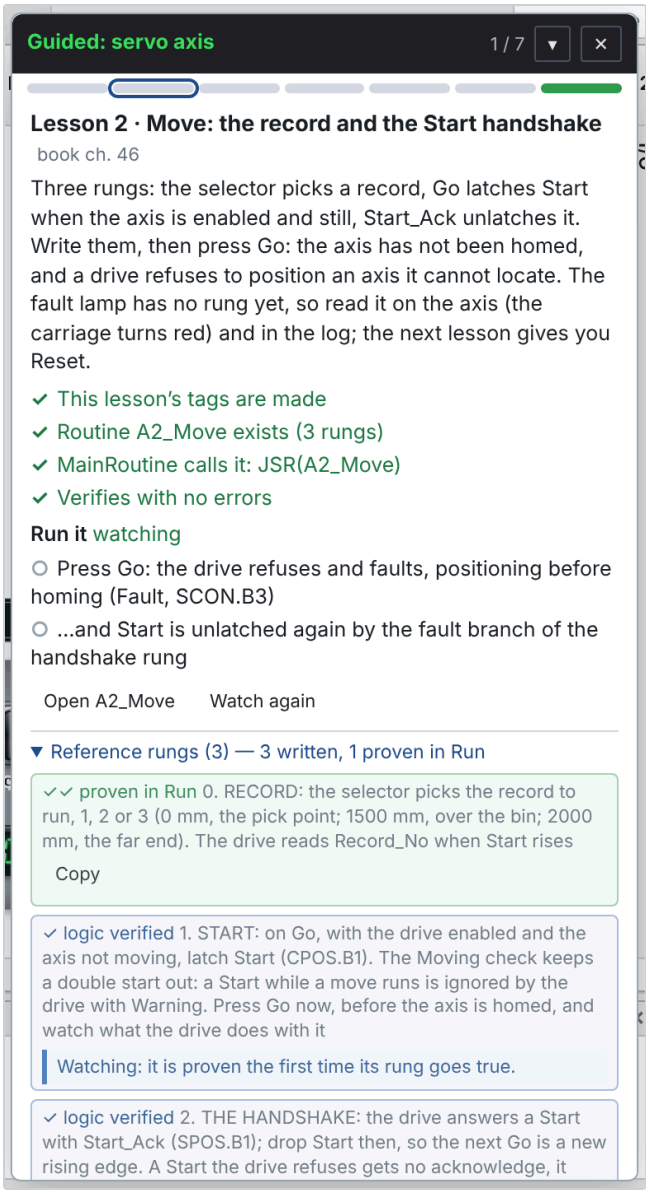


Figure 202. The guide panel at lesson 2: the three rungs of the move, the run checks waiting for the fault a Go before homing causes, the reference rungs folded below.

Lesson	Routine	What you write	What the guide watches for
1 Power	`A1_Power`	Enable, Stop and Halt from Axis_On; Hold (NC, maintained) drops Halt	Op_Enabled comes on; Halt drops while Hold is in

Lesson	Routine	What you write	What the guide watches for
2 Move	`A2_Move`	The record from the selector; Go latches Start when enabled and still; Start_Ack *or a fault* unlatches it	A Go before homing faults the drive, and the fault drops the latch
3 Reset	`A3_Reset`	Reset from the button; the fault lamp from SCON.B3	The lamp lights on the fault; Reset clears it and Op_Enabled returns
4 Home	`A4_Home`	Home when enabled and still	Referenced; then record 3 runs to 2000 mm and record 1 back
5 Position	`A5_Position`	LIM windows on Actual_Pos with Motion_Complete, one lamp per end	Each lamp seen lit: in position and stopped
6 Pick	`A6_Pick`	Belt to the eye and not while holding; Grip with a part below; release over the bin in position	A bottle waits on the eye, is gripped, and lands in the bin
7 Ready	—	—	Level 10 is this with three bins and the bottle's size choosing the record

WATCH FOR: A LATCH THAT NEVER DROPS

The first version of the handshake rung dropped Start only on Start_Ack. A Start the drive refuses, because the axis is not referenced, gets no acknowledge; it gets a fault. Start then sat high, and after Reset and Home the first Go was no rising edge: the drive saw Start already on and did nothing, and the cell looked dead until somebody pressed Go a second time. The fault branch in the unlatch rung is the fix, and it is in the sandbox program and Level 10's reference as well now. A latch needs a way out for every way in.

Two things the sim does when you add rungs from the guide: it restarts the machine, so the drive comes up clean and unreferenced (Home again), and it puts the selector back. Both are what a download does to a real cell.

WATCH FOR: A RUNG TYPED WHILE ONLINE

If you type a lesson's rung with the controller in Run, it is an *online edit*: a pending (i) rung that is in the controller but not running until it is accepted and assembled, exactly as in Logix Designer. The card will tell you ('1 online edit in A4_Home not running yet') and its button assembles it; or go to Program and back to Run, which is a download. The symptom, if you miss it, is a rung that verifies and does nothing: a Home that never fires, and every Go faulting with positioning before homing. Rungs the guide adds for you go straight in.

The vacuum cup at work

From Version 136 the gripper shows what its vacuum is doing. With Grip on, the vacuum builds in about a fifth of a second: the cup's bellows squash and the ring turns blue; while there is nothing under it, suction rings run inward, which on a real cell is the hiss of a cup pulling air. A held part shows the cup's seal on its top. When Grip drops the cup gives a short blow-off puff and the part is set down. With the vacuum fault injected the cup stays grey however long Grip is on: that is the picture of *Grip 1, Gripped 0*.

Machine faults on the servo axis

Tools → Inject Machine Fault… lists the faults that belong to the parts on this floor. Each has a fault card: the symptom, what to watch in the I/O to confirm it, the repair, and what a program should do about it. **Repair** on an active fault removes its cause the way a technician would; whether the machine runs again is up to the program. The axis also carries the belts' and the eyes' faults when the cell has them.

Fault	Watch	Repair	The program should
Servo axis: load voltage lost The safety contactor in front of the drive opens: the logic supply stays, the power stage loses its load voltage. A move in progress faults.	Load_Voltage 0, Op_Enabled 0, with Enable 1.	Close the safety contactor (Repair).	Op_Enabled lost while Axis_On is on stops the cycle and lights the fault lamp; after Repair, Reset and Home again.
Servo axis: following error The mechanics are blocked: every move faults a moment after it starts.	Fault 1 a moment into every move; the log says following error.	Free the mechanics (Repair), then Reset.	Fault stops the cycle and lights the fault lamp; Reset pulses the drive's Reset; the axis is homed again before the next pick.
Servo axis: vacuum lost No vacuum at the cup: Grip does nothing and a held part drops.	Grip 1, Part_Below 1, Gripped 0.	Clear the vacuum line (Repair).	Grip on and no Gripped within half a second: release, retry once, then alarm.

Predict it: the answers

1. The rung is false at `XIC(Axis_On_Pos2)`: `Axis_On_Pos2` is off, so `Axis1:O.Enable` is **off** (an OTE writes 0 when its rung is false), `Axis1:O.Stop` is **off** (an OTE writes 0 when its rung is false) and `Axis1:O.Halt` is **off** (an OTE writes 0 when its rung is false).
2. Every condition on the way is met, so `Axis1:O.Home` is **on**.
3. Every condition on the way is met, so `Axis1:O.Record_No` becomes **3**.
4. Every condition on the way is met, so `Axis1:O.Start` is unlatched, **off**.
5. The rung is false at `XIO(Axis1:I.Fault)`: `Axis1:I.Fault` is on, so `Axis1:O.Home` is **off** (an OTE writes 0 when its rung is false).

47 Classic scenes: the elevator

Every PLC simulator worth the name has an elevator, because an elevator is the smallest machine that has all the hard parts of a big one: requests that arrive while you are busy, a direction to choose and stick to, a door that must never be open while you move, people who stand in it, and a safety chain that has the final say over the program. The classic student exercise for it, the four-floor elevator with a shaft encoder and a main routine that is nothing but an index of subroutine calls, is the shape this chapter follows. From Version 125 Logix Connect has the machine for it, under **File → New Project → Classic: elevator**: a four-floor panoramic lift with hall calls, cab buttons, a shaft encoder, a door with its end switches and photo eye, and passengers who press the buttons by themselves. The scene comes with its tags made and an empty routine: this is one to write yourself. The program below is the reference (**File → Load Reference Solution: Elevator**), built in the order the exercise asks for it.

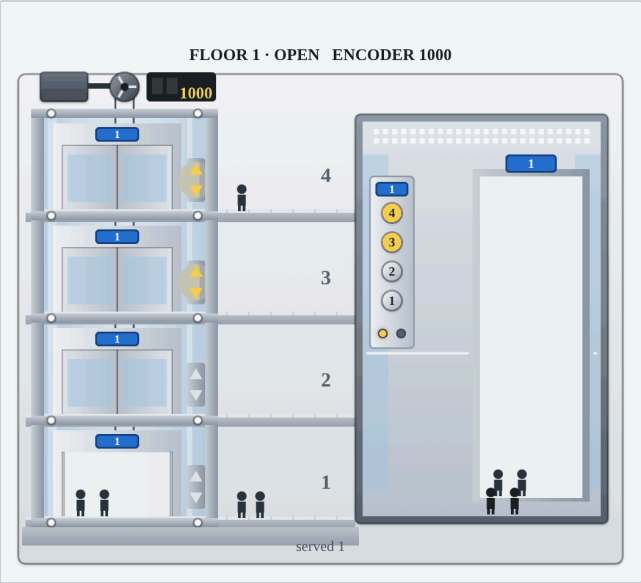


Figure 203. The Elevator part, close. Left, the glazed steel shaft from the landings: on every floor a stainless portal with its two doors, the blue display over them (the floor the car is at, with an arrow while it moves), the call plate on the jamb that lights when a call is registered, people waiting on the slab. The cab is a glass box on its ropes; the hoist, its sheave and the encoder count sit over the shaft. Right, the inside of the cab: the lit ceiling, the car door with its own display, the handrails, the operating panel with the floor buttons, the riders.

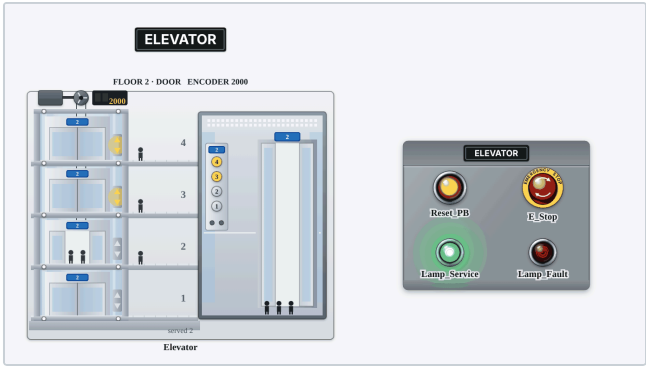


Figure 204. The scene a quarter of a minute in: the car at floor 2, doors parting, requests registered for 3 and 4, somebody waiting on every floor. The console has Reset, the E-stop and the service and fault lamps.

The machine

The *Elevator* part is under a new palette category, **Classic scenes**. Its properties are the building: the number of floors (two to six), the travel time floor to floor, the door travel time, whether passengers call by themselves and how often, and whether the safety switches (eye and overload) are wired. Its I/O is what an elevator controller sees:

Signal		Meaning
`Call_1`	input	Hall call button on floor 1 (1 while pressed)
`Call_2`	input	Hall call button on floor 2 (1 while pressed)
`Call_3`	input	Hall call button on floor 3 (1 while pressed)
`Call_4`	input	Hall call button on floor 4 (1 while pressed)
`Cab_1`	input	Cab button for floor 1 (1 while pressed)
`Cab_2`	input	Cab button for floor 2 (1 while pressed)
`Cab_3`	input	Cab button for floor 3 (1 while pressed)
`Cab_4`	input	Cab button for floor 4 (1 while pressed)
`At_1`	input	Floor switch: the cab is level with floor 1
`At_2`	input	Floor switch: the cab is level with floor 2
`At_3`	input	Floor switch: the cab is level with floor 3
`At_4`	input	Floor switch: the cab is level with floor 4
`Encoder`	analog in	Shaft encoder on the hoist: the cab position in counts, 1000 per floor (floor 1 = 1000, floor 2 = 2000 …)
`Door_Open`	input	Door fully open (end switch)
`Door_Closed`	input	Door fully closed (end switch)
`Door_Eye`	input	Door photo eye: blocked while someone is in the doorway
`Overload`	input	Cab overload switch (too many passengers)
`Up`	output	Hoist motor up
`Down`	output	Hoist motor down (never with Up: the contactors are interlocked)
`Open`	output	Door motor: open
`Close`	output	Door motor: close
`Lamp_1`	output	Built-in lamp of the floor 1 hall switch and cab button: a call is registered
`Lamp_2`	output	Built-in lamp of the floor 2 hall switch and cab button: a call is registered
`Lamp_3`	output	Built-in lamp of the floor 3 hall switch and cab button: a call is registered
`Lamp_4`	output	Built-in lamp of the floor 4 hall switch and cab button: a call is registered

Signal		Meaning
`Ind_1`	output	Floor indicator 1: the lamp above every door and in the cab that says the cab is at floor 1
`Ind_2`	output	Floor indicator 2: the lamp above every door and in the cab that says the cab is at floor 2
`Ind_3`	output	Floor indicator 3: the lamp above every door and in the cab that says the cab is at floor 3
`Ind_4`	output	Floor indicator 4: the lamp above every door and in the cab that says the cab is at floor 4
`Buzzer`	output	Alarm buzzer

Four things about it are physical, not programmable. **Up and Down together** do nothing: the contactors are mechanically interlocked, and the log says so. **Moving with the door open** moves, but the log shouts and the part counts it as unsafe: on a real elevator the door contact is in the safety chain and the hoist cannot run; the part leaves that to your program so you can see what it is for. **Levelling**: when the hoist drops inside the door zone the drive creeps the car level, as a real one does, so a stop a few counts early still lines up with the sill. And the **door eye**: a passenger boarding or leaving stands in the doorway for a second and a half, and a Close commanded then does not close (the log notes it).

Two ways to know where the car is. `At_1` to `At_4` are floor switches, made only while the car is level. `Encoder` is the shaft encoder on the hoist, an analog word that reads 1000 counts a floor: 1000 at floor 1, 4000 at floor 4, 2500 half way between 2 and 3. The classic exercise gives you only the encoder and has you find the floor values by trial; here you have both, and the reference uses each where it is best.

Passengers: on their own every nine seconds or so (*Seconds between passengers*), or when you click a call plate, somebody appears on a floor and presses the call button once, a 0.3 s press. When the car stands at their floor with the door fully open they board, one at a time through the eye, and press a cab button for where they want to go; at that floor, door open, they leave. A passenger who has waited 45 s takes the stairs and is counted as lost. The cab holds four; a fifth makes `Overload`. The displays over every door and in the car show whichever `Ind_N` you turn on; the call plates and cab buttons light with `Lamp_N`.

The elevator has three views like every scene, and it opens in the side view, the elevation above, because that is where the machine is easiest to read. The top view is a sheet of floor plans, one per floor, the way an architect draws a lift core: the lobby, the hoistway in its concrete walls with the guide rails and the counterweight at the back, the landing doors, the floor indicator, the hall call station with its lamp, and the people waiting. The car is drawn on the floor it is at, fading as it leaves, with its doors, the eye across them, the car operating panel and its riders. The hall buttons and the car's panel work in the plan as they do in the elevation.

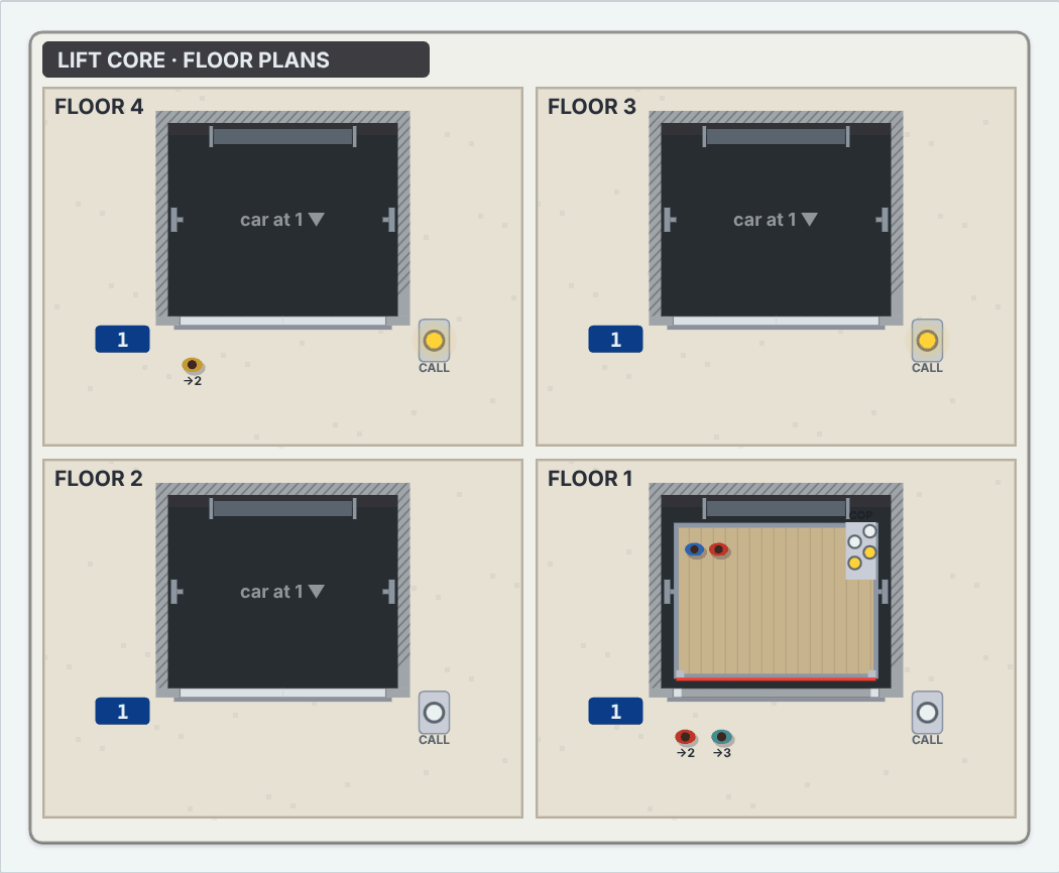


Figure 205. The top view: the floor plans of the lift core. The car at floor 1 with its doors open and someone in the doorway (the eye's beam red across them), two riders, its operating panel lit for the floors asked for; the call lamps lit on floors 4 and 3; people waiting on floors 4 and 1, each with the floor they are going to.

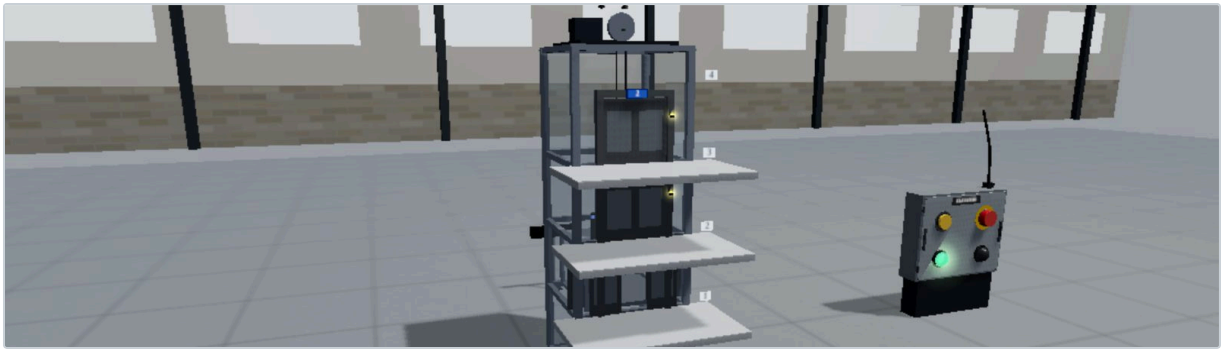


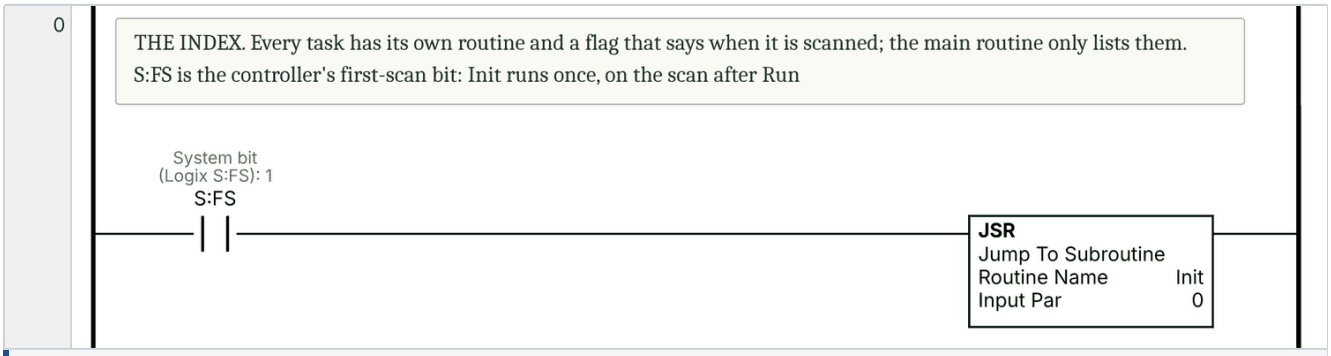
Figure 206. In 3D: the glazed shaft on its steel frame, a tiled landing with its portal, display and call plate at every floor, the glass cab on its ropes under the machine.

The program's index

The exercise starts with structure, not logic: a main routine that only lists the tasks, each a subroutine with a flag that says when it is scanned. The flags pass control round a loop: `DoNext_Or_Wait` (at a floor, door open, deciding) → `Close_And_Go` (a floor chosen) → `Car_Moving` (the hoist runs) → `Stop_And_Open`

(arrived) → back to `DoNext_Or_Wait`. Two routines are scanned every time: catching requests, and the safety chain. Read the main routine like a table of contents:

MAINROUTINE · RUNG 0




```
XIC(S:FS)JSR(Init);
```

THE INDEX. Every task has its own routine and a flag that says when it is scanned; the main routine only lists them. S:FS is the controller's first-scan bit: Init runs once, on the scan after Run

HOW THIS RUNG WORKS

In words. When `S:FS` is on, the rung runs the routine `Init`.

Left to right:

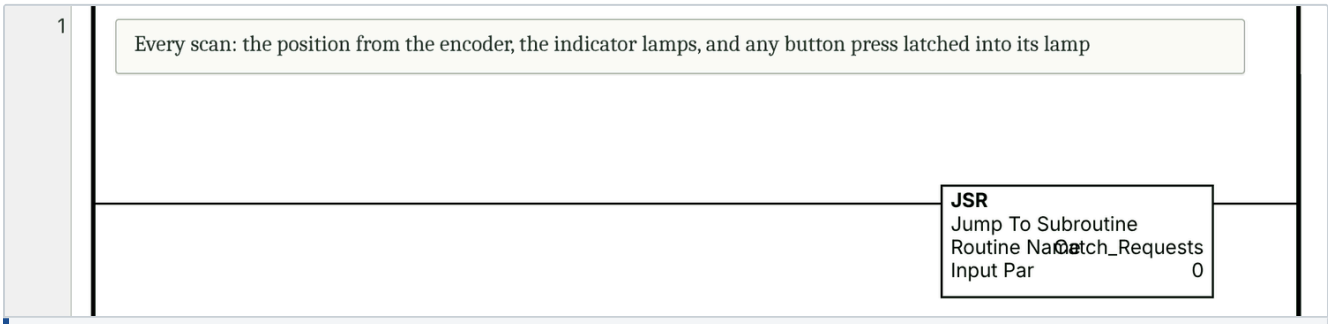
1. `XIC(S:FS)` *Examine If Closed* asks: true when `S:FS` is on.
2. `JSR(Init)` *Jump to Subroutine* does: runs the routine `Init`. The routine is scanned at this point, then the scan comes back here.

Why it is built this way:

Part of the index. The main routine is a list of calls; each routine holds one job, and the scan goes through them in this order.

> **Think of it like this:** the table of contents of a book. The main routine does no work itself: it lists the chapters and the order to read them in, every scan.

MAINROUTINE · RUNG 1




```
JSR(Catch_Requests);
```

Every scan: the position from the encoder, the indicator lamps, and any button press latched into its lamp

HOW THIS RUNG WORKS

In words. Every scan, with nothing to test, the rung runs the routine `Catch_Requests`.

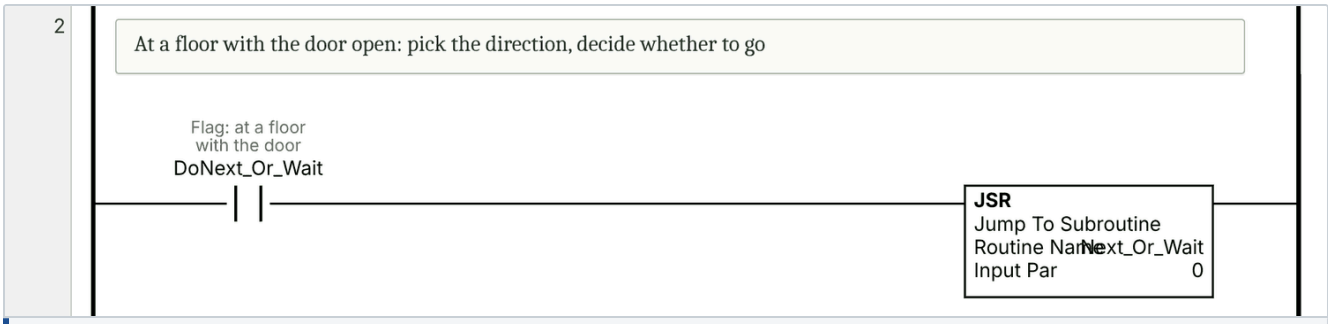
Left to right:

1. `JSR(Catch_Requests)` *Jump to Subroutine* does: runs the routine `Catch_Requests`.

Why it is built this way:

Part of the index: one call, one job.

MAINROUTINE · RUNG 2




```
XIC(DoNext_Or_Wait)JSR(Next_Or_Wait);
```

At a floor with the door open: pick the direction, decide whether to go

HOW THIS RUNG WORKS

In words. When `DoNext_Or_Wait` (Flag: at a floor with the door open; Next_Or_Wait is scanned and decides where to go next) is on, the rung runs the routine `Next_Or_Wait`.

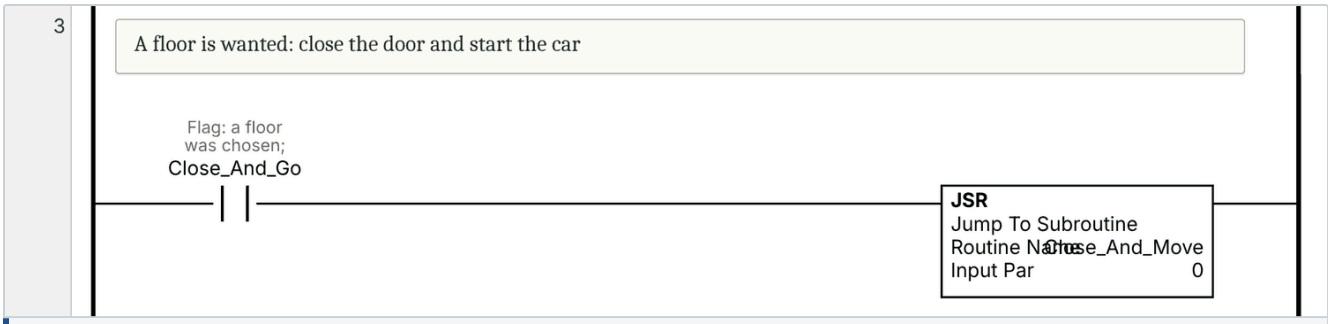
Left to right:

1. `XIC(DoNext_Or_Wait)` *Examine If Closed* asks: true when `DoNext_Or_Wait` (Flag: at a floor with the door open; Next_Or_Wait is scanned and decides where to go next) is on.
2. `JSR(Next_Or_Wait)` *Jump to Subroutine* does: runs the routine `Next_Or_Wait`.

Why it is built this way:

Part of the index: one call, one job.

MAINROUTINE · RUNG 3




```
XIC(Close_And_Go)JSR(Close_And_Move);
```

A floor is wanted: close the door and start the car

HOW THIS RUNG WORKS

In words. When `Close_And_Go` (Flag: a floor was chosen; Close_And_Move closes the door and hands over to Track_Car) is on, the rung runs the routine `Close_And_Move`.

Left to right:

1. `XIC(Close_And_Go)` *Examine If Closed* asks: true when `Close_And_Go` (Flag: a floor was chosen; Close_And_Move closes the door and hands over to Track_Car) is on.
2. `JSR(Close_And_Move)` *Jump to Subroutine* does: runs the routine `Close_And_Move`.

Why it is built this way:

Part of the index: one call, one job.

MAINROUTINE · RUNG 4

```
XIC(Car_Moving)JSR(Track_Car);
```

Moving: run the hoist, stop at the next wanted floor

HOW THIS RUNG WORKS

In words. When `Car_Moving` (Flag: the hoist runs; Track_Car watches for the floor to stop at) is on, the rung runs the routine `Track_Car`.

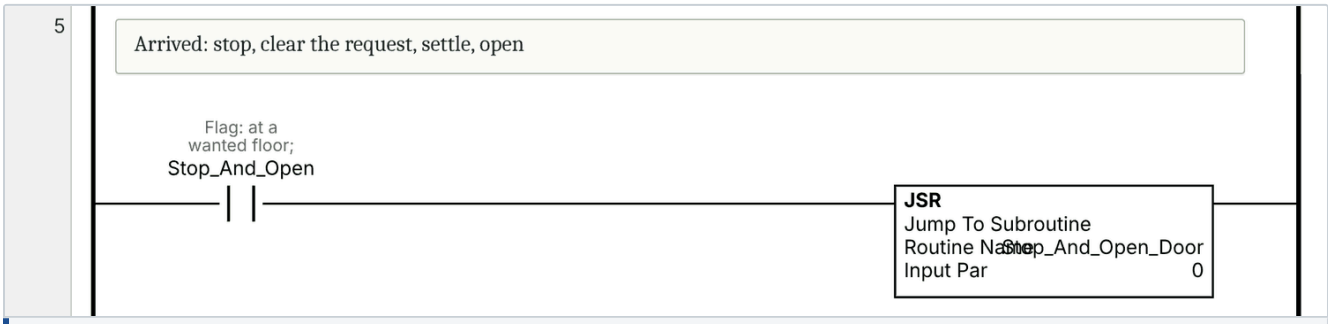
Left to right:

1. `XIC(Car_Moving)` *Examine If Closed* asks: true when `Car_Moving` (Flag: the hoist runs; Track_Car watches for the floor to stop at) is on.
2. `JSR(Track_Car)` *Jump to Subroutine* does: runs the routine `Track_Car`.

Why it is built this way:

Part of the index: one call, one job.

MAINROUTINE · RUNG 5




```
XIC(Stop_And_Open)JSR(Stop_And_Open_Door);
```

Arrived: stop, clear the request, settle, open

HOW THIS RUNG WORKS

In words. When `Stop_And_Open` (Flag: at a wanted floor; Stop_And_Open_Door stops the hoist, settles and opens) is on, the rung runs the routine `Stop_And_Open_Door`.

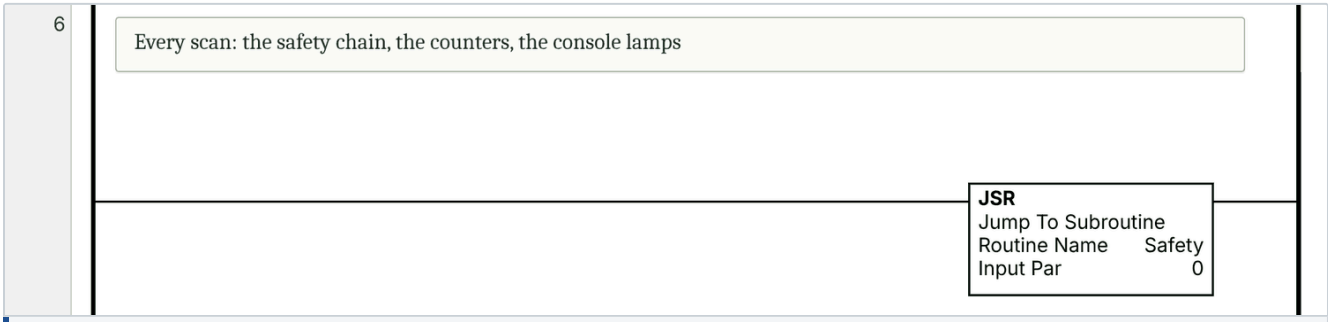
Left to right:

1. `XIC(Stop_And_Open)` *Examine If Closed* asks: true when `Stop_And_Open` (Flag: at a wanted floor; Stop_And_Open_Door stops the hoist, settles and opens) is on.
2. `JSR(Stop_And_Open_Door)` *Jump to Subroutine* does: runs the routine `Stop_And_Open_Door`.

Why it is built this way:

Part of the index: one call, one job.

MAINROUTINE · RUNG 6




```
JSR(Safety);
```

Every scan: the safety chain, the counters, the console lamps

HOW THIS RUNG WORKS

In words. Every scan, with nothing to test, the rung runs the routine `Safety`.

Left to right:

1. `JSR(Safety)` *Jump to Subroutine* does: runs the routine `Safety`.

Why it is built this way:

Part of the index: one call, one job.

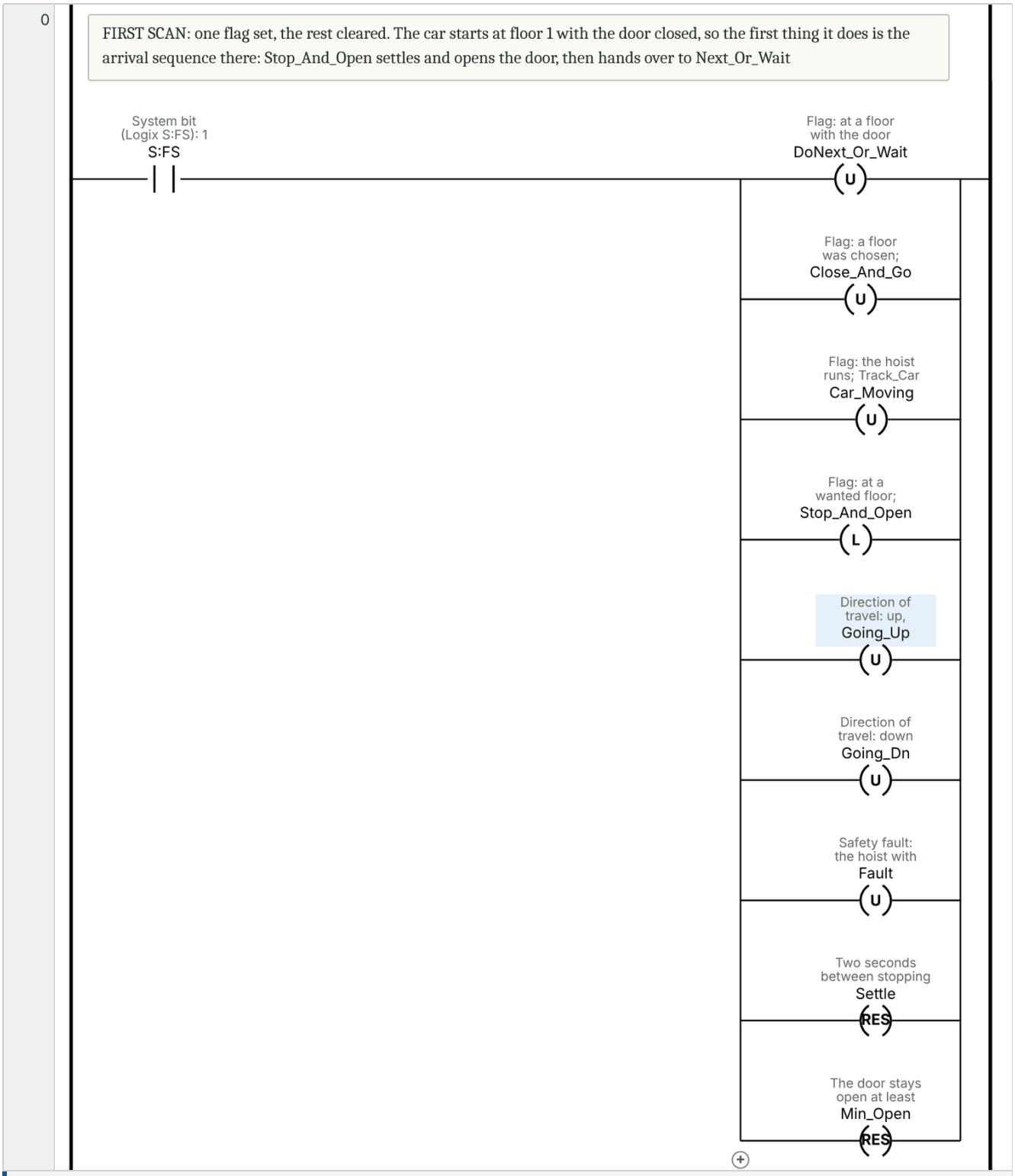
`S:FS` is the controller's first-scan bit, 1 for the first scan after a download or Program → Run and 0 after; Logix Connect has it from Version 125. These are the tags the program adds to the scene's aliases:

Tag	Type	Description
`Served_Count`	DINT	Shown on the toolbar: door cycles
`Trip_Count`	DINT	Shown on the toolbar: trips
`DoNext_Or_Wait`	BOOL	Flag: at a floor with the door open; Next_Or_Wait is scanned and decides where to go next
`Close_And_Go`	BOOL	Flag: a floor was chosen; Close_And_Move closes the door and hands over to Track_Car
`Car_Moving`	BOOL	Flag: the hoist runs; Track_Car watches for the floor to stop at
`Stop_And_Open`	BOOL	Flag: at a wanted floor; Stop_And_Open_Door stops the hoist, settles and opens
`Going_Up`	BOOL	Direction of travel: up, latched while anything is wanted above
`Going_Dn`	BOOL	Direction of travel: down
`Req_Above`	BOOL	A lit request lamp above the car
`Req_Below`	BOOL	A lit request lamp below the car
`Floor`	DINT	The floor the car is at or nearest, from the encoder (1000 counts per floor)
`Settle`	TIMER	Two seconds between stopping and opening the door
`Min_Open`	TIMER	The door stays open at least five seconds before the next request is processed
`Trip_ONS`	BOOL	One-shot storage for the trip counter
`Fault`	BOOL	Safety fault: the hoist with the door not closed, or an overload; cleared by Reset

Exercise 1: initialise

Everything latched must start somewhere known. The car begins at floor 1 with its door closed, so the first thing the program does is treat that as an arrival: `Stop_And_Open` is the one flag set, and the arrival routine settles, opens the door and hands over to the decision routine.

INIT · RUNG 0




```
XIC(S:FS)
[OTU(DoNext_Or_Wait),OTU(Close_And_Go),OTU(Car_Moving),OTL(Stop_And_Open),OTU(G
oing_Up),OTU(Going_Dn),OTU(Fault),RES(Settle),RES(Min_Open)];
```

FIRST SCAN: one flag set, the rest cleared. The car starts at floor 1 with the door closed, so the first thing it does is the arrival sequence there: Stop_And_Open settles and opens the door, then hands over to Next_Or_Wait

HOW THIS RUNG WORKS

In words. When `S:FS` is on, the rung unlatches `DoNext_Or_Wait` (Flag: at a floor with the door open; Next_Or_Wait is scanned and decides where to go next) (turns it off), unlatches `Close_And_Go` (Flag: a floor was chosen; Close_And_Move closes the door and hands over to Track_Car) (turns it off), unlatches `Car_Moving` (Flag: the hoist runs; Track_Car watches for the floor to stop at) (turns it off), latches `Stop_And_Open` (Flag: at a wanted floor; Stop_And_Open_Door stops the hoist, settles and opens) on, unlatches `Going_Up` (Direction of travel: up, latched while anything is wanted above) (turns it off), unlatches `Going_Dn` (Direction of travel: down) (turns it off), unlatches `Fault` (Safety fault: the hoist with the door not closed, or an overload; cleared by Reset) (turns it off), resets `Settle` (Two seconds between stopping and opening the door) and resets `Min_Open` (The door stays open at least five seconds before the next request is processed).

Left to right:

1. `XIC(S:FS)` *Examine If Closed* asks: true when `S:FS` is on.
2. A branch of 9 parallel paths: each path is tested on its own and does its own work.

Path 1

1. `OTU(DoNext_Or_Wait)` *Output Unlatch* does: unlatches `DoNext_Or_Wait` (Flag: at a floor with the door open; Next_Or_Wait is scanned and decides where to go next) (turns it off). It only ever clears the bit.

Path 2

1. `OTU(Close_And_Go)` *Output Unlatch* does: unlatches `Close_And_Go` (Flag: a floor was chosen; Close_And_Move closes the door and hands over to Track_Car) (turns it off).

Path 3

1. `OTU(Car_Moving)` *Output Unlatch* does: unlatches `Car_Moving` (Flag: the hoist runs; Track_Car watches for the floor to stop at) (turns it off).

Path 4

1. `OTL(Stop_And_Open)` *Output Latch* does: latches `Stop_And_Open` (Flag: at a wanted floor; Stop_And_Open_Door stops the hoist, settles and opens) on. It only ever sets the bit; the bit stays on after the rung goes false.

Path 5

1. `OTU(Going_Up)` *Output Unlatch* does: unlatches `Going_Up` (Direction of travel: up, latched while anything is wanted above) (turns it off).

Path 6

1. `OTU(Going_Dn)` *Output Unlatch* does: unlatches `Going_Dn` (Direction of travel: down) (turns it off).

Path 7

1. `OTU(Fault)` *Output Unlatch* does: unlatches `Fault` (Safety fault: the hoist with the door not closed, or an overload; cleared by Reset) (turns it off).

Path 8

1. `RES(Settle)` *Reset* does: resets `Settle` (Two seconds between stopping and opening the door). It zeroes a timer or counter (accumulator and status bits), or a control (its position and status bits).

Path 9

1. `RES(Min_Open)` *Reset* does: resets `Min_Open` (The door stays open at least five seconds before the next request is processed).

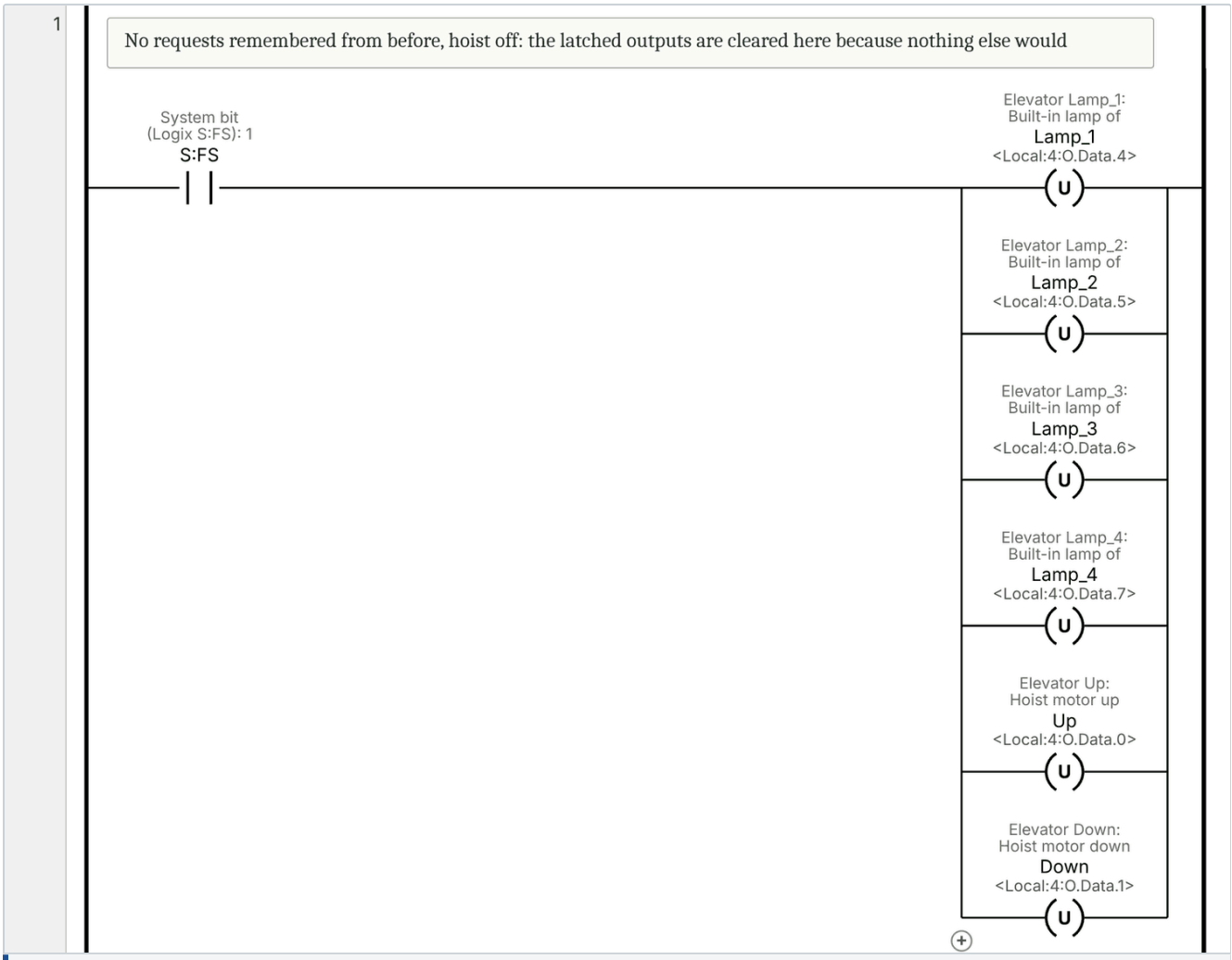
Why it is built this way:

A latch. `Stop_And_Open` stays on after this rung goes false; an `OTU` elsewhere in the program clears it.

> **Think of it like this:** a light switch instead of a doorbell. OTL flips it on and it stays on when you take your hand away; OTU flips it off. Whatever flipped it on does not have to stay there.

> **Watch for:** A latched bit survives the condition that set it: every way in needs a way out, or it stays on for good.

INIT · RUNG 1




```
XIC(S:FS)[OTU(Lamp_1),OTU(Lamp_2),OTU(Lamp_3),OTU(Lamp_4),OTU(Up),OTU(Down)];
```

No requests remembered from before, hoist off: the latched outputs are cleared here because nothing else would

HOW THIS RUNG WORKS

In words. When `S:FS` is on, the rung unlatches `Lamp_1` (Built-in lamp of the floor 1 hall switch and cab button: a call is registered) (turns it off), unlatches `Lamp_2` (Built-in lamp of the floor 2 hall switch and cab button: a call is registered) (turns it off), unlatches `Lamp_3` (Built-in lamp of the floor 3 hall switch and cab button: a call is registered) (turns it off), unlatches `Lamp_4` (Built-in lamp of the floor 4 hall switch and cab button: a call is registered) (turns it off), unlatches `Up` (Hoist motor up) (turns it off) and unlatches `Down` (Hoist motor down (never with Up: the contactors are interlocked)) (turns it off).

Left to right:

1. `XIC(S:FS)` *Examine If Closed* asks: true when `S:FS` is on.
2. A branch of 6 parallel paths: each path is tested on its own and does its own work.

   Path 1

1. `OTU(Lamp_1)` *Output Unlatch* does: unlatches `Lamp_1` (Built-in lamp of the floor 1 hall switch and cab button: a call is registered) (turns it off).

Path 2

1. `OTU(Lamp_2)` *Output Unlatch* does: unlatches `Lamp_2` (Built-in lamp of the floor 2 hall switch and cab button: a call is registered) (turns it off).

Path 3

1. `OTU(Lamp_3)` *Output Unlatch* does: unlatches `Lamp_3` (Built-in lamp of the floor 3 hall switch and cab button: a call is registered) (turns it off).

Path 4

1. `OTU(Lamp_4)` *Output Unlatch* does: unlatches `Lamp_4` (Built-in lamp of the floor 4 hall switch and cab button: a call is registered) (turns it off).

Path 5

1. `OTU(Up)` *Output Unlatch* does: unlatches `Up` (Hoist motor up) (turns it off).

Path 6

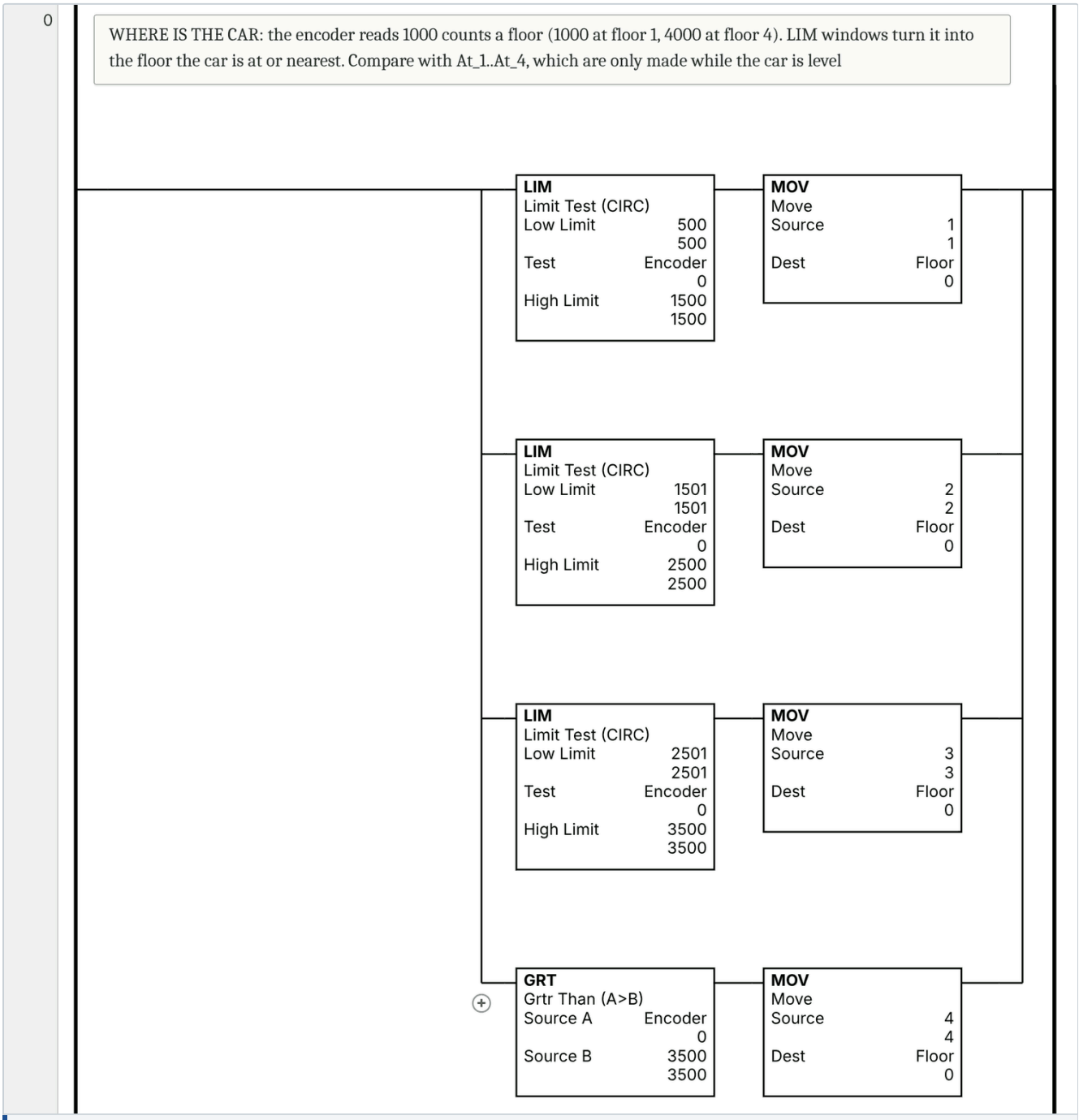
1. `OTU(Down)` *Output Unlatch* does: unlatches `Down` (Hoist motor down (never with Up: the contactors are interlocked)) (turns it off).

> **Think of it like this:** a rung is a pipe with valves in it, read left to right. Every contact is a valve: XIC is open while its bit is 1, XIO while its bit is 0. Water reaches the output at the right only if every valve in a row is open. The output does not remember anything: on every scan it is simply wet or dry.

Exercise 2: catch the requests

The exercise's trick, and the reason it is allowed to use latches: the lamp in the hall switch is the request flag. Press and it latches; the car opens at that floor and it unlatches. One output doubles as the passenger's confirmation, the program's memory of the floor, and, read against the car's position, the direction to go. The routine first turns the encoder into a floor number and lights the indicator, then latches a lamp for any button pressed for a floor the car is not already at, then sums up what is wanted above and below.

CATCH_REQUESTS · RUNG 0




```
[LIM(500,Encoder,1500)MOV(1,Floor),LIM(1501,Encoder,2500)MOV(2,Floor),LIM(2501,
Encoder,3500)MOV(3,Floor),GRT(Encoder,3500)MOV(4,Floor)];
```

WHERE IS THE CAR: the encoder reads 1000 counts a floor (1000 at floor 1, 4000 at floor 4). LIM windows turn it into the floor the car is at or nearest. Compare with At_1..At_4, which are only made while the car is level

PREDICT IT · 1 `Encoder` is 3000.5. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. Every scan, with nothing to test, the rung picks its path: if `Encoder` (Shaft encoder on the hoist: the cab position in counts, 1000 per floor (floor 1 = 1000, floor 2 = 2000 …)) is between **500** and **1500**, it puts **1** into `Floor` (The floor the car is at or nearest, from the encoder (1000 counts per floor)); if `Encoder` is between **1501** and **2500**, it puts **2** into `Floor`; if `Encoder` is between **2501** and **3500**, it puts **3** into `Floor`; if `Encoder` is greater than **3500**, it puts **4** into `Floor`.

Left to right:

1. A branch of 4 parallel paths: each path is tested on its own and does its own work.

   Path 1

   1. `LIM(500,Encoder,1500)` *Limit Test* asks: true when `Encoder` (Shaft encoder on the hoist: the cab position in counts, 1000 per floor (floor 1 = 1000, floor 2 = 2000 …)) is between **500** and **1500**. The test is inclusive at both ends.
   2. `MOV(1,Floor)` *Move* does: puts **1** into `Floor` (The floor the car is at or nearest, from the encoder (1000 counts per floor)). It writes every scan the rung is true.

   Path 2

   1. `LIM(1501,Encoder,2500)` *Limit Test* asks: true when `Encoder` is between **1501** and **2500**.
   2. `MOV(2,Floor)` *Move* does: puts **2** into `Floor`.

   Path 3

   1. `LIM(2501,Encoder,3500)` *Limit Test* asks: true when `Encoder` is between **2501** and **3500**.
   2. `MOV(3,Floor)` *Move* does: puts **3** into `Floor`.

   Path 4

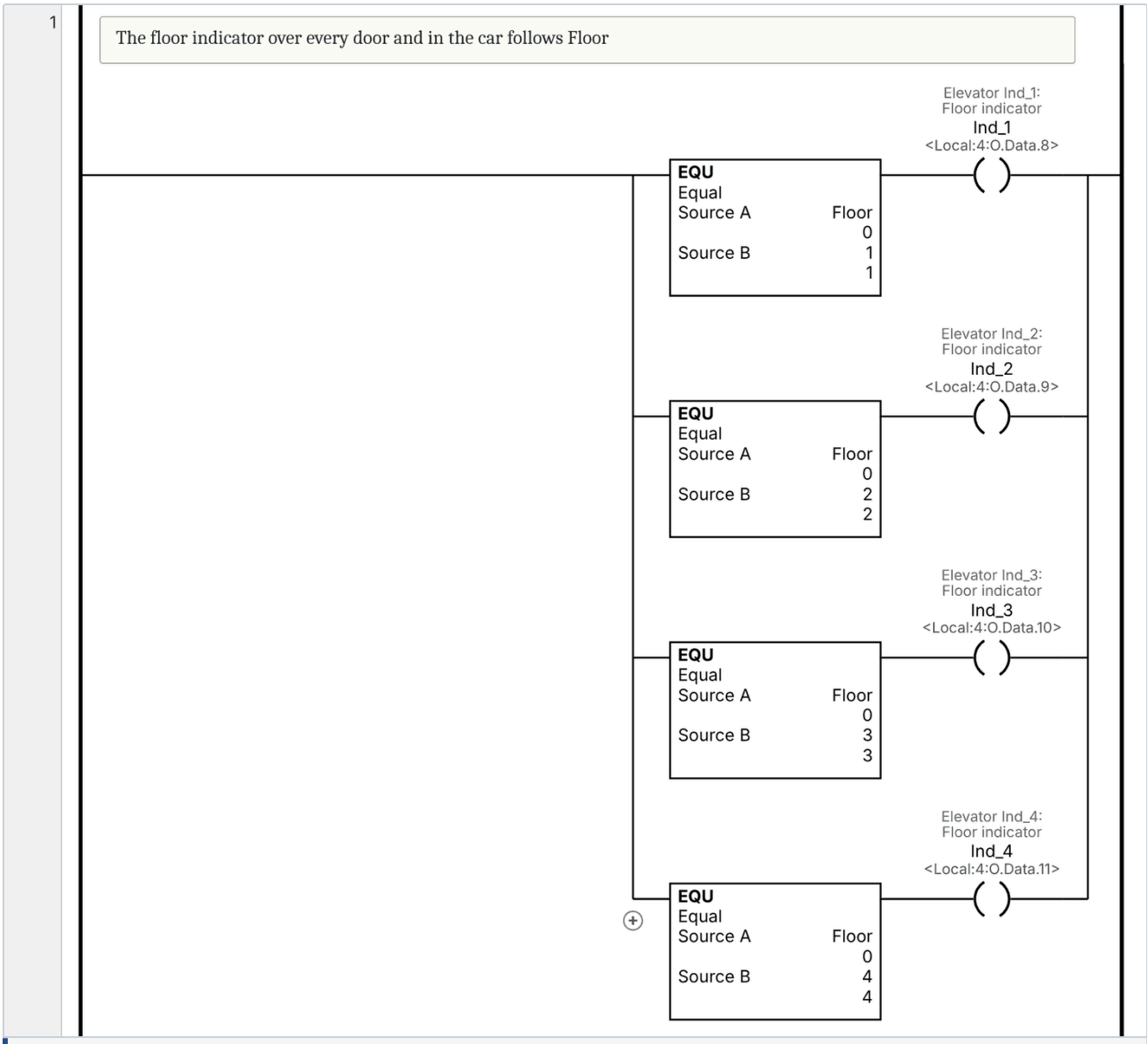
   1. `GRT(Encoder,3500)` *Greater Than* asks: true when `Encoder` is greater than **3500**.
   2. `MOV(4,Floor)` *Move* does: puts **4** into `Floor`.

Why it is built this way:

A window, not a value. A moving value can step over any single number between two scans; a window catches it on the way through.

> **Think of it like this:** meeting a train. You cannot stand at one exact inch of track and wait for the train to stop there: between two glances it rolls past. You watch the whole platform instead.

CATCH_REQUESTS · RUNG 1




```
[EQU(Floor,1)OTE(Ind_1),EQU(Floor,2)OTE(Ind_2),EQU(Floor,3)OTE(Ind_3),EQU(Floor
,4)OTE(Ind_4)];
```

The floor indicator over every door and in the car follows Floor

PREDICT IT · 2 `Floor` is 4. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. Every scan, with nothing to test, the rung picks its path: if `Floor` (The floor the car is at or nearest, from the encoder (1000 counts per floor)) equals **1**, it turns `Ind_1` (Floor indicator 1: the lamp above every door and in the cab that says the cab is at floor 1) on (and off again whenever the rung is false); if `Floor` equals **2**, it turns `Ind_2` (Floor indicator 2: the lamp above every door and in the cab that says the cab is at floor 2) on (and off again whenever the rung is

false); if `Floor` equals **3**, it turns `Ind_3` (Floor indicator 3: the lamp above every door and in the cab that says the cab is at floor 3) on (and off again whenever the rung is false); if `Floor` equals **4**, it turns `Ind_4` (Floor indicator 4: the lamp above every door and in the cab that says the cab is at floor 4) on (and off again whenever the rung is false).

Left to right:

1. A branch of 4 parallel paths: each path is tested on its own and does its own work.

   Path 1

   1. `EQU(Floor,1)` *Equal* asks: true when `Floor` (The floor the car is at or nearest, from the encoder (1000 counts per floor)) equals **1**. Both values are compared every scan.
   2. `OTE(Ind_1)` *Output Energize* does: turns `Ind_1` (Floor indicator 1: the lamp above every door and in the cab that says the cab is at floor 1) on (and off again whenever the rung is false). Every scan it writes the rung state: true on, false off.

   Path 2

   1. `EQU(Floor,2)` *Equal* asks: true when `Floor` equals **2**.
   2. `OTE(Ind_2)` *Output Energize* does: turns `Ind_2` (Floor indicator 2: the lamp above every door and in the cab that says the cab is at floor 2) on (and off again whenever the rung is false).

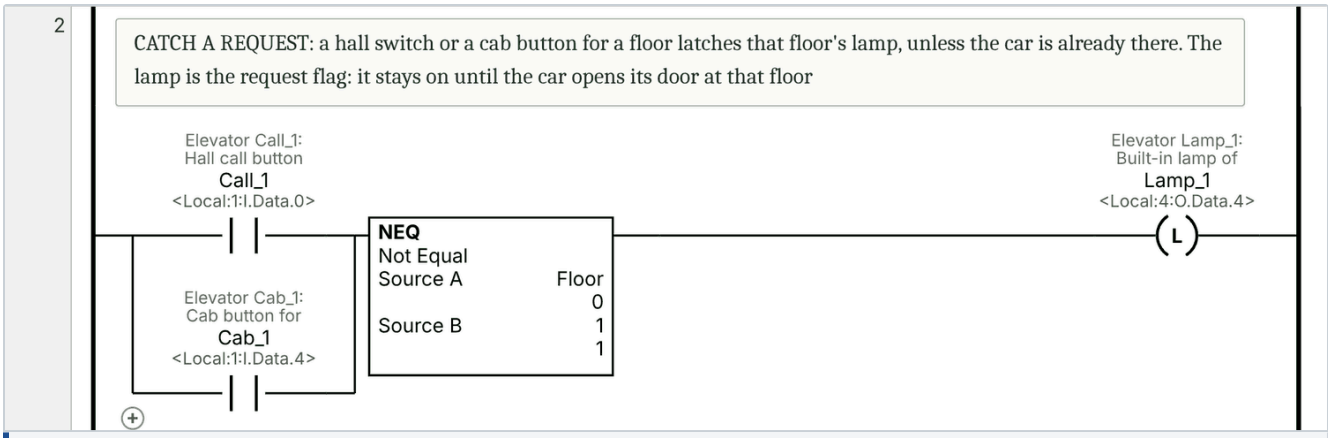
   Path 3

   1. `EQU(Floor,3)` *Equal* asks: true when `Floor` equals **3**.
   2. `OTE(Ind_3)` *Output Energize* does: turns `Ind_3` (Floor indicator 3: the lamp above every door and in the cab that says the cab is at floor 3) on (and off again whenever the rung is false).

   Path 4

   1. `EQU(Floor,4)` *Equal* asks: true when `Floor` equals **4**.
   2. `OTE(Ind_4)` *Output Energize* does: turns `Ind_4` (Floor indicator 4: the lamp above every door and in the cab that says the cab is at floor 4) on (and off again whenever the rung is false).

CATCH_REQUESTS · RUNG 2




```
[XIC(Call_1),XIC(Cab_1)]NEQ(Floor,1)OTL(Lamp_1);
```

CATCH A REQUEST: a hall switch or a cab button for a floor latches that floor's lamp, unless the car is already there. The lamp is the request flag: it stays on until the car opens its door at that floor

PREDICT IT · 3 `Call_1` is on and `Floor` is 2. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When either (`Call_1` (Hall call button on floor 1 (1 while pressed)) is on) *or* (`Cab_1` (Cab button for floor 1 (1 while pressed)) is on) and `Floor` (The floor the car is at or nearest, from the encoder (1000 counts per floor)) is not **1**, the rung latches `Lamp_1` (Built-in lamp of the floor 1 hall switch and cab button: a call is registered) on.

Left to right:

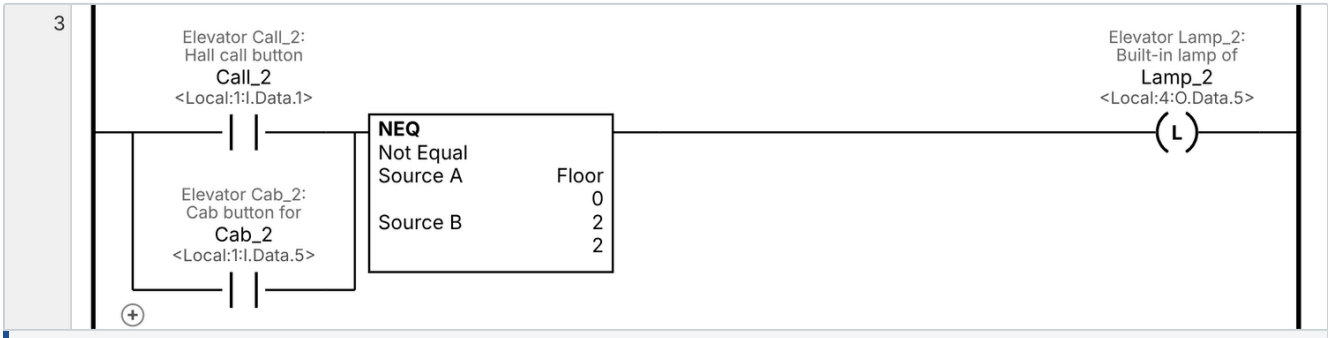
1. A branch of 2 parallel paths: power gets through if any one of them is true.
   - **Path 1**
     1. `XIC(Call_1)` *Examine If Closed* asks: true when `Call_1` (Hall call button on floor 1 (1 while pressed)) is on.
   - **Path 2**
     1. `XIC(Cab_1)` *Examine If Closed* asks: true when `Cab_1` (Cab button for floor 1 (1 while pressed)) is on.
2. `NEQ(Floor,1)` *Not Equal* asks: true when `Floor` (The floor the car is at or nearest, from the encoder (1000 counts per floor)) is not **1**.
3. `OTL(Lamp_1)` *Output Latch* does: latches `Lamp_1` (Built-in lamp of the floor 1 hall switch and cab button: a call is registered) on.

Why it is built this way:

A latch: it stays on until an OTU clears it.

> **Think of it like this:** parallel paths are two roads to the same town. If either road is open you get there; the rung only cares that power reaches the far end, not which way it went.

CATCH_REQUESTS · RUNG 3




```
[XIC(Call_2),XIC(Cab_2)]NEQ(Floor,2)OTL(Lamp_2);
```

PREDICT IT · 4 `Call_2` is on and `Floor` is 2. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When either (`Call_2` (Hall call button on floor 2 (1 while pressed)) is on) *or* (`Cab_2` (Cab button for floor 2 (1 while pressed)) is on) and `Floor` (The floor the car is at or nearest, from the encoder (1000 counts per floor)) is not **2**, the rung latches `Lamp_2` (Built-in lamp of the floor 2 hall switch and cab button: a call is registered) on.

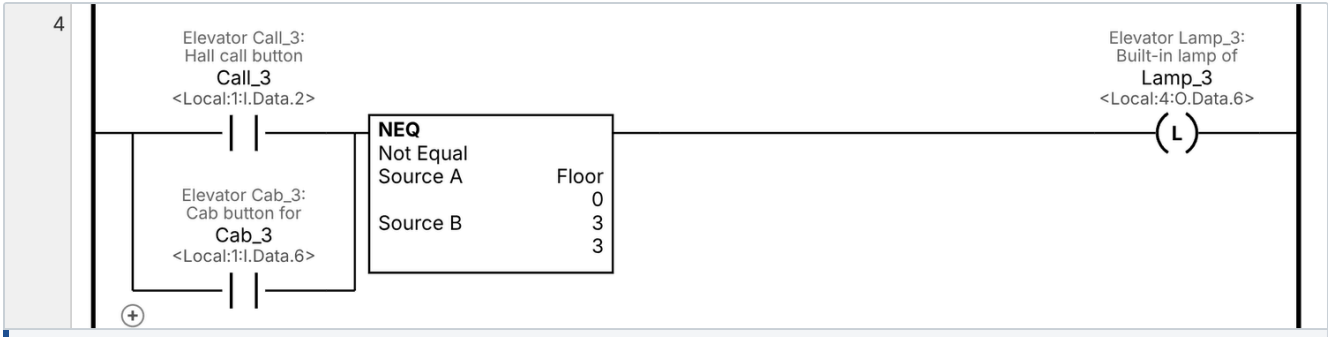
Left to right:

1. A branch of 2 parallel paths: power gets through if any one of them is true.
   - **Path 1**
     1. `XIC(Call_2)` *Examine If Closed* asks: true when `Call_2` (Hall call button on floor 2 (1 while pressed)) is on.
   - **Path 2**
     1. `XIC(Cab_2)` *Examine If Closed* asks: true when `Cab_2` (Cab button for floor 2 (1 while pressed)) is on.
2. `NEQ(Floor,2)` *Not Equal* asks: true when `Floor` (The floor the car is at or nearest, from the encoder (1000 counts per floor)) is not **2**.
3. `OTL(Lamp_2)` *Output Latch* does: latches `Lamp_2` (Built-in lamp of the floor 2 hall switch and cab button: a call is registered) on.

Why it is built this way:

A latch: it stays on until an OTU clears it.

CATCH_REQUESTS · RUNG 4




```
[XIC(Call_3),XIC(Cab_3)]NEQ(Floor,3)OTL(Lamp_3);
```

PREDICT IT · 5 `Call_3` is on and `Floor` is 4. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When either (`Call_3` (Hall call button on floor 3 (1 while pressed)) is on) *or* (`Cab_3` (Cab button for floor 3 (1 while pressed)) is on) and `Floor` (The floor the car is at or nearest, from the encoder (1000 counts per floor)) is not **3**, the rung latches `Lamp_3` (Built-in lamp of the floor 3 hall switch and cab button: a call is registered) on.

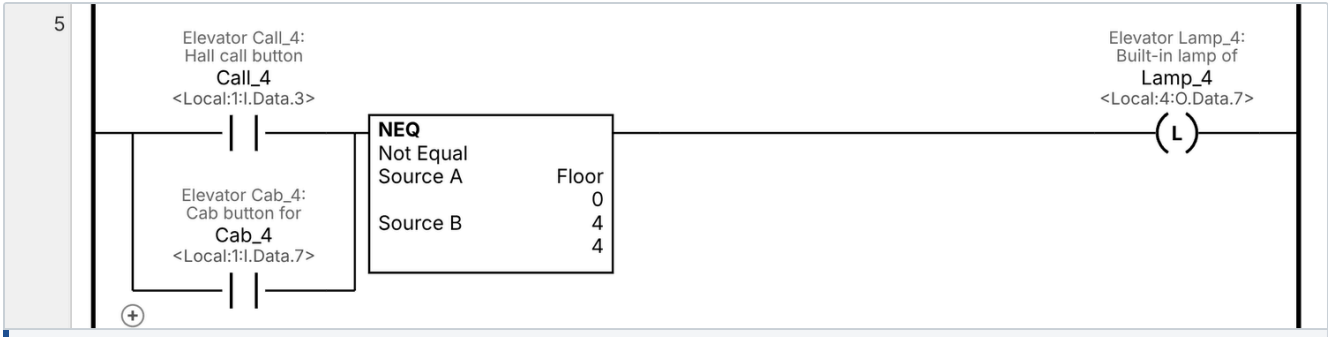
Left to right:

1. A branch of 2 parallel paths: power gets through if any one of them is true.
   - **Path 1**
     1. `XIC(Call_3)` *Examine If Closed* asks: true when `Call_3` (Hall call button on floor 3 (1 while pressed)) is on.
   - **Path 2**
     1. `XIC(Cab_3)` *Examine If Closed* asks: true when `Cab_3` (Cab button for floor 3 (1 while pressed)) is on.
2. `NEQ(Floor,3)` *Not Equal* asks: true when `Floor` (The floor the car is at or nearest, from the encoder (1000 counts per floor)) is not **3**.
3. `OTL(Lamp_3)` *Output Latch* does: latches `Lamp_3` (Built-in lamp of the floor 3 hall switch and cab button: a call is registered) on.

Why it is built this way:

A latch: it stays on until an OTU clears it.

CATCH_REQUESTS · RUNG 5




```
[XIC(Call_4),XIC(Cab_4)]NEQ(Floor,4)OTL(Lamp_4);
```

PREDICT IT · 6 `Call_4` is on and `Floor` is 4. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When either (`Call_4` (Hall call button on floor 4 (1 while pressed)) is on) *or* (`Cab_4` (Cab button for floor 4 (1 while pressed)) is on) and `Floor` (The floor the car is at or nearest, from the encoder (1000 counts per floor)) is not **4**, the rung latches `Lamp_4` (Built-in lamp of the floor 4 hall switch and cab button: a call is registered) on.

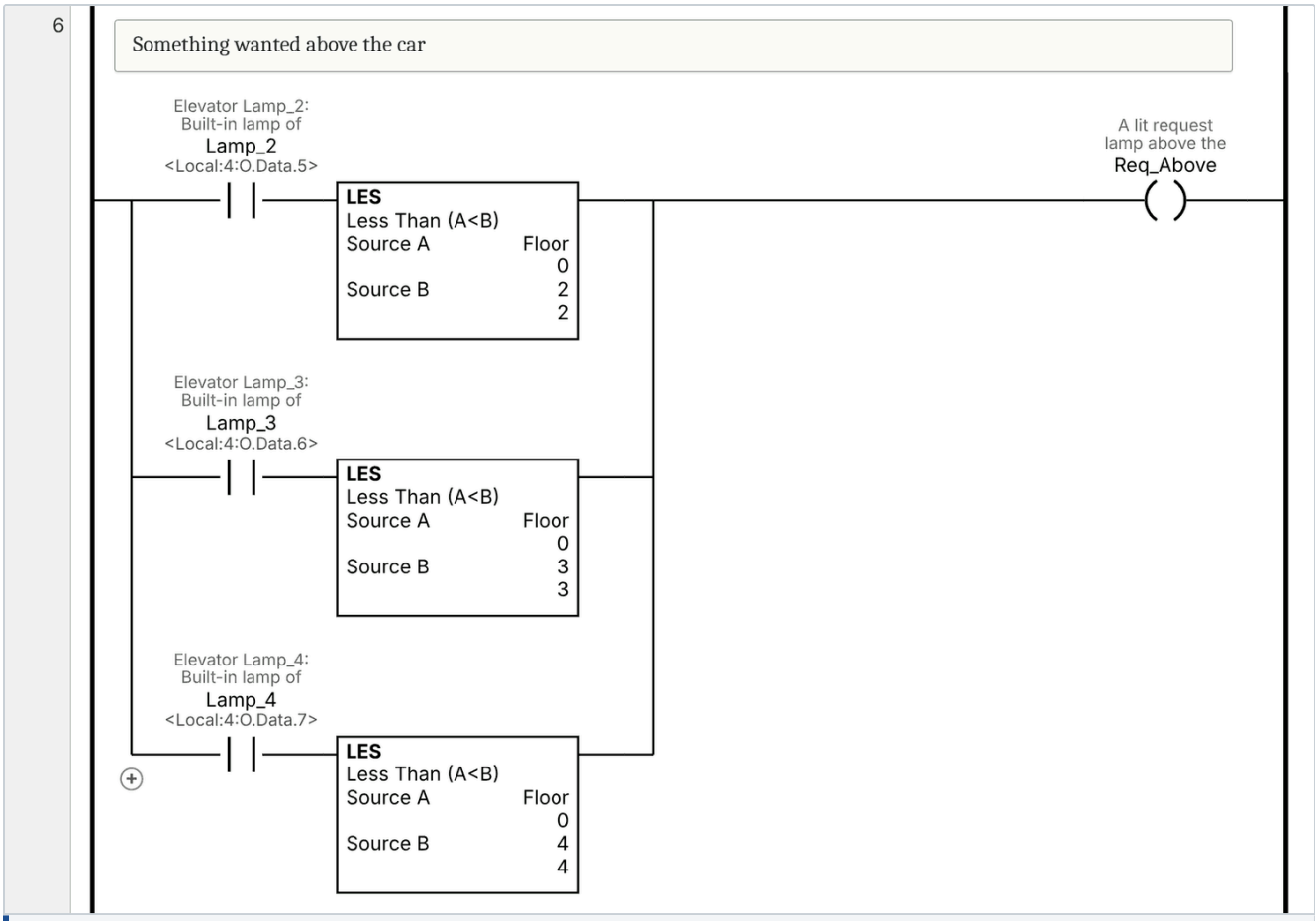
Left to right:

1. A branch of 2 parallel paths: power gets through if any one of them is true.
   - **Path 1**
     1. `XIC(Call_4)` *Examine If Closed* asks: true when `Call_4` (Hall call button on floor 4 (1 while pressed)) is on.
   - **Path 2**
     1. `XIC(Cab_4)` *Examine If Closed* asks: true when `Cab_4` (Cab button for floor 4 (1 while pressed)) is on.
2. `NEQ(Floor,4)` *Not Equal* asks: true when `Floor` (The floor the car is at or nearest, from the encoder (1000 counts per floor)) is not **4**.
3. `OTL(Lamp_4)` *Output Latch* does: latches `Lamp_4` (Built-in lamp of the floor 4 hall switch and cab button: a call is registered) on.

Why it is built this way:

A latch: it stays on until an OTU clears it.

CATCH_REQUESTS · RUNG 6




```
[XIC(Lamp_2)LES(Floor,2),XIC(Lamp_3)LES(Floor,3),XIC(Lamp_4)LES(Floor,4)]OTE(Req_Above);
```

Something wanted above the car

PREDICT IT · 7 `Lamp_2` is on and `Floor` is 1. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When either (`Lamp_2` (Built-in lamp of the floor 2 hall switch and cab button: a call is registered) is on and `Floor` (The floor the car is at or nearest, from the encoder (1000 counts per floor)) is less than **2**) *or* (`Lamp_3` (Built-in lamp of the floor 3 hall switch and cab button: a call is registered) is on and `Floor` is less than **3**) *or* (`Lamp_4` (Built-in lamp of the floor 4 hall switch and cab button: a call is registered) is on and `Floor` is less than **4**), the rung turns `Req_Above` (A lit request lamp above the car) on (and off again whenever the rung is false).

Left to right:

1. A branch of 3 parallel paths: power gets through if any one of them is true.

   Path 1

   1. `XIC(Lamp_2)` *Examine If Closed* asks: true when `Lamp_2` (Built-in lamp of the floor 2 hall switch and cab button: a call is registered) is on.

2. `LES(Floor,2)` *Less Than* asks: true when `Floor` (The floor the car is at or nearest, from the encoder (1000 counts per floor)) is less than **2**.

Path 2

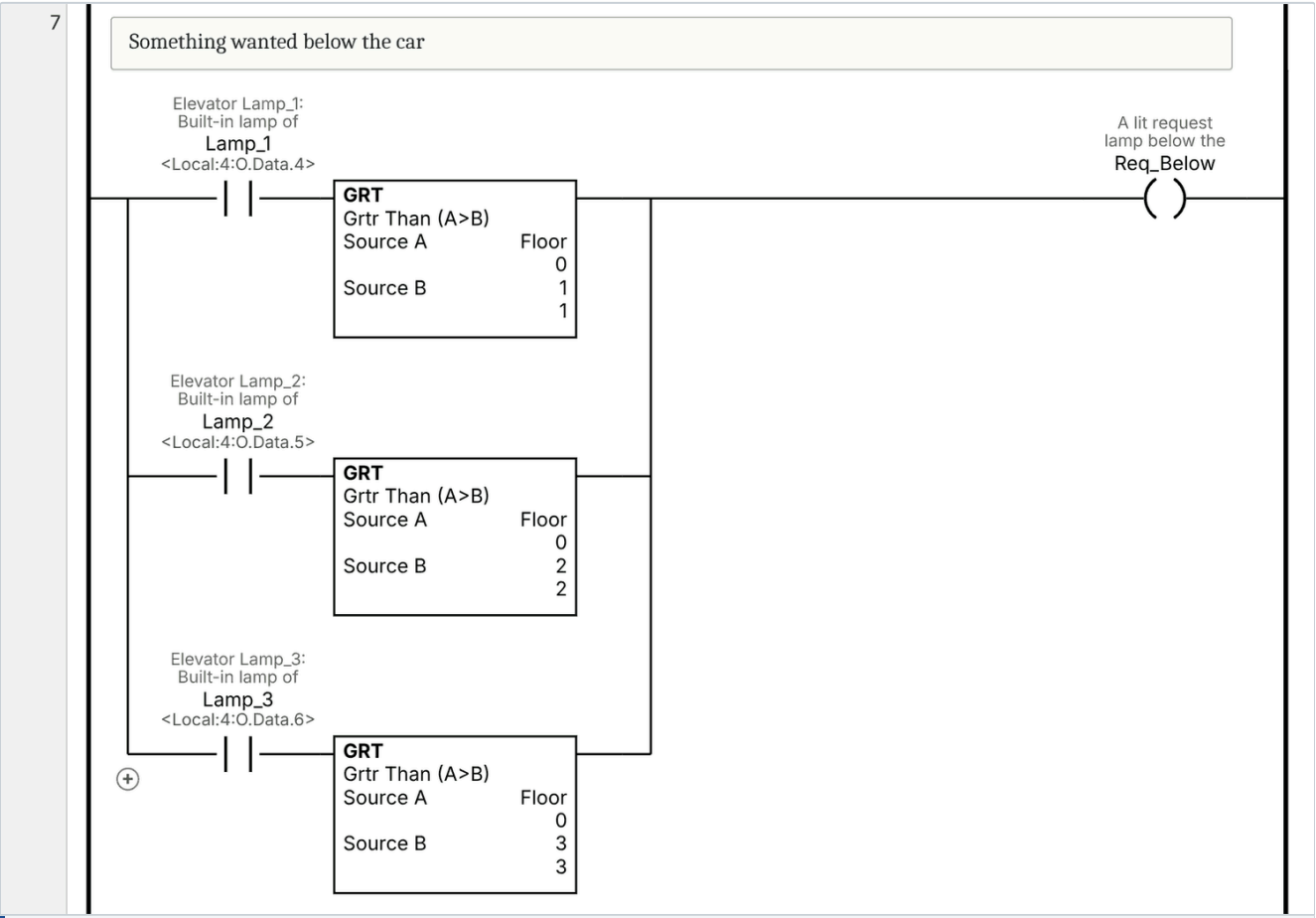
1. `XIC(Lamp_3)` *Examine If Closed* asks: true when `Lamp_3` (Built-in lamp of the floor 3 hall switch and cab button: a call is registered) is on.
2. `LES(Floor,3)` *Less Than* asks: true when `Floor` is less than **3**.

Path 3

1. `XIC(Lamp_4)` *Examine If Closed* asks: true when `Lamp_4` (Built-in lamp of the floor 4 hall switch and cab button: a call is registered) is on.
2. `LES(Floor,4)` *Less Than* asks: true when `Floor` is less than **4**.

2. `OTE(Req_Above)` *Output Energize* does: turns `Req_Above` (A lit request lamp above the car) on (and off again whenever the rung is false).

CATCH_REQUESTS · RUNG 7




```
[XIC(Lamp_1)GRT(Floor,1),XIC(Lamp_2)GRT(Floor,2),XIC(Lamp_3)GRT(Floor,3)]OTE(Req_Below);
```

Something wanted below the car

PREDICT IT · 8 `Lamp_1` is on and `Floor` is 2. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When either (`Lamp_1` (Built-in lamp of the floor 1 hall switch and cab button: a call is registered) is on and `Floor` (The floor the car is at or nearest, from the encoder (1000 counts per floor)) is greater than **1**) *or* (`Lamp_2` (Built-in lamp of the floor 2 hall switch and cab button: a call is registered) is on and `Floor` is greater than **2**) *or* (`Lamp_3` (Built-in lamp of the floor 3 hall switch and cab button: a call is registered) is on and `Floor` is greater than **3**), the rung turns `Req_Below` (A lit request lamp below the car) on (and off again whenever the rung is false).

Left to right:

1. A branch of 3 parallel paths: power gets through if any one of them is true.

   Path 1

   1. `XIC(Lamp_1)` *Examine If Closed* asks: true when `Lamp_1` (Built-in lamp of the floor 1 hall switch and cab button: a call is registered) is on.

2. `GRT(Floor,1)` *Greater Than* asks: true when `Floor` (The floor the car is at or nearest, from the encoder (1000 counts per floor)) is greater than **1**.

Path 2

1. `XIC(Lamp_2)` *Examine If Closed* asks: true when `Lamp_2` (Built-in lamp of the floor 2 hall switch and cab button: a call is registered) is on.
2. `GRT(Floor,2)` *Greater Than* asks: true when `Floor` is greater than **2**.

Path 3

1. `XIC(Lamp_3)` *Examine If Closed* asks: true when `Lamp_3` (Built-in lamp of the floor 3 hall switch and cab button: a call is registered) is on.
2. `GRT(Floor,3)` *Greater Than* asks: true when `Floor` is greater than **3**.

2. `OTE(Req_Below)` *Output Energize* does: turns `Req_Below` (A lit request lamp below the car) on (and off again whenever the rung is false).

TRY IT: THE ENCODER WINDOWS

Change the first rung's windows to the floor switches, `XIC(At_1)MOV(1,Floor)` and so on, and run. Floor now only updates while the car is level, and between floors the displays keep the last floor: that is also a correct indicator, the one most elevators have. Then make the windows narrow, 950 to 1050, and watch Floor go to 0 between floors: Req_Above and Req_Below are then wrong while moving, and the car hesitates. The wide windows are not laziness, they are what makes the direction logic right at every count.

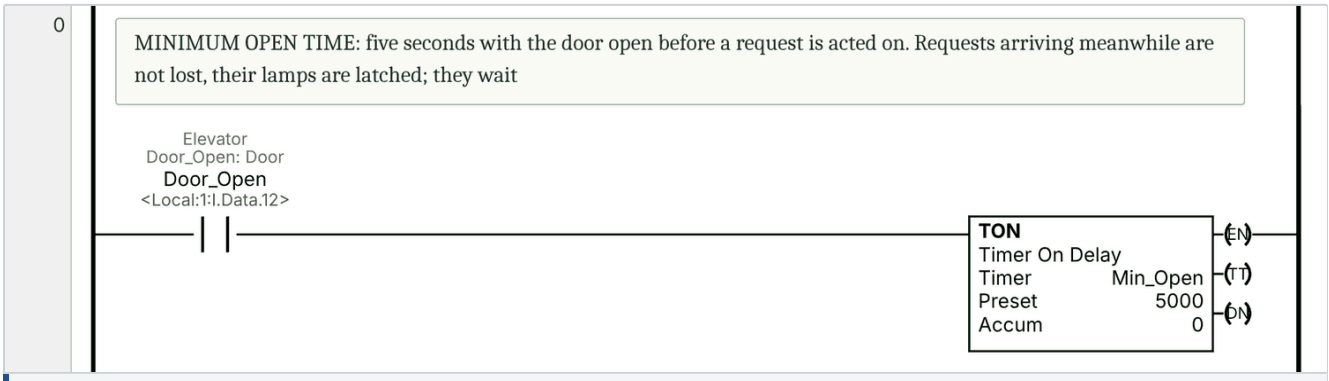
Exercise 3: to the top, and back

The exercise first asks only for this: a call from the fourth floor takes the car up and stops it there. That is one pass round the loop, and it needs every routine to exist with just enough in it. Here they are in the order the loop runs them, already complete for all four floors.

Next request, or wait

Scanned while the car stands at a floor with its door open. It holds the door five seconds (the minimum open time the exercise asks for; requests arriving meanwhile wait, latched), picks a direction, and goes.

NEXT_OR_WAIT · RUNG 0




```
XIC(Door_Open)TON(Min_Open,5000,0);
```

MINIMUM OPEN TIME: five seconds with the door open before a request is acted on. Requests arriving meanwhile are not lost, their lamps are latched; they wait

HOW THIS RUNG WORKS

In words. When `Door_Open` (Door fully open (end switch)) is on, the rung runs the timer `Min_Open` (The door stays open at least five seconds before the next request is processed) toward 5000 ms (5 s).

Left to right:

1. `XIC(Door_Open)` *Examine If Closed* asks: true when `Door_Open` (Door fully open (end switch)) is on.
2. `TON(Min_Open,5000,0)` *Timer On Delay* does: runs the timer `Min_Open` (The door stays open at least five seconds before the next request is processed) toward 5000 ms (5 s). The accumulator counts while the rung is true; .DN comes on at the preset; the moment the rung goes false the timer resets.

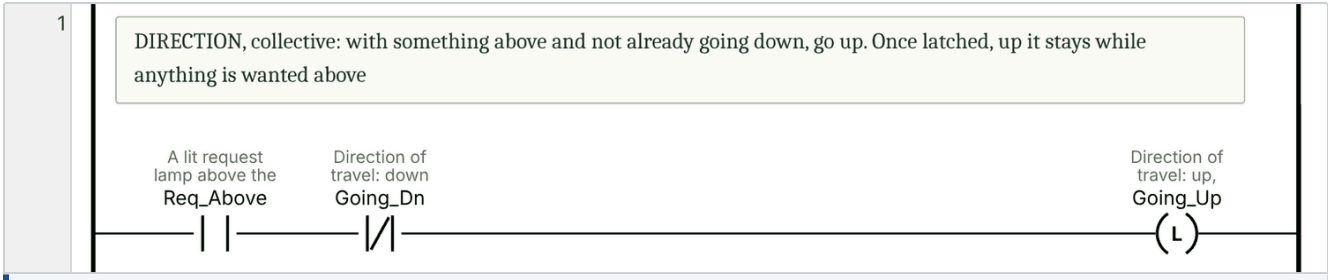
Why it is built this way:

A timer. `Min_Open` only times while this rung is true; other rungs use its `.DN` (done) or `.TT` (timing) bits. Being a TON, it starts again from zero every time the rung goes false.

> **Think of it like this:** a kitchen timer that only runs while you hold your finger on it. Take your finger off and it snaps back to zero. Hold it long enough and the done light comes on, and stays on as long as your finger is still there.

> **Watch for:** If anything in series flickers false for one scan, a TON starts over from zero. Use RTO when the time must survive an interruption.

NEXT_OR_WAIT · RUNG 1




```
XIC(Req_Above)XIO(Going_Dn)OTL(Going_Up);
```

DIRECTION, collective: with something above and not already going down, go up. Once latched, up it stays while anything is wanted above

PREDICT IT · 9 `Req_Above` is on and `Going_Dn` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Req_Above` (A lit request lamp above the car) is on and `Going_Dn` (Direction of travel: down) is off, the rung latches `Going_Up` (Direction of travel: up, latched while anything is wanted above) on.

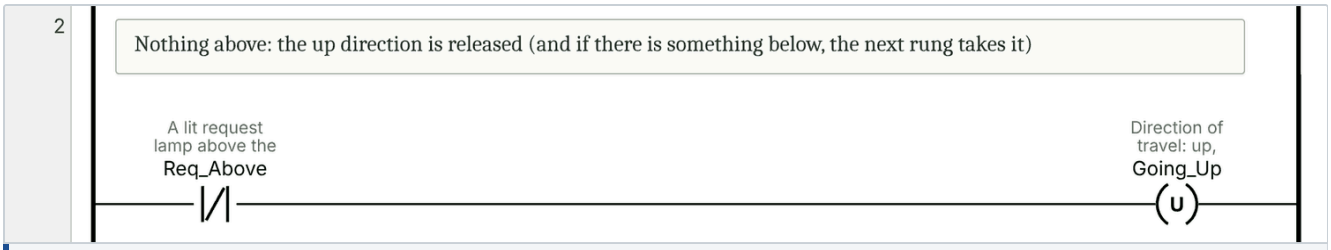
Left to right:

1. `XIC(Req_Above)` *Examine If Closed* asks: true when `Req_Above` (A lit request lamp above the car) is on.
2. `XIO(Going_Dn)` *Examine If Open* asks: true when `Going_Dn` (Direction of travel: down) is off.
3. `OTL(Going_Up)` *Output Latch* does: latches `Going_Up` (Direction of travel: up, latched while anything is wanted above) on.

Why it is built this way:

A latch: it stays on until an OTU clears it.

NEXT_OR_WAIT · RUNG 2




```
XIO(Req_Above)OTU(Going_Up);
```

Nothing above: the up direction is released (and if there is something below, the next rung takes it)

HOW THIS RUNG WORKS

In words. When `Req_Above` (A lit request lamp above the car) is off, the rung unlatches `Going_Up` (Direction of travel: up, latched while anything is wanted above) (turns it off).

Left to right:

1. `XIO(Req_Above)` *Examine If Open* asks: true when `Req_Above` (A lit request lamp above the car) is off.
2. `OTU(Going_Up)` *Output Unlatch* does: unlatches `Going_Up` (Direction of travel: up, latched while anything is wanted above) (turns it off).

NEXT_OR_WAIT · RUNG 3

```
XIC(Req_Below)XIO(Going_Up)OTL(Going_Dn);
```

PREDICT IT · 10 `Req_Below` is off and `Going_Up` is off. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Req_Below` (A lit request lamp below the car) is on and `Going_Up` (Direction of travel: up, latched while anything is wanted above) is off, the rung latches `Going_Dn` (Direction of travel: down) on.

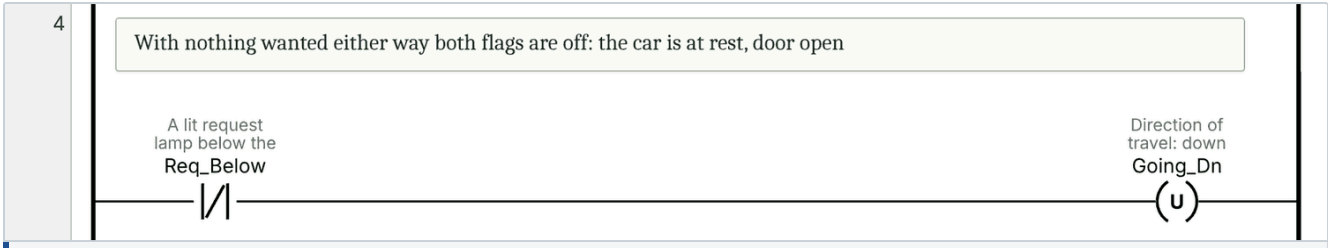
Left to right:

1. `XIC(Req_Below)` *Examine If Closed* asks: true when `Req_Below` (A lit request lamp below the car) is on.
2. `XIO(Going_Up)` *Examine If Open* asks: true when `Going_Up` (Direction of travel: up, latched while anything is wanted above) is off.
3. `OTL(Going_Dn)` *Output Latch* does: latches `Going_Dn` (Direction of travel: down) on.

Why it is built this way:

A latch: it stays on until an OTU clears it.

NEXT_OR_WAIT · RUNG 4




```
XIO(Req_Below)OTU(Going_Dn);
```

With nothing wanted either way both flags are off: the car is at rest, door open

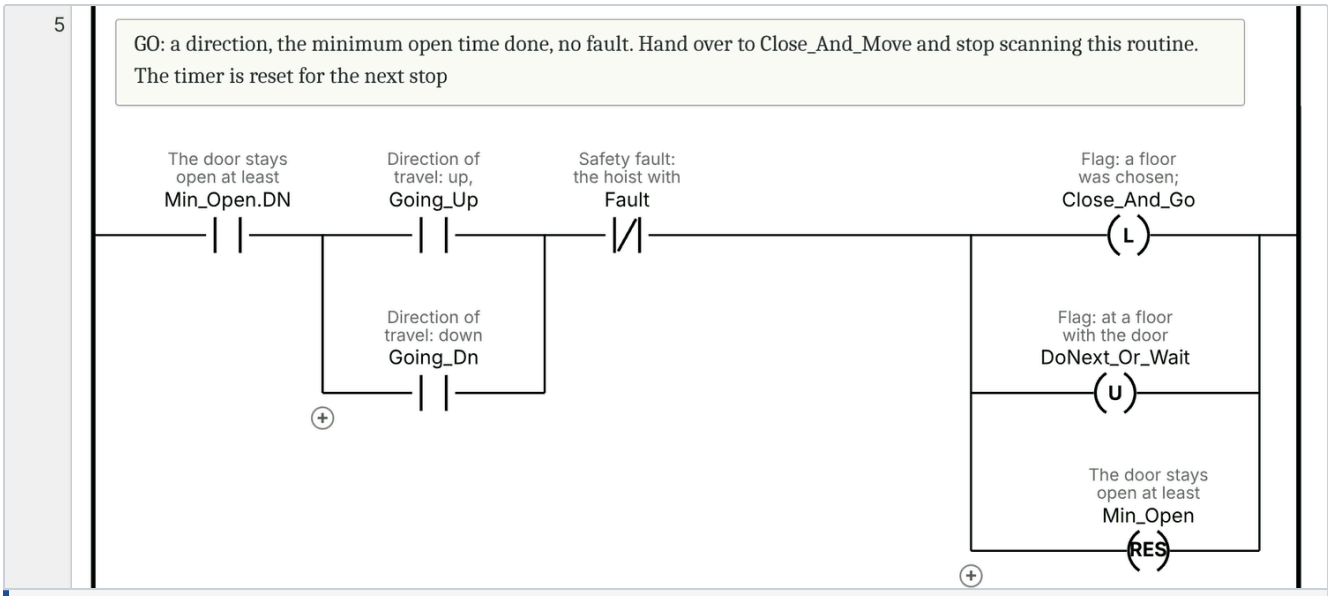
HOW THIS RUNG WORKS

In words. When `Req_Below` (A lit request lamp below the car) is off, the rung unlatches `Going_Dn` (Direction of travel: down) (turns it off).

Left to right:

1. `XIO(Req_Below)` *Examine If Open* asks: true when `Req_Below` (A lit request lamp below the car) is off.
2. `OTU(Going_Dn)` *Output Unlatch* does: unlatches `Going_Dn` (Direction of travel: down) (turns it off).

NEXT_OR_WAIT · RUNG 5




```
XIC(Min_Open.DN)[XIC(Going_Up),XIC(Going_Dn)]XIO(Fault)
[OTL(Close_And_Go),OTU(DoNext_Or_Wait),RES(Min_Open)];
```

GO: a direction, the minimum open time done, no fault. Hand over to Close_And_Move and stop scanning this routine. The timer is reset for the next stop

HOW THIS RUNG WORKS

In words. When `Min_Open.DN` (the done bit of `Min_Open` : The door stays open at least five seconds before the next request is processed) is on, either (`Going_Up` (Direction of travel: up, latched while anything is wanted above) is on) *or* (`Going_Dn` (Direction of travel: down) is on) and `Fault` (Safety fault: the hoist with the door not closed, or an overload; cleared by Reset) is off, the rung latches `Close_And_Go` (Flag: a floor was chosen; Close_And_Move closes the door and hands over to Track_Car) on, unlatches `DoNext_Or_Wait` (Flag: at a floor with the door open; Next_Or_Wait is scanned and decides where to go next) (turns it off) and resets `Min_Open` .

Left to right:

1. `XIC(Min_Open.DN)` *Examine If Closed* asks: true when `Min_Open.DN` (the done bit of `Min_Open` : The door stays open at least five seconds before the next request is processed) is on.
2. A branch of 2 parallel paths: power gets through if any one of them is true.

   Path 1

   1. `XIC(Going_Up)` *Examine If Closed* asks: true when `Going_Up` (Direction of travel: up, latched while anything is wanted above) is on.

   Path 2

   1. `XIC(Going_Dn)` *Examine If Closed* asks: true when `Going_Dn` (Direction of travel: down) is on.
3. `XIO(Fault)` *Examine If Open* asks: true when `Fault` (Safety fault: the hoist with the door not closed, or an overload; cleared by Reset) is off.
4. A branch of 3 parallel paths: each path is tested on its own and does its own work.

   Path 1

   1. `OTL(Close_And_Go)` *Output Latch* does: latches `Close_And_Go` (Flag: a floor was chosen; Close_And_Move closes the door and hands over to Track_Car) on.

Path 2

1. `OTU(DoNext_Or_Wait)` *Output Unlatch* does: unlatches `DoNext_Or_Wait` (Flag: at a floor with the door open; Next_Or_Wait is scanned and decides where to go next) (turns it off).

Path 3

1. `RES(Min_Open)` *Reset* does: resets `Min_Open`.

Why it is built this way:

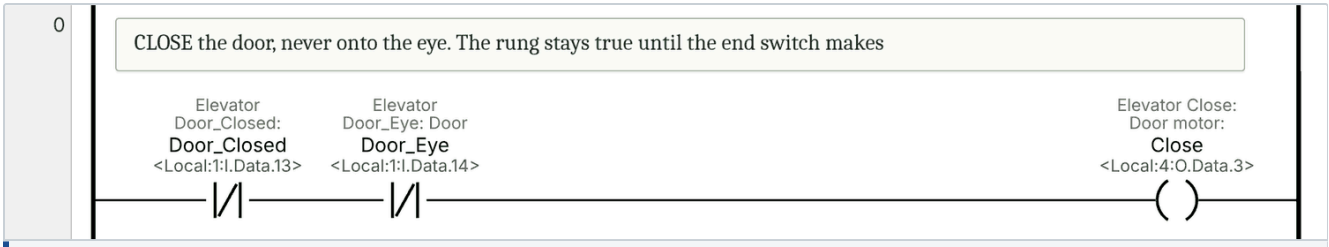
A latch: it stays on until an OTU clears it.

WATCH FOR: A DIRECTION THAT FLICKERS

Write the direction as a plain OTE, `XIC(Req_Above)OTE(Going_Up)`, and run. With requests above and below, the car serves the one above, Req_Above drops the moment it opens there, Req_Below takes over, and it goes down; fine so far. But a new call from above while it is going down makes Req_Above true again and, with an OTE, the car reverses between floors. The latch pair is what keeps the direction until the work ahead is done, which is what a passenger expects.

Close the door and move

CLOSE_AND_MOVE · RUNG 0




```
XIO(Door_Closed)XIO(Door_Eye)OTE(Close);
```

CLOSE the door, never onto the eye. The rung stays true until the end switch makes

PREDICT IT · 11 `Door_Closed` is off and `Door_Eye` is off. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

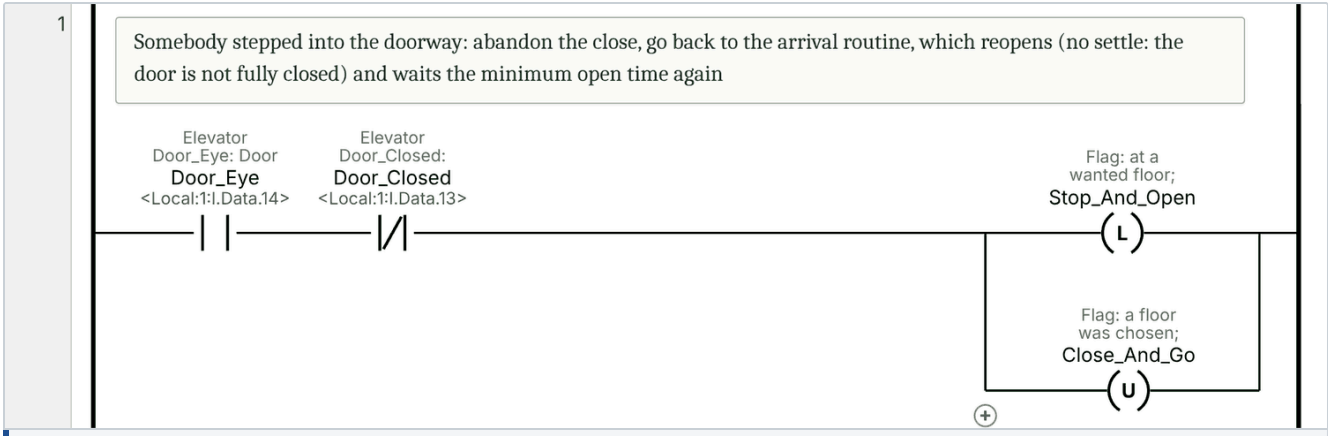
HOW THIS RUNG WORKS

In words. When `Door_Closed` (Door fully closed (end switch)) is off and `Door_Eye` (Door photo eye: blocked while someone is in the doorway) is off, the rung turns `Close` (Door motor: close) on (and off again whenever the rung is false).

Left to right:

1. `XIO(Door_Closed)` *Examine If Open* asks: true when `Door_Closed` (Door fully closed (end switch)) is off.
2. `XIO(Door_Eye)` *Examine If Open* asks: true when `Door_Eye` (Door photo eye: blocked while someone is in the doorway) is off.
3. `OTE(Close)` *Output Energize* does: turns `Close` (Door motor: close) on (and off again whenever the rung is false).

CLOSE_AND_MOVE · RUNG 1




```
XIC(Door_Eye)XIO(Door_Closed)[OTL(Stop_And_Open),OTU(Close_And_Go)];
```

Somebody stepped into the doorway: abandon the close, go back to the arrival routine, which reopens (no settle: the door is not fully closed) and waits the minimum open time again

HOW THIS RUNG WORKS

In words. When `Door_Eye` (Door photo eye: blocked while someone is in the doorway) is on and `Door_Closed` (Door fully closed (end switch)) is off, the rung latches `Stop_And_Open` (Flag: at a wanted floor; Stop_And_Open_Door stops the hoist, settles and opens) on and unlatches `Close_And_Go` (Flag: a floor was chosen; Close_And_Move closes the door and hands over to Track_Car) (turns it off).

Left to right:

1. `XIC(Door_Eye)` *Examine If Closed* asks: true when `Door_Eye` (Door photo eye: blocked while someone is in the doorway) is on.
2. `XIO(Door_Closed)` *Examine If Open* asks: true when `Door_Closed` (Door fully closed (end switch)) is off.
3. A branch of 2 parallel paths: each path is tested on its own and does its own work.

   Path 1

   1. `OTL(Stop_And_Open)` *Output Latch* does: latches `Stop_And_Open` (Flag: at a wanted floor; Stop_And_Open_Door stops the hoist, settles and opens) on.

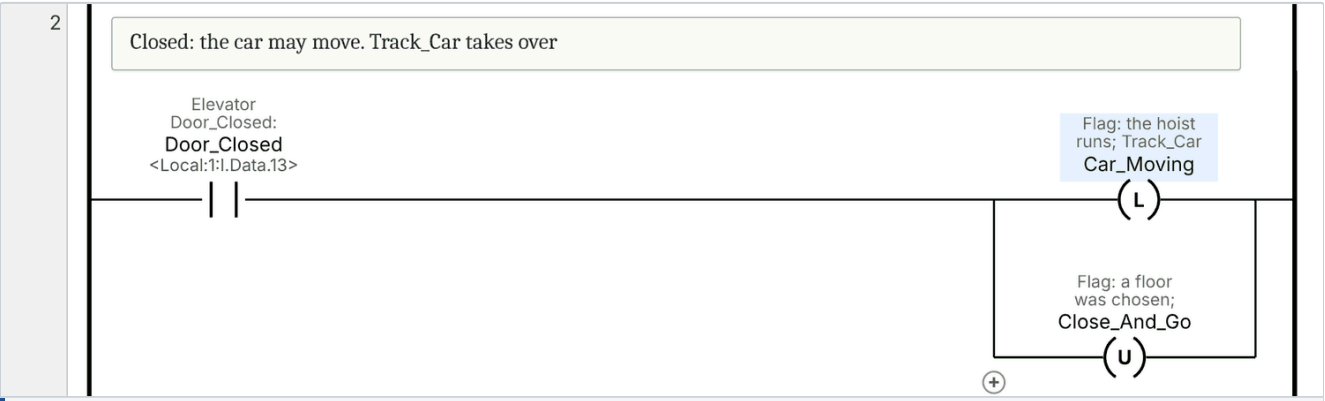
   Path 2

   1. `OTU(Close_And_Go)` *Output Unlatch* does: unlatches `Close_And_Go` (Flag: a floor was chosen; Close_And_Move closes the door and hands over to Track_Car) (turns it off).

Why it is built this way:

A latch: it stays on until an OTU clears it.

CLOSE_AND_MOVE · RUNG 2




```
XIC(Door_Closed)[OTL(Car_Moving),OTU(Close_And_Go)];
```

Closed: the car may move. Track_Car takes over

HOW THIS RUNG WORKS

In words. When `Door_Closed` (Door fully closed (end switch)) is on, the rung latches `Car_Moving` (Flag: the hoist runs; Track_Car watches for the floor to stop at) on and unlatches `Close_And_Go` (Flag: a floor was chosen; Close_And_Move closes the door and hands over to Track_Car) (turns it off).

Left to right:

1. `XIC(Door_Closed)` *Examine If Closed* asks: true when `Door_Closed` (Door fully closed (end switch)) is on.
2. A branch of 2 parallel paths: each path is tested on its own and does its own work.

   Path 1

   1. `OTL(Car_Moving)` *Output Latch* does: latches `Car_Moving` (Flag: the hoist runs; Track_Car watches for the floor to stop at) on.

   Path 2

   1. `OTU(Close_And_Go)` *Output Unlatch* does: unlatches `Close_And_Go` (Flag: a floor was chosen; Close_And_Move closes the door and hands over to Track_Car) (turns it off).

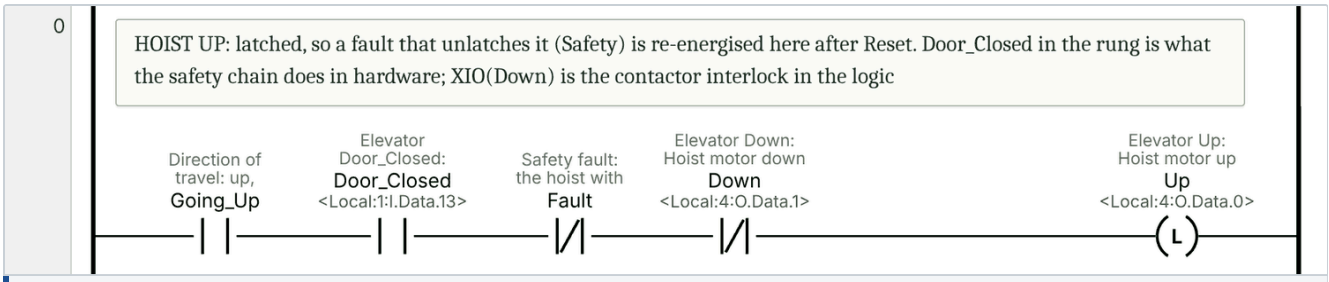
Why it is built this way:

A latch: it stays on until an OTU clears it.

Track the car

Scanned while the car moves. The hoist outputs are latched, which is unusual and deliberate: this routine stops being scanned the instant the car arrives, and an OTE in a routine that is not scanned keeps its last state. With latches, the arrival routine unlatches the hoist explicitly, and a fault that unlatches it is re-energised here after Reset.

TRACK_CAR · RUNG 0




```
XIC(Going_Up)XIC(Door_Closed)XIO(Fault)XIO(Down)OTL(Up);
```

HOIST UP: latched, so a fault that unlatches it (Safety) is re-energised here after Reset. Door_Closed in the rung is what the safety chain does in hardware; XIO(Down) is the contactor interlock in the logic

PREDICT IT · 12 `Going_Up` is on, `Door_Closed` is off, `Fault` is off and `Down` is off. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Going_Up` (Direction of travel: up, latched while anything is wanted above) is on, `Door_Closed` (Door fully closed (end switch)) is on, `Fault` (Safety fault: the hoist with the door not closed, or an overload; cleared by Reset) is off and `Down` (Hoist motor down (never with Up: the contactors are interlocked)) is off, the rung latches `Up` (Hoist motor up) on.

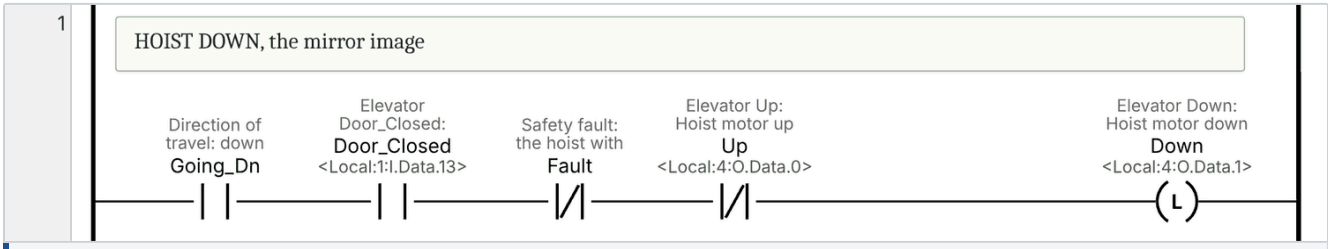
Left to right:

1. `XIC(Going_Up)` *Examine If Closed* asks: true when `Going_Up` (Direction of travel: up, latched while anything is wanted above) is on.
2. `XIC(Door_Closed)` *Examine If Closed* asks: true when `Door_Closed` (Door fully closed (end switch)) is on.
3. `XIO(Fault)` *Examine If Open* asks: true when `Fault` (Safety fault: the hoist with the door not closed, or an overload; cleared by Reset) is off.
4. `XIO(Down)` *Examine If Open* asks: true when `Down` (Hoist motor down (never with Up: the contactors are interlocked)) is off.
5. `OTL(Up)` *Output Latch* does: latches `Up` (Hoist motor up) on.

Why it is built this way:

A latch: it stays on until an OTU clears it.

TRACK_CAR · RUNG 1




```
XIC(Going_Dn)XIC(Door_Closed)XIO(Fault)XIO(Up)OTL(Down);
```

HOIST DOWN, the mirror image

PREDICT IT · 13 `Going_Dn` is on, `Door_Closed` is on, `Fault` is off and `Up` is off. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Going_Dn` (Direction of travel: down) is on, `Door_Closed` (Door fully closed (end switch)) is on, `Fault` (Safety fault: the hoist with the door not closed, or an overload; cleared by Reset) is off and `Up` (Hoist motor up) is off, the rung latches `Down` (Hoist motor down (never with Up: the contactors are interlocked)) on.

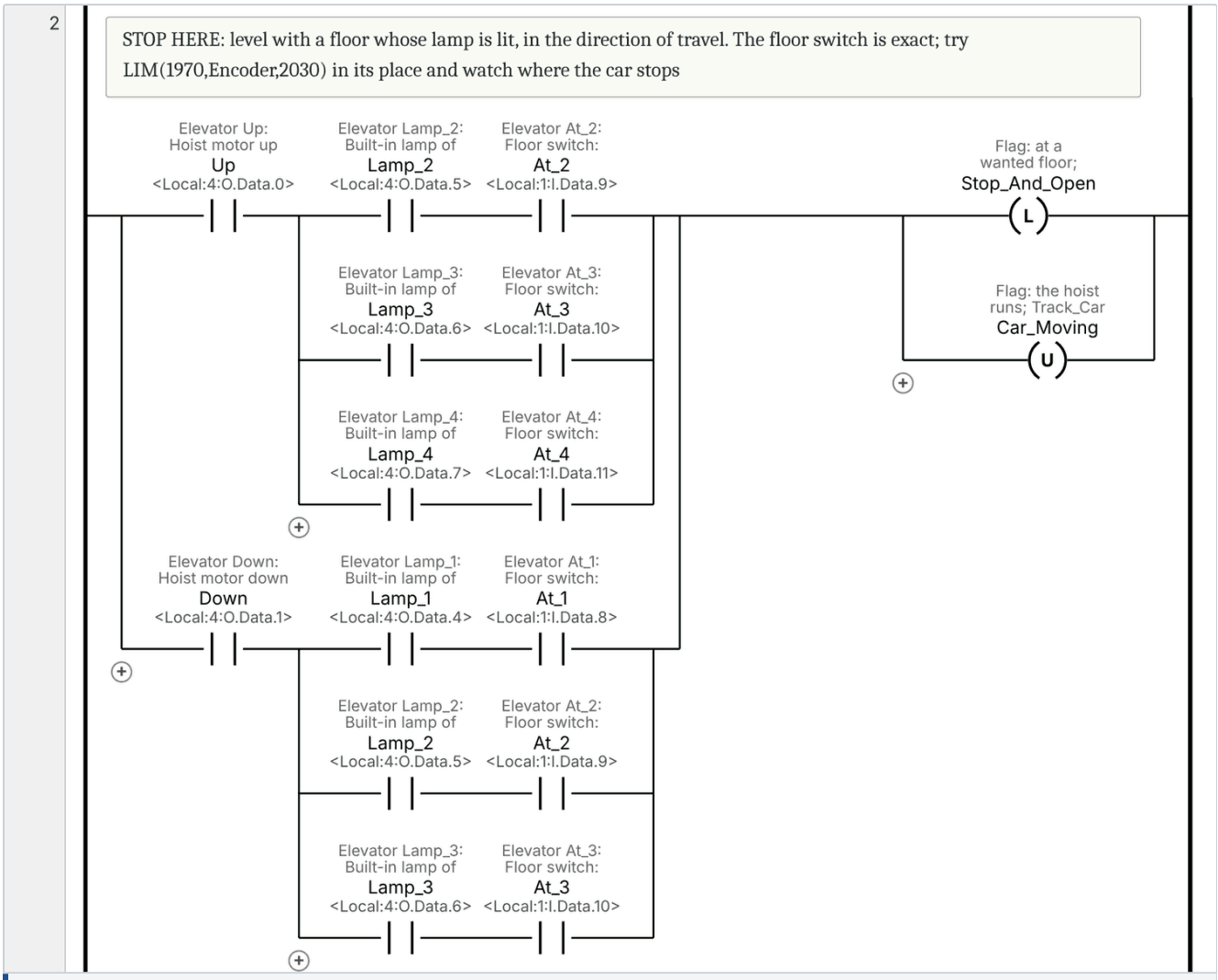
Left to right:

1. `XIC(Going_Dn)` *Examine If Closed* asks: true when `Going_Dn` (Direction of travel: down) is on.
2. `XIC(Door_Closed)` *Examine If Closed* asks: true when `Door_Closed` (Door fully closed (end switch)) is on.
3. `XIO(Fault)` *Examine If Open* asks: true when `Fault` (Safety fault: the hoist with the door not closed, or an overload; cleared by Reset) is off.
4. `XIO(Up)` *Examine If Open* asks: true when `Up` (Hoist motor up) is off.
5. `OTL(Down)` *Output Latch* does: latches `Down` (Hoist motor down (never with Up: the contactors are interlocked)) on.

Why it is built this way:

A latch: it stays on until an OTU clears it.

TRACK_CAR · RUNG 2




```
[XIC(Up)
[XIC(Lamp_2)XIC(At_2),XIC(Lamp_3)XIC(At_3),XIC(Lamp_4)XIC(At_4)],XIC(Down)
[XIC(Lamp_1)XIC(At_1),XIC(Lamp_2)XIC(At_2),XIC(Lamp_3)XIC(At_3)]]
[OTL(Stop_And_Open),OTU(Car_Moving)];
```

STOP HERE: level with a floor whose lamp is lit, in the direction of travel. The floor switch is exact; try LIM(1970,Encoder,2030) in its place and watch where the car stops

PREDICT IT · 14 `Up` is on, `Lamp_2` is on and `At_2` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When either (`Up` (Hoist motor up) is on and either (`Lamp_2` (Built-in lamp of the floor 2 hall switch and cab button: a call is registered) is on and `At_2` (Floor switch: the cab is level with floor 2) is on) *or* (`Lamp_3` (Built-in lamp of the floor 3 hall switch and cab button: a call is registered) is on and `At_3` (Floor switch: the cab is level with floor 3) is on) *or* (`Lamp_4` (Built-in lamp of the floor 4 hall switch and cab button: a call is registered) is on and `At_4` (Floor

switch: the cab is level with floor 4) is on)) *or* (`Down` (Hoist motor down (never with Up: the contactors are interlocked)) is on and either (`Lamp_1` (Built-in lamp of the floor 1 hall switch and cab button: a call is registered) is on and `At_1` (Floor switch: the cab is level with floor 1) is on) *or* (`Lamp_2` is on and `At_2` is on) *or* (`Lamp_3` is on and `At_3` is on)), the rung latches `Stop_And_Open` (Flag: at a wanted floor; Stop_And_Open_Door stops the hoist, settles and opens) on and unlatches `Car_Moving` (Flag: the hoist runs; Track_Car watches for the floor to stop at) (turns it off).

Left to right:

1. A branch of 2 parallel paths: power gets through if any one of them is true.

   Path 1

   1. `XIC(Up)` *Examine If Closed* asks: true when `Up` (Hoist motor up) is on.
   2. A branch of 3 parallel paths: power gets through if any one of them is true.

      Path 1

      1. `XIC(Lamp_2)` *Examine If Closed* asks: true when `Lamp_2` (Built-in lamp of the floor 2 hall switch and cab button: a call is registered) is on.
      2. `XIC(At_2)` *Examine If Closed* asks: true when `At_2` (Floor switch: the cab is level with floor 2) is on.

      Path 2

      1. `XIC(Lamp_3)` *Examine If Closed* asks: true when `Lamp_3` (Built-in lamp of the floor 3 hall switch and cab button: a call is registered) is on.
      2. `XIC(At_3)` *Examine If Closed* asks: true when `At_3` (Floor switch: the cab is level with floor 3) is on.

      Path 3

      1. `XIC(Lamp_4)` *Examine If Closed* asks: true when `Lamp_4` (Built-in lamp of the floor 4 hall switch and cab button: a call is registered) is on.
      2. `XIC(At_4)` *Examine If Closed* asks: true when `At_4` (Floor switch: the cab is level with floor 4) is on.

   Path 2

   1. `XIC(Down)` *Examine If Closed* asks: true when `Down` (Hoist motor down (never with Up: the contactors are interlocked)) is on.
   2. A branch of 3 parallel paths: power gets through if any one of them is true.

      Path 1

      1. `XIC(Lamp_1)` *Examine If Closed* asks: true when `Lamp_1` (Built-in lamp of the floor 1 hall switch and cab button: a call is registered) is on.
      2. `XIC(At_1)` *Examine If Closed* asks: true when `At_1` (Floor switch: the cab is level with floor 1) is on.

      Path 2

      1. `XIC(Lamp_2)` *Examine If Closed* asks: true when `Lamp_2` is on.
      2. `XIC(At_2)` *Examine If Closed* asks: true when `At_2` is on.

      Path 3

      1. `XIC(Lamp_3)` *Examine If Closed* asks: true when `Lamp_3` is on.
      2. `XIC(At_3)` *Examine If Closed* asks: true when `At_3` is on.

2. A branch of 2 parallel paths: each path is tested on its own and does its own work.

   Path 1

   1. `OTL(Stop_And_Open)` *Output Latch* does: latches `Stop_And_Open` (Flag: at a wanted floor; Stop_And_Open_Door stops the hoist, settles and opens) on.

Path 2

1. `OTU(Car_Moving)` *Output Unlatch* does: unlatches `Car_Moving` (Flag: the hoist runs; Track_Car watches for the floor to stop at) (turns it off).

Why it is built this way:

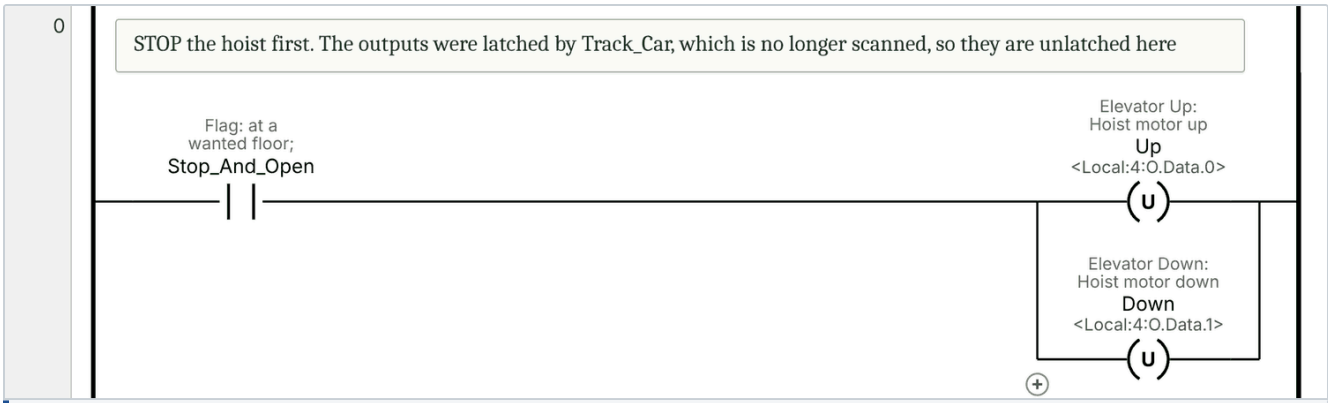
A latch: it stays on until an OTU clears it.

WATCH FOR: THE DOOR AND THE HOIST

The hoist rungs have `XIC(Door_Closed)` in them, and so does the hand-over rung of Close_And_Move. Take both out and run: the moment a request arrives while the door is open the car is flagged moving and the hoist latches with the door still closing. The log shouts MOVING WITH THE DOOR OPEN, the unsafe count on the part goes up, and the Safety routine trips Fault and drops the hoist. On a real elevator that contact is in the safety chain, in hardware; in the program it belongs in the hoist rung as well, because the two are not the same safeguard and you want both.

Stop and open

STOP_AND_OPEN_DOOR · RUNG 0




```
XIC(Stop_And_Open)[OTU(Up),OTU(Down)];
```

STOP the hoist first. The outputs were latched by Track_Car, which is no longer scanned, so they are unlatched here

HOW THIS RUNG WORKS

In words. When `Stop_And_Open` (Flag: at a wanted floor; Stop_And_Open_Door stops the hoist, settles and opens) is on, the rung unlatches `Up` (Hoist motor up) (turns it off) and unlatches `Down` (Hoist motor down (never with Up: the contactors are interlocked)) (turns it off).

Left to right:

1. `XIC(Stop_And_Open)` *Examine If Closed* asks: true when `Stop_And_Open` (Flag: at a wanted floor; Stop_And_Open_Door stops the hoist, settles and opens) is on.
2. A branch of 2 parallel paths: each path is tested on its own and does its own work.

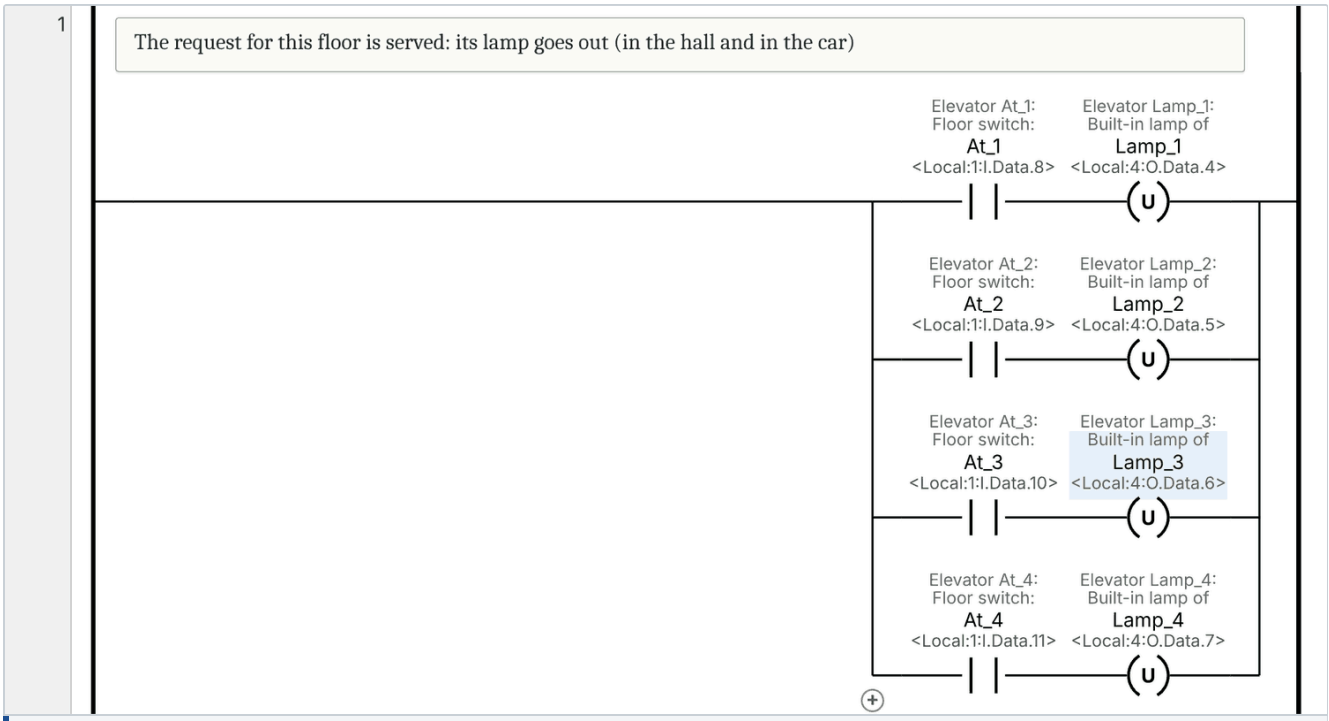
   Path 1

   1. `OTU(Up)` *Output Unlatch* does: unlatches `Up` (Hoist motor up) (turns it off).

   Path 2

   1. `OTU(Down)` *Output Unlatch* does: unlatches `Down` (Hoist motor down (never with Up: the contactors are interlocked)) (turns it off).

STOP_AND_OPEN_DOOR · RUNG 1




```
[XIC(At_1)OTU(Lamp_1),XIC(At_2)OTU(Lamp_2),XIC(At_3)OTU(Lamp_3),XIC(At_4)OTU(Lamp_4)];
```

The request for this floor is served: its lamp goes out (in the hall and in the car)

PREDICT IT · 15 `At_4` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. Every scan, with nothing to test, the rung picks its path: if `At_1` (Floor switch: the cab is level with floor 1) is on, it unlatches `Lamp_1` (Built-in lamp of the floor 1 hall switch and cab button: a call is registered) (turns it off); if `At_2` (Floor switch: the cab is level with floor 2) is on, it unlatches `Lamp_2` (Built-in lamp of the floor 2 hall switch and cab button: a call is registered) (turns it off); if `At_3` (Floor switch: the cab is level with floor 3) is on, it unlatches `Lamp_3` (Built-in lamp of the floor 3 hall switch and cab button: a call is registered) (turns it off); if `At_4` (Floor switch: the cab is level with floor 4) is on, it unlatches `Lamp_4` (Built-in lamp of the floor 4 hall switch and cab button: a call is registered) (turns it off).

Left to right:

1. A branch of 4 parallel paths: each path is tested on its own and does its own work.

   Path 1

   1. `XIC(At_1)` *Examine If Closed* asks: true when `At_1` (Floor switch: the cab is level with floor 1) is on.
   2. `OTU(Lamp_1)` *Output Unlatch* does: unlatches `Lamp_1` (Built-in lamp of the floor 1 hall switch and cab button: a call is registered) (turns it off).

   Path 2

   1. `XIC(At_2)` *Examine If Closed* asks: true when `At_2` (Floor switch: the cab is level with floor 2) is on.

2. `OTU(Lamp_2)` *Output Unlatch* does: unlatches `Lamp_2` (Built-in lamp of the floor 2 hall switch and cab button: a call is registered) (turns it off).

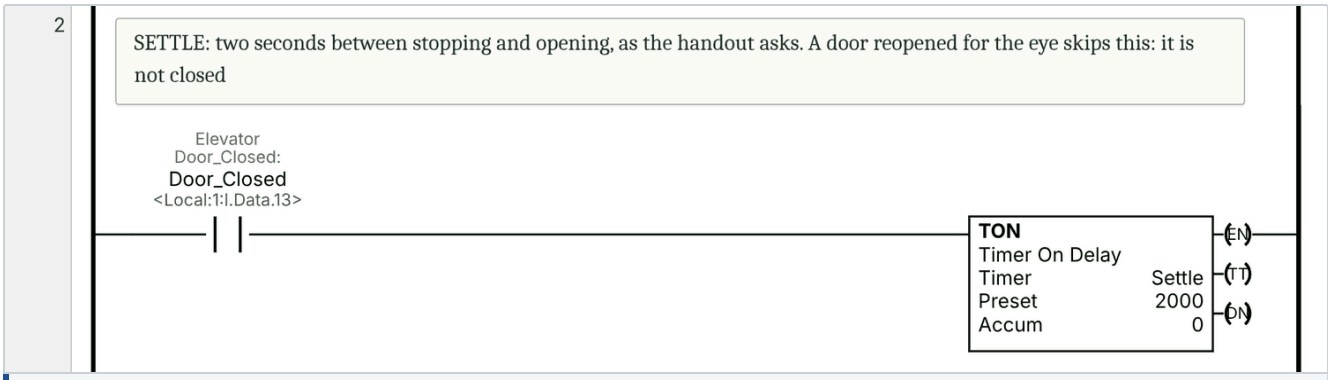
Path 3

1. `XIC(At_3)` *Examine If Closed* asks: true when `At_3` (Floor switch: the cab is level with floor 3) is on.
2. `OTU(Lamp_3)` *Output Unlatch* does: unlatches `Lamp_3` (Built-in lamp of the floor 3 hall switch and cab button: a call is registered) (turns it off).

Path 4

1. `XIC(At_4)` *Examine If Closed* asks: true when `At_4` (Floor switch: the cab is level with floor 4) is on.
2. `OTU(Lamp_4)` *Output Unlatch* does: unlatches `Lamp_4` (Built-in lamp of the floor 4 hall switch and cab button: a call is registered) (turns it off).

STOP_AND_OPEN_DOOR · RUNG 2




```
XIC(Door_Closed)TON(Settle,2000,0);
```

SETTLE: two seconds between stopping and opening, as the handout asks. A door reopened for the eye skips this: it is not closed

HOW THIS RUNG WORKS

In words. When `Door_Closed` (Door fully closed (end switch)) is on, the rung runs the timer `Settle` (Two seconds between stopping and opening the door) toward 2000 ms (2 s).

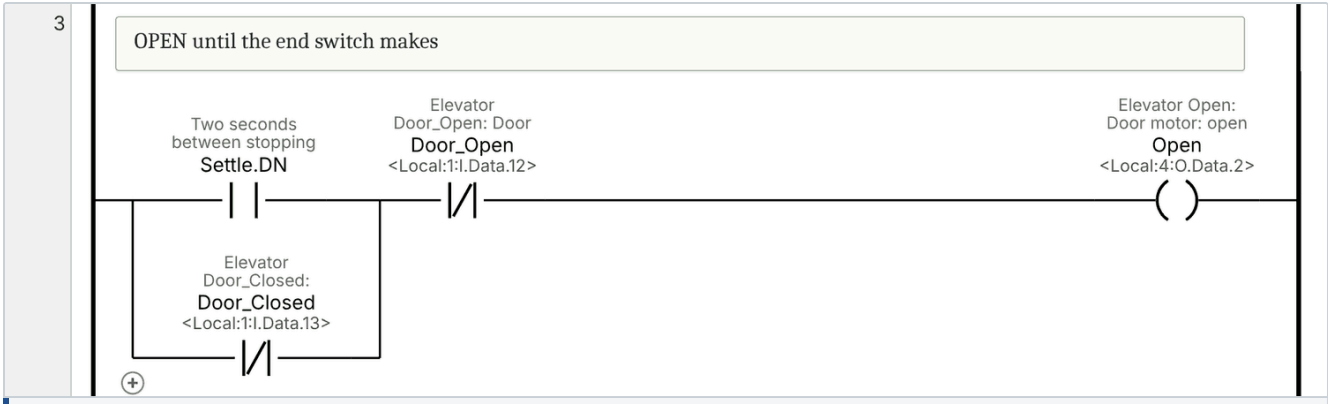
Left to right:

1. `XIC(Door_Closed)` *Examine If Closed* asks: true when `Door_Closed` (Door fully closed (end switch)) is on.
2. `TON(Settle,2000,0)` *Timer On Delay* does: runs the timer `Settle` (Two seconds between stopping and opening the door) toward 2000 ms (2 s).

Why it is built this way:

A timer: it times only while this rung is true.

STOP_AND_OPEN_DOOR · RUNG 3




```
[XIC(Settle.DN),XIO(Door_Closed)]XIO(Door_Open)OTE(Open);
```

OPEN until the end switch makes

PREDICT IT · 16 `Settle.DN` (the done bit of `Settle` : Two seconds between stopping and opening the door) is on and `Door_Open` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

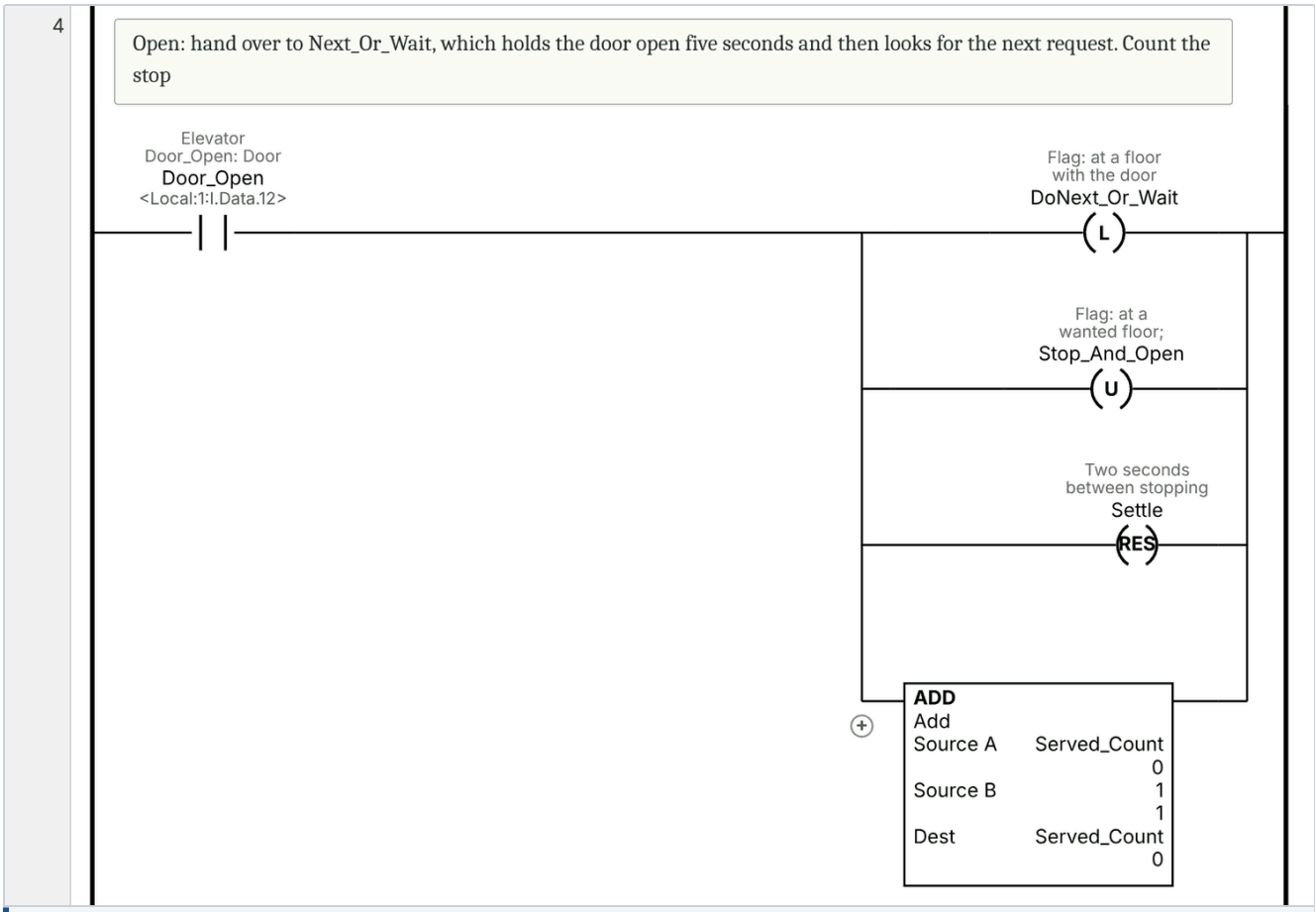
HOW THIS RUNG WORKS

In words. When either (`Settle.DN` (the done bit of `Settle` : Two seconds between stopping and opening the door) is on) *or* (`Door_Closed` (Door fully closed (end switch)) is off) and `Door_Open` (Door fully open (end switch)) is off, the rung turns `Open` (Door motor: open) on (and off again whenever the rung is false).

Left to right:

1. A branch of 2 parallel paths: power gets through if any one of them is true.
   - **Path 1**
     1. `XIC(Settle.DN)` *Examine If Closed* asks: true when `Settle.DN` (the done bit of `Settle` : Two seconds between stopping and opening the door) is on.
   - **Path 2**
     1. `XIO(Door_Closed)` *Examine If Open* asks: true when `Door_Closed` (Door fully closed (end switch)) is off.
2. `XIO(Door_Open)` *Examine If Open* asks: true when `Door_Open` (Door fully open (end switch)) is off.
3. `OTE(Open)` *Output Energize* does: turns `Open` (Door motor: open) on (and off again whenever the rung is false).

STOP_AND_OPEN_DOOR · RUNG 4




```
XIC(Door_Open)
[OTL(DoNext_Or_Wait),OTU(Stop_And_Open),RES(Settle),ADD(Served_Count,1,Served_C
ount)];
```

Open: hand over to Next_Or_Wait, which holds the door open five seconds and then looks for the next request. Count the stop

HOW THIS RUNG WORKS

In words. When `Door_Open` (Door fully open (end switch)) is on, the rung latches `DoNext_Or_Wait` (Flag: at a floor with the door open; Next_Or_Wait is scanned and decides where to go next) on, unlatches `Stop_And_Open` (Flag: at a wanted floor; Stop_And_Open_Door stops the hoist, settles and opens) (turns it off), resets `Settle` (Two seconds between stopping and opening the door) and makes `Served_Count` (Shown on the toolbar: door cycles) = `Served_Count` + **1**.

Left to right:

1. `XIC(Door_Open)` *Examine If Closed* asks: true when `Door_Open` (Door fully open (end switch)) is on.
2. A branch of 4 parallel paths: each path is tested on its own and does its own work.

   Path 1

   1. `OTL(DoNext_Or_Wait)` *Output Latch* does: latches `DoNext_Or_Wait` (Flag: at a floor with the door open; Next_Or_Wait is scanned and decides where to go next) on.

   Path 2

1. `OTU(Stop_And_Open)` *Output Unlatch* does: unlatches `Stop_And_Open` (Flag: at a wanted floor; Stop_And_Open_Door stops the hoist, settles and opens) (turns it off).

Path 3

1. `RES(Settle)` *Reset* does: resets `Settle` (Two seconds between stopping and opening the door).

Path 4

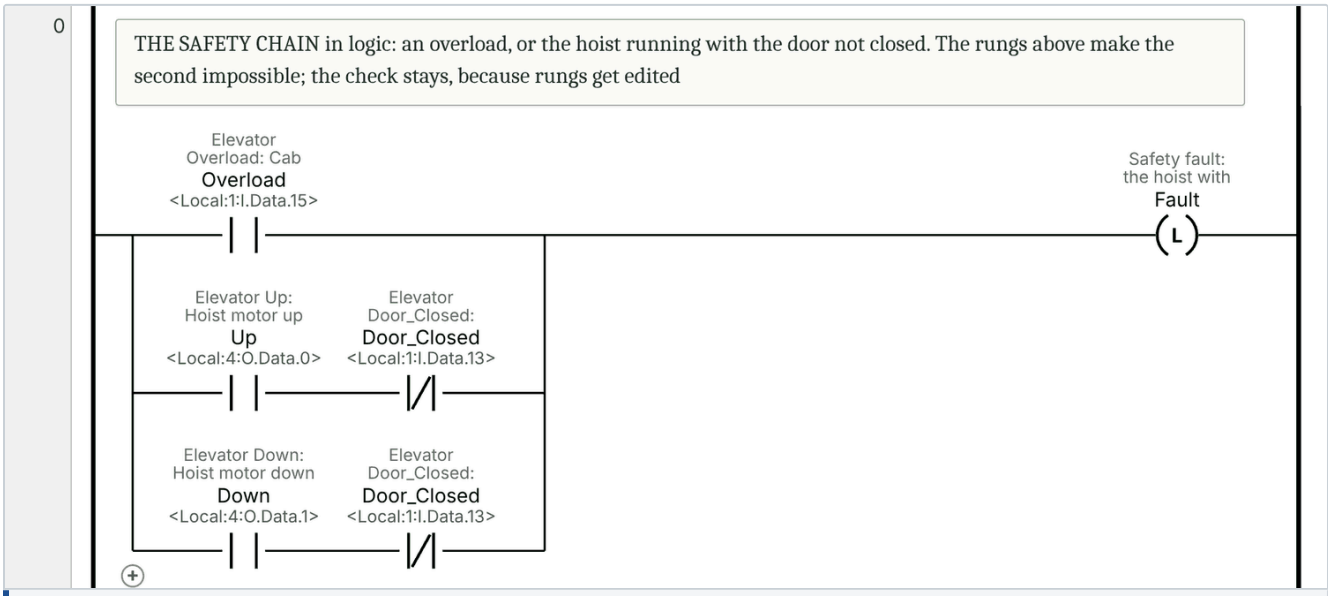
1. `ADD(Served_Count,1,Served_Count)` *Add* does: makes `Served_Count` (Shown on the toolbar: door cycles) = `Served_Count` + **1**. Every scan the rung is true; behind a one-shot it adds once per event.

Why it is built this way:

A latch: it stays on until an OTU clears it.

Exercise 4: the safety chain, the counts, the lamps

SAFETY · RUNG 0




```
[XIC(Overload),XIC(Up)XIO(Door_Closed),XIC(Down)XIO(Door_Closed)]OTL(Fault);
```

THE SAFETY CHAIN in logic: an overload, or the hoist running with the door not closed. The rungs above make the second impossible; the check stays, because rungs get edited

PREDICT IT · 17 `Overload` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When either (`Overload` (Cab overload switch (too many passengers)) is on) *or* (`Up` (Hoist motor up) is on and `Door_Closed` (Door fully closed (end switch)) is off) *or* (`Down` (Hoist motor down (never with Up: the contactors are interlocked)) is on and `Door_Closed` is off), the rung latches `Fault` (Safety fault: the hoist with the door not closed, or an overload; cleared by Reset) on.

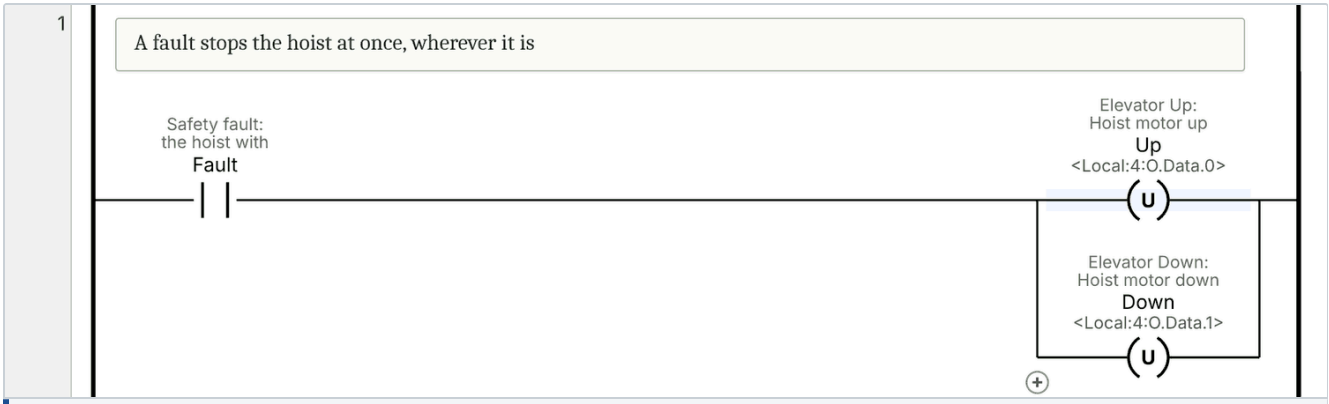
Left to right:

1. A branch of 3 parallel paths: power gets through if any one of them is true.
   - **Path 1**
     1. `XIC(Overload)` *Examine If Closed* asks: true when `Overload` (Cab overload switch (too many passengers)) is on.
   - **Path 2**
     1. `XIC(Up)` *Examine If Closed* asks: true when `Up` (Hoist motor up) is on.
     2. `XIO(Door_Closed)` *Examine If Open* asks: true when `Door_Closed` (Door fully closed (end switch)) is off.
   - **Path 3**
     1. `XIC(Down)` *Examine If Closed* asks: true when `Down` (Hoist motor down (never with Up: the contactors are interlocked)) is on.
     2. `XIO(Door_Closed)` *Examine If Open* asks: true when `Door_Closed` is off.
2. `OTL(Fault)` *Output Latch* does: latches `Fault` (Safety fault: the hoist with the door not closed, or an overload; cleared by Reset) on.

Why it is built this way:

A latch: it stays on until an OTU clears it.

SAFETY · RUNG 1




```
XIC(Fault)[OTU(Up),OTU(Down)];
```

A fault stops the hoist at once, wherever it is

HOW THIS RUNG WORKS

In words. When `Fault` (Safety fault: the hoist with the door not closed, or an overload; cleared by Reset) is on, the rung unlatches `Up` (Hoist motor up) (turns it off) and unlatches `Down` (Hoist motor down (never with Up: the contactors are interlocked)) (turns it off).

Left to right:

1. `XIC(Fault)` *Examine If Closed* asks: true when `Fault` (Safety fault: the hoist with the door not closed, or an overload; cleared by Reset) is on.
2. A branch of 2 parallel paths: each path is tested on its own and does its own work.

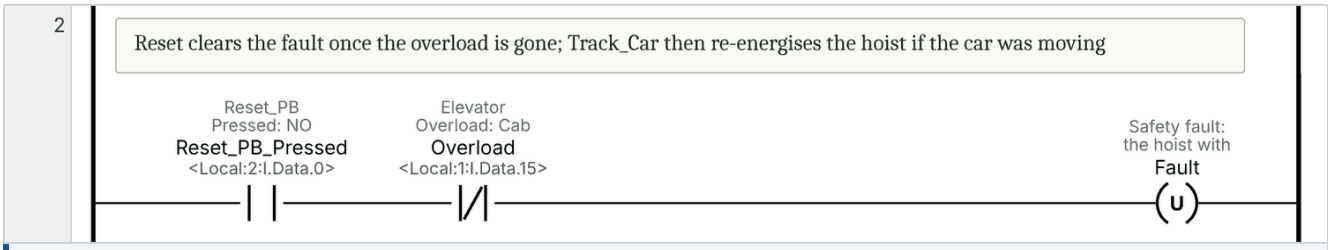
   Path 1

   1. `OTU(Up)` *Output Unlatch* does: unlatches `Up` (Hoist motor up) (turns it off).

   Path 2

   1. `OTU(Down)` *Output Unlatch* does: unlatches `Down` (Hoist motor down (never with Up: the contactors are interlocked)) (turns it off).

SAFETY · RUNG 2




```
XIC(Reset_PB_Pressed)XIO(Overload)OTU(Fault);
```

Reset clears the fault once the overload is gone; Track_Car then re-energises the hoist if the car was moving

PREDICT IT · 18 `Reset_PB_Pressed` is off and `Overload` is off. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

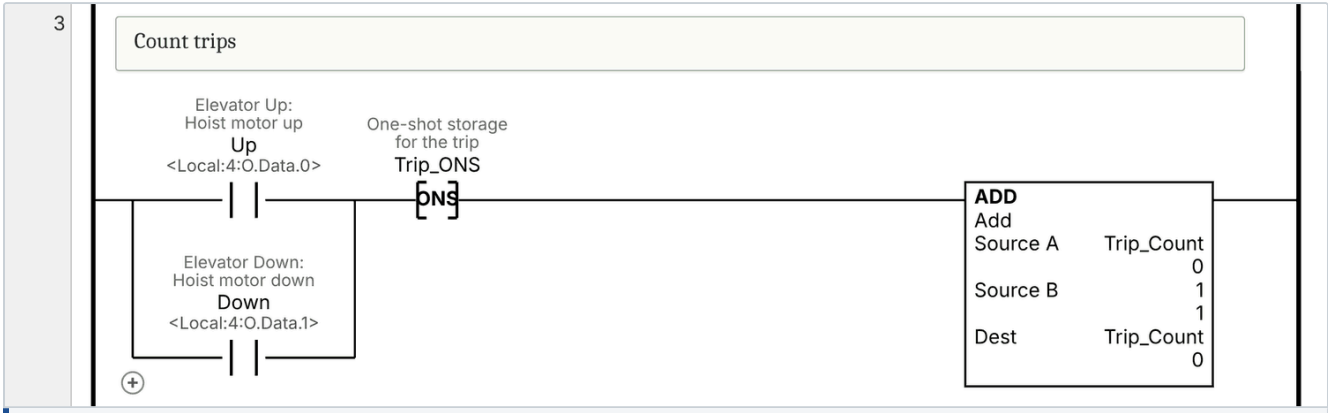
HOW THIS RUNG WORKS

In words. When `Reset_PB_Pressed` (NO contact: 1 while pressed) is on and `Overload` (Cab overload switch (too many passengers)) is off, the rung unlatches `Fault` (Safety fault: the hoist with the door not closed, or an overload; cleared by Reset) (turns it off).

Left to right:

1. `XIC(Reset_PB_Pressed)` *Examine If Closed* asks: true when `Reset_PB_Pressed` (NO contact: 1 while pressed) is on.
2. `XIO(Overload)` *Examine If Open* asks: true when `Overload` (Cab overload switch (too many passengers)) is off.
3. `OTU(Fault)` *Output Unlatch* does: unlatches `Fault` (Safety fault: the hoist with the door not closed, or an overload; cleared by Reset) (turns it off).

SAFETY · RUNG 3




```
[XIC(Up),XIC(Down)]ONS(Trip_ONS)ADD(Trip_Count,1,Trip_Count);
```

Count trips

HOW THIS RUNG WORKS

In words. When either (`Up` (Hoist motor up) is on) *or* (`Down` (Hoist motor down (never with Up: the contactors are interlocked)) is on) and it is the first scan they are true (the one-shot, remembered in `Trip_ONS` (One-shot storage for the trip counter)), the rung makes `Trip_Count` (Shown on the toolbar: trips) = `Trip_Count` + **1**.

Left to right:

1. A branch of 2 parallel paths: power gets through if any one of them is true.
   - **Path 1**
     1. `XIC(Up)` *Examine If Closed* asks: true when `Up` (Hoist motor up) is on.
   - **Path 2**
     1. `XIC(Down)` *Examine If Closed* asks: true when `Down` (Hoist motor down (never with Up: the contactors are interlocked)) is on.
2. `ONS(Trip_ONS)` *One Shot* asks: true when it is the first scan they are true (the one-shot, remembered in `Trip_ONS` (One-shot storage for the trip counter)). It passes power for one scan only, the scan its input goes from false to true; its storage bit remembers last scan.
3. `ADD(Trip_Count,1,Trip_Count)` *Add* does: makes `Trip_Count` (Shown on the toolbar: trips) = `Trip_Count` + **1**.

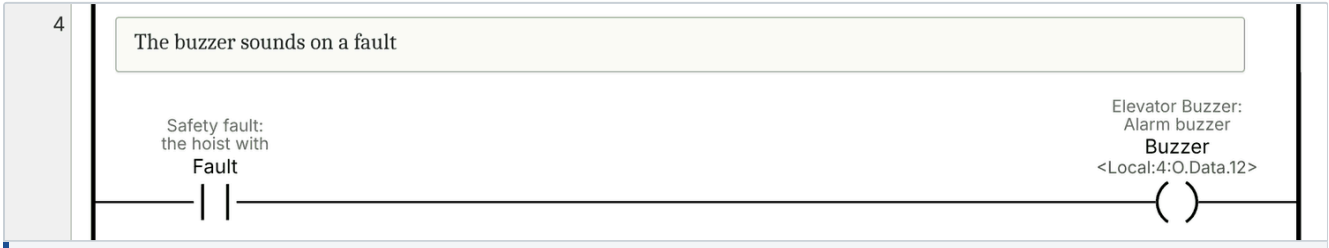
Why it is built this way:

One-shot. Everything after the ONS happens once per event, not on every scan the condition holds; `Trip_ONS` is only its memory of the last scan.

> **Think of it like this:** a doorbell that rings once however long you lean on the button. The ONS looks at what the button was doing last scan: only the change from up to down rings it.

> **Watch for:** Each ONS needs its own storage bit. Two ONS sharing one bit make one of them miss its edges.

SAFETY · RUNG 4




```
XIC(Fault)OTE(Buzzer);
```

The buzzer sounds on a fault

HOW THIS RUNG WORKS

In words. When `Fault` (Safety fault: the hoist with the door not closed, or an overload; cleared by Reset) is on, the rung turns `Buzzer` (Alarm buzzer) on (and off again whenever the rung is false).

Left to right:

1. `XIC(Fault)` *Examine If Closed* asks: true when `Fault` (Safety fault: the hoist with the door not closed, or an overload; cleared by Reset) is on.
2. `OTE(Buzzer)` *Output Energize* does: turns `Buzzer` (Alarm buzzer) on (and off again whenever the rung is false).

SAFETY · RUNG 5

```
XIC(Fault)OTE(Lamp_Fault_On);
```

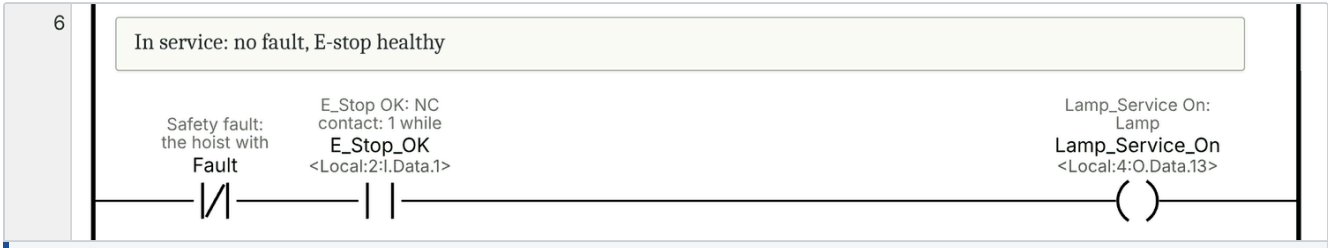
HOW THIS RUNG WORKS

In words. When `Fault` (Safety fault: the hoist with the door not closed, or an overload; cleared by Reset) is on, the rung turns `Lamp_Fault_On` (Lamp) on (and off again whenever the rung is false).

Left to right:

1. `XIC(Fault)` *Examine If Closed* asks: true when `Fault` (Safety fault: the hoist with the door not closed, or an overload; cleared by Reset) is on.
2. `OTE(Lamp_Fault_On)` *Output Energize* does: turns `Lamp_Fault_On` (Lamp) on (and off again whenever the rung is false).

SAFETY · RUNG 6




```
XIO(Fault)XIC(E_Stop_OK)OTE(Lamp_Service_On);
```

In service: no fault, E-stop healthy

PREDICT IT · 19 `Fault` is on and `E_Stop_OK` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Fault` (Safety fault: the hoist with the door not closed, or an overload; cleared by Reset) is off and `E_Stop_OK` (NC contact: 1 while healthy, 0 when pressed) is on, the rung turns `Lamp_Service_On` (Lamp) on (and off again whenever the rung is false).

Left to right:

1. `XIO(Fault)` *Examine If Open* asks: true when `Fault` (Safety fault: the hoist with the door not closed, or an overload; cleared by Reset) is off.
2. `XIC(E_Stop_OK)` *Examine If Closed* asks: true when `E_Stop_OK` (NC contact: 1 while healthy, 0 when pressed) is on.
3. `OTE(Lamp_Service_On)` *Output Energize* does: turns `Lamp_Service_On` (Lamp) on (and off again whenever the rung is false).

Exercise 5: multi-floor

The exercise's last and hardest part is already in the routines above, and it is worth seeing where. Collective control means: keep going in the direction of travel while anything is wanted that way, stop at every wanted floor on the way, turn round only when nothing is left ahead, and rest when nothing is wanted at all. The direction is the latch pair in Next_Or_Wait; the stop on the way is Track_Car's third rung, which only looks at lamps in the direction of travel; the turn-round is the unlatch rungs, which release a direction only when nothing is left that way; the rest is both flags off with the door open. Run it, click the call plates in awkward orders, and watch the displays.

Guided: the elevator

From Version 133 the scene opens with its guided track on (**File → New Project → Classic: elevator**; or **Guided → Guide This Scene: the Elevator** on a crossing whose guide was closed). Eight lessons, one routine each and in the order above rather than the index's: **Init** (the first scan opens the door), **Stop_And_Open_Door** (the car stops, the door opens, the passenger steps in), **Catch_Requests** (a hall call

lights its plate and makes a request), **Next_Or_Wait** (a direction is picked and Close_And_Go set), **Close_And_Move** (the door is proved closed before the hoist), **Track_Car** (the car travels, arrives and levels), **Safety** (the service switch trips the fault) and **Ready**. Each lesson adds its JSR to the index with the condition the reference has, `XIC(Car_Moving)JSR(Track_Car)`, not a bare JSR, because the index's conditions are the state machine: an Init called every scan would open the door forever. The reference rungs sit under the fold with hints first, so write each from its comment and ask for the shape only when the comment is not enough; the run checks are the ones a commissioning engineer would make with a hand on the call plate.

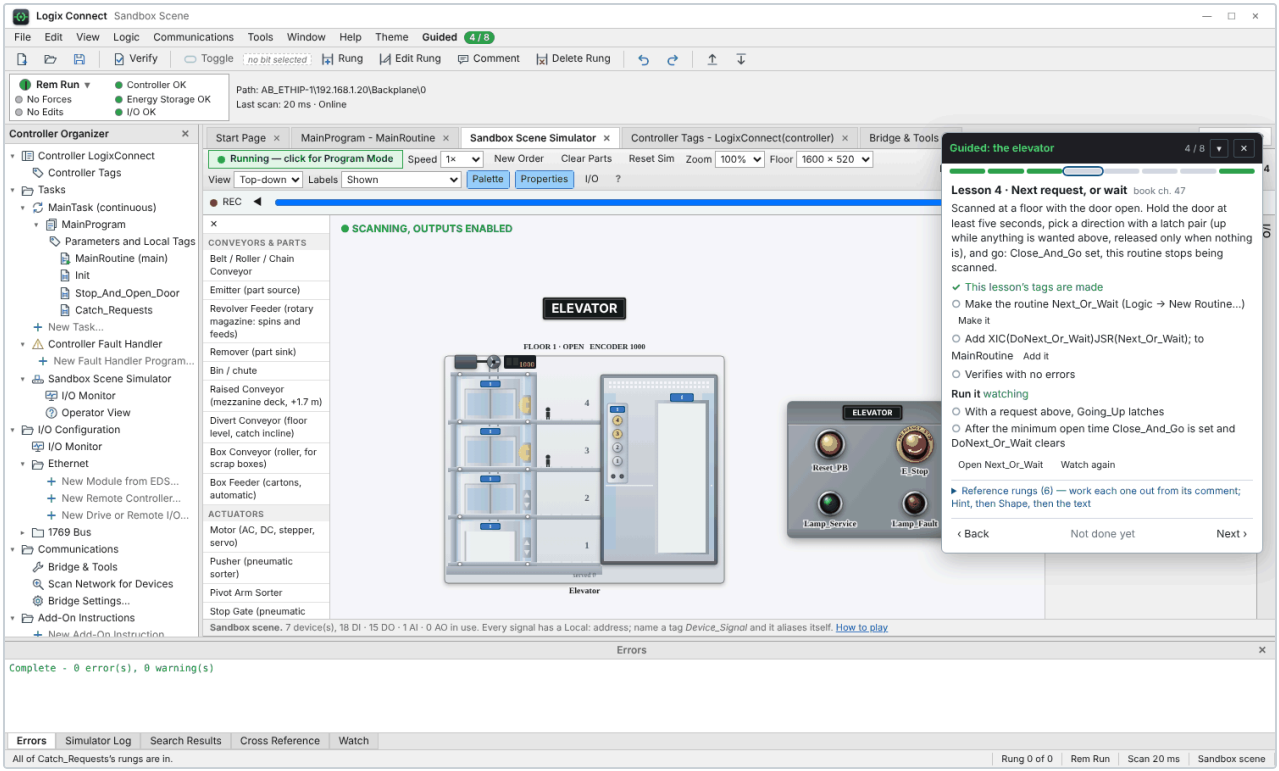


Figure 207. Lesson 4 of the elevator track: three routines in, a hall call on floor 3, the guide waiting for Going_Up to latch.

> **WATCH FOR: THE LESSON ORDER IS NOT THE SCAN ORDER**
>
> The track writes Stop_And_Open_Door second and Track_Car sixth, because a door that opens is something you can see on the first scan and a car that moves is not worth having until there is somewhere to send it. The index still runs them in the reference's order. If a lesson's run check never turns green, open the Simulator Log: the most common cause is the door check, Door_Closed, failing because the previous lesson's rung was typed while the controller was in Run and is still a pending edit.

Run it

Run mode: the car settles two seconds, opens at floor 1 and waits. Watch the first passenger: the call plate on their floor lights (the lamp latched), the car closes its door after the minimum open time, the hoist lamp over the shaft comes on, the sheave turns and the encoder counts up, the displays follow the floor, the car stops level, settles, opens; the eye flickers as they board and press a cab button; the lamp for that floor lights. Let it run five minutes and look at the count under the shaft: served and, if the car is slow, lost.

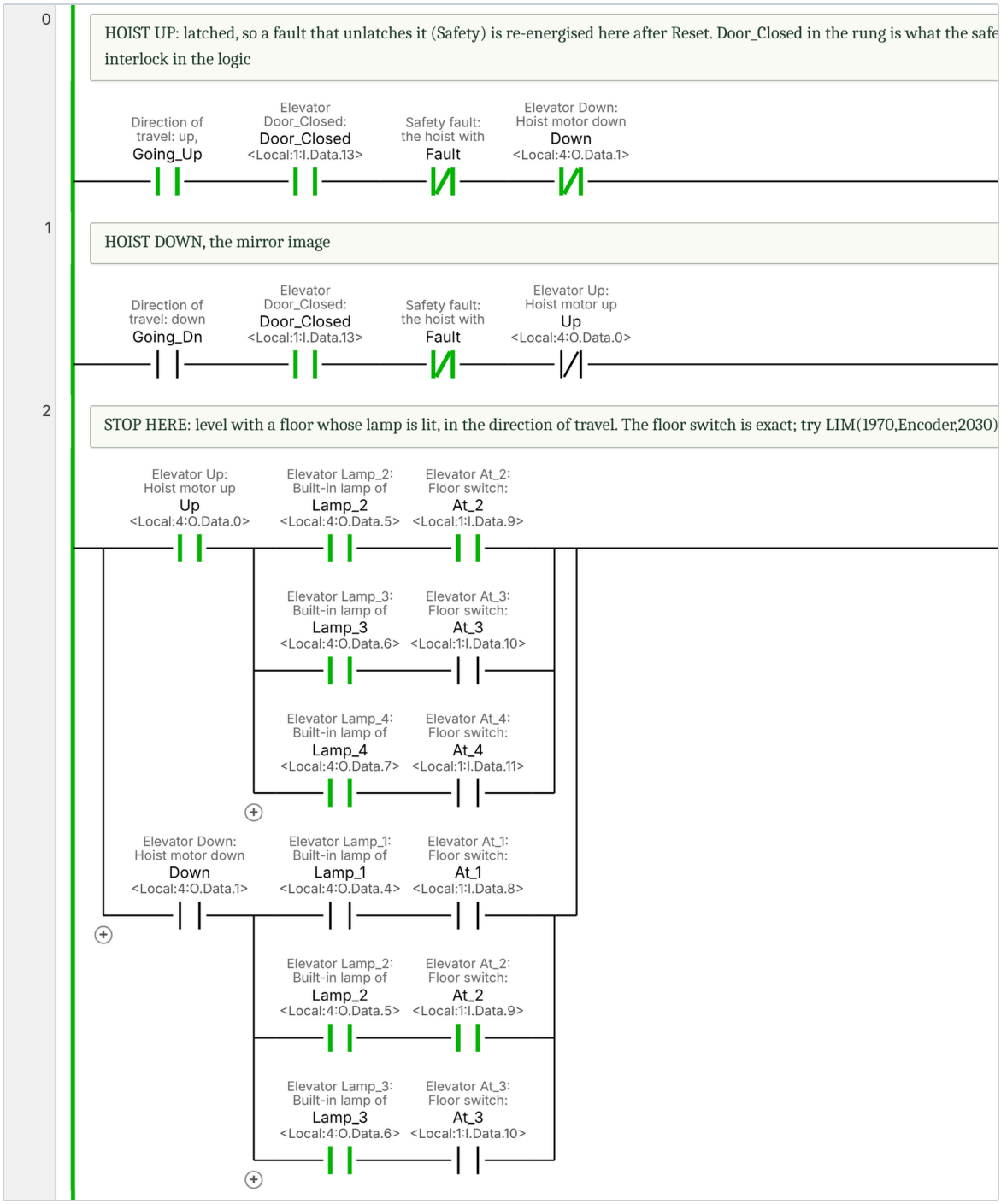


Figure 208. Track_Car online, the car travelling up to a lit floor.

TRY IT

Set *Seconds between passengers* to 4 and run: the collective control starts losing people, because a passenger on 1 going to 4 waits while the car serves everything between. Then the harder one: give each hall call a direction, two buttons per floor, and stop for a hall call only when the car is going the way that passenger wants. That is a full collective, and it is the program a real elevator runs. And the encoder exercise as set: delete the four floor switches from Track_Car's stop rung and stop on `LIM(1970,Encoder,2030)` windows instead, then find the widest window that still stops the car inside the levelling zone.

Machine faults on the elevator

Tools → Inject Machine Fault… lists the faults that belong to the parts on this floor. Each has a fault card: the symptom, what to watch in the I/O to confirm it, the repair, and what a program should do about it. **Repair** on an active fault removes its cause the way a technician would; whether the machine runs again is up to the program. None of them is caught by the reference program as it stands: the door-close timeout, the no-motion timeout and the encoder cross-check are the exercises.

Fault	Watch	Repair	The program should
Elevator: door-closed switch stuck open The doors shut but the door-closed switch never makes, so the car never proves its doors closed.	Close 1, the doors shut, Door_Closed 0; the car never leaves.	Adjust the switch (Repair).	A door-close timeout: Close on for six seconds without Door_Closed reopens the door and alarms.
Elevator: encoder dead The shaft encoder stops counting: the car moves, the count does not.	Up or Down 1, the At_ floor switches changing, Encoder frozen.	Replace the encoder (Repair).	The floor from the encoder disagreeing with an At_ switch stops the car and alarms.
Elevator: hoist overload tripped The hoist motor's overload trips: Up and Down are commanded and nothing turns.	Up (or Down) 1, Encoder not changing.	Reset the overload (Repair).	A no-motion timeout: Up or Down on and the encoder not moving for two seconds drops the hoist and alarms.
Elevator: floor 2 call button stuck in The floor 2 hall button is jammed in.	Call_2 1 with nobody there; the car keeps going back to floor 2.	Free the button (Repair).	A call input on for more than ten seconds is ignored and alarmed: no real passenger holds a button that long.

Predict it: the answers

1. Every condition on the way is met, so `Floor` becomes **3**.
2. Every condition on the way is met, so `Ind_1` is **off** (an OTE writes 0 when its rung is false), `Ind_2` is **off** (an OTE writes 0 when its rung is false), `Ind_3` is **off** (an OTE writes 0 when its rung is false) and `Ind_4` is **on**.
3. Every condition on the way is met, so `Lamp_1` latches **on**.

4. The rung is false at the NEQ: `Floor` equals **2**, so the OTL does nothing: `Lamp_2` keeps whatever it had.
5. Every condition on the way is met, so `Lamp_3` latches **on**.
6. The rung is false at the NEQ: `Floor` equals **4**, so the OTL does nothing: `Lamp_4` keeps whatever it had.
7. Every condition on the way is met, so `Req_Above` is **on**.
8. Every condition on the way is met, so `Req_Below` is **on**.
9. The rung is false at `XIO(Going_Dn)`: `Going_Dn` is on, so the OTL does nothing: `Going_Up` keeps whatever it had.
10. The rung is false at `XIC(Req_Below)`: `Req_Below` is off, so the OTL does nothing: `Going_Dn` keeps whatever it had.
11. Every condition on the way is met, so `Close` is **on**.
12. The rung is false at `XIC(Door_Closed)`: `Door_Closed` is off, so the OTL does nothing: `Up` keeps whatever it had.
13. Every condition on the way is met, so `Down` latches **on**.
14. Every condition on the way is met, so `Stop_And_Open` latches **on** and `Car_Moving` is unlatched, **off**.
15. Every condition on the way is met, so `Lamp_4` is unlatched, **off**.
16. The rung is false at `XIO(Door_Open)`: `Door_Open` is on, so `Open` is **off** (an OTE writes 0 when its rung is false).
17. Every condition on the way is met, so `Fault` latches **on**.
18. The rung is false at `XIC(Reset_PB_Pressed)`: `Reset_PB_Pressed` is off, so the OTU does nothing: `Fault` keeps whatever it had.
19. The rung is false at `XIO(Fault)`: `Fault` is on, so `Lamp_Service_On` is **off** (an OTE writes 0 when its rung is false).

48 Classic scenes: the traffic intersection

The traffic light is the other exercise every simulator has, and the reason is not the lamps: six outputs in a fixed order is a morning's work. The reason is everything around them. A pedestrian button that must be remembered until the right moment and then forgotten. Loops in the road that turn a fixed cycle into one that answers to traffic. An amber that is never shortened, an all-red that has to be long enough for the last car to clear, a WALK that has to end early enough for the slowest walker, and a box in the cabinet that will take the whole intersection away from your program the instant it shows two greens. From Version 126 Logix Connect has the street for it, under **File → New Project → Classic: traffic intersection**: a real crossing on grass, with cars and pedestrians who do what cars and pedestrians do. The scene comes with its tags made and an empty routine. The program below is the reference (**File → Load Reference Solution: Traffic intersection**), built in the order the classic exercise asks: the timed cycle, then the walk signs, then the loops, then flash.

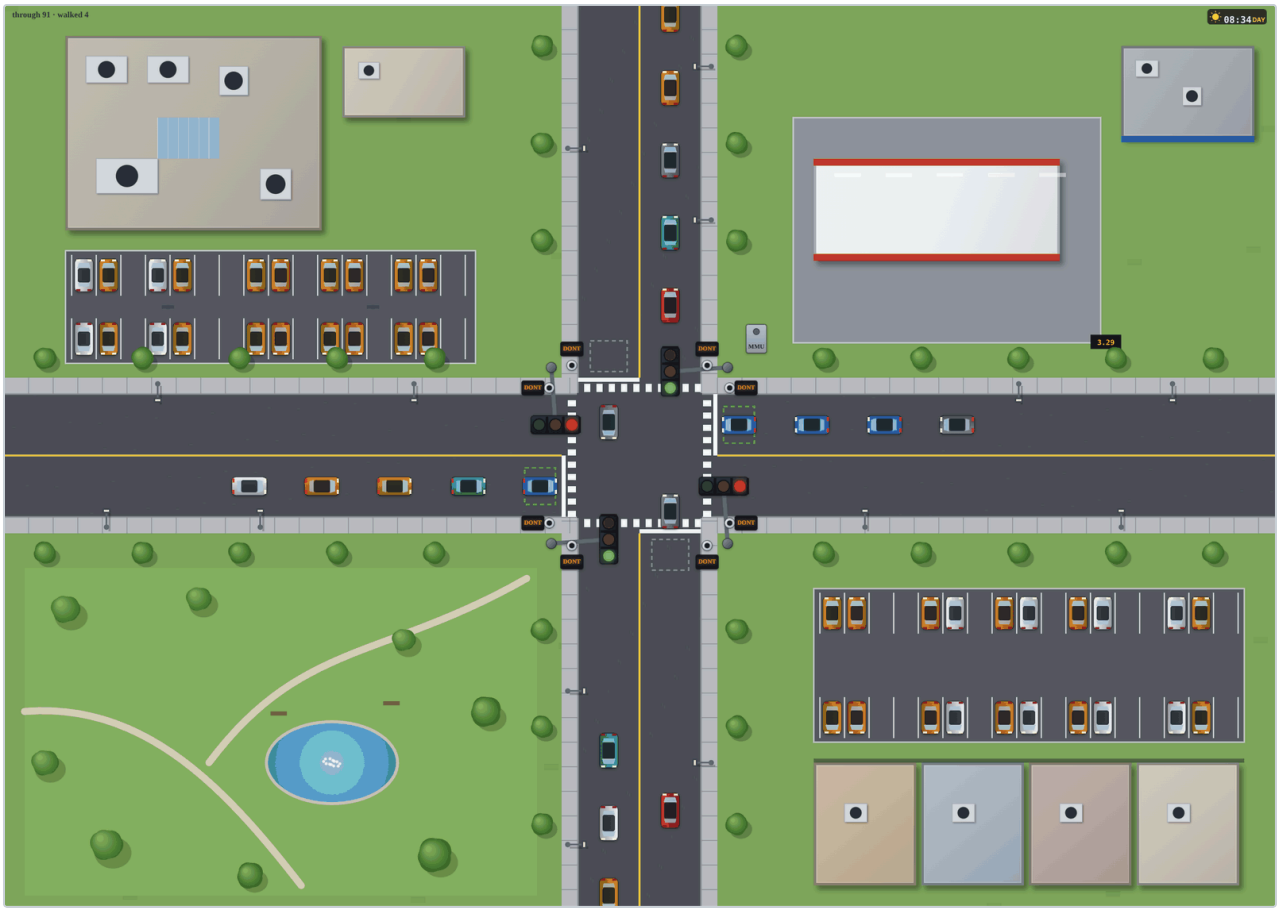


Figure 209. The Intersection part. Two roads with their kerbs, sidewalks, centre lines, stop lines and zebra crossings; a pole with a mast arm and a head over every approach (here N/S amber, E/W red); WALK / DON'T WALK signs and a button post at every crossing end; the dashed loops in the approach lanes, green when a car is on them; the cabinet with its conflict-monitor lamp; cars queued and crossing, a pedestrian waiting at a corner. Since Version 139 the roads run out to the next blocks: the office block and its car park, the filling station, the park with its pond, the row of shops, street trees and street lights. The count at the top left (cars through, pedestrians walked, conflicts), the clock at the top right.

The street

The *Intersection* part is under **Classic scenes**. Its properties are the traffic: cars per minute on each approach (16 in the scene), whether pedestrians arrive by themselves and how often, whether the cabinet has a conflict monitor, *Long roads* (on in a new scene; a scene saved before Version 139 keeps the short approaches it was built with), and the day: *Day length* and *Start hour*. Its I/O is what a signal cabinet wires, and the N/S heads share outputs the way the two heads for one road share a load switch:

Signal		Meaning
`NS_Car`	input	A car on either N/S approach loop (northbound or southbound)
`EW_Car`	input	A car on either E/W approach loop
`NS_Ped_PB`	input	Pedestrian button to cross the E/W road (walk along N/S), 1 while pressed

Signal		Meaning
`EW_Ped_PB`	input	Pedestrian button to cross the N/S road (walk along E/W), 1 while pressed
`MMU_Fault`	input	Conflict monitor tripped: the cabinet is in red flash until the MMU is reset (click the cabinet)
`NS_Red`	output	N/S heads: red
`NS_Yellow`	output	N/S heads: amber
`NS_Green`	output	N/S heads: green
`EW_Red`	output	E/W heads: red
`EW_Yellow`	output	E/W heads: amber
`EW_Green`	output	E/W heads: green
`NS_Walk`	output	WALK for the crossings along N/S (over the E/W road)
`NS_DontWalk`	output	DON'T WALK for those crossings
`EW_Walk`	output	WALK for the crossings along E/W (over the N/S road)
`EW_DontWalk`	output	DON'T WALK for those crossings

NS means along N/S. `NS_Green` is the green for cars travelling north and south; `NS_Walk` is the WALK for pedestrians travelling north and south, who cross the E/W road, and so needs the E/W road red. That is how the original exercise names them and it is the only naming that lets the conflict rule be stated in one line: a WALK may never be shown with the green of the road it crosses.

The cars are the part's physics, not programmable. They arrive at random, queue behind each other, stand on the loops, go on green and stop on red. On amber they make one decision when it appears, as a driver does: a car close to the line at speed goes through, the rest stop, and the decision is kept. A car that is through the stop line is committed and crosses whatever the lamps do. Two cars in the box from both roads at once is a collision: logged, counted, and only the program can cause it. The pedestrians press once (0.4 s), again after twenty seconds, and give up after a minute; a WALK for their crossing starts them, and kerb to kerb takes six seconds whatever the sign does meanwhile. A car through the crosswalk while they are on it is counted the same way as a collision.

The **conflict monitor** is the part of a real cabinet most people have never heard of. It is a separate box, the MMU, that watches the load switches, not the controller, and if it ever sees green or amber on both roads, or a WALK against the green that crosses it, for a third of a second, it drops the whole intersection into red flash in hardware and holds it there, whatever the controller says, until a technician resets it at the cabinet. The part does exactly that: `MMU_Fault` goes to 1, the heads flash red regardless of your outputs, and a click on the cabinet resets it. Turn it off in the properties if you want to see what your mistake would have done.

Three views, day and night

Like every scene, the crossing has three views. The side view stands on the south side of Main St and looks north up 1st Ave: a low oblique elevation in which x is the plan's x and a thing further north stands a little higher, so the cross street runs across the picture, the avenue's cars stand one behind the other up it, the northbound heads face you with their three lamps and the others are seen from the side or behind. The near side of Main St is cut away so nothing hides the crossing. The cabinet and the walk buttons work in it as they do in the plan.

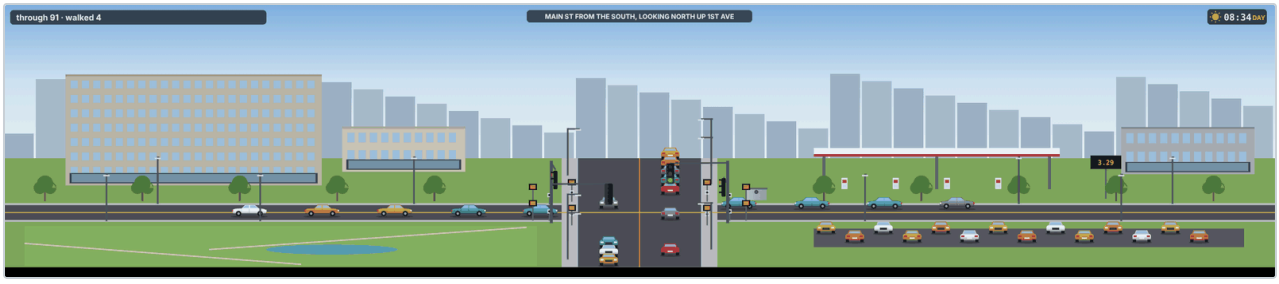


Figure 210. The side view: Main St from the south, looking north up 1st Ave. Cars on Main St in profile, the avenue's queues standing up the picture, the heads, walk signs and buttons on their corners, the office block, the filling station and the shops behind; the park and the car park flat in front.

The day runs on the run's clock, as the parking gate's does: 24 minutes of run time make a whole day by default, twelve of daylight and twelve of night, with a dusk and a dawn between. At night the lamps carry the picture: every lit signal lamp throws its colour on the road, the walk signs glow, the street lights pool on the pavement, the filling station's canopy and the shopfronts light, and every car runs with its headlights (their beams cut short by the car in front) and its tail lights, bright brake lights while it waits at the line. In 3D the sky goes to night blue, the office and shop windows light and the lamps come on. None of it changes what the program sees.

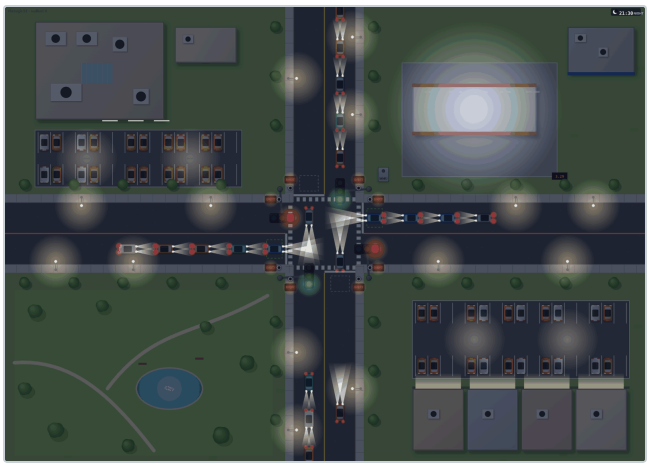


Figure 211. At night from above: queues at the red with their brake lights on, the green approach's headlights reaching across the box, street lights pooling on the sidewalks, the filling station lit.

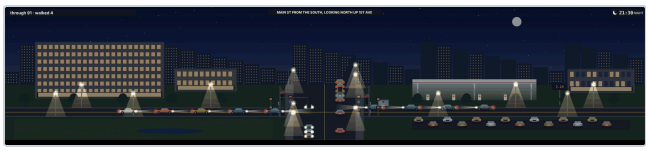


Figure 212. At night from the side: the lit windows, the street lights, the heads, the cars' lights.

Figure 213. In 3D at night: the windows of the office block and the shops, the filling station canopy, the street lamps and the cars' lights.

TRY IT: RUSH HOUR AT DUSK

Set *Cars per minute* to 30 and *Start hour* to 17, and run your program. Watch the queues on the long approaches: a fixed cycle that was fine at 8 cars a minute leaves cars waiting through two greens now, and the loops are what fix it. Then switch to the side view and watch the northbound queue stand up the avenue.

WATCH FOR: A SAVED SCENE KEEPS ITS ROADS

A traffic scene saved before Version 139 opens with the short approaches it was built with, so its console and everything around it stay where they were. Tick *Long roads* in the part's properties to have the blocks, and move the console clear of them.

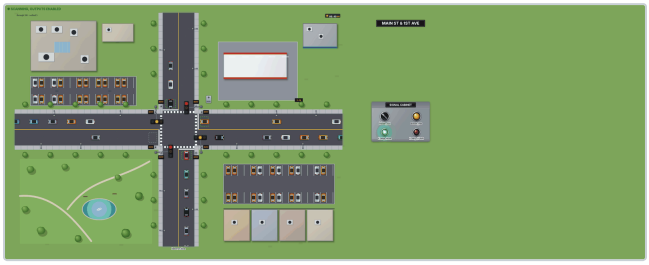


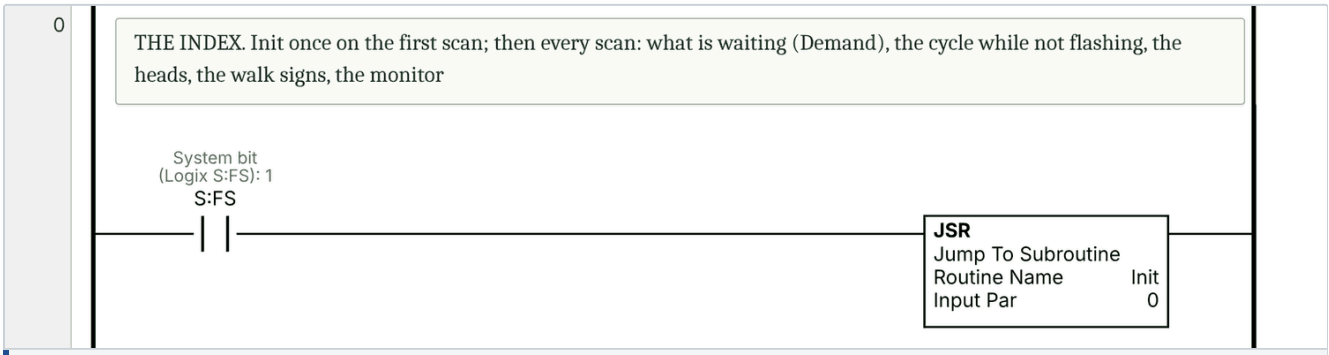
Figure 214. The scene: the crossing on its lawn and the cabinet console, AUTO / FLASH selector, Reset, run and fault lamps.

Figure 215. In 3D: the blocks around the crossing, the poles, mast arms and lit heads, the signs and button posts, the loops, cars with their lights, pedestrians on the corners.

The program's index

As with the elevator, the main routine is a table of contents. Two routines run every scan regardless (demand, and the monitor), the heads and the signs follow the step, and the cycle itself is held while the intersection flashes:

MAINROUTINE · RUNG 0




```
XIC(S:FS)JSR(Init);
```

THE INDEX. Init once on the first scan; then every scan: what is waiting (Demand), the cycle while not flashing, the heads, the walk signs, the monitor

HOW THIS RUNG WORKS

In words. When `S:FS` is on, the rung runs the routine `Init`.

Left to right:

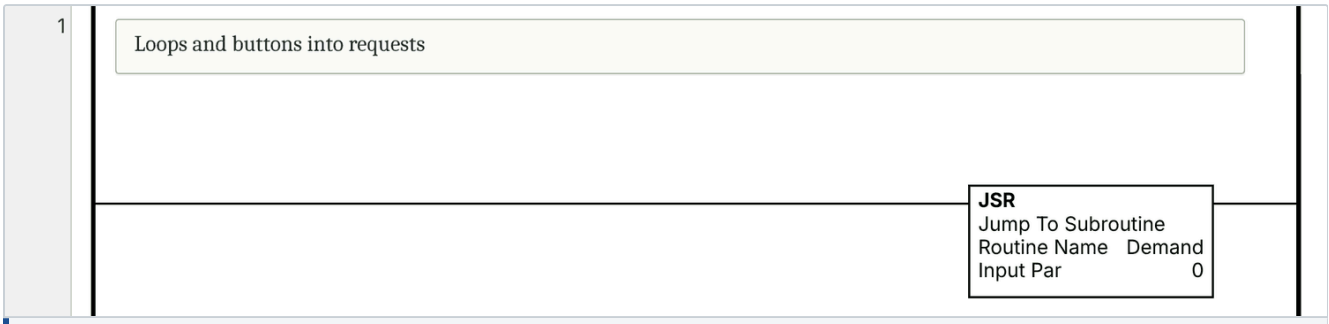
1. `XIC(S:FS)` *Examine If Closed* asks: true when `S:FS` is on.
2. `JSR(Init)` *Jump to Subroutine* does: runs the routine `Init`. The routine is scanned at this point, then the scan comes back here.

Why it is built this way:

Part of the index. The main routine is a list of calls; each routine holds one job, and the scan goes through them in this order.

> **Think of it like this:** the table of contents of a book. The main routine does no work itself: it lists the chapters and the order to read them in, every scan.

MAINROUTINE · RUNG 1




```
JSR(Demand);
```

Loops and buttons into requests

HOW THIS RUNG WORKS

In words. Every scan, with nothing to test, the rung runs the routine `Demand`.

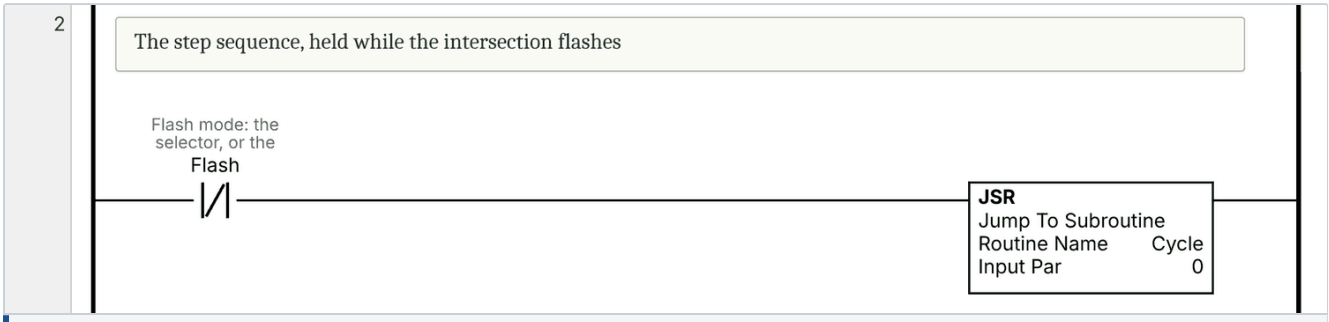
Left to right:

1. `JSR(Demand)` *Jump to Subroutine* does: runs the routine `Demand`.

Why it is built this way:

Part of the index: one call, one job.

MAINROUTINE · RUNG 2




```
XIO(Flash)JSR(Cycle);
```

The step sequence, held while the intersection flashes

HOW THIS RUNG WORKS

In words. When `Flash` (Flash mode: the selector, or the conflict monitor has tripped) is off, the rung runs the routine `Cycle`.

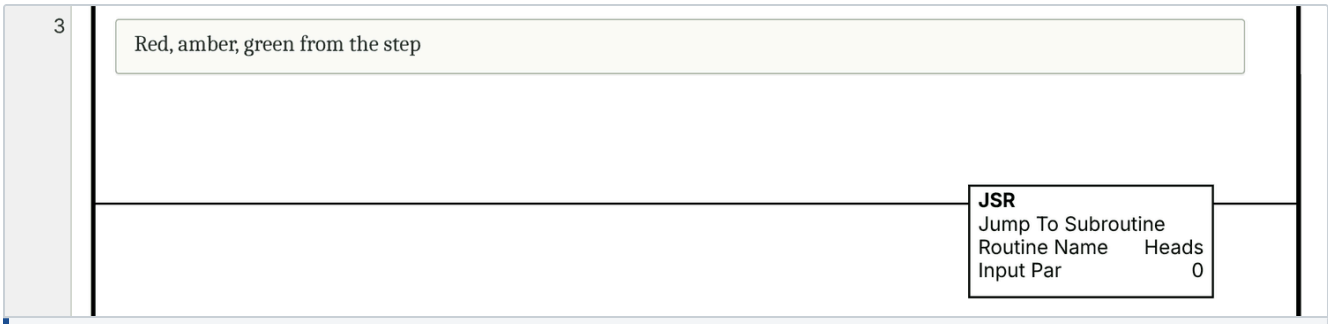
Left to right:

1. `XIO(Flash)` *Examine If Open* asks: true when `Flash` (Flash mode: the selector, or the conflict monitor has tripped) is off.
2. `JSR(Cycle)` *Jump to Subroutine* does: runs the routine `Cycle`.

Why it is built this way:

Part of the index: one call, one job.

MAINROUTINE · RUNG 3




```
JSR(Heads);
```

Red, amber, green from the step

HOW THIS RUNG WORKS

In words. Every scan, with nothing to test, the rung runs the routine `Heads`.

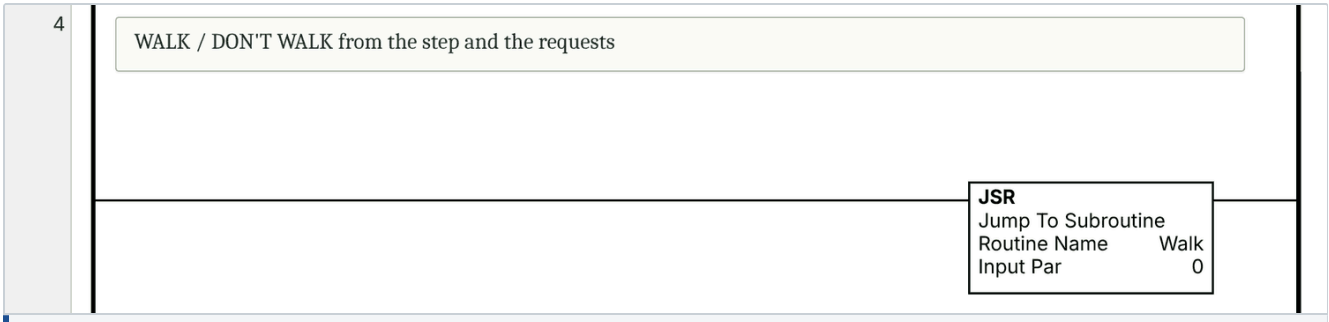
Left to right:

1. `JSR(Heads)` *Jump to Subroutine* does: runs the routine `Heads`.

Why it is built this way:

Part of the index: one call, one job.

MAINROUTINE · RUNG 4




```
JSR(Walk);
```

WALK / DON'T WALK from the step and the requests

HOW THIS RUNG WORKS

In words. Every scan, with nothing to test, the rung runs the routine `Walk`.

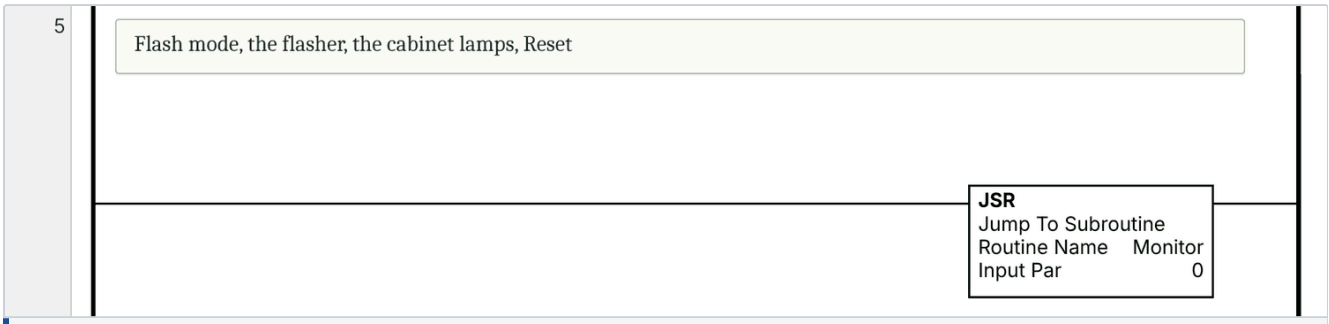
Left to right:

1. `JSR(Walk)` *Jump to Subroutine* does: runs the routine `Walk`.

Why it is built this way:

Part of the index: one call, one job.

MAINROUTINE · RUNG 5




```
JSR(Monitor);
```

Flash mode, the flasher, the cabinet lamps, Reset

HOW THIS RUNG WORKS

In words. Every scan, with nothing to test, the rung runs the routine `Monitor`.

Left to right:

1. `JSR(Monitor)` *Jump to Subroutine* does: runs the routine `Monitor`.

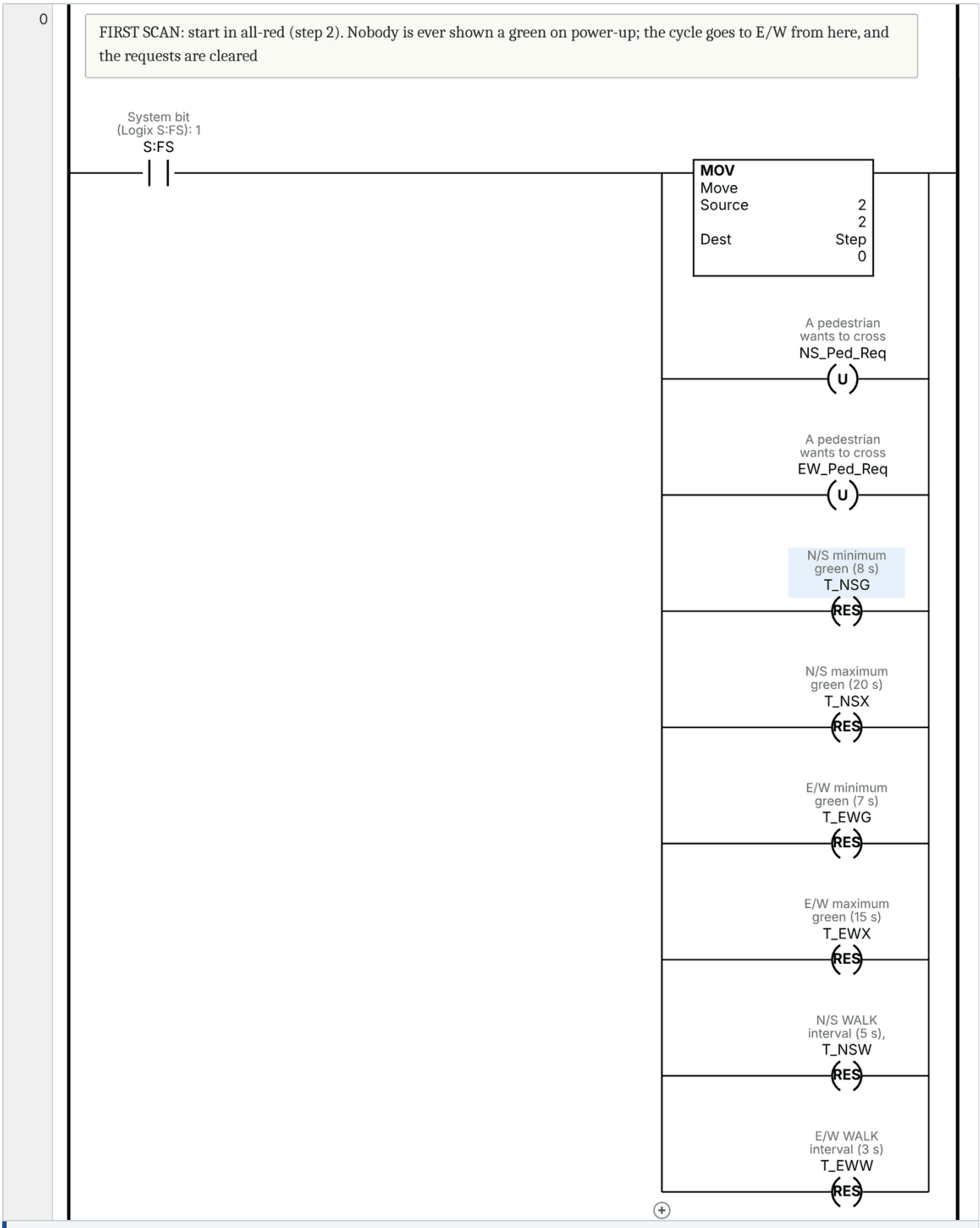
Why it is built this way:

Part of the index: one call, one job.

Tag	Type	Description
`Cycle_Count`	DINT	Shown on the toolbar: completed signal cycles
`Step`	DINT	The cycle: 0 N/S green, 1 N/S amber, 2 all red, 3 E/W green, 4 E/W amber, 5 all red
`T_NSG`	TIMER	N/S minimum green (8 s)
`T_NSX`	TIMER	N/S maximum green (20 s)
`T_NSY`	TIMER	N/S amber (3 s)
`T_AR1`	TIMER	All-red clearance after N/S (2 s)
`T_EWG`	TIMER	E/W minimum green (7 s)
`T_EWX`	TIMER	E/W maximum green (15 s)
`T_EWY`	TIMER	E/W amber (3 s)
`T_AR2`	TIMER	All-red clearance after E/W (2 s)
`T_NSW`	TIMER	N/S WALK interval (5 s), then flashing DON'T WALK for the rest of the green
`T_EWW`	TIMER	E/W WALK interval (3 s)
`NS_Ped_Req`	BOOL	A pedestrian wants to cross along N/S (latched by the button, cleared when served)
`EW_Ped_Req`	BOOL	A pedestrian wants to cross along E/W
`NS_Req`	BOOL	Demand on the N/S road: a car on a loop or a pedestrian request

Tag	Type	Description
`EW_Req`	BOOL	Demand on the E/W road
`Flash`	BOOL	Flash mode: the selector, or the conflict monitor has tripped
`T_Fl`	TIMER	Flasher, half a second on
`T_Fl2`	TIMER	Flasher, half a second off
`Fl_Bit`	BOOL	The flasher bit, 1 Hz

INIT · RUNG 0




```
XIC(S:FS)
[MOV(2,Step),OTU(NS_Ped_Req),OTU(EW_Ped_Req),RES(T_NSG),RES(T_NSX),RES(T_EWG),R
```

```
ES(T_EWX),RES(T_NSW),RES(T_EWW)];
```

FIRST SCAN: start in all-red (step 2). Nobody is ever shown a green on power-up; the cycle goes to E/W from here, and the requests are cleared

HOW THIS RUNG WORKS

In words. When `S:FS` is on, the rung puts **2** into `Step` (The cycle: 0 N/S green, 1 N/S amber, 2 all red, 3 E/W green, 4 E/W amber, 5 all red), unlatches `NS_Ped_Req` (A pedestrian wants to cross along N/S (latched by the button, cleared when served)) (turns it off), unlatches `EW_Ped_Req` (A pedestrian wants to cross along E/W) (turns it off), resets `T_NSG` (N/S minimum green (8 s)), resets `T_NSX` (N/S maximum green (20 s)), resets `T_EWG` (E/W minimum green (7 s)), resets `T_EWX` (E/W maximum green (15 s)), resets `T_NSW` (N/S WALK interval (5 s), then flashing DON'T WALK for the rest of the green) and resets `T_EWW` (E/W WALK interval (3 s)).

Left to right:

1. `XIC(S:FS)` *Examine If Closed* asks: true when `S:FS` is on.
2. A branch of 9 parallel paths: each path is tested on its own and does its own work.

Path 1

1. `MOV(2,Step)` *Move* does: puts **2** into `Step` (The cycle: 0 N/S green, 1 N/S amber, 2 all red, 3 E/W green, 4 E/W amber, 5 all red). It writes every scan the rung is true.

Path 2

1. `OTU(NS_Ped_Req)` *Output Unlatch* does: unlatches `NS_Ped_Req` (A pedestrian wants to cross along N/S (latched by the button, cleared when served)) (turns it off). It only ever clears the bit.

Path 3

1. `OTU(EW_Ped_Req)` *Output Unlatch* does: unlatches `EW_Ped_Req` (A pedestrian wants to cross along E/W) (turns it off).

Path 4

1. `RES(T_NSG)` *Reset* does: resets `T_NSG` (N/S minimum green (8 s)). It zeroes a timer or counter (accumulator and status bits), or a control (its position and status bits).

Path 5

1. `RES(T_NSX)` *Reset* does: resets `T_NSX` (N/S maximum green (20 s)).

Path 6

1. `RES(T_EWG)` *Reset* does: resets `T_EWG` (E/W minimum green (7 s)).

Path 7

1. `RES(T_EWX)` *Reset* does: resets `T_EWX` (E/W maximum green (15 s)).

Path 8

1. `RES(T_NSW)` *Reset* does: resets `T_NSW` (N/S WALK interval (5 s), then flashing DON'T WALK for the rest of the green).

Path 9

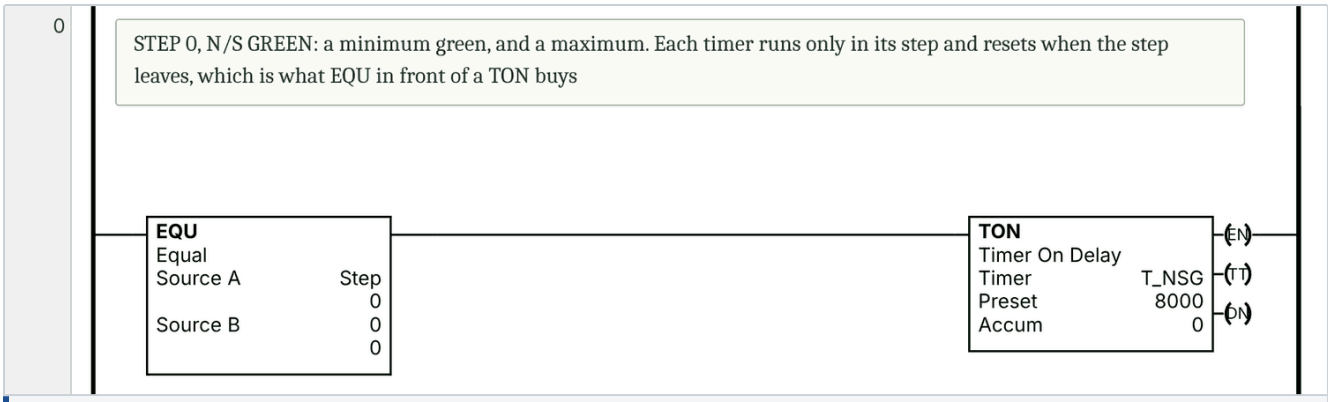
1. `RES(T_EWW)` *Reset* does: resets `T_EWW` (E/W WALK interval (3 s)).

> **Think of it like this:** a rung is a pipe with valves in it, read left to right. Every contact is a valve: XIC is open while its bit is 1, XIO while its bit is 0. Water reaches the output at the right only if every valve in a row is open. The output does not remember anything: on every scan it is simply wet or dry.

Exercise 1: the timed cycle

Six steps in a DINT, a timer for each, and a transition rung that moves the step on when its timer is done. The timers are conditioned on their step, so each resets when its step leaves and starts clean when it comes back; that is the whole trick of a step sequencer with TONs. Read the Cycle routine first with the loop and request contacts covered by your thumb: what remains is a fixed-time signal, which is the first exercise.

CYCLE · RUNG 0




```
EQU(Step,0)TON(T_NSG,8000,0);
```

STEP 0, N/S GREEN: a minimum green, and a maximum. Each timer runs only in its step and resets when the step leaves, which is what EQU in front of a TON buys

HOW THIS RUNG WORKS

In words. When `Step` (The cycle: 0 N/S green, 1 N/S amber, 2 all red, 3 E/W green, 4 E/W amber, 5 all red) equals **0**, the rung runs the timer `T_NSG` (N/S minimum green (8 s)) toward 8000 ms (8 s).

Left to right:

1. `EQU(Step,0)` *Equal* asks: true when `Step` (The cycle: 0 N/S green, 1 N/S amber, 2 all red, 3 E/W green, 4 E/W amber, 5 all red) equals **0**. Both values are compared every scan.
2. `TON(T_NSG,8000,0)` *Timer On Delay* does: runs the timer `T_NSG` (N/S minimum green (8 s)) toward 8000 ms (8 s). The accumulator counts while the rung is true; .DN comes on at the preset; the moment the rung goes false the timer resets.

Why it is built this way:

A timer. `T_NSG` only times while this rung is true; other rungs use its `.DN` (done) or `.TT` (timing) bits. Being a TON, it starts again from zero every time the rung goes false.

> **Think of it like this:** a kitchen timer that only runs while you hold your finger on it. Take your finger off and it snaps back to zero. Hold it long enough and the done light comes on, and stays on as long as your finger is still there.

> **Watch for:** If anything in series flickers false for one scan, a TON starts over from zero. Use RTO when the time must survive an interruption.

CYCLE · RUNG 1

```
EQU(Step,0)TON(T_NSX,20000,0);
```

HOW THIS RUNG WORKS

In words. When `Step` (The cycle: 0 N/S green, 1 N/S amber, 2 all red, 3 E/W green, 4 E/W amber, 5 all red) equals **0**, the rung runs the timer `T_NSX` (N/S maximum green (20 s)) toward 20000 ms (20 s).

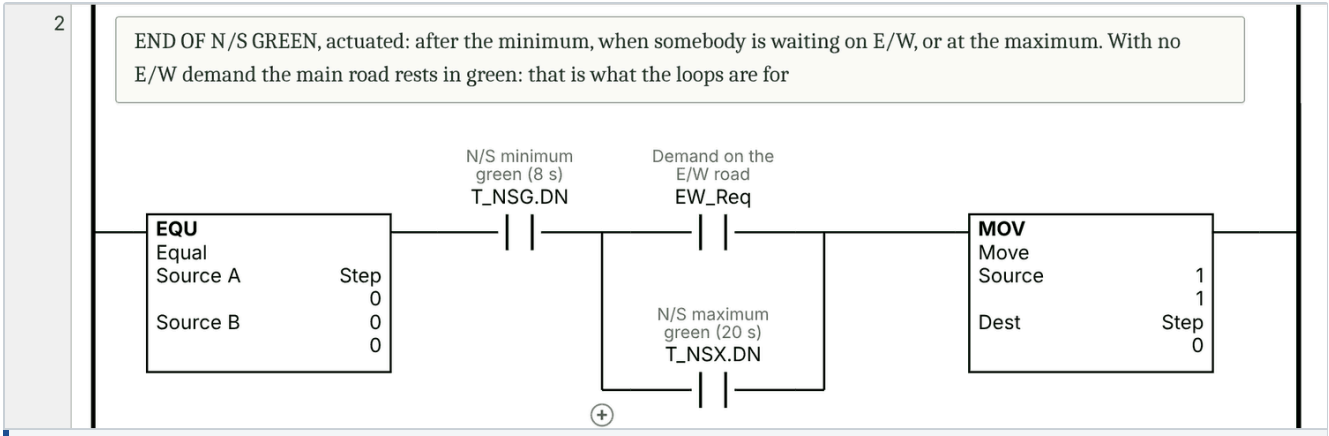
Left to right:

1. `EQU(Step,0)` *Equal* asks: true when `Step` (The cycle: 0 N/S green, 1 N/S amber, 2 all red, 3 E/W green, 4 E/W amber, 5 all red) equals **0**.
2. `TON(T_NSX,20000,0)` *Timer On Delay* does: runs the timer `T_NSX` (N/S maximum green (20 s)) toward 20000 ms (20 s).

Why it is built this way:

A timer: it times only while this rung is true.

CYCLE · RUNG 2




```
EQU(Step,0)XIC(T_NSG.DN)[XIC(EW_Req),XIC(T_NSX.DN)]MOV(1,Step);
```

END OF N/S GREEN, actuated: after the minimum, when somebody is waiting on E/W, or at the maximum. With no E/W demand the main road rests in green: that is what the loops are for

PREDICT IT · 1 `Step` is 0, `T_NSG.DN` (the done bit of `T_NSG` : N/S minimum green (8 s)) is on and `EW_Req` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Step` (The cycle: 0 N/S green, 1 N/S amber, 2 all red, 3 E/W green, 4 E/W amber, 5 all red) equals **0**, `T_NSG.DN` (the done bit of `T_NSG` : N/S minimum green (8 s)) is on and either (`EW_Req` (Demand on the E/W road) is on) *or* (`T_NSX.DN` (the done bit of `T_NSX` : N/S maximum green (20 s)) is on), the rung puts **1** into `Step` .

Left to right:

1. `EQU(Step,0)` *Equal* asks: true when `Step` (The cycle: 0 N/S green, 1 N/S amber, 2 all red, 3 E/W green, 4 E/W amber, 5 all red) equals **0**.
2. `XIC(T_NSG.DN)` *Examine If Closed* asks: true when `T_NSG.DN` (the done bit of `T_NSG` : N/S minimum green (8 s)) is on.
3. A branch of 2 parallel paths: power gets through if any one of them is true.

   Path 1

   1. `XIC(EW_Req)` *Examine If Closed* asks: true when `EW_Req` (Demand on the E/W road) is on.

   Path 2

   1. `XIC(T_NSX.DN)` *Examine If Closed* asks: true when `T_NSX.DN` (the done bit of `T_NSX` : N/S maximum green (20 s)) is on.
4. `MOV(1,Step)` *Move* does: puts **1** into `Step` .

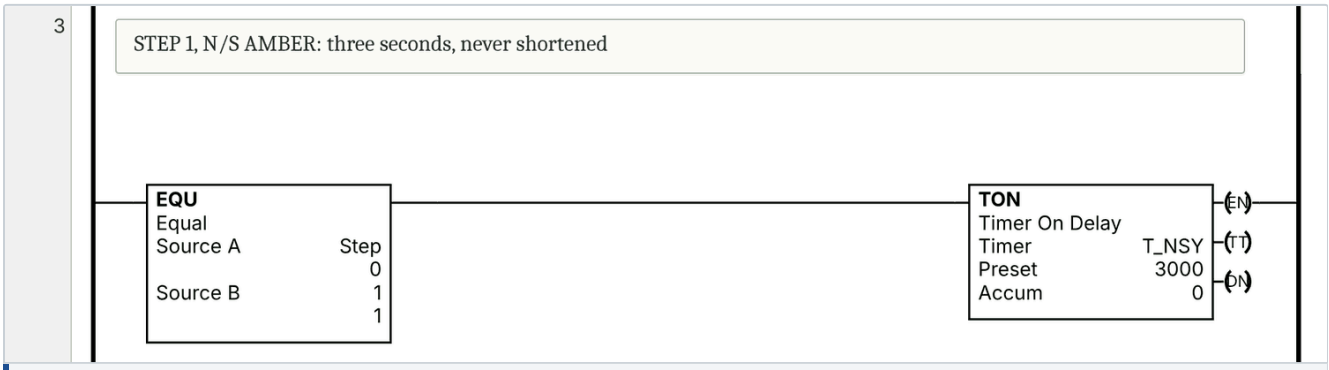
Why it is built this way:

A step of a sequence. The rung only acts while `Step` is 0; when its conditions are met it writes 1 into `Step` , and from then on the rungs that test for 1 take over. One step number at a time means one thing happening at a time.

> **Think of it like this:** a recipe card with numbered steps and one finger on it. Each rung only reads the line under the finger; when its line is done it moves the finger to the next number. Because the finger can only be in one place, only one step can ever be "doing".

> **Watch for:** A rung further down that tests for the new step number sees it in this same scan. Rung order decides whether the next step starts now or next scan.

CYCLE · RUNG 3




```
EQU(Step,1)TON(T_NSY,3000,0);
```

STEP 1, N/S AMBER: three seconds, never shortened

HOW THIS RUNG WORKS

In words. When `Step` (The cycle: 0 N/S green, 1 N/S amber, 2 all red, 3 E/W green, 4 E/W amber, 5 all red) equals **1**, the rung runs the timer `T_NSY` (N/S amber (3 s)) toward 3000 ms (3 s).

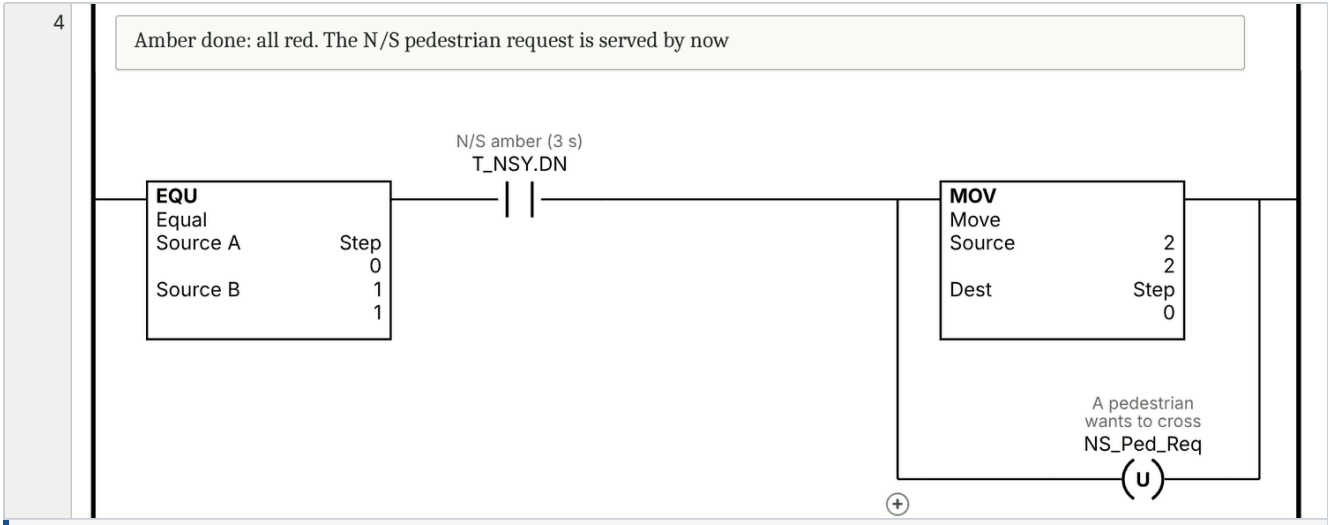
Left to right:

1. `EQU(Step,1)` *Equal* asks: true when `Step` (The cycle: 0 N/S green, 1 N/S amber, 2 all red, 3 E/W green, 4 E/W amber, 5 all red) equals **1**.
2. `TON(T_NSY,3000,0)` *Timer On Delay* does: runs the timer `T_NSY` (N/S amber (3 s)) toward 3000 ms (3 s).

Why it is built this way:

A timer: it times only while this rung is true.

CYCLE · RUNG 4




```
EQU(Step,1)XIC(T_NSY.DN)[MOV(2,Step),OTU(NS_Ped_Req)];
```

Amber done: all red. The N/S pedestrian request is served by now

PREDICT IT · 2 `Step` is 1 and `T_NSY.DN` (the done bit of `T_NSY` : N/S amber (3 s)) is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Step` (The cycle: 0 N/S green, 1 N/S amber, 2 all red, 3 E/W green, 4 E/W amber, 5 all red) equals **1** and `T_NSY.DN` (the done bit of `T_NSY` : N/S amber (3 s)) is on, the rung puts **2** into `Step` and unlatches `NS_Ped_Req` (A pedestrian wants to cross along N/S (latched by the button, cleared when served)) (turns it off).

Left to right:

1. `EQU(Step,1)` *Equal* asks: true when `Step` (The cycle: 0 N/S green, 1 N/S amber, 2 all red, 3 E/W green, 4 E/W amber, 5 all red) equals **1**.
2. `XIC(T_NSY.DN)` *Examine If Closed* asks: true when `T_NSY.DN` (the done bit of `T_NSY` : N/S amber (3 s)) is on.
3. A branch of 2 parallel paths: each path is tested on its own and does its own work.

   Path 1

   1. `MOV(2,Step)` *Move* does: puts **2** into `Step` .

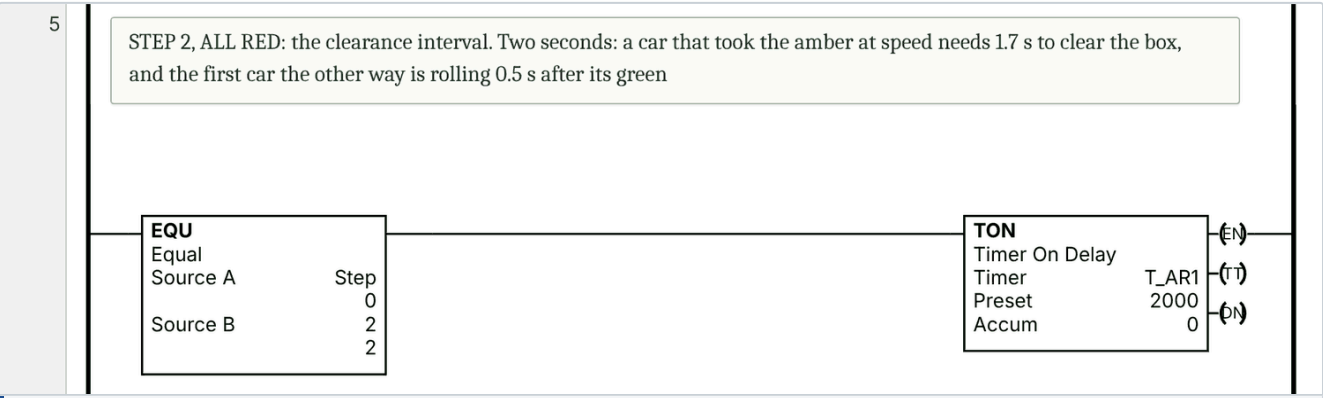
   Path 2

   1. `OTU(NS_Ped_Req)` *Output Unlatch* does: unlatches `NS_Ped_Req` (A pedestrian wants to cross along N/S (latched by the button, cleared when served)) (turns it off).

Why it is built this way:

A step of the sequence: from Step = 1 to 2.

CYCLE · RUNG 5




```
EQU(Step,2)TON(T_AR1,2000,0);
```

STEP 2, ALL RED: the clearance interval. Two seconds: a car that took the amber at speed needs 1.7 s to clear the box, and the first car the other way is rolling 0.5 s after its green

HOW THIS RUNG WORKS

In words. When `Step` (The cycle: 0 N/S green, 1 N/S amber, 2 all red, 3 E/W green, 4 E/W amber, 5 all red) equals **2**, the rung runs the timer `T_AR1` (All-red clearance after N/S (2 s)) toward 2000 ms (2 s).

Left to right:

1. `EQU(Step,2)` *Equal* asks: true when `Step` (The cycle: 0 N/S green, 1 N/S amber, 2 all red, 3 E/W green, 4 E/W amber, 5 all red) equals **2**.
2. `TON(T_AR1,2000,0)` *Timer On Delay* does: runs the timer `T_AR1` (All-red clearance after N/S (2 s)) toward 2000 ms (2 s).

Why it is built this way:

A timer: it times only while this rung is true.

CYCLE · RUNG 6

```
EQU(Step,2)XIC(T_AR1.DN)MOV(3,Step);
```

PREDICT IT · 3 `Step` is 2 and `T_AR1.DN` (the done bit of `T_AR1` : All-red clearance after N/S (2 s)) is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Step` (The cycle: 0 N/S green, 1 N/S amber, 2 all red, 3 E/W green, 4 E/W amber, 5 all red) equals **2** and `T_AR1.DN` (the done bit of `T_AR1` : All-red clearance after N/S (2 s)) is on, the rung puts **3** into `Step` .

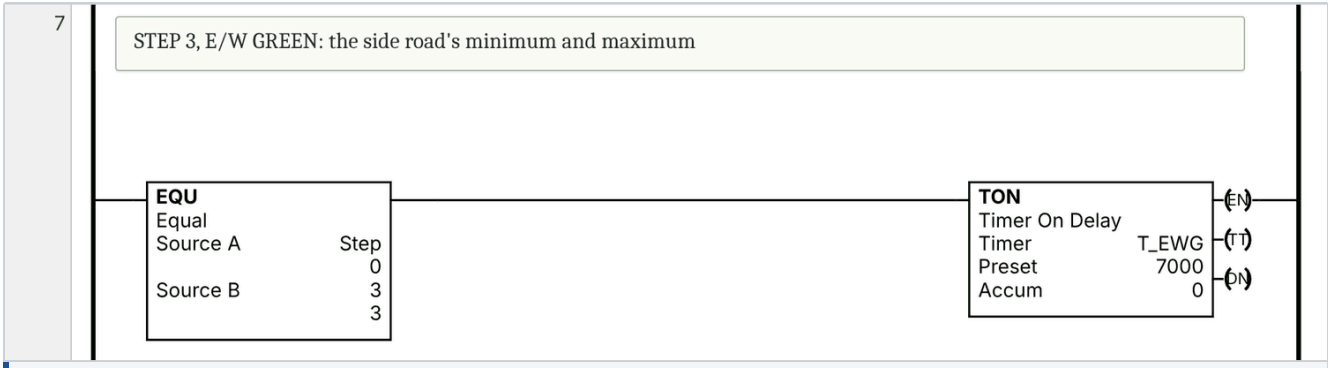
Left to right:

1. `EQU(Step,2)` *Equal* asks: true when `Step` (The cycle: 0 N/S green, 1 N/S amber, 2 all red, 3 E/W green, 4 E/W amber, 5 all red) equals **2**.
2. `XIC(T_AR1.DN)` *Examine If Closed* asks: true when `T_AR1.DN` (the done bit of `T_AR1` : All-red clearance after N/S (2 s)) is on.
3. `MOV(3,Step)` *Move* does: puts **3** into `Step` .

Why it is built this way:

A step of the sequence: from Step = 2 to 3.

CYCLE · RUNG 7




```
EQU(Step,3)TON(T_EWG,7000,0);
```

STEP 3, E/W GREEN: the side road's minimum and maximum

HOW THIS RUNG WORKS

In words. When `Step` (The cycle: 0 N/S green, 1 N/S amber, 2 all red, 3 E/W green, 4 E/W amber, 5 all red) equals **3**, the rung runs the timer `T_EWG` (E/W minimum green (7 s)) toward 7000 ms (7 s).

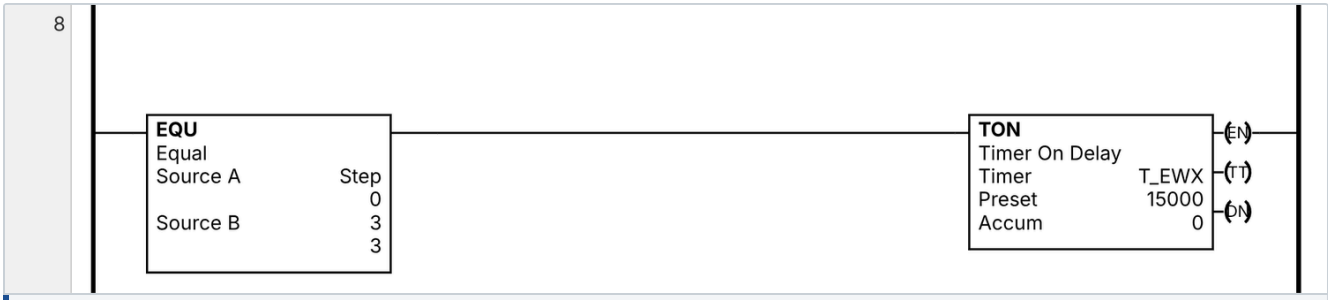
Left to right:

1. `EQU(Step,3)` *Equal* asks: true when `Step` (The cycle: 0 N/S green, 1 N/S amber, 2 all red, 3 E/W green, 4 E/W amber, 5 all red) equals **3**.
2. `TON(T_EWG,7000,0)` *Timer On Delay* does: runs the timer `T_EWG` (E/W minimum green (7 s)) toward 7000 ms (7 s).

Why it is built this way:

A timer: it times only while this rung is true.

CYCLE · RUNG 8




```
EQU(Step,3)TON(T_EWX,15000,0);
```

HOW THIS RUNG WORKS

In words. When `Step` (The cycle: 0 N/S green, 1 N/S amber, 2 all red, 3 E/W green, 4 E/W amber, 5 all red) equals **3**, the rung runs the timer `T_EWX` (E/W maximum green (15 s)) toward 15000 ms (15 s).

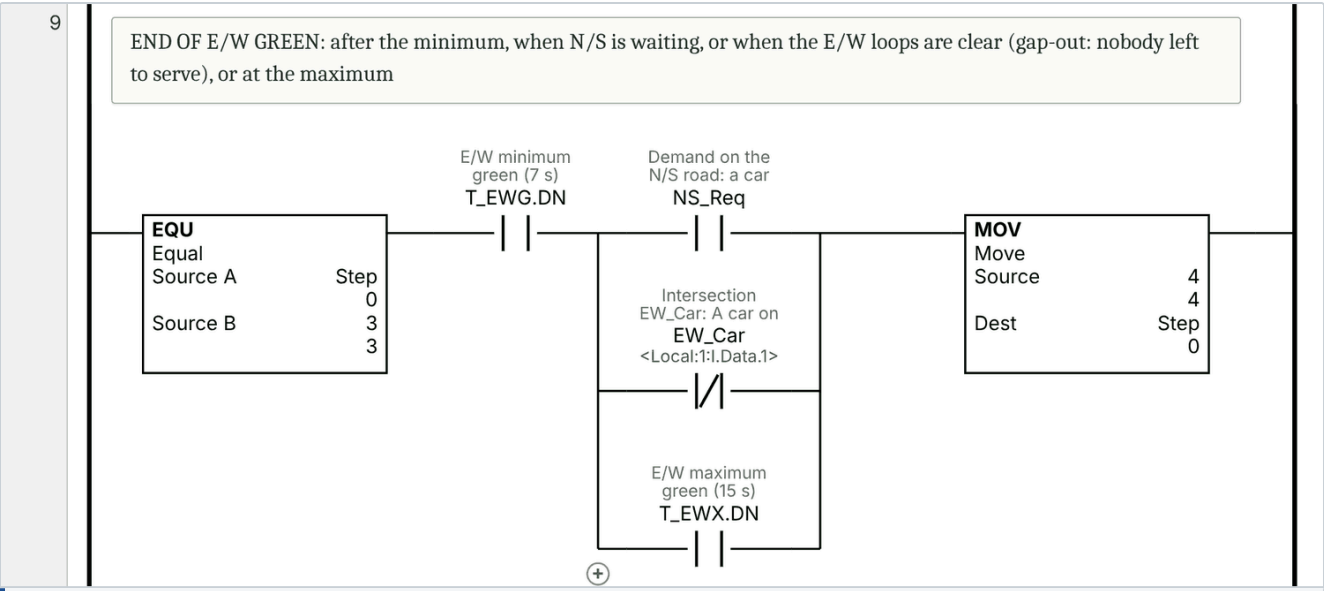
Left to right:

1. `EQU(Step,3)` *Equal* asks: true when `Step` (The cycle: 0 N/S green, 1 N/S amber, 2 all red, 3 E/W green, 4 E/W amber, 5 all red) equals **3**.
2. `TON(T_EWX,15000,0)` *Timer On Delay* does: runs the timer `T_EWX` (E/W maximum green (15 s)) toward 15000 ms (15 s).

Why it is built this way:

A timer: it times only while this rung is true.

CYCLE · RUNG 9




```
EQU(Step,3)XIC(T_EWG.DN)[XIC(NS_Req),XIO(EW_Car),XIC(T_EWX.DN)]MOV(4,Step);
```

END OF E/W GREEN: after the minimum, when N/S is waiting, or when the E/W loops are clear (gap-out: nobody left to serve), or at the maximum

PREDICT IT · 4 `Step` is 4, `T_EWG.DN` (the done bit of `T_EWG` : E/W minimum green (7 s)) is on and `NS_Req` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Step` (The cycle: 0 N/S green, 1 N/S amber, 2 all red, 3 E/W green, 4 E/W amber, 5 all red) equals **3**, `T_EWG.DN` (the done bit of `T_EWG` : E/W minimum green (7 s)) is on and either (`NS_Req` (Demand on the N/S road: a car on a loop or a pedestrian request) is on) *or* (`EW_Car` (A car on either E/W approach loop) is off) *or* (`T_EWX.DN` (the done bit of `T_EWX` : E/W maximum green (15 s)) is on), the rung puts **4** into `Step` .

Left to right:

1. `EQU(Step,3)` *Equal* asks: true when `Step` (The cycle: 0 N/S green, 1 N/S amber, 2 all red, 3 E/W green, 4 E/W amber, 5 all red) equals **3**.
2. `XIC(T_EWG.DN)` *Examine If Closed* asks: true when `T_EWG.DN` (the done bit of `T_EWG` : E/W minimum green (7 s)) is on.
3. A branch of 3 parallel paths: power gets through if any one of them is true.

   Path 1

   1. `XIC(NS_Req)` *Examine If Closed* asks: true when `NS_Req` (Demand on the N/S road: a car on a loop or a pedestrian request) is on.

   Path 2

   1. `XIO(EW_Car)` *Examine If Open* asks: true when `EW_Car` (A car on either E/W approach loop) is off.

   Path 3

   1. `XIC(T_EWX.DN)` *Examine If Closed* asks: true when `T_EWX.DN` (the done bit of `T_EWX` : E/W maximum green (15 s)) is on.

4. `MOV(4,Step)` *Move* does: puts **4** into `Step` .

Why it is built this way:

A step of the sequence: from Step = 3 to 4.

> **Think of it like this:** parallel paths are two roads to the same town. If either road is open you get there; the rung only cares that power reaches the far end, not which way it went.

CYCLE · RUNG 10

```
EQU(Step,4)TON(T_EWY,3000,0);
```

STEP 4, E/W AMBER

HOW THIS RUNG WORKS

In words. When `Step` (The cycle: 0 N/S green, 1 N/S amber, 2 all red, 3 E/W green, 4 E/W amber, 5 all red) equals **4**, the rung runs the timer `T_EWY` (E/W amber (3 s)) toward 3000 ms (3 s).

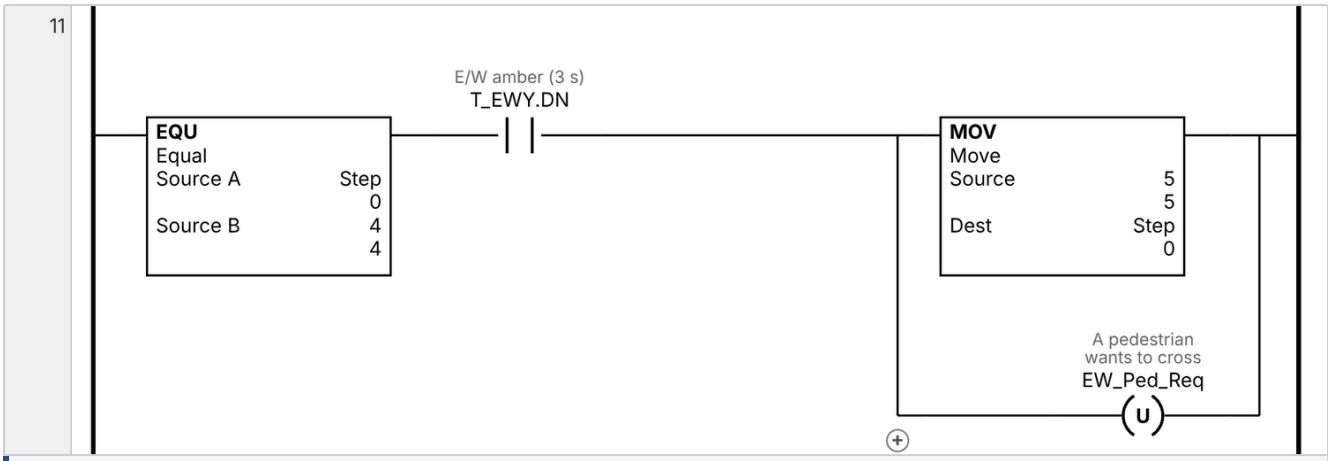
Left to right:

1. `EQU(Step,4)` *Equal* asks: true when `Step` (The cycle: 0 N/S green, 1 N/S amber, 2 all red, 3 E/W green, 4 E/W amber, 5 all red) equals **4**.
2. `TON(T_EWY,3000,0)` *Timer On Delay* does: runs the timer `T_EWY` (E/W amber (3 s)) toward 3000 ms (3 s).

Why it is built this way:

A timer: it times only while this rung is true.

CYCLE · RUNG 11




```
EQU(Step,4)XIC(T_EWY.DN)[MOV(5,Step),OTU(EW_Ped_Req)];
```

HOW THIS RUNG WORKS

In words. When `Step` (The cycle: 0 N/S green, 1 N/S amber, 2 all red, 3 E/W green, 4 E/W amber, 5 all red) equals **4** and `T_EWY.DN` (the done bit of `T_EWY` : E/W amber (3 s)) is on, the rung puts **5** into `Step` and unlatches `EW_Ped_Req` (A pedestrian wants to cross along E/W) (turns it off).

Left to right:

1. `EQU(Step,4)` *Equal* asks: true when `Step` (The cycle: 0 N/S green, 1 N/S amber, 2 all red, 3 E/W green, 4 E/W amber, 5 all red) equals **4**.
2. `XIC(T_EWY.DN)` *Examine If Closed* asks: true when `T_EWY.DN` (the done bit of `T_EWY` : E/W amber (3 s)) is on.
3. A branch of 2 parallel paths: each path is tested on its own and does its own work.

   Path 1

   1. `MOV(5,Step)` *Move* does: puts **5** into `Step` .

   Path 2

   1. `OTU(EW_Ped_Req)` *Output Unlatch* does: unlatches `EW_Ped_Req` (A pedestrian wants to cross along E/W) (turns it off).

Why it is built this way:

A step of the sequence: from Step = 4 to 5.

CYCLE · RUNG 12

```
EQU(Step,5)TON(T_AR2,2000,0);
```

STEP 5, ALL RED

HOW THIS RUNG WORKS

In words. When `Step` (The cycle: 0 N/S green, 1 N/S amber, 2 all red, 3 E/W green, 4 E/W amber, 5 all red) equals **5**, the rung runs the timer `T_AR2` (All-red clearance after E/W (2 s)) toward 2000 ms (2 s).

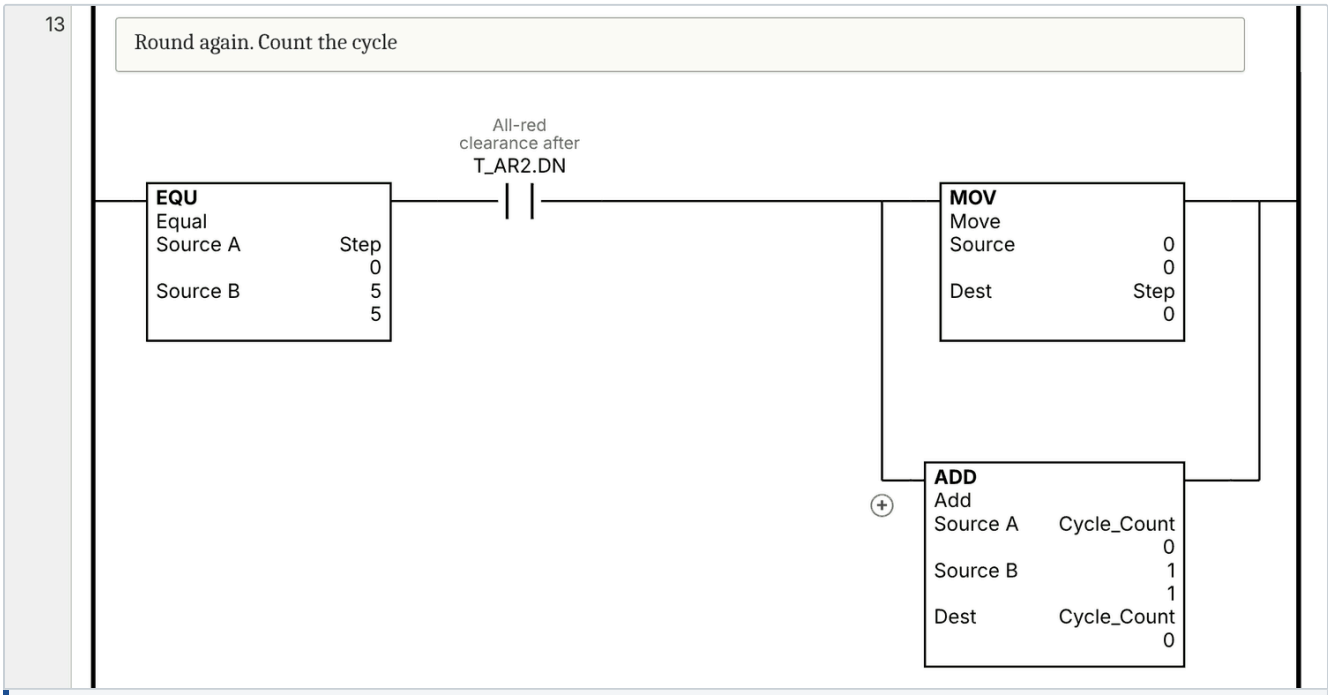
Left to right:

1. `EQU(Step,5)` *Equal* asks: true when `Step` (The cycle: 0 N/S green, 1 N/S amber, 2 all red, 3 E/W green, 4 E/W amber, 5 all red) equals **5**.
2. `TON(T_AR2,2000,0)` *Timer On Delay* does: runs the timer `T_AR2` (All-red clearance after E/W (2 s)) toward 2000 ms (2 s).

Why it is built this way:

A timer: it times only while this rung is true.

CYCLE · RUNG 13




```
EQU(Step,5)XIC(T_AR2.DN)[MOV(0,Step),ADD(Cycle_Count,1,Cycle_Count)];
```

Round again. Count the cycle

HOW THIS RUNG WORKS

In words. When `Step` (The cycle: 0 N/S green, 1 N/S amber, 2 all red, 3 E/W green, 4 E/W amber, 5 all red) equals **5** and `T_AR2.DN` (the done bit of `T_AR2` : All-red clearance after E/W (2 s)) is on, the rung puts **0** into `Step` and makes `Cycle_Count` (Shown on the toolbar: completed signal cycles) = `Cycle_Count` + **1**.

Left to right:

1. `EQU(Step,5)` *Equal* asks: true when `Step` (The cycle: 0 N/S green, 1 N/S amber, 2 all red, 3 E/W green, 4 E/W amber, 5 all red) equals **5**.
2. `XIC(T_AR2.DN)` *Examine If Closed* asks: true when `T_AR2.DN` (the done bit of `T_AR2` : All-red clearance after E/W (2 s)) is on.
3. A branch of 2 parallel paths: each path is tested on its own and does its own work.

   Path 1

   1. `MOV(0,Step)` *Move* does: puts **0** into `Step` .

   Path 2

   1. `ADD(Cycle_Count,1,Cycle_Count)` *Add* does: makes `Cycle_Count` (Shown on the toolbar: completed signal cycles) = `Cycle_Count` + **1**. Every scan the rung is true; behind a one-shot it adds once per event.

Why it is built this way:

A step of the sequence: from Step = 5 to 0.

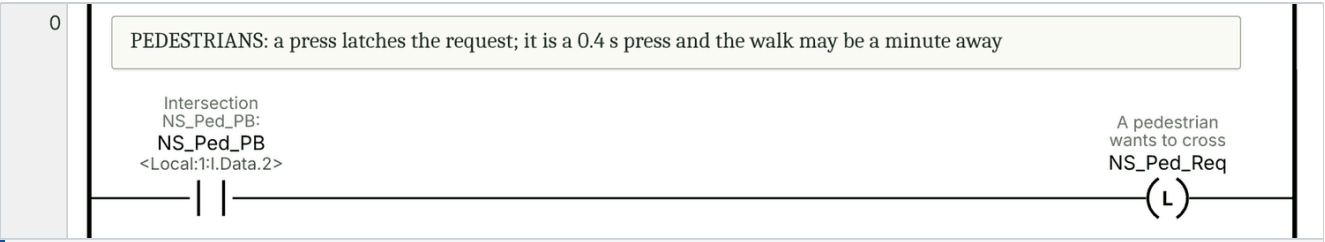
WATCH FOR: THE CLEARANCE SUMS

Two numbers in this routine are safety, not timing. The **amber** is three seconds and nothing shortens it. The **all-red** is two: a car that took the amber at speed crosses the stop line 0.4 s into it and needs 1.7 s more to clear the box, and the first car the other way is rolling half a second after its green. Set T_AR1 to 500 ms and run at twenty cars a minute: within a few cycles the log says CARS IN THE BOX FROM BOTH ROADS. The conflict monitor does not catch this one, because it only watches the lamps, and the lamps were never wrong. Only the sums were.

Exercise 2: the walk signs

A pedestrian request is a latch: pressed for 0.4 s, wanted for up to a minute. It is served only in the right step, only for a *walk interval*, and only if it existed when the step began; after the interval the sign flashes DON'T WALK for the rest of the green, the clearance for whoever is still crossing, and the request is cleared when the step ends.

DEMAND · RUNG 0




```
XIC(NS_Ped_PB)OTL(NS_Ped_Req);
```

PEDESTRIANS: a press latches the request; it is a 0.4 s press and the walk may be a minute away

HOW THIS RUNG WORKS

In words. When `NS_Ped_PB` (Pedestrian button to cross the E/W road (walk along N/S), 1 while pressed) is on, the rung latches `NS_Ped_Req` (A pedestrian wants to cross along N/S (latched by the button, cleared when served)) on.

Left to right:

1. `XIC(NS_Ped_PB)` *Examine If Closed* asks: true when `NS_Ped_PB` (Pedestrian button to cross the E/W road (walk along N/S), 1 while pressed) is on.
2. `OTL(NS_Ped_Req)` *Output Latch* does: latches `NS_Ped_Req` (A pedestrian wants to cross along N/S (latched by the button, cleared when served)) on. It only ever sets the bit; the bit stays on after the rung goes false.

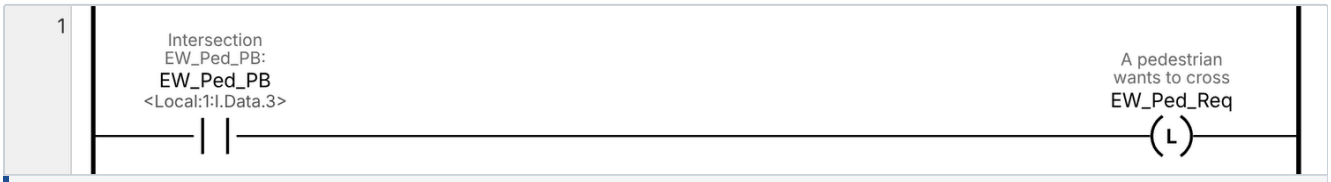
Why it is built this way:

A latch. `NS_Ped_Req` stays on after this rung goes false; an `OTU` elsewhere in the program clears it.

> **Think of it like this:** a light switch instead of a doorbell. OTL flips it on and it stays on when you take your hand away; OTU flips it off. Whatever flipped it on does not have to stay there.

> **Watch for:** A latched bit survives the condition that set it: every way in needs a way out, or it stays on for good.

DEMAND · RUNG 1




```
XIC(EW_Ped_PB)OTL(EW_Ped_Req);
```

HOW THIS RUNG WORKS

In words. When `EW_Ped_PB` (Pedestrian button to cross the N/S road (walk along E/W), 1 while pressed) is on, the rung latches `EW_Ped_Req` (A pedestrian wants to cross along E/W) on.

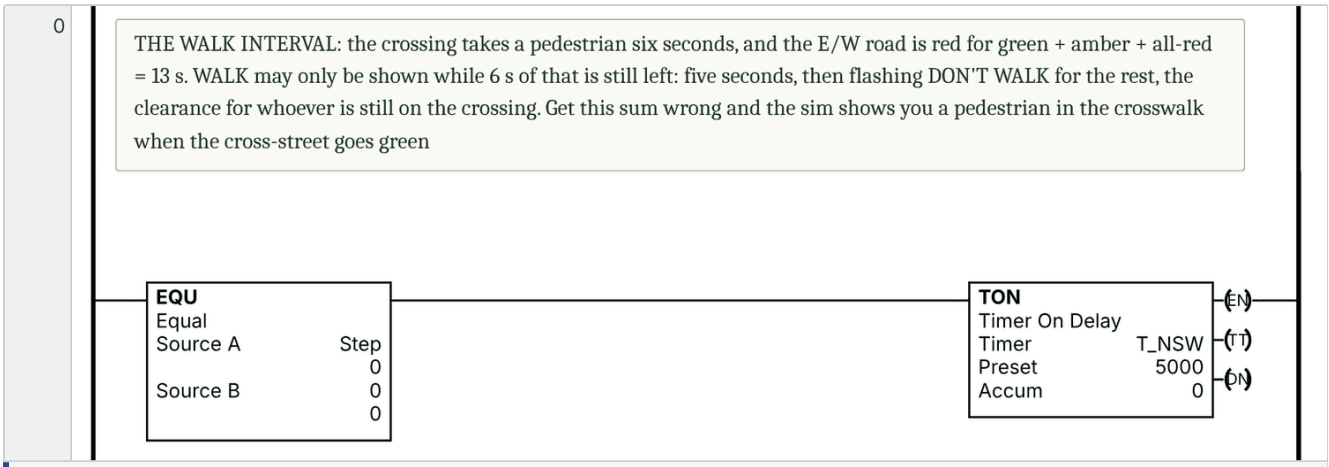
Left to right:

1. `XIC(EW_Ped_PB)` *Examine If Closed* asks: true when `EW_Ped_PB` (Pedestrian button to cross the N/S road (walk along E/W), 1 while pressed) is on.
2. `OTL(EW_Ped_Req)` *Output Latch* does: latches `EW_Ped_Req` (A pedestrian wants to cross along E/W) on.

Why it is built this way:

A latch: it stays on until an OTU clears it.

WALK · RUNG 0




```
EQU(Step,0)TON(T_NSW,5000,0);
```

THE WALK INTERVAL: the crossing takes a pedestrian six seconds, and the E/W road is red for green + amber + all-red = 13 s. WALK may only be shown while 6 s of that is still left: five seconds, then flashing DON'T WALK for the rest, the clearance for whoever is still on the crossing. Get this sum wrong and the sim shows you a pedestrian in the crosswalk when the cross-street goes green

HOW THIS RUNG WORKS

In words. When `Step` (The cycle: 0 N/S green, 1 N/S amber, 2 all red, 3 E/W green, 4 E/W amber, 5 all red) equals **0**, the rung runs the timer `T_NSW` (N/S WALK interval (5 s), then flashing DON'T WALK for the rest of the green) toward 5000 ms (5 s).

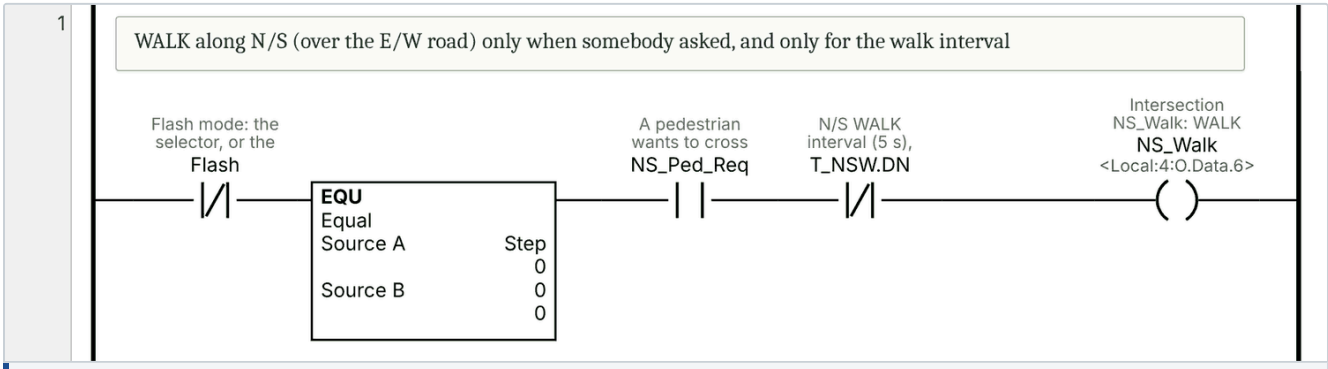
Left to right:

1. `EQU(Step,0)` *Equal* asks: true when `Step` (The cycle: 0 N/S green, 1 N/S amber, 2 all red, 3 E/W green, 4 E/W amber, 5 all red) equals **0**.
2. `TON(T_NSW,5000,0)` *Timer On Delay* does: runs the timer `T_NSW` (N/S WALK interval (5 s), then flashing DON'T WALK for the rest of the green) toward 5000 ms (5 s).

Why it is built this way:

A timer: it times only while this rung is true.

WALK · RUNG 1




```
XIO(Flash)EQU(Step,0)XIC(NS_Ped_Req)XIO(T_NSW.DN)OTE(NS_Walk);
```

WALK along N/S (over the E/W road) only when somebody asked, and only for the walk interval

PREDICT IT · 5 `Flash` is on, `Step` is 0, `NS_Ped_Req` is on and `T_NSW.DN` (the done bit of `T_NSW`: N/S WALK interval (5 s), then flashing DON'T WALK for the rest of the green) is off. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

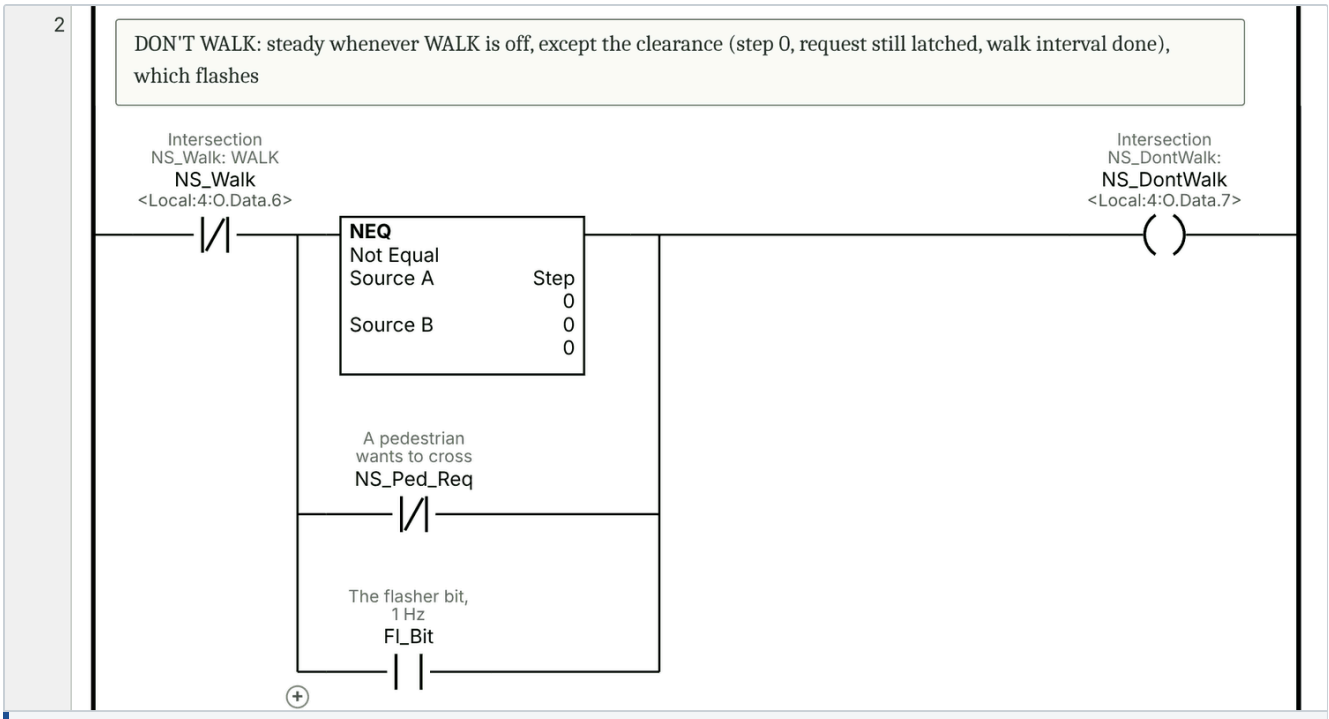
HOW THIS RUNG WORKS

In words. When `Flash` (Flash mode: the selector, or the conflict monitor has tripped) is off, `Step` (The cycle: 0 N/S green, 1 N/S amber, 2 all red, 3 E/W green, 4 E/W amber, 5 all red) equals **0**, `NS_Ped_Req` (A pedestrian wants to cross along N/S (latched by the button, cleared when served)) is on and `T_NSW.DN` (the done bit of `T_NSW`: N/S WALK interval (5 s), then flashing DON'T WALK for the rest of the green) is off, the rung turns `NS_Walk` (WALK for the crossings along N/S (over the E/W road)) on (and off again whenever the rung is false).

Left to right:

1. `XIO(Flash)` *Examine If Open* asks: true when `Flash` (Flash mode: the selector, or the conflict monitor has tripped) is off.
2. `EQU(Step,0)` *Equal* asks: true when `Step` (The cycle: 0 N/S green, 1 N/S amber, 2 all red, 3 E/W green, 4 E/W amber, 5 all red) equals **0**.
3. `XIC(NS_Ped_Req)` *Examine If Closed* asks: true when `NS_Ped_Req` (A pedestrian wants to cross along N/S (latched by the button, cleared when served)) is on.
4. `XIO(T_NSW.DN)` *Examine If Open* asks: true when `T_NSW.DN` (the done bit of `T_NSW`: N/S WALK interval (5 s), then flashing DON'T WALK for the rest of the green) is off.
5. `OTE(NS_Walk)` *Output Energize* does: turns `NS_Walk` (WALK for the crossings along N/S (over the E/W road)) on (and off again whenever the rung is false). Every scan it writes the rung state: true on, false off.

WALK · RUNG 2




```
XIO(NS_Walk)[NEQ(Step,0),XIO(NS_Ped_Req),XIC(Fl_Bit)]OTE(NS_DontWalk);
```

DON'T WALK: steady whenever WALK is off, except the clearance (step 0, request still latched, walk interval done), which flashes

PREDICT IT · 6 `NS_Walk` is off and `Step` is 1. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `NS_Walk` (WALK for the crossings along N/S (over the E/W road)) is off and either (`Step` (The cycle: 0 N/S green, 1 N/S amber, 2 all red, 3 E/W green, 4 E/W amber, 5 all red) is not **0**) *or* (`NS_Ped_Req` (A pedestrian wants to cross along N/S (latched by the button, cleared when served)) is off) *or* (`Fl_Bit` (The flasher bit, 1 Hz) is on), the rung turns `NS_DontWalk` (DON'T WALK for those crossings) on (and off again whenever the rung is false).

Left to right:

1. `XIO(NS_Walk)` *Examine If Open* asks: true when `NS_Walk` (WALK for the crossings along N/S (over the E/W road)) is off.
2. A branch of 3 parallel paths: power gets through if any one of them is true.

   Path 1

   1. `NEQ(Step,0)` *Not Equal* asks: true when `Step` (The cycle: 0 N/S green, 1 N/S amber, 2 all red, 3 E/W green, 4 E/W amber, 5 all red) is not **0**.

   Path 2

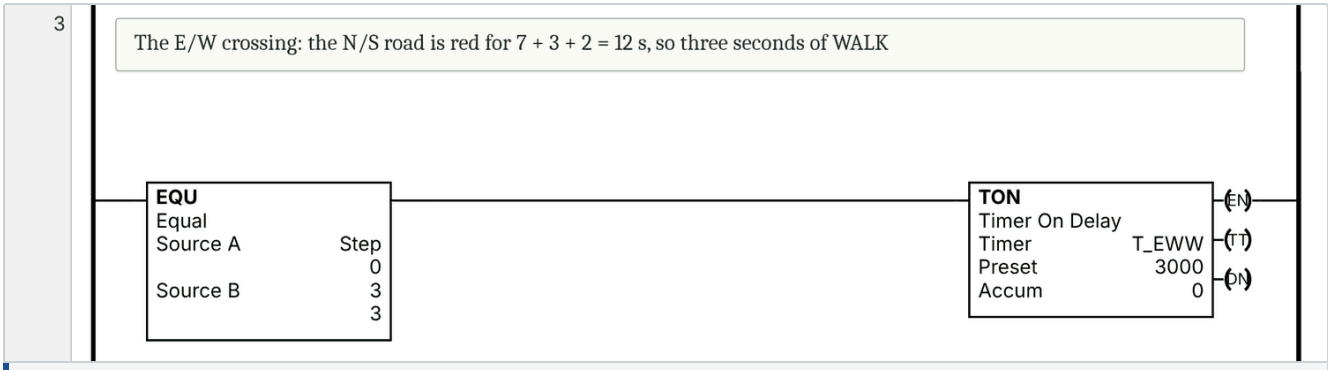
   1. `XIO(NS_Ped_Req)` *Examine If Open* asks: true when `NS_Ped_Req` (A pedestrian wants to cross along N/S (latched by the button, cleared when served)) is off.

   Path 3

   1. `XIC(Fl_Bit)` *Examine If Closed* asks: true when `Fl_Bit` (The flasher bit, 1 Hz) is on.

3. `OTE(NS_DontWalk)` *Output Energize* does: turns `NS_DontWalk` (DON'T WALK for those crossings) on (and off again whenever the rung is false).

WALK · RUNG 3




```
EQU(Step,3)TON(T_EWW,3000,0);
```

The E/W crossing: the N/S road is red for 7 + 3 + 2 = 12 s, so three seconds of WALK

HOW THIS RUNG WORKS

In words. When `Step` (The cycle: 0 N/S green, 1 N/S amber, 2 all red, 3 E/W green, 4 E/W amber, 5 all red) equals **3**, the rung runs the timer `T_EWW` (E/W WALK interval (3 s)) toward 3000 ms (3 s).

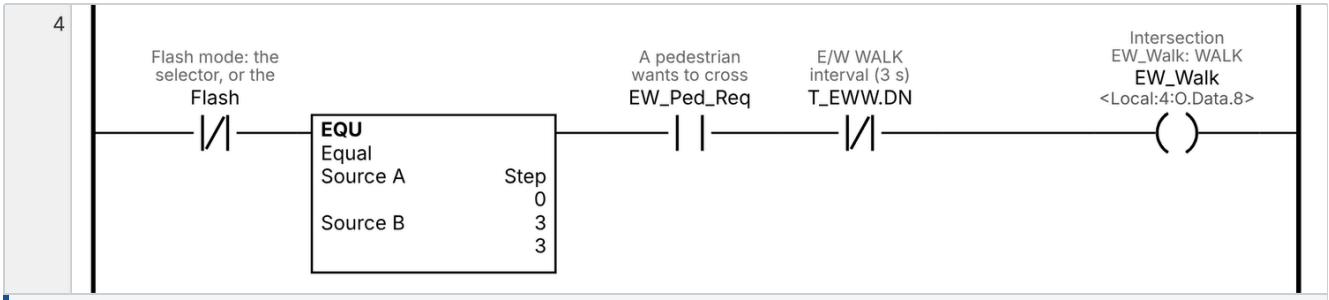
Left to right:

1. `EQU(Step,3)` *Equal* asks: true when `Step` (The cycle: 0 N/S green, 1 N/S amber, 2 all red, 3 E/W green, 4 E/W amber, 5 all red) equals **3**.
2. `TON(T_EWW,3000,0)` *Timer On Delay* does: runs the timer `T_EWW` (E/W WALK interval (3 s)) toward 3000 ms (3 s).

Why it is built this way:

A timer: it times only while this rung is true.

WALK · RUNG 4




```
XIO(Flash)EQU(Step,3)XIC(EW_Ped_Req)XIO(T_EWW.DN)OTE(EW_Walk);
```

PREDICT IT · 7 `Flash` is off, `Step` is 3, `EW_Ped_Req` is on and `T_EWW.DN` (the done bit of `T_EWW`: E/W WALK interval (3 s)) is off. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

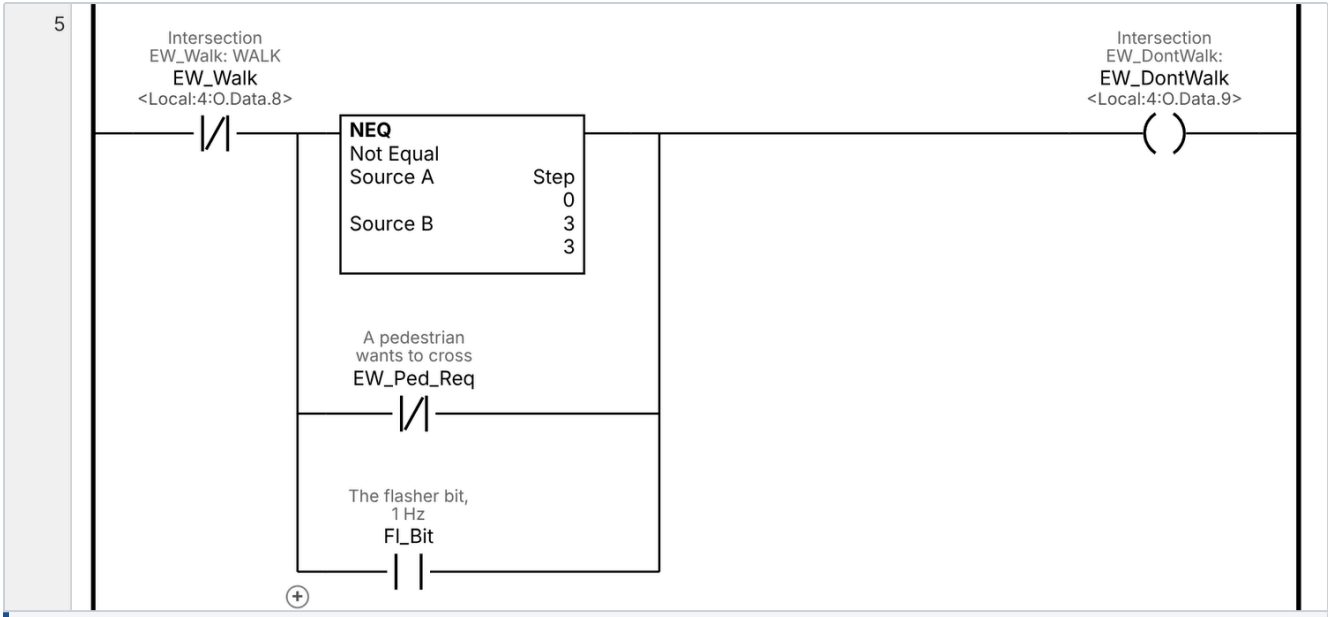
HOW THIS RUNG WORKS

In words. When `Flash` (Flash mode: the selector, or the conflict monitor has tripped) is off, `Step` (The cycle: 0 N/S green, 1 N/S amber, 2 all red, 3 E/W green, 4 E/W amber, 5 all red) equals **3**, `EW_Ped_Req` (A pedestrian wants to cross along E/W) is on and `T_EWW.DN` (the done bit of `T_EWW`: E/W WALK interval (3 s)) is off, the rung turns `EW_Walk` (WALK for the crossings along E/W (over the N/S road)) on (and off again whenever the rung is false).

Left to right:

1. `XIO(Flash)` *Examine If Open* asks: true when `Flash` (Flash mode: the selector, or the conflict monitor has tripped) is off.
2. `EQU(Step,3)` *Equal* asks: true when `Step` (The cycle: 0 N/S green, 1 N/S amber, 2 all red, 3 E/W green, 4 E/W amber, 5 all red) equals **3**.
3. `XIC(EW_Ped_Req)` *Examine If Closed* asks: true when `EW_Ped_Req` (A pedestrian wants to cross along E/W) is on.
4. `XIO(T_EWW.DN)` *Examine If Open* asks: true when `T_EWW.DN` (the done bit of `T_EWW`: E/W WALK interval (3 s)) is off.
5. `OTE(EW_Walk)` *Output Energize* does: turns `EW_Walk` (WALK for the crossings along E/W (over the N/S road)) on (and off again whenever the rung is false).

WALK · RUNG 5




```
XIO(EW_Walk)[NEQ(Step,3),XIO(EW_Ped_Req),XIC(Fl_Bit)]OTE(EW_DontWalk);
```

PREDICT IT · 8 `EW_Walk` is on and `Step` is 4. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `EW_Walk` (WALK for the crossings along E/W (over the N/S road)) is off and either (`Step` (The cycle: 0 N/S green, 1 N/S amber, 2 all red, 3 E/W green, 4 E/W amber, 5 all red) is not **3**) *or* (`EW_Ped_Req` (A pedestrian wants to cross along E/W) is off) *or* (`Fl_Bit` (The flasher bit, 1 Hz) is on), the rung turns `EW_DontWalk` (DON'T WALK for those crossings) on (and off again whenever the rung is false).

Left to right:

1. `XIO(EW_Walk)` *Examine If Open* asks: true when `EW_Walk` (WALK for the crossings along E/W (over the N/S road)) is off.
2. A branch of 3 parallel paths: power gets through if any one of them is true.

   Path 1

   1. `NEQ(Step,3)` *Not Equal* asks: true when `Step` (The cycle: 0 N/S green, 1 N/S amber, 2 all red, 3 E/W green, 4 E/W amber, 5 all red) is not **3**.

   Path 2

   1. `XIO(EW_Ped_Req)` *Examine If Open* asks: true when `EW_Ped_Req` (A pedestrian wants to cross along E/W) is off.

   Path 3

   1. `XIC(Fl_Bit)` *Examine If Closed* asks: true when `Fl_Bit` (The flasher bit, 1 Hz) is on.
3. `OTE(EW_DontWalk)` *Output Energize* does: turns `EW_DontWalk` (DON'T WALK for those crossings) on (and off again whenever the rung is false).

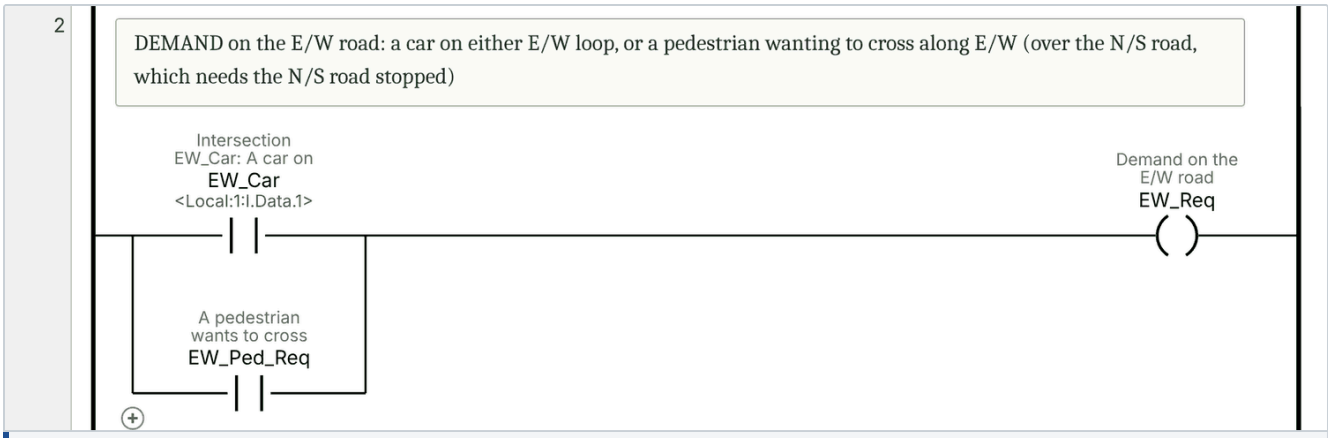
WATCH FOR: THE WALK INTERVAL

Why five seconds and not the whole minimum green? The E/W road is red for the N/S green, amber and all-red: 8 + 3 + 2 = 13 s. A pedestrian needs six. WALK may therefore only invite them while at least six seconds of that red remain, which is the first seven seconds; five leaves a margin. Set T_NSW to 8000 and run with pedestrians every ten seconds: a walker who steps off at 7.9 s is still on the crossing when E/W goes green at 13 s, and the log says A CAR THROUGH THE CROSSWALK. This sum is on the drawing of every real intersection, and the number that goes into the controller is the one somebody worked out from the width of the road.

Exercise 3: vehicle actuation

The loops turn the fixed cycle into a responsive one, and the change is small: two contacts in two transition rungs. The main road *rests in green*: its green ends after the minimum only when the side road has demand (a car on a loop or a pedestrian request), or at the maximum. The side road's green ends after its minimum when the main road is waiting, or when its own loops have been clear (gap-out: nobody left to serve), or at its maximum. With nobody about, the main road stays green all night.

DEMAND · RUNG 2




```
[XIC(EW_Car),XIC(EW_Ped_Req)]OTE(EW_Req);
```

DEMAND on the E/W road: a car on either E/W loop, or a pedestrian wanting to cross along E/W (over the N/S road, which needs the N/S road stopped)

PREDICT IT · 9 `EW_Car` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When either (`EW_Car` (A car on either E/W approach loop) is on) *or* (`EW_Ped_Req` (A pedestrian wants to cross along E/W) is on), the rung turns `EW_Req` (Demand on the E/W road) on (and off again whenever the rung is false).

Left to right:

1. A branch of 2 parallel paths: power gets through if any one of them is true.

   Path 1

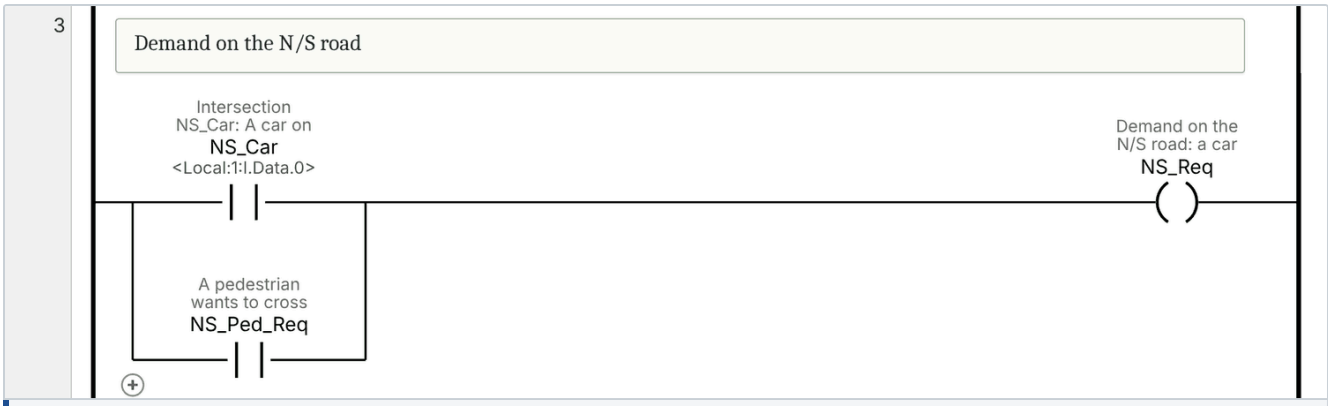
   1. `XIC(EW_Car)` *Examine If Closed* asks: true when `EW_Car` (A car on either E/W approach loop) is on.

   Path 2

   1. `XIC(EW_Ped_Req)` *Examine If Closed* asks: true when `EW_Ped_Req` (A pedestrian wants to cross along E/W) is on.

2. `OTE(EW_Req)` *Output Energize* does: turns `EW_Req` (Demand on the E/W road) on (and off again whenever the rung is false).

DEMAND · RUNG 3




```
[XIC(NS_Car),XIC(NS_Ped_Req)]OTE(NS_Req);
```

Demand on the N/S road

PREDICT IT · 10 `NS_Car` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When either (`NS_Car` (A car on either N/S approach loop (northbound or southbound)) is on) *or* (`NS_Ped_Req` (A pedestrian wants to cross along N/S (latched by the button, cleared when served)) is on), the rung turns `NS_Req` (Demand on the N/S road: a car on a loop or a pedestrian request) on (and off again whenever the rung is false).

Left to right:

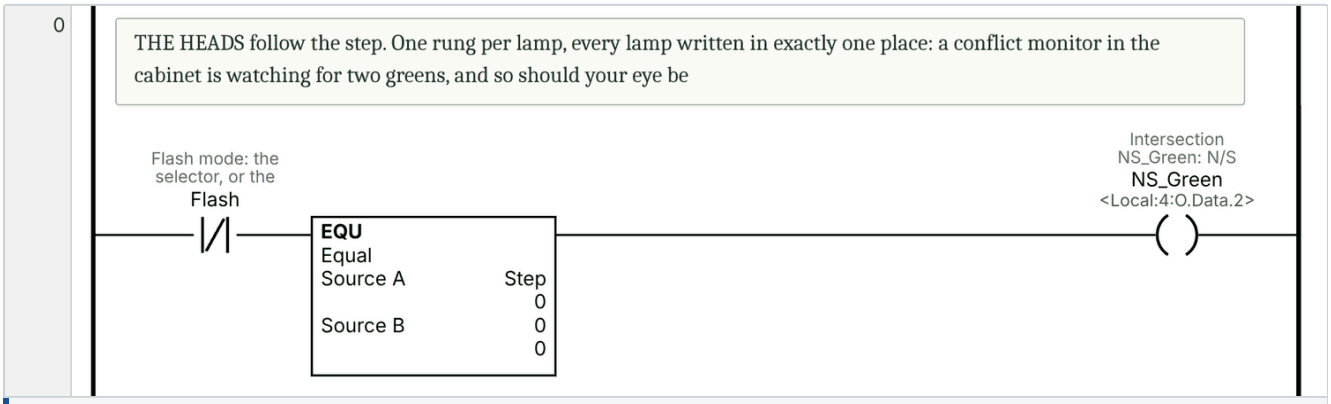
1. A branch of 2 parallel paths: power gets through if any one of them is true.
   - **Path 1**
     1. `XIC(NS_Car)` *Examine If Closed* asks: true when `NS_Car` (A car on either N/S approach loop (northbound or southbound)) is on.
   - **Path 2**
     1. `XIC(NS_Ped_Req)` *Examine If Closed* asks: true when `NS_Ped_Req` (A pedestrian wants to cross along N/S (latched by the button, cleared when served)) is on.
2. `OTE(NS_Req)` *Output Energize* does: turns `NS_Req` (Demand on the N/S road: a car on a loop or a pedestrian request) on (and off again whenever the rung is false).

TRY IT: TWO KINDS OF BUSY

Set Cars per minute to 30 and watch the maximum timers do the work: both roads saturate and the cycle settles into fixed time at the maxima. Then set it to 2: the main road sits in green, the odd side-road car trips its loop and gets a green within the minimum, and the main road gets it back the moment the side road gaps out. Then click a loop to park a car on it and see what a stuck detector does to the cycle: that is the LogixPro Park box, and a real fault you will meet.

The heads

HEADS · RUNG 0




```
XIO(Flash)EQU(Step,0)OTE(NS_Green);
```

THE HEADS follow the step. One rung per lamp, every lamp written in exactly one place: a conflict monitor in the cabinet is watching for two greens, and so should your eye be

PREDICT IT · 11 `Flash` is off and `Step` is 0. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Flash` (Flash mode: the selector, or the conflict monitor has tripped) is off and `Step` (The cycle: 0 N/S green, 1 N/S amber, 2 all red, 3 E/W green, 4 E/W amber, 5 all red) equals **0**, the rung turns `NS_Green` (N/S heads: green) on (and off again whenever the rung is false).

Left to right:

1. `XIO(Flash)` *Examine If Open* asks: true when `Flash` (Flash mode: the selector, or the conflict monitor has tripped) is off.
2. `EQU(Step,0)` *Equal* asks: true when `Step` (The cycle: 0 N/S green, 1 N/S amber, 2 all red, 3 E/W green, 4 E/W amber, 5 all red) equals **0**.
3. `OTE(NS_Green)` *Output Energize* does: turns `NS_Green` (N/S heads: green) on (and off again whenever the rung is false).

HEADS · RUNG 1

```
XIO(Flash)EQU(Step,1)OTE(NS_Yellow);
```

PREDICT IT · 12 `Flash` is on and `Step` is 1. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

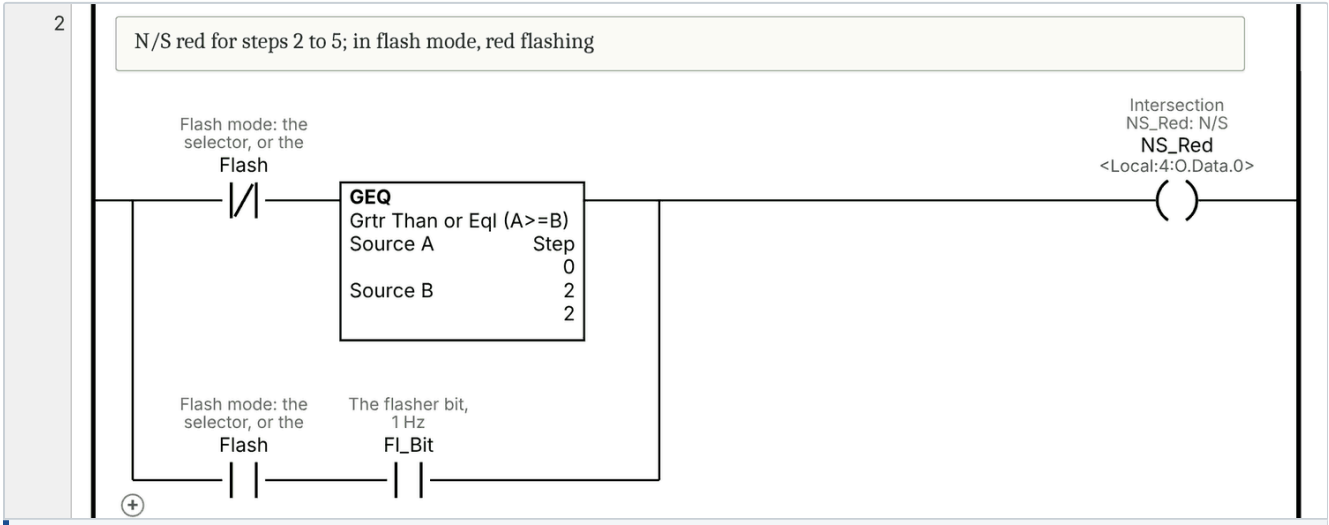
HOW THIS RUNG WORKS

In words. When `Flash` (Flash mode: the selector, or the conflict monitor has tripped) is off and `Step` (The cycle: 0 N/S green, 1 N/S amber, 2 all red, 3 E/W green, 4 E/W amber, 5 all red) equals **1**, the rung turns `NS_Yellow` (N/S heads: amber) on (and off again whenever the rung is false).

Left to right:

1. `XIO(Flash)` *Examine If Open* asks: true when `Flash` (Flash mode: the selector, or the conflict monitor has tripped) is off.
2. `EQU(Step,1)` *Equal* asks: true when `Step` (The cycle: 0 N/S green, 1 N/S amber, 2 all red, 3 E/W green, 4 E/W amber, 5 all red) equals **1**.
3. `OTE(NS_Yellow)` *Output Energize* does: turns `NS_Yellow` (N/S heads: amber) on (and off again whenever the rung is false).

HEADS · RUNG 2




```
[XIO(Flash)GEQ(Step,2),XIC(Flash)XIC(Fl_Bit)]OTE(NS_Red);
```

N/S red for steps 2 to 5; in flash mode, red flashing

PREDICT IT · 13 `Flash` is off and `Step` is 2. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When either (`Flash` (Flash mode: the selector, or the conflict monitor has tripped) is off and `Step` (The cycle: 0 N/S green, 1 N/S amber, 2 all red, 3 E/W green, 4 E/W amber, 5 all red) is at least **2**) *or* (`Flash` is on and `Fl_Bit` (The flasher bit, 1 Hz) is on), the rung turns `NS_Red` (N/S heads: red) on (and off again whenever the rung is false).

Left to right:

1. A branch of 2 parallel paths: power gets through if any one of them is true.

   Path 1

   1. `XIO(Flash)` *Examine If Open* asks: true when `Flash` (Flash mode: the selector, or the conflict monitor has tripped) is off.
   2. `GEQ(Step,2)` *Greater Than or Equal* asks: true when `Step` (The cycle: 0 N/S green, 1 N/S amber, 2 all red, 3 E/W green, 4 E/W amber, 5 all red) is at least **2**.

   Path 2

   1. `XIC(Flash)` *Examine If Closed* asks: true when `Flash` is on.
   2. `XIC(Fl_Bit)` *Examine If Closed* asks: true when `Fl_Bit` (The flasher bit, 1 Hz) is on.

2. `OTE(NS_Red)` *Output Energize* does: turns `NS_Red` (N/S heads: red) on (and off again whenever the rung is false).

HEADS · RUNG 3

```
XIO(Flash)EQU(Step,3)OTE(EW_Green);
```

PREDICT IT · 14 `Flash` is off and `Step` is 4. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Flash` (Flash mode: the selector, or the conflict monitor has tripped) is off and `Step` (The cycle: 0 N/S green, 1 N/S amber, 2 all red, 3 E/W green, 4 E/W amber, 5 all red) equals **3**, the rung turns `EW_Green` (E/W heads: green) on (and off again whenever the rung is false).

Left to right:

1. `XIO(Flash)` *Examine If Open* asks: true when `Flash` (Flash mode: the selector, or the conflict monitor has tripped) is off.
2. `EQU(Step,3)` *Equal* asks: true when `Step` (The cycle: 0 N/S green, 1 N/S amber, 2 all red, 3 E/W green, 4 E/W amber, 5 all red) equals **3**.
3. `OTE(EW_Green)` *Output Energize* does: turns `EW_Green` (E/W heads: green) on (and off again whenever the rung is false).

HEADS · RUNG 4

```
XIO(Flash)EQU(Step,4)OTE(EW_Yellow);
```

PREDICT IT · 15 `Flash` is off and `Step` is 4. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

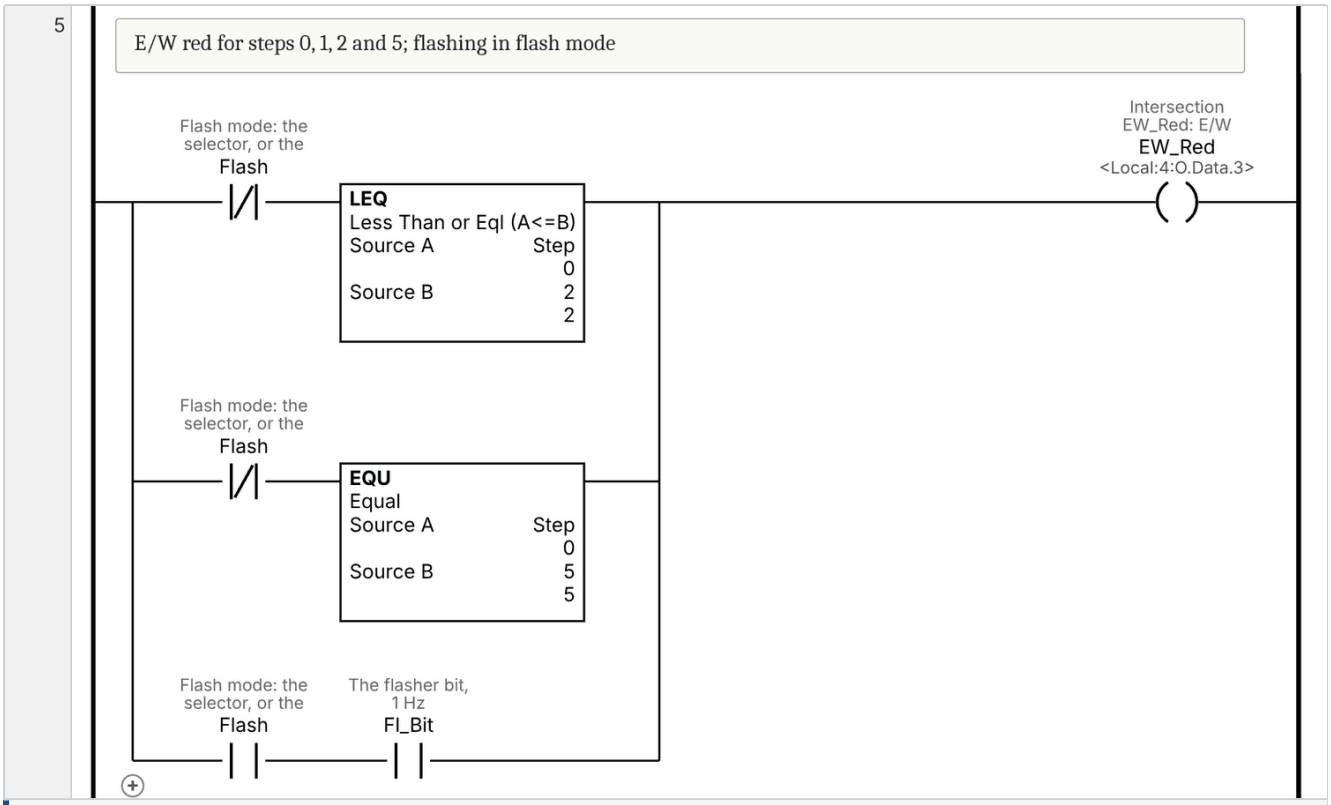
HOW THIS RUNG WORKS

In words. When `Flash` (Flash mode: the selector, or the conflict monitor has tripped) is off and `Step` (The cycle: 0 N/S green, 1 N/S amber, 2 all red, 3 E/W green, 4 E/W amber, 5 all red) equals **4**, the rung turns `EW_Yellow` (E/W heads: amber) on (and off again whenever the rung is false).

Left to right:

1. `XIO(Flash)` *Examine If Open* asks: true when `Flash` (Flash mode: the selector, or the conflict monitor has tripped) is off.
2. `EQU(Step,4)` *Equal* asks: true when `Step` (The cycle: 0 N/S green, 1 N/S amber, 2 all red, 3 E/W green, 4 E/W amber, 5 all red) equals **4**.
3. `OTE(EW_Yellow)` *Output Energize* does: turns `EW_Yellow` (E/W heads: amber) on (and off again whenever the rung is false).

HEADS · RUNG 5




```
[XIO(Flash)LEQ(Step,2),XIO(Flash)EQU(Step,5),XIC(Flash)XIC(Fl_Bit)]OTE(EW_Red);
```

E/W red for steps 0, 1, 2 and 5; flashing in flash mode

PREDICT IT · 16 `Flash` is off and `Step` is 2. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When either (`Flash` (Flash mode: the selector, or the conflict monitor has tripped) is off and `Step` (The cycle: 0 N/S green, 1 N/S amber, 2 all red, 3 E/W green, 4 E/W amber, 5 all red) is at most **2**) *or* (`Flash` is off and `Step` equals **5**) *or* (`Flash` is on and `Fl_Bit` (The flasher bit, 1 Hz) is on), the rung turns `EW_Red` (E/W heads: red) on (and off again whenever the rung is false).

Left to right:

1. A branch of 3 parallel paths: power gets through if any one of them is true.

   Path 1

   1. `XIO(Flash)` *Examine If Open* asks: true when `Flash` (Flash mode: the selector, or the conflict monitor has tripped) is off.
   2. `LEQ(Step,2)` *Less Than or Equal* asks: true when `Step` (The cycle: 0 N/S green, 1 N/S amber, 2 all red, 3 E/W green, 4 E/W amber, 5 all red) is at most **2**.

   Path 2

   1. `XIO(Flash)` *Examine If Open* asks: true when `Flash` is off.
   2. `EQU(Step,5)` *Equal* asks: true when `Step` equals **5**.

Path 3

1. `XIC(Flash)` *Examine If Closed* asks: true when `Flash` is on.
2. `XIC(Fl_Bit)` *Examine If Closed* asks: true when `Fl_Bit` (The flasher bit, 1 Hz) is on.

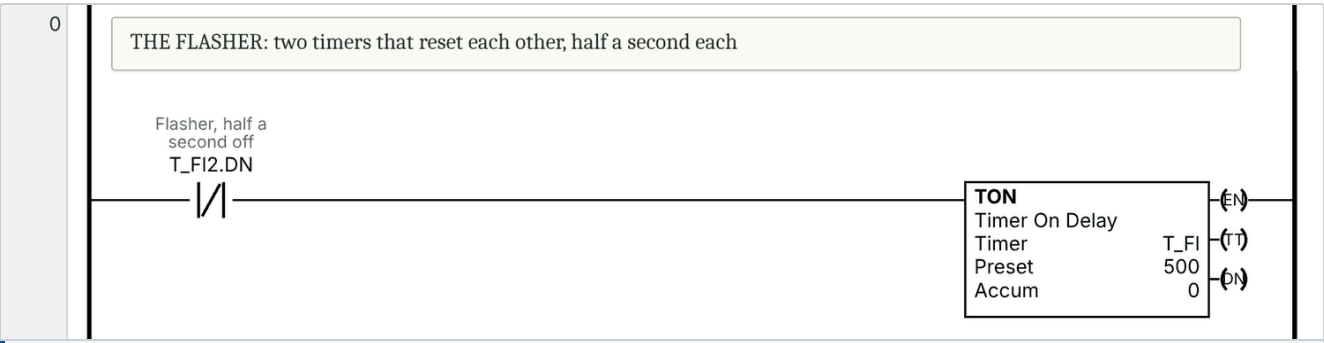
2. `OTE(EW_Red)` *Output Energize* does: turns `EW_Red` (E/W heads: red) on (and off again whenever the rung is false).

WATCH FOR: TWO GREENS

Each lamp is written in exactly one rung, and the two green rungs can never be true together because they test the same tag for different values. Break that on purpose: change the E/W green rung to `EQU(Step,0)`. Run. Within a third of a second the heads flash red, MMU_Fault comes on, the fault lamp lights and the log explains. Click the cabinet to reset it and it trips again at once, because the program is still wrong. That is what the MMU is for, and it is why a traffic controller is one of the few PLC jobs where the program is not the last line of defence.

Exercise 4: flash

MONITOR · RUNG 0




```
XIO(T_Fl2.DN)TON(T_Fl,500,0);
```

THE FLASHER: two timers that reset each other, half a second each

HOW THIS RUNG WORKS

In words. When `T_Fl2.DN` (the done bit of `T_Fl2` : Flasher, half a second off) is off, the rung runs the timer `T_Fl` (Flasher, half a second on) toward 500 ms (0.5 s).

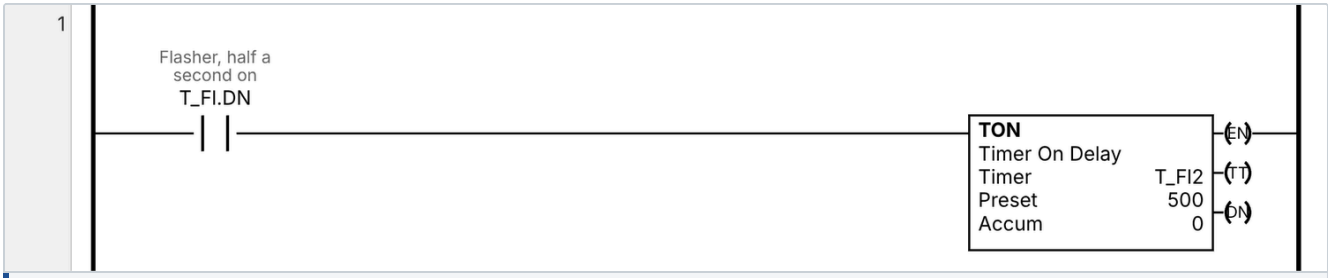
Left to right:

1. `XIO(T_Fl2.DN)` *Examine If Open* asks: true when `T_Fl2.DN` (the done bit of `T_Fl2` : Flasher, half a second off) is off.
2. `TON(T_Fl,500,0)` *Timer On Delay* does: runs the timer `T_Fl` (Flasher, half a second on) toward 500 ms (0.5 s).

Why it is built this way:

A timer: it times only while this rung is true.

MONITOR · RUNG 1




```
XIC(T_Fl.DN)TON(T_Fl2,500,0);
```

HOW THIS RUNG WORKS

In words. When `T_Fl.DN` (the done bit of `T_Fl` : Flasher, half a second on) is on, the rung runs the timer `T_Fl2` (Flasher, half a second off) toward 500 ms (0.5 s).

Left to right:

1. `XIC(T_Fl.DN)` *Examine If Closed* asks: true when `T_Fl.DN` (the done bit of `T_Fl` : Flasher, half a second on) is on.
2. `TON(T_Fl2,500,0)` *Timer On Delay* does: runs the timer `T_Fl2` (Flasher, half a second off) toward 500 ms (0.5 s).

Why it is built this way:

A timer: it times only while this rung is true.

MONITOR · RUNG 2

```
XIC(T_Fl.DN)OTE(Fl_Bit);
```

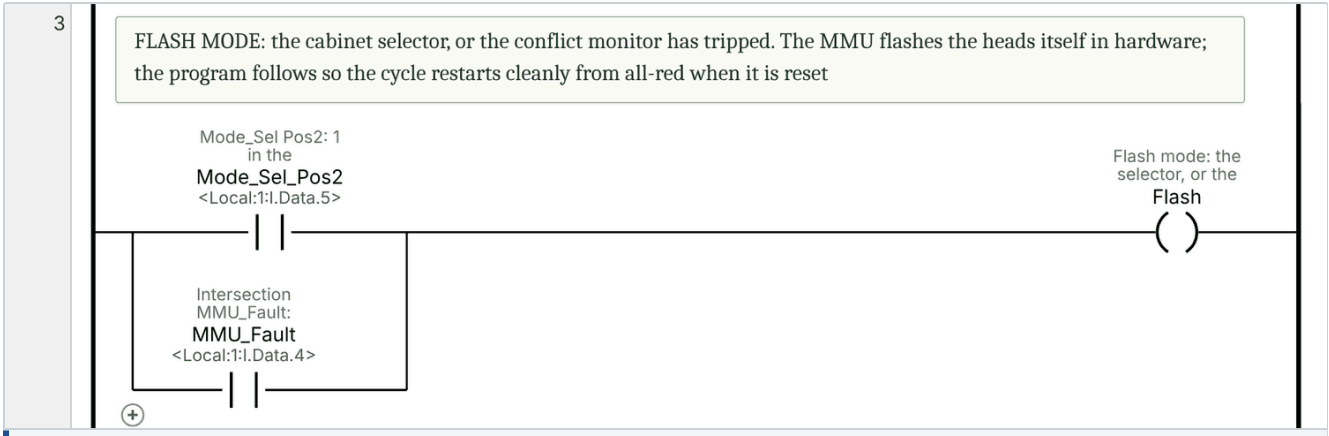
HOW THIS RUNG WORKS

In words. When `T_Fl.DN` (the done bit of `T_Fl` : Flasher, half a second on) is on, the rung turns `Fl_Bit` (The flasher bit, 1 Hz) on (and off again whenever the rung is false).

Left to right:

1. `XIC(T_Fl.DN)` *Examine If Closed* asks: true when `T_Fl.DN` (the done bit of `T_Fl` : Flasher, half a second on) is on.
2. `OTE(Fl_Bit)` *Output Energize* does: turns `Fl_Bit` (The flasher bit, 1 Hz) on (and off again whenever the rung is false).

MONITOR · RUNG 3




```
[XIC(Mode_Sel_Pos2),XIC(MMU_Fault)]OTE(Flash);
```

FLASH MODE: the cabinet selector, or the conflict monitor has tripped. The MMU flashes the heads itself in hardware; the program follows so the cycle restarts cleanly from all-red when it is reset

PREDICT IT · 17 `Mode_Sel_Pos2` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When either (`Mode_Sel_Pos2` (1 in the right-hand position) is on) *or* (`MMU_Fault` (Conflict monitor tripped: the cabinet is in red flash until the MMU is reset (click the cabinet)) is on), the rung turns `Flash` (Flash mode: the selector, or the conflict monitor has tripped) on (and off again whenever the rung is false).

Left to right:

1. A branch of 2 parallel paths: power gets through if any one of them is true.

   Path 1

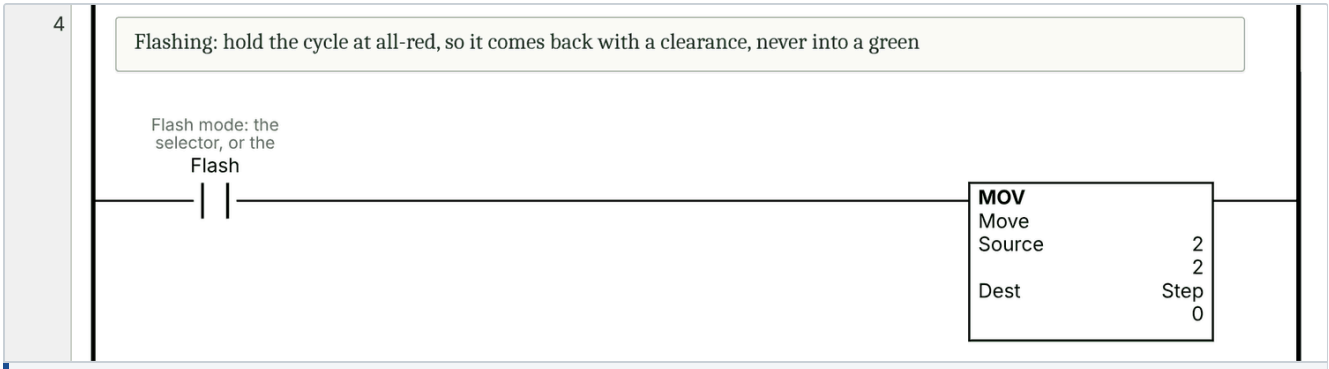
   1. `XIC(Mode_Sel_Pos2)` *Examine If Closed* asks: true when `Mode_Sel_Pos2` (1 in the right-hand position) is on.

   Path 2

   1. `XIC(MMU_Fault)` *Examine If Closed* asks: true when `MMU_Fault` (Conflict monitor tripped: the cabinet is in red flash until the MMU is reset (click the cabinet)) is on.

2. `OTE(Flash)` *Output Energize* does: turns `Flash` (Flash mode: the selector, or the conflict monitor has tripped) on (and off again whenever the rung is false).

MONITOR · RUNG 4




```
XIC(Flash)MOV(2,Step);
```

Flashing: hold the cycle at all-red, so it comes back with a clearance, never into a green

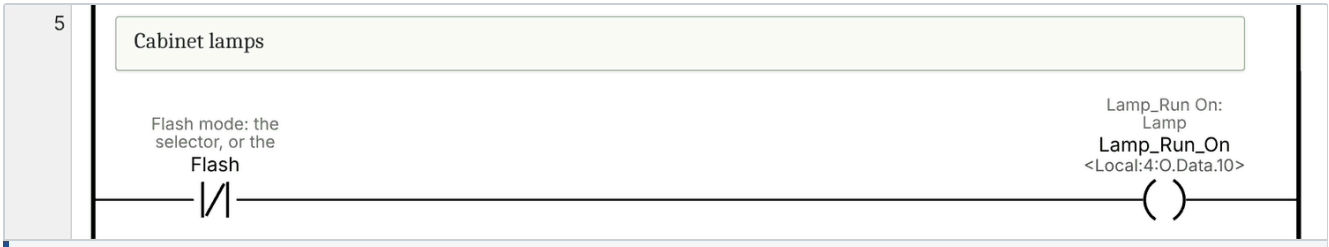
HOW THIS RUNG WORKS

In words. When `Flash` (Flash mode: the selector, or the conflict monitor has tripped) is on, the rung puts **2** into `Step` (The cycle: 0 N/S green, 1 N/S amber, 2 all red, 3 E/W green, 4 E/W amber, 5 all red).

Left to right:

1. `XIC(Flash)` *Examine If Closed* asks: true when `Flash` (Flash mode: the selector, or the conflict monitor has tripped) is on.
2. `MOV(2,Step)` *Move* does: puts **2** into `Step` (The cycle: 0 N/S green, 1 N/S amber, 2 all red, 3 E/W green, 4 E/W amber, 5 all red).

MONITOR · RUNG 5




```
XIO(Flash)OTE(Lamp_Run_On);
```

Cabinet lamps

HOW THIS RUNG WORKS

In words. When `Flash` (Flash mode: the selector, or the conflict monitor has tripped) is off, the rung turns `Lamp_Run_On` (Lamp) on (and off again whenever the rung is false).

Left to right:

1. `XIO(Flash)` *Examine If Open* asks: true when `Flash` (Flash mode: the selector, or the conflict monitor has tripped) is off.
2. `OTE(Lamp_Run_On)` *Output Energize* does: turns `Lamp_Run_On` (Lamp) on (and off again whenever the rung is false).

MONITOR · RUNG 6

```
XIC(MMU_Fault)OTE(Lamp_Fault_On);
```

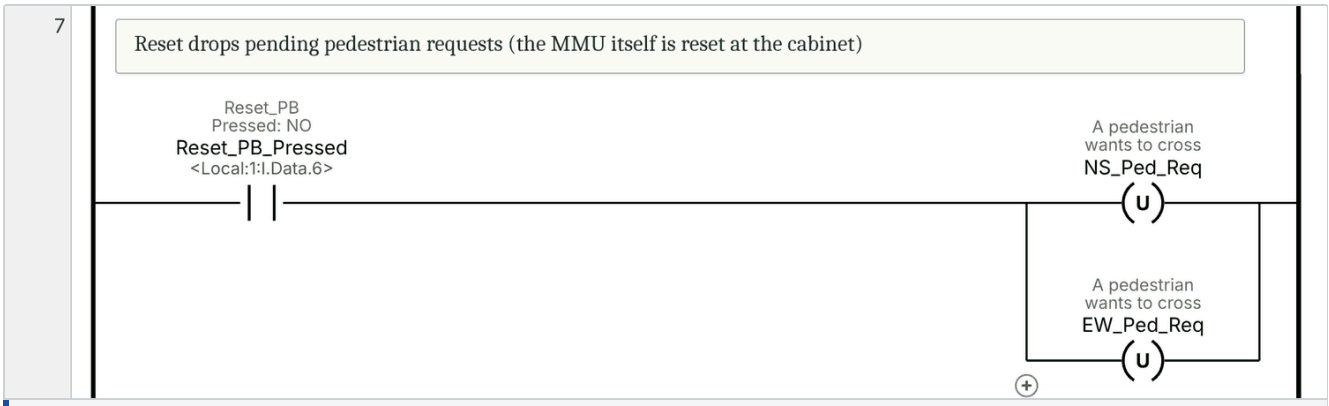
HOW THIS RUNG WORKS

In words. When `MMU_Fault` (Conflict monitor tripped: the cabinet is in red flash until the MMU is reset (click the cabinet)) is on, the rung turns `Lamp_Fault_On` (Lamp) on (and off again whenever the rung is false).

Left to right:

1. `XIC(MMU_Fault)` *Examine If Closed* asks: true when `MMU_Fault` (Conflict monitor tripped: the cabinet is in red flash until the MMU is reset (click the cabinet)) is on.
2. `OTE(Lamp_Fault_On)` *Output Energize* does: turns `Lamp_Fault_On` (Lamp) on (and off again whenever the rung is false).

MONITOR · RUNG 7




```
XIC(Reset_PB_Pressed)[OTU(NS_Ped_Req),OTU(EW_Ped_Req)];
```

Reset drops pending pedestrian requests (the MMU itself is reset at the cabinet)

HOW THIS RUNG WORKS

In words. When `Reset_PB_Pressed` (NO contact: 1 while pressed) is on, the rung unlatches `NS_Ped_Req` (A pedestrian wants to cross along N/S (latched by the button, cleared when served)) (turns it off) and unlatches `EW_Ped_Req` (A pedestrian wants to cross along E/W) (turns it off).

Left to right:

1. `XIC(Reset_PB_Pressed)` *Examine If Closed* asks: true when `Reset_PB_Pressed` (NO contact: 1 while pressed) is on.
2. A branch of 2 parallel paths: each path is tested on its own and does its own work.

   Path 1

   1. `OTU(NS_Ped_Req)` *Output Unlatch* does: unlatches `NS_Ped_Req` (A pedestrian wants to cross along N/S (latched by the button, cleared when served)) (turns it off).

   Path 2

   1. `OTU(EW_Ped_Req)` *Output Unlatch* does: unlatches `EW_Ped_Req` (A pedestrian wants to cross along E/W) (turns it off).

Flash is two things here. The selector on the cabinet puts the intersection into flashing red on purpose (night mode, or a technician working); the MMU puts it there by force. In both the program holds the cycle at all-red, so when flash ends the drivers get a clearance interval and then a green, never a green out of nowhere.

Guided: the intersection

The crossing opens with its guided track on as well (**File → New Project → Classic: traffic intersection**, or **Guided → Guide This Scene: the Intersection**). Seven lessons: **Init** (all-red on power-up), **Heads** (the reds from the step: both red, nothing else), **Cycle** (E/W gets its green and the cycle goes round), **Demand** (a car parked on a loop and a button press make requests), **Walk** (WALK comes with the matching green, DON'T WALK otherwise), **Monitor** (the run lamp, and the selector puts the heads in flash) and **Ready**. Heads comes before Cycle, which is backwards from the exercise order above and deliberate: with the

lamps written first, the first thing the Cycle lesson shows you is a green, and from then on every rung is checked against a crossing you can watch. The Cycle's JSR goes into the index as `XIO(Flash)JSR(Cycle)`, so the flash lesson at the end has something to hold.

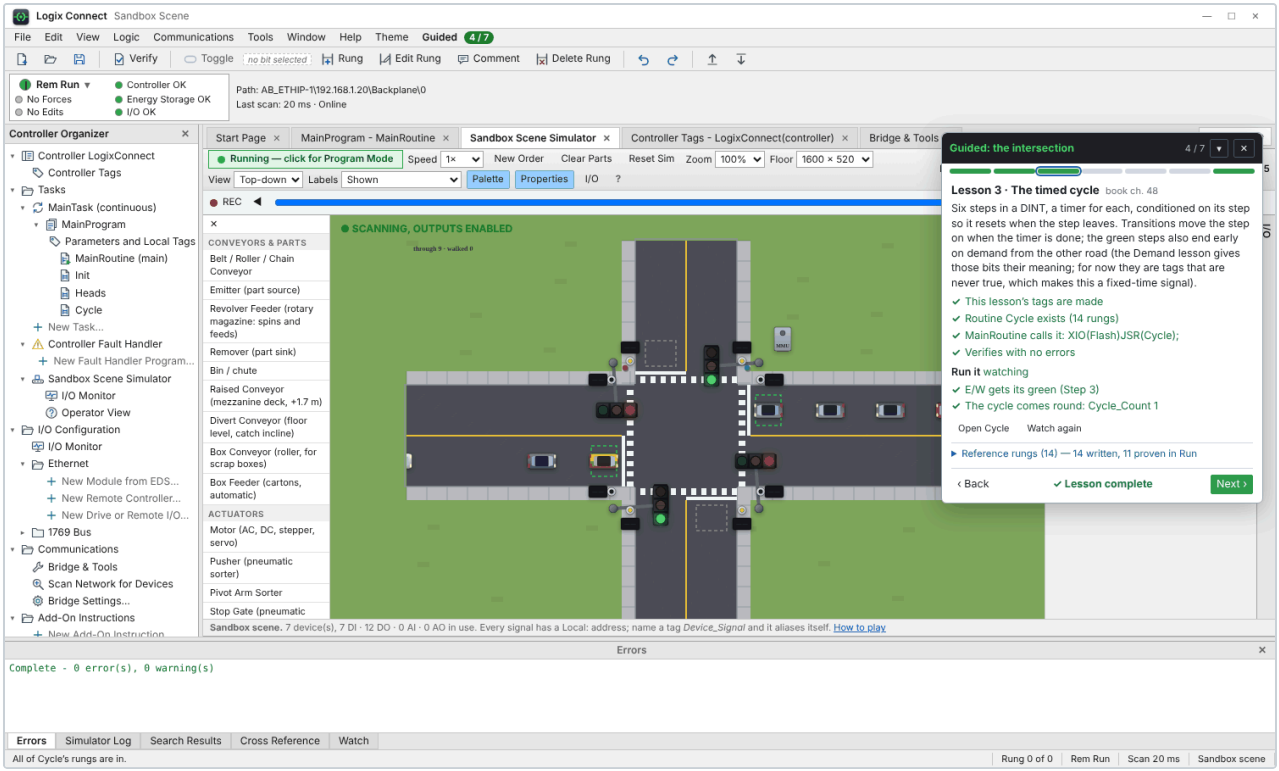


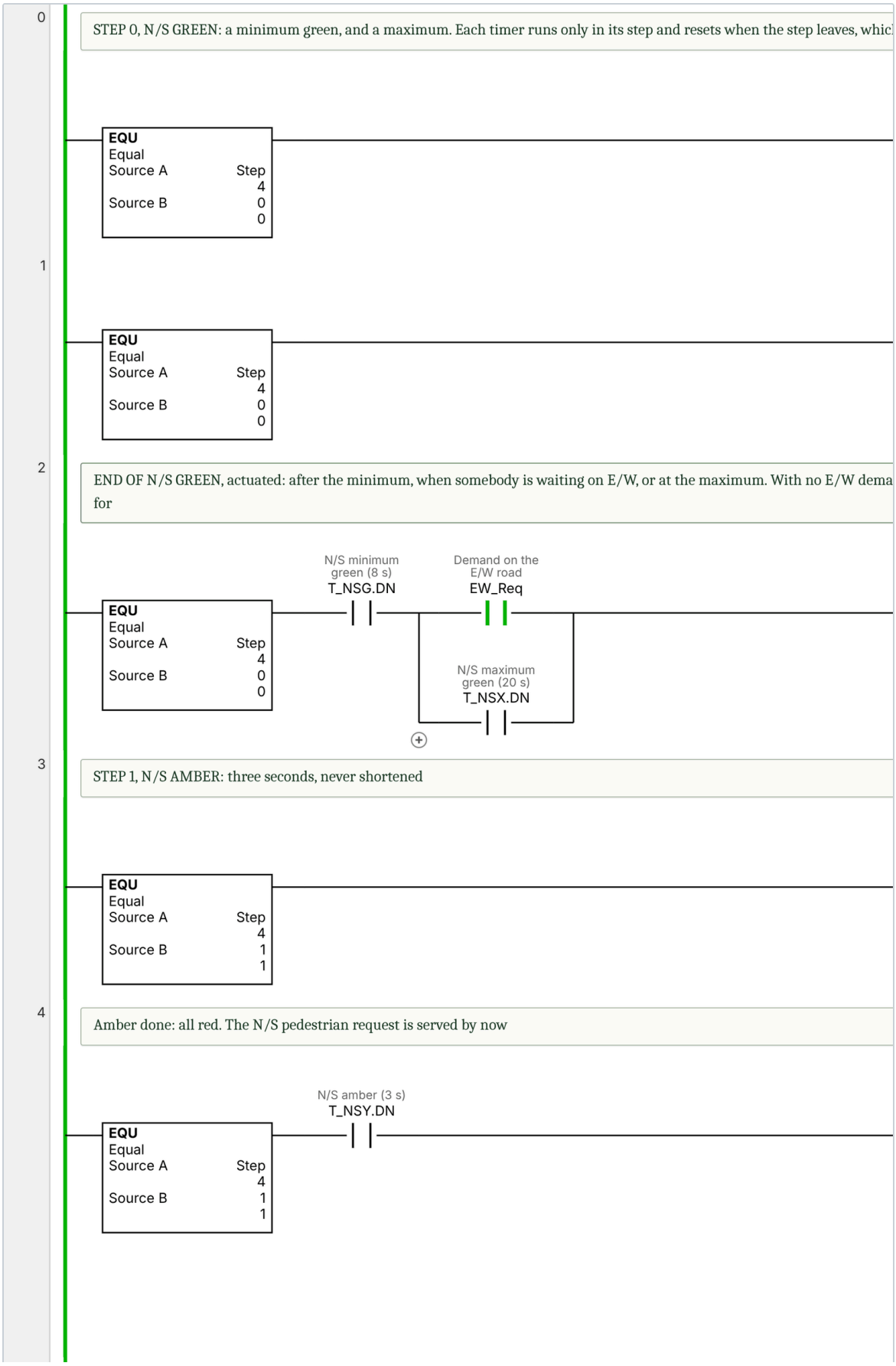
Figure 216. Lesson 3 of the intersection track: Init and Heads in, the Cycle routine running, E/W in its green.

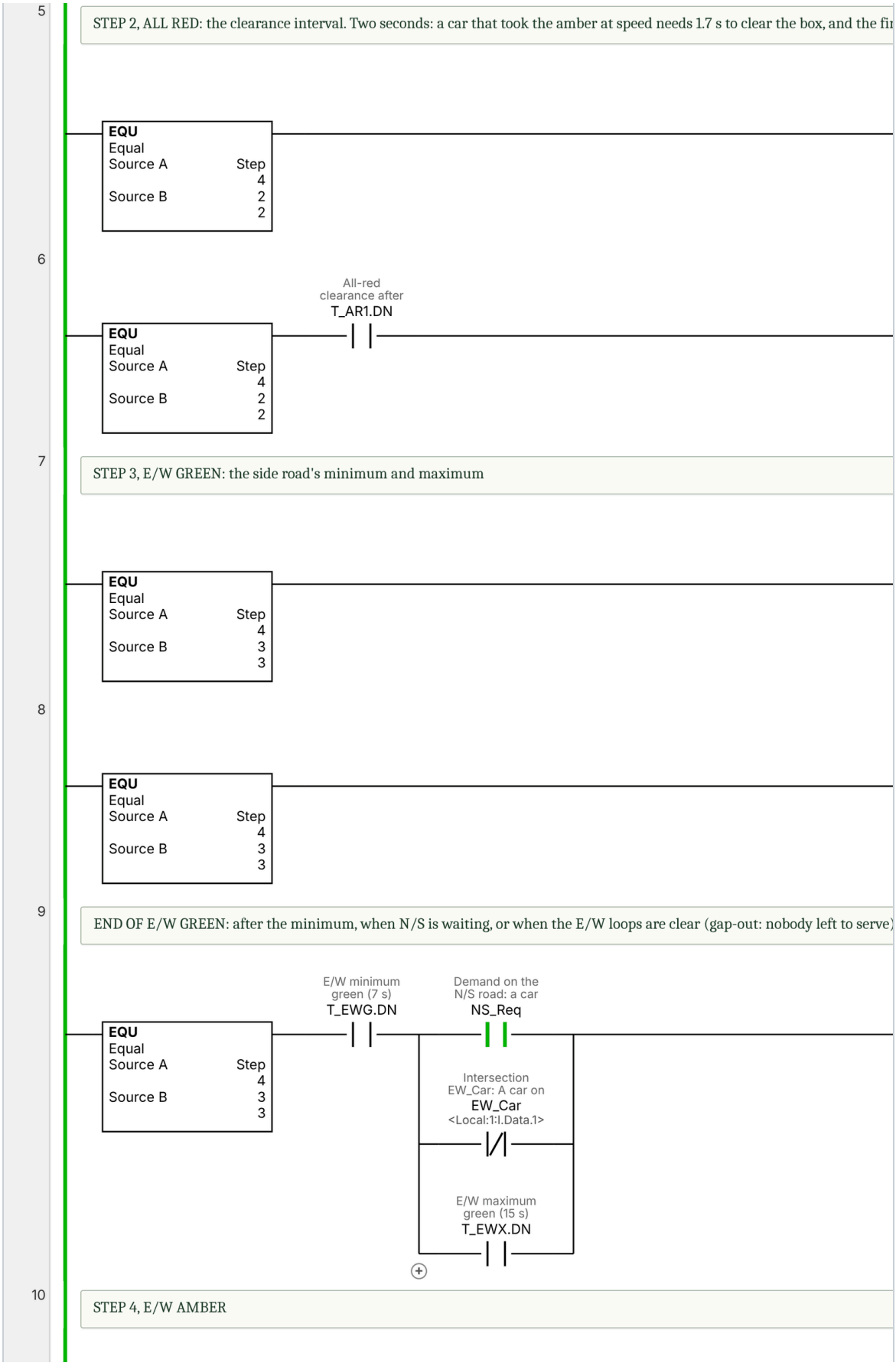
TRY IT: HINTS ONLY

Do the Cycle lesson without pressing Show rung once. The comments give the step numbers and the times; Hint gives the timers and the compares; the shape of a transition rung, `EQU(?,?)XIC(?)[XIC(?),XIC(?)]MOV(?,?)`, is the same rung four times with different numbers. The tally at the routine's heading should read 14 written (14 without the text), and if it does, you have written a step sequencer, which is most of industrial programming.

Run it

Run mode: both roads red for two seconds, then E/W green. Cars arrive, queue, and the main road settles into green; the first side-road car trips its loop, N/S goes amber, all-red, and the side road gets its green. A pedestrian presses at a corner; the WALK comes with the next matching green, they cross, the sign flashes DON'T WALK behind them. Let it run ten minutes and read the count at the top of the part: through, walked, and no conflicts.


0
STEP 0, N/S GREEN: a minimum green, and a maximum. Each timer runs only in its step and resets when the step leaves, whic
EQU
Equal
Source A
Step
4
Source B
0
0
1
EQU
Equal
Source A
Step
4
Source B
0
0
2
END OF N/S GREEN, actuated: after the minimum, when somebody is waiting on E/W, or at the maximum. With no E/W dema
for
N/S minimum green (8 s)
T_NSG.DN
Demand on the E/W road
EW_Req
EQU
Equal
Source A
Step
4
Source B
0
0
N/S maximum green (20 s)
T_NSX.DN
3
STEP 1, N/S AMBER: three seconds, never shortened
EQU
Equal
Source A
Step
4
Source B
1
1
4
Amber done: all red. The N/S pedestrian request is served by now
N/S amber (3 s)
T_NSY.DN
EQU
Equal
Source A
Step
4
Source B
1
1


5
STEP 2, ALL RED: the clearance interval. Two seconds: a car that took the amber at speed needs 1.7 s to clear the box, and the fi
EQU
Equal
Source A
Step
4
Source B
2
2
6
All-red
clearance after
T_AR1.DN
EQU
Equal
Source A
Step
4
Source B
2
2
7
STEP 3, E/W GREEN: the side road's minimum and maximum
EQU
Equal
Source A
Step
4
Source B
3
3
8
EQU
Equal
Source A
Step
4
Source B
3
3
9
END OF E/W GREEN: after the minimum, when N/S is waiting, or when the E/W loops are clear (gap-out: nobody left to serve)
E/W minimum
green (7 s)
T_EWG.DN
Demand on the
N/S road: a car
NS_Req
EQU
Equal
Source A
Step
4
Source B
3
3
Intersection
EW_Car: A car on
EW_Car
<Local:1:I.Data.1>
E/W maximum
green (15 s)
T_EWX.DN
10
STEP 4, E/W AMBER

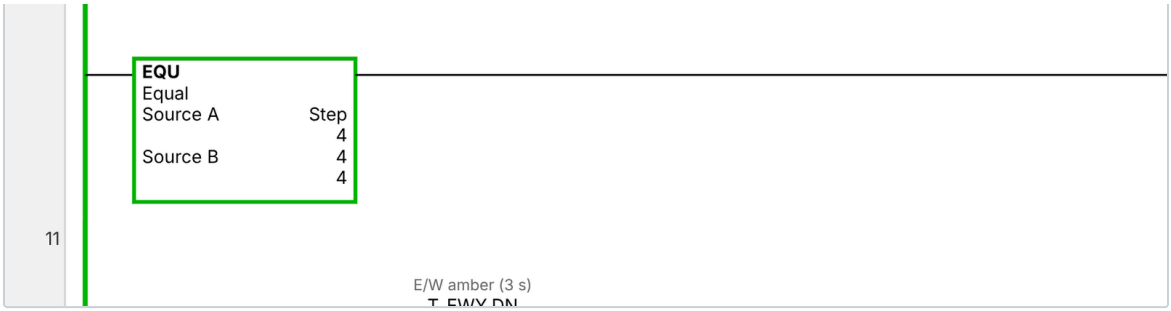


Figure 217. The Cycle routine online, N/S in its green with the maximum timer running and a car on an E/W loop.

> **TRY IT**
>
> Add a left-turn phase: a protected N/S left on a fourth lamp (a green arrow) before the N/S through green, with its own minimum and its own all-red, and work out what the MMU would need to know about it. Then the exercise as the original sets it: throw away the Cycle routine and build the six steps with a single SQO sequencer and one timer. The car wash chapter, when it comes, is that instruction's home.

Machine faults on the intersection

Tools → Inject Machine Fault… lists the faults that belong to the parts on this floor. Each has a fault card: the symptom, what to watch in the I/O to confirm it, the repair, and what a program should do about it. **Repair** on an active fault removes its cause the way a technician would; whether the machine runs again is up to the program. The failed red load switch is the one the cabinet catches for you: the conflict monitor sees the dark approach (RED FAIL) and flashes the intersection, and the reference's Monitor routine follows it into flash. The detectors and the button are yours to catch.

Fault	Watch	Repair	The program should
Intersection: N/S red load switch failed The program commands N/S red but no voltage reaches the heads: the N/S approach goes dark. The conflict monitor detects it (RED FAIL) and puts the cabinet into flash.	NS_Red 1 from the program, the N/S heads dark, then MMU_Fault 1.	Replace the load switch (Repair), then reset the conflict monitor at the cabinet.	MMU_Fault: flash mode, the cycle held at all-red, the fault lamp. The Monitor routine of the reference does exactly this.
Intersection: E/W loop detector stuck on The E/W detector reports a car whether there is one or not.	EW_Car 1 with the E/W loops empty; the main road's green is cut short every cycle.	Retune the detector (Repair).	A detector on for longer than any car sits (two minutes) is treated as failed: alarm, and run that road on its maximum (max recall).
Intersection: E/W loop detector dead The E/W detector never reports a car.	Cars queued on E/W, EW_Car 0: they wait for the main road's maximum.	Replace the detector (Repair).	A detector that never changes for a long time is treated as failed: give its road a fixed recall so nobody waits for ever.

Fault	Watch	Repair	The program should
Intersection: N/S push button stuck in The N/S pedestrian button is jammed in.	NS_Ped_PB 1 all the time: a WALK every N/S green.	Free the button (Repair).	A button held for more than ten seconds is ignored and alarmed.

Predict it: the answers

1. Every condition on the way is met, so `Step` becomes **1**.
2. Every condition on the way is met, so `Step` becomes **2** and `NS_Ped_Req` is unlatched, **off**.
3. Every condition on the way is met, so `Step` becomes **3**.
4. The rung is false at the EQU: `Step` is not **3**, so the MOV does nothing: `Step` keeps its value.
5. The rung is false at `XIO(Flash)`: `Flash` is on, so `NS_Walk` is **off** (an OTE writes 0 when its rung is false).
6. Every condition on the way is met, so `NS_DontWalk` is **on**.
7. Every condition on the way is met, so `EW_Walk` is **on**.
8. The rung is false at `XIO(EW_Walk)`: `EW_Walk` is on, so `EW_DontWalk` is **off** (an OTE writes 0 when its rung is false).
9. Every condition on the way is met, so `EW_Req` is **on**.
10. Every condition on the way is met, so `NS_Req` is **on**.
11. Every condition on the way is met, so `NS_Green` is **on**.
12. The rung is false at `XIO(Flash)`: `Flash` is on, so `NS_Yellow` is **off** (an OTE writes 0 when its rung is false).
13. Every condition on the way is met, so `NS_Red` is **on**.
14. The rung is false at the EQU: `Step` is not **3**, so `EW_Green` is **off** (an OTE writes 0 when its rung is false).
15. Every condition on the way is met, so `EW_Yellow` is **on**.
16. Every condition on the way is met, so `EW_Red` is **on**.
17. Every condition on the way is met, so `Flash` is **on**.

49 The sample sorter, programmed

The sample sorter is the first scene most people open: **File → New Project → Sandbox: sample sorter scene**. An emitter drops cartons and the odd metal box onto two belts; an entry eye, an inductive prox, a pusher over a bin, a colour sensor, a pivot arm over a second bin and a remover at the end; a panel with Start, Stop, an E-stop, a HAND / AUTO selector, a stack light with a buzzer, a fault lamp, a count display and a speed pot; and a wash tank with its floats and a mixer. It has always come with no logic. From Version 134 it has a reference program (**File → Load Reference Solution: Sample sorter**) and a guided track (**File → New Project → Guided: sample sorter**, or **Guided → Guide This Scene: the Sample Sorter** on the plain scene). This chapter is that program, in the order a sorting line is commissioned: make it run and stop, feed it, count it, sort it, protect it.

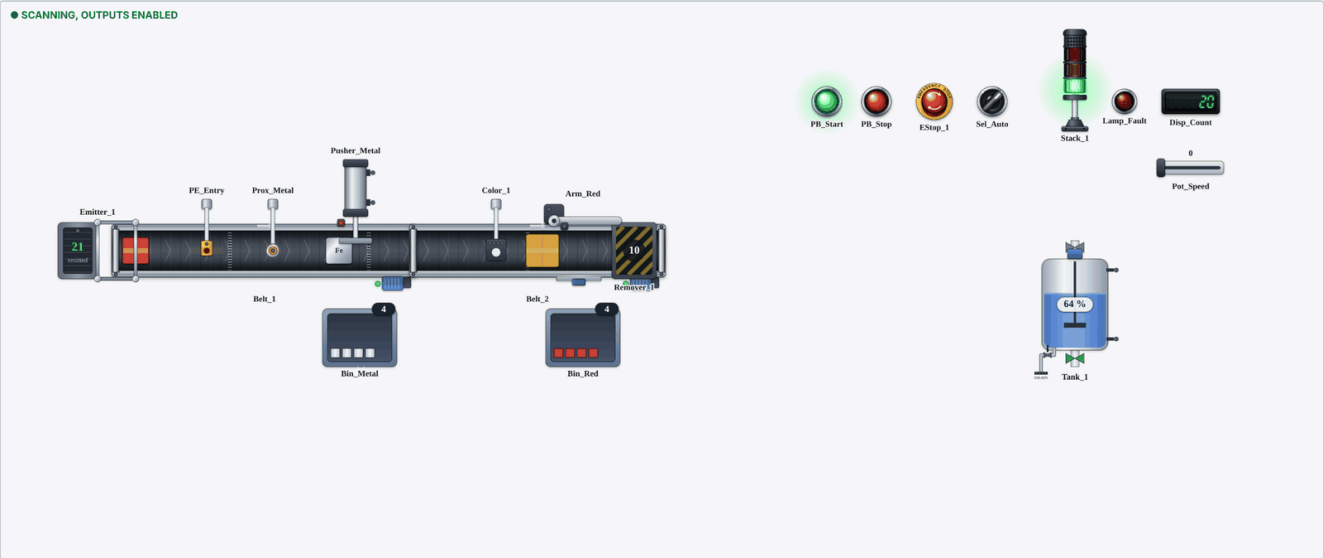


Figure 218. The sample sorter running the reference in AUTO: metal boxes in Bin_Metal, red ones in Bin_Red, the rest to the remover, the count on the display, the stack light green, the wash tank mixing.

The tags

Every signal gets an alias named *Device_Signal*, which is what **Alias all I/O** in the scene's properties makes, so `PB_Stop_Pressed` is the Stop button's contact and `Pusher_Metal_Extended` the pusher's front reed switch. The guided project makes them for you. The program's own tags:

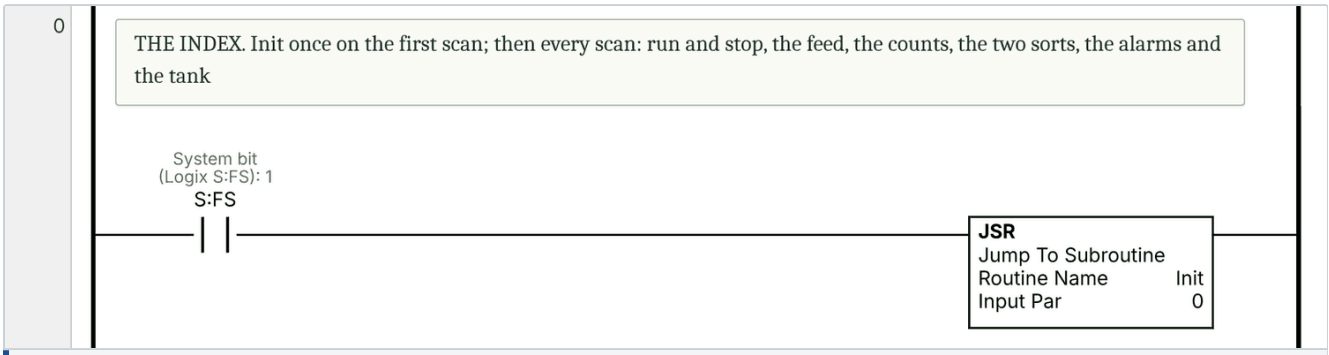
Tag	Type	Description
`Run`	BOOL	The line is running: Start sealed in, Stop and the E-stop healthy, no bin full
`Feed_ms`	DINT	Time between parts in AUTO, from the speed pot: 2000 + 40 x pot (2 to 6 s)
`T_Feed`	TIMER	Feed interval timer; its ACC is compared with Feed_ms
`Feed_Pulse`	BOOL	One scan: release a part (AUTO)

Tag	Type	Description
`Hand_ONS`	BOOL	One-shot storage for the HAND release
`Hand_Feed`	BOOL	One scan: release a part (HAND, on a Start press)
`Entry_ONS`	BOOL	One-shot storage for the entry count
`Out_ONS`	BOOL	One-shot storage for the pass-through count
`Parts_In`	DINT	Parts seen by PE_Entry, shown on Disp_Count
`Parts_Out`	DINT	Parts that reached the remover unsorted
`Metal_Seen`	BOOL	A metal part has passed Prox_Metal and is on its way to the pusher
`Red_Seen`	BOOL	A red part has passed Color_1 and is on its way to the arm
`Red_At_Arm`	BOOL	That red part has reached the arm's eye
`T_Arm`	TIMER	The arm stays in the lane 1 s after the red part leaves its eye
`Bin_Full`	BOOL	Either bin is full: the line stops until it is emptied
`T_Buzz`	TIMER	The buzzer sounds for the first 5 s of a bin-full stop

The index

Like the elevator and the intersection, the main routine is a table of contents: one JSR per job, Init only on the first scan.

MAINROUTINE · RUNG 0




```
XIC(S:FS)JSR(Init);
```

THE INDEX. Init once on the first scan; then every scan: run and stop, the feed, the counts, the two sorts, the alarms and the tank

HOW THIS RUNG WORKS

In words. When `S:FS` is on, the rung runs the routine `Init`.

Left to right:

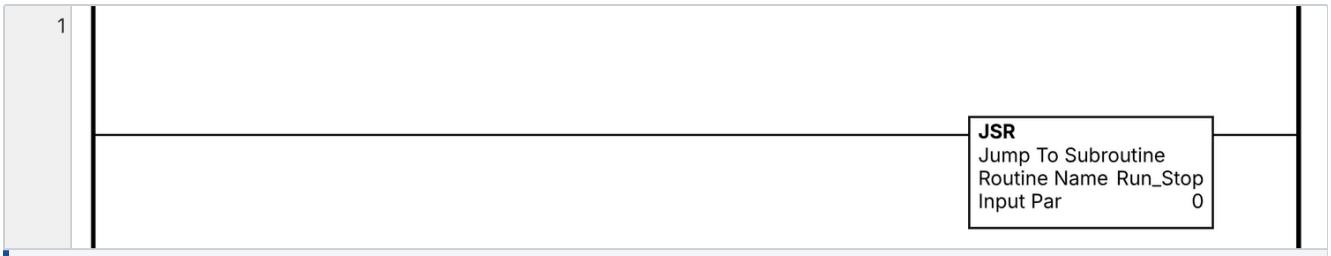
1. `XIC(S:FS)` *Examine If Closed* asks: true when `S:FS` is on.
2. `JSR(Init)` *Jump to Subroutine* does: runs the routine `Init`. The routine is scanned at this point, then the scan comes back here.

Why it is built this way:

Part of the index. The main routine is a list of calls; each routine holds one job, and the scan goes through them in this order.

> **Think of it like this:** the table of contents of a book. The main routine does no work itself: it lists the chapters and the order to read them in, every scan.

MAINROUTINE · RUNG 1




```
JSR(Run_Stop);
```

HOW THIS RUNG WORKS

In words. Every scan, with nothing to test, the rung runs the routine `Run_Stop`.

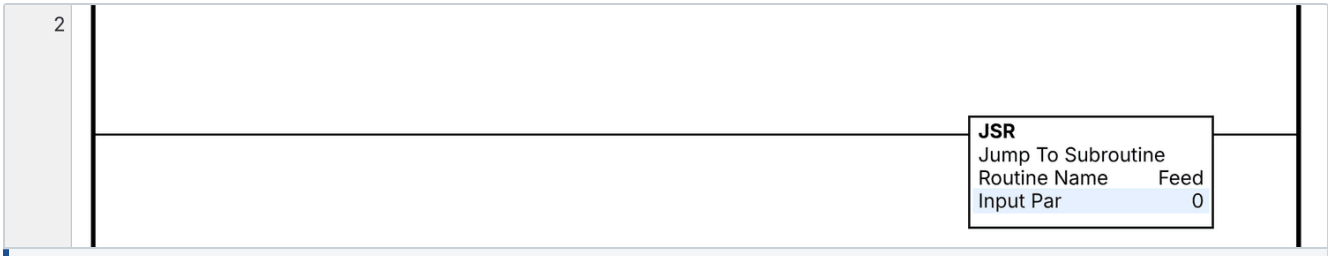
Left to right:

1. `JSR(Run_Stop)` *Jump to Subroutine* does: runs the routine `Run_Stop`.

Why it is built this way:

Part of the index: one call, one job.

MAINROUTINE · RUNG 2




```
JSR(Feed);
```

HOW THIS RUNG WORKS

In words. Every scan, with nothing to test, the rung runs the routine `Feed`.

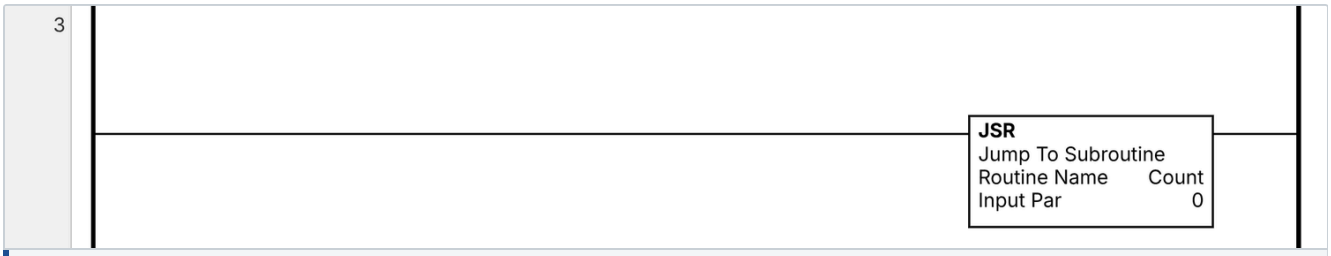
Left to right:

1. `JSR(Feed)` *Jump to Subroutine* does: runs the routine `Feed`.

Why it is built this way:

Part of the index: one call, one job.

MAINROUTINE · RUNG 3




```
JSR(Count);
```

HOW THIS RUNG WORKS

In words. Every scan, with nothing to test, the rung runs the routine `Count`.

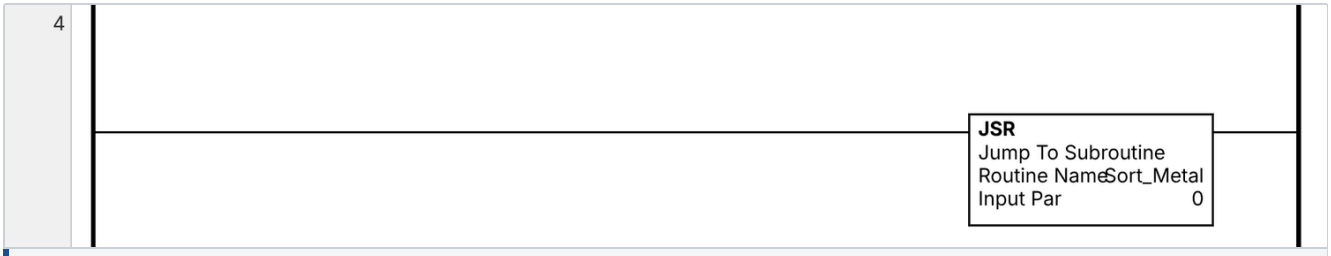
Left to right:

1. `JSR(Count)` *Jump to Subroutine* does: runs the routine `Count`.

Why it is built this way:

Part of the index: one call, one job.

MAINROUTINE · RUNG 4




```
JSR(Sort_Metal);
```

HOW THIS RUNG WORKS

In words. Every scan, with nothing to test, the rung runs the routine `Sort_Metal`.

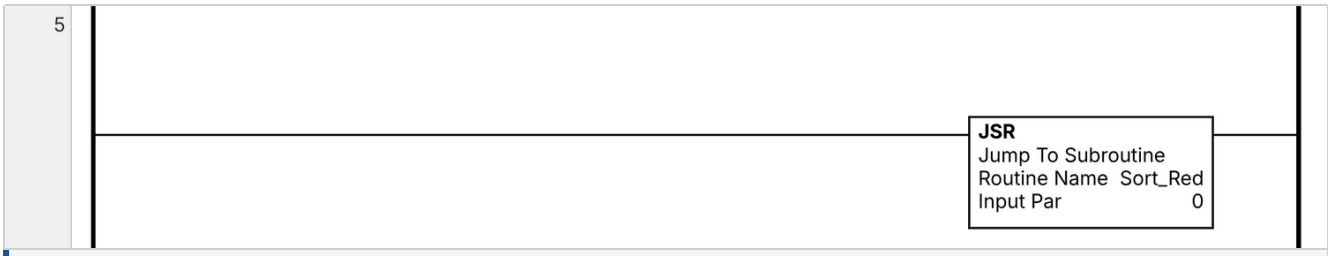
Left to right:

1. `JSR(Sort_Metal)` *Jump to Subroutine* does: runs the routine `Sort_Metal`.

Why it is built this way:

Part of the index: one call, one job.

MAINROUTINE · RUNG 5




```
JSR(Sort_Red);
```

HOW THIS RUNG WORKS

In words. Every scan, with nothing to test, the rung runs the routine `Sort_Red`.

Left to right:

1. `JSR(Sort_Red)` *Jump to Subroutine* does: runs the routine `Sort_Red`.

Why it is built this way:

Part of the index: one call, one job.

MAINROUTINE · RUNG 6

```
JSR(Alarms);
```

HOW THIS RUNG WORKS

In words. Every scan, with nothing to test, the rung runs the routine `Alarms`.

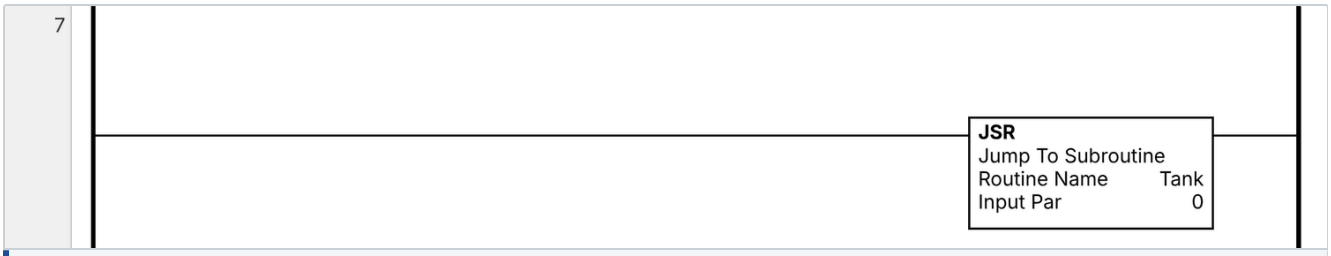
Left to right:

1. `JSR(Alarms)` *Jump to Subroutine* does: runs the routine `Alarms`.

Why it is built this way:

Part of the index: one call, one job.

MAINROUTINE · RUNG 7




```
JSR(Tank);
```

HOW THIS RUNG WORKS

In words. Every scan, with nothing to test, the rung runs the routine `Tank`.

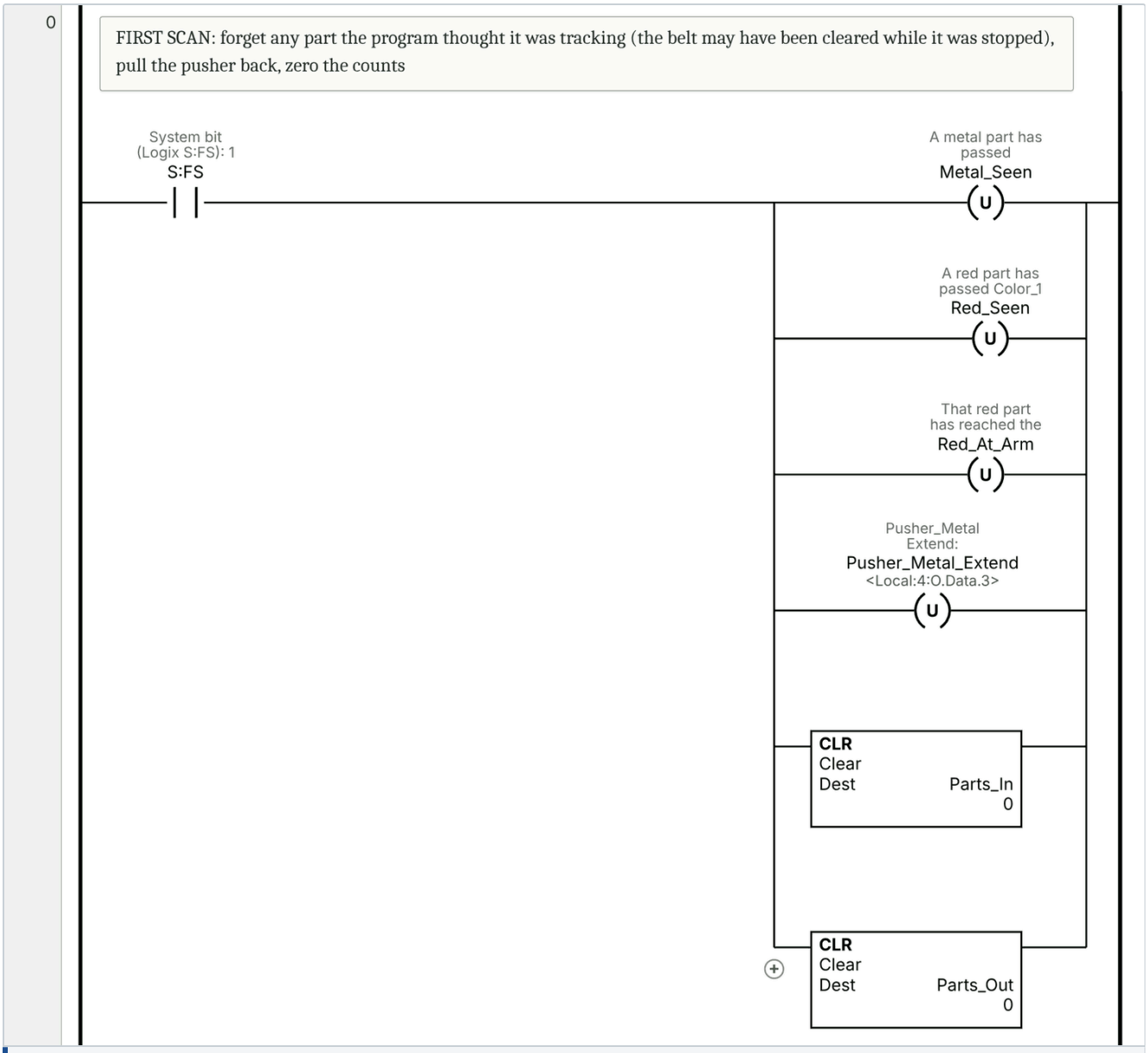
Left to right:

1. `JSR(Tank)` *Jump to Subroutine* does: runs the routine `Tank`.

Why it is built this way:

Part of the index: one call, one job.

INIT · RUNG 0




```
XIC(S:FS)
[OTU(Metal_Seen),OTU(Red_Seen),OTU(Red_At_Arm),OTU(Pusher_Metal_Extend),CLR(Par
ts_In),CLR(Parts_Out)];
```

FIRST SCAN: forget any part the program thought it was tracking (the belt may have been cleared while it was stopped), pull the pusher back, zero the counts

HOW THIS RUNG WORKS

In words. When `S:FS` is on, the rung unlatches `Metal_Seen` (A metal part has passed Prox_Metal and is on its way to the pusher) (turns it off), unlatches `Red_Seen` (A red part has passed Color_1 and is on its way to the arm) (turns it off), unlatches `Red_At_Arm` (That red part has reached the arm's eye) (turns it off), unlatches `Pusher_Metal_Extend` (Solenoid: extend cylinder) (turns it off), clears `Parts_In` (Parts seen by PE_Entry, shown on Disp_Count) to 0 and clears `Parts_Out` (Parts that reached the remover unsorted) to 0.

Left to right:

1. `XIC(S:FS)` *Examine If Closed* asks: true when `S:FS` is on.
2. A branch of 6 parallel paths: each path is tested on its own and does its own work.

   Path 1

   1. `OTU(Metal_Seen)` *Output Unlatch* does: unlatches `Metal_Seen` (A metal part has passed Prox_Metal and is on its way to the pusher) (turns it off). It only ever clears the bit.

   Path 2

   1. `OTU(Red_Seen)` *Output Unlatch* does: unlatches `Red_Seen` (A red part has passed Color_1 and is on its way to the arm) (turns it off).

   Path 3

   1. `OTU(Red_At_Arm)` *Output Unlatch* does: unlatches `Red_At_Arm` (That red part has reached the arm's eye) (turns it off).

   Path 4

   1. `OTU(Pusher_Metal_Extend)` *Output Unlatch* does: unlatches `Pusher_Metal_Extend` (Solenoid: extend cylinder) (turns it off).

   Path 5

   1. `CLR(Parts_In)` *Clear* does: clears `Parts_In` (Parts seen by PE_Entry, shown on Disp_Count) to 0.

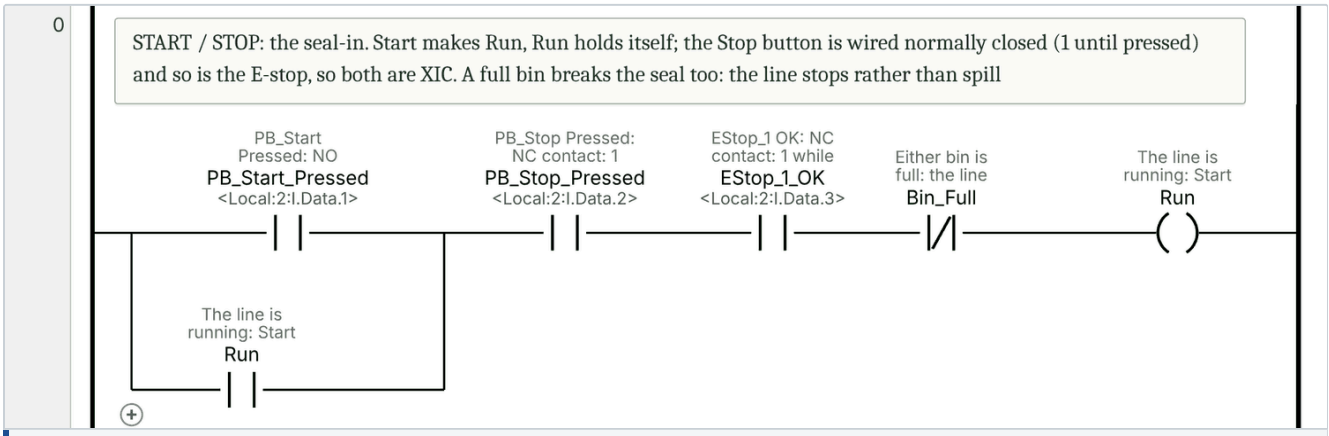
   Path 6

   1. `CLR(Parts_Out)` *Clear* does: clears `Parts_Out` (Parts that reached the remover unsorted) to 0.

> **Think of it like this:** a rung is a pipe with valves in it, read left to right. Every contact is a valve: XIC is open while its bit is 1, XIO while its bit is 0. Water reaches the output at the right only if every valve in a row is open. The output does not remember anything: on every scan it is simply wet or dry.

Start and stop

RUN_STOP · RUNG 0




```
[XIC(PB_Start_Pressed),XIC(Run)]XIC(PB_Stop_Pressed)XIC(EStop_1_OK)XIO(Bin_Full
)OTE(Run);
```

START / STOP: the seal-in. Start makes Run, Run holds itself; the Stop button is wired normally closed (1 until pressed) and so is the E-stop, so both are XIC. A full bin breaks the seal too: the line stops rather than spill

PREDICT IT · 1 `PB_Start_Pressed` is on, `PB_Stop_Pressed` is on, `EStop_1_OK` is on and `Bin_Full` is off. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When either (`PB_Start_Pressed` (NO contact: 1 while pressed) is on) *or* (`Run` (The line is running: Start sealed in, Stop and the E-stop healthy, no bin full) is on), `PB_Stop_Pressed` (NC contact: 1 until pressed) is on, `EStop_1_OK` (NC contact: 1 while healthy, 0 when pressed) is on and `Bin_Full` (Either bin is full: the line stops until it is emptied) is off, the rung turns `Run` on (and off again whenever the rung is false).

Left to right:

1. A branch of 2 parallel paths: power gets through if any one of them is true.
   - **Path 1**
     1. `XIC(PB_Start_Pressed)` *Examine If Closed* asks: true when `PB_Start_Pressed` (NO contact: 1 while pressed) is on.
   - **Path 2**
     1. `XIC(Run)` *Examine If Closed* asks: true when `Run` (The line is running: Start sealed in, Stop and the E-stop healthy, no bin full) is on.
2. `XIC(PB_Stop_Pressed)` *Examine If Closed* asks: true when `PB_Stop_Pressed` (NC contact: 1 until pressed) is on.
3. `XIC(EStop_1_OK)` *Examine If Closed* asks: true when `EStop_1_OK` (NC contact: 1 while healthy, 0 when pressed) is on.
4. `XIO(Bin_Full)` *Examine If Open* asks: true when `Bin_Full` (Either bin is full: the line stops until it is emptied) is off.
5. `OTE(Run)` *Output Energize* does: turns `Run` on (and off again whenever the rung is false). Every scan it writes the rung state: true on, false off.

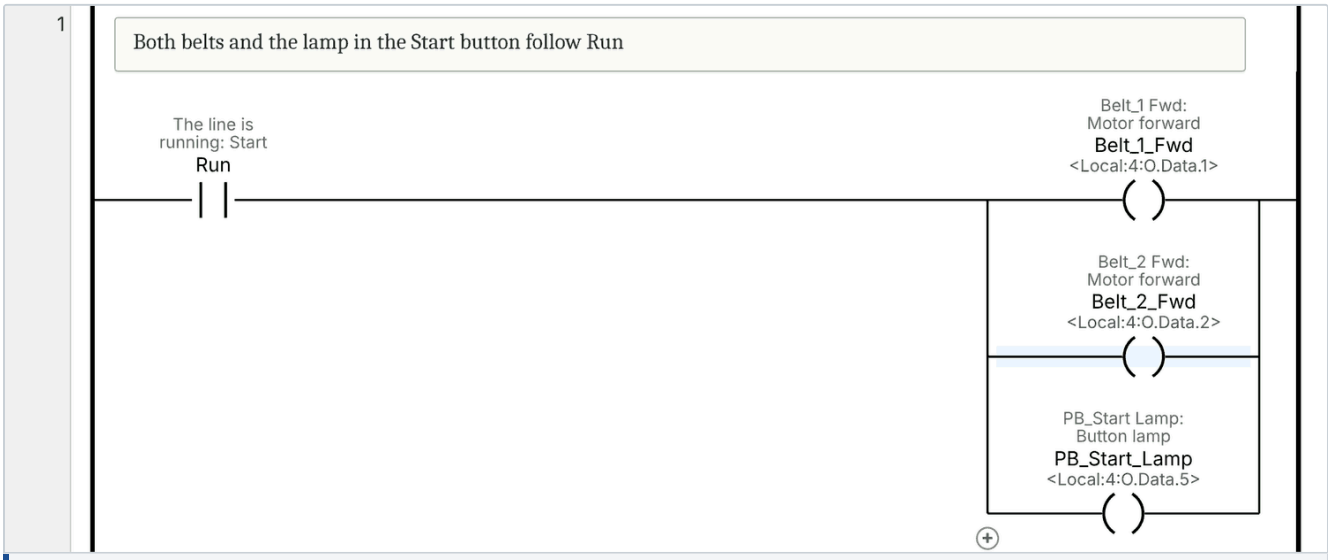
Why it is built this way:

Seal-in. The start condition turns `Run` on, and `Run`'s own contact in the parallel path keeps the rung true after the start goes away. Anything in series outside the branch (a stop, a fault) breaks it, and then it stays off until the start is made again.

> **Think of it like this:** a doorbell button and a hand. The start button is the doorbell, pressed for a moment. The seal-in contact is a hand that grabs the circuit as soon as the output comes on and keeps holding after you let go. The stop button, in series, is the only thing that can knock that hand away.

> **Watch for:** The stop has to be in series outside the branch. Put it inside the start path only and the seal-in path bypasses it: the stop no longer stops.

RUN_STOP · RUNG 1




```
XIC(Run)[OTE(Belt_1_Fwd),OTE(Belt_2_Fwd),OTE(PB_Start_Lamp)];
```

Both belts and the lamp in the Start button follow Run

HOW THIS RUNG WORKS

In words. When `Run` (The line is running: Start sealed in, Stop and the E-stop healthy, no bin full) is on, the rung turns `Belt_1_Fwd` (Motor forward) on (and off again whenever the rung is false), turns `Belt_2_Fwd` (Motor forward) on (and off again whenever the rung is false) and turns `PB_Start_Lamp` (Button lamp) on (and off again whenever the rung is false).

Left to right:

1. `XIC(Run)` *Examine If Closed* asks: true when `Run` (The line is running: Start sealed in, Stop and the E-stop healthy, no bin full) is on.
2. A branch of 3 parallel paths: each path is tested on its own and does its own work.

   Path 1

   1. `OTE(Belt_1_Fwd)` *Output Energize* does: turns `Belt_1_Fwd` (Motor forward) on (and off again whenever the rung is false).

   Path 2

   1. `OTE(Belt_2_Fwd)` *Output Energize* does: turns `Belt_2_Fwd` (Motor forward) on (and off again whenever the rung is false).

   Path 3

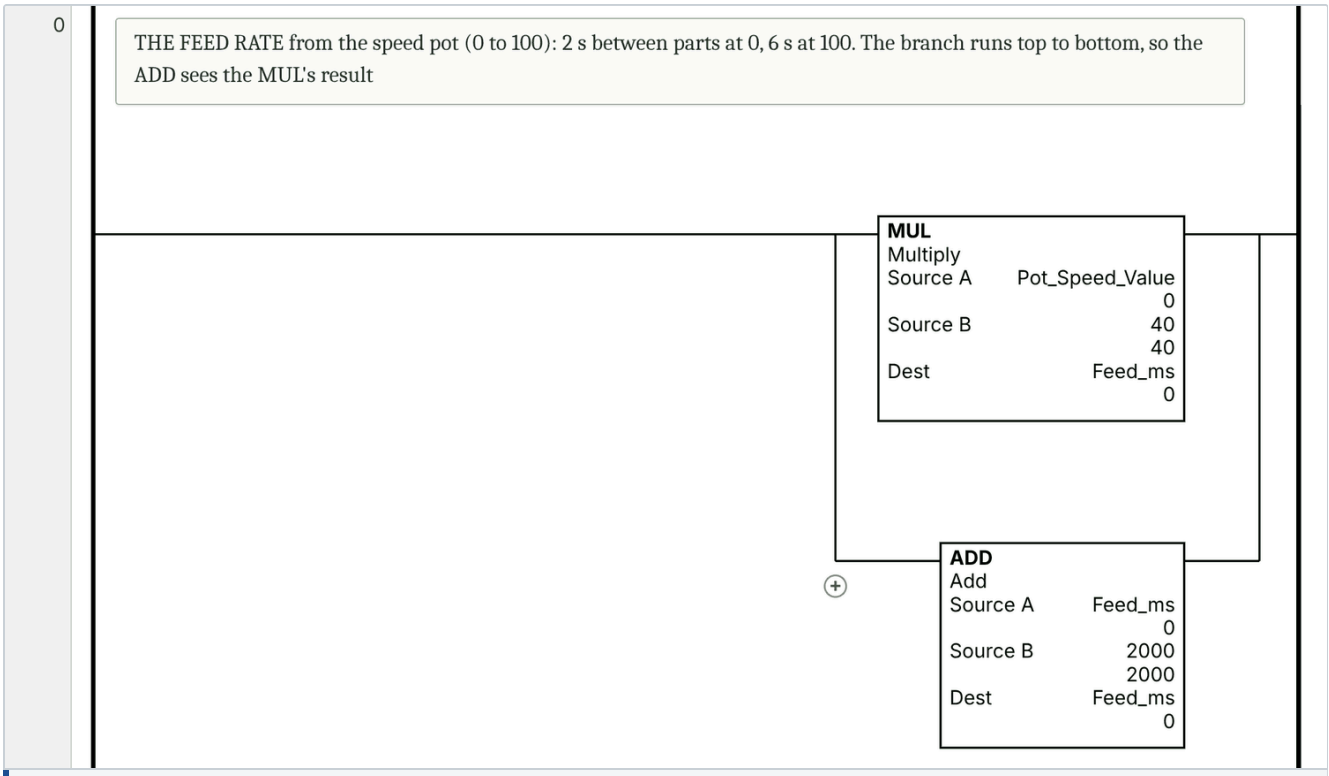
   1. `OTE(PB_Start_Lamp)` *Output Energize* does: turns `PB_Start_Lamp` (Button lamp) on (and off again whenever the rung is false).

> **WATCH FOR: THE STOP BUTTON IS AN XIC**
>
> Stop and the E-stop are wired normally closed: the contact is made, the input is 1, until somebody presses. So the rung tests them with XIC, and a broken wire stops the line exactly as a press would. Write Stop as XIO and the line runs only while Stop is held. On the panel that is the difference between a red button that works and one that is a Start button in disguise.

The feed

The emitter releases a part on a rising edge of `Emit`, and only while its exit is clear (`PartReady`). In HAND each Start press releases one part; in AUTO a timer releases one every `Feed_ms`, which the pot sets between two and six seconds. The timer's preset is a ceiling, not the interval: the interval is the GEQ on its accumulator, which is how you get a timer whose time comes from an analog input without writing to .PRE.

FEED · RUNG 0




```
[MUL(Pot_Speed_Value,40,Feed_ms),ADD(Feed_ms,2000,Feed_ms)];
```

THE FEED RATE from the speed pot (0 to 100): 2 s between parts at 0, 6 s at 100. The branch runs top to bottom, so the ADD sees the MUL's result

HOW THIS RUNG WORKS

In words. Every scan, with nothing to test, the rung makes `Feed_ms` (Time between parts in AUTO, from the speed pot: 2000 + 40 x pot (2 to 6 s)) = `Pot_Speed_Value` (0-100) × **40** and makes `Feed_ms` = `Feed_ms` + **2000**.

Left to right:

1. A branch of 2 parallel paths: each path is tested on its own and does its own work.

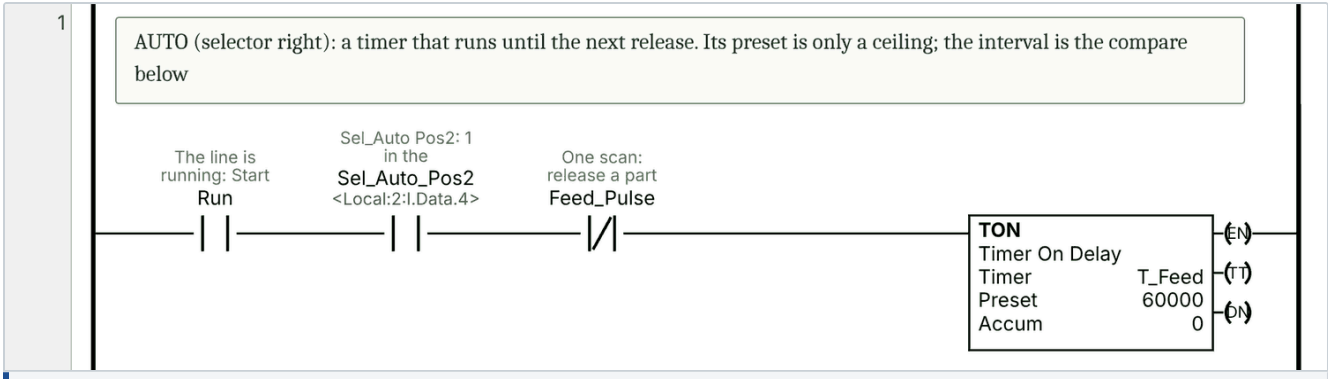
   Path 1

   1. `MUL(Pot_Speed_Value,40,Feed_ms)` *Multiply* does: makes `Feed_ms` (Time between parts in AUTO, from the speed pot: 2000 + 40 x pot (2 to 6 s)) = `Pot_Speed_Value` (0-100) × **40**.

   Path 2

   1. `ADD(Feed_ms,2000,Feed_ms)` *Add* does: makes `Feed_ms` = `Feed_ms` + **2000**. Every scan the rung is true; behind a one-shot it adds once per event.

FEED · RUNG 1




```
XIC(Run)XIC(Sel_Auto_Pos2)XIO(Feed_Pulse)TON(T_Feed,60000,0);
```

AUTO (selector right): a timer that runs until the next release. Its preset is only a ceiling; the interval is the compare below

PREDICT IT · 2 `Run` is off, `Sel_Auto_Pos2` is on and `Feed_Pulse` is off. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Run` (The line is running: Start sealed in, Stop and the E-stop healthy, no bin full) is on, `Sel_Auto_Pos2` (1 in the right-hand position) is on and `Feed_Pulse` (One scan: release a part (AUTO)) is off, the rung runs the timer `T_Feed` (Feed interval timer; its ACC is compared with Feed_ms) toward 60000 ms (60 s).

Left to right:

1. `XIC(Run)` *Examine If Closed* asks: true when `Run` (The line is running: Start sealed in, Stop and the E-stop healthy, no bin full) is on.
2. `XIC(Sel_Auto_Pos2)` *Examine If Closed* asks: true when `Sel_Auto_Pos2` (1 in the right-hand position) is on.
3. `XIO(Feed_Pulse)` *Examine If Open* asks: true when `Feed_Pulse` (One scan: release a part (AUTO)) is off.
4. `TON(T_Feed,60000,0)` *Timer On Delay* does: runs the timer `T_Feed` (Feed interval timer; its ACC is compared with Feed_ms) toward 60000 ms (60 s). The accumulator counts while the rung is true; .DN comes on at the preset; the moment the rung goes false the timer resets.

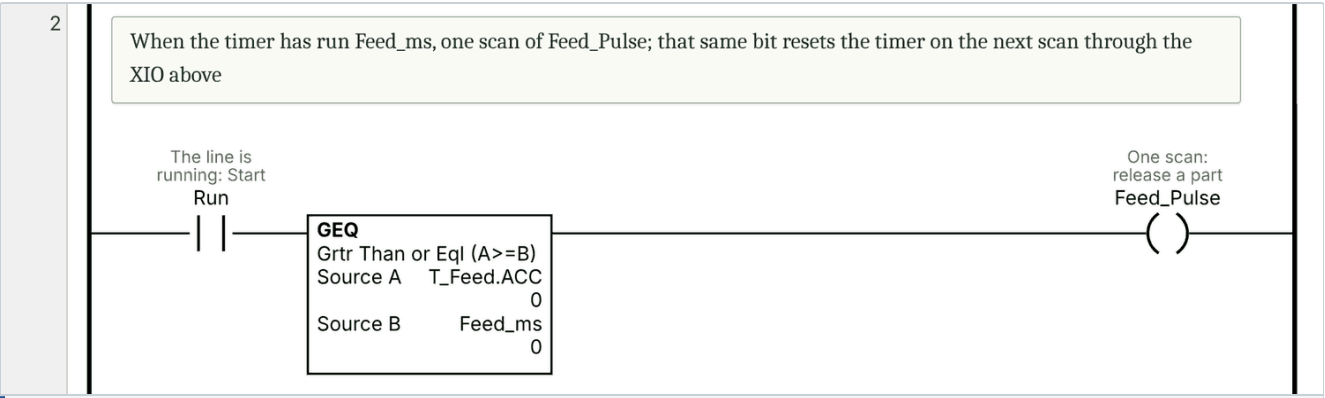
Why it is built this way:

A timer. `T_Feed` only times while this rung is true; other rungs use its `.DN` (done) or `.TT` (timing) bits. Being a TON, it starts again from zero every time the rung goes false.

> **Think of it like this:** a kitchen timer that only runs while you hold your finger on it. Take your finger off and it snaps back to zero. Hold it long enough and the done light comes on, and stays on as long as your finger is still there.

> **Watch for:** If anything in series flickers false for one scan, a TON starts over from zero. Use RTO when the time must survive an interruption.

FEED · RUNG 2




```
XIC(Run)GEQ(T_Feed.ACC,Feed_ms)OTE(Feed_Pulse);
```

When the timer has run Feed_ms, one scan of Feed_Pulse; that same bit resets the timer on the next scan through the XIO above

PREDICT IT · 3 `Run` is on, `T_Feed.ACC` (the accumulated value of `T_Feed` : Feed interval timer; its ACC is compared with Feed_ms) is 500 and `Feed_ms` is 500. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

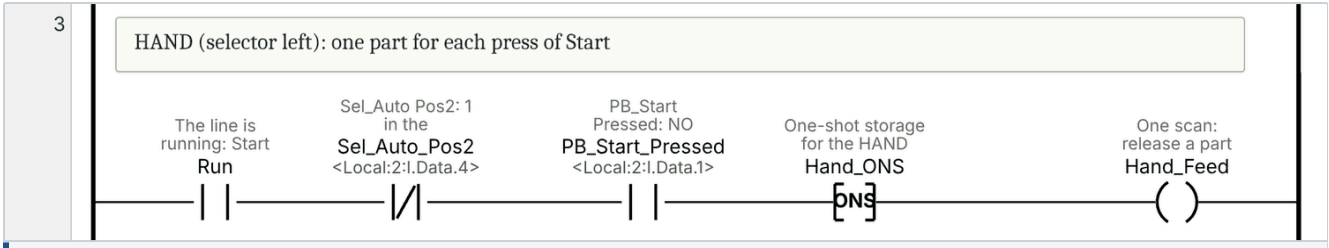
HOW THIS RUNG WORKS

In words. When `Run` (The line is running: Start sealed in, Stop and the E-stop healthy, no bin full) is on and `T_Feed.ACC` (the accumulated value of `T_Feed` : Feed interval timer; its ACC is compared with Feed_ms) is at least `Feed_ms` (Time between parts in AUTO, from the speed pot: 2000 + 40 x pot (2 to 6 s)), the rung turns `Feed_Pulse` (One scan: release a part (AUTO)) on (and off again whenever the rung is false).

Left to right:

1. `XIC(Run)` *Examine If Closed* asks: true when `Run` (The line is running: Start sealed in, Stop and the E-stop healthy, no bin full) is on.
2. `GEQ(T_Feed.ACC,Feed_ms)` *Greater Than or Equal* asks: true when `T_Feed.ACC` (the accumulated value of `T_Feed` : Feed interval timer; its ACC is compared with Feed_ms) is at least `Feed_ms` (Time between parts in AUTO, from the speed pot: 2000 + 40 x pot (2 to 6 s)).
3. `OTE(Feed_Pulse)` *Output Energize* does: turns `Feed_Pulse` (One scan: release a part (AUTO)) on (and off again whenever the rung is false).

FEED · RUNG 3




```
XIC(Run)XIO(Sel_Auto_Pos2)XIC(PB_Start_Pressed)ONS(Hand_ONS)OTE(Hand_Feed);
```

HAND (selector left): one part for each press of Start

HOW THIS RUNG WORKS

In words. When `Run` (The line is running: Start sealed in, Stop and the E-stop healthy, no bin full) is on, `Sel_Auto_Pos2` (1 in the right-hand position) is off, `PB_Start_Pressed` (NO contact: 1 while pressed) is on and it is the first scan they are true (the one-shot, remembered in `Hand_ONS` (One-shot storage for the HAND release)), the rung turns `Hand_Feed` (One scan: release a part (HAND, on a Start press)) on (and off again whenever the rung is false).

Left to right:

1. `XIC(Run)` *Examine If Closed* asks: true when `Run` (The line is running: Start sealed in, Stop and the E-stop healthy, no bin full) is on.
2. `XIO(Sel_Auto_Pos2)` *Examine If Open* asks: true when `Sel_Auto_Pos2` (1 in the right-hand position) is off.
3. `XIC(PB_Start_Pressed)` *Examine If Closed* asks: true when `PB_Start_Pressed` (NO contact: 1 while pressed) is on.
4. `ONS(Hand_ONS)` *One Shot* asks: true when it is the first scan they are true (the one-shot, remembered in `Hand_ONS` (One-shot storage for the HAND release)). It passes power for one scan only, the scan its input goes from false to true; its storage bit remembers last scan.
5. `OTE(Hand_Feed)` *Output Energize* does: turns `Hand_Feed` (One scan: release a part (HAND, on a Start press)) on (and off again whenever the rung is false).

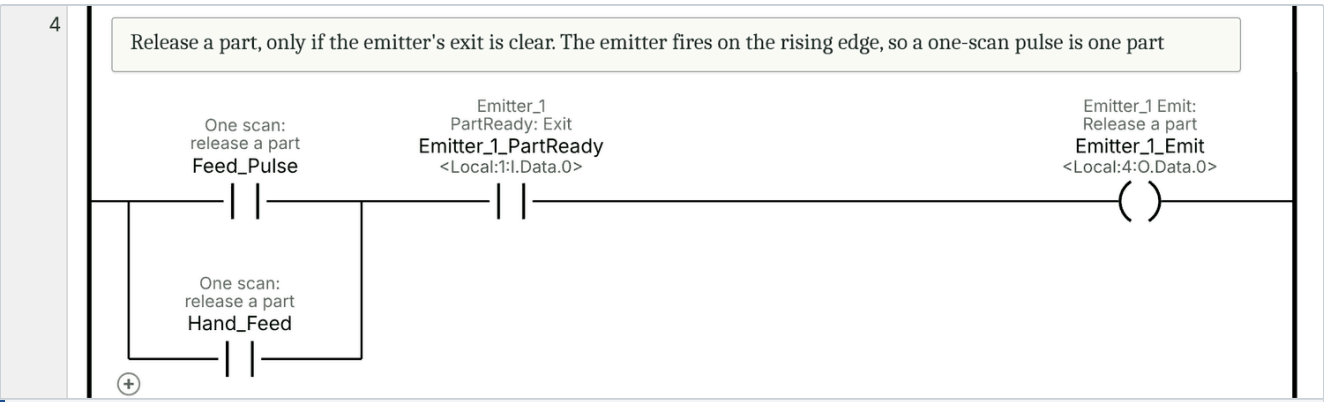
Why it is built this way:

One-shot. Everything after the ONS happens once per event, not on every scan the condition holds; `Hand_ONS` is only its memory of the last scan.

> **Think of it like this:** a doorbell that rings once however long you lean on the button. The ONS looks at what the button was doing last scan: only the change from up to down rings it.

> **Watch for:** Each ONS needs its own storage bit. Two ONS sharing one bit make one of them miss its edges.

FEED · RUNG 4




```
[XIC(Feed_Pulse),XIC(Hand_Feed)]XIC(Emitter_1_PartReady)OTE(Emitter_1_Emit);
```

Release a part, only if the emitter's exit is clear. The emitter fires on the rising edge, so a one-scan pulse is one part

PREDICT IT · 4 `Feed_Pulse` is on and `Emitter_1_PartReady` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When either (`Feed_Pulse` (One scan: release a part (AUTO)) is on) *or* (`Hand_Feed` (One scan: release a part (HAND, on a Start press)) is on) and `Emitter_1_PartReady` (Exit clear, ready to emit) is on, the rung turns `Emitter_1_Emit` (Release a part (rising edge, or continuous while on)) on (and off again whenever the rung is false).

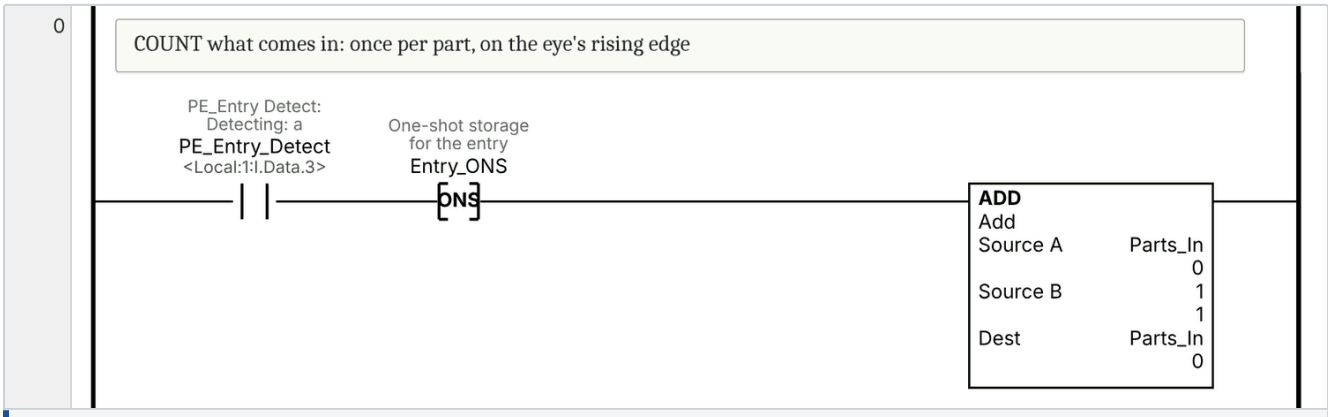
Left to right:

1. A branch of 2 parallel paths: power gets through if any one of them is true.
   - **Path 1**
     1. `XIC(Feed_Pulse)` *Examine If Closed* asks: true when `Feed_Pulse` (One scan: release a part (AUTO)) is on.
   - **Path 2**
     1. `XIC(Hand_Feed)` *Examine If Closed* asks: true when `Hand_Feed` (One scan: release a part (HAND, on a Start press)) is on.
2. `XIC(Emitter_1_PartReady)` *Examine If Closed* asks: true when `Emitter_1_PartReady` (Exit clear, ready to emit) is on.
3. `OTE(Emitter_1_Emit)` *Output Energize* does: turns `Emitter_1_Emit` (Release a part (rising edge, or continuous while on)) on (and off again whenever the rung is false).

> **Think of it like this:** parallel paths are two roads to the same town. If either road is open you get there; the rung only cares that power reaches the far end, not which way it went.

Count

COUNT · RUNG 0




```
XIC(PE_Entry_Detect)ONS(Entry_ONS)ADD(Parts_In,1,Parts_In);
```

COUNT what comes in: once per part, on the eye's rising edge

HOW THIS RUNG WORKS

In words. When `PE_Entry_Detect` (Detecting: a solid part is within range) is on and it is the first scan they are true (the one-shot, remembered in `Entry_ONS` (One-shot storage for the entry count)), the rung makes `Parts_In` (Parts seen by PE_Entry, shown on Disp_Count) = `Parts_In` + **1**.

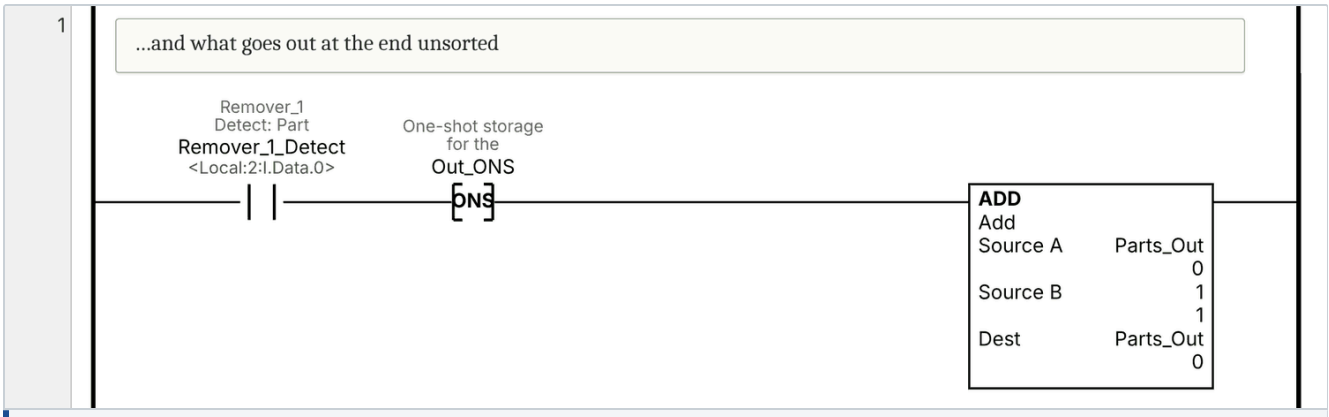
Left to right:

1. `XIC(PE_Entry_Detect)` *Examine If Closed* asks: true when `PE_Entry_Detect` (Detecting: a solid part is within range) is on.
2. `ONS(Entry_ONS)` *One Shot* asks: true when it is the first scan they are true (the one-shot, remembered in `Entry_ONS` (One-shot storage for the entry count)).
3. `ADD(Parts_In,1,Parts_In)` *Add* does: makes `Parts_In` (Parts seen by PE_Entry, shown on Disp_Count) = `Parts_In` + **1**.

Why it is built this way:

One-shot: once per event, not once per scan.

COUNT · RUNG 1




```
XIC(Remover_1_Detect)ONS(Out_ONS)ADD(Parts_Out,1,Parts_Out);
```

...and what goes out at the end unsorted

HOW THIS RUNG WORKS

In words. When `Remover_1_Detect` (Part entering the remover) is on and it is the first scan they are true (the one-shot, remembered in `Out_ONS` (One-shot storage for the pass-through count)), the rung makes `Parts_Out` (Parts that reached the remover unsorted) = `Parts_Out` + **1**.

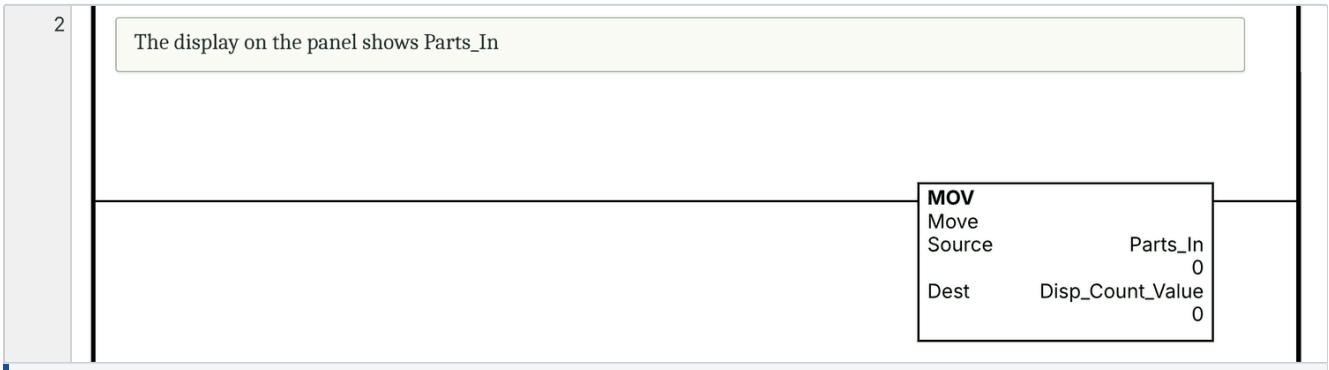
Left to right:

1. `XIC(Remover_1_Detect)` *Examine If Closed* asks: true when `Remover_1_Detect` (Part entering the remover) is on.
2. `ONS(Out_ONS)` *One Shot* asks: true when it is the first scan they are true (the one-shot, remembered in `Out_ONS` (One-shot storage for the pass-through count)).
3. `ADD(Parts_Out,1,Parts_Out)` *Add* does: makes `Parts_Out` (Parts that reached the remover unsorted) = `Parts_Out` + **1**.

Why it is built this way:

One-shot: once per event, not once per scan.

COUNT · RUNG 2




```
MOV(Parts_In,Disp_Count_Value);
```

The display on the panel shows Parts_In

HOW THIS RUNG WORKS

In words. Every scan, with nothing to test, the rung puts `Parts_In` (Parts seen by PE_Entry, shown on Disp_Count) into `Disp_Count_Value` (Number to show).

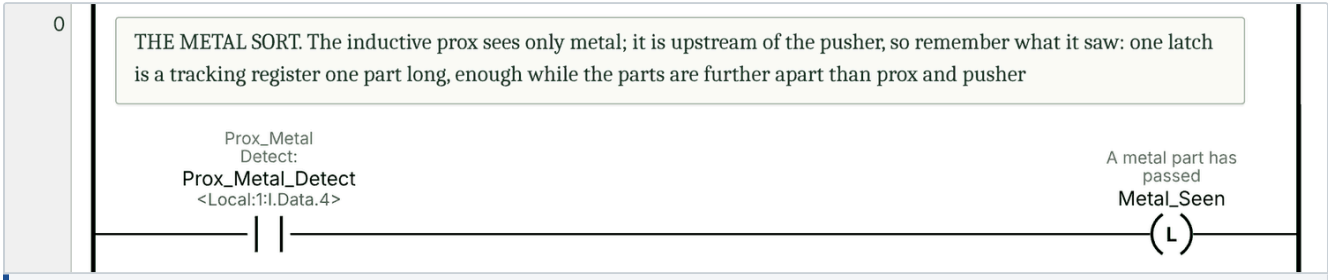
Left to right:

1. `MOV(Parts_In,Disp_Count_Value)` *Move* does: puts `Parts_In` (Parts seen by PE_Entry, shown on Disp_Count) into `Disp_Count_Value` (Number to show). It writes every scan the rung is true.

Sort the metal

The prox is upstream of the pusher, so the program has to remember what it saw for the hundred pixels between them. One latch is a tracking register one part long, and it is enough because at the fastest feed the parts are two seconds apart and the belt covers prox-to-pusher in under one. The day the parts come closer than the sensor is to the actuator, the latch becomes a shift register (Part IV's Track routine does exactly that).

SORT_METAL · RUNG 0




```
XIC(Prox_Metal_Detect)OTL(Metal_Seen);
```

THE METAL SORT. The inductive prox sees only metal; it is upstream of the pusher, so remember what it saw: one latch is a tracking register one part long, enough while the parts are further apart than prox and pusher

HOW THIS RUNG WORKS

In words. When `Prox_Metal_Detect` (Detecting: a conductive (metal) part is within range) is on, the rung latches `Metal_Seen` (A metal part has passed Prox_Metal and is on its way to the pusher) on.

Left to right:

1. `XIC(Prox_Metal_Detect)` *Examine If Closed* asks: true when `Prox_Metal_Detect` (Detecting: a conductive (metal) part is within range) is on.
2. `OTL(Metal_Seen)` *Output Latch* does: latches `Metal_Seen` (A metal part has passed Prox_Metal and is on its way to the pusher) on. It only ever sets the bit; the bit stays on after the rung goes false.

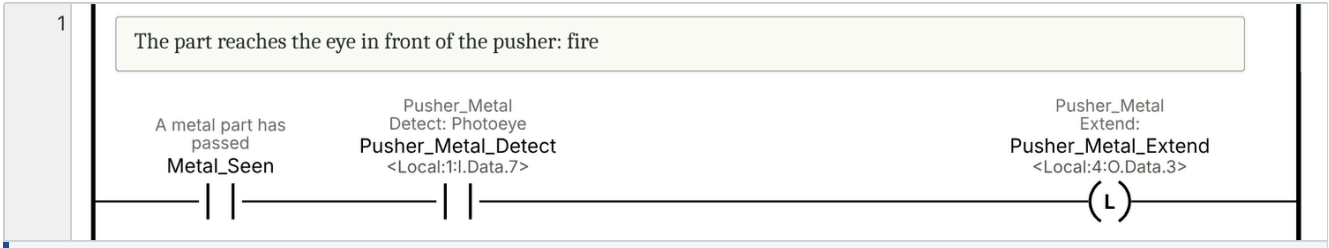
Why it is built this way:

A latch. `Metal_Seen` stays on after this rung goes false; an `OTU` elsewhere in the program clears it.

> **Think of it like this:** a light switch instead of a doorbell. OTL flips it on and it stays on when you take your hand away; OTU flips it off. Whatever flipped it on does not have to stay there.

> **Watch for:** A latched bit survives the condition that set it: every way in needs a way out, or it stays on for good.

SORT_METAL · RUNG 1




```
XIC(Metal_Seen)XIC(Pusher_Metal_Detect)OTL(Pusher_Metal_Extend);
```

The part reaches the eye in front of the pusher: fire

PREDICT IT · 5 `Metal_Seen` is off and `Pusher_Metal_Detect` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Metal_Seen` (A metal part has passed Prox_Metal and is on its way to the pusher) is on and `Pusher_Metal_Detect` (Photoeye in front of the pusher) is on, the rung latches `Pusher_Metal_Extend` (Solenoid: extend cylinder) on.

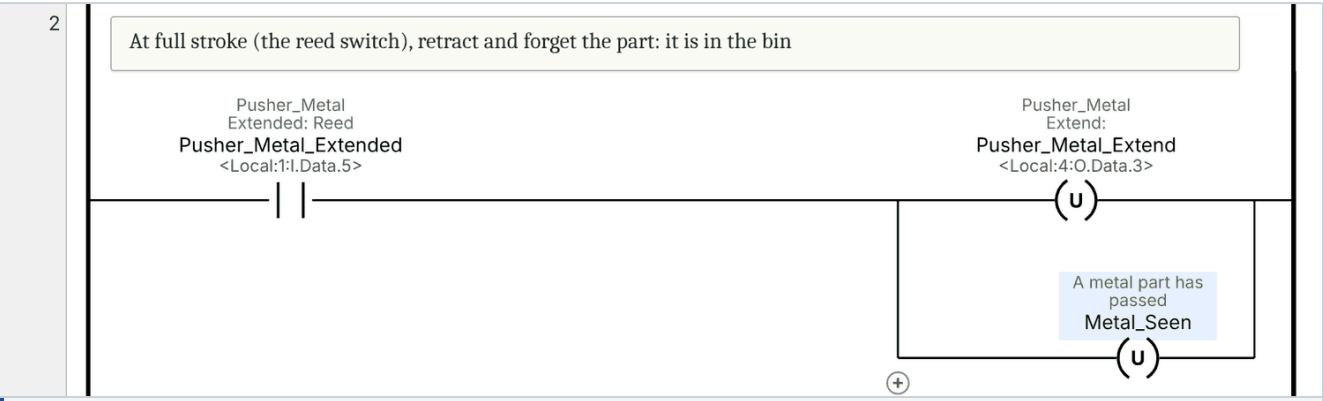
Left to right:

1. `XIC(Metal_Seen)` *Examine If Closed* asks: true when `Metal_Seen` (A metal part has passed Prox_Metal and is on its way to the pusher) is on.
2. `XIC(Pusher_Metal_Detect)` *Examine If Closed* asks: true when `Pusher_Metal_Detect` (Photoeye in front of the pusher) is on.
3. `OTL(Pusher_Metal_Extend)` *Output Latch* does: latches `Pusher_Metal_Extend` (Solenoid: extend cylinder) on.

Why it is built this way:

A latch: it stays on until an OTU clears it.

SORT_METAL · RUNG 2




```
XIC(Pusher_Metal_Extended)[OTU(Pusher_Metal_Extend),OTU(Metal_Seen)];
```

At full stroke (the reed switch), retract and forget the part: it is in the bin

HOW THIS RUNG WORKS

In words. When `Pusher_Metal_Extended` (Reed switch: cylinder extended) is on, the rung unlatches `Pusher_Metal_Extend` (Solenoid: extend cylinder) (turns it off) and unlatches `Metal_Seen` (A metal part has passed Prox_Metal and is on its way to the pusher) (turns it off).

Left to right:

1. `XIC(Pusher_Metal_Extended)` *Examine If Closed* asks: true when `Pusher_Metal_Extended` (Reed switch: cylinder extended) is on.
2. A branch of 2 parallel paths: each path is tested on its own and does its own work.

   Path 1

   1. `OTU(Pusher_Metal_Extend)` *Output Unlatch* does: unlatches `Pusher_Metal_Extend` (Solenoid: extend cylinder) (turns it off).

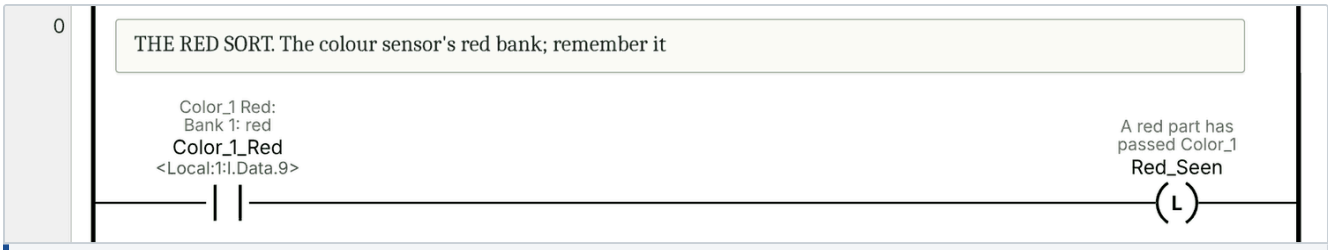
   Path 2

   1. `OTU(Metal_Seen)` *Output Unlatch* does: unlatches `Metal_Seen` (A metal part has passed Prox_Metal and is on its way to the pusher) (turns it off).

Sort the red

A pusher fires when the part is in front of it; an arm has to be across the lane before the part gets there. So the arm swings as soon as red is seen and comes back a second after the part has left its eye.

SORT_RED · RUNG 0




```
XIC(Color_1_Red)OTL(Red_Seen);
```

THE RED SORT. The colour sensor's red bank; remember it

HOW THIS RUNG WORKS

In words. When `Color_1_Red` (Bank 1: red) is on, the rung latches `Red_Seen` (A red part has passed Color_1 and is on its way to the arm) on.

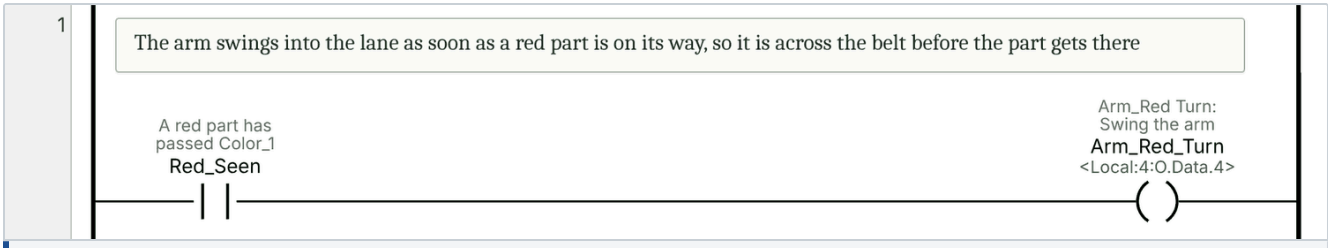
Left to right:

1. `XIC(Color_1_Red)` *Examine If Closed* asks: true when `Color_1_Red` (Bank 1: red) is on.
2. `OTL(Red_Seen)` *Output Latch* does: latches `Red_Seen` (A red part has passed Color_1 and is on its way to the arm) on.

Why it is built this way:

A latch: it stays on until an OTU clears it.

SORT_RED · RUNG 1




```
XIC(Red_Seen)OTE(Arm_Red_Turn);
```

The arm swings into the lane as soon as a red part is on its way, so it is across the belt before the part gets there

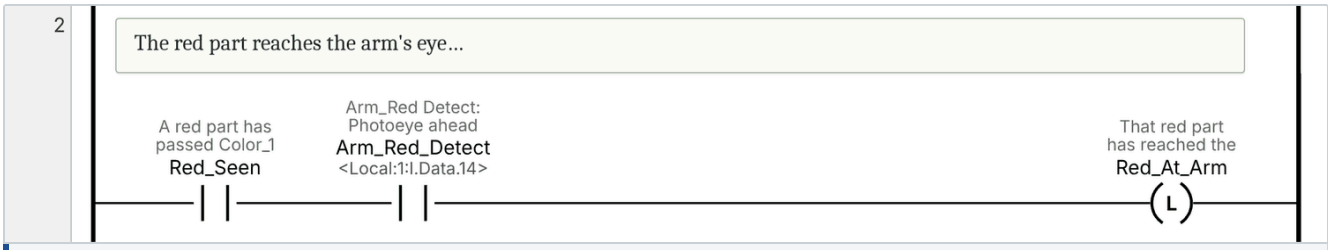
HOW THIS RUNG WORKS

In words. When `Red_Seen` (A red part has passed Color_1 and is on its way to the arm) is on, the rung turns `Arm_Red_Turn` (Swing the arm into the lane) on (and off again whenever the rung is false).

Left to right:

1. `XIC(Red_Seen)` *Examine If Closed* asks: true when `Red_Seen` (A red part has passed Color_1 and is on its way to the arm) is on.
2. `OTE(Arm_Red_Turn)` *Output Energize* does: turns `Arm_Red_Turn` (Swing the arm into the lane) on (and off again whenever the rung is false).

SORT_RED · RUNG 2




```
XIC(Red_Seen)XIC(Arm_Red_Detect)OTL(Red_At_Arm);
```

The red part reaches the arm's eye...

PREDICT IT · 6 `Red_Seen` is off and `Arm_Red_Detect` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Red_Seen` (A red part has passed Color_1 and is on its way to the arm) is on and `Arm_Red_Detect` (Photoeye ahead of the arm) is on, the rung latches `Red_At_Arm` (That red part has reached the arm's eye) on.

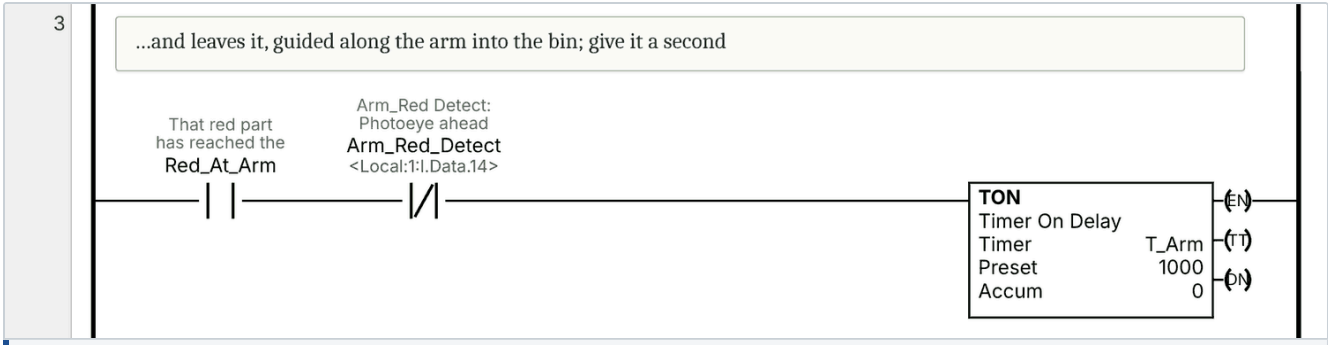
Left to right:

1. `XIC(Red_Seen)` *Examine If Closed* asks: true when `Red_Seen` (A red part has passed Color_1 and is on its way to the arm) is on.
2. `XIC(Arm_Red_Detect)` *Examine If Closed* asks: true when `Arm_Red_Detect` (Photoeye ahead of the arm) is on.
3. `OTL(Red_At_Arm)` *Output Latch* does: latches `Red_At_Arm` (That red part has reached the arm's eye) on.

Why it is built this way:

A latch: it stays on until an OTU clears it.

SORT_RED · RUNG 3




```
XIC(Red_At_Arm)XIO(Arm_Red_Detect)TON(T_Arm,1000,0);
```

...and leaves it, guided along the arm into the bin; give it a second

PREDICT IT · 7 `Red_At_Arm` is on and `Arm_Red_Detect` is off. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Red_At_Arm` (That red part has reached the arm's eye) is on and `Arm_Red_Detect` (Photoeye ahead of the arm) is off, the rung runs the timer `T_Arm` (The arm stays in the lane 1 s after the red part leaves its eye) toward 1000 ms (1 s).

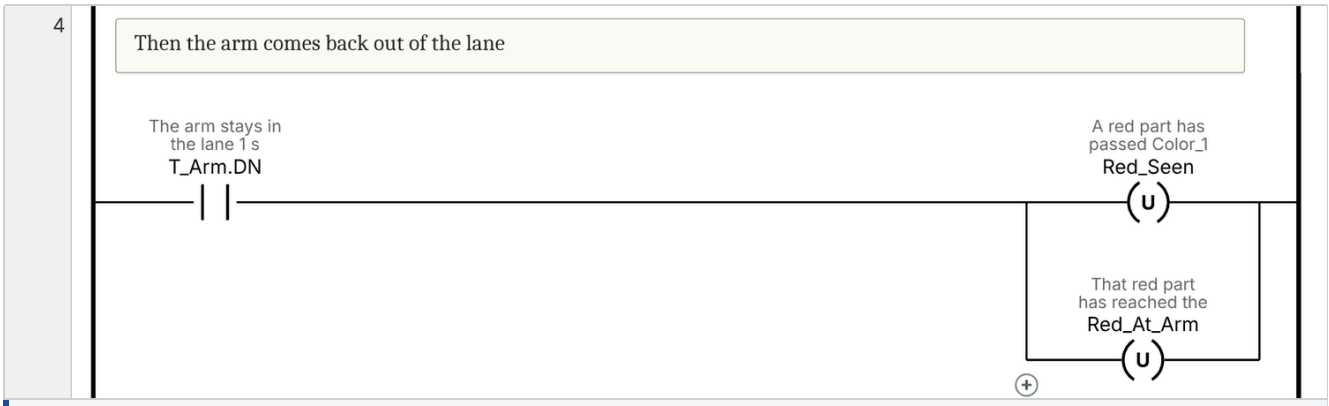
Left to right:

1. `XIC(Red_At_Arm)` *Examine If Closed* asks: true when `Red_At_Arm` (That red part has reached the arm's eye) is on.
2. `XIO(Arm_Red_Detect)` *Examine If Open* asks: true when `Arm_Red_Detect` (Photoeye ahead of the arm) is off.
3. `TON(T_Arm,1000,0)` *Timer On Delay* does: runs the timer `T_Arm` (The arm stays in the lane 1 s after the red part leaves its eye) toward 1000 ms (1 s).

Why it is built this way:

A timer: it times only while this rung is true.

SORT_RED · RUNG 4




```
XIC(T_Arm.DN)[OTU(Red_Seen),OTU(Red_At_Arm)];
```

Then the arm comes back out of the lane

HOW THIS RUNG WORKS

In words. When `T_Arm.DN` (the done bit of `T_Arm`: The arm stays in the lane 1 s after the red part leaves its eye) is on, the rung unlatches `Red_Seen` (A red part has passed Color_1 and is on its way to the arm) (turns it off) and unlatches `Red_At_Arm` (That red part has reached the arm's eye) (turns it off).

Left to right:

1. `XIC(T_Arm.DN)` *Examine If Closed* asks: true when `T_Arm.DN` (the done bit of `T_Arm`: The arm stays in the lane 1 s after the red part leaves its eye) is on.
2. A branch of 2 parallel paths: each path is tested on its own and does its own work.

   Path 1

   1. `OTU(Red_Seen)` *Output Unlatch* does: unlatches `Red_Seen` (A red part has passed Color_1 and is on its way to the arm) (turns it off).

   Path 2

   1. `OTU(Red_At_Arm)` *Output Unlatch* does: unlatches `Red_At_Arm` (That red part has reached the arm's eye) (turns it off).

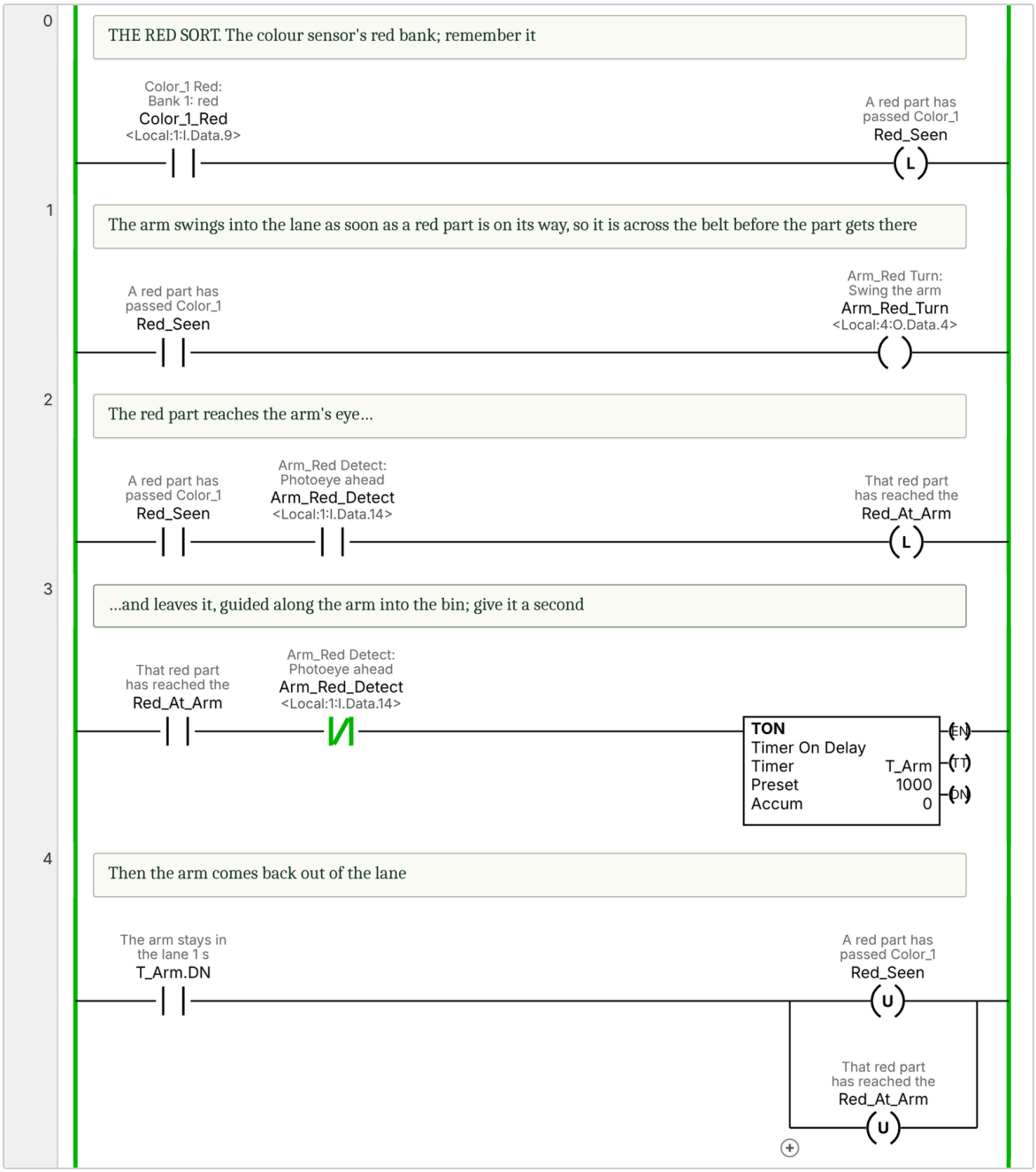
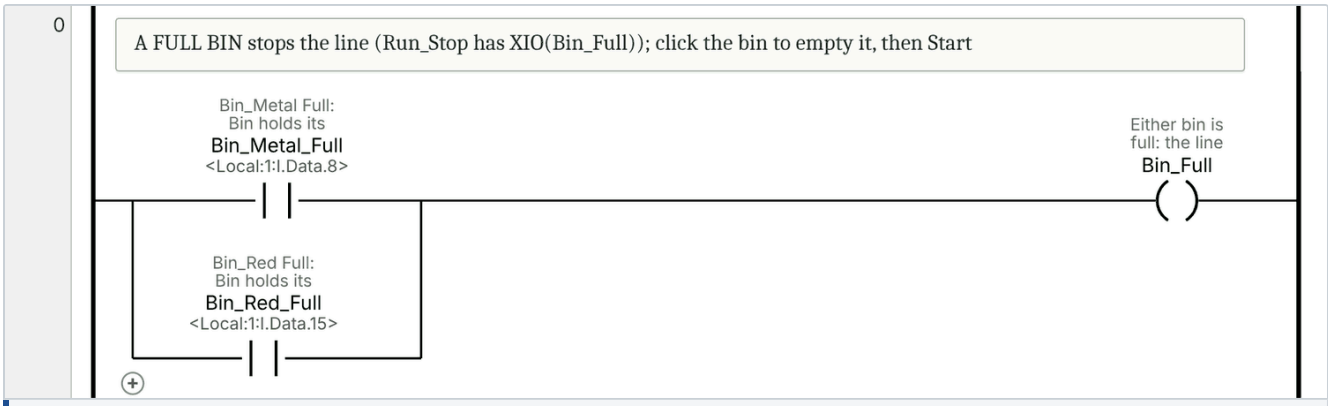


Figure 219. Sort_Red online, a red carton on its way to the arm.

The alarms and the stack light

ALARMS · RUNG 0




```
[XIC(Bin_Metal_Full),XIC(Bin_Red_Full)]OTE(Bin_Full);
```

A FULL BIN stops the line (Run_Stop has XIO(Bin_Full)); click the bin to empty it, then Start

PREDICT IT · 8 `Bin_Metal_Full` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

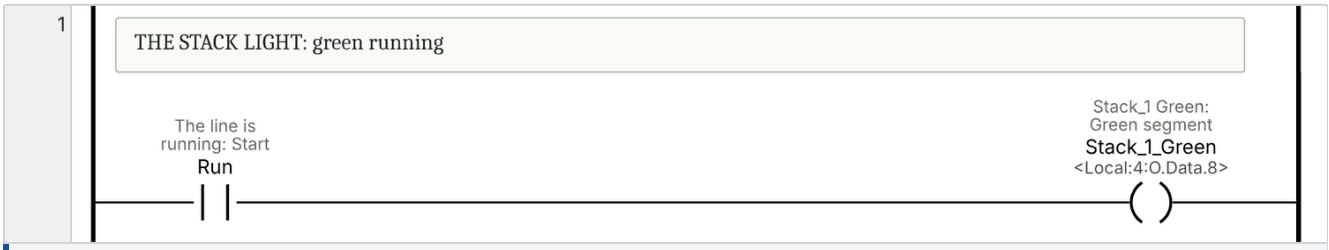
HOW THIS RUNG WORKS

In words. When either (`Bin_Metal_Full` (Bin holds its capacity) is on) *or* (`Bin_Red_Full` (Bin holds its capacity) is on), the rung turns `Bin_Full` (Either bin is full: the line stops until it is emptied) on (and off again whenever the rung is false).

Left to right:

1. A branch of 2 parallel paths: power gets through if any one of them is true.
   - **Path 1**
     1. `XIC(Bin_Metal_Full)` *Examine If Closed* asks: true when `Bin_Metal_Full` (Bin holds its capacity) is on.
   - **Path 2**
     1. `XIC(Bin_Red_Full)` *Examine If Closed* asks: true when `Bin_Red_Full` (Bin holds its capacity) is on.
2. `OTE(Bin_Full)` *Output Energize* does: turns `Bin_Full` (Either bin is full: the line stops until it is emptied) on (and off again whenever the rung is false).

ALARMS · RUNG 1




```
XIC(Run)OTE(Stack_1_Green);
```

THE STACK LIGHT: green running

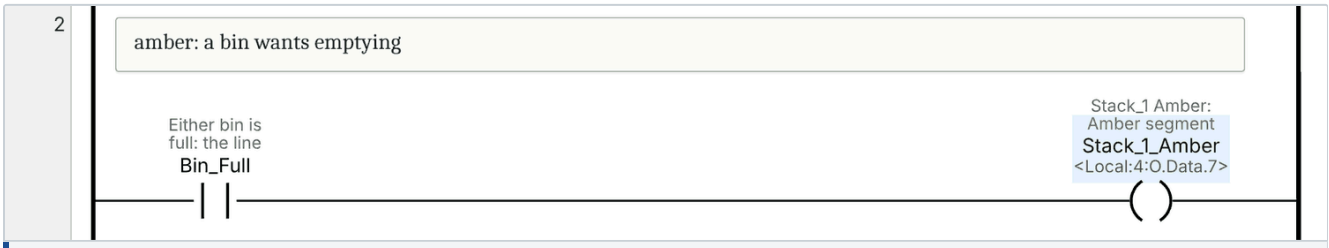
HOW THIS RUNG WORKS

In words. When `Run` (The line is running: Start sealed in, Stop and the E-stop healthy, no bin full) is on, the rung turns `Stack_1_Green` (Green segment) on (and off again whenever the rung is false).

Left to right:

1. `XIC(Run)` *Examine If Closed* asks: true when `Run` (The line is running: Start sealed in, Stop and the E-stop healthy, no bin full) is on.
2. `OTE(Stack_1_Green)` *Output Energize* does: turns `Stack_1_Green` (Green segment) on (and off again whenever the rung is false).

ALARMS · RUNG 2




```
XIC(Bin_Full)OTE(Stack_1_Amber);
```

amber: a bin wants emptying

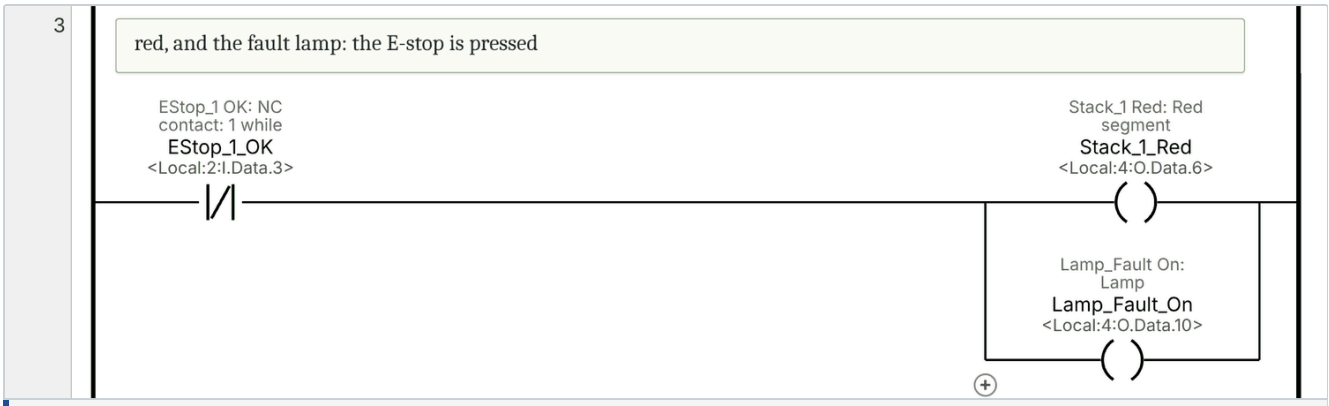
HOW THIS RUNG WORKS

In words. When `Bin_Full` (Either bin is full: the line stops until it is emptied) is on, the rung turns `Stack_1_Amber` (Amber segment) on (and off again whenever the rung is false).

Left to right:

1. `XIC(Bin_Full)` *Examine If Closed* asks: true when `Bin_Full` (Either bin is full: the line stops until it is emptied) is on.
2. `OTE(Stack_1_Amber)` *Output Energize* does: turns `Stack_1_Amber` (Amber segment) on (and off again whenever the rung is false).

ALARMS · RUNG 3




```
XIO(EStop_1_OK)[OTE(Stack_1_Red),OTE(Lamp_Fault_On)];
```

red, and the fault lamp: the E-stop is pressed

HOW THIS RUNG WORKS

In words. When `EStop_1_OK` (NC contact: 1 while healthy, 0 when pressed) is off, the rung turns `Stack_1_Red` (Red segment) on (and off again whenever the rung is false) and turns `Lamp_Fault_On` (Lamp) on (and off again whenever the rung is false).

Left to right:

1. `XIO(EStop_1_OK)` *Examine If Open* asks: true when `EStop_1_OK` (NC contact: 1 while healthy, 0 when pressed) is off.
2. A branch of 2 parallel paths: each path is tested on its own and does its own work.

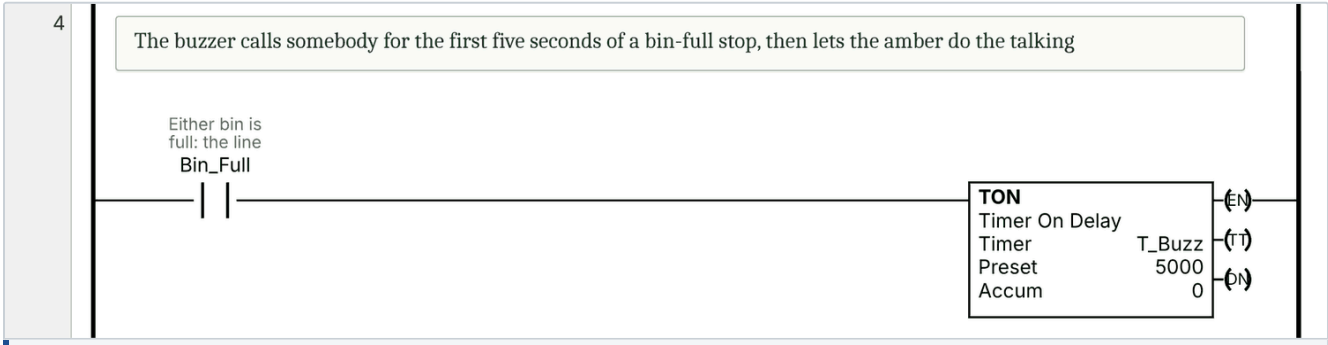
   Path 1

   1. `OTE(Stack_1_Red)` *Output Energize* does: turns `Stack_1_Red` (Red segment) on (and off again whenever the rung is false).

   Path 2

   1. `OTE(Lamp_Fault_On)` *Output Energize* does: turns `Lamp_Fault_On` (Lamp) on (and off again whenever the rung is false).

ALARMS · RUNG 4




```
XIC(Bin_Full)TON(T_Buzz,5000,0);
```

The buzzer calls somebody for the first five seconds of a bin-full stop, then lets the amber do the talking

HOW THIS RUNG WORKS

In words. When `Bin_Full` (Either bin is full: the line stops until it is emptied) is on, the rung runs the timer `T_Buzz` (The buzzer sounds for the first 5 s of a bin-full stop) toward 5000 ms (5 s).

Left to right:

1. `XIC(Bin_Full)` *Examine If Closed* asks: true when `Bin_Full` (Either bin is full: the line stops until it is emptied) is on.
2. `TON(T_Buzz,5000,0)` *Timer On Delay* does: runs the timer `T_Buzz` (The buzzer sounds for the first 5 s of a bin-full stop) toward 5000 ms (5 s).

Why it is built this way:

A timer: it times only while this rung is true.

ALARMS · RUNG 5

```
XIC(Bin_Full)XIO(T_Buzz.DN)OTE(Stack_1_Buzzer);
```

PREDICT IT · 9 `Bin_Full` is off and `T_Buzz.DN` (the done bit of `T_Buzz` : The buzzer sounds for the first 5 s of a bin-full stop) is off. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Bin_Full` (Either bin is full: the line stops until it is emptied) is on and `T_Buzz.DN` (the done bit of `T_Buzz` : The buzzer sounds for the first 5 s of a bin-full stop) is off, the rung turns `Stack_1_Buzzer` (Buzzer) on (and off again whenever the rung is false).

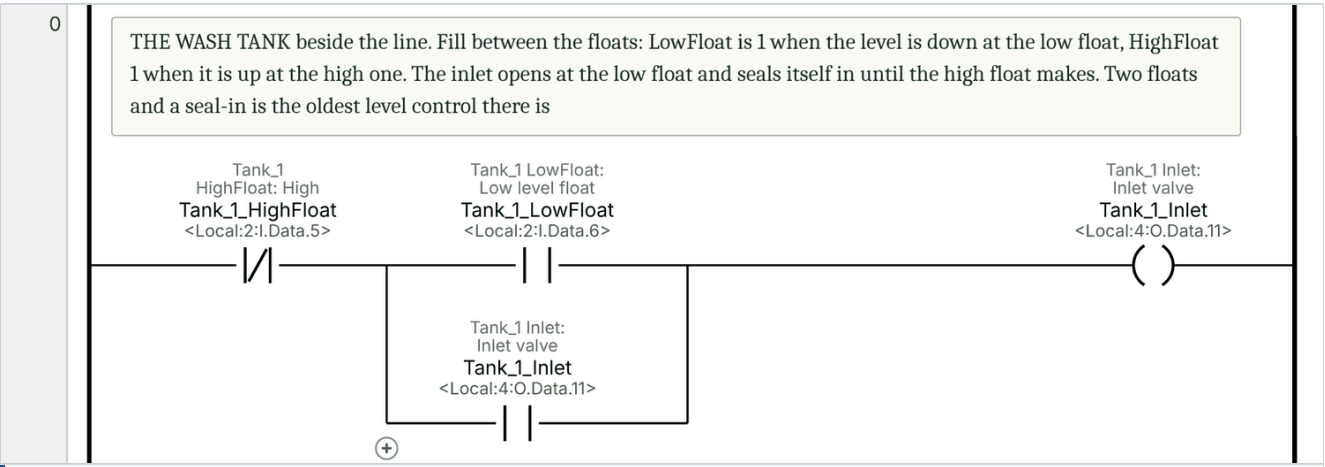
Left to right:

1. `XIC(Bin_Full)` *Examine If Closed* asks: true when `Bin_Full` (Either bin is full: the line stops until it is emptied) is on.
2. `XIO(T_Buzz.DN)` *Examine If Open* asks: true when `T_Buzz.DN` (the done bit of `T_Buzz` : The buzzer sounds for the first 5 s of a bin-full stop) is off.
3. `OTE(Stack_1_Buzzer)` *Output Energize* does: turns `Stack_1_Buzzer` (Buzzer) on (and off again whenever the rung is false).

A full bin stops the line through the XIO in the seal-in; click the bin to empty it (in a sandbox scene a click on a bin empties it and the log says so; a level's bins are its score and do not) and press Start.

The wash tank

TANK · RUNG 0




```
XIO(Tank_1_HighFloat)[XIC(Tank_1_LowFloat),XIC(Tank_1_Inlet)]OTE(Tank_1_Inlet);
```

THE WASH TANK beside the line. Fill between the floats: LowFloat is 1 when the level is down at the low float, HighFloat 1 when it is up at the high one. The inlet opens at the low float and seals itself in until the high float makes. Two floats and a seal-in is the oldest level control there is

PREDICT IT · 10 `Tank_1_HighFloat` is off and `Tank_1_LowFloat` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Tank_1_HighFloat` (High level float switch) is off and either (`Tank_1_LowFloat` (Low level float switch) is on) *or* (`Tank_1_Inlet` (Inlet valve) is on), the rung turns `Tank_1_Inlet` on (and off again whenever the rung is false).

Left to right:

1. `XIO(Tank_1_HighFloat)` *Examine If Open* asks: true when `Tank_1_HighFloat` (High level float switch) is off.
2. A branch of 2 parallel paths: power gets through if any one of them is true.

   Path 1

   1. `XIC(Tank_1_LowFloat)` *Examine If Closed* asks: true when `Tank_1_LowFloat` (Low level float switch) is on.

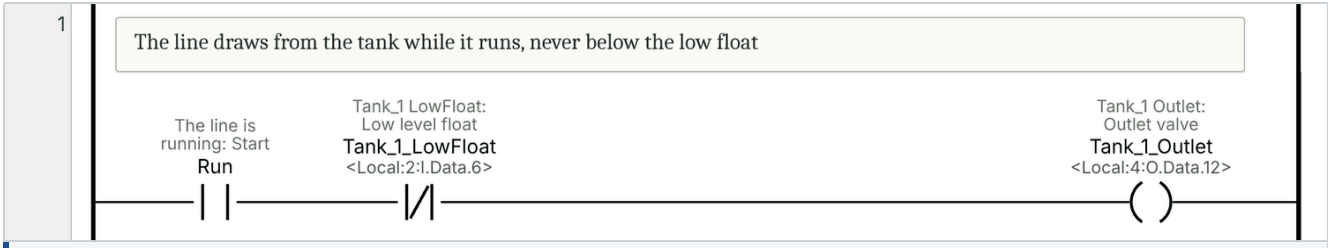
   Path 2

   1. `XIC(Tank_1_Inlet)` *Examine If Closed* asks: true when `Tank_1_Inlet` (Inlet valve) is on.
3. `OTE(Tank_1_Inlet)` *Output Energize* does: turns `Tank_1_Inlet` on (and off again whenever the rung is false).

Why it is built this way:

Seal-in, as before: the output's own contact holds the rung once the start has been made.

TANK · RUNG 1




```
XIC(Run)XIO(Tank_1_LowFloat)OTE(Tank_1_Outlet);
```

The line draws from the tank while it runs, never below the low float

PREDICT IT · 11 `Run` is on and `Tank_1_LowFloat` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

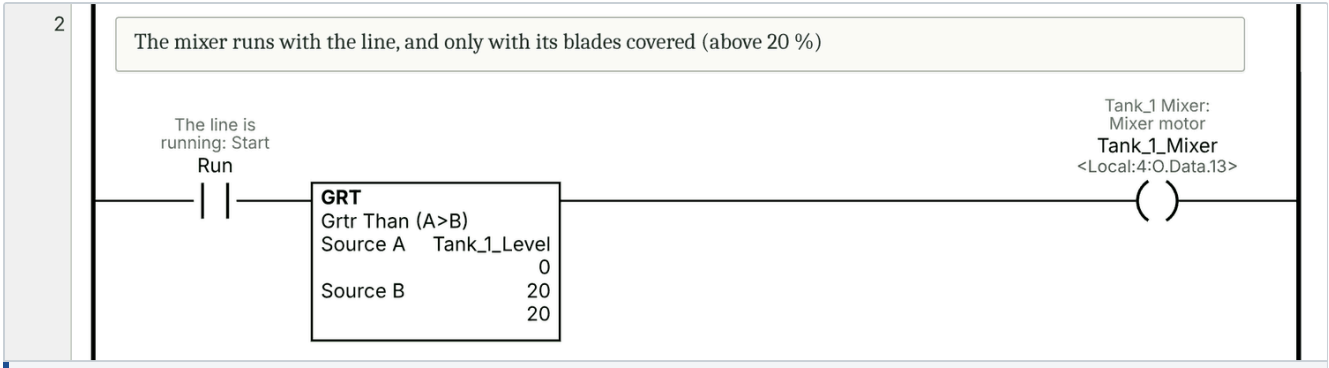
HOW THIS RUNG WORKS

In words. When `Run` (The line is running: Start sealed in, Stop and the E-stop healthy, no bin full) is on and `Tank_1_LowFloat` (Low level float switch) is off, the rung turns `Tank_1_Outlet` (Outlet valve) on (and off again whenever the rung is false).

Left to right:

1. `XIC(Run)` *Examine If Closed* asks: true when `Run` (The line is running: Start sealed in, Stop and the E-stop healthy, no bin full) is on.
2. `XIO(Tank_1_LowFloat)` *Examine If Open* asks: true when `Tank_1_LowFloat` (Low level float switch) is off.
3. `OTE(Tank_1_Outlet)` *Output Energize* does: turns `Tank_1_Outlet` (Outlet valve) on (and off again whenever the rung is false).

TANK · RUNG 2




```
XIC(Run)GRT(Tank_1_Level,20)OTE(Tank_1_Mixer);
```

The mixer runs with the line, and only with its blades covered (above 20 %)

PREDICT IT · 12 `Run` is on and `Tank_1_Level` is 21. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Run` (The line is running: Start sealed in, Stop and the E-stop healthy, no bin full) is on and `Tank_1_Level` (Level 0-100 % (bind it to a REAL tag for the unrounded value)) is greater than **20**, the rung turns `Tank_1_Mixer` (Mixer motor) on (and off again whenever the rung is false).

Left to right:

1. `XIC(Run)` *Examine If Closed* asks: true when `Run` (The line is running: Start sealed in, Stop and the E-stop healthy, no bin full) is on.
2. `GRT(Tank_1_Level,20)` *Greater Than* asks: true when `Tank_1_Level` (Level 0-100 % (bind it to a REAL tag for the unrounded value)) is greater than **20**.
3. `OTE(Tank_1_Mixer)` *Output Energize* does: turns `Tank_1_Mixer` (Mixer motor) on (and off again whenever the rung is false).

WATCH FOR: WHICH WAY A FLOAT READS

The tank's LowFloat is 1 when the level is *down at* the low float, HighFloat 1 when it is *up at* the high one. The fill rung opens the inlet on LowFloat and holds it with its own contact until HighFloat. Read LowFloat the other way round and the tank never fills: the first draft of this rung did exactly that, and the empty tank sat at 0 % with the inlet shut.

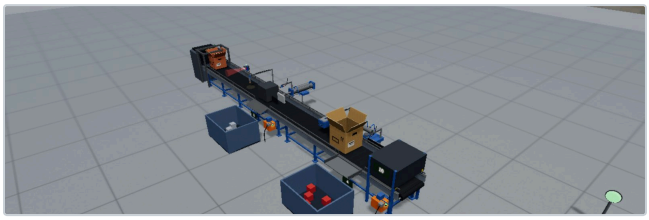


Figure 220. In 3D: the pusher, the colour sensor and the arm over their bins.

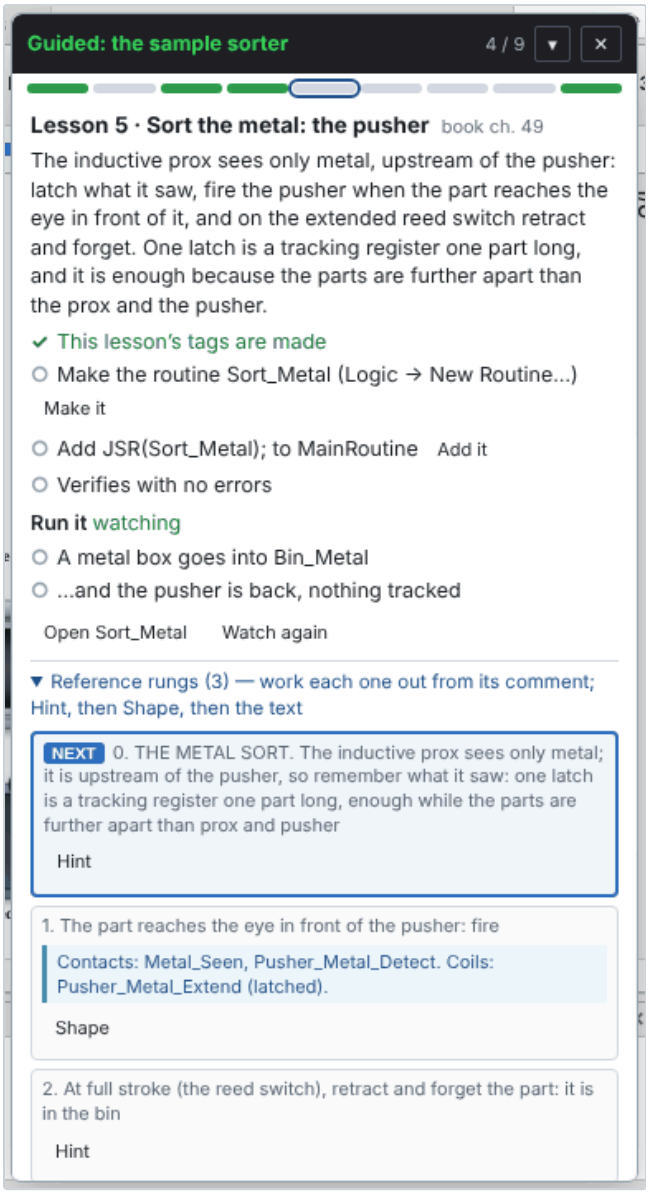


Figure 221. Guided: the sample sorter, the metal lesson with the reference rungs at their comments.

Guided: the sample sorter

Nine lessons, one routine each, in the order above: **Init**, **Run_Stop** (Start seals in and lights the button; Stop drops it), **Feed** (a part per press in HAND, then AUTO), **Count** (Parts_In on the display), **Sort_Metal** (a metal box in its bin, the pusher back), **Sort_Red** (a red box in its bin, the arm back), **Alarms** (green running; the E-stop gives red and the fault lamp), **Tank** (the inlet fills, the mixer turns) and **Ready**. The reference rungs come hints first, as everywhere in guided mode: the comment, then what the rung uses, then its shape, then the text.

TRY IT: BREAK THE TRACKING

Turn the pot to 0 (two seconds between parts) and, in Program mode, drag Prox_Metal back towards the emitter until it is further from the pusher than two parts are from each other. Run: a metal box followed closely by a carton fires the pusher at the carton. Then replace Metal_Seen with a BSL on a DINT, clocked by the entry eye, and read the bit for the pusher's position. That is the step from a latch to a tracking register, and the reason Part IV is built the way it is.

Machine faults on the sample sorter

Tools → Inject Machine Fault… lists the faults that belong to the parts on this floor. Each has a fault card: the symptom, what to watch in the I/O to confirm it, the repair, and what a program should do about it. **Repair** on an active fault removes its cause the way a technician would; whether the machine runs again is up to the program. The sorter also has the belts' and the eyes' faults (a jam, an overload, a dirty lens).

Fault	Watch	Repair	The program should
Pusher: extended reed switch dead The cylinder goes out and comes back as commanded, but the front reed switch never makes.	Extend 1 and the cylinder out on the floor, Extended 0.	Replace the reed switch (Repair).	A stroke-time alarm: Extend on without Extended for a second. Retract, stop the line, light the fault lamp.
Air lost: the cylinder does not move No air at the valve: the solenoid is energized and nothing moves.	Extend (or Turn) 1, the cylinder still back: Retracted 1, Extended (Turned) 0.	Open the air supply (Repair).	The same stroke-time alarm: commanded and not there in time.
Colour sensor: lens fouled, no colour Every bank stays off while parts pass under it.	Red, Blue, Green and Yellow all 0 while parts pass.	Clean the lens (Repair).	A part reaching the next eye with no colour seen is an alarm (or counted as unknown).
High float stuck down The high-level float switch never makes: a fill that stops on it never stops.	The level at the top, HighFloat 0, Inlet still 1.	Free the float (Repair).	Cross-check with the level transmitter: Level above 95 % with HighFloat 0 closes the inlet and alarms.
Bin full switch stuck on The bin reports full when it is not.	Full 1 with the bin empty (Count 0).	Free the switch (Repair).	Full with Count well below the capacity is a sensor alarm, not a reason to wait for somebody to empty the bin.

Inject a Machine Fault

Faults happen to the machine, not the controller: your ladder has to notice them. Each one is injected into the running simulator (Run mode), logged in the Simulator Log and outlined in red. Repair removes one cause the way a technician would (Clear Machine Faults removes them all); whether the machine runs again is up to your program. Every fault has a card: the symptom, what to watch in the I/O, the repair, and what a program should do about it.

Active now

Pusher: extended reed switch dead on Pusher_Metal — Repair

Symptom The cylinder goes out and comes back as commanded, but the front reed switch never makes.
Watch Extend 1 and the cylinder out on the floor, Extended 0.
Repair Replace the reed switch (Repair).
The program should A stroke-time alarm: Extend on without Extended for a second. Retract, stop the line, light the fault lamp.

Inject a fault on this machine

Jam: a part sticks on the belt The part furthest along this belt sticks; the belt runs on under it and parts pile up behind. ▼ Fault card **Symptom** The part furthest along this belt sticks; the belt runs on under it and parts pile up behind. **Watch** The eye at the jam stays made, parts queue behind it, the belt's Running stays 1. **Repair** Click the stuck part in Run mode, or Repair. **The program should** Time how long an eye stays blocked: longer than a part takes to pass means a jam. Stop the infeed and alarm.	Belt_1	Inject
Dirty sensor: stuck ON Dirt on the lens: Detect stays on whatever is in front of it. ▶ Fault card	PE_Entry	Inject
Dirty sensor: stuck OFF Detect stays off whatever is in front of it. ▶ Fault card	PE_Entry	Inject
Motor overload: the belt trips The motor overload trips: the belt stops and Running drops while Fwd is still commanded. ▶ Fault card	Belt_1	Inject
Pusher: extended reed switch dead The cylinder goes out and comes back as commanded, but the front reed switch never makes. ▶ Fault card	Pusher_Metal	Inject
Air lost: the cylinder does not move No air at the valve: the solenoid is energized and nothing moves. ▶ Fault card	Pusher_Metal	Inject
Colour sensor: lens fouled, no colour Every bank stays off while parts pass under it. ▶ Fault card	Color_1	Inject
High float stuck down The high-level float switch never makes: a fill that stops on it never stops. ▶ Fault card	Tank_1	Inject
Bin full switch stuck on The bin reports full when it is not. ▶ Fault card	Bin_Metal	Inject

18 more fault kinds belong to parts that are not on this floor.

☐ Drill: inject a random fault from this list every 2 minutes of run time

Figure 222. Tools → Inject Machine Fault... on the sample sorter: the pusher's reed switch has been killed, its fault card is open with the Repair button; below it, the faults this floor's parts can have, each with its card.

> **TRY IT: A STROKE-TIME ALARM**
>
> Inject *Pusher: extended reed switch dead* with the reference running: the pusher goes out and stays commanded out for ever, because nothing in Sort_Metal ever times the stroke. Add the rung a real machine has, `XIC(Pusher_Metal_Extend)XIO(Pusher_Metal_Extended)TON(T_Stroke,1000,0)`, put `XIC(T_Stroke.DN)` into the fault lamp and the Run seal, and inject it again. The same rung catches *Air lost*. That is the whole idea of fault handling: time what you commanded against what you were told happened.

Predict it: the answers

1. Every condition on the way is met, so `Run` is **on**.
2. The rung is false at `XIC(Run)`: `Run` is off, so `T_Feed` is reset to zero (a TON resets when its rung goes false).
3. Every condition on the way is met, so `Feed_Pulse` is **on**.
4. Every condition on the way is met, so `Emitter_1_Emit` is **on**.
5. The rung is false at `XIC(Metal_Seen)`: `Metal_Seen` is off, so the OTL does nothing: `Pusher_Metal_Extend` keeps whatever it had.
6. The rung is false at `XIC(Red_Seen)`: `Red_Seen` is off, so the OTL does nothing: `Red_At_Arm` keeps whatever it had.
7. Every condition on the way is met, so `T_Arm` times toward its preset.
8. Every condition on the way is met, so `Bin_Full` is **on**.
9. The rung is false at `XIC(Bin_Full)`: `Bin_Full` is off, so `Stack_1_Buzzer` is **off** (an OTE writes 0 when its rung is false).
10. Every condition on the way is met, so `Tank_1_Inlet` is **on**.
11. The rung is false at `XIO(Tank_1_LowFloat)`: `Tank_1_LowFloat` is on, so `Tank_1_Outlet` is **off** (an OTE writes 0 when its rung is false).
12. Every condition on the way is met, so `Tank_1_Mixer` is **on**.

50 Classic scenes: doors and gates

A barrier at a car park and a roll-up door at a loading dock are the two machines most people walk past every day without thinking of them as machines. Both are a motor between two limit switches, which is a morning's work. What makes them worth a chapter is the same thing that made the traffic light worth one: everything around the motor. Each moves a heavy thing across a place where people and vehicles go, so each has to know when something is in the way, and each has to do something better than stop when it finds out. From Version 137 both are in Logix Connect as classic scenes, under **File → New Project**, each opening with its guided track and its own machine faults.

The parking entry gate

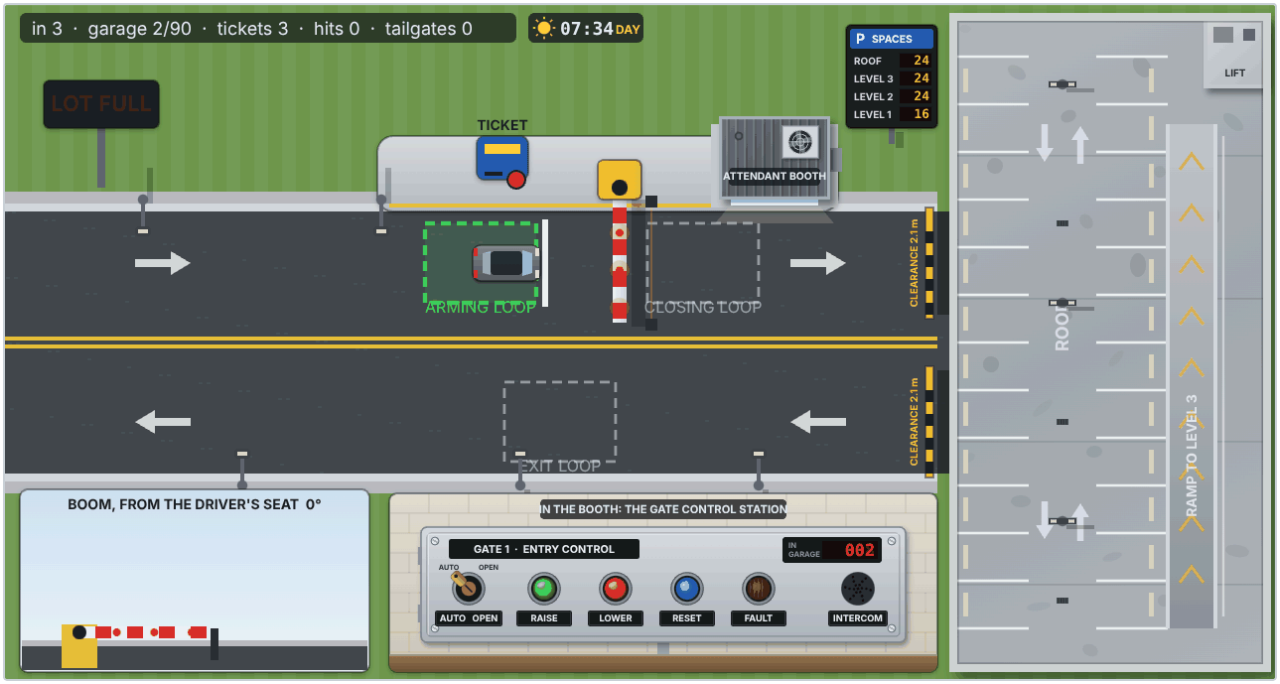


Figure 223. The Parking Entry Gate part, from above. The entry lane with its arming loop under a car at the ticket kiosk; the island with the kiosk (PRESS FOR TICKET lamp, button, slot) and the boom column; the boom across the lane, foreshortened as it rises; the photo-eye just past it; the closing loop beyond; the garage's roof deck on the right, the road running in under it past the clearance bars; the SPACES sign and the attendant booth with its roof on; the exit lane with its own loop. Bottom left, the boom in elevation as the driver sees it; beside it, the gate control station on the booth's wall. The counts across the top, and the clock.

The machine

Cars arrive by themselves (*Cars per minute*), queue, and the first one stops at the kiosk with its wheels on the arming loop. The driver presses for a ticket a second later, takes it a second after it appears, and waits for the boom to be all the way up before driving in. On the closing loop past the boom the car is through; in the garage it takes a stall. The garage has four floors of 24 stalls, 96 in all, and fills from the street up:

LEVEL 1, LEVEL 2, LEVEL 3, then the ROOF, the only floor you see from above. Parked cars leave by the exit lane at random (*Seconds between cars leaving*), over the exit loop, with no barrier. The boom is a motor between two limit switches: 2.4 s for its 90 degrees. Lowered onto a car it rests on the roof, and that is logged and counted. Driven against its stop (or a roof) for four seconds, the motor's thermal trips and it will not run for twenty. Turn on *Some drivers tailgate* and a few drivers will drive straight under a raised boom without a ticket, which is the problem every car park has.

Signal		Meaning
`Arming_Loop`	input	A car stands over the loop at the ticket kiosk
`Ticket_PB`	input	The kiosk's PRESS FOR TICKET button, 1 while pressed
`Ticket_Taken`	input	The driver has pulled the ticket from the slot (a short pulse)
`Arm_Up`	input	Boom up limit switch: the boom is fully raised
`Arm_Down`	input	Boom down limit switch: the boom is fully lowered
`Gate_Eye`	input	Photo-eye across the lane just past the boom: 1 while the beam is blocked
`Closing_Loop`	input	A car stands over the loop past the boom
`Exit_Loop`	input	A car is leaving the lot over the exit lane's loop
`Arm_Raise`	output	Boom motor: raise
`Arm_Lower`	output	Boom motor: lower
`Ticket_Out`	output	Dispense a ticket (on the rising edge)
`Ticket_Lamp`	output	PRESS FOR TICKET lamp on the kiosk
`Boom_Lights`	output	The red lamps along the boom
`Full_Sign`	output	LOT FULL sign at the entrance

Figure 224. The scene: the gate outdoors and the booth panel's devices: the AUTO / OPEN key, Raise, Lower, Reset, the fault lamp and the count in the garage.

Figure 225. In 3D: the island with the kiosk and the column, the boom raised for a car, the booth, the loops, the garage's decks behind.

The attendant's controls are where an attendant would find them: on the wall of the booth, in a grey steel station with an engraved nameplate and 22 mm operators in chrome bezels, a black legend plate under each. The key switch is the scene's `Mode_Sel` (AUTO or OPEN), RAISE, LOWER and RESET are `Raise_PB`, `Lower_PB` and `Reset_PB`, the red pilot light is `Lamp_Fault`, and the IN GARAGE counter shows `Lot_Count`. Click them in the inset and the program sees exactly what it would from the console devices, because they are the same devices.

Figure 226. The side view: an elevation from the south at the plan's scale. The town behind, the LOT FULL and SPACES signs, the kiosk, the boom coming toward you as it lowers (end-on when it is down), the booth with the attendant at the window, the cars on both lanes in profile; the garage with its four floors, the amber LED header over each one counting the cars parked on it. Under the road, the pavement in section with the three loops' saw cuts.

Every scene has three views now, and they are the same machine: the side view is not a picture of the gate but the gate drawn from the side, so every x in it is the x of the plan, the loops light in the pavement when the plan lights them, and the control station works from either view.

Day and night

The car park runs through the day and the night on the run's own clock. *Day length* is how many minutes of run time make a whole 24-hour day: 24 by default, so twelve minutes of daylight, then twelve of night, with a dusk and a dawn between; *Start hour* sets the clock when the run starts (07:00). A clock beside the counts shows the time. The clock is the recorder's time, so scrubbing back through a run shows the light as it was. Nothing in the program depends on it: the loops, the eye and the limits work the same in the dark, which is the point. What changes is what you can see, and that is what a night shift on a real gate is like.

Figure 227. At night, from above: street lights pool on the road, the cars waiting at the kiosk show their headlights and their brake lights, the booth's window throws light on the lane, the garage's roof poles light the deck.

Figure 228. At night from the side: every floor of the garage lit, the LED headers bright, headlight beams on the road reaching to the car in front, tail lights, the moon over the town.

TRY IT: DUSK AT THE KIOSK

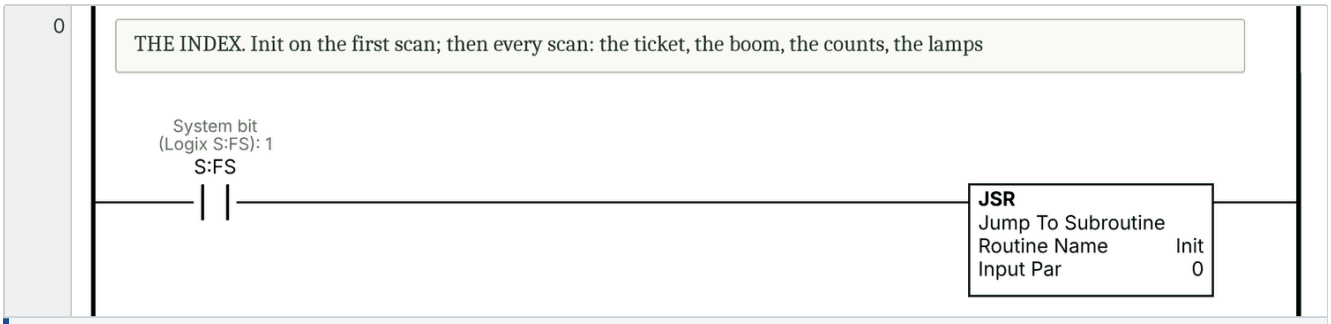
Set *Start hour* to 17 and watch dusk come down over the queue at the kiosk, in the plan, the side view and 3D. Then set *Day length* to 4 and watch a whole day go by in four minutes: the counts and the program do not care.

WATCH FOR: A FULL FLOOR IS NOT A FULL GARAGE

The SPACES sign and the LED headers count the cars on each floor; the program knows only `Lot_Count` against `Lot_Capacity`. LEVEL 1 FULL with 72 stalls free upstairs is a sign for drivers, not a reason to stop selling tickets.

The program's index

MAINROUTINE · RUNG 0




```
XIC(S:FS)JSR(Init);
```

THE INDEX. Init on the first scan; then every scan: the ticket, the boom, the counts, the lamps

HOW THIS RUNG WORKS

In words. When `S:FS` is on, the rung runs the routine `Init`.

Left to right:

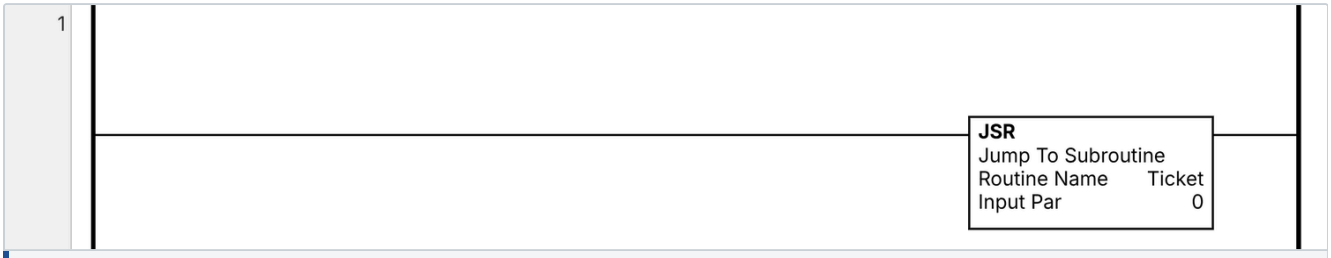
1. `XIC(S:FS)` *Examine If Closed* asks: true when `S:FS` is on.
2. `JSR(Init)` *Jump to Subroutine* does: runs the routine `Init`. The routine is scanned at this point, then the scan comes back here.

Why it is built this way:

Part of the index. The main routine is a list of calls; each routine holds one job, and the scan goes through them in this order.

> **Think of it like this:** the table of contents of a book. The main routine does no work itself: it lists the chapters and the order to read them in, every scan.

MAINROUTINE · RUNG 1




```
JSR(Ticket);
```

HOW THIS RUNG WORKS

In words. Every scan, with nothing to test, the rung runs the routine `Ticket`.

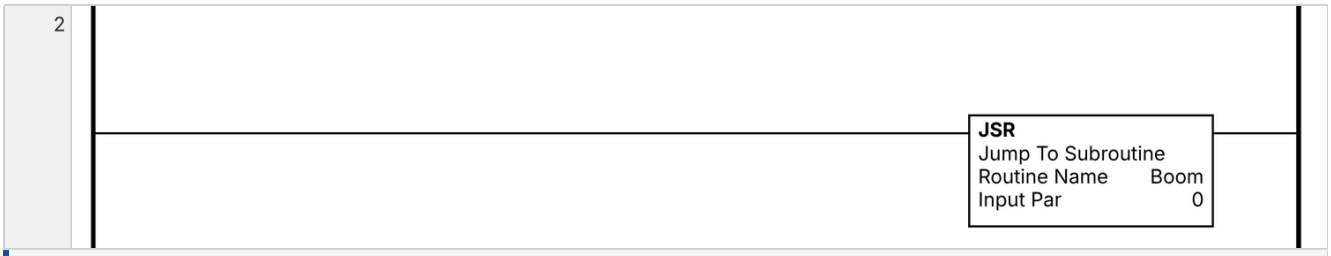
Left to right:

1. `JSR(Ticket)` *Jump to Subroutine* does: runs the routine `Ticket`.

Why it is built this way:

Part of the index: one call, one job.

MAINROUTINE · RUNG 2




```
JSR(Boom);
```

HOW THIS RUNG WORKS

In words. Every scan, with nothing to test, the rung runs the routine `Boom`.

Left to right:

1. `JSR(Boom)` *Jump to Subroutine* does: runs the routine `Boom`.

Why it is built this way:

Part of the index: one call, one job.

MAINROUTINE · RUNG 3

```
JSR(Count);
```

HOW THIS RUNG WORKS

In words. Every scan, with nothing to test, the rung runs the routine `Count`.

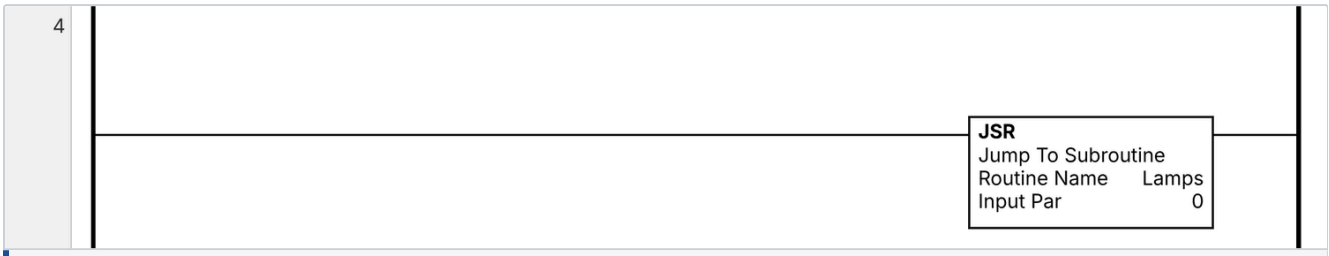
Left to right:

1. `JSR(Count)` *Jump to Subroutine* does: runs the routine `Count`.

Why it is built this way:

Part of the index: one call, one job.

MAINROUTINE · RUNG 4




```
JSR(Lamps);
```

HOW THIS RUNG WORKS

In words. Every scan, with nothing to test, the rung runs the routine `Lamps`.

Left to right:

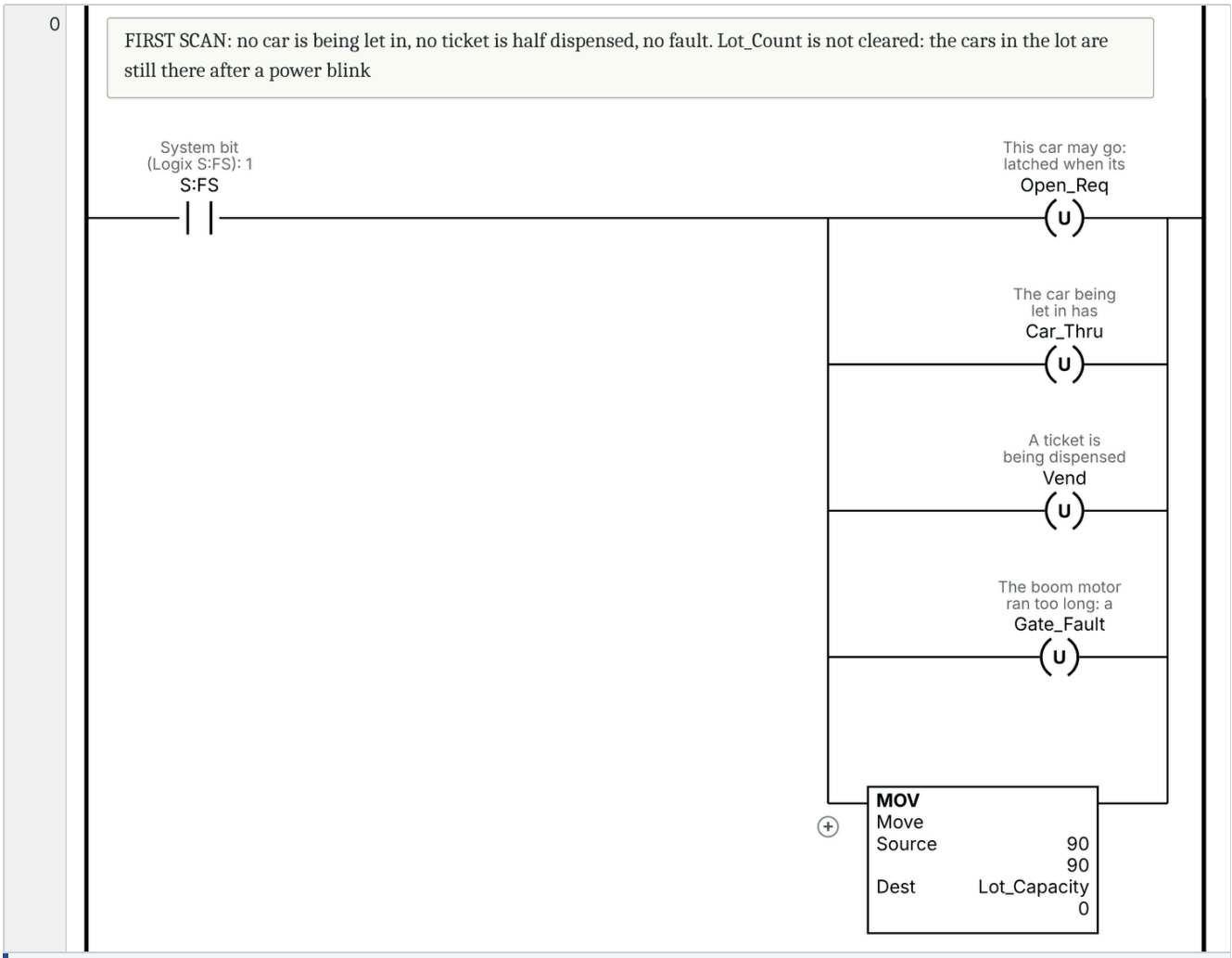
1. `JSR(Lamps)` *Jump to Subroutine* does: runs the routine `Lamps`.

Why it is built this way:

Part of the index: one call, one job.

Tag	Type	Description
`Open_Req`	BOOL	This car may go: latched when its ticket is taken (or by Raise), dropped when it is through
`Car_Thru`	BOOL	The car being let in has reached the closing loop
`Vend`	BOOL	A ticket is being dispensed
`PB_ONS`	BOOL	One-shot storage for the ticket button
`T_Vend`	TIMER	The dispense pulse, half a second
`Gate_Fault`	BOOL	The boom motor ran too long: a limit switch or the motor has failed. Reset clears it
`T_Motor`	TIMER	Boom motor watchdog: 4 s (a full stroke takes 2.4 s)
`Lot_Count`	DINT	Cars in the lot: up on the closing loop, down on the exit loop. Kept through a power cycle
`In_Count`	DINT	Cars let in, all time
`Lot_Capacity`	DINT	Stalls in the garage (90: 18 on LEVEL 1, 24 on each floor above)
`Full`	BOOL	The lot is full
`CL_ONS`	BOOL	One-shot storage, closing loop
`EX_ONS`	BOOL	One-shot storage, exit loop
`T_F1`	TIMER	Flasher, half a second on
`T_F2`	TIMER	Flasher, half a second off
`Fl_Bit`	BOOL	The flasher bit, 1 Hz
`T_Eye`	TIMER	The eye blocked for 30 s: a car stalled under the boom, or the eye has failed

INIT · RUNG 0




```
XIC(S:FS)
[OTU(Open_Req),OTU(Car_Thru),OTU(Vend),OTU(Gate_Fault),MOV(90,Lot_Capacity)];
```

FIRST SCAN: no car is being let in, no ticket is half dispensed, no fault. Lot_Count is not cleared: the cars in the lot are still there after a power blink

HOW THIS RUNG WORKS

In words. When `S:FS` is on, the rung unlatches `Open_Req` (This car may go: latched when its ticket is taken (or by Raise), dropped when it is through) (turns it off), unlatches `Car_Thru` (The car being let in has reached the closing loop) (turns it off), unlatches `Vend` (A ticket is being dispensed) (turns it off), unlatches `Gate_Fault` (The boom motor ran too long: a limit switch or the motor has failed. Reset clears it) (turns it off) and puts **90** into `Lot_Capacity` (Stalls in the garage (90: 18 on LEVEL 1, 24 on each floor above)).

Left to right:

1. `XIC(S:FS)` *Examine If Closed* asks: true when `S:FS` is on.
2. A branch of 5 parallel paths: each path is tested on its own and does its own work.

   Path 1

1. `OTU(Open_Req)` *Output Unlatch* does: unlatches `Open_Req` (This car may go: latched when its ticket is taken (or by Raise), dropped when it is through) (turns it off). It only ever clears the bit.

Path 2

1. `OTU(Car_Thru)` *Output Unlatch* does: unlatches `Car_Thru` (The car being let in has reached the closing loop) (turns it off).

Path 3

1. `OTU(Vend)` *Output Unlatch* does: unlatches `Vend` (A ticket is being dispensed) (turns it off).

Path 4

1. `OTU(Gate_Fault)` *Output Unlatch* does: unlatches `Gate_Fault` (The boom motor ran too long: a limit switch or the motor has failed. Reset clears it) (turns it off).

Path 5

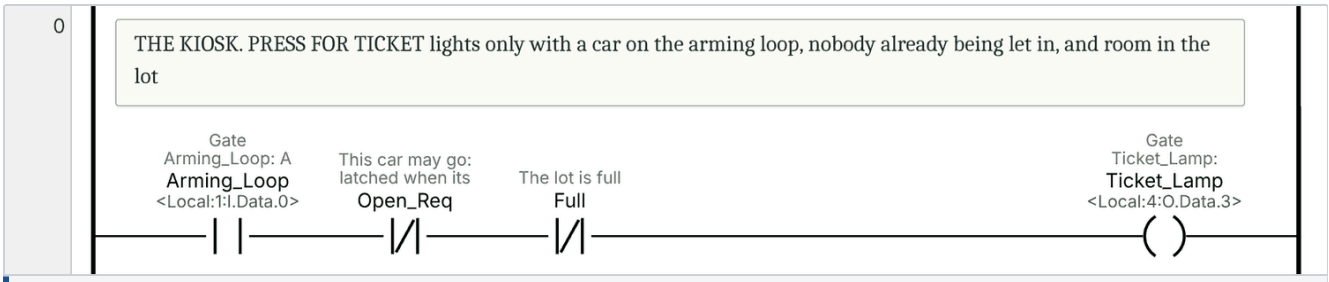
1. `MOV(90,Lot_Capacity)` *Move* does: puts **90** into `Lot_Capacity` (Stalls in the garage (90: 18 on LEVEL 1, 24 on each floor above)). It writes every scan the rung is true.

> **Think of it like this:** a rung is a pipe with valves in it, read left to right. Every contact is a valve: XIC is open while its bit is 1, XIO while its bit is 0. Water reaches the output at the right only if every valve in a row is open. The output does not remember anything: on every scan it is simply wet or dry.

Exercise 1: a ticket for a car

Three conditions sell a ticket: a car on the arming loop, nobody already being let in, room in the lot. The press makes a one-shot, the one-shot latches a half-second dispense pulse, and the driver taking the ticket latches `Open_Req`: this car may go. The arming loop in the press rung is the whole point of an arming loop: a ticket handed to somebody walking past is a free car later.

TICKET · RUNG 0




```
XIC(Arming_Loop)XIO(Open_Req)XIO(Full)OTE(Ticket_Lamp);
```

THE KIOSK. PRESS FOR TICKET lights only with a car on the arming loop, nobody already being let in, and room in the lot

PREDICT IT · 1 `Arming_Loop` is off, `Open_Req` is off and `Full` is off. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

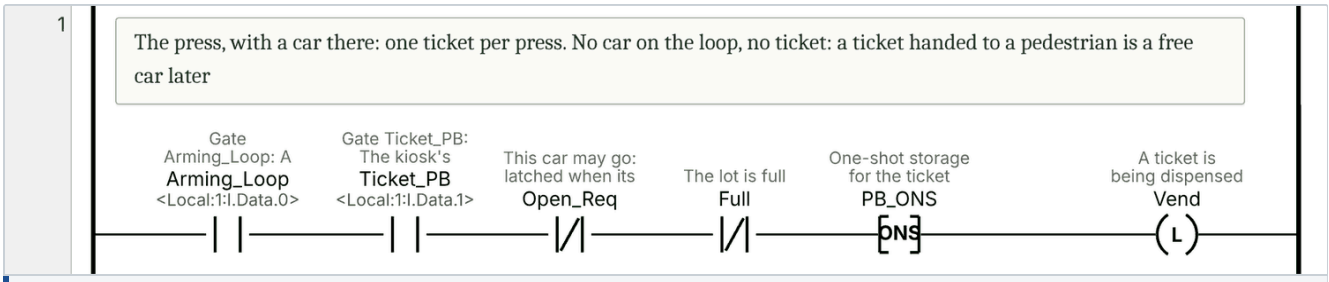
HOW THIS RUNG WORKS

In words. When `Arming_Loop` (A car stands over the loop at the ticket kiosk) is on, `Open_Req` (This car may go: latched when its ticket is taken (or by Raise), dropped when it is through) is off and `Full` (The lot is full) is off, the rung turns `Ticket_Lamp` (PRESS FOR TICKET lamp on the kiosk) on (and off again whenever the rung is false).

Left to right:

1. `XIC(Arming_Loop)` *Examine If Closed* asks: true when `Arming_Loop` (A car stands over the loop at the ticket kiosk) is on.
2. `XIO(Open_Req)` *Examine If Open* asks: true when `Open_Req` (This car may go: latched when its ticket is taken (or by Raise), dropped when it is through) is off.
3. `XIO(Full)` *Examine If Open* asks: true when `Full` (The lot is full) is off.
4. `OTE(Ticket_Lamp)` *Output Energize* does: turns `Ticket_Lamp` (PRESS FOR TICKET lamp on the kiosk) on (and off again whenever the rung is false). Every scan it writes the rung state: true on, false off.

TICKET · RUNG 1




```
XIC(Arming_Loop)XIC(Ticket_PB)XIO(Open_Req)XIO(Full)ONS(PB_ONS)OTL(Vend);
```

The press, with a car there: one ticket per press. No car on the loop, no ticket: a ticket handed to a pedestrian is a free car later

HOW THIS RUNG WORKS

In words. When `Arming_Loop` (A car stands over the loop at the ticket kiosk) is on, `Ticket_PB` (The kiosk's PRESS FOR TICKET button, 1 while pressed) is on, `Open_Req` (This car may go: latched when its ticket is taken (or by Raise), dropped when it is through) is off, `Full` (The lot is full) is off and it is the first scan they are true (the one-shot, remembered in `PB_ONS` (One-shot storage for the ticket button)), the rung latches `Vend` (A ticket is being dispensed) on.

Left to right:

1. `XIC(Arming_Loop)` *Examine If Closed* asks: true when `Arming_Loop` (A car stands over the loop at the ticket kiosk) is on.
2. `XIC(Ticket_PB)` *Examine If Closed* asks: true when `Ticket_PB` (The kiosk's PRESS FOR TICKET button, 1 while pressed) is on.
3. `XIO(Open_Req)` *Examine If Open* asks: true when `Open_Req` (This car may go: latched when its ticket is taken (or by Raise), dropped when it is through) is off.
4. `XIO(Full)` *Examine If Open* asks: true when `Full` (The lot is full) is off.
5. `ONS(PB_ONS)` *One Shot* asks: true when it is the first scan they are true (the one-shot, remembered in `PB_ONS` (One-shot storage for the ticket button)). It passes power for one scan only, the scan its input goes from false to true; its storage bit remembers last scan.
6. `OTL(Vend)` *Output Latch* does: latches `Vend` (A ticket is being dispensed) on. It only ever sets the bit; the bit stays on after the rung goes false.

Why it is built this way:

One-shot. Everything after the ONS happens once per event, not on every scan the condition holds; `PB_ONS` is only its memory of the last scan.

A latch. `Vend` stays on after this rung goes false; an `OTU` elsewhere in the program clears it.

> **Think of it like this:** a doorbell that rings once however long you lean on the button. The ONS looks at what the button was doing last scan: only the change from up to down rings it.

> **Watch for:** Each ONS needs its own storage bit. Two ONS sharing one bit make one of them miss its edges.

TICKET · RUNG 2

```
XIC(Vend)TON(T_Vend,500,0);
```

The dispense pulse: half a second

HOW THIS RUNG WORKS

In words. When `Vend` (A ticket is being dispensed) is on, the rung runs the timer `T_Vend` (The dispense pulse, half a second) toward 500 ms (0.5 s).

Left to right:

1. `XIC(Vend)` *Examine If Closed* asks: true when `Vend` (A ticket is being dispensed) is on.
2. `TON(T_Vend,500,0)` *Timer On Delay* does: runs the timer `T_Vend` (The dispense pulse, half a second) toward 500 ms (0.5 s). The accumulator counts while the rung is true; .DN comes on at the preset; the moment the rung goes false the timer resets.

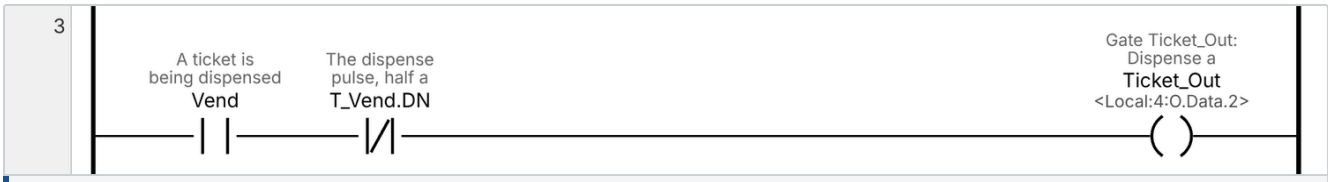
Why it is built this way:

A timer. `T_Vend` only times while this rung is true; other rungs use its `.DN` (done) or `.TT` (timing) bits. Being a TON, it starts again from zero every time the rung goes false.

> **Think of it like this:** a kitchen timer that only runs while you hold your finger on it. Take your finger off and it snaps back to zero. Hold it long enough and the done light comes on, and stays on as long as your finger is still there.

> **Watch for:** If anything in series flickers false for one scan, a TON starts over from zero. Use RTO when the time must survive an interruption.

TICKET · RUNG 3




```
XIC(Vend)XIO(T_Vend.DN)OTE(Ticket_Out);
```

PREDICT IT · 2 `Vend` is on and `T_Vend.DN` (the done bit of `T_Vend` : The dispense pulse, half a second) is off. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Vend` (A ticket is being dispensed) is on and `T_Vend.DN` (the done bit of `T_Vend` : The dispense pulse, half a second) is off, the rung turns `Ticket_Out` (Dispense a ticket (on the rising edge)) on (and off again whenever the rung is false).

Left to right:

1. `XIC(Vend)` *Examine If Closed* asks: true when `Vend` (A ticket is being dispensed) is on.
2. `XIO(T_Vend.DN)` *Examine If Open* asks: true when `T_Vend.DN` (the done bit of `T_Vend` : The dispense pulse, half a second) is off.
3. `OTE(Ticket_Out)` *Output Energize* does: turns `Ticket_Out` (Dispense a ticket (on the rising edge)) on (and off again whenever the rung is false).

TICKET · RUNG 4

```
XIC(T_Vend.DN)OTU(Vend);
```

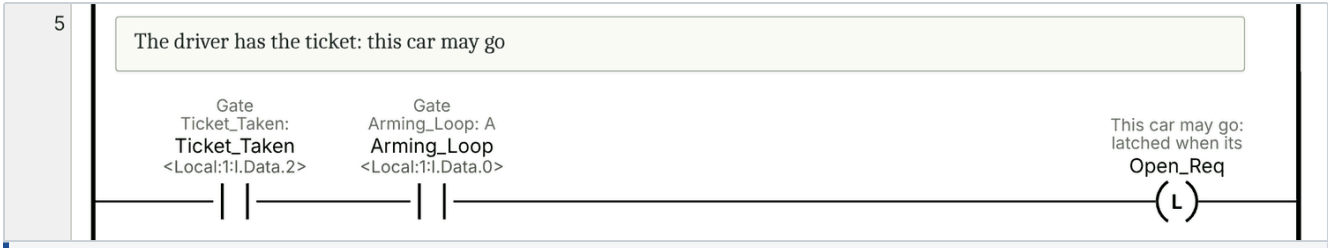
HOW THIS RUNG WORKS

In words. When `T_Vend.DN` (the done bit of `T_Vend` : The dispense pulse, half a second) is on, the rung unlatches `Vend` (A ticket is being dispensed) (turns it off).

Left to right:

1. `XIC(T_Vend.DN)` *Examine If Closed* asks: true when `T_Vend.DN` (the done bit of `T_Vend` : The dispense pulse, half a second) is on.
2. `OTU(Vend)` *Output Unlatch* does: unlatches `Vend` (A ticket is being dispensed) (turns it off).

TICKET · RUNG 5




```
XIC(Ticket_Taken)XIC(Arming_Loop)OTL(Open_Req);
```

The driver has the ticket: this car may go

PREDICT IT · 3 `Ticket_Taken` is on and `Arming_Loop` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Ticket_Taken` (The driver has pulled the ticket from the slot (a short pulse)) is on and `Arming_Loop` (A car stands over the loop at the ticket kiosk) is on, the rung latches `Open_Req` (This car may go: latched when its ticket is taken (or by Raise), dropped when it is through) on.

Left to right:

1. `XIC(Ticket_Taken)` *Examine If Closed* asks: true when `Ticket_Taken` (The driver has pulled the ticket from the slot (a short pulse)) is on.
2. `XIC(Arming_Loop)` *Examine If Closed* asks: true when `Arming_Loop` (A car stands over the loop at the ticket kiosk) is on.
3. `OTL(Open_Req)` *Output Latch* does: latches `Open_Req` (This car may go: latched when its ticket is taken (or by Raise), dropped when it is through) on.

Why it is built this way:

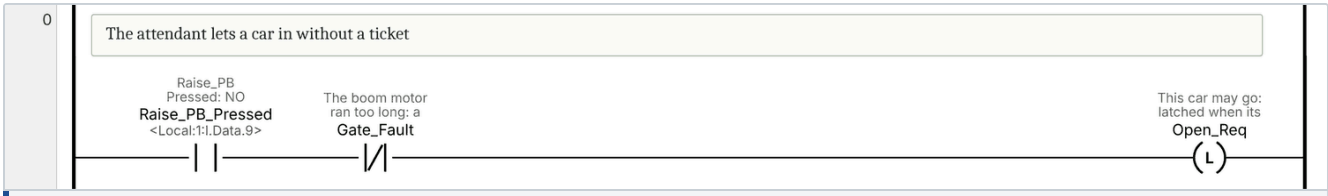
A latch: it stays on until an OTU clears it.

> **Watch for:** A latched bit survives the condition that set it: every way in needs a way out, or it stays on for good.

Exercise 2: up for the car, down behind it

The boom goes up while `Open_Req` is set, to its up limit. The car is through when it stands on the closing loop; when it has left the loop and the eye, `Open_Req` drops and the boom comes straight down behind it, before the next car can follow it in. The lower rung has the two contacts that keep a boom off a car: the eye just past the arm and the closing loop. A watchdog times the motor: a stroke takes 2.4 s, so a motor still running at 4 s has a limit switch that did not make, and the program drops it before its own thermal does.

BOOM · RUNG 0




```
XIC(Raise_PB_Pressed)XIO(Gate_Fault)OTL(Open_Req);
```

The attendant lets a car in without a ticket

PREDICT IT · 4 `Raise_PB_Pressed` is off and `Gate_Fault` is off. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Raise_PB_Pressed` (NO contact: 1 while pressed) is on and `Gate_Fault` (The boom motor ran too long: a limit switch or the motor has failed. Reset clears it) is off, the rung latches `Open_Req` (This car may go: latched when its ticket is taken (or by Raise), dropped when it is through) on.

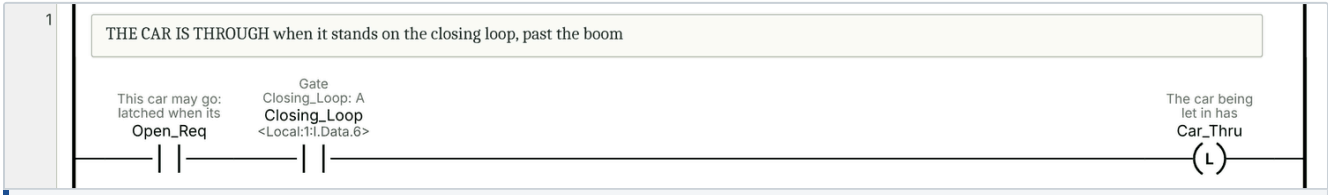
Left to right:

1. `XIC(Raise_PB_Pressed)` *Examine If Closed* asks: true when `Raise_PB_Pressed` (NO contact: 1 while pressed) is on.
2. `XIO(Gate_Fault)` *Examine If Open* asks: true when `Gate_Fault` (The boom motor ran too long: a limit switch or the motor has failed. Reset clears it) is off.
3. `OTL(Open_Req)` *Output Latch* does: latches `Open_Req` (This car may go: latched when its ticket is taken (or by Raise), dropped when it is through) on.

Why it is built this way:

A latch: it stays on until an OTU clears it.

BOOM · RUNG 1




```
XIC(Open_Req)XIC(Closing_Loop)OTL(Car_Thru);
```

THE CAR IS THROUGH when it stands on the closing loop, past the boom

PREDICT IT · 5 `Open_Req` is on and `Closing_Loop` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Open_Req` (This car may go: latched when its ticket is taken (or by Raise), dropped when it is through) is on and `Closing_Loop` (A car stands over the loop past the boom) is on, the rung latches `Car_Thru` (The car being let in has reached the closing loop) on.

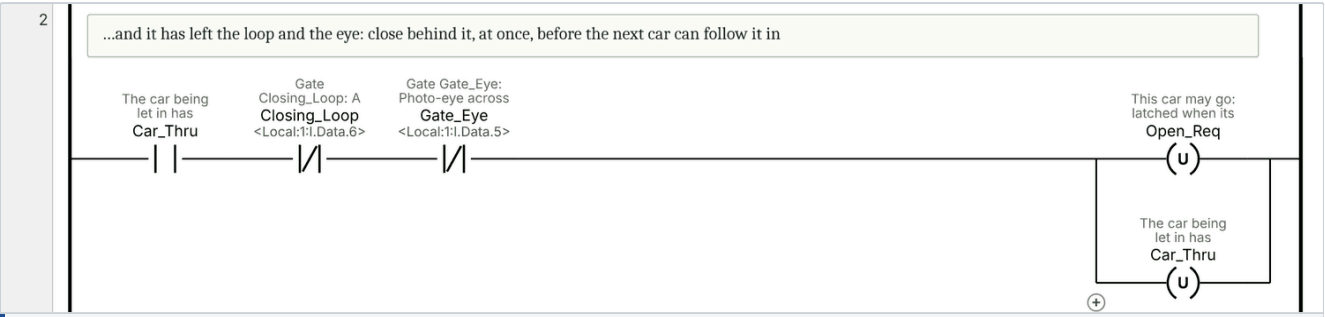
Left to right:

1. `XIC(Open_Req)` *Examine If Closed* asks: true when `Open_Req` (This car may go: latched when its ticket is taken (or by Raise), dropped when it is through) is on.
2. `XIC(Closing_Loop)` *Examine If Closed* asks: true when `Closing_Loop` (A car stands over the loop past the boom) is on.
3. `OTL(Car_Thru)` *Output Latch* does: latches `Car_Thru` (The car being let in has reached the closing loop) on.

Why it is built this way:

A latch: it stays on until an OTU clears it.

BOOM · RUNG 2




```
XIC(Car_Thru)XIO(Closing_Loop)XIO(Gate_Eye)[OTU(Open_Req),OTU(Car_Thru)];
```

...and it has left the loop and the eye: close behind it, at once, before the next car can follow it in

HOW THIS RUNG WORKS

In words. When `Car_Thru` (The car being let in has reached the closing loop) is on, `Closing_Loop` (A car stands over the loop past the boom) is off and `Gate_Eye` (Photo-eye across the lane just past the boom: 1 while the beam is blocked) is off, the rung unlatches `Open_Req` (This car may go: latched when its ticket is taken (or by Raise), dropped when it is through) (turns it off) and unlatches `Car_Thru` (turns it off).

Left to right:

1. `XIC(Car_Thru)` *Examine If Closed* asks: true when `Car_Thru` (The car being let in has reached the closing loop) is on.
2. `XIO(Closing_Loop)` *Examine If Open* asks: true when `Closing_Loop` (A car stands over the loop past the boom) is off.
3. `XIO(Gate_Eye)` *Examine If Open* asks: true when `Gate_Eye` (Photo-eye across the lane just past the boom: 1 while the beam is blocked) is off.
4. A branch of 2 parallel paths: each path is tested on its own and does its own work.

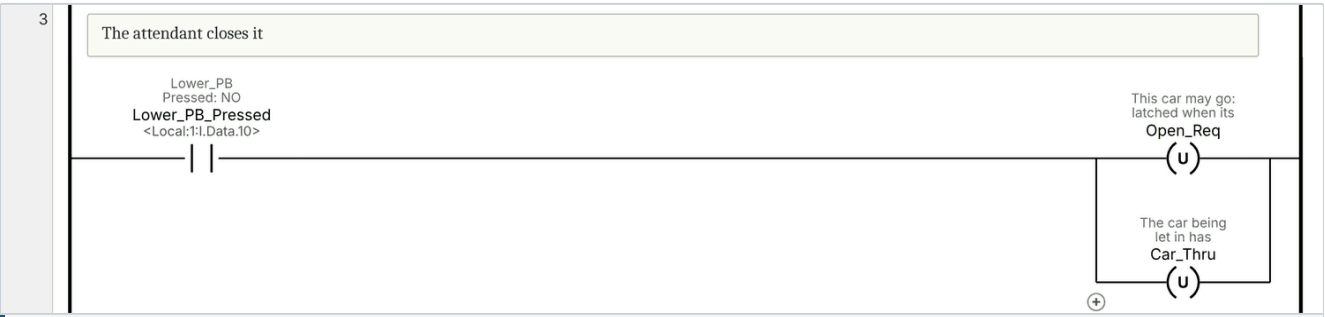
   Path 1

   1. `OTU(Open_Req)` *Output Unlatch* does: unlatches `Open_Req` (This car may go: latched when its ticket is taken (or by Raise), dropped when it is through) (turns it off).

   Path 2

   1. `OTU(Car_Thru)` *Output Unlatch* does: unlatches `Car_Thru` (turns it off).

BOOM · RUNG 3




```
XIC(Lower_PB_Pressed)[OTU(Open_Req),OTU(Car_Thru)];
```

The attendant closes it

HOW THIS RUNG WORKS

In words. When `Lower_PB_Pressed` (NO contact: 1 while pressed) is on, the rung unlatches `Open_Req` (This car may go: latched when its ticket is taken (or by Raise), dropped when it is through) (turns it off) and unlatches `Car_Thru` (The car being let in has reached the closing loop) (turns it off).

Left to right:

1. `XIC(Lower_PB_Pressed)` *Examine If Closed* asks: true when `Lower_PB_Pressed` (NO contact: 1 while pressed) is on.
2. A branch of 2 parallel paths: each path is tested on its own and does its own work.

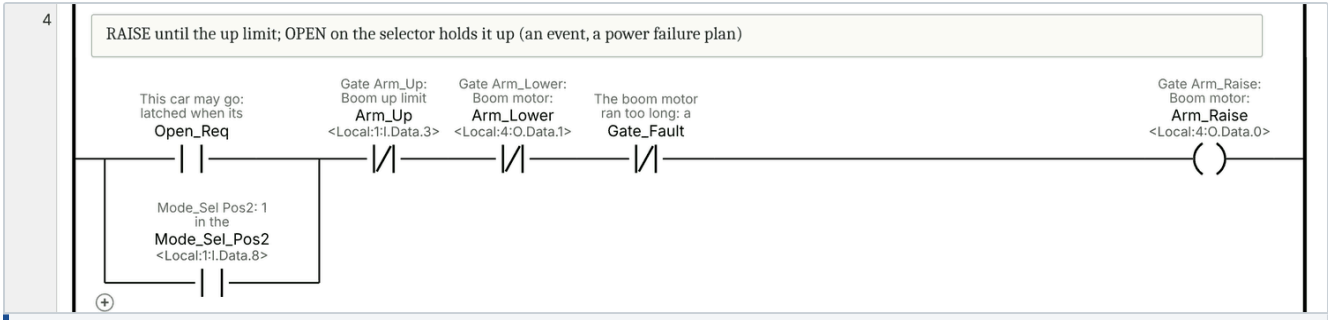
   Path 1

   1. `OTU(Open_Req)` *Output Unlatch* does: unlatches `Open_Req` (This car may go: latched when its ticket is taken (or by Raise), dropped when it is through) (turns it off).

   Path 2

   1. `OTU(Car_Thru)` *Output Unlatch* does: unlatches `Car_Thru` (The car being let in has reached the closing loop) (turns it off).

BOOM · RUNG 4




```
[XIC(Open_Req),XIC(Mode_Sel_Pos2)]XIO(Arm_Up)XIO(Arm_Lower)XIO(Gate_Fault)OTE(Arm_Raise);
```

RAISE until the up limit; OPEN on the selector holds it up (an event, a power failure plan)

PREDICT IT · 6 `Open_Req` is on, `Arm_Up` is off, `Arm_Lower` is off and `Gate_Fault` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When either (`Open_Req` (This car may go: latched when its ticket is taken (or by Raise), dropped when it is through) is on) *or* (`Mode_Sel_Pos2` (1 in the right-hand position) is on), `Arm_Up` (Boom up limit switch: the boom is fully raised) is off, `Arm_Lower` (Boom motor: lower) is off and `Gate_Fault` (The boom motor ran too long: a limit switch or the motor has failed. Reset clears it) is off, the rung turns `Arm_Raise` (Boom motor: raise) on (and off again whenever the rung is false).

Left to right:

1. A branch of 2 parallel paths: power gets through if any one of them is true.
   - **Path 1**
     1. `XIC(Open_Req)` *Examine If Closed* asks: true when `Open_Req` (This car may go: latched when its ticket is taken (or by Raise), dropped when it is through) is on.
   - **Path 2**
     1. `XIC(Mode_Sel_Pos2)` *Examine If Closed* asks: true when `Mode_Sel_Pos2` (1 in the right-hand position) is on.
2. `XIO(Arm_Up)` *Examine If Open* asks: true when `Arm_Up` (Boom up limit switch: the boom is fully raised) is off.
3. `XIO(Arm_Lower)` *Examine If Open* asks: true when `Arm_Lower` (Boom motor: lower) is off.
4. `XIO(Gate_Fault)` *Examine If Open* asks: true when `Gate_Fault` (The boom motor ran too long: a limit switch or the motor has failed. Reset clears it) is off.
5. `OTE(Arm_Raise)` *Output Energize* does: turns `Arm_Raise` (Boom motor: raise) on (and off again whenever the rung is false).

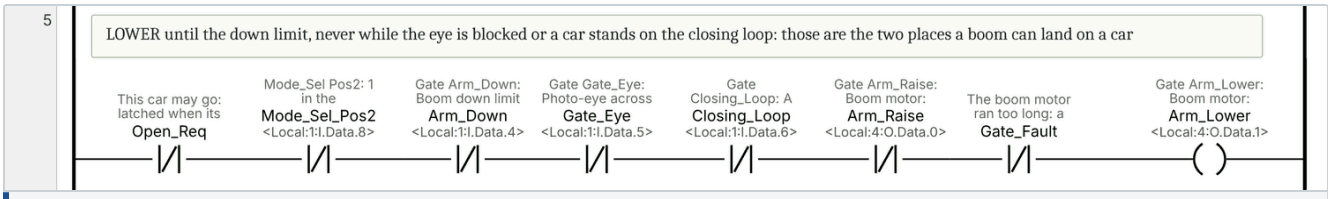
Why it is built this way:

Interlock. `Arm_Raise` can only come on while `Arm_Lower` is off, and the rung for `Arm_Lower` checks the same the other way round. The two can never be on together.

> **Think of it like this:** the two doors of an airlock. Each door checks that the other one is shut before it will open, so whichever moves first, the other has to wait.

> **Watch for:** Both rungs need the interlock. With it on one side only, the other can still come on first.

BOOM · RUNG 5




```
XIO(Open_Req)XIO(Mode_Sel_Pos2)XIO(Arm_Down)XIO(Gate_Eye)XIO(Closing_Loop)XIO(Arm_Raise)XIO(Gate_Fault)OTE(Arm_Lower);
```

LOWER until the down limit, never while the eye is blocked or a car stands on the closing loop: those are the two places a boom can land on a car

PREDICT IT · 7 `Open_Req` is off, `Mode_Sel_Pos2` is off, `Arm_Down` is off, `Gate_Eye` is off, `Closing_Loop` is off, `Arm_Raise` is off and `Gate_Fault` is off. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Open_Req` (This car may go: latched when its ticket is taken (or by Raise), dropped when it is through) is off, `Mode_Sel_Pos2` (1 in the right-hand position) is off, `Arm_Down` (Boom down limit switch: the boom is fully lowered) is off, `Gate_Eye` (Photo-eye across the lane just past the boom: 1 while the beam is blocked) is off, `Closing_Loop` (A car stands over the loop past the boom) is off, `Arm_Raise` (Boom motor: raise) is off and `Gate_Fault` (The boom motor ran too long: a limit switch or the motor has failed. Reset clears it) is off, the rung turns `Arm_Lower` (Boom motor: lower) on (and off again whenever the rung is false).

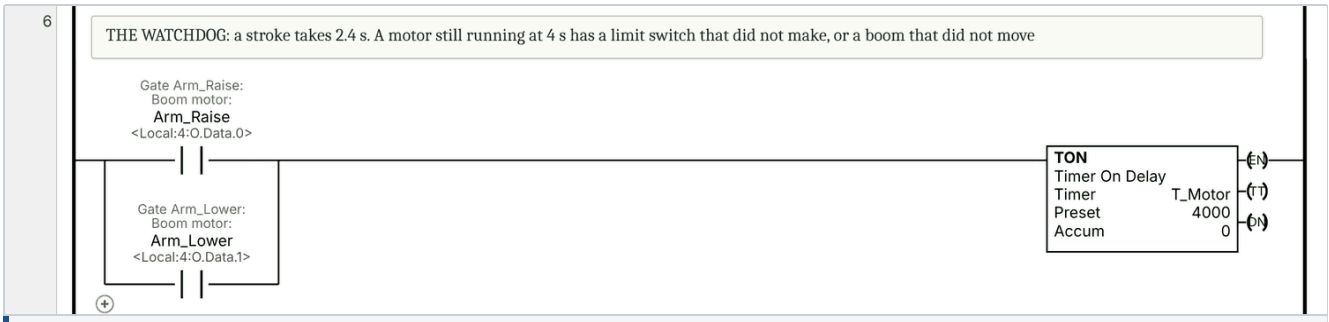
Left to right:

1. `XIO(Open_Req)` *Examine If Open* asks: true when `Open_Req` (This car may go: latched when its ticket is taken (or by Raise), dropped when it is through) is off.
2. `XIO(Mode_Sel_Pos2)` *Examine If Open* asks: true when `Mode_Sel_Pos2` (1 in the right-hand position) is off.
3. `XIO(Arm_Down)` *Examine If Open* asks: true when `Arm_Down` (Boom down limit switch: the boom is fully lowered) is off.
4. `XIO(Gate_Eye)` *Examine If Open* asks: true when `Gate_Eye` (Photo-eye across the lane just past the boom: 1 while the beam is blocked) is off.
5. `XIO(Closing_Loop)` *Examine If Open* asks: true when `Closing_Loop` (A car stands over the loop past the boom) is off.
6. `XIO(Arm_Raise)` *Examine If Open* asks: true when `Arm_Raise` (Boom motor: raise) is off.
7. `XIO(Gate_Fault)` *Examine If Open* asks: true when `Gate_Fault` (The boom motor ran too long: a limit switch or the motor has failed. Reset clears it) is off.
8. `OTE(Arm_Lower)` *Output Energize* does: turns `Arm_Lower` (Boom motor: lower) on (and off again whenever the rung is false).

Why it is built this way:

Interlock: one of the pair, never both.

BOOM · RUNG 6




```
[XIC(Arm_Raise),XIC(Arm_Lower)]TON(T_Motor,4000,0);
```

THE WATCHDOG: a stroke takes 2.4 s. A motor still running at 4 s has a limit switch that did not make, or a boom that did not move

PREDICT IT · 8 `Arm_Raise` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When either (`Arm_Raise` (Boom motor: raise) is on) *or* (`Arm_Lower` (Boom motor: lower) is on), the rung runs the timer `T_Motor` (Boom motor watchdog: 4 s (a full stroke takes 2.4 s)) toward 4000 ms (4 s).

Left to right:

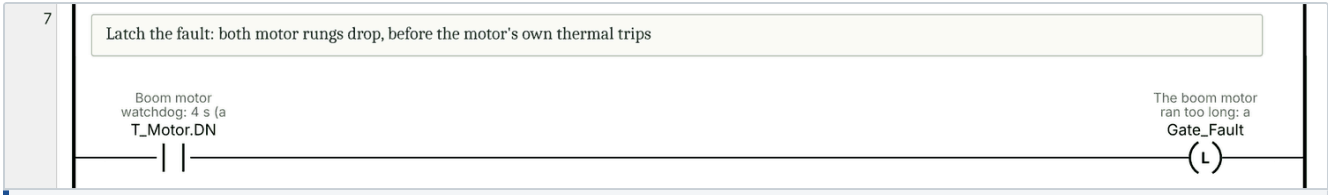
1. A branch of 2 parallel paths: power gets through if any one of them is true.
   - **Path 1**
     1. `XIC(Arm_Raise)` *Examine If Closed* asks: true when `Arm_Raise` (Boom motor: raise) is on.
   - **Path 2**
     1. `XIC(Arm_Lower)` *Examine If Closed* asks: true when `Arm_Lower` (Boom motor: lower) is on.
2. `TON(T_Motor,4000,0)` *Timer On Delay* does: runs the timer `T_Motor` (Boom motor watchdog: 4 s (a full stroke takes 2.4 s)) toward 4000 ms (4 s).

Why it is built this way:

A timer: it times only while this rung is true.

> **Think of it like this:** parallel paths are two roads to the same town. If either road is open you get there; the rung only cares that power reaches the far end, not which way it went.

BOOM · RUNG 7




```
XIC(T_Motor.DN)OTL(Gate_Fault);
```

Latch the fault: both motor rungs drop, before the motor's own thermal trips

HOW THIS RUNG WORKS

In words. When `T_Motor.DN` (the done bit of `T_Motor` : Boom motor watchdog: 4 s (a full stroke takes 2.4 s)) is on, the rung latches `Gate_Fault` (The boom motor ran too long: a limit switch or the motor has failed. Reset clears it) on.

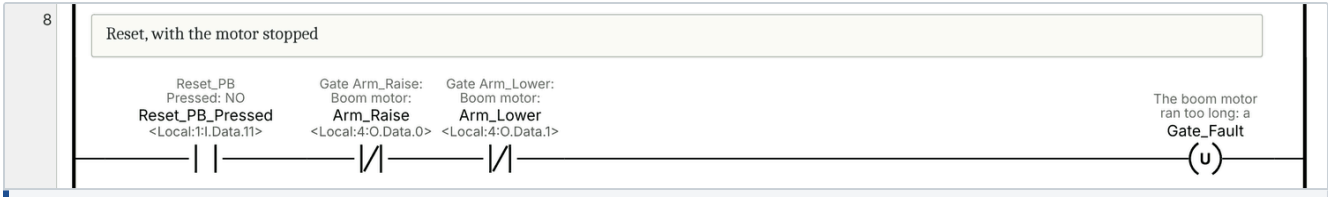
Left to right:

1. `XIC(T_Motor.DN)` *Examine If Closed* asks: true when `T_Motor.DN` (the done bit of `T_Motor` : Boom motor watchdog: 4 s (a full stroke takes 2.4 s)) is on.
2. `OTL(Gate_Fault)` *Output Latch* does: latches `Gate_Fault` (The boom motor ran too long: a limit switch or the motor has failed. Reset clears it) on.

Why it is built this way:

A latch: it stays on until an OTU clears it.

BOOM · RUNG 8




```
XIC(Reset_PB_Pressed)XIO(Arm_Raise)XIO(Arm_Lower)OTU(Gate_Fault);
```

Reset, with the motor stopped

PREDICT IT · 9 `Reset_PB_Pressed` is on, `Arm_Raise` is on and `Arm_Lower` is off. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Reset_PB_Pressed` (NO contact: 1 while pressed) is on, `Arm_Raise` (Boom motor: raise) is off and `Arm_Lower` (Boom motor: lower) is off, the rung unlatches `Gate_Fault` (The boom motor ran too long: a limit switch or the motor has failed. Reset clears it) (turns it off).

Left to right:

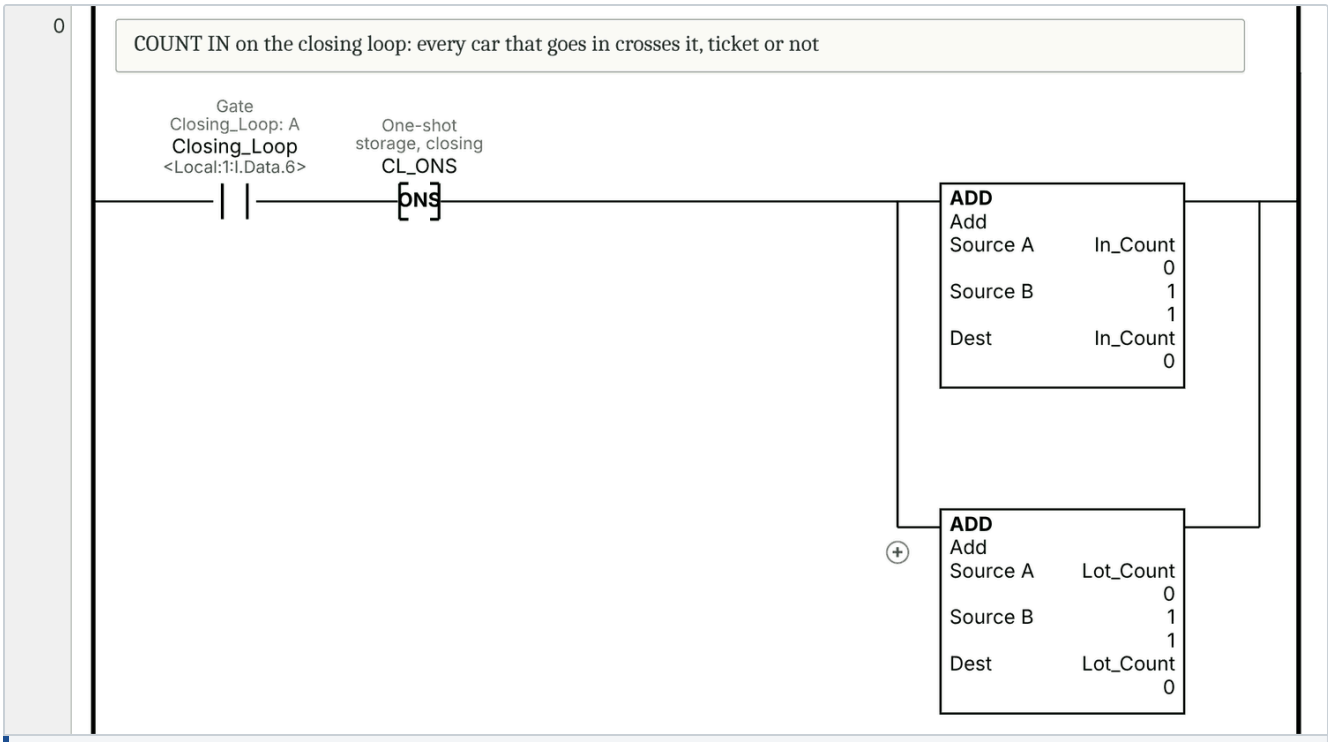
1. `XIC(Reset_PB_Pressed)` *Examine If Closed* asks: true when `Reset_PB_Pressed` (NO contact: 1 while pressed) is on.
2. `XIO(Arm_Raise)` *Examine If Open* asks: true when `Arm_Raise` (Boom motor: raise) is off.
3. `XIO(Arm_Lower)` *Examine If Open* asks: true when `Arm_Lower` (Boom motor: lower) is off.
4. `OTU(Gate_Fault)` *Output Unlatch* does: unlatches `Gate_Fault` (The boom motor ran too long: a limit switch or the motor has failed. Reset clears it) (turns it off).

WATCH FOR: CLOSING ON THE SECOND CAR

Turn on tailgating and watch the boom come down the moment the first car clears the eye. A tailgater close behind is between the kiosk and the eye, under the arm, and neither the eye nor the closing loop can see it there: the log says THE BOOM CAME DOWN ON A CAR. Real gates handle this with a third detector, a safety loop directly under the arm, or by refusing to start lowering while the arming loop is still occupied. Add the second: `XIO(Arming_Loop)` in the lower rung costs a few seconds per car and nobody's roof.

Exercise 3: count in, count out

COUNT · RUNG 0




```
XIC(Closing_Loop)ONS(CL_ONS)
[ADD(In_Count,1,In_Count),ADD(Lot_Count,1,Lot_Count)];
```

COUNT IN on the closing loop: every car that goes in crosses it, ticket or not

HOW THIS RUNG WORKS

In words. When `Closing_Loop` (A car stands over the loop past the boom) is on and it is the first scan they are true (the one-shot, remembered in `CL_ONS` (One-shot storage, closing loop)), the rung makes `In_Count` (Cars let in, all time) = `In_Count` + **1** and makes `Lot_Count` (Cars in the lot: up on the closing loop, down on the exit loop. Kept through a power cycle) = `Lot_Count` + **1**.

Left to right:

1. `XIC(Closing_Loop)` *Examine If Closed* asks: true when `Closing_Loop` (A car stands over the loop past the boom) is on.
2. `ONS(CL_ONS)` *One Shot* asks: true when it is the first scan they are true (the one-shot, remembered in `CL_ONS` (One-shot storage, closing loop)).
3. A branch of 2 parallel paths: each path is tested on its own and does its own work.

   Path 1

   1. `ADD(In_Count,1,In_Count)` *Add* does: makes `In_Count` (Cars let in, all time) = `In_Count` + **1**. Every scan the rung is true; behind a one-shot it adds once per event.

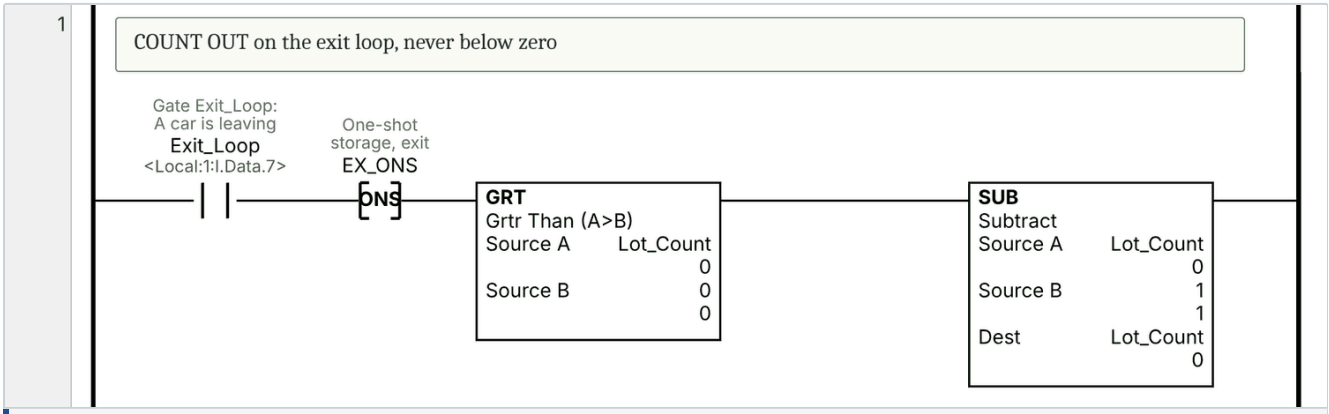
   Path 2

   1. `ADD(Lot_Count,1,Lot_Count)` *Add* does: makes `Lot_Count` (Cars in the lot: up on the closing loop, down on the exit loop. Kept through a power cycle) = `Lot_Count` + **1**.

Why it is built this way:

One-shot: once per event, not once per scan.

COUNT · RUNG 1




```
XIC(Exit_Loop)ONS(EX_ONS)GRT(Lot_Count,0)SUB(Lot_Count,1,Lot_Count);
```

COUNT OUT on the exit loop, never below zero

HOW THIS RUNG WORKS

In words. When `Exit_Loop` (A car is leaving the lot over the exit lane's loop) is on, it is the first scan they are true (the one-shot, remembered in `EX_ONS` (One-shot storage, exit loop)) and `Lot_Count` (Cars in the lot: up on the closing loop, down on the exit loop. Kept through a power cycle) is greater than **0**, the rung makes `Lot_Count = Lot_Count − 1`.

Left to right:

1. `XIC(Exit_Loop)` *Examine If Closed* asks: true when `Exit_Loop` (A car is leaving the lot over the exit lane's loop) is on.
2. `ONS(EX_ONS)` *One Shot* asks: true when it is the first scan they are true (the one-shot, remembered in `EX_ONS` (One-shot storage, exit loop)).
3. `GRT(Lot_Count,0)` *Greater Than* asks: true when `Lot_Count` (Cars in the lot: up on the closing loop, down on the exit loop. Kept through a power cycle) is greater than **0**.
4. `SUB(Lot_Count,1,Lot_Count)` *Subtract* does: makes `Lot_Count = Lot_Count − 1`.

Why it is built this way:

One-shot: once per event, not once per scan.

COUNT · RUNG 2

```
GEQ(Lot_Count,Lot_Capacity)OTE(Full);
```

FULL at the capacity

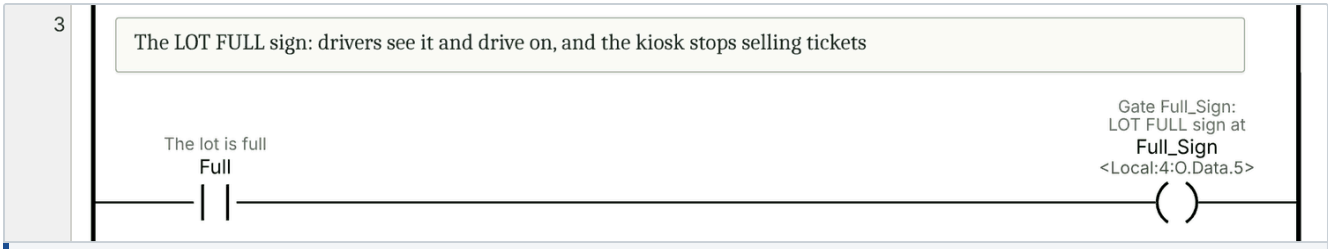
HOW THIS RUNG WORKS

In words. When `Lot_Count` (Cars in the lot: up on the closing loop, down on the exit loop. Kept through a power cycle) is at least `Lot_Capacity` (Stalls in the garage (90: 18 on LEVEL 1, 24 on each floor above)), the rung turns `Full` (The lot is full) on (and off again whenever the rung is false).

Left to right:

1. `GEQ(Lot_Count,Lot_Capacity)` *Greater Than or Equal* asks: true when `Lot_Count` (Cars in the lot: up on the closing loop, down on the exit loop. Kept through a power cycle) is at least `Lot_Capacity` (Stalls in the garage (90: 18 on LEVEL 1, 24 on each floor above)).
2. `OTE(Full)` *Output Energize* does: turns `Full` (The lot is full) on (and off again whenever the rung is false).

COUNT · RUNG 3




```
XIC(Full)OTE(Full_Sign);
```

The LOT FULL sign: drivers see it and drive on, and the kiosk stops selling tickets

HOW THIS RUNG WORKS

In words. When `Full` (The lot is full) is on, the rung turns `Full_Sign` (LOT FULL sign at the entrance) on (and off again whenever the rung is false).

Left to right:

1. `XIC(Full)` *Examine If Closed* asks: true when `Full` (The lot is full) is on.
2. `OTE(Full_Sign)` *Output Energize* does: turns `Full_Sign` (LOT FULL sign at the entrance) on (and off again whenever the rung is false).

The count goes up on the closing loop and not on the ticket, because a car that comes in without one (an attendant's Raise, a tailgater) still takes a stall. It is a DINT, not a counter instruction, and it is not cleared at power-up: the cars are still in the lot when the power comes back.

Exercise 4: the lamps

LAMPS · RUNG 0

```
XIO(T_F2.DN)TON(T_F1,500,0);
```

THE FLASHER

HOW THIS RUNG WORKS

In words. When `T_F2.DN` (the done bit of `T_F2` : Flasher, half a second off) is off, the rung runs the timer `T_F1` (Flasher, half a second on) toward 500 ms (0.5 s).

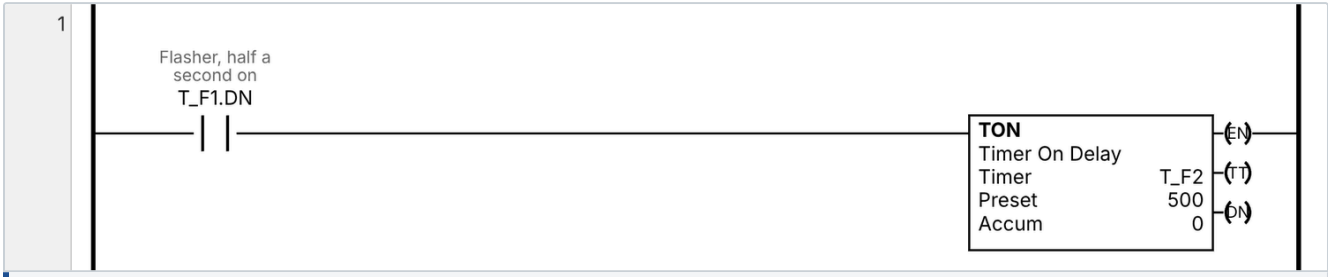
Left to right:

1. `XIO(T_F2.DN)` *Examine If Open* asks: true when `T_F2.DN` (the done bit of `T_F2` : Flasher, half a second off) is off.
2. `TON(T_F1,500,0)` *Timer On Delay* does: runs the timer `T_F1` (Flasher, half a second on) toward 500 ms (0.5 s).

Why it is built this way:

A timer: it times only while this rung is true.

LAMPS · RUNG 1




```
XIC(T_F1.DN)TON(T_F2,500,0);
```

HOW THIS RUNG WORKS

In words. When `T_F1.DN` (the done bit of `T_F1` : Flasher, half a second on) is on, the rung runs the timer `T_F2` (Flasher, half a second off) toward 500 ms (0.5 s).

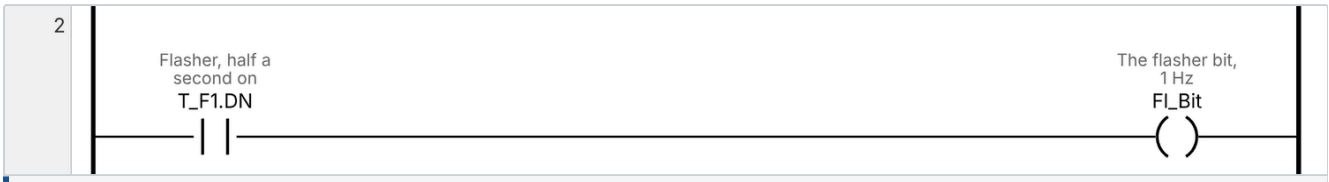
Left to right:

1. `XIC(T_F1.DN)` *Examine If Closed* asks: true when `T_F1.DN` (the done bit of `T_F1` : Flasher, half a second on) is on.
2. `TON(T_F2,500,0)` *Timer On Delay* does: runs the timer `T_F2` (Flasher, half a second off) toward 500 ms (0.5 s).

Why it is built this way:

A timer: it times only while this rung is true.

LAMPS · RUNG 2




```
XIC(T_F1.DN)OTE(Fl_Bit);
```

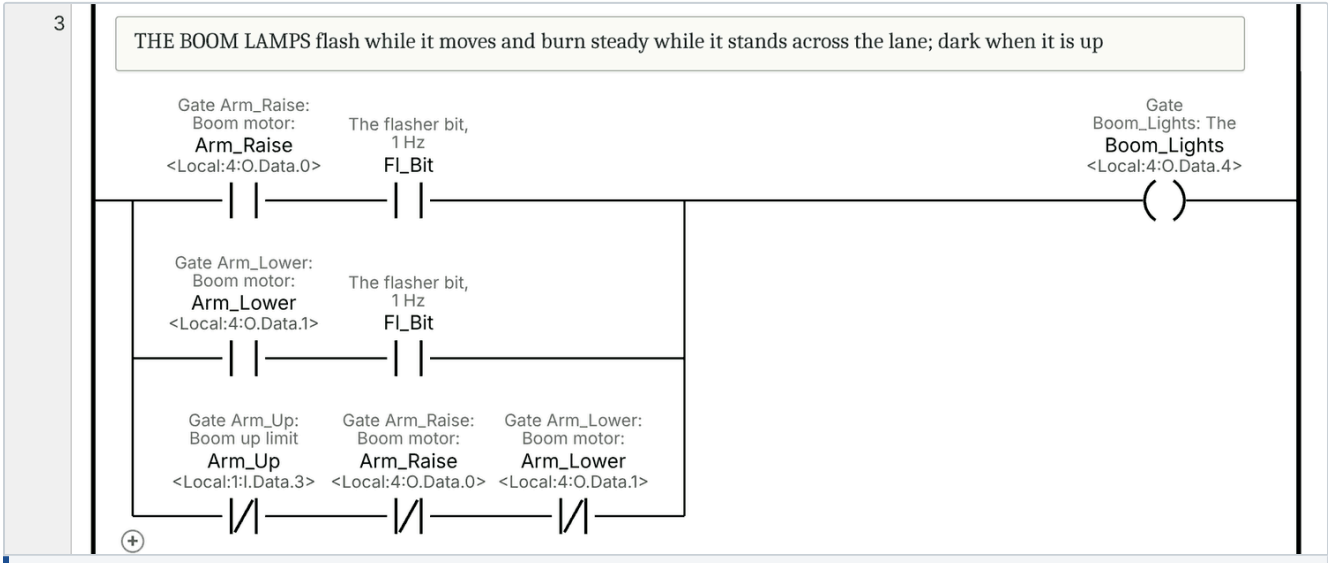
HOW THIS RUNG WORKS

In words. When `T_F1.DN` (the done bit of `T_F1` : Flasher, half a second on) is on, the rung turns `Fl_Bit` (The flasher bit) on (and off again whenever the rung is false).

Left to right:

1. `XIC(T_F1.DN)` *Examine If Closed* asks: true when `T_F1.DN` (the done bit of `T_F1` : Flasher, half a second on) is on.
2. `OTE(Fl_Bit)` *Output Energize* does: turns `Fl_Bit` (The flasher bit) on (and off again whenever the rung is false).

LAMPS · RUNG 3




```
[XIC(Arm_Raise)XIC(Fl_Bit),XIC(Arm_Lower)XIC(Fl_Bit),XIO(Arm_Up)XIO(Arm_Raise)X
IO(Arm_Lower)]OTE(Boom_Lights);
```

THE BOOM LAMPS flash while it moves and burn steady while it stands across the lane; dark when it is up

PREDICT IT · 10 `Arm_Raise` is on and `Fl_Bit` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When either (`Arm_Raise` (Boom motor: raise) is on and `Fl_Bit` (The flasher bit) is on) *or* (`Arm_Lower` (Boom motor: lower) is on and `Fl_Bit` is on) *or* (`Arm_Up` (Boom up limit switch: the boom is fully raised) is off, `Arm_Raise` is off and `Arm_Lower` is off), the rung turns `Boom_Lights` (The red lamps along the boom) on (and off again whenever the rung is false).

Left to right:

1. A branch of 3 parallel paths: power gets through if any one of them is true.

   Path 1

   1. `XIC(Arm_Raise)` *Examine If Closed* asks: true when `Arm_Raise` (Boom motor: raise) is on.
   2. `XIC(Fl_Bit)` *Examine If Closed* asks: true when `Fl_Bit` (The flasher bit) is on.

   Path 2

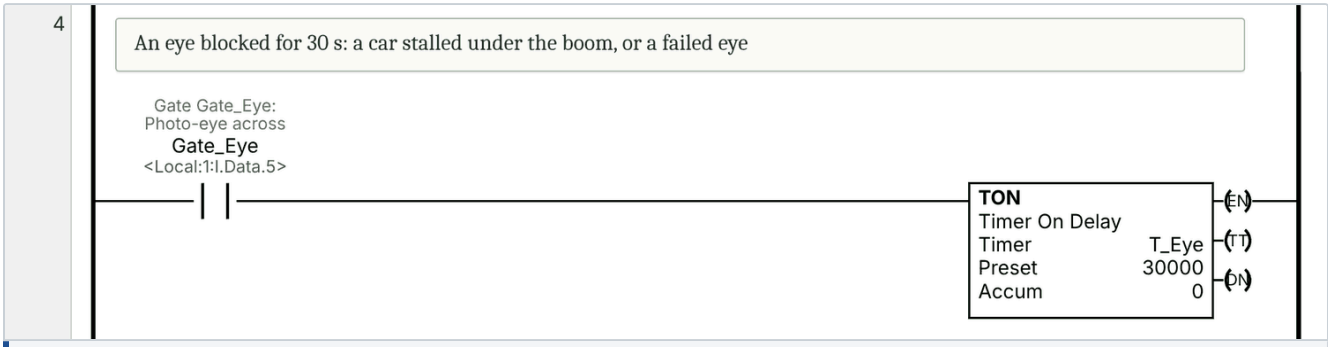
   1. `XIC(Arm_Lower)` *Examine If Closed* asks: true when `Arm_Lower` (Boom motor: lower) is on.
   2. `XIC(Fl_Bit)` *Examine If Closed* asks: true when `Fl_Bit` is on.

   Path 3

   1. `XIO(Arm_Up)` *Examine If Open* asks: true when `Arm_Up` (Boom up limit switch: the boom is fully raised) is off.
   2. `XIO(Arm_Raise)` *Examine If Open* asks: true when `Arm_Raise` is off.
   3. `XIO(Arm_Lower)` *Examine If Open* asks: true when `Arm_Lower` is off.

2. `OTE(Boom_Lights)` *Output Energize* does: turns `Boom_Lights` (The red lamps along the boom) on (and off again whenever the rung is false).

LAMPS · RUNG 4




```
XIC(Gate_Eye)TON(T_Eye,30000,0);
```

An eye blocked for 30 s: a car stalled under the boom, or a failed eye

HOW THIS RUNG WORKS

In words. When `Gate_Eye` (Photo-eye across the lane just past the boom: 1 while the beam is blocked) is on, the rung runs the timer `T_Eye` (The eye blocked for 30 s with no forklift waiting: something is standing in the doorway, or the eye has failed) toward 30000 ms (30 s).

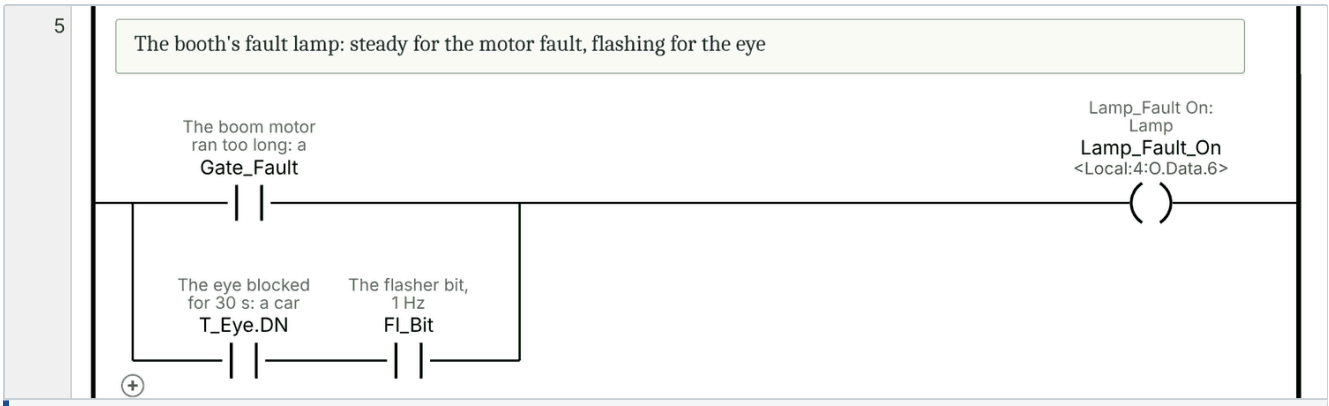
Left to right:

1. `XIC(Gate_Eye)` *Examine If Closed* asks: true when `Gate_Eye` (Photo-eye across the lane just past the boom: 1 while the beam is blocked) is on.
2. `TON(T_Eye,30000,0)` *Timer On Delay* does: runs the timer `T_Eye` (The eye blocked for 30 s with no forklift waiting: something is standing in the doorway, or the eye has failed) toward 30000 ms (30 s).

Why it is built this way:

A timer: it times only while this rung is true.

LAMPS · RUNG 5




```
[XIC(Gate_Fault),XIC(T_Eye.DN)XIC(Fl_Bit)]OTE(Lamp_Fault_On);
```

The booth's fault lamp: steady for the motor fault, flashing for the eye

PREDICT IT · 11 `Gate_Fault` is on. What does the rung do?

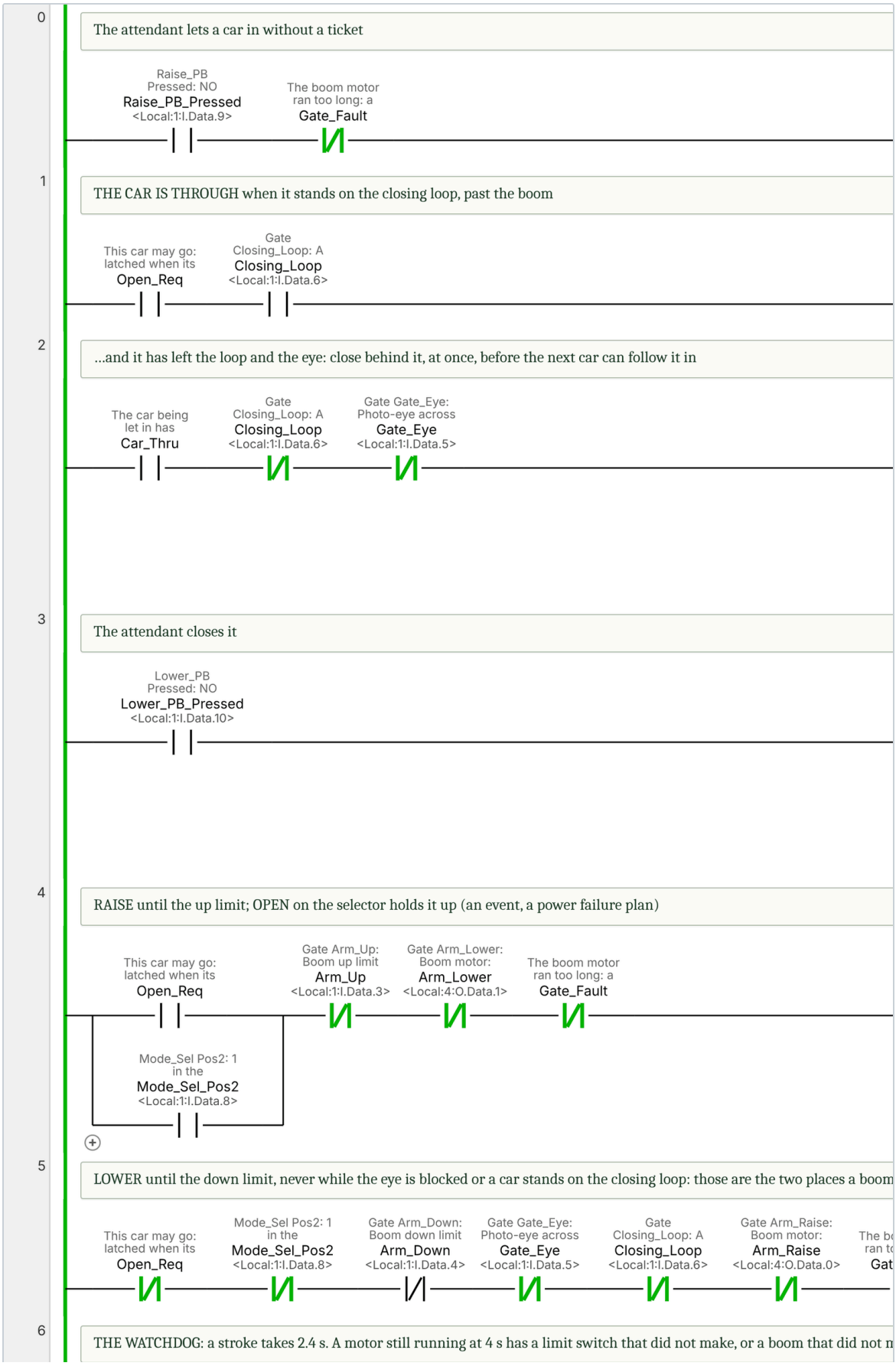
Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When either (`Gate_Fault` (The boom motor ran too long: a limit switch or the motor has failed. Reset clears it) is on) *or* (`T_Eye.DN` (the done bit of `T_Eye` : The eye blocked for 30 s with no forklift waiting: something is standing in the doorway, or the eye has failed) is on and `Fl_Bit` (The flasher bit) is on), the rung turns `Lamp_Fault_On` (Lamp) on (and off again whenever the rung is false).

Left to right:

1. A branch of 2 parallel paths: power gets through if any one of them is true.
   - **Path 1**
     1. `XIC(Gate_Fault)` *Examine If Closed* asks: true when `Gate_Fault` (The boom motor ran too long: a limit switch or the motor has failed. Reset clears it) is on.
   - **Path 2**
     1. `XIC(T_Eye.DN)` *Examine If Closed* asks: true when `T_Eye.DN` (the done bit of `T_Eye` : The eye blocked for 30 s with no forklift waiting: something is standing in the doorway, or the eye has failed) is on.
     2. `XIC(Fl_Bit)` *Examine If Closed* asks: true when `Fl_Bit` (The flasher bit) is on.
2. `OTE(Lamp_Fault_On)` *Output Energize* does: turns `Lamp_Fault_On` (Lamp) on (and off again whenever the rung is false).


0
The attendant lets a car in without a ticket
Raise_PB
Pressed: NO
Raise_PB_Pressed
<Local:1:I.Data.9>
The boom motor
ran too long: a
Gate_Fault
1
THE CAR IS THROUGH when it stands on the closing loop, past the boom
This car may go:
latched when its
Open_Req
Gate
Closing_Loop: A
Closing_Loop
<Local:1:I.Data.6>
2
...and it has left the loop and the eye: close behind it, at once, before the next car can follow it in
The car being
let in has
Car_Thru
Gate
Closing_Loop: A
Closing_Loop
<Local:1:I.Data.6>
Gate Gate_Eye:
Photo-eye across
Gate_Eye
<Local:1:I.Data.5>
3
The attendant closes it
Lower_PB
Pressed: NO
Lower_PB_Pressed
<Local:1:I.Data.10>
4
RAISE until the up limit; OPEN on the selector holds it up (an event, a power failure plan)
This car may go:
latched when its
Open_Req
Gate Arm_Up:
Boom up limit
Arm_Up
<Local:1:I.Data.3>
Gate Arm_Lower:
Boom motor:
Arm_Lower
<Local:4:O.Data.1>
The boom motor
ran too long: a
Gate_Fault
Mode_Sel Pos2: 1
in the
Mode_Sel_Pos2
<Local:1:I.Data.8>
5
LOWER until the down limit, never while the eye is blocked or a car stands on the closing loop: those are the two places a boom
This car may go:
latched when its
Open_Req
Mode_Sel Pos2: 1
in the
Mode_Sel_Pos2
<Local:1:I.Data.8>
Gate Arm_Down:
Boom down limit
Arm_Down
<Local:1:I.Data.4>
Gate Gate_Eye:
Photo-eye across
Gate_Eye
<Local:1:I.Data.5>
Gate
Closing_Loop: A
Closing_Loop
<Local:1:I.Data.6>
Gate Arm_Raise:
Boom motor:
Arm_Raise
<Local:4:O.Data.0>
6
THE WATCHDOG: a stroke takes 2.4 s. A motor still running at 4 s has a limit switch that did not make, or a boom that did not

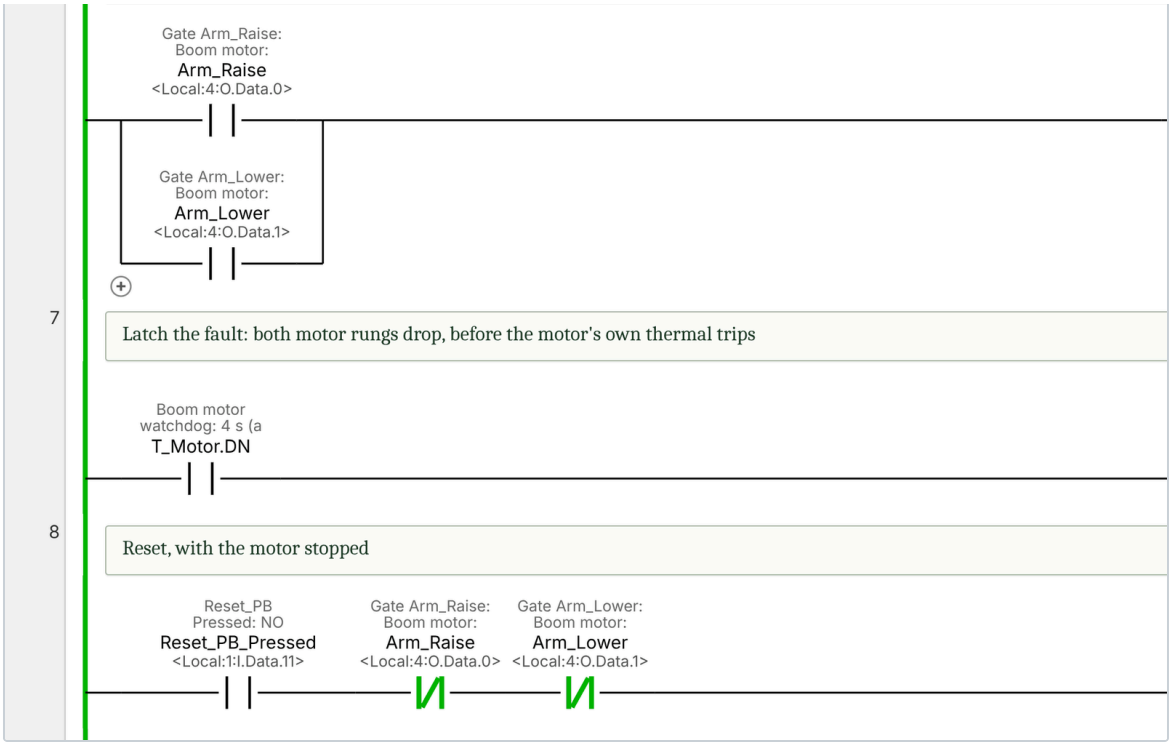


Figure 229. The Boom routine online, a car on its way in.

Machine faults on the parking gate

Tools → Inject Machine Fault… lists the faults that belong to the parts on this floor. Each has a fault card: the symptom, what to watch in the I/O to confirm it, the repair, and what a program should do about it. **Repair** on an active fault removes its cause the way a technician would; whether the machine runs again is up to the program. The reference catches the dead up limit (its watchdog) and flags the blocked eye after 30 s; the dead closing loop and the ticket jam are yours to catch.

Fault	Watch	Repair	The program should
Parking gate: up limit switch dead The boom goes up but its up limit switch never makes: a program that raises until Arm_Up drives the motor against its stop.	Arm_Raise 1, the boom fully up, Arm_Up 0; after 4 s the motor's thermal trips.	Replace the limit switch (Repair).	A motor watchdog: Arm_Raise or Arm_Lower on for longer than a stroke takes (the reference: 4 s) latches a fault and drops the motor before its thermal does.
Parking gate: photo-eye blocked or misaligned The eye across the lane reports blocked all the time.	Gate_Eye 1 with nothing in the lane; the boom never comes down.	Clean and align the eye (Repair).	An eye blocked for longer than any car takes to pass (the reference: 30 s) is an alarm (the fault lamp flashes). The boom stays up: that is the safe side.

Fault	Watch	Repair	The program should
Parking gate: ticket jammed in the dispenser Ticket_Out pulses and no ticket comes out of the slot, so the driver never takes one.	Ticket_Out pulses, Ticket_Taken never comes; the driver presses again and then leans on the horn.	Clear the dispenser (Repair).	A ticket not taken within a few seconds of Ticket_Out is a dispenser alarm; the attendant lets the car in with Raise.
Parking gate: closing loop dead The loop past the boom never reports a car.	A car drives in, Closing_Loop stays 0; the count does not go up; the boom stays up behind it.	Replace the loop detector (Repair).	Cross-check: the eye saw a car pass and the loop did not, or Open_Req has been set for a minute: an alarm, and close on the eye alone once the car has cleared it.

Guided: the parking gate

The scene opens with its guided track: **Init**, **Ticket** (a car presses and gets one), **Boom** (up for it, down behind it), **Count**, **Lamps** and **Ready**, with the reference rungs hints first. The boom is a lesson after the ticket on purpose: the first car sits at the kiosk with its ticket in its hand until you write the rung that lets it in.

The roll-up door

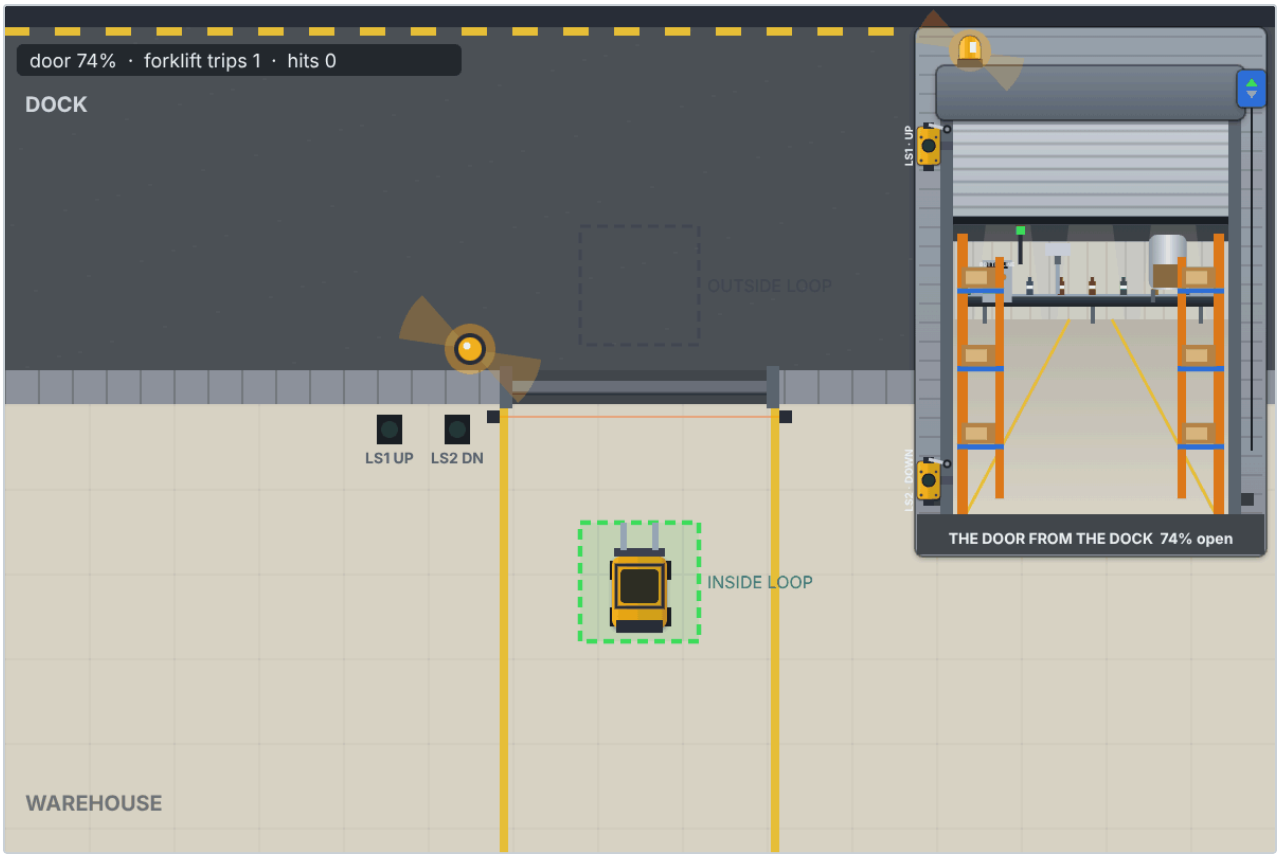


Figure 230. The Roll-up Door part. In plan: the wall with its opening, the dock outside, the warehouse floor inside with the aisle lines through the door, the floor loops either side, the photo-eye across the inside face, the curtain in the opening, the forklift. Top right, the door in elevation from the dock: the curtain coming down out of its hood between the guides, the bottom bar with the safety edge, the operator and its hand chain, the beacon, the eyes at the foot of the guides, and whatever is standing in the doorway.

The machine

The curtain runs on an operator between an up and a down limit, seven seconds for the full height. A forklift drives out through the door every so often, loads on the dock and comes back; it waits on the floor loop on either side until the door is fully open, honks if it waits half a minute, and once it starts through it keeps going. The door coming down onto its mast (or onto a pallet left in the doorway: click the doorway) rests on it with the safety edge pressed and stops there, because a curtain cannot go through a forklift. Kept driving down, that is a hit. Driven against a stop for four seconds, the operator's thermal trips.

Signal		Meaning
`Door_Up`	input	Up limit switch: the door is fully open
`Door_Down`	input	Down limit switch: the door is fully closed
`Eye`	input	Photo-eye across the opening, inside face: 1 while the beam is blocked
`Edge`	input	Safety edge on the curtain's bottom bar: 1 while it is pressed against something

Signal		Meaning
`Inside_Loop`	input	A forklift waits on the floor loop inside the door
`Outside_Loop`	input	A forklift waits on the loop outside the door
`Door_Pos`	output	Door position from the encoder on the operator: 0 shut to 100 fully open (%)
`Motor_Up`	output	Door operator: open
`Motor_Down`	output	Door operator: close
`Beacon`	output	Amber warning beacon over the door

Figure 231. The side view of the door: a section through the wall, looking west, so the forklift drives across the page. The curtain rolled on its drum in the hood (the roll is fatter the more the door is open) and coming down the guide on the inside face to the bottom bar and its safety edge; LS1 · UP and LS2 · DOWN on the guide with their LEDs; the operator with its direction arrows; the beacon on the dock side; the photo-eye low on the jamb; the forklift in profile with its mast; the trailer at the dock; the racking and lights inside; the loops' saw cuts in the slab. Click the doorway to leave a pallet there, click the forklift to send it.

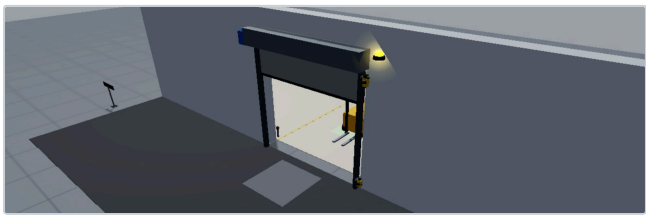


Figure 233. In 3D, from the dock: the curtain up in its hood, the beacon, the forklift coming out.

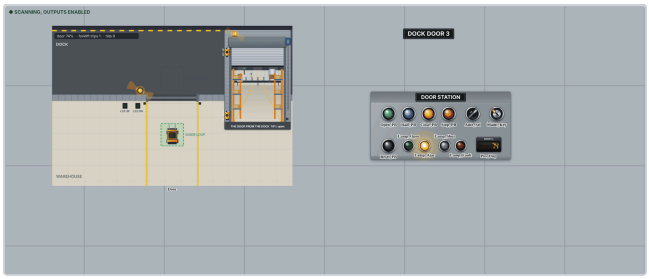
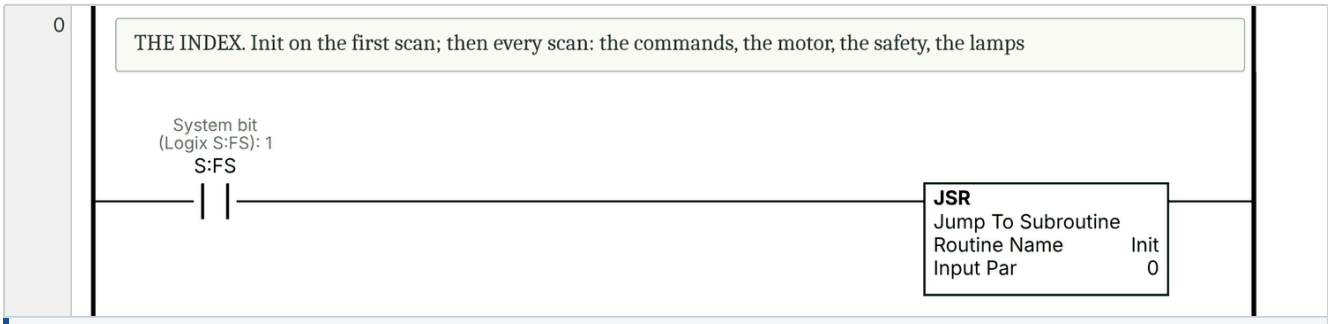


Figure 232. The scene: the door in the warehouse and its station: OPEN, CLOSE, STOP (normally closed), HAND / AUTO CLOSE, Reset, the open and fault lamps.

The program's index

MAINROUTINE · RUNG 0




```
XIC(S:FS)JSR(Init);
```

THE INDEX. Init on the first scan; then every scan: the commands, the motor, the safety, the lamps

HOW THIS RUNG WORKS

In words. When `S:FS` is on, the rung runs the routine `Init`.

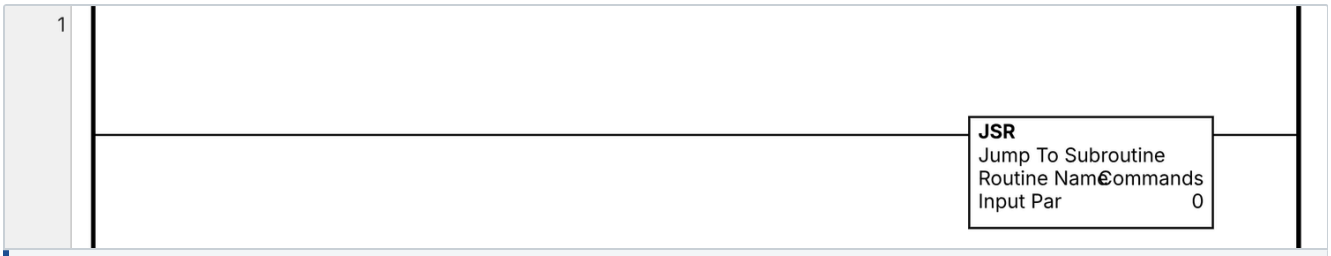
Left to right:

1. `XIC(S:FS)` *Examine If Closed* asks: true when `S:FS` is on.
2. `JSR(Init)` *Jump to Subroutine* does: runs the routine `Init`.

Why it is built this way:

Part of the index: one call, one job.

MAINROUTINE · RUNG 1




```
JSR(Commands);
```

HOW THIS RUNG WORKS

In words. Every scan, with nothing to test, the rung runs the routine `Commands`.

Left to right:

1. `JSR(Commands)` *Jump to Subroutine* does: runs the routine `Commands`.

Why it is built this way:

Part of the index: one call, one job.

MAINROUTINE · RUNG 2

```
JSR(Motor);
```

HOW THIS RUNG WORKS

In words. Every scan, with nothing to test, the rung runs the routine `Motor`.

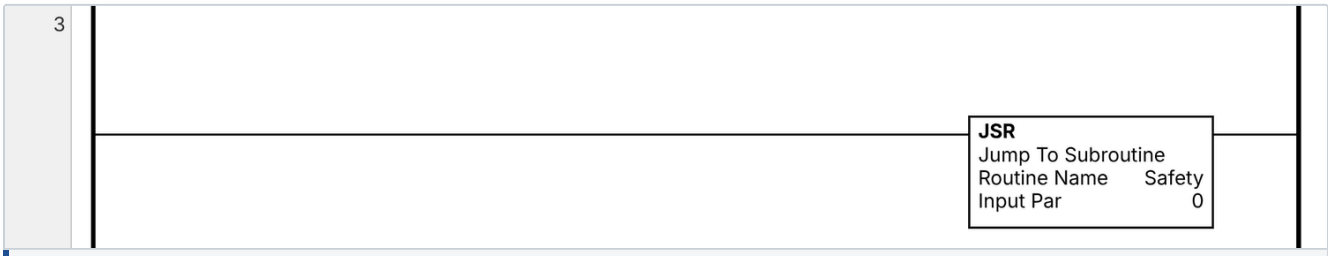
Left to right:

1. `JSR(Motor)` *Jump to Subroutine* does: runs the routine `Motor`.

Why it is built this way:

Part of the index: one call, one job.

MAINROUTINE · RUNG 3




```
JSR(Safety);
```

HOW THIS RUNG WORKS

In words. Every scan, with nothing to test, the rung runs the routine `Safety`.

Left to right:

1. `JSR(Safety)` *Jump to Subroutine* does: runs the routine `Safety`.

Why it is built this way:

Part of the index: one call, one job.

MAINROUTINE · RUNG 4

```
JSR(Lamps);
```

HOW THIS RUNG WORKS

In words. Every scan, with nothing to test, the rung runs the routine `Lamps`.

Left to right:

1. `JSR(Lamps)` *Jump to Subroutine* does: runs the routine `Lamps`.

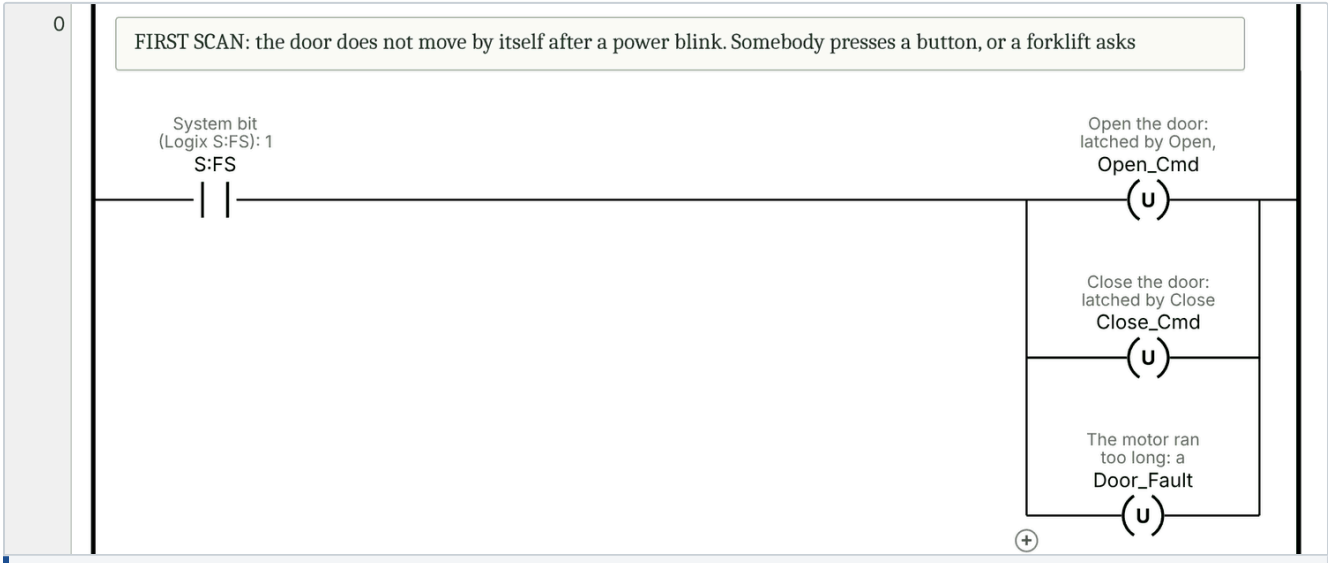
Why it is built this way:

Part of the index: one call, one job.

Tag	Type	Description
`Open_Cmd`	BOOL	Open the door: latched by Open, a forklift on a loop, or a reversal; dropped at the up limit
`Half_Cmd`	BOOL	Go to half open: latched by HALF, dropped inside 48..52 % of Door_Pos
`Close_Cmd`	BOOL	Close the door: latched by Close or the auto-close; dropped at the down limit or by a reversal
`Door_Fault`	BOOL	The motor ran too long: a limit switch, the chain or the motor has failed. Reset clears it

Tag	Type	Description
`T_Auto`	TIMER	Auto-close: 6 s open with the doorway and both loops clear
`T_Run`	TIMER	Motor watchdog: 9 s (a full stroke takes 7 s)
`T_Eye`	TIMER	The eye blocked for 30 s with no forklift waiting: something is standing in the doorway, or the eye has failed
`T_Enc`	TIMER	A limit made with Door_Pos far from it for 1 s: the encoder or the limit has failed
`Up_ONS`	BOOL	One-shot storage for the open count
`Open_Count`	DINT	Times the door has opened
`T_F1`	TIMER	Flasher, half a second on
`T_F2`	TIMER	Flasher, half a second off
`Fl_Bit`	BOOL	The flasher bit

INIT · RUNG 0




```
XIC(S:FS)[OTU(Open_Cmd),OTU(Close_Cmd),OTU(Door_Fault)];
```

FIRST SCAN: the door does not move by itself after a power blink. Somebody presses a button, or a forklift asks

HOW THIS RUNG WORKS

In words. When `S:FS` is on, the rung unlatches `Open_Cmd` (Open the door: latched by Open, a forklift on a loop, or a reversal; dropped at the up limit) (turns it off), unlatches `Close_Cmd` (Close the door: latched by Close or the auto-close; dropped at the down limit or by a reversal) (turns it off) and unlatches `Door_Fault` (The motor ran too long: a limit switch, the chain or the motor has failed. Reset clears it) (turns it off).

Left to right:

1. `XIC(S:FS)` *Examine If Closed* asks: true when `S:FS` is on.
2. A branch of 3 parallel paths: each path is tested on its own and does its own work.

   Path 1

   1. `OTU(Open_Cmd)` *Output Unlatch* does: unlatches `Open_Cmd` (Open the door: latched by Open, a forklift on a loop, or a reversal; dropped at the up limit) (turns it off).

   Path 2

   1. `OTU(Close_Cmd)` *Output Unlatch* does: unlatches `Close_Cmd` (Close the door: latched by Close or the auto-close; dropped at the down limit or by a reversal) (turns it off).

   Path 3

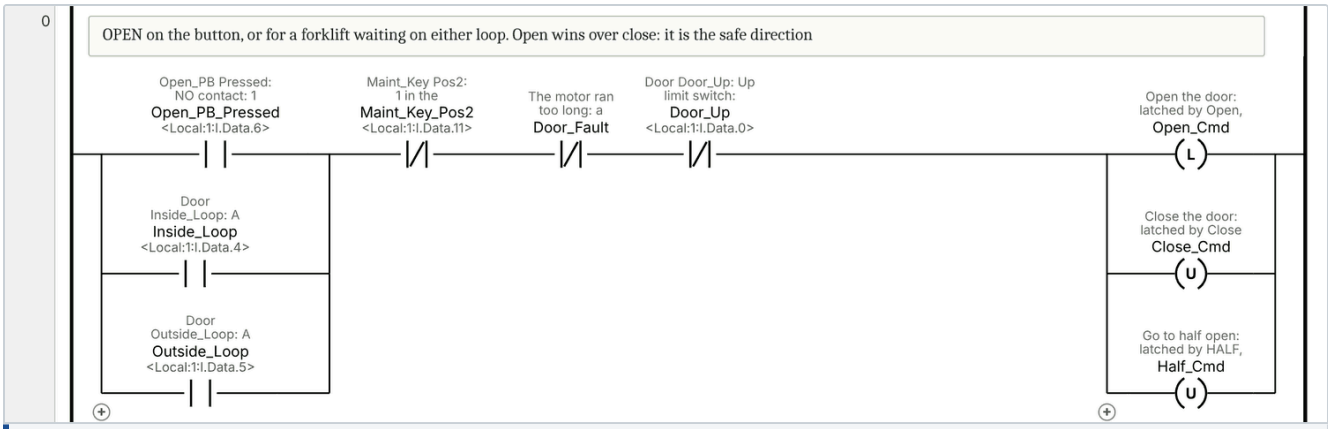
   1. `OTU(Door_Fault)` *Output Unlatch* does: unlatches `Door_Fault` (The motor ran too long: a limit switch, the chain or the motor has failed. Reset clears it) (turns it off).

Exercise 1: the commands

Two latched commands, open and close, each cancelling the other, so the door always knows which way it was last told to go. Open comes from the button or from a forklift on either loop, and it wins: it is the safe direction. Close comes from the button or the auto-close, never into a blocked eye. Stop, wired normally

closed, drops both. The limits end them.

COMMANDS · RUNG 0




```
[XIC(Open_PB_Pressed),XIC(Inside_Loop),XIC(Outside_Loop)]XIO(Maint_Key_Pos2)XIO
(Door_Fault)XIO(Door_Up)[OTL(Open_Cmd),OTU(Close_Cmd),OTU(Half_Cmd)];
```

OPEN on the button, or for a forklift waiting on either loop. Open wins over close: it is the safe direction

PREDICT IT · 12 `Open_PB_Pressed` is on, `Maint_Key_Pos2` is off, `Door_Fault` is off and `Door_Up` is off. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When either (`Open_PB_Pressed` (NO contact: 1 while pressed) is on) *or* (`Inside_Loop` (A forklift waits on the floor loop inside the door) is on) *or* (`Outside_Loop` (A forklift waits on the loop outside the door) is on), `Maint_Key_Pos2` (1 in the right-hand position) is off, `Door_Fault` (The motor ran too long: a limit switch, the chain or the motor has failed. Reset clears it) is off and `Door_Up` (Up limit switch: the door is fully open) is off, the rung latches `Open_Cmd` (Open the door: latched by Open, a forklift on a loop, or a reversal; dropped at the up limit) on, unlatches `Close_Cmd` (Close the door: latched by Close or the auto-close; dropped at the down limit or by a reversal) (turns it off) and unlatches `Half_Cmd` (Go to half open: latched by HALF, dropped inside 48..52 % of Door_Pos) (turns it off).

Left to right:

1. A branch of 3 parallel paths: power gets through if any one of them is true.
   - **Path 1**
     1. `XIC(Open_PB_Pressed)` *Examine If Closed* asks: true when `Open_PB_Pressed` (NO contact: 1 while pressed) is on.
   - **Path 2**
     1. `XIC(Inside_Loop)` *Examine If Closed* asks: true when `Inside_Loop` (A forklift waits on the floor loop inside the door) is on.
   - **Path 3**
     1. `XIC(Outside_Loop)` *Examine If Closed* asks: true when `Outside_Loop` (A forklift waits on the loop outside the door) is on.
2. `XIO(Maint_Key_Pos2)` *Examine If Open* asks: true when `Maint_Key_Pos2` (1 in the right-hand position) is off.
3. `XIO(Door_Fault)` *Examine If Open* asks: true when `Door_Fault` (The motor ran too long: a limit switch, the chain or the motor has failed. Reset clears it) is off.

4. `XIO(Door_Up)` *Examine If Open* asks: true when `Door_Up` (Up limit switch: the door is fully open) is off.
5. A branch of 3 parallel paths: each path is tested on its own and does its own work.

   Path 1

   1. `OTL(Open_Cmd)` *Output Latch* does: latches `Open_Cmd` (Open the door: latched by Open, a forklift on a loop, or a reversal; dropped at the up limit) on.

   Path 2

   1. `OTU(Close_Cmd)` *Output Unlatch* does: unlatches `Close_Cmd` (Close the door: latched by Close or the auto-close; dropped at the down limit or by a reversal) (turns it off).

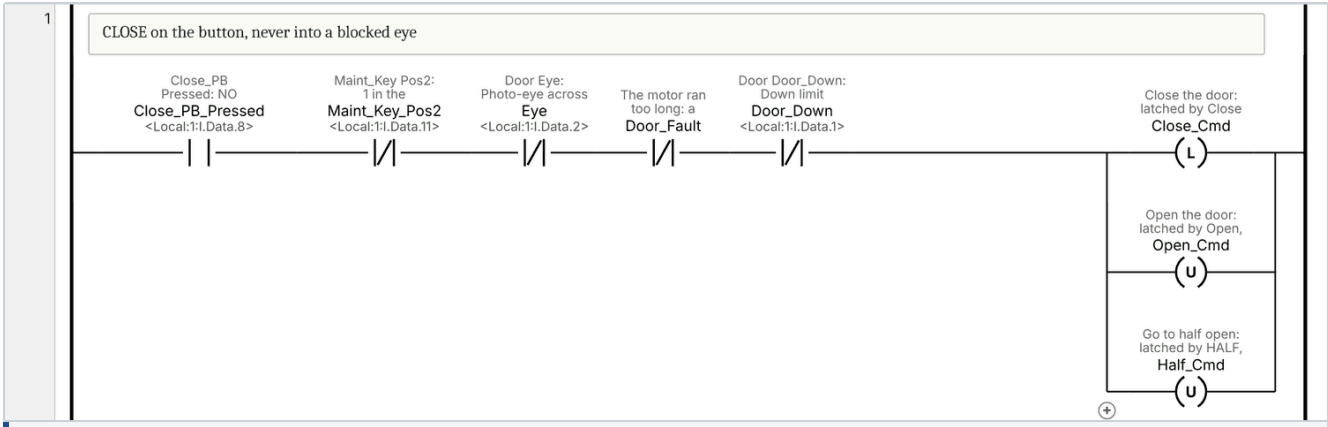
   Path 3

   1. `OTU(Half_Cmd)` *Output Unlatch* does: unlatches `Half_Cmd` (Go to half open: latched by HALF, dropped inside 48..52 % of Door_Pos) (turns it off).

Why it is built this way:

A latch: it stays on until an OTU clears it.

COMMANDS · RUNG 1




```
XIC(Close_PB_Pressed)XIO(Maint_Key_Pos2)XIO(Eye)XIO(Door_Fault)XIO(Door_Down)
[OTL(Close_Cmd),OTU(Open_Cmd),OTU(Half_Cmd)];
```

CLOSE on the button, never into a blocked eye

HOW THIS RUNG WORKS

In words. When `Close_PB_Pressed` (NO contact: 1 while pressed) is on, `Maint_Key_Pos2` (1 in the right-hand position) is off, `Eye` (Photo-eye across the opening, inside face: 1 while the beam is blocked) is off, `Door_Fault` (The motor ran too long: a limit switch, the chain or the motor has failed. Reset clears it) is off and `Door_Down` (Down limit switch: the door is fully closed) is off, the rung latches `Close_Cmd` (Close the door: latched by Close or the auto-close; dropped at the down limit or by a reversal) on, unlatches `Open_Cmd` (Open the door: latched by Open, a forklift on a loop, or a reversal; dropped at the up limit) (turns it off) and unlatches `Half_Cmd` (Go to half open: latched by HALF, dropped inside 48..52 % of Door_Pos) (turns it off).

Left to right:

1. `XIC(Close_PB_Pressed)` *Examine If Closed* asks: true when `Close_PB_Pressed` (NO contact: 1 while pressed) is on.
2. `XIO(Maint_Key_Pos2)` *Examine If Open* asks: true when `Maint_Key_Pos2` (1 in the right-hand position) is off.
3. `XIO(Eye)` *Examine If Open* asks: true when `Eye` (Photo-eye across the opening, inside face: 1 while the beam is blocked) is off.
4. `XIO(Door_Fault)` *Examine If Open* asks: true when `Door_Fault` (The motor ran too long: a limit switch, the chain or the motor has failed. Reset clears it) is off.
5. `XIO(Door_Down)` *Examine If Open* asks: true when `Door_Down` (Down limit switch: the door is fully closed) is off.
6. A branch of 3 parallel paths: each path is tested on its own and does its own work.

   Path 1

   1. `OTL(Close_Cmd)` *Output Latch* does: latches `Close_Cmd` (Close the door: latched by Close or the auto-close; dropped at the down limit or by a reversal) on.

   Path 2

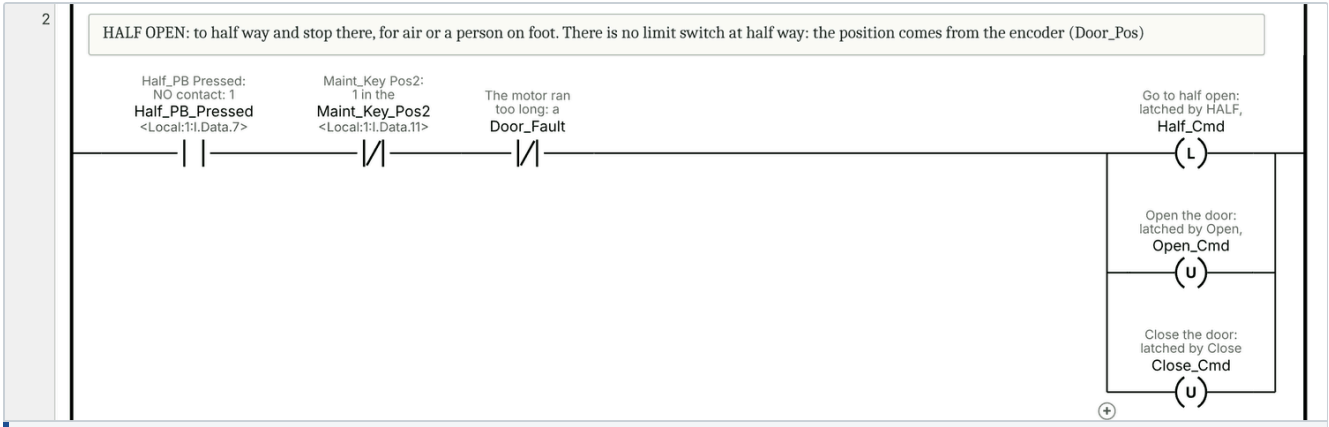
   1. `OTU(Open_Cmd)` *Output Unlatch* does: unlatches `Open_Cmd` (Open the door: latched by Open, a forklift on a loop, or a reversal; dropped at the up limit) (turns it off).

   Path 3

   1. `OTU(Half_Cmd)` *Output Unlatch* does: unlatches `Half_Cmd` (Go to half open: latched by HALF, dropped inside 48..52 % of Door_Pos) (turns it off).

> **Why it is built this way:**
>
> **A latch**: it stays on until an OTU clears it.

COMMANDS · RUNG 2




```
XIC(Half_PB_Pressed)XIO(Maint_Key_Pos2)XIO(Door_Fault)
[OTL(Half_Cmd),OTU(Open_Cmd),OTU(Close_Cmd)];
```

HALF OPEN: to half way and stop there, for air or a person on foot. There is no limit switch at half way: the position comes from the encoder (Door_Pos)

PREDICT IT · 13 `Half_PB_Pressed` is on, `Maint_Key_Pos2` is off and `Door_Fault` is off. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Half_PB_Pressed` (NO contact: 1 while pressed) is on, `Maint_Key_Pos2` (1 in the right-hand position) is off and `Door_Fault` (The motor ran too long: a limit switch, the chain or the motor has failed. Reset clears it) is off, the rung latches `Half_Cmd` (Go to half open: latched by HALF, dropped inside 48..52 % of Door_Pos) on, unlatches `Open_Cmd` (Open the door: latched by Open, a forklift on a loop, or a reversal; dropped at the up limit) (turns it off) and unlatches `Close_Cmd` (Close the door: latched by Close or the auto-close; dropped at the down limit or by a reversal) (turns it off).

Left to right:

1. `XIC(Half_PB_Pressed)` *Examine If Closed* asks: true when `Half_PB_Pressed` (NO contact: 1 while pressed) is on.
2. `XIO(Maint_Key_Pos2)` *Examine If Open* asks: true when `Maint_Key_Pos2` (1 in the right-hand position) is off.
3. `XIO(Door_Fault)` *Examine If Open* asks: true when `Door_Fault` (The motor ran too long: a limit switch, the chain or the motor has failed. Reset clears it) is off.
4. A branch of 3 parallel paths: each path is tested on its own and does its own work.

   Path 1

   1. `OTL(Half_Cmd)` *Output Latch* does: latches `Half_Cmd` (Go to half open: latched by HALF, dropped inside 48..52 % of Door_Pos) on.

   Path 2

   1. `OTU(Open_Cmd)` *Output Unlatch* does: unlatches `Open_Cmd` (Open the door: latched by Open, a forklift on a loop, or a reversal; dropped at the up limit) (turns it off).

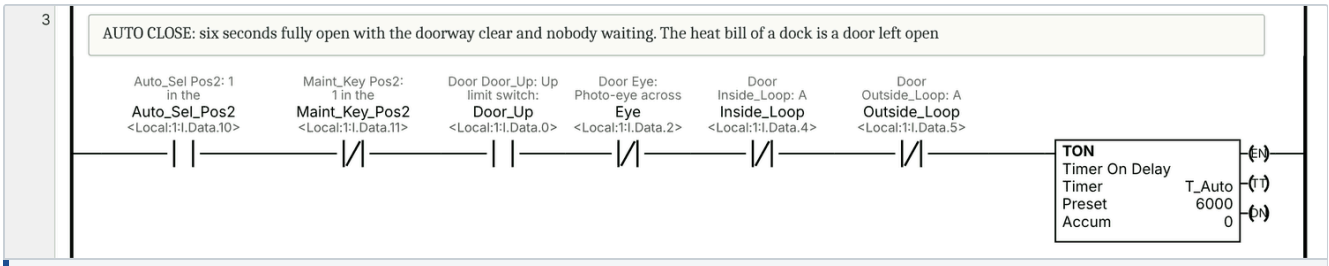
   Path 3

1. `OTU(Close_Cmd)` *Output Unlatch* does: unlatches `Close_Cmd` (Close the door: latched by Close or the auto-close; dropped at the down limit or by a reversal) (turns it off).

Why it is built this way:

A latch: it stays on until an OTU clears it.

COMMANDS · RUNG 3




```
XIC(Auto_Sel_Pos2)XIO(Maint_Key_Pos2)XIC(Door_Up)XIO(Eye)XIO(Inside_Loop)XIO(Outside_Loop)TON(T_Auto,6000,0);
```

AUTO CLOSE: six seconds fully open with the doorway clear and nobody waiting. The heat bill of a dock is a door left open

PREDICT IT · 14 `Auto_Sel_Pos2` is on, `Maint_Key_Pos2` is off, `Door_Up` is off, `Eye` is off, `Inside_Loop` is off and `Outside_Loop` is off. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Auto_Sel_Pos2` (1 in the right-hand position) is on, `Maint_Key_Pos2` (1 in the right-hand position) is off, `Door_Up` (Up limit switch: the door is fully open) is on, `Eye` (Photo-eye across the opening, inside face: 1 while the beam is blocked) is off, `Inside_Loop` (A forklift waits on the floor loop inside the door) is off and `Outside_Loop` (A forklift waits on the loop outside the door) is off, the rung runs the timer `T_Auto` (Auto-close: 6 s open with the doorway and both loops clear) toward 6000 ms (6 s).

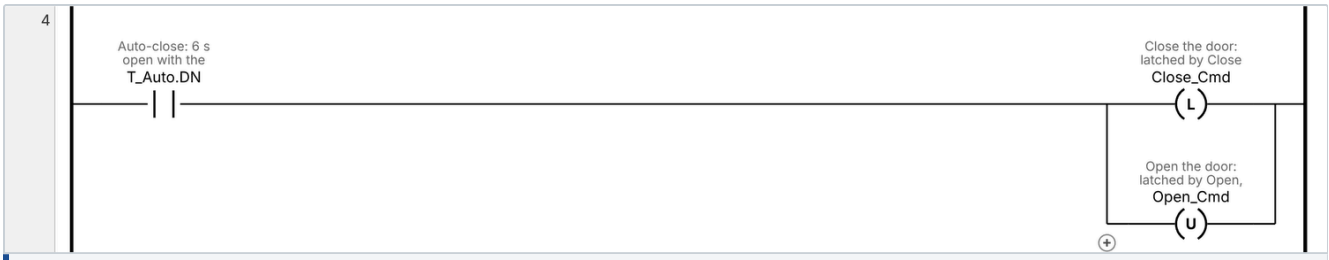
Left to right:

1. `XIC(Auto_Sel_Pos2)` *Examine If Closed* asks: true when `Auto_Sel_Pos2` (1 in the right-hand position) is on.
2. `XIO(Maint_Key_Pos2)` *Examine If Open* asks: true when `Maint_Key_Pos2` (1 in the right-hand position) is off.
3. `XIC(Door_Up)` *Examine If Closed* asks: true when `Door_Up` (Up limit switch: the door is fully open) is on.
4. `XIO(Eye)` *Examine If Open* asks: true when `Eye` (Photo-eye across the opening, inside face: 1 while the beam is blocked) is off.
5. `XIO(Inside_Loop)` *Examine If Open* asks: true when `Inside_Loop` (A forklift waits on the floor loop inside the door) is off.
6. `XIO(Outside_Loop)` *Examine If Open* asks: true when `Outside_Loop` (A forklift waits on the loop outside the door) is off.
7. `TON(T_Auto,6000,0)` *Timer On Delay* does: runs the timer `T_Auto` (Auto-close: 6 s open with the doorway and both loops clear) toward 6000 ms (6 s).

Why it is built this way:

A timer: it times only while this rung is true.

COMMANDS · RUNG 4




```
XIC(T_Auto.DN)[OTL(Close_Cmd),OTU(Open_Cmd)];
```

HOW THIS RUNG WORKS

In words. When `T_Auto.DN` (the done bit of `T_Auto` : Auto-close: 6 s open with the doorway and both loops clear) is on, the rung latches `Close_Cmd` (Close the door: latched by Close or the auto-close; dropped at the down limit or by a reversal) on and unlatches `Open_Cmd` (Open the door: latched by Open, a forklift on a loop, or a reversal; dropped at the up limit) (turns it off).

Left to right:

1. `XIC(T_Auto.DN)` *Examine If Closed* asks: true when `T_Auto.DN` (the done bit of `T_Auto` : Auto-close: 6 s open with the doorway and both loops clear) is on.
2. A branch of 2 parallel paths: each path is tested on its own and does its own work.

   Path 1

   1. `OTL(Close_Cmd)` *Output Latch* does: latches `Close_Cmd` (Close the door: latched by Close or the auto-close; dropped at the down limit or by a reversal) on.

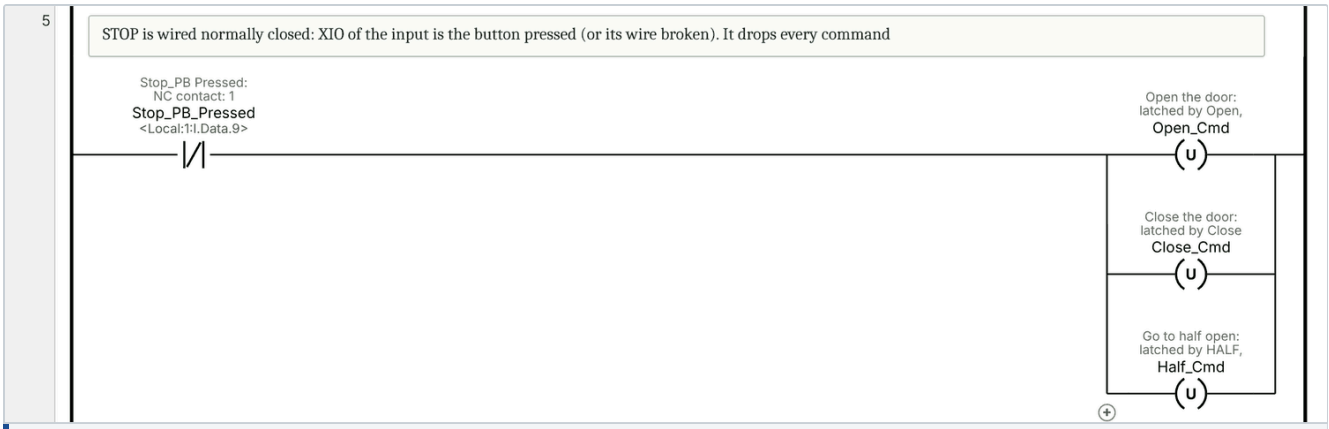
   Path 2

   1. `OTU(Open_Cmd)` *Output Unlatch* does: unlatches `Open_Cmd` (Open the door: latched by Open, a forklift on a loop, or a reversal; dropped at the up limit) (turns it off).

Why it is built this way:

A latch: it stays on until an OTU clears it.

COMMANDS · RUNG 5




```
XIO(Stop_PB_Pressed)[OTU(Open_Cmd),OTU(Close_Cmd),OTU(Half_Cmd)];
```

STOP is wired normally closed: XIO of the input is the button pressed (or its wire broken). It drops every command

HOW THIS RUNG WORKS

In words. When `Stop_PB_Pressed` (NC contact: 1 until pressed) is off, the rung unlatches `Open_Cmd` (Open the door: latched by Open, a forklift on a loop, or a reversal; dropped at the up limit) (turns it off), unlatches `Close_Cmd` (Close the door: latched by Close or the auto-close; dropped at the down limit or by a reversal) (turns it off) and unlatches `Half_Cmd` (Go to half open: latched by HALF, dropped inside 48..52 % of Door_Pos) (turns it off).

Left to right:

1. `XIO(Stop_PB_Pressed)` *Examine If Open* asks: true when `Stop_PB_Pressed` (NC contact: 1 until pressed) is off.
2. A branch of 3 parallel paths: each path is tested on its own and does its own work.

   Path 1

   1. `OTU(Open_Cmd)` *Output Unlatch* does: unlatches `Open_Cmd` (Open the door: latched by Open, a forklift on a loop, or a reversal; dropped at the up limit) (turns it off).

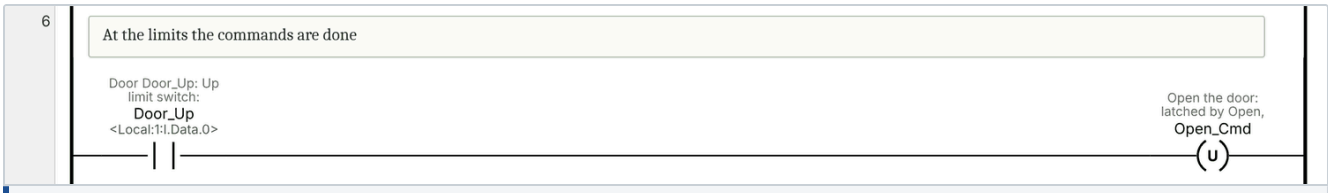
   Path 2

   1. `OTU(Close_Cmd)` *Output Unlatch* does: unlatches `Close_Cmd` (Close the door: latched by Close or the auto-close; dropped at the down limit or by a reversal) (turns it off).

   Path 3

   1. `OTU(Half_Cmd)` *Output Unlatch* does: unlatches `Half_Cmd` (Go to half open: latched by HALF, dropped inside 48..52 % of Door_Pos) (turns it off).

COMMANDS · RUNG 6




```
XIC(Door_Up)OTU(Open_Cmd);
```

At the limits the commands are done

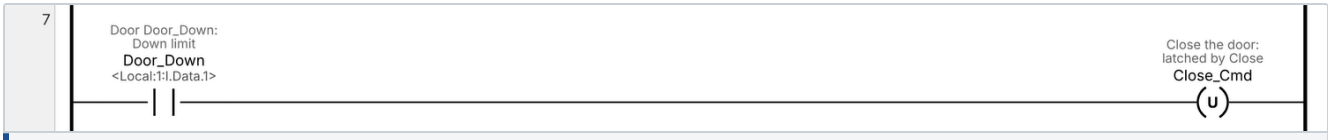
HOW THIS RUNG WORKS

In words. When `Door_Up` (Up limit switch: the door is fully open) is on, the rung unlatches `Open_Cmd` (Open the door: latched by Open, a forklift on a loop, or a reversal; dropped at the up limit) (turns it off).

Left to right:

1. `XIC(Door_Up)` *Examine If Closed* asks: true when `Door_Up` (Up limit switch: the door is fully open) is on.
2. `OTU(Open_Cmd)` *Output Unlatch* does: unlatches `Open_Cmd` (Open the door: latched by Open, a forklift on a loop, or a reversal; dropped at the up limit) (turns it off).

COMMANDS · RUNG 7




```
XIC(Door_Down)OTU(Close_Cmd);
```

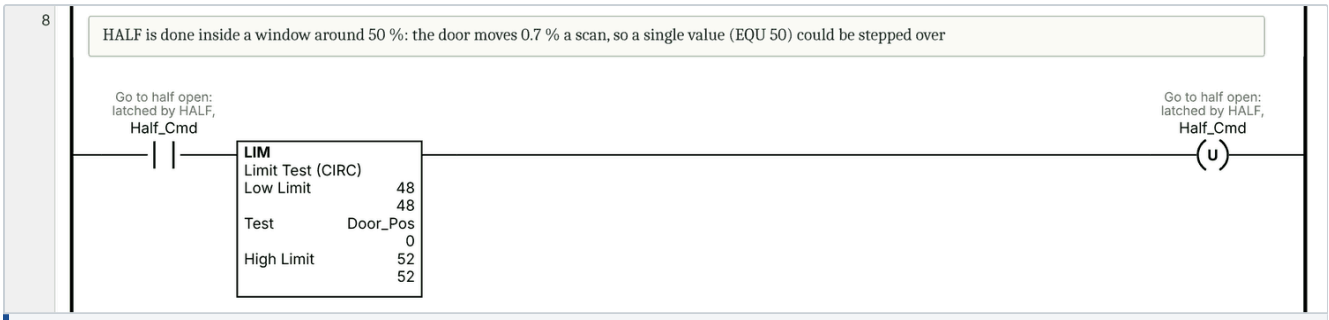
HOW THIS RUNG WORKS

In words. When `Door_Down` (Down limit switch: the door is fully closed) is on, the rung unlatches `Close_Cmd` (Close the door: latched by Close or the auto-close; dropped at the down limit or by a reversal) (turns it off).

Left to right:

1. `XIC(Door_Down)` *Examine If Closed* asks: true when `Door_Down` (Down limit switch: the door is fully closed) is on.
2. `OTU(Close_Cmd)` *Output Unlatch* does: unlatches `Close_Cmd` (Close the door: latched by Close or the auto-close; dropped at the down limit or by a reversal) (turns it off).

COMMANDS · RUNG 8




```
XIC(Half_Cmd)LIM(48,Door_Pos,52)OTU(Half_Cmd);
```

HALF is done inside a window around 50 %: the door moves 0.7 % a scan, so a single value (EQU 50) could be stepped over

PREDICT IT · 15 `Half_Cmd` is on and `Door_Pos` is 50. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Half_Cmd` (Go to half open: latched by HALF, dropped inside 48..52 % of Door_Pos) is on and `Door_Pos` (Door position from the encoder on the operator: 0 shut to 100 fully open (%)) is between **48** and **52**, the rung unlatches `Half_Cmd` (turns it off).

Left to right:

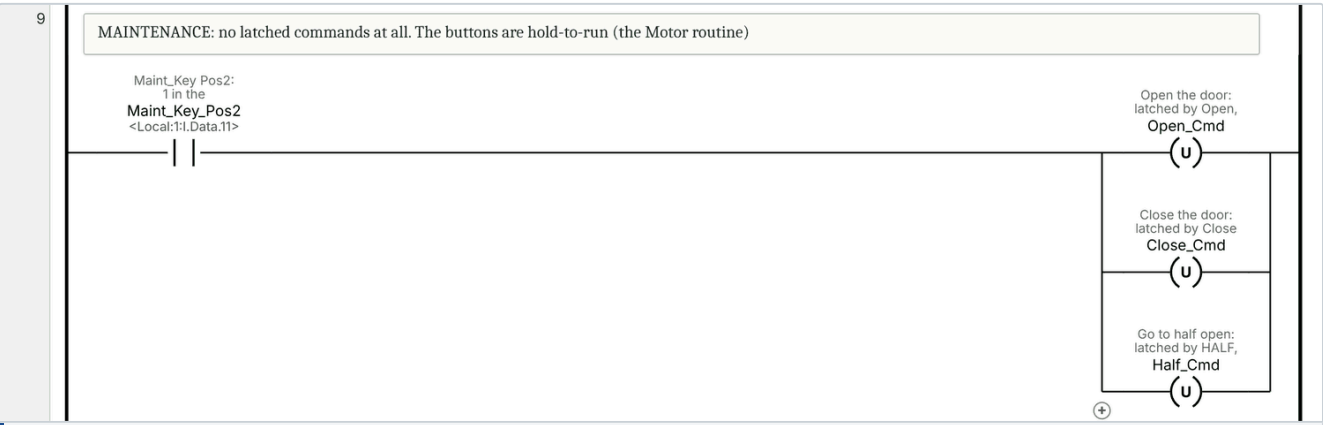
1. `XIC(Half_Cmd)` *Examine If Closed* asks: true when `Half_Cmd` (Go to half open: latched by HALF, dropped inside 48..52 % of Door_Pos) is on.
2. `LIM(48,Door_Pos,52)` *Limit Test* asks: true when `Door_Pos` (Door position from the encoder on the operator: 0 shut to 100 fully open (%)) is between **48** and **52**. The test is inclusive at both ends.
3. `OTU(Half_Cmd)` *Output Unlatch* does: unlatches `Half_Cmd` (turns it off).

Why it is built this way:

A window, not a value. A moving value can step over any single number between two scans; a window catches it on the way through.

> **Think of it like this:** meeting a train. You cannot stand at one exact inch of track and wait for the train to stop there: between two glances it rolls past. You watch the whole platform instead.

COMMANDS · RUNG 9




```
XIC(Maint_Key_Pos2)[OTU(Open_Cmd),OTU(Close_Cmd),OTU(Half_Cmd)];
```

MAINTENANCE: no latched commands at all. The buttons are hold-to-run (the Motor routine)

HOW THIS RUNG WORKS

In words. When `Maint_Key_Pos2` (1 in the right-hand position) is on, the rung unlatches `Open_Cmd` (Open the door: latched by Open, a forklift on a loop, or a reversal; dropped at the up limit) (turns it off), unlatches `Close_Cmd` (Close the door: latched by Close or the auto-close; dropped at the down limit or by a reversal) (turns it off) and unlatches `Half_Cmd` (Go to half open: latched by HALF, dropped inside 48..52 % of Door_Pos) (turns it off).

Left to right:

1. `XIC(Maint_Key_Pos2)` *Examine If Closed* asks: true when `Maint_Key_Pos2` (1 in the right-hand position) is on.
2. A branch of 3 parallel paths: each path is tested on its own and does its own work.

   Path 1

   1. `OTU(Open_Cmd)` *Output Unlatch* does: unlatches `Open_Cmd` (Open the door: latched by Open, a forklift on a loop, or a reversal; dropped at the up limit) (turns it off).

   Path 2

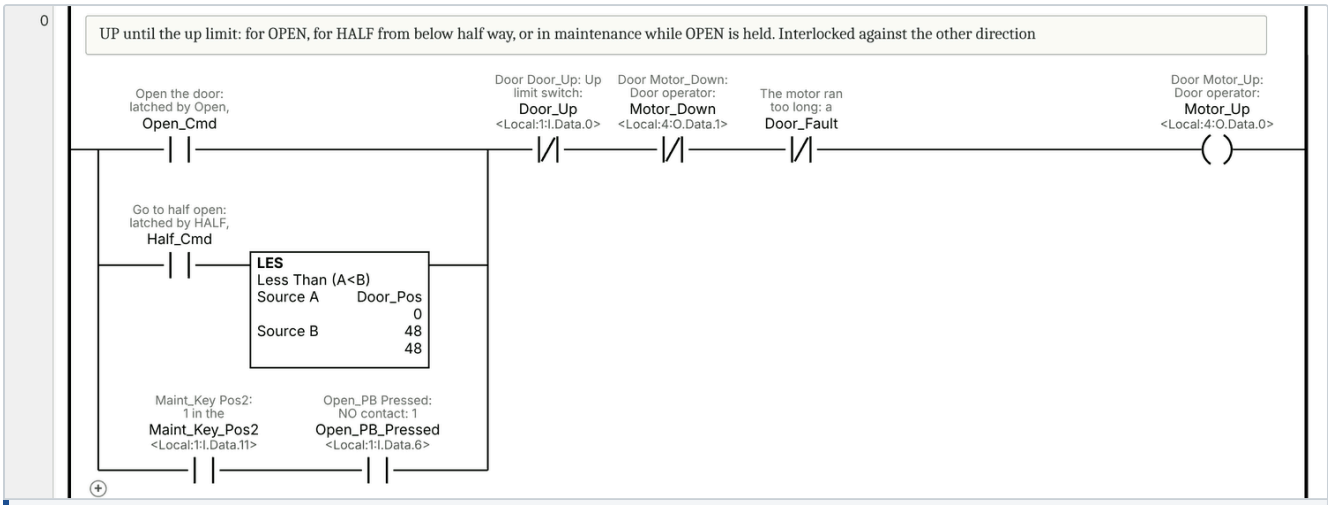
   1. `OTU(Close_Cmd)` *Output Unlatch* does: unlatches `Close_Cmd` (Close the door: latched by Close or the auto-close; dropped at the down limit or by a reversal) (turns it off).

   Path 3

   1. `OTU(Half_Cmd)` *Output Unlatch* does: unlatches `Half_Cmd` (Go to half open: latched by HALF, dropped inside 48..52 % of Door_Pos) (turns it off).

Exercise 2: the motor

MOTOR · RUNG 0




```
[XIC(Open_Cmd),XIC(Half_Cmd)LES(Door_Pos,48),XIC(Maint_Key_Pos2)XIC(Open_PB_Pressed)]XIO(Door_Up)XIO(Motor_Down)XIO(Door_Fault)OTE(Motor_Up);
```

UP until the up limit: for OPEN, for HALF from below half way, or in maintenance while OPEN is held. Interlocked against the other direction

PREDICT IT · 16 `Open_Cmd` is on, `Door_Up` is off, `Motor_Down` is off and `Door_Fault` is off. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When either (`Open_Cmd` (Open the door: latched by Open, a forklift on a loop, or a reversal; dropped at the up limit) is on) *or* (`Half_Cmd` (Go to half open: latched by HALF, dropped inside 48..52 % of Door_Pos) is on and `Door_Pos` (Door position from the encoder on the operator: 0 shut to 100 fully open (%)) is less than **48**) *or* (`Maint_Key_Pos2` (1 in the right-hand position) is on and `Open_PB_Pressed` (NO contact: 1 while pressed) is on), `Door_Up` (Up limit switch: the door is fully open) is off, `Motor_Down` (Door operator: close) is off and `Door_Fault` (The motor ran too long: a limit switch, the chain or the motor has failed. Reset clears it) is off, the rung turns `Motor_Up` (Door operator: open) on (and off again whenever the rung is false).

Left to right:

1. A branch of 3 parallel paths: power gets through if any one of them is true.

   Path 1

   1. `XIC(Open_Cmd)` *Examine If Closed* asks: true when `Open_Cmd` (Open the door: latched by Open, a forklift on a loop, or a reversal; dropped at the up limit) is on.

   Path 2

   1. `XIC(Half_Cmd)` *Examine If Closed* asks: true when `Half_Cmd` (Go to half open: latched by HALF, dropped inside 48..52 % of Door_Pos) is on.
   2. `LES(Door_Pos,48)` *Less Than* asks: true when `Door_Pos` (Door position from the encoder on the operator: 0 shut to 100 fully open (%)) is less than **48**.

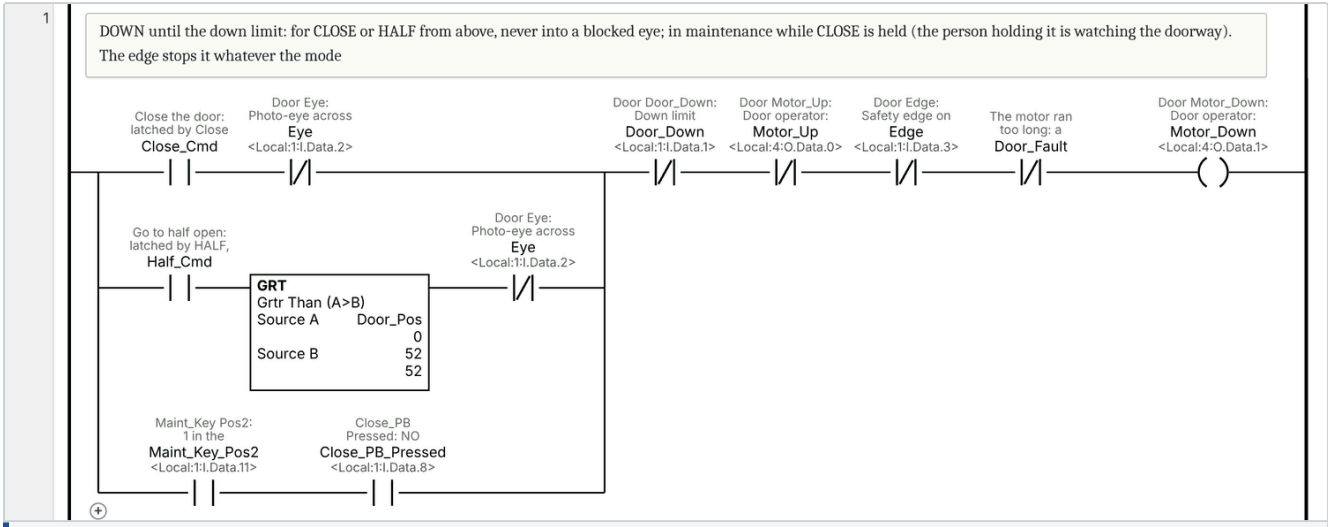
   Path 3

    1. `XIC(Maint_Key_Pos2)` *Examine If Closed* asks: true when `Maint_Key_Pos2` (1 in the right-hand position) is on.
    2. `XIC(Open_PB_Pressed)` *Examine If Closed* asks: true when `Open_PB_Pressed` (NO contact: 1 while pressed) is on.
2. `XIO(Door_Up)` *Examine If Open* asks: true when `Door_Up` (Up limit switch: the door is fully open) is off.
3. `XIO(Motor_Down)` *Examine If Open* asks: true when `Motor_Down` (Door operator: close) is off.
4. `XIO(Door_Fault)` *Examine If Open* asks: true when `Door_Fault` (The motor ran too long: a limit switch, the chain or the motor has failed. Reset clears it) is off.
5. `OTE(Motor_Up)` *Output Energize* does: turns `Motor_Up` (Door operator: open) on (and off again whenever the rung is false).

Why it is built this way:

Interlock: one of the pair, never both.

MOTOR · RUNG 1




```
[XIC(Close_Cmd)XIO(Eye),XIC(Half_Cmd)GRT(Door_Pos,52)XIO(Eye),XIC(Maint_Key_Pos2)XIC(Close_PB_Pressed)]XIO(Door_Down)XIO(Motor_Up)XIO(Edge)XIO(Door_Fault)OTE(Motor_Down);
```

DOWN until the down limit: for CLOSE or HALF from above, never into a blocked eye; in maintenance while CLOSE is held (the person holding it is watching the doorway). The edge stops it whatever the mode

PREDICT IT · 17 `Close_Cmd` is on, `Eye` is off, `Door_Down` is on, `Motor_Up` is off, `Edge` is off and `Door_Fault` is off. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When either (`Close_Cmd` (Close the door: latched by Close or the auto-close; dropped at the down limit or by a reversal) is on and `Eye` (Photo-eye across the opening, inside face: 1 while the beam is blocked) is off) *or* (`Half_Cmd` (Go to half open: latched by HALF, dropped inside 48..52 % of Door_Pos) is on, `Door_Pos` (Door position from the encoder on the operator: 0 shut to 100 fully open (%)) is greater than **52** and `Eye` is off) *or* (`Maint_Key_Pos2` (1 in the right-hand position) is on and `Close_PB_Pressed` (NO contact: 1 while pressed) is on), `Door_Down` (Down limit switch: the door is fully closed) is off, `Motor_Up` (Door operator: open) is off, `Edge` (Safety edge on the curtain's bottom bar: 1 while it is pressed against something) is off and `Door_Fault` (The motor ran too long: a limit switch, the chain or the motor has failed. Reset clears it) is off, the rung turns `Motor_Down` (Door operator: close) on (and off again whenever the rung is false).

Left to right:

1. A branch of 3 parallel paths: power gets through if any one of them is true.

   Path 1

   1. `XIC(Close_Cmd)` *Examine If Closed* asks: true when `Close_Cmd` (Close the door: latched by Close or the auto-close; dropped at the down limit or by a reversal) is on.
   2. `XIO(Eye)` *Examine If Open* asks: true when `Eye` (Photo-eye across the opening, inside face: 1 while the beam is blocked) is off.

   Path 2

1. `XIC(Half_Cmd)` *Examine If Closed* asks: true when `Half_Cmd` (Go to half open: latched by HALF, dropped inside 48..52 % of Door_Pos) is on.
2. `GRT(Door_Pos,52)` *Greater Than* asks: true when `Door_Pos` (Door position from the encoder on the operator: 0 shut to 100 fully open (%)) is greater than **52**.
3. `XIO(Eye)` *Examine If Open* asks: true when `Eye` is off.

Path 3

1. `XIC(Maint_Key_Pos2)` *Examine If Closed* asks: true when `Maint_Key_Pos2` (1 in the right-hand position) is on.
2. `XIC(Close_PB_Pressed)` *Examine If Closed* asks: true when `Close_PB_Pressed` (NO contact: 1 while pressed) is on.

2. `XIO(Door_Down)` *Examine If Open* asks: true when `Door_Down` (Down limit switch: the door is fully closed) is off.
3. `XIO(Motor_Up)` *Examine If Open* asks: true when `Motor_Up` (Door operator: open) is off.
4. `XIO(Edge)` *Examine If Open* asks: true when `Edge` (Safety edge on the curtain's bottom bar: 1 while it is pressed against something) is off.
5. `XIO(Door_Fault)` *Examine If Open* asks: true when `Door_Fault` (The motor ran too long: a limit switch, the chain or the motor has failed. Reset clears it) is off.
6. `OTE(Motor_Down)` *Output Energize* does: turns `Motor_Down` (Door operator: close) on (and off again whenever the rung is false).

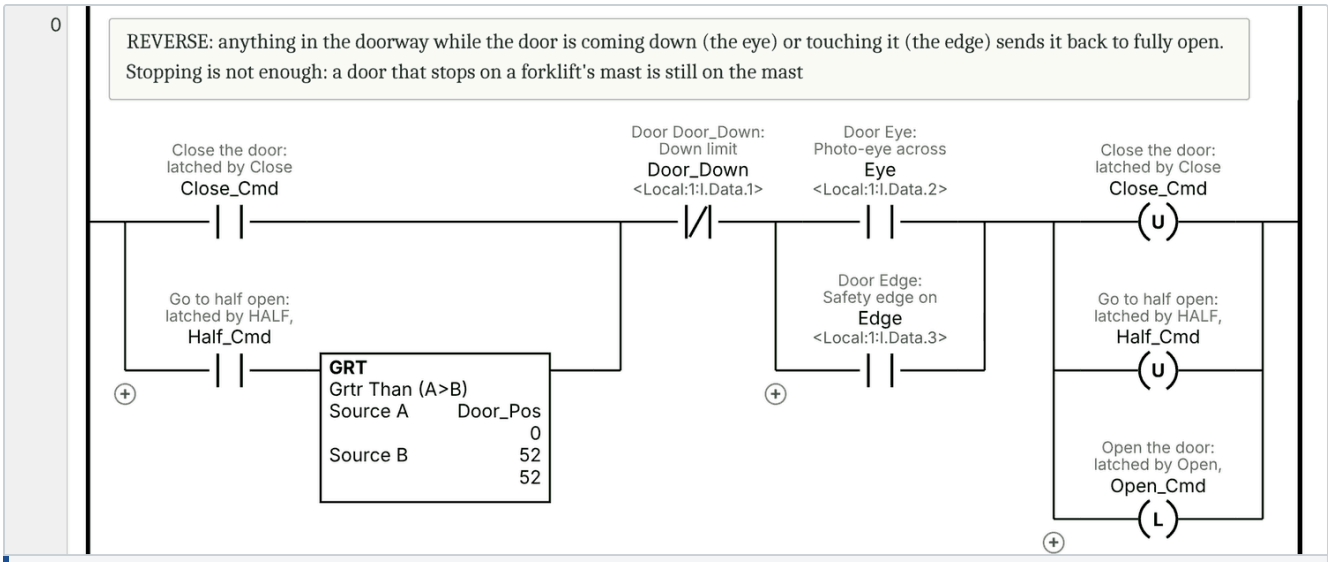
Why it is built this way:

Interlock: one of the pair, never both.

Exercise 3: reverse, and the watchdog

This is the routine the door is for. Anything in the doorway while the door is coming down sends it back to fully open: the eye sees it before the curtain gets there, the edge feels it if it does. Not stop: reverse. A door that stops on a forklift's mast is still on the mast, and the driver cannot back out from under it.

SAFETY · RUNG 0




```
[XIC(Close_Cmd),XIC(Half_Cmd)GRT(Door_Pos,52)]XIO(Door_Down)
[XIC(Eye),XIC(Edge)][OTU(Close_Cmd),OTU(Half_Cmd),OTL(Open_Cmd)];
```

REVERSE: anything in the doorway while the door is coming down (the eye) or touching it (the edge) sends it back to fully open. Stopping is not enough: a door that stops on a forklift's mast is still on the mast

PREDICT IT · 18 `Close_Cmd` is on, `Door_Down` is off and `Eye` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When either (`Close_Cmd` (Close the door: latched by Close or the auto-close; dropped at the down limit or by a reversal) is on) *or* (`Half_Cmd` (Go to half open: latched by HALF, dropped inside 48..52 % of Door_Pos) is on and `Door_Pos` (Door position from the encoder on the operator: 0 shut to 100 fully open (%)) is greater than **52**), `Door_Down` (Down limit switch: the door is fully closed) is off and either (`Eye` (Photo-eye across the opening, inside face: 1 while the beam is blocked) is on) *or* (`Edge` (Safety edge on the curtain's bottom bar: 1 while it is pressed against something) is on), the rung unlatches `Close_Cmd` (turns it off), unlatches `Half_Cmd` (turns it off) and latches `Open_Cmd` (Open the door: latched by Open, a forklift on a loop, or a reversal; dropped at the up limit) on.

Left to right:

1. A branch of 2 parallel paths: power gets through if any one of them is true.

   Path 1

   1. `XIC(Close_Cmd)` *Examine If Closed* asks: true when `Close_Cmd` (Close the door: latched by Close or the auto-close; dropped at the down limit or by a reversal) is on.

   Path 2

   1. `XIC(Half_Cmd)` *Examine If Closed* asks: true when `Half_Cmd` (Go to half open: latched by HALF, dropped inside 48..52 % of Door_Pos) is on.
   2. `GRT(Door_Pos,52)` *Greater Than* asks: true when `Door_Pos` (Door position from the encoder on the operator: 0 shut to 100 fully open (%)) is greater than **52**.

2. `XIO(Door_Down)` *Examine If Open* asks: true when `Door_Down` (Down limit switch: the door is fully closed) is off.
3. A branch of 2 parallel paths: power gets through if any one of them is true.

   Path 1

   1. `XIC(Eye)` *Examine If Closed* asks: true when `Eye` (Photo-eye across the opening, inside face: 1 while the beam is blocked) is on.

   Path 2

   1. `XIC(Edge)` *Examine If Closed* asks: true when `Edge` (Safety edge on the curtain's bottom bar: 1 while it is pressed against something) is on.
4. A branch of 3 parallel paths: each path is tested on its own and does its own work.

   Path 1

   1. `OTU(Close_Cmd)` *Output Unlatch* does: unlatches `Close_Cmd` (turns it off).

   Path 2

   1. `OTU(Half_Cmd)` *Output Unlatch* does: unlatches `Half_Cmd` (turns it off).

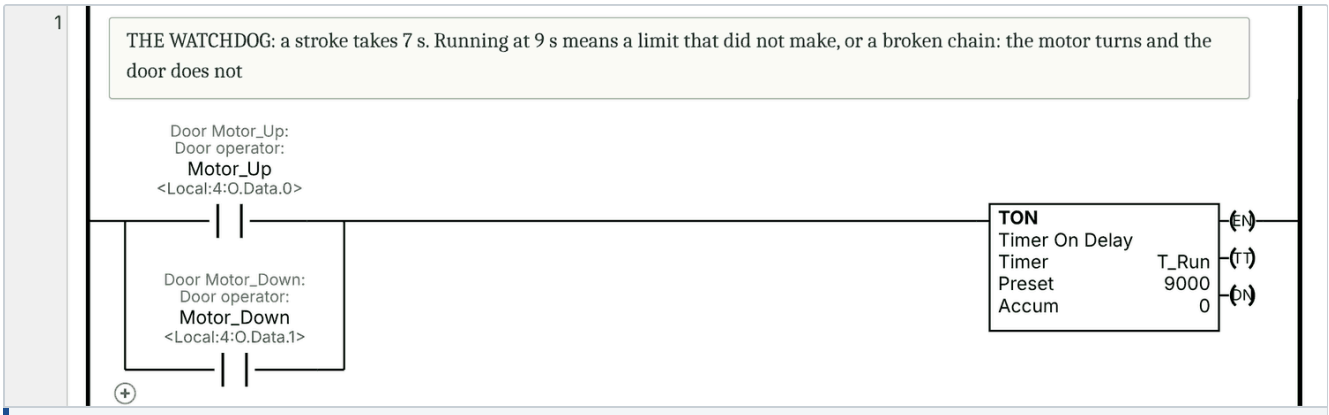
   Path 3

   1. `OTL(Open_Cmd)` *Output Latch* does: latches `Open_Cmd` (Open the door: latched by Open, a forklift on a loop, or a reversal; dropped at the up limit) on.

Why it is built this way:

A latch: it stays on until an OTU clears it.

SAFETY · RUNG 1




```
[XIC(Motor_Up),XIC(Motor_Down)]TON(T_Run,9000,0);
```

THE WATCHDOG: a stroke takes 7 s. Running at 9 s means a limit that did not make, or a broken chain: the motor turns and the door does not

PREDICT IT · 19 `Motor_Up` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When either (`Motor_Up` (Door operator: open) is on) *or* (`Motor_Down` (Door operator: close) is on), the rung runs the timer `T_Run` (Motor watchdog: 9 s (a full stroke takes 7 s)) toward 9000 ms (9 s).

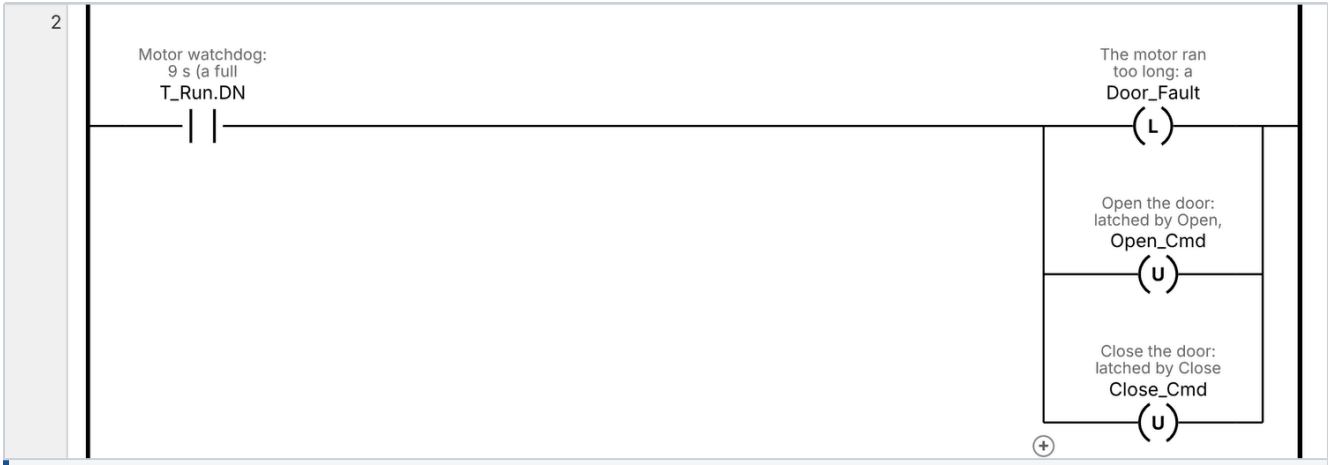
Left to right:

1. A branch of 2 parallel paths: power gets through if any one of them is true.
   - **Path 1**
     1. `XIC(Motor_Up)` *Examine If Closed* asks: true when `Motor_Up` (Door operator: open) is on.
   - **Path 2**
     1. `XIC(Motor_Down)` *Examine If Closed* asks: true when `Motor_Down` (Door operator: close) is on.
2. `TON(T_Run,9000,0)` *Timer On Delay* does: runs the timer `T_Run` (Motor watchdog: 9 s (a full stroke takes 7 s)) toward 9000 ms (9 s).

Why it is built this way:

A timer: it times only while this rung is true.

SAFETY · RUNG 2




```
XIC(T_Run.DN)[OTL(Door_Fault),OTU(Open_Cmd),OTU(Close_Cmd)];
```

HOW THIS RUNG WORKS

In words. When `T_Run.DN` (the done bit of `T_Run` : Motor watchdog: 9 s (a full stroke takes 7 s)) is on, the rung latches `Door_Fault` (The motor ran too long: a limit switch, the chain or the motor has failed. Reset clears it) on, unlatches `Open_Cmd` (Open the door: latched by Open, a forklift on a loop, or a reversal; dropped at the up limit) (turns it off) and unlatches `Close_Cmd` (Close the door: latched by Close or the auto-close; dropped at the down limit or by a reversal) (turns it off).

Left to right:

1. `XIC(T_Run.DN)` *Examine If Closed* asks: true when `T_Run.DN` (the done bit of `T_Run` : Motor watchdog: 9 s (a full stroke takes 7 s)) is on.
2. A branch of 3 parallel paths: each path is tested on its own and does its own work.

   Path 1

   1. `OTL(Door_Fault)` *Output Latch* does: latches `Door_Fault` (The motor ran too long: a limit switch, the chain or the motor has failed. Reset clears it) on.

   Path 2

   1. `OTU(Open_Cmd)` *Output Unlatch* does: unlatches `Open_Cmd` (Open the door: latched by Open, a forklift on a loop, or a reversal; dropped at the up limit) (turns it off).

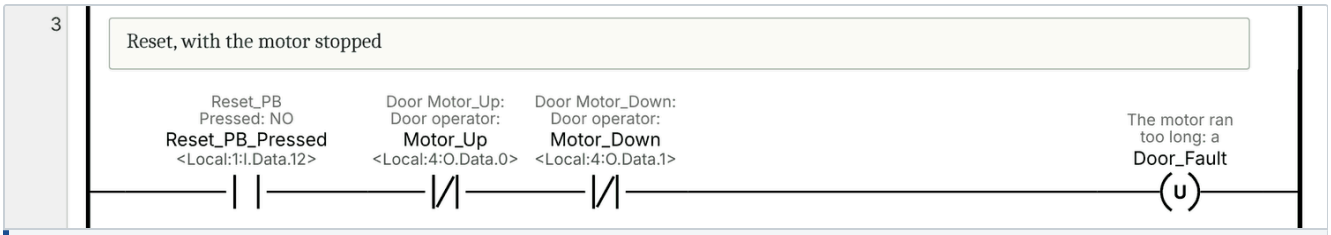
   Path 3

   1. `OTU(Close_Cmd)` *Output Unlatch* does: unlatches `Close_Cmd` (Close the door: latched by Close or the auto-close; dropped at the down limit or by a reversal) (turns it off).

Why it is built this way:

A latch: it stays on until an OTU clears it.

SAFETY · RUNG 3




```
XIC(Reset_PB_Pressed)XIO(Motor_Up)XIO(Motor_Down)OTU(Door_Fault);
```

Reset, with the motor stopped

PREDICT IT · 20 `Reset_PB_Pressed` is on, `Motor_Up` is off and `Motor_Down` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

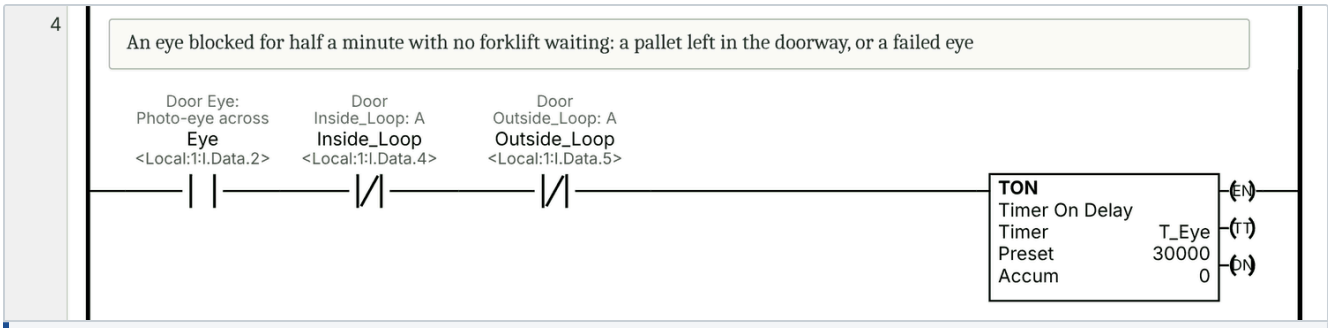
HOW THIS RUNG WORKS

In words. When `Reset_PB_Pressed` (NO contact: 1 while pressed) is on, `Motor_Up` (Door operator: open) is off and `Motor_Down` (Door operator: close) is off, the rung unlatches `Door_Fault` (The motor ran too long: a limit switch, the chain or the motor has failed. Reset clears it) (turns it off).

Left to right:

1. `XIC(Reset_PB_Pressed)` *Examine If Closed* asks: true when `Reset_PB_Pressed` (NO contact: 1 while pressed) is on.
2. `XIO(Motor_Up)` *Examine If Open* asks: true when `Motor_Up` (Door operator: open) is off.
3. `XIO(Motor_Down)` *Examine If Open* asks: true when `Motor_Down` (Door operator: close) is off.
4. `OTU(Door_Fault)` *Output Unlatch* does: unlatches `Door_Fault` (The motor ran too long: a limit switch, the chain or the motor has failed. Reset clears it) (turns it off).

SAFETY · RUNG 4




```
XIC(Eye)XIO(Inside_Loop)XIO(Outside_Loop)TON(T_Eye,30000,0);
```

An eye blocked for half a minute with no forklift waiting: a pallet left in the doorway, or a failed eye

PREDICT IT · 21 `Eye` is on, `Inside_Loop` is off and `Outside_Loop` is off. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Eye` (Photo-eye across the opening, inside face: 1 while the beam is blocked) is on, `Inside_Loop` (A forklift waits on the floor loop inside the door) is off and `Outside_Loop` (A forklift waits on the loop outside the door) is off, the rung runs the timer `T_Eye` (The eye blocked for 30 s with no forklift waiting: something is standing in the doorway, or the eye has failed) toward 30000 ms (30 s).

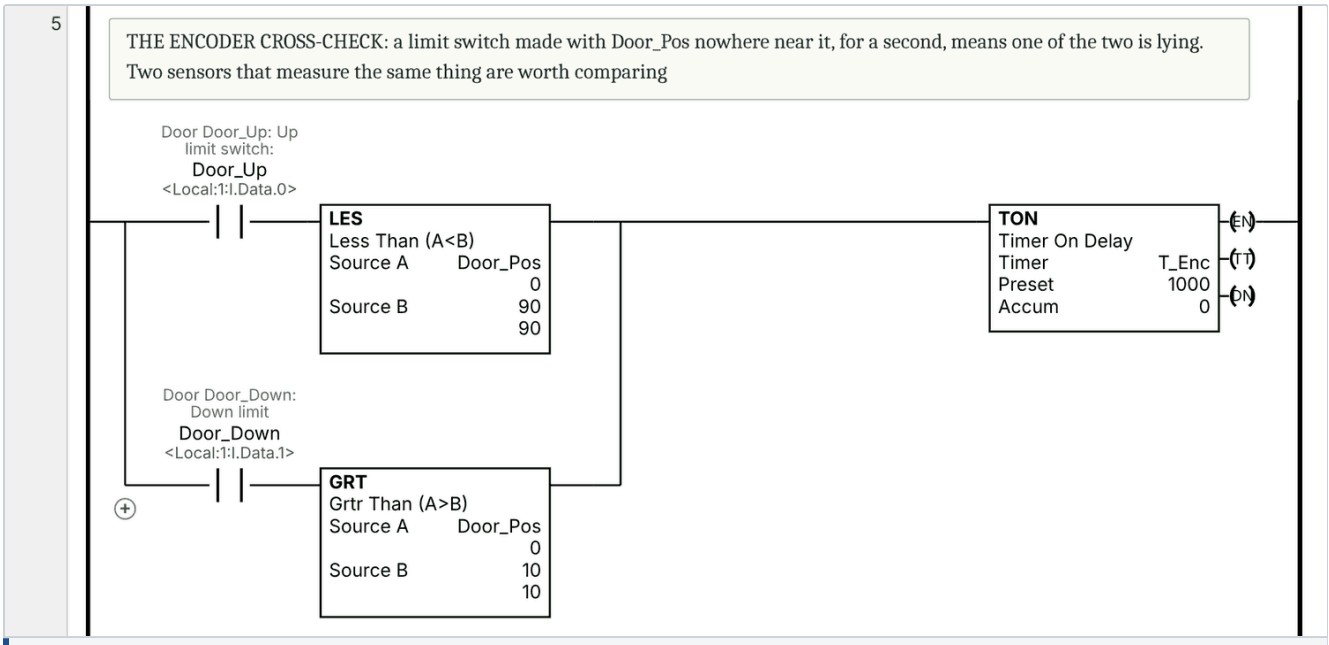
Left to right:

1. `XIC(Eye)` *Examine If Closed* asks: true when `Eye` (Photo-eye across the opening, inside face: 1 while the beam is blocked) is on.
2. `XIO(Inside_Loop)` *Examine If Open* asks: true when `Inside_Loop` (A forklift waits on the floor loop inside the door) is off.
3. `XIO(Outside_Loop)` *Examine If Open* asks: true when `Outside_Loop` (A forklift waits on the loop outside the door) is off.
4. `TON(T_Eye,30000,0)` *Timer On Delay* does: runs the timer `T_Eye` (The eye blocked for 30 s with no forklift waiting: something is standing in the doorway, or the eye has failed) toward 30000 ms (30 s).

Why it is built this way:

A timer: it times only while this rung is true.

SAFETY · RUNG 5




```
[XIC(Door_Up)LES(Door_Pos,90),XIC(Door_Down)GRT(Door_Pos,10)]TON(T_Enc,1000,0);
```

THE ENCODER CROSS-CHECK: a limit switch made with Door_Pos nowhere near it, for a second, means one of the two is lying. Two sensors that measure the same thing are worth comparing

PREDICT IT · 22 `Door_Up` is on and `Door_Pos` is 89. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When either (`Door_Up` (Up limit switch: the door is fully open) is on and `Door_Pos` (Door position from the encoder on the operator: 0 shut to 100 fully open (%)) is less than **90**) *or* (`Door_Down` (Down limit switch: the door is fully closed) is on and `Door_Pos` is greater than **10**), the rung runs the timer `T_Enc` (A limit made with Door_Pos far from it for 1 s: the encoder or the limit has failed) toward 1000 ms (1 s).

Left to right:

1. A branch of 2 parallel paths: power gets through if any one of them is true.

   Path 1

   1. `XIC(Door_Up)` *Examine If Closed* asks: true when `Door_Up` (Up limit switch: the door is fully open) is on.
   2. `LES(Door_Pos,90)` *Less Than* asks: true when `Door_Pos` (Door position from the encoder on the operator: 0 shut to 100 fully open (%)) is less than **90**.

   Path 2

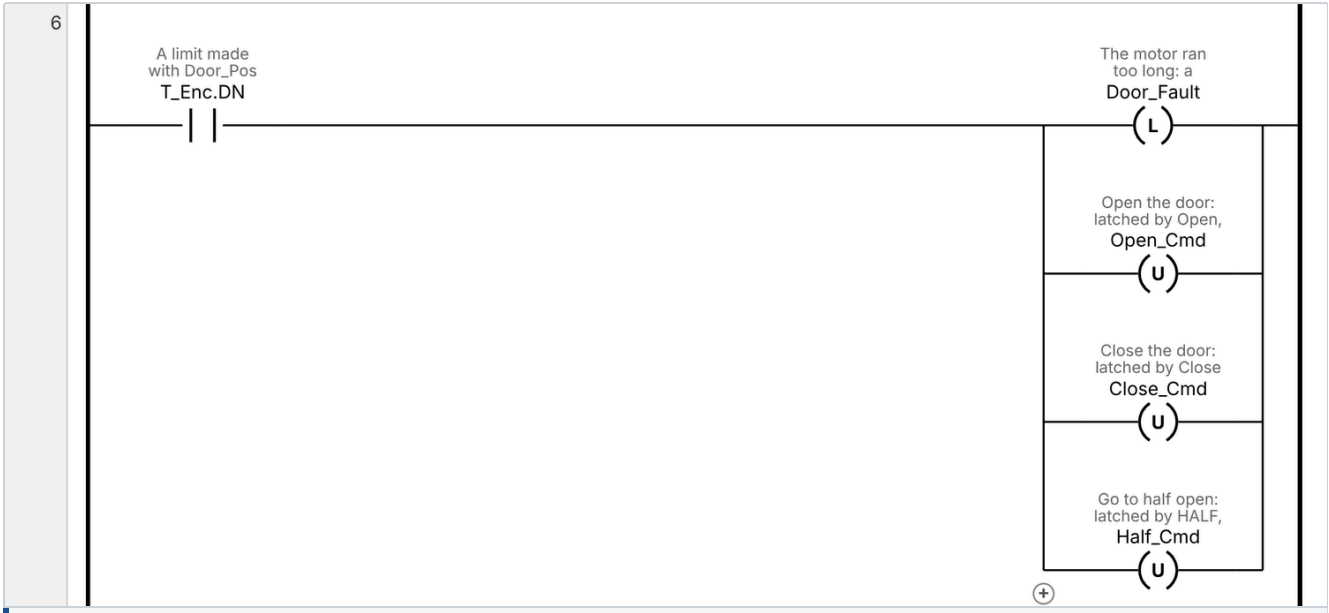
   1. `XIC(Door_Down)` *Examine If Closed* asks: true when `Door_Down` (Down limit switch: the door is fully closed) is on.
   2. `GRT(Door_Pos,10)` *Greater Than* asks: true when `Door_Pos` is greater than **10**.

2. `TON(T_Enc,1000,0)` *Timer On Delay* does: runs the timer `T_Enc` (A limit made with Door_Pos far from it for 1 s: the encoder or the limit has failed) toward 1000 ms (1 s).

Why it is built this way:

> **A timer**: it times only while this rung is true.

SAFETY · RUNG 6




```
XIC(T_Enc.DN)[OTL(Door_Fault),OTU(Open_Cmd),OTU(Close_Cmd),OTU(Half_Cmd)];
```

HOW THIS RUNG WORKS

In words. When `T_Enc.DN` (the done bit of `T_Enc` : A limit made with Door_Pos far from it for 1 s: the encoder or the limit has failed) is on, the rung latches `Door_Fault` (The motor ran too long: a limit switch, the chain or the motor has failed. Reset clears it) on, unlatches `Open_Cmd` (Open the door: latched by Open, a forklift on a loop, or a reversal; dropped at the up limit) (turns it off), unlatches `Close_Cmd` (Close the door: latched by Close or the auto-close; dropped at the down limit or by a reversal) (turns it off) and unlatches `Half_Cmd` (Go to half open: latched by HALF, dropped inside 48..52 % of Door_Pos) (turns it off).

Left to right:

1. `XIC(T_Enc.DN)` *Examine If Closed* asks: true when `T_Enc.DN` (the done bit of `T_Enc` : A limit made with Door_Pos far from it for 1 s: the encoder or the limit has failed) is on.
2. A branch of 4 parallel paths: each path is tested on its own and does its own work.

   Path 1

   1. `OTL(Door_Fault)` *Output Latch* does: latches `Door_Fault` (The motor ran too long: a limit switch, the chain or the motor has failed. Reset clears it) on.

   Path 2

   1. `OTU(Open_Cmd)` *Output Unlatch* does: unlatches `Open_Cmd` (Open the door: latched by Open, a forklift on a loop, or a reversal; dropped at the up limit) (turns it off).

   Path 3

   1. `OTU(Close_Cmd)` *Output Unlatch* does: unlatches `Close_Cmd` (Close the door: latched by Close or the auto-close; dropped at the down limit or by a reversal) (turns it off).

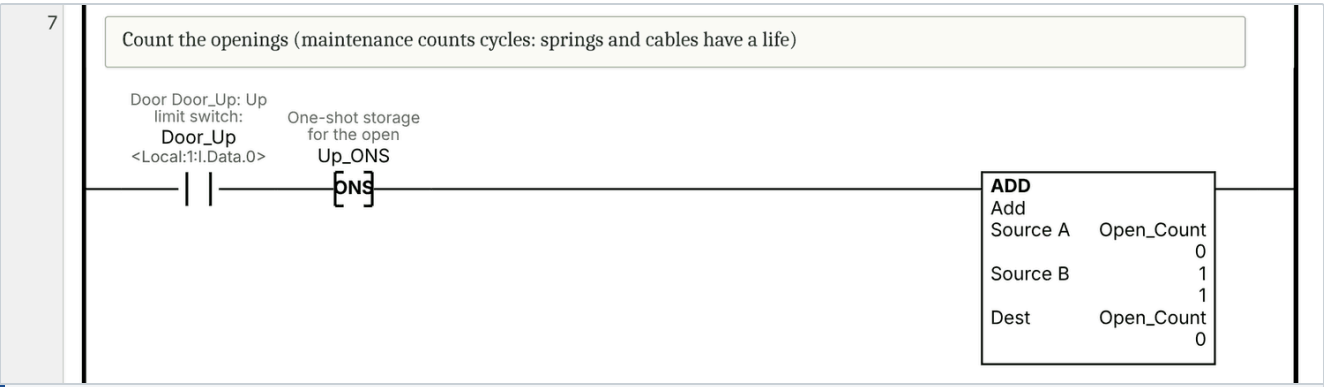
   Path 4

   1. `OTU(Half_Cmd)` *Output Unlatch* does: unlatches `Half_Cmd` (Go to half open: latched by HALF, dropped inside 48..52 % of Door_Pos) (turns it off).

Why it is built this way:

A latch: it stays on until an OTU clears it.

SAFETY · RUNG 7




```
XIC(Door_Up)ONS(Up_ONS)ADD(Open_Count,1,Open_Count);
```

Count the openings (maintenance counts cycles: springs and cables have a life)

HOW THIS RUNG WORKS

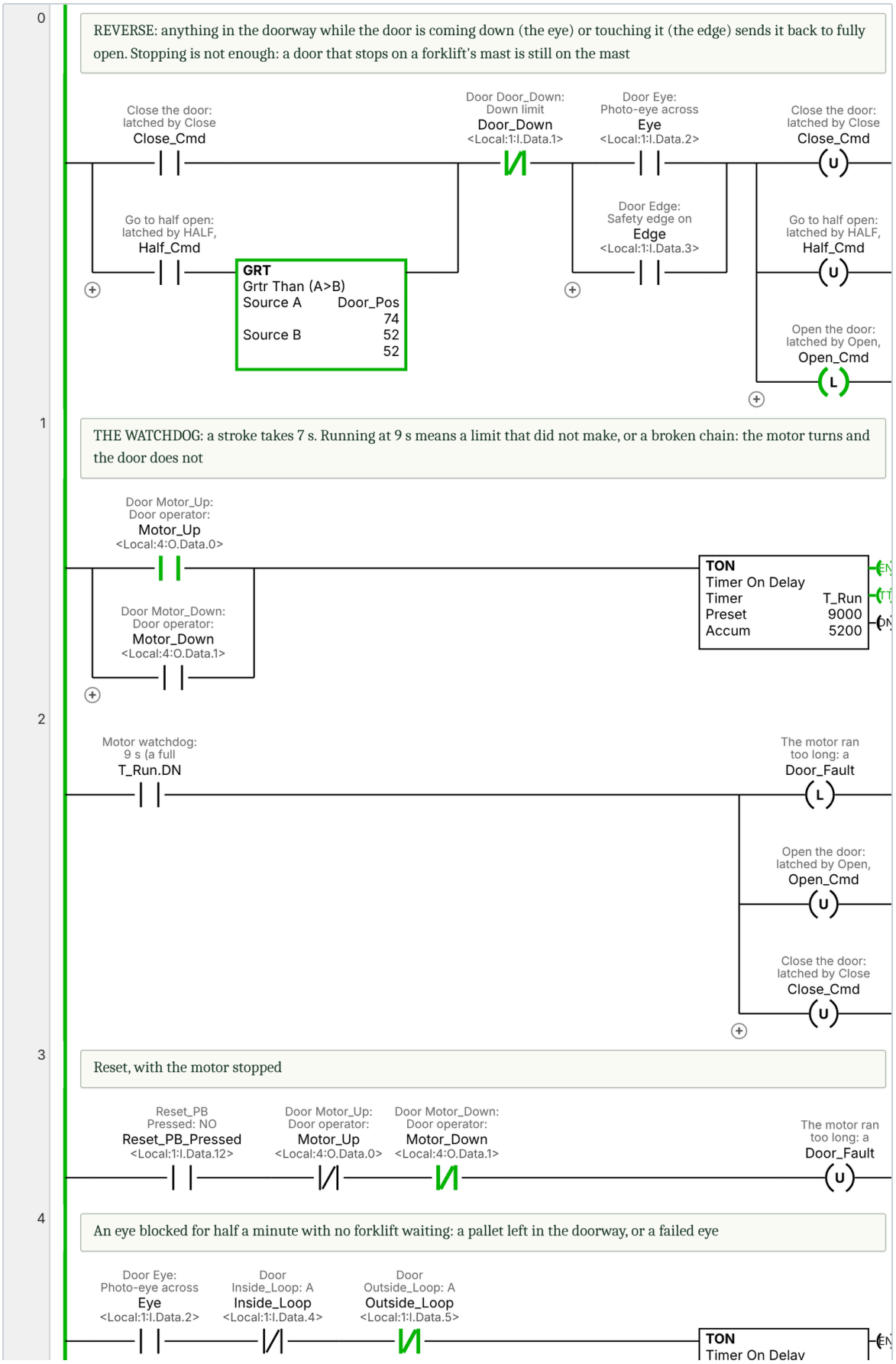
In words. When `Door_Up` (Up limit switch: the door is fully open) is on and it is the first scan they are true (the one-shot, remembered in `Up_ONS` (One-shot storage for the open count)), the rung makes `Open_Count` (Times the door has opened) = `Open_Count` + **1**.

Left to right:

1. `XIC(Door_Up)` *Examine If Closed* asks: true when `Door_Up` (Up limit switch: the door is fully open) is on.
2. `ONS(Up_ONS)` *One Shot* asks: true when it is the first scan they are true (the one-shot, remembered in `Up_ONS` (One-shot storage for the open count)).
3. `ADD(Open_Count,1,Open_Count)` *Add* does: makes `Open_Count` (Times the door has opened) = `Open_Count` + **1**.

Why it is built this way:

One-shot: once per event, not once per scan.


0
REVERSE: anything in the doorway while the door is coming down (the eye) or touching it (the edge) sends it back to fully open. Stopping is not enough: a door that stops on a forklift's mast is still on the mast
Close the door: latched by Close
Close_Cmd
Door Door_Down: Down limit
Door_Down
<Local:1:I.Data.1>
Door Eye: Photo-eye across
Eye
<Local:1:I.Data.2>
Close the door: latched by Close
Close_Cmd
U
Go to half open: latched by HALF,
Half_Cmd
GRT
Grtr Than (A>B)
Source A Door_Pos
74
Source B 52
52
Door Edge: Safety edge on
Edge
<Local:1:I.Data.3>
Go to half open: latched by HALF,
Half_Cmd
U
Open the door: latched by Open,
Open_Cmd
L
1
THE WATCHDOG: a stroke takes 7 s. Running at 9 s means a limit that did not make, or a broken chain: the motor turns and the door does not
Door Motor_Up: Door operator:
Motor_Up
<Local:4:O.Data.0>
Door Motor_Down: Door operator:
Motor_Down
<Local:4:O.Data.1>
TON
Timer On Delay
Timer T_Run
Preset 9000
Accum 5200
2
Motor watchdog: 9 s (a full
T_Run.DN
The motor ran too long: a
Door_Fault
L
Open the door: latched by Open,
Open_Cmd
U
Close the door: latched by Close
Close_Cmd
U
3
Reset, with the motor stopped
Reset_PB Pressed: NO
Reset_PB_Pressed
<Local:1:I.Data.12>
Door Motor_Up: Door operator:
Motor_Up
<Local:4:O.Data.0>
Door Motor_Down: Door operator:
Motor_Down
<Local:4:O.Data.1>
The motor ran too long: a
Door_Fault
U
4
An eye blocked for half a minute with no forklift waiting: a pallet left in the doorway, or a failed eye
Door Eye: Photo-eye across
Eye
<Local:1:I.Data.2>
Door Inside_Loop: A
Inside_Loop
<Local:1:I.Data.4>
Door Outside_Loop: A
Outside_Loop
<Local:1:I.Data.5>
TON
Timer On Delay

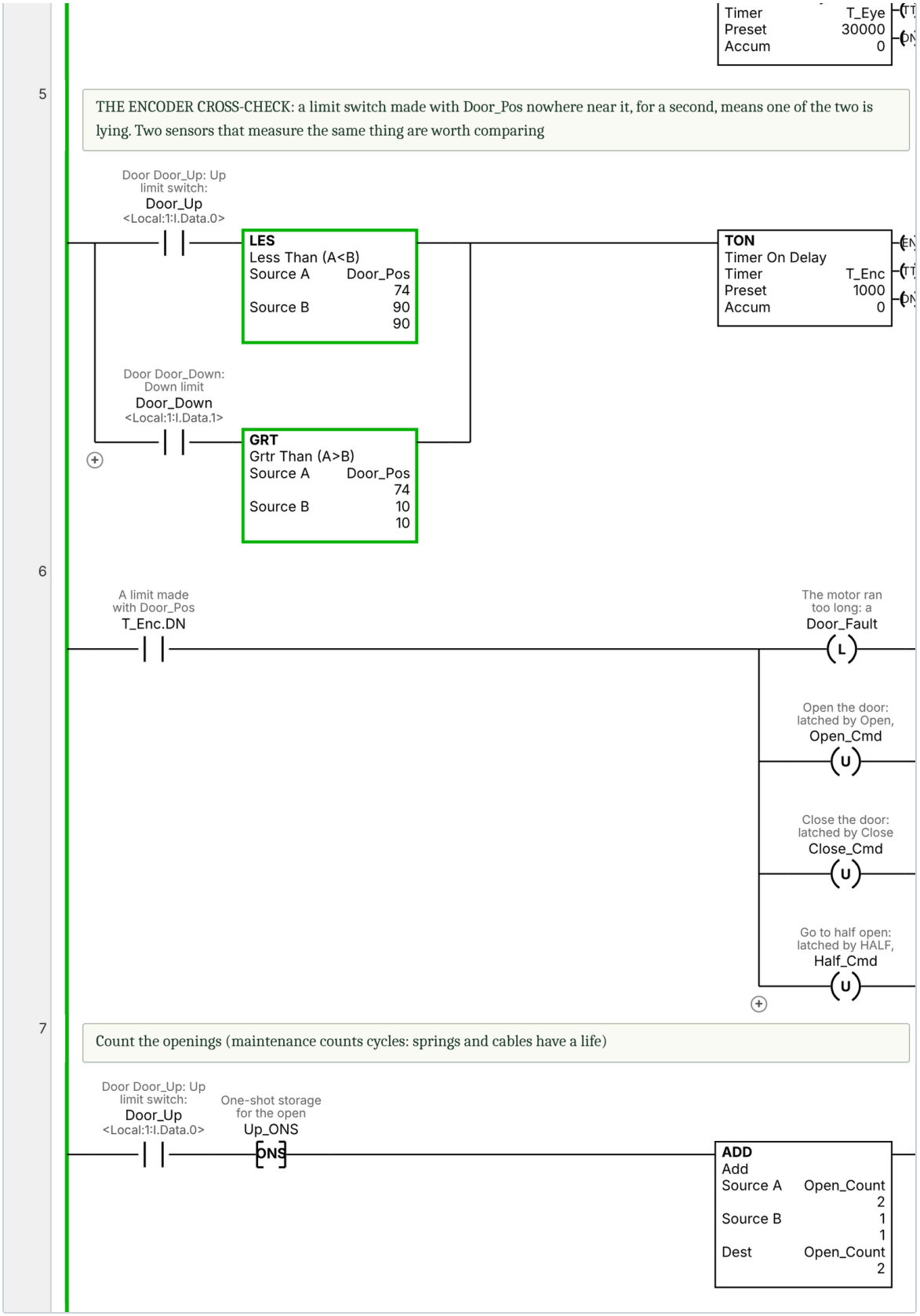


Figure 234. The Safety routine online.

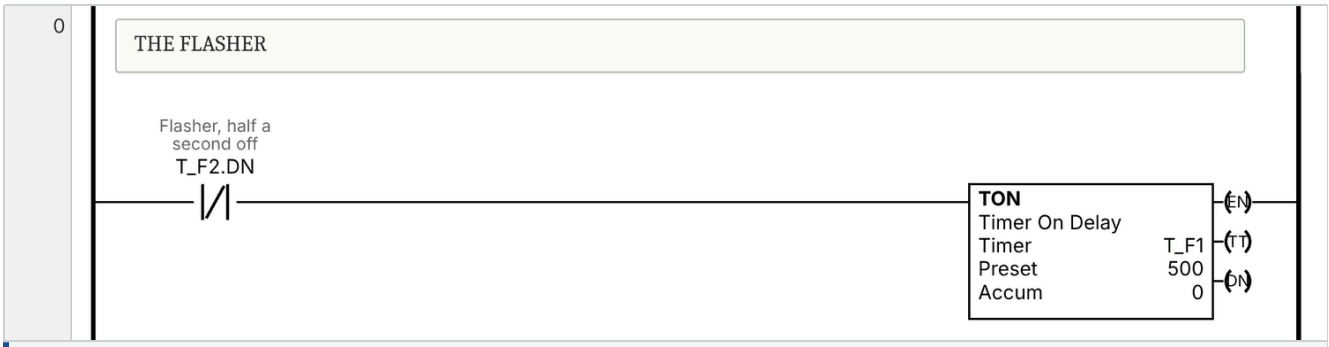
> **WATCH FOR: THE AUTO-CLOSE AND THE FORKLIFT COMING BACK**
>
> Six seconds after the forklift has gone through, the auto-close starts the door down. The forklift comes back from the dock a few seconds later and arrives on the outside loop while the door is half closed. The open command wins, the door reverses, the forklift waits for it to be fully up and comes in. Lengthen the dwell outside or shorten the auto-close and watch the door go up and down for it; take the loops out of the open rung and watch the driver sit and sound the horn.

Beyond the classic exercise: position, half open, maintenance

The classic door has two limit switches and three buttons. A real dock door has a little more, and from Version 138 so does this one. The two limit switches are drawn as what they are: roller-lever switches on brackets on the guide, LS1 at the top and LS2 at the floor, each tripped by a striker on the curtain's bottom bar and each with a bright LED that is lit while it is made. They are the only things that tell the program where the door is, unless the operator also has an **encoder**: this one does, and `Door_Pos` says 0 shut to 100 fully open. With it the station has a **HALF OPEN** button (half way, for air or a person on foot, with no switch at half way: the door stops on a LIM window around 50 %, because it moves 0.7 % a scan and an EQU could be stepped over) and a position display in colour bands. A **RUN / MAINT** key turns the buttons into hold-to-run, the way a technician wants them while adjusting the door: nothing latches, the door moves while the button is held, and the safety edge still stops it. And two sensors that measure the same thing are worth comparing: a limit switch made with Door_Pos nowhere near it means one of them is lying, which the program treats as a fault.

Exercise 4: the beacon and the lamps

LAMPS · RUNG 0




```
XIO(T_F2.DN)TON(T_F1,500,0);
```

THE FLASHER

HOW THIS RUNG WORKS

In words. When `T_F2.DN` (the done bit of `T_F2` : Flasher, half a second off) is off, the rung runs the timer `T_F1` (Flasher, half a second on) toward 500 ms (0.5 s).

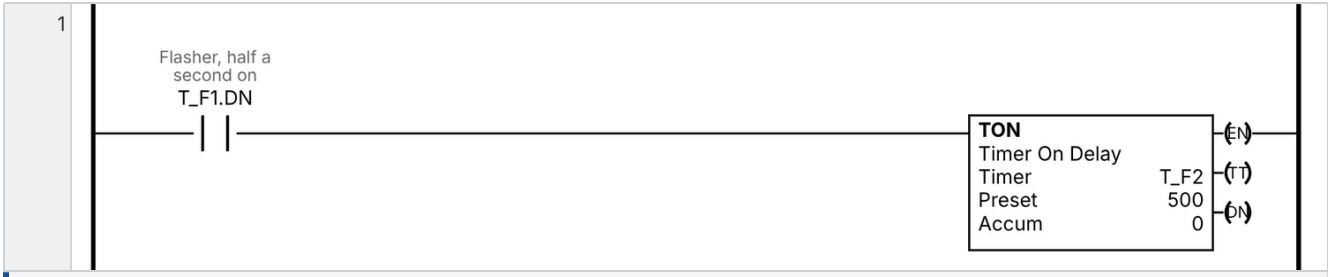
Left to right:

1. `XIO(T_F2.DN)` *Examine If Open* asks: true when `T_F2.DN` (the done bit of `T_F2` : Flasher, half a second off) is off.
2. `TON(T_F1,500,0)` *Timer On Delay* does: runs the timer `T_F1` (Flasher, half a second on) toward 500 ms (0.5 s).

Why it is built this way:

A timer: it times only while this rung is true.

LAMPS · RUNG 1




```
XIC(T_F1.DN)TON(T_F2,500,0);
```

HOW THIS RUNG WORKS

In words. When `T_F1.DN` (the done bit of `T_F1` : Flasher, half a second on) is on, the rung runs the timer `T_F2` (Flasher, half a second off) toward 500 ms (0.5 s).

Left to right:

1. `XIC(T_F1.DN)` *Examine If Closed* asks: true when `T_F1.DN` (the done bit of `T_F1` : Flasher, half a second on) is on.
2. `TON(T_F2,500,0)` *Timer On Delay* does: runs the timer `T_F2` (Flasher, half a second off) toward 500 ms (0.5 s).

Why it is built this way:

A timer: it times only while this rung is true.

LAMPS · RUNG 2

```
XIC(T_F1.DN)OTE(Fl_Bit);
```

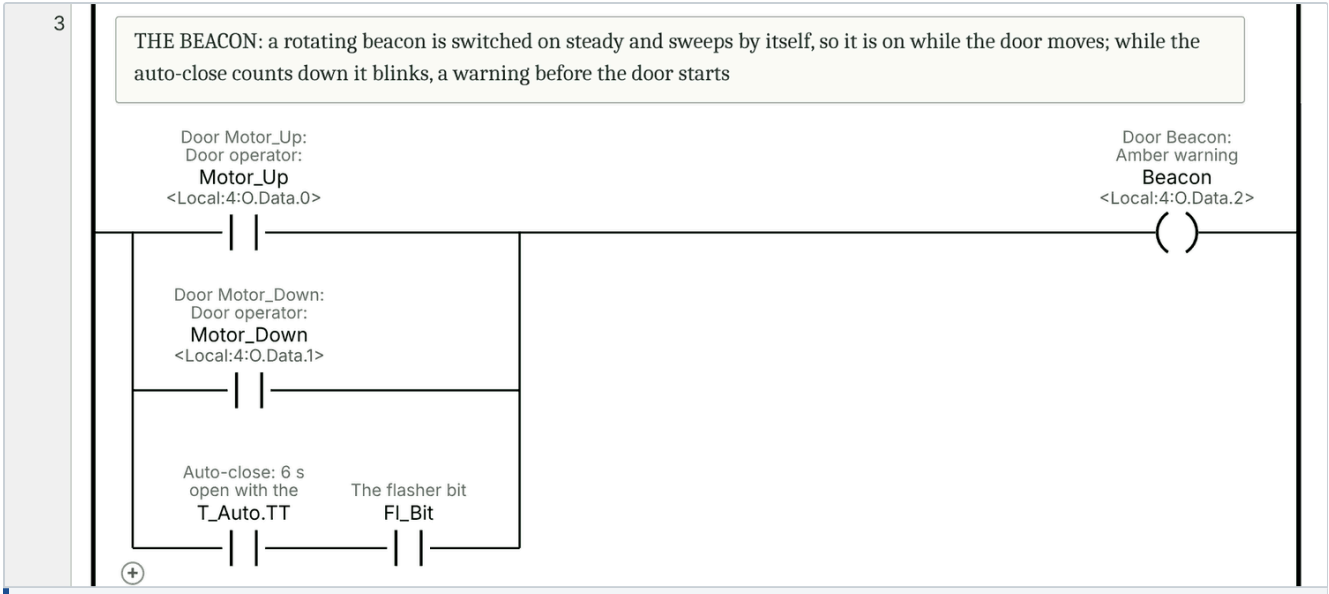
HOW THIS RUNG WORKS

In words. When `T_F1.DN` (the done bit of `T_F1` : Flasher, half a second on) is on, the rung turns `Fl_Bit` (The flasher bit) on (and off again whenever the rung is false).

Left to right:

1. `XIC(T_F1.DN)` *Examine If Closed* asks: true when `T_F1.DN` (the done bit of `T_F1` : Flasher, half a second on) is on.
2. `OTE(Fl_Bit)` *Output Energize* does: turns `Fl_Bit` (The flasher bit) on (and off again whenever the rung is false).

LAMPS · RUNG 3




```
[XIC(Motor_Up),XIC(Motor_Down),XIC(T_Auto.TT)XIC(Fl_Bit)]OTE(Beacon);
```

THE BEACON: a rotating beacon is switched on steady and sweeps by itself, so it is on while the door moves; while the auto-close counts down it blinks, a warning before the door starts

PREDICT IT · 23 `Motor_Up` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

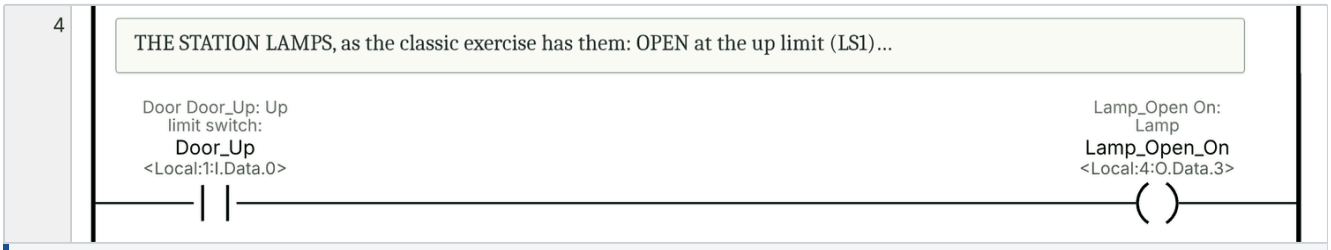
HOW THIS RUNG WORKS

In words. When either (`Motor_Up` (Door operator: open) is on) *or* (`Motor_Down` (Door operator: close) is on) *or* (`T_Auto.TT` (the timing bit of `T_Auto` : Auto-close: 6 s open with the doorway and both loops clear) is on and `Fl_Bit` (The flasher bit) is on), the rung turns `Beacon` (Amber warning beacon over the door) on (and off again whenever the rung is false).

Left to right:

1. A branch of 3 parallel paths: power gets through if any one of them is true.
   - **Path 1**
     1. `XIC(Motor_Up)` *Examine If Closed* asks: true when `Motor_Up` (Door operator: open) is on.
   - **Path 2**
     1. `XIC(Motor_Down)` *Examine If Closed* asks: true when `Motor_Down` (Door operator: close) is on.
   - **Path 3**
     1. `XIC(T_Auto.TT)` *Examine If Closed* asks: true when `T_Auto.TT` (the timing bit of `T_Auto` : Auto-close: 6 s open with the doorway and both loops clear) is on.
     2. `XIC(Fl_Bit)` *Examine If Closed* asks: true when `Fl_Bit` (The flasher bit) is on.
2. `OTE(Beacon)` *Output Energize* does: turns `Beacon` (Amber warning beacon over the door) on (and off again whenever the rung is false).

LAMPS · RUNG 4




```
XIC(Door_Up)OTE(Lamp_Open_On);
```

THE STATION LAMPS, as the classic exercise has them: OPEN at the up limit (LS1)...

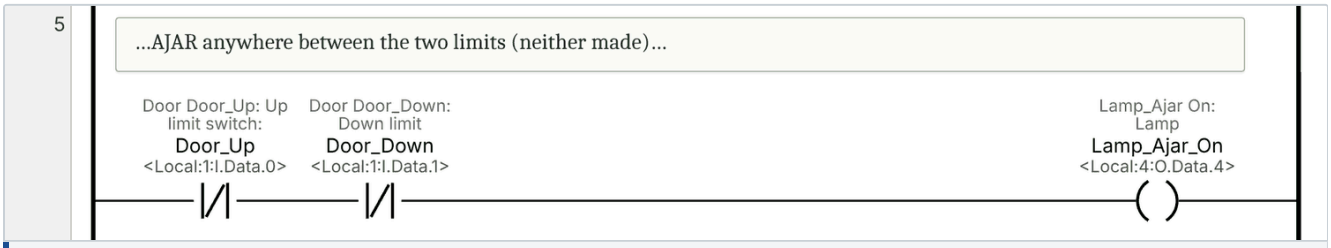
HOW THIS RUNG WORKS

In words. When `Door_Up` (Up limit switch: the door is fully open) is on, the rung turns `Lamp_Open_On` (Lamp) on (and off again whenever the rung is false).

Left to right:

1. `XIC(Door_Up)` *Examine If Closed* asks: true when `Door_Up` (Up limit switch: the door is fully open) is on.
2. `OTE(Lamp_Open_On)` *Output Energize* does: turns `Lamp_Open_On` (Lamp) on (and off again whenever the rung is false).

LAMPS · RUNG 5




```
XIO(Door_Up)XIO(Door_Down)OTE(Lamp_Ajar_On);
```

...AJAR anywhere between the two limits (neither made)...

PREDICT IT · 24 `Door_Up` is off and `Door_Down` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

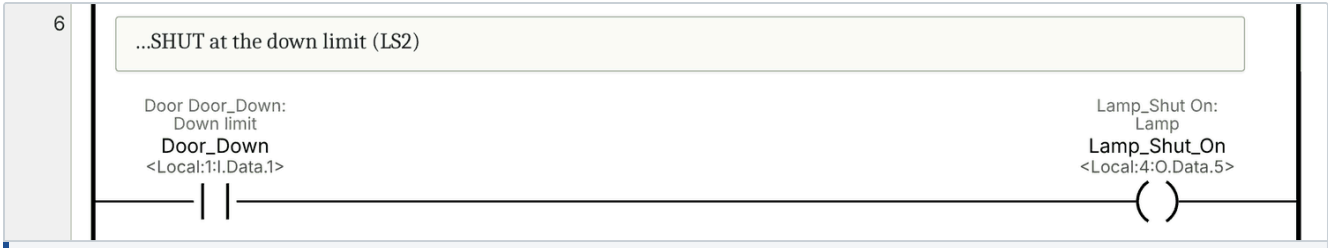
HOW THIS RUNG WORKS

In words. When `Door_Up` (Up limit switch: the door is fully open) is off and `Door_Down` (Down limit switch: the door is fully closed) is off, the rung turns `Lamp_Ajar_On` (Lamp) on (and off again whenever the rung is false).

Left to right:

1. `XIO(Door_Up)` *Examine If Open* asks: true when `Door_Up` (Up limit switch: the door is fully open) is off.
2. `XIO(Door_Down)` *Examine If Open* asks: true when `Door_Down` (Down limit switch: the door is fully closed) is off.
3. `OTE(Lamp_Ajar_On)` *Output Energize* does: turns `Lamp_Ajar_On` (Lamp) on (and off again whenever the rung is false).

LAMPS · RUNG 6




```
XIC(Door_Down)OTE(Lamp_Shut_On);
```

...SHUT at the down limit (LS2)

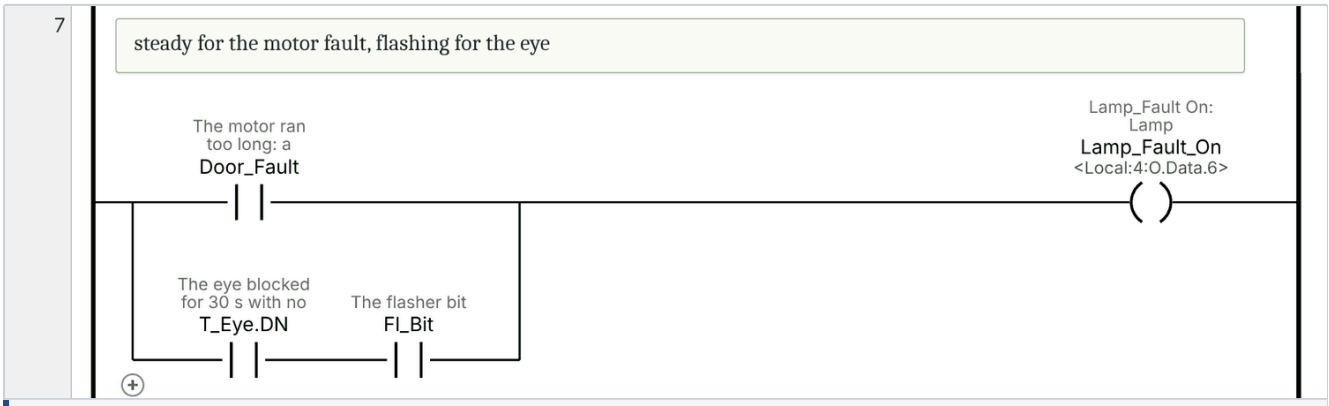
HOW THIS RUNG WORKS

In words. When `Door_Down` (Down limit switch: the door is fully closed) is on, the rung turns `Lamp_Shut_On` (Lamp) on (and off again whenever the rung is false).

Left to right:

1. `XIC(Door_Down)` *Examine If Closed* asks: true when `Door_Down` (Down limit switch: the door is fully closed) is on.
2. `OTE(Lamp_Shut_On)` *Output Energize* does: turns `Lamp_Shut_On` (Lamp) on (and off again whenever the rung is false).

LAMPS · RUNG 7




```
[XIC(Door_Fault),XIC(T_Eye.DN)XIC(Fl_Bit)]OTE(Lamp_Fault_On);
```

steady for the motor fault, flashing for the eye

PREDICT IT · 25 `Door_Fault` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When either (`Door_Fault` (The motor ran too long: a limit switch, the chain or the motor has failed. Reset clears it) is on) *or* (`T_Eye.DN` (the done bit of `T_Eye` : The eye blocked for 30 s with no forklift waiting: something is standing in the doorway, or the eye has failed) is on and `Fl_Bit` (The flasher bit) is on), the rung turns `Lamp_Fault_On` (Lamp) on (and off again whenever the rung is false).

Left to right:

1. A branch of 2 parallel paths: power gets through if any one of them is true.
   - **Path 1**
     1. `XIC(Door_Fault)` *Examine If Closed* asks: true when `Door_Fault` (The motor ran too long: a limit switch, the chain or the motor has failed. Reset clears it) is on.
   - **Path 2**
     1. `XIC(T_Eye.DN)` *Examine If Closed* asks: true when `T_Eye.DN` (the done bit of `T_Eye` : The eye blocked for 30 s with no forklift waiting: something is standing in the doorway, or the eye has failed) is on.
     2. `XIC(Fl_Bit)` *Examine If Closed* asks: true when `Fl_Bit` (The flasher bit) is on.
2. `OTE(Lamp_Fault_On)` *Output Energize* does: turns `Lamp_Fault_On` (Lamp) on (and off again whenever the rung is false).

Machine faults on the roll-up door

Tools → Inject Machine Fault… lists the faults that belong to the parts on this floor. Each has a fault card: the symptom, what to watch in the I/O to confirm it, the repair, and what a program should do about it. **Repair** on an active fault removes its cause the way a technician would; whether the machine runs again is up to the program. The reference's watchdog catches the dead up limit and the broken chain; the stuck edge makes it reverse every close, which keeps everybody safe and the door open, and the exercise is to tell an edge fault from an obstruction.

Fault	Watch	Repair	The program should
Roll-up door: up limit switch dead The door goes fully up but its up limit never makes: a program that opens until Door_Up drives the operator against its stop.	Motor_Up 1, the door fully open, Door_Up 0; the forklift waits; after 4 s against the stop the operator's thermal trips.	Replace the limit switch (Repair).	A motor watchdog: Motor_Up or Motor_Down on for longer than a stroke takes (the reference: 9 s for a 7 s stroke) latches a fault and drops the motor.
Roll-up door: photo-eye blocked or misaligned The eye across the opening reports blocked all the time.	Eye 1 with the doorway clear; the door will not close and does not auto-close.	Clean and align the eye (Repair).	An eye blocked for half a minute with nobody waiting is an alarm (the reference flashes the fault lamp). The door stays open: the safe side.
Roll-up door: safety edge stuck pressed The bottom bar's safety edge reports contact all the time.	Edge 1 with the curtain clear of everything; every close reverses at once.	Replace the edge (Repair).	An edge pressed while the door is fully open (it cannot be touching anything up there) is an edge fault, not an obstruction: alarm it.
Roll-up door: position encoder stuck The encoder on the operator stops counting: Door_Pos freezes where it was.	The door moves, the limits change, Door_Pos does not; HALF OPEN runs the door to a limit (or never stops).	Replace the encoder (Repair).	The watchdog catches a HALF that never arrives; cross-check Door_Pos against the limits (Door_Up made with Door_Pos well below 100 is an encoder fault).
Roll-up door: drive chain broken The operator's motor runs and the curtain does not move.	Motor_Up (or Motor_Down) 1, neither limit changing, the door where it was.	Replace the chain (Repair).	The motor watchdog catches it: commanded for longer than a stroke and no limit reached.

Guided: the roll-up door

The scene opens with its guided track: **Init**, **Commands**, **Motor** (fully open, fully closed), **Safety** (close it onto a pallet and it reverses), **Lamps** and **Ready**. The forklift runs throughout, so from the Commands lesson on it asks for the door every half a minute: the loops in the open rung are what answer it.

> **TRY IT: ONE PROGRAM, BOTH MACHINES**
>
> The two programs have the same skeleton: a latched request, a motor between two limits interlocked against itself, something that must never be under the moving part, a watchdog. Write the boom's logic again in the shape of the door's (an Open_Cmd and a Close_Cmd, a reverse on the eye and the closing loop) and compare. Which one would you rather troubleshoot at two in the morning?

Predict it: the answers

1. The rung is false at `XIC(Arming_Loop)`: `Arming_Loop` is off, so `Ticket_Lamp` is **off** (an OTE writes 0 when its rung is false).

2. Every condition on the way is met, so `Ticket_Out` is **on**.
3. Every condition on the way is met, so `Open_Req` latches **on**.
4. The rung is false at `XIC(Raise_PB_Pressed)`: `Raise_PB_Pressed` is off, so the OTL does nothing: `Open_Req` keeps whatever it had.
5. Every condition on the way is met, so `Car_Thru` latches **on**.
6. The rung is false at `XIO(Gate_Fault)`: `Gate_Fault` is on, so `Arm_Raise` is **off** (an OTE writes 0 when its rung is false).
7. Every condition on the way is met, so `Arm_Lower` is **on**.
8. Every condition on the way is met, so `T_Motor` times toward its preset.
9. The rung is false at `XIO(Arm_Raise)`: `Arm_Raise` is on, so the OTU does nothing: `Gate_Fault` keeps whatever it had.
10. Every condition on the way is met, so `Boom_Lights` is **on**.
11. Every condition on the way is met, so `Lamp_Fault_On` is **on**.
12. Every condition on the way is met, so `Open_Cmd` latches **on**, `Close_Cmd` is unlatched, **off** and `Half_Cmd` is unlatched, **off**.
13. Every condition on the way is met, so `Half_Cmd` latches **on**, `Open_Cmd` is unlatched, **off** and `Close_Cmd` is unlatched, **off**.
14. The rung is false at `XIC(Door_Up)`: `Door_Up` is off, so `T_Auto` is reset to zero (a TON resets when its rung goes false).
15. Every condition on the way is met, so `Half_Cmd` is unlatched, **off**.
16. Every condition on the way is met, so `Motor_Up` is **on**.
17. The rung is false at `XIO(Door_Down)`: `Door_Down` is on, so `Motor_Down` is **off** (an OTE writes 0 when its rung is false).
18. Every condition on the way is met, so `Close_Cmd` is unlatched, **off**, `Half_Cmd` is unlatched, **off** and `Open_Cmd` latches **on**.
19. Every condition on the way is met, so `T_Run` times toward its preset.
20. The rung is false at `XIO(Motor_Down)`: `Motor_Down` is on, so the OTU does nothing: `Door_Fault` keeps whatever it had.
21. Every condition on the way is met, so `T_Eye` times toward its preset.
22. Every condition on the way is met, so `T_Enc` times toward its preset.
23. Every condition on the way is met, so `Beacon` is **on**.
24. The rung is false at `XIO(Door_Down)`: `Door_Down` is on, so `Lamp_Ajar_On` is **off** (an OTE writes 0 when its rung is false).
25. Every condition on the way is met, so `Lamp_Fault_On` is **on**.

51 Classic scene: the car wash

A car wash is a short machine that does the same thing every time, in the same order, to every car: soap, brushes, rinse, wax, dry. A machine like that can be written as a list of rungs, one per step, each latching the next; you have done that on the bottle line. It can also be written as a **table**: one row a step, each row saying which outputs are on and what the step waits for, and one instruction that walks down the table. That is what a sequencer is, and this chapter is about the two Logix instructions that make one, **SQO** (Sequencer Output) and **SQI** (Sequencer Input). From Version 141 the car wash is in Logix Connect as a classic scene, under **File → New Project**, opening with its guided track and its own machine faults. It takes its look from the PLCLogix 5000 car wash: a glass tunnel on an orange steel frame, a solar roof, dark portals, a pay station with a boom and yellow bollards.

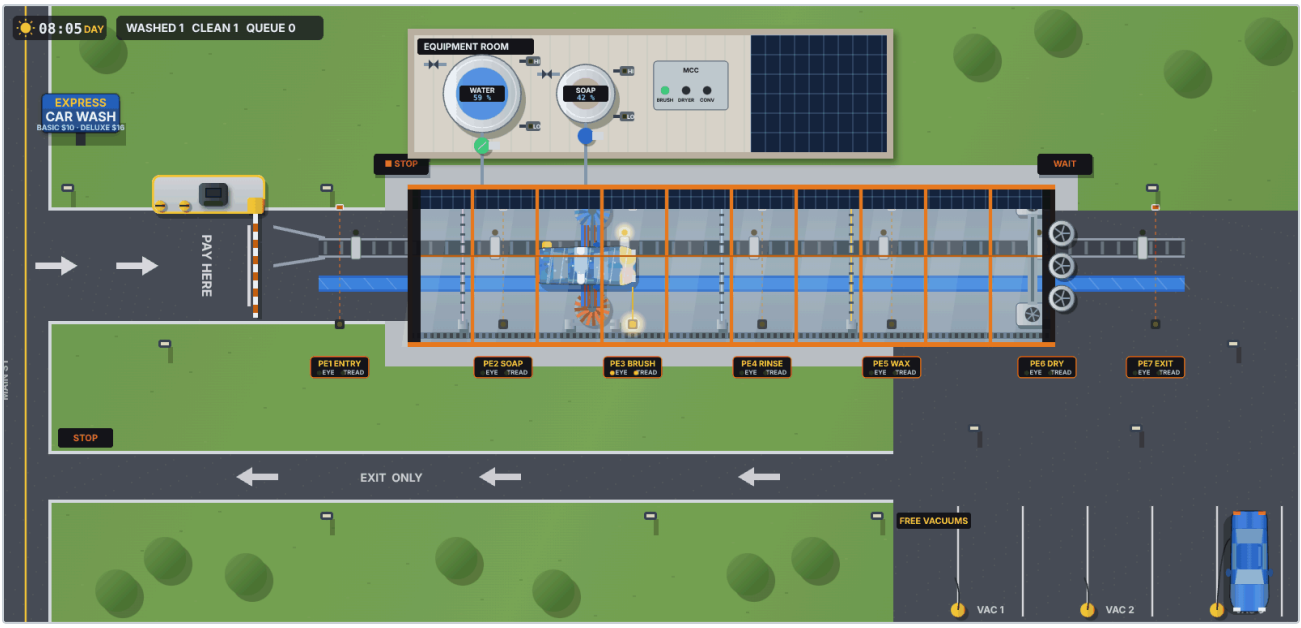


Figure 235. The Car Wash part from above, a car in the brush bay. Left to right: the street, the pylon sign, the pay island (terminal, bollards, the boom), the conveyor trough under the left wheels and the blue track under the right, the glass tunnel on its orange frame (solar panels along the north eave, dark portals at the ends, DRIVE ON and DRIVE OUT over them); inside, the soap arch, the wrap and top brushes, the rinse and wax arches, and the dryer at the exit portal with its three producers on the portal itself. Every station has its photo-eye across the bay (the beam dashed red, amber when a car blocks it) and a treadle in the trough; the name tags along the south wall carry both LEDs. Behind the tunnel the equipment room with the two tanks, their HI and LO switches, the pumps and fill valves, the MCC. MAIN ST down the left edge; after the wash the lot with three vacuum bays, and the one-way EXIT ONLY lane back to the street.

The machine

Drivers arrive by themselves (*Cars per minute*) and stop at the pay terminal. When it says PAY HERE they pay, BASIC or DELUXE (*Drivers who buy DELUXE*; DELUXE adds wax), and wait for the boom. With the boom up they pull forward while the sign over the entry says DRIVE ON, until the front wheel stands

on the first treadle, TS1, at the mouth of the conveyor, and put the car in neutral. From then on the conveyor moves the car: a roller pops up behind the left front wheel and pushes. It is a stop-and-go wash: the conveyor carries the car to a station, the program stops it with the wheel on that station's treadle, the station works, the conveyor carries it on. The last station is the dryer, built into the exit portal: blowers each side and three producers on the portal pointing down at the car. Then the sign says DRIVE OUT and the driver pulls off the conveyor into the lot. About half the drivers (*Drivers who stop at the free vacuums*) take a free vacuum bay, vacuum the car for half a minute or so and back out; everybody leaves by the one-way EXIT ONLY lane to MAIN ST. Each car leaves the log a verdict: clean and dry, or what it missed (no soap, brushed dry, still soapy, still wet, no wax on a DELUXE).

Every sensor is where the car stands, and you can see it. At each of the seven stations a photo-eye pair crosses the bay at bumper height, the emitter on the south side and the reflector on the north, and a treadle sits in the trough where the front wheel stops. The name tags along the wall, PE1 ENTRY to PE7 EXIT, light an LED for each. Behind the tunnel, the equipment room: a water tank and a soap tank, each with a LO and a HI level switch, a pump to the arches and a fill valve; they show their level in percent and on the analog inputs.

Signal		Meaning
`Pay_OK`	input	The pay terminal took the money: a short pulse
`Deluxe_Sel`	input	DELUXE (with wax) was bought: on together with Pay_OK
`TS1_Entry`	input	Wheel treadle 1 (ENTRY): the front wheel stands on it
`TS2_Soap`	input	Wheel treadle 2 (SOAP): the front wheel stands on it
`TS3_Brush`	input	Wheel treadle 3 (BRUSH): the front wheel stands on it
`TS4_Rinse`	input	Wheel treadle 4 (RINSE): the front wheel stands on it
`TS5_Wax`	input	Wheel treadle 5 (WAX): the front wheel stands on it
`TS6_Dry`	input	Wheel treadle 6 (DRY): the front wheel stands on it
`TS7_Exit`	input	Wheel treadle 7 (EXIT): the front wheel stands on it
`PE1_Entry`	input	Photo-eye 1 (ENTRY) across the bay at bumper height: 1 while a car blocks it
`PE2_Soap`	input	Photo-eye 2 (SOAP) across the bay at bumper height: 1 while a car blocks it
`PE3_Brush`	input	Photo-eye 3 (BRUSH) across the bay at bumper height: 1 while a car blocks it
`PE4_Rinse`	input	Photo-eye 4 (RINSE) across the bay at bumper height: 1 while a car blocks it
`PE5_Wax`	input	Photo-eye 5 (WAX) across the bay at bumper height: 1 while a car blocks it
`PE6_Dry`	input	Photo-eye 6 (DRY) across the bay at bumper height: 1 while a car blocks it
`PE7_Exit`	input	Photo-eye 7 (EXIT) across the bay at bumper height: 1 while a car blocks it
`Brush_OL`	input	Brush motor overload relay: 1 when it has tripped

Signal		Meaning
`Water_Low`	input	Water tank low level switch: 1 below 20 %
`Water_High`	input	Water tank high level switch: 1 above 90 %
`Soap_Low`	input	Soap tank low level switch: 1 below 20 %
`Soap_High`	input	Soap tank high level switch: 1 above 90 %
`Gate_Open`	output	Raise the entry boom (it comes down when this drops)
`Pay_Lamp`	output	PAY HERE on the pay terminal: the wash will take the next car
`Enter_Sign`	output	The sign over the entry: DRIVE ON (green) or STOP (red)
`Conveyor`	output	Tunnel conveyor motor: the roller pushes the front wheel
`Soap_Valve`	output	Soap arch valve
`Brushes`	output	Wrap and top brush motors
`Rinse_Valve`	output	Rinse arch valve
`Wax_Valve`	output	Wax arch valve
`Dryer`	output	Dryer blowers
`Exit_Sign`	output	The sign over the exit: DRIVE OUT (green) or WAIT (red)
`Water_Pump`	output	Water pump to the brushes and the rinse arch
`Soap_Pump`	output	Soap pump to the soap arch
`Water_Fill`	output	Water tank fill valve (city water)
`Soap_Fill`	output	Soap tank fill valve (from the bulk drum)
`Water_Level`	analog in	Water tank level, %
`Soap_Level`	analog in	Soap tank level, %

Figure 236. The side view: an elevation from the south through the glass at the plan's scale. The pay terminal and the boom (it rises toward you), the car in profile on the conveyor in the brush bay, the arches and the brushes inside (the top brush seen end on), the dryer's blowers, the photo-eyes at bumper height on the near wall and the treadles in the floor, the name tags under the wall. The equipment room is behind the tunnel, so the inset at the top left cuts it open: both tanks, their switches, valves and pumps.

Figure 237. The scene: the wash outdoors and its console: START, STOP (normally closed), RESET, RUN, FAULT, SUPPLY LOW, the car count and the step number.

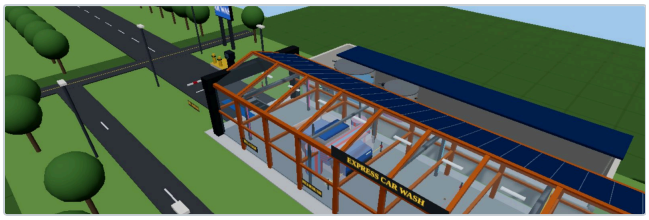


Figure 238. In 3D: the glass tunnel on its orange portal frames, the solar roof, a car under the brushes, the equipment room and its tanks behind.

Day and night

Like the parking gate, the wash runs through the day and the night on the run's own clock (*Day length*, 24 minutes by default; *Start hour*). At night a car wash is its own advertisement: the tunnel glows through its glass, the arches light up in their colours (soap pink, brushes blue, rinse cyan, wax amber), the signs and the pylon shine, the cars come in with their headlights on.

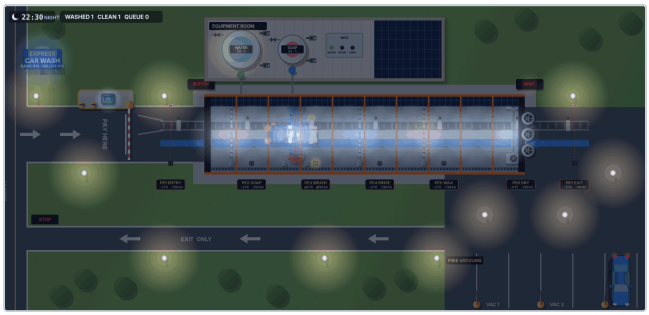


Figure 239. At night from above.

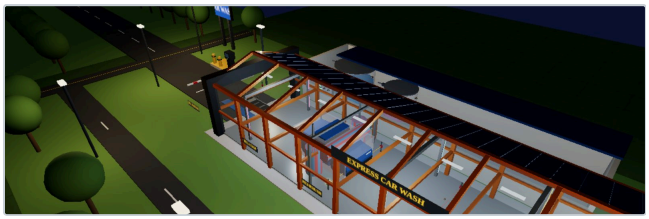


Figure 240. At night in 3D.

TRY IT: CLICK THE MACHINE

Click the pay terminal and the waiting driver pays at once. Click a tank (in the plan, in the side view's inset, or in 3D) and it drains to 12 %. Click the road on the left and another car pulls in. Then drain the soap tank while a car is on its way to SOAP, and watch it wait.

The step table

Write the wash down the way the operator would describe it, a step at a time, and every step has three things: what is on, what it waits for, how long. Number the steps from 1, keep step 0 as HOME (nothing on), and you have a table of fourteen rows:

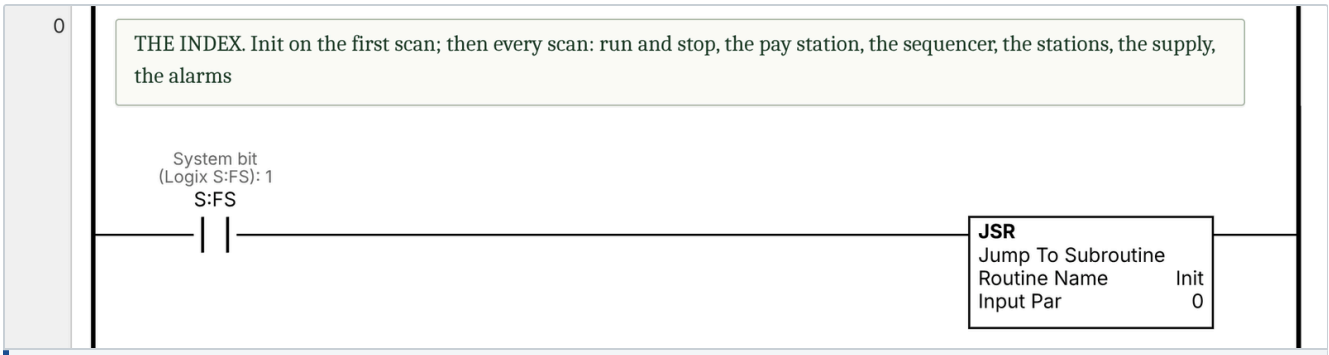
POS	Step	Dwell	Exit	Dry	Wax	Rinse	Brush	Soap	Conv	Enter	Done	Paid	TS7	TS6	TS5	TS4	TS3	TS2	TS1	Dwell
		Out_Table: bit 8 … bit 0									In_Table: bit 8 … bit 0									ms
0	HOME	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
1	LOAD	0	0	0	0	0	0	0	0	1	0	1	0	0	0	0	0	0	1	
2	TO SOAP	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	1	0	
3	SOAP	1	0	0	0	0	0	1	0	0	1	0	0	0	0	0	0	1	0	4000
4	TO BRUSH	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	1	0	0	
5	BRUSH	1	0	0	0	0	1	0	0	0	1	0	0	0	0	0	1	0	0	6000
6	TO RINSE	0	0	0	0	0	0	0	1	0	0	0	0	0	0	1	0	0	0	
7	RINSE	1	0	0	0	1	0	0	0	0	1	0	0	0	0	1	0	0	0	4000
8	TO WAX	0	0	0	0	0	0	0	1	0	0	0	0	0	1	0	0	0	0	
9	WAX	1	0	0	1	0	0	0	0	0	1	0	0	0	1	0	0	0	0	3000
10	TO DRY	0	0	0	0	0	0	0	1	0	0	0	0	1	0	0	0	0	0	
11	DRY	1	0	1	0	0	0	0	0	0	1	0	0	1	0	0	0	0	0	5000
12	TO EXIT	0	0	0	0	0	0	0	1	0	0	0	1	0	0	0	0	0	0	
13	EXIT	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

The output word, `Wash_Out`, has one bit for each thing a step can turn on: 0 the DRIVE ON sign, 1 the conveyor, 2 soap, 3 brushes, 4 rinse, 5 wax, 6 dryer, 7 DRIVE OUT, and bit 8, *Dwell*, which is not an output at all: it tells the dwell timer to run. The input word, `Wash_In`, has a bit for everything a step can wait for: the seven treadles in bits 0 to 6, the payment in bit 7, the dwell timer done in bit 8. Read the SOAP row: out `2#1_0000_0100` is Dwell and Soap; in `2#1_0000_0010` is TS2 and the timer done: soap the car standing on TS2 until the timer runs out. The underscores are only for your eyes, like the commas in 1,000.

> **THINK OF IT LIKE THIS: A PLAYER PIANO**
>
> A player piano's roll has a row of holes for every moment of the tune and a hole for each key. The piano does not know the tune; it reads the next row and plays whatever keys have holes. `Out_Table` is the roll, `Wash_Out` is the row under the reading head, `Wash_Ctl.POS` is how far the roll has turned. SQO turns the roll on one row. SQI is the pianist's foot: it waits for the moment to turn the roll.

The program's index

MAINROUTINE · RUNG 0




```
XIC(S:FS)JSR(Init);
```

THE INDEX. Init on the first scan; then every scan: run and stop, the pay station, the sequencer, the stations, the supply, the alarms

HOW THIS RUNG WORKS

In words. When `S:FS` is on, the rung runs the routine `Init`.

Left to right:

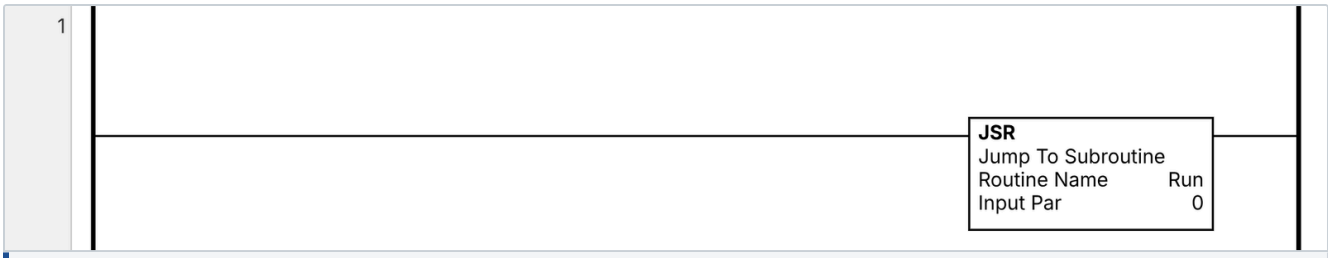
1. `XIC(S:FS)` *Examine If Closed* asks: true when `S:FS` is on.
2. `JSR(Init)` *Jump to Subroutine* does: runs the routine `Init`. The routine is scanned at this point, then the scan comes back here.

Why it is built this way:

Part of the index. The main routine is a list of calls; each routine holds one job, and the scan goes through them in this order.

> **Think of it like this:** the table of contents of a book. The main routine does no work itself: it lists the chapters and the order to read them in, every scan.

MAINROUTINE · RUNG 1




```
JSR(Run);
```

HOW THIS RUNG WORKS

In words. Every scan, with nothing to test, the rung runs the routine `Run`.

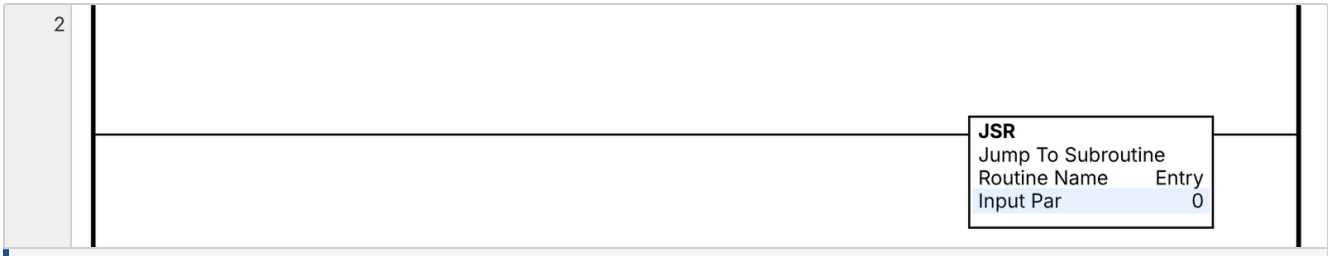
Left to right:

1. `JSR(Run)` *Jump to Subroutine* does: runs the routine `Run`.

Why it is built this way:

Part of the index: one call, one job.

MAINROUTINE · RUNG 2




```
JSR(Entry);
```

HOW THIS RUNG WORKS

In words. Every scan, with nothing to test, the rung runs the routine `Entry`.

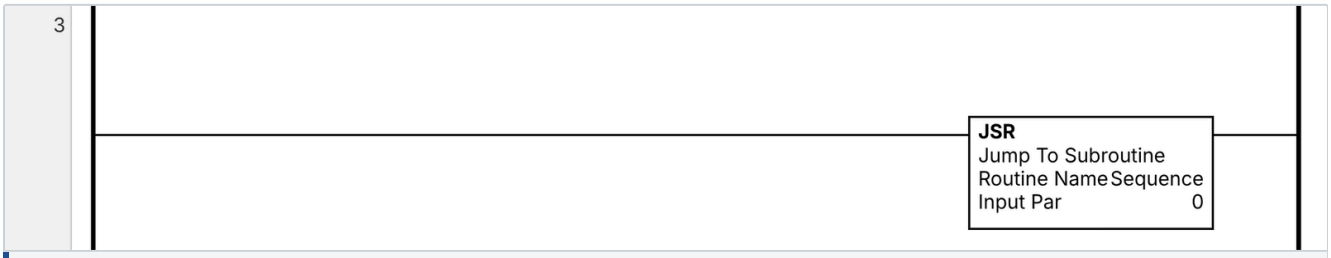
Left to right:

1. `JSR(Entry)` *Jump to Subroutine* does: runs the routine `Entry`.

Why it is built this way:

Part of the index: one call, one job.

MAINROUTINE · RUNG 3




```
JSR(Sequence);
```

HOW THIS RUNG WORKS

In words. Every scan, with nothing to test, the rung runs the routine `Sequence`.

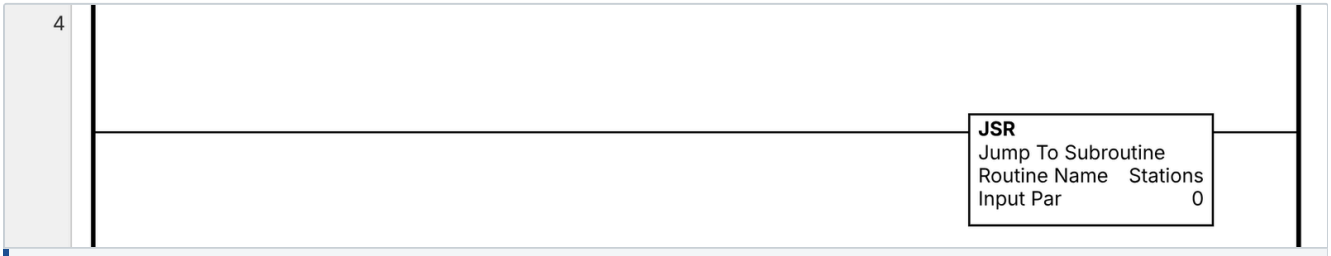
Left to right:

1. `JSR(Sequence)` *Jump to Subroutine* does: runs the routine `Sequence`.

Why it is built this way:

Part of the index: one call, one job.

MAINROUTINE · RUNG 4




```
JSR(Stations);
```

HOW THIS RUNG WORKS

In words. Every scan, with nothing to test, the rung runs the routine `Stations`.

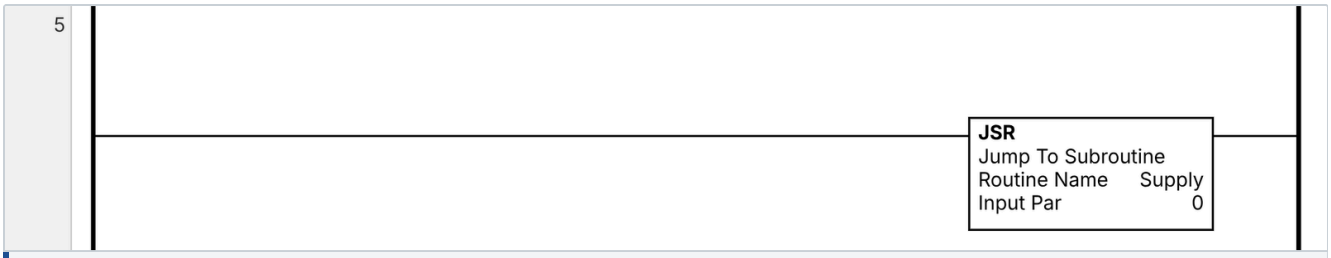
Left to right:

1. `JSR(Stations)` *Jump to Subroutine* does: runs the routine `Stations`.

Why it is built this way:

Part of the index: one call, one job.

MAINROUTINE · RUNG 5




```
JSR(Supply);
```

HOW THIS RUNG WORKS

In words. Every scan, with nothing to test, the rung runs the routine `Supply`.

Left to right:

1. `JSR(Supply)` *Jump to Subroutine* does: runs the routine `Supply`.

Why it is built this way:

Part of the index: one call, one job.

MAINROUTINE · RUNG 6

```
JSR(Alarms);
```

HOW THIS RUNG WORKS

In words. Every scan, with nothing to test, the rung runs the routine `Alarms`.

Left to right:

1. `JSR(Alarms)` *Jump to Subroutine* does: runs the routine `Alarms`.

Why it is built this way:

Part of the index: one call, one job.

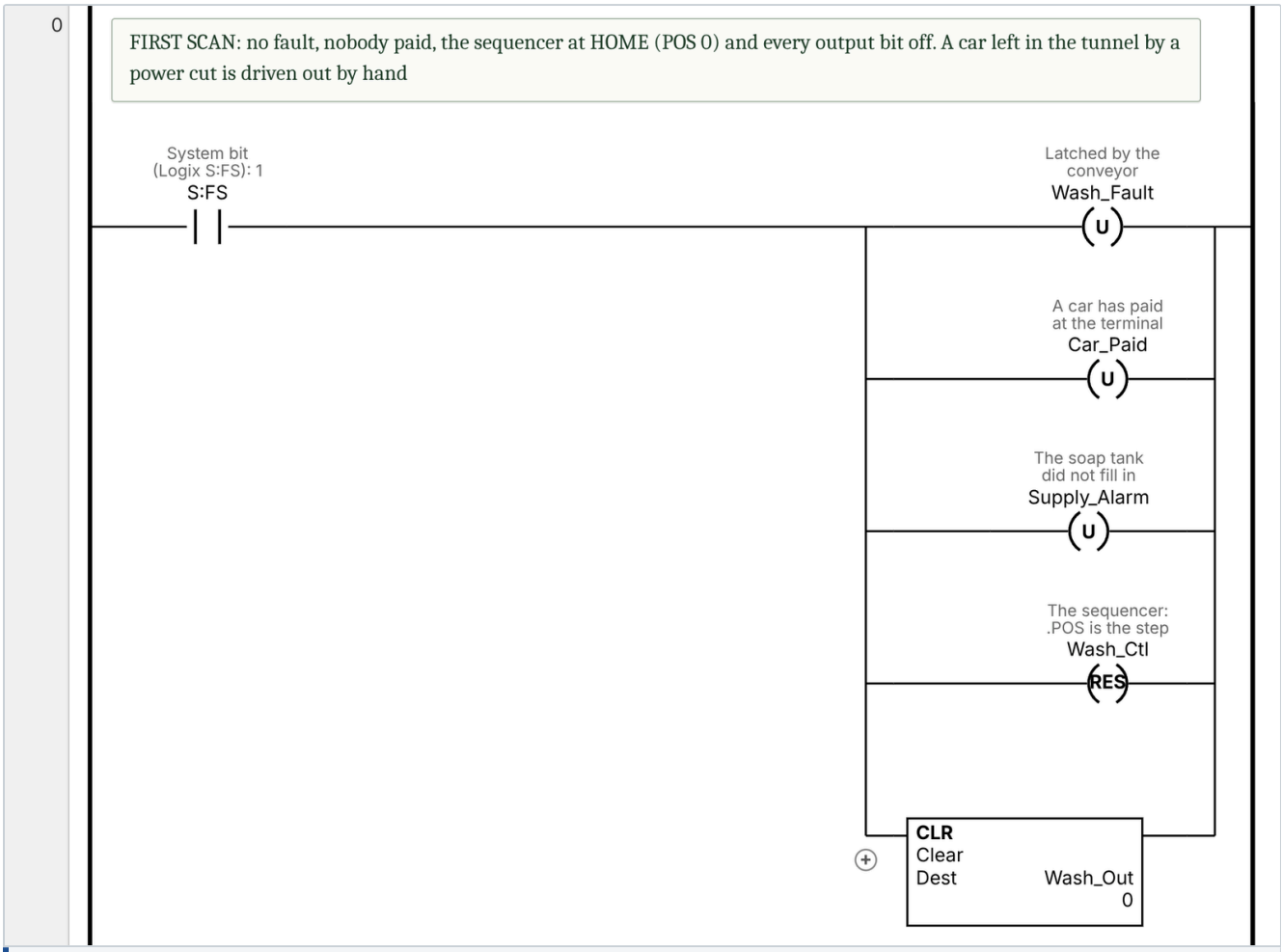
Tag	Type	Description
`Running`	BOOL	The wash is running: START seals it in, STOP (normally closed) or a fault drops it
`Wash_Fault`	BOOL	Latched by the conveyor watchdog or the brush overload; RESET clears it
`Car_Paid`	BOOL	A car has paid at the terminal and may come on (only while the sequencer is at LOAD)
`Deluxe_Car`	BOOL	The car being washed bought DELUXE: the wax arch runs for it

Tag	Type	Description
`Wash_Ctl`	CONTROL	The sequencer: .POS is the step (0 HOME, 1 LOAD ... 13 EXIT), .LEN 13; SQO moves it, SQI reads it
`Wash_Out`	DINT	The step's outputs, one bit each: 0 Enter, 1 Conveyor, 2 Soap, 3 Brushes, 4 Rinse, 5 Wax, 6 Dryer, 7 Exit, 8 Dwell
`Wash_In`	DINT	The inputs a step waits for, gathered into one word: bits 0-6 the treadles TS1-TS7, 7 Car_Paid, 8 the dwell timer done
`Out_Table`	DINT	The output word of every step (element 0 = HOME)
`In_Table`	DINT	The input word every step waits for
`Dwell`	DINT	How long each station runs, ms (0 for the moves)
`Step_Done`	BOOL	The inputs match the step's row (or HOME, or a BASIC car at WAX): step on
`Soap_Hold`	BOOL	The soap tank is low with a car at SOAP: the dwell waits and the pump stays off until it is filled
`T_Step`	TIMER	The station dwell: its preset comes from Dwell[] for the step
`T_Move`	TIMER	Conveyor watchdog: 8 s (a move takes about 3.5 s)
`T_Eye`	TIMER	A wheel on a station's treadle and that bay's eye seeing nothing for 1.5 s: one of the two is lying
`T_SoapFill`	TIMER	Soap fill watchdog: 40 s without reaching the high switch is a supply alarm
`Supply_Alarm`	BOOL	The soap tank did not fill in time (a leak, an empty drum)
`Car_Count`	DINT	Cars washed
`Pay_ONS`	BOOL	One-shot storage, payment
`Exit_ONS`	BOOL	One-shot storage, car counted out
`T_F1`	TIMER	Flasher, half a second on
`T_F2`	TIMER	Flasher, half a second off
`Fl_Bit`	BOOL	The flasher bit

Exercise 1: Init and the table

Init runs on the first scan. Its first rung clears what a power cut leaves half done and sends the sequencer HOME: `RES` on a CONTROL clears its position and its status bits, as it does on a timer. Then one rung per row of the table: three MOVs each. In a plant you would type the values into the tag editor once and never see them in the ladder; here they are written on the first scan so you can read the table where the logic is.

INIT · RUNG 0




```
XIC(S:FS)
[OTU(Wash_Fault),OTU(Car_Paid),OTU(Supply_Alarm),RES(Wash_Ctl),CLR(Wash_Out)];
```

FIRST SCAN: no fault, nobody paid, the sequencer at HOME (POS 0) and every output bit off. A car left in the tunnel by a power cut is driven out by hand

HOW THIS RUNG WORKS

In words. When `S:FS` is on, the rung unlatches `Wash_Fault` (Latched by the conveyor watchdog or the brush overload; RESET clears it) (turns it off), unlatches `Car_Paid` (A car has paid at the terminal and may come on (only while the sequencer is at LOAD)) (turns it off), unlatches `Supply_Alarm` (The soap tank did not fill in time (a leak, an empty drum)) (turns it off), resets `Wash_Ctl` (The sequencer: .POS is the step (0 HOME, 1 LOAD ... 13 EXIT), .LEN 13; SQO moves it, SQI reads it) and clears `Wash_Out` (The step's outputs, one bit each: 0 Enter, 1 Conveyor, 2 Soap, 3 Brushes, 4 Rinse, 5 Wax, 6 Dryer, 7 Exit, 8 Dwell) to 0.

Left to right:

1. `XIC(S:FS)` *Examine If Closed* asks: true when `S:FS` is on.
2. A branch of 5 parallel paths: each path is tested on its own and does its own work.

   Path 1

1. `OTU(Wash_Fault)` *Output Unlatch* does: unlatches `Wash_Fault` (Latched by the conveyor watchdog or the brush overload; RESET clears it) (turns it off). It only ever clears the bit.

Path 2

1. `OTU(Car_Paid)` *Output Unlatch* does: unlatches `Car_Paid` (A car has paid at the terminal and may come on (only while the sequencer is at LOAD)) (turns it off).

Path 3

1. `OTU(Supply_Alarm)` *Output Unlatch* does: unlatches `Supply_Alarm` (The soap tank did not fill in time (a leak, an empty drum)) (turns it off).

Path 4

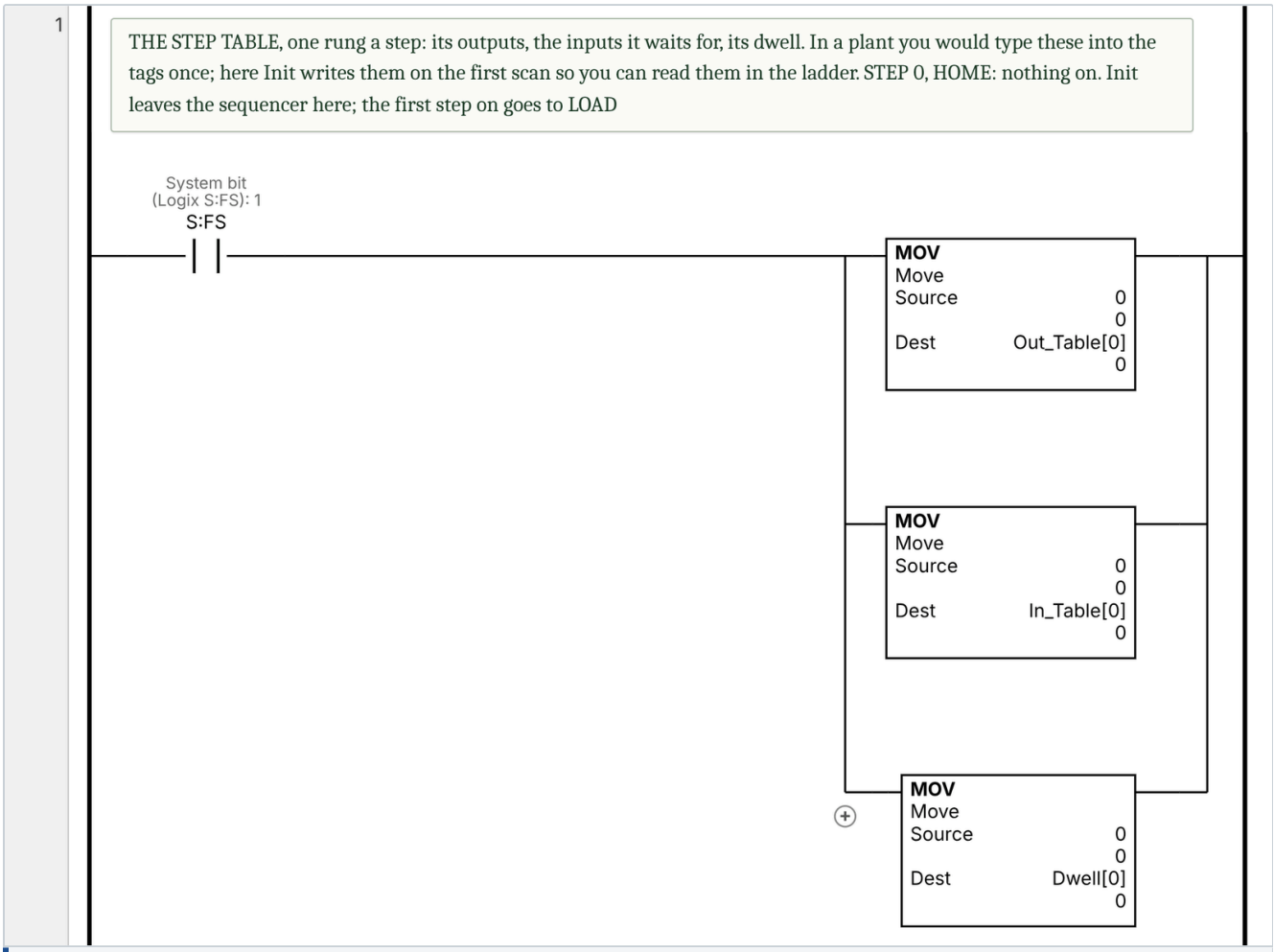
1. `RES(Wash_Ctl)` *Reset* does: resets `Wash_Ctl` (The sequencer: .POS is the step (0 HOME, 1 LOAD ... 13 EXIT), .LEN 13; SQO moves it, SQI reads it). It zeroes a timer or counter (accumulator and status bits), or a control (its position and status bits).

Path 5

1. `CLR(Wash_Out)` *Clear* does: clears `Wash_Out` (The step's outputs, one bit each: 0 Enter, 1 Conveyor, 2 Soap, 3 Brushes, 4 Rinse, 5 Wax, 6 Dryer, 7 Exit, 8 Dwell) to 0.

> **Think of it like this:** a rung is a pipe with valves in it, read left to right. Every contact is a valve: XIC is open while its bit is 1, XIO while its bit is 0. Water reaches the output at the right only if every valve in a row is open. The output does not remember anything: on every scan it is simply wet or dry.

INIT · RUNG 1




```
XIC(S:FS)
[MOV(2#0_0000_0000,Out_Table[0]),MOV(2#0_0000_0000,In_Table[0]),MOV(0,Dwell[0])
];
```

THE STEP TABLE, one rung a step: its outputs, the inputs it waits for, its dwell. In a plant you would type these into the tags once; here Init writes them on the first scan so you can read them in the ladder. STEP 0, HOME: nothing on. Init leaves the sequencer here; the first step on goes to LOAD

HOW THIS RUNG WORKS

In words. When `S:FS` is on, the rung puts `2#0_0000_0000` into `Out_Table[0]`, puts `2#0_0000_0000` into `In_Table[0]` and puts **0** into `Dwell[0]`.

Left to right:

1. `XIC(S:FS)` *Examine If Closed* asks: true when `S:FS` is on.
2. A branch of 3 parallel paths: each path is tested on its own and does its own work.

   Path 1

   1. `MOV(2#0_0000_0000,Out_Table[0])` *Move* does: puts `2#0_0000_0000` into `Out_Table[0]`. It writes every scan the rung is true.

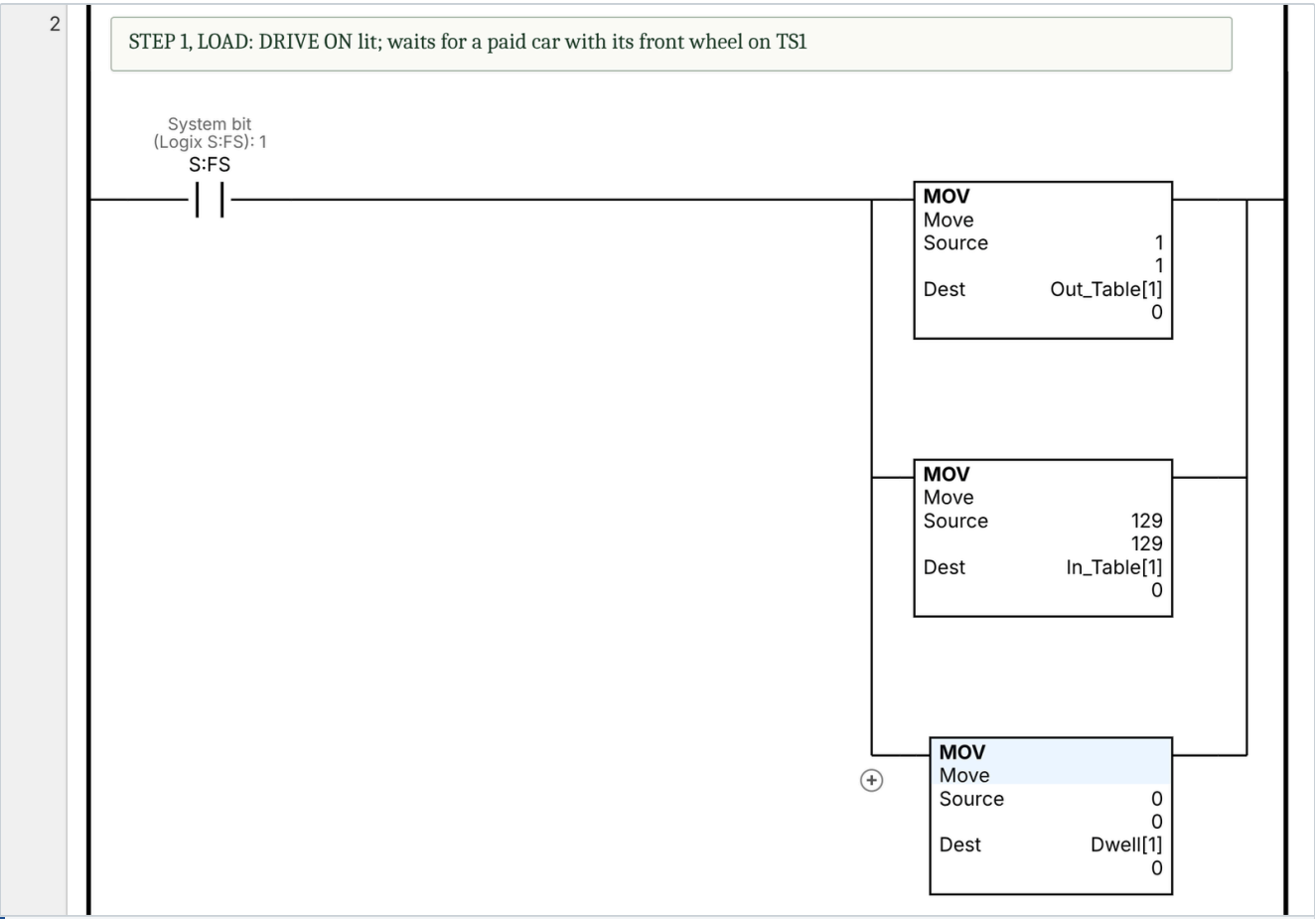
   Path 2

1. `MOV(2#0_0000_0000,In_Table[0])` *Move* does: puts `2#0_0000_0000` into `In_Table[0]`.

Path 3

1. `MOV(0,Dwell[0])` *Move* does: puts **0** into `Dwell[0]`.

INIT · RUNG 2




```
XIC(S:FS)
[MOV(2#0_0000_0001,Out_Table[1]),MOV(2#0_1000_0001,In_Table[1]),MOV(0,Dwell[1])
];
```

STEP 1, LOAD: DRIVE ON lit; waits for a paid car with its front wheel on TS1

HOW THIS RUNG WORKS

In words. When `S:FS` is on, the rung puts `2#0_0000_0001` into `Out_Table[1]`, puts `2#0_1000_0001` into `In_Table[1]` and puts **0** into `Dwell[1]`.

Left to right:

1. `XIC(S:FS)` *Examine If Closed* asks: true when `S:FS` is on.
2. A branch of 3 parallel paths: each path is tested on its own and does its own work.

   Path 1

   1. `MOV(2#0_0000_0001,Out_Table[1])` *Move* does: puts `2#0_0000_0001` into `Out_Table[1]`.

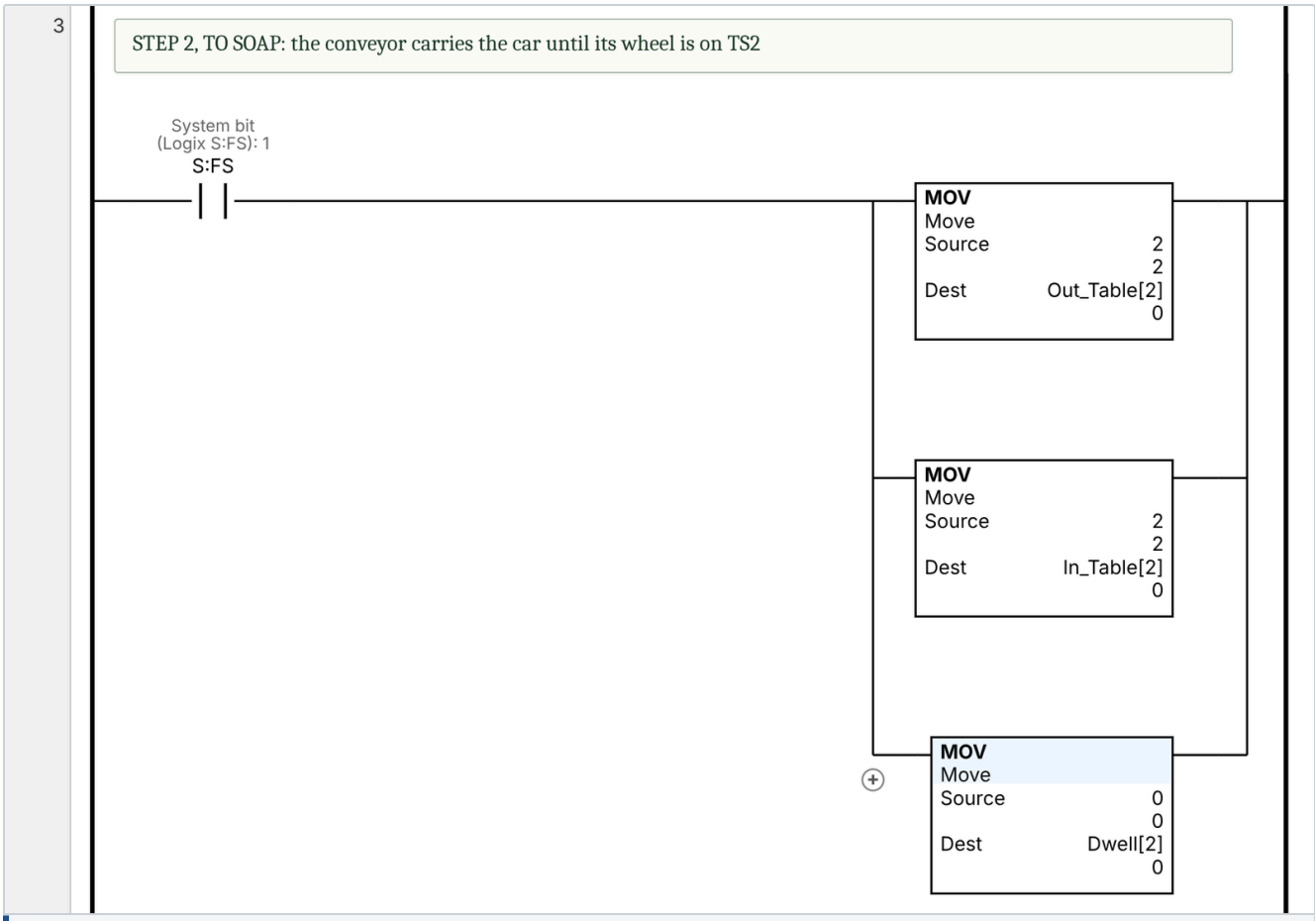
   Path 2

   1. `MOV(2#0_1000_0001,In_Table[1])` *Move* does: puts `2#0_1000_0001` into `In_Table[1]`.

   Path 3

   1. `MOV(0,Dwell[1])` *Move* does: puts **0** into `Dwell[1]`.

INIT · RUNG 3




```
XIC(S:FS)
[MOV(2#0_0000_0010,Out_Table[2]),MOV(2#0_0000_0010,In_Table[2]),MOV(0,Dwell[2])
];
```

STEP 2, TO SOAP: the conveyor carries the car until its wheel is on TS2

HOW THIS RUNG WORKS

In words. When `S:FS` is on, the rung puts `2#0_0000_0010` into `Out_Table[2]`, puts `2#0_0000_0010` into `In_Table[2]` and puts **0** into `Dwell[2]`.

Left to right:

1. `XIC(S:FS)` *Examine If Closed* asks: true when `S:FS` is on.
2. A branch of 3 parallel paths: each path is tested on its own and does its own work.

Path 1

1. `MOV(2#0_0000_0010,Out_Table[2])` *Move* does: puts `2#0_0000_0010` into `Out_Table[2]`.

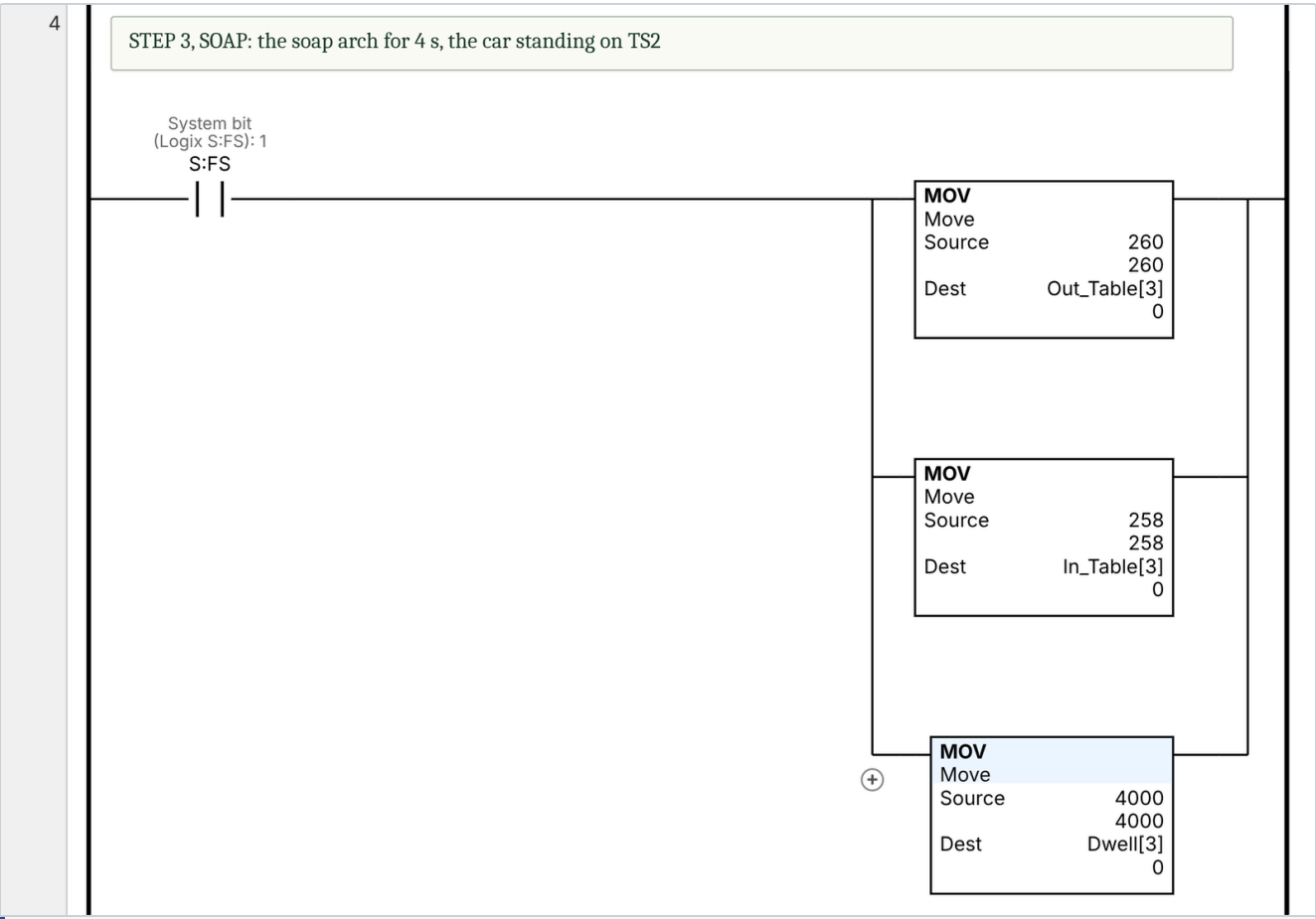
Path 2

1. `MOV(2#0_0000_0010,In_Table[2])` *Move* does: puts `2#0_0000_0010` into `In_Table[2]`.

Path 3

1. `MOV(0,Dwell[2])` *Move* does: puts **0** into `Dwell[2]`.

INIT · RUNG 4




```
XIC(S:FS)
[MOV(2#1_0000_0100,Out_Table[3]),MOV(2#1_0000_0010,In_Table[3]),MOV(4000,Dwell[
3])];
```

STEP 3, SOAP: the soap arch for 4 s, the car standing on TS2

HOW THIS RUNG WORKS

In words. When `S:FS` is on, the rung puts `2#1_0000_0100` into `Out_Table[3]`, puts `2#1_0000_0010` into `In_Table[3]` and puts **4000** into `Dwell[3]`.

Left to right:

1. `XIC(S:FS)` *Examine If Closed* asks: true when `S:FS` is on.
2. A branch of 3 parallel paths: each path is tested on its own and does its own work.

Path 1

1. `MOV(2#1_0000_0100,Out_Table[3])` *Move* does: puts `2#1_0000_0100` into `Out_Table[3]`.

Path 2

1. `MOV(2#1_0000_0010,In_Table[3])` *Move* does: puts `2#1_0000_0010` into `In_Table[3]`.

Path 3

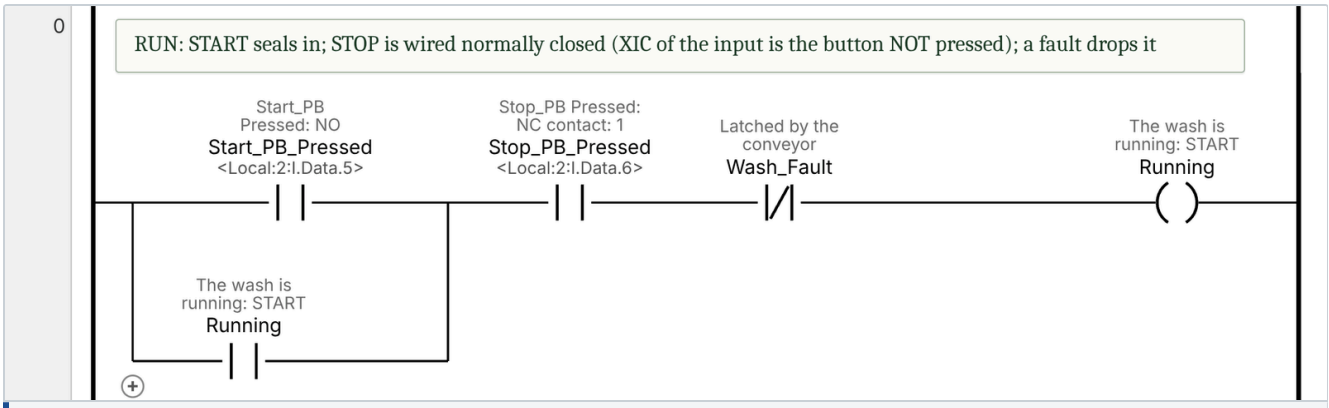
1. `MOV(4000,Dwell[3])` *Move* does: puts **4000** into `Dwell[3]`.

…and so on down to EXIT; the full table is above.

> **WATCH FOR: ELEMENT 0**
>
> SQO never writes element 0 while it walks: it steps POS from 1 to LEN and wraps back to 1. Element 0 is the HOME row, the one Init and RESET leave you in. A table with a real step in element 0 would run it once, after a reset, and never again.

Exercise 2: run, stop and reset

RUN · RUNG 0




```
[XIC(Start_PB_Pressed),XIC(Running)]XIC(Stop_PB_Pressed)XIO(Wash_Fault)OTE(Running);
```

RUN: START seals in; STOP is wired normally closed (XIC of the input is the button NOT pressed); a fault drops it

PREDICT IT · 1 `Start_PB_Pressed` is on, `Stop_PB_Pressed` is off and `Wash_Fault` is off. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When either (`Start_PB_Pressed` (NO contact: 1 while pressed) is on) *or* (`Running` (The wash is running: START seals it in, STOP (normally closed) or a fault drops it) is on), `Stop_PB_Pressed` (NC contact: 1 until pressed) is on and `Wash_Fault` (Latched by the conveyor watchdog or the brush overload; RESET clears it) is off, the rung turns `Running` on (and off again whenever the rung is false).

Left to right:

1. A branch of 2 parallel paths: power gets through if any one of them is true.
   - **Path 1**
     1. `XIC(Start_PB_Pressed)` *Examine If Closed* asks: true when `Start_PB_Pressed` (NO contact: 1 while pressed) is on.
   - **Path 2**
     1. `XIC(Running)` *Examine If Closed* asks: true when `Running` (The wash is running: START seals it in, STOP (normally closed) or a fault drops it) is on.
2. `XIC(Stop_PB_Pressed)` *Examine If Closed* asks: true when `Stop_PB_Pressed` (NC contact: 1 until pressed) is on.
3. `XIO(Wash_Fault)` *Examine If Open* asks: true when `Wash_Fault` (Latched by the conveyor watchdog or the brush overload; RESET clears it) is off.
4. `OTE(Running)` *Output Energize* does: turns `Running` on (and off again whenever the rung is false). Every scan it writes the rung state: true on, false off.

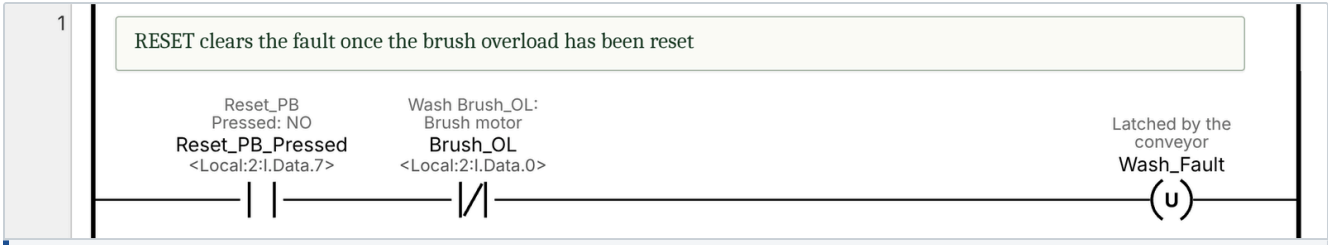
Why it is built this way:

Seal-in. The start condition turns `Running` on, and `Running` 's own contact in the parallel path keeps the rung true after the start goes away. Anything in series outside the branch (a stop, a fault) breaks it, and then it stays off until the start is made again.

> **Think of it like this:** a doorbell button and a hand. The start button is the doorbell, pressed for a moment. The seal-in contact is a hand that grabs the circuit as soon as the output comes on and keeps holding after you let go. The stop button, in series, is the only thing that can knock that hand away.

> **Watch for:** The stop has to be in series outside the branch. Put it inside the start path only and the seal-in path bypasses it: the stop no longer stops.

RUN · RUNG 1




```
XIC(Reset_PB_Pressed)XIO(Brush_OL)OTU(Wash_Fault);
```

RESET clears the fault once the brush overload has been reset

PREDICT IT · 2 `Reset_PB_Pressed` is on and `Brush_OL` is off. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

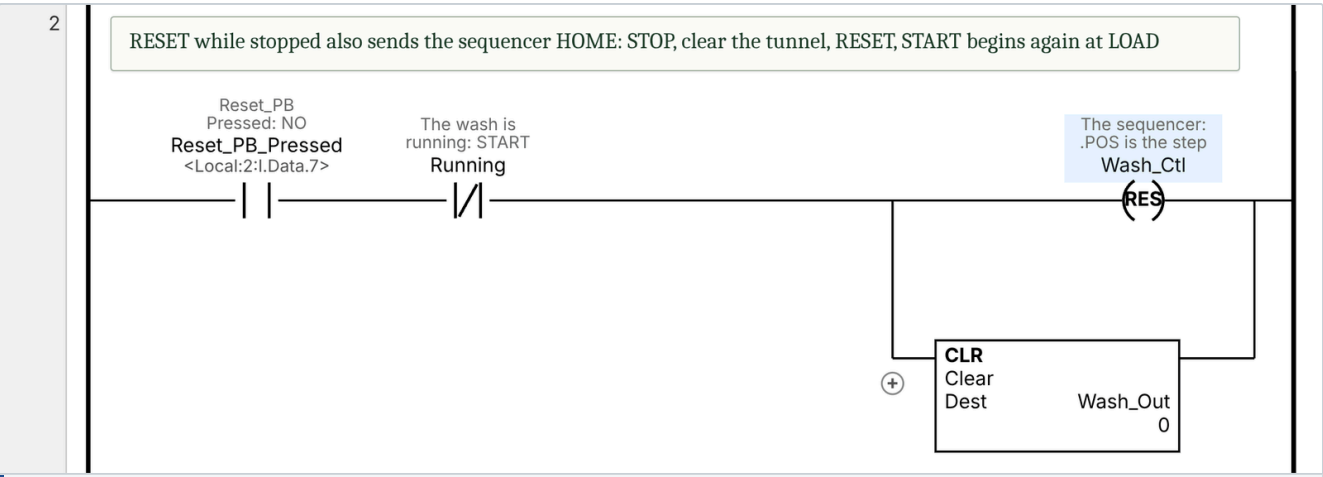
HOW THIS RUNG WORKS

In words. When `Reset_PB_Pressed` (NO contact: 1 while pressed) is on and `Brush_OL` (Brush motor overload relay: 1 when it has tripped) is off, the rung unlatches `Wash_Fault` (Latched by the conveyor watchdog or the brush overload; RESET clears it) (turns it off).

Left to right:

1. `XIC(Reset_PB_Pressed)` *Examine If Closed* asks: true when `Reset_PB_Pressed` (NO contact: 1 while pressed) is on.
2. `XIO(Brush_OL)` *Examine If Open* asks: true when `Brush_OL` (Brush motor overload relay: 1 when it has tripped) is off.
3. `OTU(Wash_Fault)` *Output Unlatch* does: unlatches `Wash_Fault` (Latched by the conveyor watchdog or the brush overload; RESET clears it) (turns it off).

RUN · RUNG 2




```
XIC(Reset_PB_Pressed)XIO(Running)[RES(Wash_Ctl),CLR(Wash_Out)];
```

RESET while stopped also sends the sequencer HOME: STOP, clear the tunnel, RESET, START begins again at LOAD

HOW THIS RUNG WORKS

In words. When `Reset_PB_Pressed` (NO contact: 1 while pressed) is on and `Running` (The wash is running: START seals it in, STOP (normally closed) or a fault drops it) is off, the rung resets `Wash_Ctl` (The sequencer: .POS is the step (0 HOME, 1 LOAD ... 13 EXIT), .LEN 13; SQO moves it, SQI reads it) and clears `Wash_Out` (The step's outputs, one bit each: 0 Enter, 1 Conveyor, 2 Soap, 3 Brushes, 4 Rinse, 5 Wax, 6 Dryer, 7 Exit, 8 Dwell) to 0.

Left to right:

1. `XIC(Reset_PB_Pressed)` *Examine If Closed* asks: true when `Reset_PB_Pressed` (NO contact: 1 while pressed) is on.
2. `XIO(Running)` *Examine If Open* asks: true when `Running` (The wash is running: START seals it in, STOP (normally closed) or a fault drops it) is off.
3. A branch of 2 parallel paths: each path is tested on its own and does its own work.

   Path 1

   1. `RES(Wash_Ctl)` *Reset* does: resets `Wash_Ctl` (The sequencer: .POS is the step (0 HOME, 1 LOAD ... 13 EXIT), .LEN 13; SQO moves it, SQI reads it).

   Path 2

   1. `CLR(Wash_Out)` *Clear* does: clears `Wash_Out` (The step's outputs, one bit each: 0 Enter, 1 Conveyor, 2 Soap, 3 Brushes, 4 Rinse, 5 Wax, 6 Dryer, 7 Exit, 8 Dwell) to 0.

Exercise 3: SQI looks, SQO steps

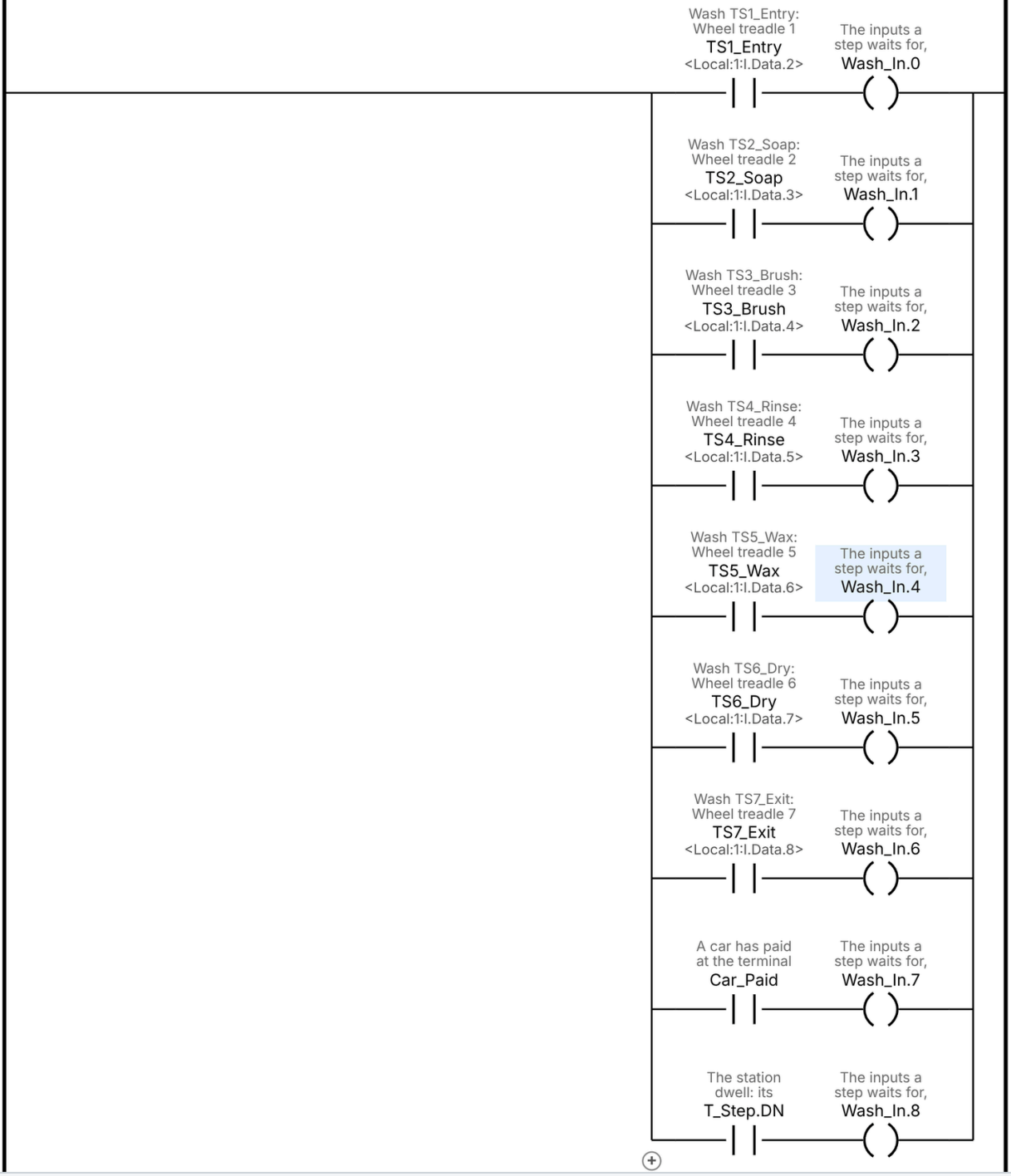
This is the routine the chapter is about. The first rung gathers the inputs into `Wash_In`, one bit each, because SQI compares one word. The second asks SQI whether that word, through the mask, equals the step's row of `In_Table`; SQI reads the step from the same CONTROL the SQO uses and never moves it. The mask, `16#01FF`, is nine ones: compare all nine bits. If they match, `Step_Done`. Then SQO: on the

change of its rung from false to true it steps the position on and copies the new row of `Out_Table` into `Wash_Out`, through its own mask. Last, the dwell: the step’s time from `Dwell[Wash_Ctl.POS]` into the timer’s preset, and the timer runs while the step’s Dwell bit is on.

SEQUENCE · RUNG 0

0

THE INPUT WORD. SQI compares one word, so the inputs a step can wait for are gathered into one: the seven treadles, the payment, the dwell timer




```
[XIC(TS1_Entry)OTE(Wash_In.0),XIC(TS2_Soap)OTE(Wash_In.1),XIC(TS3_Brush)OTE(Was
h_In.2),XIC(TS4_Rinse)OTE(Wash_In.3),XIC(TS5_Wax)OTE(Wash_In.4),XIC(TS6_Dry)OTE
(Wash_In.5),XIC(TS7_Exit)OTE(Wash_In.6),XIC(Car_Paid)OTE(Wash_In.7),XIC(T_Step.
DN)OTE(Wash_In.8)];
```

THE INPUT WORD. SQI compares one word, so the inputs a step can wait for are gathered into one: the seven treadles, the payment, the dwell timer

PREDICT IT · 3 `Car_Paid` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. Every scan, with nothing to test, the rung picks its path: if `TS1_Entry` (Wheel treadle 1 (ENTRY): the front wheel stands on it) is on, it turns `Wash_In.0` on (and off again whenever the rung is false); if `TS2_Soap` (Wheel treadle 2 (SOAP): the front wheel stands on it) is on, it turns `Wash_In.1` on (and off again whenever the rung is false); if `TS3_Brush` (Wheel treadle 3 (BRUSH): the front wheel stands on it) is on, it turns `Wash_In.2` on (and off again whenever the rung is false); if `TS4_Rinse` (Wheel treadle 4 (RINSE): the front wheel stands on it) is on, it turns `Wash_In.3` on (and off again whenever the rung is false); if `TS5_Wax` (Wheel treadle 5 (WAX): the front wheel stands on it) is on, it turns `Wash_In.4` on (and off again whenever the rung is false); if `TS6_Dry` (Wheel treadle 6 (DRY): the front wheel stands on it) is on, it turns `Wash_In.5` on (and off again whenever the rung is false); if `TS7_Exit` (Wheel treadle 7 (EXIT): the front wheel stands on it) is on, it turns `Wash_In.6` on (and off again whenever the rung is false); if `Car_Paid` (A car has paid at the terminal and may come on (only while the sequencer is at LOAD)) is on, it turns `Wash_In.7` on (and off again whenever the rung is false); if `T_Step.DN` (the done bit of `T_Step` : The station dwell: its preset comes from Dwell[] for the step) is on, it turns `Wash_In.8` on (and off again whenever the rung is false).

Left to right:

1. A branch of 9 parallel paths: each path is tested on its own and does its own work.

   Path 1

   1. `XIC(TS1_Entry)` *Examine If Closed* asks: true when `TS1_Entry` (Wheel treadle 1 (ENTRY): the front wheel stands on it) is on.
   2. `OTE(Wash_In.0)` *Output Energize* does: turns `Wash_In.0` on (and off again whenever the rung is false).

   Path 2

   1. `XIC(TS2_Soap)` *Examine If Closed* asks: true when `TS2_Soap` (Wheel treadle 2 (SOAP): the front wheel stands on it) is on.
   2. `OTE(Wash_In.1)` *Output Energize* does: turns `Wash_In.1` on (and off again whenever the rung is false).

   Path 3

   1. `XIC(TS3_Brush)` *Examine If Closed* asks: true when `TS3_Brush` (Wheel treadle 3 (BRUSH): the front wheel stands on it) is on.
   2. `OTE(Wash_In.2)` *Output Energize* does: turns `Wash_In.2` on (and off again whenever the rung is false).

   Path 4

   1. `XIC(TS4_Rinse)` *Examine If Closed* asks: true when `TS4_Rinse` (Wheel treadle 4 (RINSE): the front wheel stands on it) is on.
   2. `OTE(Wash_In.3)` *Output Energize* does: turns `Wash_In.3` on (and off again whenever the rung is false).

   Path 5

   1. `XIC(TS5_Wax)` *Examine If Closed* asks: true when `TS5_Wax` (Wheel treadle 5 (WAX): the front wheel stands on it) is on.
   2. `OTE(Wash_In.4)` *Output Energize* does: turns `Wash_In.4` on (and off again whenever the rung is false).

   Path 6

   1. `XIC(TS6_Dry)` *Examine If Closed* asks: true when `TS6_Dry` (Wheel treadle 6 (DRY): the front wheel stands on it) is on.

2. `OTE(Wash_In.5)` *Output Energize* does: turns `Wash_In.5` on (and off again whenever the rung is false).

Path 7

1. `XIC(TS7_Exit)` *Examine If Closed* asks: true when `TS7_Exit` (Wheel treadle 7 (EXIT): the front wheel stands on it) is on.
2. `OTE(Wash_In.6)` *Output Energize* does: turns `Wash_In.6` on (and off again whenever the rung is false).

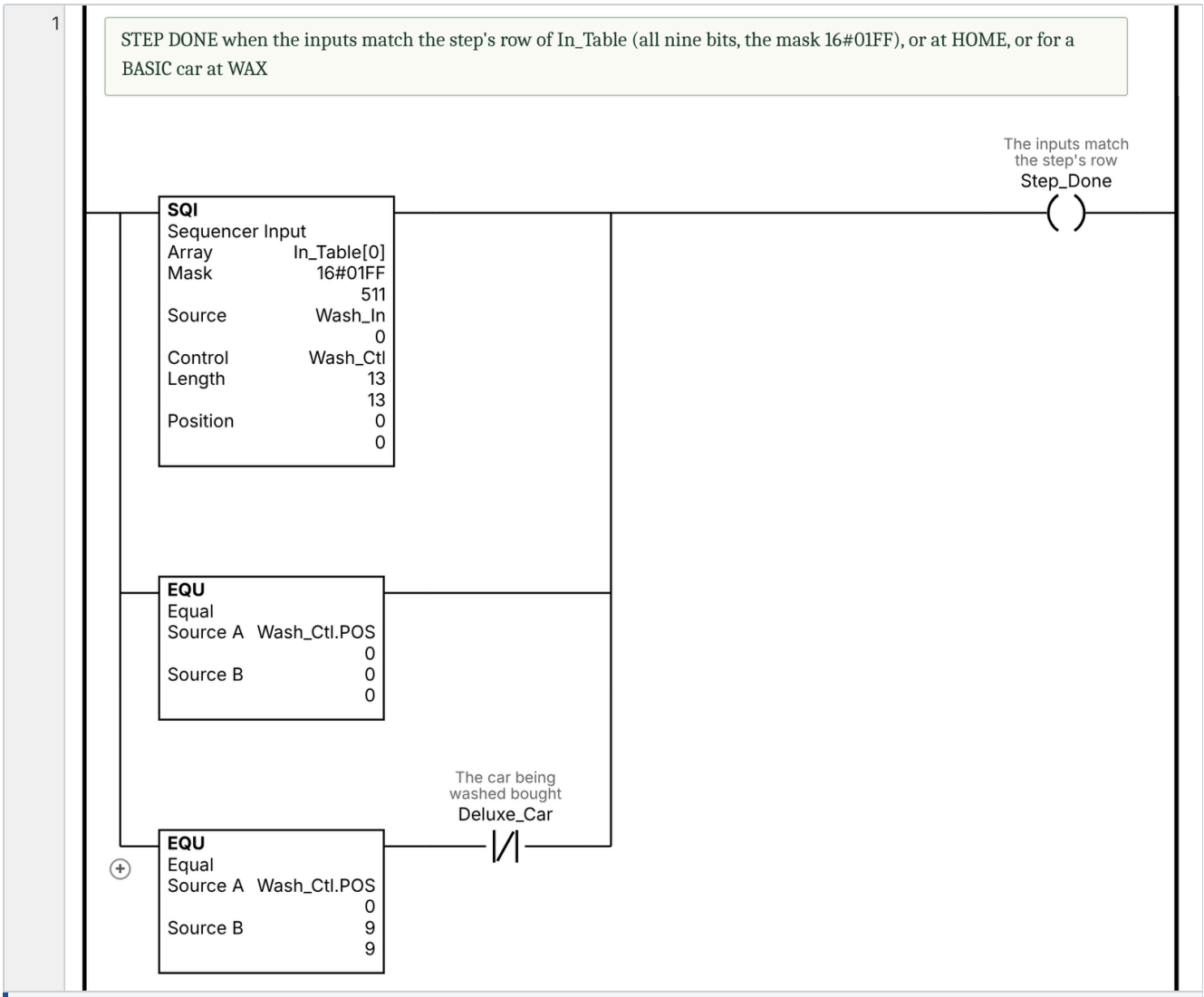
Path 8

1. `XIC(Car_Paid)` *Examine If Closed* asks: true when `Car_Paid` (A car has paid at the terminal and may come on (only while the sequencer is at LOAD)) is on.
2. `OTE(Wash_In.7)` *Output Energize* does: turns `Wash_In.7` on (and off again whenever the rung is false).

Path 9

1. `XIC(T_Step.DN)` *Examine If Closed* asks: true when `T_Step.DN` (the done bit of `T_Step` : The station dwell: its preset comes from Dwell[] for the step) is on.
2. `OTE(Wash_In.8)` *Output Energize* does: turns `Wash_In.8` on (and off again whenever the rung is false).

SEQUENCE · RUNG 1




```
[SQI(In_Table[0],16#01FF,Wash_In,Wash_Ctl,13,0),EQU(Wash_Ctl.POS,0),EQU(Wash_Ctl.POS,9)XIO(Deluxe_Car)]OTE(Step_Done);
```

STEP DONE when the inputs match the step's row of In_Table (all nine bits, the mask 16#01FF), or at HOME, or for a BASIC car at WAX

HOW THIS RUNG WORKS

In words. When either (`Wash_In` (The inputs a step waits for, gathered into one word: bits 0-6 the treadles TS1-TS7, 7 Car_Paid, 8 the dwell timer done), through the mask 16#01FF, equals the row of `In_Table[0]` at the step in `Wash_Ctl` (The sequencer: .POS is the step (0 HOME, 1 LOAD ... 13 EXIT), .LEN 13; SQO moves it, SQI reads it)) *or* (`Wash_Ctl.POS` (the position of `Wash_Ctl`) equals **0**) *or* (`Wash_Ctl.POS` (the position of `Wash_Ctl`) equals **9** and `Deluxe_Car` (The car being washed bought DELUXE: the wax arch runs for it) is off), the rung turns `Step_Done` (The inputs match the step's row (or HOME, or a BASIC car at WAX): step on) on (and off again whenever the rung is false).

Left to right:

1. A branch of 3 parallel paths: power gets through if any one of them is true.

   Path 1

1. `SQI(In_Table[0],16#01FF,Wash_In,Wash_Ctl,13,0)` *Sequencer Input* asks: true when `Wash_In` (The inputs a step waits for, gathered into one word: bits 0-6 the treadles TS1-TS7, 7 Car_Paid, 8 the dwell timer done), through the mask 16#01FF, equals the row of `In_Table[0]` at the step in `Wash_Ctl` (The sequencer: .POS is the step (0 HOME, 1 LOAD ... 13 EXIT), .LEN 13; SQO moves it, SQI reads it). It reads the position from the control and never moves it.

Path 2

1. `EQU(Wash_Ctl.POS,0)` *Equal* asks: true when `Wash_Ctl.POS` (the position of `Wash_Ctl`) equals **0**. Both values are compared every scan.

Path 3

1. `EQU(Wash_Ctl.POS,9)` *Equal* asks: true when `Wash_Ctl.POS` (the position of `Wash_Ctl`) equals **9**.
2. `XIO(Deluxe_Car)` *Examine If Open* asks: true when `Deluxe_Car` (The car being washed bought DELUXE: the wax arch runs for it) is off.

2. `OTE(Step_Done)` *Output Energize* does: turns `Step_Done` (The inputs match the step's row (or HOME, or a BASIC car at WAX): step on) on (and off again whenever the rung is false).

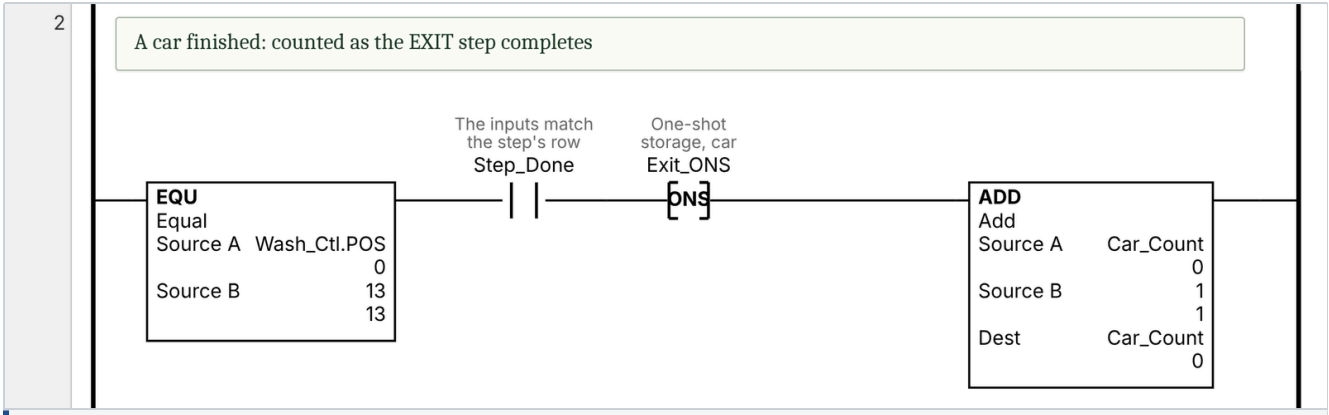
Why it is built this way:

A sequencer. The order of the machine is a table, one row a step; SQI asks whether the step's inputs have come, SQO turns the table on one row and writes its outputs. Change the recipe and you change numbers, not rungs.

> **Think of it like this:** a player piano: the roll has a row of holes for every moment and a hole for each key; the piano plays whatever row is under the head, and turning the roll on is all it ever decides.

> **Watch for:** SQO steps on a transition. A step that is complete the moment it starts needs the rung broken for a scan (XIO of the control's .EN), or the sequencer stops there.

SEQUENCE · RUNG 2




```
EQU(Wash_Ctl.POS,13)XIC(Step_Done)ONS(Exit_ONS)ADD(Car_Count,1,Car_Count);
```

A car finished: counted as the EXIT step completes

HOW THIS RUNG WORKS

In words. When `Wash_Ctl.POS` (the position of `Wash_Ctl`: The sequencer: .POS is the step (0 HOME, 1 LOAD ... 13 EXIT), .LEN 13; SQO moves it, SQI reads it) equals **13**, `Step_Done` (The inputs match the step's row (or HOME, or a BASIC car at WAX): step on) is on and it is the first scan they are true (the one-shot, remembered in `Exit_ONS` (One-shot storage, car counted out)), the rung makes `Car_Count` (Cars washed) = `Car_Count` + **1**.

Left to right:

1. `EQU(Wash_Ctl.POS,13)` *Equal* asks: true when `Wash_Ctl.POS` (the position of `Wash_Ctl`: The sequencer: .POS is the step (0 HOME, 1 LOAD ... 13 EXIT), .LEN 13; SQO moves it, SQI reads it) equals **13**.
2. `XIC(Step_Done)` *Examine If Closed* asks: true when `Step_Done` (The inputs match the step's row (or HOME, or a BASIC car at WAX): step on) is on.
3. `ONS(Exit_ONS)` *One Shot* asks: true when it is the first scan they are true (the one-shot, remembered in `Exit_ONS` (One-shot storage, car counted out)). It passes power for one scan only, the scan its input goes from false to true; its storage bit remembers last scan.
4. `ADD(Car_Count,1,Car_Count)` *Add* does: makes `Car_Count` (Cars washed) = `Car_Count` + **1**. Every scan the rung is true; behind a one-shot it adds once per event.

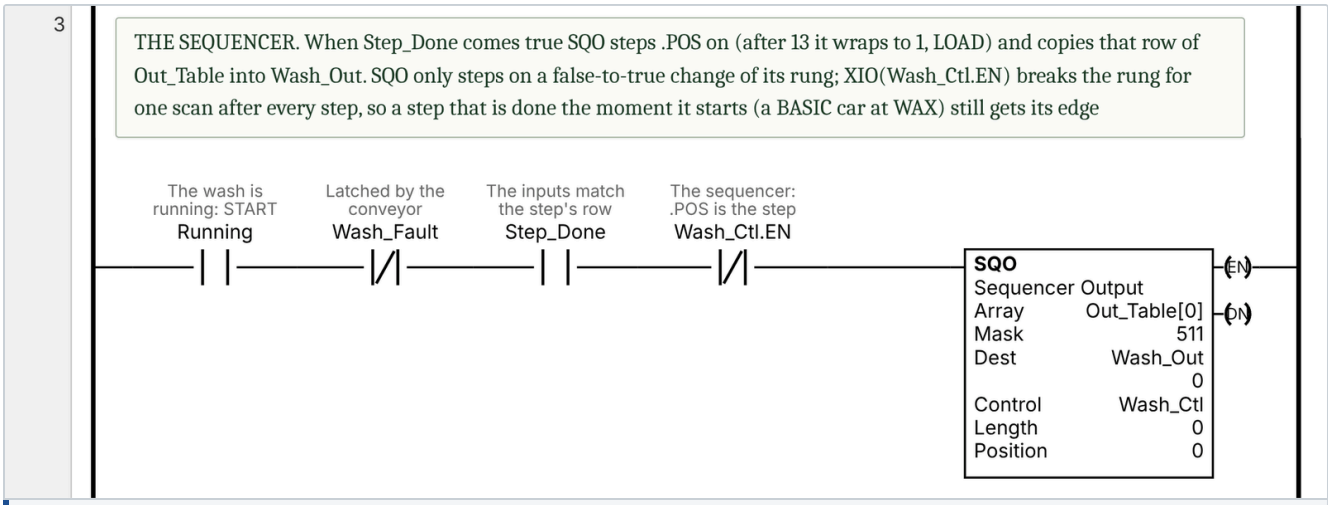
Why it is built this way:

One-shot. Everything after the ONS happens once per event, not on every scan the condition holds; `Exit_ONS` is only its memory of the last scan.

> **Think of it like this:** a doorbell that rings once however long you lean on the button. The ONS looks at what the button was doing last scan: only the change from up to down rings it.

> **Watch for:** Each ONS needs its own storage bit. Two ONS sharing one bit make one of them miss its edges.

SEQUENCE · RUNG 3




```
XIC(Running)XIO(Wash_Fault)XIC(Step_Done)XIO(Wash_Ctl.EN)SQO(Out_Table[0],16#01FF,Wash_Out,Wash_Ctl,13,0);
```

THE SEQUENCER. When Step_Done comes true SQO steps .POS on (after 13 it wraps to 1, LOAD) and copies that row of Out_Table into Wash_Out. SQO only steps on a false-to-true change of its rung; XIO(Wash_Ctl.EN) breaks the rung for one scan after every step, so a step that is done the moment it starts (a BASIC car at WAX) still gets its edge

HOW THIS RUNG WORKS

In words. When `Running` (The wash is running: START seals it in, STOP (normally closed) or a fault drops it) is on, `Wash_Fault` (Latched by the conveyor watchdog or the brush overload; RESET clears it) is off, `Step_Done` (The inputs match the step's row (or HOME, or a BASIC car at WAX): step on) is on and `Wash_Ctl.EN` (the enable bit of `Wash_Ctl`: The sequencer: .POS is the step (0 HOME, 1 LOAD ... 13 EXIT), .LEN 13; SQO moves it, SQI reads it) is off, the rung steps `Wash_Ctl` on one row and copies that row of `Out_Table[0]` into `Wash_Out` (The step's outputs, one bit each: 0 Enter, 1 Conveyor, 2 Soap, 3 Brushes, 4 Rinse, 5 Wax, 6 Dryer, 7 Exit, 8 Dwell) (through the mask 16#01FF).

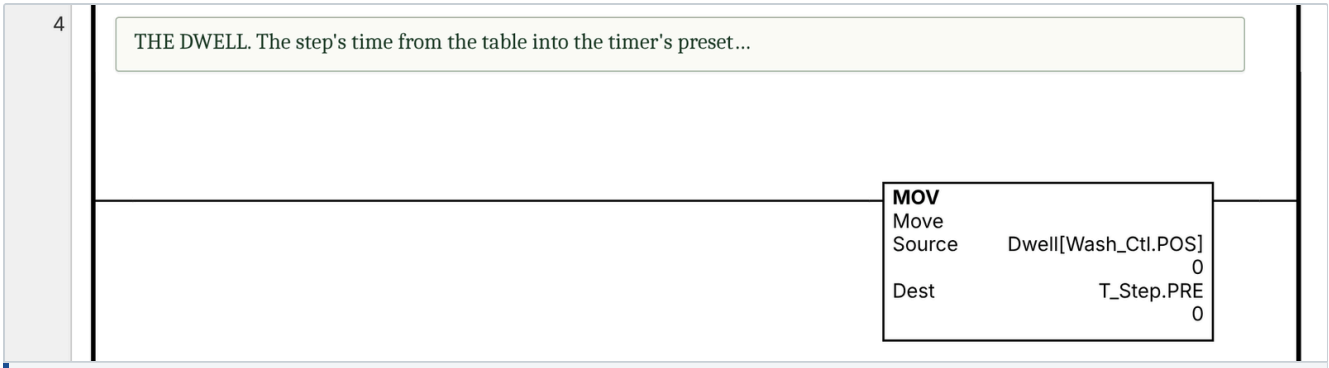
Left to right:

1. `XIC(Running)` *Examine If Closed* asks: true when `Running` (The wash is running: START seals it in, STOP (normally closed) or a fault drops it) is on.
2. `XIO(Wash_Fault)` *Examine If Open* asks: true when `Wash_Fault` (Latched by the conveyor watchdog or the brush overload; RESET clears it) is off.
3. `XIC(Step_Done)` *Examine If Closed* asks: true when `Step_Done` (The inputs match the step's row (or HOME, or a BASIC car at WAX): step on) is on.
4. `XIO(Wash_Ctl.EN)` *Examine If Open* asks: true when `Wash_Ctl.EN` (the enable bit of `Wash_Ctl`: The sequencer: .POS is the step (0 HOME, 1 LOAD ... 13 EXIT), .LEN 13; SQO moves it, SQI reads it) is off.
5. `SQO(Out_Table[0],16#01FF,Wash_Out,Wash_Ctl,13,0)` *Sequencer Output* does: steps `Wash_Ctl` on one row and copies that row of `Out_Table[0]` into `Wash_Out` (The step's outputs, one bit each: 0 Enter, 1 Conveyor, 2 Soap, 3 Brushes, 4 Rinse, 5 Wax, 6 Dryer, 7 Exit, 8 Dwell) (through the mask 16#01FF). One step per false-to-true transition of the rung; after the last step it wraps to step 1.

Why it is built this way:

A sequencer: the table holds the order.

SEQUENCE · RUNG 4




```
MOV(Dwell[Wash_Ctl.POS],T_Step.PRE);
```

THE DWELL. The step's time from the table into the timer's preset...

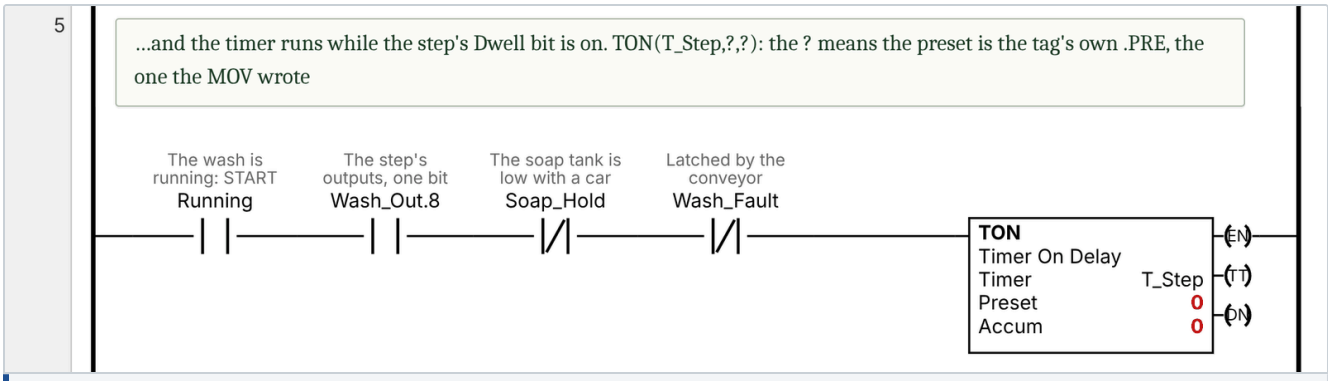
HOW THIS RUNG WORKS

In words. Every scan, with nothing to test, the rung puts `Dwell[Wash_Ctl.POS]` into `T_Step.PRE` (the preset of `T_Step` : The station dwell: its preset comes from Dwell[] for the step).

Left to right:

1. `MOV(Dwell[Wash_Ctl.POS],T_Step.PRE)` *Move* does: puts `Dwell[Wash_Ctl.POS]` into `T_Step.PRE` (the preset of `T_Step` : The station dwell: its preset comes from Dwell[] for the step).

SEQUENCE · RUNG 5




```
XIC(Running)XIC(Wash_Out.8)XIO(Soap_Hold)XIO(Wash_Fault)TON(T_Step,?,?);
```

...and the timer runs while the step's Dwell bit is on. TON(T_Step,?,?): the ? means the preset is the tag's own .PRE, the one the MOV wrote

PREDICT IT · 4 `Running` is on, `Wash_Out.8` is on, `Soap_Hold` is on and `Wash_Fault` is off. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Running` (The wash is running: START seals it in, STOP (normally closed) or a fault drops it) is on, `Wash_Out.8` is on, `Soap_Hold` (The soap tank is low with a car at SOAP: the dwell waits and the pump stays off until it is filled) is off and `Wash_Fault` (Latched by the conveyor watchdog or the brush overload; RESET clears it) is off, the rung runs the timer `T_Step` (The station dwell: its preset comes from Dwell[] for the step) toward its own preset (whatever is in its .PRE).

Left to right:

1. `XIC(Running)` *Examine If Closed* asks: true when `Running` (The wash is running: START seals it in, STOP (normally closed) or a fault drops it) is on.
2. `XIC(Wash_Out.8)` *Examine If Closed* asks: true when `Wash_Out.8` is on.
3. `XIO(Soap_Hold)` *Examine If Open* asks: true when `Soap_Hold` (The soap tank is low with a car at SOAP: the dwell waits and the pump stays off until it is filled) is off.
4. `XIO(Wash_Fault)` *Examine If Open* asks: true when `Wash_Fault` (Latched by the conveyor watchdog or the brush overload; RESET clears it) is off.
5. `TON(T_Step,?,?)` *Timer On Delay* does: runs the timer `T_Step` (The station dwell: its preset comes from Dwell[] for the step) toward its own preset (whatever is in its .PRE). The accumulator counts while the rung is true; .DN comes on at the preset; the moment the rung goes false the timer resets.

Why it is built this way:

A timer. `T_Step` only times while this rung is true; other rungs use its `.DN` (done) or `.TT` (timing) bits. Being a TON, it starts again from zero every time the rung goes false.

> **Think of it like this:** a kitchen timer that only runs while you hold your finger on it. Take your finger off and it snaps back to zero. Hold it long enough and the done light comes on, and stays on as long as your finger is still there.

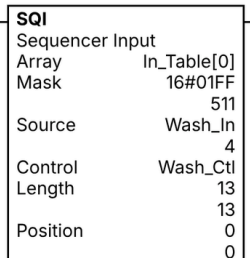
> **Watch for:** If anything in series flickers false for one scan, a TON starts over from zero. Use RTO when the time must survive an interruption.

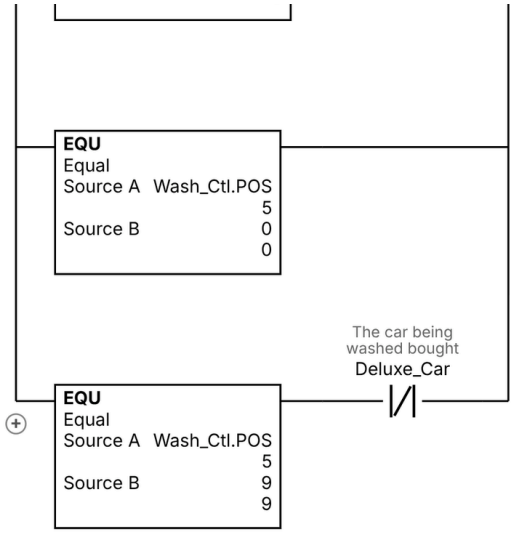
0

THE INPUT WORD. SQI compares one word, so the inputs a step can wait for are gathered into one: the seven treadles, the payr

1

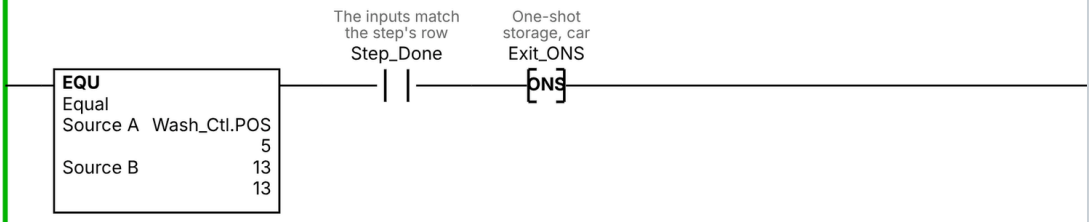
STEP DONE when the inputs match the step's row of In_Table (all nine bits, the mask 16#01FF), or at HOME, or for a BASIC car





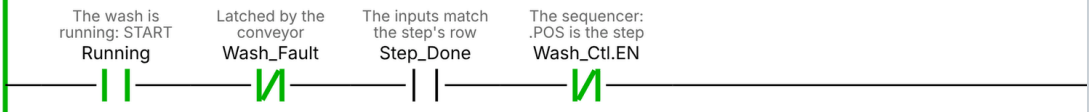
2

A car finished: counted as the EXIT step completes



3

THE SEQUENCER. When Step_Done comes true SQO steps .POS on (after 13 it wraps to 1, LOAD) and copies that row of Out_Tabl
of its rung; XIO(Wash_Ctl.EN) breaks the rung for one scan after every step, so a step that is done the moment it starts (a BASIC



4

THE DWELL. The step's time from the table into the timer's preset…

5

…and the timer runs while the step's Dwell bit is on. TON(T_Step,?,?): the ? means the preset is the tag's own .PRE, the one the

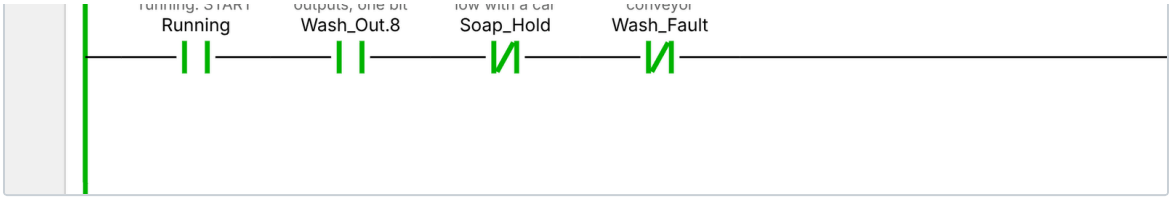


Figure 241. The Sequence routine online: the car standing on TS3 in the BRUSH step (POS 5), the dwell timer running towards its 6 s.

WATCH FOR: A STEP THAT IS DONE THE MOMENT IT STARTS

SQO steps on an edge, not a level. A BASIC car arrives at WAX and the WAX step is done at once (the branch for a car without wax). Without `XIO(Wash_Ctl.EN)` the SQO rung would go from true (TO WAX done) to true (WAX done) without ever being false, SQO would never see another edge, and the car would wait at the wax arch for ever. SQO sets `.EN` when it fires; the XIO breaks the rung for that one scan, and the next edge can come. Take the XIO out and run a BASIC car.

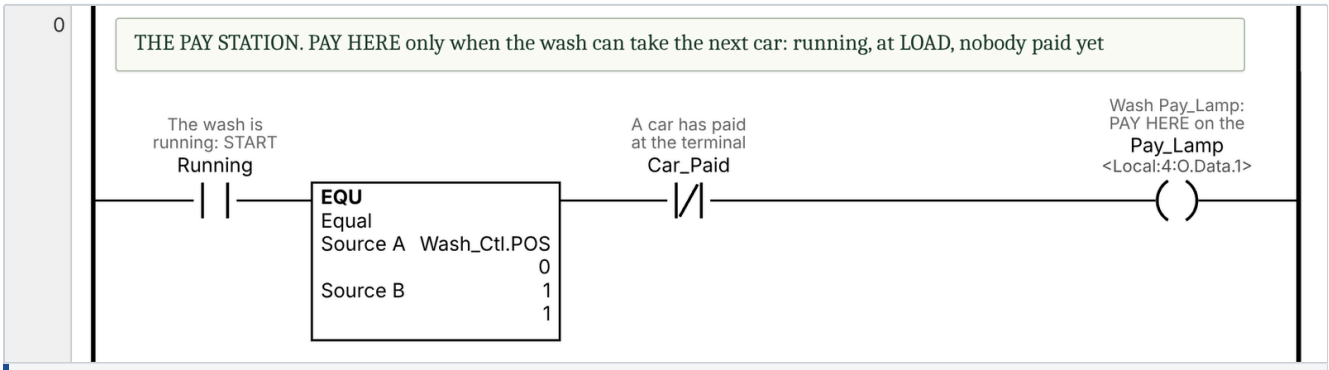
THINK OF IT LIKE THIS: TON(T_STEP,?,?)

A timer's preset is a number on its face. `TON(T_Step,5000,0)` prints 5000 on the face. `TON(T_Step,?,?)`, which is how Logix itself writes it out, says: use whatever is on the face now. The MOV just above paints the step's own time on the face before the timer looks.

Exercise 4: pay, the boom, onto the conveyor

PAY HERE lights only when the wash can take the next car: running, at LOAD, nobody paid yet. The payment latches `Car_Paid`, which is bit 7 of `Wash_In`: LOAD waits for a paid car with its wheel on TS1. Once the sequencer has moved on, `Car_Paid` is cleared so the next car has to pay for itself. The boom goes up for the paid car and down once its front wheel is on TS1, by which time its back bumper is well past the arm.

ENTRY · RUNG 0




```
XIC(Running)EQU(Wash_Ctl.POS,1)XIO(Car_Paid)OTE(Pay_Lamp);
```

THE PAY STATION. PAY HERE only when the wash can take the next car: running, at LOAD, nobody paid yet

PREDICT IT · 5 `Running` is on, `Wash_Ctl.POS` (the position of `Wash_Ctl`: The sequencer: .POS is the step (0 HOME, 1 LOAD ... 13 EXIT), .LEN 13; SQO moves it, SQI reads it) is 1 and `Car_Paid` is off. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

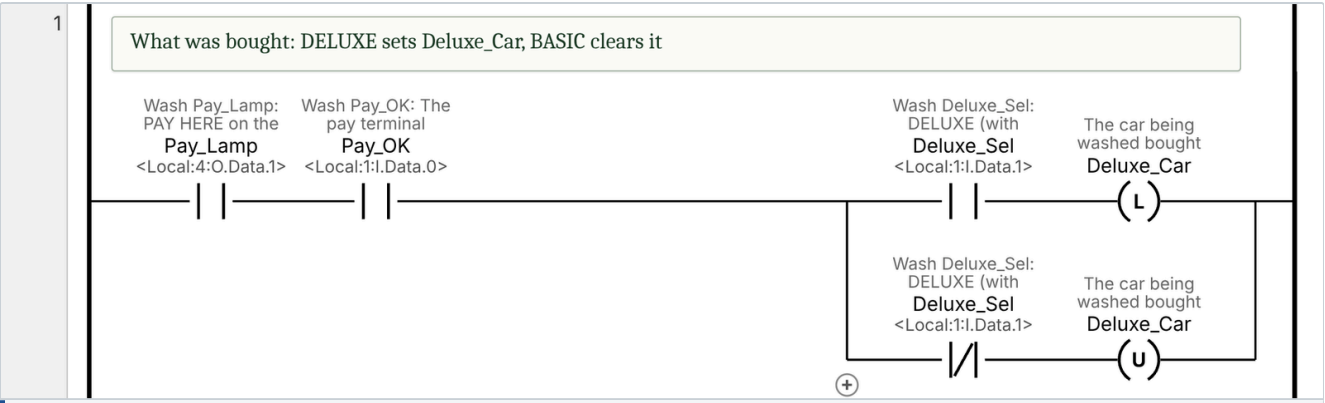
HOW THIS RUNG WORKS

In words. When `Running` (The wash is running: START seals it in, STOP (normally closed) or a fault drops it) is on, `Wash_Ctl.POS` (the position of `Wash_Ctl`: The sequencer: .POS is the step (0 HOME, 1 LOAD ... 13 EXIT), .LEN 13; SQO moves it, SQI reads it) equals **1** and `Car_Paid` (A car has paid at the terminal and may come on (only while the sequencer is at LOAD)) is off, the rung turns `Pay_Lamp` (PAY HERE on the pay terminal: the wash will take the next car) on (and off again whenever the rung is false).

Left to right:

1. `XIC(Running)` *Examine If Closed* asks: true when `Running` (The wash is running: START seals it in, STOP (normally closed) or a fault drops it) is on.
2. `EQU(Wash_Ctl.POS,1)` *Equal* asks: true when `Wash_Ctl.POS` (the position of `Wash_Ctl`: The sequencer: .POS is the step (0 HOME, 1 LOAD ... 13 EXIT), .LEN 13; SQO moves it, SQI reads it) equals **1**.
3. `XIO(Car_Paid)` *Examine If Open* asks: true when `Car_Paid` (A car has paid at the terminal and may come on (only while the sequencer is at LOAD)) is off.
4. `OTE(Pay_Lamp)` *Output Energize* does: turns `Pay_Lamp` (PAY HERE on the pay terminal: the wash will take the next car) on (and off again whenever the rung is false).

ENTRY · RUNG 1




```
XIC(Pay_Lamp)XIC(Pay_OK)
[XIC(Deluxe_Sel)OTL(Deluxe_Car),XIO(Deluxe_Sel)OTU(Deluxe_Car)];
```

What was bought: DELUXE sets Deluxe_Car, BASIC clears it

HOW THIS RUNG WORKS

In words. When `Pay_Lamp` (PAY HERE on the pay terminal: the wash will take the next car) is on and `Pay_OK` (The pay terminal took the money: a short pulse) is on, the rung picks its path: if `Deluxe_Sel` (DELUXE (with wax) was bought: on together with Pay_OK) is on, it latches `Deluxe_Car` (The car being washed bought DELUXE: the wax arch runs for it) on; if `Deluxe_Sel` is off, it unlatches `Deluxe_Car` (turns it off).

Left to right:

1. `XIC(Pay_Lamp)` *Examine If Closed* asks: true when `Pay_Lamp` (PAY HERE on the pay terminal: the wash will take the next car) is on.
2. `XIC(Pay_OK)` *Examine If Closed* asks: true when `Pay_OK` (The pay terminal took the money: a short pulse) is on.
3. A branch of 2 parallel paths: each path is tested on its own and does its own work.

   Path 1

   1. `XIC(Deluxe_Sel)` *Examine If Closed* asks: true when `Deluxe_Sel` (DELUXE (with wax) was bought: on together with Pay_OK) is on.
   2. `OTL(Deluxe_Car)` *Output Latch* does: latches `Deluxe_Car` (The car being washed bought DELUXE: the wax arch runs for it) on. It only ever sets the bit; the bit stays on after the rung goes false.

   Path 2

   1. `XIO(Deluxe_Sel)` *Examine If Open* asks: true when `Deluxe_Sel` is off.
   2. `OTU(Deluxe_Car)` *Output Unlatch* does: unlatches `Deluxe_Car` (turns it off).

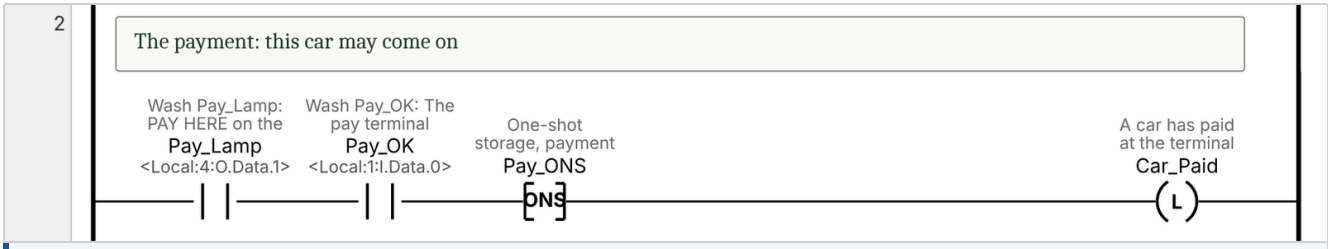
Why it is built this way:

A latch. `Deluxe_Car` stays on after this rung goes false; an `OTU` elsewhere in the program clears it.

> **Think of it like this:** a light switch instead of a doorbell. OTL flips it on and it stays on when you take your hand away; OTU flips it off. Whatever flipped it on does not have to stay there.

> **Watch for:** A latched bit survives the condition that set it: every way in needs a way out, or it stays on for good.

ENTRY · RUNG 2




```
XIC(Pay_Lamp)XIC(Pay_OK)ONS(Pay_ONS)OTL(Car_Paid);
```

The payment: this car may come on

HOW THIS RUNG WORKS

In words. When `Pay_Lamp` (PAY HERE on the pay terminal: the wash will take the next car) is on, `Pay_OK` (The pay terminal took the money: a short pulse) is on and it is the first scan they are true (the one-shot, remembered in `Pay_ONS` (One-shot storage, payment)), the rung latches `Car_Paid` (A car has paid at the terminal and may come on (only while the sequencer is at LOAD)) on.

Left to right:

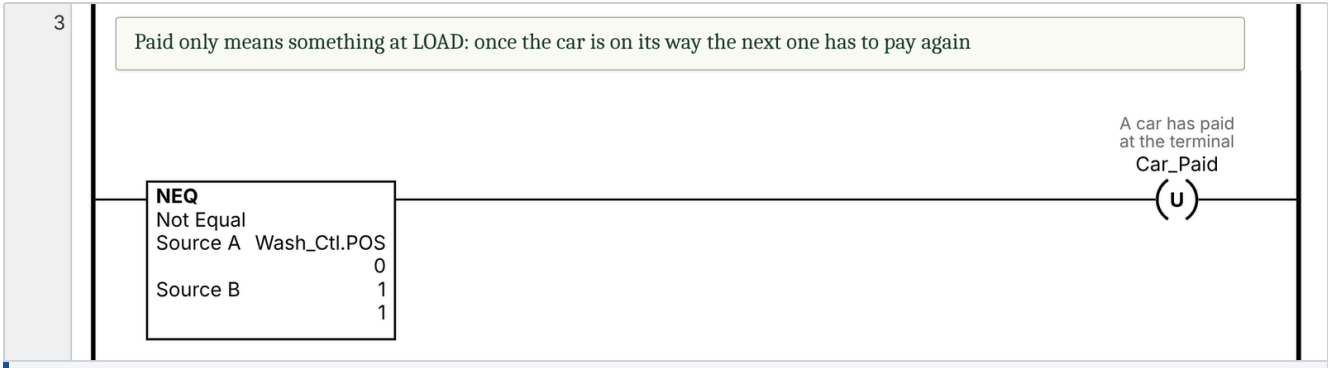
1. `XIC(Pay_Lamp)` *Examine If Closed* asks: true when `Pay_Lamp` (PAY HERE on the pay terminal: the wash will take the next car) is on.
2. `XIC(Pay_OK)` *Examine If Closed* asks: true when `Pay_OK` (The pay terminal took the money: a short pulse) is on.
3. `ONS(Pay_ONS)` *One Shot* asks: true when it is the first scan they are true (the one-shot, remembered in `Pay_ONS` (One-shot storage, payment)).
4. `OTL(Car_Paid)` *Output Latch* does: latches `Car_Paid` (A car has paid at the terminal and may come on (only while the sequencer is at LOAD)) on.

Why it is built this way:

One-shot: once per event, not once per scan.

A latch: it stays on until an OTU clears it.

ENTRY · RUNG 3




```
NEQ(Wash_Ctl.POS,1)OTU(Car_Paid);
```

Paid only means something at LOAD: once the car is on its way the next one has to pay again

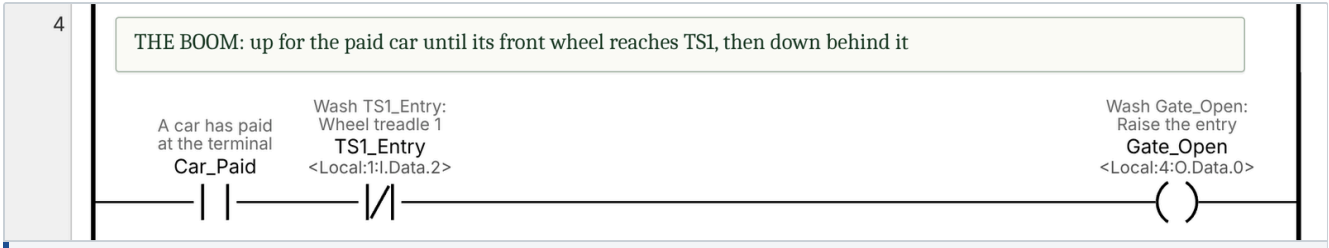
HOW THIS RUNG WORKS

In words. When `Wash_Ctl.POS` (the position of `Wash_Ctl`: The sequencer: .POS is the step (0 HOME, 1 LOAD ... 13 EXIT), .LEN 13; SQO moves it, SQI reads it) is not **1**, the rung unlatches `Car_Paid` (A car has paid at the terminal and may come on (only while the sequencer is at LOAD)) (turns it off).

Left to right:

1. `NEQ(Wash_Ctl.POS,1)` *Not Equal* asks: true when `Wash_Ctl.POS` (the position of `Wash_Ctl`: The sequencer: .POS is the step (0 HOME, 1 LOAD ... 13 EXIT), .LEN 13; SQO moves it, SQI reads it) is not **1**.
2. `OTU(Car_Paid)` *Output Unlatch* does: unlatches `Car_Paid` (A car has paid at the terminal and may come on (only while the sequencer is at LOAD)) (turns it off).

ENTRY · RUNG 4




```
XIC(Car_Paid)XIO(TS1_Entry)OTE(Gate_Open);
```

THE BOOM: up for the paid car until its front wheel reaches TS1, then down behind it

PREDICT IT · 6 `Car_Paid` is on and `TS1_Entry` is off. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Car_Paid` (A car has paid at the terminal and may come on (only while the sequencer is at LOAD)) is on and `TS1_Entry` (Wheel treadle 1 (ENTRY): the front wheel stands on it) is off, the rung turns `Gate_Open` (Raise the entry boom (it comes down when this drops)) on (and off again whenever the rung is false).

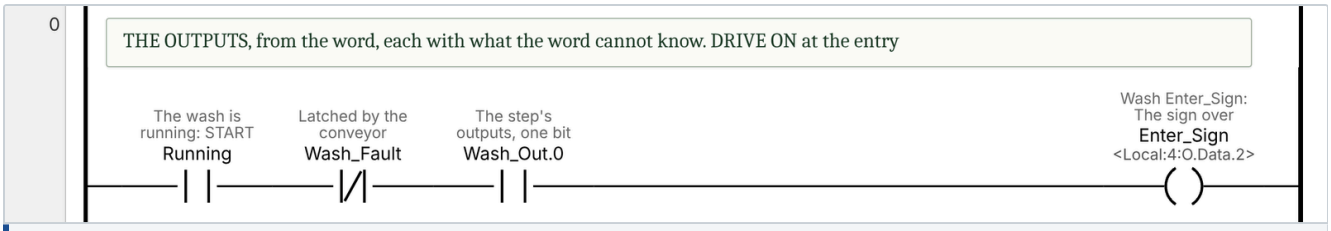
Left to right:

1. `XIC(Car_Paid)` *Examine If Closed* asks: true when `Car_Paid` (A car has paid at the terminal and may come on (only while the sequencer is at LOAD)) is on.
2. `XIO(TS1_Entry)` *Examine If Open* asks: true when `TS1_Entry` (Wheel treadle 1 (ENTRY): the front wheel stands on it) is off.
3. `OTE(Gate_Open)` *Output Energize* does: turns `Gate_Open` (Raise the entry boom (it comes down when this drops)) on (and off again whenever the rung is false).

Exercise 5: the stations, and what a table cannot know

Each output takes its bit of `Wash_Out`, and each adds what a table cannot hold. Running and no fault on all of them. A car in front of the arch: the eye. Soap in the tank before the soap pump runs: a pump run dry is a pump replaced. Water before the brushes turn: brushes on a dry car leave swirl marks in the paint. The brush motor's overload. DELUXE for the wax. The water pump runs for whatever needs water.

STATIONS · RUNG 0




```
XIC(Running)XIO(Wash_Fault)XIC(Wash_Out.0)OTE(Enter_Sign);
```

THE OUTPUTS, from the word, each with what the word cannot know. DRIVE ON at the entry

PREDICT IT · 7 `Running` is on, `Wash_Fault` is on and `Wash_Out.0` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

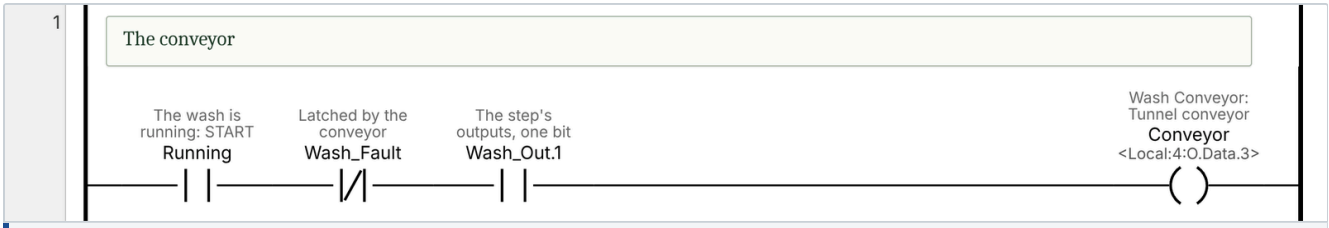
HOW THIS RUNG WORKS

In words. When `Running` (The wash is running: START seals it in, STOP (normally closed) or a fault drops it) is on, `Wash_Fault` (Latched by the conveyor watchdog or the brush overload; RESET clears it) is off and `Wash_Out.0` is on, the rung turns `Enter_Sign` (The sign over the entry: DRIVE ON (green) or STOP (red)) on (and off again whenever the rung is false).

Left to right:

1. `XIC(Running)` *Examine If Closed* asks: true when `Running` (The wash is running: START seals it in, STOP (normally closed) or a fault drops it) is on.
2. `XIO(Wash_Fault)` *Examine If Open* asks: true when `Wash_Fault` (Latched by the conveyor watchdog or the brush overload; RESET clears it) is off.
3. `XIC(Wash_Out.0)` *Examine If Closed* asks: true when `Wash_Out.0` is on.
4. `OTE(Enter_Sign)` *Output Energize* does: turns `Enter_Sign` (The sign over the entry: DRIVE ON (green) or STOP (red)) on (and off again whenever the rung is false).

STATIONS · RUNG 1




```
XIC(Running)XIO(Wash_Fault)XIC(Wash_Out.1)OTE(Conveyor);
```

The conveyor

PREDICT IT · 8 `Running` is on, `Wash_Fault` is off and `Wash_Out.1` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

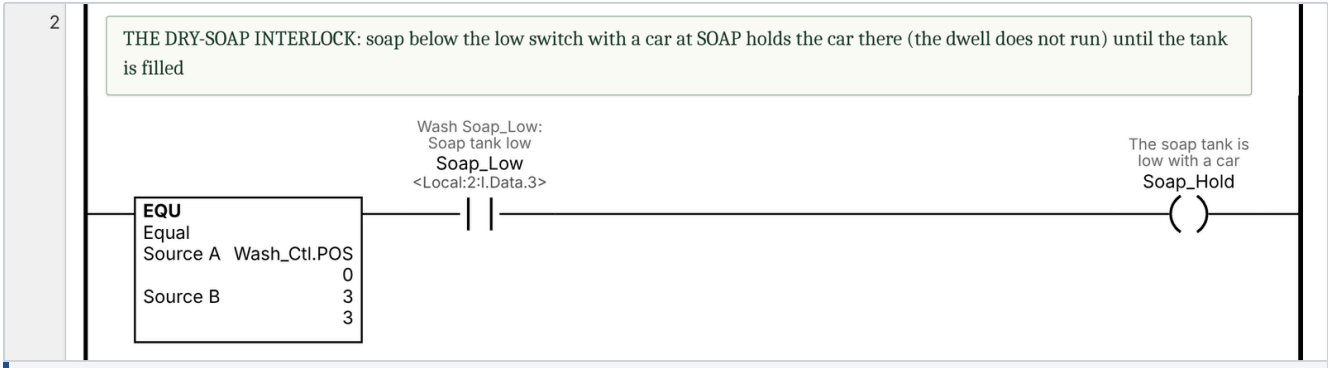
HOW THIS RUNG WORKS

In words. When `Running` (The wash is running: START seals it in, STOP (normally closed) or a fault drops it) is on, `Wash_Fault` (Latched by the conveyor watchdog or the brush overload; RESET clears it) is off and `Wash_Out.1` is on, the rung turns `Conveyor` (Tunnel conveyor motor: the roller pushes the front wheel) on (and off again whenever the rung is false).

Left to right:

1. `XIC(Running)` *Examine If Closed* asks: true when `Running` (The wash is running: START seals it in, STOP (normally closed) or a fault drops it) is on.
2. `XIO(Wash_Fault)` *Examine If Open* asks: true when `Wash_Fault` (Latched by the conveyor watchdog or the brush overload; RESET clears it) is off.
3. `XIC(Wash_Out.1)` *Examine If Closed* asks: true when `Wash_Out.1` is on.
4. `OTE(Conveyor)` *Output Energize* does: turns `Conveyor` (Tunnel conveyor motor: the roller pushes the front wheel) on (and off again whenever the rung is false).

STATIONS · RUNG 2




```
EQU(Wash_Ctl.POS,3)XIC(Soap_Low)OTE(Soap_Hold);
```

THE DRY-SOAP INTERLOCK: soap below the low switch with a car at SOAP holds the car there (the dwell does not run) until the tank is filled

PREDICT IT · 9 `Wash_Ctl.POS` (the position of `Wash_Ctl`: The sequencer: .POS is the step (0 HOME, 1 LOAD ... 13 EXIT), .LEN 13; SQO moves it, SQI reads it) is 4 and `Soap_Low` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

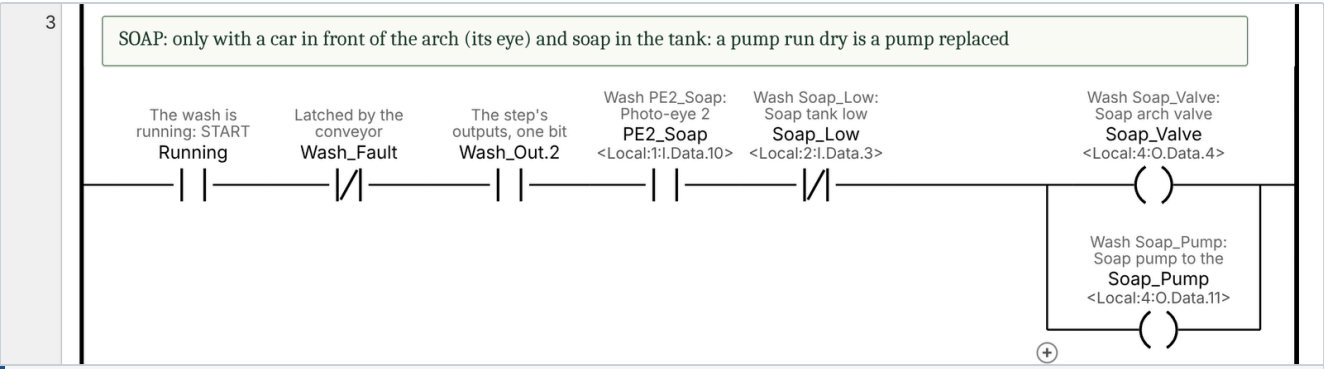
HOW THIS RUNG WORKS

In words. When `Wash_Ctl.POS` (the position of `Wash_Ctl`: The sequencer: .POS is the step (0 HOME, 1 LOAD ... 13 EXIT), .LEN 13; SQO moves it, SQI reads it) equals **3** and `Soap_Low` (Soap tank low level switch: 1 below 20 %) is on, the rung turns `Soap_Hold` (The soap tank is low with a car at SOAP: the dwell waits and the pump stays off until it is filled) on (and off again whenever the rung is false).

Left to right:

1. `EQU(Wash_Ctl.POS,3)` *Equal* asks: true when `Wash_Ctl.POS` (the position of `Wash_Ctl`: The sequencer: .POS is the step (0 HOME, 1 LOAD ... 13 EXIT), .LEN 13; SQO moves it, SQI reads it) equals **3**.
2. `XIC(Soap_Low)` *Examine If Closed* asks: true when `Soap_Low` (Soap tank low level switch: 1 below 20 %) is on.
3. `OTE(Soap_Hold)` *Output Energize* does: turns `Soap_Hold` (The soap tank is low with a car at SOAP: the dwell waits and the pump stays off until it is filled) on (and off again whenever the rung is false).

STATIONS · RUNG 3




```
XIC(Running)XIO(Wash_Fault)XIC(Wash_Out.2)XIC(PE2_Soap)XIO(Soap_Low)
[OTE(Soap_Valve),OTE(Soap_Pump)];
```

SOAP: only with a car in front of the arch (its eye) and soap in the tank: a pump run dry is a pump replaced

PREDICT IT · 10 `Running` is on, `Wash_Fault` is off, `Wash_Out.2` is on, `PE2_Soap` is on and `Soap_Low` is off. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Running` (The wash is running: START seals it in, STOP (normally closed) or a fault drops it) is on, `Wash_Fault` (Latched by the conveyor watchdog or the brush overload; RESET clears it) is off, `Wash_Out.2` is on, `PE2_Soap` (Photo-eye 2 (SOAP) across the bay at bumper height: 1 while a car blocks it) is on and `Soap_Low` (Soap tank low level switch: 1 below 20 %) is off, the rung turns `Soap_Valve` (Soap arch valve) on (and off again whenever the rung is false) and turns `Soap_Pump` (Soap pump to the soap arch) on (and off again whenever the rung is false).

Left to right:

1. `XIC(Running)` *Examine If Closed* asks: true when `Running` (The wash is running: START seals it in, STOP (normally closed) or a fault drops it) is on.
2. `XIO(Wash_Fault)` *Examine If Open* asks: true when `Wash_Fault` (Latched by the conveyor watchdog or the brush overload; RESET clears it) is off.
3. `XIC(Wash_Out.2)` *Examine If Closed* asks: true when `Wash_Out.2` is on.
4. `XIC(PE2_Soap)` *Examine If Closed* asks: true when `PE2_Soap` (Photo-eye 2 (SOAP) across the bay at bumper height: 1 while a car blocks it) is on.
5. `XIO(Soap_Low)` *Examine If Open* asks: true when `Soap_Low` (Soap tank low level switch: 1 below 20 %) is off.
6. A branch of 2 parallel paths: each path is tested on its own and does its own work.

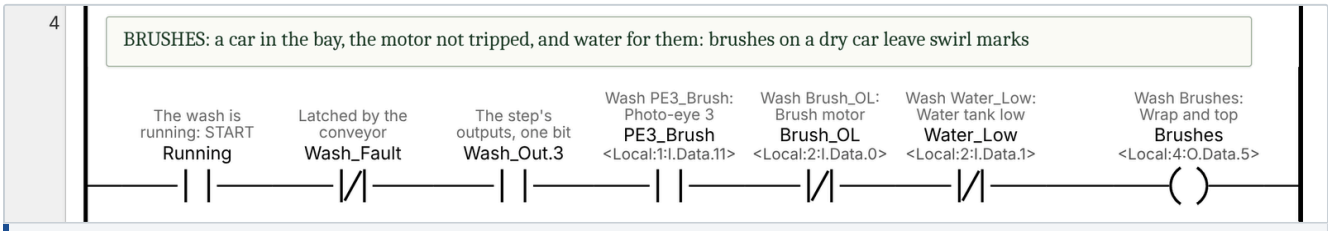
   Path 1

   1. `OTE(Soap_Valve)` *Output Energize* does: turns `Soap_Valve` (Soap arch valve) on (and off again whenever the rung is false).

   Path 2

   1. `OTE(Soap_Pump)` *Output Energize* does: turns `Soap_Pump` (Soap pump to the soap arch) on (and off again whenever the rung is false).

STATIONS · RUNG 4




```
XIC(Running)XIO(Wash_Fault)XIC(Wash_Out.3)XIC(PE3_Brush)XIO(Brush_OL)XIO(Water_Low)OTE(Brushes);
```

BRUSHES: a car in the bay, the motor not tripped, and water for them: brushes on a dry car leave swirl marks

PREDICT IT · 11 `Running` is on, `Wash_Fault` is off, `Wash_Out.3` is on, `PE3_Brush` is off, `Brush_OL` is off and `Water_Low` is off. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

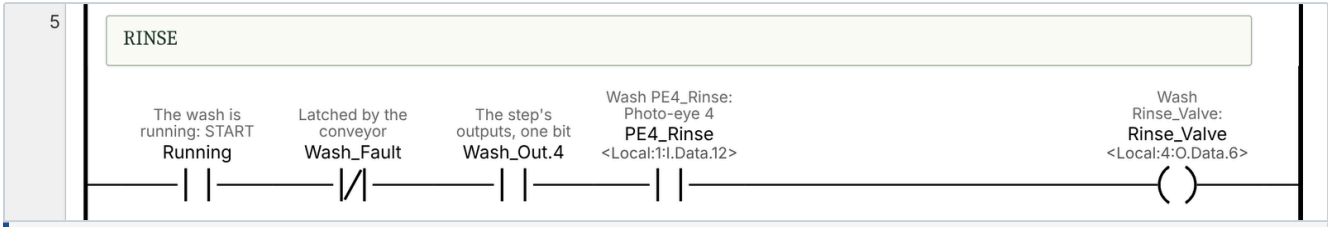
HOW THIS RUNG WORKS

In words. When `Running` (The wash is running: START seals it in, STOP (normally closed) or a fault drops it) is on, `Wash_Fault` (Latched by the conveyor watchdog or the brush overload; RESET clears it) is off, `Wash_Out.3` is on, `PE3_Brush` (Photo-eye 3 (BRUSH) across the bay at bumper height: 1 while a car blocks it) is on, `Brush_OL` (Brush motor overload relay: 1 when it has tripped) is off and `Water_Low` (Water tank low level switch: 1 below 20 %) is off, the rung turns `Brushes` (Wrap and top brush motors) on (and off again whenever the rung is false).

Left to right:

1. `XIC(Running)` *Examine If Closed* asks: true when `Running` (The wash is running: START seals it in, STOP (normally closed) or a fault drops it) is on.
2. `XIO(Wash_Fault)` *Examine If Open* asks: true when `Wash_Fault` (Latched by the conveyor watchdog or the brush overload; RESET clears it) is off.
3. `XIC(Wash_Out.3)` *Examine If Closed* asks: true when `Wash_Out.3` is on.
4. `XIC(PE3_Brush)` *Examine If Closed* asks: true when `PE3_Brush` (Photo-eye 3 (BRUSH) across the bay at bumper height: 1 while a car blocks it) is on.
5. `XIO(Brush_OL)` *Examine If Open* asks: true when `Brush_OL` (Brush motor overload relay: 1 when it has tripped) is off.
6. `XIO(Water_Low)` *Examine If Open* asks: true when `Water_Low` (Water tank low level switch: 1 below 20 %) is off.
7. `OTE(Brushes)` *Output Energize* does: turns `Brushes` (Wrap and top brush motors) on (and off again whenever the rung is false).

STATIONS · RUNG 5




```
XIC(Running)XIO(Wash_Fault)XIC(Wash_Out.4)XIC(PE4_Rinse)OTE(Rinse_Valve);
```

RINSE

PREDICT IT · 12 `Running` is on, `Wash_Fault` is off, `Wash_Out.4` is on and `PE4_Rinse` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

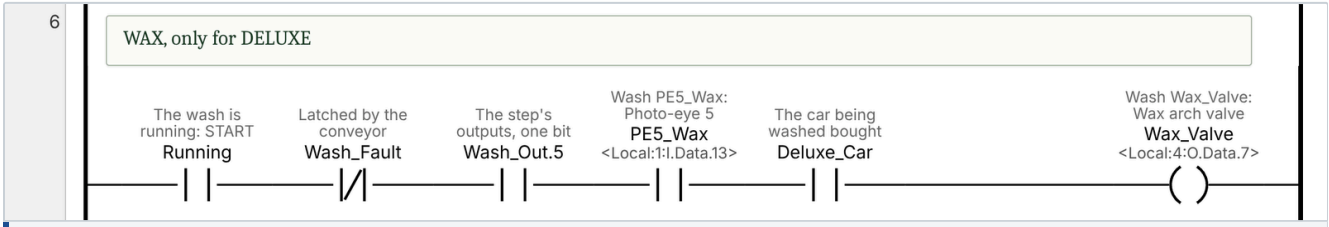
HOW THIS RUNG WORKS

In words. When `Running` (The wash is running: START seals it in, STOP (normally closed) or a fault drops it) is on, `Wash_Fault` (Latched by the conveyor watchdog or the brush overload; RESET clears it) is off, `Wash_Out.4` is on and `PE4_Rinse` (Photo-eye 4 (RINSE) across the bay at bumper height: 1 while a car blocks it) is on, the rung turns `Rinse_Valve` (Rinse arch valve) on (and off again whenever the rung is false).

Left to right:

1. `XIC(Running)` *Examine If Closed* asks: true when `Running` (The wash is running: START seals it in, STOP (normally closed) or a fault drops it) is on.
2. `XIO(Wash_Fault)` *Examine If Open* asks: true when `Wash_Fault` (Latched by the conveyor watchdog or the brush overload; RESET clears it) is off.
3. `XIC(Wash_Out.4)` *Examine If Closed* asks: true when `Wash_Out.4` is on.
4. `XIC(PE4_Rinse)` *Examine If Closed* asks: true when `PE4_Rinse` (Photo-eye 4 (RINSE) across the bay at bumper height: 1 while a car blocks it) is on.
5. `OTE(Rinse_Valve)` *Output Energize* does: turns `Rinse_Valve` (Rinse arch valve) on (and off again whenever the rung is false).

STATIONS · RUNG 6




```
XIC(Running)XIO(Wash_Fault)XIC(Wash_Out.5)XIC(PE5_Wax)XIC(Deluxe_Car)OTE(Wax_Valve);
```

WAX, only for DELUXE

PREDICT IT · 13 `Running` is on, `Wash_Fault` is on, `Wash_Out.5` is on, `PE5_Wax` is on and `Deluxe_Car` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

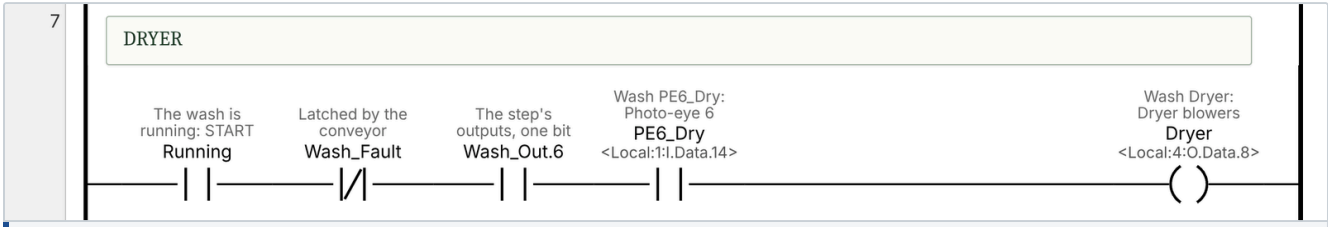
HOW THIS RUNG WORKS

In words. When `Running` (The wash is running: START seals it in, STOP (normally closed) or a fault drops it) is on, `Wash_Fault` (Latched by the conveyor watchdog or the brush overload; RESET clears it) is off, `Wash_Out.5` is on, `PE5_Wax` (Photo-eye 5 (WAX) across the bay at bumper height: 1 while a car blocks it) is on and `Deluxe_Car` (The car being washed bought DELUXE: the wax arch runs for it) is on, the rung turns `Wax_Valve` (Wax arch valve) on (and off again whenever the rung is false).

Left to right:

1. `XIC(Running)` *Examine If Closed* asks: true when `Running` (The wash is running: START seals it in, STOP (normally closed) or a fault drops it) is on.
2. `XIO(Wash_Fault)` *Examine If Open* asks: true when `Wash_Fault` (Latched by the conveyor watchdog or the brush overload; RESET clears it) is off.
3. `XIC(Wash_Out.5)` *Examine If Closed* asks: true when `Wash_Out.5` is on.
4. `XIC(PE5_Wax)` *Examine If Closed* asks: true when `PE5_Wax` (Photo-eye 5 (WAX) across the bay at bumper height: 1 while a car blocks it) is on.
5. `XIC(Deluxe_Car)` *Examine If Closed* asks: true when `Deluxe_Car` (The car being washed bought DELUXE: the wax arch runs for it) is on.
6. `OTE(Wax_Valve)` *Output Energize* does: turns `Wax_Valve` (Wax arch valve) on (and off again whenever the rung is false).

STATIONS · RUNG 7




```
XIC(Running)XIO(Wash_Fault)XIC(Wash_Out.6)XIC(PE6_Dry)OTE(Dryer);
```

DRYER

PREDICT IT · 14 `Running` is on, `Wash_Fault` is off, `Wash_Out.6` is on and `PE6_Dry` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

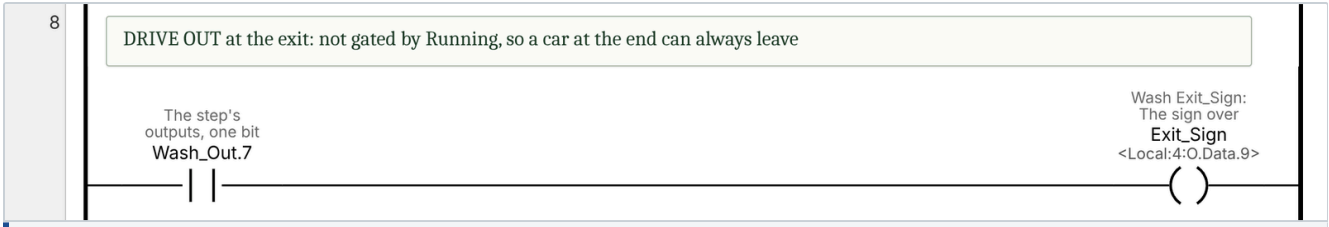
HOW THIS RUNG WORKS

In words. When `Running` (The wash is running: START seals it in, STOP (normally closed) or a fault drops it) is on, `Wash_Fault` (Latched by the conveyor watchdog or the brush overload; RESET clears it) is off, `Wash_Out.6` is on and `PE6_Dry` (Photo-eye 6 (DRY) across the bay at bumper height: 1 while a car blocks it) is on, the rung turns `Dryer` (Dryer blowers) on (and off again whenever the rung is false).

Left to right:

1. `XIC(Running)` *Examine If Closed* asks: true when `Running` (The wash is running: START seals it in, STOP (normally closed) or a fault drops it) is on.
2. `XIO(Wash_Fault)` *Examine If Open* asks: true when `Wash_Fault` (Latched by the conveyor watchdog or the brush overload; RESET clears it) is off.
3. `XIC(Wash_Out.6)` *Examine If Closed* asks: true when `Wash_Out.6` is on.
4. `XIC(PE6_Dry)` *Examine If Closed* asks: true when `PE6_Dry` (Photo-eye 6 (DRY) across the bay at bumper height: 1 while a car blocks it) is on.
5. `OTE(Dryer)` *Output Energize* does: turns `Dryer` (Dryer blowers) on (and off again whenever the rung is false).

STATIONS · RUNG 8




```
XIC(Wash_Out.7)OTE(Exit_Sign);
```

DRIVE OUT at the exit: not gated by Running, so a car at the end can always leave

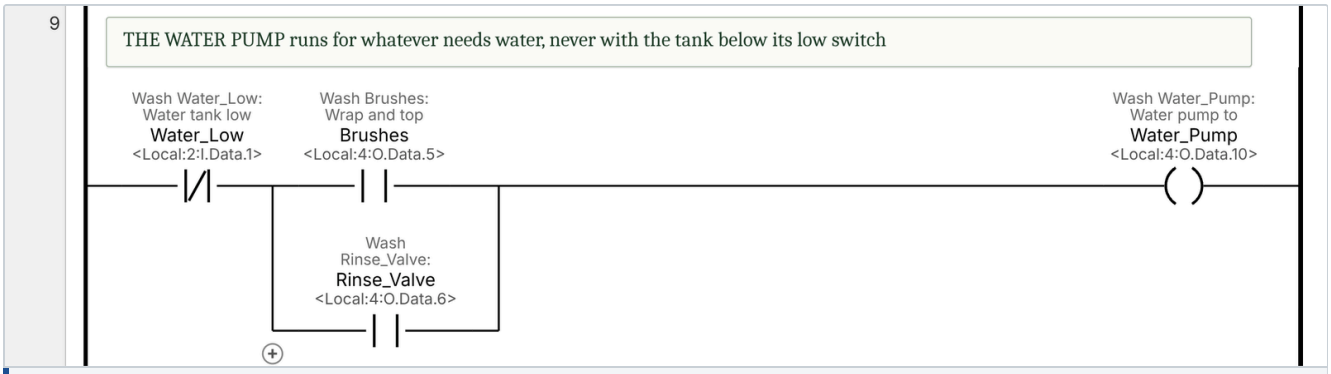
HOW THIS RUNG WORKS

In words. When `Wash_Out.7` is on, the rung turns `Exit_Sign` (The sign over the exit: DRIVE OUT (green) or WAIT (red)) on (and off again whenever the rung is false).

Left to right:

1. `XIC(Wash_Out.7)` *Examine If Closed* asks: true when `Wash_Out.7` is on.
2. `OTE(Exit_Sign)` *Output Energize* does: turns `Exit_Sign` (The sign over the exit: DRIVE OUT (green) or WAIT (red)) on (and off again whenever the rung is false).

STATIONS · RUNG 9




```
XIO(Water_Low)[XIC(Brushes),XIC(Rinse_Valve)]OTE(Water_Pump);
```

THE WATER PUMP runs for whatever needs water, never with the tank below its low switch

PREDICT IT · 15 `Water_Low` is off and `Brushes` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Water_Low` (Water tank low level switch: 1 below 20 %) is off and either (`Brushes` (Wrap and top brush motors) is on) *or* (`Rinse_Valve` (Rinse arch valve) is on), the rung turns `Water_Pump` (Water pump to the brushes and the rinse arch) on (and off again whenever the rung is false).

Left to right:

1. `XIO(Water_Low)` *Examine If Open* asks: true when `Water_Low` (Water tank low level switch: 1 below 20 %) is off.
2. A branch of 2 parallel paths: power gets through if any one of them is true.
   - **Path 1**
     1. `XIC(Brushes)` *Examine If Closed* asks: true when `Brushes` (Wrap and top brush motors) is on.
   - **Path 2**
     1. `XIC(Rinse_Valve)` *Examine If Closed* asks: true when `Rinse_Valve` (Rinse arch valve) is on.
3. `OTE(Water_Pump)` *Output Energize* does: turns `Water_Pump` (Water pump to the brushes and the rinse arch) on (and off again whenever the rung is false).

> **Think of it like this:** parallel paths are two roads to the same town. If either road is open you get there; the rung only cares that power reaches the far end, not which way it went.

THINK OF IT LIKE THIS: THE TABLE AND THE INTERLOCK

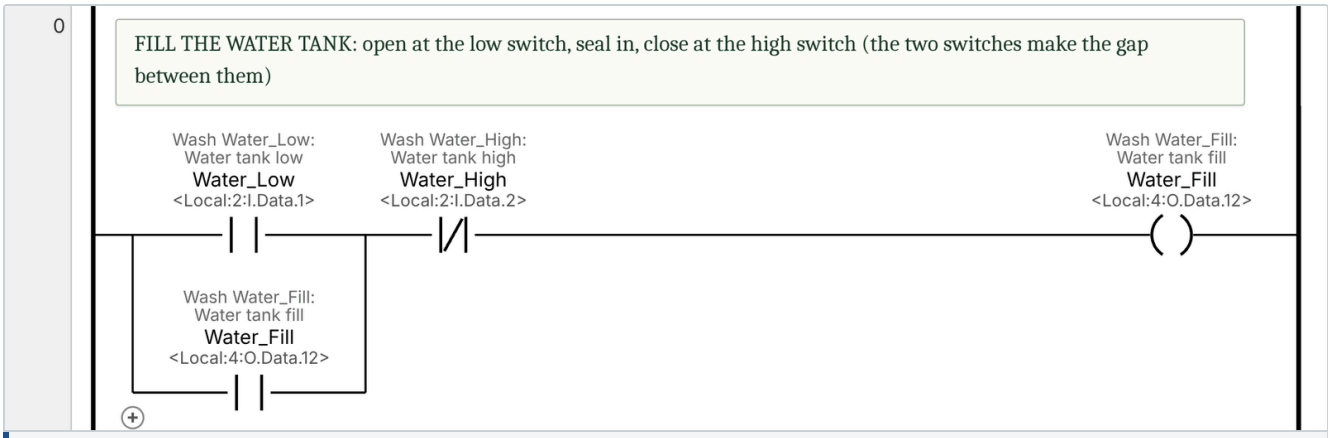
The table is the recipe; the interlocks are the cook. The recipe says "now the soap"; the cook looks in the soap bottle first. A step table holds the order of things, which is the same for every car. An interlock holds a condition of the moment, which no table can know in advance. Keep each where it belongs and the table stays a table.

WATCH FOR: THE DRY-SOAP INTERLOCK

Drain the soap tank as a car heads for SOAP. Without `Soap_Hold` the dwell runs out with the pump held off and the car goes on unsoaped; the log says *no soap*. With it, the car waits at SOAP until the fill has taken the tank off its low switch. A wash that refuses to wash is better than a wash that pretends to.

Exercise 6: the tanks, between their switches

SUPPLY · RUNG 0




```
[XIC(Water_Low),XIC(Water_Fill)]XIO(Water_High)OTE(Water_Fill);
```

FILL THE WATER TANK: open at the low switch, seal in, close at the high switch (the two switches make the gap between them)

PREDICT IT · 16 `Water_Low` is on and `Water_High` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When either (`Water_Low` (Water tank low level switch: 1 below 20 %) is on) *or* (`Water_Fill` (Water tank fill valve (city water)) is on) and `Water_High` (Water tank high level switch: 1 above 90 %) is off, the rung turns `Water_Fill` on (and off again whenever the rung is false).

Left to right:

1. A branch of 2 parallel paths: power gets through if any one of them is true.

   Path 1

   1. `XIC(Water_Low)` *Examine If Closed* asks: true when `Water_Low` (Water tank low level switch: 1 below 20 %) is on.

   Path 2

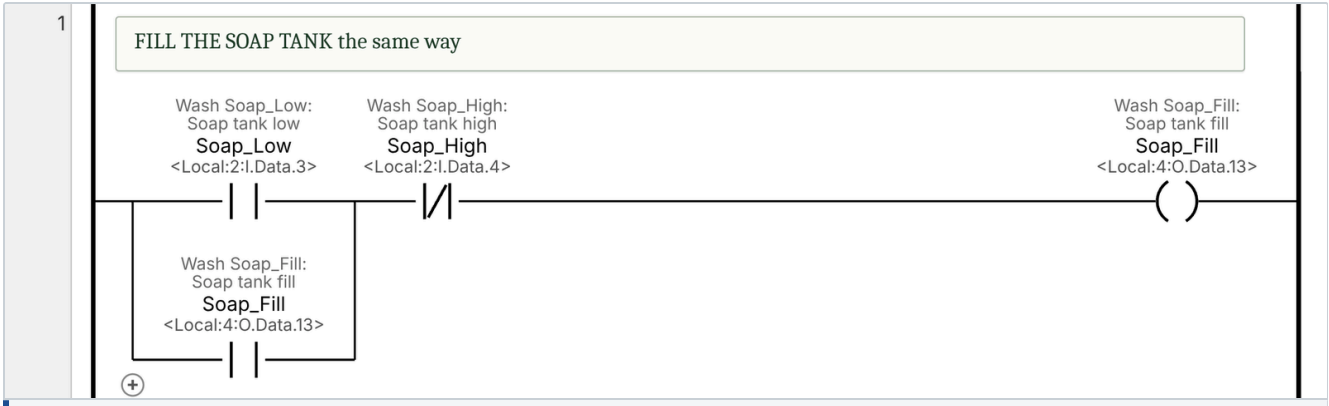
   1. `XIC(Water_Fill)` *Examine If Closed* asks: true when `Water_Fill` (Water tank fill valve (city water)) is on.

2. `XIO(Water_High)` *Examine If Open* asks: true when `Water_High` (Water tank high level switch: 1 above 90 %) is off.
3. `OTE(Water_Fill)` *Output Energize* does: turns `Water_Fill` on (and off again whenever the rung is false).

Why it is built this way:

Seal-in, as before: the output's own contact holds the rung once the start has been made.

SUPPLY · RUNG 1




```
[XIC(Soap_Low),XIC(Soap_Fill)]XIO(Soap_High)OTE(Soap_Fill);
```

FILL THE SOAP TANK the same way

PREDICT IT · 17 `Soap_Low` is on and `Soap_High` is off. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When either (`Soap_Low` (Soap tank low level switch: 1 below 20 %) is on) *or* (`Soap_Fill` (Soap tank fill valve (from the bulk drum)) is on) and `Soap_High` (Soap tank high level switch: 1 above 90 %) is off, the rung turns `Soap_Fill` on (and off again whenever the rung is false).

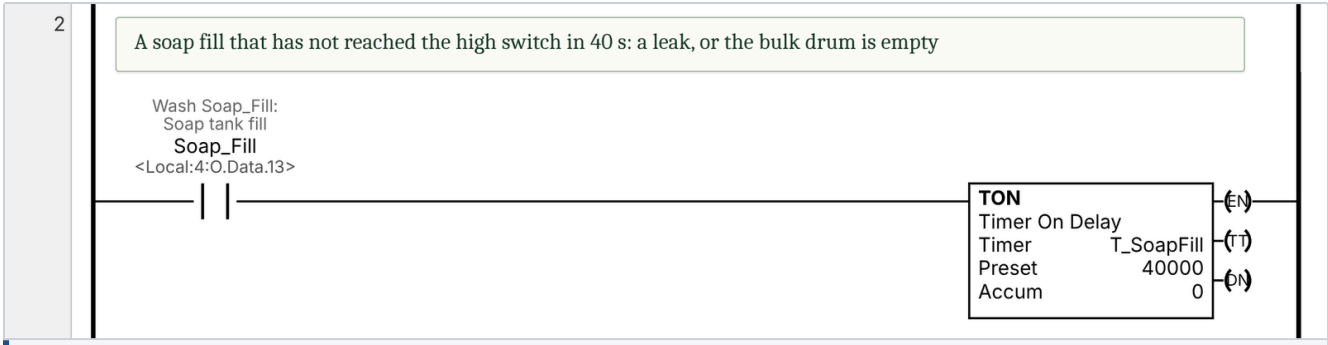
Left to right:

1. A branch of 2 parallel paths: power gets through if any one of them is true.
   - **Path 1**
     1. `XIC(Soap_Low)` *Examine If Closed* asks: true when `Soap_Low` (Soap tank low level switch: 1 below 20 %) is on.
   - **Path 2**
     1. `XIC(Soap_Fill)` *Examine If Closed* asks: true when `Soap_Fill` (Soap tank fill valve (from the bulk drum)) is on.
2. `XIO(Soap_High)` *Examine If Open* asks: true when `Soap_High` (Soap tank high level switch: 1 above 90 %) is off.
3. `OTE(Soap_Fill)` *Output Energize* does: turns `Soap_Fill` on (and off again whenever the rung is false).

Why it is built this way:

Seal-in, as before: the output's own contact holds the rung once the start has been made.

SUPPLY · RUNG 2




```
XIC(Soap_Fill)TON(T_SoapFill,40000,0);
```

A soap fill that has not reached the high switch in 40 s: a leak, or the bulk drum is empty

HOW THIS RUNG WORKS

In words. When `Soap_Fill` (Soap tank fill valve (from the bulk drum)) is on, the rung runs the timer `T_SoapFill` (Soap fill watchdog: 40 s without reaching the high switch is a supply alarm) toward 40000 ms (40 s).

Left to right:

1. `XIC(Soap_Fill)` *Examine If Closed* asks: true when `Soap_Fill` (Soap tank fill valve (from the bulk drum)) is on.
2. `TON(T_SoapFill,40000,0)` *Timer On Delay* does: runs the timer `T_SoapFill` (Soap fill watchdog: 40 s without reaching the high switch is a supply alarm) toward 40000 ms (40 s).

Why it is built this way:

A timer: it times only while this rung is true.

SUPPLY · RUNG 3

```
XIC(T_SoapFill.DN)OTL(Supply_Alarm);
```

HOW THIS RUNG WORKS

In words. When `T_SoapFill.DN` (the done bit of `T_SoapFill`: Soap fill watchdog: 40 s without reaching the high switch is a supply alarm) is on, the rung latches `Supply_Alarm` (The soap tank did not fill in time (a leak, an empty drum)) on.

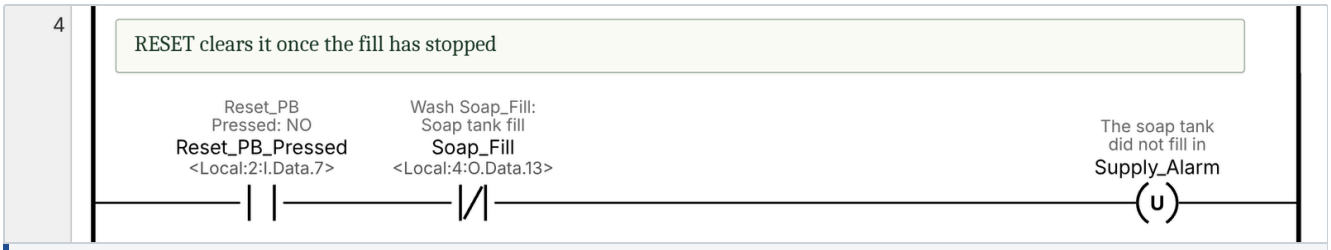
Left to right:

1. `XIC(T_SoapFill.DN)` *Examine If Closed* asks: true when `T_SoapFill.DN` (the done bit of `T_SoapFill`: Soap fill watchdog: 40 s without reaching the high switch is a supply alarm) is on.
2. `OTL(Supply_Alarm)` *Output Latch* does: latches `Supply_Alarm` (The soap tank did not fill in time (a leak, an empty drum)) on.

Why it is built this way:

A latch: it stays on until an OTU clears it.

SUPPLY · RUNG 4




```
XIC(Reset_PB_Pressed)XIO(Soap_Fill)OTU(Supply_Alarm);
```

RESET clears it once the fill has stopped

PREDICT IT · 18 `Reset_PB_Pressed` is off and `Soap_Fill` is off. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When `Reset_PB_Pressed` (NO contact: 1 while pressed) is on and `Soap_Fill` (Soap tank fill valve (from the bulk drum)) is off, the rung unlatches `Supply_Alarm` (The soap tank did not fill in time (a leak, an empty drum)) (turns it off).

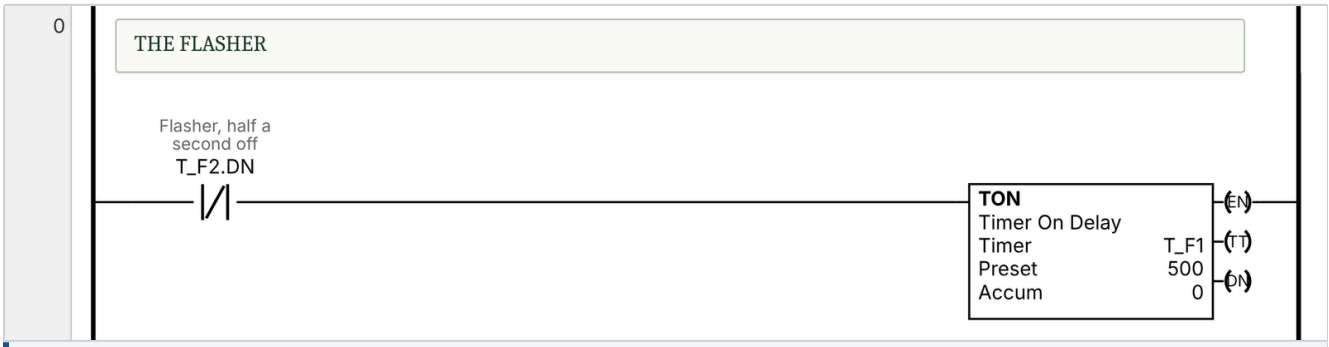
Left to right:

1. `XIC(Reset_PB_Pressed)` *Examine If Closed* asks: true when `Reset_PB_Pressed` (NO contact: 1 while pressed) is on.
2. `XIO(Soap_Fill)` *Examine If Open* asks: true when `Soap_Fill` (Soap tank fill valve (from the bulk drum)) is off.
3. `OTU(Supply_Alarm)` *Output Unlatch* does: unlatches `Supply_Alarm` (The soap tank did not fill in time (a leak, an empty drum)) (turns it off).

Two switches make a band: open the fill at LO, keep it open (the seal-in) until HI. With one switch the valve would chatter on and off at the switch point.

Exercise 7: the watchdog, the cross-check, the lamps

ALARMS · RUNG 0




```
XIO(T_F2.DN)TON(T_F1,500,0);
```

THE FLASHER

HOW THIS RUNG WORKS

In words. When `T_F2.DN` (the done bit of `T_F2` : Flasher, half a second off) is off, the rung runs the timer `T_F1` (Flasher, half a second on) toward 500 ms (0.5 s).

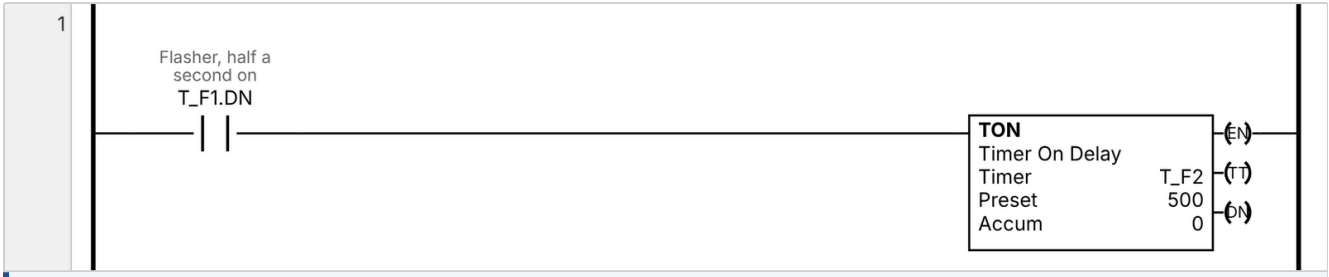
Left to right:

1. `XIO(T_F2.DN)` *Examine If Open* asks: true when `T_F2.DN` (the done bit of `T_F2` : Flasher, half a second off) is off.
2. `TON(T_F1,500,0)` *Timer On Delay* does: runs the timer `T_F1` (Flasher, half a second on) toward 500 ms (0.5 s).

Why it is built this way:

A timer: it times only while this rung is true.

ALARMS · RUNG 1




```
XIC(T_F1.DN)TON(T_F2,500,0);
```

HOW THIS RUNG WORKS

In words. When `T_F1.DN` (the done bit of `T_F1` : Flasher, half a second on) is on, the rung runs the timer `T_F2` (Flasher, half a second off) toward 500 ms (0.5 s).

Left to right:

1. `XIC(T_F1.DN)` *Examine If Closed* asks: true when `T_F1.DN` (the done bit of `T_F1` : Flasher, half a second on) is on.
2. `TON(T_F2,500,0)` *Timer On Delay* does: runs the timer `T_F2` (Flasher, half a second off) toward 500 ms (0.5 s).

Why it is built this way:

A timer: it times only while this rung is true.

ALARMS · RUNG 2

```
XIC(T_F1.DN)OTE(Fl_Bit);
```

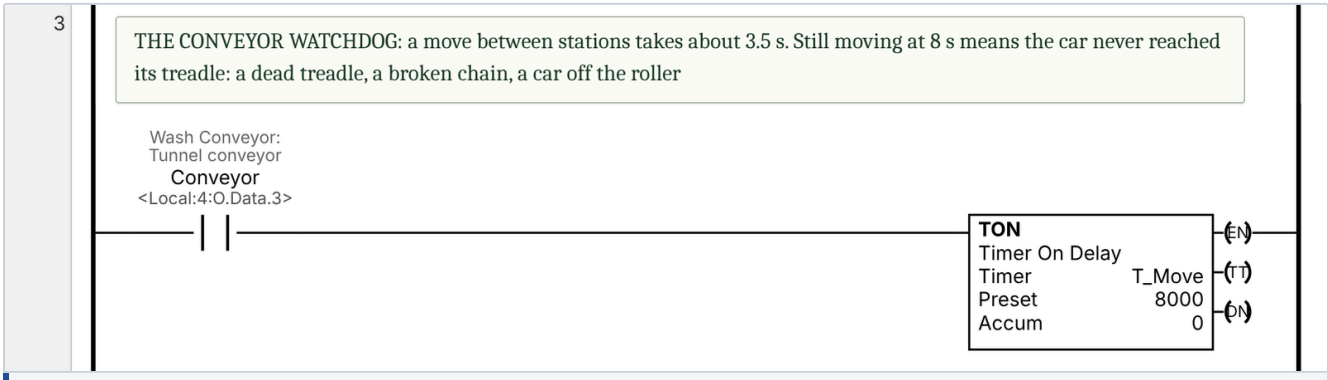
HOW THIS RUNG WORKS

In words. When `T_F1.DN` (the done bit of `T_F1` : Flasher, half a second on) is on, the rung turns `Fl_Bit` (The flasher bit) on (and off again whenever the rung is false).

Left to right:

1. `XIC(T_F1.DN)` *Examine If Closed* asks: true when `T_F1.DN` (the done bit of `T_F1` : Flasher, half a second on) is on.
2. `OTE(Fl_Bit)` *Output Energize* does: turns `Fl_Bit` (The flasher bit) on (and off again whenever the rung is false).

ALARMS · RUNG 3




```
XIC(Conveyor)TON(T_Move,8000,0);
```

THE CONVEYOR WATCHDOG: a move between stations takes about 3.5 s. Still moving at 8 s means the car never reached its treadle: a dead treadle, a broken chain, a car off the roller

HOW THIS RUNG WORKS

In words. When `Conveyor` (Tunnel conveyor motor: the roller pushes the front wheel) is on, the rung runs the timer `T_Move` (Conveyor watchdog: 8 s (a move takes about 3.5 s)) toward 8000 ms (8 s).

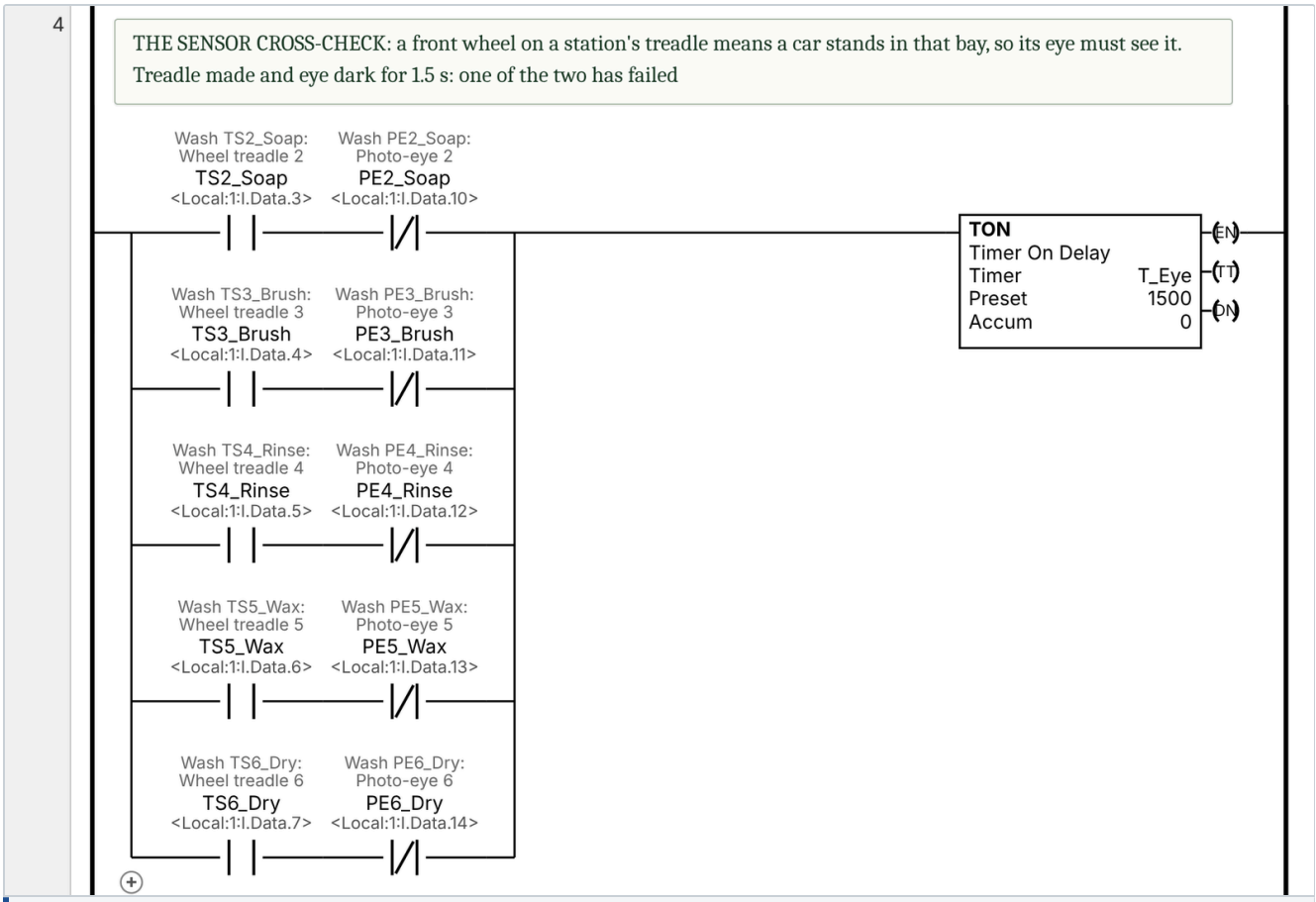
Left to right:

1. `XIC(Conveyor)` *Examine If Closed* asks: true when `Conveyor` (Tunnel conveyor motor: the roller pushes the front wheel) is on.
2. `TON(T_Move,8000,0)` *Timer On Delay* does: runs the timer `T_Move` (Conveyor watchdog: 8 s (a move takes about 3.5 s)) toward 8000 ms (8 s).

Why it is built this way:

A timer: it times only while this rung is true.

ALARMS · RUNG 4




```
[XIC(TS2_Soap)XIO(PE2_Soap),XIC(TS3_Brush)XIO(PE3_Brush),XIC(TS4_Rinse)XIO(PE4_Rinse),XIC(TS5_Wax)XIO(PE5_Wax),XIC(TS6_Dry)XIO(PE6_Dry)]TON(T_Eye,1500,0);
```

THE SENSOR CROSS-CHECK: a front wheel on a station's treadle means a car stands in that bay, so its eye must see it. Treadle made and eye dark for 1.5 s: one of the two has failed

PREDICT IT · 19 `TS2_Soap` is on and `PE2_Soap` is off. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When either (`TS2_Soap` (Wheel treadle 2 (SOAP): the front wheel stands on it) is on and `PE2_Soap` (Photo-eye 2 (SOAP) across the bay at bumper height: 1 while a car blocks it) is off) *or* (`TS3_Brush` (Wheel treadle 3 (BRUSH): the front wheel stands on it) is on and `PE3_Brush` (Photo-eye 3 (BRUSH) across the bay at bumper height: 1 while a car blocks it) is off) *or* (`TS4_Rinse` (Wheel treadle 4 (RINSE): the front wheel stands on it) is on and `PE4_Rinse` (Photo-eye 4 (RINSE) across the bay at bumper height: 1 while a car blocks it) is off) *or* (`TS5_Wax` (Wheel treadle 5 (WAX): the front wheel stands on it) is on and `PE5_Wax` (Photo-eye 5 (WAX) across the bay at bumper height: 1 while a car blocks it) is off) *or* (`TS6_Dry` (Wheel treadle 6 (DRY): the front wheel stands on it) is on and `PE6_Dry` (Photo-eye 6 (DRY) across the bay at bumper height: 1 while a car blocks it) is off), the rung runs the timer `T_Eye` (A wheel on a station's treadle and that bay's eye seeing nothing for 1.5 s: one of the two is lying) toward 1500 ms (1.5 s).

Left to right:

1. A branch of 5 parallel paths: power gets through if any one of them is true.

   Path 1

   1. `XIC(TS2_Soap)` *Examine If Closed* asks: true when `TS2_Soap` (Wheel treadle 2 (SOAP): the front wheel stands on it) is on.
   2. `XIO(PE2_Soap)` *Examine If Open* asks: true when `PE2_Soap` (Photo-eye 2 (SOAP) across the bay at bumper height: 1 while a car blocks it) is off.

   Path 2

   1. `XIC(TS3_Brush)` *Examine If Closed* asks: true when `TS3_Brush` (Wheel treadle 3 (BRUSH): the front wheel stands on it) is on.
   2. `XIO(PE3_Brush)` *Examine If Open* asks: true when `PE3_Brush` (Photo-eye 3 (BRUSH) across the bay at bumper height: 1 while a car blocks it) is off.

   Path 3

   1. `XIC(TS4_Rinse)` *Examine If Closed* asks: true when `TS4_Rinse` (Wheel treadle 4 (RINSE): the front wheel stands on it) is on.
   2. `XIO(PE4_Rinse)` *Examine If Open* asks: true when `PE4_Rinse` (Photo-eye 4 (RINSE) across the bay at bumper height: 1 while a car blocks it) is off.

   Path 4

   1. `XIC(TS5_Wax)` *Examine If Closed* asks: true when `TS5_Wax` (Wheel treadle 5 (WAX): the front wheel stands on it) is on.
   2. `XIO(PE5_Wax)` *Examine If Open* asks: true when `PE5_Wax` (Photo-eye 5 (WAX) across the bay at bumper height: 1 while a car blocks it) is off.

   Path 5

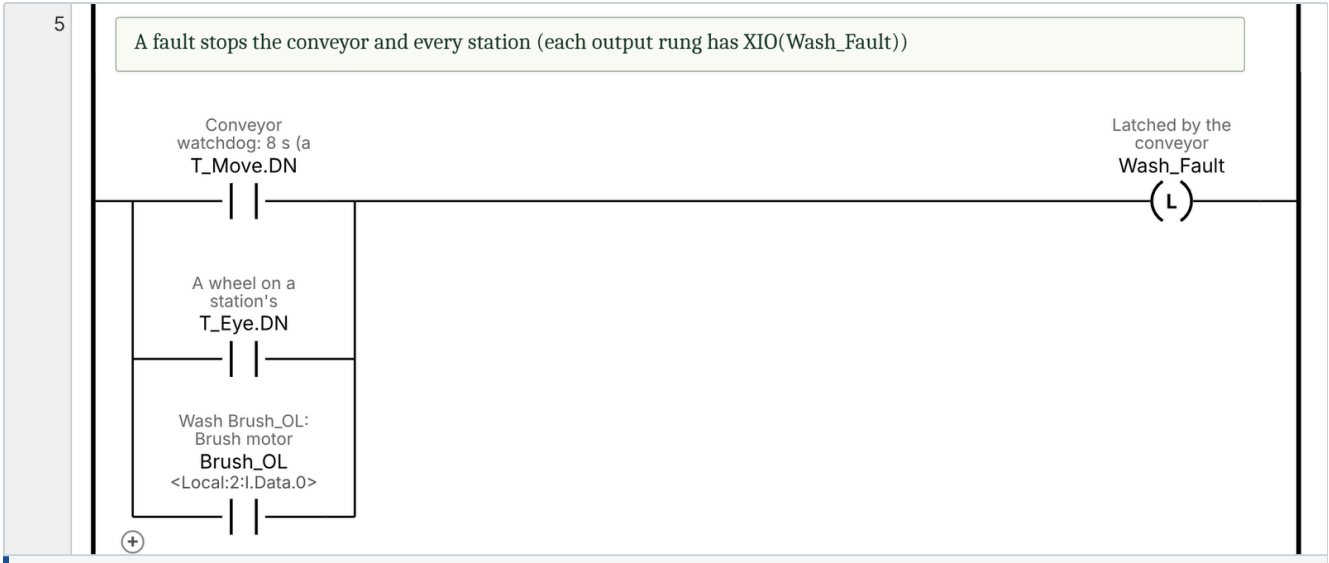
   1. `XIC(TS6_Dry)` *Examine If Closed* asks: true when `TS6_Dry` (Wheel treadle 6 (DRY): the front wheel stands on it) is on.
   2. `XIO(PE6_Dry)` *Examine If Open* asks: true when `PE6_Dry` (Photo-eye 6 (DRY) across the bay at bumper height: 1 while a car blocks it) is off.

2. `TON(T_Eye,1500,0)` *Timer On Delay* does: runs the timer `T_Eye` (A wheel on a station's treadle and that bay's eye seeing nothing for 1.5 s: one of the two is lying) toward 1500 ms (1.5 s).

Why it is built this way:

A timer: it times only while this rung is true.

ALARMS · RUNG 5




```
[XIC(T_Move.DN),XIC(T_Eye.DN),XIC(Brush_OL)]OTL(Wash_Fault);
```

A fault stops the conveyor and every station (each output rung has XIO(Wash_Fault))

PREDICT IT · 20 `T_Move.DN` (the done bit of `T_Move` : Conveyor watchdog: 8 s (a move takes about 3.5 s)) is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When either (`T_Move.DN` (the done bit of `T_Move` : Conveyor watchdog: 8 s (a move takes about 3.5 s)) is on) *or* (`T_Eye.DN` (the done bit of `T_Eye` : A wheel on a station's treadle and that bay's eye seeing nothing for 1.5 s: one of the two is lying) is on) *or* (`Brush_OL` (Brush motor overload relay: 1 when it has tripped) is on), the rung latches `Wash_Fault` (Latched by the conveyor watchdog or the brush overload; RESET clears it) on.

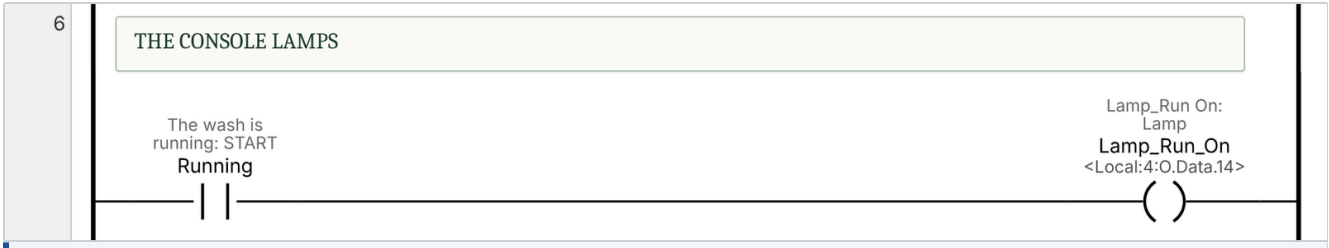
Left to right:

1. A branch of 3 parallel paths: power gets through if any one of them is true.
   - **Path 1**
     1. `XIC(T_Move.DN)` *Examine If Closed* asks: true when `T_Move.DN` (the done bit of `T_Move` : Conveyor watchdog: 8 s (a move takes about 3.5 s)) is on.
   - **Path 2**
     1. `XIC(T_Eye.DN)` *Examine If Closed* asks: true when `T_Eye.DN` (the done bit of `T_Eye` : A wheel on a station's treadle and that bay's eye seeing nothing for 1.5 s: one of the two is lying) is on.
   - **Path 3**
     1. `XIC(Brush_OL)` *Examine If Closed* asks: true when `Brush_OL` (Brush motor overload relay: 1 when it has tripped) is on.
2. `OTL(Wash_Fault)` *Output Latch* does: latches `Wash_Fault` (Latched by the conveyor watchdog or the brush overload; RESET clears it) on.

Why it is built this way:

A latch: it stays on until an OTU clears it.

ALARMS · RUNG 6




```
XIC(Running)OTE(Lamp_Run_On);
```

THE CONSOLE LAMPS

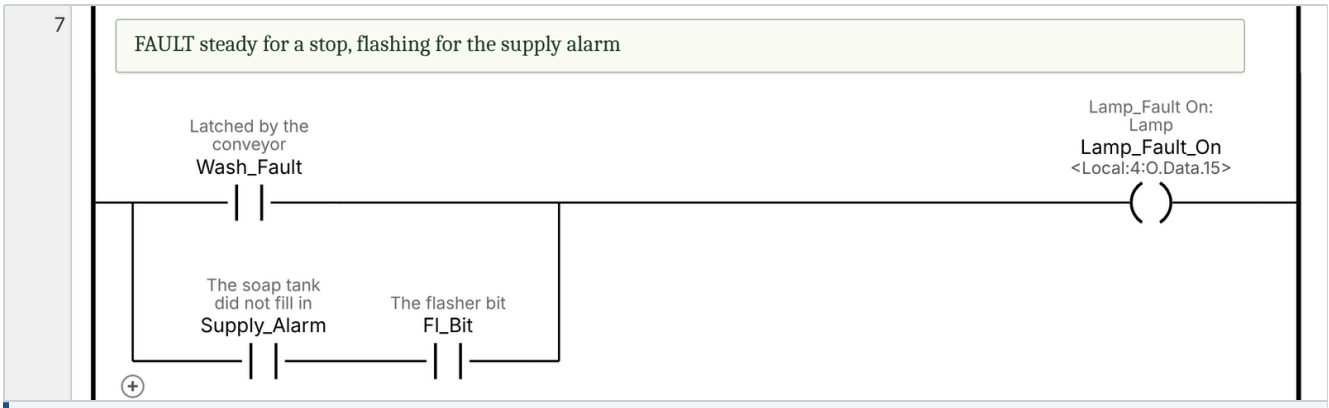
HOW THIS RUNG WORKS

In words. When `Running` (The wash is running: START seals it in, STOP (normally closed) or a fault drops it) is on, the rung turns `Lamp_Run_On` (Lamp) on (and off again whenever the rung is false).

Left to right:

1. `XIC(Running)` *Examine If Closed* asks: true when `Running` (The wash is running: START seals it in, STOP (normally closed) or a fault drops it) is on.
2. `OTE(Lamp_Run_On)` *Output Energize* does: turns `Lamp_Run_On` (Lamp) on (and off again whenever the rung is false).

ALARMS · RUNG 7




```
[XIC(Wash_Fault),XIC(Supply_Alarm)XIC(Fl_Bit)]OTE(Lamp_Fault_On);
```

FAULT steady for a stop, flashing for the supply alarm

PREDICT IT · 21 `Wash_Fault` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

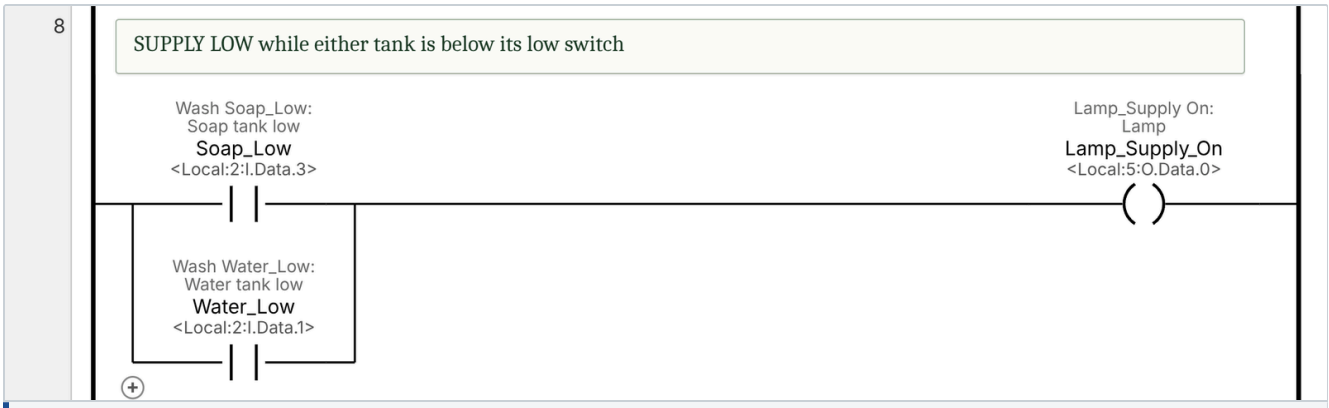
HOW THIS RUNG WORKS

In words. When either (`Wash_Fault` (Latched by the conveyor watchdog or the brush overload; RESET clears it) is on) *or* (`Supply_Alarm` (The soap tank did not fill in time (a leak, an empty drum)) is on and `Fl_Bit` (The flasher bit) is on), the rung turns `Lamp_Fault_On` (Lamp) on (and off again whenever the rung is false).

Left to right:

1. A branch of 2 parallel paths: power gets through if any one of them is true.
   - **Path 1**
     1. `XIC(Wash_Fault)` *Examine If Closed* asks: true when `Wash_Fault` (Latched by the conveyor watchdog or the brush overload; RESET clears it) is on.
   - **Path 2**
     1. `XIC(Supply_Alarm)` *Examine If Closed* asks: true when `Supply_Alarm` (The soap tank did not fill in time (a leak, an empty drum)) is on.
     2. `XIC(Fl_Bit)` *Examine If Closed* asks: true when `Fl_Bit` (The flasher bit) is on.
2. `OTE(Lamp_Fault_On)` *Output Energize* does: turns `Lamp_Fault_On` (Lamp) on (and off again whenever the rung is false).

ALARMS · RUNG 8




```
[XIC(Soap_Low),XIC(Water_Low)]OTE(Lamp_Supply_On);
```

SUPPLY LOW while either tank is below its low switch

PREDICT IT · 22 `Soap_Low` is on. What does the rung do?

Work it out from the rung before you read on. The answer is at the end of this chapter.

HOW THIS RUNG WORKS

In words. When either (`Soap_Low` (Soap tank low level switch: 1 below 20 %) is on) *or* (`Water_Low` (Water tank low level switch: 1 below 20 %) is on), the rung turns `Lamp_Supply_On` (Lamp) on (and off again whenever the rung is false).

Left to right:

1. A branch of 2 parallel paths: power gets through if any one of them is true.
   - **Path 1**
     1. `XIC(Soap_Low)` *Examine If Closed* asks: true when `Soap_Low` (Soap tank low level switch: 1 below 20 %) is on.
   - **Path 2**
     1. `XIC(Water_Low)` *Examine If Closed* asks: true when `Water_Low` (Water tank low level switch: 1 below 20 %) is on.
2. `OTE(Lamp_Supply_On)` *Output Energize* does: turns `Lamp_Supply_On` (Lamp) on (and off again whenever the rung is false).

WATCH FOR: TWO SENSORS THAT SEE THE SAME CAR

A front wheel on a station's treadle means a car stands in that bay, so that bay's eye has to see it. The cross-check rung costs one timer and catches a dead eye or a dead treadle that nothing else would: the dead brush eye would otherwise send every car out dirty, the brushes waiting politely for a car they cannot see.

Machine faults on the car wash

Tools → Inject Machine Fault… lists the faults that belong to the parts on this floor. Each has a fault card: the symptom, what to watch in the I/O to confirm it, the repair, and what a program should do about it. **Repair** on an active fault removes its cause the way a technician would; whether the machine runs again is up to the program. The reference catches all five: the conveyor watchdog the dead treadle and the broken chain, the overload by its own contact, the dead eye by the cross-check, the leak by the fill watchdog.

Fault	Watch	Repair	The program should
Car wash: brush treadle (TS3) dead The treadle at the brush station never reports the wheel.	The car rolls over TS3 and on past the brushes: TS3_Brush stays 0, the sequencer waits at TO BRUSH, the conveyor keeps going.	Replace the treadle switch (Repair).	A conveyor watchdog: a move between stations takes about 3.5 s, so the conveyor still running at 8 s latches a fault and stops everything. STOP, drive the car out, RESET.
Car wash: conveyor chain broken The conveyor motor runs and the roller does not move: the car stands where it is.	Conveyor 1, no treadle changes, the car still.	Replace the chain (Repair).	The conveyor watchdog catches it the same way: commanded for 8 s and no treadle reached.
Car wash: brush motor overload tripped The brush motor's overload relay trips: the brushes stop and Brush_OL reports it.	Brush_OL 1, the brushes still, Brushes 1 from a program that does not look at it.	Reset the overload (Repair), then RESET at the console.	Brush_OL in the brush rung (XIO) and in the fault latch: the wash stops with the car in the brush bay rather than sending it on unbrushed.
Car wash: brush photo-eye dead PE3 at the brush station never sees a car.	A car on TS3, PE3_Brush 0: the brushes never start (their rung waits for the eye) and the car is sent on dirty.	Replace the eye (Repair).	Cross-check two sensors that see the same car: a wheel on a station's treadle with that bay's eye dark for 1.5 s is a fault.
Car wash: soap tank leaking The soap tank loses soap faster than the fill puts it back.	Soap_Level falling with the pump off, Soap_Fill on and the tank never reaching Soap_High.	Repair the leak (Repair).	A fill watchdog: Soap_Fill on for 40 s without the high switch is a supply alarm; the dry-soap interlock holds the car at SOAP rather than wash it without soap.

Guided: the car wash

The scene opens with its guided track: **Init** (the table), **Run**, **Sequence** (watch POS go from HOME to LOAD and wait), **Entry** (a car pays and comes on; the sequencer moves although nothing moves yet), **Stations** (now the car goes through), **Supply** (drain the soap tank and watch it fill), **Alarms** and **Ready**. The sequencer is a lesson before the stations on purpose: you see the table being walked before anything is wired to it.

> **TRY IT: CHANGE THE RECIPE WITHOUT TOUCHING THE LOGIC**
>
> Give the brushes nine seconds: `MOV(9000,Dwell[5])` in its row of Init. Then let the dryer keep blowing while the car is carried out: change TO EXIT's output row from `2#0_0000_0010` to `2#0_0100_0010`, conveyor and dryer. Run it: no output rung changed, the dryer rung still only blows while PE6 sees the car. On a program written as a latch per step, count the rungs you would have touched.

Predict it: the answers

1. The rung is false at `XIC(Stop_PB_Pressed)`: `Stop_PB_Pressed` is off, so `Running` is **off** (an OTE writes 0 when its rung is false).
2. Every condition on the way is met, so `Wash_Fault` is unlatched, **off**.
3. Every condition on the way is met, so `Wash_In.0` is **off** (an OTE writes 0 when its rung is false), `Wash_In.1` is **off** (an OTE writes 0 when its rung is false), `Wash_In.2` is **off** (an OTE writes 0 when its rung is false), `Wash_In.3` is **off** (an OTE writes 0 when its rung is false), `Wash_In.4` is **off** (an OTE writes 0 when its rung is false), `Wash_In.5` is **off** (an OTE writes 0 when its rung is false), `Wash_In.6` is **off** (an OTE writes 0 when its rung is false), `Wash_In.7` is **on** and `Wash_In.8` is **off** (an OTE writes 0 when its rung is false).
4. The rung is false at `XIO(Soap_Hold)`: `Soap_Hold` is on, so `T_Step` is reset to zero (a TON resets when its rung goes false).
5. Every condition on the way is met, so `Pay_Lamp` is **on**.
6. Every condition on the way is met, so `Gate_Open` is **on**.
7. The rung is false at `XIO(Wash_Fault)`: `Wash_Fault` is on, so `Enter_Sign` is **off** (an OTE writes 0 when its rung is false).
8. Every condition on the way is met, so `Conveyor` is **on**.
9. The rung is false at the EQU: `Wash_Ctl.POS` (the position of `Wash_Ctl`: The sequencer: .POS is the step (0 HOME, 1 LOAD ... 13 EXIT), .LEN 13; SQO moves it, SQI reads it) is not **3**, so `Soap_Hold` is **off** (an OTE writes 0 when its rung is false).
10. Every condition on the way is met, so `Soap_Valve` is **on** and `Soap_Pump` is **on**.
11. The rung is false at `XIC(PE3_Brush)`: `PE3_Brush` is off, so `Brushes` is **off** (an OTE writes 0 when its rung is false).
12. Every condition on the way is met, so `Rinse_Valve` is **on**.
13. The rung is false at `XIO(Wash_Fault)`: `Wash_Fault` is on, so `Wax_Valve` is **off** (an OTE writes 0 when its rung is false).
14. Every condition on the way is met, so `Dryer` is **on**.
15. Every condition on the way is met, so `Water_Pump` is **on**.
16. The rung is false at `XIO(Water_High)`: `Water_High` is on, so `Water_Fill` is **off** (an OTE writes 0 when its rung is false).

17. Every condition on the way is met, so `Soap_Fill` is **on**.
18. The rung is false at `XIC(Reset_PB_Pressed)`: `Reset_PB_Pressed` is off, so the OTU does nothing: `Supply_Alarm` keeps whatever it had.
19. Every condition on the way is met, so `T_Eye` times toward its preset.
20. Every condition on the way is met, so `Wash_Fault` latches **on**.
21. Every condition on the way is met, so `Lamp_Fault_On` is **on**.
22. Every condition on the way is met, so `Lamp_Supply_On` is **on**.

52 Where to go next

Build cells of your own

Every part in Part III works in an empty sandbox the way it does on the template, and the program's test suite builds a scrap cell from the parts list in an empty floor every release: a Raised Conveyor with a Bottle Feeder, a Broken-bottle Eye, a pivot gate over a Grinder, a Box Conveyor with a Box Feeder and Box Photoeye, a second gate over a Divert Conveyor, and six rungs — a run bit, the belts, a gate on its own photoeye, the box conveyor on LS, the shredder on an empty box. It runs. Start there, then:

- **A continuous line.** No indexing, no shift registers: every station acts on its own photoeye (`Detect` on a gate is the eye just ahead of the arm). Compare the two styles.
- **A second labeler.** The line's labeler (step 10) is RH with a wrap-around roller. Put a second one on the other side (LH, tamp pad) two stops further and give it its own routine — the two handshakes are identical; only the register bits move. Then a photoeye set to *labeled bottles* at the end of the line counts the finished product.
- **A Revolver Feeder cell.** Feed one part per pulse from the rotary magazine: `XIO(T1.DN)TON(T1,1000,0); XIC(T1.DN)XIC(Rev_PartReady)OTE(Rev_Index);`, run it dry, let it reload.
- **A sorter.** Inductive and capacitive sensors tell metal from plastic, a colour sensor tells red from blue, a light array measures height, a weighing conveyor weighs; pushers and bins sort. The levels are this with a score.
- **A filling cell with a Dispenser** and a Fill Level Sensor instead of the fill station: the level sensor tells you when to close the valve.
- **Vision and RFID.** The Vision Sensor can be a camera module from an EDS file (I/O Configuration → Ethernet) with trigger, results and inspection-complete tags; the RFID reader reads and writes a tag's memory on each part.

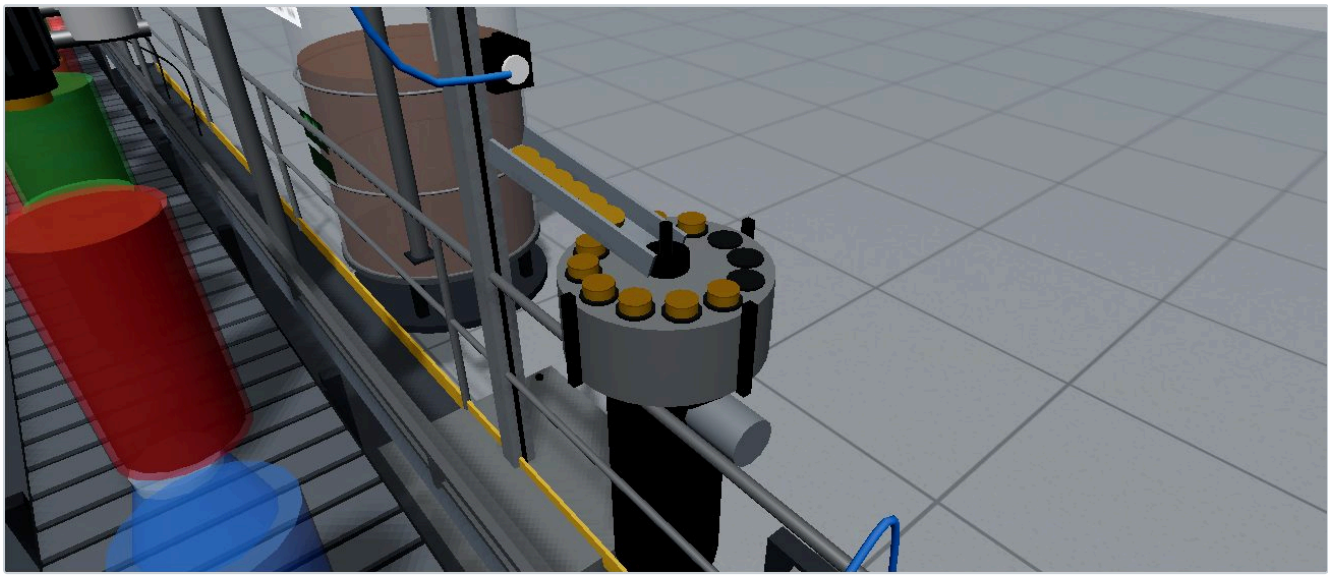


Figure 194. The rotary cap magazine, loaded, indexing a pocket per cap: the same mechanism is the Revolver Feeder part, which feeds any part one per pulse.

Real hardware

Bridge & Tools runs a small Python bridge on your PC that connects the page to the outside: a TCP socket a vision system or a test bench talks to (the *TCP Client Socket* part), and EtherNet/IP devices. Communications → Start Bridge, then the socket part's settings. Help → Bridge & Tools covers it.

Studio 5000

Export L5X and import the file into Studio 5000: the tags, aliases and routines come across, and the program that ran the simulated line is the program you would download to a CompactLogix driving the real one — the I/O addresses are already in the right shape. Going the other way, Import L5X brings a real project's routines in to run against a sandbox machine you lay out to match.

Appendix A — The bottle line tag list

Tag	Type	Alias for	Description
`Start`	BOOL	`Local:3:I.Data.9`	LogixPro I:1/0 · Start push button, NO
`Stop`	BOOL	`Local:3:I.Data.10`	LogixPro I:1/1 · Stop push button, NC (1 until pressed)
`LS1`	BOOL	`Local:1:I.Data.4`	LogixPro I:1/6 · Exist sensor: a bottle at the start of the line
`LS2`	BOOL	`Local:1:I.Data.5`	LogixPro I:1/7 · Size check: beam set so only a large bottle breaks it
`LS3`	BOOL	`Local:1:I.Data.6`	LogixPro I:1/8 · Broken check: sees broken bottles only
`LS4`	BOOL	`Local:1:I.Data.9`	LogixPro I:1/9 · Fill tube down
`LS5`	BOOL	`Local:1:I.Data.10`	LogixPro I:1/10 · Large charge delivered
`LS6`	BOOL	`Local:1:I.Data.11`	LogixPro I:1/11 · Small charge delivered
`LS7`	BOOL	`Local:1:I.Data.13`	LogixPro I:1/12 · Cap ram down
`LS8`	BOOL	`Local:2:I.Data.7`	LogixPro I:1/13 · Bottle at the scrap gate
`LS9`	BOOL	`Local:2:I.Data.11`	LogixPro I:1/14 · Bottle at the divert gate
`LS10`	BOOL	`Local:3:I.Data.0`	LogixPro I:1/15 · Scrap box under the grinder chute
`Main_Conveyor`	BOOL	`Local:4:O.Data.1`	LogixPro O:2/0 · Main conveyor motor
`Scrap_Conveyor`	BOOL	`Local:5:O.Data.4`	LogixPro O:2/1 · Box conveyor under the grinder
`Divert_Conveyor`	BOOL	`Local:5:O.Data.3`	LogixPro O:2/2 · Divert conveyor for large bottles
`Grinder`	BOOL	`Local:5:O.Data.1`	LogixPro O:2/3 · Grinder motor
`Scrap_Gate`	BOOL	`Local:5:O.Data.0`	LogixPro O:2/4 · Scrap gate: swings a bottle into the grinder
`Divert_Gate`	BOOL	`Local:5:O.Data.2`	LogixPro O:2/5 · Divert gate: swings a bottle onto the divert conveyor
`Fill_Tube`	BOOL	`Local:4:O.Data.6`	LogixPro O:2/6 · Lower the fill tube
`Large_Fill`	BOOL	`Local:4:O.Data.7`	LogixPro O:2/7 · Large fill valve
`Small_Fill`	BOOL	`Local:4:O.Data.8`	LogixPro O:2/8 · Small fill valve
`Cap_Ram`	BOOL	`Local:4:O.Data.9`	LogixPro O:2/9 · Cap ram
`Run`	BOOL	`Local:5:O.Data.8`	LogixPro O:2/10 · Run lamp

Tag	Type	Alias for	Description
`Cap_Feeder`	BOOL	`Local:4:O.Data.10`	Capper Feeder: runs the vibratory cap bowl, refilling the chute
`Label_Apply`	BOOL	`Local:4:O.Data.13`	Labeler Start: the applicator comes in and presses a label on
`Label_PadIn`	BOOL	`Local:2:I.Data.0`	Labeler applicator at the end of its stroke (label pressed on)
`Label_Ready`	BOOL	`Local:2:I.Data.1`	Labeler ready: web loaded, head home, no fault
`Label_Error`	BOOL	`Local:2:I.Data.2`	Labeler fault: end of roll, or the last cycle found no label
`Label_Low`	BOOL	`Local:2:I.Data.3`	Labeler roll low
`Label_Present`	BOOL	`Local:2:I.Data.4`	A label is peeled and waiting at the dispensing edge
`Label_Done`	BOOL	`Local:2:I.Data.5`	The bottle under the applicator already carries a label
`Label_Green`	BOOL	`Local:4:O.Data.14`	Labeler green indicator
`Label_Amber`	BOOL	`Local:4:O.Data.15`	Labeler amber indicator
`LS13`	BOOL	`Local:3:I.Data.2`	Box at the hazard-tape dispenser
`Haz_Apply`	BOOL	`Local:5:O.Data.5`	Hazard-tape dispenser: start the wipe-on cycle
`Haz_PadIn`	BOOL	`Local:3:I.Data.3`	Hazard-tape dispenser: applicator at the end of its stroke
`Haz_Ready`	BOOL	`Local:3:I.Data.4`	Hazard-tape dispenser ready: web loaded, applicator home, no fault
`Haz_Error`	BOOL	`Local:3:I.Data.5`	Hazard-tape dispenser fault: end of roll, or the last cycle found no tape
`Haz_Low`	BOOL	`Local:3:I.Data.6`	Hazard-tape roll low
`Haz_Present`	BOOL	`Local:3:I.Data.7`	A tape band is peeled and waiting at the dispensing edge
`Haz_Done`	BOOL	`Local:3:I.Data.8`	The box under the applicator already carries its hazard tape
`Haz_Green`	BOOL	`Local:5:O.Data.6`	Hazard-tape dispenser green indicator
`Haz_Amber`	BOOL	`Local:5:O.Data.7`	Hazard-tape dispenser amber indicator
`Haz_Req`	BOOL		Hazard tape requested for the box at LS13 (latched while the applicator cycles)
`Haz_Fault`	BOOL		Hazard-tape dispenser fault, latched until Enter
`Cap_Green`	BOOL	`Local:4:O.Data.11`	Capper green indicator (column)

Tag	Type	Alias for	Description
`Cap_Amber`	BOOL	`Local:4:O.Data.12`	Capper amber indicator (column)
`Cap_Low`	BOOL	`Local:1:I.Data.14`	Capper CapLow: the chute is down to its low count
`Cap_Present`	BOOL	`Local:1:I.Data.15`	Capper CapPresent: a cap is waiting at the chuck
`Cap_Fill_T`	TIMER		Cap feeder run-on after CapLow clears
`Thumbwheel`	INT	`Local:9:I.Ch0Data`	LogixPro I:3 · Thumbwheel switch, read as a number (BCD on the real one)
`Enter`	BOOL	`Local:3:I.Data.13`	Thumbwheel Enter button
`LS11`	BOOL	`Local:3:I.Data.11`	End-of-line eye at 1240: verifies the tracking (only large, whole bottles reach it)
`Main_Conv_Running`	BOOL	`Local:1:I.Data.1`	Main conveyor running feedback (motor contactor)
`Divert_Conv_Running`	BOOL	`Local:2:I.Data.12`	Divert conveyor running feedback
`Fault_Lamp`	BOOL	`Local:5:O.Data.9`	Red fault lamp on the console: flashing = unacknowledged alarm, steady = alarm still active
`Reset_PB`	BOOL	`Local:3:I.Data.12`	Blue Reset button: acknowledge alarms and clear the ones whose cause is gone
`Case_Run`	BOOL	`Local:5:O.Data.13`	Case lane drive: run (two-wire)
`Case_VFD_Reset`	BOOL	`Local:5:O.Data.15`	Case lane drive: fault reset (rising edge)
`Case_Speed_Ref`	INT	`Local:8:O.Ch0Data`	Case lane drive speed reference: 0-10000 = 0-10 V = 0-60 Hz
`Case_VFD_Ready`	BOOL	`Local:10:I.Data.10`	Case lane drive ready (not tripped)
`Case_VFD_Running`	BOOL	`Local:10:I.Data.11`	Case lane drive output frequency above zero
`Case_At_Speed`	BOOL	`Local:10:I.Data.12`	Case lane drive at the commanded speed
`Case_VFD_Fault`	BOOL	`Local:10:I.Data.13`	Case lane drive tripped
`Case_Hz_Fbk`	INT	`Local:9:I.Ch3Data`	Case lane drive output frequency: 0-10000 = 0-10 V = 0-60 Hz
`Case_Slow`	BOOL	`Local:10:I.Data.9`	Slow-down eye on the case lane, 100 px before the packer head
`Case_In_Place`	BOOL	`Local:10:I.Data.1`	A case is centred under the packer head
`Case_Full`	BOOL	`Local:10:I.Data.2`	The case under the head has been packed
`Group_Ready`	BOOL	`Local:3:I.Data.15`	The packer infeed holds a case worth (4)

Tag	Type	Alias for	Description
`Infeed_Full`	BOOL	`Local:10:I.Data.0`	Packer infeed buffer full: bottles arriving now wait on the belt
`Infeed_Count`	INT	`Local:9:I.Ch1Data`	Bottles in the packer infeed
`Packer_Ready`	BOOL	`Local:3:I.Data.14`	Packer head home, not cycling
`Pack_Drop`	BOOL	`Local:5:O.Data.10`	Packer: pick a case worth from the infeed and place it in the case (rising edge)
`Pick_Lower`	BOOL	`Local:5:O.Data.11`	Packer: the next Drop picks from the lower lane (small bottles) instead of the deck infeed
`Lower_Ready`	BOOL	`Local:10:I.Data.3`	The packer lower lane holds a case worth (4 small bottles)
`Lower_Full`	BOOL	`Local:10:I.Data.4`	Packer lower lane full: small bottles arriving now wait at the stop
`Lower_Count`	INT	`Local:9:I.Ch2Data`	Small bottles waiting in the packer lower lane
`Case_Small`	BOOL	`Local:10:I.Data.5`	The packed case under the head holds small bottles
`LS12`	BOOL	`Local:2:I.Data.13`	Stop eye at the end of the divert conveyor, just before the packer lower lane
`Large_Bottle_Count`	DINT		LogixPro O:4/0 · Large bottle count, shown on the LARGE display
`Small_Bottle_Count`	DINT		LogixPro O:4/1 · Small bottle count, shown on the SMALL display
`Scrap_Bottle_Count`	DINT		LogixPro O:4/2 · Scrap bottle count, shown on the SCRAP display
`Scrap_Box_Count`	DINT		LogixPro O:4/3 · Scrap box count, shown on the BOXES display
`Main_Conv_A`	BOOL	`Local:1:I.Data.2`	Main_Conv encoder channel A: one rising edge per 40 px pitch
`Bottle_Feed_Emit`	BOOL	`Local:4:O.Data.0`	Bottle_Feed Emit: one bottle per rising edge
`Line_Run`	BOOL		Seal-in run bit (LogixPro B3:0/0)
`Conv_Index`	BOOL		Latched while the belt steps one pitch
`A_ONS`	BOOL		One-shot storage for the encoder edge
`Dwell`	TIMER		Belt stopped: sensors read, feeder drops a bottle
`Exist`	DINT[2]		Bottle present, one bit per stop (LogixPro B3:10). During the dwell: filler 6, capper 9, scrap gate 13, divert gate 20; after the shift (Dwell.DN) one bit higher

Tag	Type	Alias for	Description
`Large`	DINT[2]		Large bottle (LogixPro B3:20). During the dwell: filler 5, capper 8, scrap gate 12, divert gate 19; after the shift one bit higher
`Broken`	DINT[2]		Broken bottle (LogixPro B3:30). During the dwell: filler 4, capper 7, scrap gate 11, divert gate 18; after the shift one bit higher
`Exist_Ctl`	CONTROL		LogixPro R6:1
`Large_Ctl`	CONTROL		LogixPro R6:2
`Broken_Ctl`	CONTROL		LogixPro R6:3
`Fill_Req`	BOOL		A bottle stands under the filler and still needs its charge
`Fill_Done`	BOOL		Latched when the charge is in; cleared when the belt steps
`Cap_Req`	BOOL		A bottle stands under the cap ram and has not been capped
`Cap_Done`	BOOL		Latched when the ram has been down; cleared when the belt steps
`Label_Req`	BOOL		Latched while the labeler is asked to apply; cleared at PadIn
`Label_Applied`	BOOL		Latched when the label is pressed on; cleared when the belt steps
`Label_Fault`	BOOL		Latched on the labeler Error input (end of roll or a missed label); Enter clears it
`Alarm_Word`	DINT		Latched alarms, one bit each: 0 main conveyor overload, 1 divert conveyor overload, 2 end-of-line check failed (a jam, a lost bottle, tracking out of step), 3 LS1 dark too long (dirty lens or feeder empty), 4 labeler fault, 5 reservoir inlet open but level not rising, 6 case lane drive tripped, 7 packer infeed full
`Alarm_Unack`	BOOL		New alarm, not yet acknowledged (Reset)
`Main_OL_T`	TIMER		Main conveyor overload timer (no running feedback)
`Div_OL_T`	TIMER		Divert conveyor overload timer (no running feedback)
`EOL_Missing`	BOOL		LS11 dark when the registers said a large, whole bottle was there
`EOL_Extra`	BOOL		LS11 saw a bottle the registers did not expect

Tag	Type	Alias for	Description
`EOL_Inhibit`	BOOL		Tracking cleared by Reset: the end-of-line check waits until the first bottle tracked since then reaches LS11
`EOL_ONS`	BOOL		
`LS1_Dark_T`	TIMER		LS1 dark too long timer
`Inlet_T`	TIMER		Inlet not filling timer
`Flash_T`	TIMER		Fault lamp flasher
`OL_ONS`	BOOL		
`DOL_ONS`	BOOL		
`BLK_ONS`	BOOL		
`Reg_Clear`	BOOL		
`DRK_ONS`	BOOL		
`LBL_ONS`	BOOL		
`INL_ONS`	BOOL		
`RST_ONS`	BOOL		
`VFD_ONS`	BOOL		
`INF_ONS`	BOOL		
`Case_Hz`	REAL		Case lane speed the program wants, in Hz
`Case_Hz_Actual`	REAL		Case lane drive output frequency read back, in Hz
`Case_Count`	DINT		Cases packed, shown on the CASES display
`Case_Creep`	BOOL		Latched from the slow-down eye until an empty case stands under the head
`Drop_ONS`	BOOL		
`Case_ONS`	BOOL		
`Box_Full`	BOOL	`Local:2:I.Data.9`	Grinder BoxFull: the box under the scrap drop holds its share
`Box_Level`	INT	`Local:7:I.Ch3Data`	Grinder BoxLevel: bottles in the box under the chute (the IN BOX display)
`Scrap_Waiting`	BOOL	`Local:2:I.Data.8`	Grinder Detect: a broken bottle is waiting in the hopper
`Scrap_ONS`	BOOL		

Tag	Type	Alias for	Description
`Divert_ONS`	BOOL		
`Small_ONS`	BOOL		
`Large_ONS`	BOOL		
`Box_ONS`	BOOL		
`Enter_ONS`	BOOL		
`Batch_Done`	BOOL		The large-bottle count has reached the thumbwheel preset
`Tank_Level`	INT	`Local:7:I.Ch2Data`	Filler head tank, 0-100 %
`Tank_Low`	BOOL	`Local:1:I.Data.12`	Filler head tank below 15 %
`Supply_Low`	BOOL	`Local:1:I.Data.8`	Supply_Tank low float
`Supply_High`	BOOL	`Local:1:I.Data.7`	Supply_Tank high float
`Supply_Inlet`	BOOL	`Local:4:O.Data.2`	Supply_Tank inlet valve
`Supply_Level`	INT	`Local:7:I.Ch1Data`	Supply_Tank level, 0-100 %
`Mix_PumpA`	BOOL	`Local:6:O.Data.0`	Mix_Tank pump A: water
`Mix_PumpB`	BOOL	`Local:6:O.Data.1`	Mix_Tank pump B: syrup
`Mix_PumpOut`	BOOL	`Local:6:O.Data.2`	Mix_Tank outlet pump: transfers to Supply_Tank
`Mix_Heater`	BOOL	`Local:6:O.Data.3`	Mix_Tank heater
`Mix_Mixer`	BOOL	`Local:6:O.Data.4`	Mix_Tank mixer
`Mix_FlowA`	BOOL	`Local:12:I.Data.0`	Mix_Tank flowmeter A: one pulse per 20 units
`Mix_FlowOut`	BOOL	`Local:12:I.Data.2`	Mix_Tank outlet flowmeter: one pulse per 20 units
`Mix_Hi`	BOOL	`Local:12:I.Data.3`	Mix_Tank HI probe (90 %)
`Mix_Lo`	BOOL	`Local:12:I.Data.4`	Mix_Tank LO probe (15 %)
`Mix_Thermostat`	BOOL	`Local:12:I.Data.5`	Mix_Tank high-limit thermostat (180 °F)
`Mix_Level`	INT	`Local:11:I.Ch0Data`	Mix_Tank level, 0-100 %
`Mix_Temp`	INT	`Local:11:I.Ch1Data`	Mix_Tank temperature, °F
`Water_Low`	BOOL	`Local:10:I.Data.14`	Supply_A (water tote) at its low switch
`Conc_Low`	BOOL	`Local:10:I.Data.15`	Supply_B (syrup tote) at its low switch
`Mix_Start`	BOOL		START on Tank_HMI (pulsed by the touch screen)
`Mix_Stop`	BOOL		STOP on Tank_HMI (pulsed by the touch screen)

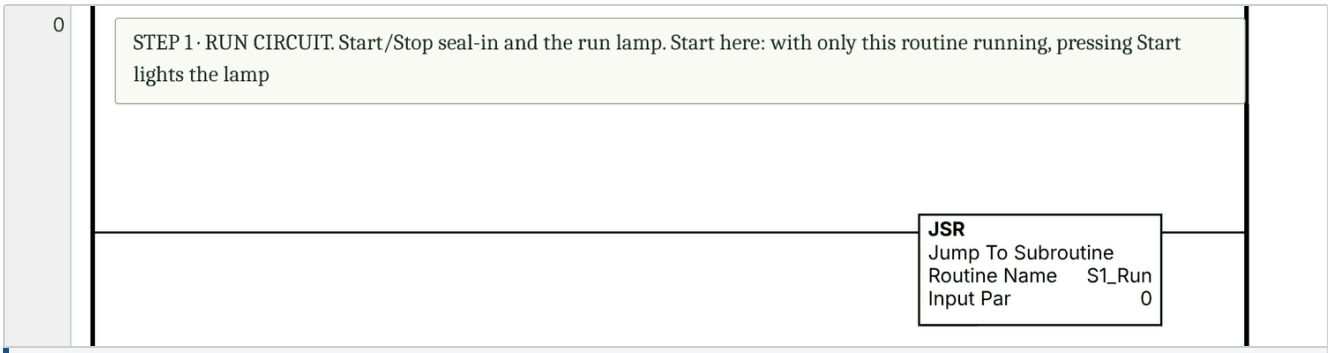
Tag	Type	Alias for	Description
`Mix_Run`	BOOL		The mix kitchen is running (its own seal-in)
`Mix_Auto_Start`	BOOL		One scan when the line starts: starts the kitchen too
`Mix_Line_ONS`	BOOL		
`Mix_Step`	DINT		Kitchen sequence: 0 empty, 1 dose water, 2 syrup to HI, 3 heat and mix, 4 ready / transfer
`Dose_Pre`	DINT		Water dose for this batch, in flowmeter pulses
`Mix_Ms`	DINT		Mixing time in ms (Mix_Time × 1000)
`Dose_A`	COUNTER		Water dose: flowmeter A pulses
`Mix_Timer`	TIMER		Mixing time at temperature
`Mix_Empty_T`	TIMER		LO uncovered this long: the tank is empty
`Mix_Heat`	BOOL		Heater wanted (latched under the band, unlatched at the setpoint)
`Mix_Target_T`	REAL		Batch temperature, °F: the setpoint on Tank_HMI (TEMP page)
`Mix_Lo_T`	REAL		Bottom of the heater band: 2 °F under Mix_Target_T
`Mix_Ratio`	REAL		Water share of the batch, %: A SHARE on the RECIPE page
`Mix_Time`	DINT		Mixing time, s: MIX TIME on the RECIPE page
`Carb_CO2`	BOOL	`Local:6:O.Data.5`	Carbonator CO2 inlet valve
`Carb_Press`	INT	`Local:11:I.Ch2Data`	Carbonator tank pressure, psi
`Carb_Vol`	INT	`Local:11:I.Ch3Data`	CO2 in the soda going to Supply_Tank, hundredths of a volume (350 = 3.50)
`Case_Large_Count`	DINT		Cases of large bottles packed (CASES LARGE on Pack_HMI)
`Case_Small_Count`	DINT		Cases of small bottles packed (CASES SMALL on Pack_HMI)
`Pack_Start`	BOOL		START on Pack_HMI (pulsed by the touch screen)
`Pack_Stop`	BOOL		STOP on Pack_HMI (pulsed by the touch screen)
`Pack_Run`	BOOL		The packer is running (its own seal-in)
`Pack_Auto_Start`	BOOL		One scan when the line starts: starts the packer too

Tag	Type	Alias for	Description
`Pack_Line_ONS`	BOOL		
`Tank_Red`	BOOL	`Local:4:O.Data.3`	Tank_Light red: reservoir at the low float (flashes)
`Tank_Amber`	BOOL	`Local:4:O.Data.4`	Tank_Light amber: inlet open, refilling
`Tank_Green`	BOOL	`Local:4:O.Data.5`	Tank_Light green: level healthy

Appendix B — The complete program

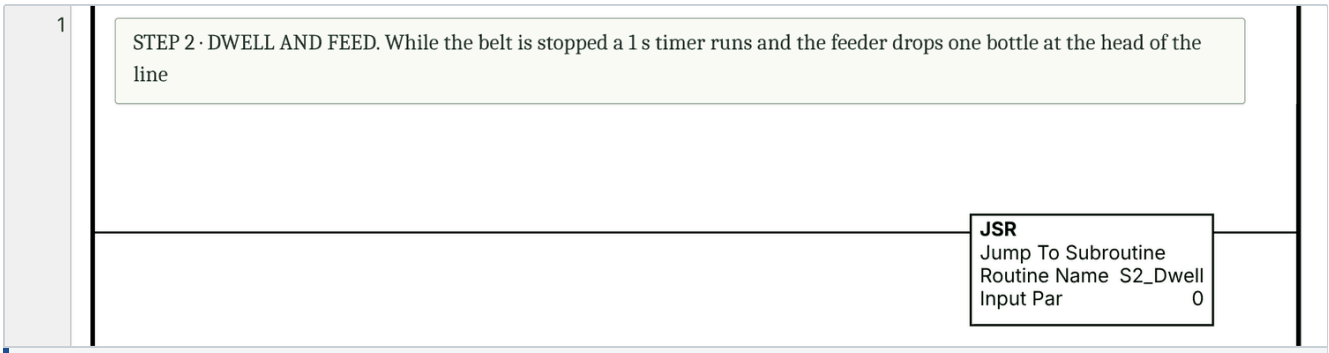
MainRoutine

RUNG 0



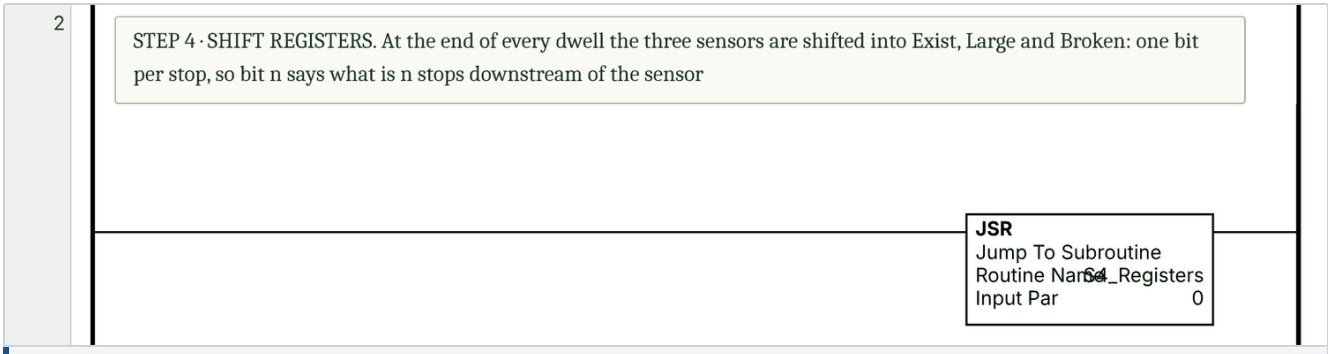

```
JSR(S1_Run);
```

RUNG 1



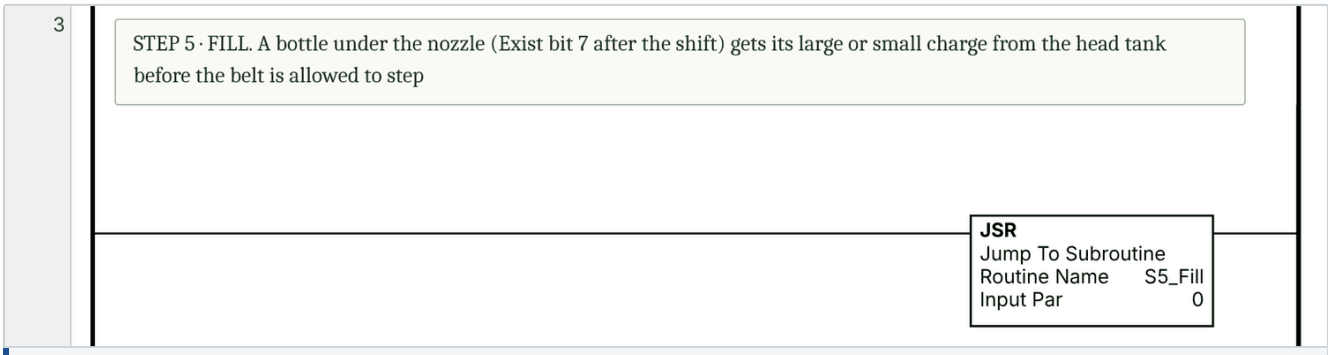

```
JSR(S2_Dwell);
```

RUNG 2



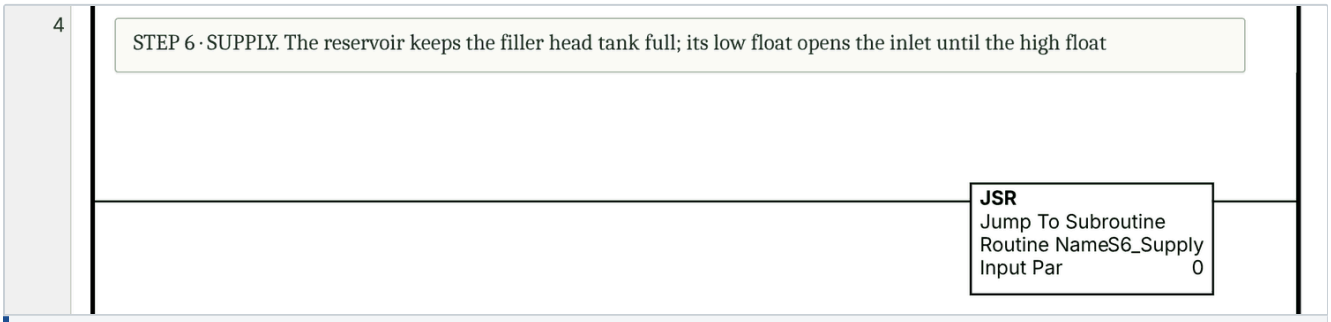

```
JSR(S4_Registers);
```

RUNG 3



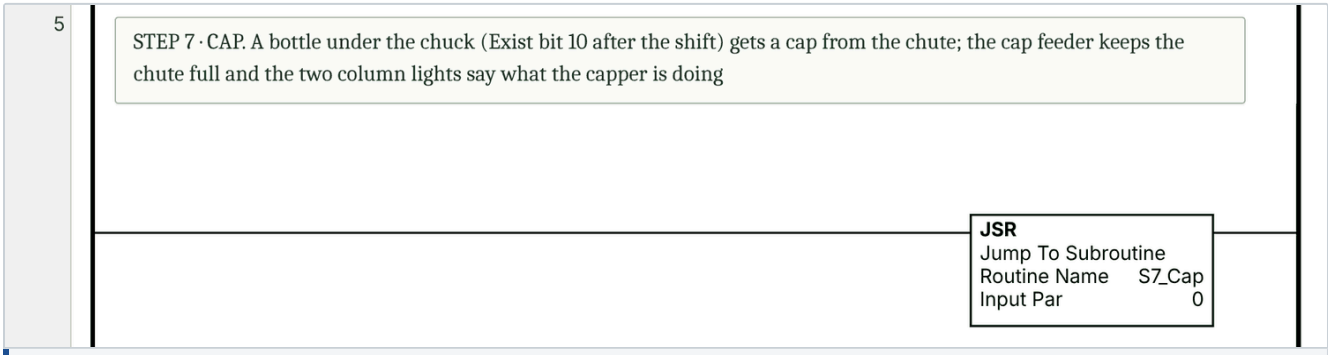

```
JSR(S5_Fill);
```

RUNG 4



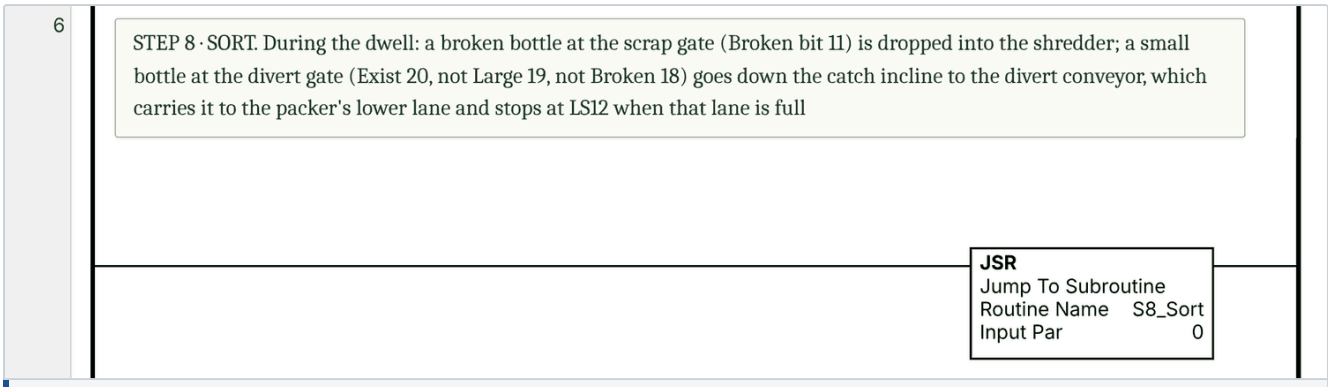

```
JSR(S6_Supply);
```

RUNG 5



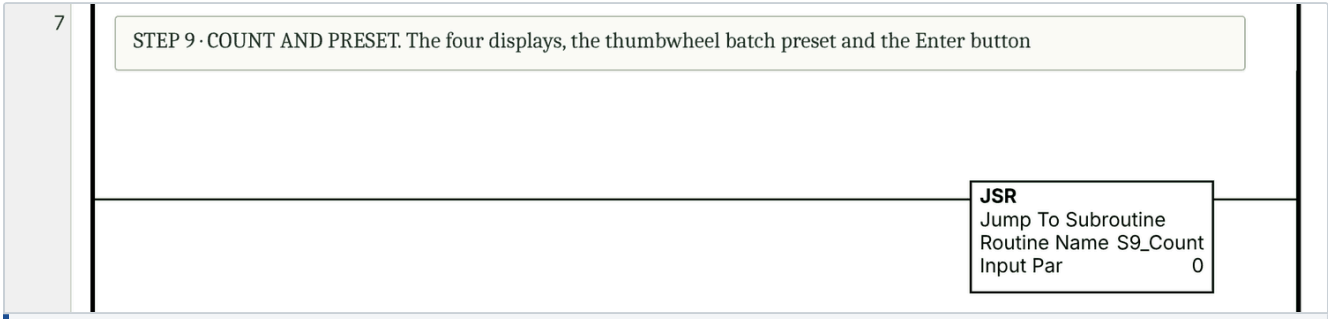

```
JSR(S7_Cap);
```

RUNG 6



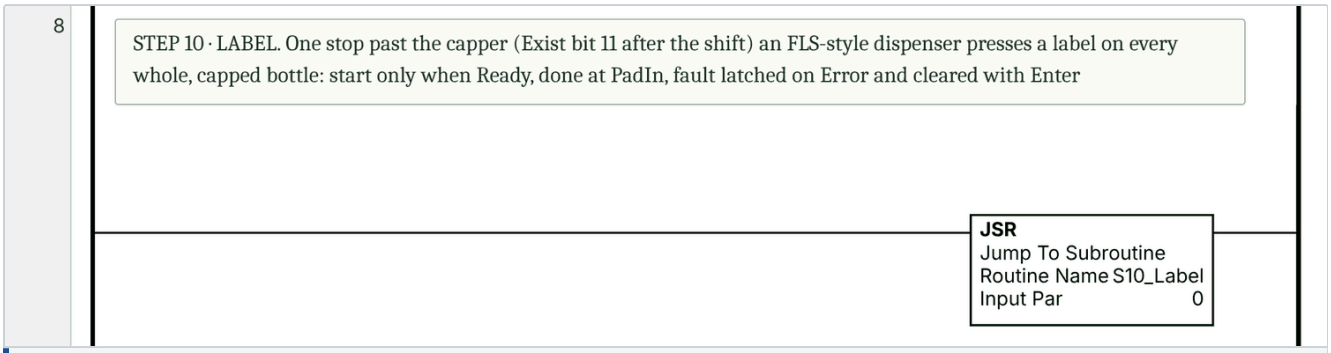

```
JSR(S8_Sort);
```

RUNG 7



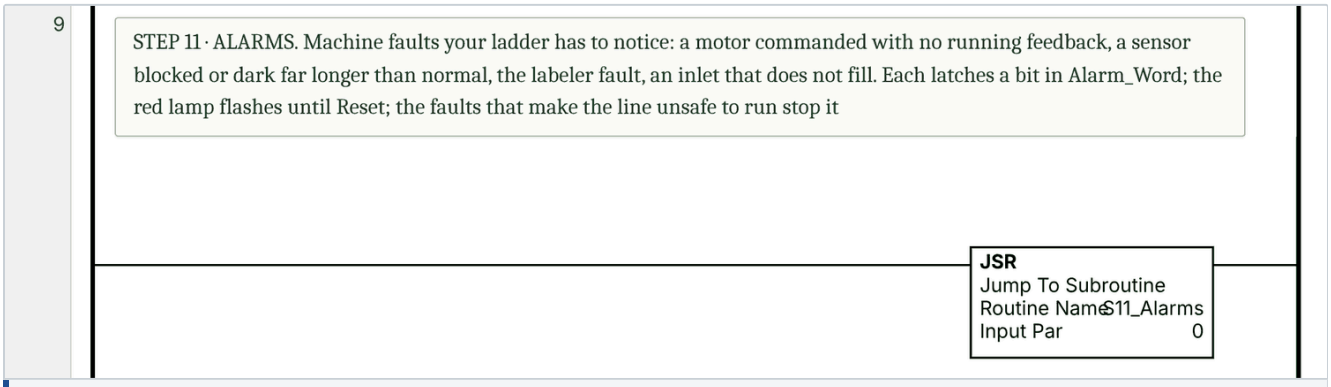

```
JSR(S9_Count);
```

RUNG 8



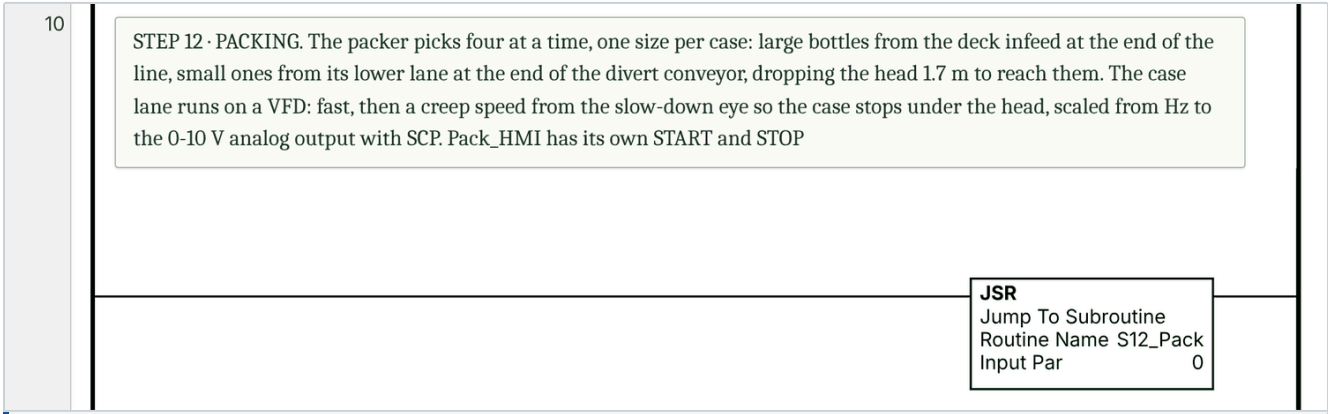

```
JSR(S10_Label);
```

RUNG 9



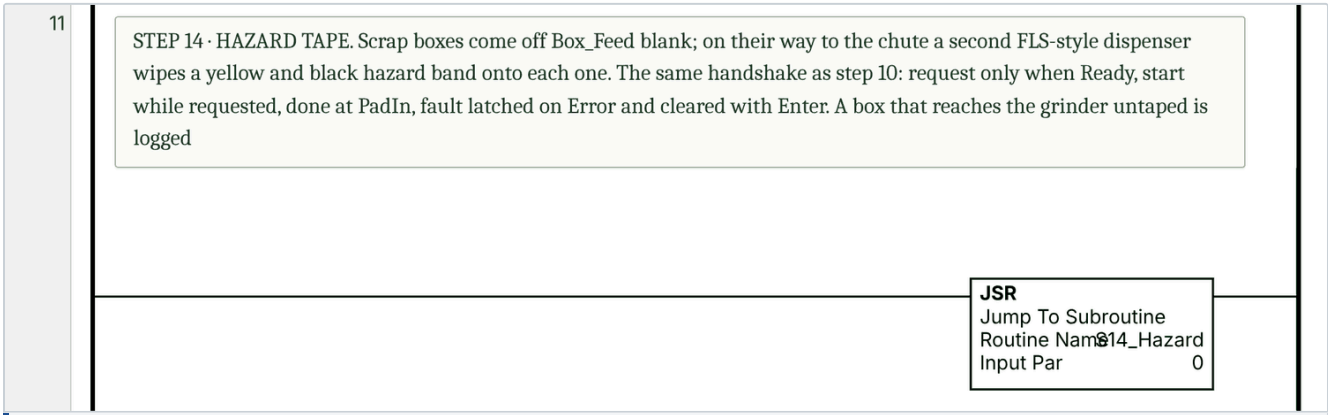

```
JSR(S11_Alarms);
```

RUNG 10



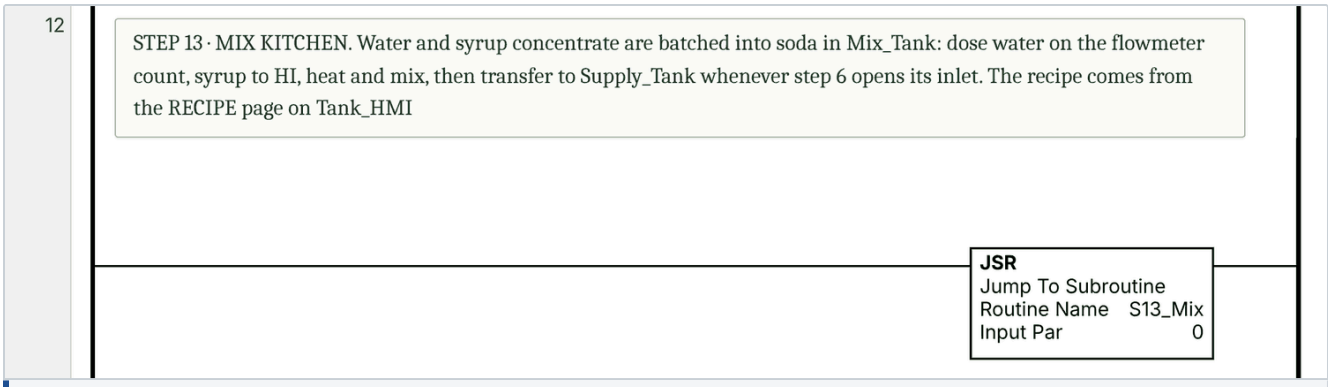

```
JSR(S12_Pack);
```

RUNG 11



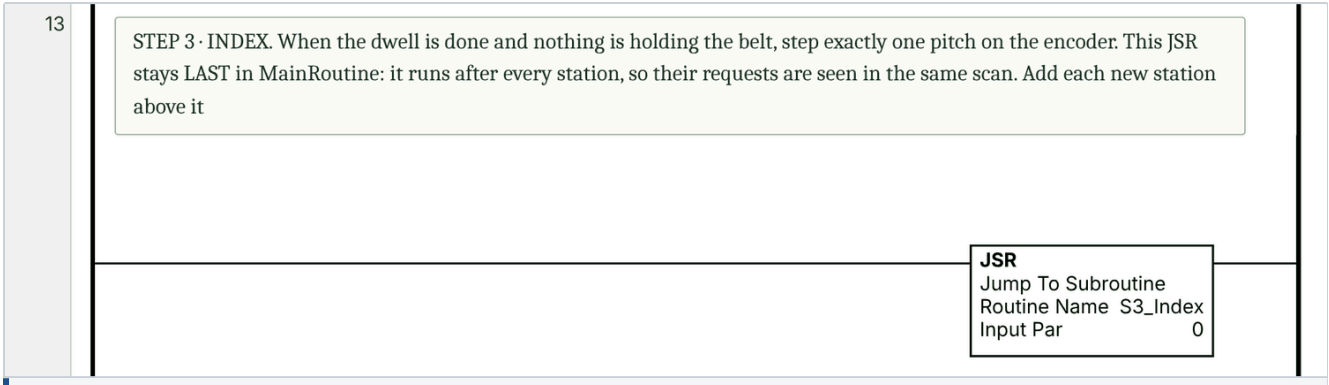

```
JSR(S14_Hazard);
```

RUNG 12




```
JSR(S13_Mix);
```

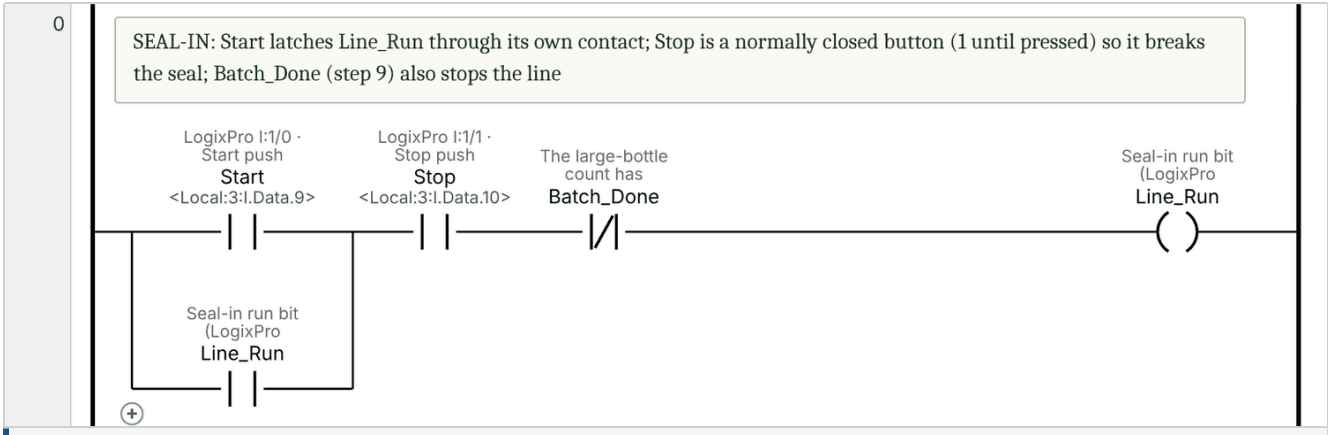
RUNG 13




```
JSR(S3_Index);
```

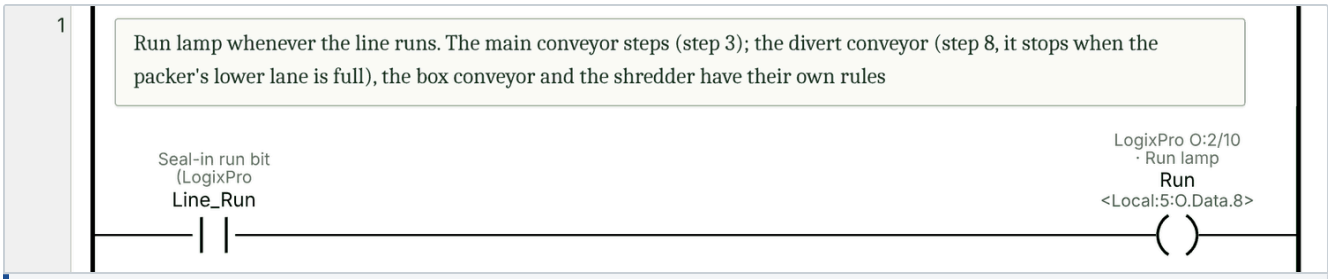
S1_Run

RUNG 0



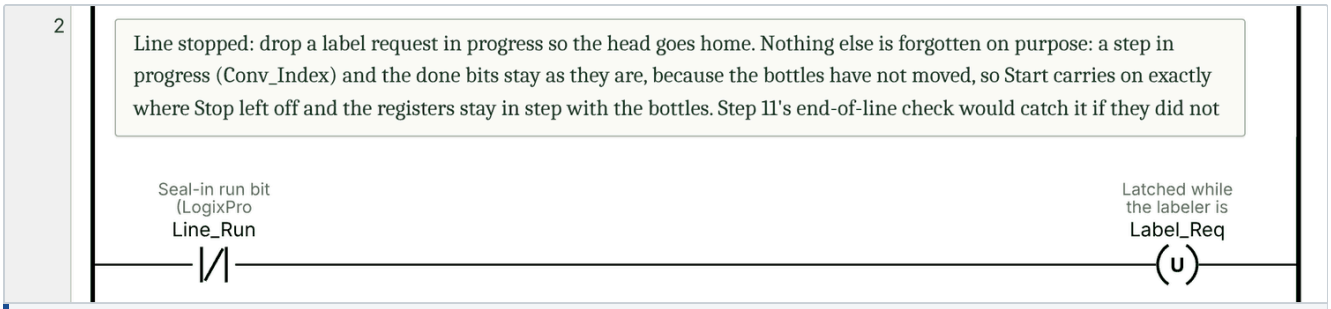

```
[XIC(Start),XIC(Line_Run)]XIC(Stop)XIO(Batch_Done)OTE(Line_Run);
```

RUNG 1




```
XIC(Line_Run)OTE(Run);
```

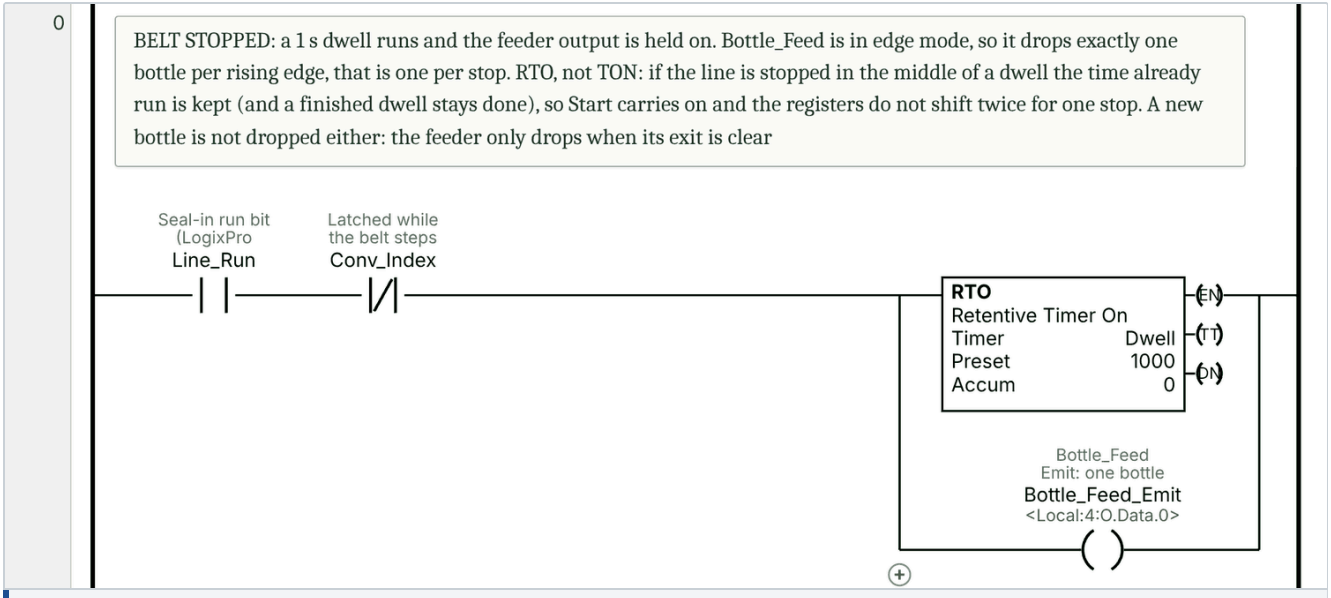
RUNG 2




```
XIO(Line_Run)OTU(Label_Req);
```

S2_Dwell

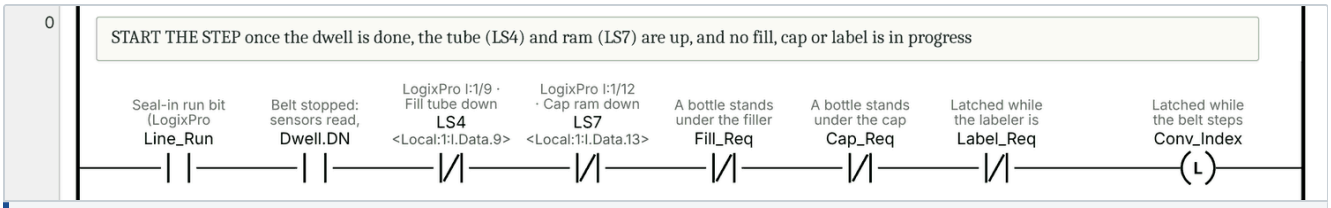
RUNG 0




```
XIC(Line_Run)XIO(Conv_Index)[RTO(Dwell,1000,0),OTE(Bottle_Feed_Emit)];
```

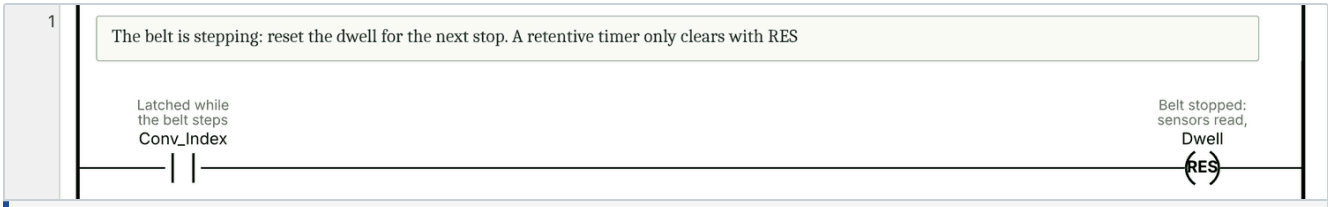
S3_Index

RUNG 0



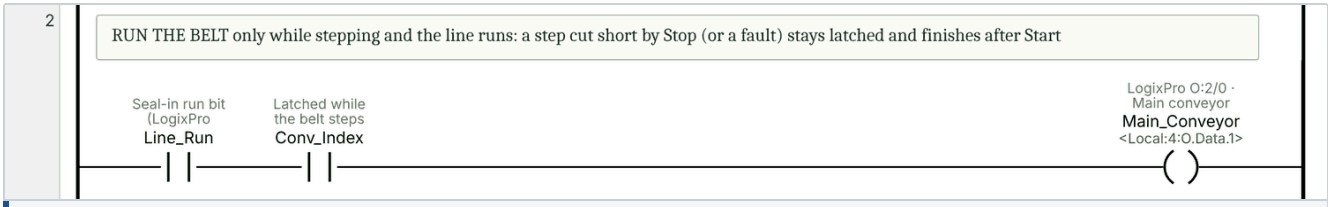

```
XIC(Line_Run)XIC(Dwell.DN)XIO(LS4)XIO(LS7)XIO(Fill_Req)XIO(Cap_Req)XIO(Label_Req)OTL(Conv_Index);
```

RUNG 1



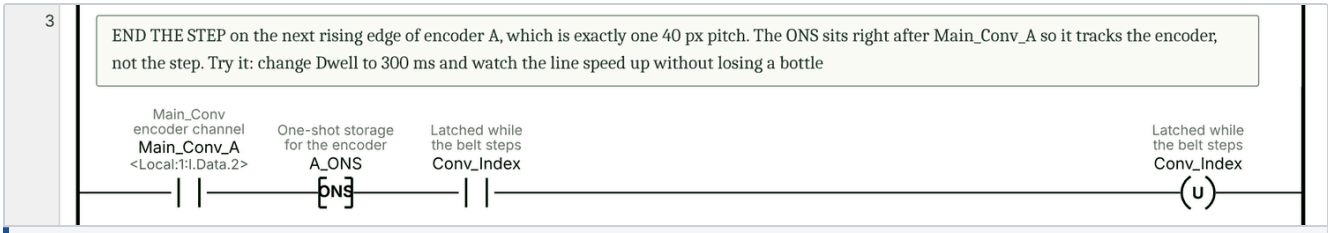

```
XIC(Conv_Index)RES(Dwell);
```

RUNG 2




```
XIC(Line_Run)XIC(Conv_Index)OTE(Main_Conveyor);
```

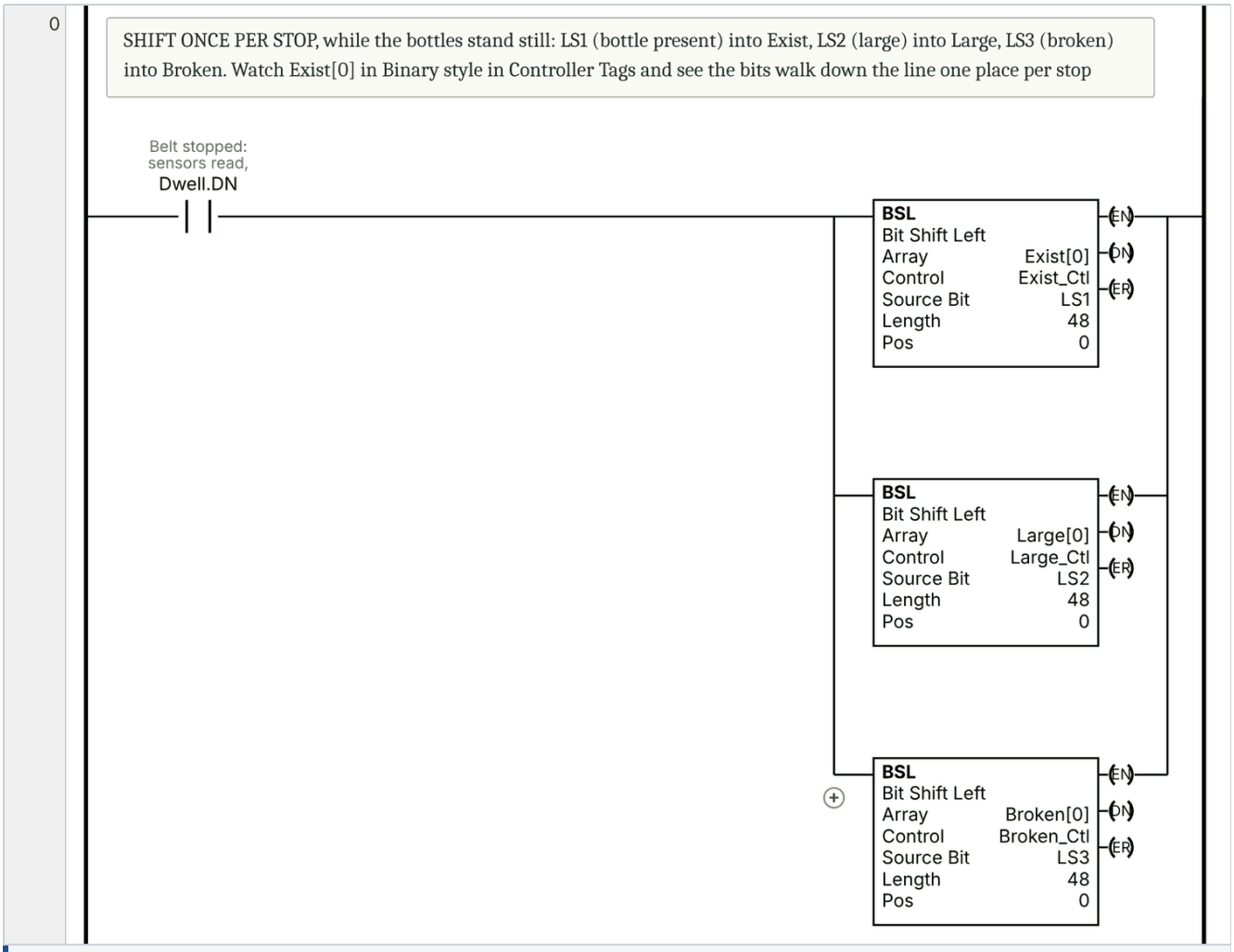
RUNG 3




```
XIC(Main_Conv_A)ONS(A_ONS)XIC(Conv_Index)OTU(Conv_Index);
```

S4_Registers

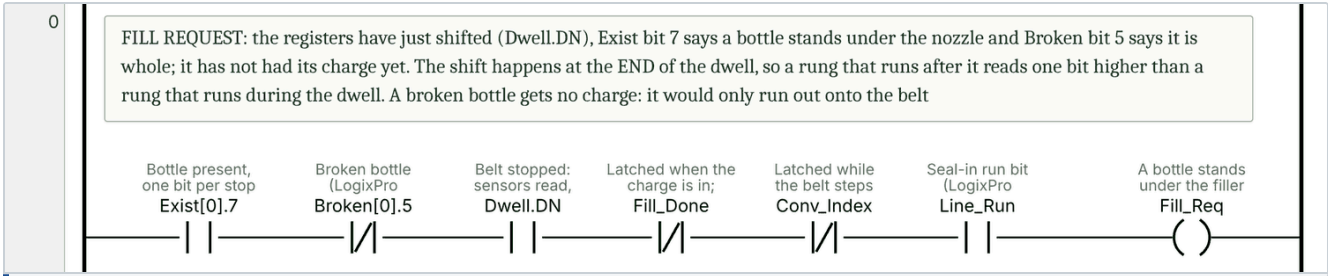
RUNG 0




```
XIC(Dwell.DN)
[BSL(Exist[0],Exist_Ctl,LS1,48),BSL(Large[0],Large_Ctl,LS2,48),BSL(Broken[0],Br
oken_Ctl,LS3,48)];
```

S5_Fill

RUNG 0

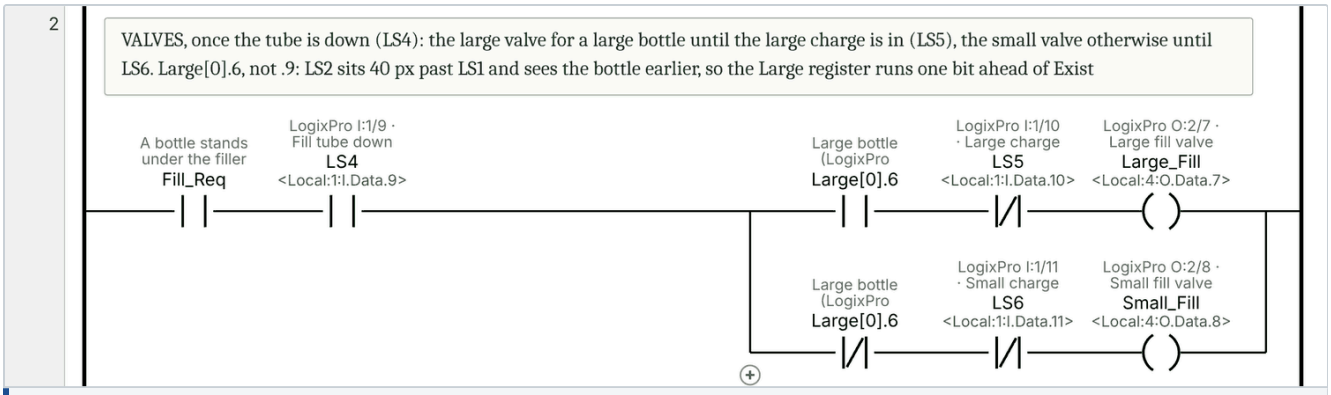



```
XIC(Exist[0].7)XIO(Broken[0].5)XIC(Dwell.DN)XIO(Fill_Done)XIO(Conv_Index)XIC(Li
ne_Run)OTE(Fill_Req);
```

RUNG 1

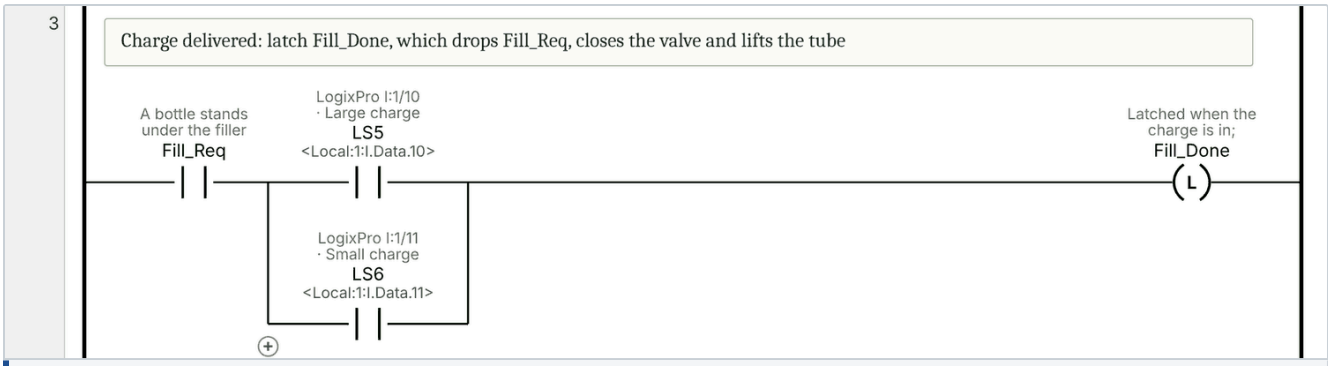
```
XIC(Fill_Req)OTE(Fill_Tube);
```

RUNG 2



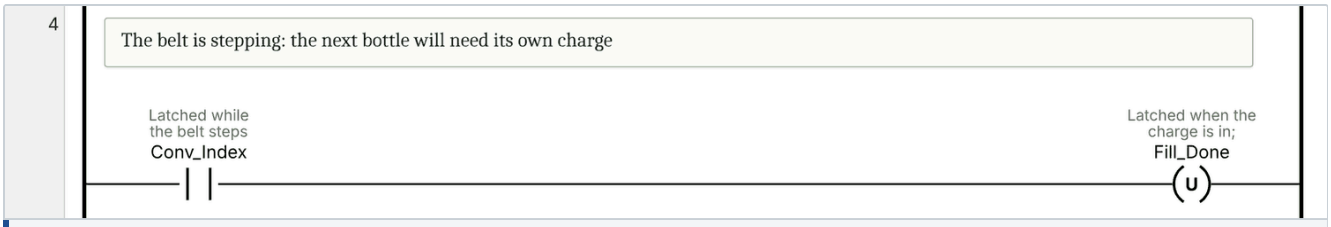

```
XIC(Fill_Req)XIC(LS4)
[XIC(Large[0].6)XIO(LS5)OTE(Large_Fill),XIO(Large[0].6)XIO(LS6)OTE(Small_Fill)]
;
```

RUNG 3




```
XIC(Fill_Req)[XIC(LS5),XIC(LS6)]OTL(Fill_Done);
```

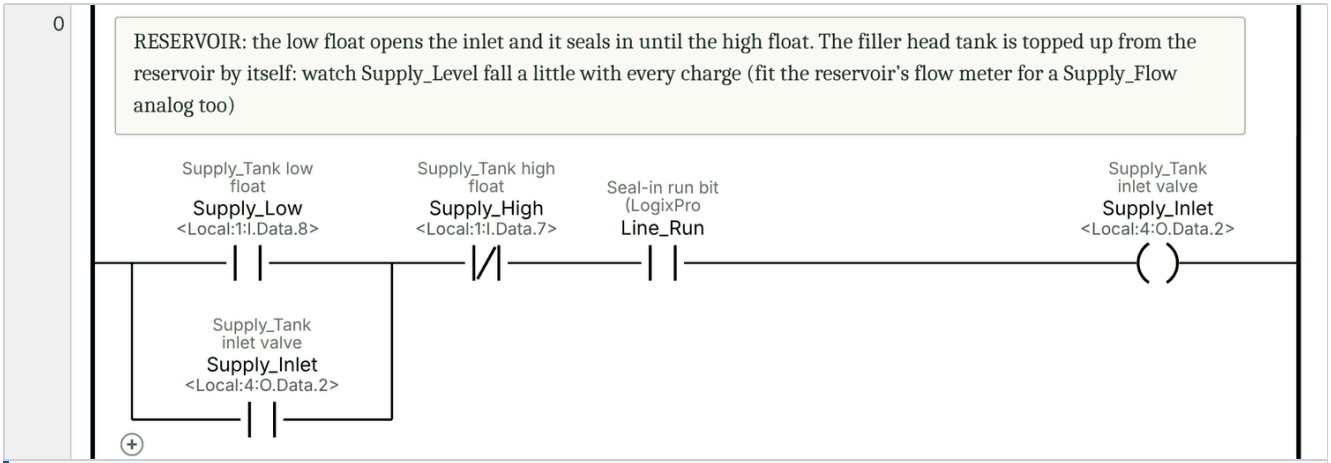
RUNG 4




```
XIC(Conv_Index)OTU(Fill_Done);
```

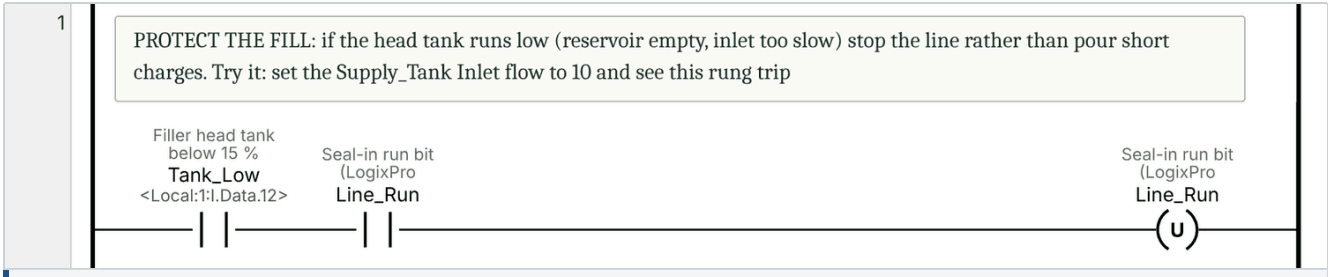
S6_Supply

RUNG 0



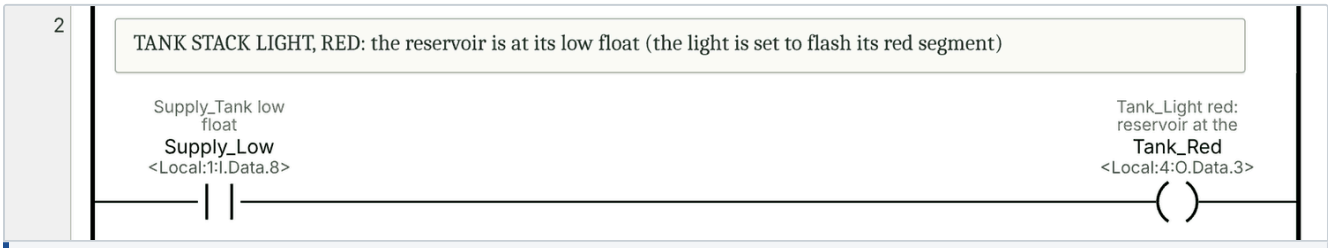

```
[XIC(Supply_Low),XIC(Supply_Inlet)]XIO(Supply_High)XIC(Line_Run)OTE(Supply_Inlet);
```

RUNG 1



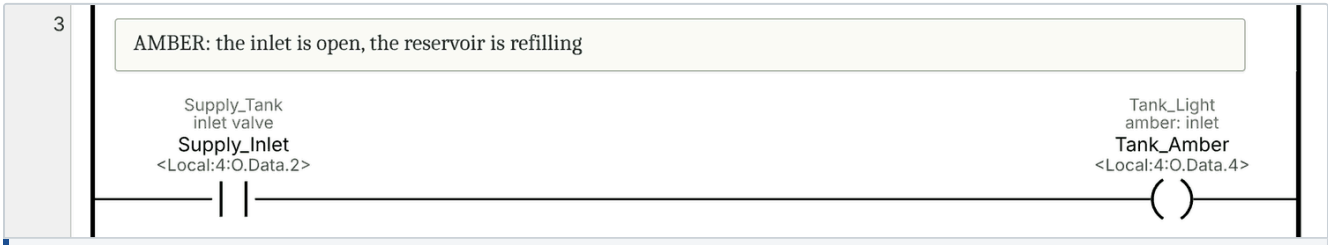

```
XIC(Tank_Low)XIC(Line_Run)OTU(Line_Run);
```

RUNG 2



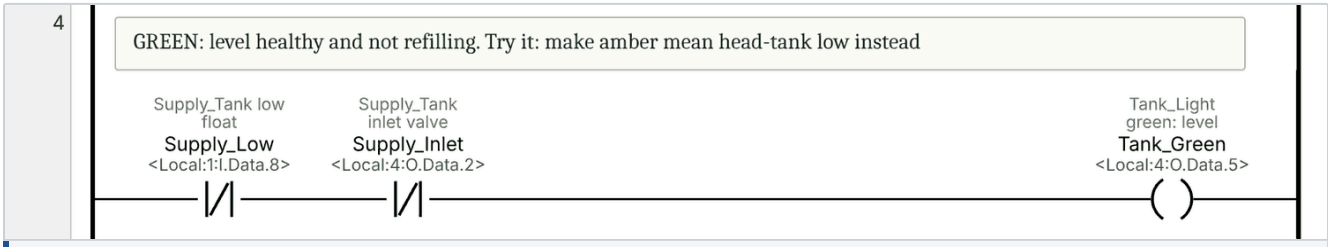

```
XIC(Supply_Low)OTE(Tank_Red);
```

RUNG 3




```
XIC(Supply_Inlet)OTE(Tank_Amber);
```

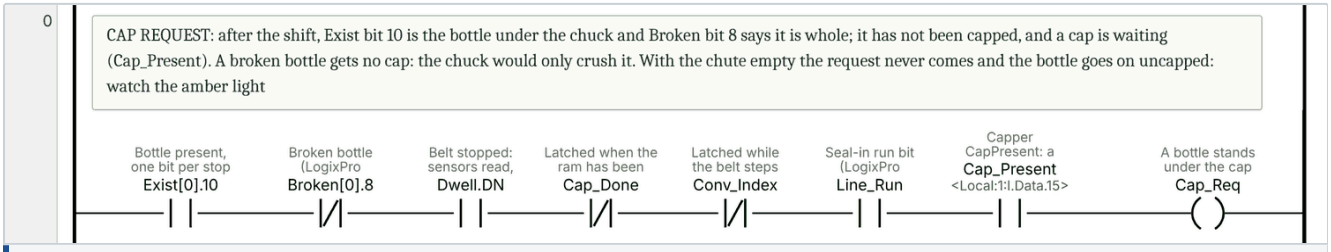
RUNG 4




```
XIO(Supply_Low)XIO(Supply_Inlet)OTE(Tank_Green);
```

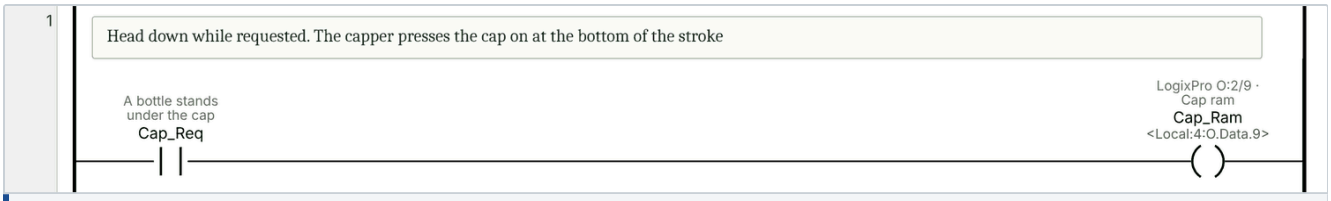
S7_Cap

RUNG 0



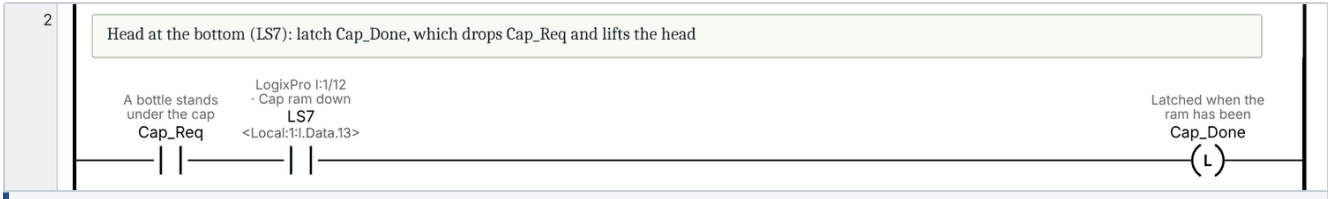

```
XIC(Exist[0].10)XIO(Broken[0].8)XIC(Dwell.DN)XIO(Cap_Done)XIO(Conv_Index)XIC(Line_Run)XIC(Cap_Present)OTE(Cap_Req);
```

RUNG 1



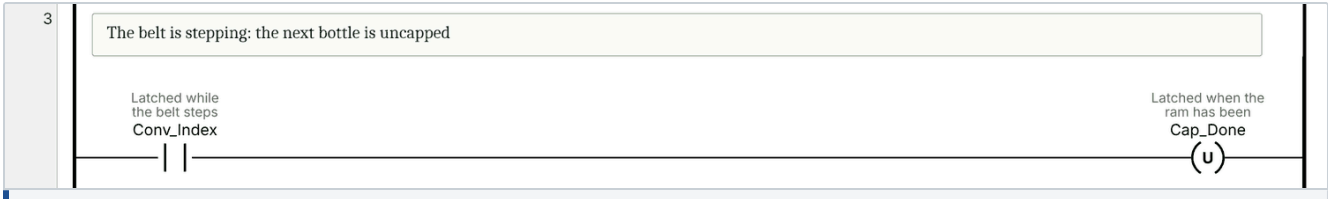

```
XIC(Cap_Req)OTE(Cap_Ram);
```

RUNG 2



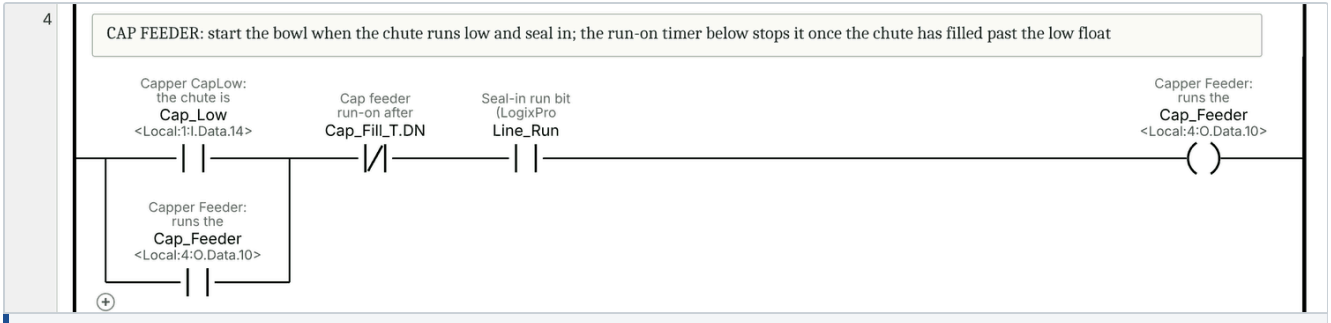

```
XIC(Cap_Req)XIC(LS7)OTL(Cap_Done);
```

RUNG 3



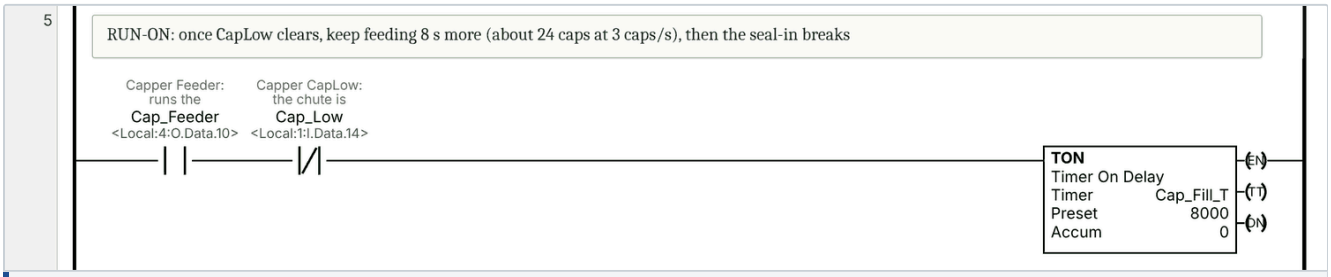

```
XIC(Conv_Index)OTU(Cap_Done);
```

RUNG 4



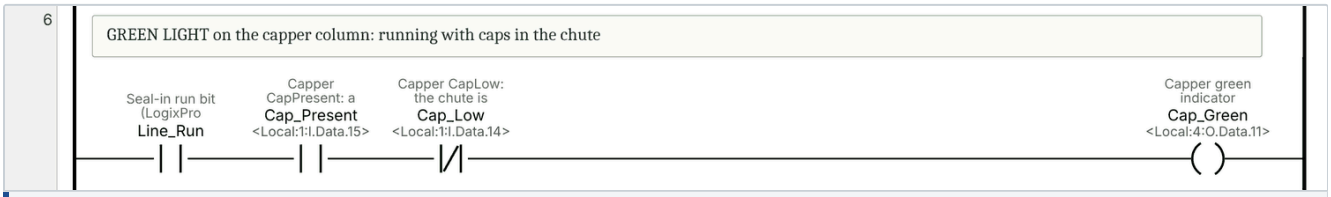

```
[XIC(Cap_Low),XIC(Cap_Feeder)]XIO(Cap_Fill_T.DN)XIC(Line_Run)OTE(Cap_Feeder);
```

RUNG 5



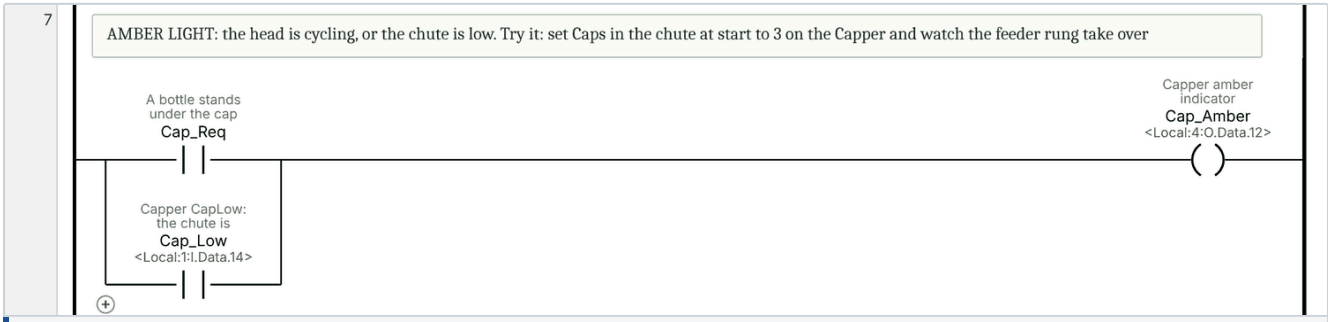

```
XIC(Cap_Feeder)XIO(Cap_Low)TON(Cap_Fill_T,8000,0);
```

RUNG 6




```
XIC(Line_Run)XIC(Cap_Present)XIO(Cap_Low)OTE(Cap_Green);
```

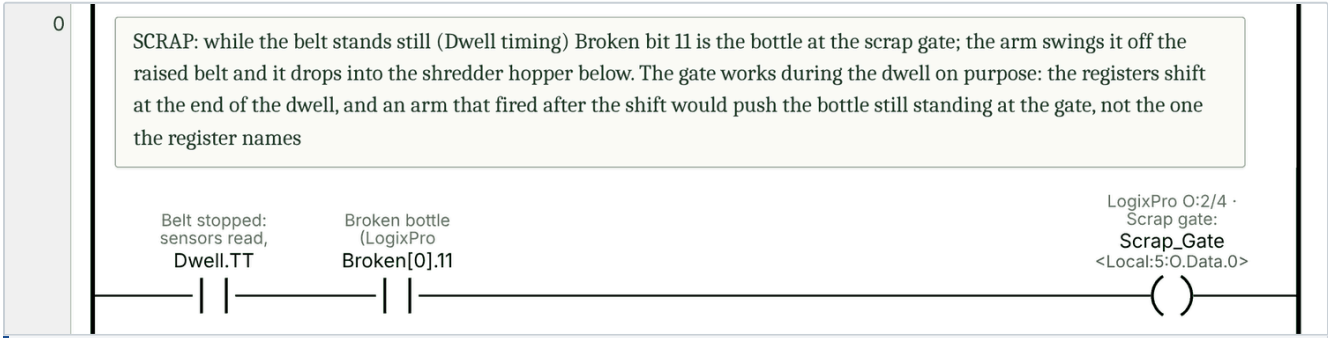
RUNG 7




```
[XIC(Cap_Req),XIC(Cap_Low)]OTE(Cap_Amber);
```

S8_Sort

RUNG 0




```
XIC(Dwell.TT)XIC(Broken[0].11)OTE(Scrap_Gate);
```

RUNG 1

1

DIVERT: a small, unbroken bottle at the divert gate (Exist bit 20, not Large 19, not Broken 18, all in the dwell frame) is swung off the raised belt and rolls down the catch incline onto the divert conveyor. Large bottles carry on to the end of the line. The Broken bit matters: a broken bottle was already scrapped, but its Exist bit is still set, so without it the arm would fire at an empty slot

```
XIC(Dwell.TT)XIC(Exist[0].20)XIO(Large[0].19)XIO(Broken[0].18)OTE(Divert_Gate);
```

RUNG 2

2

DIVERT CONVEYOR: runs with the line and carries the small bottles to the packer's lower lane. When that lane is full (Lower_Full) it keeps running only until the next bottle reaches the stop eye LS12, then stops, so the bottles close up at the stop instead of riding against it. It starts again the moment the packer takes four

```
XIC(Line_Run)[XIO(Lower_Full),XIO(LS12)]OTE(Divert_Conveyor);
```

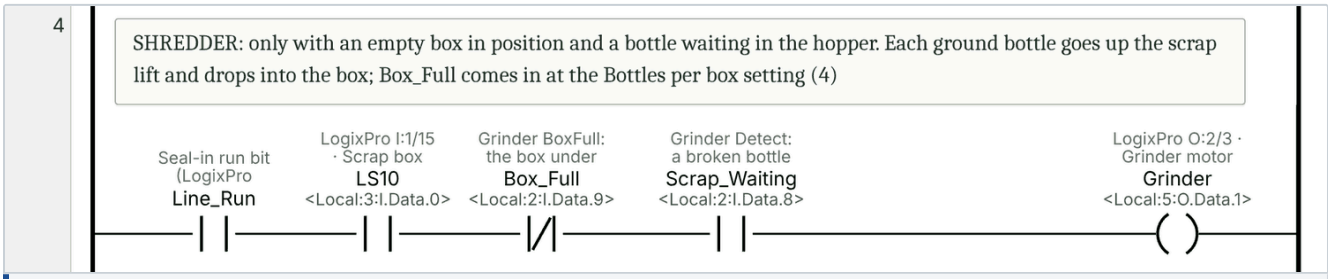
RUNG 3

3

BOX CONVEYOR: run until a box reaches LS10, which puts it under the scrap drop; run again once that box is full to carry it out and bring the next one in

```
XIC(Line_Run)[XIO(LS10),XIC(Box_Full)]OTE(Scrap_Conveyor);
```

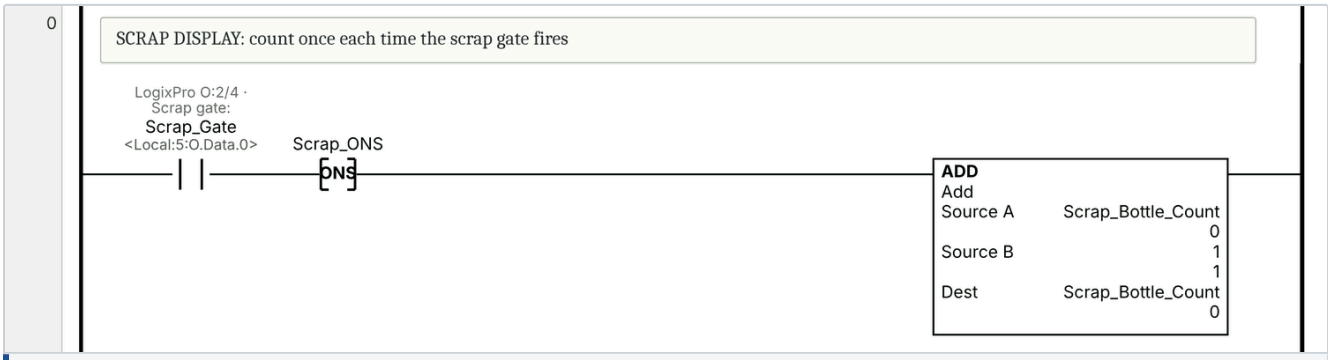
RUNG 4




```
XIC(Line_Run)XIC(LS10)XIO(Box_Full)XIC(Scrap_Waiting)OTE(Grinder);
```

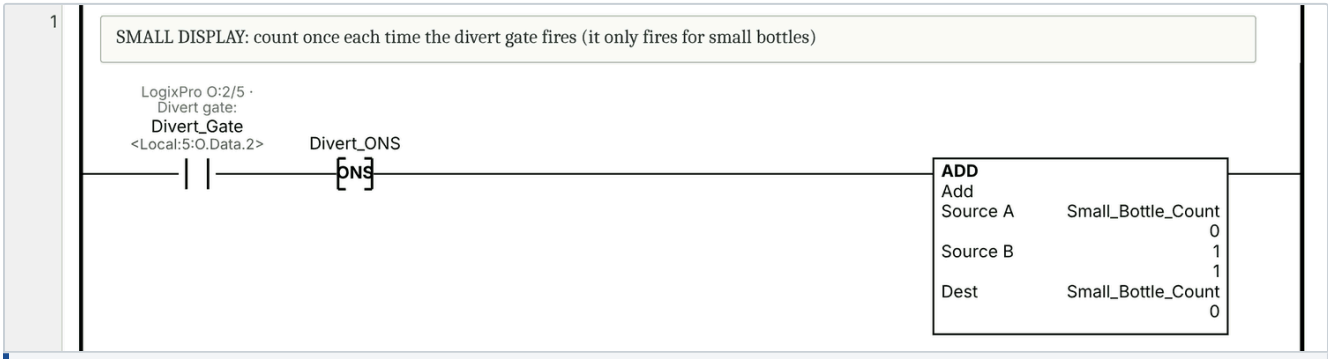
S9_Count

RUNG 0



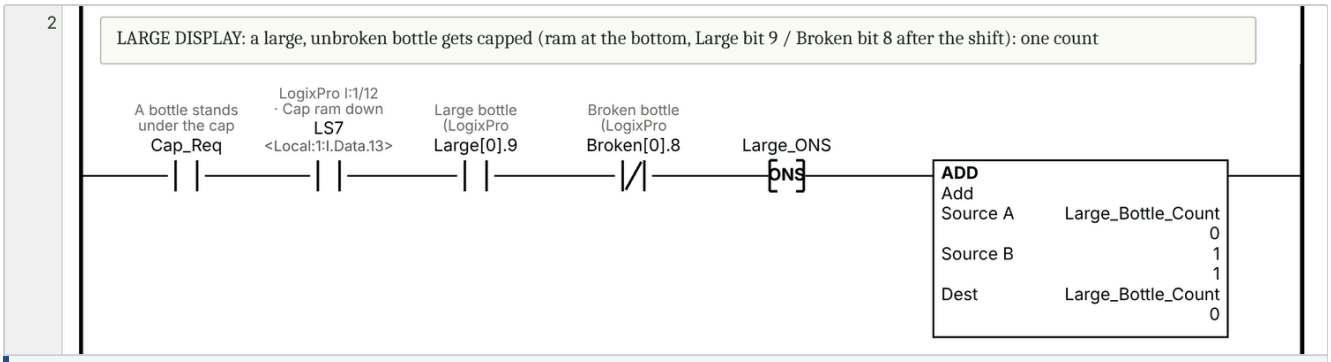

```
XIC(Scrap_Gate)ONS(Scrap_ONS)ADD(Scrap_Bottle_Count,1,Scrap_Bottle_Count);
```

RUNG 1



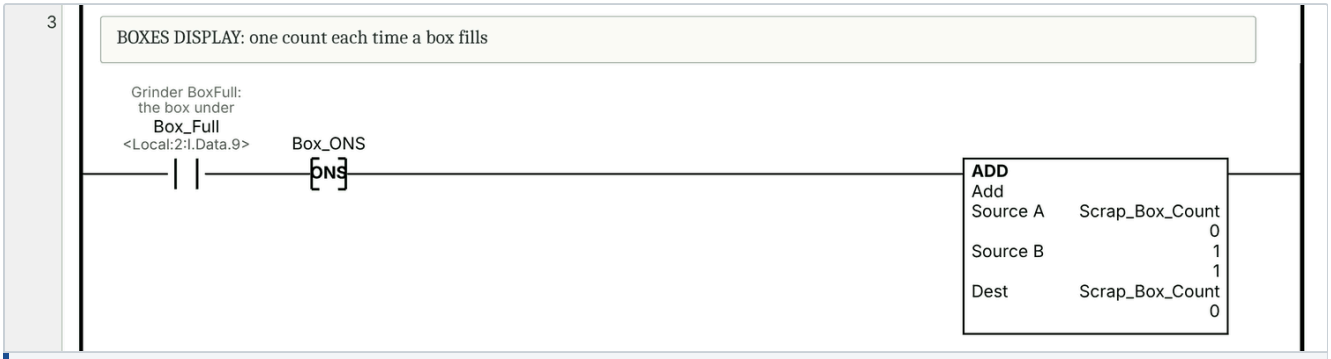

```
XIC(Divert_Gate)ONS(Divert_ONS)ADD(Small_Bottle_Count,1,Small_Bottle_Count);
```

RUNG 2



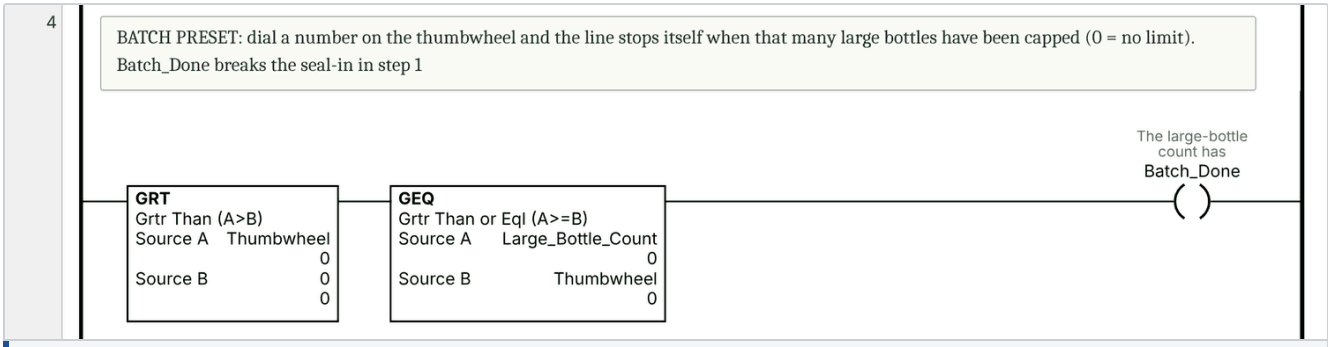

```
XIC(Cap_Req)XIC(LS7)XIC(Large[0].9)XIO(Broken[0].8)ONS(Large_ONS)ADD(Large_Bott
le_Count,1,Large_Bottle_Count);
```

RUNG 3



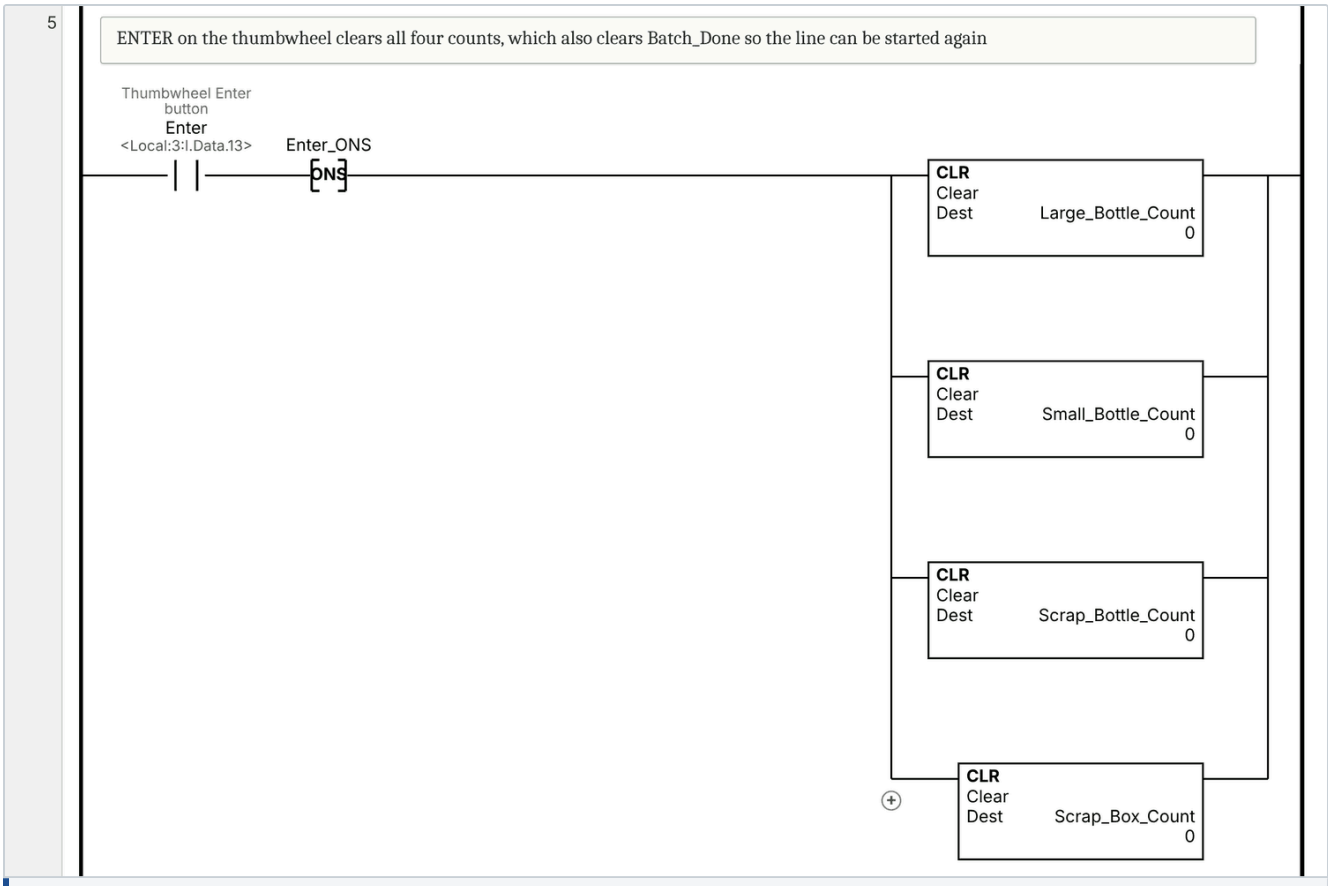

```
XIC(Box_Full)ONS(Box_ONS)ADD(Scrap_Box_Count,1,Scrap_Box_Count);
```

RUNG 4




```
GRT(Thumbwheel,0)GEQ(Large_Bottle_Count,Thumbwheel)OTE(Batch_Done);
```

RUNG 5




```
XIC(Enter)ONS(Enter_ONS)
[CLR(Large_Bottle_Count),CLR(Small_Bottle_Count),CLR(Scrap_Bottle_Count),CLR(Sc
rap_Box_Count)];
```

Appendix C — The register bit map

Station	x	Stops after LS1 / LS2 / LS3	Exist bit	Large bit	Broken bit	Frame
Filler	440	7 / 6 / 5	7	6	5	after the shift (Dwell.DN)
Capper	560	10 / 9 / 8	10	9	8	after the shift
Labeler	600	11 / 10 / 9	11	10	9	after the shift
Scrap gate	720	14 / 13 / 12	13	12	11	during the dwell (Dwell.TT)
Divert gate	1000	21 / 20 / 19	20	19	18	during the dwell
LS11 (end-of-line check)	1240	27 / 26 / 25	26	25	24	during the dwell, ACC ≥ 500

During the dwell a register describes the stop one pitch behind its count (it was last shifted before the previous step); after the shift it describes the count itself. Pitch 40 px; LS1 at 160, LS2 at 200, LS3 at 240; the registers are 48 bits long.

Appendix D — Sensor reference

Sensors detect parts, measure distance and tell part types apart. Every sensor is placed over a conveyor and looks at the lane beneath it; **60 px = 1 m**, so the default ranges follow the real devices they are modelled on. Rotate a sensor with its right-click menu or R; drag it along a conveyor and it snaps to the lane. Each sensor below lists its LEDs, what it detects, its range, and a table of the controller tags for every configuration (the Output setting in the property sheet).

Weighing Conveyor

- **Settings:** Speed (mm/s)

Tag	Controller I/O	Type	Description
Device_Fwd	Output	BOOL	Motor forward
Device_Weight	Input	INT	Weight on the scale, kg
Device_Running	Input	BOOL	Motor aux contact

Fill Level Sensor (liquid in the bottle)

- **Settings:** Fill line (px above the belt; a small bottle is 40 tall, a large 56; the liquid tops out at about 80 % of that); Sensor; Analog output (adds a Percent analog input: fill of the bottle under the sensor, 0-100 % of its label)

Tag	Controller I/O	Type	Description
Device_Detect	Input	BOOL	Liquid at or above the fill line in the bottle under the sensor

Diffuse Sensor (photoelectric)

Diffuse photoelectric sensor: its own light reflects off the part, so no reflector is needed. The shaded band on the lane shows the sensing range.

- **LED:** Red: detecting.
- **Detectable:** Solids: any part.
- **Sensing range:** 0 – 100 px, default 20 px (≈ 0.3 m). Factory I/O: 0 – 1.6 m.
- **Settings:** Sensing range (px; 60 px = 1 m); Sensitivity (%): the gain. 100 sees every part at full range; lower it and the range shrinks and dull targets (glass bottles, broken glass, dark cartons) drop out before bright ones (metal, cans, white and yellow cartons); Beam; Sees

Tag	Controller I/O	Type	Description
Device_Detect	Input	BOOL	Detecting: a solid part is within range

Retroreflective Sensor and Reflector

The sensor and its reflector face each other across the conveyor. **Detect** is 1 while the beam is interrupted. Use the beam height to ignore low parts: a lid (8 px) passes under a 12 px beam, a carton does not.

- **LED:** Green: aligned with the reflector. Yellow: beam not interrupted (goes out while a part blocks it).
- **Detectable:** Solids taller than the beam height.
- **Sensing range:** Across the lane (≈ 1 m here; the real device reaches 6 m).
- **Settings:** Beam height (px above the belt)

Tag	Controller I/O	Type	Description
Device_Detect	Input	BOOL	Light beam interrupted

Light Array (8 beams)

Eight parallel beams form a light curtain. Beam 1 is lowest. **Digital** gives one BOOL per beam, **Numerical** packs the beams into one INT (bit 0 = beam 1), **Analog** gives 10 V × interrupted beams ÷ 8 as hundredths of a volt. With the default 6 px spacing a small carton (22 px) breaks beams 1-4, a medium one 5, a large one 7, a crate all 8, a lid only beam 1.

- **LED:** Green: emitter and receiver synchronised. One red LED per interrupted beam.
- **Detectable:** Solids; measures how tall a part is.
- **Sensing range:** Across the lane; 8 beams spaced by the beam-spacing setting.
- **Settings:** Output; Beam spacing (px of part height)

Configuration: digital

Tag	Controller I/O	Type	Description
Device_Beam1	Input	BOOL	Beam 1 interrupted (3 px above the belt)
Device_Beam2	Input	BOOL	Beam 2 interrupted (9 px above the belt)
Device_Beam3	Input	BOOL	Beam 3 interrupted (15 px above the belt)
Device_Beam4	Input	BOOL	Beam 4 interrupted (21 px above the belt)
Device_Beam5	Input	BOOL	Beam 5 interrupted (27 px above the belt)
Device_Beam6	Input	BOOL	Beam 6 interrupted (33 px above the belt)
Device_Beam7	Input	BOOL	Beam 7 interrupted (39 px above the belt)
Device_Beam8	Input	BOOL	Beam 8 interrupted (45 px above the belt)

Configuration: numerical

Tag	Controller I/O	Type	Description
Device_Value	Input	INT	Each bit is a beam: bit 0 = beam 1 (lowest) … bit 7 = beam 8; 1 = interrupted

Configuration: analog

Tag	Controller I/O	Type	Description
Device_Volts	Input	INT	10 V × interrupted beams / 8, as hundredths of a volt (0–1000)

Inductive Sensor (metal)

Proximity sensor for metal. **Digital** gives Detect. **Analog** gives Volts: 0 V with the part at the sensor, rising to 10 V (1000) at the edge of the range, and 1000 when nothing metal is in range.

- **LED:** Red: detecting.
- **Detectable:** Conductive parts only: metal blocks, metal stock, metal bases and lids, cans.
- **Sensing range:** 0 – 60 px, default 10 px (≈ 0.17 m). Factory I/O: 0 – 0.1 m.
- **Settings:** Output; Sensing range (px; 60 px = 1 m)

Configuration: digital

Tag	Controller I/O	Type	Description
Device_Detect	Input	BOOL	Detecting: a conductive (metal) part is within range

Configuration: analog

Tag	Controller I/O	Type	Description
Device_Volts	Input	INT	0–10 V as hundredths of a volt: distance from the sensor to the nearest metal part, 1000 when none in range

Capacitive Sensor (any material)

Short-range proximity sensor for anything. **Digital** gives Detect. **Analog** gives Volts exactly like the inductive sensor, without the material filter.

- **LED:** Green: detecting.
- **Detectable:** Any material: solids (and liquids on the real device).
- **Sensing range:** 0 – 60 px, default 12 px (≈ 0.2 m), same as the real sensor.
- **Settings:** Output; Sensing range (px; 60 px = 1 m)

Configuration: digital

Tag	Controller I/O	Type	Description
Device_Detect	Input	BOOL	Detecting: any solid part is within range

Configuration: analog

Tag	Controller I/O	Type	Description
Device_Volts	Input	INT	0–10 V as hundredths of a volt: distance from the sensor to the nearest part, 1000 when none in range

Color Sensor (R/B/G/Y banks)

Four colour banks, one BOOL each.

- **LED:** The head shows the colour it sees.
- **Detectable:** Coloured plastic parts: red, blue, green, yellow. Metal gives no bank.
- **Sensing range:** Directly beneath the head.

Tag	Controller I/O	Type	Description
Device_Red	Input	BOOL	Bank 1: red
Device_Blue	Input	BOOL	Bank 2: blue
Device_Green	Input	BOOL	Bank 3: green
Device_Yellow	Input	BOOL	Bank 4: yellow

Vision Sensor

Recognises the part under it. **Digital**: one BOOL per ticked part. **Bits**: the part code as five binary inputs Bit0-Bit4 (Factory I/O's "All Digital"). **Numerical**: one INT with the part code (the number next to each part in the list; 0 = none or not ticked). **ID**: the serial number of the part in view plus its code, usable like a barcode. **Size**: the older Small/Medium/Large/Metal outputs. Tick **Triggered** for a Trigger output with Busy and Complete, like a real smart camera: the result then holds until the next trigger. Set **Inspection job** to **Bottle inspection** and it checks a bottle's cap, label and fill instead: Pass is a bottle with all three, and on a camera module the result code is bit 0 no cap, bit 1 no label, bit 2 short fill, bit 3 no part in view, with the fill in % of the label in byte 9 of Inspection_Results. On a camera module, **Buffer_Results_Enable** queues results up to the buffer depth until the PLC acknowledges each one (Results_Buffer_Overrun when one is lost). A Process HMI set to show the sensor draws its camera screen. **Card mode** (bottle job): the bottle cards SN-4801 to SN-4804 decide each bottle. The hopper sends plain good bottles; when a result comes back, the bottle under the camera takes the card's verdict (no cap, no label, short fill; no card in view makes it a reject) and moves on. Each bottle waits at the camera until its result is presented, so there is time to swap the card. With the simulated camera, pick the card in **Card under the lens** or on the camera screen's CARDS page; with the camera module live on a real In-Sight, the real card decides: its result code, the fill in byte 9 and the card ID in byte 10 of Inspection_Results.

- **LED:** Green: a part in view. Amber: result valid for a selected part. The ring light flashes on a trigger.
- **Detectable:** Every part in the catalogue; tick the ones to detect.
- **Sensing range:** Directly beneath the camera.
- **Settings:** Output; Camera module (its name in I/O Configuration; blank = wired signals only); Housing; Parts to Detect; Triggered (adds Trigger input, Busy and Complete); Inspection time (ms); Inspection job; Bottle job: the fill passes at (% of the label); Results buffer depth (camera module, with Buffer_Results_Enable on); Card mode (bottle job): the card under the camera decides what is wrong with each bottle, and each bottle waits at the camera for its result; Camera jobs, in job ID order: PRODUCT:min fill %, … (Level 9; blank = the one job above); Card under the lens (simulated camera; a live camera reads the real card); Live image: after each inspection of a live camera, fetch the image it

acquired (image.bmp, over FTP through the bridge) and show it on the camera screen; Live image: FTP user on the camera (a user with FTP Read); Live image: FTP password (blank for the In-Sight default admin); Live image: at most one image every (s) — each image is work for the camera; Live image: width to fetch (px; the bridge scales the image down to this); Live camera: where the inspection ID and result code are read

Configuration: digital

Tag	Controller I/O	Type	Description
Device_Detect	Input	BOOL	A part is in the field of view
Device_Pass	Input	BOOL	The part is one of the selected parts
Device_CartonS	Input	BOOL	Carton (S) in view
Device_CartonM	Input	BOOL	Carton (M) in view
Device_CartonL	Input	BOOL	Carton (L) in view
Device_MetalM	Input	BOOL	Metal Block (M) in view

Configuration: bits

Tag	Controller I/O	Type	Description
Device_Detect	Input	BOOL	A part is in the field of view
Device_Pass	Input	BOOL	The part is one of the selected parts
Device_Bit0	Input	BOOL	Part code bit 0 (the code of the part in view, 1..21, as 5 binary digits; all 0 = none or not selected)
Device_Bit1	Input	BOOL	Part code bit 1 (the code of the part in view, 1..21, as 5 binary digits; all 0 = none or not selected)
Device_Bit2	Input	BOOL	Part code bit 2 (the code of the part in view, 1..21, as 5 binary digits; all 0 = none or not selected)
Device_Bit3	Input	BOOL	Part code bit 3 (the code of the part in view, 1..21, as 5 binary digits; all 0 = none or not selected)
Device_Bit4	Input	BOOL	Part code bit 4 (the code of the part in view, 1..21, as 5 binary digits; all 0 = none or not selected)

Configuration: numerical

Tag	Controller I/O	Type	Description
Device_Detect	Input	BOOL	A part is in the field of view
Device_Pass	Input	BOOL	The part is one of the selected parts
Device_Code	Input	INT	Part code: 1 = Carton (S), 2 = Carton (M), 3 = Carton (L), 7 = Metal Block (M); 0 = none or not selected

Configuration: id

Tag	Controller I/O	Type	Description
Device_Detect	Input	BOOL	A part is in the field of view
Device_Pass	Input	BOOL	The part is one of the selected parts
Device_ID	Input	INT	Serial number of the part in view (every part gets its own; 0 = none)
Device_Code	Input	INT	Part code, as in Numerical mode

Configuration: size

Tag	Controller I/O	Type	Description
Device_Detect	Input	BOOL	A part is in the field of view
Device_Pass	Input	BOOL	The part is one of the selected parts
Device_Small	Input	BOOL	Small part
Device_Medium	Input	BOOL	Medium part
Device_Large	Input	BOOL	Large part
Device_Metal	Input	BOOL	Metal part

Configuration: camera tags only

Tag	Controller I/O	Type	Description

RFID Reader

Single-trigger command interface: set **Command**, pulse **Execute**; when **CommandID** increments the command has finished and **Status** says how it went (0 = success). Command 0 only checks for a tag, 1 reads the serial into ReadData, 2 reads the word at MemoryIndex, 3 writes WriteData there, 4 clears the memory. Two tags in range give status 2. Write a route code into a part at one station and sort on it at the next.

- **LED:** Green: idle. Orange: executing a command. Red: the last command failed.
- **Detectable:** RFID tags. Every part carries one except raw stock, housing bases and housing lids. Each tag has a unique serial number and 16 words of memory.
- **Sensing range:** 0 – 60 px, default 16 px.
- **Settings:** Reading range (px); Command time (ms)

Tag	Controller I/O	Type	Description
Device_Command	Output	INT	Command to execute: 0 = is a tag available?, 1 = read serial number, 2 = read data, 3 = write data, 4 = clear all data
Device_Execute	Output	BOOL	Rising edge executes the command
Device_MemoryIndex	Output	INT	Tag memory index for read/write (0-15)
Device_WriteData	Output	INT	Data written by command 3
Device_CommandID	Input	INT	Number of commands executed so far; changes when the current one has finished
Device_Status	Input	INT	0 = success, 1 = no tag, 2 = too many tags, 3 = invalid index, 4 = invalid command
Device_ReadData	Input	INT	Result of command 1 (serial) or 2 (data)
Device_TagPresent	Input	BOOL	Exactly one tag is within range

Appendix E — Keyboard shortcuts

Insert	Add a rung after the selected one	Enter	Edit the selected rung
Delete	Delete the selected rung (or the selected tag in Controller Tags)	Esc	Cancel the edit
↑ / ↓	Select the previous / next rung	Ctrl+↑ / Ctrl+↓	Move the rung
Drag a tab	Rearrange the document tabs; the order is remembered	Right-click a tab	Move Left / Right / to Start / to End, Reset Tab Order, Close Others
Ctrl+D	Add or edit the comment on the selected rung (also the Comment button; double-click a comment to edit it)	Enter / Shift+Enter / Esc	In a comment: save / new line / cancel
Ctrl+C / Ctrl+X	Copy / cut the selected instruction or branch (or the whole rung when no instruction is selected)	Ctrl+V / Ctrl+Shift+V	Paste after / before the selection. Works across rungs, routines, programs and projects
Ctrl+Enter	In a Structured Text routine: accept the text	Tab	In a Structured Text routine: indent
Alt+3	Open or close the Quick Watch window		
Ctrl+E	Cross Reference for the selected instruction or tag	Ctrl+G	Go To a routine and rung
Ctrl+F / F3 / Shift+F3	Find / find next / find previous (on this page Ctrl+F goes to the search box)	Ctrl+H	Find and Replace
Ctrl+Z / Ctrl+Y	Undo / redo	Ctrl+T	Toggle the selected bit
Ctrl+S / Ctrl+O	Save the project file / open one	Double-click a tab	Maximize / restore the document

Appendix F — Glossary

Accumulator (.ACC)

The running value of a timer (milliseconds elapsed) or a counter (counts so far). It is a DINT, so it shows up for compare and math instructions, not for XIC.

Adapter

The EtherNet/IP device that is owned and answers the scanner: a camera, a drive, a rack of remote I/O.

Add-On Instruction (AOI)

An instruction you define yourself: parameters, local tags and a ladder routine, packaged under one name and used like TON or MOV. Every call needs its own backing tag. See the Add-On Instructions section.

Alias tag

A second name for another tag or an I/O point. Start_PB aliased to Local:1:I.Data.0 is that input, under a readable name. Cross Reference treats the alias and its target as the same thing.

Analog I/O

A channel that carries a number instead of on/off: a level, a weight, a speed reference. Here it is an INT on the 1769-IF4 and 1769-OF4 cards.

Array

A tag with a Dim: Name[0] to Name[Dim-1], all of one data type. Used by the file, shift, FIFO and sequencer instructions.

Assemble (edits)

The last step of an online edit: the accepted edits become the program and the original rungs are removed.

Assembly (CIP)

A block of bytes an EtherNet/IP device exposes as one object, class 4. The input assembly is what the device produces, the output assembly is what it consumes. The EDS file gives the instance numbers and sizes.

Backing tag

The tag of an Add-On Instruction's own data type that goes with one call. It holds that call's parameters and local tags, so two calls with two backing tags run independently.

Backplane

The bus the controller and its local cards sit on. Local:3:O.Data.0 means slot 3 on the backplane.

Base tag

The real tag or I/O address behind an alias.

BOOL

One bit: 0 or 1. What XIC, XIO and OTE work on.

Branch

A parallel path in a rung, written [leg , leg]. Power gets through if any leg is true.

Bridge

The small server (TcpBridge, built into LogixConnect.exe) that gives the page real TCP and UDP sockets. Everything that talks to hardware goes through it.

Bumpless transfer

Switching a PID loop between manual and automatic without a jump in the output.

CIP

Common Industrial Protocol: the object model (classes, instances, attributes, services) that EtherNet/IP carries.

Class 1 connection

A cyclic I/O connection: after a Forward Open, both ends send their data over UDP 2222 every RPI. Also called implicit messaging.

CompactLogix

The Allen-Bradley controller family this workstation is modelled on (1769-L33ER).

Consumed tag

A controller tag whose value is read from another controller at a fixed interval (RPI).

Continuous task

The task that runs over and over as fast as it can. Here MainTask scans every 20 ms.

CONTROL

The structure the file instructions use to keep their place: .LEN, .POS and the status bits .EN .DN .ER .EM .UL.

Control variable (CV)

The output a control loop moves: a valve position, a heater power, a speed reference.

Controller Organizer

The tree on the left: controller tags, tasks, programs and routines, Add-On Instructions, data types, I/O configuration.

Controller tags

Tags every program can use. All I/O and alias tags are controller tags. Program tags belong to one program.

Counter

CTU and CTD count rung transitions into .ACC and set .DN when .ACC reaches .PRE. RES clears it.

Cross Reference

A list of every instruction that reads or writes a tag, across all routines, with the ones that write it marked Destructive. Ctrl+E.

Data context

Which call of an Add-On Instruction you are looking at when its logic is open: the definition, or one backing tag's live values.

Data type

What a tag holds: BOOL, SINT, INT, DINT, REAL, STRING, a structure such as TIMER, a user-defined type or an Add-On Instruction's type.

Deadband

A range around the setpoint inside which the error is treated as zero, so the output does not chase noise.

Destructive reference

A place where an instruction writes a tag (OTE, MOV destination, a timer instruction on its timer). Cross Reference marks these with Y.

Destructive reference

An instruction that writes the tag (OTE, MOV destination, TON on its timer) rather than only reading it. Two destructive references to one bit is the classic double-coil bug.

DINT

A 32-bit signed integer. The default number type in Logix.

Done bit (.DN)

Set by a timer when .ACC reaches .PRE, by a counter when the count is reached, by a message when it completes.

EDS file

Electronic Data Sheet: the text file a vendor ships that describes an EtherNet/IP device. The identity, the assemblies, the connections and their sizes come from it.

Enable bit (.EN)

True while the instruction's rung-in condition is true (timers, messages, file instructions).

EnableIn / EnableOut

The two built-in parameters of every Add-On Instruction: the rung-in condition it was called with, and the rung-out condition it passes on.

EtherNet/IP

CIP carried over standard Ethernet: explicit messages on TCP 44818 and cyclic I/O on UDP 2222.

Event task

A task that runs when something happens: an EVENT instruction, a trigger tag going true or changing, or a timeout with no trigger.

Exclusive owner

The one scanner allowed to write a device's outputs. A second one is refused with an ownership conflict.

Explicit messaging

Request and reply, one at a time, on TCP: read this attribute, write that tag. The MSG instruction does this.

Expression

A formula written out, as in CPT, CMP, FAL, FSC and Structured Text: Level > 50 AND NOT Fault, or Src[C.POS] * 2 + 1.

Fault handler

A program under Controller Fault Handler that runs when a major fault happens. Here it runs once and the fault is cleared.

Fault, major

An error that stops the controller: mode shows Faulted and logic stops until Clear Faults, unless a fault handler program clears it. Integer divide by zero is one.

Fault, minor

An error that is logged while the scan carries on, such as arithmetic overflow.

FBD_TIMER, FBD_COUNTER, FBD_ONESHOT

The structures the Structured Text timers, counter and one-shots work on (TONR, TOFR, RTOR, CTUD, OSRI, OSFI). You set their inputs, such as .TimerEnable and .PRE, then call the instruction.

FIFO

First in, first out: FFL loads a value into an array, FFU unloads the oldest one.

File instruction

An instruction that works through an array one element at a time using a CONTROL tag: FAL, FSC, AVE, SRT, STD, COP, FLL.

Force

An override on an I/O point from the programming software, on top of the device (inputs) or the logic (outputs). Forces act only while enabled.

Forward Open

The CIP request that opens a connection. The scanner asks for sizes and an RPI, the device answers with the connection IDs, then cyclic data starts.

FreqCommand / OutputFreq

A drive's speed reference and its actual output frequency, in hundredths of a hertz: 6000 is 60.00 Hz.

GSV / SSV

Get / Set System Value: read or write controller data such as the wall clock, task scan times or a module's EntryStatus.

I/O Configuration

The part of the organizer listing the local cards and the Ethernet modules the controller owns.

I/O Monitor

The live list of every input and output point with its value, the device wired to it and the rungs that use it.

Implicit messaging

Cyclic I/O data with no request: both ends just send at the RPI. See Class 1 connection.

In-Sight

Cognex's vision system family. Over EtherNet/IP it exposes trigger and status bits and an inspection results area.

Indirect addressing

Choosing an array element with a tag instead of a number: Recipe[Step]. The element is picked each time the instruction runs. A subscript outside the array is a major fault.

Inhibit

Configured but switched off: an inhibited task or module stays in the project and does not run.

Input / Output / InOut (parameter)

How an Add-On Instruction parameter is passed. Input: the value is copied in. Output: the value is copied out. InOut: the logic works on the caller's tag directly.

Instruction

One element of a rung: a contact, a coil or a box such as TON, MOV or an Add-On Instruction.

INT

A 16-bit signed integer.

Interlock

A condition in series that must be true before an output may come on.

JSR / SBR / RET

Jump To Subroutine, and the Subroutine and Return instructions that receive and hand back its parameters.

L5X

The XML file Studio 5000 uses to export and import a project or a piece of one. File → Import L5X and Export L5X read and write it.

Ladder Diagram (LD)

The relay-style language of rungs between two power rails. The only language here.

Latch / Unlatch

OTL sets a bit and it stays set until OTU clears it, whatever the rungs do in between.

LIFO

Last in, first out: LFL loads a value into an array, LFU unloads the newest one.

Listen-only

A connection that receives a device's inputs without owning its outputs. In-Sight cameras do not offer one.

Local tag

A tag that belongs to an Add-On Instruction and exists once per backing tag: its timers, counters and working values.

Main routine

The routine a program runs each scan. Other routines run only when a JSR calls them.

MCR zone

The rungs between two MCR instructions. While the first MCR rung is false, every rung in the zone is scanned as false, so ordinary outputs drop. It is not a substitute for a hard-wired safety relay.

Module

A device in the I/O configuration: a local card, or an EtherNet/IP device created from an EDS file.

MSG

The Message instruction: one explicit request to a device or another controller, with .EN .DN .ER status bits.

Neutral text

The text form of a rung, for example XIC(Start)OTE(Motor); . Press Enter on a rung to edit it this way.

One-shot

ONS, OSR, OSF: true for exactly one scan when the rung goes true (or false), so something happens once per event.

Online

Connected to the controller, so you see live values. Here the workstation is always online with its own controller unless you go offline.

Online edit

Changing logic while the controller runs. The change is pending (i), then accepted (I), then tested, then assembled. See the Online edits section.

Operand

A value or tag an instruction works on: the bit of an XIC, the source and destination of a MOV.

OTE

Output Energize: the coil. The bit follows the rung, every scan.

Overlap

A task triggered again before its previous run had started or finished. Counted in GSV(Task,name,OverlapCount).

Ownership conflict

The error a device returns when a second scanner asks for a connection it has already given to another.

Pen

One tag plotted on a trend. A trend holds up to 8, each with its own colour.

Pending edit

An online edit that is still only in the editor: the i rung. The controller has not been given it yet.

Periodic task

A task that runs at a fixed period in milliseconds, for logic that must run at a known rate.

PID

Proportional, Integral, Derivative: the standard closed-loop control instruction. It compares the process variable with the setpoint and moves the control variable to close the gap.

PLC

Programmable Logic Controller: the industrial computer that reads inputs, runs the logic and writes outputs, over and over.

Power flow

Whether the rung is true up to a given point. The green highlighting on contacts and wires.

Pre-emption

A higher-priority task interrupting a lower one part-way through. Here it happens between rungs; the interrupted task then carries on from where it stopped.

Preset (.PRE)

The target of a timer (milliseconds) or counter (counts).

Priority (task)

1 is highest, 15 lowest. Periodic and event tasks all outrank the continuous task.

Process variable (PV)

The measured value a control loop is controlling: a level, a temperature, a speed.

Produced / consumed tag

A tag one controller broadcasts and another receives cyclically. Not supported here.

Produced tag

A controller tag that is sent to another controller at a fixed interval (RPI).

Program

A group of routines with its own tags, scheduled in a task.

Program mode

Controller mode where inputs update but no logic runs and outputs are off.

Program tags (local tags)

Tags that belong to one program, listed under Parameters and Local Tags. Only that program's routines can use them, and inside that program they win over a controller tag of the same name.

Quick Watch

Your own list of tags with live values that you can change, shown in a floating window (Alt+3) and in the Watch tab of the output window. Saved with the project.

REAL

A 32-bit floating-point number.

Remote controller

Another controller this one shares produced and consumed tags with: simulated inside the project, or a real one reached through the bridge.

Remote I/O

Inputs and outputs on a block out on the network instead of in the controller chassis. Their tags are named after the module, such as RIO1:I.Pt00.

Requested Packet Interval (RPI)

How often, in milliseconds, each end of a cyclic connection sends its data.

Retentive

Keeps its value when the rung goes false. RTO is a retentive timer; only RES clears it.

Routine

A list of rungs. Each program has a main routine and may have more.

Run mode

Controller mode where logic scans and outputs are driven.

Rung

One line of ladder: conditions on the left, outputs on the right.

Rung comment

Free text above a rung saying what it does. Ctrl+D.

Sandbox

The free-form project where you place your own devices and wire them to I/O.

Scan

One pass: read inputs, run the tasks, write outputs. 20 ms here.

Scanner

The EtherNet/IP device that opens connections and owns the adapters: normally the PLC, here the workstation.

Scope

Where a tag lives and who can see it: the controller (everyone) or one program (only its routines). The Scope box in the tag editor switches between them.

Seal-in

A branch around a start contact using the output's own bit, so a momentary start keeps the output on until a stop breaks the rung.

Sequencer

SQO steps through an array of words and copies each to an output through a mask.

Setpoint (SP)

The value a control loop is trying to hold.

SINT

An 8-bit signed integer.

Slot

A position on the backplane. In a route, slot 0 is the CompactLogix controller.

STRING

Text of up to 82 characters with a .LEN and the characters in .DATA.

Structure

A tag made of members: TIMER, COUNTER, CONTROL, MESSAGE, a user-defined type or an Add-On Instruction tag.

Structured Text (ST)

A text language for routines: assignments with :=, IF, CASE, FOR, WHILE and REPEAT. Good for math, loops and decisions that are clumsy in ladder. See the Structured Text section.

Tag

A named piece of controller memory. Logix addresses everything by tag name.

Task

What schedules programs: continuous, periodic or event. Each has a priority; a higher-priority task interrupts a lower one.

TCP Client Socket

A sandbox device that opens a plain TCP connection through the bridge to send and receive text, for equipment that speaks ASCII.

Test (edits)

Running the accepted edits in place of the original rungs without committing to them. Untest goes back.

Test mode

Controller mode where logic scans but outputs stay off.

Timer

TON, TOF, RTO: time a condition in milliseconds into .ACC and set .DN at .PRE. .TT is true while timing.

Timing bit (.TT)

True while a timer is counting: enabled and not done yet.

Trend

A chart of tag values against controller time. Each plotted tag is a pen. See the Trends section.

Trigger

The signal that makes a camera acquire an image. Over EtherNet/IP: set Trigger Enable, pulse Trigger, wait for the acknowledge and Results Valid.

UCMM

Unconnected Message Manager: explicit messages sent without opening a connection first.

UID / UIE

User Interrupt Disable and Enable. Rungs between them cannot be interrupted by another task.

User-Defined Data Type (UDT)

A structure you define: named members of any type. A tag of that type gets Tag.Member for each.

Variable frequency drive (VFD)

A drive that runs an AC motor at a commanded frequency. The built-in one uses the PowerFlex 525 tags: Start, Stop, FreqCommand out; Ready, Active, Faulted, OutputFreq in.

Verify

Check every rung for errors without running anything. The results are in the Errors window.

Watchdog

The limit on how long a task may run. Here a loop that never ends is stopped with a major fault (type 6) instead of freezing the controller.

XIC

Examine If Closed: a contact that is true when the bit is 1.

XIO

Examine If Open: a contact that is true when the bit is 0.

Zone markers (i I r R d D)

The letters in the rung margin during an online edit. Lower case is pending, upper case is in the controller: i/I the new rung, r/R the original it replaces, d/D a rung being deleted.

Appendix G — Instruction set

Instructions the rung editor accepts (parameter counts as in Logix): `XIC`, `XIO`, `OTE`, `OTL`, `OTU`, `ONS`, `OSR`, `OSF`, `TON`, `TOF`, `RTO`, `CTU`, `CTD`, `RES`, `MOV`, `ADD`, `SUB`, `EQU`, `NEQ`, `GRT`, `GEQ`, `LES`, `LEQ`, `LIM`, `MEQ`, `JSR`, `SBR`, `RET`, `JMP`, `LBL`, `AFI`, `NOP`, `MSG`, `GSV`, `SSV`, `COP`, `CPS`, `FLL`, `MUL`, `DIV`, `MOD`, `SQR`, `NEG`, `ABS`, `CLR`, `CPT`, `DTOS`, `STOD`, `RTOS`, `STOR`, `CONCAT`, `UPPER`, `LOWER`, `MID`, `FIND`, `DELETE`, `INSERT`, `BSL`, `BSR`, `FFL`, `FFU`, `SQO`, `SQI`, `MVM`, `BTD`, `SWPB`, `AND`, `OR`, `XOR`, `NOT`, `CMP`, `XPY`, `TRN`, `SCP`, `SIN`, `COS`, `TAN`, `ASN`, `ACS`, `ATN`, `LN`, `LOG`, `DEG`, `RAD`, `FAL`, `FSC`, `AVE`, `STD`, `SRT`, `SIZE`, `LFL`, `LFU`, `SQL`, `PID`, `EVENT`, `UID`, `UIE`, `TND`, `MCR`, `BRK`, `FOR`, `TONR`, `TOFR`, `RTOR`, `CTUD`, `OSRI`, `OSFI`.

Appendix H — Walkthrough log

For the first reader. Work through Part IV in order; after each lesson's *Run it*, tick the row if the check passed as written, and note anything that was unclear, out of order, or did not behave as the text says — a sentence is enough. The log is what the next revision of this book is written from.

Reader: _____ Version of the page: _____ Date started: _____

#	Lesson	The check	Passed	Time taken	Where I stalled / what was unclear
1	Lay out the machine (ch. 16)	43 devices; Scene sheet reads ≈ 63 DI, 34 DO, 10 AI in use; pipe, incline and latch doors visible in Side / 3D	☐		
2	The tag list (ch. 17)	Monitor Tags: Start goes to 1 when pressed; Stop is already 1; toggling Bottle_Feed_Emit drops a bottle and LS1 goes to 1	☐		
3	Step 1 — run circuit (ch. 18)	Start: Run lamp on, Line_Run = 1. Stop drops it. Start while holding Stop: nothing	☐		
4	Step 2 — dwell and feeder (ch. 19)	One bottle drops at the head; Dwell.ACC counts to 1000 and holds; Dwell.DN stays on	☐		
5	Step 3 — registers (ch. 20)	Exist bit 0 = 1 after the first dwell; toggling Conv_Index twice walks it to bit 1	☐		
6	Step 4 — filling (ch. 21)	Verifies clean; filler head tank reads 100 %; lamps dark (nothing fills yet)	☐		
7	Step 5 — capping (ch. 22)	Verifies clean; green column lamp on when the line runs	☐		
8	Step 6 — index (ch. 23)	Belt steps one pitch per dwell; bottles fill at stop 7 and cap at stop 10; Simulator Log shows fills and caps	☐		
9	Step 7 — sort (ch. 24)	Broken bottles into the shredder; a box under the chute; IN BOX counts to 4; small bottles down the incline and on to the packer's lower lane; with the lane full the divert conveyor stops at LS12; no "lost", no "no box" log lines	☐		

#	Lesson	The check	Passed	Time taken	Where I stalled / what was unclear
10	Step 8 — count and preset (ch. 25)	Dial 0020: line stops at LARGE = 20; Enter zeroes; Start runs again	☐		
11	Step 9 — supply (ch. 26)	Tank falls to 20 %, red flashes, inlet opens, level climbs to 90 %	☐		
12	Step 10 — labeler (ch. 27)	Every capped bottle leaves labeled; with 3 labels on the roll: fault latched, amber, line keeps running; click reloads, Enter clears	☐		
13	Step 11 — alarms (ch. 28)	Three minutes and random Stop / Start: Alarm_Word 0. Each injected fault caught by its bit; lamp flashes, steady after Reset; Clear, Reset, Start recovers	☐		
14	Step 12 — packing (ch. 29)	Cases come in fast, creep onto the mark, are packed with four of one size and leave; both lanes picked; Case_Speed_Ref 10000 / 2500; CASES counts, and CASES LARGE / SMALL on Pack_HMI; Pack_HMI STOP and START; a drive trip latches bit 6 and Reset clears it	☐		
15	Step 13 — the mix kitchen (ch. 30)	Mix_Step 4 when Supply_Tank calls; the transfer runs while the inlet is open; the kitchen tops up and is ready again within a minute; STOP on Tank_HMI stops only the kitchen; seven minutes with Alarm_Word 0	☐		
16	Commissioning (ch. 31)	All checklist items pass; compare with the template via Export / Copy as Text	☐		
17	Levels 1–5 (ch. 33–37)	Each level completes with every bottle in the right bin, under par	☐		
18	Level 6 — Pack it (ch. 38)	Complete, every good bottle packed and the broken ones in the scrap bin, under 140 s	☐		
19	Level 7 — Batch Mix (ch. 39)	Three batches A, B, C on spec (ratio, temperature, mixing time), no overflow or scorch, under 240 s	☐		
20	Level 8 — Vision check (ch. 40)	Every bottle inspected, every defective one rejected and every good one shipped, no missed acquisition or overrun, under 45 s	☐		

Overall. Which chapter of Part II did you have to re-read before Part IV made sense? _____
Which lesson was the hardest? _____

What would you add, cut or move?